



CartesianSchool

PYTHON 3.14 · С НУЛЯ

Python с нуля

программирование, графика,
приложения и игры

Siergej Sobolewski

Software & AI Engineer, основатель Cartesian School

PYTHON 3.14 · С НУЛЯ

Python с нуля

программирование, графика, приложения и
игры

Siergej Sobolewski

Software & AI Engineer, основатель Cartesian School

Python с нуля: программирование, графика, приложения и игры

Siergej Sobolewski — Software & AI Engineer, основатель
Cartesian School

Издание Cartesian School, 2026. Python 3.14.

Электронное издание. Онлайн-версия курса, интерактивная практика в браузере и исходный код всех проектов — cartesianschool.org

© Siergej Sobolewski / Cartesian School

Текст книги и оригинальные учебные материалы: [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-nc-sa/4.0/) (CC BY-NC-SA 4.0).

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Программный код — включая фрагменты кода и листинги внутри текста книги, а не только отдельные проекты и скрипты, — если прямо не указано иное, распространяется на условиях MIT License.

Оглавление

ВВОДНЫЕ МАТЕРИАЛЫ

Об авторе	6
О техническом рецензенте	7
Введение	8
Глава 1. А вы знали?	10
Глава 2. Давайте установим Python!	36
Глава 3. Ваша первая программа на Python	89
Глава 4. Python любит числа	149
Глава 5. Давайте поиграем с числами!	219
Глава 6. Рисуем классные вещи с помощью Turtle	291
Глава 7. Глубокое погружение в Turtle	364
Глава 8. Играем с буквами и словами	471
Глава 9. Выполняй мою команду!	527
Глава 10. Немного автоматизации!	605
Глава 11. Очень много информации!	685
Глава 12. Множество увлекательных мини-проектов!	773
Глава 13. Автоматизация с помощью функций	856
Глава 14. Создаём объекты реального мира	961
Глава 15. Python и файлы	1043
Глава 16. Создаём классные приложения с Tkinter	1149
Глава 17. Проект: игра «Крестики-нолики» с Tkinter	1273
Глава 18. Проект: приложение для рисования с Tkinter	1366
Глава 19. Проект: игра «Змейка» с Turtle	1465

Глава 20. Разработка игр с Pygame	1553
Глава 21. Проект: космический шутер с Pygame	1657
Глава 22. Веб-разработка с Python	1723
Глава 23. Первый проект на GitHub: SafeSort	1830
Глава 24. Что дальше? Roadmap Python-разработчика	2023

ПРОЕКТЫ

Рисовалка	2082
Змейка	2083
Прыгающий мяч	2084
Космический шутер	2085
Список задач	2086
Калькулятор	2087
Генератор случайных историй	2088
Камень, ножницы, бумага	2089
Отскакивающий мяч — версия с классом	2090
Преобразование температуры	2091
Заметки	2092
Крестики-нолики	2093
SafeSort	2094

СПРАВОЧНИК

Предметный указатель	2095
----------------------	------

Об авторе

Сергей Соболевски (Siergej Sobolewski) — инженер с более чем двадцатилетней практикой в системном программировании и AI, основатель Cartesian School. Его опыт — от embedded и авионики до production AI/ML — лёг в основу инженерного метода этого курса: объяснение должно быть не просто понятным, а точным и проверяемым в реальном коде. Эта книга — попытка показать программирование так, как его показывают инженеру, которого готовят к настоящей работе, но с самого первого шага.

Сергей Соболевски (Siergej Sobolewski), основатель Cartesian School. Эта книга выросла из практического вопроса: как объяснить программирование человеку, который никогда раньше не писал ни строчки кода, не превращая объяснение ни в скучный справочник, ни в поверхностную «книжку с картинками».

Cartesian School — образовательный проект, в основе которого лежит идея, что современный инженерный подход и понятное объяснение для начинающих не противоречат друг другу: одно и то же занятие может быть одновременно точным и увлекательным.

О техническом рецензенте

Открытый пункт

Имя технического рецензента будет добавлено сюда после того, как Product Owner назначит рецензента издания. Раздел сохранён по канонической структуре оглавления и не удалён — при этом мы не стали придумывать несуществующего человека.

Каждая глава этой книги проходит техническую проверку: весь код автоматически компилируется, проверяется линтером `ruff`, а примеры в формате Jupyter Notebook реально выполняются от начала до конца — результаты вычислений в тексте книги получены не «на глаз», а настоящим запуском кода на Python 3.14.

Введение

Коротко о том, как устроена эта книга и как получить от неё максимум.

Для кого эта книга

Эта книга — для тех, кто раньше не писал код. Совсем. Неважно, сколько вам лет: десять, двадцать пять или пятьдесят — если вы умеете пользоваться компьютером и готовы разбираться, книга проведёт вас от первой напечатанной строки до собственных игр, приложений и рисунков на экране.

Как устроена каждая глава

Почти в каждом разделе вы встретите один и тот же порядок: сначала — зачем это нужно, затем — простое объяснение идеи, потом — рабочий пример, который можно запустить прямо сейчас, и в конце — практика и типичные ошибки. Мы намеренно не начинаем с сухого определения — сначала должно быть понятно, зачем оно вам, а термин появится следом.

Классический подход и современный Python 3.14

Python существует больше тридцати лет, и за это время в языке появлялись более удобные способы делать привычные вещи. Там, где это важно, книга сначала показывает классический приём — потому что именно его вы встретите в старом коде, статьях и ответах на форумах — а затем современный вариант для Python 3.14 и объясняет, чем они отличаются и что использовать сегодня. Это отмечено специальным блоком «Классический подход → современный Python».

Теория на сайте, практика в Jupyter Notebook

Теорию вы читаете здесь, на сайте Cartesian School — с подсветкой кода, поиском и навигацией. А для практики к каждому разделу прилагается отдельный файл Jupyter Notebook: в нём можно менять код и сразу видеть результат, не открывая ничего дополнительного, кроме Visual Studio Code или PyCharm.

Уровни сложности практики

Задания отмечены звёздами: ★ — базовая практика, повторяющая пример из раздела; ★★ — самостоятельная задача, требующая немного подумать; ★★★ — задача повышенной сложности для тех, кто хочет большего. Не в каждом разделе есть все три уровня — только там, где это оправдано.

Что понадобится

Python 3.14, установленный по инструкции из главы 2, и один из двух редакторов кода: Visual Studio Code или PyCharm — оба подробно описаны в книге. Больше почти ничего не нужно: все проекты в этой книге можно запустить на обычном ноутбуке.

А вы знали?

Первая глава книги «Python с нуля» — что такое программирование, откуда взялся Python, как устроена его экосистема и почему он отлично подходит для первого языка.

1.1 Что такое программирование?

Алгоритм, инструкция, выражение

Интерпретатор и компилятор

Почему вашим детям стоит научиться программировать?

Почему Python?

1.2 История Python

1.3 python.org, документация и PyPI

1.4 Сообщество и философия Python

1.5 Python — это весело!

Где применяется Python

1.6 Первые переменные и первые ошибки

1.7 Как получить максимум от этой книги

Итоги

Что такое программирование?

Прежде чем писать код, полезно понять, что вообще происходит, когда компьютер «выполняет программу» — и почему для первого языка мы выбрали именно Python.

После этой главы вы сможете: различать программу, исходный код, алгоритм, инструкцию и выражение; объяснить роль байт-кода в CPython; расположить основные этапы истории Python; различать python.org, документацию, PyPI, PSF и PEP; назвать области применения и границы Python; запустить `print(...)` и прочитать сообщение об ошибке.

Что такое программирование?

Компьютер невероятно быстрый, но абсолютно несамостоятельный. Сам по себе он не умеет вообще ничего — ни открыть игру, ни показать фотографию, ни посчитать сдачу в магазине. Всё, что делает компьютер, он делает потому, что кто-то заранее написал для него точную последовательность команд. Эта последовательность и называется **программой**, а процесс её написания — **программированием**.

Текст программы называется **исходным кодом** (source code). Процессор не выполняет его напрямую: используемый в книге **CPython** подготавливает код и выполняет его. На первом уровне будем называть этот процесс работой **интерпретатора**; точная схема с байт-кодом приведена ниже.



От идеи до результата: путь программы

Что происходит

Когда вы нажимаете «Выполнить», происходит эта упрощённая цепочка: ваш текст → реализация Python → наблюдаемый результат (например, вывод текста на экран). Ниже мы уточним внутренний этап CPython с байт-кодом.

Алгоритм, инструкция, выражение

Несколько слов, которые будут встречаться в каждой главе этой книги — полезно один раз договориться, что они означают:

- **Алгоритм** — конечная, однозначно заданная последовательность шагов, которая для допустимых входных данных приводит к результату или завершению. Результат может зависеть от входных данных или случайности — он не обязан всегда быть одинаковым.
- **Инструкция** (statement) — одна законченная команда программе, например «выведи текст на экран» или «сохрани число в память».
- **Выражение** (expression) — часть кода, у которой есть значение, например `2 + 2` или `"Привет" + "!"`. Выражения часто находятся внутри инструкций.
- **Программа** — упорядоченный набор инструкций на языке программирования, реализующий один или несколько алгоритмов.
- **Машинный код** — последовательность чисел, которую напрямую выполняет процессор. Люди почти никогда не пишут его руками — именно для этого и нужны языки вроде Python и переводчики, которые превращают удобный для человека код в машинный.

Интерпретатор и компилятор

Уровень 1 (достаточно для начала). Реализация Python принимает исходный код и организует его выполнение согласно правилам управления потоком. Её обычно называют **интерпретатором**. Это намеренно упрощённая модель.

Для сравнения: у некоторых языков (например, C или Rust) есть отдельный шаг — **компилятор** заранее переводит весь исходный код целиком в машинный код, и только потом готовую программу запускают. При обычном запуске CPython пользователь не собирает заранее машинный файл, поэтому Python называют *интерпретируемым*. Это описание способа запуска, а не утверждение, что компиляции нет.

Распространённая неточность

Говорить «Python вообще никогда не компилируется» — не совсем точно. На самом деле происходит немного больше — см. уровень 2 ниже. Для начала уровня 1 полностью достаточно, чтобы писать и понимать код.

Уровень 2 (если интересно, что происходит на самом деле). Официальная («эталонная») реализация Python, CPython, на самом деле сначала незаметно компилирует ваш исходный код в промежуточную форму — **байт-код** (компактные инструкции, непонятные человеку, но быстрые для выполнения), а затем выполняет этот байт-код с помощью Python VM (виртуальной машины). Всё это происходит автоматически и незаметно для вас — просто теперь вы знаете, что «интерпретируемый» не значит «без единого шага компиляции».



Подробнее об устройстве CPython — в разделе «Сообщество и философия Python» этой же главы.

Почему вашим детям стоит научиться программировать?

Программирование — не заявка на профессию «программист», а способ мышления, который приговаривается в любой области. Это касается и взрослых, которые садятся за первую программу сами. Три причины, почему этому стоит учиться в любом возрасте:

- **Ошибка — это нормально.** Программа почти никогда не работает с первого раза, и это не повод расстраиваться — это часть процесса. Программирование учит спокойно находить причину и пробовать снова, вместо того чтобы бояться ошибиться.
- **Идея превращается в результат.** Мысль «а что если сделать вот так» можно проверить за несколько минут, написав и запустив код, — не нужно ничьё разрешение.
- **Это переносимый навык.** Умение разложить сложную задачу на простые шаги приговаривается далеко за пределами программирования — в учёбе, работе и повседневных задачах.

Почему Python?

Языков программирования — сотни. Python выбран для первого языка по трём практическим причинам:

- **Читается почти как обычный текст.** Строка `if age >= 18:` понятна без перевода. Меньше отвлечённого синтаксиса — больше внимания на саму идею.
- **Один язык — очень разные результаты.** На Python пишут игры, рисуют графику, собирают настольные и веб-приложения, обучают модели искусственного интеллекта. Всё, о чём вы прочитаете в этой книге, — на одном и том же языке.
- **Огромное сообщество.** Если у вас возникнет вопрос — скорее всего, кто-то уже задавал его и получил ответ. Готовых инструментов (их называют библиотеками или пакетами) для Python написано огромное количество — подробнее об этом чуть позже в этой же главе.

История Python

Python появился не случайно и не сразу — короткая, но настоящая история о том, почему язык устроен именно так, как устроен сегодня.

Python

До Python: CWI и язык ABC

Python появился не на пустом месте. В 1980-х годах в нидерландском исследовательском институте **CWI** (Centrum Wiskunde & Informatica, «Центр математики и информатики») разрабатывался язык **ABC** — он должен был быть простым и удобным для обучения программированию непрофессионалов. У ABC были хорошие идеи (например, отступы вместо фигурных скобок), но были и серьёзные ограничения — он не был предназначен для больших «настоящих» программ и плохо расширялся.

Рождение Python: 1989–1991

Один из разработчиков ABC, нидерландский программист **Гвидо ван Россум** (Guido van Rossum), хорошо знал сильные и слабые стороны ABC. В конце декабря 1989 года, во время рождественских каникул, он начал писать новый язык — как хобби-проект, чтобы занять себя на праздниках, и как способ сохранить понравившиеся идеи ABC, избавившись от его ограничений.

Гвидо ван Россум на конференции PyCon US

Гвидо ван Россум, создатель Python, на PyCon US 2024. Фото: Kushal Das, лицензия CC BY-SA 4.0 (изображение обрезано и сжато для сайта).

Первый публичный релиз, **Python 0.9.0**, вышел в феврале 1991 года. В нём уже были функции, обработка исключений, классы с наследованием и встроенные типы данных вроде списков и словарей — многое из того, что вы будете использовать в этой книге.

Идея

Python начинался как личный проект одного человека для собственного удовольствия. Не обязательно с самого начала «менять индустрию» — иногда достаточно решить свою собственную небольшую задачу.

Откуда взялось название «Python»

Несмотря на логотип со змеями, язык назван вовсе не в честь пресмыкающегося. Гвидо ван Россум — большой поклонник британского комедийного шоу «**Летающий цирк Монти Пайтона**» (Monty Python's Flying Circus), и ему хотелось короткое, немного необычное и

запоминающееся имя. Змеиная тема логотипа появилась уже позже, как визуальная ассоциация со словом «Python».

Python 1.x и Python 2.0

Версия **Python 1.0** вышла в январе 1994 года. Язык продолжал расти: появлялись новые модули стандартной библиотеки, росло сообщество разработчиков.

В октябре 2000 года вышел **Python 2.0**, принёсший важные для языка вещи — например, списковые включения (list comprehensions) и полноценный сборщик мусора, умеющий находить и освобождать память даже у объектов, ссылающихся друг на друга по кругу.

В 2001 году была создана **Python Software Foundation (PSF)** — некоммерческая организация, которой с тех пор принадлежат права на Python и которая направляет развитие языка. Подробнее о PSF — в следующем разделе этой главы.

Python 3: зачем понадобилась несовместимая версия

К середине 2000-х в языке накопились особенности, которые нельзя было исправить, не сломав часть старого кода, — например, то, как Python 2 обрабатывал текст и байты, было источником постоянной путаницы, особенно при работе с не-английскими языками (в том числе русским). Разработчики Python сознательно пошли на выпуск **Python 3.0** (декабрь 2008) — версии, которая местами *намеренно несовместима* со старым кодом ради того, чтобы исправить эти накопившиеся проблемы: единая модель текста (Unicode по умолчанию), `print()` стала обычной функцией, а не особым словом языка, целочисленное деление стало более предсказуемым, и многое другое.

Официальный источник

Общий план Python 3000 описан в PEP 3000, а изменения, заметные пользователю, — в официальном разделе «What's New In Python 3.0». PEP 3100 «Miscellaneous Python 3.0 Plans» дополняет их перечнем отдельных запланированных изменений, но не является полным списком причин перехода.

Переход занял намного дольше, чем ожидалось: сообществу требовалось время, чтобы переписать старый код и библиотеки. Официальная поддержка **Python 2 закончилась 1 января 2020 года** — с этого момента для Python 2 больше не выпускают даже исправления критических уязвимостей.

Путь к современному Python и версия 3.14

После 2008 года Python продолжил развиваться заметными, но уже совместимыми шагами: f-строки для удобного форматирования текста (3.6), оператор «морж» `:=` (3.8), более понятные и точные сообщения об ошибках и структурное сопоставление с образцом (3.10), заметный прирост скорости выполнения в нескольких последних релизах. Именно это непрерывное, но осторожное развитие и определяет то, что сегодня называют «современным Python».

Эта книга целиком написана и проверена на **Python 3.14** — актуальном стабильном релизе на момент написания.

- 
- CWI, Нидерланды**
исследовательский институт, где рождается идея
 - Язык ABC**
предшественник Python — простой, но со своими ограничениями
 - Конец 1989**
Гвидо ван Россум начинает Python на рождественских каникулах
 - 1991 • Python 0.9.0**
первый публичный релиз
 - 1994 • Python 1.0**
первая полноценная версия
 - 2000 • Python 2.0**
списковые включения, сборщик мусора с циклами
 - 2001 • создана PSF**
Python Software Foundation — некоммерческая организация
 - 2008 • Python 3.0**
осознанно несовместимые улучшения языка
 - 2020 • конец Python 2**
официальная поддержка Python 2 прекращена
 - Сегодня • Python 3.14**
актуальный стабильный релиз, база этой книги

Краткая хронология Python

python.org, документация и PyPI

Три места, с которыми полезно познакомиться в самом начале: официальный сайт, официальная документация и каталог сторонних пакетов.

Python

python.org — официальный сайт

python.org — официальный сайт языка Python, которым управляет Python Software Foundation. Это первое место, где стоит искать официальную информацию: как установить Python, где почитать документацию, что нового в последнем релизе.

Главная страница python.org с разделами Get Started, Download, Docs и Jobs

python.org — официальный сайт Python. Обратите внимание на верхнее меню (About, Downloads, Documentation, Community) и блок «Download» с актуальной версией.

На главной странице стоит запомнить четыре раздела верхнего меню:

- **Downloads** — установщики Python для Windows, macOS и Linux (этим вы воспользуетесь уже в главе 2).
- **Documentation** — ссылка на docs.python.org, официальную документацию (подробнее ниже).
- **Community** — форумы, списки рассылки, локальные сообщества Python-разработчиков.
- **PSF** (вкладка вверх страницы) — раздел о самой организации, Python Software Foundation.

Типичная ошибка

В интернете много сайтов с похожими названиями и «упрощённой установкой Python», которые на самом деле не имеют отношения к официальному проекту. Надёжный признак подлинности — домен `python.org` в адресной строке браузера.

Официальная документация: `docs.python.org`

Документация — это не что-то, что читают только опытные разработчики. Хорошая документация — рабочий инструмент на каждый день, которым пользуются абсолютно все, включая авторов самого языка.

Главная страница `docs.python.org 3.14` с разделами Tutorial, Library reference, Language reference

`docs.python.org` — официальная документация. Три раздела в левой верхней части страницы стоит различать с самого начала.

На странице документации сразу видно три разных раздела, и стоит понимать разницу между ними:

- **Tutorial** — учебное введение в язык, написанное для тех, кто уже немного знаком с программированием. Хороший источник для повторения, но не замена этой книге для полного новичка.
- **Library Reference** — подробное описание стандартной библиотеки: что умеет каждый встроенный модуль и функция. Именно сюда вы будете заглядывать чаще всего, когда захотите узнать, что ещё умеет, например, `print()`.
- **Language Reference** — формальное, самое строгое описание правил самого языка. Обычно не нужен новичку — рассчитан на очень точные технические вопросы.

Идея

Пока не нужно понимать документацию целиком. Цель этого раздела — просто перестать бояться слова «документация» и один раз увидеть, как она устроена.

PyPI: сторонние пакеты Python

В Python уже встроено много полезного — это называется **стандартной библиотекой**. Но существует ещё огромное количество готовых инструментов, написанных другими разработчиками и опубликованных отдельно — их называют **пакетами** (или библиотеками). Они расширяют возможности Python далеко за пределы того, что есть «из коробки».



Главное хранилище таких пакетов называется **PyPI** — Python Package Index (pypi.org). Когда кто-то говорит «поставь пакет через `pip`» — `pip` как раз и загружает пакет именно с PyPI.

Главная страница pypi.org с полем поиска и статистикой количества проектов

pypi.org — Python Package Index, каталог сторонних пакетов Python.

Официальный источник

Подробное практическое руководство по установке пакетов — packaging.python.org. Само практическое умение пользоваться `pip` и виртуальными окружениями понадобится вам позже в этой книге — пока достаточно понимать саму идею экосистемы пакетов.

Сообщество и философия Python

Кто решает, каким становится Python, какими принципами он руководствуется — и почему «Python» и «CPython» не совсем одно и то же.

Python Software Foundation (PSF)

Python Software Foundation — некоммерческая организация в США, основанная в 2001 году. Ей принадлежат права на Python, она поддерживает развитие языка, организует конференции (например, ежегодную PyCon US) и финансирует инфраструктуру сообщества — включая сам сайт python.org и PyPI.

PSF не продаёт Python и не выдаёт официальных сертификатов от имени языка — её роль скорее административная и организационная: следить, чтобы у Python было будущее и чтобы развитием занималось открытое сообщество, а не одна компания.

PEP: как Python принимает решения об изменениях

Важные изменения в языке не происходят стихийно. Для этого существует формальный процесс — **PEP** (Python Enhancement Proposal, «предложение по улучшению Python»). Это документ, в котором кто-то подробно описывает предлагаемое изменение, объясняет, зачем оно нужно, и его обсуждает сообщество, прежде чем принять или отклонить.

Все PEP пронумерованы и опубликованы на peps.python.org. Два PEP стоит знать по имени уже сейчас, просто чтобы узнавать их в будущем:

- **PEP 8** — руководство по стилю оформления кода Python (как называть переменные, сколько оставлять отступов и пробелов). Мы вернёмся к нему подробно позже в этой книге.
- **PEP 20** — он же «дзен Python», о котором ниже.

Дзен Python

У Python есть собственный короткий список принципов, объясняющих, каким язык старается быть. Его написал разработчик Тим Питерс, и увидеть его можно, выполнив в интерпретаторе Python одну строку:

```
dzen.py
```

```
import this
```

Своими словами, а не дословной цитатой, идея сводится к нескольким вещам, которые Python ценит больше всего:

- **Читаемость важнее краткости.** Понятный код лучше кода, который уместился в одну строку ценой ясности.
- **Явное лучше неявного.** Код должен по возможности прямо показывать, что он делает, а не полагаться на скрытую «магию».

- **Простота лучше сложности — но не ценой возможностей.** Если задача сложная, решение может быть сложным, но не сложнее, чем должно быть.

★ Базовая практика

Попробуем

Откройте практику этого раздела и выполните ячейку с `import this`. Прочитайте вывод целиком — эти строки будут появляться в подсказках книги ещё не раз.

CPython и другие реализации

Полезно различать две вещи: **Python как язык** (правила синтаксиса и поведения, которые описывает Language Reference) и **конкретную программу, которая этот язык выполняет** — её называют реализацией.

Самая распространённая реализация — **CPython**, написанная на языке C. Её вы используете на протяжении всей этой книги, и её же скачивают с python.org по умолчанию. Существуют и другие реализации, созданные для особых задач:



Python — это язык; у языка есть несколько реализаций

- **PyPy** — реализация с JIT-компиляцией (Just-In-Time), которая на некоторых задачах выполняет код заметно быстрее CPython за счёт компиляции «на лету» во время выполнения.
- **MicroPython** — компактная реализация Python для микроконтроллеров и встраиваемых устройств с очень ограниченной памятью — от роботов до умных датчиков.

Идея

Для этой книги достаточно знать одно: когда где-то написано «Python 3.14.2», почти всегда имеется в виду именно CPython — реализация по умолчанию, с которой вы уже работаете в практике этой главы.

Python — это весело!

Короткая экскурсия по тому, что вы будете строить своими руками на страницах этой книги, — и честный взгляд на то, где Python применяется в реальном мире.

Игры!

В главах 17, 19 и 21 вы соберёте три настоящие игры целиком: «Крестики-нолики», «Змейку» и космический шутер — от пустого окна до готовой игры с очками и правилами. Каждая из них строится маленькими понятными шагами, а не появляется одним большим куском кода.

Графика и анимация

Уже в главе 6 вы начнёте рисовать на экране с помощью модуля `turtle` — виртуальной черепашки с пером, которая двигается по вашим командам. Вот как выглядит код, рисующий квадрат — просто для примера, разбирать его подробно будем позже:

```
podglyadyvaem_vpered.py
```

```
import turtle

artist = turtle.Turtle()
for _ in range(4):
    artist.forward(100)    # едем вперёд
    artist.right(90)       # поворачиваем на 90°
```

Забегая вперёд

Не пытайтесь понять этот код прямо сейчас — `for` (цикл) появится только в главе 10. Здесь достаточно увидеть, что настоящий рисунок получается всего из нескольких строк.

Веб-сайты

В главе 22 вы узнаете, как устроен веб — что делают HTML, CSS и JavaScript в браузере, какую роль играет Python на сервере, — и запустите собственный работающий сайт на Flask.

Приложения

С помощью модуля `Tkinter` (главы 16–18) вы соберёте настоящие оконные приложения с кнопками и полями ввода — калькулятор чаевых и собственное приложение для рисования, — а не только программы, которые выводят текст в терминал.

Где применяется Python в реальном мире

Игры, графика, приложения и сайты из этой книги — только часть картины. За пределами учебного курса Python используют в очень разных областях:

- **Автоматизация и скрипты** — переименовать сотни файлов, обработать таблицу, запустить рутинную задачу по расписанию.
- **Веб-разработка** — серверная часть крупных сайтов (Django, Flask и похожие инструменты).
- **Анализ данных и наука** — обработка больших таблиц данных, статистика, научные вычисления (pandas, NumPy).
- **Искусственный интеллект и машинное обучение** — обучение и запуск моделей (PyTorch, TensorFlow и другие).
- **Серверные и backend-сервисы** — API, очереди задач, внутренние инструменты компаний.
- **Тестирование программ** — автоматические тесты для другого софта, независимо от того, на каком языке он написан.
- **DevOps и инфраструктура** — скрипты развёртывания, автоматизация серверов.
- **Образование** — один из самых популярных первых языков программирования в мире, в том числе в этой книге.
- **Инструменты кибербезопасности** — сканеры, автоматизация анализа уязвимостей.

- **Встраиваемые системы** — MicroPython на микроконтроллерах и небольших устройствах.

Где Python — не лучший выбор

Честности ради: Python не подходит одинаково хорошо для абсолютно всего. Несколько примеров, где обычно выбирают другие инструменты:

- **Системы жёсткого реального времени** — там, где опоздание в доли миллисекунды недопустимо (например, некоторые встроенные системы управления), обычно выбирают более предсказуемые по времени выполнения языки.
- **Некоторые нативные мобильные приложения** — для iOS и Android чаще используют инструменты, «родные» для этих платформ.
- **Низкоуровневая работа с ядром ОС и драйверами устройств** — эта область обычно требует языков вроде C, дающих прямой контроль над памятью и железом.
- **Сильно ограниченные по памяти окружения** — там, где даже MicroPython оказывается слишком «тяжёлым», выбирают C или ассемблер.

Это нормально — ни один язык программирования не universal. Знать сильные и слабые стороны инструмента — часть профессионального роста.

Практика: первый запуск кода

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/01-01/index.html) → (<https://www.cartesianschool.org/practice/01-01/index.html>)

Первые переменные и первые ошибки

Один правильный мысленный образ переменной — и первая спокойная встреча с сообщением об ошибке, задолго до того, как оно застанет вас врасплох.

Первые переменные: как Python на самом деле их хранит

Уже в следующей главе вы начнёте активно пользоваться переменными. Один маленький, но важный мысленный образ стоит освоить прямо сейчас — он убережёт от путаницы позже.

```
переменная.py
```

```
name = "Anna"  
print(name)
```

Легко (и ошибочно) представлять переменную как «коробку», в которую положили значение. На самом деле удобнее думать иначе: **имя — это ярлык, который указывает на значение**, а не контейнер, где значение физически лежит.

name



"Anna"

name не «содержит» текст, а указывает на него

Что происходит

`name = "Анна"` означает «пусть имя `name` теперь указывает на значение `"Анна"`», а не «положи текст внутрь коробки по имени `name`». Подробное устройство памяти Python — тема для более поздних глав; сейчас важно только не привыкнуть к неверному образу «коробки».

★ Базовая практика

Эксперимент

В практике этого раздела создайте переменную `name` со своим именем и выведите её через `print(name)`. Затем присвойте `name` другое значение и выведите снова — обратите внимание, что «ярлык» просто стал указывать на другое значение.

Ошибки — это нормально

Рано или поздно любая программа выдаёт ошибку — и это не признак того, что что-то пошло катастрофически не так. Разберём это на примере прямо сейчас, пока ставки низкие.

```
oшибka.py
```

```
print("Hello"
```

В этом коде не хватает закрывающей скобки. Если выполнить такую строку, Python не угадывает, что вы имели в виду, — вместо этого он останавливается и печатает подробное сообщение об ошибке, которое называется **traceback** («трассировка»): в нём указано, в какой строке произошла проблема и какого рода она была (в данном случае — незакрытая скобка).

Типичная ошибка новичка

Увидев длинный текст ошибки, легко запаниковать и решить, что «что-то сломалось навсегда». На самом деле `traceback` — это не угроза, а подсказка: он почти всегда прямо указывает, в какой строке искать проблему.

Полезная привычка при виде ошибки — прочитать **последнюю строку** `traceback`: там обычно кратко написано, что именно пошло не так. Всё, что выше, — это путь, которым Python дошёл до проблемы.

★★ Самостоятельная задача

Debug Lab

В практике этого раздела найдёте специально сломанную строку кода. Прочитайте `traceback`, найдите строку с проблемой и исправьте её самостоятельно — так же, как в разделе «Типичная ошибка» предыдущей практики этой главы.

Как получить максимум от этой книги

Несколько привычек, которые сделают чтение куда эффективнее, — и полные итоги главы 1.

Четыре простые привычки сделают чтение книги гораздо эффективнее:

- **Печатайте код руками**, а не копируйте. Опечатка, которую вы находите и исправляете сами, учит куда лучше, чем код, который просто сработал с первого раза.
- **Меняйте числа и слова в примерах** — что произойдёт, если поставить не 100, а 300? Предсказание результата *до* запуска — лучшая тренировка.
- **Не пропускайте раздел «Типичная ошибка»**. Там разобраны настоящие ошибки начинающих — увидеть их заранее полезнее, чем встретить их вслепую.

- **Открывайте ноутбук практики** к каждому разделу — теория на сайте объясняет идею, а ноутбук — то место, где вы её проверяете руками.

Если что-то не работает

Это не признак того, что программирование не для вас — это самая обычная часть процесса. В главе 3 и далее в книге разобрана целая система: как читать сообщение об ошибке и находить причину, а не бояться её.

Итоги

Что мы узнали в этой главе

- Программа — это точная последовательность команд для компьютера; исходный код превращает в действия **интерпретатор**, а внутри CPython код по пути ещё и компилируется в байт-код.
- Python появился в CWI в конце 1989 года — его создал Гвидо ван Россум как развитие идей языка ABC; название — в честь «Летающего цирка Монти Пайтона».
- Python 3 намеренно несовместим местами с Python 2 — эта цена была осознанным выбором ради исправления давних проблем языка; поддержка Python 2 закончилась в 2020 году.
- python.org — официальный сайт, docs.python.org — официальная документация (Tutorial/Library Reference/Language Reference), PyPI — каталог сторонних пакетов.
- Python Software Foundation направляет развитие языка; важные изменения проходят через процесс PEP; «дзен Python» (`import this`) описывает ценности языка.
- CPython — основная реализация Python, но не единственная: есть также PyPy и MicroPython для разных задач.
- Python используется в самых разных областях — но не является лучшим выбором абсолютно везде, и это нормально для любого языка.
- Имя переменной указывает на значение, а не «содержит» его физически, — и ошибки (traceback) — обычная, полезная часть работы, а не повод для паники.
- В главе 2 вы установите Python на свой компьютер и начнёте писать код за пределами браузера.

Давайте установим Python!

Полноценная сборка рабочего места Python-разработчика: установка на Windows, macOS и Linux, терминал и PATH, VS Code и PyCharm, виртуальные окружения, pip/pipx/venv/uv/conda — и как всё это связано между собой.

- 2.1 Что именно мы устанавливаем?
- 2.2 Терминал, shell и PATH
- 2.3 Установка Python на Windows
- 2.4 Установка Python на macOS
- 2.5 Установка Python на Linux
- 2.6 Какой Python действительно запущен
- 2.7 VS Code: установка и расширения
- 2.8 VS Code: интерпретатор и рабочий процесс
- 2.9 PyCharm: проект, интерпретатор, окружение
- 2.10 Виртуальные окружения — зачем они нужны
- 2.11 Создаём .venv

2.12 Устанавливаем первый пакет

2.13 pip, pipx, venv, virtualenv, uv

2.14 conda, Miniconda, Miniforge, Anaconda

2.15 Как IDE, Python и окружение связаны

2.16 Типичные проблемы и диагностика

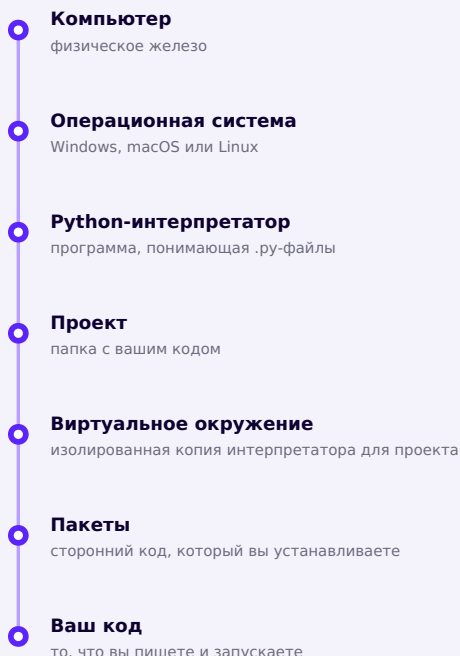
2.17 Рекомендации Cartesian School и итоги

Что именно мы устанавливаем?

Прежде чем ставить галочки в установщиках, полезно понять, из каких слоёв вообще состоит рабочее место Python-разработчика.

Компьютер невероятно быстрый, но абсолютно несамостоятельный. Сам по себе он не умеет вообще ничего — ни открыть игру, ни показать фотографию, ни посчитать сдачу в магазине. Всё, что делает компьютер, он делает потому, что кто-то заранее написал для него точную последовательность команд. Эта последовательность и называется **программой**, а процесс её написания — **программированием**.

Когда вы устанавливаете Python, вы устанавливаете **интерпретатор** — программу, которая умеет читать и выполнять код на Python (мы говорили об этом в главе 1). Но между «Python установлен» и «мой код работает» на самом деле выстраивается целая цепочка слоёв, и эта глава — про то, чтобы честно разобрать каждый из них.



Семь слоёв между «железом» и вашей первой программой

Четыре вещи, которые легко перепутать

Это, пожалуй, самое важное различие во всей главе — и одна из главных причин, почему начинающие путаются в установке Python:

- **Python-интерпретатор ≠ VS Code.** VS Code — это редактор кода. Он не умеет выполнять Python сам по себе — он находит и запускает установленный на компьютере интерпретатор.

- **Python-интерпретатор $\neq$ PyCharm.** То же самое: PyCharm — среда разработки, которая тоже использует внешний интерпретатор, а не содержит его «внутри себя».
- **Python-интерпретатор $\neq$ pip.** pip — это программа для установки пакетов; она сама работает *поверх* интерпретатора, а не является им.
- **Python-интерпретатор $\neq$ виртуальное окружение.** Виртуальное окружение (о нём подробно — в разделах 2.10–2.11) — это лёгкая, изолированная копия интерпретатора для конкретного проекта, а не что-то отдельное от него.

Идея

VS Code и PyCharm — это инструменты, которые *используют* Python-интерпретатор, а не заменяют его. Установка редактора кода и установка Python — это два разных, независимых шага.

Дальше в этой главе мы пройдем весь путь по порядку: установим сам интерпретатор на вашей операционной системе (разделы 2.3–2.5), научимся проверять, какой именно Python реально запускается (2.6), настроим редактор кода (2.7–2.9), разберёмся с виртуальными окружениями (2.10–2.12) и с зоопарком инструментов вокруг них (2.13–2.14), а в конце — соберём всё вместе и потренируемся находить и чинить типичные ошибки (2.15–2.17).

Терминал, shell и PATH

Одно из самых важных объяснений этой главы: что на самом деле происходит, когда вы вводите команду и нажимаете Enter.

Терминал и shell

Терминал — окно программы, где вы вводите текстовые команды и видите текстовый ответ компьютера. **Shell** (командная оболочка) — программа, которая внутри терминала на самом деле читает вашу команду и решает,

что с ней делать. На Windows это обычно PowerShell или классическая cmd, на macOS и Linux — чаще всего `zsh` или `bash`.

Команда обычно состоит из имени программы и, при необходимости, **аргументов** — дополнительных слов после неё: в `python --version` само `python` — команда, а `--version` — аргумент.

Путь, файловая система и текущая папка

Shell всегда «стоит» в какой-то одной папке — она называется **текущей рабочей папкой** (current directory). **Путь** (path) — это адрес файла или папки в файловой системе. Путь бывает:

- **абсолютным** — начинается от корня диска, например `C:\Users\anna\project` или `/home/anna/project`, и работает откуда угодно;
- **относительным** — отсчитывается от текущей папки, например `./project` или просто `project`.

Переменная окружения PATH

Когда вы вводите короткую команду вроде `python`, компьютер не ищет файл по всему диску — это заняло бы слишком много времени. Вместо этого shell смотрит в специальный список папок — **переменную окружения PATH** — и проверяет их по очереди в заданном порядке, пока не найдёт исполняемый файл с таким именем.

PATH (Linux/macOS, упрощённо)

```
/usr/local/bin
/usr/bin
/bin
~/local/bin
```

Если вы наберёте `python`, shell проверит `/usr/local/bin/python`, затем `/usr/bin/python` и так далее — и запустит первый найденный файл. Если ни в одной из папок PATH такого файла нет — вы увидите ошибку вроде `command not found` (macOS/Linux) или `'python' is not recognized...` (Windows).

- 
- Вы вводите команду**
например, `python`
 - Shell получает текст**
программа-командная оболочка
 - Поиск по PATH**
shell проверяет папки по очереди
 - Исполняемый файл найден**
первое совпадение побеждает
 - Команда запускается**
или «`command not found`», если нигде не нашлось

Что происходит, когда вы нажимаете Enter после команды

Типичная ошибка

Если на компьютере установлено **два разных Python**, а в PATH они перечислены не в том порядке, который вы ожидаете, — команда `python` может запускать совсем не ту версию, которую вы только что установили. Раздел 2.6 научит это проверять.

Официальный источник

Подробное объяснение переменных окружения — в документации вашей операционной системы; для самого Python — раздел «Using Python on Unix platforms» / «Using Python on Windows» на docs.python.org.

Установка Python на Windows

Четыре шага: скачать с python.org, поставить галочку PATH, установить, проверить.

На Windows есть два официальных пути поставить Python: классический установщик `.exe`, который существует уже много лет, и новый **Python install manager** — облегчённый инструмент для управления несколькими версиями Python, который Python Software Foundation продвигает как основной способ установки на Windows начиная с недавних релизов. Traditional-установщик **остаётся доступным на протяжении веток 3.14 и 3.15**, так что оба варианта — не ошибка, а два официальных пути.

Для этого курса

Мы будем использовать классический установщик `.exe` — он нагляднее для первого знакомства и даёт точь-в-точь тот же результат: работающую команду `python` в терминале.

Шаг 1. Скачайте установщик

Откройте python.org/downloads — единственный источник, которому стоит доверять при загрузке Python. Сайт сам определяет вашу операционную систему и предлагает актуальную стабильную версию.

Страница загрузки Python для Windows на python.org

python.org/downloads/windows — официальная страница загрузки для Windows.

Скачивайте только с python.org

Сторонние сайты иногда предлагают «удобные» установщики Python с добавленным рекламным ПО. Официальный установщик — бесплатный, безопасный и всегда доступен напрямую на python.org.

Шаг 2. Запустите установщик

Запустите скачанный файл. На первом экране установщика — самая важная галочка во всей установке:

Не пропустите: «Add python.exe to PATH»

Внизу первого экрана установщика есть флажок **Add python.exe to PATH**. Обязательно поставьте его. Если пропустить этот шаг, Python установится, но команда `python` в терминале работать не будет — а причина будет совершенно не очевидна, если вы ещё не знаете, что такое PATH (мы разбирали это в разделе 2.2).

Затем нажмите **Install Now** для стандартной установки — она подходит для подавляющего большинства случаев и устанавливает Python, pip и стандартную библиотеку в папку профиля пользователя.



Четыре шага установки Python на Windows

Шаг 3. Проверьте установку

Откройте PowerShell (или обычную командную строку) и введите:

```
PowerShell
```

```
python --version
```

Вы должны увидеть что-то вроде `Python 3.14.7`. Если вместо этого терминал отвечает, что команда не распознана — скорее всего, вы пропустили галочку «Add python.exe to PATH». Решение — запустить установщик заново и выбрать **Modify**, снова отметив эту опцию, либо переустановить с нуля.

python или py?

На Windows часто работает ещё и команда `py` — это отдельный «лаунчер Python», который штатно ставится вместе с интерпретатором и умеет запускать нужную версию, даже если их установлено несколько (например, `py -3.14`). Для этого курса используйте `python` — но не пугайтесь, если увидите `py` в чужих инструкциях.

Про Microsoft Store

В Microsoft Store тоже есть Python — это официальный пакет, но у него есть особенности работы с правами доступа к файлам, которые иногда удивляют новичков. Для этого курса рекомендуем установщик с python.org — он предсказуемее и ближе к тому, с чем вы столкнётесь на реальных проектах и других операционных системах.

Установка Python на macOS

Официальный универсальный установщик — и почему системный Python лучше не трогать.

На современном macOS уже есть команда `/usr/bin/python3` — но это не «ваш» Python, а **Apple-контролируемый инструмент**: он поставляется вместе с Xcode / Command Line Tools for Xcode и предназначен для внутренних нужд самой macOS и стороннего ПО, которое на него полагается. Обычно это заметно более старая и неполная сборка Python, чем актуальный релиз с python.org. Устанавливать в него сторонние пакеты, обновлять или удалять его — плохая идея: вы можете сломать инструменты, от него зависящие.

Не трогайте Apple-Python

Никогда не устанавливайте пакеты «поверх» `/usr/bin/python3` и не удаляйте его — это официальная рекомендация docs.python.org: этот Python Apple-контролируемый и используется системным и сторонним ПО. Вместо этого поставьте отдельную, «вашу собственную» версию Python с python.org — так делают профессиональные разработчики, и оба Python спокойно сосуществуют на одном компьютере.

Шаг 1. Скачайте установщик

Откройте python.org/downloads/macos. Официальный установщик для macOS — **универсальный (universal2)** пакет `.pkg`: один и тот же файл работает и на Mac с процессором Apple Silicon (M1/M2/M3/M4), и на Mac с процессором Intel.

Страница загрузки Python для macOS на python.org

python.org/downloads/macos — официальная страница загрузки для macOS.

Шаг 2. Запустите установщик

Откройте скачанный `.pkg`-файл и пройдите стандартный мастер установки macOS (Введение → Лицензия → Место установки → Установка). В отличие от Windows, здесь нет отдельной галочки про PATH — установщик macOS сам настраивает всё необходимое.

Шаг 3. Проверьте установку

Откройте приложение **Terminal** и введите:

```
zsh
```

```
python3 --version
```

Почему `python3`, а не `python`?

На современном macOS (и на Linux) команда `python` без суффикса обычно вообще не существует. Так было не всегда: на очень старых версиях macOS (до Catalina, 2019 год) команда `python` исторически указывала на системный Python 2 — но это осталось в прошлом, а не текущее поведение. Официальный установщик `python.org` ставит команду именно как `python3`. Мы разберём, как сделать команду `python` удобной внутри виртуального окружения, в разделе 2.10.

Вы должны увидеть что-то вроде `Python 3.14.7`. Если терминал показывает более старую версию (например, `Python 3.9.6`) — это, вероятно, Apple-контролируемый `/usr/bin/python3`, а не тот интерпретатор, что вы только что установили; проверьте порядок PATH (раздел 2.4) или явно укажите путь через `sys.executable` (раздел 2.6), чтобы убедиться, какой именно `python3` реально запускается.

Установка Python на Linux

Linux — полноценная платформа первого класса для Python, со своими правилами хорошего тона: системный менеджер пакетов и обязательные виртуальные окружения.

Linux — такая же первоклассная платформа для Python, как Windows и macOS, и во многих профессиональных командах именно Linux — основная рабочая система (а ещё почти все серверы, на которых работает ваш будущий код, тоже Linux). Как и macOS, большинство дистрибутивов Linux уже включают системный Python — и здесь действует то же правило: **не трогайте системный Python**.

PEP 668: «externally-managed-environment»

На современных дистрибутивах (Ubuntu 23.04+, Debian 12+ и других) команда `pip install` без виртуального окружения специально **откажется работать** и покажет ошибку `error: externally-managed-environment`. Это не баг — это защита, описанная в официальном стандарте PEP 668, чтобы вы случайно не сломали системные инструменты, которые тоже написаны на Python. Правильный ответ на эту ошибку — не «обойти» её флагом, а создать виртуальное окружение (раздел 2.10) и ставить пакеты туда.

Установка через менеджер пакетов

На Linux Python принято устанавливать через системный менеджер пакетов дистрибутива, а не скачивать установщик отдельно — это стандартный, официально рекомендуемый способ.

Debian / Ubuntu

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
```

Fedora

```
sudo dnf install python3 python3-pip
```

Arch Linux

```
sudo pacman -S python python-pip
```

Зачем отдельно ставить python3-venv?

На Debian/Ubuntu модуль для создания виртуальных окружений иногда выносят в отдельный пакет `python3-venv` — без него команда `python3 -m venv` (раздел 2.11) не работает. Если вы планируете использовать виртуальные окружения (а мы будем), поставьте его сразу.

Проверка установки

bash

```
python3 --version  
pip3 --version
```

Почему это так строго на Linux

Сама операционная система на Linux нередко написана частично на Python и использует системный интерпретатор для своих инструментов (менеджера пакетов, системных утилит). Если вы командой `pip install` без окружения замените версию библиотеки, от которой зависит сама система, — можно сломать что-то важное в ОС. PEP 668 — это общепромышленный стандарт, который защищает именно от этого сценария, а не прихоть конкретного дистрибутива.

Практический вывод

На Linux правило «всегда работайте внутри виртуального окружения» — не совет, а во многих случаях техническое требование. Хорошая новость: как только вы освоите виртуальные окружения (раздел 2.10–2.12), это правило станет привычкой, а не препятствием.

Какой Python действительно запущен

Один и тот же вопрос спасёт вас от половины будущих проблем с окружениями: «а какой именно интерпретатор сейчас работает?»

Если на компьютере может быть несколько Python (системный + установленный вами, а позже ещё и виртуальные окружения), рано или поздно встанет вопрос: **какой именно Python сейчас запускается командой `python`**? Хорошая новость — сам Python умеет отвечать на этот вопрос.

`sys.executable` — самый честный ответ

Каждый запущенный интерпретатор Python знает точный путь к своему собственному исполняемому файлу. Он хранится в переменной `sys.executable`.

```
python
```

```
import sys
print(sys.executable)
print(sys.version)
```

Запустите это двумя способами и сравните результат — из обычного терминала и (когда дойдём до VS Code, раздел 2.8) из встроенного терминала редактора. Если пути отличаются — значит, в этих двух местах

запускаются *разные* интерпретаторы, и это многое объясняет, если пакет «установлен», а импорт всё равно падает (см. диагностику в разделе 2.16).

which / where — короткая версия из терминала

Не запуская Python вовсе, можно спросить у самого shell, какой файл он найдёт первым по PATH (вспомните раздел 2.2):

```
macOS / Linux
```

```
which python3
```

```
Windows (PowerShell)
```

```
where.exe python
```

Лаборатория: «два Python»

Если на вашем компьютере одновременно есть системный Python и Python с `python.org`, выполните `which python3` (или `where.exe python`) и сравните путь с тем, что печатает `sys.executable` внутри Python. Они должны совпадать — если нет, значит переменная PATH ссылается не на тот интерпретатор, который вы ожидали.

Официальный источник

Подробнее про `sys.executable` и `sys.version` — в модуле `sys` на `docs.python.org`.

VS Code: установка и расширения

VS Code сам по себе не понимает Python — эту способность ему дают расширения. Разберём, что ставить и зачем каждое из них нужно.

VS Code (Visual Studio Code) — бесплатный редактор кода от Microsoft. Сам по себе, «из коробки», он ничего не знает о Python — это универсальный текстовый редактор для десятков языков. Понимание Python ему дают **расширения** (extensions) — отдельные маленькие плагины, которые вы устанавливаете поверх редактора.

Расширение ≠ пакет

Это разные вещи, хоть слова и похожи. **Расширение VS Code** — плагин, который устанавливается в сам редактор и добавляет ему новые возможности (подсветку синтаксиса, автодополнение). **Пакет Python** (раздел 2.12) — библиотека кода, которую вы устанавливаете через `pip` внутрь конкретного окружения, чтобы использовать её функции в своей программе. Расширения не видят и не заменяют пакеты, и наоборот.

Установка VS Code

Скачайте установщик с официального сайта **code.visualstudio.com** для вашей операционной системы и пройдите обычный мастер установки — здесь нет особых ловушек вроде флажка PATH из раздела 2.3.

Расширение Python — и что оно ставит само

Откройте вкладку Extensions (значок из четырёх квадратов на боковой панели, или `Ctrl+Shift+X`) и найдите расширение **Python** от Microsoft.

Страница расширения Python (Microsoft) в VS Code Marketplace

Официальное расширение Python от Microsoft на VS Code Marketplace — свыше 232 млн установок.

Когда вы устанавливаете это расширение, VS Code Marketplace автоматически подтягивает ещё три расширения вместе с ним:

- **Pylance** — быстрый анализатор кода (автодополнение, подсказки типов, переход к определению);
- **Python Debugger** — отдельный движок для пошаговой отладки;
- **Python Environments** — новый интерфейс для создания и переключения виртуальных окружений прямо из VS Code.

Расширение Jupyter

Если вы планируете работать с блокнотами (notebooks, файлы `.ipynb` — с ними мы уже встречались в интерактивной практике этого курса), понадобится расширение **Jupyter**.

Страница расширения Jupyter в VS Code Marketplace

Официальное расширение Jupyter от Microsoft — свыше 108 млн установок.

Расширение Jupyter — это не сам Jupyter

Разработчики честно предупреждают на странице расширения: это **не ядро Jupyter** (not a Jupyter kernel). Расширение — это только интерфейс внутри VS Code; чтобы реально запускать блокноты, нужно ещё и окружение Python с установленным пакетом `jupyter` (раздел 2.15) — точно так же, как редактор и интерпретатор разделены в разделе 2.1.

Расширение Ruff

Ruff — стремительно ставший стандартом инструмент проверки и форматирования кода от Astral Software (создателей uv, раздел 2.13). Он заменяет собой сразу несколько старых инструментов — Flake8, Black, isort — одним быстрым.

Страница расширения Ruff в VS Code Marketplace

Официальное расширение Ruff от Astral Software — свыше 4,3 млн установок.

Что ставить: сводная таблица

Расширение	Издатель	Зачем нужно	Статус
Python	Microsoft	Базовая поддержка языка: подсветка, запуск, отладка, выбор интерпретатора	Обязательно
Pylance	Microsoft	Быстрый анализ кода, автодополнение, подсказки типов	Ставится автоматически вместе с Python
Python Debugger	Microsoft	Отдельный движок пошаговой отладки (breakpoints, шаг за шагом)	Ставится автоматически вместе с Python

Расширение	Издатель	Зачем нужно	Статус
Python Environment s	Microsoft	Новый интерфейс создания и переключения виртуальных окружений	Ставится автоматически вместе с Python
Jupyter	Microsoft	Работа с блокнотами .ipynb внутри VS Code	Рекомендуется
Ruff	Astral Software	Мгновенная проверка стиля и автоформатирование кода	Рекомендуется

Минимальный набор для этого курса

Достаточно поставить одно расширение **Python** — оно автоматически подтянет Pylance, Python Debugger и Python Environments. Jupyter и Ruff — по желанию, но мы рекомендуем оба.

VS Code: интерпретатор и рабочий процесс

Расширение установлено — но VS Code ещё не знает, какой Python использовать для вашего конкретного проекта. Разберём, как это указать и как это устроено.

Установленное расширение Python ещё не знает, *какой именно* интерпретатор использовать для вашего проекта — особенно если на компьютере их несколько (системный, с python.org, а скоро ещё и виртуальные окружения). Это нужно указать явно — один раз на проект.

Выбор интерпретатора

Откройте папку проекта в VS Code, затем откройте любой файл `.py`. В правом нижнем углу окна появится надпись с текущей выбранной версией Python — нажмите на неё (или откройте палитру команд `Ctrl+Shift+P` / `Cmd+Shift+P` и наберите «Python: Select Interpreter»).

Появится список всех интерпретаторов, которые VS Code нашёл на компьютере — с полными путями к каждому. Выберите нужный.

Проверка боем

Выбранный интерпретатор должен совпадать с тем путём, что печатает `sys.executable` (раздел 2.6), если запустить файл через VS Code. Откройте встроенный терминал VS Code (`Ctrl+``) и сравните.

USER-настройки и WORKSPACE-настройки

У VS Code есть два уровня настроек с одинаковыми именами полей, но разным охватом:

- **User settings** — применяются ко всем проектам, которые вы открываете на этом компьютере;
- **Workspace settings** — применяются только к текущей открытой папке (проекту) и хранятся в файле `.vscode/settings.json` внутри самого проекта.



Когда вы выбираете интерпретатор через «Select Interpreter», выбор привязывается именно к этому проекту, а не «навсегда» ко всему компьютеру — у разных проектов на компьютере вполне может быть разный Python. Технически расширение **Python Environments** (мы ставили его вместе с расширением Python в разделе 2.7) хранит это назначение в `.vscode/settings.json` проекта — но не как прямой путь к файлу, а как ссылку на нужное окружение, которую расширение само находит заново при каждом открытии проекта:

`.vscode/settings.json`

```
{
  "python-envs.pythonProjects": [
    { "path": ".", "envManager": "ms-
python.python:venv" }
  ]
}
```

А как же `python.defaultInterpreterPath`?

Более старая настройка

`python.defaultInterpreterPath` — это не запись о вашем выборе, а **запасной вариант**: расширение читает её только тогда, когда для проекта ещё не назначено никакое окружение никаким другим способом. Ручной выбор через «Select Interpreter» её не создаёт и не изменяет — поэтому не удивляйтесь, если не найдёте эту строку в своём `settings.json` даже после того, как выбрали интерпретатор.

Встроенный терминал

Терминал внутри VS Code (`Ctrl+``) — это тот же самый shell, что и обычный терминал операционной системы (раздел 2.2), просто открытый прямо внутри редактора, в папке проекта. Когда у проекта выбрано виртуальное окружение, VS Code обычно **автоматически активирует его** в новом встроенном терминале — вы увидите `(.venv)` перед приглашением командной строки (подробнее в разделе 2.11).

Запуск и отладка файла

Кнопка  («Run Python File») в правом верхнем углу запускает текущий файл именно тем интерпретатором, который выбран для проекта. Кнопка отладки (значок с жуком) делает то же самое, но позволяет ставить точки

останова (breakpoints) — кликом слева от номера строки — и останавливать выполнение построчно, чтобы увидеть текущие значения переменных.

«Run» использует не тот Python»

Если запуск файла через VS Code ведёт себя иначе, чем запуск того же файла в обычном терминале — почти всегда причина в том, что выбран не тот интерпретатор. Откройте «Select Interpreter» ещё раз и сравните выбранный путь с тем, что печатает `sys.executable` (раздел 2.6).

PyCharm: проект, интерпретатор, окружение

Специализированная IDE для Python — с другой моделью работы, чем у VS Code, но теми же базовыми идеями внутри.

PyCharm (компания JetBrains) — специализированная среда разработки (IDE) именно для Python, в отличие от VS Code, который универсален и понимает Python через расширения. Начиная с версии 2025.1 PyCharm — это **единый продукт**: прежнего деления на Community- и Professional-редакции больше нет. Основные, «повседневные» возможности (включая поддержку Jupyter-блокнотов) бесплатны для всех, а платная подписка **Pro** добавляет более продвинутые инструменты (например, для веб-разработки и баз данных); при первой установке каждому пользователю даётся бесплатный пробный период Pro. Для этого курса полностью достаточно бесплатных возможностей.

Если встретите «PyCharm Community» в старых материалах

До объединения в 2025 году у PyCharm действительно были отдельные редакции Community и Professional — вы ещё можете встретить эти названия в старых статьях и видео. Суть это не меняет: бесплатные базовые возможности сегодняшнего PyCharm — прямой преемник прежней Community-редакции.

IDE ≠ интерпретатор

Как и с VS Code (раздел 2.1): PyCharm сам не является Python-интерпретатором. Это программа, которая находит на компьютере уже установленный интерпретатор (или помогает создать новое виртуальное окружение) и использует его для запуска и анализа вашего кода.

Проект и его интерпретатор

В PyCharm единица работы — **проект** (Project): папка с кодом плюс настройки, привязанные именно к ней. При создании нового проекта PyCharm сразу спрашивает, какой интерпретатор использовать, и предлагает удобную кнопку — создать новое виртуальное окружение специально для этого проекта (мы разберём, что это значит, в разделе 2.10).

Изменить интерпретатор позже можно через **Settings → Project → Python Interpreter** — там же виден полный список всех пакетов, установленных в текущее окружение, что удобно для быстрой проверки.

Плагины PyCharm

Как и у VS Code, у PyCharm есть система плагинов — но большая часть базовой поддержки Python в нём уже встроена «из коробки», без установки дополнительных расширений. Через **Settings → Plugins** можно добавить

поддержку других языков и инструментов; поддержка Jupyter-блокнотов в единый продукт PyCharm тоже входит бесплатно.

VS Code или PyCharm?

	VS Code	PyCharm (бесплатные возможности)
Что это	Универсальный редактор + расширения	IDE специально для Python
Поддержка Python	Через расширения (раздел 2.7)	Встроена изначально
Вес и скорость запуска	Легче	Тяжелее
Другие языки	Отлично — десятки языков	В основном Python (+плагины)
Для этого курса	Рекомендуем	Тоже подходит

Какой выбрать

Оба варианта — правильный выбор, и оба используются профессионалами. Если вы уже работали в VS Code — оставайтесь в нём. Если хочется среды, «заточенной» только под Python из коробки, — попробуйте PyCharm: для этого курса хватит его бесплатных возможностей, без подписки Pro. Материалы курса не зависят от выбора редактора.

Виртуальные окружения — зачем они нужны

Прежде чем вводить команду создания окружения — разберём проблему, которую оно решает. Без этого команда — просто ещё одна строчка для заучивания наизусть.

Прежде чем показывать команду для создания виртуального окружения, разберём, зачем оно вообще нужно — на конкретной проблеме, с которой рано или поздно сталкивается каждый.

Представьте такую ситуацию

Вы работаете над двумя проектами на одном компьютере:

- **Проект А** — старый учебный проект, который использует библиотеку `requests` версии 2.25 (написан давно, и обновлять его пока незачем);
- **Проект Б** — новый проект, для которого нужна свежая `requests` версии 2.32 — там есть функции, которых не было в старой версии.

Если бы существовал только один, общий на весь компьютер Python со своим единственным набором пакетов — установить сразу обе версии одной и той же библиотеки было бы невозможно. Установка новой версии для проекта Б автоматически сломала бы проект А.

Это не гипотетический сценарий

Ровно эта проблема — главная причина, по которой виртуальные окружения стали стандартом в мире Python. Без них любые два проекта на одном компьютере вынуждены делить одни и те же версии всех библиотек — а любое обновление одного проекта рискует тихо сломать другой.

Решение: своя копия окружения для каждого проекта

Виртуальное окружение (virtual environment, сокращённо `venv`) — это лёгкая, изолированная «копия» интерпретатора Python вместе с собственным, отдельным набором установленных пакетов, привязанная к одному конкретному проекту.



Технически это не полная копия самого интерпретатора (не тратится гигабайт места на каждый проект) — `venv` переиспользует основной установленный Python, но заводит для проекта отдельную, независимую папку с пакетами. Для вас как для разработчика разница неощутима: внутри активированного окружения пакеты, установленные для одного проекта, попросту не видны другому.

Аналогия

Виртуальное окружение — как отдельный, промаркированный ящик с инструментами для каждого проекта, а не одна общая полка на всех. Взяли молоток из ящика проекта А — проект Б об этом даже не узнает.

Не только про конфликты версий

У виртуальных окружений есть и другие практические плюсы:

- **Воспроизводимость** — список пакетов проекта можно сохранить в файл и точно восстановить такое же окружение на другом компьютере;
- **Чистота** — удалить окружение проекта так же просто, как удалить одну папку, не затрагивая остальную систему;
- **Безопасность системы** — на macOS и особенно на Linux (раздел 2.5) это ещё и способ вообще не трогать системный Python.

В следующем разделе — команда для создания окружения и разбор того, что она реально создаёт на диске.

Создаём .venv

От команды — к тому, что реально появляется на диске, и как включить и выключить окружение.

Теперь, когда понятно зачем — создадим первое виртуальное окружение. Модуль `venv` входит в стандартную библиотеку Python — отдельно ставить ничего не нужно (кроме Linux, где иногда требуется системный пакет `python3-venv`, см. раздел 2.5).

Создание окружения

Откройте терминал в папке вашего проекта и выполните:

```
Windows / macOS / Linux
```

```
python -m venv .venv
```

Что означает точка перед именем

Имя `.venv` начинается с точки — на macOS и Linux это стандартное соглашение для «скрытых» файлов и папок: обычный файловый менеджер и команда `ls` без флагов их не показывают, чтобы не загромождать список служебными файлами. Папку можно назвать и без точки (например, просто `venv`), но `.venv` — самое распространённое соглашение, и его же по умолчанию узнают VS Code и PyCharm.

Что появилось на диске

Команда создала папку `.venv` со своей внутренней структурой:

```
.venv/ (macOS / Linux)
```

```
.venv/
├── bin/
│   ├── python -> /usr/local/bin/python3.14
│   ├── pip
│   └── activate
├── lib/
│   └── python3.14/site-packages/
└── pyvenv.cfg
```

```
.venv\ (Windows)
```

```
.venv\  
├── Scripts\  
│   ├── python.exe  
│   ├── pip.exe  
│   └── activate.bat  
├── Lib\  
│   └── site-packages\  
└── pyvenv.cfg
```

Главное здесь — `site-packages` : именно сюда попадут пакеты, которые вы установите для этого проекта, и именно эта папка изолирована от других проектов (раздел 2.10). Файл `pyvenv.cfg` хранит ссылку на «родительский» интерпретатор, из которого было создано окружение.

Активация окружения

Само по себе создание окружения ещё не значит, что оно используется — его нужно **активировать** в текущем терминале:

```
macOS / Linux (bash/zsh)
```

```
source .venv/bin/activate
```

```
Windows (PowerShell)
```

```
.venv\Scripts\Activate.ps1
```

```
Windows (cmd.exe)
```

```
.venv\Scripts\activate.bat
```

После активации приглашение командной строки изменится — перед ним появится `(.venv)`. Это визуальный сигнал: теперь команды `python` и `pip` в этом терминале указывают именно на интерпретатор и пакеты этого окружения, а не на системный или глобально установленный Python.

Терминал после активации

```
(.venv) anna@laptop project %
```

Активация — на каждую новую вкладку/окно терминала

Активация действует только в том окне терминала, где вы её выполнили. Открыли новую вкладку терминала — активируйте окружение заново. VS Code, как правило, делает это автоматически для встроенного терминала (раздел 2.8), если для проекта выбран этот интерпретатор.

Деактивация

Чтобы вернуться к обычному, «глобальному» состоянию терминала:

Любая ОС

```
deactivate
```

★ Базовая практика

Создайте и активируйте своё первое окружение

Создайте папку с любым именем, откройте в ней терминал, выполните `python -m venv .venv`, активируйте его командой для вашей ОС и убедитесь, что приглашение командной строки изменилось на `(.venv)`.

Устанавливаем первый пакет

`pip install` — и диагностика самой частой ошибки новичков: «установлено, но почему-то не импортируется».

С активированным окружением (раздел 2.11) можно установить первый пакет. Мы возьмём `requests` — популярную библиотеку для запросов к веб-серверам, её мы позже используем в проектах курса.

`pip install`

Терминал (окружение активировано)

```
(.venv) $ pip install requests
```

`pip` скачивает пакет с **PyPI** (Python Package Index — мы познакомились с ним в главе 1) и распаковывает его прямо в `site-packages` вашего активного окружения (раздел 2.11).

Проверка установки

Терминал

```
pip show requests  
pip list
```

python

```
import requests  
print(requests.__version__)
```

Лаборатория: «Установлено, но import не работает»

Одна из самых частых и самых запутывающих проблем новичков: вы только что установили пакет, видите его в `pip list` — а `import` в коде всё равно выдаёт `ModuleNotFoundError`.

Почти всегда причина одна

Пакет установлен в **одно** окружение, а код запускается через **другое**. Например: вы активировали `.venv` в терминале и поставили туда пакет — но VS Code запускает файл через глобальный интерпретатор, потому что для проекта выбран не тот Python (раздел 2.8).

Порядок диагностики:

1. Проверьте `sys.executable` (раздел 2.6) прямо из места, где падает импорт — какой интерпретатор реально используется?
2. Проверьте, что приглашение терминала показывает `(.venv)` — окружение точно активировано?
3. Выполните `pip show requests` в этом же терминале — пакет точно виден отсюда?
4. Если вы в VS Code — откройте «Select Interpreter» (раздел 2.8) и убедитесь, что выбран путь именно к `.venv`, а не к системному или глобальному Python.

★★ Самостоятельная задача

Найдите и почините «два окружения»

Создайте новое окружение `.venv2` рядом с существующим `.venv` (но не активируйте его). В активном `.venv` установите `requests`. Деактивируйте окружение, активируйте `.venv2` и попробуйте `import requests` — вы должны увидеть `ModuleNotFoundError`. Это тот самый сценарий «установлено, но не импортируется» — только вы теперь понимаете, почему.

pip, pipx, venv, virtualenv, uv

Пять похожих по звучанию инструментов — и очень разные задачи, которые каждый из них решает.

К этому моменту вы уже пользовались `pip` и `venv` (разделы 2.11–2.12). В экосистеме Python есть ещё несколько похожих по звучанию инструментов — разберём каждый отдельно, а не единым списком, чтобы не путать их друг с другом.

pip

pip устанавливает, обновляет и удаляет пакеты в уже существующее активное окружение. Он ничего не создаёт — только наполняет то, что уже есть.

Терминал

```
pip install requests
pip install requests==2.31.0
pip uninstall requests
```

venv

venv — модуль стандартной библиотеки (раздел 2.11), который делает ровно одну вещь: создаёт новое изолированное окружение. Он входит в сам Python — устанавливать его отдельно не нужно.

virtualenv

virtualenv — сторонний пакет, предшественник `venv` (модуль `venv` в стандартной библиотеке во многом вырос именно из него). Он решает ту же задачу, но исторически работает быстрее и поддерживает чуть больше сценариев (например, старые версии Python, где встроенного `venv` ещё не было). Для этого курса вам полностью хватит обычного `venv` — `virtualenv` стоит знать по имени, если встретите его в чужом проекте.

pipx

pipx решает другую задачу — не «пакет для моего проекта», а «программа с интерфейсом командной строки, которой я хочу пользоваться из любой папки». Пример: инструмент форматирования кода `black` — вы не импортируете его в свой код, вы запускаете его как команду. **pipx** автоматически создаёт для каждой такой программы своё отдельное окружение, но делает саму команду доступной глобально, из любой папки — без активации.

Терминал

```
pipx install black
black my_script.py
```

uv

uv (от компании Astral — они же делают Ruff, раздел 2.7) — относительно новый инструмент, который сам себя описывает как единую замену сразу для `pip`, `pip-tools`, `pipx`, `poetry`, `pyenv`, `virtualenv` и других. Его главное преимущество — скорость: **uv** в 10–100 раз быстрее `pip` на типичных операциях, потому что написан на Rust и агрессивно кеширует пакеты.

Терминал

```
uv venv
uv pip install requests
uv run my_script.py
```

Официальный источник

docs.astral.sh/uv — актуальная документация; на момент написания курса стабильная версия — 0.12.5.

Сравнение

Инструмент	Что делает	Устанавливает пакеты	Создаёт окружения
pip	Устанавливает пакеты в текущее активное окружение	Да	Нет
venv	Создаёт изолированные окружения (входит в стандартную библиотеку)	Нет	Да
virtualenv	Более старый и быстрый сторонний аналог venv	Нет	Да
pipx	Устанавливает CLI-приложения на Python глобально, каждое — в своё изолированное окружение	Да (для приложений)	Да (по одному на приложение, автоматически)
uv	Один быстрый инструмент, заменяющий pip, venv, virtualenv, pipx и другие	Да	Да



Что использовать в этом курсе

Мы будем показывать примеры через `pip` и `venv` — это стандарт, который работает везде «из коробки» и на котором проще всего понять, что происходит на самом деле. Если позже вам понравится скорость `uv` — переключиться легко: команды почти зеркальные.

conda, Miniconda, Miniforge, Anaconda

Ещё одна семья инструментов, родом из data science — со своей моделью окружений и своим взглядом на пакеты.

conda — ещё один менеджер пакетов и окружений, изначально созданный для data science и научных вычислений компанией Anaconda, Inc. Его ключевое отличие от `pip`: `conda` умеет устанавливать не только Python-пакеты, но и **непитоновские** зависимости — например, компиляторы, библиотеки CUDA для видеокарт, системные библиотеки для научных вычислений — то, с чем `pip` в одиночку не справляется.

Anaconda, Miniconda, Miniforge — в чём разница

Все три — это *способы получить conda на компьютер*, а не три разных инструмента:



- **Anaconda** — самый «тяжёлый» вариант: сразу ставит conda и несколько сотен популярных пакетов для анализа данных (несколько гигабайт на диске);
- **Miniconda** — официальный минимальный установщик: только сам conda, остальное вы ставите по мере необходимости;
- **Miniforge** — похож на Miniconda, но по умолчанию использует сообщество-репозиторий conda-forge вместо канала по умолчанию от Anaconda, Inc.

Окружения в conda

Терминал

```
conda create -n my-project python=3.14
conda activate my-project
conda install numpy pandas
```

Идея точно та же, что и у `venv` (раздел 2.10) — изолированное окружение на проект. Отличие в деталях: `conda` хранит окружения не в папке проекта, а в общем месте на диске, и вы обращаетесь к ним по имени (`-n my-project`), а не по пути.

Как `conda` и `pip` уживаются вместе

Внутри активированного `conda`-окружения команда `pip` тоже работает — и часто используется для пакетов, которых нет в репозиториях `conda`. Правило хорошего тона: сначала ставьте через `conda install` всё, что доступно этим способом, и только оставшееся — через `pip install`, чтобы не запутать собственную систему разрешения зависимостей `conda`.

Не смешивайте `venv` и `conda` для одного окружения

`conda` и `venv` — два независимых способа создавать изолированные окружения. Не пытайтесь активировать `venv` *внутри* `conda`-окружения (или наоборот) для одного и того же проекта — выберите один инструмент на проект.

Когда выбирать `conda`

Практический совет

Если вы занимаетесь анализом данных, машинным обучением или научными вычислениями и используете библиотеки со сложными непитоновскими зависимостями (например, некоторые версии `PyTorch` с поддержкой GPU) — `conda` часто удобнее. Для этого курса и для большинства обычных проектов на Python вполне достаточно связки `venv` + `pip` из разделов 2.10–2.13.

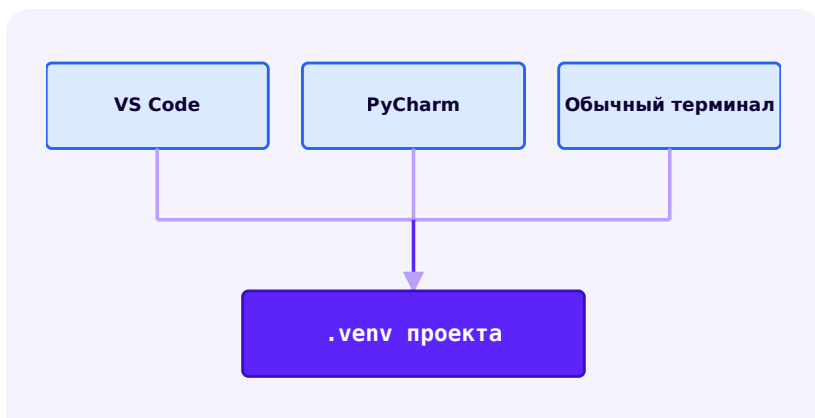
Как IDE, Python и окружение связаны

Собираем все части главы в одну целостную картину — включая частый источник путаницы: разницу между блокнотом и ядром Jupyter.

Мы установили Python (2.3–2.5), редактор кода (2.7–2.9) и научились создавать окружения (2.10–2.12). Теперь соберём всё это в одну целостную картину — как эти части на самом деле связаны друг с другом.

Главная идея: IDE не владеет окружением

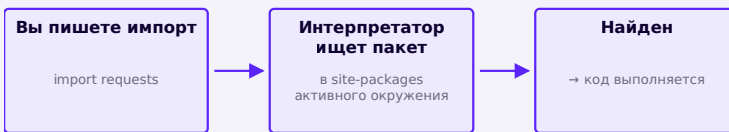
Это стоит проговорить прямо: **VS Code и PyCharm не «содержат» в себе Python и пакеты** — они лишь находят на диске уже существующий интерпретатор (и его окружение) и запускают его от своего имени. Одно и то же окружение `.venv` можно с одинаковым успехом использовать хоть из VS Code, хоть из PyCharm, хоть из обычного терминала — и результат будет одинаковым, потому что реально работает не редактор, а интерпретатор.



Три разных «фронтенда» — одно и то же окружение
позади них

Простая модель процесса

Когда вы запускаете файл — не важно, из какого редактора или терминала — происходит одно и то же:



Простая модель: интерпретатор ищет пакет только
в своём собственном окружении

Блокноты: **notebook** и **kernel** — тоже не одно и то же

С блокнотами (`.ipynb`) добавляется ещё один слой, который часто путает: **notebook** — это сам файл с ячейками кода и текста, а **kernel** (ядро) — это работающий в фоне процесс Python, который реально выполняет код из ячеек. Один и тот же файл блокнота можно открыть и подключить к разным ядрам — например, к ядру из окружения проекта А или проекта Б.

Расширение Jupyter в VS Code (раздел 2.7) прямо предупреждает: оно **не является ядром** само по себе — это лишь интерфейс, который показывает ячейки и отправляет их код выбранному ядру. Чтобы ядро вообще появилось в списке выбора, в соответствующем окружении должен быть установлен пакет `jupyter` :

Терминал (окружение активировано)

```
pip install jupyter
```

После этого при открытии `.ipynb`-файла в VS Code в правом верхнем углу можно выбрать ядро — в списке появится что-то вроде `Python 3.14.7 ('.venv': venv)`, явно показывая, из какого именно окружения оно взято.

«В блокноте другие пакеты, чем в терминале»

Если `import` работает в обычном `.py`-файле, но не в блокноте (или наоборот) — почти всегда для блокнота выбрано другое ядро, то есть другое окружение. Проверьте выбор ядра в правом верхнем углу блокнота так же, как проверяли «Select Interpreter» в разделе 2.8.

Итоговая картина главы

Кто есть кто

- **Интерпретатор Python** — программа, которая реально выполняет код.
- **Виртуальное окружение** — изолированный набор пакетов, привязанный к интерпретатору.
- **VS Code / PyCharm** — редакторы, которые находят и используют интерпретатор + окружение, но не содержат их внутри себя.
- **pip / uv** — устанавливают пакеты в текущее активное окружение.
- **Notebook** — файл с ячейками; **kernel** — процесс Python, который их выполняет.

Типичные проблемы и диагностика

Восемь именованных лабораторий — справочник частых проблем этой главы и способов их решить.

Собрали восемь самых частых проблем этой главы в одном месте — как справочник, к которому можно будет вернуться, когда что-то пойдёт не так. Каждая уже разбиралась по ходу главы; здесь — компактная версия для быстрой диагностики.

Лаборатория 1. «command not found: python»

Симптом: Команда `python` не найдена в терминале.

Причина и решение: Python не установлен, либо не добавлен в PATH (Windows: галочка «Add python.exe to PATH», раздел 2.3), либо на macOS/Linux нужно использовать `python3` вместо `python` (раздел 2.4).

Лаборатория 2. «externally-managed-environment»

Симптом: `pip install` отказывается работать на Linux/macOS.

Причина и решение: Это защита PEP 668 (раздел 2.5): создайте и активируйте виртуальное окружение (раздел 2.11) и ставьте пакеты туда — не в системный Python.

Лаборатория 3. `ModuleNotFoundError` после установки

Симптом: Пакет установлен, но `import` падает.

Причина и решение: Пакет установлен не в то окружение, из которого запускается код. Сверьте `sys.executable` (раздел 2.6) с активным окружением терминала и с интерпретатором, выбранным в VS Code (раздел 2.8) — см. лабораторию раздела 2.12.

Лаборатория 4. VS Code запускает «не тот» Python

Симптом: Поведение при запуске из VS Code отличается от запуска из терминала.

Причина и решение: Проверьте «Select Interpreter» (раздел 2.8) — выбран не тот путь. Сверьтесь с `.vscode/settings.json` проекта.

Лаборатория 5. Забыли активировать окружение

Симптом: Приглашение терминала не показывает `(.venv)`, пакеты «пропали».

Причина и решение: Окружение не активировано в этом окне терминала — активация не сохраняется между вкладками (раздел 2.11). Выполните команду активации для вашей ОС заново.

Лаборатория 6. Два Python — конфликт версий

Симптом: `python --version` показывает не ту версию, что вы только что установили.

Причина и решение: На компьютере несколько интерпретаторов, и PATH находит не тот, что ожидалось (раздел 2.2). Проверьте порядок через `which` / `where.exe` (раздел 2.6).

Лаборатория 7. Блокнот использует не то ядро

Симптом: В `.ipynb` импорт не находит пакет, который точно установлен.

Причина и решение: У блокнота выбрано другое ядро (kernel) — не то окружение, куда вы ставили пакет (раздел 2.15). Смените ядро в правом верхнем углу блокнота.

Лаборатория 8. «pip: command not found» при активном venv

Симптом: Внутри активированного окружения команда `pip` всё равно не находится.

Причина и решение: Редкий случай повреждённого окружения — пересоздайте его: удалите папку `.venv` и выполните `python -m venv .venv` заново (раздел 2.11).

Общий метод диагностики

Почти все проблемы этой главы сводятся к одному вопросу: **какой именно интерпретатор и какое окружение сейчас реально используются?**

Проверяйте это в первую очередь — через `sys.executable` (раздел 2.6), приглашение терминала (`.venv`) и выбор интерпретатора в редакторе (раздел 2.8) — прежде чем подозревать что-то более сложное.

Рекомендации Cartesian School и итоги

Три пути настройки рабочего места — и финальная практика: полный чек-лист сборки рабочего места на вашем собственном компьютере.

Три рекомендованных пути Cartesian School

Все инструменты этой главы — легитимные, рабочие варианты. Вот три сочетания, которые мы рекомендуем в зависимости от ваших целей:

Путь	Редактор	Управление пакетами	Кому подходит
Стандартный	VS Code	venv + pip	Рекомендуем для этого курса — минимум движущихся частей, максимум понимания
Быстрый	VS Code или любой другой	uv	Если хочется скорости и единого инструмента с первого дня
Data science	PyCharm, VS Code или Jupyter	conda / Miniforge	Если вы уже нацелены на анализ данных, ML или научные вычисления

Если сомневаетесь

Начните со **стандартного пути** — VS Code + venv + pip. Это ровно то, что мы использовали во всех примерах этой главы, и на нём проще всего понять, что реально происходит «под капотом». К uv или conda всегда можно перейти позже — идеи от этого не изменятся.

Практика: соберите рабочее место (local-required)

Эта практика — **обязательная для выполнения на вашем собственном компьютере**. В отличие от интерактивных упражнений в браузере из других глав, установку Python нельзя (да и не нужно) эмулировать онлайн — смысл именно в том, чтобы у вас на диске появилось настоящее, работающее рабочее место.

local-required

Пройдите каждый пункт по порядку на своём компьютере и отметьте выполненное. Ничего отправлять на проверку не нужно — это чек-лист для вас самих.

★★★ Обязательная практика главы

Чек-лист рабочего места (18 шагов)

- ☐ Python установлен, и `python --version` (или `python3 --version`) печатает версию 3.x
- ☐ Команда работает из **обычного** терминала операционной системы (не только из редактора)
- ☐ `sys.executable` печатает ожидаемый путь к интерпретатору
- ☐ VS Code (или PyCharm) установлен и открывается без ошибок
- ☐ Расширение Python установлено в VS Code (или подтверждена встроенная поддержка PyCharm)
- ☐ В боковой панели VS Code видно, что вместе с Python подтянулись Pylance, Python Debugger и Python Environments
- ☐ Создана тестовая папка проекта
- ☐ В этой папке выполнено `python -m venv .venv`
- ☐ Окружение `.venv` успешно активировано (в приглашении терминала видно `(.venv)`)
- ☐ В активном окружении выполнено `pip install requests` без ошибок
- ☐ `import requests` работает в интерактивном `python`
- ☐ В VS Code для тестовой папки выбран интерпретатор именно из `.venv` («Select Interpreter», раздел 2.8)
- ☐ Создан файл `main.py` с `import requests`, и запуск через кнопку ▶ в VS Code проходит без ошибок
- ☐ Тот же файл успешно запускается и из встроенного терминала VS Code
- ☐ Поставлена хотя бы одна точка останова, и отладчик VS Code успешно останавливается на ней

- Окружение деактивировано командой `deactivate`, и после этого `import requests` в обычном терминале уже не работает (если `requests` не был установлен глобально) — это подтверждает изоляцию
- Окружение активировано заново
- Вы можете своими словами объяснить разницу между интерпретатором, редактором, `pip` и виртуальным окружением — вслух, партнёру по учёбе или самому себе

Итоги главы

Что мы теперь умеем

- Понимаем семь слоёв между железом и своим кодом — и что интерпретатор, редактор, `pip` и окружение — четыре разные вещи.
- Понимаем, что такое терминал, `shell` и переменная `PATH`, и как компьютер находит команду по имени.
- Установили Python на своей операционной системе — Windows, macOS или Linux — и знаем её особенности (`PATH`-галочка, системный Python, PEP 668).
- Умеем проверять, какой именно интерпретатор реально запущен, через `sys.executable`.
- Настроили редактор кода — VS Code или PyCharm — и умеем выбирать интерпретатор для проекта.
- Понимаем, зачем нужны виртуальные окружения, умеем их создавать, активировать и устанавливать в них пакеты.
- Различаем `pip`, `pipx`, `venv`, `virtualenv` и `uv` — и знаем, для какой задачи каждый из них.
- Знаем, как работает `conda` и семейство Anaconda/Miniconda/Miniforge, и когда его стоит выбрать.
- Умеем диагностировать восемь самых частых проблем установки и настройки.
- Собрали полноценное, работающее рабочее место Python-разработчика на своём компьютере.

Что дальше

С готовым рабочим местом вы полностью подготовлены к главе 3 — там мы начнём писать настоящий, содержательный код.

Python VS Code PyCharm Jupyter PySH

Технологии главы: официальные логотипы использованы для идентификации, без заявления о партнёрстве

ГЛАВА 3 · СТР. 89

Ваша первая программа на Python

Как по-настоящему разговаривать с Python: файлы и их запуск, терминал и командные оболочки, PySH, интерактивный REPL, `print()`/`input()`, имена, ошибки и `traceback`, отладчик, Jupyter — и первый маленький проект.

3.1 Создание и запуск программ на Python

3.2 Терминал, shell и Python REPL

3.3 Семейство командных оболочек

3.4 PySH: Python-first оболочка

3.5 Интерактивный режим Python (REPL)

Ваша оболочка умеет считать

3.6 REPL как инструмент исследования

3.7 Вывод данных с помощью Python

3.8 `input()`: первый диалог

3.9 Имена и значения

3.10 Комментарии и читаемый код

3.11 Режим сценариев IDLE

3.12 Практика: выведите своё имя

3.13 Ошибки, traceback и как их читать

3.14 Лаборатории отладки

3.15 Первый отладчик в IDE

3.16 Notebook и kernel

3.17 Мини-проект и итоги главы

Создание и запуск программ на Python

Программа на Python начинается с обычного текстового файла. Разберём, что это значит на самом деле — и как создать такой файл и запустить его.

Программа на Python — это **обычный текстовый файл**. Ничего волшебного: те же байты, что и в любом `.txt`-файле, только Python умеет их читать и превращать в действия.

Что такое файл программы `.py`

У файла программы есть несколько частей, которые стоит различать:

- **Исходный код** (source code) — сам текст программы, который пишет человек;

- **Имя файла** — например, `privet.py`; расширение `.py` — соглашение, по которому редакторы и сам Python узнают файл с кодом на Python;
- **Папка проекта** — место на диске, где лежит файл (и, возможно, другие файлы того же проекта);
- **Кодировка** — то, как текст хранится в виде байтов. Python по умолчанию читает исходный код в **UTF-8** — универсальной кодировке, которая поддерживает практически любой алфавит, включая кириллицу. Именно поэтому в исходном коде Python можно свободно писать русские строки и комментарии — никаких специальных настроек не нужно.

Файл — это не программа, которая «уже работает»

Пока файл просто лежит на диске, ничего не происходит — это только текст. Программа начинает работать только в момент, когда её ЗАПУСКАЮТ — передают интерпретатору Python на выполнение. Разница между «файл существует» и «файл выполняется» — то же самое различие, что между рецептом на бумаге и настоящим приготовлением блюда.

Текущая папка и путь к файлу

Чтобы запустить файл, shell (раздел 3.2) должен знать, где он лежит. Если вы находитесь *в той же папке*, что и файл, достаточно его имени. Если нет — нужен путь: полный (от корня диска) или относительный (от текущей папки). Мы разбирали путь и текущую папку подробнее в главе 2.

Простая модель: как Python запускает программу



Внутри «Python работает» скрывается ещё один уровень, который полезно знать хотя бы в общих чертах:



Что происходит на самом деле

CPython (реализация Python, которой мы пользуемся) сначала переводит ваш исходный код в промежуточный, более компактный формат — **байт-код** — а затем выполняет именно его на собственной виртуальной машине. Это не значит, что Python «никогда не компилируется» — как раз наоборот: компиляция в байт-код происходит практически всегда, просто незаметно для вас, за долю секунды перед выполнением.

А как же `__pycache__`?

Когда вы позже начнёте *импортировать* свои файлы как модули в другие файлы (главы про функции и модули), вы заметите папку `__pycache__` — туда Python сохраняет уже скомпилированный байт-код, чтобы не пересчитывать его заново при каждом запуске. Для файла, который вы просто запускаете напрямую (как `privet.py` ниже), эта папка обычно не создаётся — не удивляйтесь, если сейчас её не будет видно.

Ваша первая программа

```
privet.py
```

```
print("Привет, Python!")
```

Разберём команду запуска по частям:

```
Терминал
```

```
python privet.py
```

- `python` — исполняемый файл интерпретатора (мы устанавливали его в главе 2);
- `privet.py` — аргумент: какой именно файл выполнить.

После запуска вы должны увидеть:

```
вывод программы
```

```
Привет, Python!
```

Создание и запуск в VS Code

1. Откройте папку проекта через **File → Open Folder** (мы настраивали VS Code в главе 2 — интерпретатор уже должен быть выбран для этой папки).
2. Создайте новый файл `privet.py`.
3. Наберите код и нажмите на треугольник ▷ «Run Python File» в правом верхнем углу — либо откройте встроенный терминал (`Ctrl+``) и наберите `python privet.py`.

Создание и запуск в PyCharm

1. Откройте или создайте проект с выбранным интерпретатором Python (глава 2).
2. Щёлкните правой кнопкой по папке проекта в панели слева → **New → Python File**, назовите его `privet`.
3. Наберите код и запустите через **Run → Run 'privet'** в верхнем меню, либо кликните правой кнопкой в редакторе и выберите **Run**.

Про зелёный треугольник рядом с `if __name__`

В некоторых чужих самоучителях просят искать зелёный треугольник ▷ именно рядом со строкой `if __name__ == "__main__":` — но для такой простой программы, как `privet.py`, эта конструкция вообще не нужна. Пока запускайте файл через **Run** в верхнем меню или клик правой кнопкой — этого достаточно. Что означает `if __name__ == "__main__":`, мы разберём позже, когда до неё дойдёт очередь по-настоящему.

Практика: создайте и запустите свою первую программу

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/03-01/index.html>)

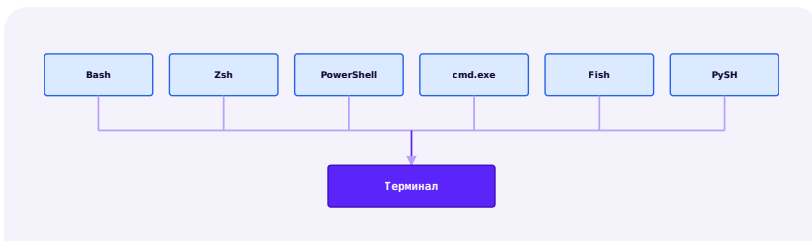
Терминал, shell и Python REPL — это разные вещи

Одна из самых частых причин путаницы у новичков — эти три понятия. Разберём их раз и навсегда.

Мы уже встречали слова «терминал» и «shell» в главе 2 — но, чтобы уверенно двигаться дальше, важно окончательно закрепить это различие и добавить к нему третье понятие: сам **Python REPL**. Путаница между этими тремя вещами — одна из самых частых причин, почему новичкам кажется, что «командная строка» — это что-то одно большое и запутанное.

Три разные вещи

- **Терминал** (terminal) — окно приложения, где вообще происходит текстовый ввод/вывод. Сам по себе терминал ничего не «понимает» — это просто экран и клавиатура.
- **Shell** (командная оболочка) — программа ВНУТРИ терминала, которая читает ваши команды операционной системы: `cd`, `ls`, запуск программ. Bash, Zsh, PowerShell, cmd.exe, Fish, PySH — это всё разные shell.
- **Python REPL** (он же Python Shell) — это отдельная программа (сам интерпретатор Python в интерактивном режиме), которую вы ЗАПУСКАЕТЕ из shell командой `python`. У неё своё собственное приглашение `>>>`, и она понимает уже не команды операционной системы, а код на Python.



Разные shell — один и тот же терминал-«экран»
вокруг них



Отдельный путь: терминал → интерпретатор →
Python REPL

Почему все путают «shell» и «Shell»

Есть досадное историческое совпадение: интерактивный режим Python в англоязычной документации часто называют «Python Shell» — тем же словом, что и обычную командную оболочку (Bash Shell, PowerShell). Это два РАЗНЫХ значения одного и того же слова. Когда сомневаетесь — смотрите на приглашение: `$` или `>` обычно означает обычный shell операционной системы, а `>>>` — именно Python REPL.

Как отличить на глаз

	Обычный shell (Bash/ PowerShell/...)	Python REPL
Приглашение	<code>\$</code> , <code>%</code> или <code>></code>	<code>>>></code>
Понимает	Команды ОС: <code>cd</code> , <code>ls</code> , запуск про- грамм	Код на Python: вы- ражения, присваи- вания, вызовы функций

	Обычный shell (Bash/ PowerShell/...)	Python REPL
Как попасть	Открыть терминал — вы уже внутри	Набрать <code>python</code> внутри shell
Как выйти	Закрыть терминал или <code>exit</code>	<code>exit()</code> или <code>Ctrl+D</code> / <code>Ctrl+Z</code>

Проверьте сами

Откройте терминал и наберите `python`.
Приглашение изменится с обычного (например, `$`)
на `>>>` — это верный признак, что вы только что
перешли из shell операционной системы в Python
REPL. Наберите `exit()`, чтобы вернуться обратно.

Семейство командных оболочек

`cmd.exe`, `PowerShell`, `Bash`, `Zsh`, `Fish` — коротко о том, где вы их встретите и что это вообще такое.

Раз уж мы заговорили про shell — коротко познакомимся с самыми распространёнными. Цель этого раздела не в том, чтобы выучить синтаксис каждой из них, а в том, чтобы узнавать их по имени и понимать, где вы их встретите.

Shell	Где встречается	Что это
<code>cmd.exe</code>	Windows (классическая, «Командная строка»)	Старейшая командная оболочка Windows; всё ещё встречается в старых инструкциях и корпоративных системах

Shell	Где встречается	Что это
PowerShell	Windows (современная по умолчанию)	Более мощная оболочка от Microsoft с объектно-ориентированным подходом к командам; то, что мы использовали в главе 2 для Windows
Bash	Linux (часто по умолчанию), macOS (более старые версии)	Один из самых распространённых shell в мире — почти любой сервер и учебник её знает
Zsh	macOS (по умолчанию с 2019 года), популярна на Linux	Похожа на Bash, но с более удобными возможностями «из коробки» (автодополнение, темы оформления)
Fish	Linux/macOS (по выбору пользователя)	Дружелюбная к новичкам оболочка с подсказками и подсветкой прямо во время набора команды
PySH	Linux/macOS/Windows (устанавливается отдельно)	Python-first оболочка — вместо собственного мини-языка команд использует настоящий Python (раздел 3.4)

Зачем Python-разработчику вообще это знать

Вы не обязаны учить синтаксис всех shell — но полезно узнавать их по приглашению и названию, когда встречаете в чужих статьях, логах CI/CD или инструкциях по установке. Часто инструкция говорит «выполните в Bash» или «в PowerShell» — и теперь вы точно знаете, что имеется в виду.

Логотип PySH

PySH — последняя строка таблицы выше, и ей посвящён следующий раздел: единственная оболочка здесь, написанная как Python-first альтернатива, а не как ещё один вариант классического shell-языка.

Мы не будем подробно разбирать синтаксис каждой оболочки — это не входит в задачи курса Python. Вместо этого в следующем разделе познакомимся с PySH — оболочкой, которая устроена иначе: вместо своего мини-языка команд она использует настоящий Python.

PySH: Python-first оболочка

Ещё один взгляд на командную строку — что если вместо мини-языка команд использовать прямо настоящий Python?

Логотип PySH

Официальный логотип проекта PySH (pysh-shell.com) — использован для идентификации технологии, обсуждаемой в этом курсе, без заявления о партнёрстве или спонсорстве.

PySH — это реальная, установленная на этом компьютере программа, и все примеры на этой странице — настоящий, проверенный вывод именно этой установки, а не выдуманный текст.

Главная страница официального сайта pysh-shell.com

pysh-shell.com — официальный сайт проекта PySH («The Python-Native Automation Platform»). Версия на сайте (0.8.2) совпадает с версией, установленной на этом компьютере.

Только этот проект

В мире существует несколько пакетов с похожим названием «pysh». Этот раздел — именно про проект pysh-shell.com (пакет на PyPI называется `pysh-shell`, команда — `pysh`), а не про другие одноимённые инструменты.

Что такое PySH

PySH — это интерактивная командная оболочка (shell — раздел 3.3), написанная на чистом Python и позиционирующая себя как **Python-first альтернатива** традиционным shell вроде Bash. Официальное описание с сайта: «The Python-Native Automation Platform for Developers, System Administrators, and DevOps Engineers».

Два подхода к командной строке



Традиционный подход: терминал → shell-язык → shell-скрипт



Традиционный shell отлично подходит для коротких интерактивных команд — но чем сложнее становится скрипт, тем сильнее чувствуется нехватка настоящего языка программирования: структурных исключений, тестов, читаемых модулей. PySH предлагает другое направление: писать автоматизацию сразу на Python, не переключаясь между двумя разными языками.

Не путайте с полной заменой Bash

PySH сам честно указывает в официальной документации: это **не** полная POSIX-совместимая замена `/bin/sh`, не клон Zsh и не гарантированно совместим с любым существующим Bash-скриптом. Это самостоятельное направление с собственным, Python-ориентированным подходом — а не притворство, что внутри спрятан настоящий Bash.

Запуск PySH — реальный вывод

На этом компьютере PySH уже установлен. Проверим это теми же командами, которыми вы проверяли бы любую другую программу:

Терминал

```
command -v pysh
pysh --version
```

Реальный вывод

```
/usr/bin/pysh
pysh 0.8.2
```

Запустим PySH и посмотрим на настоящий баннер и приглашение:

Терминал — реальная сессия PySH

```
PySH 0.8.2 | Python 3.13.5 | GPL-2.0-only
System: Debian GNU/Linux 13 | Kernel 6.12.101 | 11th
Gen Intel Core i3-1115G4 | RAM 19 GiB
Type 'exit' or press Ctrl+D to quit.
└─ astra@soi — [~/Projects/Python_001] — git:feat/
curriculum-v2-chapter-03
| py3.13 · uv0.11.24 · ruff0.15.20 · rust1.85.0 ·
node26.3.1 · npm11.16.0
└─>
```

О скриншотах на этой странице

Эта страница показывает настоящий, дословно скопированный вывод реальной установленной PySH — терминальный вывод воспроизведён как текст, а не как фотография экрана, потому что окружение, в котором собирался этот курс, технически не может делать снимки экрана графических окон. Каждая команда на этой странице была выполнена по-настоящему.

Приглашение PySH — двухстрочное и информативное: имя пользователя, хост, текущая папка, ветка git — и вторая строка с версиями инструментов проекта (Python,

uv, ruff, и другие, если они найдены). Это тот же дух, что и у «продвинутых» тем оформления для Bash/Zsh, только встроенный по умолчанию.

PySH понимает обычные команды

Несмотря на Python-first философию, привычные команды тоже работают — PySH умеет запускать внешние программы, пайпы и базовые операторы, как обычный shell:

Реальная сессия

```
└─> pwd
/home/astra/Projects/Python_001
└─> ls scripts | head -3
build_book.py
build_chapter_01.py
build_chapter_02.py
```

Python прямо в командной строке

А вот это уже отличает PySH от обычного shell: команда `py` выполняет настоящий Python прямо на месте, в постоянном (сохраняющемся между вызовами) контексте:

Реальная сессия

```
└─> py print("hello from python")
hello from python
└─> py x = 10
└─> py print(x)
10
```

Переменные и импорты сохраняются между отдельными вызовами `py` — `x` из первой команды доступен во второй. Для более длинного кода есть и многострочный блок:

Реальная сессия

```

└─> py {
import os
targets = [p for p in os.environ.get("PATH",
 "").split(":") if p]
print(f"PATH entries: {len(targets)}")
}
PATH entries: 43

```

Есть и третий, более «тяжёлый» режим — полноценный вложенный Python REPL со своим `>>>` приглашением, который включается командой `#py` прямо из обычного приглашения PySH:

Реальная сессия

```

└─> #py
PySH Python Command Execution Layer | GPL-2.0-only
Python 3.13.5
Type #help for commands, Ctrl+D or #exit to return to
PySH.

>>> 2 + 2
4
>>> print("Привет, PySH!")
Привет, PySH!
>>> #exit

```

Bash-скрипт и PySH-подход: один пример

Небольшая иллюстрация разницы в стиле — не чтобы объявить один способ «плохим», а чтобы показать, откуда берётся Python-first идея.

Задача: посчитать файлы в текущей папке

Bash

```
count=$(ls | wc -l)
echo "Файлов: $count"
```

PySH (py-блок)

```
py {
    import os
    count =
len(os.listdir('.'))
    print(f'Файлов:
{count}')
```

Что использовать сегодня: Для короткой разовой команды Bash по-прежнему быстрее набрать. Но как только автоматизация обростает условиями, обработкой ошибок и тестами — читаемый Python начинает выигрывать. PySH даёт этот путь, не заставляя переключаться в отдельный .py-файл.

Локальная лаборатория: попробуйте PySH (необязательно)

Это — необязательное упражнение для вашего собственного компьютера. Оно не проверяется автоматически и не является обязательным условием для прохождения курса Python — PySH стоит **знать**, а пользоваться им дальше или нет — решать вам.

Необязательно · локально на вашем компьютере

Чек-лист: первый запуск PySH

- ❑ Установите PySH: `pip install pysh-shell` (или найдите готовую установку через `command -v pysh`)
- ❑ Запустите `pysh` и посмотрите на настоящий баннер и приглашение
- ❑ Выполните одну простую команду, например `pwd` или `ls`
- ❑ Попробуйте команду `py print("hello")` — Python прямо из приглашения PySH
- ❑ Выйдите командой `exit` или `Ctrl+D`

Официальные источники

pysh-shell.com — сайт проекта; pypi.org/project/pysh-shell — страница пакета (`pip install pysh-shell`); полная документация — в репозитории проекта, ссылка указана на PyPI-странице.

Интерактивный режим Python (Python REPL)

Второй способ работать с Python — вводить код по одной строке и сразу видеть результат. Разберём, как он устроен на самом деле.

Кроме запуска файлов (раздел 3.1), у Python есть второй режим работы — **интерактивный**. В нём вы вводите код по одной строке и сразу видите результат, без файла и без явного «запуска» в привычном смысле. Мы уже называли это Python REPL или Python Shell в разделе 3.2 — теперь разберём его по-настоящему подробно.

Запуск

Откройте терминал (в VS Code, PyCharm или отдельным приложением — раздел 3.2) и наберите:

```
Терминал
```

```
python
```

Вы увидите что-то вроде:

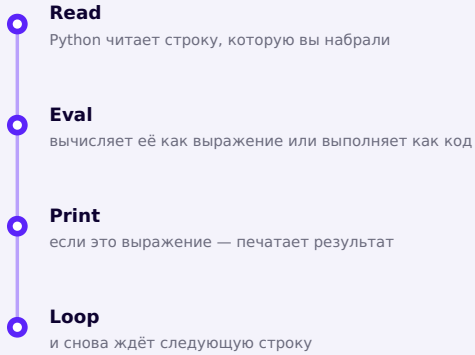
```
Реальный вывод
```

```
Python 3.14.7 (main, Aug 5 2026, 00:00:00) [GCC ...]  
on linux  
Type "help", "copyright", "credits" or "license" for  
more information.  
>>>
```

Приглашение `>>>` означает: Python готов и ждёт вашу команду.

Что означает R-E-P-L

REPL — сокращение от **Read-Eval-Print Loop** («прочитать — выполнить — напечатать — повторить»). Именно так и работает интерактивный режим:



REPL = Read - Eval - Print - Loop

Первые примеры

Python REPL

```
>>> 2 + 2
4
>>> "Py" + "thon"
'Python'
>>> name = "Анна"
>>> name
'Анна'
```

Обратите внимание: строка `name = "Анна"` НЕ показала результат — присваивание само по себе не является выражением, значение которого нужно печатать. А вот следующая строка, где мы просто написали `name`, — это уже выражение, и REPL сразу же его вычислил и напечатал.

Почему REPL печатает сам, а файл — нет

REPL автоматически показывает результат *последнего вычисленного выражения* в каждой введённой строке — это удобство именно интерактивного режима. Обычный .py -файл выполняется целиком и молча — если нигде явно не вызвать `print()`, на экране ничего не появится, даже если где-то в середине файла встретилось выражение вроде `2 + 2`.

Ваша оболочка умеет считать

Раз REPL сразу показывает результат каждой строки, его удобно использовать как быстрый калькулятор — без единого `print()`:

Python REPL

```
>>> 2 + 2
4
>>> 10 / 4
2.5
>>> 3 * 7
21
```

Это работает только в самой оболочке: в обычном .py -файле строка `2 + 2` сама по себе ничего не выведет.

Практика: интерактивный режим в ноутбуке

Интерактивный ноутбук прямо в браузере — сравните ячейку Jupyter с Python REPL

[Открыть практику](https://www.cartesianschool.org/practice/03-02/index.html) → (https://www.cartesianschool.org/practice/03-02/index.html)

Практика: используйте Python как калькулятор

Интерактивный ноутбук прямо в браузере — сложение, вычитание, умножение, деление

[Открыть практику → \(https://www.cartesianschool.org/practice/03-03/index.html\)](https://www.cartesianschool.org/practice/03-03/index.html)

Два вида приглашения

- `>>>` — **основное** приглашение: Python ждёт начало новой команды;
- `...` — **приглашение продолжения**: Python видит, что текущая команда ещё не закончена (например, открыта скобка или начат блок), и ждёт остальное.

Python REPL — многострочный пример

```
>>> total = (  
...     2  
...     + 2  
... )  
>>> total  
4
```

Как выйти

Наберите `exit()` и нажмите Enter. Также работают клавиатурные сочетания: `Ctrl+Z` затем Enter на Windows, или `Ctrl+D` на macOS/Linux.

REPL как инструмент исследования

`type()`, `help()`, `dir()` — как задавать Python вопросы прямо во время работы, и чем в итоге отличаются script, REPL, notebook и IDE.

REPL — это не только способ запуска кода, но и удобный инструмент для того, чтобы задавать Python вопросы прямо во время работы. Три встроенные команды особенно полезны для исследования.

type() — какого типа это значение?

Python REPL

```
>>> type(42)
<class 'int'>
>>> type("Python")
<class 'str'>
```

help() — встроенная справка

Python REPL

```
>>> help(print)
```

Появится описание функции прямо из официальной документации Python — работает даже без интернета, потому что справка встроена в сам интерпретатор.

dir() — что вообще есть у значения

Python REPL

```
>>> dir(42)
```

Длинный список — это нормально

`dir()` покажет длинный список имён вроде `__add__`, `__class__` и другие — сейчас понимать каждое из них не нужно. Сама привычка «спросить у Python напрямую» гораздо важнее, чем немедленное понимание каждого имени в ответе.

Практика: предскажите и проверьте

Интерактивный ноутбук прямо в браузере — `type()`, `help()`, `dir()` на практике

[Открыть практику → \(https://www.cartesianschool.org/practice/03-05/index.html\)](https://www.cartesianschool.org/practice/03-05/index.html)

Script vs REPL vs Notebook vs IDE

Мы уже встречали все четыре понятия по отдельности — соберём их в одну таблицу, чтобы видеть общую картину:

	Script (.py)	REPL	Notebook (.ipynb)	IDE/редактор
Что это	Сохранённый файл с кодом	Временный интерактивный сеанс	Ячейки кода + текст + результаты	Инструмент для редактирования/запуска/отладки
Хранится?	Да, на диске	Нет — исчезает при выходе	Да, в файле .ipynb	—
Лучше всего для	Готовых программ	Быстрых экспериментов	Обучения, исследования данных	Написания и отладки кода
Python-язык?	Да	Да	Да	Нет — это инструмент, использующий Python

А как насчёт браузерной практики этого курса?

Интерактивные упражнения, которые вы проходите прямо на этой странице курса, — это ещё один, пятый вариант: настоящий Python выполняется у вас в браузере через технологию **Pyodide** (подробнее — в разделе 3.16 про notebook и kernel). Это тоже полноценный Python — просто с другой средой выполнения, без установки на компьютер.

Вывод данных с помощью Python

`print()` умеет больше, чем кажется на первый взгляд — несколько значений сразу, свои разделители и управление концом строки.

В файле `.py` ничего не выводится на экран само по себе — нужно явно попросить об этом командой `print()`. Мы уже пользовались ей в главе 1 и в REPL (раздел 3.5), теперь разберём подробнее — это первый настоящий инструмент, которым мы будем пользоваться в каждой программе этого курса.

Несколько значений в одной команде

```
print_demo.py
print("Возраст:", 10, "лет")
```

Через запятую можно передать сколько угодно значений — `print()` сам вставит между ними пробел и выведет всё в одну строку.

Параметр `sep` — разделитель

По умолчанию значения разделяются пробелом. Это можно изменить параметром `sep`:

```
print_sep.py
```

```
print("2024", "01", "15", sep="- ")
```

вывод программы

2024-01-15

Параметр `end` — чем закончить строку

По умолчанию каждый `print()` завершается переводом строки (`\n`) — поэтому следующий вызов начинается с новой строки. Параметр `end` позволяет это изменить:

```
print_end.py
```

```
print("Загрузка", end="")
print("...", end="")
print("готово!")
```

вывод программы

Загрузка...готово!

Пустая строка

Вызов `print()` без аргументов просто выводит пустую строку — удобно, чтобы отделить блоки текста друг от друга.

Официальный источник

Полное описание всех параметров `print()` — в разделе «Built-in Functions» на docs.python.org.

Практика: `sep`, `end` и форматирование вывода

Интерактивный ноутбук прямо в браузере — параметры `print()` на практике

[Открыть практику → \(https://www.cartesianschool.org/practice/03-04/index.html\)](https://www.cartesianschool.org/practice/03-04/index.html)

`input()`: программа начинает разговаривать с человеком

Первая команда, которая делает программу по-настоящему интерактивной.

До сих пор наши программы только говорили — теперь научим их слушать. `input()` — первая команда, которая делает программу по-настоящему интерактивной: она приостанавливает выполнение и ждёт, пока человек что-то напечатает.

Первый диалог

`dialog.py`

```
name = input("Как вас зовут? ")  
print("Привет,", name)
```

При запуске в терминале появится приглашение, программа будет ждать ввода, а затем поприветствует вас:

Терминал

```
Как вас зовут? Анна  
Привет, Анна
```

Что происходит по шагам

○ **Программа печатает приглашение**
`input("Как вас зовут? ")`

○ **Программа ждёт**
выполнение приостановлено

○ **Пользователь печатает текст**
и нажимает Enter

○ **`input()` возвращает текст**
как строку

○ **Имя указывает на этот текст**
`name = ...`

○ **`print()` использует значение**
`print("Привет,", name)`

Что происходит при вызове `input()`

`input()` всегда возвращает текст

Даже если пользователь вводит цифры, `input()` возвращает **строку** (текст), а не число. Чтобы получить настоящее число, текст нужно явно преобразовать — например, командой `int(...)`. Подробно числа и преобразование типов мы разберём в следующей главе; здесь достаточно запомнить сам факт: результат `input()` — всегда текст.

Пример преобразования (забегая вперёд)

```
age_text = input("Сколько вам лет? ")
print(int(age_text) + 1)
```

Практика: input() и первый диалог

Интерактивный ноутбук прямо в браузере — настоящее поле ввода

Открыть практику → (<https://www.cartesianschool.org/practice/03-06/index.html>)

Первые имена и значения

Правильная мысленная модель переменной в Python — не коробка с данными, а имя, указывающее на значение.

Мы уже создавали имена вроде `name` в предыдущих разделах — настало время разобрать, что это на самом деле означает.

Не «коробка с данными»

Частая, но не совсем точная метафора: «переменная — это коробка, в которую кладут значение». Она сбивает с толку в Python, потому что подразумевает, будто у каждого имени своя отдельная ячейка памяти. На самом деле правильнее думать иначе:

`city`



`'Warsaw'`

Имя указывает на значение — а не «содержит» его

Имя — это, скорее, стрелка, указывающая на значение. Присваивание `city = "Warsaw"` не «кладёт» строку внутрь `city` — оно заставляет имя `city` указывать на объект-строку `"Warsaw"`, который существует сам по себе.

Создание и чтение имени

names.py

```
city = "Warsaw"
print(city)
```

Части этой строки:

- `city` — имя (переменная);
- `=` — оператор присваивания: «пусть имя слева указывает на значение справа»;
- `"Warsaw"` — значение (объект).

Несколько имён — один объект

Поскольку имя — это просто указатель, у одного и того же объекта может быть несколько имён одновременно. Возьмём пример:

score.py

```
score = 100
best_score = score
print(score)
print(best_score)
```

Здесь **два имени** — `score` и `best_score` — но **один объект**, на который оба указывают:



Частая ошибка мышления

НЕ думайте, что Python «скопировал 100 в две переменные-коробки» — это ровно та модель, которую мы отвергли выше. На самом деле объект один, а связей (ссылок) с ним — две. Правильный термин для стрелки на диаграмме — **ссылка** (reference): «имя связано с объектом», а не «имя содержит значение».

Переприсваивание не меняет старое имя

Продолжим тот же пример дальше:

`score2.py`

```
score = 100
best_score = score
score = 120
print(score)      # 120
print(best_score) # всё ещё 100
```

Строка `score = 120` — это **переприсваивание** (rebinding): имя `score` начинает указывать на другой объект. Существующий объект `100` при этом никак не меняется — `best_score` по-прежнему указывает на него:

`score`

120

`best_score`

100

После `score = 120`: у каждого имени теперь своя, независимая связь

Это важная идея, к которой мы ещё не раз вернёмся — при разговоре о неизменяемости чисел и строк (глава 4), о списках и их отличии от чисел, о том, как один и тот же объект может быть доступен из нескольких мест, и о том, как значения передаются в функции. Пока достаточно запомнить сам принцип: **переприсваивание меняет связь имени, а не объект.**

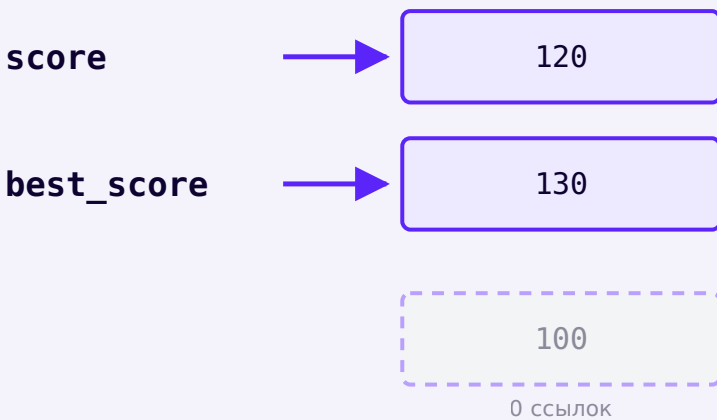
Когда объект «исчезает»?

Логичный вопрос: если `score` больше не указывает на `100`, что происходит с самим объектом `100`? Продолжим пример ещё на шаг:

score3.py

```
best_score = 130
print(score)      # 120
print(best_score) # 130
```

Теперь ни `score`, ни `best_score` не указывают на объект `100` — у него не осталось ни одной связи:



`100` стал недостижим — ни одно имя больше на него не указывает

Объект `100` стал **недостижимым** (unreachable) — до него нельзя «дотянуться» ни через одно имя в программе. Именно в этот момент, а не раньше, Python может освободить память, которую он занимал.

Так писать неточно

«Как только переменная перестаёт указывать на значение, сборщик мусора немедленно удаляет это значение» — эта формулировка вводит в заблуждение. Удаление одной связи НЕ уничтожает объект, если на него по-прежнему указывают другие имена (как `best_score` указывал на `100` даже после того, как `score` переприсвоили). Объект становится кандидатом на удаление только тогда, когда **ни одной** связи с ним не остаётся.

Сборщик мусора: автоматическое управление памятью

Вам, как правило, никогда не придётся вручную «освободить» память в Python. За этим следит **сборщик мусора** (garbage collector, GC) — часть самого Python, которая освобождает память недостижимых объектов автоматически.

Достаточно для повседневной работы

Python сам следит за объектами и освобождает память, когда объект больше нельзя использовать ни через одно имя в программе. Именно поэтому в Python (в отличие от языков вроде C) вы почти никогда не пишете код специально для освобождения памяти.

Что происходит глубже: подсчёт ссылок

Это необязательный, более глубокий взгляд — можно спокойно пропустить его при первом чтении и вернуться позже.

В CPython (реализация Python, которой мы пользуемся) у каждого объекта есть счётчик: сколько ссылок на него сейчас существует. Пройдём наш же пример ещё раз, но уже с этим счётчиком:

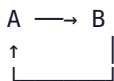
Объект 100 — счётчик ссылок

```
score = 100          # объект 100: ссылок = 1
best_score = score   # объект 100: ссылок = 2
score = 120          # объект 100: ссылок = 1 (score
теперь указывает на 120)
best_score = 130     # объект 100: ссылок = 0
(недостижим)
```

Для обычных, не входящих в циклическую ссылку объектов в CPython момент, когда счётчик достигает нуля, обычно совпадает с моментом освобождения памяти — почти сразу, без задержки. Это удобное, практичное поведение, но не гарантия, зафиксированная языком Python в целом — другие реализации Python (например, PyPy) освобождают память по другим правилам.

У подсчёта ссылок есть одна принципиальная сложность — **циклические ссылки**:

Циклическая ссылка (концептуально)



Представьте два объекта, каждый из которых ссылается на другой. Даже если ни одно имя в программе больше не указывает ни на А, ни на В, они всё ещё ссылаются друг на друга — и счётчик ссылок каждого из них не опускается до нуля. Поэтому в CPython, кроме подсчёта ссылок, есть отдельный **циклический сборщик мусора**, который периодически ищет именно такие недостижимые снаружи циклы и освобождает их.

Идём дальше — не сейчас

Мы намеренно останавливаемся здесь. К управлению памятью, времени жизни объектов и модулю `gc` мы ещё вернёмся позже в курсе, когда появятся структуры данных, которые реально могут образовывать циклы (например, объекты, ссылающиеся друг на друга). Сейчас достаточно знать, что такая ситуация существует и у Python есть механизм для неё.

Маленькая заметка про `del`

Команда `del` удаляет не объект, а конкретную связь (имя):

`del_demo.py`

```

x = 100
y = x
del x
print(y) # 100 — объект никуда не делся
  
```


`del x` убирает имя `x` из пространства имён — но НЕ означает «удали объект 100, что бы ни случилось». Поскольку `y` по-прежнему указывает на `100`, объект остаётся полностью доступным через `y`.

Пространство имён

Пока программа работает, Python хранит список всех созданных имён и того, на что каждое из них сейчас указывает, — это называется **пространством имён** (namespace). Диаграммы выше как раз и показывают срез такого пространства имён в разные моменты выполнения программы.

Обращение к ещё не созданному имени

Если обратиться к имени раньше, чем оно было создано присваиванием, Python не может догадаться, что вы имеете в виду — и сообщает об ошибке:

```
broken.py
```

```
print(favourite_city)
favourite_city = "Warsaw"
```

Ошибка

```
NameError: name 'favourite_city' is not defined
```

Подробно про ошибки и то, как читать такие сообщения, — в разделе 3.13.

Практика: имена и значения

Интерактивный ноутбук прямо в браузере — присваивание, чтение, `NameError`

[Открыть практику](https://www.cartesianschool.org/practice/03-07/index.html) → (<https://www.cartesianschool.org/practice/03-07/index.html>)

Комментарии и читаемый код

Небольшие привычки, которые сильно облегчают жизнь — себе будущему и всем, кто будет читать ваш код.

Комментарии

Строка, начинающаяся с `#`, — это **комментарий**: Python полностью её игнорирует при выполнении. Комментарии существуют не для компьютера, а для людей — включая вас самих через полгода.

```
privet.py
```

```
# Спрашиваем имя, чтобы персонализировать приветствие
name = input("Как вас зовут? ")
print("Привет,", name)
```

Хороший комментарий объясняет «почему», а не «что»

Комментарий вроде `# складываем a и b` над строкой `c = a + b` бесполезен — код и так это показывает. Полезный комментарий объясняет то, что не видно из самого кода: почему выбрано именно такое решение, какая бизнес-логика за этим стоит, на что стоит обратить внимание в будущем.

На этапе обучения совершенно нормально оставлять больше комментариев, чем оставил бы опытный разработчик в готовом проекте — они помогают вам самим проговорить, что делает каждая строка. Не переживайте, если пока хочется комментировать почти всё.

Имена файлов и переменных

Хороший стиль в именах экономит время — и вам, и всем, кто читает код после вас:

- используйте **строчные буквы**: `privet.py`, а не `Privet.PY`;
- используйте **осмысленные имена**: `calculator.py`, а не `file1.py`;
- для нескольких слов используйте **подчёркивание**: `my_first_program.py`, а не пробелы или регистр;
- избегайте пробелов в именах файлов — они удобны в графическом интерфейсе, но неудобны в терминале, где приходится экранировать каждый пробел.

Плохо	Хорошо
<code>New File FINAL 2!! .py</code>	<code>calculator.py</code>
<code>a.py</code>	<code>temperature_converter.py</code>
<code>MyFirstProgram.py</code>	<code>my_first_program.py</code>

PEP 8

Официальный гид по стилю Python-кода — PEP 8 (peps.python.org/pep-0008) — мы уже упоминали его в главе 1. Он описывает куда больше, чем имена файлов, но сейчас достаточно знать: он существует, и его рекомендации разделяет практически всё сообщество Python.

IDLE — что это и когда он полезен

Python сам приносит с собой простой редактор — IDLE. Разберёмся, как им пользоваться и когда стоит перейти на более мощный инструмент.

У Python есть встроенный простой редактор кода, который устанавливается автоматически вместе с самим Python, — **IDLE** (Integrated Development and Learning Environment). Открыть его можно, найдя «IDLE» в меню Пуск (Windows) или Launchpad (Mac), или набрав `idle3` в терминале (Linux).

Зачем IDLE вообще существует

IDLE — не «устаревший мусор», который стоит забыть. Его цель — быть простым и всегда доступным сразу после установки Python, без единого дополнительного шага. Это делает его удобным для самого первого знакомства с языком и для быстрых экспериментов, когда открывать полноценный редактор кода — избыточно.

Python Shell — окно интерактивной оболочки

При запуске IDLE открывается интерактивная оболочка — тот же самый Python REPL, что мы разбирали в разделах 3.5–3.6, только в собственном окне с небольшими удобствами вроде подсветки синтаксиса.

Режим сценариев (Script Mode)

Чтобы написать полноценную программу, нужен отдельный файл: **File** → **New File** откроет пустое окно редактора — это и есть «режим сценариев», второй режим работы IDLE.

1. Наберите код в новом окне редактора.
2. Сохраните файл: **File** → **Save** (расширение `.py` подставится само).
3. Запустите: **Run** → **Run Module**, либо просто нажмите клавишу `F5`.
4. Результат появится в окне интерактивной оболочки IDLE.

Когда стоит использовать IDLE

IDLE → современная разработка

IDLE (входит в Python «из коробки»)
`print("Привет из IDLE!")`
 # запуск: меню Run -> Run Module (F5)
 # один файл, минимум настроек

VS Code / PyCharm
`print("Привет из VS Code!")`
 # запуск: кнопка Run или встроенный терминал
 # подсветка ошибок на лету, автодополнение, # отладчик, git, множество расширений

Что использовать сегодня: IDLE отлично подходит, чтобы буквально за одну минуту после установки Python написать и запустить первую строку кода — устанавливать ничего дополнительно не нужно. Но как только проекты вырастают за пределы одного файла, заметно не хватает автодополнения, отладчика и подсветки ошибок на лету. Мы в основном пользуемся VS Code или PyCharm — но IDLE стоит знать: именно её вы увидите, если попробуете Python на чужом компьютере без дополнительной настройки.

Практика: выведите своё имя (и кое-что ещё)

Собираем воедино файл, терминал, print и выбор редактора — в одном небольшом упражнении, прежде чем двигаться дальше.

Небольшой контрольный чек-пойнт: применим файл, REPL, `print()` и выбор редактора (включая IDLE) в одном упражнении, прежде чем идти дальше — к ошибкам, отладке и первому настоящему мини-проекту.

★ Базовая практика

Выведите своё имя

Создайте файл `o_sebe.py` и одной командой `print()` выведите своё имя.

★★ Самостоятельная задача

И кое-что ещё

В том же файле добавьте ещё две строки: одну — с вашим городом, вторую — с любым числом, используя `sep`, чтобы вывести имя, город и число в одну строку через « · ».

★★★ Задача повышенной сложности

Тот же результат тремя способами

Получите один и тот же вывод тремя разными способами: запустив файл из терминала (`python o_sebe.py`), через кнопку Run в VS Code/PyCharm, и через Run Module в IDLE.

Ошибки, traceback и как их читать

Ошибки — это данные, а не повод для паники. Разберём три самых частых типа и научимся читать traceback осмысленно.

Красный текст ошибки — это не повод паниковать. Это **данные**: Python подробно объясняет, что именно пошло не так и где именно искать проблему. Научиться спокойно читать эти сообщения — один из самых полезных навыков для новичка.

SyntaxError

Python не может даже понять структуру кода — до выполнения дело не дошло вовсе. Чаще всего причина — пропущенная скобка, кавычка или двоеточие.

```
broken.py
```

```
print("Привет"
```

Traceback

```
File "broken.py", line 1
    print("Привет"
          ^
SyntaxError: '(' was never closed
```

NameError

Python встретил имя, которого ещё нигде не было создано присваиванием — часто это просто опечатка.

broken2.py

```
print(usr_name)
```

Traceback

```
Traceback (most recent call last):
  File "broken2.py", line 1, in <module>
    print(usr_name)
          ^^^^^^^
NameError: name 'usr_name' is not defined
```

IndentationError

В Python отступы — часть синтаксиса (мы вернёмся к этому подробно в главе про условия и циклы). Несогласованный отступ — тоже ошибка:

broken3.py

```
name = "Cartesian"
    print(name)
```

Traceback

```
File "broken3.py", line 2
    print(name)
IndentationError: unexpected indent
```

Анатомия traceback

Traceback лучше всего читать **снизу вверх** — самая важная информация внизу:

- **Traceback (most recent call last):**
заголовок — сообщает, что дальше будет цепочка вызовов
- **File "privet.py", line 3**
ГДЕ — какой файл и какая строка
- **print("Привет")**
сама проблемная строка кода
- **SyntaxError: ...**
ЧТО и ПОЧЕМУ — тип ошибки и объяснение

Анатомия traceback — читаем снизу вверх



Порядок работы с любой ошибкой

Не бойтесь красного текста

Ошибка — это Python, честно объясняющий, что случилось. Профессиональные разработчики видят traceback-и десятки раз в день — разница лишь в том, что они умеют быстро находить в нём нужную строку. Теперь и вы умеете.

Практика: прочитайте traceback (NameError)

Интерактивный ноутбук прямо в браузере — найдите и исправьте опечатку

Открыть практику → (<https://www.cartesianschool.org/practice/03-08/index.html>)

Лаборатории отладки

Пять коротких, специально сломанных примеров — потренируем один и тот же навык: предсказать, запустить, изучить, исправить.

Пять коротких лабораторий — специально сломанный код, чтобы потренироваться находить и чинить проблему по одному и тому же алгоритму: предсказать → запустить → изучить → исправить → объяснить себе, почему это сработало.

Лаборатория 1. Пропущенная кавычка или скобка

1. Предскажите: прочитайте код ниже — как вы думаете, что произойдёт?

сломанный код

```
print("Привет, мир!
```

2. Запустите — вы увидите: `SyntaxError: unterminated string literal`

3. Исправление:

исправлено

```
print("Привет, мир!")
```

4. Почему: у открывающей кавычки не было пары — Python не понял, где заканчивается текст.

Лаборатория 2. Неправильное имя переменной

1. Предскажите: прочитайте код ниже — как вы думаете, что произойдёт?

сломанный код

```
user_name = "Cartesian"  
print(usr_name)
```

2. Запустите — вы увидите: `NameError: name 'usr_name' is not defined`

3. Исправление:

исправлено

```
user_name = "Cartesian"  
print(user_name)
```

4. Почему: опечатка в имени — `usr_name` вместо `user_name`. Python не «догадывается» об опечатках — для него это два разных имени.

Лаборатория 3. Ошибка отступа

1. Предскажите: прочитайте код ниже — как вы думаете, что произойдёт?

сломанный код

```
print("Начало")  
    print("Продолжение")
```

2. Запустите — вы увидите: `IndentationError: unexpected indent`

3. Исправление:

исправлено

```
print("Начало")  
print("Продолжение")
```

4. Почему: у второй строки появился отступ, которого там быть не должно — вне блоков (`if/for/def`, которые встретятся в следующих главах) все строки должны начинаться с одной и той же позиции.

Лаборатория 4. Программа работает, но выводит не то

1. Предскажите: прочитайте код ниже — как вы думаете, что произойдёт?

сломанный код

```
name = "Анна"  
city = "Москва"  
print("Привет,", city)
```

2. Запустите — вы увидите: Привет, Москва (а должно быть — имя)

3. Исправление:

исправлено

```
name = "Анна"  
city = "Москва"  
print("Привет,", name)
```

4. Почему: программа выполнилась без единой ошибки — но перепутаны переменные. Это самый коварный тип бага: Python не может знать, что вы имели в виду не то, что написали.

Лаборатория 5. Не тот интерпретатор — привет из главы 2

1. Предскажите: прочитайте код ниже — как вы думаете, что произойдёт?

сломанный код

```
import requests
print("Пакет найден!")
```

2. Запустите — вы увидите:

```
ModuleNotFoundError: No module named 'requests'
```

(хотя вы точно его ставили!)

3. Исправление:

исправлено

```
# в VS Code: Ctrl+Shift+P -> "Python: Select
Interpreter"
# выбрать интерпретатор именно из активного .venv
проекта
```

4. Почему: пакет установлен в одно окружение, а код запускается через другое — классическая проблема из главы 2 (раздел 2.16). Проверьте `sys.executable` и выбранный интерпретатор в редакторе.

Практика: прочитайте traceback (SyntaxError)

Интерактивный ноутбук прямо в браузере — найдите пропущенную скобку и кавычку

Открыть практику → (<https://www.cartesianschool.org/practice/03-09/index.html>)

Первый отладчик в IDE

Отладчик умеет то, что не умеет `print()` — приостановить программу и заглянуть внутрь неё.

Отладчик (debugger) — инструмент, который умеет **приостанавливать** выполняющуюся программу в конкретной строке и показывать, чему в этот момент равны все имена (раздел 3.9). Это гораздо мощнее, чем расставлять `print()` по всему коду и запускать заново.



Общая идея отладчика — одинаковая в VS Code и в PyCharm

Возьмём один и тот же крошечный пример для обеих сред:

debug_demo.py

```

name = "Анна"
city = "Варшава"
greeting = "Привет, " + name
print(greeting)
    
```

Отладчик в VS Code

1. Кликните слева от номера строки `greeting = "Привет, " + name` — появится красная точка (breakpoint, точка останова).
2. Откройте вкладку **Run and Debug** на боковой панели (или нажмите `F5`).
3. Программа запустится и остановится ровно на точке останова — строка подсветится.
4. В панели **Variables** слева видно текущие значения: `name` уже существует, а `greeting` — ещё нет (мы остановились ДО выполнения этой строки).
5. **Step Over** (значок со стрелкой вверх точки) выполняет ровно одну строку и снова останавливается — теперь `greeting` тоже появится в списке переменных.
6. **Continue** (▷) отпускает программу до конца или до следующей точки останова.

Отладчик в PyCharm

Идея та же самая — отличаются только названия кнопок:

Действие	VS Code	PyCharm
Поставить точку останова	Клик слева от номера строки	Клик слева от номера строки
Запустить с отладкой	Run and Debug / <code>F5</code>	Значок с жуком рядом с Run
Выполнить одну строку	Step Over	Step Over (<code>F8</code>)
Продолжить до конца/следующей точки	Continue	Resume Program

Действие	VS Code	PyCharm
Посмотреть значения переменных	Панель Variables	Панель Variables (внизу)

О скриншотах на этой странице

Эта страница описывает реальный, стандартный интерфейс отладчика VS Code и PyCharm текстом и диаграммой, а не скриншотом — окружение, в котором собирался этот курс, не может делать снимки экрана графических приложений. Сама последовательность действий (точка останова → запуск с отладкой → пауза → осмотр значений → Step Over/Continue) полностью совпадает с тем, что вы увидите на экране.

Главная идея

Отладчик позволяет ПРИОСТАНОВИТЬ работающую программу и заглянуть внутрь неё — увидеть реальные значения имён в реальный момент выполнения, а не гадать по коду, что должно было произойти.

Notebook и kernel

Что на самом деле происходит, когда вы запускаете ячейку — в блокноте и в браузерной практике этого курса.

Мы уже проходили практику этого курса в браузере — самое время объяснить, что там на самом деле происходит, с точки зрения выполнения кода.

Notebook — это не то же самое, что kernel

- **notebook** (`.ipynb`) — сам файл: набор ячеек с кодом, текстом и уже сохранёнными результатами прошлого запуска;

- **kernel** (ядро) — отдельный работающий процесс Python, который реально выполняет код из ячеек, когда вы их запускаете.



От файла блокнота до результата — через ядро

Один и тот же файл блокнота можно открыть и подключить к разным ядрам — например, к ядру с одним набором установленных пакетов или с другим (мы говорили об этом в главе 2, раздел 2.15, применительно к VS Code).

Важная особенность: состояние сохраняется между ячейками

В отличие от обычного скрипта, который каждый раз выполняется **с самого начала**, в блокноте переменные, созданные в одной ячейке, остаются доступны в следующих — пока работает то же самое ядро.

Ячейка 1

```
x = 10
```

Ячейка 2 (запущена ПОСЛЕ ячейки 1)

```
print(x) # 10 — x всё ещё существует
```

Порядок запуска ячеек — это не порядок ячеек на экране

Если запустить ячейки не по порядку (например, вторую раньше первой) или перезапустить ядро, не выполнив всё заново, — результат может не совпасть с тем, что вы видите на экране. Это одна из самых частых причин путаницы у новичков в блокнотах: то, что показано на экране, — результат ПОСЛЕДНЕГО запуска, а не гарантия того, что получится при выполнении сверху вниз прямо сейчас.

Как устроена браузерная практика этого курса

Все интерактивные упражнения курса — настоящий Python, а не имитация. Устроено это так:



Как устроена браузерная практика этого курса

Pyodide — это реальный интерпретатор Python, скомпилированный в WebAssembly и запускающийся прямо в вашем браузере, без установки на компьютер. Это ещё одна среда выполнения — со своими ограничениями: например, у неё нет доступа к нативным графическим окнам операционной системы, поэтому пакеты вроде `tkinter` в чистом виде там не работают (в курсе такие упражнения честно помечены как «локальные» — вы уже видели это в главе 2).

Официальный источник

Подробнее о самом Jupyter — jupyter.org; о Pyodide — pyodide.org.

Мини-проект и итоги главы

Собираем всё пройденное в один небольшой, но настоящий проект — и подводим итоги главы.

Пора собрать почти всё, что мы прошли в этой главе, в один небольшой, но настоящий проект.

Мини-проект: визитная карточка в терминале

Программа спрашивает имя, город и то, что вы хотите создать на Python — а затем печатает небольшой персонализированный профиль.

card.py

```
name = input("Ваше имя: ")
city = input("Ваш город: ")
goal = input("Что вы хотите создать на Python? ")

print("=====")
print(" МОЯ PYTHON-КАРТОЧКА")
print("=====")
print("Имя:", name)
print("Город:", city)
print("Хочу создать:", goal)
print("=====")
```

Пример того, что должно получиться:

Пример вывода

```
=====
МОЯ PYTHON-КАРТОЧКА
=====
Имя: Анна
Город: Варшава
Хочу создать: игру
=====
```

Только то, что уже знакомо

Весь проект построен исключительно на инструментах этой главы: `input()` , присваивание, `print()` . Условия и циклы, которые сделали бы код гибче, придут в следующих главах — а пока достаточно и этого, чтобы получить настоящую, законченную маленькую программу.

★★★ **Задача повышенной сложности**

Challenge: своё оформление

Придумайте собственное оформление визитки — другая рамка (например, из `*`), выравнивание через `sep` , или добавьте четвёртую строку с любимым языком программирования.

Практика: мини-проект «Python-визитка»

Интерактивный ноутбук прямо в браузере — соберите всё вместе

[Открыть практику](https://www.cartesianschool.org/practice/03-10/index.html) → (<https://www.cartesianschool.org/practice/03-10/index.html>)

Итоги главы

Что мы теперь умеем

- Понимаем, что программа на Python — обычный текстовый .py-файл, и как CPython превращает исходный код в байт-код, а затем выполняет его.
- Чётко различаем терминал, shell операционной системы и Python REPL — три разные вещи под похожими именами.
- Знаем основные командные оболочки (Bash, Zsh, PowerShell, cmd.exe, Fish) и познакомились с PySH — Python-first альтернативой на реальном локальном примере.
- Уверенно пользуемся Python REPL: R-E-P-L, приглашения `>>>` и `...`, `type()/help()/dir()` как инструменты исследования.
- Различаем script, REPL, notebook и IDE — и знаем, для чего каждый из них лучше подходит.
- Подробно освоили `print()` (несколько значений, `sep`, `end`) и `input()` (первый диалог, текст как результат).
- Понимаем имена как указатели на значения, а не как «коробки с данными».
- Умеем писать понятные комментарии и выбирать читаемые имена файлов и переменных.
- Спокойно читаем `SyntaxError`, `NameError` и `IndentationError`, понимаем анатомию `traceback` и прошли пять лабораторий отладки.
- Знаем, как поставить точку останова и использовать отладчик в VS Code и PyCharm.
- Понимаем разницу между notebook и kernel — и как устроена браузерная практика этого курса на Pyodide.

- Собрали первый настоящий мини-проект — интерактивную визитную карточку.

Что дальше

В главе 4 мы вплотную займёмся числами — их видами, преобразованием и математикой на Python.

Python любит числа

Полноценные числовые основы Python 3.14: что компьютер называет числом, `int` и системы счисления, операторы и их порядок, деление и остаток, `float` и почему $0.1 + 0.2$ не равно 0.3 , округление, `Decimal`, `Fraction`, `complex`, `math`, `random/secrets`, `statistics` — и несколько мини-проектов.

4.1 Что такое число для компьютера

4.2 `int` — целые числа без страха

4.3 Системы счисления

4.4 Комментарии в вычислениях

4.5 Арифметические операторы

4.6 Порядок выполнения операций

4.7 Деление и остаток

4.8 Степени и `abs()`

4.9 Карта числовых типов

4.10 `float` — дробные числа

4.11 Почему $0.1 + 0.2$ не равно 0.3

4.12 Сравниваем float правильно

4.13 Округление: round, floor, ceil, trunc

4.14 Decimal — точная десятичная арифметика

4.15 Fraction — точные дроби

4.16 complex и cmath

4.17 Преобразование типов чисел

4.18 Модуль math

4.19 random и secrets

4.20 statistics, inf и nan

4.21 Числовые ошибки и отладка

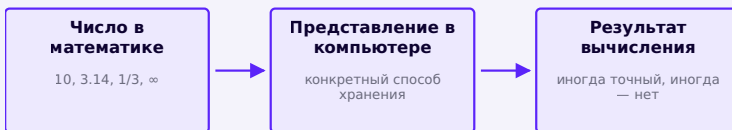
4.22 Мини-проекты и итоги

Что такое число для компьютера

Прежде чем изучать типы чисел — разберёмся, что вообще значит «число» с точки зрения компьютера, и как имена связаны с числовыми объектами.

Человек видит числа как абстрактные понятия: `10`, `3.14`, `1/3`, даже `∞`. Компьютер же не «понимает» числа в математическом смысле — он хранит их

представление: конкретную последовательность байтов по конкретным правилам.



Математическое число и его представление в компьютере — связаны, но не всегда совпадают

Для целых чисел это различие почти незаметно — представление в компьютере ведёт себя так же, как число в математике. Но для дробных чисел разница становится важной — и мы подробно разберём её в разделах 4.10–4.11, когда дойдём до `float`. Пока достаточно запомнить сам принцип: то, что вы пишете в коде, и то, что реально хранится внутри, — связанные, но не обязательно тождественные вещи.

Имена указывают на числовые объекты

В главе 3 мы разобрали важный принцип: имя (переменная) — это не «коробка», в которую кладут значение, а скорее стрелка, указывающая на объект. Этот же принцип полностью относится и к числам.

```
age.py
```

```
age = 10  
print(age)
```

age



10

age указывает на числовой объект 10 — не «содержит» его

Что происходит при `age = age + 1`

Числа в Python — **неизменяемые** (immutable) объекты: сам объект 10 невозможно «отредактировать» в 11. Когда мы пишем:

age2.py

```
age = 10
age = age + 1
print(age) # 11
```

происходит не редактирование, а переприсваивание — Python вычисляет новое значение и заставляет имя age указывать на **НОВЫЙ** объект:

age (до)



10

ДО: age указывает на 10

вычисление: $10 + 1 \rightarrow 11$

age (после)



11

ПОСЛЕ: age указывает на новый объект 11 —
старый объект 10 больше не нужен

Точная формулировка важна

Python не «изменил число 10 на 11». Целые числа **неизменяемы**: Python вычислил результат `10 + 1`, получил новый объект `11` и переприсвоил имя `age` этому новому объекту. Это ровно та же модель «имя → объект» и «переприсваивание меняет связь, а не сам объект», которую мы разбирали в главе 3.

Мы будем возвращаться к этому принципу неизменяемости на протяжении всего курса — он объясняет поведение строк, кортежей и (в отличие от них) то, почему списки ведут себя иначе, когда мы дойдём до них в следующих главах.

Попробуем

Наберите этот пример в браузерной практике или в REPL (глава 3) — и после каждой строки проверяйте `id(age)`. Число (адрес объекта в памяти для CPython) изменится после переприсваивания — наглядное подтверждение того, что объект действительно стал другим.

Практика: числа как объекты, а не коробки

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/04-01/index.html>)

int — целые числа без страха больших значений

Целые числа в Python могут расти намного больше, чем в большинстве других языков — и остаются при этом неизменяемыми объектами.

`int` (integer — целое число) — тип для чисел без дробной части: положительных, отрицательных и нуля.

```
celye.py
```

```
print(0)
print(42)
print(-17)
print(type(42))
```

Числа практически без ограничения размера

В отличие от многих других языков программирования, где целые числа ограничены фиксированным диапазоном (например, 32 или 64 бита), в Python `int` может расти сколь угодно — точность **произвольная**, ограниченная практически только доступной памятью компьютера, а не языком.

```
bolshoe_chislo.py
```

```
huge = 10 ** 100
print(huge)
```

Точная формулировка

Правильно говорить «произвольная точность, ограниченная доступными ресурсами», а не «неограниченный размер». Формально предел есть — но на практике вы почти никогда его не достигнете в обычных программах.

Числа неизменяемы — продолжение

Мы уже видели это в разделе 4.1 на примере `age`. Ещё раз, уже с двумя именами:

```
a_b.py
```

```
a = 10
b = a
a += 1
print(a) # 11
print(b) # 10 — b не изменился
```

a



11

b



10

После `a += 1`: у `a` — новая связь; `b` по-прежнему указывает на прежний объект

Не используйте `id()` как учебный трюк для «кеша целых чисел»

Вы можете где-то встретить сравнение `id(256)` и `id(257)` как «доказательство» особого поведения маленьких чисел. Это деталь реализации CPython (оптимизация кеширования маленьких целых чисел), а не гарантия языка Python — полагаться на неё в реальном коде нельзя, и как обучающий пример она скорее запутывает, чем помогает.

bool — особый родственник int

Значения `True` и `False` образуют тип `bool` — и исторически он тесно связан с `int`: `True` ведёт себя как `1`, а `False` — как `0`.

```
bool_int.py
```

```
print(int(True))    # 1
print(int(False))   # 0
print(True + True)  # 2
```

Но в обычном коде — это логические значения

То, что `bool` технически «числовой» — деталь языка, а не приглашение складывать `True + True` в реальном коде. Полноценно об условиях и логике — в главе про условия; здесь достаточно знать связь с числами, если вы её где-то встретите.

Разделители-подчёркивания для читаемости

В длинных числах легко потерять счёт нулям. Python позволяет разбивать число подчёркиваниями — они не влияют на значение, только на читаемость:

```
naselenie.py
```

```
population = 38_000_000
print(population) # 38000000
```

Сокращённое присваивание

Операторы вроде `+=` — это компактная запись «пересчитать и переприсвоить», а не «изменить число на месте»:

augmented.py

```
x = 10
x += 1  # то же, что x = x + 1
x -= 2  # то же, что x = x - 2
x *= 3  # то же, что x = x * 3
```

Практика: большие числа и неизменяемость int

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/04-06/index.html\)](https://www.cartesianschool.org/practice/04-06/index.html)

Как записывать числа: системы счисления

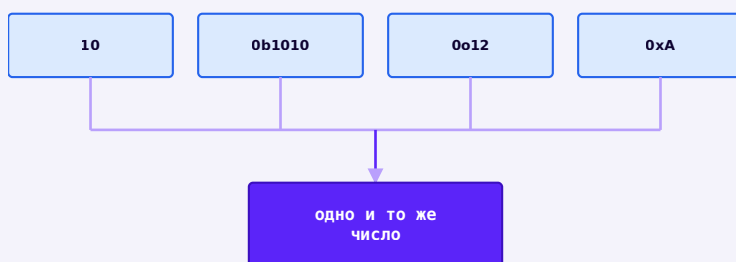
Десятичная система — не единственная. Разберём двоичную, восьмеричную и шестнадцатеричную запись — и зачем они вообще нужны программисту.

Мы привыкли записывать числа в **десятичной** системе (base 10) — десять цифр, от 0 до 9. Но это не единственный способ, и компьютеры часто используют другие системы счисления.

Четыре системы, с которыми вы встретитесь

Система	Основание	Цифры	Python-литерал
Десятичная (decimal)	10	0-9	10
Двоичная (binary)	2	0-1	0b1010

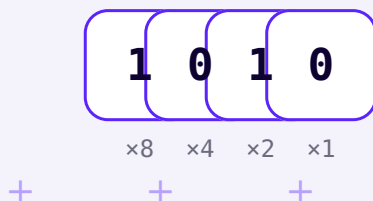
Система	Основание	Цифры	Python-литерал
Восьмеричная (octal)	8	0-7	<code>0o12</code>
Шестнадцатеричная (hexadecimal)	16	0-9, A-F	<code>0xA</code>



Четыре записи — один и тот же результат

Как читать позиционную запись

В любой системе счисления каждая позиция цифры имеет свой «вес» — степень основания. Возьмём двоичное число `1010`:



$$= 10$$

$$1010_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 = 10$$

Ровно тот же принцип работает и для десятичной системы, которой вы пользуетесь каждый день — просто вес позиций там 1000, 100, 10, 1 (степени десяти), а не степени двойки.

Зачем программисту шестнадцатеричная система

Шестнадцатеричная запись встречается на практике куда чаще, чем кажется:

- **Цвета** в вебе и графике: `#FF0000` — ярко-красный;
- **Байтовые/битовые шаблоны** — компактная запись двоичных данных;
- **Сетевые протоколы и форматы файлов** — MAC-адреса, хеш-суммы;
- **Символы Unicode** — код символа часто записывают в hex (забегая вперёд, подробно про Unicode — в главе про строки).

```
cvet.py
```

```
red = 0xFF
print(red) # 255
```

Функции `bin()`, `oct()`, `hex()`

Переводят число ИЗ десятичной записи В строку с другой системой счисления:

```
v_druguyu_sistemu.py
```

```
print(bin(10)) # '0b1010'
print(oct(10)) # '0o12'
print(hex(10)) # '0xa'
```

int() с указанием основания — обратное преобразование

А функция `int()` с двумя аргументами делает обратное — читает текст в указанной системе счисления и возвращает обычное (десятичное) число:

```
iz_teksta.py
```

```
print(int("1010", 2)) # 10
print(int("FF", 16)) # 255
```

Попробуем

Переведите свой год рождения в двоичную и шестнадцатеричную запись через `bin()` и `hex()` — а затем переведите результат обратно через `int(..., 2)` и `int(..., 16)`, чтобы убедиться, что получили то же число.

Практика: переводчик систем счисления

Интерактивный ноутбук прямо в браузере — `bin()`, `oct()`, `hex()`, `int(x, base)`

Открыть практику → (<https://www.cartesianschool.org/practice/04-07/index.html>)

Комментарии в вычислениях: единицы, формулы и намерение

Числа сами по себе не несут смысла — комментарии и осмысленные имена помогают зафиксировать, что именно они означают.

Комментарии мы уже подробно разбирали в главе 3 — здесь не будем повторять весь урок, а сосредоточимся на том, чем комментарии особенно полезны именно в вычислениях: единицы измерения, формулы и скрытые предположения.

Число само по себе не несёт единицу измерения

Взгляните на эту строку:

```
distance.py
```

```
distance = 120 # км
```

Комментарий здесь помогает — но остаётся хрупким: он живёт отдельно от значения и никто не проверяет, что он всё ещё верен. Более надёжный (хоть и не идеальный) приём — включить единицу прямо в имя:

```
distance2.py
```

```
distance_km = 120
```

Так яснее видно из самого кода, что именно хранится — без необходимости искать комментарий рядом. Мы вернёмся к более строгим способам связывать значения с их смыслом (типы, структуры данных) в следующих главах — здесь достаточно осознать саму проблему: **Python не знает, что число 120 означает**

километры. Это знание существует только в голове того, кто написал код — и в лучшем случае в имени или комментарии.

Комментарии для формул и предположений

Отдельная, очень полезная роль комментариев — объяснить формулу или зафиксировать предположение, которое иначе останется «магическим числом»:

`nds.py`

```
# НДС в учебном примере: 23 %
vat_rate = 0.23
price = 100
total = price * (1 + vat_rate)
print(total)
```

Хороший комментарий объясняет то, чего не видно из кода

`vat_rate = 0.23` — само число не объясняет, откуда оно взялось и что означает. Комментарий `# НДС в учебном примере: 23 %` отвечает именно на этот вопрос — «почему именно это число», а не «что делает эта строка».

Этот же приём пригодится буквально в каждом разделе этой главы — формулы для площади круга, конвертации времени, скидок и процентов читаются заметно понятнее, если рядом зафиксировано, что означает каждое число.

Практика: единицы измерения и комментарии к формулам

Интерактивный ноутбук прямо в браузере — сделайте числовой код понятным

Открыть практику → (<https://www.cartesianschool.org/practice/04-02/index.html>)

Арифметические операторы

Полный набор операторов для работы с числами — от сложения до возведения в степень.

Python поддерживает полный набор арифметических операторов — большинство из них уже знакомы из математики, но у нескольких есть особое, «программистское» поведение.

Оператор	Название	Пример	Результат
<code>+</code>	Сложение	<code>7 + 3</code>	10
<code>-</code>	Вычитание	<code>7 - 3</code>	4
<code>*</code>	Умножение	<code>7 * 3</code>	21
<code>/</code>	Деление (всегда float)	<code>7 / 3</code>	2.333...
<code>//</code>	Целочислен- ное деление	<code>7 // 3</code>	2
<code>%</code>	Остаток от деления	<code>7 % 3</code>	1
<code>**</code>	Возведение в степень	<code>7 ** 3</code>	343

Унарные + и -

Плюс и минус работают и как унарные операторы — перед одним числом, без второго операнда:

unarnye.py

```
x = 5
print(-x)    # -5
print(+x)    # 5 (унарный + почти не используется, но
              существует)
```

Операторы в связке с реальной задачей

Возьмём цену товара со скидкой:

skidka.py

```
price = 1000
discount_percent = 15
discount_amount = price * discount_percent / 100
final_price = price - discount_amount
print(final_price) # 850.0
```

Практика: арифметические операторы

Интерактивный ноутбук прямо в браузере — предскажите и проверьте результат

[Открыть практику → \(https://www.cartesianschool.org/practice/04-08/index.html\)](https://www.cartesianschool.org/practice/04-08/index.html)

Порядок выполнения операций

$2 + 3 * 4$ — это 14 или 20? Разберём, в каком порядке Python на самом деле вычисляет выражения.

Как и в математике, в Python операции выполняются не строго слева направо — у каждого оператора есть свой **приоритет**.

primer.py

```
print(2 + 3 * 4)      # 14, а не 20
print((2 + 3) * 4)    # 20 — скобки меняют порядок
```

()

скобки — всегда
выполняются первыми

**

возведение в степень

+X -X

унарный плюс/минус

* / //
%

умножение, деление,
целочисленное деление, остаток

+ -

сложение, вычитание

Порядок операций этой главы — от высшего
приоритета к низшему

Это не полная таблица

Официальная таблица приоритетов Python включает намного больше операторов — сравнения, логические, битовые и другие, до которых мы ещё не дошли. Здесь — только то, что относится к арифметике этой главы. Полную таблицу можно найти в разделе «Operator precedence» на docs.python.org, когда она вам понадобится целиком.

Правило хорошего тона

Даже когда вы точно помните приоритет операторов, скобки почти ничего не стоят, а читаемость улучшают ощутимо. Если есть сомнение, как прочтает выражение другой человек (или вы сами через полгода) — добавьте скобки.

Практика: почините порядок операций

Интерактивный ноутбук прямо в браузере — предскажите результат, затем расставьте скобки

[Открыть практику](https://www.cartesianschool.org/practice/04-09/index.html) → (<https://www.cartesianschool.org/practice/04-09/index.html>)

Деление: /, //, % и divmod()

Три родственника оператора деления — и почему отрицательное целочисленное деление работает не так, как кажется на первый взгляд.

У деления в Python на самом деле три родственника, но разных операторов — и путаница между ними одна из самых частых у новичков.

17 конфет на 5 детей

Представим задачу: у нас 17 конфет, и мы делим их поровну между 5 детьми.

konfety.py

```
candies = 17
children = 5

print(candies / children)  # 3.4 — точное дробное
                           деление
print(candies // children) # 3   — по 3 конфеты
                           каждому ребёнку
print(candies % children)  # 2   — 2 конфеты
                           остались нераспределены
```

Каждый ребёнок получает `candies // children` целых конфет, а `candies % children` — сколько осталось «в остатке». Удобно получить оба числа сразу:

divmod_demo.py

```
print(divmod(17, 5))  # (3, 2)
```

Чуть глубже: отрицательное floor-деление

С отрицательными числами `//` ведёт себя не так, как «просто отбросить дробную часть» — а округляет результат **в сторону минус бесконечности** (floor):

otricatelnoe.py

```
print(-7 // 3) # -3, а не -2
print(-7 % 3)  # 2
```



`-7 // 3` округляется вниз, к -3 (в сторону минус бесконечности), а не к -2

Почему так, а не иначе

`//` в Python определён как «floor division» — округление результата деления вниз, к ближайшему целому В СТОРОНУ минус бесконечности. Для положительных чисел это выглядит как обычное отбрасывание дробной части, но для отрицательных — даёт число МЕНЬШЕ, а не ближе к нулю. Это не баг, а осознанное определение оператора, одинаковое во всех версиях Python.

Практика: деление, остаток и `divmod()`

Интерактивный ноутбук прямо в браузере — включая отрицательное floor-деление

Открыть практику → (<https://www.cartesianschool.org/practice/04-10/index.html>)

Степени, `pow()` и `abs()`

Возведение в степень — и функция `abs()`, которая пригодится буквально в каждой следующей главе курса.

Возведение в степень в Python — оператор `**` или функция `pow()` :

stepeni.py

```
print(2 ** 10)      # 1024
print(pow(2, 10))   # 1024 — то же самое
```

Необязательный третий аргумент pow()

pow() умеет ещё и третий, необязательный аргумент — модуль:

pow_mod.py

```
print(pow(2, 10, 1000))
# 24 — то же, что (2 ** 10) % 1000, но эффективнее для
# больших чисел
```

Такое «модульное возведение в степень» — важный строительный блок в алгоритмах и криптографии (мы не будем изучать криптографию сейчас — просто полезно знать, что этот инструмент существует и эффективен даже для огромных чисел).

abs() — модуль числа

abs() возвращает число без знака — расстояние до нуля:

abs_demo.py

```
print(abs(-7))    # 7
print(abs(7))     # 7
print(abs(-3.5))  # 3.5
```

Практика: степени, pow() и abs()

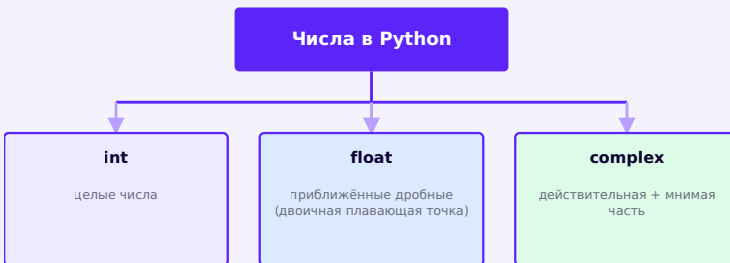
Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/04-11/index.html>)

Карта числовых типов Python

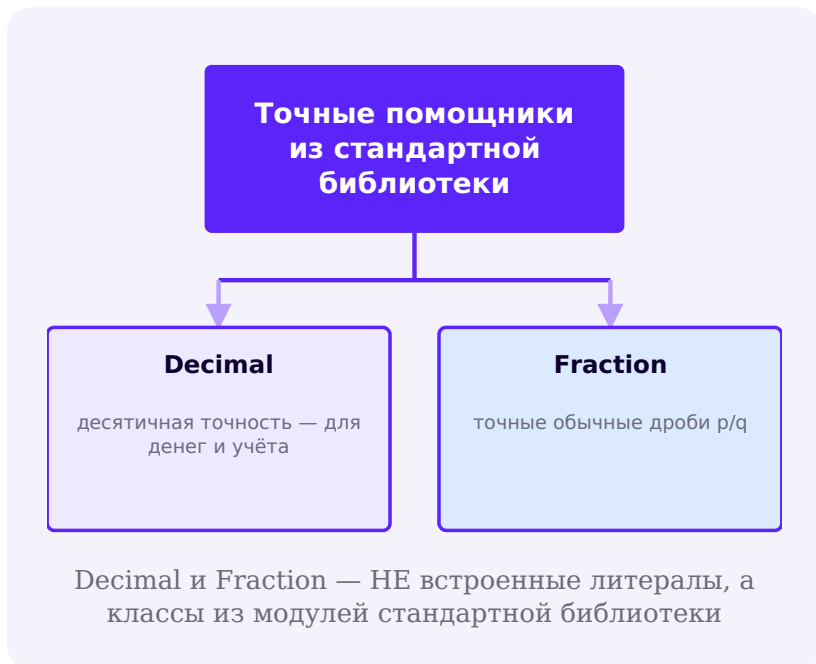
Прежде чем идти глубже в каждый тип по отдельности — увидим общую картину целиком.

Прежде чем идти глубже — соберём общую карту. В Python есть три **встроенных** числовых типа, литералы которых понимает сам язык:



Три встроенных числовых типа — литералы понимает сам язык

Кроме них, в стандартной библиотеке (модули, которые устанавливаются вместе с Python, но требуют явного `import`) есть ещё два важных числовых инструмента:



Важное различие

`int`, `float` и `complex` — часть самого языка: их литералы (`42`, `3.14`, `3+4j`) работают без единого `import`. `Decimal` и `Fraction` — обычные классы Python из модулей `decimal` и `fractions`, и без `import` они недоступны. Мы разберём оба подробно в разделах 4.14–4.15.

Дальше в этой главе мы пройдем по каждому из пяти инструментов подробно — а в самом конце главы (раздел 4.22) соберём их все в одну финальную карту выбора: какой тип нужен для какой задачи.

float — дробные числа

Первое знакомство с float — прежде чем разобраться, почему его поведение иногда удивляет (следующий раздел).

float — тип для чисел с дробной частью: измерения, цены, доли, научные величины. В отличие от int, у которого одна прямолинейная запись, у float их две.

Обычная запись

float_obychnaya.py

```
pi = 3.14
print(type(pi))

whole = 2.0
print(type(whole)) # float, несмотря на нулевую
                    # дробную часть
```

Точка делает число float

Даже 2.0 — float, а не int, хотя математически это целое число. Тип определяет запись в коде, а не математический смысл значения.

Научная (экспоненциальная) запись

Для очень больших или очень маленьких чисел удобна научная запись — e означает «умножить на 10 в степени»:

nauchnaya.py

```
print(1e6)      # 1000000.0   (1 × 10^6)
print(2.5e-3)   # 0.0025     (2.5 × 10^-3)
```


float в вычислениях

Оператор `/` (обычное деление, раздел 4.7) всегда возвращает `float`, даже если числа делятся нацело:

```
delenie_float.py
```

```
print(10 / 2) # 5.0, а не 5
```

Практика: float — запись и первые вычисления

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/04-12/index.html>)

Почему $0.1 + 0.2$ не равно равно 0.3

Самый знаменитый «сюрприз» `float` — и совершенно логичное, точное объяснение, почему он происходит.

Один из самых знаменитых «сюрпризов» Python — да и практически любого языка программирования:

```
surpriz.py
```

```
print(0.1 + 0.2)
```

Реальный вывод

```
0.30000000000000004
```

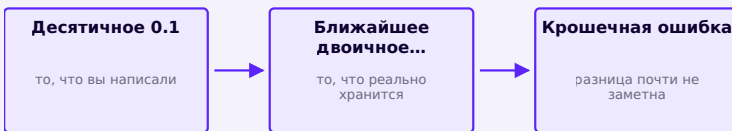
Это не баг Python

Точно такое же поведение вы увидите в JavaScript, Java, C, C++ и почти любом другом распространённом языке. Причина — не ошибка Python, а то, как ЛЮБОЙ компьютер хранит дробные числа в двоичном виде.

Простая аналогия: $1/3$ в десятичной записи

В десятичной системе $1/3$ нельзя записать конечным числом цифр — только $0.333333\dots$, до бесконечности. Не потому, что десятичная система «плохая» — просто у неё есть числа, которые она не может выразить точно конечной записью.

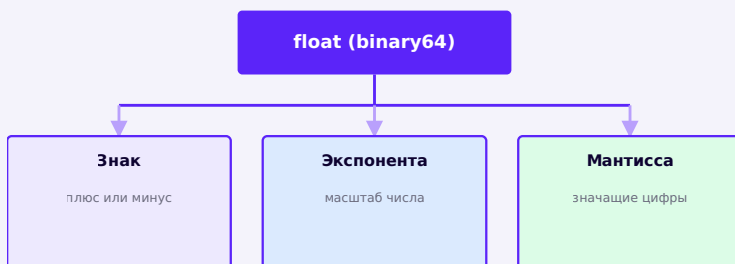
У компьютера в двоичной системе — точно такая же проблема, только с другими числами. 0.1 в десятичной записи выглядит «простым» — но в двоичной системе он превращается в бесконечно повторяющуюся дробь, совсем как $1/3$ в десятичной. Компьютер вынужден её обрезать до конечного числа битов — отсюда и крошечная погрешность.



0.1 не хранится ровно — хранится ближайшее представимое двоичное приближение

Что происходит глубже

На большинстве современных компьютеров и в стандартном CPython `float` соответствует аппаратному формату двойной точности, известному как **binary64** (часть стандарта IEEE 754). Концептуально число хранится тремя частями:



Из чего состоит типичный float на современном CPython (стандарт binary64)

Не нужно запоминать наизусть

Точное число битов на каждую часть (1/11/52 в стандарте binary64) — деталь, которую не нужно заучивать прямо сейчас. Важно понимать сам принцип: значение хранится приближённо, а не то, что Python «плохо считает». Это распространённое, хорошо изученное поведение — стандарт binary64, а не собственное правило языка Python.

Python может показать точное сохранённое значение

Хорошая новость: приближение не случайно и не загадочно — Python может показать вам, что именно хранится:

tochnoe.py

```
print((0.1).as_integer_ratio())
# (3602879701896397, 36028797018963968) — точная
# дробь, которая реально хранится
```

hex_predstavlenie.py

```
print((0.1).hex())
# '0x1.999999999999ap-4' — точное представление в
# шестнадцатеричном виде
```

Главный вывод раздела

0.1 хранится не «примерно как получится», а как совершенно точное, предсказуемое и объяснимое приближение — просто не то десятичное число 0.1, которое вы написали. Это приближение, а не случайность.

Практика: $0.1 + 0.2$ — исследуем приближение

Интерактивный ноутбук прямо в браузере —
`as_integer_ratio()`, `hex()` и предсказание результата

Открыть практику → (<https://www.cartesianschool.org/practice/04-13/index.html>)

Сравниваем float правильно

`==` почти никогда не подходит для float. Разберём
 правильный инструмент — `math.isclose()`.

Раз `0.1 + 0.2` не равно ровно `0.3` (раздел 4.11), прямое
 сравнение через `==` для float — плохая идея:

```
sravnenie_ploho.py
```

```
print(0.1 + 0.2 == 0.3) # False!
```

`math.isclose()` — сравнение «достаточно близко»

Правильный инструмент — сравнение с допуском
 (tolerance): вместо «равно точно» проверяем «достаточно
 близко»:

```
isclose_demo.py
```

```
from math import isclose
```

```
print(isclose(0.1 + 0.2, 0.3)) # True
```

Не изобретайте свой допуск на глаз

Часто встречается самодельная проверка вроде `abs(a - b) < 0.000001`. Она работает не всегда — для очень больших или очень маленьких чисел фиксированный порог может оказаться слишком строгим или слишком мягким. `math.isclose()` по умолчанию использует **относительный** допуск (пропорциональный размеру самих чисел), что гораздо надёжнее для чисел разного масштаба.

`otnositelnij_dopusk.py`

```
from math import isclose

print(isclose(1000.0001, 1000.0002))
# True — разница мала относительно размера чисел
print(isclose(0.0001, 0.0002))          # False — та же
абсолютная разница, но огромна относительно размера
```

Практика: почините сравнение float

Интерактивный ноутбук прямо в браузере — замените `==` на `math.isclose()`

Открыть практику → (<https://www.cartesianschool.org/practice/04-14/index.html>)

Округление: `round()`, `floor()`, `ceil()`, `trunc()`

Несколько способов превратить дробное число в целое — и почему для отрицательных чисел они дают разные ответы.

Есть сразу несколько способов превратить дробное число в целое — и они ведут себя по-разному, особенно для отрицательных чисел.

round() — обычное округление

round_demo.py

```
print(round(3.14159, 2)) # 3.14
print(round(7.5))        # 8
```

Округление «до чётного» на границе .5

Внимательный взгляд на округление ровно посередине даёт неожиданный на первый взгляд результат:

granica.py

```
print(round(2.5)) # 2, а не 3!
print(round(3.5)) # 4
```

Округление до ближайшего чётного

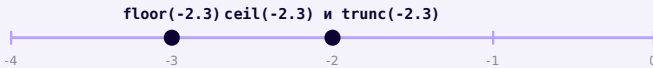
Это не ошибка и не случайность: `round()` в Python округляет значение ровно между двумя целыми к БЛИЖАЙШЕМУ ЧЁТНОМУ — стандартный статистический приём («банковское округление»), который снижает систематический перекося при округлении множества значений. Для большинства повседневных задач это почти незаметно — но полезно знать, если результат округления вас удивил.

math.floor(), math.ceil(), math.trunc()

floor_ceil_trunc.py

```
from math import floor, ceil, trunc

print(floor(-2.3)) # -3 — вниз, к меньшему целому
print(ceil(-2.3))  # -2 — вверх, к большему целому
print(trunc(-2.3)) # -2 — просто отбросить дробную часть
```



floor округляет вниз, к -3; ceil и trunc здесь совпадают — оба дают -2

Для отрицательных чисел это три разных ответа

Для положительных чисел `floor()` и `trunc()` часто совпадают, поэтому разница легко ускользает от внимания. Для отрицательных — не совпадают почти никогда, как видно на числовой прямой выше.

Практика: round, floor, ceil, trunc

Интерактивный ноутбук прямо в браузере — предскажите результат для отрицательных чисел

Открыть практику → (<https://www.cartesianschool.org/practice/04-15/index.html>)

Decimal — когда нужна точная десятичная арифметика

Для денег и учёта даже крошечная погрешность `float` может быть неприемлема — разберём инструмент, который её убирает.

Мы уже видели, что `float` хранит приближение (раздел 4.11). Для большинства задач это не проблема — но для денег даже крошечная погрешность накапливается и может стать заметной. Для этого в стандартной библиотеке есть модуль `decimal`.

`decimal_demo.py`

```
from decimal import Decimal

print(Decimal("0.1") + Decimal("0.2"))
# 0.3 — точно, без погрешности
```

Создавайте Decimal из строки, не из float

Это едва ли не самая важная практическая деталь во всём разделе:

`stroka_vs_float.py`

```
from decimal import Decimal

print(Decimal("19.99")) # Decimal('19.99') — именно
то, что вы написали
print(Decimal(19.99))   #
Decimal('19.990000000000000000002131628...') — унаследовал
погрешность float!
```

Почему так происходит

`Decimal(19.99)` сначала создаёт обычный `float` `19.99` — уже с тем самым приближением из раздела 4.11 — а потом заворачивает это приближённое значение в `Decimal`. Точность к этому моменту уже потеряна. `Decimal("19.99")` читает текст напрямую и не проходит через `float` вообще — поэтому сохраняет именно то значение, которое вы написали.

Пример: покупка

`покупка.py`

```
from decimal import Decimal

price = Decimal("19.99")
quantity = 3
total = price * quantity
print(total) # 59.97 — точно
```

Decimal — не «всегда лучше»

	<code>float</code>	<code>Decimal</code>
Скорость	Быстрый — аппаратная поддержка	Медленнее — программная реализация
Типичное применение	Научные расчёты, измерения, общего назначения	Деньги, бухгалтерия, где важна десятичная точность
Точность	Двоичное приближение	Точная десятичная запись

Не заменяйте каждый float на Decimal

Decimal — правильный выбор для денег и учёта, где важна десятичная точность и предсказуемое округление. Для большинства научных и повседневных вычислений обычный float быстрее и абсолютно достаточен.

Практика: Decimal и деньги

Интерактивный ноутбук прямо в браузере — `Decimal(str)` vs `Decimal(float)`

Открыть практику → (<https://www.cartesianschool.org/practice/04-16/index.html>)

Fraction — точные обычные дроби

Когда нужна не десятичная, а самая обычная точная дробь — числитель через знаменатель, без приближения.

Ещё один точный числовой инструмент из стандартной библиотеки — `Fraction`, обычная рациональная дробь «числитель / знаменатель», без какого-либо приближения.

```
fraction_demo.py
```

```
from fractions import Fraction

third = Fraction(1, 3)
print(third) # 1/3
```



1/3

`Fraction(1, 3)` — ровно одна треть, без приближения

Арифметика с `Fraction` — точная

`fraction_math.py`

```
from fractions import Fraction

result = Fraction(1, 3) + Fraction(1, 6)
print(result) # 1/2 — точно, не приближённо
```

`Fraction` из текста и из `float` — тоже разница

`fraction_stroka.py`

```
from fractions import Fraction

print(Fraction("0.1")) # 1/10 — прочитано из
# десятичного текста точно
print(Fraction(0.1)) #
# 3602879701896397/36028797018963968 — унаследовал
# приближение float!
```

Та же самая логика, что и с `Decimal` в разделе 4.14: если сначала создать `float`, приближение уже произошло раньше, чем `Fraction` успел его получить.

float, Decimal и Fraction рядом

Инструмент	Представляет	Точен для	Типичное применение
float	Двоичное приближение	Степеней двойки и их сумм	Наука, измерения, общие расчёты
Decimal	Десятичную дробь	Десятичных значений	Деньги, бухгалтерия
Fraction	Рациональное число p/q	Любой обычной дроби	Точная рациональная арифметика

float vs Decimal vs Fraction

Нужен обычный расчёт или измерение? → **float**

Нужна десятичная точность (деньги)? → **Decimal**

Нужна точная дробь p/q ? → **Fraction**

Три инструмента для трёх разных видов точности

Практика: Fraction и точные дроби

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/04-17/index.html>)

complex — комплексные числа

Последний из трёх встроенных числовых типов — для инженерных и научных задач, где нужна и действительная, и мнимая часть.

`complex` — тип для комплексных чисел: значений вида «действительная часть + мнимая часть». В Python мнимую единицу записывают буквой `j` (а не `i`, как в математике — инженерное соглашение, чтобы не путать с обозначением тока).

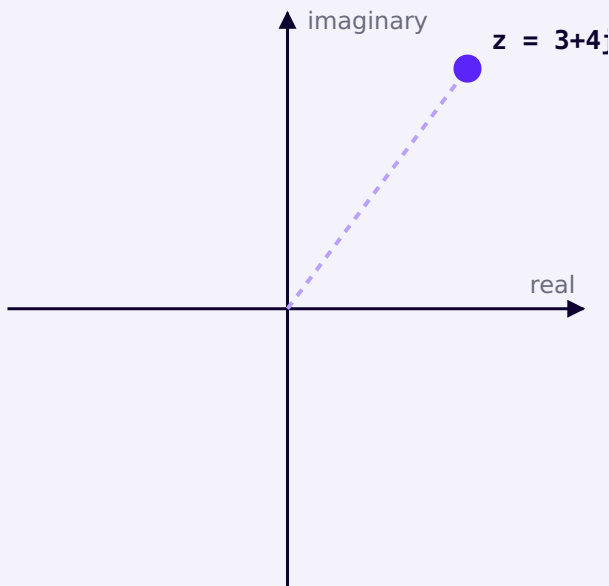
`complex_demo.py`

```
z = 3 + 4j
print(z)
print(type(z))
```

Действительная и мнимая часть

`real_imag.py`

```
z = 3 + 4j
print(z.real) # 3.0
print(z.imag) # 4.0
print(abs(z)) # 5.0 — длина вектора от начала координат
```



$z = 3 + 4j$ на комплексной плоскости

Где встречается на практике

В обычном прикладном коде комплексные числа — редкость. Но в некоторых областях они незаменимы:

- **Электротехника** — расчёт переменного тока;
- **Обработка сигналов** — преобразование Фурье и связанные вычисления;
- **Системы управления** — анализ устойчивости;
- **Физика** — квантовая механика, волновые процессы.

Если ни одна из этих областей не входит в ваши ближайшие планы — вполне нормально просто знать, что `complex` существует, и вернуться к этому разделу, когда он действительно понадобится.

cmath — математика для комплексных чисел

Обычный модуль `math` (раздел 4.18) не умеет работать с комплексными числами — для них есть отдельный модуль `cmath`:

```
cmath_demo.py
```

```
import cmath

z = 3 + 4j
print(cmath.phase(z)) # угол вектора z в радианах
```

Практика: комплексная плоскость

Интерактивный ноутбук прямо в браузере — `real`, `imag`, `abs()` для комплексных чисел

Открыть практику → (<https://www.cartesianschool.org/practice/04-18/index.html>)

Преобразование типов чисел

Числа можно превращать друг в друга и в текст — но не всегда наоборот, и не всегда так, как кажется на первый взгляд.

У каждого числового типа есть своя функция-преобразователь с тем же именем: `int()`, `float()`, `complex()`. Кроме того, любое число можно превратить в текст функцией `str()` — и наоборот, текст, похожий на число, можно превратить обратно.


```
preobrazovanie.py
```

```
whole = int(3.99)
print(whole)
# 3 – дробная часть отбрасывается, а не округляется

decimal = float(7)
print(decimal)    # 7.0

text = str(42)
print(text, type(text))    # "42" <class 'str'>
```

int() отбрасывает дробную часть, а не округляет

int(3.99) равно 3, а не 4 — это **усечение** (truncation) в сторону нуля, а не округление. Для настоящего округления нужна отдельная функция round() (раздел 4.13).

Направление усечения важно и для отрицательных чисел — int() всегда идёт к нулю, а не «вниз» в математическом смысле:

```
int_otricatelnoe.py
```

```
print(int(3.9))    # 3
print(int(-3.9))   # -3, а не -4
```

Преобразование текста в число

```
tekst_v_chislo.py
```

```
age_text = "10"
age = int(age_text)
print(age + 5)    # 15 – теперь это настоящее число
```

Не любой текст получится преобразовать

`int("десять")` вызовет `ValueError` — Python не умеет читать числа словами. Преобразовать можно только текст, который выглядит как число.

Разбор текста с указанием системы счисления

Мы уже видели это в разделе 4.3 — `int()` умеет читать текст не только в десятичной системе:

`s_osnovaniem.py`

```
print(int("1010", 2)) # 10
print(int("FF", 16)) # 255
```

Сборка строки из чисел: конкатенация → f-строка

Классический подход

```
age = 10
message = "Возраст: " +
str(age) + " лет"
print(message)
# обязательно вручную
оборачивать число в
str()
```

Современный Python 3.14

```
age = 10
message = f"Возраст:
{age} лет"
print(message)
# f-строка сама
выполняет преобразование
внутри {}
```

Что использовать сегодня: f-строки (f-strings), появившиеся в Python 3.6 и с тех пор ставшие стандартом. Они короче, меньше подвержены ошибкам (не нужно помнить о `str()`) и позволяют сразу писать выражения внутри фигурных скобок.

Практика: int(), float(), str() и парсинг с основанием

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/04-04/index.html>)

Модуль math

Готовый набор математических инструментов из стандартной библиотеки — организованный по смыслу, а не по алфавиту.

Модуль `math` из стандартной библиотеки — набор готовых математических инструментов, которые не нужно писать самостоятельно.

```
import.py
```

```
import math
```

```
print(math.sqrt(16)) # 4.0
```

math

КОНСТАНТЫ

`pi` · `e` · `tau` · `inf` · `nan`

КОРНИ И СТЕПЕНИ

`sqrt` · `isqrt` · `pow`

ОКРУГЛЕНИЕ

`floor` · `ceil` · `trunc`

ЦЕЛОЧИСЛЕННАЯ МАТЕМАТИКА

`factorial` · `gcd` · `lcm`
`comb` · `perm`

ГЕОМЕТРИЯ

`hypot` · `dist`

ТРИГОНОМЕТРИЯ

`sin` · `cos` · `tan`
`radians` · `degrees`

`math` как организованный набор возможностей, а не случайный список имён

Константы

konstanty.py

```
import math

print(math.pi)      # 3.141592653589793
print(math.e)        # 2.718281828459045
print(math.tau)      # 6.283185307179586 – это 2 * pi
```

Корни и степени

korni.py

```
import math

print(math.sqrt(16))    # 4.0 – квадратный корень
                        # (float)
print(math.isqrt(16))   # 4 – целочисленный
                        # квадратный корень (int)
```

Целочисленная математика

celochislennaya.py

```
import math

print(math.factorial(5)) # 120 – 5!
print(math.gcd(12, 18))  # 6 – наибольший общий
                        # делитель
print(math.lcm(4, 6))    # 12 – наименьшее общее
                        # кратное
print(math.comb(5, 2))   # 10 – число сочетаний
print(math.perm(5, 2))   # 20 – число размещений
```

Геометрия

geometriya.py

```
import math

print(math.hypot(3, 4))          # 5.0 – гипотенуза
    (длина вектора)
print(math.dist((0, 0), (3, 4))) # 5.0 – расстояние
    между точками
```

Тригонометрия — краткий обзор

Тригонометрические функции работают в **радианах**, а не градусах:

trig.py

```
import math

angle_deg = 90
angle_rad = math.radians(angle_deg)
print(math.sin(angle_rad)) # 1.0
```

Если сейчас тригонометрия не входит в ваши задачи — вполне нормально вернуться к этому подразделу позже.

Практика: модуль math

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/04-19/index.html>)

Случайные числа: random и secrets

Два модуля для случайности — и правило, которое стоит запомнить с первого дня: не всякая случайность одинаково безопасна.

Модуль `random` из стандартной библиотеки генерирует **псевдослучайные** числа — они выглядят случайными и достаточно хороши для игр, симуляций и учебных примеров.

```
random_demo.py
```

```
import random

print(random.randint(1, 6))  # случайное целое от 1
                             # до 6 включительно
print(random.random())
# случайное число от 0.0 до 1.0
print(random.choice(["орёл", "решка"])) # случайный
                                         # выбор из списка
```

Воспроизводимость через seed

Иногда нужно, чтобы «случайная» последовательность повторялась — например, для отладки. Для этого есть `seed()` :

```
seed_demo.py
```

```
import random

random.seed(42)
print(random.randint(1, 100)) # всегда одно и то же
                               # число при одном и том же seed
```



Выбор модуля зависит от того, поставлена ли
безопасность на карту

БЕЗОПАСНОСТЬ: random — не для паролей и токенов

Не генерируйте пароли и токены модулем random

`random` оптимизирован для скорости и статистического качества, а не для непредсказуемости против злоумышленника — его результат в принципе можно предсказать, зная внутреннее состояние генератора. Для всего «чувствительного к безопасности» (пароли, токены сессий, ключи) используйте модуль `secrets`.

secrets_demo.py

```
import secrets

token = secrets.token_hex(16)
print(token) # криптографически стойкий случайный
токен
```

Практика: random vs secrets

Интерактивный ноутбук прямо в браузере — выберите правильный инструмент для задачи

[Открыть практику](https://www.cartesianschool.org/practice/04-20/index.html) → (<https://www.cartesianschool.org/practice/04-20/index.html>)

statistics, inf и nan

Готовые статистические инструменты — и особые числовые состояния, которые полезно уметь распознавать.

Модуль statistics

Простые статистические расчёты уже есть в стандартной библиотеке — не нужно писать их вручную:

```
statistics_demo.py
```

```
import statistics

scores = [85, 90, 78, 92, 88]

print(statistics.mean(scores))    # 86.6 — среднее
print(statistics.median(scores))  # 88   — медиана
(среднее значение по порядку)
```

Мода и стандартное отклонение

mode_stdev.py

```
import statistics

scores = [85, 90, 78, 92, 88]

print(statistics.mode(scores))    # самое частое
                                  # значение
print(statistics.pstdev(scores))  # стандартное
                                  # отклонение генеральной совокупности
```

Это лишь маленькая часть модуля `statistics` — задача этого раздела не превратить курс в курс статистики, а показать: типовые статистические расчёты уже есть готовыми в Python, и не нужно писать формулу среднего вручную каждый раз.

Особые значения: `inf` и `nan`

Кроме обычных чисел, `float` умеет представлять несколько особых состояний:

```
inf_nan_demo.py
```

```
positive_infinity = float("inf")  
negative_infinity = float("-inf")  
not_a_number = float("nan")  
  
print(positive_infinity) # inf  
print(not_a_number)     # nan
```

Особые значения float

INF

$+\infty$

бесконечность

`math.isinf()`

-INF

$-\infty$

минус бесконечность

`math.isinf()`

NAN

не число

not a number

`math.isnan()`

Три особых состояния, которые может принимать float — проверяются тремя разными функциями

nan не равен даже самому себе

Одно из самых неожиданных свойств nan : сравнение `nan == nan` возвращает `False`. Это не ошибка Python, а часть стандарта IEEE 754 (тот же стандарт, что мы упоминали в разделе про float) — nan специально определён как «не равный ничему, включая себя».

nan_sravnenie.py

```
x = float("nan")
print(x == x) # False!
```

Как проверять эти состояния правильно

proverki.py

```
import math

x = float("nan")
y = float("inf")

print(math.isnan(x))      # True
print(math.isinf(y))      # True
print(math.isfinite(5.0)) # True — обычное конечное
                           число
```

Такие значения не выдумка — они реально появляются в данных и вычислениях: деление, дающее переполнение, отсутствующие данные при анализе, результат неопределённой операции. Полезно уметь распознать их и обработать осознанно, а не удивляться, откуда они взялись.

Практика: statistics, inf и nan

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/
practice/04-21/index.html\)](https://www.cartesianschool.org/practice/04-21/index.html)

Числовые ошибки и отладка

Девять коротких лабораторий — от настоящих исключений до валидных, но неожиданных числовых результатов.

Собрали девять коротких лабораторий — часть из них про настоящие ошибки с исключением, часть — про валидный, но неожиданный числовой результат. Важно уметь отличать эти два сценария.

Что пошло не так?

Python остановился с исключением

ZeroDivisionError
ValueError
TypeError



Читаем traceback и исправляем причину

Код выполнился без исключения

Но результат выглядит неожиданно



Проверяем математическую модель и
представление чисел



**Исправляем код, ожидания или выбор
числового типа**

Лаборатория 1. Почему $10 / 2$ даёт 5.0?

предскажите и запустите

```
print(10 / 2)
```

Наблюдение: результат — 5.0, а не 5.

Объяснение: оператор `/` в Python ВСЕГДА возвращает float (раздел 4.7), даже при делении нацело.

Лаборатория 2. Почему `int(3.99)` равно 3?

предскажите и запустите

```
print(int(3.99))
```

Наблюдение: результат — 3, а не 4.

Объяснение: `int()` отбрасывает дробную часть (усечение к нулю), а не округляет (раздел 4.16).

Лаборатория 3. Почему сравнение $0.1 + 0.2 == 0.3$ не срабатывает?

предскажите и запустите

```
print(0.1 + 0.2 == 0.3)
```

Наблюдение: результат — `False` .

Объяснение: `float` хранит приближение (раздел 4.11) — используйте `math.isclose()` вместо `==` .

Лаборатория 4. Почему `Decimal(0.1)` выглядит странно?

предскажите и запустите

```
from decimal import Decimal
print(Decimal(0.1))
```

Наблюдение: результат — длинная некруглая дробь, а не `0.1`.

Объяснение: аргумент — `float`, а значит приближение из раздела 4.11 уже произошло ДО создания `Decimal`. Используйте `Decimal("0.1")`.

Лаборатория 5. Почему $7 // 3$ отличается от $7 / 3$?

предскажите и запустите

```
print(7 // 3)
print(7 / 3)
```

Наблюдение: 2 против 2.333...

Объяснение: `//` — целочисленное деление (отбрасывает дробную часть результата), `/` — обычное деление, всегда float.

Лаборатория 6. Предскажите: $-7 // 3$ и $-7 \% 3$

предскажите и запустите

```
print(-7 // 3)
print(-7 % 3)
```

Наблюдение: -3 и 2 — не то, что многие ожидают.

Объяснение: `//` округляет к минус бесконечности, а не к нулю (раздел 4.7).

Лаборатория 7. Исправьте: `int("FF", 16)`

предскажите и запустите

```
print(int("FF"))
```

Наблюдение: `ValueError: invalid literal for int() with base 10: 'FF'`

Объяснение: без указания основания `int()` ожидает десятичную запись. Нужно: `int("FF", 16)`.

Лаборатория 8. Почините ZeroDivisionError

предскажите и запустите

```
price = 100  
quantity = 0  
print(price / quantity)
```

Наблюдение: ZeroDivisionError: division by zero.

Объяснение: перед делением стоит проверить делитель на ноль, если он может быть нулевым — подробно про условия будет в следующей главе; здесь достаточно узнать сам тип ошибки.

Лаборатория 9. Выберите верный числовой тип

предскажите и запустите

Задача: посчитать точную сумму чека в рублях и копейках

Наблюдение: `float` способен дать погрешность в дробных суммах.

Объяснение: для денег правильный выбор — `Decimal` (раздел 4.14), а не `float`.

Практика: отладка числовых ошибок

Интерактивный ноутбук прямо в браузере — найдите и исправьте типичные числовые проблемы

Открыть практику → (<https://www.cartesianschool.org/practice/04-22/index.html>)

Мини-проекты и итоги главы

Собираем всё пройденное в три небольших, но настоящих проекта — и подводим итоги главы финальной картой выбора числового типа.

Три небольших, но настоящих проекта — каждый использует только то, что мы уже прошли в этой главе (и в главе 3).

Проект А: калькулятор покупки

Вводим цену и количество, считаем сумму. Сравним float и Decimal на одном и том же примере:

```
kalkulyator_pokupki.py
```

```
from decimal import Decimal

# Версия на float — обычная, но с риском погрешности
price_float = 19.99
quantity = 3
print(price_float * quantity) # 59.97 — здесь
                              совпало, но не всегда так повезёт

# Версия на Decimal — предсказуемая точность для денег
price_decimal = Decimal("19.99")
print(price_decimal * quantity) # 59.97 — точно,
                              гарантированно
```

★ Базовая практика

Challenge

Добавьте налог (например, 23%) к сумме — версией на Decimal.

Проект В: конвертер времени

Вводим общее число секунд — считаем часы, минуты и секунды через // и % (раздел 4.7) — здесь у этих операторов появляется настоящее применение:

konverter_vremeni.py

```
total_seconds = 3725

hours = total_seconds // 3600
minutes = (total_seconds % 3600) // 60
seconds = total_seconds % 60

print(f"{hours}ч {minutes}м {seconds}с") # 1ч 2м 5с
```

★★ Самостоятельная задача

Challenge

Проверьте свою функцию на 0 секунд и на 90000 секунд (больше суток).

Проект С: геометрический калькулятор

Вводим радиус окружности — считаем диаметр, длину окружности и площадь через `math.pi`:

geometrisheskij.py

```
import math

radius = 5
diameter = radius * 2
circumference = 2 * math.pi * radius
area = math.pi * radius ** 2

print(f"Диаметр: {diameter}")
print(f"Длина окружности: {circumference:.2f}")
print(f"Площадь: {area:.2f}")
```

★★★ Задача повышенной сложности

Challenge · системы счисления

Дополнительно: напишите мини-«переводчик» — по введённому десятичному числу выведите его `bin()`, `oct()` и `hex()` (раздел 4.3).

Практика: мини-проекты главы 4

Интерактивный ноутбук прямо в браузере — калькулятор покупки, конвертер времени, геометрия

Открыть практику → (<https://www.cartesianschool.org/practice/04-05/index.html>)

Финальная карта: какой числовой тип мне нужен?

Какой числовой тип/инструмент мне нужен?

Нужен целый счёт?

→ **int**

Нужно обычное дробное измерение?

→ **float**

Нужна точная десятичная сумма (деньги)?

→ **Decimal**

Нужна точная дробь p/q ?

→ **Fraction**

Нужна действительная + мнимая часть?

→ **complex**

Нужны готовые
математические
функции?

→ **math / cmath**

Нужна случайность для игр/
симуляций?

→ **random**

Нужна случайность для
безопасности?

→ **secrets**

Главная сводная карта главы 4 — возвращайтесь к
ней в будущих проектах

Итоги главы

Что мы теперь умеем

- Понимаем разницу между математическим числом и его представлением в компьютере, и что имена указывают на неизменяемые числовые объекты — не «коробки».
- Уверенно работаем с `int`: произвольная точность, связь с `bool`, подчёркивания-разделители.
- Понимаем системы счисления — `decimal/binary/octal/hex` — и умеем конвертировать между ними.
- Знаем полный набор арифметических операторов и их порядок выполнения.
- Различаем `/`, `//` и `%` — и понимаем, как работает отрицательное `floor`-деление.
- Понимаем, почему `0.1 + 0.2` не равно ровно `0.3`, и умеем сравнивать `float` через `math.isclose()`.
- Умеем округлять числа через `round()`, `floor()`, `ceil()` и `trunc()` — и знаем их разницу для отрицательных чисел.
- Знаем, когда использовать `Decimal` (деньги) и `Fraction` (точные дроби) вместо `float`.
- Понимаем `complex` и комплексную плоскость, и умеем пользоваться `math/cmath`, `random/secrets` и `statistics`.
- Умеем распознавать `inf` и `nan`, и отлаживать типичные числовые ошибки.
- Собрали три мини-проекта, использующих числа Python для реальных задач.

Что дальше

В главе 5 мы продолжим работать с числами — сравнения, логические операции и первые условные конструкции.

Давайте поиграем с числами!

Глубокое погружение в математические вычисления Python: выражения, операторы и их приоритет, перевод формул из математики в код, модуль `math` (корни, геометрия, тригонометрия, логарифмы), модуль `random` (псевдослучайность, `seed`, воспроизводимость) и отладка вычислений.

5.1 Выражения и основные операции

5.2 Деление с остатком

5.3 Отрицательное floor-деление

5.4 Степени и корни

5.5 Унарные операторы

5.6 Присваивание и порядок вычислений

5.7 Ассоциативность операторов

5.8 Скобки и формулы из переменных

5.9 Перевод формул: математика → Python

5.10 Модуль `math` — карта возможностей

5.11 Корни и расстояния

5.12 gcd, lcm, факториал, comb, perm

5.13 Геометрия с math

5.14 Тригонометрия без страха

5.15 Логарифмы и экспоненты

5.16 Модуль random: псевдослучайность

5.17 randint, randrange, uniform

5.18 choice, choices, sample, shuffle

5.19 seed и воспроизводимость

5.20 Отладка вычислений

5.21 Мини-проекты и итоги

Выражения и основные операции

Выражение — операнды плюс оператор. Начнём с четырёх операций, знакомых из школы — но теперь увидим их не только в коде, а и на числовой прямой, и в виде прямоугольника.

В главе 4 мы разобрались, что такое числа. Теперь научим Python по-настоящему с ними работать — считать, сравнивать, комбинировать. Начнём с самого главного слова этой главы: **выражение**.

Что такое выражение

$5 + 3$ — это **выражение**: у него есть **операнды** (значения, с которыми работаем — 5 и 3) и **оператор** (действие — $+$). У любого выражения есть результат — само выражение можно подставить туда, где ожидается значение:

vyrazhenie.py

```
print(5 + 3)           # выражение прямо внутри print()
summa = 5 + 3          # результат выражения сохранён в
                        # переменную
print(summa * 2)       # summa — тоже операнд в новом
                        # выражении
```

Выражение — не то же самое, что команда (statement)

`summa = 5 + 3` целиком — это **инструкция** (присваивание), а $5 + 3$ внутри неё — **выражение**: у выражения всегда есть значение, у инструкции — нет. Именно поэтому `print(x = 5)` — ошибка (`x = 5` ничего не возвращает), а `print(x == 5)` — нормальный код.

Сложение и вычитание — движение по числовой прямой

Самая простая модель для $+$ и $-$ — прыжок по числовой прямой: сложение прыгает вправо, вычитание — влево.



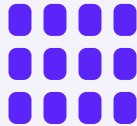
$3 + 4 = 7$ — прыжок на 4 шага вправо от точки 3

slozhenie.py

```
print(3 + 4)    # 7 — прыжок вправо
print(3 - 4)    # -1 — прыжок влево, можно уйти в
                отрицательные числа
```

Умножение — прямоугольник из клеток

Умножение $a * b$ — это площадь прямоугольника со сторонами a и b : ряды по a клеток, и таких рядов b штук.



$$4 \times 3 = 12$$

$4 \times 3 = 12$ — 3 ряда по 4 клетки

umnozhenie.py

```
print(4 * 3)    # 12
print(4 * 0)    # 0 — прямоугольник без высоты не
                имеет площади
```

Деление — всегда дробное число

В Python `/` — это **обычное деление**, и оно **всегда** возвращает `float`, даже если числа делятся нацело:

delenie.py

```
print(6 / 3)    # 2.0 – float, а не 2
print(7 / 2)    # 3.5
```

На ноль делить нельзя

5 / 0 вызывает ZeroDivisionError — Python не придумывает результат сам, а честно сообщает, что операция не имеет смысла.

Операторы и реальная жизнь

Оператор	Что делает	Пример из жизни
+	сложение	сумма чеков в корзине
-	вычитание	остаток на счету после покупки
*	умножение	цена × количество товара
/	деление	сумма счёта, делённая между друзьями

Про // (целочисленное деление) и % (остаток) — в следующем разделе: это отдельная и очень важная тема.

Практика: выражения и +, -, *, /

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-01/index.html>)

Деление с остатком

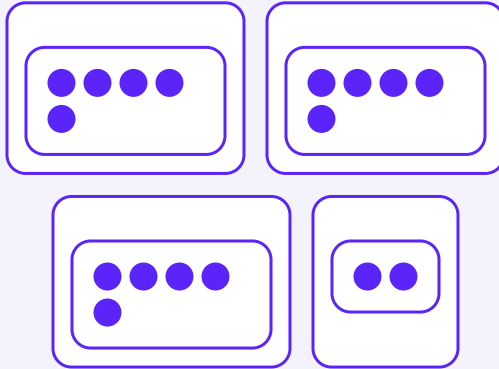
Раскладываем числа по группам — и знакомимся с двумя операторами, которых нет на обычном калькуляторе.

Кроме обычного деления `/`, у Python есть два оператора, которых нет на школьном калькуляторе, но без них не обходится почти ни одна программа.

Целочисленное деление `//`

Возвращает только целую часть результата — «сколько раз одно число помещается в другое целиком».

Представим 17 конфет, которые раскладываем по пакетикам по 5 штук:



$$17 // 5 = 3, 17 \% 5 = 2$$

17 конфет по 5 в пакетике — 3 полных пакетика и 2 конфеты остались

celochislennoe.py

```
print(17 // 5)    # 3 – три полных пакетика
print(17 / 5)     # 3.4 – обычное деление, для
                 # сравнения
```

Остаток от деления %

Возвращает то, что **осталось** после раскладывания по пакетикам — на схеме выше это 2 конфеты. Один из самых полезных операторов в программировании: например, для проверки чётности числа.

ostatok.py

```
print(17 % 5)     # 2 – именно столько осталось
print(10 % 2)     # 0 – если остаток 0, число чётное
print(11 % 2)     # 1 – а если 1 – нечётное
```

Проверка чётности — классический приём

число % 2 == 0 — самый частый способ проверить, чётное ли число. Этот приём встретится ещё много раз в курсе.

divmod() — оба результата сразу

Если нужны и целая часть, и остаток одновременно, не обязательно писать // и % отдельно — есть встроенная функция divmod():

divmod_primer.py

```
print(divmod(17, 5)) # (3, 2) – кортеж: целая часть
                    # и остаток
chast, ostatok = divmod(17, 5)
print(chast, ostatok) # 3 2
```

Реальные примеры

Задача	Выражение	Результат
Сколько полных команд по 4 человека из 22	<code>22 // 4</code>	5
Сколько человек останется без команды	<code>22 % 4</code>	2
Сколько часов и минут в 137 минутах	<code>divmod(137, 60)</code>	(2, 17)

Практика: `//`, `%` и `divmod()`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-07/index.html>)

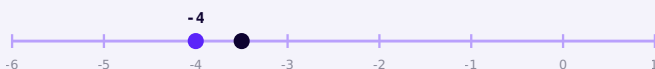
Отрицательное floor-деление

Самая частая неожиданность в этой теме — что делает `//` с отрицательными числами. Разбираемся на числовой прямой.

Что будет, если разделить с остатком **отрицательное** число? Здесь Python ведёт себя иначе, чем многие ожидают из школы — и это важно понять сейчас, а не в момент отладки.

Округление всегда вниз — то есть к минус бесконечности

// в Python называется *floor division* — «деление с округлением к полу» (floor). Пол — это округление в сторону **меньшего** числа, и для отрицательных чисел «меньшее» означает «более отрицательное»:



$-7 // 2 = -4$ — округление к полу означает движение ВЛЕВО по числовой прямой, к $-\infty$

otricatelnoe_floor.py

```
print(-7 // 2)    # -4, а не -3! Python округляет к
минус бесконечности
print(7 // 2)     # 3 — для положительных чисел
разницы с округлением 'к нулю' нет
print(-7 / 2)    # -3.5 — обычное деление честно
показывает дробную часть
```

Не 'округление к нулю', как в некоторых языках

В части других языков программирования `-7 // 2` дало бы `-3` (округление к нулю). В Python результат — `-4`: это осознанный выбор языка, чтобы формула ниже работала всегда, без исключений для отрицательных чисел.

Остаток при отрицательном делении

Знак остатка в Python всегда совпадает со знаком делителя:

```
otricatelnyj_ostatok.py
```

```
print(-7 % 2)      # 1 – остаток положителен, потому
                  # что делитель (2) положителен
print(7 % -2)      # -1 – остаток отрицателен, потому
                  # что делитель (-2) отрицателен
```

Тождество, которое работает всегда

Как бы ни менялись знаки, для любых `a` и `b` (кроме деления на 0) верно:

$$\begin{aligned}
 a &== (a // b) * b + a \% b \\
 &\downarrow \\
 -7 &== (-7 // 2) * 2 + (-7 \% 2) \\
 &\downarrow \\
 -7 &== (-4) * 2 + 1 \\
 &\downarrow \\
 -7 &== -8 + 1 \\
 &\downarrow \\
 -7 &== -7 \checkmark
 \end{aligned}$$

Тождество деления с остатком проверено на примере -7 и 2

tozhdestvo.py

```
a, b = -7, 2
print((a // b) * b + a % b == a)  # True — тождество
работает всегда
```

Практика: отрицательное floor-деление

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/05-08/index.html) → (https://www.cartesianschool.org/
practice/05-08/index.html)

Степени и корни

Возведение в степень и извлечение корня — одна и та же геометрическая картинка, прочитанная в двух направлениях.

Теперь — операция, у которой самая красивая геометрическая картинка из всех: возведение в степень.

Квадрат числа — это площадь квадрата

$x ** 2$ — площадь квадрата со стороной x :



16

сторона = 4

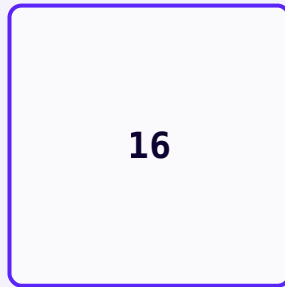
$4 ** 2 = 16$ — площадь квадрата со стороной 4

kvadrat.py

```
print(4 ** 2)    # 16
print(4 ** 3)    # 64 — куб: та же идея, но в трёх
                 # измерениях (объём)
```

Квадратный корень — обратная операция: от площади к стороне

Если известна площадь квадрата, можно найти длину его стороны — это и есть квадратный корень. Та же картинка, только читаем её в обратную сторону: у нас есть число 16 внутри квадрата, и мы ищем сторону.



сторона = 4

$\sqrt{16} = 4$ — обратная задача: дана площадь, ищем сторону

koren.py

```
print(16 ** 0.5)      # 4.0 — степень 0.5 и есть
                      # квадратный корень
import math
print(math.sqrt(16))  # 4.0 — то же самое, но явно и
                      # понятно для читателя
```

math.sqrt() читается лучше, чем ** 0.5

Оба варианта работают одинаково правильно, но `math.sqrt(16)` сразу говорит читателю кода: «здесь ищут корень». `16 ** 0.5` заставляет вспоминать, что 0.5 — это корень.

Три способа возвести в степень — и в чём разница

Способ	Что возвращает	Пример
**	int, если оба операнда int; иначе float	<code>2 ** 10</code> → 1024
pow(a, b)	то же самое, что <code>a ** b</code>	<code>pow(2, 10)</code> → 1024
math.pow(a, b)	всегда float, даже для целых чисел	<code>math.pow(2, 10)</code> → 1024.0

У pow() есть и третий, необязательный аргумент

`pow(a, b, m)` считает $(a ** b) \% m$, но намного быстрее — для больших чисел это ускоряет работу в тысячи раз. Используется, например, в криптографии. Пока достаточно знать, что такая возможность существует.

Практика: степени и корни

Тот же ноутбук, что закрепляет `//`, `%` и `**` вместе — самое время попрактиковать все специальные операторы разом

Открыть практику → (<https://www.cartesianschool.org/practice/05-02/index.html>)

Унарные операторы

Один операнд, а не два — и одна очень известная ловушка приоритета операций.

`-5` — это не «минус пять как отдельное число», а применение **унарного** оператора `-` к числу `5`. Унарный — значит «с одним операндом», в отличие от привычного бинарного вычитания `(8 - 3)`.

unarnyj.py

```
x = 5
print(-x)      # -5 — унарный минус меняет знак
print(+x)      # 5 — унарный плюс почти ничего не
               # делает (редко используется)
print(8 - 3)   # 5 — а это бинарное вычитание, два
               # операнда
```

Классическая ловушка: `-2 ** 2`

Многие ожидают `-4`, но подозревают `4` (как будто сначала считается `(-2) ** 2`). Проверим:

lovushka.py

```
print(-2 ** 2)   # -4
print((-2) ** 2) # 4 — другое число!
```

()

скобки — всегда первыми

****** возведение в степень

+X -X унарный плюс/минус

*** / //** умножение, деление,
% целочисленное деление, остаток

+ - сложение, вычитание

****** стоит **ВЫШЕ** унарного минуса — поэтому $-2 ** 2$
сначала возводит 2 в квадрат

$-2 ** 2$



$-(2 ** 2)$



$-(4)$



-4

****** выполняется раньше унарного минуса — сначала
 $2 ** 2 = 4$, потом применяется минус

**** — единственное исключение такого рода**

У всех остальных операторов приоритет ведёт себя ожидаемо. Именно из-за этой особенности `**` опытные разработчики почти всегда пишут `(-2)**2` явно, скобками — даже когда приоритет формально понятен.

Практика: унарные операторы

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-09/index.html>)

Присваивание и порядок вычислений

Более короткая запись для изменения переменных — и полная карта того, что выполняется раньше, а что позже.

Операции присваивания

Изменить переменную «на основе самой себя» — очень частое действие: например, увеличить счёт в игре. Писать `score = score + 10` можно, но Python предлагает более короткую запись:

`priswaivanie.py`

```
score = 0
score += 10 # то же самое, что score = score + 10
print(score) # 10

score -= 3 # score = score - 3
print(score) # 7
```

Это создаёт НОВЫЙ объект, а не меняет старый

Как мы выяснили в главах 3 и 4, числа в Python неизменяемы. `score += 10` не «дописывает» 10 к существующему объекту 0 — Python вычисляет `score + 10`, создаёт новый объект 10 и переподвязывает имя `score` к нему. Снаружи это выглядит как изменение, но внутри это ровно тот же механизм, что мы видели в главе 4 для `age = age + 1`.

Такие сокращения существуют для всех основных операторов:

Сокращённая запись	Полная запись
<code>x += n</code>	<code>x = x + n</code>
<code>x -= n</code>	<code>x = x - n</code>
<code>x *= n</code>	<code>x = x * n</code>
<code>x /= n</code>	<code>x = x / n</code>
<code>x //= n</code>	<code>x = x // n</code>
<code>x %= n</code>	<code>x = x % n</code>
<code>x **= n</code>	<code>x = x ** n</code>

Что выполняется первым?

Мы уже видели кусочки этой картины по отдельности — теперь соберём полную карту приоритета операторов главы 5, от высшего к низшему:

()

скобки — всегда первыми

****** возведение в степень

+X -X унарный плюс/минус

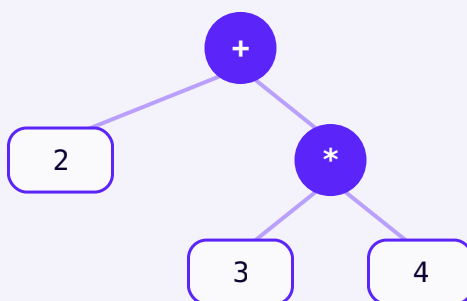
*** / //** умножение, деление,
% целочисленное деление, остаток

+ - сложение, вычитание

Полная карта приоритета операторов этой главы —
от высшего к низшему

poryadok.py

```
print(2 + 3 * 4)      # 14 — сначала 3 * 4 = 12, потом
+ 2                  # 14 + 2 = 16
print((2 + 3) * 4)    # 20 — скобки меняют порядок
```



$2 + 3 * 4 = 14$ — умножение выполняется глубже в дереве, то есть раньше

Дерево показывает то же самое, что и приоритет: `*` вложено глубже, поэтому вычисляется первым — его результат становится одним из операндов `+`.

Сомневаетесь — используйте скобки

Даже когда порядок формально понятен, скобки делают код честнее для человека, который будет его читать (в том числе для вас через полгода).

$(a + b) * c$ читается быстрее, чем приходится вспоминать правила приоритета.

Практика: присваивание и порядок вычислений

Тот же ноутбук, что и в разделе «Степени и корни» — он охватывает и эту тему

Открыть практику → (<https://www.cartesianschool.org/practice/05-02/index.html>)

Ассоциативность операторов

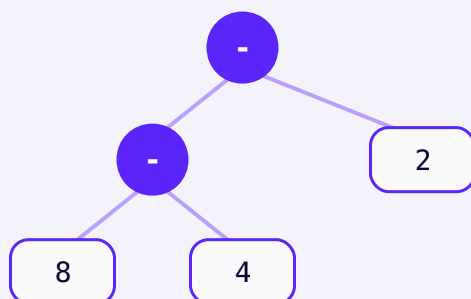
Приоритет говорит, какой оператор раньше.

Ассоциативность говорит, в каком порядке считать несколько одинаковых по приоритету операторов подряд.

Приоритет отвечает на вопрос «какой оператор раньше», а **ассоциативность** — на вопрос «в каком порядке считать несколько операторов ОДНОГО приоритета подряд».

Большинство операторов — слева направо

$8 - 4 - 2$ считается как $(8 - 4) - 2$, то есть слева направо. Это называется **левой ассоциативностью**, и так ведут себя почти все операторы Python: `+` `-` `*` `/` `//` `%`.



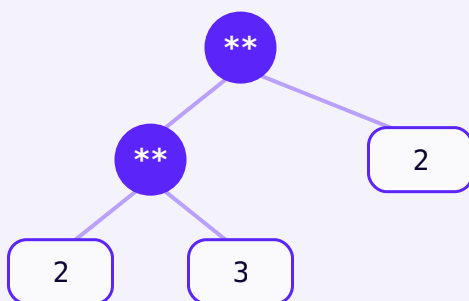
8 - 4 - 2 — левоассоциативно: сначала левая пара (8 - 4), потом результат минус 2

levaya.py

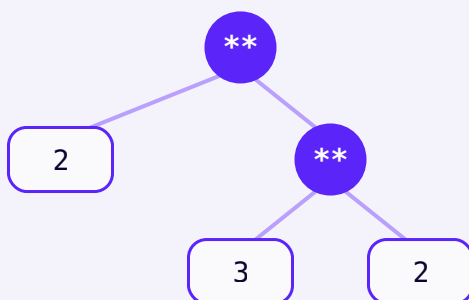
```
print(8 - 4 - 2)
# 2, потому что это (8 - 4) - 2 = 4 - 2
print((8 - 4) - 2) # 2 — то же самое явно
print(8 - (4 - 2)) # 6 — а вот это другое число!
```

Исключение: ** — справа налево

Возведение в степень — единственный оператор в этой главе с **правой** ассоциативностью. Сравним, что дал бы каждый вариант для `2 ** 3 ** 2`:



Если бы ** было левоассоциативным: $(2 ** 3) ** 2 = 64$ — но это НЕ то, что делает Python



На самом деле ** правоассоциативно: $2 ** (3 ** 2) = 512$

```
stepen_associativnost.py
```

```
print(2 ** 3 ** 2)
# 512 – Python считает как 2 ** (3 ** 2)
print((2 ** 3) ** 2) # 64 – левоассоциативный
вариант дал бы другое число
print(2 ** (3 ** 2)) # 512 – совпадает с реальным
поведением Python
```

2 ** 3 ** 2



2 ** (3 ** 2)



2 ** 9



512

Правая ассоциативность: сначала считается
ВЕРХНИЙ (правый) показатель степени

Зачем ** вообще так сделали

Правая ассоциативность степени совпадает с математической традицией «башни степеней» (*tetration*): в математике a^{b^c} всегда читается как $a^{(b^c)}$, а не наоборот. Python здесь просто следует давнему математическому соглашению.

Практика: ассоциативность операторов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-10/index.html>)

Скобки и формулы из переменных

От отдельных операторов — к настоящим формулам с понятными именами, как в реальном коде.

Мы уже видели, что скобки меняют порядок вычислений. Но у скобок есть и вторая роль — не менее важная: они делают формулы понятными для **человека**, даже когда приоритет и так дал бы правильный результат.

Скобки для людей, а не только для Python

skobki_dlya_lyudej.py

```
# Технически скобки не нужны — приоритет и так верный:
srednee = 4 + 7 + 9 / 3
print(srednee)      # 14.0 — НЕПРАВИЛЬНО! Это 4 + 7 +
                    # (9 / 3), не то, что имелось в виду
```

```
# А вот с явными скобками результат соответствует
замыслу:
srednee = (4 + 7 + 9) / 3
print(srednee)      # 6.666... — среднее трёх чисел
```

Забытые скобки — источник реальных ошибок

Пример выше — не выдуманная ловушка: «забыть скобки вокруг суммы перед делением» — одна из самых частых ошибок начинающих при переводе формулы в код.

Формулы из именованных переменных

Настоящий код почти никогда не пишут с «голыми» числами — используют переменные с понятными именами. Это делает формулу читаемой и легко изменяемой:

```
formula_pryamougolnika.py
```

```
shirina = 6
vysota = 3

perimetr = 2 * (shirina + vysota)
ploshad = shirina * vysota

print(f"Периметр: {shirina + shirina}")
print(f"Периметр: {perimetr}")
print(f"Площадь: {ploshad}")
```

Опечатка выше — специально

Обратите внимание на первую строку `print()` в примере: `shirina + shirina` вместо `perimetr` — она не выдаёт ошибку, но выдаёт неверное число.

Формула, которая не падает, но считает не то, что задумано — самая коварная ошибка в вычислениях. Мы разберём такие ошибки подробно в разделе про отладку вычислений.

Ещё три классические формулы

eshche_formuly.py

```
# скорость = расстояние / время
rasstoyanie_km = 120
vremya_ch = 2
skorost = rasstoyanie_km / vremya_ch
print(f"Скорость: {skorost} км/ч")

# перевод температуры из Цельсия в Фаренгейт
celsius = 20
fahrenheit = celsius * 9 / 5 + 32
print(f"{celsius}°C = {fahrenheit}°F")

# простые проценты: сумма * ставка * годы / 100
summa = 1000
stavka = 5
gody = 3
procenty = summa * stavka * gody / 100
print(f"Проценты: {procenty}")
```



1/3

Среднее трёх равных долей — та же идея, что и деление на 3 в формуле выше

Практика: скобки и формулы из переменных

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-11/index.html>)

Перевод формул: математика → Python

Учебник записывает формулы иначе, чем Python. Разбираем правила перевода — и самые частые ошибки при этом переводе.

Формулы в учебниках математики записаны иначе, чем код Python — умножение часто вообще не пишут, степень поднимают над строкой, деление рисуют чертой. Переводить такие формулы в Python нужно осторожно.

Таблица перевода

В математике	В Python	Частая ошибка
$2x$	<code>2 * x</code>	написать <code>2x</code> — <code>SyntaxError</code> , умножение никогда не подразумевается
x^2	<code>x ** 2</code>	написать <code>x^2</code> — сработает, но даст совсем не то число!
$a+bc$	<code>(a + b) / c</code>	написать <code>a + b / c</code> — разделится только <code>b</code> , не вся сумма
x	<code>x ** 0.5</code> или <code>math.sqrt(x)</code>	забыть, что корень — это тоже степень (0.5)
πr^2	<code>math.pi * radius ** 2</code>	забыть импортировать <code>math</code> , или написать <code>pi</code> вместо <code>math.pi</code>

В математике	В Python	Частая ошибка
$a(b+c)$	<code>a * (b + c)</code>	написать <code>a(b + c)</code> — Python решит, что <code>a</code> это функция, и попытается её вызвать
$a+bc+d$	<code>(a + b) / (c + d)</code>	написать <code>a + b / c + d</code> — разделится только <code>b</code> на <code>c</code>

^ в Python — это НЕ степень

В математике $\wedge$ иногда означает возведение в степень. В Python $\wedge$ — совсем другой оператор (побитовое исключающее ИЛИ, разберём его в главе про биты): `5 ^ 3` равно `6`, а не `125`. Для степени всегда используйте `**`.

```
proverka_karety.py
```

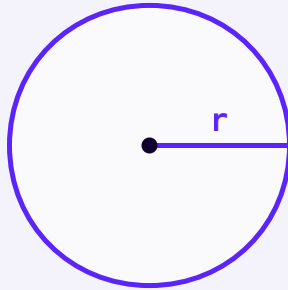
```
print(5 ** 3)    # 125 — степень
print(5 ^ 3)     # 6 — совсем другая операция!
```

От математики — через геометрию — к Python

Таблица показывает перевод построчно. Но у формулы $A=\pi r^2$ есть третий, самый наглядный уровень — сама геометрическая фигура, которую формула описывает:

$$A=\pi r^2$$

$A = \pi r^2$ — площадь круга через его радиус



Радиус r соединяет центр круга с его краем — из него вычисляется площадь $A = \pi r^2$

ploshad_kruga_perevod.py

```
import math

radius = 5
ploshad = math.pi * radius ** 2
print(round(ploshad, 2))  # 78.54
```

radius ** 2, а не (radius) ** 2

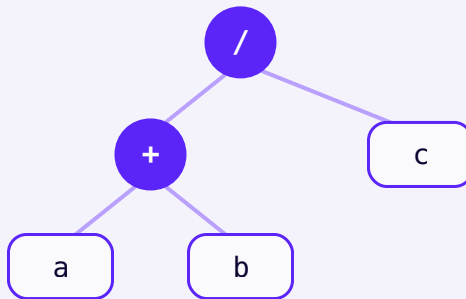
Скобки вокруг одиночной переменной не нужны — ** уже применяется только к `radius`, а не ко всему выражению `math.pi * radius` (умножение и степень — разные уровни приоритета, степень выше).

Почему скобки вокруг суммы обязательны

Вернёмся к дроби $a+bc$ и посмотрим на неё с трёх сторон: как формулу, как дерево вычислений и как код.

$$a+bc$$

Черта дроби ГРУППИРУЕТ $a + b$ в один числитель
— это визуально очевидно



$(a + b) / c$ — дерево показывает, что вся сумма должна оказаться ПОД чертой дроби

Дерево показывает то же самое, что и черта дроби: $a + b$ — единый блок, который целиком становится числителем. В Python черты нет, поэтому эту группировку нужно обозначить явно — круглыми скобками:

```
pochemu_skobki.py
```

```
a, b, c = 8, 4, 3

# ПРАВИЛЬНО — скобки воспроизводят черту дроби:
print((a + b) / c)    # 4.0

# НЕПРАВИЛЬНО — без скобок делится только b:
print(a + b / c)      # 9.333... — это a + (b / c),
# совсем другая формула!
```

Пример перевода: формула среднего

Формула среднего трёх чисел $a+b+c$ переводится в Python по тому же правилу — вся сумма должна попасть в скобки, прежде чем делить:

```
srednee_perevod.py
```

```
a, b, c = 4, 7, 9
srednee = (a + b + c) / 3
print(srednee)    # 6.666...
```

Практика: перевод формул из математики в Python

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-12/index.html>)

Модуль math — от каталога к инструментам

Что такое модуль, зачем нужен `import` и что означает точка в `math.sqrt()` — прежде чем нырять в конкретные функции, разберёмся, как вообще устроен этот ящик с инструментами.

Python уже кое-что умеет считать без всякой подготовки:

vstroennoe.py

```
print(abs(-7))      # 7
print(round(4.5))   # 4
print(pow(2, 10))   # 1024
```

Но настоящей математики в мире куда больше, чем четыре встроенные функции: корни, синусы, логарифмы, работа с углами и расстояниями. Если бы **все** такие инструменты были встроены прямо в язык — Python превратился бы в загромождённый список из сотен имён, и запомнить, что вообще доступно, стало бы невозможно.

Модуль — это отдельный, подписанный ящик с инструментами

Вместо этого стандартная библиотека Python организует связанные инструменты в **модули** — отдельные «ящики», которые лежат рядом, но не смешиваются с языком напрямую и друг с другом. Один из таких ящиков называется `math` — в нём собраны математические инструменты: корни, тригонометрия, логарифмы и многое другое.

Python: то, что встроено, и то, что лежит в модулях

ВСТРОЕНО В ЯЗЫК — ДОСТУПНО ВСЕГДА

```
print() · abs()
round() · pow()
не требует import
```

МОДУЛЬ MATH — ОТДЕЛЬНЫЙ ЯЩИК С ИНСТРУМЕНТАМИ

```
sqrt() · hypot() · gcd()
sin() · log() · pi
требуется import math
```

Модуль — это отдельный, заранее подписанный ящик с инструментами, а не часть самого языка

import math — открываем ящик

Чтобы воспользоваться инструментами из ящика `math`, его нужно сначала явно **открыть** — командой `import`:

`import_math.py`

```
import math
```

```
# теперь всё содержимое ящика math доступно в этой
программе
print(math.sqrt(2))
```

import math — это не «включить математику»

Строка `import math` примерно означает: «сделай содержимое модуля `math` доступным в этой программе». Без неё `math.sqrt(25)` вызовет `NameError` — Python ещё не знает, где искать `math`.

Что означает точка

Разберём запись `math.sqrt(25)` по частям:

math . sqrt (25)

модуль

где искать

инструмент

что делать

аргумент

с чем работать

Точка означает «возьми sqrt ИЗ модуля math» — это не украшение синтаксиса, а указание адреса

Точка здесь значит «возьми sqrt **из** модуля math» — так Python отличает, например, math.sqrt от какой-нибудь другой функции с похожим именем из совсем другого модуля: у каждого инструмента есть свой явный адрес.

А что если аргумент не нужен: math.pi

math.pi — не функция, а готовое **значение** (константа): число π уже вычислено и лежит внутри модуля под именем pi. Поэтому у него нет круглых скобок — скобки означают «вызови функцию», а pi вызывать не нужно, его нужно просто прочитать:

```
math_pi.py
```

```
import math
```

```
print(math.pi)           # 3.141592653589793 — значение,  
не вызов
```

```
print(math.sqrt(25))     # 5.0 — а это вызов функции,  
поэтому есть скобки
```

Первые эксперименты

Несколько маленьких примеров — код и его результат:

```
eksperiment_1.py
```

```
import math  
  
print(math.sqrt(25))
```

РЕЗУЛЬТАТ: 5.0 — квадратный корень из 25.

```
eksperiment_2.py
```

```
import math  
  
print(math.pi)
```

РЕЗУЛЬТАТ: 3.141592653589793 — число π с точностью float.

```
eksperiment_3.py
```

```
import math  
  
print(math.gcd(18, 24))
```

РЕЗУЛЬТАТ: 6 — наибольший общий делитель 18 и 24.

```
eksperiment_4.py
```

```
import math  
  
print(math.hypot(3, 4))
```

РЕЗУЛЬТАТ: 5.0 — длина гипотенузы прямоугольного треугольника со сторонами 3 и 4.

Встроенное vs math — не путаем

Ещё раз проведём чёткую границу — какие инструменты встроены в язык напрямую, а какие нужно явно импортировать, и откуда именно:

Инструмент	Откуда	Нужен import?
<code>abs()</code> , <code>round()</code> , <code>pow()</code>	встроены в язык	нет
<code>math.sqrt()</code> , <code>math.floor()</code> , <code>math.sin()</code> , <code>math.log()</code>	модуль <code>math</code>	<code>import math</code>
<code>Decimal</code>	модуль <code>decimal</code> (глава 4)	<code>from decimal</code> <code>import Decimal</code>
<code>Fraction</code>	модуль <code>fractions</code> (глава 4)	<code>from fractions</code> <code>import Fraction</code>

Decimal и Fraction — НЕ часть math

Это частая путаница: `Decimal` и `Fraction` из главы 4 — инструменты для точных чисел, но живут в СВОИХ собственных модулях (`decimal` и `fractions`), отдельно от `math`. Три разных ящика — три разных импорта.

Полная карта: что умеет math

Теперь, когда понятно, что такое модуль и зачем нужна точка, можно спокойно посмотреть на карту того, что `math` предлагает — следующие разделы главы разберут каждую карточку подробно, с реальными задачами:

Что умеет модуль math

КОРНИ И РАССТОЯНИЯ

`sqrt` · `isqrt`

hypot · dist

ЦЕЛОЧИСЛЕННАЯ МАТЕМАТИКА

gcd · lcm

factorial · comb · perm

ГЕОМЕТРИЯ

pi

hypot · dist

ТРИГОНОМЕТРИЯ

sin · cos · tan

radians · degrees

ЛОГАРИФМЫ И РОСТ

log · log2 · log10 · exp

Каждая карточка — отдельный раздел ниже, с реальными задачами и картинками

Как искать то, чего нет на карте

У `math` около полусотни функций — эта карта показывает только самые полезные для начала. Никто не держит в голове весь список, и это нормально: профессиональный навык — не «знать всё наизусть», а знать **примерно**, что существует, и уметь быстро найти точную деталь.

Официальная документация — не враг, а инструмент

Полный и точный список всего, что есть в `math`, — на странице официальной документации Python 3.14: docs.python.org/3.14/library/math (Standard Library → `math`). Опытные разработчики заглядывают туда постоянно — это не признак незнания, а нормальная часть работы.

Практика: модуль `math` — что такое модуль и первые вызовы

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-04/index.html>)

Корни и расстояния

От теоремы Пифагора вручную — к готовым функциям, которые считают расстояние за вас.

`isqrt()` — целочисленный корень

`math.sqrt()` всегда возвращает `float`. Если нужен именно `int` — например, для чисел настолько больших, что `float` начинает терять точность — есть `math.isqrt()`: он отбрасывает дробную часть и работает только с целыми числами.

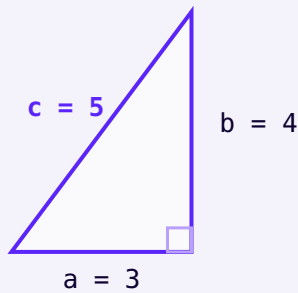
`isqrt.py`

```
import math

print(math.sqrt(10))      # 3.1622776601683795
print(math.isqrt(10))     # 3 — целая часть корня, без
                          # дробей и без потери точности
```

`hypot()` — гипотенуза без Пифагора вручную

Если известны длины двух катетов прямоугольного треугольника, длину гипотенузы можно найти формулой `c = sqrt(a**2 + b**2)` — но `math` уже умеет это одной функцией:



`hypot(3, 4) = 5` — классический прямоугольный треугольник 3-4-5

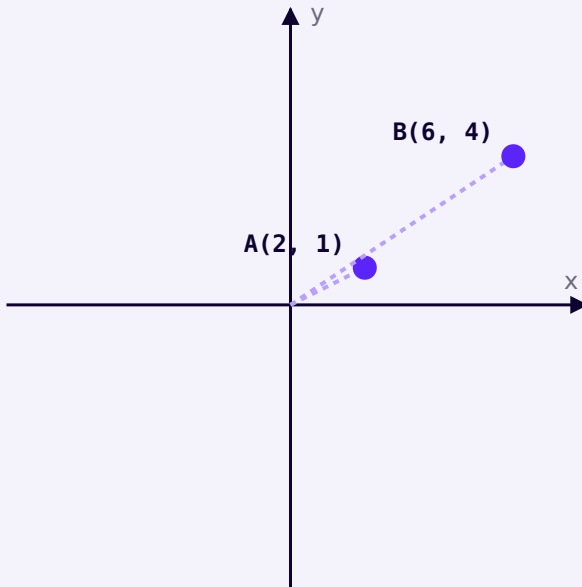
`hypot.py`

```
import math

print(math.sqrt(3 ** 2 + 4 ** 2))
# 5.0 — вручную по теореме Пифагора
print(math.hypot(3, 4))           # 5.0 — то же
# самое, короче и точнее
```

`dist()` — расстояние между двумя точками

`math.dist()` — это `hypot()`, но для двух точек на плоскости (или в пространстве), а не для одного треугольника: он сам вычисляет катеты как разницу координат.



Расстояние между A и B — это гипотенуза прямоугольного треугольника, построенного на их координатах

dist.py

```
import math

A = (2, 1)
B = (6, 4)
print(math.dist(A, B))  # 5.0 — то же самое
                        расстояние, посчитанное из координат
```

Откуда взялось число 5.0

Разница по x: $6 - 2 = 4$. Разница по y: $4 - 1 = 3$. Это ровно катеты треугольника 3-4-5 выше — `dist()` под капотом делает именно то же самое, что `hypot()`, только сам считает катеты за вас.

Практика: корни и расстояния

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-13/index.html>)

gcd, lcm, факториал, comb, perm

Пять функций для работы с целыми числами и подсчёта вариантов — с реальными задачами для каждой.

gcd() — наибольший общий делитель

Наибольший общий делитель (НОД) двух чисел — самое большое число, на которое оба делятся без остатка.

Классическое применение — сокращение дробей:

gcd.py

```
import math

print(math.gcd(12, 18))    # 6 — и 12, и 18 делятся на
                           # 6 без остатка

# сократить дробь 12/18:
chislitel, znamenatel = 12, 18
nod = math.gcd(chislitel, znamenatel)
print(f"{chislitel // nod}/{znamenatel // nod}")    #
2/3 — та же дробь, но в простейшем виде
```

lcm() — наименьшее общее кратное

Наименьшее общее кратное (НОК) — наименьшее число, которое делится на оба числа сразу. Пригодится, например, чтобы понять, когда снова совпадут два повторяющихся события:

lcm.py

```
import math

print(math.lcm(4, 6))
# 12 — автобус №1 ходит каждые 4 минуты, автобус №2 —
# каждые 6
# оба будут на остановке
одновременно каждые 12 минут
```

factorial() — считаем перестановки

Факториал $n!$ — произведение всех целых чисел от 1 до n . Отвечает на вопрос «сколькими способами можно расставить n разных предметов по порядку»:

factorial.py

```
import math

print(math.factorial(5))  # 120 — 5 книг на полке
                           можно расставить 120 разными способами
```

comb() и perm() — выбор и порядок

comb(n, k) — сколькими способами выбрать k предметов из n, если порядок выбора **не важен**.
 perm(n, k) — то же самое, но когда порядок **важен**.

comb_perm.py

```
import math

# сколько разных троек чисел можно выбрать из
# лотерейных 49, не важен порядок:
print(math.comb(49, 3))  # 18424

# сколько разных подиумов (1, 2, 3 место) можно
# составить из 10 бегунов, порядок важен:
print(math.perm(10, 3))  # 720
```

perm() всегда больше или равен comb()

Для одних и тех же n и k perm() считает КАЖДЫЙ порядок отдельно, а comb() — только уникальные наборы. Если сомневаетесь, какую функцию использовать, задайте себе вопрос: «важен ли порядок?» — да → perm(), нет → comb() .

Практика: gcd, lcm, факториал, comb, perm

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

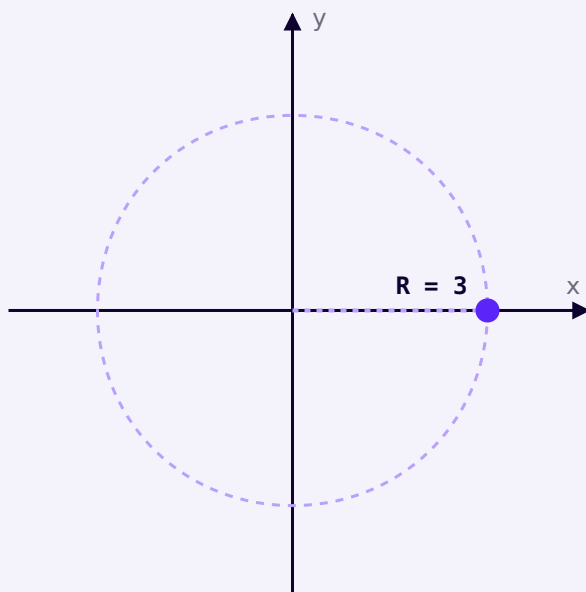
Открыть практику → (<https://www.cartesianschool.org/practice/05-14/index.html>)

Геометрия с math

math не рисует круги — но даёт всё, чтобы их точно посчитать.

Модуль `math` не умеет рисовать круги, но даёт всё нужное, чтобы их **посчитать**: константу `math.pi` и обычные арифметические операторы, которые мы уже знаем.

Площадь и длина окружности



Окружность радиусом 3 — площадь и длина считаются через её радиус

krug.py

```
import math

radius = 3
ploshad = math.pi * radius ** 2
dlina_okruzhnosti = 2 * math.pi * radius

print(f"Площадь: {round(ploshad, 2)}")
# 28.27
print(f"Длина окружности: {round(dlina_okruzhnosti, 2)}") # 18.85
```

math.pi — это float с ограниченной точностью

math.pi хранит π с точностью, достаточной для практических задач (15-16 значащих цифр) — но это не «настоящее» бесконечное π , а его float-представление. Мы подробно разбирали, почему у float есть предел точности, в главе 4.

Комбинируем несколько формул: площадь кольца

Настоящие геометрические задачи часто требуют комбинировать несколько формул. Площадь кольца между двумя окружностями — площадь большого круга минус площадь маленького:

kolco.py

```
import math

vneshnij_radius = 5
vnutrennij_radius = 3

ploshad_kolca = math.pi * vneshnij_radius ** 2 -
math.pi * vnutrennij_radius ** 2
print(round(ploshad_kolca, 2)) # 50.27
```

Полезные помощники: `prod()` и `fsum()`

Для перемножения или точного суммирования нескольких чисел сразу — например, объёма параллелепипеда по трём измерениям:

```
prod_fsum.py
```

```
import math

izmereniya = [4, 5, 6]
obyom = math.prod(izmereniya)
print(obyom)      # 120 — как 4 * 5 * 6

# fsum складывает float точнее, чем обычный sum() —
# меньше накопленной ошибки округления
chisla = [1, 1e16, -1e16]
print(sum(chisla))      # 0.0 — обычный sum()
# потерял единицу из-за порядка сложения
print(math.fsum(chisla)) # 1.0 — fsum() считает
# точнее именно для таких случаев
```

Практика: геометрия с `math`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-15/index.html>)

Тригонометрия без страха

$\sin$ и $\cos$ — это просто координаты точки на окружности радиусом 1. Вся тригонометрия строится вокруг этой одной картинки.

Тригонометрия пугает многих не самой математикой, а обозначениями. На деле идея простая: для любой точки на окружности радиусом 1 (единичной окружности) координаты этой точки — это и есть `cos` и `sin` угла.

Градусы и радианы

Python (как и почти вся математика за пределами школьной геометрии) считает углы в **радианах**, не в градусах. Полный круг — это 360° или, что то же самое, 2π радиан:

```
gradusy_radiany.py
```

```
import math

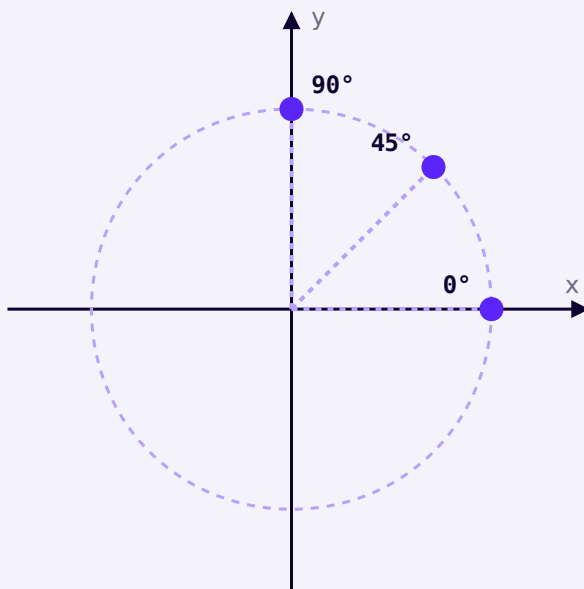
print(math.radians(180))    # 3.141592653589793 — то
    есть  $\pi$ 
print(math.degrees(math.pi)) # 180.0
```

math.sin(90) — это НЕ то, что вы думаете

`math.sin(90)` считает синус угла в 90 **радиан** (это огромный угол, много полных оборотов), а не 90 градусов! Перед тригонометрическими функциями почти всегда нужен `math.radians()` :

```
math.sin(math.radians(90)) .
```

Единичная окружность



Единичная окружность (радиус 1) — координаты точки равны ($\cos$ угла, $\sin$ угла)

edinichnaya_okrzhnost.py

```
import math

ugol_gradusy = 30
ugol_radiany = math.radians(ugol_gradusy)

print(round(math.cos(ugol_radiany), 3)) # 0.866 —
координата x точки на окружности
print(round(math.sin(ugol_radiany), 3)) # 0.5 —
координата y точки на окружности
```

round() здесь не случаен

Без `round()` `math.sin(math.radians(30))` выдаёт `0.49999999999999994`, а не ровно `0.5` — та же особенность `float`, которую мы разбирали в главе 4. Для сравнения углов тоже стоит использовать допуск (например, `math.isclose()`), а не `==`.

tan() и обратные функции

`tan_obratnye.py`

```
import math

print(round(math.tan(math.radians(45)), 3))    # 1.0

# обратная задача: известен tan, ищем угол
ugol = math.degrees(math.atan(1))
print(ugol)    # 45.0
```

Практика: тригонометрия без страха

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-16/index.html>)

Логарифмы и экспоненты

Логарифм — просто обратная операция к степени: не «результат возведения», а «какая нужна степень».

Логарифм отвечает на вопрос, обратный степени: не «2 в какой степени даёт 8», а «в какую степень нужно возвести 2, чтобы получить 8».

$$2^{**} x = 8$$



$$x = \log_2(8)$$



$$x = 3$$

Логарифм — обратная операция к степени: ищем показатель, а не результат

log.py

```
import math

print(2 ** 3)           # 8 — прямая задача: степень
                        # известна, ищем результат
print(math.log2(8))    # 3.0 — обратная задача:
                        # результат известен, ищем степень
```

Три варианта основания

Функция	Основание	Когда используется
<code>math.log2(x)</code>	2	информатика: сколько битов нужно для x зна- чений
<code>math.log10(x)</code>	10	порядок величины числа (сколько в нём разрядов)
<code>math.log(x)</code>	$e \approx 2.718$ (нату- ральный)	рост/распад, фи- нансы, наука
<code>math.log(x, base)</code>	любое, задаётся вторым аргумен- том	произвольное основание

osnovaniya.py

```
import math

print(math.log10(1000))    # 3.0 — в числе 1000 три
                          # нуля
print(math.log(8, 2))      # 3.0 — то же самое, что
                          # log2(8), но через общую функцию
```

exp() — обратная операция к логарифму

`math.exp(x)` считает $e ** x$ — то, во что логарифм и степень превращаются друг в друга:

exp.py

```
import math

x = 2
print(math.exp(x))        # 7.38905609893065
print(math.log(math.exp(x))) # 2.0 — log и exp
                          # отменяют друг друга
```

Где логарифмы встретятся снова

Логарифмы понадобятся не сразу, но встретятся ещё не раз: в оценке сложности алгоритмов (почему поиск в отсортированном списке быстрый), в анализе данных, в шкалах вроде децибел или землетрясений по шкале Рихтера. Пока достаточно понимать саму идею — «обратная операция к степени».

Практика: логарифмы и экспоненты

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/05-17/index.html) (<https://www.cartesianschool.org/practice/05-17/index.html>)

Модуль random: псевдослучайность

Без капли случайности сложно представить хоть одну игру — но эта «случайность» устроена куда интереснее, чем кажется.

Модуль `random` добавляет в Python элемент случайности — без него не обходится почти ни одна игра: случайный противник, случайная карта, случайное число для игры «Угадай число» в главе 9.

«Случайное» число — на самом деле не совсем случайное

Компьютер не умеет создавать настоящую случайность из ничего — вместо этого `random` использует **псевдослучайный генератор**: формулу, которая по одному числу («состоянию») вычисляет следующее число, выглядящее беспорядочным, а затем обновляет своё состояние для следующего вызова. Числа выглядят случайными, но получены полностью детерминированной формулой.

```
psevdosluchajnost.py
```

```
import random

print(random.random())    # случайное дробное число от
                           # 0.0 до 1.0 (не включая 1.0)
print(random.random())    # другое число – состояние
                           # генератора обновилось
```

Зачем это знать, если можно просто пользоваться?

Потому что из этого следует важное практическое свойство: генератор можно **перезапустить** с того же самого состояния — и тогда он выдаст ту же самую последовательность чисел заново. Это называется воспроизводимостью, и мы разберём её подробно через несколько разделов — она незаменима при отладке программ со случайностью.

Что дальше в этом разделе**RANDINT · RANDRANGE · UNIFORM**

случайные числа в диапазоне

CHOICE · CHOICES · SAMPLE · SHUFFLE

случайный выбор из набора

SEED

воспроизводимость и random vs secrets

Каждая карточка — отдельный раздел ниже

Практика: основы random

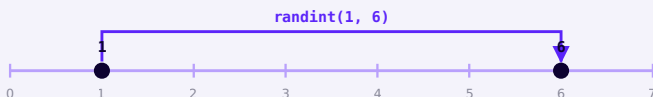
Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-05/index.html>)

randint, randrange, uniform

Три похожих функции с тремя разными правилами
на счёт границ диапазона — и одна классическая ошибка,
которая случается, если их перепутать.

**randint(a, b) — случайное целое, обе
границы включены**



randint(1, 6) может вернуть 1, может вернуть 6 —
обе границы включены

randint.py

```
import random
```

```
kubik = random.randint(1, 6)  # может быть 1, 2, 3,  
                               4, 5 ИЛИ 6  
print(kubik)
```

Частая ошибка «на единицу» (off-by-one)

Если нужны числа от 1 до 5 (не включая 6), а написать `random.randint(1, 6)` — программа иногда будет неверно выдавать 6. Обе границы `randint()` включены — это стоит держать в голове каждый раз.

`randrange(start, stop, step)` — как `range()`, но случайно

В отличие от `randint()`, у `randrange()` верхняя граница **не включена** — совсем как у обычного `range()`, который мы разберём подробно в главе 10. Плюс есть необязательный шаг:

`randrange.py`

```
import random

print(random.randrange(1, 7))      # от 1 до 6
                                   # включительно — 7 НЕ входит, аналог randint(1, 6)
print(random.randrange(0, 10, 2))  # случайное чётное число: 0, 2, 4, 6 или 8
```

Функция	Верхняя граница	Шаг
<code>randint(a, b)</code>	включена	нет — только соседние целые
<code>randrange(a, b)</code>	НЕ включена	нет (по умолчанию 1)
<code>randrange(a, b, step)</code>	НЕ включена	да — можно перескакивать

`uniform(a, b)` — случайное дробное число

Работает как `randint()`, но для `float` — любое дробное значение в диапазоне, а не только целые:

uniform.py

```
import random

temperatura = random.uniform(18.0, 24.0)
print(round(temperatura, 1))  # например, 21.3
```

Практика: randint, randrange, uniform

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-18/index.html>)

choice, choices, sample, shuffle

Четыре похожих инструмента для случайного выбора — отличаются тем, сколько элементов выбирают и можно ли повторяться.

Кроме случайных чисел, random умеет случайно выбирать из **готового набора** значений — списка вариантов, колоды карт, списка имён.

Четыре похожих, но разных инструмента

Функция	Сколько выбирает	Повторы возможны?	Меняет исходный набор?
<code>choice(seq)</code>	1 элемент	—	нет
<code>choices(seq, k=n)</code>	n элементов	да — один и тот же элемент может повториться	нет

Функция	Сколько выбирает	Повторы возможны?	Меняет исходный набор?
<code>sample(seq, k=n)</code>	n элементов	нет — все разные	нет
<code>shuffle(seq)</code>	все элементы	—	да, перемешивает на месте

`choice()` — один случайный элемент

choice.py

```
import random

varianty = ["камень", "ножницы", "бумага"]
print(random.choice(varianty))
```

`choices()` — несколько, с возможными повторами

Представим колесо рулетки с призами — один и тот же приз может выпасть снова:

choices.py

```
import random

prizy = [" ", " ", " ", " "]
vypavshie = random.choices(prizy, k=5)
print(vypavshie) # например, [" ", " ", " ", " ", " "],
                  " ", " " — повторы допустимы
```

У choices() есть необязательные веса

`random.choices(prizy, weights=[1, 1, 10], k=5)` сделает третий приз в десять раз вероятнее остальных — полезно для «нечестных» кубиков или систем с разной редкостью наград.

sample() — несколько, без повторов

Представим розыгрыш призов среди участников — один и тот же участник не может выиграть дважды в одном розыгрыше:

sample.py

```
import random

uchastniki = ["Аня", "Борис", "Вера", "Глеб", "Дана"]
pobediteli = random.sample(uchastniki, k=2)
print(pobediteli)  # например, ["Вера", "Аня"] — все
разные
```

k не может быть больше длины набора

`random.sample(uchastniki, k=10)` при 5 участниках вызовет `ValueError` — без повторов невозможно выбрать больше элементов, чем есть в наборе. У `choices()` такого ограничения нет, потому что повторы разрешены.

shuffle() — перемешать всё на месте

В отличие от трёх функций выше, `shuffle()` не выбирает подмножество — она перемешивает **весь** список и ничего не возвращает (`None`), потому что изменяет список прямо «на месте»:

shuffle.py

```
import random

koloda = [1, 2, 3, 4, 5]
random.shuffle(koloda)
print(koloda)  # например, [3, 1, 5, 2, 4] — тот же
               # список, новый порядок
```

shuffle() возвращает None — не переприсваивайте результат

`koloda = random.shuffle(koloda)` — частая ошибка: после неё `koloda` станет `None`, хотя перемешивание уже прошло успешно ДО присваивания. Просто вызовите `random.shuffle(koloda)` отдельной строкой.

Практика: choice, choices, sample, shuffle

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-19/index.html>)

seed и воспроизводимость

Псевдослучайность можно «заморозить» — и это не баг, а важнейшая возможность для отладки и тестирования.

seed() — задаём начальное состояние генератора

Раз псевдослучайный генератор — это формула с состоянием, то, задав одно и то же начальное состояние («зерно», `seed`), можно заставить его выдать ровно ту же последовательность чисел снова:

seed.py

```
import random

random.seed(42)
print(random.randint(1, 100))    # всегда одно и то же
                                  # число при seed(42)

random.seed(42)
# сбрасываем состояние заново
print(random.randint(1, 100))    # то же самое число,
                                  # что и в первый раз
```

Зачем это нужно на практике

Воспроизводимость критична при отладке: если баг проявляется только «иногда», `random.seed()` позволяет зафиксировать конкретную «случайную» последовательность и повторно воспроизвести баг столько раз, сколько нужно для его исправления. Она же используется для автоматической проверки практик этой главы — без фиксированного seed автоматически проверить случайный результат было бы невозможно.

Независимые генераторы: `random.Random()`

Все функции модуля `random`, которые мы использовали, работают с одним общим, «глобальным» генератором. Если нужен отдельный, независимый генератор — например, чтобы одна часть программы не влияла на случайность другой — можно создать собственный экземпляр:

```
random_instance.py
```

```
import random

generator_a = random.Random(1)
generator_b = random.Random(1)

print(generator_a.randint(1, 100))  # оба генератора
# созданы с одним и тем же seed —
print(generator_b.randint(1, 100))  # значит, дадут
# одинаковый результат независимо друг от друга
```

random — это не безопасно для паролей и ключей

Раз `random` — **предсказуемый** генератор (тот же seed → та же последовательность), его нельзя использовать там, где случайность защищает от взлома: пароли, токены сессий, ключи шифрования. Для этого в стандартной библиотеке Python есть отдельный модуль — `secrets`, спроектированный именно для безопасности, а не для скорости.

random или secrets?

Нужна
случайность для
игры,
симуляции,
тестового
примера?

→ **используйте** `random`

Нужна
случайность
для пароля,
токена, ключа
безопасности?

→ **используйте secrets**

random оптимизирован для скорости и
предсказуемости при том же seed — это НЕ
подходит для защиты данных

secrets_primer.py

```
import secrets
```

```
token = secrets.token_hex(16)  # криптографически  
надёжный случайный токен  
print(token)
```

Мы не будем глубоко изучать secrets в этой главе

Достаточно запомнить сам принцип: random — для игр, симуляций и учебных задач; secrets — там, где случайность защищает что-то ценное. Подробнее о безопасности мы поговорим в более поздних главах.

Практика: seed и воспроизводимость

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/05-20/index.html) → (https://www.cartesianschool.org/practice/05-20/index.html)

Отладка вычислений

Неверное число без единой ошибки — самый коварный тип бага. Разберём три причины и надёжный способ их искать.

Формула, которая выдаёт неверное число, — гораздо коварнее, чем ошибка, которая роняет программу: Python не подскажет, что что-то не так. Разберём, как искать такие проблемы осознанно, а не методом «поменял и посмотрел».



1. Синтаксическая ошибка

Самый простой случай — программа не запускается вовсе, и Python указывает на конкретную строку и символ ещё до начала выполнения:

`sintaksicheskaya.py`

```
# itog = (a + b * c
# SyntaxError: '(' was never closed — не хватает
# закрывающей скобки
```

Считайте скобки парами

Самый надёжный способ найти незакрытую скобку — считать открывающие и закрывающие скобки в подозрительной строке отдельно; их должно быть поровну.

2. Ошибка в формуле

Самая опасная категория — программа выполняется без единой ошибки, но результат неверен. Обычно за этим стоит одна из трёх причин, которые мы уже разбирали в этой главе:

Причина	Пример	Где мы это уже видели
забытые скобки	<code>4 + 7 + 9 / 3</code> вместо <code>(4 + 7 + 9) / 3</code>	раздел «Скобки и формулы»
не та переменная	<code>print(shirina + shirina)</code> вместо <code>print(perimetr)</code>	раздел «Скобки и формулы»
перепутанный приоритет	<code>-2 ** 2</code> — не то же самое, что <code>(-2) ** 2</code>	раздел «Унарные операторы»

Приём: трассировка формулы через именованные шаги

Когда одна длинная формула даёт подозрительный результат, самый надёжный способ найти проблему — разбить её на именованные промежуточные шаги и напечатать каждый:

trassirovka.py

```
# было — одна строка, непонятно, что пошло не так:
itog = (cena - skidka) * kolichество + dostavka /
kolichество

# стало — видно каждый промежуточный результат:
cena_so_skidkoj = cena - skidka
print("Цена со скидкой:", cena_so_skidkoj)

summa_za_tovar = cena_so_skidkoj * kolichество
print("Сумма за товар:", summa_za_tovar)

dostavka_na_edinicu = dostavka / kolichество
print("Доставка на единицу:", dostavka_na_edinicu)

itog = summa_za_tovar + dostavka_na_edinicu
print("Итог:", itog)
```

$$\text{итог} = (100 - 10) \times 3 + 15 / 3$$


цена со скидкой = 90



сумма за товар = 270



доставка на единицу = 5.0



итог = 275.0

Трассировка превращает одну загадочную строку в последовательность понятных, проверяемых шагов

Это временный код, а не постоянный

Разбиение на именованные шаги — приём именно для **отладки**. Когда формула заработает верно, промежуточные `print()` обычно убирают, но иногда осмысленные имена промежуточных переменных стоит оставить — они делают код понятнее, даже если он больше не в отладке.

3. Особенность представления чисел

Третья категория — код синтаксически верен, формула тоже верна, но результат всё равно удивляет. Обычно причина в том, что мы уже подробно разбирали в главе 4:

```
float_lovushka.py
```

```
print(0.1 + 0.2 == 0.3)           # False! — не баг, а
особенность float
print(round(0.1 + 0.2, 10) == 0.3)
# True — сравнение с допуском работает верно
```

Подробности — в главе 4

Почему так происходит и как сравнивать `float` правильно — подробно разобрано в разделе «Сравниваем `float` правильно» главы 4. Здесь достаточно узнавать симптом: сравнение `==` между `float` ведёт себя неожиданно — значит, дело в представлении чисел, а не в ошибке формулы.

Предскажите, прежде чем запускать

Лучшая привычка для отладки — формировать её ещё до появления багов: прежде чем запускать код, предскажите результат. Если предсказание не совпало с реальностью — вы либо нашли баг, либо неверно понимаете код. Оба случая стоит разобрать, не запуская код дальше не глядя.

★ Базовая практика

Предскажите результат

Не запуская код, определите результат: `итог = 2 + 3 * 4 ** 2 // 5`. Затем проверьте себя.

★★ Самостоятельная задача

Найдите ошибку в формуле

В коде `srednyaya_ocenka = ocenka1 + ocenka2 + ocenka3 / 3` есть ошибка приоритета. Найдите и исправьте её.

★★★ Задача повышенной сложности

Трассируйте формулу

Возьмите формулу расчёта итоговой суммы заказа с учётом скидки и доставки из примера выше и добавьте к ней трассировку с понятными именами промежуточных переменных.

Практика: отладка вычислений

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-21/index.html>)

Мини-проекты и итоги

Собираем всю главу в пяти небольших, но настоящих проектах — и подводим итоги.

Пять небольших, но настоящих проектов — каждый использует только то, что мы уже прошли в этой главе.

Проект 1: умный калькулятор

Считаем результат в зависимости от того, какой оператор выбран — `+`, `-`, `*` или `/`:

Забегаям вперёд

Здесь использована конструкция `if / elif` — мы разберём её подробно в главе 8. Пока достаточно прочитать её как обычный текст: «если оператор такой-то — сделай так, а если другой — иначе».

```
umnyj_kalkulyator.py
```

```
a, b = 12, 4
operator = "+"

if operator == "+":
    rezultat = a + b
elif operator == "-":
    rezultat = a - b
elif operator == "*":
    rezultat = a * b
elif operator == "/":
    rezultat = a / b

print(rezultat)    # 16
```

★ Базовая практика**Добавьте `//`, `%` и `**`**

Расширьте калькулятор так, чтобы он умел также целочисленное деление, остаток и возведение в степень.

Проект 2: конвертер времени — версия короче

Мы уже решали эту задачу в главе 4 — через `//` и `%` по отдельности. Теперь, когда мы знаем `divmod()` (раздел «Деление с остатком»), можно решить её вдвое короче:

```
konverter_vremeni_divmod.py
```

```
total_seconds = 3725

hours, ostatok = divmod(total_seconds, 3600)
minutes, seconds = divmod(ostatok, 60)

print(f"{hours}ч {minutes}м {seconds}с")    # 1ч 2м 5с
```

★★ Самостоятельная задача

Challenge

Проверьте свой конвертер на 0 секунд и на 90000 секунд (больше суток).

Проект 3: лаборатория кубика

Бросаем два кубика и считаем сумму и среднее — используем `random.randint()` и уже знакомые операторы:

```
laboratoriya_kubika.py
```

```
import random

kub1 = random.randint(1, 6)
kub2 = random.randint(1, 6)
summa = kub1 + kub2
srednee = summa / 2

print(f"Кубик 1: {kub1}, кубик 2: {kub2}")
print(f"Сумма: {summa}, среднее: {srednee}")
```

★★ Самостоятельная задача

Дубль!

Добавьте проверку: если оба кубика выпали одинаковыми — выведите «Дубль!» (используйте `==` из главы 3).

Проект 4: случайный челлендж — кратные числа

Программа, которая находит все числа, кратные случайно выбранному — используем `random` и `%` вместе:

```
kratnye_chisla.py
```

```
import random

# случайное число, кратности которого мы ищем
kratnoe_chemu = random.randint(2, 9)
print(f"Ищем числа, кратные {kratnoe_chemu}, от 1 до 50")

chislo = 1
while chislo <= 50:
    if chislo % kratnoe_chemu == 0:
        print(chislo)
    chislo += 1
```

Забегает вперёд ещё раз

Цикл `while` мы разберём подробно в главе 10. Здесь важно увидеть, как оператор `%` из этой главы находит все числа, кратные заданному: остаток от деления равен нулю именно тогда, когда число делится нацело.

★★★ Задача повышенной сложности

Свой диапазон

Измените границы поиска на 1–100 вместо 1–50.

Проект 5: геометрическая лаборатория

Собираем сразу несколько формул этой главы: расстояние между двумя точками и площадь круга такого радиуса:

```
geometricheskaya_laboratoriya.py
```

```
import math

tochka_a = (0, 0)
tochka_b = (3, 4)

radius = math.dist(tochka_a, tochka_b)
ploshad_kruga = math.pi * radius ** 2

print(f"Расстояние между точками: {radius}")
print(f"Площадь круга такого радиуса: {round(ploshad_kruga, 2)}")
```

★★★ Задача повышенной сложности

Периметр треугольника

Добавьте третью точку и посчитайте периметр треугольника, образованного всеми тремя точками, используя `math.dist()` трижды.

Практика: мини-проекты главы 5

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/05-06/index.html>)

Финальная карта главы

Какой инструмент главы 5 мне нужен?

Нужен остаток или проверка кратности?

→ // и %

Нужна степень
или корень?

→ `**` или `math.sqrt()`

Формула не
помещается
в одну
понятную
строку?

→ **именованные промежуточные шаги**

Нужна
геометрия
—
расстояние,
площадь,
угол?

→ `math.hypot()` / `math.dist()` / `math.pi`

Нужна случайность для игры?

→ `random`

Нужно повторить
один и тот же
случайный результат?

→ `random.seed()`

Возвращайтесь к этой карте в будущих проектах —
она собирает всю главу в шесть вопросов

Итоги главы

Что мы теперь умеем

- Выражение — операнды плюс оператор; отличаем выражение от инструкции (statement).
- Основные операторы `+` `-` `*` `/` и их геометрические модели: числовая прямая для `+`/`-`, прямоугольник для умножения.
- `//`, `%` и `divmod()` — включая то, как они работают с отрицательными числами (округление к минус бесконечности).
- `**`, `pow()` и `math.pow()` — и классическая ловушка `-2 ** 2`.
- Сокращённые операторы присваивания, полная карта приоритета и ассоциативность (включая правоассоциативность `**`).
- Перевод формул из математической записи в Python — и типичные ошибки перевода.
- Глубокий модуль `math`: корни и расстояния, `gcd/lcm/factorial/comb/perm`, геометрия, тригонометрия, логарифмы.
- Глубокий модуль `random`: псевдослучайность, `randint/range/uniform`, `choice/choices/sample/shuffle`, `seed` и `random` vs `secrets`.
- Отладка вычислений: три типа ошибок и приём трассировки формулы через именованные шаги.

Что дальше

В главе 6 мы возьмём числа, которые теперь умеем считать, и начнём их **рисовать** — познакомимся с модулем `turtle` и нарисуем первые фигуры на экране. Координаты, углы и расстояния из этой главы там сразу пригодятся.

Рисуем классные вещи с помощью Turtle

Первая настоящая графика на Python: координаты, движение и повороты, многоугольники, перо и заливка, круги и цвета, случайная графика — и мандала в финале. Каждый пример показывает по-настоящему выполненный рисунок, а не описание того, что должно получиться.

6.1 Приступаем

6.2 Координаты: центр (0, 0)

6.3 Заставляем Turtle двигаться

6.4 Направление и угол

6.5 Меняем направление

6.6 Первые фигуры и формула $360/n$

6.7 Мини-проекты: квадрат и шестиугольник

6.8 Сокращённые приёмы

6.9 Поднять и опустить перо

6.10 goto() и координатная панель

6.11 Цвет, толщина и внешний вид

6.12 Заливка, круг, дуга и точка

6.13 Случайные точки на экране

6.14 Случайное движение

6.15 Рисуем по координатам

6.16 Отладка Turtle

6.17 Мини-проекты

6.18 Мандала и итоги

Приступаем

Знакомимся с двумя главными объектами графики Turtle — экраном и самой черепашкой — и с идеей, что у черепашки есть состояние.

Модуль `turtle` входит в стандартную поставку Python — ничего дополнительно устанавливать не нужно.

Немного истории

Идея «черепашьюй графики» появилась в конце 1960-х в языке Logo, который придумали Сеймур Пейперт и его коллеги специально для обучения детей программированию. Мысль была простой и сильной: пусть код будет виден — не абстрактные числа на экране, а линия, которую оставляет за собой воображаемая черепашка. Эта идея настолько удачная, что дожила до наших дней почти без изменений — модуль `turtle` в Python прямой потомок той самой Logo-черепашки.

Экран и черепашка — два разных объекта

Прежде чем рисовать, нужны два объекта: **экран** (`turtle.Screen()`) — окно, в котором происходит рисование, и **черепашка** (`turtle.Turtle()`) — то, что, собственно, рисует.

```
screen_i_cherepashka.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=360)
artist = turtle.Turtle()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Белое окно Turtle с маленьким чёрным треугольником-черепашкой в центре, направленным вправо

Пустое окно 480×360 с черепашкой в начале координат, курс направлен вправо

Маленький чёрный треугольник в центре — это и есть черепашка в исходном положении. Курс (направление) черепашки по умолчанию направлен вправо — на восток.

СОСТОЯНИЕ ЧЕРЕПАШКИ

position	(0, 0)
----------	--------

heading	0° (восток)
pen	down

У черепашки есть состояние

Заметьте: у черепашки в любой момент есть несколько характеристик одновременно — где она находится (*позиция*), куда смотрит (*курс*), и рисует ли она сейчас (*перо*). Каждая команда, которую мы будем изучать в этой главе, меняет какую-то одну из этих характеристик, не трогая остальные. Мы будем возвращаться к этой панели состояния на протяжении всей главы.

Зачем два отдельных объекта?

Разделение экрана и черепашки позволяет иметь на одном экране сразу **несколько** черепашек — у каждой будет своё собственное состояние, независимое от остальных.

Практика: начинаем с настройки экрана

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/06-02/index.html) (<https://www.cartesianschool.org/practice/06-02/index.html>)

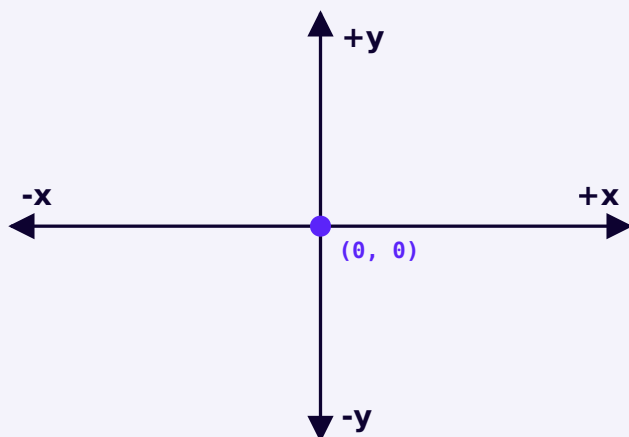
Координаты: центр (0, 0)

Числа из главы 5 становятся местом на экране — знакомимся с координатной системой окна Turtle.

Мы уже встречали координаты в главе 5 — точки на плоскости с координатами x и y . Окно Turtle устроено ровно по той же системе, и она будет буквально управлять тем, что рисует черепашка.

Центр экрана — точка (0, 0)

В отличие от многих других графических систем (где ось Y растёт *вниз*), Turtle использует привычную математическую систему координат:



Центр окна — точка (0, 0). Ось X растёт вправо, ось Y растёт вверх.

- **x** — горизонталь: положительные значения вправо, отрицательные влево.
- **y** — вертикаль: положительные значения вверх, отрицательные вниз.
- Центр окна — точка (0, 0), где черепашка появляется по умолчанию.

Координата	Куда указывает
(100, 0)	вправо от центра

Координата	Куда указывает
<code>(-100, 0)</code>	влево от центра
<code>(0, 100)</code>	вверх от центра
<code>(0, -100)</code>	вниз от центра

Координаты на самом деле управляют окном

Это не просто абстрактная схема — координатная система по-настоящему определяет, куда черепашка попадёт, если её туда отправить:

koordinaty.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=480)
artist = turtle.Turtle()
artist.hideturtle()
artist.speed(0)

# ось X
artist.penup(); artist.goto(-200, 0); artist.pendown()
artist.pencolor("#0D0230"); artist.pensize(2)
artist.goto(200, 0)

# ось Y
artist.penup(); artist.goto(0, -200); artist.pendown()
artist.goto(0, 200)

# отметим начало координат и одну точку
artist.penup(); artist.goto(0, 0)
artist.dot(10, "#5B24F9")
artist.goto(120, 80)
artist.dot(10, "#5B24F9")
artist.write("  (120, 80)", font=("Arial", 12,
"normal"))
artist.goto(0, -20)
artist.write("  (0, 0)", font=("Arial", 12, "normal"))

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Окно Turtle с горизонтальной и вертикальной осью, пересекающимися в центре, и двумя подписанными точками: (0, 0) и (120, 80)

Оси, начало координат (0, 0) и точка (120, 80), отмеченные прямо в окне Turtle

write() — подписать точку на экране

В примере выше использована команда `artist.write("текст")` — она печатает текст в текущей позиции черепашки. Удобно для подписи точек на схемах, как здесь.

Практика: координаты и goto()

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/06-09/index.html>)

Движение вперёд и назад

Первое, что должна уметь черепашка — двигаться. В этом разделе вы напишете свою первую программу, которая по-настоящему рисует.

Зачем это нужно

Всё, что вы нарисуете дальше — многоугольники, дома, мандалы — строится из одной и той же пары действий: **«проехать немного»** и **«повернуть на какой-то угол»**. Освоив четыре команды этого раздела, вы получите строительные блоки для всей главы.

Синтаксис

- `forward(расстояние)` — проехать вперёд на указанное число пикселей (короткая форма — `fd`)
- `backward(расстояние)` — проехать назад, не поворачиваясь (короткие формы — `bk`, `back`)
- `right(угол)` — повернуть по часовой стрелке на угол в градусах (короткая форма — `rt`)

- `left(угол)` — повернуть против часовой стрелки на угол в градусах (короткая форма — `lt`)

Первый шаг: просто вперёд

vpered_120.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=300)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.forward(120)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Прямая фиолетовая линия длиной 120 пикселей от центра окна вправо, с черепашкой на конце

`artist.forward(120)` — черепашка проехала 120 пикселей вправо, оставив линию

Вперёд, потом назад

vpered_nazad.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=300)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.forward(120)
artist.backward(50)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Га же линия, но короче — черепашка вернулась на 50 пикселей назад по прямой

Тот же `forward(120)`, затем `backward(50)` — черепашка отступила назад по той же линии

Совет

Числа в `forward()` тоже могут быть отрицательными. `forward(-50)` и `backward(50)` делают одно и то же.

Первая настоящая программа: буква «Г»

Проедем вперёд, повернём и проедем ещё раз:

bukva_g.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=360)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.forward(120)
artist.left(90)
artist.forward(80)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Уголок из двух отрезков в форме буквы Г,
нарисованный черепашкой

`forward(120), left(90), forward(80)` — получилась буква
«Г»

Разбор по шагам

- `turtle.Screen()` открывает окно для рисования.
- `turtle.Turtle()` создаёт саму черепашку — в начале координат, курс направлен вправо.
- `artist.forward(120)` — черепашка проезжает 120 пикселей в направлении курса, оставляя линию.
- `artist.left(90)` — курс поворачивается на 90° против часовой стрелки.
- Второй вызов `forward()` едет уже в новом направлении — отсюда и уголок буквы «Г».

Типичная ошибка

Внимание

Начинающие часто путают `right()` / `left()` с движением — кажется, что черепашка должна сместиться в сторону. На самом деле эти команды **только поворачивают** курс и не двигают черепашку ни на пиксель.

Диагностика: если после поворота фигура выглядит «сплюсненной» или линии накладываются друг на друга — скорее всего, вы забыли вызвать `forward()` после поворота, либо перепутали местами поворот и движение.

Попробуйте изменить

★ Базовая практика

Другое расстояние и угол

Измените расстояние 120 на 250 и угол 90 на 45. Перед запуском попробуйте предсказать: в какую сторону теперь «смотрит» черепашка после поворота?

★★ Самостоятельная задача

Третий отрезок

Добавьте третью пару команд `left(90)` и `forward(120)` в конец программы. Какая фигура получится?

Современный вариант

В старом коде черепашку часто рисуют без явного объекта — модуль `turtle` автоматически создаёт «черепашку по умолчанию».

Классический подход → современный Python 3.14**Классический подход**

```
import turtle
```

```
# без явного объекта –  
# используется скрытая  
# черепашка по умолчанию  
turtle.forward(120)  
turtle.left(90)  
turtle.forward(80)  
turtle.done()
```

Современный Python 3.14

```
import turtle
```

```
# явный объект –  
понятно,  
# какая черепашка  
рисует,  
# легко добавить вторую  
artist = turtle.Turtle()  
artist.forward(120)  
artist.left(90)  
artist.forward(80)  
turtle.done()
```

Что использовать сегодня: явный объект

`turtle.Turtle()` . Он ничего не усложняет, зато сразу готовит вас к главе про объекты и легко масштабируется — если позже понадобится нарисовать сцену из нескольких черепашек, у каждой будет своё имя и своё состояние. Модульные функции (`turtle.forward()` без объекта) всё ещё встречаются в старом коде и учебниках — знать их полезно, но начинайте привычку сразу с объекта.

Практика: эксперименты, задание и самостоятельная практика

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/06-02/index.html\)](https://www.cartesianschool.org/practice/06-02/index.html)

Что запомнить

- `forward()` / `backward()` двигают черепашку по прямой; `left()` / `right()` только поворачивают курс.
- Позиция и курс — это два независимых свойства черепашки.
- Современный стиль — создавать явный объект `turtle.Turtle()`, а не полагаться на скрытую черепашку по умолчанию.

Направление и угол

Черепашка может стоять на месте и смотреть в четыре разные стороны — разбираемся, чем курс отличается от позиции.

Важно различать две вещи, которые легко перепутать: **позицию** черепашки (где она находится) и её **курс** (куда она смотрит). Черепашка может стоять в одной и той же точке и при этом смотреть в четыре совершенно разные стороны.

Четыре состояния в одной точке

chetyre_napravleniya.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=480)
artist = turtle.Turtle()
artist.shape("classic")
artist.speed(0)
artist.penup()

headings = [(0, "восток · 0°"), (90, "север · 90°"),
(180, "запад · 180°"), (270, "юг · 270°")]
colors = ["#5B24F9", "#DB2777", "#059669", "#D97706"]
for (angle, label), color in zip(headings, colors):
    artist.setheading(angle)
    artist.goto(0, 0)
    artist.pencolor(color)
    artist.pendown()
    artist.forward(90)
    artist.penup()
    artist.write("  " + label, font=("Arial", 11,
"normal"))
    artist.goto(0, 0)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Четыре цветных отрезка из центра окна в четырёх направлениях — вправо, вверх, влево, вниз — подписанные 0°, 90°, 180°, 270°

Одна и та же точка (0, 0) — четыре разных курса, отмеченных отрезком и штампом черепашки

Поворот меняет курс, но не позицию

Вот что происходит с состоянием черепашки при повороте — обратите внимание, что `position` не изменилась ни на пиксель:

СОСТОЯНИЕ ЧЕРЕПАШКИ

<code>position</code>	<code>(0, 0)</code>
-----------------------	---------------------

<code>heading</code>	<code>0° (восток)</code>
----------------------	--------------------------

<code>pen</code>	<code>down</code>
------------------	-------------------

→ `left(90)` →

СОСТОЯНИЕ ЧЕРЕПАШКИ

<code>position</code>	<code>(0, 0)</code>
-----------------------	---------------------

<code>heading</code>	<code>90° (север)</code>
----------------------	--------------------------

<code>pen</code>	<code>down</code>
------------------	-------------------

Углы из главы 5

Мы уже встречали эти углы в главе 5, когда говорили о тригонометрии и единичной окружности — теперь они управляют направлением черепашки напрямую:

Угол	Что означает для черепашки
360°	полный оборот — черепашка снова смотрит туда же, откуда начала
180°	разворот — черепашка смотрит строго в противоположную сторону
90°	четверть оборота — прямой угол, курс становится перпендикулярным
45°	половина прямого угла — курс «по диагонали»

Направление в градусах, начиная с востока

В Turtle 0° — это восток (курс по умолчанию), и градусы растут против часовой стрелки: 90° — север, 180° — запад, 270° — юг. Это тот же принцип, что и на единичной окружности из главы 5.

Практика: направление и состояние черепашки

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/06-10/index.html) → (<https://www.cartesianschool.org/practice/06-10/index.html>)

Практика: предскажите курс и координату (без Turtle)

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/06-19/index.html) → (<https://www.cartesianschool.org/practice/06-19/index.html>)

Заставляем черепашку менять направление

Кроме относительных поворотов `left()/right()`, у черепашки есть способ задать направление абсолютно — вне зависимости от того, куда она смотрела раньше.

Команды `left()` / `right()` из предыдущего раздела поворачивают черепашку **относительно** её текущего курса. Иногда удобнее задать направление **абсолютно** — «смотри строго на север», независимо от того, куда черепашка смотрела раньше.

Абсолютный курс: `setheading()`

setheading.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=400)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.setheading(90)
artist.forward(100)

artist.setheading(180)
artist.forward(100)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Два перпендикулярных отрезка — один вверх, один влево — образующие прямой угол, начатые из одной точки

`setheading(90)` → `forward(100)`, затем `setheading(180)` → `forward(100)` — курс задан абсолютно, независимо от предыдущего направления

```
setheading_kod.py
```

```
artist.setheading(90)  # смотрим строго вверх,
                       # неважно, куда смотрели раньше
artist.forward(100)

artist.setheading(180) # смотрим строго влево
artist.forward(100)
```

Возврат домой: `home()`

Возвращает черепашку в исходную точку (0, 0) с исходным курсом (0°) — удобно, чтобы начать рисунок заново, не создавая нового объекта:

```
home.py
```

```
artist.home()
```

heading() — узнать текущий курс

Если нужно не задать курс, а *узнать* его — используйте `artist.heading()`: она возвращает текущее направление в градусах, не меняя его. У `setheading()` есть и короткий псевдоним — `seth()`.

Практика: `setheading()`, `heading()` и `home()`

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/06-03/index.html) → (https://www.cartesianschool.org/practice/06-03/index.html)

Первые фигуры и формула 360/n

От отдельных отрезков — к настоящим многоугольникам, и к формуле, которая объясняет их все разом.

Применим движение и повороты, чтобы нарисовать первые настоящие фигуры — пока без циклов, просто повторяя одну и ту же пару команд нужное число раз.

Треугольник

У треугольника три стороны и три угла поворота. Сумма всех внешних углов любого многоугольника всегда равна 360° , поэтому один поворот здесь — $360 / 3 = 120$ градусов:

треugolnik.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=400)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.forward(120)
artist.right(120)
artist.forward(120)
artist.right(120)
artist.forward(120)
artist.right(120)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Равносторонний треугольник, нарисованный сплошной фиолетовой линией

Правильный треугольник — три поворота по 120°

Прямоугольник

У прямоугольника углы всё ещё прямые (90°), но стороны чередуются — длинная, короткая, длинная, короткая:

```
pryamougolnik.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=360)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.forward(160)
artist.right(90)
artist.forward(90)
artist.right(90)
artist.forward(160)
artist.right(90)
artist.forward(90)
artist.right(90)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Прямоугольник шире, чем выше, нарисованный сплошной фиолетовой линией

Прямоугольник — стороны разной длины, все повороты по 90°

Пятиугольник

pyatiugolnik.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=440)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.forward(90)
artist.right(72)
artist.forward(90)
artist.right(72)
artist.forward(90)
artist.right(72)
artist.forward(90)
artist.right(72)
artist.forward(90)
artist.right(72)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Правильный пятиугольник, нарисованный сплошной фиолетовой линией

Правильный пятиугольник — пять поворотов по 72°

Формула для любого правильного многоугольника

Общее правило, которое работает для всех фигур выше:

поворот= $360/n$

где n — число сторон. Каждый раз, когда черепашка обходит правильный многоугольник по кругу и возвращается в исходную точку с исходным курсом, сумма всех поворотов обязана составить ровно один полный оборот — 360° .

Фигура	Сторон (n)	Поворот = $360 / n$
Треугольник	3	120°
Квадрат	4	90°
Пятиугольник	5	72°
Шестиугольник	6	60°

Квадрат и шестиугольник — в следующем разделе

Мы намеренно оставили квадрат ($360 / 4 = 90^\circ$) и шестиугольник ($360 / 6 = 60^\circ$) для следующего мини-проекта — там разберём их подробнее.

Практика: первые фигуры

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/06-11/index.html>)

Практика: формула $360/n$ (без Turtle)

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/06-20/index.html>)

Мини-проект — рисуем квадрат и шестиугольник

Применяем формулу $360/n$ к двум самым узнаваемым многоугольникам.

Применим формулу $360 / n$ из предыдущего раздела к двум самым узнаваемым фигурам.

Мини-проект — рисуем квадрат

У квадрата четыре одинаковые стороны и четыре угла по 90° : $360 / 4 = 90$. Значит, нужно четыре раза повторить одну и ту же пару команд: проехать вперёд, повернуть на 90° .

kvadrat.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=400, height=400)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Правильный квадрат, нарисованный сплошной фиолетовой линией

Квадрат: четыре стороны по 100 пикселей, четыре поворота по 90°

Заметили повторение?

Четыре одинаковых блока подряд — явный признак того, что напрашивается цикл `for`. Мы вернёмся к этому же квадрату в главе про циклы и перепишем его в 2 строки вместо 8.

Мини-проект — рисуем шестиугольник

У шестиугольника шесть равных сторон: $360 / 6 = 60$ градусов на каждом шаге.

shestiugolnik.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=400)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.forward(80)
artist.right(60)
artist.forward(80)
artist.right(60)
artist.forward(80)
artist.right(60)
artist.forward(80)
artist.right(60)
artist.forward(80)
artist.right(60)
artist.forward(80)
artist.right(60)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Правильный шестиугольник, нарисованный сплошной фиолетовой линией

Шестиугольник: шесть сторон по 80 пикселей, шесть поворотов по 60°

Практика: квадрат, шестиугольник и другие многоугольники

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/06-04/index.html>)

Сокращённые приёмы

Каждая команда движения имеет более короткий псевдоним — полезно уметь читать оба варианта.

У каждой команды движения, которую мы изучили, есть более короткий псевдоним — они делают абсолютно то же самое, только короче печатать:

- `forward()` → `fd()`
- `backward()` → `bk()` (или `back()`)
- `left()` → `lt()`
- `right()` → `rt()`

Полные имена команд → короткие псевдонимы

Полные имена

```
artist.forward(100)
artist.backward(50)
artist.left(90)
artist.right(90)
```

Короткие псевдонимы

```
artist.fd(100)
artist.bk(50)
artist.lt(90)
artist.rt(90)
```

Что использовать сегодня: оба варианта официальные и делают ровно одно и то же — псевдонимы существуют в модуле turtle специально для более быстрого написания. В этой книге мы обычно используем полные имена, потому что они понятнее при первом чтении, но короткие формы стоит узнавать: в чужом коде и примерах в интернете вы будете встречать оба варианта.

Первый взгляд на оформление линии

Помимо движения, есть ещё две команды, которые встретятся уже в следующих разделах — познакомимся с ними коротко прямо сейчас:

```
oformlenie.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=300)
artist = turtle.Turtle()

artist.pencolor("purple")
artist.pensize(6)
artist.fd(200)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Толстая фиолетовая горизонтальная линия

`pencolor("purple")` и `pensize(6)` — толстая фиолетовая линия вместо тонкой чёрной по умолчанию

```
oformlenie_kod.py
```

```
artist.pencolor("purple") # цвет линии
artist.pensize(6)         # толщина линии
```

Практика: включает практику с короткими командами

Тот же ноутбук, что и в разделе «Мини-проекты: фигуры» — он охватывает и эту тему

[Открыть практику → \(https://www.cartesianschool.org/practice/06-04/index.html\)](https://www.cartesianschool.org/practice/06-04/index.html)

Поднять и опустить перо

Иногда нужно переместиться, ничего не рисуя — знакомимся с `penup()` и `pendown()`.

До сих пор каждое движение черепашки оставляло линию. Но иногда нужно переместиться, ничего не рисуя — например, чтобы начать новую фигуру в другом месте экрана, не соединяя её с предыдущей.

Мысленная модель

Представьте настоящую ручку на бумаге: пока она касается листа, любое движение руки оставляет след. Поднимите ручку — и рука может двигаться куда угодно, не оставляя ни одной линии.

penup() И pendown()

pero.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=260)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.forward(60)      # перо опущено – рисует
artist.penup()
artist.forward(60)      # перо поднято – просто
                        # перемещение, без линии
artist.pendown()
artist.forward(60)      # перо снова опущено – рисует

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Два коротких фиолетовых отрезка на одной прямой с явным пробелом между ними — след от поднятого пера

Линия — пробел — линия: penup() между двумя движениями оставляет видимый разрыв

pero_kod.py

```

artist.forward(60)      # перо опущено – рисует
artist.penup()
artist.forward(60)      # перо поднято – просто
                        # перемещение, без линии
artist.pendown()
artist.forward(60)      # перо снова опущено – рисует

```

Не забудьте включить перо обратно

Самая частая ошибка с `penup()` — забыть вызвать `pendown()` после него. Тогда всё, что рисуется дальше в программе, окажется невидимым — черепашка честно двигается, просто ничего не оставляет на экране.

`isdown()` — проверить состояние пера

Если нужно узнать, опущено ли перо прямо сейчас — `artist.isdown()` вернёт `True` или `False`, не меняя само состояние.

Практика: `penup()` и `pendown()`

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/06-12/index.html>)

`goto()` и координатная панель

Тот же квадрат, что и раньше, — но на этот раз через прямое указание координат каждого угла.

Вернёмся к квадрату из раздела про мини-проекты — на этот раз нарисуем его без единого поворота, задавая напрямую четыре угловые точки командой `goto()`.

Абсолютное перемещение: goto(x, y)

kvadrat_goto.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=400, height=400)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.goto(100, 0)
artist.goto(100, 100)
artist.goto(0, 100)
artist.goto(0, 0)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Правильный квадрат, нарисованный через goto() — визуально идентичен квадрату из предыдущего раздела

Тот же квадрат — на этот раз каждая вершина задана координатой напрямую, без единого поворота

kvadrat_goto_kod.py

```
artist.goto(100, 0)
artist.goto(100, 100)
artist.goto(0, 100)
artist.goto(0, 0)
```

Два способа — один результат

`forward()` + `right()` описывают движение *относительно черепашки* («проехать, повернуть»);
`goto()` описывает движение *относительно экрана* («окажись в этой точке»). `goto()` удобен, когда координаты фигуры уже известны заранее.

Навигационная панель черепашки

Кроме `goto()`, у черепашки есть ещё несколько команд для работы с координатами напрямую:

Команда	Что делает
<code>position()</code> (или <code>pos()</code>)	вернуть текущие координаты (x, y)
<code>xcor()</code>	вернуть только координату x
<code>ycor()</code>	вернуть только координату y
<code>setx(x)</code>	переместиться, изменив только x
<code>sety(y)</code>	переместиться, изменив только y
<code>home()</code>	вернуться в (0, 0) с курсом 0°

navigacionnaya_panel.py

```

import turtle

screen = turtle.Screen()
screen.setup(width=480, height=360)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.penup()
artist.dot(8, "#B9A0FC")
artist.write("  старт (0, 0)", font=("Arial", 11,
"normal"))
artist.setx(150)
artist.sety(80)
artist.dot(8, "#5B24F9")
artist.write("  после setx(150), sety(80)",
font=("Arial", 11, "normal"))

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Две точки на экране: одна в центре подписана «старт (0, 0)», вторая правее и выше подписана координатами после setx/sety

Точка старта (0, 0) и точка после setx(150), sety(80) — обе отмечены и подписаны

```
navigaciya_kod.py
```

```
print(artist.position())    # текущие координаты

artist.setx(150)
artist.sety(80)
print(artist.position())    # координаты изменились по
одной оси за раз
```

Практика: рисуем фигуры через координаты

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/06-07/index.html>)

Цвет, толщина и внешний вид

Управляем тем, как выглядит линия и сама черепашка — и разбираемся, что `speed()` меняет, а что нет.

До сих пор мы рисовали фиолетовой линией по умолчанию. Пора научиться управлять тем, как именно выглядит рисунок — и как выглядит сама черепашка.

Толщина и цвет линии

tonkaya_chernaya.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=200)
artist = turtle.Turtle()

artist.pensize(1)
artist.pencolor("black")
artist.forward(220)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Тонкая чёрная горизонтальная линия

`pensize(1), pencolor("black")` — тонкая чёрная линия
по умолчанию

tolstaya_sinyaya.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=200)
artist = turtle.Turtle()

artist.pensize(10)
artist.pencolor("#2563EB")
artist.forward(220)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Толстая синяя горизонтальная линия той же длины

`pensize(10), pencolor("#2563EB")` — та же линия, но
толстая и синяя

cvet_i_tolschina.py

```
artist.pensize(10)
artist.pencolor("#2563EB")
artist.forward(220)
```

Цвет можно задавать по-разному

`pencolor("blue")` (имя цвета), `pencolor("#2563EB")` (шестнадцатеричный код, как в CSS) — оба варианта работают одинаково. Имена цветов проще запомнить, коды дают точный оттенок.

Внешний вид черепашки и фон окна

```
vid_i_fon.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=300)
screen.bgcolor("#FAFAFC")
artist = turtle.Turtle()
artist.shape("turtle")
artist.pencolor("#5B24F9")
artist.pensize(2)
artist.forward(120)
artist.left(90)
artist.forward(60)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Уголок из двух отрезков, нарисованный черепашкой в форме силуэта черепахи вместо треугольника

`shape("turtle")` меняет курсор на силуэт черепашки;
`bgcolor()` задаёт фон окна

```
vid_kod.py
```

```
screen.bgcolor("#FAFAFC")
artist.shape("turtle")
# вместо треугольника-стрелки – силуэт черепахи
```

Команда	Что меняет
<code>screen.title("...")</code>	заголовок окна
<code>screen.bgcolor("...")</code>	цвет фона
<code>artist.shape("turtle")</code>	форма курсора черепашки
<code>artist.hideturtle()</code> / <code>showturtle()</code>	скрыть/показать курсор (сама линия не исчезает)

Скорость — это про анимацию, не про рисунок

`artist.speed(0)` ускоряет анимацию рисования до максимума (0 — самая быстрая, 1 — самая медленная, 6 — обычная). Это касается только того, **как быстро** линия появляется на экране, — итоговый рисунок будет совершенно одинаковым что на скорости 1, что на скорости 0. Статичная картинка не может показать разницу в скорости — её видно только вживую, при запуске кода.

`skorost.py`

```
artist.speed(0)    # максимально быстро – удобно для
                  сложных узоров
```

Практика: цвет, толщина и внешний вид

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/06-13/index.html\)](https://www.cartesianschool.org/practice/06-13/index.html)

Заливка, круг, дуга и точка

От пустого контура — к закрашенным фигурам, готовым окружностям, дугам и точкам.

Заливка фигур

Контур — это ещё не всё: часто фигуру хочется закрасить внутри. Для этого черепашке нужно «запомнить», где начинается контур, и «отпустить», когда контур замкнётся.

kontur_treugolnika.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=360)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.forward(140)
artist.left(120)
artist.forward(140)
artist.left(120)
artist.forward(140)
artist.left(120)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Равносторонний треугольник, нарисованный только контуром, без заливки внутри

Треугольник без заливки — только контурная линия

`zalityj_treugolnik.py`

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=360)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")
artist.fillcolor("#B9A0FC")

artist.begin_fill()
artist.forward(140)
artist.left(120)
artist.forward(140)
artist.left(120)
artist.forward(140)
artist.left(120)
artist.end_fill()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Гот же равносторонний треугольник, но с закрашенной светло-фиолетовой заливкой внутри контура

Тот же треугольник, но `begin_fill()/end_fill()` закрасили область внутри контура

```
zalivka_kod.py
```

```
artist.pencolor("#5B24F9")
artist.fillcolor("#B9A0FC")

artist.begin_fill()
artist.forward(140)
artist.left(120)
artist.forward(140)
artist.left(120)
artist.forward(140)
artist.left(120)
artist.end_fill()
```

Как это работает

`begin_fill()` запоминает точку старта. Затем обычные команды рисования (`forward()`, `left()` и другие) вычерчивают замкнутый контур, как обычно. `end_fill()` закрашивает область внутри этого контура цветом `fillcolor()`.

`circle()` — готовая команда для окружностей

Рисовать круг вручную (много маленьких прямых под небольшим углом) не нужно — есть готовая команда:

`krug.py`

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=380)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.circle(80)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Правильная окружность, нарисованная сплошной фиолетовой линией

`circle(80)` — полная окружность радиусом 80 пикселей

`krug_kod.py`

```
artist.circle(80)  # радиус в пикселях
```

Дуга — часть окружности

Второй, необязательный аргумент `circle()` — это `extent`, сколько градусов окружности рисовать:

duga.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=300)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.circle(80, 90)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Дуга — четверть окружности, нарисованная сплошной фиолетовой линией

`circle(80, 90)` — та же окружность, но только четверть (90° из 360°)

duga_kod.py

```
artist.circle(80, 90)  # радиус 80, но только 90° из 360°
```

`dot()` — точка без движения

В отличие от всех предыдущих команд, `dot()` не двигает черепашку ни на пиксель — она просто ставит закрашенный кружок в текущей позиции:

tochki.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=260)
artist = turtle.Turtle()
artist.hideturtle()
artist.penup()

artist.goto(-100, 0); artist.dot(20, "#5B24F9")
artist.goto(0, 0); artist.dot(30, "#DB2777")
artist.goto(100, 0); artist.dot(40, "#059669")

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Три цветных точки разного размера в ряд

dot() рисует точку заданного диаметра и цвета — без всякого движения черепашки

tochki_kod.py

```
artist.dot(20, "#5B24F9")
artist.forward(100)
artist.dot(30, "#DB2777")
```

Практика: заливка, круг, дуга и точка

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/06-14/index.html) (<https://www.cartesianschool.org/practice/06-14/index.html>)

Переходим к случайным точкам на экране

`goto()` перемещает черепашку в любую точку экрана напрямую — вместе со случайными числами получается интересный эффект.

До сих пор черепашка двигалась только относительно своей текущей позиции или по заранее известным координатам. Но можно сразу «телепортировать» её в конкретную точку экрана командой `goto(x, y)` — а вместе со случайными числами из главы 5 это открывает интересные возможности.

Случайные точки на экране

```
sluchaynye_tochki.py
```

```
import turtle
import random

random.seed(7)
screen = turtle.Screen()
screen.setup(width=420, height=340)
artist = turtle.Turtle()
artist.hideturtle()
artist.penup()

for _ in range(10):
    x = random.randint(-200, 200)
    y = random.randint(-150, 150)
    artist.goto(x, y)
    artist.dot(14, "#5B24F9")

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Десять фиолетовых точек, разбросанных по окну в случайных, но воспроизводимых местах

10 случайных точек — при `random.seed(7)` каждый запуск даёт одну и ту же картину

```
sluchaynye_tochki_kod.py
```

```
import turtle
import random

random.seed(7)  # фиксируем seed – картинка будет
одинаковой при каждом запуске
screen = turtle.Screen()
artist = turtle.Turtle()
artist.penup()

for _ in range(10):
    x = random.randint(-200, 200)
    y = random.randint(-150, 150)
    artist.goto(x, y)
    artist.dot(14, "#5B24F9")
```

penup() перед goto()

Без `penup()` черепашка будет рисовать линию при каждом перемещении `goto()` — что обычно не то, что нужно для «прыжка» в новую точку перед тем, как поставить точку.

Зачем здесь random.seed(7)

Мы разбирали `random.seed()` в главе 5 — здесь она нужна именно затем, чтобы КАЖДЫЙ запуск этого кода давал одну и ту же картину, а не новую каждый раз. Уберите строку с `seed` — и точки станут по-настоящему случайными при каждом запуске.

Практика: goto(), penup()/pendown() и random

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/06-06/index.html>)

Случайное движение

Маленький шаг и случайный поворот, повторённые много раз, — и получается извилистая, неповторяющаяся траектория.

Комбинируя маленький шаг вперёд со случайным поворотом на каждой итерации, можно получить неповторяющуюся, органично выглядящую траекторию — приём, который называется «случайным блужданием» (random walk).

Идея — сначала без кода

Представьте: сделать маленький шаг, слегка повернуть в случайную сторону, снова шаг, снова случайный поворот — и так много раз подряд. Никакого плана, только повторение одного и того же маленького решения.

```
sluchaynoe_dvizhenie.py
```

```
import turtle
import random

random.seed(3)
screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.pensize(2)
artist.pencolor("#5B24F9")
artist.speed(0)

for _ in range(40):
    artist.forward(15)
    artist.right(random.randint(-60, 60))

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Извилистая фиолетовая линия, блуждающая по окну без определённого направления

«Случайное блуждание»: 40 маленьких шагов, каждый раз со случайным поворотом от -60° до 60°

Забегает вперёд

Код ниже использует цикл `for` — мы разберём его подробно позже. Пока не нужно понимать синтаксис цикла целиком: важна сама идея — маленький шаг и случайный поворот, повторённые много раз.

sluchaynoe_dvizhenie_kod.py

```
import turtle
import random

random.seed(3)  # фиксируем seed для воспроизводимой
картинки
screen = turtle.Screen()
artist = turtle.Turtle()
artist.pensize(2)
artist.pencolor("#5B24F9")
artist.speed(0)

for _ in range(40):
    artist.forward(15)
    artist.right(random.randint(-60, 60))
```

Попробуйте изменить

Уберите `random.seed(3)` — траектория станет новой при каждом запуске. Измените диапазон `randint(-60, 60)` на более узкий, например `(-15, 15)` — путь станет более прямым и предсказуемым.

Практика: случайное движение

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/06-15/index.html>)

Рисуем по координатам

План на бумаге в координатах — и тот же план, переведённый в код `goto()`.

До сих пор мы строили фигуры движением («проехать, повернуть») или уже готовыми командами (`circle()`). Но можно спланировать рисунок сразу в координатах — на бумаге или в голове — и просто перечислить точки для `goto()`.

Сначала — план на координатной плоскости

Прежде чем писать код, решим, где будут находиться вершины треугольника:

Вершина	Координата
Нижний левый угол	(-60, -50)
Нижний правый угол	(60, -50)
Верхний угол	(0, 70)

Теперь — код

```
risuem_po_koordinatam_kod.py
```

```
artist.fillcolor("#B9A0FC")
artist.penup()
artist.goto(-60, -50)
artist.pendown()

artist.begin_fill()
artist.goto(60, -50)
artist.goto(0, 70)
artist.goto(-60, -50)
artist.end_fill()
```

И, наконец, результат

```
risuem_po_koordinatam.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=380)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")
artist.fillcolor("#B9A0FC")
artist.penup()
artist.goto(-60, -50)
artist.pendown()

artist.begin_fill()
artist.goto(60, -50)
artist.goto(0, 70)
artist.goto(-60, -50)
artist.end_fill()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Закрашенный светло-фиолетовый треугольник,
построенный по трём явным координатам

Треугольник, построенный по трём заранее
продуманным координатам, с заливкой

Планирование в координатах — мощный приём

Такой подход особенно удобен, когда фигура сложная и должна получиться **точно** такой, как задумано — гораздо надёжнее, чем подбирать углы и расстояния на глаз. В следующем разделе мы соберём несколько фигур в одну композицию именно так — план сначала, код потом.

Практика: рисуем по координатам

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/06-16/index.html>)

Отладка Turtle

Графика добавляет несколько новых способов ошибиться — разберём их по схеме «симптом → причина → исправление».

Графика добавляет несколько новых, специфичных для Turtle способов что-то сделать не так — разберём самые частые из них по схеме «симптом → причина → исправление».

Симптом	Причина	Исправление
Окно открывается и сразу же закрывается	Программа закончилась, и Python закрыл окно вместе с ней	Добавьте <code>screen.exitonclick()</code> (ждёт клика) или <code>turtle.done()</code> в самом конце программы

Симптом	Причина	Исправление
Черепашка «смотрит не туда», хотя код кажется верным	Перепутаны <code>left()</code> / <code>right()</code> , или забыт один из поворотов в последовательности	Перечитайте программу построчно и для каждой команды спросите: «это движение или поворот?»
Часть рисунка не появляется на экране	Забыт <code>pendown()</code> после <code>penup()</code> — черепашка честно двигается, просто ничего не рисует	Проверьте, что перед каждым «настоящим» движением перо снова опущено
Фигура нарисовалась, но заливки нет	Забыт <code>end_fill()</code> , либо контур не замкнулся в точности там, где начался	Убедитесь, что путь между <code>begin_fill()</code> и <code>end_fill()</code> возвращается в стартовую точку
Часть рисунка «пропадает» за краем окна	Координаты вышли за пределы видимой области — окно не бесконечное	Проверьте <code>screen.setup()</code> и не выходите за половину ширины/высоты окна от центра
TclError или окно вовсе не открывается на сервере/Linux без монитора	Turtle использует Tk — нативный GUI, которому нужен настоящий (или виртуальный) дисплей	Запускайте Turtle на компьютере с обычным рабочим столом; для автоматизации без монитора нужен виртуальный дисплей вроде Xvfb — это уже не начальный уровень

Симптом	Причина	Исправление
Практика Turtle не открывается прямо в браузере на этом сайте	Браузерная среда Python (Pyodide) не умеет открывать нативные Tk-окна — это техническое ограничение, а не ошибка в вашем коде	Такие практики отмечены «требуется локальный Python» — скачайте .ipynb и запустите его в VS Code, PyCharm или Jupyter на своём компьютере

Почему Turtle нельзя запустить прямо в браузере на этом сайте

Обычный Python (CPython) рисует Turtle через Tk — библиотеку нативных окон операционной системы. Браузерная версия Python, на которой работают интерактивные практики этого курса, не имеет доступа к нативным окнам вообще — поэтому практики Turtle помечены как локальные с самого начала главы, а не потому что что-то не работает.

Предскажите, прежде чем запускать

Как и с числами в главе 5, лучшая привычка — предсказать курс и примерные координаты черепашки **до** запуска, а не после. Если предсказание разошлось с реальностью — вы либо нашли баг, либо неточно понимаете команды. Оба случая стоит разобрать.

★ Базовая практика

Предскажите курс

Черепашка стоит на (0, 0), курс 0°. Выполнены команды: left(90), right(45). Каков итоговый курс? Проверьте себя, выполнив это в реальном окне Turtle.

★★ Самостоятельная задача**Найдите ошибку**

В программе, рисующей квадрат, после третьего поворота получается пятиугольник, а не квадрат. Сколько раз в коде повторяется пара `forward()/right(90)`? Сколько должно быть?

Практика: отладка Turtle

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/06-17/index.html) → (<https://www.cartesianschool.org/practice/06-17/index.html>)

Мини-проекты

Четыре небольших, но по-настоящему готовых проекта — собираем всё, что прошли в этой главе, в законченные картинки.

Четыре небольших, но по-настоящему готовых проекта — каждый использует только то, что мы уже прошли в этой главе. Пятый проект, мандала, ждёт в самом конце главы — это финал.

Проект А: цветной дом

Три отдельные фигуры (стены, крыша, дверь), каждая со своей заливкой, собранные в одну композицию с помощью `penup()` / `goto()` между ними:

```
cvetnoj_dom.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.pensize(3)
artist.speed(0)

# стены
artist.penup(); artist.goto(-80, -80);
artist.pendown()
artist.pencolor("#5B24F9");
artist.fillcolor("#EDE9FE")
artist.begin_fill()
for _ in range(4):
    artist.forward(160)
    artist.left(90)
artist.end_fill()

# крыша
artist.penup(); artist.goto(-100, 80);
artist.pendown()
artist.pencolor("#B91C1C");
artist.fillcolor("#FCA5A5")
artist.begin_fill()
artist.goto(0, 160)
artist.goto(100, 80)
artist.goto(-100, 80)
artist.end_fill()

# дверь
artist.penup(); artist.goto(-20, -80);
artist.pendown()
artist.pencolor("#78350F");
artist.fillcolor("#92400E")
artist.begin_fill()
for _ in range(2):
    artist.forward(40)
```

```
artist.left(90)
artist.forward(70)
artist.left(90)
artist.end_fill()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Домик из фиолетовых стен, красной треугольной крыши и коричневой прямоугольной двери

Проект А: Цветной дом — стены, крыша и дверь, три отдельные заливки

★ Базовая практика

Окно

Добавьте квадратное окно в стене дома — ещё один маленький `begin_fill()/end_fill()` блок.

Проект В: мишень

Четыре окружности одна внутри другой, с уменьшающимся радиусом и чередующимися цветами:

mishen.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=380)
artist = turtle.Turtle()
artist.speed(0)
artist.hideturtle()

rings = [(90, "#DC2626"), (65, "#FAFAFC"), (40,
"#DC2626"), (15, "#FAFAFC")]
for radius, color in rings:
    artist.penup()
    artist.goto(0, -radius)
    artist.pendown()
    artist.fillcolor(color)
    artist.pencolor("#0D0230")
    artist.begin_fill()
    artist.circle(radius)
    artist.end_fill()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Круглая мишень из чередующихся красных и белых концентрических колец

Проект В: Мишень — четыре концентрические окружности с чередующейся заливкой

mishen_kod.py

```

rings = [(90, "#DC2626"), (65, "#FAFAFC"), (40,
"#DC2626"), (15, "#FAFAFC")]
for radius, color in rings:
    artist.penup()
    artist.goto(0, -radius)
    artist.pendown()
    artist.fillcolor(color)
    artist.begin_fill()
    artist.circle(radius)
    artist.end_fill()

```

Забегая вперёд

Здесь использован цикл `for` по списку пар (радиус, цвет) — мы разберём списки и циклы подробно позже. Пока достаточно увидеть идею: одна и та же последовательность действий повторяется для каждой пары значений.

★★ Самостоятельная задача

Свои цвета

Замените цвета колец на свои — например, оттенки синего и жёлтого.

Проект С: звёздное небо

Тёмный фон, много точек случайного размера в случайных местах — тот же приём, что и в разделе про случайные точки, но с другим настроением:

```
zvezdnoe_nebo.py
```

```
import turtle
import random

random.seed(11)
screen = turtle.Screen()
screen.setup(width=420, height=340)
screen.bgcolor("#0D0230")
artist = turtle.Turtle()
artist.hideturtle()
artist.penup()

for _ in range(35):
    x = random.randint(-200, 200)
    y = random.randint(-150, 150)
    size = random.choice([6, 8, 10])
    artist.goto(x, y)
    artist.dot(size, "#FDE68A")

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Тёмно-фиолетовое небо с рассыпанными по нему жёлтыми точками-звёздами разного размера

Проект С: Звёздное небо — тёмный фон и случайно расположенные точки-звёзды

★★ Самостоятельная задача

Луна

Добавьте одну большую белую точку в углу экрана — «луну», нарисованную заранее, до случайных звёзд.

Проект D: геометрическая звезда

Пятиконечная звезда рисуется одной линией, без единого отрыва пера — секрет в том, что поворот на 144° (а не на 72°) заставляет линию пересекать саму себя:

```
geometricheskaya_zvezda.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=380, height=380)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")
artist.speed(0)

artist.forward(150)
artist.right(144)
artist.forward(150)
artist.right(144)
artist.forward(150)
artist.right(144)
artist.forward(150)
artist.right(144)
artist.forward(150)
artist.right(144)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Пятиконечная звезда, нарисованная одной непрерывной фиолетовой линией

Проект D: Геометрическая звезда — пять отрезков по 150 пикселей, поворот 144° каждый раз

```
geometrisheskaya_zvezda_kod.py
```

```
for _ in range(5):
    artist.forward(150)
    artist.right(144)
```

Откуда взялось число 144

144° — это $2 \times 360 / 5$: чтобы звезда «сложилась» правильно, черепашка должна обойти вокруг центра дважды за пять шагов, а не один раз, как в обычном пятиугольнике.

★★★ Задача повышенной сложности

Звезда о семи концах

Замените 5 на 7 и 144° на подходящий угол (подсказка: тот же принцип — обойти дважды за семь шагов). Что получится?

Практика: мини-проекты

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/06-18/index.html>)

Мини-проект — рисуем мандалу только из прямых линий

Собираем все приёмы главы в одном узоре — от первого мотива до полного результата — и подводим итоги.

Соберём все приёмы главы в одном финальном узоре: мандале, составленной только из прямых линий, — множество лучей одинаковой длины, каждый раз повернутых на небольшой угол.

Планирование: что вообще происходит

Идея простая: встать в одну точку, посмотреть в направлении `ugol`, проехать вперёд и сразу же вернуться назад по той же линии — а затем немного повернуть курс и повторить. Если повторить это достаточно много раз, покрывая все 360° вокруг центра, лучи сольются в симметричный узор.

Один мотив

Сначала — всего несколько лучей, чтобы увидеть сам приём до того, как он превратится в насыщенный узор:

`motiv.py`

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")

shag_ugla = 10
ugol = 0
while ugol < 60:
    artist.setheading(ugol)
    artist.forward(150)
    artist.backward(150)
    ugol += shag_ugla

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Веер из нескольких фиолетовых лучей, выходящих из одной точки под небольшим углом друг к другу

Один мотив: несколько лучей под небольшим углом друг к другу — ещё не полная мандала

```
motiv_kod.py
```

```
shag_ugla = 10
ugol = 0
while ugol < 60:
    artist.setheading(ugol)
    artist.forward(150)
    artist.backward(150)
    ugol += shag_ugla
```

Забегая вперёд

Цикл `while` подробно разберём позже — здесь достаточно увидеть, что `setheading()` позволяет нарисовать сложный узор всего в нескольких строках, перебирая углы по кругу.

Полный оборот — готовая мандала

Тот же код, только цикл теперь идёт не до 60° , а до полных 360° :

`mandala.py`

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")

shag_ugla = 10
ugol = 0
while ugol < 360:
    artist.setheading(ugol)
    artist.forward(150)
    artist.backward(150)
    ugol += shag_ugla

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Симметричная мандала из множества прямых фиолетовых лучей, расходящихся из центра во все стороны

Тот же приём, доведённый до полного оборота 360° — мандала из прямых линий

```
mandala_kod.py
```

```
shag_ugla = 10
ugol = 0
while ugol < 360:
    artist.setheading(ugol)
    artist.forward(150)
    artist.backward(150)
    ugol += shag_ugla
```

★ Базовая практика

Другой шаг угла

Измените shag_ugla на 5 или на 30 — как меняется узор?

★★ Самостоятельная задача

Другая длина луча

Измените длину forward(150) — попробуйте 80 и 250.

★★★ Задача повышенной сложности

Цветная мандала

Добавьте artist.pencolor("violet") перед циклом и поэкспериментируйте с другими цветами.

Практика: мандала из прямых линий

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/06-08/index.html>)

Финальная карта главы

Какая команда Turtle мне нужна?

Нужно
узнать, где
сейчас
черепашка? → `position() / xcor() / ycor()`

Нужно
переместиться,
не рисуя? → `penup() / pendown()`

Нужно попасть в
конкретную точку? → `goto(x, y)`

Нужен
правильный
многоугольник? → `поворот = 360 / n`

Нужна
закрашенная
фигура? → `begin_fill() / end_fill()`

Нужна
окружность
или дуга? → `circle(радиус, [extent])`

Нужна повторяемая
случайность? → `random.seed()`

Главная сводная карта главы 6 — возвращайтесь к
ней в будущих проектах

Итоги

Что мы узнали в этой главе

- Экран (`turtle.Screen()`) и черепашка (`turtle.Turtle()`) — два разных объекта; у черепашки есть состояние: позиция, курс, перо, цвет.
- Окно Turtle использует ту же координатную систему, что и математика: центр (0, 0), x вправо, y вверх.
- `forward()` / `backward()` двигают черепашку; `left()` / `right()` поворачивают курс относительно текущего направления; `setheading()` задаёт направление абсолютно.
- Угол поворота для правильного многоугольника — $360 / n$.
- `penup()` / `pendown()` управляют тем, рисуется ли линия при движении; `goto(x, y)` перемещает черепашку в конкретную точку экрана напрямую.
- `begin_fill()` / `end_fill()` закрашивают замкнутый контур; `circle()` рисует окружности и дуги; `dot()` ставит точку без движения.
- `random.seed()` делает случайную графику воспроизводимой — полезно и для документации, и для отладки.

Что дальше

В главе 7 мы вернёмся к Turtle и настроим её ещё тоньше — научимся точно настраивать экран, рисовать текст, освоим дуги подробнее и соберём собственного смайлика. Всё, что вы узнали здесь — координаты, углы, перо, заливка, — станет тем фундаментом, на котором строится более тонкая настройка.

Глубокое погружение в Turtle

Turtle как настоящая графическая система: окно и холст, цвет и режимы `colormode`, перо и заливка отдельно, глубокая геометрия окружности, штампы, текст и координаты, несколько черепашек и `clone()`, отладка графики — и четыре мини-проекта, включая часы и координатную мишень.

7.1 Экран как графическая система

7.2 `colormode` и цвет

7.3 Настраиваем саму Turtle

7.4 Перо и заливка — не одно и то же

7.5 `speed`, `tracer` и `update`

7.6 Фигуры без линий, окружности, точки

7.7 Геометрия окружности

7.8 Дуги

7.9 `circle(steps=...)` — от круга к многоугольнику

7.10 Штампы и другие приёмы

7.11 Рисуем текст на экране

7.12 Выравнивание и шрифт

7.13 Текст и координатная система

7.14 Черепашка как графический объект

7.15 Несколько черепашек

7.16 `clone()` — копируем состояние

7.17 `clear()`, `reset()` и `home()`

7.18 Отладка графики

7.19 Мини-проект — окружность в квадрате

7.20 Меняем направление рисования

7.21 Мини-проект — часы без времени

7.22 Мини-проект — координатная мишень

7.23 Мини-проект — смайлик и итоги

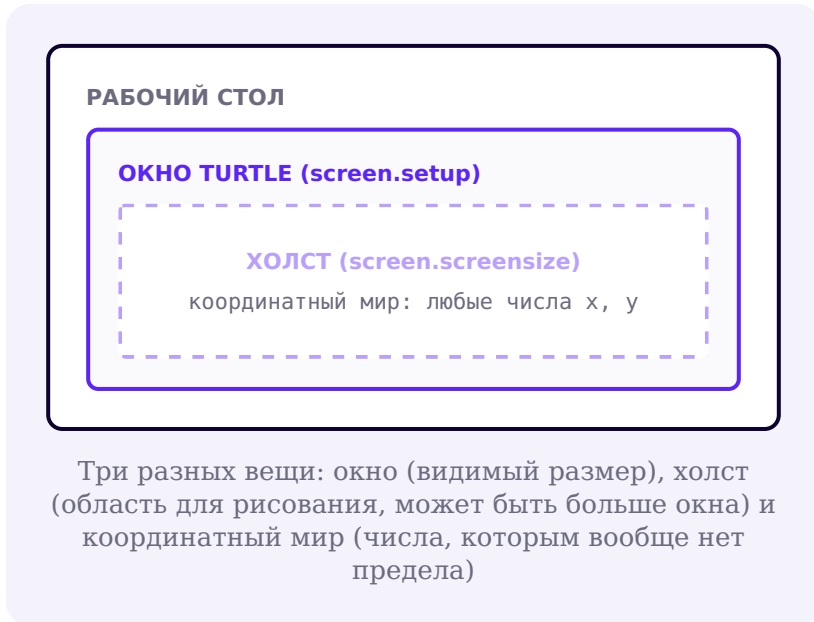
Экран как графическая система

Прежде чем рисовать что-то сложнее квадратов, разберёмся, что вообще значит «размер» в Turtle — окно, холст и координатный мир не одно и то же.

В главе 6 мы уже создавали окно и черепашку — но не разбирались, что вообще внутри происходит. Начнём с того, что Turtle — это не просто «нарисовать линию», а настоящая графическая система с несколькими слоями.

Три разные вещи, которые легко перепутать

Когда мы говорим о «размере» в Turtle, на самом деле речь может идти о трёх разных вещах:



Три разных вещи: окно (видимый размер), холст (область для рисования, может быть больше окна) и координатный мир (числа, которым вообще нет предела)

- **Окно** — то, что видно на экране компьютера, размер задаёт `screen.setup()`.
- **Холст** — область, на которой можно рисовать; обычно совпадает с окном, но может быть и больше (тогда появляются полосы прокрутки) — размер задаёт `screen.screensize()`.
- **Координатный мир** — числа `x` и `y`, которыми мы оперируем в коде; они ничем не ограничены и не обязаны совпадать с пикселями окна один в один.

Для начала достаточно setup()

В подавляющем большинстве учебных программ довольно одного `screen.setup()` — окно и холст совпадают, и думать о разнице не приходится.

`screensize()` нужен реже — например, когда рисунок больше, чем разумно показывать на экране целиком.

Настраиваем окно: до и после

Три команды экрана — заголовок, цвет фона и размер:

svetlaya_tema.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=500, height=360)
screen.title("Cartesian Turtle Studio")
screen.bgcolor("#F5F2FF")
artist = turtle.Turtle()
artist.pencolor("#5B24F9")
artist.pensize(3)
artist.forward(150)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Светлое окно Turtle светло-фиолетового оттенка с заголовком Cartesian Turtle Studio и короткой фиолетовой линией

`setup(width=500, height=360)`, `title(...)` и светлый `bgcolor()`

```
svetlaya_tema_kod.py
```

```
screen.setup(width=700, height=500)
screen.title("Cartesian Turtle Studio")
screen.bgcolor("#F5F2FF")
```

Меняем только цвет фона — вся остальная логика рисования остаётся той же:

```
temnaya_tema.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=500, height=360)
screen.title("Cartesian Turtle Studio – Dark")
screen.bgcolor("#0D0230")
artist = turtle.Turtle()
artist.pencolor("#B9A0FC")
artist.pensize(3)
artist.forward(150)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Тёмно-синее окно Turtle с той же линией, нарисованной светло-фиолетовым цветом

Тот же код, но с тёмным bgcolor() — тема окна не меняет ничего в логике рисования

Практика: настройка экрана как системы

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/07-01/index.html\)](https://www.cartesianschool.org/practice/07-01/index.html)

colormode и цвет

Название, HEX-код или тройка чисел RGB — три способа сказать Turtle одно и то же: «вот этот цвет».

Мы уже задавали цвета по имени — "purple", "gold". Но у Turtle есть и числовой способ — через компоненты красного, зелёного и синего (RGB).

Два режима: 0.0-1.0 и 0-255

`screen.colormode()` определяет, в каком диапазоне записываются числа RGB:

Режим	Диапазон каждого числа	Пример красного
<code>colormode(1.0)</code> (по умолчанию)	0.0 — 1.0	(1.0, 0.0, 0.0)
<code>colormode(255)</code>	0 — 255	(255, 0, 0)

colormode_kod.py

```
screen.colormode(255)
artist.pencolor(255, 0, 0)  # красный, компоненты 0-255

screen.colormode(1.0)
artist.pencolor(1.0, 0, 0)  # тот же красный,
                             компоненты 0.0-1.0
```

Числа вне диапазона — ошибка

Если установлен `colormode(1.0)`, а вы передадите `(255, 0, 0)` — Turtle не поймёт, что вы имели в виду красный: 255 вне диапазона 0.0–1.0, и будет ошибка. Режим и числа должны совпадать.

RGB-палитра целиком

```
rgb_svatchi.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=460, height=200)
screen.colormode(255)
artist = turtle.Turtle()
artist.hideturtle()
artist.penup()

swatches = [
    ((255, 0, 0), "red"),
    ((0, 200, 0), "green"),
    ((0, 0, 255), "blue"),
    ((255, 210, 0), "yellow"),
]
x = -170
for color, label in swatches:
    artist.goto(x, 0)
    artist.fillcolor(color)
    artist.pencolor("#0D0230")
    artist.pendown()
    artist.begin_fill()
    for _ in range(4):
        artist.forward(70)
        artist.left(90)
    artist.end_fill()
    artist.penup()
    artist.goto(x, -25)
    artist.write(label, font=("Arial", 12, "normal"))
    x += 110

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Четыре цветных квадрата в ряд — красный, зелёный, синий и жёлтый — с подписями под каждым

`colormode(255)` — четыре цвета, заданных тройками (r, g, b) от 0 до 255

```
rgb_svatchi_kod.py
```

```
screen.colormode(255)
```

```
artist.fillcolor(255, 0, 0)      # красный
artist.fillcolor(0, 200, 0)     # зелёный
artist.fillcolor(0, 0, 255)     # синий
artist.fillcolor(255, 210, 0)   # жёлтый
```

Три способа задать один и тот же цвет

Способ	Пример	Когда удобно
Название	"red"	быстро, для стандартных цветов
HEX-код	"#FF0000"	точный оттенок, как в CSS/дизайне
RGB-кортеж	(255, 0, 0)	когда цвет вычисляется в коде (например, случайный)

Не углубляемся в теорию цвета

Этого достаточно для практики этой главы. Как RGB устроен «под капотом» — тема для отдельного курса по графике, а не для этой книги.

Практика: colormode и RGB

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/07-10/index.html\)](https://www.cartesianschool.org/practice/07-10/index.html)

Настраиваем саму Turtle

Экран настроен — теперь настроим саму черепашку: скорость, толщина, цвет, форма указателя.

В прошлом разделе мы настраивали **экран**. Теперь настроим саму **черепашку**: скорость рисования, толщину и цвет линии, форму указателя.

```
nastrojka_grafiki.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=260)
artist = turtle.Turtle()

artist.speed(3)
artist.pensize(4)
artist.pencolor("purple")
artist.shape("turtle")
artist.forward(150)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Голстая фиолетовая линия, нарисованная черепашкой с формой курсора в виде силуэта черепахи

```
speed(3), pensize(4), pencolor("purple"), shape("turtle")
```

— четыре настройки черепашки разом

```
nastrojka_grafiki_kod.py
```

```
artist.speed(3)
# скорость от 1 (медленно) до 10 (быстро), 0 –
# мгновенно
artist.pensize(4) # толщина линии в пикселях
artist.pencolor("purple") # цвет линии
artist.shape("turtle") # форма указателя –
# черепашка вместо стрелки
```

speed(0) — режим художника

Когда важен результат, а не сам процесс рисования (например, в сложных узорах вроде мандалы из главы 6), удобно поставить `artist.speed(0)` — черепашка рисует мгновенно, без анимации движения. Мы разберём, почему это не влияет на итоговый рисунок, в разделе «speed, tracer и update».

Практика: включает настройку графики черепашки

Тот же ноутбук, что и в разделе «Экран как система» — он охватывает и эту тему

Открыть практику → (<https://www.cartesianschool.org/practice/07-01/index.html>)

Перо и заливка — не одно и то же

Контур фигуры и её внутренняя область красятся разными командами — разбираемся, какая за что отвечает.

Мы уже пользовались и `pencolor()`, и `fillcolor()` по отдельности — но важно чётко понимать, что это **два разных** цвета, отвечающих за разное.

Контур — это не то же самое, что заливка

Команда	За что отвечает
<code>pencolor()</code>	цвет ЛИНИИ — границы фигуры
<code>fillcolor()</code>	цвет ЗАЛИВКИ — области внутри фигуры (виден только между <code>begin_fill()/end_fill()</code>)

Команда	За что отвечает
<code>color()</code>	задаёт оба сразу: color(контур, заливка)

tolko_pencolor.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=320)
artist = turtle.Turtle()
artist.pensize(4)
artist.pencolor("blue")

for _ in range(4):
    artist.forward(120)
    artist.right(90)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Квадрат, нарисованный только синим контуром, без заливки внутри

pencolor("blue") без fillcolor — только контур, заливки нет

tolko_pencolor_kod.py

```

artist.pencolor("blue")

for _ in range(4):
    artist.forward(120)
    artist.right(90)

```

Оба цвета одной командой

color_vmeste.py

```

import turtle

screen = turtle.Screen()
screen.setup(width=340, height=320)
artist = turtle.Turtle()
artist.pensize(4)
artist.color("blue", "gold")

artist.begin_fill()
for _ in range(4):
    artist.forward(120)
    artist.right(90)
artist.end_fill()

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Гот же квадрат с синим контуром, но теперь с золотой заливкой внутри

`color("blue", "gold")` задаёт И контур, И заливку одной командой

color_vmeste_kod.py

```

artist.color("blue", "gold")  # (контур, заливка)

artist.begin_fill()
for _ in range(4):
    artist.forward(120)
    artist.right(90)
artist.end_fill()

```

color() без аргументов — узнать текущие цвета

artist.color() без аргументов возвращает пару (pencolor, fillcolor) — удобно для отладки, если непонятно, почему фигура выглядит не так, как ожидалось.

Практика: перо и заливка

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/07-11/index.html) (<https://www.cartesianschool.org/practice/07-11/index.html>)

speed, tracer и update

Скорость анимации и итоговая геометрия рисунка — совершенно разные вещи. А для по-настоящему сложных узоров есть приём куда мощнее, чем просто speed(0).

speed() меняет анимацию, а не результат

artist.speed() управляет тем, **как быстро** видно рисование — не тем, что в итоге получится. Финальный рисунок при speed(1) и при speed(0) получится геометрически **абсолютно одинаковым** — разница только в том, видите ли вы сам процесс или сразу готовый результат. Статичная картинка физически не может показать разницу в скорости — её видно только живую, при запуске кода.

tracer() и update(): рисовать пакетами

Для действительно сложных рисунков (сотни и тысячи команд) даже `speed(0)` может ощущаться медленным — экран обновляется после **каждой** команды.

`screen.tracer(0)` отключает промежуточные обновления совсем, а `screen.update()` обновляет экран один раз, вручную, когда рисунок готов:

ОБЫЧНЫЙ РЕЖИМ

команда → экран обновился

команда → экран обновился

команда → экран обновился

ПАКЕТНЫЙ РЕЖИМ — `tracer(0)`

`tracer(0)`

команда, команда, команда, ...

`update()` → экран обновился **ОДИН** раз

tracer_batch.py

```

import turtle

screen = turtle.Screen()
screen.setup(width=420, height=420)
screen.tracer(0)
artist = turtle.Turtle()
artist.hideturtle()
artist.pencolor("#5B24F9")

for i in range(36):
    artist.circle(80)
    artist.right(10)
screen.update()

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Сложный симметричный узор из 36 наложенных друг на друга окружностей, образующих цветочный паттерн

36 окружностей, каждая повернута на 10° — нарисованы мгновенно благодаря `tracer(0) + update()`

```
tracer_batch_kod.py
```

```
screen.tracer(0)  # отключаем промежуточные
                  обновления

for i in range(36):
    artist.circle(80)
    artist.right(10)

screen.update()   # один финальный апдейт — и весь
                  узор появляется разом
```

Зачем это нужно

Для 10 команд разницы почти не заметно. Но для тысяч операций `tracer(0) + update()` может сделать рисование в разы быстрее — экран просто не тратит время на перерисовку после каждого шага.

`delay()` — отдельная настройка

`screen.delay()` задаёт задержку в миллисекундах между кадрами анимации — концептуально похоже на `speed()`, но работает на уровне экрана, а не отдельной черепашки:

```
delay_kod.py
```

```
screen.delay(15)  # миллисекунды между кадрами
                  анимации
print(screen.delay()) # без аргумента — вернёт
                     текущее значение
```

Классическая ловушка отладки

tracer(0) без update() — пустой экран

Если вызвать `tracer(0)` и что-то нарисовать, но забыть `update()` — экран останется пустым! Это одна из самых частых и неочевидных ошибок с Turtle — код выполняется без единой ошибки, а на экране ничего нет. Подробнее разберём в разделе про отладку графики.

Практика: speed, tracer и update

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/07-12/index.html>)

Фигуры без линий, окружности, точки

Три инструмента из главы 6, но теперь с реальным выполненным результатом — и мостом к более глубокой геометрии окружности впереди.

Три инструмента, с которыми мы уже вскользь встречались в главе 6 — теперь наведём порядок и посмотрим на них внимательнее, прежде чем идти глубже.

Фигуры без линий: заливка цветом

Чтобы нарисовать не просто контур, а закрашенную фигуру, оберните рисование между `begin_fill()` и `end_fill()`:

zalivka.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=320)
artist = turtle.Turtle()
artist.fillcolor("gold")

artist.begin_fill()
for _ in range(4):
    artist.forward(100)
    artist.right(90)
artist.end_fill()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Золотой покрашенный квадрат

`begin_fill()/end_fill()` закрашивают замкнутый контур

zalivka_kod.py

```
artist.fillcolor("gold")
artist.begin_fill()
for _ in range(4):
    artist.forward(100)
    artist.right(90)
artist.end_fill()
```

Окружности

Рисовать окружность вручную через множество коротких отрезков не нужно — у Turtle есть готовая команда

`circle(радиус)` :

okruzhnost.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=320)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.circle(80)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Фиолетовая окружность радиусом 80 пикселей

`circle(80)` — готовая команда для окружности, без
единого поворота вручную

Точки

`dot()` — маленький (или не очень) закрашенный кружок
в текущей позиции, без движения черепашки:

```
tochka.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=260, height=200)
artist = turtle.Turtle()
artist.hideturtle()

artist.dot(50, "red")

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Одна крупная красная точка диаметром 50 пикселей

`dot(50, "red")` — точка без движения черепашки

```
tochka_kod.py
```

```
artist.dot(50, "red")  # точка диаметром 50, красная
```

Впереди — намного глубже

Это только знакомство. В следующих трёх разделах разберём геометрию окружности по-настоящему подробно — откуда берётся центр, что значит отрицательный радиус, и как окружность превращается в многоугольник.

Практика: заливка, окружности и точки

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/07-03/index.html\)](https://www.cartesianschool.org/practice/07-03/index.html)

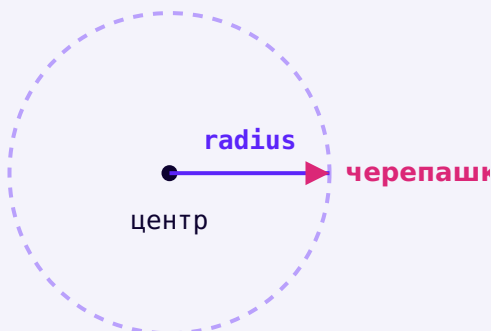
Геометрия окружности

`circle()` — не магия: у неё есть точная геометрия, которую полезно понимать, а не просто запомнить как заклинание.

`circle()` выглядит как одна простая команда, но за ней стоит настоящая геометрия — разберём её по-настоящему, а не как «магическую» функцию.

Где на самом деле центр

Официальное поведение `circle(radius)`: центр окружности находится на расстоянии `radius` **слева** от черепашки (относительно направления её курса) — не в текущей позиции черепашки, как можно было бы подумать:



Центр окружности находится на расстоянии `radius` слева от черепашки — не там, где сама черепашка стоит

Один конец дуги — там, где стоит черепашка

Если нарисовать не полную окружность, а дугу (`extent` меньше 360°) — одним из концов дуги всегда будет текущая позиция черепашки. Это удобно: не нужно заранее вычислять, откуда начнётся дуга.

Положительный и отрицательный радиус

Положительный радиус рисует окружность против часовой стрелки; отрицательный — по часовой, с тем же радиусом по модулю:


```
krug_polozhitelnyj.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=320)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.circle(80)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Окружность, нарисованная фиолетовым цветом
против часовой стрелки

`circle(80)` — положительный радиус, против часовой
стрелки

```
krug_otricatelnyj.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=320)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#DB2777")

artist.circle(-80)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Окружность того же размера, нарисованная розовым цветом по часовой стрелке

`circle(-80)` — тот же радиус по модулю, но по часовой стрелке

```
napravlenie_kod.py
```

```
artist.circle(80)    # против часовой стрелки
artist.circle(-80)   # по часовой стрелке — тот же
размер, другое направление
```

Направление меняет и курс черепашки в конце

После полной окружности курс черепашки возвращается к исходному в обоих случаях. Но если рисовать дугу (не полный круг), финальный курс будет РАЗНЫМ для положительного и отрицательного радиуса — черепашка развернётся в противоположные стороны.

Практика: геометрия окружности

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/07-13/index.html>)

Дуги

`circle()` умеет рисовать не только полные окружности, но и их части — дуги заданного размера.

Дуга — это часть окружности. У `circle()` есть второй, необязательный аргумент `extent` — сколько градусов окружности нужно нарисовать.

От четверти до полного круга

dugi_podpisannye.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=460, height=460)
artist = turtle.Turtle()
artist.speed(0)
artist.hideturtle()

arcs = [(60, "#5B24F9"), (90, "#DB2777"), (180,
"#059669"), (270, "#D97706"), (360, "#0D0230")]
for extent, color in arcs:
    artist.penup()
    artist.goto(0, 0)
    artist.setheading(0)
    artist.pendown()
    artist.pencolor(color)
    artist.pensize(3)
    artist.circle(90, extent)
    artist.penup()
    artist.write(f" {extent}°", font=("Arial", 11,
"bold"))

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Пять цветных дуг разной длины, выходящих из одной точки, каждая подписана своим значением extent в градусах

Пять дуг одного радиуса, но разного extent — от 60° до полной окружности 360°

dugi_kod.py

```
arcs = [(60, "#5B24F9"), (90, "#DB2777"), (180,
"#059669"), (270, "#D97706"), (360, "#0D0230")]
for extent, color in arcs:
    artist.pencolor(color)
    artist.circle(90, extent)
```

extent	Что рисуется
90°	четверть окружности
180°	половина окружности (полукруг)
270°	три четверти окружности
360° (или не указан)	полная окружность

Полукруг + прямая = купол

Комбинируя дугу с обычной линией, легко получить составные фигуры — например, полукруглый купол домика или арку. Мы используем именно такую дугу в мини-проекте «смайлик» в конце главы — улыбка окажется дугой в 120°.

Практика: дуги разного размера

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/07-04/index.html) → (https://www.cartesianschool.org/practice/07-04/index.html)

circle(steps=...) — от круга к многоугольнику

Секрет circle(): под капотом это всегда многоугольник — просто иногда с очень большим числом сторон.

Вот секрет, который многое объясняет: `circle()` на самом деле **не** умеет рисовать идеальную окружность — она приближает её правильным многоугольником со множеством маленьких сторон. Третий аргумент, `steps`, позволяет управлять числом этих сторон явно.

От треугольника до шестиугольника — тот же радиус

`steps_3.py`

```
import turtle

screen = turtle.Screen()
screen.setup(width=320, height=300)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.circle(80, steps=3)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Треугольник, вписанный в невидимую окружность радиусом 80

`circle(80, steps=3)` — окружность, приближённая треугольником

steps_4.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=320, height=300)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.circle(80, steps=4)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Квадрат, вписанный в невидимую окружность
радиусом 80

`circle(80, steps=4)` — тот же радиус, но 4 стороны
вместо 3

```
steps_6.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=320, height=300)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

artist.circle(80, steps=6)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Правильный шестиугольник, вписанный в невидимую окружность радиусом 80

`circle(80, steps=6)` — 6 сторон, уже больше похоже на окружность

```
circle_steps_kod.py
```

```
artist.circle(80, steps=3)  # треугольник
artist.circle(80, steps=4)  # квадрат
artist.circle(80, steps=6)  # шестиугольник
```

Мост к формуле $360/n$ из главы 6

Это та же самая идея правильного многоугольника, что мы строили вручную в главе 6 через `forward() / right(360 / n) . circle(radius, steps=n)` — просто более короткий способ получить тот же результат, если известен нужный радиус описанной окружности.

А если `steps` не указан?

Если `steps` не задан, Turtle сам подбирает достаточно большое число сторон, чтобы многоугольник выглядел гладкой окружностью — обычно несколько десятков. Мы просто не замечаем, что это тоже многоугольник.

Практика: `circle(steps=...)` как многоугольник

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/07-14/index.html>)

Практика: предскажите многоугольник по `steps` (без Turtle)

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/07-24/index.html>)

Штампы и другие приёмы

`stamp()` — новая идея: отпечаток формы черепашки без движения и без линии.

Штамп — это не линия и не курсор

`stamp()` оставляет отпечаток текущей формы черепашки на экране, не рисуя линию — важное отличие от обычного движения. Полезно для множества одинаковых объектов на экране (звёзд неба, врагов в игре, отметок на графике).

shtampy.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=460, height=200)
artist = turtle.Turtle()
artist.shape("circle")
artist.color("#5B24F9")
artist.penup()

for _ in range(6):
    artist.stamp()
    artist.forward(60)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Шесть одинаковых фиолетовых кружков-штампов в ряд, без соединяющей их линии

`stamp()` шесть раз подряд — оставляет отпечаток формы, не рисуя линию между ними

```
shtampy_kod.py
```

```
artist.shape("circle")
artist.color("#5B24F9")
artist.penup()

for _ in range(6):
    artist.stamp()
    artist.forward(60)
```

stamp() возвращает номер — запомните его

Каждый вызов `stamp()` возвращает целое число — идентификатор именно этого штампа. Без него нельзя удалить конкретный штамп позже.

Удаляем конкретные штампы

clearstamp.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=460, height=200)
artist = turtle.Turtle()
artist.shape("circle")
artist.color("#5B24F9")
artist.penup()

stamp_ids = []
for _ in range(6):
    stamp_ids.append(artist.stamp())
    artist.forward(60)

artist.clearstamp(stamp_ids[0])
artist.clearstamp(stamp_ids[2])
artist.clearstamp(stamp_ids[4])

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Гри оставшихся фиолетовых кружка-штампа из первоначальных шести, промежутки между удалёнными штампами пустые

Те же шесть штампов, но три из них удалены через `clearstamp(id)`

clearstamp_kod.py

```
stamp_ids = []
for _ in range(6):
    stamp_ids.append(artist.stamp())
    artist.forward(60)

artist.clearstamp(stamp_ids[0])
artist.clearstamp(stamp_ids[2])
artist.clearstamp(stamp_ids[4])
```

Команда	Что делает
clearstamp(id)	удаляет ОДИН конкретный штамп по его номеру
clearstamps(n)	удаляет первые n штампов (или последние n, если n отрицательное)
clearstamps()	удаляет ВСЕ штампы черепашки

Скрыть и показать черепашку

skryt.py

```
artist.hideturtle() # спрятать указатель черепашки
                     (сама линия останется)
artist.showturtle() # показать обратно
```

Зачем прятать черепашку?

В готовых рисунках указатель черепашки может визуально мешать. `hideturtle()` в конце программы делает финальный результат чище — мы используем это во всех мини-проектах главы.

Отмена последнего действия: `undo()`

`undo.py`

```
artist.forward(100)
artist.undo()    # отменяет последнее движение, как
Ctrl+Z
```

Практика: включает штампы и другие приёмы

Тот же ноутбук, что и в разделе «Фигуры, окружности, точки» — он охватывает и эту тему

Открыть практику → (<https://www.cartesianschool.org/practice/07-03/index.html>)

Рисуем текст на экране

Turtle умеет не только линии — научим её подписывать собственные рисунки.

Turtle умеет не только рисовать линии, но и выводить текст прямо на холсте — командой `write()`.

`tekst_bazovyj.py`

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=200)
artist = turtle.Turtle()
artist.hideturtle()

artist.write("Привет, Turtle!", font=("Arial", 20,
"normal"))

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Текст «Привет, Turtle!» шрифтом Arial размером 20, выведенный на белом фоне

`write()` выводит текст в текущей позиции черепашки

`tekst_bazovyj_kod.py`

```
artist.write("Привет, Turtle!", font=("Arial", 20,
"normal"))
```

Параметр `font` — это набор из трёх значений вместе: название шрифта, размер и начертание (`"normal"`, `"bold"` или `"italic"`). Такую группировку нескольких значений в одно (*кортеж*) мы разберём формально в главе про коллекции — пока достаточно писать её по образцу, как в примере.

Текст появляется от текущей позиции

`write()` не двигает черепашку и не поднимает/опускает перо — он просто печатает текст, начиная от текущей координаты. Чтобы разместить текст в конкретном месте, сначала переместитесь туда через `goto()` (обычно с `penup()` , чтобы не провести лишнюю линию).

Практика: `write()` и параметры шрифта

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/07-06/index.html>)

Выравнивание и шрифт

Текст можно не просто вывести, а управлять тем, как он растёт от точки — и как выглядит.

У `write()` есть два параметра, которые превращают текст из подписи «куда получится» в управляемый элемент интерфейса — важно для меток и подписей.

`align` — откуда растёт текст

Точка, куда мы переместили черепашку — это не всегда начало текста. Параметр `align` определяет, как текст расположен ОТНОСИТЕЛЬНО этой точки:

```
vyravniwanie.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=460, height=260)
artist = turtle.Turtle()
artist.hideturtle()
artist.penup()

for y, align, text in [(60, "left", "left"), (0,
"center", "center"), (-60, "right", "right")]:
    artist.goto(0, y)
    artist.dot(6, "#5B24F9")
    artist.write(text, align=align, font=("Arial",
16, "normal"))

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Три строки текста (left, center, right), каждая выровнена по-своему относительно отмеченной точки в начале строки

Одна и та же точка (0, y), три разных align — текст растёт в разные стороны от неё

```
vyravnivanie_kod.py
```

```
for y, align, text in [(60, "left", "left"), (0,
"center", "center"), (-60, "right", "right")]:
    artist.goto(0, y)
    artist.dot(6, "#5B24F9")
    artist.write(text, align=align, font=("Arial",
16, "normal"))
```

align	Где точка относительно текста
"left" (по умолчанию)	точка — начало текста, текст растёт вправо
"center"	точка — центр текста
"right"	точка — конец текста, текст растёт влево

font — размер и начертание

shrift.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=220)
artist = turtle.Turtle()
artist.hideturtle()
artist.penup()

artist.goto(-180, 40)
artist.write("Arial 12 normal", font=("Arial", 12,
"normal"))
artist.goto(-180, -30)
artist.write("Arial 26 bold", font=("Arial", 26,
"bold"))

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Две строки текста разного размера: маленький
обычный текст и крупный жирный текст

Два разных шрифта и размера — тот же метод write(),
другие параметры font

shrift_kod.py

```
artist.write("Arial 12 normal", font=("Arial", 12,
"normal"))
artist.write("Arial 26 bold", font=("Arial", 26,
"bold"))
```

`align="center"` — то, что нужно для подписей на графиках

Когда мы подписываем деления координатной оси или центр фигуры, `align="center"` почти всегда выглядит аккуратнее — иначе подпись съезжает в сторону от точки, которую описывает.

Практика: выравнивание и шрифт текста

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/07-15/index.html>)

Текст и координатная система

Три инструмента порознь — просто команды. Вместе они превращаются в подписанный координатный график.

Соберём вместе три инструмента из этой главы — `goto()`, `dot()` и `write()` — и координаты из главы 6, чтобы получить нечто похожее на настоящий график с подписанными делениями.

Настоящий график начинается с плана

План простой: нарисовать две оси, затем в нескольких точках вдоль каждой оси поставить точку-деление и подписать её числом.

koordinatnyj_chart.py

```

import turtle

screen = turtle.Screen()
screen.setup(width=460, height=460)
artist = turtle.Turtle()
artist.hideturtle()
artist.speed(0)

artist.penup()
artist.goto(-180, 0)
artist.pendown()
artist.goto(180, 0)
artist.penup()
artist.goto(0, -180)
artist.pendown()
artist.goto(0, 180)

artist.penup()
for value in [-100, -50, 50, 100]:
    artist.goto(value, 0)
    artist.dot(6, "#5B24F9")
    artist.goto(value, -20)
    artist.write(str(value), align="center",
font=("Arial", 11, "normal"))
    artist.goto(0, value)
    artist.dot(6, "#5B24F9")
    artist.goto(15, value)
    artist.write(str(value), font=("Arial", 11,
"normal"))
artist.goto(0, 0)
artist.dot(8, "#0D0230")
artist.goto(10, 6)
artist.write("0", font=("Arial", 11, "normal"))

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Координатная плоскость с осями X и Y, точками-делениями через каждые 50 пикселей и подписанными числами -100, -50, 0, 50, 100

Оси с числовыми подписями делений — goto(), dot() и write() вместе

koordinatnyj_chart_kod.py

```
# оси
artist.penup(); artist.goto(-180, 0);
artist.pendown(); artist.goto(180, 0)
artist.penup(); artist.goto(0, -180);
artist.pendown(); artist.goto(0, 180)

# деления и подписи
artist.penup()
for value in [-100, -50, 50, 100]:
    artist.goto(value, 0)
    artist.dot(6, "#5B24F9")
    artist.goto(value, -20)
    artist.write(str(value), align="center",
font=("Arial", 11, "normal"))
    artist.goto(0, value)
    artist.dot(6, "#5B24F9")
    artist.goto(15, value)
    artist.write(str(value), font=("Arial", 11,
"normal"))
```

Три главы в одном рисунке

Эта картинка — не случайность: числа и координаты из главы 5, идея координатного мира из главы 6, и write() / dot() из этой главы соединились в один полезный инструмент. Мы используем эту же идею в мини-проекте «координатная мишень» в конце главы.

Практика: текст и координатная система

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/07-16/index.html\)](https://www.cartesianschool.org/practice/07-16/index.html)

Черепашка как графический объект

Форма, размер и наклон курсора — настройки внешнего вида, независимые от того, куда и как черепашка на самом деле движется.

До сих пор форма черепашки была для нас просто «указателем». Но у неё есть собственные настройки внешнего вида — размер и наклон — независимые от логики движения.

Встроенные формы

galereya_form.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=560, height=180)
artist = turtle.Turtle()
artist.penup()
artist.pencolor("#5B24F9")

shapes = ["arrow", "turtle", "circle", "square",
         "triangle", "classic"]
x = -220
for shape_name in shapes:
    artist.goto(x, 20)
    artist.shape(shape_name)
    artist.stamp()
    artist.goto(x, -25)
    artist.write(shape_name, align="center",
font=("Arial", 10, "normal"))
    x += 85

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Шесть разных значков-курсоров черепашки в ряд:
arrow, turtle, circle, square, triangle, classic — каждый
подписан своим именем

Все встроенные формы курсора черепашки — от
arrow до classic

galereya_form_kod.py

```
shapes = ["arrow", "turtle", "circle", "square",  
         "triangle", "classic"]  
for shape_name in shapes:  
    artist.shape(shape_name)  
    artist.stamp()
```

shapesize() — размер курсора, не размер рисунка

razmer_formy.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=220)
artist = turtle.Turtle()
artist.shape("turtle")
artist.pencolor("#5B24F9")
artist.penup()

sizes = [(0.5, -140), (1, -20), (2, 140)]
for factor, x in sizes:
    artist.goto(x, 0)
    artist.shapesize(factor, factor)
    artist.stamp()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Гри силуэта черепахи разного размера — маленький, обычный и увеличенный вдвое

shapesize() растягивает вид курсора — 0.5×, 1× и 2× одной и той же формы

```
razmer_formy_kod.py
```

```
artist.shape("turtle")
for factor in [0.5, 1, 2]:
    artist.shapesize(factor, factor)
    artist.stamp()
```

shapesize() не меняет координаты

Увеличенная черепашка выглядит больше, но это только внешний вид курсора — все координаты, повороты и расстояния в коде остаются точно такими же, как были.

tilt() — наклон вида, а не курс движения

Здесь важно различать два похожих, но разных понятия:

Понятие	Что означает
heading (курс)	куда черепашка ДВИЖЕТСЯ — это меняют left()/right()/setheading()
tilt (наклон)	как повёрнут ВИД курсора на экране — это меняет только tilt()/tiltangle()

`naklon.py`

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=220)
artist = turtle.Turtle()
artist.shape("arrow")
artist.shapesize(3, 3)
artist.pencolor("#5B24F9")
artist.penup()

artist.goto(-100, 0)
artist.setheading(0)
artist.stamp()

artist.goto(100, 0)
artist.tilt(45)
artist.stamp()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Два стрелочных курсора одинакового размера — левый смотрит прямо, правый наклонён на 45 градусов

`tilt(45)` поворачивает ВИД курсора на 45°, не меняя направление движения

naklon_kod.py

```
artist.shape("arrow")
artist.shapesize(3, 3)

artist.stamp()           # обычный вид

artist.tilt(45)           # только поворачиваем ВИД,
                          # курс движения не изменился
artist.stamp()
```

Зачем это может понадобиться

Представьте самолёт или машинку в игре: она может ЛЕТЕТЬ строго вправо (курс фиксирован), но слегка покачиваться визуально — это и есть разница между heading и tilt. Для базовых учебных рисунков tilt() нужен редко, но полезно знать, что курс движения и внешний вид — разные вещи.

Практика: черепашка как графический объект

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/07-17/index.html) (https://www.cartesianschool.org/practice/07-17/index.html)

Несколько черепашек

Один экран может держать сколько угодно черепашек — и у каждой будет своё, полностью независимое состояние.

Мы создавали одну черепашку на весь скрипт. Но ничто не мешает создать сразу несколько — у каждой будет своё, полностью независимое состояние.

Один экран, разные черепашки

dve_cherepashki.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=320)

alice = turtle.Turtle()
alice.color("#5B24F9")
alice.shape("turtle")
alice.penup()
alice.goto(-100, -80)
alice.pendown()
alice.setheading(70)
alice.forward(140)

bob = turtle.Turtle()
bob.color("#DB2777")
bob.shape("turtle")
bob.penup()
bob.goto(100, -80)
bob.pendown()
bob.setheading(110)
bob.forward(140)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Два отрезка разного цвета из разных точек — фиолетовый от Alice и розовый от Bob

alice и bob — два независимых объекта Turtle на одном экране

dve_cherepashki_kod.py

```
alice = turtle.Turtle()
alice.color("#5B24F9")
alice.penup(); alice.goto(-100, -80); alice.pendown()
alice.setheading(70)
alice.forward(140)

bob = turtle.Turtle()
bob.color("#DB2777")
bob.penup(); bob.goto(100, -80); bob.pendown()
bob.setheading(110)
bob.forward(140)
```

SCREEN

```
└─ alice
   │  └─ position
   │  └─ heading
   │  └─ color
   │
   └─ bob
      │  └─ position
      │  └─ heading
      └─ color
```

Своё состояние — значит, независимое поведение

Изменение курса или цвета `alice` НИКАК не влияет на `bob` — они существуют на одном экране, но живут полностью раздельно. Это и есть суть того, что `turtle.Turtle()` — не просто функция, а фабрика самостоятельных объектов.

```
sравнение_sostoyaniya.py
```

```
print(alice.position(), alice.heading())
print(bob.position(), bob.heading())
```

Название для этого — впереди

Позже, в главе про объектно-ориентированное программирование, мы дадим этому явлению формальное имя и разберём его гораздо глубже. Пока достаточно самого наблюдения: каждый `Turtle()` — отдельный, независимый объект со своим состоянием.

Практика: несколько черепашек

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/07-18/index.html>)

clone() — копируем состояние

Не «новая черепашка с нуля», а «точная копия существующей» — а дальше они полностью независимы.

`clone()` — ещё один способ получить вторую черепашку, но принципиально другой: не «создать с нуля», а «скопировать состояние существующей».

Копия стартует с того же места

clone.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=320)

original = turtle.Turtle()
original.color("#5B24F9")
original.penup()
original.goto(0, -100)
original.setheading(90)

copy = original.clone()
copy.color("#DB2777")

original.pendown()
original.left(30)
original.forward(160)

copy.pendown()
copy.right(30)
copy.forward(160)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Два отрезка, выходящих из одной общей нижней точки в разные стороны — один синий, один розовый

original и copy стартуют из одной точки с одинаковым состоянием, затем расходятся

```
clone_kod.py
```

```
original = turtle.Turtle()
original.color("#5B24F9")
original.penup(); original.goto(0, -100);
original.pendown()
original.setheading(90)

copy = original.clone()  # тот же цвет, та же
                          # позиция, тот же курс
copy.color("#DB2777")
# меняем только цвет — чтобы отличить на рисунке

original.left(30)
original.forward(160)

copy.right(30)
copy.forward(160)
```

clone() ≠ ссылка на ту же черепашку

copy — полностью НОВЫЙ, независимый объект. Он начинает с теми же позицией, курсом и настройками, что и original в момент клонирования — но дальнейшие изменения одной черепашки никак не затрагивают другую, как мы уже видели с alice и bob.

Когда это полезно

Представьте узор с несколькими одинаковыми лучами, но повернутыми под разными углами — вместо того, чтобы настраивать каждую черепашку с нуля, можно один раз настроить "эталонную" черепашку и клонировать её сколько угодно раз.

Практика: clone() и независимые копии

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/07-19/index.html) (<https://www.cartesianschool.org/practice/07-19/index.html>)

clear(), reset() и home()

Три похожие по звучанию команды — но каждая стирает и двигает что-то своё.

Три похожих по звучанию команды — но с разным эффектом. Разберём их одну за другой, используя ОДИН и тот же сценарий: нарисовать квадрат, отойти в сторону, затем очистить.

Команда	Стирает рисунок?	Двигает черепашку?
<code>clear()</code>	да	нет — черепашка остаётся, где была
<code>reset()</code>	да	да — возвращает в (0, 0) с курсором по умолчанию
<code>home()</code>	нет	да — возвращает в (0, 0), но НЕ трогает рисунок

`clear()` — стирает рисунок, не двигает черепашку

```
clear_effekt.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=260)
artist = turtle.Turtle()
artist.pencolor("#5B24F9")

for _ in range(4):
    artist.forward(100)
    artist.right(90)
artist.penup()
artist.goto(120, 80)
artist.clear()
# clear() стирает рисунок, но НЕ трогает позицию
# черепашки —
# точка появится там же, куда мы её только что
# переместили
artist.dot(14, "#DB2777")

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Пустой экран с розовой точкой в правом верхнем углу и чёрным курсором черепашки рядом с ней — квадрат стёрт, но позиция черепашки осталась прежней

После `clear()` рисунок исчез, но точка появилась там же, куда мы переместились ДО очистки — позиция не сбросилась

clear_effekt_kod.py

```
for _ in range(4):
    artist.forward(100)
    artist.right(90)

artist.penup()
artist.goto(120, 80)
artist.clear()
# clear() стирает рисунок, но НЕ трогает позицию
# черепашки –
# точка появится там же, куда мы её только что
# переместили
artist.dot(14, "#DB2777")
```

reset() — стирает рисунок И возвращает домой

```
reset_effekt.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=260)
artist = turtle.Turtle()
artist.pencolor("#5B24F9")

for _ in range(4):
    artist.forward(100)
    artist.right(90)
artist.penup()
artist.goto(120, 80)
artist.reset()
# reset() стирает рисунок И возвращает черепашку в (0,
# 0) —
# точка появится в начале координат, а не там, где мы
# были
artist.dot(14, "#DB2777")

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Пустой экран с розовой точкой точно в центре — квадрат стёрт, и черепашка вернулась в начало координат

После reset() рисунок исчез И черепашка вернулась в (0, 0) — точка появилась в центре


```
reset_effekt_kod.py
```

```
for _ in range(4):
    artist.forward(100)
    artist.right(90)

artist.penup()
artist.goto(120, 80)
artist.reset()
# reset() стирает рисунок И возвращает черепашку в (0,
# 0) —
# точка появится в начале координат, а не там, где мы
# были
artist.dot(14, "#DB2777")
```

home() — только движение, рисунок остаётся

Мы уже встречали `home()` в главе 6 — она перемещает черепашку в (0, 0) с курсом по умолчанию, но, в отличие от `reset()`, НЕ стирает то, что уже нарисовано:

```
home_kod.py
```

```
artist.forward(100)
artist.left(45)
artist.forward(60)
artist.home() # рисунок остаётся на месте, домой
              возвращается только черепашка
```

Как выбрать нужную команду

Нужно стереть рисунок, но остаться на месте? `clear()`. Нужно полностью начать заново — и рисунок, и позицию? `reset()`. Нужно просто вернуться в центр, ничего не стирая? `home()`.

Практика: clear, reset, home

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/07-20/index.html) → (<https://www.cartesianschool.org/practice/07-20/index.html>)

Практика: предскажите эффект clear/reset/home (без Turtle)

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/07-25/index.html) → (<https://www.cartesianschool.org/practice/07-25/index.html>)

Отладка графики

Более продвинутая графика добавляет новые, специфичные способы ошибиться — разберём их по схеме «симптом → причина → исправление».

Графика добавляет несколько новых, специфичных для более продвинутого Turtle способов что-то сделать не так — разберём их по схеме «симптом → причина → исправление».

Симптом	Причина	Исправление
Окружность рисуется не в ту сторону, чем ожидалось	Радиус имеет не тот знак — положительный вместо отрицательного или наоборот	Проверьте знак: положительный радиус — против часовой стрелки, отрицательный — по
Текст появился совсем не там, где ожидалось	Забыли переместить черепашку через <code>goto()</code> / <code>penup()</code> перед <code>write()</code> — текст выводится от ТЕКУЩЕЙ позиции	Переместитесь в нужную точку до вызова <code>write()</code>

Симптом	Причина	Исправление
Заливка не закрасилась, хотя fillcolor() был задан	Контур между <code>begin_fill()</code> и <code>end_fill()</code> не замкнулся — черепашка не вернулась в стартовую точку	Проверьте, что путь образует ЗАМКНУТУЮ фигуру — конец совпадает с началом
RGB-цвет вызывает ошибку вроде «недопустимое значение цвета»	Числа не соответствуют текущему <code>colormode</code> — например, <code>(255, 0, 0)</code> при <code>colormode(1.0)</code>	Сверьте числа с активным режимом — 0-255 или 0.0-1.0
Код выполняется без ошибок, но на экране совсем ничего не появилось	Вызван <code>screen.tracer(0)</code> , но забыт <code>screen.update()</code> — экран так и не обновился ни разу	Добавьте <code>screen.update()</code> после всех команд рисования
На экране остался штамп, хотя казалось, что он должен был исчезнуть	<code>clearstamp()</code> вызван с неверным <code>id</code> , либо <code>id</code> вообще не был сохранён при вызове <code>stamp()</code>	Сохраняйте <code>id</code> каждого <code>stamp()</code> в переменную/список сразу при создании
Две черепашки двигаются одинаково, хотя должны были разойтись	Обе переменные на самом деле ссылаются на ОДНУ и ту же черепашку (например, <code>bob = alice</code> вместо <code>bob = turtle.Turtle()</code>)	Создавайте каждую черепашку отдельным вызовом <code>turtle.Turtle()</code> или через <code>clone()</code>

Симптом	Причина	Исправление
Курсор черепашки выглядит повернутым не в ту сторону, хотя движется верно	Спутаны <code>heading</code> (направление движения) и <code>tilt</code> (визуальный поворот курсора) — они независимы	<code>heading</code> меняют <code>left()/right()/setheading()</code> ; внешний вид — только <code>tilt()/tiltangle()</code>
Часть рисунка не видна, хотя код точно её рисует	Координаты вышли за пределы окна — оно не бесконечное	Проверьте <code>screen.setup()</code> / <code>screensize()</code> и держите координаты в пределах видимой области

`tracer(0)` без `update()` — разберём подробно

Это настолько частая и настолько незаметная ошибка, что заслуживает отдельного разбора. Программа выполняется полностью, без единой ошибки — но экран остаётся девственно пустым:

`tracer_bug.py`

```
screen.tracer(0)
```

```
for _ in range(4):
    artist.forward(100)
    artist.right(90)
```

если здесь нет `screen.update()` — окно так и останется пустым!

Мысленная модель

`tracer(0)` — это не «выключить рисование», а «не показывать его на экране, пока не попросят». Рисование продолжает происходить по-настоящему, просто оно невидимо, пока не вызван `update()`.

Практика: отладка графики (без Turtle)

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/07-23/index.html\)](https://www.cartesianschool.org/practice/07-23/index.html)

Мини-проект — окружность внутри квадрата

Результат сначала, план потом, шаги по одному — учимся строить композицию из фигур осознанно, а не методом подбора.

Что мы построим

okruzhnost_v_kvadrate.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=340)
artist = turtle.Turtle()
artist.pensize(3)

razmer = 150
artist.pencolor("#5B24F9")
artist.penup()
artist.goto(-razmer / 2, -razmer / 2)
artist.pendown()
for _ in range(4):
    artist.forward(razmer)
    artist.left(90)

artist.pencolor("#DB2777")
artist.penup()
artist.goto(0, -razmer / 2)
artist.setheading(0)
artist.pendown()
artist.circle(razmer / 2)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Синий квадрат со стороной 150 пикселей и розовая окружность, вписанная точно внутри него

Готовый результат: квадрат со стороной 150 и вписанная в него окружность

Разложим на части

Прежде чем писать код, разберём рисунок на простые фигуры: квадрат со стороной `razmer` , и окружность радиусом `razmer / 2` , вписанная точно по центру.

Координатный план

Чтобы окружность идеально вписалась в квадрат, её нужно начать рисовать из точки на полпути вдоль нижней стороны квадрата — не из угла и не из центра:

Что рисуем	Откуда начинаем	Курс в начале
Квадрат	нижний левый угол: <code>(-razmer/2, -razmer/2)</code>	0° (вправо)
Окружность	середина нижней стороны: <code>(0, -razmer/2)</code>	0° (вправо)

Шаг 1: только квадрат

kvadrat_plan.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=340)
artist = turtle.Turtle()
artist.pensize(3)
artist.pencolor("#5B24F9")

razmer = 150
artist.penup()
artist.goto(-razmer / 2, -razmer / 2)
artist.pendown()
for _ in range(4):
    artist.forward(razmer)
    artist.left(90)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Один синий квадрат со стороной 150 пикселей, без каких-либо других фигур

Шаг 1 — сначала просто квадрат, без окружности

kvadrat_plan_kod.py

```

razmer = 150
artist.penup()
artist.goto(-razmer / 2, -razmer / 2)
artist.pendown()
for _ in range(4):
    artist.forward(razmer)
    artist.left(90)

```

Шаг 2: добавляем вписанную окружность

okruzhnost_v_kvadrate_kod.py

```

artist.pencolor("#DB2777")
artist.penup()
artist.goto(0, -razmer / 2)
artist.setheading(0)
artist.pendown()
artist.circle(razmer / 2)

```

Откуда взялась эта точка

Радиус вписанной окружности — ровно половина стороны квадрата. А центр окружности находится слева от черепашки (см. раздел «Геометрия окружности») — значит, чтобы центр совпал с центром квадрата, черепашка должна начать рисовать РОВНО из точки на нижней стороне, на расстоянии radius от центра.

Задание-вызов

koncentricheskie.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=340)
artist = turtle.Turtle()
artist.pensize(3)

razmer = 150
artist.pencolor("#5B24F9")
artist.penup()
artist.goto(-razmer / 2, -razmer / 2)
artist.pendown()
for _ in range(4):
    artist.forward(razmer)
    artist.left(90)

colors = ["#DB2777", "#059669", "#D97706"]
for i, color in enumerate(colors):
    radius = razmer / 2 - i * 20
    artist.pencolor(color)
    artist.penup()
    artist.goto(0, -radius)
    artist.setheading(0)
    artist.pendown()
    artist.circle(radius)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Гот же квадрат с тремя разноцветными концентрическими окружностями внутри вместо одной

Задание-вызов: несколько окружностей одна в другой
вместо одной

★★ Самостоятельная задача

Свой набор колец

Измените код так, чтобы получить свой набор концентрических окружностей — другие радиусы, другие цвета.

★★ Самостоятельная задача

Окружность в шестиугольнике

Замените квадрат на шестиугольник из главы 6 и впишите в него окружность подходящего радиуса.

★★★ Задача повышенной сложности

Толстый контур

Увеличьте `pensize()` квадрата и окружности — как меняется восприятие фигуры?

Практика: вписанная окружность

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/07-07/index.html>)

Меняем направление рисования

Направление, знак радиуса и режим измерения углов — три связанные идеи, собранные вместе перед финальными мини-проектами.

Мы уже разобрали геометрию окружности подробно в начале главы. Здесь соберём три связанные идеи вместе — направление рисования, знак радиуса и режим измерения углов — и закрепим их перед финальными мини-проектами.

**По часовой или против — определяет
знак радиуса**

```
cw_ccw.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=560, height=340)
artist = turtle.Turtle()
artist.speed(0)
artist.pensize(3)

# circle(70) – центр на 70 px СЛЕВА от курса: старт
# ниже центра на 70,
# чтобы сам круг встал вокруг (-150, 0), не упираясь в
# край окна
artist.penup()
artist.goto(-150, -70)
artist.pendown()
artist.pencolor("#5B24F9")
artist.circle(70)
artist.penup()
artist.goto(-150, 115)
artist.write("circle(70)", align="center",
font=("Arial", 13, "bold"))
artist.goto(-150, 95)
artist.write("против часовой (+)", align="center",
font=("Arial", 11, "normal"))

# circle(-70) – центр на 70 px СПРАВА от курса: старт
# выше центра на 70,
# чтобы круг встал вокруг (150, 0) – симметрично
# первому
artist.goto(150, 70)
artist.pendown()
artist.pencolor("#DB2777")
artist.circle(-70)
artist.penup()
artist.goto(150, 115)
artist.write("circle(-70)", align="center",
font=("Arial", 13, "bold"))
artist.goto(150, 95)
```

```
artist.write("по часовой (-)", align="center",
font=("Arial", 11, "normal"))

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Две окружности рядом, каждая со стрелкой направления — левая подписана «против часовой (+)», правая «по часовой (-)»

Против часовой (радиус положительный) и по часовой (радиус отрицательный) — рядом, для прямого сравнения

`cw_ccw_kod.py`

```
artist.circle(60)      # против часовой стрелки –
                        положительный радиус
artist.circle(-60)     # по часовой стрелке –
                        отрицательный радиус
```

Режимы измерения углов

По умолчанию Turtle измеряет углы в градусах. Если по какой-то причине удобнее работать в радианах (например, для совместимости с формулами из модуля `math` из главы 5) — есть команды `degrees()` и `radians()`:

```
radiany.py
```

```
import math

artist.radians()
artist.left(math.pi / 2)  # поворот на 90°,
                           выраженный в радианах

artist.degrees()
# возвращаемся к привычным градусам
```

Не забудьте вернуться в градусы

Если вызвать `radians()` и забыть `degrees()` в конце — все последующие повороты в программе будут интерпретироваться как радианы, а не градусы, что почти всегда приводит к неожиданным результатам.

Практика: включает практику с направлением рисования

Тот же ноутбук, что и в разделе «Окружность в квадрате» — он охватывает и эту тему

Открыть практику → (<https://www.cartesianschool.org/practice/07-07/index.html>)

Мини-проект — часы без времени

Узнаваемый циферблат — просто геометрия: деления через равные углы, числа, две стрелки на фиксированных направлениях.

Что мы построим

«Часы без времени» — статичный циферблат аналоговых часов. Модуль `datetime` нам пока не нужен — стрелки встанут на фиксированный угол, просто чтобы показать, как из геометрии собирается узнаваемый объект.


```
chasy_polnye.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#0D0230")
artist.pensize(3)

# circle(150) центрирует круг в 150 px СЛЕВА от
# текущего курса —
# поэтому стартуем на 150 ниже (0, -150), чтобы центр
# циферблата попал в (0, 0)
artist.penup()
artist.goto(0, -150)
artist.pendown()
artist.circle(150)

for hour in range(12):
    artist.penup()
    artist.goto(0, 0)
    artist.setheading(90 - hour * 30)
    artist.forward(135)
    artist.pendown()
    artist.forward(15)
    artist.penup()

for hour, label in [(0, "12"), (3, "3"), (6, "6"),
(9, "9")]:
    artist.goto(0, 0)
    artist.setheading(90 - hour * 30)
    artist.forward(110)
    artist.write(label, align="center",
font=("Arial", 16, "bold"))

artist.pensize(5)
artist.pencolor("#5B24F9")
artist.goto(0, 0)
```

```

artist.setheading(90 - 12 * 30)
artist.pendown()
artist.forward(80)
artist.penup()

artist.pensize(3)
artist.pencolor("#DB2777")
artist.goto(0, 0)
artist.setheading(90 - 2 * 30)
artist.pendown()
artist.forward(120)

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Циферблат аналоговых часов с делениями на 12 часов, числами 12, 3, 6, 9 и двумя стрелками, показывающими фиксированное время

Готовый результат: циферблат с делениями, числами 12/3/6/9 и двумя стрелками

Геометрический план

12 делений циферблата расположены по кругу через равные промежутки — знакомая формула $360 / 12 = 30$ градусов между соседними часами. Но в отличие от обычного многоугольника, здесь удобнее задавать угол каждого деления АБСОЛЮТНО, через `setheading()`, а не накапливать поворотами:

`ugol_deleniya.py`

```
# час 0 (12 часов) — направление вверх,  $90^\circ$   
# час 3           — направление вправо,  $0^\circ$   
# час 6           — направление вниз,  $-90^\circ$   
# час 9           — направление влево,  $180^\circ$   
# в общем виде:  $ugol = 90 - hour * 30$ 
```

Шаг 1: окружность и деления

chasy_delenia.py

```

import turtle

screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#0D0230")
artist.pensize(3)

# circle(150) центрирует круг в 150 px СЛЕВА от
# текущего курса –
# поэтому стартуем на 150 ниже (0, -150), чтобы центр
# циферблата попал в (0, 0)
artist.penup()
artist.goto(0, -150)
artist.pendown()
artist.circle(150)

for hour in range(12):
    artist.penup()
    artist.goto(0, 0)
    artist.setheading(90 - hour * 30)
    artist.forward(135)
    artist.pendown()
    artist.forward(15)
    artist.penup()

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Круглый циферблат с двенадцатью короткими штрихами-делениями по кругу, без цифр и стрелок

Шаг 1 — окружность и 12 делений, без чисел и стрелок

chasy_delenia_kod.py

```
artist.circle(150)

for hour in range(12):
    artist.penup()
    artist.goto(0, 0)
    artist.setheading(90 - hour * 30)
    artist.forward(135)
    artist.pendown()
    artist.forward(15)
    artist.penup()
```

Почему goto(0, 0) перед каждым делением

Каждое деление начинается заново от центра — иначе черепашка рисовала бы деления одно за другим по кругу, а не как независимые лучи от центра. penup()/pendown() вокруг каждого шага гарантирует, что линии между делениями не будет.

Шаг 2: числа и стрелки

chasy_chisla_strelki_kod.py

```
for hour, label in [(0, "12"), (3, "3"), (6, "6"),
(9, "9")]:
    artist.goto(0, 0)
    artist.setheading(90 - hour * 30)
    artist.forward(110)
    artist.write(label, align="center",
font=("Arial", 16, "bold"))

# часовая стрелка — короче, толще, фиксирована на 12
artist.pensize(5)
artist.pencolor("#5B24F9")
```

```
artist.goto(0, 0)
artist.setheading(90 - 12 * 30)
artist.pendown()
artist.forward(80)
artist.penup()
```

```
# минутная стрелка — длиннее, тоньше, фиксирована на 2
artist.pensize(3)
artist.pencolor("#DB2777")
artist.goto(0, 0)
artist.setheading(90 - 2 * 30)
artist.pendown()
artist.forward(120)
```

Готовый результат


```
chasy_polnye.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#0D0230")
artist.pensize(3)

# circle(150) центрирует круг в 150 px СЛЕВА от
# текущего курса —
# поэтому стартуем на 150 ниже (0, -150), чтобы центр
# циферблата попал в (0, 0)
artist.penup()
artist.goto(0, -150)
artist.pendown()
artist.circle(150)

for hour in range(12):
    artist.penup()
    artist.goto(0, 0)
    artist.setheading(90 - hour * 30)
    artist.forward(135)
    artist.pendown()
    artist.forward(15)
    artist.penup()

for hour, label in [(0, "12"), (3, "3"), (6, "6"),
(9, "9")]:
    artist.goto(0, 0)
    artist.setheading(90 - hour * 30)
    artist.forward(110)
    artist.write(label, align="center",
font=("Arial", 16, "bold"))

artist.pensize(5)
artist.pencolor("#5B24F9")
artist.goto(0, 0)
```

```

artist.setheading(90 - 12 * 30)
artist.pendown()
artist.forward(80)
artist.penup()

artist.pensize(3)
artist.pencolor("#DB2777")
artist.goto(0, 0)
artist.setheading(90 - 2 * 30)
artist.pendown()
artist.forward(120)

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Циферблат аналоговых часов с делениями на 12 часов, числами 12, 3, 6, 9 и двумя стрелками, показывающими фиксированное время

Готовый результат: циферблат с делениями, числами 12/3/6/9 и двумя стрелками

★ Базовая практика

Все 12 чисел

Добавьте оставшиеся 8 чисел (1, 2, 4, 5, 7, 8, 10, 11) по тому же принципу.

★★ Самостоятельная задача

Секундная стрелка

Добавьте третью, самую тонкую стрелку — секундную — на любой фиксированный угол.

★★★ Задача повышенной сложности

Другое время

Измените углы часовой и минутной стрелки, чтобы часы показывали другое (тоже фиксированное) время.

Практика: часы без времени

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/07-21/index.html) (<https://www.cartesianschool.org/practice/07-21/index.html>)

Мини-проект — координатная мишень

Настоящая маленькая инфографика — мишень, оси, случайные точки и подпись, собранные из уже знакомых инструментов.

Что мы построим

```
koordinatnaya_mishen.py
```

```
import turtle
import random

random.seed(4)
screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)
artist.hideturtle()

rings = [(140, "#DC2626"), (100, "#FAFAFC"), (60,
"#DC2626"), (20, "#FAFAFC")]
for radius, color in rings:
    artist.penup()
    artist.goto(0, -radius)
    artist.pendown()
    artist.fillcolor(color)
    artist.pencolor("#0D0230")
    artist.begin_fill()
    artist.circle(radius)
    artist.end_fill()

artist.penup()
artist.goto(-160, 0)
artist.pendown()
artist.goto(160, 0)
artist.penup()
artist.goto(0, -160)
artist.pendown()
artist.goto(0, 160)

artist.penup()
for _ in range(6):
    x = random.randint(-120, 120)
    y = random.randint(-120, 120)
    artist.goto(x, y)
    artist.dot(10, "#059669")
```

```

artist.goto(-190, 170)
artist.write("Координатная мишень", font=("Arial",
13, "bold"))

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Мишень из красных и белых концентрических колец с чёрным перекрестьем осей, шестью зелёными точками-попаданиями и заголовком «Координатная мишень»

Готовый результат: концентрическая мишень, перекрестье осей, случайные точки-попадания и подпись

Полноценная небольшая инфографика — не просто мишень, а мишень С осями координат, точками «попаданий» и подписью. Соберём её из того, что уже знаем: концентрические окружности с заливкой, оси через `goto()`, случайные точки из главы 6 и подпись через `write()`.

Разложим на слои

Слой	Из чего состоит	Что уже знакомо
Кольца мишени	4 закрашенные окружности убывающего радиуса	<code>begin_fill()/circle()/end_fill()</code>
Перекрестье	две линии через центр	<code>goto()</code> из главы 6
Точки попаданий	случайные координаты	<code>random + dot()</code> из главы 6

Слой	Из чего состоит	Что уже знакомо
Подпись	текст в углу	write() из этой главы

Код целиком

koordinatnaya_mishen_kod.py

```
import random

random.seed(4)  # фиксированный seed – точки
                 одинаковые при каждом запуске

# кольца мишени
rings = [(140, "#DC2626"), (100, "#FAF AFC"), (60,
"#DC2626"), (20, "#FAF AFC")]
for radius, color in rings:
    artist.penup()
    artist.goto(0, -radius)
    artist.pendown()
    artist.fillcolor(color)
    artist.begin_fill()
    artist.circle(radius)
    artist.end_fill()

# перекрестье осей
artist.penup(); artist.goto(-160, 0);
artist.pendown(); artist.goto(160, 0)
artist.penup(); artist.goto(0, -160);
artist.pendown(); artist.goto(0, 160)

# случайные точки попаданий
artist.penup()
for _ in range(6):
    x = random.randint(-120, 120)
    y = random.randint(-120, 120)
    artist.goto(x, y)
    artist.dot(10, "#059669")

# подпись
```

```
artist.goto(-190, 170)
artist.write("Координатная мишень", font=("Arial",
13, "bold"))
```

Почему `random.seed(4)`

Как и в главе 6, фиксированный seed делает картинку воспроизводимой — при каждом запуске точки окажутся ровно там же. Уберите эту строку, чтобы получить новый набор точек при каждом запуске.

★★ Самостоятельная задача

Подпись координат

Рядом с каждой точкой попадания добавьте `write()` с её координатами.

★★★ Задача повышенной сложности

Счёт попаданий в яблочко

Посчитайте (напечатайте в консоль), сколько случайных точек попало в центральное красное кольцо (радиус < 60) — потребуется формула расстояния из главы 5.

Практика: координатная мишень

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/07-22/index.html>)

Мини-проект — смайлик

Собираем все приёмы главы в одном дружелюбном рисунке, поэтапно — и подводим итоги.

Что мы построим

Финальный мини-проект главы — и всего блока о Turtle. Соберём дружелюбное лицо из примитивов, которые мы разбирали на протяжении всей главы: заливка, точки, дуга.

smajlik.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=340)
artist = turtle.Turtle()
artist.speed(0)

artist.penup()
artist.goto(0, -100)
artist.pendown()
artist.fillcolor("yellow")
artist.pencolor("#0D0230")
artist.pensize(2)
artist.begin_fill()
artist.circle(100)
artist.end_fill()

artist.penup()
artist.goto(-35, 35)
artist.pendown()
artist.dot(20, "black")
artist.penup()
artist.goto(35, 35)
artist.pendown()
artist.dot(20, "black")

artist.penup()
artist.goto(-45, -25)
artist.setheading(-60)
artist.pendown()
artist.pensize(4)
artist.circle(55, 120)

artist.hideturtle()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Жёлтый закрашенный круг с двумя чёрными точками-глазами и изогнутой улыбкой из толстой линии

Готовый результат: жёлтое лицо, два глаза и улыбка-дуга

Разложим лицо на примитивы

Деталь	Из чего состоит	Инструмент
Лицо	закрашенная окружность радиусом 100	fillcolor() + circle() + begin_fill()/end_fill()
Глаза	две точки	dot()
Улыбка	дуга окружности радиусом 60, extent 120°	circle(radius, extent)

Координатный план

Деталь	Координата/угол
Левый глаз	(-35, 120)
Правый глаз	(35, 120)
Старт улыбки	(-50, 60), курс -60°

Этап 1: лицо

smajlik_lico.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=340)
artist = turtle.Turtle()
artist.speed(0)

# circle(100) центрирует круг в 100 px СЛЕВА от курса
# — стартуем на
# 100 ниже (0, -100), чтобы лицо встало ровно по
# центру окна (0, 0)
artist.penup()
artist.goto(0, -100)
artist.pendown()
artist.fillcolor("yellow")
artist.pencolor("#0D0230")
artist.pensize(2)
artist.begin_fill()
artist.circle(100)
artist.end_fill()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Жёлтый покрашенный круг без каких-либо деталей лица

Этап 1 — лицо: покрашенная окружность

`smajlik_lico_kod.py`

```
artist.fillcolor("yellow")
artist.pencolor("#0D0230")
artist.pensize(2)

artist.begin_fill()
artist.circle(100)
artist.end_fill()
```

Этап 2: глаза

smajlik_glaza.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=340)
artist = turtle.Turtle()
artist.speed(0)

artist.penup()
artist.goto(0, -100)
artist.pendown()
artist.fillcolor("yellow")
artist.pencolor("#0D0230")
artist.pensize(2)
artist.begin_fill()
artist.circle(100)
artist.end_fill()

artist.penup()
artist.goto(-35, 35)
artist.pendown()
artist.dot(20, "black")
artist.penup()
artist.goto(35, 35)
artist.pendown()
artist.dot(20, "black")

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Гот же жёлтый круг, теперь с двумя чёрными точками-глазами сверху

Этап 2 — добавляем глаза: две точки

```
smajlik_glaza_kod.py
```

```
artist.penup()  
artist.goto(-35, 120)  
artist.pendown()  
artist.dot(20, "black")  
  
artist.penup()  
artist.goto(35, 120)  
artist.pendown()  
artist.dot(20, "black")
```

Этап 3: улыбка — и готово

```
smajlik_ulybka_kod.py
```

```
artist.penup()  
artist.goto(-50, 60)  
artist.setheading(-60)  
artist.pendown()  
artist.pensize(4)  
artist.circle(60, 120)  
  
artist.hideturtle()
```

smajlik.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=340)
artist = turtle.Turtle()
artist.speed(0)

artist.penup()
artist.goto(0, -100)
artist.pendown()
artist.fillcolor("yellow")
artist.pencolor("#0D0230")
artist.pensize(2)
artist.begin_fill()
artist.circle(100)
artist.end_fill()

artist.penup()
artist.goto(-35, 35)
artist.pendown()
artist.dot(20, "black")
artist.penup()
artist.goto(35, 35)
artist.pendown()
artist.dot(20, "black")

artist.penup()
artist.goto(-45, -25)
artist.setheading(-60)
artist.pendown()
artist.pensize(4)
artist.circle(55, 120)

artist.hideturtle()

screen.exitonclick()
```


РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Жёлтый закрашенный круг с двумя чёрными точками-глазами и изогнутой улыбкой из толстой линии

Готовый результат: жёлтое лицо, два глаза и улыбка-дуга

★ Базовая практика

Другое настроение

Измените дугу улыбки на дугу нахмуренных бровей — переверните направление extent.

★★ Самостоятельная задача

Цветной смайлик

Смените fillcolor лица на любой другой цвет.

★★★ Задача повышенной сложности

Щёчки

Добавьте два маленьких розовых dot() под глазами — получатся румяные щёчки.

Практика: рисуем смайлик

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/07-09/index.html>)

Итоги главы

Что мы узнали в этой главе

- Окно, холст и координатный мир — три разные вещи, которые Turtle позволяет настраивать отдельно.
- `colormode(1.0)` и `colormode(255)` — два способа записать один и тот же RGB-цвет.
- `pencolor()` красит контур, `fillcolor()` — заливку; `color()` задаёт оба сразу.
- `speed()` влияет только на анимацию, не на итоговую геометрию; `tracer(0) + update()` ускоряют сложную графику, рисуя её пакетом.
- `circle()` рисует вписанный многоугольник — `center` находится слева от черепашки, знак радиуса определяет направление, а `steps` управляет числом сторон явно.
- `stamp()` оставляет отпечаток формы без движения и без линии.
- `write()` выводит текст с настраиваемым `align` и `font`, начиная от текущей позиции.
- Несколько объектов `turtle.Turtle()` живут на одном экране независимо — у каждого своё состояние; `clone()` копирует стартовое состояние, а дальше объекты расходятся.
- `clear()`, `reset()` и `home()` стирают и двигают разные комбинации рисунка и позиции.

Что дальше

Turtle-графика на этом не заканчивается насовсем — но дальше курс пойдёт в сторону новых инструментов языка. В главе 8 мы начнём работать с текстом по-настоящему — со строками и словами, которые до сих пор появлялись у нас только как подписи на экране.

Играем с буквами и словами

Строки в Python на полную глубину: создание и кавычки, экранирование, многострочные и raw-строки, индексы и срезы с наглядными диаграммами, неизменяемость, десятки методов, форматирование f-строками, ввод от пользователя, юникод, отладка — и восемь мини-проектов.

8.1 Что такое строки?

8.2 Экранирование: `\n`, `\t` и другие

8.3 Многострочные и raw-строки

8.4 Кавычки, склеивание, повторение

8.5 Длина строки: `len()`

8.6 Индексы строки

8.7 Срезы строки

8.8 Строки нельзя изменить

8.9 Методы строк: регистр и пробелы

8.10 Методы строк: поиск и разбор

8.11 Методы проверки: isalpha и другие

8.12 in, сравнение и истинность

8.13 Перебираем строку в цикле

8.14 Форматирование строк

8.15 Ввод от пользователя

8.16 Кириллица, юникод и эмодзи

8.17 Отладка проблем со строками

8.18 Мини-проект — текст Turtle

8.19 Мини-проект — приветствие и ФИО

8.20 Мини-проекты — крик и переворот

8.21 Мини-проект — пароль и e-mail

8.22 Мини-проект — счётчик слов

8.23 Мини-проект — динамическая математика

Итоги

Что такое строки?

Текст в Python — это строки: последовательность символов, с которой можно работать так же уверенно, как с числами.

Текст — тоже данные

Почти любая программа работает с текстом: имя пользователя, сообщение в чате, название файла, адрес сайта, команда терминала. В Python текст хранится в специальном типе данных — **строке** (`str`). Строка — это **последовательность символов**: букв, цифр, знаков препинания, пробелов, эмодзи — идущих друг за другом по порядку, как бусины на нитке.

primery_strok.py

```

inya = "Ада"
soobshchenie = "Привет, как дела?"
faijl = "otchet_2026.pdf"
adres = "https://cartesianschool.org"
komanda = "git commit -m fix"

```

Всё, что вы вводите с клавиатуры через `input()` (раздел 8.15), тоже приходит в виде строки — даже если это выглядит как число.

Создаём строку

Строку заключают в кавычки — одинарные `'...'` или двойные `"..."`. В Python они полностью равнозначны: выбирайте любые, главное — не смешивать тип кавычек внутри одной пары.

sozdaem_stroki.py

```

greeting = "Привет"
name = 'Cartesian'
print(greeting, name)
# Привет Cartesian

```

Когда какие кавычки удобнее

Разницы для Python нет, но людям иногда удобнее один тип: если в тексте будет апостроф — `it's` — проще обернуть его двойными кавычками; если внутри будет прямая речь в кавычках — проще обернуть одинарными. Подробно об этом — в разделе 8.4.

Пустая строка

Строка может не содержать ни одного символа — это тоже полноценная строка, просто **пустая**: `""` или `''`. Она часто служит стартовым значением, к которому потом что-то добавляют.

pustaya_stroka.py

```
pusto = ""
print(pusto)           # (ничего не выведется – строка
                        # пуста)
print(len(pusto))      # 0
```

type() — как узнать, что перед вами строка

proverka_tipa.py

```
print(type("Python")) # <class 'str'>
print(type(5))         # <class 'int'>
print(type("5"))       # <class 'str'> – цифра в
                        # кавычках всё равно строка!
```

"5" — это не число

Строка `"5"` состоит из символа «пятёрка», а не хранит числовое значение 5. С ней нельзя напрямую выполнять арифметику — подробнее в разделе 8.15, когда мы будем разбирать `input()`.

Практика: создание строк и type()

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/08-01/index.html\)](https://www.cartesianschool.org/practice/08-01/index.html)

Экранирование: \n, \t и другие

Как вставить в строку символ, у которого нет своей клавиши — и почему написанное в коде не всегда совпадает с напечатанным на экране.

Символы, которые нельзя напечатать прямо

Иногда внутри строки нужен символ, у которого нет своей клавиши — например, «конец строки» (перенос) или «табуляция». Для них в Python есть **служебные последовательности** (escape-последовательности) — обратная косая черта \ плюс буква:

Пишем	Означает
<code>\n</code>	перенос строки (new line)
<code>\t</code>	табуляция (отступ)
<code>\\</code>	сама обратная косая черта
<code>\"</code>	двойная кавычка внутри строки в двойных кавычках
<code>\'</code>	одинарная кавычка внутри строки в одинарных кавычках

Исходный текст vs напечатанный результат

Ключевая идея: то, что вы **пишете** в коде (с обратными чертами), и то, что реально **печатается** на экране (уже без них), — два разных текста:

```
escape_primer.py
```

```
print("Первая строка\nВторая строка")
# Первая строка
# Вторая строка

print("Имя:\tВозраст:")
print("Ада:\t28")
# Имя:      Возраст:
# Ада:      28
```

Символ `\n` в исходном коде — это ДВА символа (косая черта и «n»), но Python понимает их как ОДИН специальный символ «перенос строки», который при печати превращается в настоящий перенос — на экране обратной черты уже не видно.

Зачем вообще экранировать

Обратная косая черта — служебный символ Python внутри строк. Экранирование объясняет Python: «дальше идёт не команда, а именно этот символ». Без этого механизма было бы невозможно вставить в строку кавычку того же типа, что обрамляет саму строку, или напечатать перенос строки одной командой `print()`.

repr() — увидеть служебные символы «как есть»

`repr()` печатает строку так, как она выглядит в коде — со всеми обратными чертами, а не с их результатом. Это отличный инструмент отладки, к которому мы ещё вернёмся в разделе 8.17:

`repr_vs_print.py`

```
text = "Строка 1\nСтрока 2"
print(text)          # печатает ДВЕ настоящие строки
print(repr(text))
# 'Строка 1\nСтрока 2' — видно \n как текст
```

★ Базовая практика

Табличка из трёх строк

Одной командой `print()` с `\n` выведите три строки: «Имя: Ада», «Профессия: программист», «Год: 1843».

Практика: экранирование и `repr()`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/08-11/index.html\)](https://www.cartesianschool.org/practice/08-11/index.html)

Многострочные и raw-строки

Два особых способа создать строку: тройные кавычки — для текста в несколько строк, буква `r` — для текста без экранирования.

Многострочные строки: тройные кавычки

Вместо того чтобы вставлять `\n` вручную, можно написать текст сразу в несколько строк — если обернуть его **тройными кавычками** `"""` или `'''`. Все переносы строк внутри сохраняются такими, какие они есть:

```
mnogostrochnaya.py
```

```
поет = """Код за кодом,  
шаг за шагом —  
так рождается  
программа."""  
print(поет)  
# Код за кодом,  
# шаг за шагом —  
# так рождается  
# программа.
```

Осторожно с отступами

Если код внутри функции или блока имеет отступ, а тройная строка написана с тем же отступом, эти пробелы попадут ВНУТРЬ текста — Python не убирает их автоматически. Для простых учебных программ, где тройная строка начинается с начала строки кода, это не проблема; в реальных проектах для этого существуют специальные приёмы (например, модуль `textwrap`), которые нам пока не нужны.

Raw-строки: `r"..."`

Иногда обратных чёрточек в тексте должно быть МНОГО, и экранировать каждую — утомительно и трудно читать. Классический пример — путь к файлу в Windows:

bez_raw.py

```
path = "C:\\Users\\Cartesian\\Documents"
print(path)
# C:\Users\Cartesian\Documents
```

Каждую обратную черту пришлось задваивать. Если поставить перед открывающей кавычкой букву `r` (от *raw* — «сырой»), Python перестаёт понимать `\` как начало служебной последовательности и берёт текст ровно таким, каким он написан:

s_raw.py

```
path = r"C:\Users\Cartesian\Documents"
print(path)
# C:\Users\Cartesian\Documents — тот же результат, но
без задвоенных чёрточек
```

Raw-строки убирают «шум» экранирования

Raw-строки часто используют для путей к файлам и для шаблонов регулярных выражений (тема для будущих глав) — там обратных чёрточек особенно много, и без `r"..."` код было бы тяжело читать.

★ Базовая практика

Свой путь

Выведите путь `r"D:\Projects\python-from-zero\notebooks"` через raw-строку и через обычную строку с экранированием — убедитесь, что `print()` печатает одно и то же.

Практика: тройные и raw-строки

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/08-12/index.html>)

В моей строке есть кавычки! :O

Кавычки внутри строки, склеивание строк оператором `+` и их повторение оператором `*`.

В моей строке есть кавычки! :O

Что делать, если внутри текста нужна кавычка того же типа, что обрамляет строку? Самый простой способ — обрамить строку кавычками **другого** типа:

kavychki.py

```
quote = "Она сказала: 'Привет!'"
quote2 = 'Она сказала: "Привет!"'
print(quote)
print(quote2)
# Она сказала: 'Привет!'
# Она сказала: "Привет!"
```

Если нужны именно такие же кавычки, как обрамляющие, — их экранируют (раздел 8.2) обратной косой чертой `\`:

ekranirovanie_kavychek.py

```
quote = "Она сказала: \"Привет!\""
print(quote)
# Она сказала: "Привет!"
```

Объединяем строки: +

Строки, как и числа, можно складывать оператором `+` — это называется **конкатенацией** («склеиванием»):

```
konkatenaciya.py
```

```
first_name = "Ада"
last_name = "Лавлейс"
full_name = first_name + " " + last_name
print(full_name)
# Ада Лавлейс
```

Строки с числом напрямую не сложить

"Возраст: " + 10 вызовет `TypeError` — Python не знает, что вы имели в виду: приклеить цифру «10» как текст или сложить числа. Нужно явно сказать: либо `str(10)`, либо f-строка (раздел 8.14).

```
tipeerror_i_ispravlenie.py
```

```
# print("Возраст: " + 10)      # TypeError!
print("Возраст: " + str(10))  # Возраст: 10 –
                             исправлено
```

Повторяем строку: *

Строку можно умножить на целое число — она повторится столько раз:

```
povtorenie.py
```

```
stroka = "Ha" * 3
print(stroka)      # HaHaHa

razdelitel = "-" * 20
print(razdelitel) # -----
```

Конкатенация в print()

Напомним: у `print()` есть более простой способ вывести несколько значений — через запятую, без явного `+`:

```
print_konkatenaciya.py
```

```
print(first_name, last_name)           # Ада Лавлейс –
сам добавит пробел
print(first_name + " " + last_name)    # Ада Лавлейс –
пробел нужно добавлять самому
```

★ Базовая практика

Рамка из повторения

Соберите строкой из символа «=», умноженного на 30, верхнюю и нижнюю рамку для заголовка «ГЛАВА 8», и выведите три строки: рамка, заголовок, рамка.

Практика: кавычки, конкатенация и повторение

Тот же ноутбук, что и в разделе «Что такое строки?» — он охватывает и эту тему

Открыть практику → (<https://www.cartesianschool.org/practice/08-01/index.html>)

Длина строки: len()

Считаем символы строки — включая пробелы — и готовимся к индексам.

Сколько символов в строке?

Функция `len()` («length» — длина) считает, сколько символов в строке — и пробелы считаются наравне с буквами:

```
dlina.py
```

```
word = "Python"
print(len(word))           # 6

phrase = "Python с нуля"
print(len(phrase))        # 13 – пробелы тоже
считаются!
```

Посчитайте пробелы сами

«Python с нуля» — это П-у-т-х-о-н (6) + пробел (1) + с (1) + пробел (1) + н-у-л-я (4) = 13 символов. Легко случайно забыть посчитать пробелы «на глаз» — `len()` никогда не ошибается.

Зачем нужна длина строки

`len()` пригодится уже в следующем разделе: чтобы понимать, какие индексы вообще существуют у строки. У строки длиной `n` символов индексы идут от 0 до `n - 1` — это и есть главная причина знать длину строки заранее.

`dlina_i_indeks.py`

```
word = "Python"
print(len(word))      # 6
print(word[len(word) - 1])  # n — последний символ,
индекс "длина минус один"
```

★ Базовая практика**Самое длинное слово**

Даны три строки: "Python", "программирование", "код". Выведите длину каждой и определите (по выведенным числам), какая строка самая длинная.

Практика: `len()` и границы строки

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/08-13/index.html) → (<https://www.cartesianschool.org/practice/08-13/index.html>)

Индексы строки

Каждый символ строки можно достать по номеру — с начала или с конца. Разберём это по-настоящему наглядно.

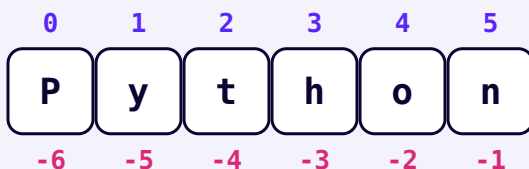
Индекс — это номер места

Строка — это последовательность символов, и у каждого символа есть свой **порядковый номер** — **индекс**.

Индекс пишут в квадратных скобках сразу после строки: строка[индекс] .

Счёт начинается с нуля!

Это самая важная деталь этого раздела. Первый символ строки имеет индекс **0**, а не 1. Второй символ — индекс 1. И так далее. Такой способ счёта называется **индексацией с нуля** (zero-based indexing) — его используют почти все языки программирования, и к нему быстро привыкаешь.



«Python»: сверху — индекс с начала (0, 1, 2, ...),
снизу — индекс с конца (-6, -5, ..., -1)

indeksy.py

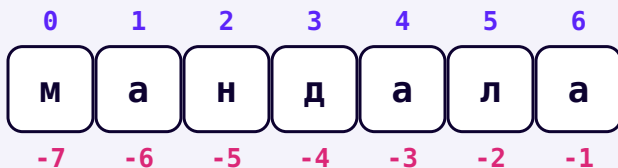
```
word = "Python"
print(word[0])    # P — первый символ (индекс 0)
print(word[1])    # y — второй символ (индекс 1)
print(word[5])    # n — шестой символ (индекс 5 —
                  # последний, т.к. всего 6 символов)
```


Отрицательные индексы: считаем с конца

Индекс `-1` означает «последний символ», `-2` — «предпоследний», и так далее. Это тот же самый ряд символов, просто прочитанный с другого конца — обратите внимание на нижний ряд чисел на диаграмме выше.

```
otricatelnye_indeksy.py
```

```
word = "Python"
print(word[-1])    # n — последний символ
print(word[-2])    # o — предпоследний
print(word[-6])    # P — тот же символ, что и word[0]!
```



Тот же принцип работает и для кириллицы — «мандала», 7 символов, индексы `0...6` и `-7...-1`

IndexError — индекс за пределами строки

`word[10]` для шестибуквенного слова вызовет `IndexError: string index out of range`. Индексы существуют только от `0` до `len(word) - 1` (раздел 8.5), и от `-1` до `-len(word)` с конца.

indexerror.py

```
word = "Python"
# word[10] # IndexError: string index out of range
print(len(word)) # 6 — значит, максимальный
индекс 5
```

★ Базовая практика

Первая и последняя буква

Для слова «Cartesian» выведите его первую букву через индекс 0 и последнюю — двумя способами: через положительный индекс (с `len()`) и через отрицательный индекс -1. Все три способа должны дать одинаковый результат для последней буквы.

Практика: индексы и отрицательные индексы

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/08-03/index.html\)](https://www.cartesianschool.org/practice/08-03/index.html)

Срезы строки

Срез достаёт из строки сразу целый кусок — а не один символ. Разберём границы среза наглядно, по коробочкам.

Срез — это кусок строки

Срез (slice) достаёт из строки сразу несколько символов подряд: `строка[start:stop]`. Здесь `start` — индекс, с которого срез **начинается** (включительно), а `stop` — индекс, на котором срез **заканчивается** (НЕ включительно).

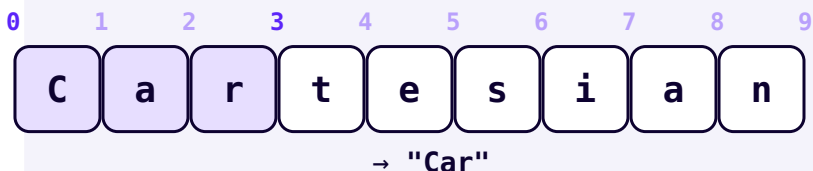
stop не включается — и это специально

Правая граница среза не входит в результат. Звучит странно, но у этого есть удобное следствие: длина среза всегда равна `stop - start`. Мысленно удобно представлять индексы не как номера БУКВ, а как номера ГРАНИЦ МЕЖДУ буквами — посмотрите на числа над коробками ниже: они стоят ровно в промежутках.

Первые три символа

```
srez_pervye_tri.py
```

```
word = "Cartesian"
print(word[0:3])    # Car
```



`word[0:3]` — от границы 0 до границы 3 (не включая её)

Последние три символа

```
srez_poslednie_tri.py
```

```
word = "Cartesian"
print(word[6:9])    # ian
```



Кусок из середины

`srez_seredina.py`

```
word = "Cartesian"
print(word[3:6])    # tes
```



Пропущенные границы

Если `start` пропустить — срез начинается с самого начала строки. Если пропустить `stop` — срез идёт до самого конца. Пропустить можно и обе границы сразу — тогда получится копия всей строки:

```
propushchennye_granicy.py
```

```
word = "Cartesian"
print(word[:3])      # Car — то же самое, что word[0:3]
print(word[6:])      # ian — то же самое, что word[6:9]
print(word[:])       # Cartesian — копия строки целиком
```

Шаг среза

У среза есть и третий, необязательный параметр — **шаг**: строка[start:stop:step]. Шаг 2 значит «берём каждый второй символ»:

```
shag_sreza.py
```

```
word = "Cartesian"
print(word[::2])     # Cgaea — каждый второй символ, с
                     # начала до конца
```

Разворот строки: [::-1]

Отрицательный шаг проходит строку в обратном направлении. Если пропустить и `start`, и `stop`, а шаг поставить `-1` — получится строка задом наперёд:

```
razvorot_sreza.py
```

```
word = "Cartesian"
print(word[::-1])    # aisetraC
```

Самый короткий способ развернуть строку

строка[::-1] — классический приём Python. Третий параметр среза — «шаг»; шаг `-1` проходит строку с конца в начало. Мы воспользуемся этим в мини-проекте 8.20.

★★ Самостоятельная задача

Средние буквы через шаг

Для слова «программирование» получите срезом каждую третью букву (шаг 3), а затем — само слово, развёрнутое задом наперёд.

Практика: срезы строки

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/08-14/index.html>)

Строки нельзя изменить

Строку невозможно поменять «на месте» — но можно построить новую. Разберём, почему это так и как это влияет на все методы строк.

Строку нельзя изменить «на месте»

Может показаться, что раз мы умеем доставать символ по индексу (`word[0]`), то можно и **заменить** его тем же способом. Это не так — строки в Python **неизменяемы** (immutable):

```
попытка_изменит.py
```

```
word = "Cat"
word[0] = "B"
# TypeError: 'str' object does not support item
assignment
```

Почему так странно?

Это осознанное устройство языка, а не ограничение. Неизменяемые строки проще и безопаснее передавать между разными частями программы — можно быть уверенным, что никакая другая часть кода «незаметно» не поменяет ваш текст, пока вы с ним работаете. Мы ещё вернёмся к этой идее в главе про списки, которые, наоборот, изменяемы.

Правильный способ: создать новую строку

Вместо изменения «на месте» строим НОВУЮ строку — из кусочков старой — и сохраняем её в переменную (можно в ту же самую):

```
pravilnyj_sposob.py
```

```
word = "Cat"
word = "B" + word[1:]      # склеиваем "B" с "at" —
                             получаем новую строку
print(word)                # Bat
```

Именно поэтому все методы строк — `upper()`, `replace()` и другие (разделы 8.9–8.10) — не меняют исходную строку, а **возвращают новую**. Если результат нужно сохранить, его нужно явно присвоить переменной:

```
metody_ne_menyayut.py
```

```
text = "python"
text.upper()
# результат создан, но никуда не сохранён — и потерян!
print(text)           # python — исходная строка не
                        изменилась

text = text.upper()   # а вот так — сохранили
                        результат обратно в text
print(text)           # PYTHON
```

★ Базовая практика

Замена первой буквы

Для строки «home» постройте новую строку «dome», заменив первый символ через срез и конкатенацию — так, как показано выше для «Cat» → «Bat».

Практика: неизменяемость строк

Тот же ноутбук, что и в разделе «Срезы строки» — он охватывает и эту тему

Открыть практику → (<https://www.cartesianschool.org/practice/08-14/index.html>)

Методы строк — магия работы со строками!

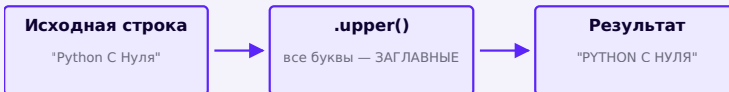
Готовые действия над строками, вызываемые через точку — от смены регистра до чистки пробелов.

У каждой строки в Python есть встроенные **методы** — готовые действия, которые можно выполнить над ней через точку: `строка.метод()`. Как мы выяснили в разделе 8.8, все они возвращают **НОВУЮ** строку, не трогая исходную.

Регистр букв: upper, lower, title, capitalize, swapcase

registr.py

```
text = "Python с нуля"
print(text.upper())      # PYTHON С НУЛЯ
print(text.lower())      # python с нуля
print(text.title())      # Python С Нуля — первая
                          # буква каждого слова заглавная
print(text.capitalize()) # Python с нуля — заглавная
                          # только у самого первого слова
print(text.swapcase())   # pYTHON С НУЛЯ — регистр
                          # каждой буквы наоборот
```



`upper()` возвращает НОВУЮ строку (раздел 8.8) — исходная переменная не меняется

Лишние пробелы: strip, lstrip, rstrip

Текст, введенный пользователем или взятый из файла, часто содержит случайные пробелы по краям. `strip()` убирает их с обеих сторон, `lstrip()` — только слева (left), `rstrip()` — только справа (right):

probely.py

```
text = "   Python   "
print(repr(text.strip()))      # 'Python'
print(repr(text.lstrip()))     # 'Python   '
print(repr(text.rstrip()))     # '   Python'
```



`strip()` убирает пробелы только по краям — не внутри строки

repr() снова пригодился

Мы используем `repr()` из раздела 8.2, чтобы увидеть пробелы по краям строки — обычный `print()` их «съедает» визуально, и не всегда понятно, остались ли они.

★★ Самостоятельная задача

Чистим и оформляем имя

Дана строка « ада лавлейс » (с лишними пробелами и строчными буквами). Одной цепочкой методов приведите её к виду «Ада Лавлейс» — уберите пробелы по краям и примените нужный метод регистра.

Практика: методы регистра и пробелов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/08-04/index.html>)

Методы строк: поиск и разбор

Заменяем подстроки, разбиваем строку на части и снова склеиваем — а ещё ищем подстроки внутри текста.

replace() — замена подстроки

replace.py

```
text = "я люблю Java"
print(text.replace("Java", "Python"))  # я люблю
Python
```



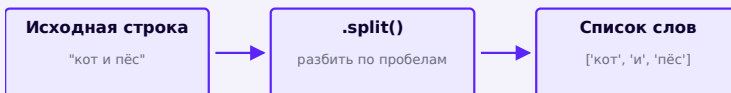
`text.replace("Java", "Python")` — заменяет ВСЕ вхождения подстроки

split() — разбить строку на части

`split()` без аргументов разбивает строку по пробелам (и группам пробелов) на список отдельных слов:

split.py

```
sentence = "кот и пёс"
words = sentence.split()
print(words)  # ['кот', 'и', 'пёс']
print(len(words))  # 3 слова
```



`split()` превращает строку в список слов (со списками подробно познакомимся в главе 10)

Можно указать свой разделитель — например, запятую:

`split_razdelitel.py`

```

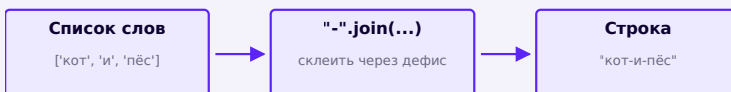
csv_row = "Ада,28,программист"
fields = csv_row.split(",")
print(fields)           # ['Ада', '28', 'программист']
  
```

`join()` — обратная операция: склеить список в строку

`join.py`

```

words = ["кот", "и", "пёс"]
result = "-".join(words)
print(result)           # КОТ-И-ПЁС
  
```



`join()` — действие, обратное `split()`: склеивает список обратно в строку

count() — сколько раз встречается

count.py

```
text = "МИССИСИПИ"
print(text.count("с"))      # 4
print(text.count("си"))    # 2
```

find() и index() — где встречается

Оба метода ищут подстроку и возвращают индекс её первого символа. Разница — в поведении, когда подстрока не найдена:

find_i_index.py

```
text = "Python"
print(text.find("th"))      # 2 — нашли, начинается с
                             индекса 2
print(text.find("zz"))      # -1 — не нашли, find()
                             возвращает -1

print(text.index("th"))     # 2 — то же самое, что
                             find(), если найдено
# text.index("zz")          # ValueError: substring not
                             found — index() «падает» с ошибкой!
```

Когда что выбрать

Если подстроки может не оказаться в тексте — используйте `find()` и проверяйте результат на `-1`. Если вы уверены, что подстрока обязана быть (и хотите узнать сразу, если это не так), — `index()` сообщит об ошибке явно, а не тихо вернёт `-1`.

startswith() и endswith()

startswith_endswith.py

```
filename = "otchet_2026.pdf"
print(filename.startswith("otchet")) # True
print(filename.endswith(".pdf"))     # True
print(filename.endswith(".docx"))    # False
```

★★ Самостоятельная задача

Разбираем имя файла

Для строки «report_final_v2.pdf» проверьте `endswith(".pdf")`, найдите позицию символа «_» методом `find()`, и через `split("_")` получите список частей имени.

Практика: поиск и разбор строк

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/08-16/index.html\)](https://www.cartesianschool.org/practice/08-16/index.html)

Методы проверки: isalpha и другие

Методы, которые отвечают True/False о том, из чего состоит строка — незаменимы перед преобразованием текста в число.

Методы, которые отвечают True или False

У строк есть целая группа методов, которые проверяют, ИЗ ЧЕГО состоит строка, и отвечают True или False. Они особенно полезны для проверки ввода пользователя (раздел 8.15) — до того, как пытаться преобразовать текст в число.

```
metody_proverki.py
```

```
print("Python".isalpha())      # True – только буквы
print("Python3".isalpha())     # False – есть цифра

print("2026".isdigit())        # True – только цифры
print("28 лет".isdigit())      # False – есть пробел и
                                буквы

print("Python3".isalnum())     # True – только буквы и/
                                или цифры
print("Python 3".isalnum())    # False – есть пробел

print(" ".isspace())           # True – только
                                пробельные символы
print("").isspace()            # False – пустая строка
                                не считается
```

Метод	Проверяет
isalpha()	только буквы (любого алфавита)
isdigit()	только цифры 0–9
isalnum()	только буквы и/или цифры
isspace()	только пробелы/табуляции/переносы

isnumeric() и isdecimal() — родственники isdigit()

Это более редкие варианты той же идеи:

isdecimal() — самый строгий (только обычные цифры 0-9), **isdigit()** — чуть шире (включает и некоторые специальные цифровые символы), **isnumeric()** — самый широкий (понимает и текстовые числительные вроде римских цифр в некоторых системах письма). Для учебных задач и проверки обычного пользовательского ввода **isdigit()** достаточно почти всегда.

Практическое применение: проверка перед преобразованием

```
proverka_pered_int.py
```

```
age_text = "28"
if age_text.isdigit():
    age = int(age_text)
    print(f"Через 5 лет: {age + 5}")
else:
    print("Это не похоже на число")
```

Полный оператор `if` подробно разберём в главе 9 — но уже сейчас видно, как `isdigit()` помогает не дать программе упасть с ошибкой при попытке `int()` от нечислового текста.

★ Базовая практика

Что за строка?

Для трёх строк «Python3», «2026», « » выведите результат `isalpha()`, `isdigit()` и `isspace()` для каждой — и убедитесь, что понимаете, почему получился именно такой результат.

Практика: методы проверки строки

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/08-17/index.html>)

in, сравнение строк и истинность

Проверяем, входит ли одна строка в другую, сравниваем строки друг с другом — и выясняем, когда строка ведёт себя как «ложь».

in и not in — проверка вхождения

Оператор `in` проверяет, содержится ли одна строка внутри другой — очень частая и полезная проверка:

```
in_i_not_in.py
```

```
print("thon" in "Python")      # True – "thon"  
                               встречается внутри "Python"  
print("java" in "Python")      # False  
print("java" not in "Python")  # True – обратная  
                               проверка
```

Сравнение строк

Строки можно сравнивать друг с другом оператором `==` — результатом всегда будет `True` или `False`:

```
sравнение_strok.py
```

```
print("Python" == "Python")    # True – строки  
                               совпадают полностью  
print("Python" == "python")    # False – регистр  
                               важен!
```

== сравнивает значение, is сравнивает объект

Для строк почти всегда нужен именно `==`. Оператор `is` проверяет, один ли это объект в памяти, — гораздо более редкий и специфичный случай, к которому мы вернёмся в главе 14.

Пустая строка — это «ложь»

В логических проверках (полноценный `if` — в главе 9) пустая строка ведёт себя как `False`, а любая непустая строка — как `True`:

pustaya_stroka_logika.py

```
print(bool(""))           # False
print(bool("Python"))     # True
print(bool(" "))          # True — пробел — это тоже
СИМВОЛ!
```

★ Базовая практика

Проверка домена

Для адреса «support@cartesianschool.org» проверьте через `in`, содержится ли в нём «@» и содержится ли «.ru».

Практика: `in`, сравнение и истинность строк

Тот же ноутбук, что и в разделе «Индексы строки» — он охватывает и эту тему

Открыть практику → (<https://www.cartesianschool.org/practice/08-03/index.html>)

Перебираем строку в цикле

Строка — последовательность символов, по которой можно пройти один за другим — первое знакомство с идеей цикла.

Забегает вперёд

Мы ещё не проходили циклы подробно — это тема главы 9. Здесь достаточно понимать `for ch in text:` буквально: «повтори блок кода для каждого символа `text` по очереди, каждый раз кладя очередной символ в переменную `ch`». Полное устройство цикла `for` разберём позже.

Строка — это последовательность, по которой можно пройти

Раз строка — это набор символов по порядку (раздел 8.1), по ней можно **перебирать** — то есть заглянуть в каждый символ по очереди:

```
perebor_stroki.py
```

```
word = "Python"
for ch in word:
    print(ch)
# P
# y
# t
# h
# o
# n
```

Практический пример: считаем гласные

```
schitaem_glasnye.py
```

```
text = "программирование"
glasnye = "aeёиоуыёя"
count = 0
for ch in text:
    if ch in glasnye:
        count += 1
print(f"Гласных букв: {count}")
# Гласных букв: 7
```

Здесь мы уже заглянули немного вперёд и на `if` (глава 9), но сама идея «пройтись по каждому символу и что-то с ним сделать» — это именно перебор строки, ключевая мысль этого раздела.

★ Базовая практика

Считаем конкретную букву

Переберите строку «миссисипи» в цикле `for` и посчитайте, сколько раз встречается буква «и» — вручную, без использования `count()` из раздела 8.10.

Практика: перебор строки в цикле

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/08-18/index.html>)

Форматирование строк

f-строки — главный современный инструмент вставки значений в текст. Разберём их по-настоящему полно: от простой подстановки до выравнивания в столбик.

Мы уже пользовались f-строками начиная с главы 4 — теперь разберём форматирование строк по-настоящему полно.

f-строка — простая подстановка

Перед открывающей кавычкой ставится буква `f`, а внутри фигурных скобок `{}` — имя переменной. Python сам подставит туда значение:

```
f_stroka_prostaya.py
```

```
name = "Ада"
print(f"Привет, {name}!")
# Привет, Ада!
```

Внутри `{}` можно писать выражения

Это не просто подстановка имени — внутри скобок работает ЛЮБОЕ Python-выражение: арифметика, вызов метода, что угодно:

f_stroka_vyrazheniya.py

```
age = 5
print(f"Через год будет {age + 1}.")
# Через год будет 6.

name = "cartesian"
print(f"Заглавными: {name.upper()}")           #
Заглавными: CARTESIAN
```

Форматирование чисел: точность и разделители

После значения можно поставить двоеточие и **спецификатор формата** — как именно отобразить число:

format_specifikatory.py

```
pi = 3.14159265
print(f"Пи округлённо: {pi:.2f}")    # Пи округлённо:
3.14 — 2 знака после запятой
print(f"{1234567:,}")                # 1,234,567 —
разделитель тысяч
print(f"{0.4567:.1%}")
# 45.7% — доля как проценты
```

`.2f` читается так: `f` — число с плавающей точкой, `.2` — округлить до 2 знаков после запятой.

Выравнивание и ширина

Число после двоеточия (без точки) задаёт минимальную ширину поля — удобно для выравнивания текста в столбик:

```
vyravniwanie.py
```

```
for name, score in [("Ада", 98), ("Алан", 87),
                    ("Грейс", 100)]:
    print(f"{name:<8} {score:>5}")
# Ада          98
# Алан         87
# Грейс        100
```

Спецификатор	Значение
<code>{value:<10}</code>	по левому краю, ширина поля 10
<code>{value:>10}</code>	по правому краю, ширина поля 10
<code>{value:^10}</code>	по центру, ширина поля 10
<code>{value:.2f}</code>	дробное число, 2 знака после запятой
<code>{value:,}</code>	разделитель тысяч запятой

Отладочный приём: `f"{{value=}}"`

Если внутри f-строки после выражения поставить знак `=` — Python выведет и само выражение, и его значение. Отлично подходит для быстрой проверки: `print(f"{{age=}}")` выведет `age=28` — без ручного набора текста «`age =`».

Форматирование строк: % → .format() → f-строки

Классический подход (%) и .format())

```
name = "Cartesian"
age = 5
```

```
# оператор % — самый
старый способ
print("Привет, %s! Тебе
%d лет." % (name, age))
```

```
# .format() — более
новый, но всё ещё
многословный
print("Привет, {}! Тебе
{} лет.".format(name,
age))
```

Современный Python 3.14 (f-строки)

```
name = "Cartesian"
age = 5
```

```
# f-строка — значения
прямо внутри {}
print(f"Привет, {name}!
Тебе {age} лет.")
```

```
# работают и выражения,
не только переменные:
print(f"Через год будет
{age + 1}.")
```

Что использовать сегодня: f-строки — они появились в Python 3.6 и с тех пор являются стандартом. Оператор % — самый старый способ, доставшийся от языка Си, всё ещё встречается в старом коде. .format() — промежуточный шаг между ними. f-строки читаются лучше всего: значение видно прямо там, где оно будет подставлено, и можно писать любые выражения, а не только имена переменных.

★★ Самостоятельная задача

Чек «на столе»

Выведите строку «Итого: 1234.5 → 1 234.50 Р» (используя :.2f) для суммы 1234.5, и отдельно — «Скидка: 15.0%» для доли 0.15 (используя :.1%).

Практика: %, .format() и f-строки

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/08-06/index.html) → (https://www.cartesianschool.org/practice/08-06/index.html)

Получение ввода от пользователей

Начало автоматизации: программа, которая спрашивает — и реагирует на ответ.

До сих пор все данные в программах были «защиты» прямо в код. Пора научиться получать данные от самого пользователя во время выполнения программы — с этого момента ваши программы становятся по-настоящему интерактивными.

`vvod.py`

```
name = input("Как вас зовут? ")
print(f"Привет, {name}!")
```

```
# Диалог в терминале:
# Как вас зовут? Ада
# Привет, Ада!
```

`input()` останавливает программу и ждёт, пока пользователь наберёт текст и нажмёт Enter — то, что он ввёл, возвращается как обычная строка.

`input()` в браузерной практике — по-настоящему интерактивный

Наш браузерный ноутбук практики умеет по-настоящему ждать ваш ответ на `input()` — прямо как обычный `.py`-файл, запущенный на компьютере: программа останавливается, показывает подсказку и ждёт, пока вы наберёте текст.

Простое правило: `input` → строка → преобразовать, если нужно число

`input()` **всегда** возвращает строку — даже если пользователь ввёл число. Чтобы посчитать что-то с этим значением, его нужно преобразовать (как мы делали в главе 4):

vvod_chisla.py

```
age_text = input("Сколько вам лет? ")
print(type(age_text))      # <class 'str'> — даже если
                             ввели "28"

age = int(age_text)
print(f"Через 5 лет вам будет {age + 5}.")
```

Частая ошибка новичков

Если забыть про `int()` и написать `age + 5`, где `age` — строка из `input()`, Python выдаст `TypeError`: складывать строку и число напрямую нельзя.

Ещё несколько диалогов

dialogi.py

```
city = input("В каком городе вы живёте? ")
print(f"{city} — отличный город!")

height_text = input("Ваш рост в см? ")
height = float(height_text)
print(f"Ваш рост в метрах: {height / 100:.2f}")
```

★ Базовая практика**Приветствие с проверкой**

Спросите имя через `input()`, а затем проверьте (через `isalpha()` из раздела 8.11), состоит ли оно только из букв — выведите разный текст в зависимости от результата.

Практика: `input()` и преобразование типов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/08-07/index.html) (<https://www.cartesianschool.org/practice/08-07/index.html>)

Кириллица, юникод и эмодзи

Строки Python одинаково хорошо понимают любой язык мира — и даже эмодзи.

Python дружит с любым языком

Строки Python умеют хранить символы ЛЮБОГО языка мира — кириллицу, латиницу, иероглифы, даже эмодзи — благодаря системе кодирования **Юникод** (Unicode), встроенной в Python «из коробки». Не нужно ничего специально настраивать:

```
unikod_primery.py
```

```
russskij = "Привет, мир!"
english = "Hello, world!"
smajlik = "Python — это    и    "
print(russskij)
print(english)
print(smajlik)
```

Длина строки с эмодзи

```
dlina_s_emodzi.py
```

```
text = "код    "
print(len(text))    # 6 — "к", "о", "д", " ", " " — но
                    пробел + эмодзи считаются отдельно
```

Достаточно знать на этом уровне

Полная теория кодировок (UTF-8, байты, таблицы символов) — тема для будущих, более продвинутых глав. Сейчас важно только одно: в Python 3 кириллица, латиница и эмодзи работают одинаково хорошо и без специальной настройки — можно свободно писать код и тексты программ на русском.

Всё, что мы уже умеем, работает и здесь

metody_na_kirillice.py

```
text = "Привет, Мир"
print(text.upper())      # ПРИВЕТ, МИР
print(text.lower())      # привет, мир
print(text[::-1])        # риМ ,тевиРП
```

★ Базовая практика

Эмодзи-приветствие

Соберите строку с помощью конкатенации из трёх частей: «Добро пожаловать» + пробел + эмодзи по вашему выбору, и выведите её длину через `len()`.

Практика: строки на разных языках

Тот же ноутбук, что и в разделе «Что такое строки?» — он охватывает и эту тему

Открыть практику → (<https://www.cartesianschool.org/practice/08-01/index.html>)

Отладка проблем со строками

Строки часто выглядят правильными на глаз, а ломаются из-за одного невидимого символа. Учимся находить и чинить такие ошибки.

Строки — источник особенно коварных ошибок: они часто выглядят правильно на глаз, а ломаются из-за одного невидимого символа. Разберём самые частые ситуации.

Невидимые пробелы

nevidimye_probely.py

```
password = "секрет "      # случайный пробел в конце!
print(password == "секрет") # False – а на глаз не
                             # отличить
print(repr(password))     # 'секрет ' – repr()
                             # выдаёт пробел
```

repr() — ваш главный инструмент

Мы уже видели `repr()` в разделе 8.2 — он показывает строку «как есть», включая пробелы и служебные символы, которые `print()` визуальнo скрывает.

Забытая кавычка

zabytaya_kavychka.py

```
# text = "Привет, мир!    # забыли закрывающую кавычку
# SyntaxError: unterminated string literal
```

Неправильное экранирование

nepravilnoe_ekranirovanie.py

```
# path = "C:\new_folder"  # \n здесь превратится в
                             # перенос строки!
print("C:\new_folder")
# C:
# ew_folder – совсем не то, что хотели

print(r"C:\new_folder")   # C:\new_folder – raw-
                             # строка (раздел 8.3) решает проблему
```

Склеивание строки с числом

stroka_plyus_chislo.py

```
score = 95
# print("Результат: " + score) # TypeError
print("Результат: " + str(score)) # исправлено –
явное преобразование
print(f"Результат: {score}")      # ещё лучше – f-
строка сама преобразует
```

find() vs index() при отсутствии подстроки

find_vs_index_oshibka.py

```
text = "Python"
print(text.find("java"))
# -1 – тихо возвращает -1, легко пропустить в коде
# text.index("java") # ValueError – а вот
index() сразу сообщает об ошибке
```

Частая ошибка — забыть, что `find()` при отсутствии подстроки возвращает `-1`, а не `False` или `None`. Проверка вроде `if text.find("java"):` работает неверно, потому что `-1` — это истина в логической проверке! Правильно — сравнивать явно: `if text.find("java") != -1:`.

input() «на самом деле» не число

input_ne_chislo.py

```
age = input("Возраст? ") # даже если ввели "28" –
это строка "28"
# print(age + 1) # TypeError – забыли int()
print(int(age) + 1) # правильно
```

Итоговая шпаргалка отладки строк

Симптом	Причина	Как исправить
Сравнение <code>==</code> неожиданно <code>False</code>	невидимый пробел или другой регистр	<code>repr()</code> , <code>.strip()</code> , <code>.lower()</code>
<code>SyntaxError: unterminated string</code>	забыта закрывающая кавычка	проверить парность кавычек
Текст «сломался», появился неожиданный перенос строки	случайный <code>\n</code> в пути/тексте	raw-строка <code>r"..."</code>
<code>TypeError: can only concatenate str</code>	сложение строки с числом через <code>+</code>	<code>str()</code> или f-строка
<code>if text.find(...)</code> ведёт себя странно	<code>-1</code> — это истина в булевой проверке	сравнивать <code>!= -1</code> явно
<code>TypeError</code> при арифметике с <code>input()</code>	<code>input()</code> всегда возвращает строку	<code>int()</code> / <code>float()</code>

★★ Самостоятельная задача

Найдите ошибку

В строке ``total = "Сумма: " + 100`` есть ошибка. Определите её тип, объясните причину и перепишите строку двумя разными способами, чтобы она работала.

Практика: отладка строковых ошибок

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/08-20/index.html) (<https://www.cartesianschool.org/practice/08-20/index.html>)

Мини-проект — выводим текст Turtle на новый уровень!

Объединяем `input()` из раздела 8.15 с `write()` из главы 7 — персональное приветствие прямо на холсте.

Соединим строки, ввод от пользователя и Turtle из глав 6–7: спросим имя и выведем персональное приветствие прямо на холсте.

`turtle_tekst_imya.py`

```
import turtle

screen = turtle.Screen()
artist = turtle.Turtle()
artist.hideturtle()

name = input("Как вас зовут? ")

artist.penup()
artist.goto(0, 0)
artist.write(f"Привет, {name}!", align="center",
font=("Arial", 24, "bold"))

screen.exitonclick()
```

★★ Самостоятельная задача**Приветствие по времени суток**

Спросите у пользователя число (час дня) через `input()` + `int()`, и выведите разный текст в зависимости от значения — используя пока лишь то, что уже знаете о строках (полноценный `if` будет в главе 9, здесь достаточно вывести один текст с подставленным часом).

Практика: ввод имени (логика, без окна Turtle)

Тот же ноутбук, что и в разделе «Ввод от пользователя» — он охватывает и эту тему

Открыть практику → (<https://www.cartesianschool.org/practice/08-07/index.html>)

Мини-проект — приветствие и форматирование ФИО

Собираем f-строки, методы регистра и `strip()` в двух практичных мини-проектах.

Мини-проект — генератор приветствий

Собираем время суток и имя в одно вежливое приветствие — используя f-строки (раздел 8.14) и методы регистра (раздел 8.9):


```
generator_privetstvij.py
```

```
daytime = input("Сейчас утро, день, вечер или ночь? ")
name = input("Как вас зовут? ")

privetstviya = {
    "утро": "Доброе утро",
    "день": "Добрый день",
    "вечер": "Добрый вечер",
    "ночь": "Доброй ночи",
}
privetstvie =
privetstviya.get(daytime.lower().strip(),
"Здравствуйте")
print(f"{privetstvie}, {name.strip().title()}!")
# Доброе утро, Ада!
```

Забегая вперёд

Здесь мы уже используем словарь `{...}` — структуру «ключ → значение», которую подробно разберём в главе 11. Сейчас достаточно понимать `.get(ключ, значение_по_умолчанию)` : «найди ключ, а если его нет — верни запасной вариант».

Мини-проект — форматировщик ФИО

Частая практическая задача: привести введённое имя и фамилию к аккуратному виду независимо от того, как их ввёл пользователь — капсом, строчными, с лишними пробелами:

formatirovshik_fio.py

```

raw_first = input("Имя: ")
raw_last = input("Фамилия: ")

first = raw_first.strip().capitalize()
last = raw_last.strip().capitalize()
full_name = f"{first} {last}"
initials = f"{first[0]}. {last[0]}."

print(f"Полное имя: {full_name}")
print(f"Инициалы: {initials}")
# ввод: " ада " / "ЛАВЛЕЙС"
# Полное имя: Ада Лавлейс
# Инициалы: А. Л.

```

★★ Самостоятельная задача

Электронная подпись

Расширьте формировщик ФИО: добавьте третий `input()` для профессии, и соберите e-mail-подпись вида «Ада Лавлейс, программист» одной f-строкой.

Практика: генератор приветствий и ФИО

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/08-21/index.html>)

Мини-проект — кричим на экран

Два коротких, но показательных мини-проекта на методы и срезы строк.

Мини-проект — кричим на экран

Простая, но забавная программа: превращает любую фразу в «крик» — капсом и с кучей восклицательных знаков.

krik.py

```
phrase = input("Что вы хотите прокричать? ")
krik = phrase.upper() + "!!!"
print(krik)
```

Мини-проект — переворачиваем своё имя

Используем срез `[::-1]` из раздела 8.7, чтобы развернуть введённое имя задом наперёд:

perevorot_imeni.py

```
name = input("Как вас зовут? ")
perevernutoe = name[::-1]
print(f"Ваше имя задом наперёд: {perevernutoe}")
```

★ Базовая практика

Крик с ограничением

Измените `krik.py` так, чтобы восклицательных знаков было столько же, сколько букв во фразе (подсказка: умножение строки на число — `str * n` — повторяет её `n` раз, раздел 8.4).

★★ Самостоятельная задача

Палиндром?

Допишите `perevorot_imeni.py` так, чтобы он сообщал, является ли введённое слово палиндромом — читается ли оно одинаково в обе стороны (сравните `name` с перевёрнутой версией).

Практика: крик и переворот имени

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/08-09/index.html) (<https://www.cartesianschool.org/practice/08-09/index.html>)

Мини-проект — проверка пароля и e-mail

Собираем методы проверки строк в две практические программы валидации ввода.

Мини-проект — простая проверка пароля

Соберём несколько правил (длина, наличие цифры, наличие буквы) в одну проверку — используя методы проверки из раздела 8.11:

proverka_parolya.py

```
password = input("Придумайте пароль: ")

dostatochno_dlinnyj = len(password) >= 8
est_cifra = any(ch.isdigit() for ch in password)
est_bukva = any(ch.isalpha() for ch in password)

print(f"Длина от 8 символов: {dostatochno_dlinnyj}")
print(f"Есть цифра: {est_cifra}")
print(f"Есть буква: {est_bukva}")

nadyozhnyj = dostatochno_dlinnyj and est_cifra and est_bukva
print(f"Пароль надёжный: {nadyozhnyj}")
```

Забегаям вперёд

`any(...)` с выражением внутри — это компактная форма цикла из раздела 8.13: «есть ли хотя бы один символ, для которого условие верно». Подробно операторы `and` / `or` и такие компактные конструкции разберём в главах 9–10.

Мини-проект — простая проверка e-mail

Настоящая проверка email — сложная задача (для неё существуют специальные библиотеки), но базовые «здравые» проверки уже можно сделать тем, что мы знаем: `in` (раздел 8.12), `count()` и `find()` (раздел 8.10):

```
proverka_email.py
```

```
email = input("Ваш e-mail: ").strip()

est_sobachka = "@" in email
odna_sobachka = email.count("@") == 1
est_tochka_posle = "." in email[email.find("@):]

pohozhe_na_email = est_sobachka and odna_sobachka and
est_tochka_posle
print(f"Похоже на e-mail: {pohozhe_na_email}")
# ввод: "ada@cartesianschool.org"
# Похоже на e-mail: True
```

★★ Самостоятельная задача

Своё правило

Добавьте к проверке пароля ещё одно правило: пароль не должен содержать пробелов (используйте `isspace()` или `in " "`). Проверьте программу на пароле с пробелом и без.

Практика: проверка пароля и e-mail

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/08-22/index.html) (<https://www.cartesianschool.org/practice/08-22/index.html>)

Мини-проект — счётчик слов и чистка предложения

Считаем слова в тексте и учимся автоматически заменять нежелательные слова.

Мини-проект — счётчик слов

Считаем, сколько слов в тексте и сколько раз встречается каждое — с помощью `split()` (раздел 8.10) и подсчёта в цикле (раздел 8.13):

```
schetchik_slov.py
```

```
text = input("Введите предложение: ")
words = text.lower().split()

print(f"Всего слов: {len(words)}")

schetchik = {}
for word in words:
    schetchik[word] = schetchik.get(word, 0) + 1

for word, count in schetchik.items():
    print(f"{word}: {count}")
# Ввод: "кот и пёс и кот"
# Всего слов: 5
# кот: 2
# и: 2
# пёс: 1
```

Забегая вперёд

Словарь `schetchik` — структура «слово → сколько раз встретилось», подробно разберём в главе 11. Сейчас достаточно понимать: `.get(word, 0)` возвращает текущий счётчик слова (или 0, если слово встретилось впервые), а мы прибавляем к нему единицу.

Мини-проект — чистка предложения (текстовый цензор)

Заменяем «запрещённые» слова звёздочками той же длины — используя `replace()` (раздел 8.10) и повторение строки (раздел 8.4):

`censor.py`

```

text = input("Введите текст: ")
zapreshchennye = ["плохое_слово", "секрет"]

ochishchennyj = text
for word in zapreshchennye:
    zamen = "*" * len(word)
    ochishchennyj = ochishchennyj.replace(word,
    zamen)

print(ochishchennyj)
# ввод: "это секрет, никому не говори"
# это *****, никому не говори

```

★★ Самостоятельная задача**Самое частое слово**

Расширьте счётчик слов: найдите слово с максимальным count и выведите его отдельной строкой «Самое частое слово: ...». Подсказка — переберите словарь `schetchik.items()` и запоминайте текущего лидера.

Практика: счётчик слов и цензор текста

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/08-23/index.html) (<https://www.cartesianschool.org/practice/08-23/index.html>)

Мини-проект — красочная и динамическая математика

Собираем главу воедино: ввод пользователя, числа, строки и Turtle — в одном интерактивном мини-калькуляторе.

Финальный мини-проект главы объединяет практически всё: ввод от пользователя, числа из глав 4–5, строки и Turtle из глав 6–7 — маленький «калькулятор с характером».

```
dinamicheskaya_matematika.py
```

```
import turtle

screen = turtle.Screen()
artist = turtle.Turtle()
artist.hideturtle()

a = float(input("Первое число: "))
b = float(input("Второе число: "))

artist.penup()
artist.goto(0, 60)
artist.pencolor("purple")
artist.write(f"{a} + {b} = {a + b}", align="center",
font=("Arial", 18, "bold"))

artist.goto(0, 0)
artist.pencolor("blue")
artist.write(f"{a} * {b} = {a * b}", align="center",
font=("Arial", 18, "bold"))

screen.exitonclick()
```

★★★ Задача повышенной сложности

Ещё две операции

Добавьте на холст ещё две строки: результат вычитания и деления — своим цветом для каждой.

Практика: динамическая математика (логика, без окна Turtle)

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/08-10/index.html>)

Итоги

Что мы узнали в этой главе

- Строки создают в одинарных, двойных или тройных кавычках; тройные сохраняют переносы строк, raw-строки (`r"..."`) отключают экранирование.
- Служебные последовательности вроде `\n` и `\t` вставляют символы, у которых нет своей клавиши; `repr()` показывает строку «как в коде».
- Символы строки доступны по индексу с нуля и по отрицательному индексу с конца — первый символ всегда имеет индекс 0.
- Срез `[start:stop:step]` достаёт часть строки: `start` включён, `stop` — нет; `[::-1]` разворачивает строку.
- Строки неизменяемы: методы вроде `upper()` возвращают НОВУЮ строку, не трогая исходную.
- У строк десятки готовых методов — от регистра (`upper()` , `strip()`) до поиска и разбора (`split()` , `replace()` , `find()`) и проверки состава (`isdigit()` , `isalpha()`).
- `in` проверяет вхождение подстроки; строку можно перебрать циклом `for ch in text`.
- f-строки — современный способ форматирования, с поддержкой выражений, точности и выравнивания; `%` и `.format()` — более старые, но всё ещё встречаются в реальном коде.
- `input()` всегда возвращает строку — для чисел нужно явное преобразование `int()` / `float()`.
- Python одинаково хорошо работает с кириллицей, латиницей и эмодзи без специальной настройки.

Выполняй мою команду!

До сих пор наши программы в основном выполняли команды одну за другой. Теперь мы научим их выбирать, какую команду выполнить дальше. Разберём алгоритмы и поток управления, как условия создают развилки и как Python принимает решения с помощью True, False и if/elif/else — и соберём первую настоящую игру.

9.1 Алгоритмы и команды

9.2 Три структуры алгоритма и ветвление

9.3 Истина или ложь

9.4 Сравниваем и принимаем решение

9.5 Чем отличаются = и ==, и как сравнивать строки

9.6 Цепочки сравнений

9.7 Truthiness и None

9.8 Если это произошло — выполни команду!

9.9 elif — несколько вариантов

9.10 Несколько if $\neq$ if/elif/else

9.11 and — все условия сразу

9.12 or — хотя бы одно

9.13 not — переворачиваем условие

9.14 Больше одного условия!

9.15 Short-circuit: Python иногда ленится

9.16 in / not in как условие

9.17 is против ==

9.18 Вложенные условия

9.19 Проектируем условие

9.20 Отладка логических ошибок

9.21 Мини-проект — «Угадай число»

9.22 Мини-проекты — клуб и погода

9.23 Мини-проекты — оценки и команды

9.24 Условия накапливаются и итоги

Итоги

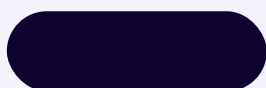
Алгоритмы и команды

Прежде чем учить программу выбирать между вариантами, разберёмся, что вообще значит «выполнять команды» — и откуда берётся сам термин «алгоритм».

До сих пор наши программы выполняли одни и те же команды в одном и том же порядке при каждом запуске. Прежде чем научить программу **выбирать**, что делать дальше, разберёмся, что вообще значит «выполнять команды» — и договоримся об одном важном инструменте: языке блок-схем.

Форма блока — не украшение

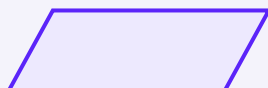
Каждая фигура на блок-схеме имеет строгий смысл: она подсказывает, какого рода шаг алгоритма перед вами. Мы будем пользоваться этим языком на протяжении всей главы:



Начало / конец
терминатор



Действие
процесс / команда



Ввод / вывод
параллелограмм



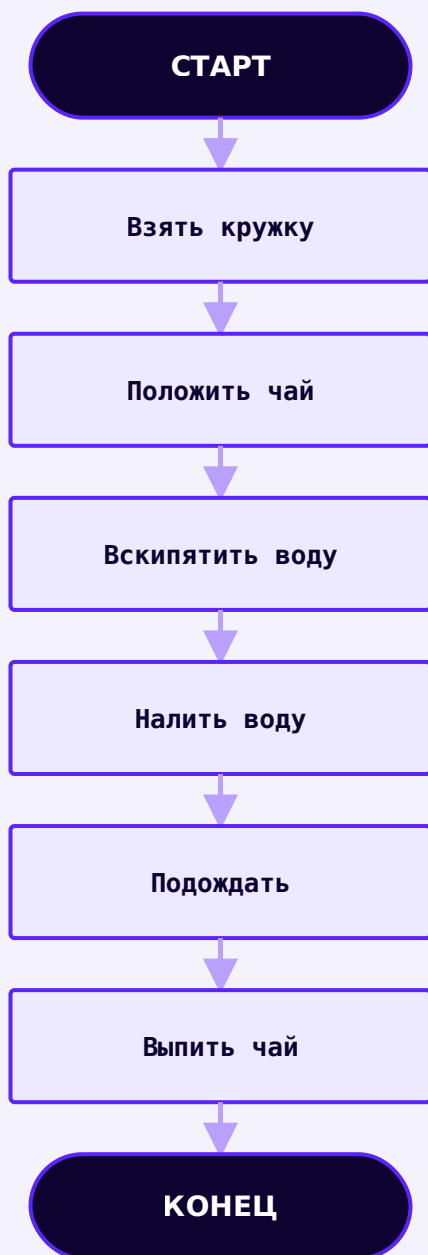
Условие?
решение (ромб)

Что такое алгоритм

Алгоритм — это понятная последовательность действий, которая приводит нас от исходной ситуации к нужному результату. Мы пользуемся алгоритмами каждый день, даже не называя их так:

АЛГОРИТМ: приготовить чай

1. Взять кружку.
2. Положить чай.
3. Вскипятить воду.
4. Налить воду.
5. Подождать.
6. Выпить чай.



Алгоритм «приготовить чай» — все шесть действий, от старта до результата

Каждый отдельный шаг — это **команда** (инструкция): указание компьютеру (или человеку) выполнить конкретное действие. Весь упорядоченный набор шагов — это и есть алгоритм.

Команда и выражение — это разные вещи

Мы уже пользовались командами с самой первой программы в главе 1 — только не называли их так:

```
znakomye_komandy.py
```

```
print("Привет")

name = input("Как вас зовут? ")

score = 10
```

- `print(...)` — команда «показать что-то на экране».
- `input(...)` — команда «запросить информацию у пользователя».
- `score = 10` — команда «связать имя с значением» (присваивание).

2 + 2 — это выражение, а не команда

`2 + 2` само по себе ничего не «делает» — это **выражение**: оно вычисляется и производит значение (4). А вот `print(2 + 2)` — уже команда: она берёт значение выражения и что-то с ним делает (показывает на экране). Формальное различие пока не обязательно запоминать дословно — но не стоит путать «вычислить значение» и «выполнить действие».

Программа — это последовательность команд

Рассмотрим простую программу и проследим, как Python выполняет её команда за командой. Обратите внимание: `input()` нарисован как ВВОД, а `print()` — как ВЫВОД, разными фигурами, а не одинаковыми прямоугольниками:

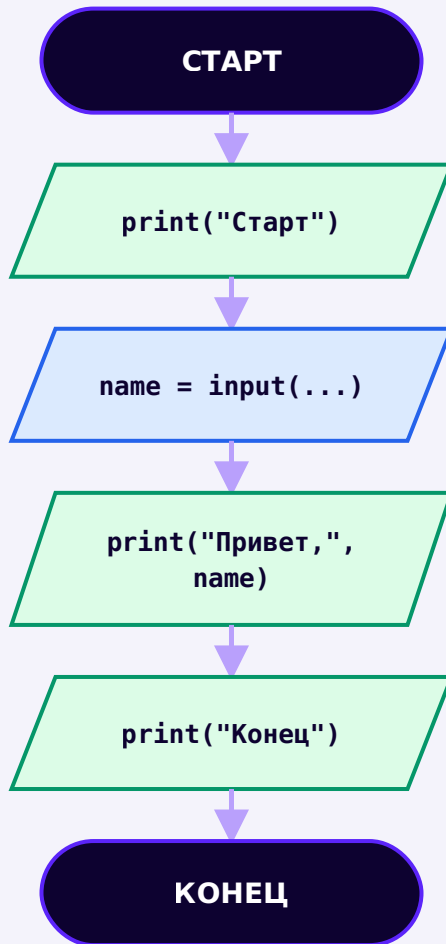
```
posledovatel'naya_programma.py
```

```
print("Старт")

name = input("Как вас зовут? ")

print("Привет,", name)

print("Конец")
```



Тот же код как блок-схема — ввод и вывод нарисованы по-разному, не как одинаковые прямоугольники

Обычно Python выполняет команды строго **сверху вниз**, одну за другой. Это называется **последовательным** (или линейным) выполнением: каждый следующий шаг идёт сразу после предыдущего, без пропусков и без выбора.

Практика: алгоритмы, команды и последовательное выполнение

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/09-07/index.html) → (https://www.cartesianschool.org/practice/09-07/index.html)

Три структуры алгоритма и ветвление

Любой алгоритм собирается из трёх идей: последовательность, ветвление, повторение. Эта глава — про вторую из них.

Три структуры, из которых строится любой алгоритм

Почти любой алгоритм — от рецепта чая до огромной программы — собирается всего из трёх базовых идей:

Какая структура нужна?

Команды
идут
одна за
другой,
без
выбора?

→ **Последовательность** — уже знакомо

Нужно
выбрать
ОДИН путь
из
нескольких,
в
зависимости
от условия?

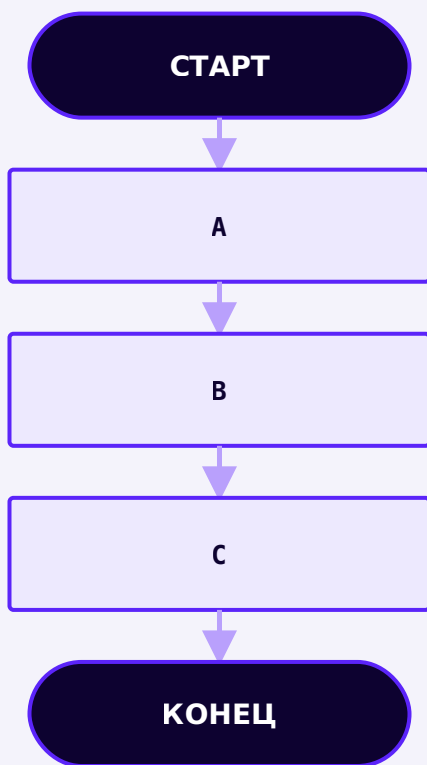
→ **Ветвление – эта глава**

Нужно
повторить
действие
несколько
раз?

→ **Повторение – глава 10**

Последовательность

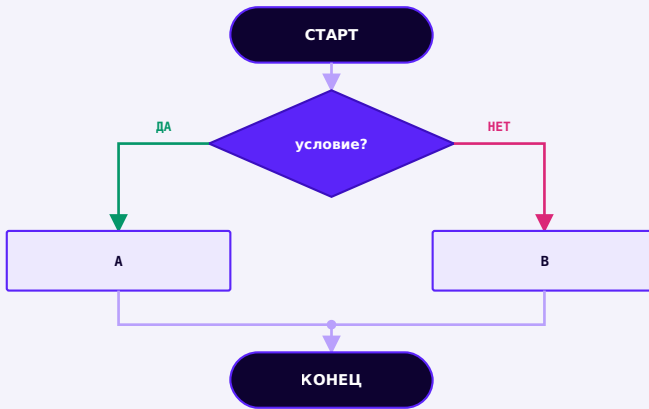
Команды выполняются одна за другой, без выбора и возврата назад.



Последовательность — каждый шаг идёт сразу после предыдущего

Ветвление

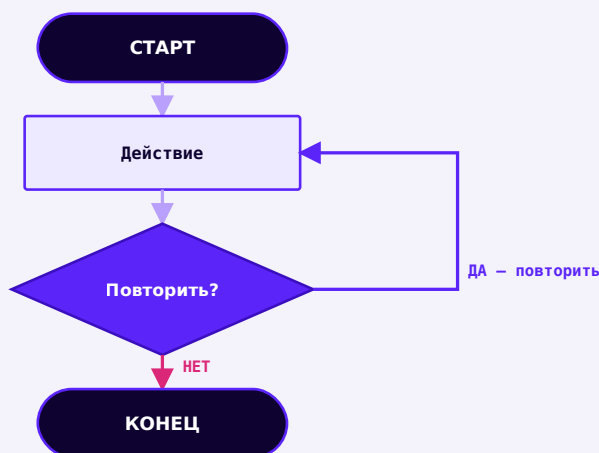
Логический вопрос выбирает один из двух путей; после выполнения ветки пути сходятся.



Ветвление — выбирается ровно один путь, оба варианта затем сходятся

Повторение

После действия логический вопрос решает: повторить действие или завершить цикл.



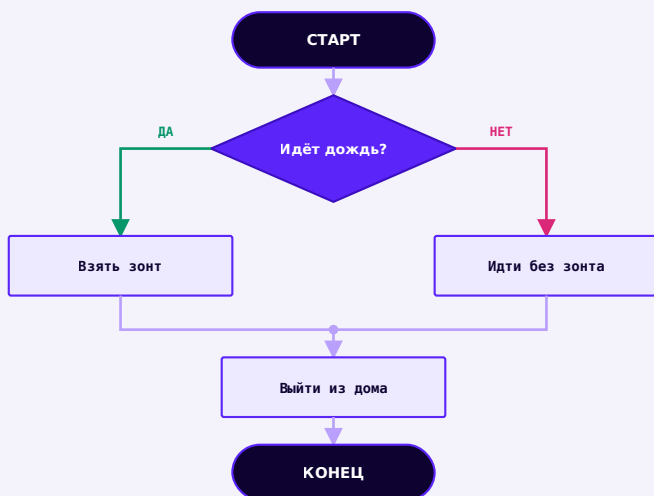
Повторение — предварительная схема, синтаксис циклов изучим в главе 10

Когда одной последовательности не хватает

Простой линейный алгоритм — «выйти из дома, идти гулять» — прекрасно работает, пока в реальной жизни не появляется вопрос. Идёт дождь?

Что такое ветвление

Ветвление — это место в алгоритме, где дальнейшие действия зависят от ответа на вопрос. Каждая развилка начинается с **условия** — вопроса, на который можно ответить «да» или «нет». Программа не выбирает ветку случайно: она вычисляет условие, и результат определяет путь. Какую бы ветку ни выбрала программа, дальше выполнение обычно продолжается с одного и того же следующего шага:



Ветвление, а затем продолжение: обе ветки сходятся и алгоритм идёт дальше

Это уже не чисто последовательный алгоритм — он **ветвится**, а затем обе ветки снова сходятся к шагу «Выйти из дома».

От «да/нет» к True/False

Человеческие «да» и «нет» в Python превращаются в два особых значения:

Человек	Python
да	True
нет	False

Именно поэтому следующий раздел — про `True` и `False` — теперь не абстрактная тема «из ниоткуда», а прямое продолжение идеи ветвления.

Практика: три структуры алгоритма и ветвление

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/09-08/index.html\)](https://www.cartesianschool.org/practice/09-08/index.html)

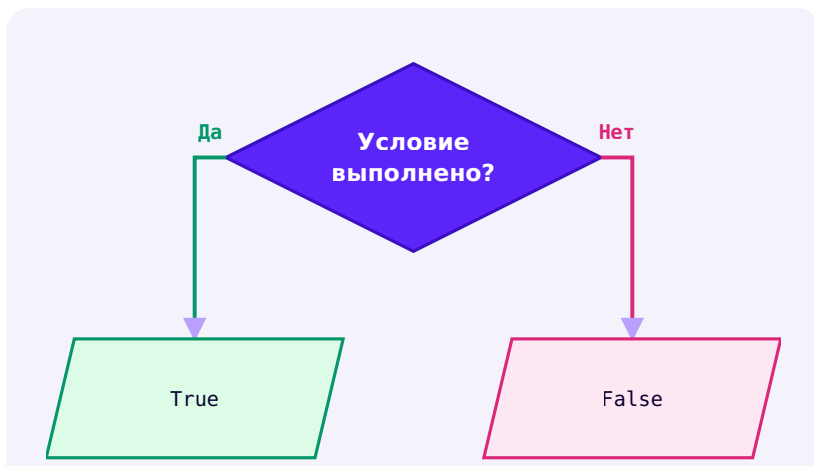
Истина или ложь

Прежде чем программа сможет принимать решения, ей нужен способ хранить сам результат решения — логический тип `bool`.

Мы выяснили: программа отвечает на вопросы «да» или «нет» значениями `True` и `False`. Пора познакомиться с типом данных, который их хранит.

`bool` — логический тип с двумя значениями

Тип `bool` имеет ровно два возможных значения:



Логический вопрос всегда даёт один из двух результатов: True или False

Обратите внимание: True и False пишутся с заглавной буквы — это ключевые слова Python, а не обычный текст.

bool.py

```
is_sunny = True
is_raining = False
print(is_sunny, type(is_sunny))
# True <class 'bool'>
```

bool — это не строка!

Легко перепутать True (значение типа bool) со строкой "True" — но это совершенно разные вещи:

bool_ne_stroka.py

```
print(type(True))      # <class 'bool'>
print(type("True"))    # <class 'str'>
print(True == "True")  # False — разные типы, разные значения
```

"True" — это просто текст из четырёх букв

Строка "True" не имеет никакого особого смысла для Python — это обычный непустой текст, который, как мы увидим дальше, при проверке истинности сам ведёт себя как True, но это два разных явления, которые легко перепутать.

Как чаще всего получают bool

Вручную писать True / False приходится редко — обычно их получают в результате сравнения (следующий раздел):

```
sravnenie_bool.py
```

```
print(5 > 3)      # True  
print(5 == 3)     # False
```

Практика: тип bool и True/False

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

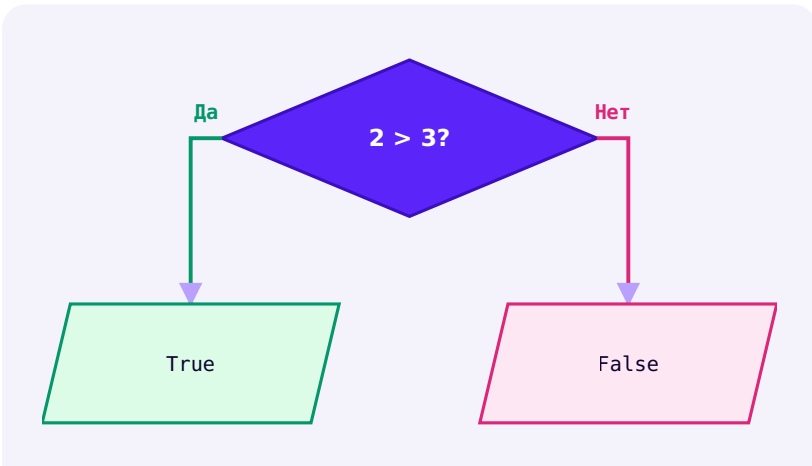
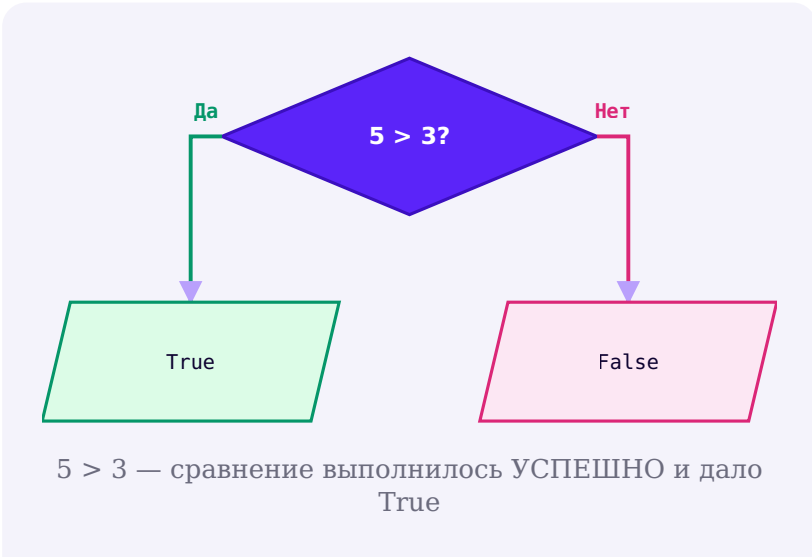
Открыть практику → ([https://www.cartesianschool.org/
practice/09-01/index.html](https://www.cartesianschool.org/practice/09-01/index.html))

Сравниваем и принимаем решение

Шесть операторов, которые превращают два значения в один bool — и числовая прямая, чтобы увидеть их границы.

Сравнение — это шаг алгоритма, который производит bool

Возьмите два значения, примените оператор сравнения — и получите ровно одно из двух: `True` или `False`. Оба исхода — **одинаково успешный** результат: сравнение не «падает» и не «не срабатывает», когда ответ `False` — оно просто честно отвечает «нет».



$2 > 3$ — сравнение ТОЖЕ выполнилось успешно,
просто дало False

sravnenie.py

```
print(5 > 3)      # True
print(5 == 3)     # False
```

False — тоже правильный ответ

$2 > 3$ выполняется так же успешно, как $5 > 3$ — просто их результаты разные. Не путайте «условие ложно» с «программа не сработала»: это два совершенно разных явления, и мы вернёмся к этому различию в разделе про `if`.

Все шесть операторов сравнения

Вопрос	Математика	Python	Пример	Результат
равно?	=	==	<code>5 == 5</code>	True
не равно?	≠	!=	<code>5 != 3</code>	True
больше?	>	>	<code>5 > 3</code>	True
меньше?	<	<	<code>5 < 3</code>	False
больше или равно?	≥	>=	<code>5 >= 5</code>	True

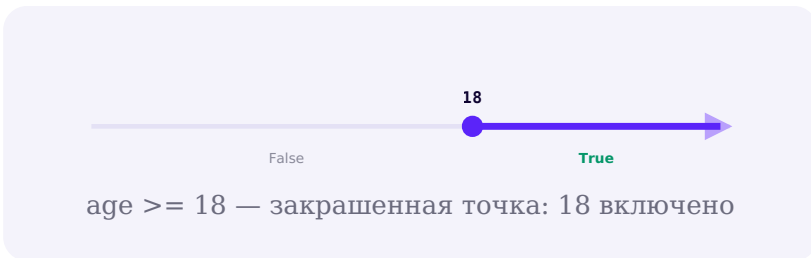
Вопрос	Математика	Python	Пример	Результат
меньше или равно?	$\leq$	<code><=</code>	<code>5 <= 3</code>	False

operatory_sravneniya.py

```
print(5 == 5)    # равно
print(5 != 3)    # не равно
print(5 > 3)     # больше
print(5 < 3)     # меньше
print(5 >= 5)    # больше или равно
print(5 <= 3)    # меньше или равно
```

Числовая прямая — увидеть сравнение

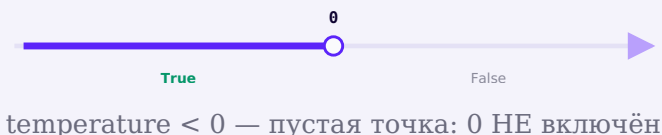
`age >= 18` означает «age больше ИЛИ равно 18». Гораздо нагляднее увидеть это на числовой прямой:



Слева от 18 — область, где условие ложно; сама точка 18 и всё, что правее, — область, где условие истинно. Точка на 18 **закрашена**: значит, 18 входит в истинную область (`>=` включает границу).

Открытая и закрытая граница

Сравните с `temperature < 0`:



Здесь точка на 0 **пустая** (не закрашена) — граница НЕ включена: значение 0 не удовлетворяет условию `< 0`. Условимся так на протяжении всей главы:

- **закрашенная точка** — граница включена (`>=`, `<=`)
- **пустая точка** — граница исключена (`>`, `<`)

Практика: шесть операторов сравнения и границы

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-02/index.html>)

Чем отличаются `=` и `==`, и как сравнивать строки

Одна из самых частых ошибок новичков — и как строки из главы 8 участвуют в условиях.

`=` — это не то же самое, что `==`

Этот раздел заслуживает отдельного внимания — путаница между одиночным и двойным «равно» встречается постоянно.

Два разных оператора: = и ==

= (один знак) —
ПРИСВАИВАНИЕ

```
name = "Anna"
```

```
# СМЫСЛ: СВЯЗАТЬ ИМЯ
# name
# со значением "Anna".
# Ничего не
# сравнивается!
```

== (два знака) —
СРАВНЕНИЕ

```
name == "Anna"
```

```
# СМЫСЛ: РАВНЫ ЛИ
# значения?
# результат — True или
# False,
# сама переменная не
# меняется
```

Что использовать сегодня: Перепутать `=` и `==` — одна из самых частых ошибок у новичков в любом языке программирования. Если в условии `if` случайно написать одиночный `=`, Python сразу сообщит об ошибке синтаксиса — присваивание нельзя использовать как условие, так что эта конкретная ошибка, к счастью, не проходит незамеченной.

!= — «не равно»

`!=` — оператор, обратный `==`:

```
ne_ravno.py
```

```
password = ""
print(password != "") # False — пароль как раз
# пустой

password = "secret123"
print(password != "") # True — пароль НЕ пустой
```


Почему != , а не ≠

В математике «не равно» пишут одним значком — $\neq$. На клавиатуре такого значка нет, и Python (как и C, Java, JavaScript и почти все языки программирования) не понимает его как код — писать нужно ровно два обычных символа подряд: восклицательный знак и знак равенства, != . Это два разных, но эквивалентных обозначения одной и той же идеи: $\neq$ — запись для человека на бумаге, != — запись для Python. Если в коде на этом сайте != выглядит как единый перечёркнутый значок — это лишь особенность шрифта редактора (лигатура), которая красиво *отображает* два символа рядом; печатать при этом всё равно нужно ! и = по отдельности — отдельного «значка $\neq$ » на клавиатуре искать не нужно, его там нет.

Сравнение строк

Мы видели в главе 8: строки сравниваются по символам, и регистр важен:

```
sравнение_strok.py
```

```
print("Python" == "python")    # False – регистр важен
print("cat" < "dog")           # True
```

«cat» < «dog» — не совсем «по алфавиту»

Python сравнивает строки по **кодовым позициям символов**, а не по «человеческому» алфавитному порядку словаря. Для обычных латинских букв в одном регистре результат обычно совпадает с интуицией, но при смешении регистров или разных алфавитов это уже не всегда «алфавитный порядок» в привычном смысле.

ЧУТЬ ГЛУБЖЕ — код символа

Каждому символу соответствует число — его код. Функция `ord()` показывает его: `ord("A") → 65`, `ord("a") → 97`. Именно эти числа Python на самом деле сравнивает. Для этой главы знать точные числа не обязательно — важно лишь понимать, что сравнение строк работает через них, а не через словарь.

Нормализация перед сравнением

Пользователь может ввести «Да», «ДА», « да » — с разным регистром и пробелами. Методы строк из главы 8 решают эту проблему одной строкой:

```
normalizaciya.py
```

```
answer = input("Продолжить? ").strip().lower()

if answer == "да":
    print("Продолжаем")
```

`.strip()` убирает случайные пробелы по краям,
`.lower()` приводит к нижнему регистру — теперь
 неважно, как именно пользователь набрал ответ.

Практика: == против =, != и сравнение строк

Интерактивный ноутбук прямо в браузере — Python 3.14
 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-09/index.html>)

Цепочки сравнений

Как проверить, что значение лежит в диапазоне, — одной изящной строкой вместо двух отдельных сравнений.

Диапазон значений — цепочка сравнений

Частая задача: проверить, что значение находится МЕЖДУ двумя границами. В Python это можно записать одной изящной цепочкой:

серочка.py

```
age = 30
print(18 <= age <= 65)    # True
```



18 <= age <= 65 — включённый диапазон

Это ровно то же самое, что:

серочка_razvernuto.py

```
age = 30
print(age >= 18 and age <= 65)    # тот же результат
```

но цепочка `18 <= age <= 65` читается ближе к тому, как мы формулируем это на человеческом языке — «age между 18 и 65».

Ещё примеры

primery_срочек.py

```
print(0 <= score <= 100)      # score от 0 до 100
                               # включительно
print(-10 < temperature < 30) # температура строго
                               # между -10 и 30
print(1 <= month <= 12)      # month — допустимый
                               # номер месяца
```



Обратите внимание: здесь обе границы **открытые** (строгие `<`) — значения -10 и 30 сами НЕ входят в диапазон, что видно по пустым точкам на прямой.

★ Базовая практика

Допустимый месяц

Даны значения `month = 0`, `month = 1`, `month = 12`, `month = 13`. Для каждого предскажите результат `1 <= month <= 12`, затем проверьте в коде.

Практика: цепочки сравнений

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-10/index.html>)

Truthiness и None

Условию не всегда нужно явное сравнение — Python умеет спросить о любом значении, истинно оно или ложно.

Условию не всегда нужно явное сравнение

Мы уже видели в главе 8: пустая строка ведёт себя как `False`, а непустая — как `True`. Это работает не только для строк — Python умеет спросить о ЛЮБОМ значении: «ведёшь ли ты себя как истина или как ложь?» Это называется **truthiness** («истинностью») значения.

Falsy — значения, которые ведут себя как False

Значение	<code>bool(...)</code>	Почему
<code>False</code>	<code>False</code>	уже ложь
<code>None</code>	<code>False</code>	«здесь нет значения»
<code>0</code> , <code>0.0</code>	<code>False</code>	ноль считается ложью
<code>""</code>	<code>False</code>	пустая строка
<code>[]</code> , <code>{}</code>	<code>False</code>	пустые коллекции — подробно изучим позже

Truthy — почти всё остальное

Значение	bool(...)	Почему
True	True	уже истина
-10, 1, 42	True	любое ненулевое число
"нет", "0", "False"	True	любая непустая строка — даже такая!

truthiness.py

```
print(bool(0))      # False
print(bool(1))      # True
print(bool(-10))    # True — отрицательное тоже не
                    # ноль!
print(bool(""))      # False
print(bool("False")) # True — это непустая СТРОКА, а
                    # не значение False
print(bool("0"))     # True — тоже непустая строка
```

Текст "False" — это не значение False!

`bool("False")` равно `True`, потому что строка `"False"` состоит из пяти символов — она непустая, а значит `truthy`. То же самое с `"0"`: это текст из одного символа, а не число ноль.

None — «здесь нет значения»

`None` — особое значение, означающее примерно «здесь нет значения» (а не 0, не `False` и не пустая строка):

none.py

```
value = None

print(value is None)      # True
print(bool(value))        # False — None тоже falsy
print(value == 0)         # False — None это не 0
print(value == "")        # False — None это не
                          # пустая строка
```

Обратите внимание на оператор `is` вместо `==` — к разнице между ними мы вернёмся в разделе 9.17.

Практика: truthiness и None

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

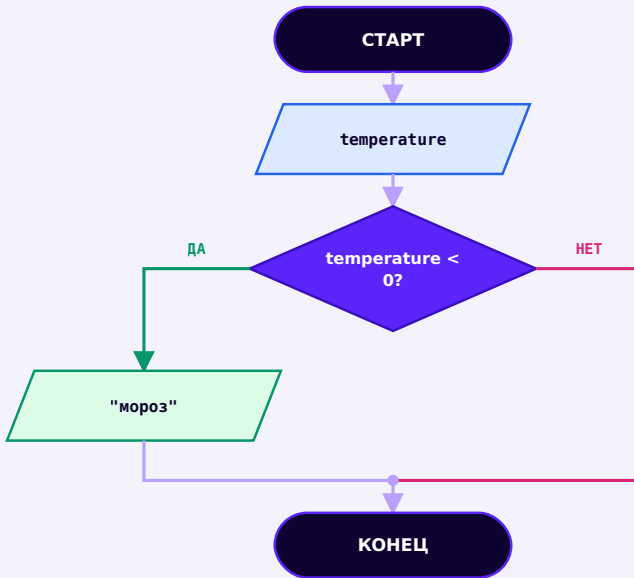
Открыть практику → (<https://www.cartesianschool.org/practice/09-11/index.html>)

Если это произошло — выполни команду!

Первый настоящий условный оператор: код, который выполняется только при определённых обстоятельствах — сначала как схема, потом на Python.

Сначала схема, потом код

Возьмём задачу: «если температура ниже нуля, показать „мороз“». Прежде чем писать Python, решим её как схему:



Сначала решаем задачу как схему — потом переводим на Python

Где здесь вопрос? Какая ветка выполнится? Что произойдёт, если условие ложно? Только когда ответы понятны — переводим схему на Python.

Первый if

```
if_prosto.py
```

```
temperature = -5

if temperature < 0:
    print("Мороз")
```


Разберём по словам:

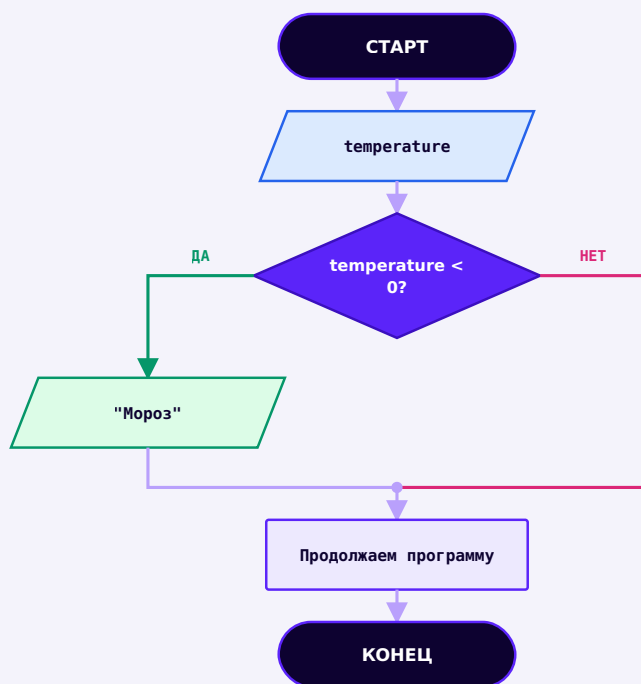
- `if` — «если»
- `temperature < 0` — условие (вопрос)
- `:` — здесь начинается блок команд
- строка с отступом — команда, принадлежащая этому блоку

Полный пример с обеими ветками

`if.py`

```
age = 20
```

```
if age >= 18:  
    print("Доступ разрешён.")
```



Первый if — при False блок просто пропускается, выполнение идёт дальше

Если условие ложно, блок `if` просто **пропускается** — программа не «падает», не выдаёт ошибку, а идёт дальше как ни в чём не бывало. Ложное условие не означает «программа остановилась»: она продолжает выполнение со следующего шага.

Отступ — это часть программы

В Python отступ — не просто оформление для красоты. Именно отступом Python определяет, какие строки относятся к блоку `if`, а какие уже нет:

`blok_komand.py`

```
if temperature < 0:
    print("На улице мороз.")
    print("Наденьте тёплую куртку.")
    print("Не забудьте перчатки.")

print("Программа закончена")
```

Первые три команды имеют одинаковый отступ — все они относятся к блоку `if`. Последняя строка отступа не имеет — она выполнится **всегда**, независимо от условия.

Рекомендация: 4 пробела

Стандартный отступ в Python — 4 пробела на один уровень вложенности. Не смешивайте пробелы и табуляцию в одном файле — Python в некоторых случаях сочтёт это ошибкой.

IndentationError

Если забыть про отступ — Python не сможет понять, что относится к блоку:

`indentationerror.py`

```
if age >= 18:
    print("OK")
# IndentationError: expected an indented block after
'if' statement
```

Исправление: добавить отступ перед `print`.

А иначе? — if/else

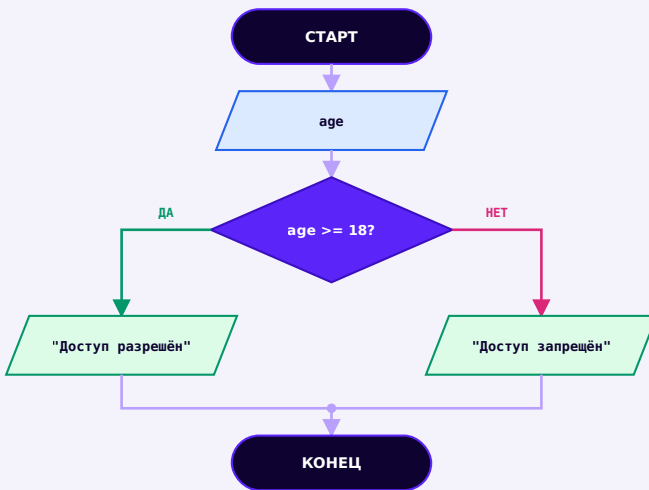
`else` задаёт блок кода, который выполнится, если условие оказалось ложным:

```
if_else.py
```

```
age = 15
```

```
if age >= 18:  
    print("Доступ разрешён.")
```

```
else:  
    print("Доступ запрещён — вам ещё нет 18.")
```



if/else: два пути — один из них выполнится обязательно, и оба сходятся дальше

Ветки снова сходятся

Какая бы ветка ни выполнялась — `if` или `else` — дальше программа обычно продолжается с одного и того же следующего шага. На схеме выше это видно по стрелкам, сходящимся в общую точку перед **КОНЕЦ**.

Практика: первый `if`, отступ и `if/else`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

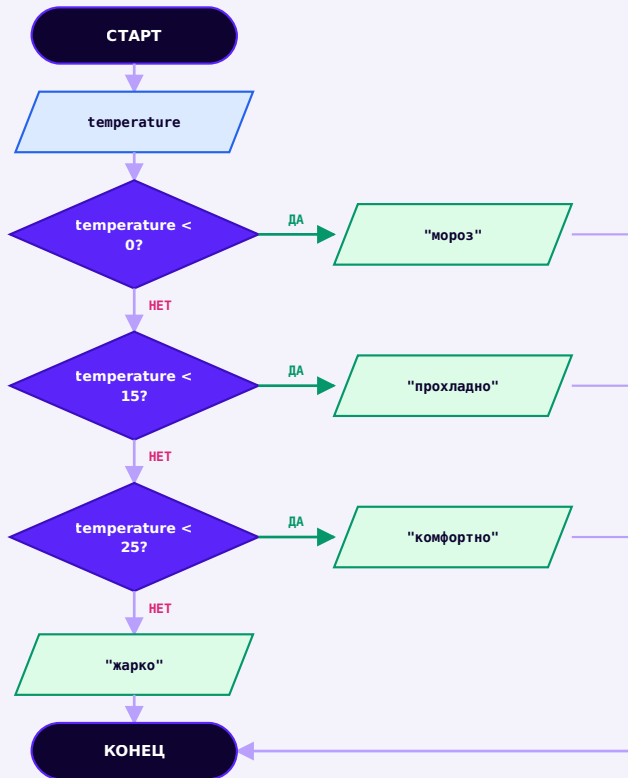
Открыть практику → (<https://www.cartesianschool.org/practice/09-03/index.html>)

`elif` — несколько вариантов

Когда исходов больше двух — `elif` проверяет условия по очереди, и первое истинное побеждает.

Когда исходов больше двух

Температура: ниже нуля — «мороз», от 0 до 14 — «прохладно», от 15 до 24 — «комфортно», иначе — «жарко». Четыре исхода, а не два — здесь `if/else` уже не хватает.



if/elif/else как лестница условий — первое истинное условие выигрывает, остальное пропускается

```
elif_цепочка.py
```

```
temperature = 10

if temperature < 0:
    print("мороз")
elif temperature < 15:
    print("прохладно")
elif temperature < 25:
    print("комфортно")
else:
    print("жарко")
```

`elif` — сокращение от *else if*, «а иначе, если». Он добавляет ещё одно условие между `if` и `else`.

Первое истинное условие побеждает

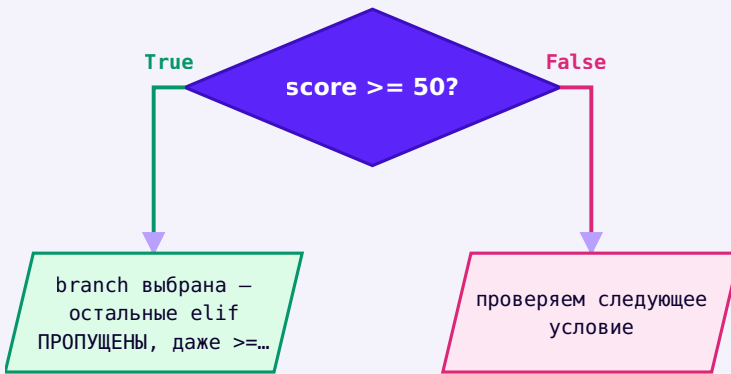
Python проверяет условия **по порядку, сверху вниз**, и останавливается на первом истинном — даже если следующие условия тоже подошли бы. Оставшаяся часть цепочки просто пропускается.

Порядок условий имеет значение

```
nepravilnyj_poryadok.py
```

```
score = 95

if score >= 50:
    буква = "D"
elif score >= 90:
    буква = "A"
# никогда не выполнится для score >= 50!
```



score = 95: если ≥ 50 стоит первым, программа никогда не дойдёт до ≥ 90

score = 95 попадёт в «D», а не в «A»

Раз `score >= 50` стоит первым и `95 >= 50` истинно, Python выбирает именно эту ветку и даже не проверяет `score >= 90`.

Исправление — расположить условия от более строгого к менее строгому:

pravilnyj_poryadok.py

```

score = 95

if score >= 90:
    буква = "A"
elif score >= 75:
    буква = "B"
elif score >= 50:
    буква = "C"
else:
    буква = "D"
  
```


★★ Самостоятельная задача

Три ступени скидки

Постройте elif-лестницу: сумма покупки ≥ 5000 — скидка 15%, ≥ 2000 — скидка 10%, ≥ 500 — скидка 5%, иначе — скидки нет. Проверьте на сумме 3000 — какая скидка выигрывает?

Практика: elif-лестница и порядок условий

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-12/index.html>)

Несколько if ≠ if/elif/else

Очень частая путаница у новичков: несколько отдельных if могут работать все разом, а в цепочке elif — только один.

Блок команд и невидимый указатель

Ветка условия может содержать несколько команд — мы это уже видели. Полезно представлять, что Python держит невидимый «указатель»: какую команду выполнить следующей. В линейной программе он просто идёт вниз строка за строкой; при if он выбирает, в какую ветку «зайти».

Это упрощённая мысленная модель

Внутри компьютера существует похожая идея (её называют «поток управления» — control flow), но для наших целей достаточно этой простой картинки: команды идут по порядку, а if/elif/else временно направляет поток в одну из веток.

Поток управления

Поток управления — это порядок, в котором на самом деле выполняются команды программы. До этой главы он был почти всегда прямой линией сверху вниз. Теперь он может **разветвляться**. В главе 10 он научится ещё и возвращаться назад — повторяться.

Несколько if — это НЕ то же самое, что if/elif/else

Это одна из самых частых путаниц у новичков — разберём её на конкретном примере:

Независимые if против цепочки if/elif

Два НЕЗАВИСИМЫХ if
temperature = 30

```
if temperature > 20:
    print("тепло")
```

```
if temperature > 25:
    print("жарко")
```

ОБА условия истинны —
ОБЕ строки
напечатаются!

Цепочка if / elif
temperature = 30

```
if temperature > 25:
    print("жарко")
elif temperature > 20:
    print("тепло")
```

только ОДНА ветка
в этой цепочке
выполнится

Что использовать сегодня: Каждый самостоятельный if проверяется независимо от остальных — Python может выполнить несколько таких блоков подряд. А в цепочке if/elif/else выполняется РОВНО ОДНА ветка — первая истинная. Это очень частая путаница у новичков: если нужен «выбор одного варианта из нескольких» — нужна именно цепочка elif, а не несколько отдельных if.

★★ Самостоятельная задача**Найдите разницу**

Возьмите `score = 85`. Постройте вариант с тремя независимыми `if` (`score > 50`, `score > 70`, `score > 90`) и вариант с `elif`-цепочкой на тех же условиях. Сравните, сколько строк напечатает каждый вариант.

Практика: независимые `if` против `elif`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/09-13/index.html) → (<https://www.cartesianschool.org/practice/09-13/index.html>)

and — все условия сразу

Иногда решение зависит не от одного, а сразу от всех перечисленных условий.

Все условия должны быть True

Реальная ситуация: вас пускают в зал, только если вам есть 18 лет И у вас есть билет. Оба условия обязательны:

Вход в зал разрешён только если ОБА условия истинны

`age >= 18?`

→ **нужно ДА**

`has_ticket?`

→ **нужно ДА тоже**

```
and_primer.py
```

```
age = 20
has_ticket = True

if age >= 18 and has_ticket:
    print("Проходите в зал.")
```

Таблица истинности and

A	B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

Результат `and` истинен, только когда истинны **оба** операнда — во всех остальных трёх случаях результат ложен.

Переведите на человеческий язык

Полезный навык — читать условие вслух как предложение:

```
chitaem_vsluh.py
```

```
age >= 18 and country == "PL"
# "age не меньше 18 И country равно PL"
```

★ Базовая практика

Своё условие с and

Напишите условие: `can_drive` — можно водить машину, если `age >= 18` И `has_license` равно `True`.
Проверьте на `age=20, has_license=False`.

Практика: and и таблица истинности

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/09-14/index.html\)](https://www.cartesianschool.org/practice/09-14/index.html)

or — хотя бы одно

Иногда для срабатывания достаточно, чтобы истинным было хотя бы одно из перечисленных условий.

Достаточно одного True

Скидка положена студентам ИЛИ пенсионерам — достаточно подходить хотя бы под одну категорию:

Скидка положена, если истинно ХОТЯ БЫ ОДНО условие

student?

→ достаточно ДА

senior?

→ или ДА здесь

```
or_primer.py
```

```
is_student = False
is_senior = True

if is_student or is_senior:
    print("Скидка применена.")
```

Таблица истинности or

A	B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

Результат `or` ложен только тогда, когда ложны **оба** операнда — во всех остальных случаях он истинен.

★ Базовая практика

Выходной или праздник

Напишите условие: `can_rest` — можно не работать, если `is_weekend` равно `True` ИЛИ `is_holiday` равно `True`. Проверьте на `is_weekend=False`, `is_holiday=True`.

Практика: or и таблица истинности

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-15/index.html>)

not — переворачиваем условие

Простейший логический оператор: превращает True в False и обратно.

Переворачиваем логическое значение

`not` — самый простой логический оператор: он просто переворачивает `True` в `False` и наоборот.

A	not A
True	False
False	True

```
not_primer.py
```

```
is_raining = False

if not is_raining:
    print("Можно гулять")
```

Читаем вслух: «если НЕ идёт дождь». Условие `not is_raining` истинно ровно тогда, когда `is_raining` ложно.

```
not_s_sravneniem.py
```

```
age = 15
print(not (age >= 18))
# True — потому что age >= 18 само по себе False
```

★ Базовая практика

Двойное not

Предскажите результат `not not True` — затем проверьте в коде. Объясните своими словами, почему получился именно такой ответ.

Практика: not и инверсия условий

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/09-16/index.html\)](https://www.cartesianschool.org/practice/09-16/index.html)

Больше одного условия! :O

`and`, `or` и `not` вместе — приоритет операторов и перевод человеческой фразы в Python-условие.

Мы разобрали `and`, `or` и `not` по отдельности — теперь соберём их вместе и разберём, как Python понимает более сложные условия.

```
logicheskie_operatory.py
```

```
age = 20
has_ticket = True

if age >= 18 and has_ticket:
    print("Проходите в зал.")

is_weekend = False
is_holiday = True
if is_weekend or is_holiday:
    print("Сегодня можно не ходить на работу.")

if not has_ticket:
    print("Сначала нужно купить билет.")
```


Приоритет операторов

Когда в одном условии встречаются `and` и `or` вместе, Python сначала вычисляет `not`, затем `and`, и только потом `or` — совсем как умножение выполняется раньше сложения в арифметике:

not выполняется первым

and выполняется вторым

or выполняется последним

Приоритет логических операторов — как $+$ / $-$ и $*$ / $÷$ в арифметике

prioritet.py

```
print(True or False and False)
# and выполнится первым: False and False -> False
# затем or: True or False -> True
```

Скобки помогают читать сложные условия

Не заставляйте читателя (и себя в будущем) запоминать таблицу приоритетов. Когда условий много, скобки делают порядок явным: `is_admin or (is_owner and is_active)` читается однозначно, в отличие от варианта без скобок.

Переводим человеческий язык в условие — и обратно

Человеческая фраза	Python-условие
«возраст не меньше 18»	<code>age >= 18</code>
«температура ниже нуля»	<code>temperature < 0</code>
«имя не пустое»	<code>name</code> (достаточно самого имени — <code>truthiness</code>)
«ответ — да или yes»	<code>answer in ("да", "yes")</code>
«совершеннолетний и с билетом»	<code>age >= 18 and has_ticket</code>

Это фундаментальный навык: научиться формулировать вопрос на человеческом языке, а затем перевести его в Python-условие — и наоборот, читать готовое условие как предложение.

Практика: **and, or, not** вместе и приоритет

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/09-04/index.html) (<https://www.cartesianschool.org/practice/09-04/index.html>)

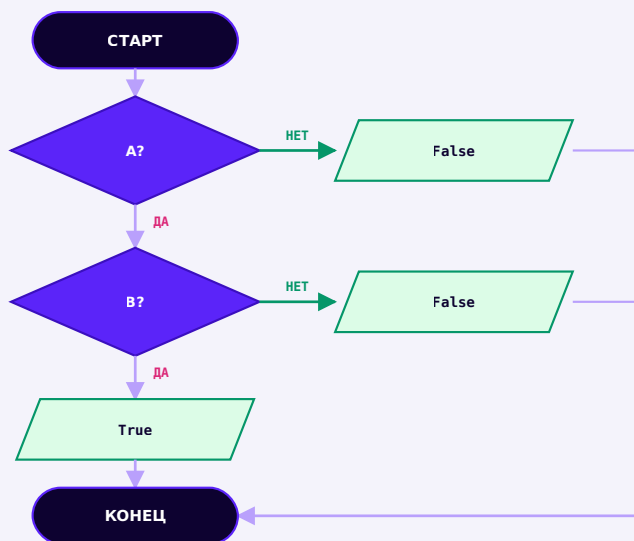
Short-circuit: Python иногда ленится

`and` и `or` не всегда вычисляют оба операнда — и это свойство можно использовать, чтобы защититься от ошибок.

Python иногда не проверяет всё условие

У `and` и `or` есть важное свойство — **short-circuit** («короткое замыкание»): Python вычисляет второй операнд, только если это действительно нужно для ответа.

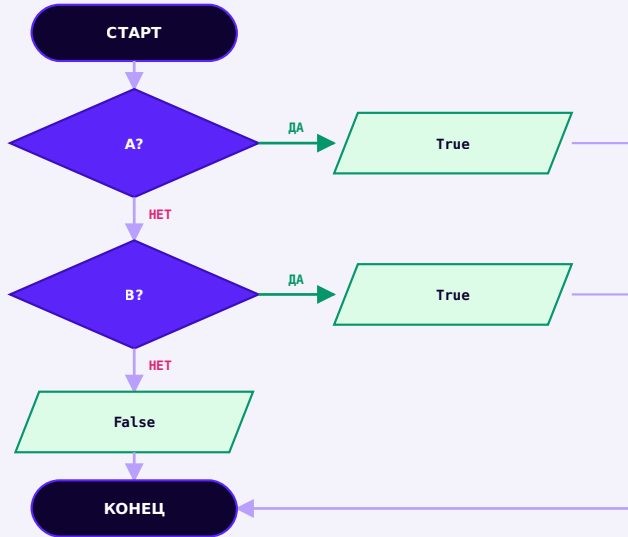
`and`: если `A` уже `False`



`and`: НЕТ на любом шаге сразу даёт `False` — второе условие проверяется, только если первое истинно

Если `A` ложно, весь `A and B` уже обязан быть ложным — независимо от `B`. Python это понимает и не тратит время на вычисление `B`.

or: если A уже True



or: ДА на любом шаге сразу даёт True — второе условие проверяется, только если первое ложно

Аналогично: если A истинно, весь A or B уже обязан быть истинным.

Реальная польза: защита от ошибки

Это не просто оптимизация скорости — иногда именно она спасает программу от падения:

```
short_circuit_zashchita.py
```

```
name = ""

if name and name[0] == "A":
    print("Имя начинается на A")
else:
    print("Условие не выполнено")
```

Если `name` — пустая строка, она сама по себе `falsy` (глава 8), поэтому `name and ...` уже ложно, и Python **даже не пытается** вычислить `name[0]`. Если бы порядок был обратным...

```
bez_zashchity.py
```

```
name = ""
print(name[0] == "A" and name)
# IndexError: string index out of range!
```

Порядок операндов имеет значение

`name and name[0] == "A"` — безопасно: сначала проверяется, что `name` вообще непустая. `name[0] == "A" and name` — опасно: Python попытается вычислить `name[0]` ещё до всякой проверки на пустоту, и получит `IndexError` для пустой строки (глава 8, раздел про индексы).

★★ Самостоятельная задача

Безопасная проверка последнего символа

Напишите условие, которое безопасно проверяет, заканчивается ли непустая строка `text` на символ `!` — так, чтобы для пустой строки не возникало `IndexError`.

Практика: short-circuit и защита от ошибок

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-17/index.html>)

in / not in как условие

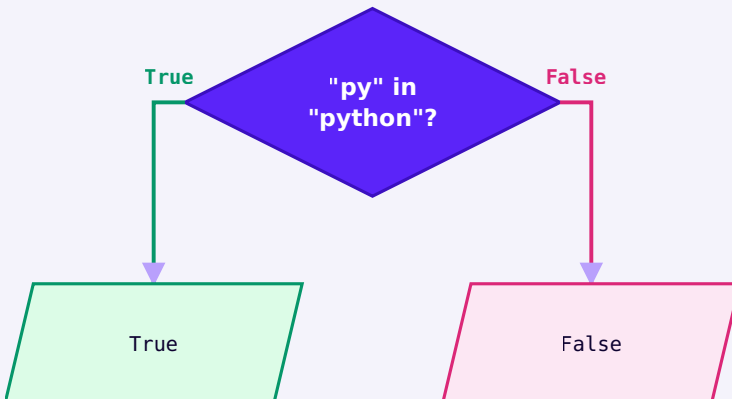
Проверка вхождения из главы 8 — теперь прямо внутри условий if.

Проверка вхождения как условие

Мы познакомились с `in / not in` в главе 8 — теперь используем их прямо в условиях `if`.

```
in_kak_uslovie.py
```

```
text = "python"  
print("py" in text)           # True  
print("java" not in text)     # True
```



`in` — вопрос «найдено ли значение внутри»

Группа допустимых ответов

Частая задача: проверить, что ответ пользователя входит в набор принятых вариантов. Пока мы не изучали списки подробно (это будет в главе 10-11), но уже можем использовать простую группу значений в скобках — **кортеж**:

```
gruppa_otvetov.py
```

```
answer = input("Продолжить? ").strip().lower()

if answer in ("да", "yes", "y"):
    print("Продолжаем")
elif answer in ("нет", "no", "n"):
    print("Останавливаемся")
else:
    print("Не понял ответ")
```

Что такое ("да", "yes", "y")

Это **кортеж** — группа значений в круглых скобках. `answer in (...)` проверяет: «совпадает ли `answer` хотя бы с одним из перечисленных значений?»

Списки и кортежи подробно разберём в следующих главах — сейчас достаточно уметь применять этот полезный приём.

Осторожно с `in` для строки-подстроки

"да" `in` "давай" тоже даёт `True` — потому что `in` для строки ищет ПОДСТРОКУ, а не точное совпадение целого слова. Для сравнения с набором вариантов используйте кортеж/список значений, как в примере выше, а не проверку «одна строка внутри другой».

Практика: in / not in в условиях

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-18/index.html>)

is против ==

Два похожих оператора спрашивают о совершенно разных вещах — и есть ровно один правильный повод использовать is прямо сейчас.

Два разных вопроса

`==` и `is` выглядят похоже, но спрашивают о совершенно разном:

==

«Равны ли ЗНАЧЕНИЯ?»

is

«Это ОДИН И ТОТ ЖЕ объект?»

Главное практическое применение — None

В подавляющем большинстве случаев для чисел, строк и вообще обычных сравнений нужен именно `==`.

Правильное и по-настоящему важное применение `is` на этом этапе — проверка на `None`:


```
is_none.py
```

```
value = None
```

```
print(value is None)      # True — правильный способ
print(value is not None)  # False
```

Почему не `value == None`

Технически `value == None` в большинстве случаев тоже работает — но `is None` считается более правильным стилем в Python: он однозначно спрашивает «это именно `None`?», а не «равно ли значение чему-то, что ведёт себя как `None`?». Договоримся всегда использовать `is None` / `is not None`.

Не используйте `is` для чисел и строк

`256 is 256` или `"a" is "a"` могут вести себя неожиданно — их результат зависит от внутренних деталей реализации Python, которые не гарантированы и могут отличаться в разных ситуациях. Это не то, чему стоит учиться сейчас: для сравнения значений всегда используйте `==`. Идея «сравнение по объекту» подробно вернётся в главе про ООП.

Практика: `is` против `==`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

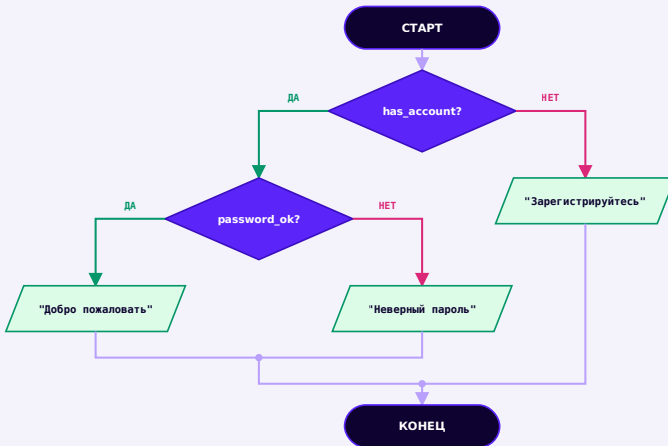
Открыть практику → (<https://www.cartesianschool.org/practice/09-19/index.html>)

Вложенные условия

Условие внутри условия — и когда его стоит упростить одним `and` вместо глубокой вложенности.

Условие внутри условия

Внутри блока `if` может быть ещё один `if` — это называется **вложенным условием**. Решим задачу входа в аккаунт целиком, одной схемой.



Вложенное условие: `password_ok` проверяется только внутри ветки `has_account = True`

`vlozhennyye_usloviya.py`

```

has_account = True
password_ok = False

if has_account:
    if password_ok:
        print("Добро пожаловать")
    else:
        print("Неверный пароль")
else:
    print("Зарегистрируйтесь")
  
```

Уровни отступа = уровни вложенности

Каждый дополнительный уровень отступа означает более глубокий уровень условной проверки:

```
urovni_otstupa.py
```

```
# уровень 0
if has_account:
    # уровень 1
    if password_ok:
        # уровень 2
        print("...")
```

Не увлекайтесь глубокой вложенностью

Три-четыре уровня вложенных if подряд («пирамида») читать становится тяжело. Часто часть условий можно объединить оператором `and` — как показано ниже.

Упрощаем: вложенность → and

Вложенные условия vs один and

Вложенные if

```
if has_account:
    if password_ok:
        print("Login")
```

Один if с and

```
if has_account and
password_ok:
    print("Login")
```

Что использовать сегодня: Оба варианта корректны и в этом простом случае дают одинаковый результат. Вложенный вариант удобен, когда каждой ветке нужна СВОЯ отдельная обработка (например, разные сообщения для «нет аккаунта» и «неверный пароль», как в примере ниже). Один `and` компактнее, когда обе ветки нужны только вместе — выбор между ними определяется читаемостью, а не «правильностью» одного варианта.

★★ Самостоятельная задача

Третий уровень

Добавьте третий уровень вложенности: внутри «Добро пожаловать» проверьте ещё и `is_active` (аккаунт не заблокирован) — если `False`, выведите «Аккаунт заблокирован».

Практика: вложенные условия

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/09-20/index.html\)](https://www.cartesianschool.org/practice/09-20/index.html)

Проектируем условие: ввод, валидация, границы

Повторяемый способ подходить к любому условию — от требования до протестированного граничного значения.

Валидация — это не то же самое, что решение

Прежде чем принимать решение по введённым данным, нужно проверить, что данные вообще осмысленны. Это два разных шага:

- **Валидация** — допустим ли ввод? (например, это вообще число?)
- **Решение** — что делать с допустимым вводом?

validaciya_i_reshenie.py

```
age_text = input("Возраст: ").strip()

if age_text.isdigit():           # 1. валидация
    age = int(age_text)
    if age >= 18:                 # 2. решение
        print("Доступ разрешён")
    else:
        print("Доступ запрещён")
else:
    print("Это не похоже на число")
```

Полноценная обработка ошибок — впереди

Здесь мы проверяем ввод вручную через `isdigit()` (глава 8). Настоящий механизм обработки исключений — `try/except` — придёт в отдельной главе позже; пока достаточно этого простого подхода.

Границы — источник большинства ошибок

«Разрешены значения от 1 до 10 включительно» — как записать это правильно?

granicy.py

```
number = 1
print(1 <= number <= 10)    # True — правильно,
                             # включает края
print(1 < number < 10)      # False — ЛОВУШКА:
                             # отвергает 1 и 10!
```

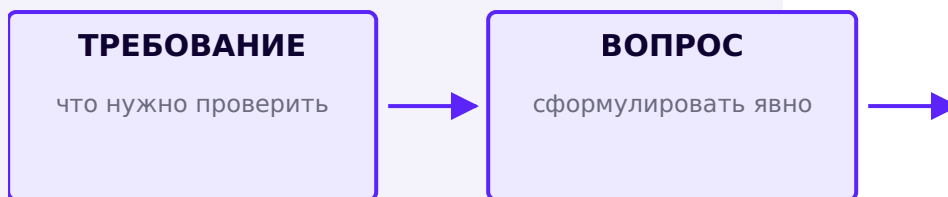
Значение	<code>1 <= number <= 10</code>	Комментарий
0	False	ниже границы

Значение	<code>1 <= number <= 10</code>	Комментарий
1	True	ровно на границе — входит
10	True	ровно на границе — входит
11	False	выше границы

Off-by-one — классическая ошибка

Если формулировка говорит «включительно», а в коде стоит строгое `< / >` вместо `<= / >=`, граничные значения незаметно отбрасываются. Такую ошибку называют **off-by-one** («ошибка на единицу») — она встречается настолько часто, что заслуживает отдельного имени.

Повторяемый способ проектировать условие



Повторяемый workflow проектирования условия — от требования до протестированного кода

Перед тем как писать код:

1. Какой вопрос мы задаём?
2. Какие значения в нём участвуют?
3. Что означает результат True?
4. Что делать, если True? А если False?
5. Есть ли больше двух исходов?
6. Какие здесь граничные значения?

Всегда тестируйте **значение чуть ниже границы, ровно на границе, и чуть выше** — это простое правило ловит огромную долю логических ошибок ещё до того, как они попадут к пользователю.

★★ Самостоятельная задача

Спроектируйте сами

Требование: «Скидка положена при сумме покупки от 1000 рублей». Пройдите весь workflow — от вопроса до протестированного кода — и проверьте граничные значения 999, 1000, 1001.

Практика: валидация, границы и off-by-one

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-21/index.html>)

Отладка логических ошибок

Программа может выполняться без единой ошибки — и всё равно давать неверный результат. Учимся находить такие ошибки методично.

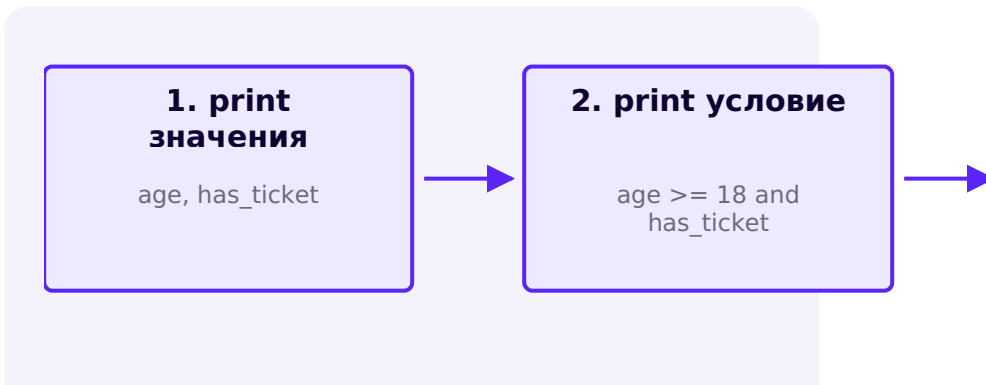
Логическая ошибка — это не ошибка Python

Программа `if age > 18:` выполняется прекрасно и не выдаёт никакой ошибки. Но если по условию задачи «доступ с 18 лет включительно», это — **логическая ошибка**: Python не может знать, что вы имели в виду.

Тип ошибки	Что происходит	Пример
SyntaxError	Python не может даже начать выполнение кода	<code>if age >= 18</code> (забыто двоеточие)
Runtime-ошибка	программа стартует, но падает на середине	<code>age_text + 5</code> (str + int)
Логическая ошибка	программа выполняется успешно, но результат неверный	<code>if age > 18</code> вместо <code>>=</code>

Метод отладки условий

Когда результат условия кажется неправильным, не гадайте — выведите промежуточные значения:



Метод отладки условий — от вывода значений до проверки каждой части отдельно

```
otladka_uslovij.py
```

```
age = 20
has_ticket = False
eligible = age >= 18 and has_ticket

print("age >= 18:", age >= 18)
print("has_ticket:", has_ticket)
print("eligible:", eligible)
# age >= 18: True
# has_ticket: False
# eligible: False
```

Разбив сложное условие на части и выведя каждую отдельно, сразу видно, ЧТО именно сделало итог ложным.

Называйте логические переменные осмысленно

Хорошее имя	Плохое имя
<code>is_ready</code>	<code>flag1</code>
<code>has_ticket</code>	<code>thing</code>
<code>can_enter</code>	<code>value2</code>
<code>needs_update</code>	<code>x</code>

Приставки `is_`, `has_`, `can_`, `needs_` — не обязательный синтаксис, а полезное соглашение: сразу видно, что переменная хранит `True / False`.

Сохраняйте сложное условие в переменную

```
imenovannoe_uslovie.py
```

```
can_enter = age >= 18 and has_ticket

if can_enter:
    print("Проходите")
```

`can_enter` читается сразу — не нужно заново разбирать выражение `age >= 18 and has_ticket` каждый раз, когда вы возвращаетесь к этому коду.

ЧУТЬ ГЛУБЖЕ — условное выражение

После того как `if/else` хорошо усвоен, полезно знать компактную форму для ПРОСТОГО выбора значения: `status = "adult" if age >= 18 else "minor"`. Это не замена обычному `if/else` — только удобство для одной строки.

match/case — на будущее

В Python есть и структурное сопоставление `match/case` для некоторых многовариантных ситуаций. Мы изучим его подробно позже — сейчас достаточно знать, что оно существует.

Практика: отладка логических ошибок

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-22/index.html>)

Мини-проект — игра «Угадай число»

Первая настоящая игра книги — компьютер загадывает число, вы пытаетесь угадать.

Первая настоящая игра в этой книге! Компьютер загадывает число, вы вводите один вариант — программа говорит, угадали вы или нет.

Сначала — как это выглядит для игрока

```
terminal_dialog.txt
```

Я загадал число от 1 до 10.

Ваш вариант: 7
Слишком большое!
Я загадал: 4

Или, если угадали с первой попытки:

```
terminal_dialog_win.txt
```

Я загадал число от 1 до 10.

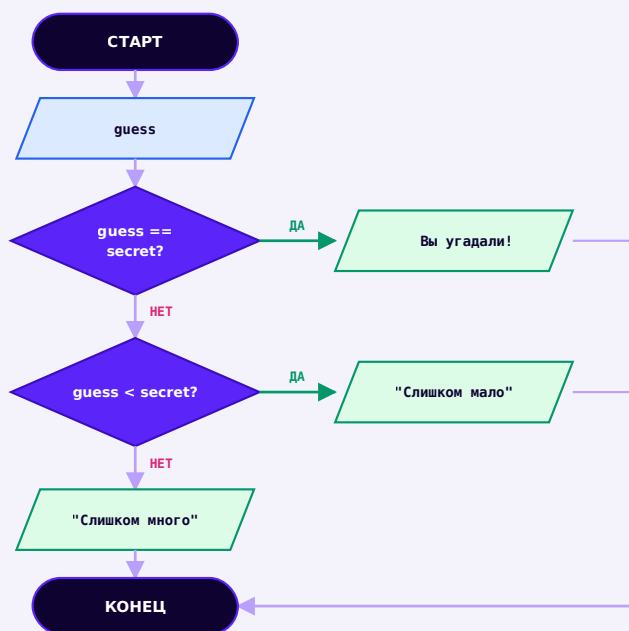
Ваш вариант: 4

Вы угадали!

Одна попытка — и это нормально

Пока мы не проходили циклы (глава 10), поэтому у игрока ровно одна попытка. Полная, переигрываемая версия с повтором до угадывания появится там же — уже сейчас держите это в голове как мотивацию для следующей главы.

Решающее дерево игры



Решающее дерево игры — переводим прямо в if / elif / else

reshenie_v_kod.py

```
if guess == secret:
    print("    Вы угадали!")
elif guess < secret:
    print("Слишком мало!")
else:
    print("Слишком много!")
```

Строим игру по шагам

Шаг 1 — загадываем число

shag_1.py

```
import random

secret = random.randint(1, 10)
```

Шаг 2 — читаем ответ игрока

shag_2.py

```
guess_text = input("Ваш вариант: ")
```

Шаг 3 — преобразуем в число

shag_3.py

```
guess = int(guess_text)
```

Забытый int() — источник ошибки

Если сравнить `guess_text < secret` напрямую (строку с числом), Python выдаст `TypeError` — сравнивать строку и число нельзя. Преобразование `int()` обязательно.

Шаг 4 — проверяем точное совпадение

shag_4.py

```
if guess == secret:
    print("    Вы угадали!")
```

Шаг 5 — добавляем «слишком мало»

shag_5.py

```
if guess == secret:
    print("    Вы угадали!")
elif guess < secret:
    print("Слишком мало!")
```

Шаг 6 — else означает «слишком много»

shag_6.py

```
if guess == secret:
    print("    Вы угадали!")
elif guess < secret:
    print("Слишком мало!")
else:
    print("Слишком много!")
```

Шаг 7 — финальная версия

ugadaj_chislo.py

```
import random

secret = random.randint(1, 10)
guess = int(input("Ваш вариант: "))

if guess == secret:
    print("    Вы угадали!")
elif guess < secret:
    print("Слишком мало!")
else:
    print("Слишком много!")

print(f"Загаданное число было: {secret}")
```

Фиксированное число для примеров курса

В примерах документации иногда используют фиксированное число вместо `random.randint(...)`, чтобы вывод был предсказуем при показе. В настоящей игре это не нужно — случайность там как раз то, что делает игру интересной.

Debug-**лаборатория: пять багов****Баг 1 — забыт `int()`****bug_1.py**

```
guess = input("Ваш вариант: ")
if guess < secret:
    ...
# TypeError: '<' not supported between instances of
# 'str' and 'int'
```

Исправление: `guess = int(input(...))`.**Баг 2 — неверный порядок веток****bug_2.py**

```
if guess < secret:
    ...
elif guess == secret:
    ...
```

Само по себе не сломается, но порядок «сначала неточные, потом точное» менее логичен — сравните с шагами 4-6 выше.

Баг 3 — = вместо ==

bug_3.py

```
if guess = secret:
    ...
# SyntaxError: invalid syntax
```

Одиночный `=` в условии — синтаксическая ошибка, Python даже не запустит программу (раздел 9.5).

Баг 4 — неверная граница

bug_4.py

```
secret = random.randint(1, 10)
# но подсказка написана как "от 1 до 9" — граница не
совпадает с кодом
```

Баг 5 — два независимых if вместо elif

bug_5.py

```
if guess < secret:
    print("Слишком мало!")
if guess > secret:
    print("Слишком много!")
# при guess == secret НИ ОДНО сообщение не покажется —
а if/elif/else это учитывает
```

★★ Самостоятельная задача

Подсказка «горячо/холодно»

Добавьте четвёртое условие: если разница между guess и secret меньше 3 (используйте `abs()`) — выведите «Очень близко!» перед основным сообщением.

Практика: собираем игру «Угадай число» целиком

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-05/index.html>)

Мини-проекты — клуб и советчик погоды

Два практичных мини-проекта на комбинирование условий из разделов 9.11–9.17.

Соберём два коротких, но показательных проекта на комбинирование условий.

Мини-проект — «Клуб Python»

Вход в клуб разрешён только совершеннолетним посетителям с билетом. Это чисто учебная логическая задача — не настоящая система безопасности!

klub_python.py

```
age = int(input("Ваш возраст: "))
has_ticket = input("У вас есть билет? (да/нет): ").strip().lower() == "да"

if age >= 18 and has_ticket:
    print("Добро пожаловать в «Клуб Python»!")
elif age < 18:
    print("Вход только с 18 лет.")
else:
    print("Нужен билет.")
```

★★ Самостоятельная задача

VIP без билета

Добавьте переменную `is_vip`. VIP-гости проходят даже без билета — используйте `or` для этого исключения.

Мини-проект — советчик погоды

Программа даёт рекомендацию на основе температуры и того, идёт ли дождь:

sovetchik_pogody.py

```
temperature = int(input("Температура: "))
is_raining = input("Идёт дождь? (да/нет): ")
is_raining = is_raining.strip().lower() == "да"

if temperature < 10 and is_raining:
    print("Тёплая куртка и зонт")
elif temperature < 10:
    print("Тёплая куртка")
elif is_raining:
    print("Просто зонт")
else:
    print("Лёгкая одежда, зонт не нужен")
```

Порядок условий продуман заранее

Обратите внимание: самое специфичное условие (холодно И дождь) стоит первым — точно по тому же принципу «первое истинное побеждает» из раздела 9.9.

★★ Самостоятельная задача

Ветреная погода

Добавьте третий фактор `is_windy`. При сильном ветре добавьте в рекомендацию «и шапку» к любому из четырёх исходов.

Практика: клуб и советчик погоды

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-23/index.html>)

Мини-проекты — классификатор оценок и интерпретатор команд

Упорядоченные elif-цепочки и связь строк из главы 8 с условиями этой главы.

Мини-проект — классификатор баллов

Переводим числовой балл (0–100) в уровень — учебный пример, границы курса условны и не отражают реальные образовательные стандарты:

score	уровень
0-49	неудовлетворительно
50-74	удовлетворительно
75-89	хорошо
90-100	отлично

```
klassifikator_ballov.py
```

```
score = int(input("Ваш балл (0-100): "))

if score >= 90:
    level = "отлично"
elif score >= 75:
    level = "хорошо"
elif score >= 50:
    level = "удовлетворительно"
else:
    level = "неудовлетворительно"

print(f"Уровень: {level}")
```

★★ Самостоятельная задача

Проверьте все границы

Прогоните классификатор на значениях 0, 49, 50, 74, 75, 89, 90, 100 — убедитесь, что каждое попадает в ожидаемый уровень (раздел 9.19 про границы).

Мини-проект — текстовый интерпретатор команд

Пользователь вводит текстовую команду — программа реагирует. Прямая связь между строками из главы 8 и условиями этой главы:

interpreter_komand.py

```
command = input("Введите команду (start/stop/help):")
command = command.strip().lower()

if command == "start":
    print("Запуск...")
elif command == "stop":
    print("Остановка...")
elif command == "help":
    print("Доступные команды: start, stop, help")
else:
    print(f"Неизвестная команда: {command}")
```

В следующей главе

Сейчас программа реагирует на ОДНУ команду и завершается. С циклом `while` (глава 10) она сможет ждать команду за командой, как настоящая мини-консоль — до тех пор, пока пользователь не введёт «exit».

★ Базовая практика

Регистронезависимость

Проверьте интерпретатор на вводе «START», « Stop », «HELP» — убедитесь, что `.strip().lower()` делает программу нечувствительной к регистру и пробелам.

Практика: классификатор оценок и интерпретатор команд

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-24/index.html>)

Условия продолжают накапливаться!

Итоговая карта: какой инструмент выбрать для какой задачи — и что дальше.

Условия могут накапливаться сколько угодно — `elif`, `and` / `or` и вложенность позволяют выразить сколь угодно сложное решение. Подведём итоги главы.

Карта принятия решений

Карта принятия решений — что использовать?

Нужно задать один вопрос да/нет? → `if`

Нужны две альтернативы? → `if / else`

Нужно несколько
взаимоисключающих вариантов? → **if / elif / else**

Нужно, чтобы ВСЕ условия были
истинны? → **and**

Достаточно ХОТЯ БЫ ОДНОГО
истинного условия? → **or**

Нужно перевернуть условие? → **not**

Нужно проверить вхождение в
текст/группу? → **in**

Нужно сравнить с None? → **is**

Нужно
сравнить значения? → **==, !=, <, >, <=, >=**

Нужно проверить
диапазон? → **цепочка сравнений**

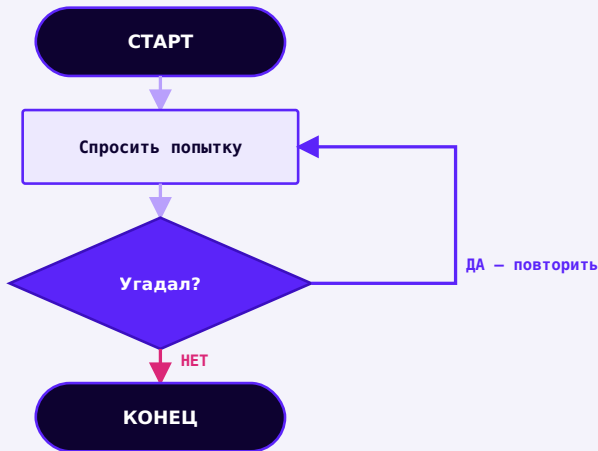
Что мы теперь умеем

Что мы узнали в этой главе

- Алгоритм — понятная последовательность действий; собирается из трёх структур: последовательность, ветвление, повторение.
- Тип `bool` хранит `True` или `False`; шесть операторов сравнения превращают значения в `bool`.
- Truthiness: ноль и пустые значения ведут себя как ложь, всё остальное — как истина (но текст `"False"` — это истина!).
- `if / elif / else` выбирают ровно ОДНУ ветку — в отличие от нескольких независимых `if`, которые могут сработать все разом.
- `and`, `or`, `not` комбинируют условия; у `and/or` есть short-circuit — Python не всегда вычисляет второй операнд.
- `is` и `==` — разные вопросы; `is None` — правильный способ проверки на `None`.
- Условия можно вкладывать, но часто их стоит упростить через `and`; всегда проверяйте граничные значения.

Мост к главе 10

Теперь программа умеет принимать ОДНО решение. Но игры и настоящие приложения обычно принимают решения СНОВА И СНОВА. «Угадай число» пока даёт только одну попытку — в следующей главе мы научим программу **повторять** действия, и игра сможет продолжаться, пока число не будет угадано.



Глава 10: тот же backward-стрелка добавляет повтор — «Угадай число» сможет продолжаться, пока число не будет угадано

Практика: карта принятия решений

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/09-06/index.html>)

Немного автоматизации!

Циклы `for` и `while` как третья структура алгоритма после ветвления из главы 9, `range()` и `enumerate()` без мифов, `break/continue/loop-else`, вложенные циклы, отладка типичных ошибок циклов — и наконец-то настоящая автоматизация всех фигур из глав 6-7.

10.1 Повторение и первый цикл `for`

Три структуры алгоритма: вспоминаем

Канонический flowchart цикла `for`

10.2 `range()` подробно

10.3 Индекс, значение и `enumerate()`

Мини-проект: анализатор текста

10.4 `if` внутри циклов `for`

Вложенные циклы `for`

Мини-проект: таблица умножения

10.5 Перебор строк, циклы `while`

Бесконечный цикл и `while True`

10.6 `break`, `continue` и `loop-else`

Мини-проект: цикл команд

10.7 Поиск, фильтрация и суммирование

10.8 Проверка ввода в цикле

10.9 Мини-проект — «Угадай число», версия 2

10.10 Отладка циклов: 10 типичных ошибок

10.11 Мини-проект — автоматизируем фигуры

10.12 Мини-проект — автоматически рисуем мандалу

10.13 Случайные узоры Turtle

10.14 Сетка фигур: вложенные циклы в Turtle

10.15 Спирали из дуг и итоги

Итоги главы

Волшебные циклы! Циклы `for`

Наконец-то настоящая автоматизация: вместо повторения кода вручную — просим Python повторить его самостоятельно.

Прежде чем Python: повторение вокруг нас

Мы повторяем действия постоянно, даже не задумываясь: чистим зубы одинаковыми движениями, маршируем одинаковыми шагами, поём куплет за куплетом одной и той же мелодии. **Повторение — это отдельный, самостоятельный способ устроить действие**, наравне с «сделать одно за другим» и «выбрать один из двух путей».

Три структуры алгоритма: вспоминаем

В главе 9 мы разобрали первые две из трёх фундаментальных структур, из которых складывается любой алгоритм. Третья — **повторение** — и есть тема этой главы.

Три структуры, из которых строится любой алгоритм

1 · ПОСЛЕДОВАТЕЛЬНОСТЬ

Шаги идут один за другим

«Сначала... потом... потом...»

Глава 9

2 · ВЕТВЛЕНИЕ

Выбор одного из путей по условию

`if / elif / else`

Глава 9

3 · ПОВТОРЕНИЕ

Один и тот же блок — снова и снова

`for` / `while`

Эта глава

Что не так с этим кодом?

Вот квадрат из главы 6 — уже знакомый код: четыре одинаковых блока «шаг вперёд, поворот», записанных вручную:

`kvadrat_bez_cikla.py`

```
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
```

А теперь представьте, что нужен не квадрат, а **двадцатиугольник**. Копировать эти две строки ещё 16 раз вручную — долго и легко ошибиться. Именно для таких повторяющихся блоков и существуют **циклы**: способ сказать Python «повтори это N раз» вместо того, чтобы копировать код руками.

Термины цикла

Прежде чем писать первый цикл, договоримся о словах, которыми будем его описывать — дальше они встретятся во всей главе:

Термин	Что это
цикл	конструкция, которая повторяет один и тот же блок кода несколько раз
тело цикла	тот самый блок кода, который повторяется (строки с отступом под <code>for / while</code>)
итерация	один проход тела цикла — один «раз» из «повтори N раз»
условие	то, что цикл проверяет, чтобы решить, продолжать или остановиться
счётчик	переменная, которая считает номер итерации или меняется с каждым шагом

Циклы for

Самый частый вид цикла в Python — `for`. Вместе с `range()` он умеет повторить блок кода заданное число раз:

```
cikl_for.py
```

```
for i in range(5):  
    print(i)
```

Разбираем по словам

Часть кода	Что означает
<code>for</code>	«начинаем цикл»
<code>i</code>	переменная цикла — на каждом шаге получает следующее значение из <code>range(5)</code>
<code>in range(5)</code>	источник значений: числа от 0 до 4
<code>:</code>	дальше начинается тело цикла (с отступом)
<code>print(i)</code>	тело цикла — то, что повторяется на каждой итерации

● **Итерация 1**
i = 0 → print(0)

● **Итерация 2**
i = 1 → print(1)

● **Итерация 3**
i = 2 → print(2)

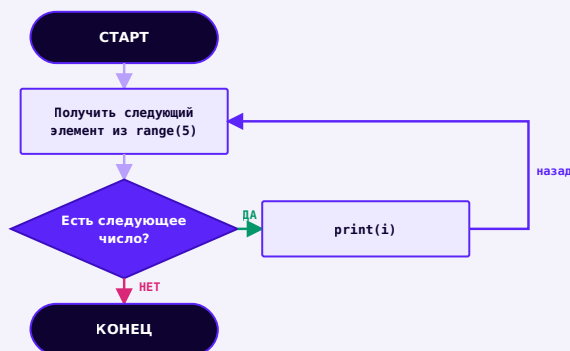
● **Итерация 4**
i = 3 → print(3)

● **Итерация 5**
i = 4 → print(4)

`for i in range(5): print(i)` — пять итераций одного и того же тела цикла

Канонический flowchart цикла for

Важно сразу привыкнуть к правильной картине происходящего: `for` не «выполняет тело N раз по волшебству» — на каждом шаге он спрашивает «есть ли следующий элемент?», и если да — выполняет тело и спрашивает снова:



`for i in range(5): print(i)` — цикл каждый раз запрашивает следующий элемент

Трасса выполнения

Таблица трассировки — способ построчно проследить, что происходит на каждой итерации. Мы будем пользоваться такой таблицей всю главу:

Итерация	i	Вывод (print)
1	0	0
2	1	1
3	2	2
4	3	3

Итерация	i	Вывод (print)
5	4	4

range() с разными аргументами — коротко

range(5) — от 0 до 4. range(2, 8) — от 2 до 7.
 range(0, 10, 2) — от 0 до 8 с шагом 2: 0, 2, 4, 6, 8.
 Подробный разбор — на следующей странице.

Когда переменная цикла не нужна

Если само число повторов важно, а не значение счётчика — по традиции его называют `_` (нижнее подчёркивание), сигнализируя «это значение сознательно не используется»:

Квадрат из главы 6: 8 строк → 2 строки

Без цикла (глава 6)

```
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
```

С циклом for

```
for _ in range(4):
    artist.forward(100)
    artist.right(90)
```

Что использовать сегодня: цикл `for`. Он не «современнее» в смысле версии Python — циклы существуют с самых первых версий языка, — но именно ради него стоило дочитать до этой главы: то же самое поведение, в 4 раза короче, и легко изменить число повторов одной цифрой.

_ — соглашение, а не специальный синтаксис

`_` — обычное имя переменной, Python не обрабатывает его как-то по-особому. Это просто договорённость между программистами: «здесь мы намеренно не используем значение цикла». Написать `for x in range(4):` тоже сработает — но будет менее понятно читателю кода.

Практика: цикл for и терминология

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/10-01/index.html>)

range() подробно

`range()` — не список, а генератор чисел по требованию. Разбираем все три его формы и то, откуда взялось правило «stop не включён».

range() — не список

`range()` — одна из самых частых причин путаницы у новичков. Разберёмся сразу и точно.

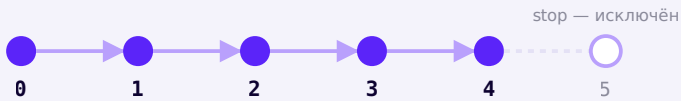
range() НЕ создаёт список

`range(5)` создаёт особый объект-генератор чисел — он выдаёт числа по одному, по требованию, а не хранит их все сразу в памяти как список.

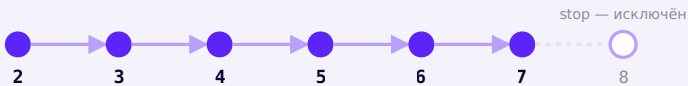
`list(range(5))` действительно даст `[0, 1, 2, 3, 4]` — но это отдельное, явное превращение, а не то, чем является сам `range`.

Три уровня сигнатуры

Вызов	Что означает	Пример
<code>range(stop)</code>	от 0 до stop, не включая stop	<code>range(5)</code> → 0, 1, 2, 3, 4
<code>range(start, stop)</code>	от start до stop, не включая stop	<code>range(2, 8)</code> → 2, 3, 4, 5, 6, 7
<code>range(start, stop, step)</code>	от start до stop с шагом step, не включая stop	<code>range(0, 10, 2)</code> → 0, 2, 4, 6, 8



`range(5)` — пять значений: 0, 1, 2, 3, 4; сам `stop=5` в результат не входит



`range(2, 8)` — от 2 до 7 включительно; 8 не входит

Почему stop не включается

Это не произвольная прихоть языка — это то же самое правило, что мы уже видели в главе 8 у срезов строк.

"Python"[0:3] берёт символы с индексами 0, 1, 2 — и не включает символ с индексом 3. range(0, 3) устроен ровно так же: 0, 1, 2, без 3. Python последователен: «конец диапазона» везде означает «до этого места, не включая его».

Полезное следствие

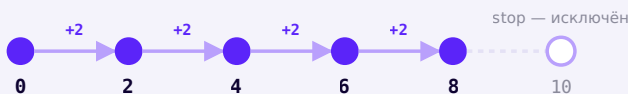
Благодаря этому правилу range(n) всегда даёт ровно n чисел, а длина среза s[a:b] всегда равна b - a — считать количество элементов получается без дополнительных +1/-1 в уме.

Шаг: step

Третий аргумент задаёт, на сколько увеличивается (или уменьшается) число на каждом шаге:

```
shag.py
```

```
for number in range(0, 10, 2):  
    print(number)
```



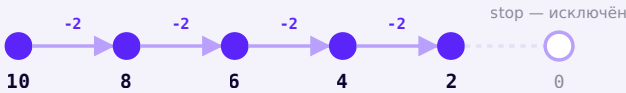
range(0, 10, 2) — шаг 2: 0, 2, 4, 6, 8

Отрицательный шаг

Шаг может быть отрицательным — тогда числа идут в обратную сторону, но start и stop нужно тоже поменять местами по смыслу: start должен быть больше, чем stop.

```
otricatelnyj_shag.py
```

```
for number in range(10, 0, -2):
    print(number)
```



`range(10, 0, -2)` — отсчёт назад с шагом 2: 10, 8, 6, 4, 2

Отрицательный range без отрицательного шага — пустой

`range(10, 0)` (без шага) не выдаст ничего — шаг по умолчанию равен `+1`, а с ним никогда не попасть от 10 вниз к 0. Чтобы диапазон шёл вниз, шаг обязательно должен быть отрицательным и соответствовать направлению.

Шаг 0 — ошибка, а не бесконечный цикл

`range(0, 10, 0)` не зависит и не превращается в бесконечный цикл — Python сразу поднимает `ValueError: range() arg 3 must not be zero`, потому что с нулевым шагом никогда не сдвинуться от start к stop.

Практика: range() — старт, стоп, шаг

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/10-09/index.html>)

Индекс, значение и enumerate()

for умеет перебирать не только числа — а счётчик и накопитель превращают простой перебор в настоящий инструмент анализа данных.

for умеет перебирать не только числа

Строка — это последовательность символов (глава 8), а значит, for умеет перебирать и её — символ за символом, без индексов:

```
perebor_strok.py
```

```
for letter in "Python":
    print(letter)
```

То же самое работает для списков и любых других **итерируемых** объектов — так называют всё, что for умеет перебирать по одному элементу. Строка, список, диапазон range() — всё это итерируемые объекты; точный технический механизм, как именно Python это делает «под капотом», мы разберём в следующих главах, когда будем подробно изучать списки.

Индекс против значения

В цикле for letter in "Python": мы получаем сам символ («значение»), но не его номер по счёту («индекс»). Иногда номер важен — например, чтобы вывести «символ №1: P».

Классический вариант — цикл по индексам

```
cikl_po_indeksam.py
```

```
slovo = "Python"
for i in range(len(slovo)):
    print(i, slovo[i])
```

«Питонический» вариант — enumerate()

```
enumerate_variant.py
```

```
slovo = "Python"
for i, letter in enumerate(slovo):
    print(i, letter)
```

i	letter
0	P
1	y
2	t
3	h
4	o
5	n

Когда использовать enumerate()

enumerate() нужен именно тогда, когда важны и индекс, и значение одновременно. Если индекс вообще не нужен — используйте `for letter in slovo:`, без него. Цикл по индексам (`range(len(...))`) — не «неправильный» способ, а просто более многословный там, где `enumerate()` справится короче.

Счётчик — переменная, которая считает

Счётчик — переменная, которая с каждой итерацией увеличивается (или уменьшается) на фиксированный шаг, обычно на 1. Она отслеживает «сколько раз мы это уже сделали».

```
schetchik.py

glasnye = "aeёиоуыэюя"
slovo = "программирование"
kolichество = 0

for letter in slovo:
    if letter in glasnye:
        kolichество += 1

print(kolichество)
```

Символ	letter in glasnye?	kolichество после шага
п	нет	0
р	нет	0
о	да	1
г	нет	1
р	нет	1
а	да	2
...	...	...

Накопитель (аккумулятор) — переменная, которая копит результат

Накопитель устроен похоже на счётчик, но копит не количество шагов, а сам результат — сумму, произведение, склеенную строку:

```
nakopitel.py

chisla = [4, 8, 15, 16, 23, 42]
summa = 0

for n in chisla:
    summa += n

print(summa)
```

Итерация	n	summa после шага
1	4	4
2	8	12
3	15	27
4	16	43
5	23	66
6	42	108

Общее в счётчике и накопителе: состояние

И счётчик, и накопитель — примеры **состояния**: переменной, которая живёт снаружи цикла, но меняется на каждой итерации внутри него. Перед циклом состояние обязательно нужно инициализировать (`kolichestvo = 0`) — без этого шага `kolichestvo += 1` на первой же итерации упадёт с ошибкой, потому что переменной ещё не существует.

Мини-проект: анализатор текста

Соберём счётчик и накопитель вместе в одном небольшом инструменте — подсчитаем гласные, длину самого длинного слова и общее число слов в тексте:

```
analizator_teksta.py
```

```
tekst = "python это просто и понятно"
slova = tekst.split()

glasnye_count = 0
for letter in tekst:
    if letter in "аеёиоуыюя":
        glasnye_count += 1

samoe_dlinnoe = ""
for slovo in slova:
    if len(slovo) > len(samoe_dlinnoe):
        самое_dlinnoe = slovo

print(f"Слов: {len(slova)}, гласных: {glasnye_count},\nсамое длинное: {самое_dlinное}")
```

Три независимых накопителя рядом

Обратите внимание: `glasnye_count` и `самое_dlinnoe` — два разных накопителя, каждый со своим циклом и своей начальной инициализацией. Комбинировать несколько состояний в одной программе — обычное дело, главное — не запутать, какое состояние меняется в каком цикле.

Практика: индекс, значение и `enumerate()`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/10-10/index.html>)

Практика: мини-проект «Анализатор текста»

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/10-11/index.html>)

Условия `if` внутри циклов `for`

Циклы и условия отлично работают вместе — а циклы можно вкладывать друг в друга.

Условия `if` внутри циклов `for`

`if` из главы 9 прекрасно работает внутри цикла — на каждом шаге можно принимать своё решение:

```
if_v_cikle.py
```

```
for number in range(1, 11):
    if number % 2 == 0:
        print(number, "– чётное")
```

Вложенные циклы for

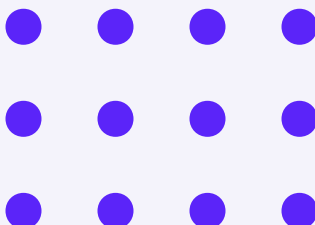
Цикл можно поместить внутрь другого цикла — тогда внутренний цикл выполняется полностью на каждом шаге внешнего. Представьте сетку: внешний цикл идёт по строкам, внутренний — по столбцам каждой строки:

vlozhennye_cikly.py

```
for row in range(3):  
    for col in range(4):  
        print(f"({row}, {col})", end=" ")  
    print() # новая строка после каждого ряда
```

↑
внешний цикл — row (0..2)

внутренний цикл — col (0..3) →



Внешний цикл — 3 строки, внутренний — 4 столбца
в каждой: $3 \times 4 = 12$ ячеек

Сколько раз выполнится внутренний цикл?

строк×столбцов=итераций тела

Формула числа итераций вложенного цикла

row	col	print(...) сработал раз №
0	0	1
0	1	2
0	2	3
0	3	4
1	0	5
...	...	...
2	3	12

Вложенные циклы — частая причина неожиданной медлительности

Внешний цикл выполняется 3 раза, внутренний — 4 раза *на каждом* шаге внешнего — итого `print(...)` внутри сработает $3 * 4 = 12$ раз. Если оба цикла идут не до 3-4, а до тысяч, итоговое число итераций перемножается — и программа может выполняться заметно дольше, чем кажется на первый взгляд.

Мини-проект: таблица умножения

Классическая задача на вложенные циклы — напечатать таблицу умножения от 1 до 5:

```
tablica_umnozheniya.py
```

```
for a in range(1, 6):
    for b in range(1, 6):
        print(a * b, end="\t")
    print()
```

Один или два примера — этого достаточно

Специально не будем множить примеры узоров на вложенных циклах — таблица умножения (и сетка выше) дают достаточно интуиции, чтобы читать и писать любые похожие конструкции самостоятельно.

Цикл + строка + условие

Вложенность работает и там, где внутренний «цикл» — это перебор символов строки внутри внешнего перебора списка слов:

```
cikl_stroka_uslovie.py
```

```
slova = ["python", "код", "цикл"]

for slovo in slova:
    bukvy_o = 0
    for letter in slovo:
        if letter == "o":
            bukvy_o += 1
    print(slovo, "--", bukvy_o)
```

Практика: условия и вложенные циклы

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/10-02/index.html>)

Практика: мини-проект «Таблица умножения»

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/10-12/index.html) → (https://www.cartesianschool.org/
practice/10-12/index.html)

Практика: сколько итераций во вложенном цикле?

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/10-24/index.html) → (https://www.cartesianschool.org/
practice/10-24/index.html)

Циклы while

`for` умеет перебирать не только числа — а `while` повторяет действие, пока условие остаётся истинным, сколько бы раз это ни потребовалось.

Когда число повторов заранее неизвестно

`for` отлично подходит, когда мы заранее знаем, сколько раз нужно повторить действие — «4 стороны квадрата», «все буквы слова». Но что делать, если число повторов заранее неизвестно — например, «повторяй, пока пользователь не угадает число»? Для таких задач существует второй вид цикла — `while`.

Циклы while

В отличие от `for`, `while` повторяется **пока истинно условие**:

```
cikl_while.py
```

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Канонический flowchart цикла while



Три части цикла while

Часть	Код	Роль
Инициализация	<code>count = 0</code>	готовит состояние ДО первой проверки условия
Условие	<code>count < 5</code>	проверяется перед каждой итерацией, включая самую первую

Часть	Код	Роль
Обновление	<code>count += 1</code>	меняет состояние так, чтобы условие рано или поздно стало ложным

Итерация	Проверка <code>count < 5</code>	Вывод	<code>count</code> после шага
1	<code>0 < 5</code> → да	0	1
2	<code>1 < 5</code> → да	1	2
3	<code>2 < 5</code> → да	2	3
4	<code>3 < 5</code> → да	3	4
5	<code>4 < 5</code> → да	4	5
—	<code>5 < 5</code> → нет	цикл завершён	—

while — это pre-test цикл

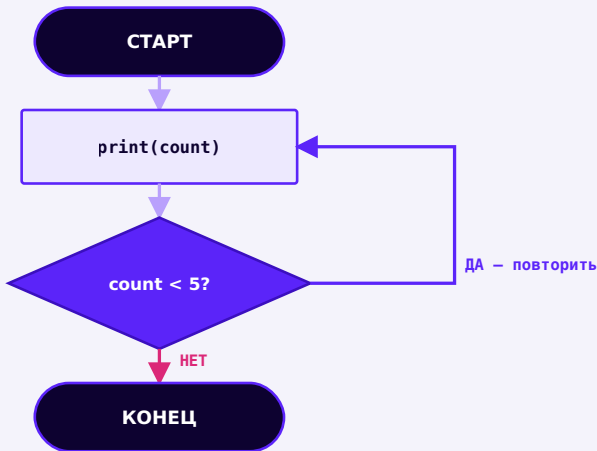
Условие в `while` проверяется **до** тела — значит, если условие изначально ложно, тело не выполнится вообще ни разу (ноль итераций). В Python нет отдельного «сначала выполни, потом проверь» цикла (в некоторых других языках он называется `do/while`) — если такое поведение нужно, его моделируют через `while True` с `break` в конце тела (мы увидим этот приём на следующей странице).

Бесконечный цикл — намеренно и случайно

Посмотрите на этот код внимательно:


```
beskonechnyj_cikl.py
```

```
count = 0
while count < 5:
    print(count)
    # забыли count += 1!
```



Состояние (count) не меняется между итерациями
— условие остаётся истинным навсегда

Как остановить зависшую программу

Если программа зависла в бесконечном цикле — в терминале нажмите `Ctrl+C` (или кнопку «Стоп»/«Interrupt kernel» в Jupyter). Всегда проверяйте, что внутри `while` есть шаг, который рано или поздно сделает условие ложным.

while True — тоже нормально

`while True` создаёт цикл, условие которого само по себе никогда не станет ложным — единственный выход из него: `break` внутри тела. Это распространённый и совершенно правильный приём, когда условие остановки естественнее проверить в середине тела, а не в его начале — мы воспользуемся им уже на следующей странице.

while True сам по себе — не ошибка

`while True:` не является «плохой практикой» сама по себе — это осознанный инструмент. Проблема возникает только тогда, когда внутри забыли поставить `break` — вот тогда цикл действительно никогда не остановится.

Перебор строк — напоминание

Строка — это последовательность символов (глава 8) — `for` умеет перебирать её напрямую, без индексов (подробный разбор индекса и значения — на странице про `enumerate()`):

```
perebor_strok.py
```

```
for letter in "Python":  
    print(letter)
```

for или while?

Ориентир, а не жёсткое правило

Число повторов известно заранее
(«нарисовать 4 стороны»,
«перебрать все буквы»)

→ **for**

Число повторов заранее
неизвестно («пока пользователь
не угадает»)

→ **while**

Естественнее
проверять
условие
остановки в
середине тела

→ **while True + break**

Нужно пройти по готовой
последовательности (строка,
список, range)

→ **for**

Оба вида цикла в Python одинаково мощные —
выбор влияет на читаемость кода, а не на то, что
вообще возможно сделать

Практика: циклы while

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → ([https://www.cartesianschool.org/
practice/10-03/index.html](https://www.cartesianschool.org/practice/10-03/index.html))

Практика: for или while — что выбрать?

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/10-23/index.html) (<https://www.cartesianschool.org/practice/10-23/index.html>)

Прервать миссию! `break` и `continue`

Два способа управлять циклом изнутри — остановить его совсем или пропустить один шаг — и особый блок `else`, который знает, был ли `break`.

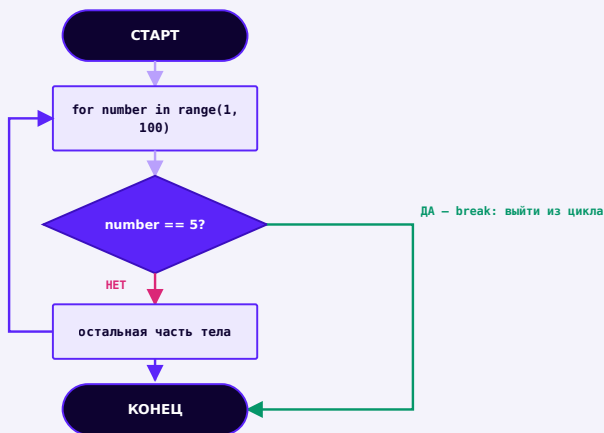
Иногда нужно выйти из цикла раньше времени или пропустить один шаг, не завершая цикл целиком — для этого есть два ключевых слова, и один особый способ узнать, был ли `break`.

`break` — прервать цикл полностью

`break.py`

```
for number in range(1, 100):  
    if number == 5:  
        break # цикл останавливается насовсем  
    print(number)
```

Выведет только 1, 2, 3, 4 — как только `number` становится равным 5, цикл прерывается, не дожидаясь оставшихся 94 значений `range()`.



`break` выходит СРАЗУ из цикла целиком — но не из программы

break не завершает программу

`break` прерывает только тот цикл, в котором находится — код после цикла продолжает выполняться как обычно. Это не аналог выхода из программы, а именно выход из повторения.

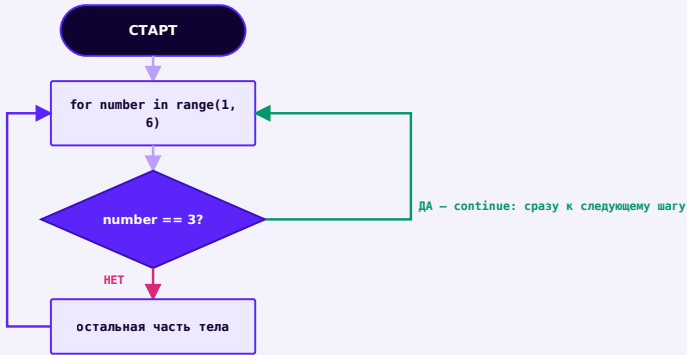
`continue` — пропустить этот шаг

`continue.py`

```

for number in range(1, 6):
    if number == 3:
        continue # пропускаем оставшуюся часть тела
    для этого шага
    print(number)
    
```

Выведет 1, 2, 4, 5 — число 3 пропущено, но цикл продолжается дальше, в отличие от `break`.



`continue` пропускает остаток ТЕЛА и переходит к следующей итерации — цикл продолжается

continue не выходит из цикла

`continue` обрывает только текущий проход тела — сам цикл как продолжался, так и продолжается со следующего элемента. Это противоположность `break`, а не его более мягкий вариант.

break и continue рядом

	break	continue
Что происходит	цикл останавливается насовсем	текущая итерация обрывается, цикл идёт дальше

	break	continue
Код после цикла	выполняется сразу	выполняется только после того, как цикл закончится сам
Частый пример	нашли то, что искали — незачем продолжать	этот элемент не подходит — пропускаем именно его

break в while True

Самое частое место для `break` — цикл `while True`, где условие остановки естественнее проверить в середине тела:

`while_true_break.py`

```
komanda = ""
while True:
    komanda = input("Введите команду (stop — выход): ")
    if komanda == "stop":
        break
    print("Выполняю:", komanda)
```

continue перед обновлением счётчика — частая ловушка

В цикле `while` размещать `continue` ДО строки, которая меняет состояние (например, `count += 1`), опасно: эта строка будет пропускаться каждый раз, когда сработал `continue`, — и цикл может никогда не завершиться. Подробнее разберём в уроке об отладке циклов.

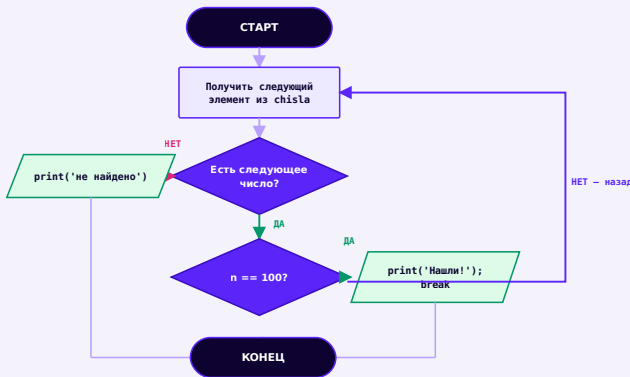
loop-else — else у цикла, а не у if

У циклов `for` и `while` есть редко используемая, но иногда очень удобная возможность — блок `else`, который выполняется, **только если цикл завершился без `break`**:

loop_else.py

```
chisla = [4, 8, 15, 16, 23, 42]

for n in chisla:
    if n == 100:
        print("Нашли 100!")
        break
else:
    print("100 не найдено ни разу")
```



`else` у цикла выполняется, только если цикл дошёл до конца БЕЗ `break`

loop-else — это НЕ if/else

Хотя слово `else` то же самое, что и у `if`, смысл совершенно другой. `else` у `if` означает «если условие ложно». `else` у цикла означает «если цикл закончился сам, а не через `break`». Если внутри цикла не было `break` вообще — блок `else` выполнится всегда, как только цикл дойдёт до конца.

Мини-проект: цикл команд

Соберём `while True`, `break` и работу со строками/условиями из глав 8-9 в маленький интерактивный цикл. Это учебная демонстрация приёма, а не замена интерпретатору PySH из введения курса:

cikl_komand.py

```
zhurnal = []
while True:
    komanda = input("Команда (help/list/stop):")
    komanda = komanda.strip().lower()
    if komanda == "stop":
        print("Завершаю работу.")
        break
    elif komanda == "help":
        print("Доступно: help, list, stop")
    elif komanda == "list":
        print("Журнал:", zhurnal)
    else:
        zhurnal.append(komanda)
        print(f"Добавлено в журнал: {komanda}")
```

Практика: break и continue

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/10-04/index.html>)

Практика: мини-проект «Цикл команд» и loop-else

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/10-13/index.html>)

Поиск, фильтрация и суммирование

Цикл в связке с `if` — это уже настоящий алгоритм: три самых частых паттерна, из которых строится большинство программ на циклах.

Цикл + `if` = настоящие алгоритмы

Большинство реальных задач с циклами — это `for` или `while` в связке с `if`. Разберём три самых частых паттерна: поиск, фильтрация, суммирование.

Поиск (search)

`poisk.py`

```
chisla = [4, 8, 15, 16, 23, 42]
iskomoe = 16
najdeno = False

for n in chisla:
    if n == iskomoe:
        najdeno = True
        break

print(najdeno)
```

Подсчёт (counting)

podschet.py

```
chisla = [4, 8, 15, 16, 23, 42, 8, 4]
chetnye = 0

for n in chisla:
    if n % 2 == 0:
        chetnye += 1

print(chetnye)
```

Суммирование: вручную и через sum()

summa_vruchnuyu.py

```
chisla = [4, 8, 15, 16, 23, 42]
summa = 0

for n in chisla:
    summa += n

print(summa)
```

summa_cherez_sum.py

```
chisla = [4, 8, 15, 16, 23, 42]
summa = sum(chisla)
print(summa)
```

sum() — не магия, а тот же цикл внутри

sum() — встроенная функция, которая делает ровно то же самое, что и ручной цикл с накопителем выше, просто в одну строку. Полезно сначала написать вручную (чтобы понять, что происходит), а потом знать, что для готового результата есть короткий путь.

Предпросмотр: поиск минимума и максимума

min_max_vruchnuyu.py

```
chisla = [4, 8, 15, 16, 23, 42]
maksimum = chisla[0]

for n in chisla:
    if n > maksimum:
        maksimum = n

print(maksimum)
# то же самое короче: print(max(chisla))
```

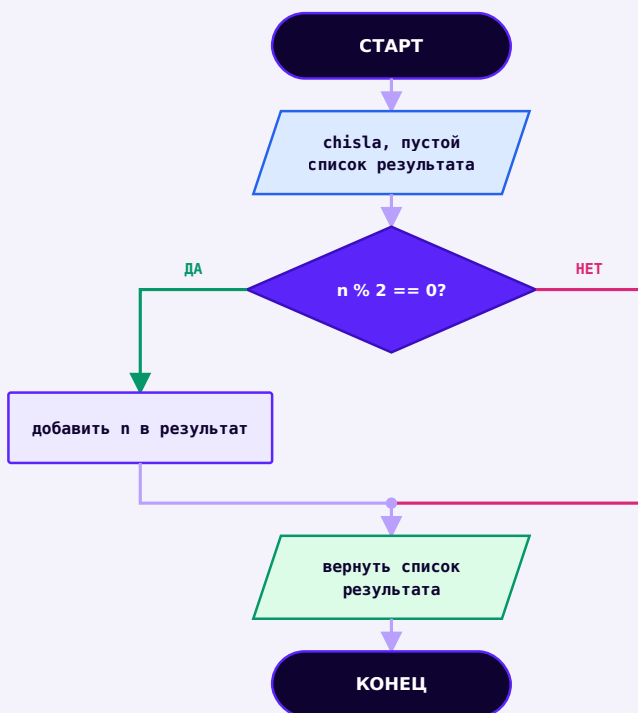
Фильтрация — собираем только нужное

filtraciya.py

```
chisla = [4, 8, 15, 16, 23, 42]
chetnye_chisla = []

for n in chisla:
    if n % 2 == 0:
        chetnye_chisla.append(n)

print(chetnye_chisla)
```



Паттерн фильтрации: пройти по всем элементам, оставить только те, что прошли условие

Сентинел-цикл — особое значение как сигнал остановки

Сентинел — специальное значение, которое само по себе не является частью данных, а сигнализирует «данные закончились»:

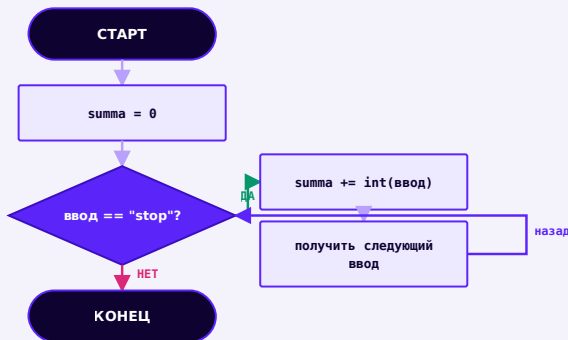
sentinel_cikl.py

```

summa = 0
while True:
    chislo = input("Введите число (stop – закончить): ")
    if chislo == "stop":
        break
    summa += int(chislo)

print("Сумма:", summa)

```



Сентинел-цикл: пользователь сам решает, когда данные закончились

Необязательно: FizzBuzz

Классическая, но необязательная задача-разминка на циклы и условия — если интересно, попробуйте написать её сами перед тем, как посмотреть решение:

fizzbuzz.py

```
for n in range(1, 21):
    if n % 15 == 0:
        print("ФиззБазз")
    elif n % 3 == 0:
        print("Фызз")
    elif n % 5 == 0:
        print("Базз")
    else:
        print(n)
```

Практика: поиск, подсчёт, суммирование, фильтрация

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/10-14/index.html) → (https://www.cartesianschool.org/practice/10-14/index.html)

Практика: сентинел-цикл

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/10-22/index.html) → (https://www.cartesianschool.org/practice/10-22/index.html)

Проверка ввода в цикле

Один из самых практичных паттернов while — переспрашивать пользователя, пока введённое не станет подходящим.

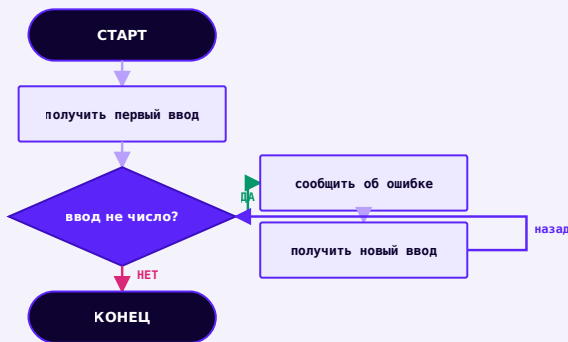
Проблема: пользователь может ввести что угодно

Программа из главы 9 ожидала число от пользователя — но что, если он введёт слово или оставит поле пустым? Цикл `while` позволяет переспрашивать, пока ввод не станет подходящим:

proverka_vvoda.py

```
vozrast = input("Введите возраст: ")
while not vozrast.isdigit():
    print("Нужно ввести число!")
    vozrast = input("Введите возраст: ")

vozrast = int(vozrast)
print("Спасибо! Вам", vozrast, "лет")
```



Цикл проверки: переспрашивать, пока ввод не станет подходящим

isdigit() — простая, но ограниченная проверка

`str.isdigit()` проверяет, что строка состоит только из цифр — этого достаточно для целых положительных чисел, но не отличает "-5" (отрицательное число) или "3.5" (дробное) от некорректного ввода. Более надёжный способ проверки — конструкция `try/except` — мы изучим её в следующих главах; сейчас достаточно уметь проверять целые положительные числа.

Ограничение количества попыток

Иногда бесконечно переспрашивать не годится — разумно ограничить число попыток:

```
proverka_s_limitom.py
```

```
popytki = 0
max_popytok = 3

while popytki < max_popytok:
    vozrast = input("Введите возраст: ")
    if vozrast.isdigit():
        print("Принято:", vozrast)
        break
    print("Это не похоже на число.")
    popytki += 1
else:
    print("Слишком много неверных попыток.")
```

Здесь снова пригодился loop-else

Блок `else` у `while` выполнится, только если попытки закончились без успешного `break` — ровно то, что нужно для сообщения «слишком много неверных попыток».

Практика: проверка ввода в цикле

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/10-15/index.html\)](https://www.cartesianschool.org/practice/10-15/index.html)

Мини-проект — игра «Угадай число», версия 2

Та же игра, что и в главе 9, — но теперь с неограниченным числом попыток, спроектированная от псевдокода и схемы до готового кода.

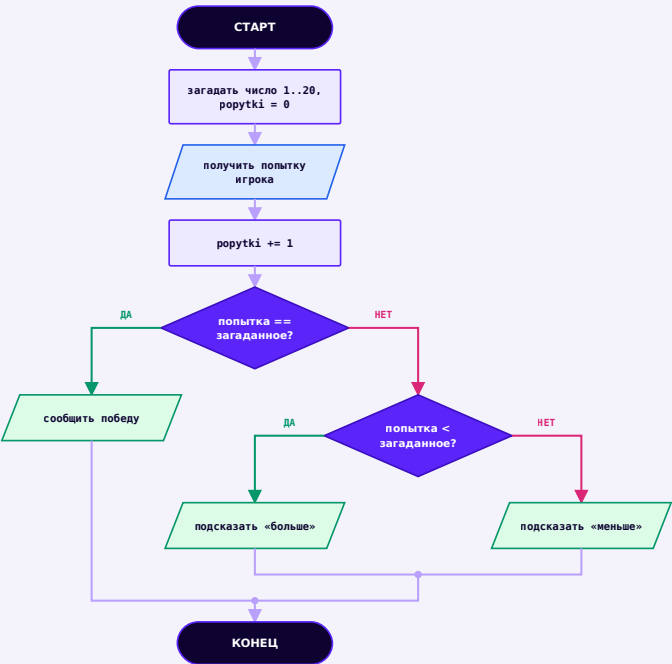
Помните игру «Угадай число» из главы 9? У неё была одна проблема — всего одна попытка. Теперь, вооружившись `while` и `break`, дадим игроку сколько угодно попыток. Спроектируем её как настоящий алгоритм — сначала словами, потом схемой, потом кодом.

Шаг 1 — псевдокод

psevdokod.txt

```
загадать случайное число от 1 до 20
счётчик попыток = 0
повторять пока не угадано:
    получить попытку игрока
    увеличить счётчик попыток
    если попытка равна загаданному — сообщить победу и
остановиться
    иначе если попытка меньше — подсказать «больше»
    иначе — подсказать «меньше»
```

Шаг 2 — flowchart



Угадай число v2 — алгоритм целиком, до единой строчки Python

Шаг 3 — таблица трассировки (пример игры)

Попытка №	Ввод игрока	Загадано	Результат
1	10	14	меньше
2	17	14	больше

Попытка №	Ввод игрока	Загадано	Результат
3	14	14	угадал!

Шаг 4 — код

```
ugadaj_chislo_v2.py
```

```
import random

zagadannoe = random.randint(1, 20)
popytki = 0

while True:
    popytka = int(input("Угадайте число от 1 до 20:
    "))
    popytki += 1

    if popytka == zagadannoe:
        print(f"Поздравляем, вы угадали за {popytki}
попыток(ки)!")
        break
    elif popytka < zagadannoe:
        print("Загаданное число больше.")
    else:
        print("Загаданное число меньше.")
```

while True — намеренно бесконечный цикл

`while True`: сам по себе никогда не остановится — единственный выход: `break` внутри. Условие остановки («игрок угадал») естественнее проверить в середине тела, а не в начале — идеальный случай для этого приёма.

Задача на закрепление: ограничение попыток и loop-else

★★ Самостоятельная задача

Ограничение попыток

Перепишите игру так, чтобы у игрока было не более 5 попыток — используйте цикл `for` по `range(5)` вместо `while True`, и блок `else` у цикла, чтобы сообщить правильный ответ, если игрок за 5 попыток не угадал.

Практика: «Угадай число» с неограниченными попытками

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/10-05/index.html>)

Практика: «Угадай число» с ограничением попыток (loop-else)

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/10-16/index.html>)

Отладка циклов

Десять типовых ошибок циклов и универсальный метод их поиска — таблица трассировки, построенная вручную, строка за строкой.

Как читать чужой (и свой) сломанный цикл

Циклы — самое частое место, где программа *работает*, но выдаёт не тот результат, или вообще зависает. Хорошая новость: подавляющее большинство таких ошибок укладывается всего в десяток типовых сценариев. Разберём их по одному.

Важно: мы НЕ будем запускать зависающий код

Некоторые примеры ниже описывают бесконечные циклы — в практике этой страницы вы будете **читать и предсказывать** поведение кода, а не запускать реальный зависающий цикл в браузере. Если код в вашей программе действительно завис — остановите его вручную (Ctrl+C в терминале, «Interrupt kernel» в Jupyter), не дожидаясь самостоятельной остановки.

10 типичных ошибок циклов

1. Забыли обновить состояние

```
bug.py

while count < 5:
    print(count)
    # забыли count += 1
```

Условие никогда не станет ложным — бесконечный цикл. Проверьте, что внутри while есть строка, меняющая переменную из условия.

2. Ошибка на единицу в range()

`bug.py`

```
for i in range(1, 10):  
    print(i) # не выведет 10!
```

`range(1, 10)` даёт 1..9 — если нужно включить 10, это `range(1, 11)`. Классическая off-by-one ошибка.

3. Неверный отступ тела

`bug.py`

```
for n in chisla:  
print(n) # SyntaxError: ожидался отступ
```

Тело цикла обязано иметь отступ. Без него Python даже не запустит программу.

4. Счётчик обнулён внутри цикла

`bug.py`

```
for n in chisla:  
    kolichество = 0  
    kolichество += 1
```

`kolichество` сбрасывается на каждой итерации — в конце в нём всегда 1, а не общее количество. Инициализация должна быть ДО цикла, не внутри.

5. `break` на неверном уровне отступа

`bug.py`

```
for n in chisla:
    if n == 5:
        break # не в теле if!
```

`break`, стоящий вне `if` (из-за отступа), сработает на первой же итерации независимо от условия. Проверяйте отступы так же внимательно, как и условия.

6. `continue` перед обновлением

`bug.py`

```
while count < 5:
    if count == 2:
        continue
    print(count)
    count += 1
```

Когда `count == 2`, `continue` пропускает `count += 1` — и `count` навсегда останется равным 2. Бесконечный цикл, и его особенно трудно заметить.

7. Путаница переменных вложенных циклов

`bug.py`

```
for i in range(3):
    for i in range(4):
        print(i)
```


Внутренний цикл использует то же имя `i`, что и внешний — внешнее значение `i` затирается и теряется. Используйте разные имена, например `i` и `j`.

8. Неверный отрицательный шаг

bug.py

```
for n in range(10, 0):  
    print(n)  # ничего не выведет
```

`range(10, 0)` с шагом по умолчанию `+1` никогда не дойдёт от 10 до 0 — нужен `range(10, 0, -1)`.

9. Несоответствие состояния и условия

bug.py

```
while spisok:  
    print(spisok[0])  
    # забыли удалить элемент из spisok
```

Условие проверяет `spisok` (пока список не пуст), но тело никогда не уменьшает список — бесконечный цикл, хотя выглядит иначе, чем счётчик.

10. Условие ложно с самого начала

`bug.py`

```
n = 10
while n < 5:
    print(n)
    n += 1
```

Тело не выполнится ни разу — ноль итераций. Это не ошибка сама по себе, но частая неожиданность, если условие не проверили заранее.

Метод отладки: таблица трассировки вручную

Когда цикл ведёт себя не так, как ожидалось, самый надёжный способ разобраться — построить ту же таблицу трассировки, что мы использовали весь этот раздел, но теперь для реального, сломанного кода:

Итерация	Условие	Переменные ДО тела	Что выводится	Переменные ПОСЛЕ тела
1	...	...	...	...
2	...	...	...	...
...	...	...	...	...

Заполняйте таблицу построчно, как будто вы — Python

Не пытайтесь угадать результат целиком. Возьмите первую итерацию, честно распишите условие, значения переменных до и после тела — и повторяйте, пока не увидите, где ожидание разошлось с тем, что происходит на самом деле.

Off-by-one — отдельный разбор

«Ошибка на единицу» — самая частая семья багов циклов: цикл выполняется на один раз больше или меньше, чем нужно.



`range(1, 10)` — последнее значение 9, не 10!



`range(1, 11)` — правильно, если нужно дойти до 10
включительно

Чек-лист граничных случаев

Вопрос	Зачем проверять
Что произойдёт при нуле повторов (пустая последовательность)?	тело цикла не выполнится вообще — не сломается ли код после цикла
Что произойдёт при одном повторе?	самый частый источник ошибок на единицу

Вопрос	Зачем проверять
Включена ли последняя граница туда, куда нужно?	stop у range() и срезов не включён по умолчанию
Совпадает ли направление шага с направлением start→stop?	положительный шаг для роста, отрицательный — для убывания

Случай	Пример	Сколько итераций
Ноль	for n in []:	0 — тело не выполнится ни разу
Один	for n in [7]:	1
Много	for n in range(1000):	1000

Переменная цикла после его завершения

peremennaya_posle_cikla.py

```
for n in range(5):
    pass

print(n)  # 4 — последнее значение, которое приняла n
```

Переменная не исчезает после for

После завершения for переменная цикла остаётся связана с последним полученным значением — в отличие от некоторых других языков, в Python у циклов нет отдельной «области видимости».

Исключение: если последовательность была пустой, тело не выполнилось ни разу, и переменная цикла вообще не была создана — обращение к ней после цикла в этом случае вызовет `NameError`.

Имена переменных цикла

Короткие имена вроде `i`, `j`, `k` — давняя традиция для простых числовых счётчиков (особенно во вложенных циклах), но как только переменная цикла имеет реальный смысл — называйте её осмысленно: `for slovo in slova:` читается понятнее, чем `for x in y:`.

Затенение переменных (shadowing)

Если внутри цикла переменной цикла присвоить новое значение вручную, или использовать имя, которое уже существовало снаружи цикла, — можно случайно затереть нужное значение. Особенно легко так ошибиться во вложенных циклах с одинаковыми именами (см. ошибку №7 выше).

Предпросмотр: изменение списка во время перебора

Не изменяйте список, который сейчас перебираете

Удаление или добавление элементов в список прямо во время цикла `for элемент in spisok:` может привести к тому, что некоторые элементы будут пропущены или обработаны дважды. Подробно разберём эту тему и безопасные способы её обхода в главе про списки — сейчас достаточно знать, что так делать не стоит.

Немного о производительности

Каждая лишняя вложенность цикла умножает число итераций (мы уже видели это на примере таблицы умножения). Пока считать нужно тысячи, а не миллионы значений — разница незаметна. Формальный разбор скорости алгоритмов ждёт вас в старших главах — сейчас достаточно интуиции: вложенный цикл внутри цикла обычно работает заметно медленнее одного цикла того же размера.

Практика: найди ошибку в цикле

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/10-17/index.html\)](https://www.cartesianschool.org/practice/10-17/index.html)

Практика: off-by-one — посчитай итерации

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/10-18/index.html\)](https://www.cartesianschool.org/practice/10-18/index.html)

Мини-проект — автоматизируем рисование квадрата

Возвращаемся к фигурам из главы 6 — на этот раз с циклами вместо ручного повторения, и с настоящими выполненными результатами.

Мини-проект — автоматизируем квадрат

Применим цикл к квадрату из главы 6. Обе версии кода ниже рисуют абсолютно одинаковый квадрат — единственная разница в том, сколько строк для этого понадобилось:

Квадрат: 8 строк вручную против 2 строк с циклом

Без цикла (глава 6)

```
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
artist.forward(100)
artist.right(90)
```

С циклом for

```
for _ in range(4):
    artist.forward(100)
    artist.right(90)
```

Что использовать сегодня: цикл — тот же самый результат, но в 4 раза короче и с одним местом для изменения числа сторон.

`avto_kvadrat.py`

```
import turtle

screen = turtle.Screen()
screen.setup(width=300, height=300)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(3)

for _ in range(4):
    artist.forward(100)
    artist.right(90)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Квадрат, нарисованный Turtle

Реальный результат — оба варианта кода выше
рисуют именно этот квадрат

Автоматизируем любую простую фигуру

Обобщим квадрат до шаблона, который рисует **любой** правильный многоугольник — используя формулу угла поворота из главы 6:

$360^\circ \div \text{количество}$ сторон = угол поворота

Чем больше сторон — тем меньше угол поворота на каждом шаге

avto_lyubaya_figura.py

```
storony = 8          # хотим восьмиугольник — просто
менеем это число
dlina = 60
ugol = 360 / storony

for _ in range(storony):
    artist.forward(dlina)
    artist.right(ugol)
```

Одна переменная — любая фигура

Поменяйте `storony` на 3, 5, 6, 20 — программа автоматически нарисует треугольник, пятиугольник, шестиугольник или почти окружность, без единого изменения остального кода. Ниже — реальный результат для нескольких значений `storony`.

Треугольник (storony = 3)

treugolnik.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=300, height=300)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(3)

storony = 3
dlina = 110
ugol = 360 / storony

for _ in range(storony):
    artist.forward(dlina)
    artist.right(ugol)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Треугольник, нарисованный Turtle

storony = 3, dlina = 110 → правильный треугольник

Пятиугольник (storony = 5)

pyatiugolnik.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=300, height=300)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(3)

storony = 5
dlina = 75
ugol = 360 / storony

for _ in range(storony):
    artist.forward(dlina)
    artist.right(ugol)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Пятиугольник, нарисованный Turtle

storony = 5, dlina = 75 → правильный пятиугольник

Шестиугольник (storony = 6)

shestiugolnik.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=300, height=300)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(3)

storony = 6
dlina = 65
ugol = 360 / storony

for _ in range(storony):
    artist.forward(dlina)
    artist.right(ugol)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Шестиугольник, нарисованный Turtle

storony = 6, dlina = 65 → правильный шестиугольник

Восьмиугольник (storony = 8)

vosmiugolnik.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=300, height=300)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(3)

storony = 8
dlina = 55
ugol = 360 / storony

for _ in range(storony):
    artist.forward(dlina)
    artist.right(ugol)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Восьмиугольник, нарисованный Turtle

storony = 8, dlina = 55 → правильный восьмиугольник

Мини-проект: студия автоматических многоугольников

★★ Самостоятельная задача

Студия многоугольников

Оформите программу выше так, чтобы `storony` запрашивалось у пользователя через `input()` (не забудьте `int()`). Проверьте на нескольких значениях — включая большие, вроде 30 или 50, — и посмотрите, во что превращается фигура.

Практика: студия автоматических многоугольников

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/10-06/index.html>)

Мини-проект — автоматически рисуем мандалу

Та же мандала, что и в главе 6, — теперь по стадиям, короче и понятнее благодаря `range()` с тремя аргументами.

В главе 6 мандала рисовалась циклом `while` с ручным увеличением угла. Теперь распишем её по стадиям — от одного лепестка до полного узора — и посмотрим на реальный результат каждой стадии.

Стадия 1 — один мотив

`circle(60)` без параметра `extent` рисует полную окружность и возвращает черепашку точно в исходную точку и с исходным направлением:

```
mandala_stadiya_1.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=300, height=300)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(2)

artist.circle(60)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Один круг мандалы

Один мотив — просто circle(60)

Стадия 2 — четыре мотива

```
mandala_stadiya_2.py
```

```
for _ in range(4):
    artist.circle(60)
    artist.left(90)
```

```
mandala_stadiya_2.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=300, height=300)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(2)

for _ in range(4):
    artist.circle(60)
    artist.left(90)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Четыре круга мандалы

4 мотива, повёрнутых на 90° друг относительно друга

Стадия 3 — двенадцать мотивов

```
mandala_stadiya_3.py
```

```
for _ in range(12):
    artist.circle(60)
    artist.left(30)
```



```
mandala_stadiya_3.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=300, height=300)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(1)

for _ in range(12):
    artist.circle(60)
    artist.left(30)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Двенадцать кругов мандалы

12 мотивов, повернутых на 30° — уже настоящий узор

Стадия 4 — полная мандала

```
mandala_polnaya.py
```

```
cvieta = ['#5B24F9', '#DB2777', '#059669']
for i in range(24):
    artist.pencolor(cvieta[i % 3])
    artist.circle(80)
    artist.left(15)
```

```
mandala_polnaya.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=340, height=340)
artist = turtle.Turtle()
artist.speed(0)
artist.pensize(1)

cvieta = ["#5B24F9", "#DB2777", "#059669"]
for i in range(24):
    artist.pencolor(cvieta[i % 3])
    artist.circle(80)
    artist.left(15)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Полная разноцветная мандала

24 мотива с чередованием трёх цветов через $i \% 3$

range() с тремя аргументами — то же, что и ручной **while**

`for ugol in range(0, 360, shag_ugla):` генерирует именно те же числа, что мы получали вручную в главе 6: `while ugol < 360: ... ugol += shag_ugla`. Цикл `for` просто делает это короче и без риска забыть увеличить счётчик.

`cvieta[i % 3]` — остаток от деления как «зацикленный» индекс

`i % 3` даёт 0, 1, 2, 0, 1, 2, ... — какое бы большое ни было `i`, результат остатка всегда укладывается в диапазон индексов списка из 3 цветов. Этот приём — циклический перебор конечного набора значений — встречается очень часто.

★★ Самостоятельная задача

Своя мандала

Поменяйте количество мотивов (24 → 36), радиус (80 → 50) и список цветов — соберите собственный узор.

Практика: мандала через `for` + `range()`, по стадиям

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/10-07/index.html>)

Случайные узоры Turtle

Каждая итерация цикла может сделать своё, заранее неизвестное движение — а `seed()` делает случайный результат воспроизводимым.

Циклы и случайность

Модуль `random` из главы 9 отлично сочетается с циклами — каждая итерация может принимать своё, заранее неизвестное решение. Чтобы результат можно было воспроизвести и проверить, в примерах ниже используется `random.seed(...)` — она делает «случайную» последовательность одинаковой при каждом запуске.

Случайное блуждание

sluchajnoe_bluzhdanie.py

```
import random
random.seed(7)

for _ in range(100):
    artist.setheading(random.randint(0, 360))
    artist.forward(10)
```

sluchajnoe_bluzhdanie.py

```
import turtle
import random

random.seed(7)
screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(2)

for _ in range(100):
    artist.setheading(random.randint(0, 360))
    artist.forward(10)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Траектория случайного блуждания

100 шагов в случайном направлении, seed(7) — при повторном запуске получится тот же путь

seed() — не про случайность фигуры, а про воспроизводимость

Без `random.seed(...)` результат был бы каждый раз новым — что интересно для творчества, но неудобно для практики, где важно проверить конкретный, повторяемый результат.

Звёздное поле

`zvezdnoe_pole.py`

```
import random
random.seed(3)

for _ in range(60):
    x = random.randint(-190, 190)
    y = random.randint(-190, 190)
    artist.goto(x, y)
    artist.dot(6, "white")
```

```
zvezdnoe_pole.py
```

```
import turtle
import random

random.seed(3)
screen = turtle.Screen()
screen.setup(width=420, height=420)
screen.bgcolor("#0D0230")
artist = turtle.Turtle()
artist.speed(0)
artist.hideturtle()
artist.penup()

for _ in range(60):
    x = random.randint(-190, 190)
    y = random.randint(-190, 190)
    artist.goto(x, y)
    artist.dot(6, "white")

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Звёздное поле из точек

60 точек в случайных координатах — тёмный фон и dot() вместо линий

★★ Самостоятельная задача

Своё случайное блуждание

Поменяйте seed на другое число и количество шагов — сравните, насколько по-разному выглядит путь.

Практика: случайное блуждание и звёздное поле

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/10-19/index.html) → (<https://www.cartesianschool.org/practice/10-19/index.html>)

Сетка фигур

Тот же паттерн «строки × столбцы», что и в таблице умножения — только теперь он рисует, а не печатает числа.

Вложенные циклы как настоящая графика

Мы уже видели вложенные циклы на числах (таблица умножения) и на тексте (буквы внутри слов). Тот же самый паттерн «строки × столбцы» — основа для сеточного искусства:

setka_figur.py

```
shag = 60
for row in range(5):
    for col in range(5):
        x = -140 + col * shag
        y = -140 + row * shag
        artist.goto(x, y)
        artist.dot(16, '#5B24F9')
```

setka_figur.py

```

import turtle

screen = turtle.Screen()
screen.setup(width=380, height=380)
artist = turtle.Turtle()
artist.speed(0)
artist.hideturtle()
artist.penup()

shag = 60
for row in range(5):
    for col in range(5):
        x = -140 + col * shag
        y = -140 + row * shag
        artist.goto(x, y)
        artist.dot(16, "#5B24F9")

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Сетка из точек 5 на 5

$5 \times 5 = 25$ точек — внешний цикл по строкам,
внутренний по столбцам

Тот же формула, что и в таблице умножения

row и col снова пробегают весь диапазон независимо друг от друга — общее число нарисованных точек равно $5 * 5 = 25$, ровно как мы считали формулой «строки $\times$ столбцы» на странице про вложенные циклы.

★★ Самостоятельная задача

Сетка фигур вместо точек

Замените `artist.dot(...)` на маленький квадрат (ещё один, третий, вложенный цикл на 4 стороны) — получится сетка из маленьких квадратов вместо точек.

Практика: сетка фигур на вложенных циклах

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/10-20/index.html>)

Мини-проект — спирали из дуг

Завершаем главу узором, который меняется на каждом шаге цикла, — и подводим итоги.

Финальный мини-проект главы: спираль из дуг — каждая следующая дуга немного больше предыдущей.

```
spirali_iz_dug.py
```

```
radius = 5

for _ in range(60):
    artist.circle(radius, 90)    # дуга в четверть
    окружности                  # окружности
    radius += 3                  # каждый раз чуть
    больше
```

```
spirali_iz_dug.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(2)

radius = 5
for _ in range(60):
    artist.circle(radius, 90)
    radius += 3

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Спираль из растущих дуг

60 дуг, радиус растёт с 5 на 3 каждый шаг —
раскручивающаяся спираль

Изменение переменной внутри цикла — обычное дело

В отличие от предыдущих примеров, здесь `radius` меняется *на каждом шаге* цикла — это и создаёт эффект нарастающей спирали, а не повторяющегося узора.

break в Turtle — остановиться, не дойдя до конца

`break` прекрасно работает и внутри циклов, рисующих графику — например, чтобы не выйти за границу экрана:

```
break_v_turtle.py
```

```
dlina = 10
for _ in range(50):
    if artist.xcor() > 150:
        break
    artist.forward(dlina)
    artist.left(90)
    dlina += 6
```

```
break_v_turtle.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(3)

dlina = 10
for _ in range(50):
    if artist.xcor() > 150:
        break
    artist.forward(dlina)
    artist.left(90)
    dlina += 6

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Квадратная спираль, обрезанная break

Квадратная спираль обрывается посередине последнего витка — как только `xcor()` превысил 150, `break` остановил цикл

50 запланировано, но сработало меньше

Цикл был готов выполниться 50 раз, но фактически завершился раньше — как только условие `artist.xcor() > 150` стало истинным. Именно поэтому последний отрезок на картинке короче, чем должен был бы быть полный виток — `break` прервал рисование строго посередине шага.

★★ Самостоятельная задача

Спираль из квадратов

Замените дугу на маленький квадрат (цикл на 4 стороны) внутри внешнего цикла — получится спираль из уменьшающихся или увеличивающихся квадратов.

Практика: спирали из дуг

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/10-08/index.html>)

Практика: break в Turtle — квадратная спираль

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/10-21/index.html>)

Итоги главы

Инструментарий циклов — на будущее

Итоговая шпаргалка главы 10

Число
повторов
известно
заранее

→ `for ... in range(n)`

Перебор готовой
последовательности
(строка, список)

→ `for элемент in последовательность`

Нужен и
индекс,
и
значение

→ `for i, значение in enumerate(...)`

Число повторов
заранее неизвестно

→ `while условие`

Условие
остановки
удобнее
проверить в
середине тела

→ `while True + break`

Нужно узнать, был
ли break

→ `for/while ... else`

Нужно пропустить один
шаг, не выходя из цикла

→ `continue`

Не жёсткие правила — ориентиры, которые становятся привычкой с практикой

Что мы узнали в этой главе

- Повторение — третья структура алгоритма после последовательности и ветвления (глава 9).
- `for ... in range(n)` повторяет блок кода заданное число раз — цикл каждый раз спрашивает «есть следующий элемент?», а не выполняет тело `N` раз «по волшебству».
- `range()` не список, а генератор чисел; его `stop` не включается — та же логика, что и у срезов строк из главы 8.
- `enumerate()` даёт и индекс, и значение сразу; счётчик и накопитель — два базовых паттерна состояния внутри цикла.
- `while` повторяет действие, пока условие остаётся истинным — используется, когда число повторов заранее неизвестно.
- `break` прерывает цикл полностью; `continue` пропускает текущий шаг; необязательный `else` у цикла срабатывает только без `break`.
- Циклы можно вкладывать друг в друга — тогда внутренний цикл выполняется полностью на каждом шаге внешнего, а общее число итераций перемножается.
- Большинство ошибок циклов укладываются в десяток типовых сценариев — таблица трассировки, построенная вручную, помогает найти любой из них.
- Все фигуры из глав 6–7, нарисованные вручную, теперь можно записать в 2–4 строки с циклом — и мы увидели их реальные, по-настоящему выполненные результаты.

Что дальше

Мы уже видели циклы, которые работают со списками чисел — `[4, 8, 15, 16, 23, 42]`. В следующей главе разберём подробно, как устроены сами коллекции данных — списки, кортежи, множества и словари — то, ради чего циклы в первую очередь и существуют в реальных программах.

Очень много информации!

Списки, кортежи, множества и словари — четыре основных способа хранить сразу много данных. Не список методов, а разбор того, зачем нужна каждая структура, как они устроены внутри (ссылки, изменяемость, копирование) и как выбирать между ними.

11.1 Зачем хранить много значений. Карта коллекций

11.2 Списки: основы

11.3 Срезы списков

11.4 Изменяем список: mutable vs immutable

11.5 append, extend, insert

11.6 remove, pop, clear, del

11.7 Мощные операции со списками

11.8 Списки и циклы

11.9 Ссылки, aliasing, == и is

11.10 Копирование списков и shallow copy

11.11 Ещё больше о списках: вложенные списки

11.12 Мини-проект — разноцветная звезда

11.13 Кортежи

11.14 `zip()` и распаковка

11.15 Множества

11.16 Операции множеств: Венн-диаграммы, хешируемость

11.17 Словари

11.18 Методы словарей

11.19 Вложенные структуры

11.20 Преобразования и comprehensions

11.21 Мини-проект — бесконечные цвета

11.22 Как выбрать правильную структуру

11.23 Отладка коллекций: 14 типичных ошибок

11.24 Мини-проект — частота слов

11.25 Мини-проекты с коллекциями

11.26 Мини-проект — перестановка имени и итоги

Итоги главы

Зачем хранить много значений

Прежде чем список методов — вопрос «зачем»: одна переменная хранит одно значение, а коллекция хранит их сразу много под одним именем.

Одна переменная — одно значение. А если их пятьсот?

До сих пор каждая переменная хранила одно значение. Представьте, что нужно сохранить оценки пяти учеников:

```
pyat_peremennyh.py
```

```
score_1 = 95
score_2 = 82
score_3 = 91
score_4 = 77
score_5 = 88
```

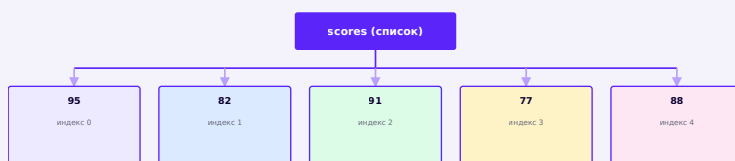
Работает. А теперь представьте, что учеников не пять, а **пятьсот**. Заводить `score_1 ... score_500` вручную — не просто долго, а по сути невозможно поддерживать: нельзя пройти по ним циклом, нельзя посчитать среднее одной строкой, нельзя добавить оценку 501-го ученика без правки кода.

Коллекция: одно имя — много значений

Для этого в Python есть **коллекции** — типы данных, которые хранят сразу много значений под одним именем:

```
odin_spisok.py
```

```
scores = [95, 82, 91, 77, 88]
print(scores)
```



Одно имя `scores` указывает на ОДИН объект-коллекцию, который хранит ссылки на пять значений

Коллекция организует ссылки на объекты, а не «складывает предметы в коробку»

Мы уже видели в главе 3, что переменная — это имя, указывающее на объект. Коллекция устроена похоже: `scores` указывает на один объект-список, а список хранит ссылки на несколько отдельных значений. Подробности внутреннего устройства CPython нам сейчас не нужны — этой картинки достаточно.

Карта коллекций Python

В Python четыре основных встроенных коллекции. Они делятся на три семейства по тому, **как** устроен доступ к значениям:

● МНОГО ЗНАЧЕНИЙ

- SEQUENCE — по позиции (индексу)
 - `list` — можно изменять
 - `tuple` — изменить нельзя
- SET — только уникальные значения

set — можно изменять

● MAPPING — по ключу

● dict — ключ → значение

Три семейства коллекций Python:
последовательность (по позиции), множество
(уникальность), отображение (по ключу)

Четыре встроенные коллекции — коротко

LIST

упорядочен

изменяем

повторы разрешены

```
[1, 2, 2]
```

TUPLE

упорядочен

неизменяем

повторы разрешены

```
(1, 2, 2)
```

SET

без индексов
изменяем
только уникальные
`{1, 2}`

DICT

по ключу
изменяем
ключи уникальны
`{"a": 1}`

Как выбрать? Пока — ориентир, не закон

Полный разбор выбора структуры будет в §11.22, но уже сейчас полезно иметь общий ориентир:

Какую коллекцию выбрать? (первый ориентир)

Нужна пара «ключ → значение»? → **dict**

Нужны только уникальные значения, порядок не важен? → **set**

Нужен порядок, и придётся
часто менять содержимое?

→ **list**

Нужен порядок, но значения
фиксированы раз и навсегда?

→ **tuple**

Это правило большого пальца, а не математический
закон — из него будут исключения

Практика: превращаем пять переменных в один список

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-11/index.html>)

Храним больше одного значения

Списки — самая универсальная коллекция Python:
упорядоченный набор значений в одной переменной.

Списки

Список (`list`) — упорядоченная коллекция значений в
квадратных скобках через запятую:

`spiski.py`

```
fruits = ["яблоко", "банан", "вишня"]  
print(fruits)  
print(type(fruits))
```

Список может хранить значения разных типов одновременно, хотя на практике чаще хранят значения одного вида — так код проще читать:

```
smeshannyj_spisok.py
```

```
smeshannyj = ["Cartesian", 5, 3.14, True]
print(smeshannyj)
```

Список любого типа — не ошибка, а выбор

Python не запрещает смешивать типы в одном списке. Но если элементы списка означают разные вещи (имя, число, флаг), почти всегда понятнее использовать словарь (§11.17) — список удобнее, когда все элементы одного смысла: имена, оценки, координаты.

Длина списка: len()

Как и у строк (глава 8), у списка есть длина:

```
dlina_spiska.py
```

```
numbers = [10, 20, 30]
print(len(numbers))    # 3
```

`len(numbers) = 3` → допустимые индексы: от 0 до `len-1 = 2`

Доступ к значениям списка по индексу

Как и у строк, у элементов списка есть индексы, начиная с нуля — с тем же правилом отрицательных индексов «с конца»:


```
names = ["Anna", "Oleg", "Maria", "Leo"]
```

```
dostup_k_spisku.py
```

```
names = ["Anna", "Oleg", "Maria", "Leo"]
print(names[0])      # Anna
print(names[-1])     # Leo — последний элемент
```

Практика: создание списков и доступ по индексу

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-01/index.html>)

Делаем срез списка!

Срезы списков работают точно так же, как срезы строк,
— уже знакомая логика в новом контексте.

Срезы списков работают точно так же, как срезы строк
из главы 8: `start` входит в срез, `stop` — нет.

```
chisla[1:3] → [20, 30] — граница 1 включена, граница  
3 нет
```

srezy_spiskov.py

```
chisla = [10, 20, 30, 40, 50]
print(chisla[1:3])    # [20, 30]
print(chisla[:2])     # [10, 20]
print(chisla[2:])     # [30, 40, 50]
print(chisla[::-1])   # [50, 40, 30, 20, 10] –
развёрнутый список
```

chisla[:2]

chisla[2:]

Срез списка — это новый список

Как и срез строки, срез списка возвращает **новый** список, не изменяя исходный. Это важно запомнить — пригодится в §11.10 «Копирование списков».

Практика: срезы списков

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/11-02/index.html) (https://www.cartesianschool.org/practice/11-02/index.html)

Изменяем список

Первый по-настоящему изменяемый тип данных: элемент списка можно заменить, не создавая новый объект.

Список можно изменить «на месте»

Это фундаментальное отличие списка от строки. Присвоим новое значение элементу по индексу:

```
izmenenie_elementa.py
```

```
numbers = [10, 20, 30]
numbers[1] = 999
print(numbers)    # [10, 999, 30]
```

До: `numbers[1]` ещё 20

После: `numbers[1] = 999` — тот же объект

Это тот же самый объект, а не новый список

После `numbers[1] = 999` список не пересоздается — изменяется содержимое того же объекта. Это станет важно в §11.9, когда у одного списка будет две ссылки-переменные сразу.

Mutable vs immutable

Мы уже встречали неизменяемые типы — числа и строки. Список — первый по-настоящему **изменяемый (mutable)** тип, который мы изучаем:

Тип	Можно ли заменить часть значения «на месте»?
<code>str</code>	Нет — <code>name[0] = "P"</code> вызывает <code>TypeError</code> , нужна новая строка
<code>tuple</code>	Нет — кортежи тоже неизменяемы (§11.13)
<code>list</code>	Да — <code>numbers[1] = 999</code> меняет тот же объект

Изменяемость — свойство типа, а не конкретного значения

Не бывает «изменяемых строк» и «неизменяемых списков» в зависимости от содержимого — изменяемость целиком определяется **типом** объекта: все `list` изменяемы, все `tuple` и `str` — нет.

Практика: изменяем элемент списка на месте

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-12/index.html>)

append, extend, insert

Три способа добавить что-то в список — и одна из самых частых ловушек новичка: `append()` и `extend()` делают совсем разные вещи.

append() — добавить один элемент

До: `numbers.append(3)`

После: `[1, 2, 3]`

`append.py`

```
numbers = [1, 2]
numbers.append(3)
print(numbers)    # [1, 2, 3]
```

`append()` всегда добавляет **ровно один** новый элемент — даже если этот элемент сам является списком.

append() vs extend() — важная ловушка

```
append_vs_extend.py
```

```
a = [1, 2]
a.append([3, 4])
print(a)    # [1, 2, [3, 4]] — список внутри списка!

b = [1, 2]
b.extend([3, 4])
print(b)    # [1, 2, 3, 4]
```

`append([3, 4])` → один НОВЫЙ элемент-список

`extend([3, 4])` → два элемента ИЗ списка

append() добавляет один объект, extend() — элементы из итерируемого

`append(x)` кладёт `x` внутрь как ОДИН новый элемент, каким бы он ни был. `extend(iterable)` разворачивает переданный список (или любой другой итерируемый объект) и добавляет каждый его элемент по отдельности. Перепутать их — классическая ошибка новичка.

insert() — вставить по индексу

insert.py

```
fruits = ["яблоко", "банан", "вишня"]
fruits.insert(1, "манго")
print(fruits)    # ["яблоко", "манго", "банан",
                 "вишня"]
```

До: insert(1, 'манго')

После: остальные элементы сдвинулись вправо

Элементы начиная с указанного индекса как будто «раздвигаются», освобождая место для нового значения — а их индексы после вставки увеличиваются на единицу.

Практика: append, extend, insert

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/11-13/index.html) → (https://www.cartesianschool.org/practice/11-13/index.html)

remove, pop, clear, del

Четыре способа что-то убрать из списка — у каждого своя роль, и путать их не стоит.

remove() — удалить по значению

remove.py

```
names = ["Anna", "Oleg", "Maria"]
names.remove("Oleg")
print(names)    # ["Anna", "Maria"]
```

pop() — удалить по позиции и вернуть значение

pop.py

```
names = ["Anna", "Oleg", "Maria"]
removed = names.pop(1)
print(names, removed)    # ["Anna", "Maria"] Oleg
```

	remove(value)	pop(index)
Удаляет по	значению	позиции (индексу)
Что возвращает	ничего (None)	удалённое значение
Если аргумент не подходит	ValueError, если значения нет	IndexError, если индекса нет

pop() без аргумента — удаляет последний элемент

`names.pop()` без индекса убирает и возвращает **последний** элемент списка. Это пригодится, если использовать список как стек — подробнее о стеках будет в следующих главах.

clear() и del — два разных способа «опустошить»

clear_del.py

```
items = [1, 2, 3]
items.clear()
print(items)           # [] — тот же список, но пустой

other = [1, 2, 3]
del other[1]
print(other)           # [1, 3] — удалили один элемент по
                        # индексу

third = [1, 2, 3]
del third               # удалили саму переменную
```

Команда	Что происходит
<code>items.clear()</code>	тот же объект-список остаётся, но становится пустым
<code>del items[i]</code>	удаляет элемент с индексом <code>i</code> (сам список остаётся)
<code>del items</code>	удаляет саму переменную-имя (глава 3) — после этого <code>items</code> не существует

После `del items` имя `items` больше не существует

`print(items)` после `del items` вызовет `NameError` — это то же самое поведение `del`, что мы видели в главе 3 для обычных переменных, только теперь применённое к списку.

Практика: `remove`, `pop`, `clear`, `del`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-14/index.html>)

Мощные операции со списками!

Списки изменяемы — у них есть целый набор методов для добавления, удаления, поиска и сортировки.

У списков (в отличие от строк) есть методы, которые **изменяют сам список** — списки, в отличие от строк, изменяемы (§11.4).

Копирование и добавление

`kopirovanie.py`

```
original = [1, 2, 3]
kopiya = original.copy()
kopiya.append(4)
print(original) # [1, 2, 3] — не изменился
print(kopiya)   # [1, 2, 3, 4]
```

kopiya = original — это не копия!

`kopiya = original` создаёт вторую переменную, указывающую **на тот же самый** список — изменение через одну переменную видно и через другую. Это называется **aliasing** — подробно разберём в §11.9. Чтобы получить настоящую независимую копию, нужен `.copy()` (или срез `original[:]`) — подробности и важный нюанс с вложенными списками в §11.10.

Подсчёт и очистка

podschet.py

```
chisla = [1, 2, 2, 3, 2]
print(chisla.count(2))
# 3 — сколько раз встречается 2
chisla.clear()
print(chisla)           # [] — список пуст
```

Конкатенация

konkatenaciya_spiskov.py

```
a = [1, 2]
b = [3, 4]
print(a + b)           # [1, 2, 3, 4]
```

Поиск и проверка наличия: `in`, `index()`, `count()`

poisk.py

```
fruits = ["яблоко", "банан", "вишня"]
print("банан" in fruits)           # True
print(fruits.index("вишня"))      # 2 — индекс
                                  элемента
```

Проверка `in` — то же логическое выражение из главы 9, только теперь слева не число, а коллекция:

membership_primer.py

```
allowed_users = ["anna", "oleg", "maria"]
username = "oleg"
if username in allowed_users:
    print("Доступ разрешён")
```

`index()` вызывает `ValueError`, если значения нет

Безопасный порядок — сначала проверить `in`, потом искать индекс: `if value in items: position = items.index(value)`.

Добавление и удаление элементов

Коротко — подробный разбор с визуальными диаграммами в §11.5 и §11.6:

dobavlenie_udalenie.py

```

fruits = ["яблоко", "банан"]
fruits.append("вишня")      # добавить в конец
fruits.insert(0, "манго")   # вставить по индексу
print(fruits)

fruits.remove("банан")      # удалить по значению
last = fruits.pop()        # удалить и вернуть
                             последний элемент
print(fruits, last)

```

sort() мутирует и возвращает None — классическая ловушка

razvorot_sortirovka.py

```

chisla = [3, 1, 4, 1, 5]
chisla.sort()
print(chisla)      # [1, 1, 3, 4, 5]
chisla.reverse()
print(chisla)      # [5, 4, 3, 1, 1]

```

sort_none_lovushka.py

```

chisla = [3, 1, 2]
chisla = chisla.sort()
print(chisla)      # None — а не отсортированный список!

```

Никогда не пишите `x = x.sort()`

`.sort()` сортирует список **на месте** и возвращает `None` — это распространённый способ Python показать «я изменил объект, а не создал новый» (тот же принцип, что и у `.append()`). Присваивание результата обратно в переменную стирает список, заменяя его на `None`.

sorted() — та же задача, но без мутации

	list.sort()	sorted(list)
Изменяет исходный список?	Да — мутирует	Нет — не трогает
Что возвращает?	None	новый отсортированный список

sorted_primer.py

```
chisla = [3, 1, 4, 1, 5]
novyj = sorted(chisla)
print(chisla)    # [3, 1, 4, 1, 5] — не изменился
print(novyj)     # [1, 1, 3, 4, 5]
```

sorted() умеет и `key` — по какому свойству сравнивать элементы:

sorted_key.py

```
names = ["Bob", "Alexandra", "Li"]
print(sorted(names, key=len))  # ['Li', 'Bob', 'Alexandra']
```

Практика: операции со списками

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-03/index.html>)

Списки и циклы

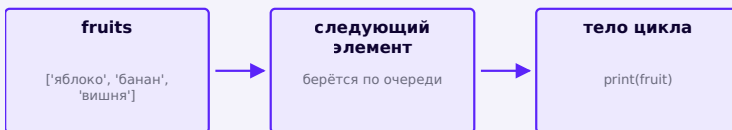
Глава 10 наконец окупается: перебираем список циклом, вместо ручной работы с индексами.

Перебор списка циклом `for`

Глава 10 наконец окупается — перебирать список циклом можно, обращаясь сразу к значениям, без ручной работы с индексами:

perebor_spiska.py

```
fruits = ["яблоко", "банан", "вишня"]
for fruit in fruits:
    print(fruit)
```



`for` в цикле сам достаёт следующий элемент — вручную считать индексы не нужно

`enumerate()` — когда нужен и индекс, и значение

enumerate_spiski.py

```
names = ["Anna", "Oleg", "Maria"]
for index, name in enumerate(names):
    print(index, name)
```

index	name
0	Anna
1	Oleg

index	name
2	Maria

Коллекция + цикл + условие

Один из самых частых паттернов в программировании — пройти по коллекции и что-то сделать только с подходящими значениями:

```
spisok_cikl_uslovie.py
```

```
scores = [95, 82, 91, 58, 77]
for score in scores:
    if score >= 90:
        print(score, "— отлично!")
```

Коллекция + цикл + условие — фундаментальная связка

Этот паттерн — «перебрать коллекцию и отфильтровать по условию» — встретится ещё десятки раз: поиск (§11.19), фильтрация, подсчёт частоты слов (§11.24). Он объединяет главу 9 (условия) и главу 10 (циклы) с этой главой (коллекции).

Практика: списки, enumerate() и фильтрация циклом

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/11-15/index.html) (<https://www.cartesianschool.org/practice/11-15/index.html>)

Ссылки, aliasing, == и is

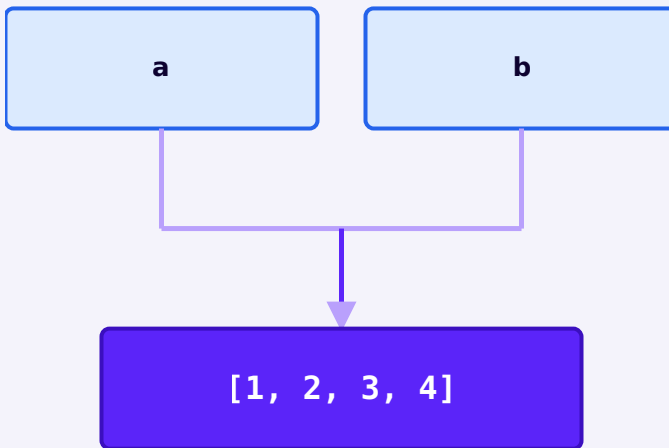
Два имени могут указывать на один и тот же список — и это не баг, а фундаментальная особенность того, как Python работает с изменяемыми объектами.

Aliasing: два имени — один и тот же объект

В §11.7 мы уже видели предупреждение: `b = a` не создаёт копию списка. Разберём подробно, что происходит на самом деле.

```
aliasing.py
```

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)    # [1, 2, 3, 4] — тоже изменился!
print(b)    # [1, 2, 3, 4]
```



а и b — это ДВА ИМЕНИ одного и того же объекта-списка, а не два списка

Мы не создали второй список — мы создали второе имя

`b = a` не копирует данные. Python просто говорит: «теперь имя `b` указывает туда же, куда и `a`». Формальный термин для этого — **aliasing** (совместная ссылка). Изменение через любое из двух имён видно через оба, потому что список-объект на самом деле один.

`==` сравнивает содержимое, `is` — сравнивает объект

Отличная возможность закрепить материал главы 9 на новом материале:

eq_vs_is.py

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)    # True – содержимое одинаковое
print(a is b)    # False – это два РАЗНЫХ объекта

c = a
print(a is c)    # True – c это то же самое имя-alias для a
```

a



[1, 2, 3]

b



[1, 2, 3]

`a` и `b` — два разных объекта с одинаковым содержимым: `a == b`, но не `a is b`

Оператор	Что проверяет	<code>a = [1,2,3]; b = [1,2,3]</code>
<code>==</code>	одинаковое ли содержимое	<code>True</code>
<code>is</code>	один ли и тот же объект в памяти	<code>False</code>

★★ Самостоятельная задача

Предсказать результат

Дано: `x = [10, 20]`, `y = x`, `y.append(30)`. Не запуская код, ответьте: чему равен `x`? А если бы вместо `y = x` было `y = x.copy()`?

Практика: `aliasing`, `==` и `is`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-16/index.html>)

Копирование списков

Скопировать внешний список легко — а вот вложенные списки внутри него по умолчанию остаются общими. Разберём, почему и что с этим делать.

Три безопасных способа скопировать список

```
sposoby_kopirovaniya.py
```

```
original = [1, 2, 3]

kopiya_1 = original.copy()
kopiya_2 = list(original)
kopiya_3 = original[:]

kopiya_1.append(4)
print(original)    # [1, 2, 3] – не изменился
print(kopiya_1)    # [1, 2, 3, 4]
```

Все три способа создают новый **ВНЕШНИЙ** список. Но что, если элементы сами являются списками?

Поверхностная копия (shallow copy) — важный нюанс

```
shallow_copy_trap.py
```

```
original = [{"Anna", 10}, {"Bob", 20}]
kopiya = original.copy()

kopiya[0][1] = 999
print(original)    # [{"Anna", 999}, {"Bob", 20}] –
тоже изменился!
```

Два **РАЗНЫХ** внешних списка, но их элементы —
ОДНИ И ТЕ ЖЕ внутренние списки

.copy() копирует только один уровень вложенности. **.copy()** (и **list(...)**, и срез **[:]**) создают новый внешний список, но **не** копируют вложенные списки внутри него — они остаются общими. Это и называется **поверхностной копией (shallow copy)**. Изменение вложенного списка через копию видно и в оригинале.

copy.deepcopy() — когда нужна полностью независимая копия

deepcopy.py

```
import copy

original = [{"Anna", 10}, {"Bob", 20}]
polnaya_kopiya = copy.deepcopy(original)
polnaya_kopiya[0][1] = 999
print(original)    # [['Anna', 10], ['Bob', 20]] — не
изменился
```

deepcopy — не всегда правильный ответ по умолчанию

copy.deepcopy() рекурсивно пытается скопировать всю вложенную структуру целиком. Это решает проблему выше, но для больших или сложных структур глубокое копирование может быть заметно медленнее и не всегда нужно — часто достаточно скопировать только тот уровень, который реально будет меняться.

Практика: копирование списков и поверхностная копия

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/11-17/index.html\)](https://www.cartesianschool.org/practice/11-17/index.html)

Ещё больше интересного со списками!

Полезные встроенные функции, вложенные списки, известная ловушка с умножением вложенного списка — и более компактный способ строить новые списки.

Ещё несколько приёмов, которые часто пригождаются при работе со списками.

`len()`, `min()`, `max()`, `sum()`

`vstroennye_funkcii.py`

```
chisla = [4, 8, 15, 16, 23, 42]
print(len(chisla))    # 6 — количество элементов
print(min(chisla))    # 4
print(max(chisla))    # 42
print(sum(chisla))    # 108
```

Списки списков (вложенные списки) — матрица

Элементом списка может быть другой список — так получают таблицы (матрицы):

`vlozhennye_spiski.py`

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(matrix[0])      # [1, 2, 3] — первая строка
print(matrix[0][1])   # 2 — второй элемент первой строки
print(matrix[1][2])   # 6 — строка 1, столбец 2
```

	столбец 0	столбец 1	столбец 2
строка 0	1	2	3
строка 1	4	5	6
строка 2	7	8	9

`matrix[1][2] → 6` — сначала строка, потом столбец

perebor_matricy.py

```
for row in matrix:
    for value in row:
        print(value, end=' ')
    print()
```

Вложенный список + вложенный цикл — прямой перенос из главы 10

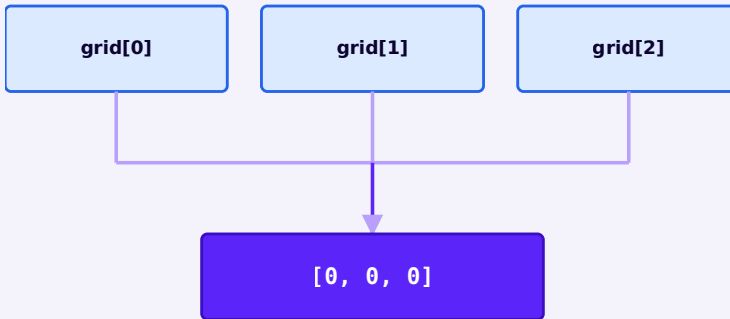
Внешний цикл идёт по строкам, внутренний — по значениям внутри строки. Это ровно тот же паттерн вложенных циклов, что и таблица умножения в главе 10, только теперь данные уже готовы в виде матрицы, а не вычисляются на лету.

Ловушка: `[[0] * 3] * 3`

Хочется создать сетку 3×3 из нулей одной строкой — но это частая и коварная ошибка:

```
lovushka_umnozheniya.py
```

```
grid = [[0] * 3] * 3
grid[0][0] = 1
print(grid)  # [[1, 0, 0], [1, 0, 0], [1, 0, 0]] —
              изменились ВСЕ строки!
```



`[[0]*3]*3` создаёт ОДИН внутренний список и трижды копирует на него ссылку

***3 повторяет ссылку, а не создаёт три независимых списка**

`[значение] * n` для неизменяемых значений (чисел, строк) работает безопасно. Но когда значение само является списком, `* n` просто копирует ссылку на один и тот же внутренний список `n` раз — ровно тот же принцип aliasing из §11.9. Правильный способ создать три НЕЗАВИСИМЫЕ строки — через comprehension:

pravilnaya_setka.py

```
grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 1
print(grid)  # [[1, 0, 0], [0, 0, 0], [0, 0, 0]] —
              только первая строка
```

Перебор списка циклом for

perebor_spiska_v4.py

```
fruits = ["яблоко", "банан", "вишня"]
for fruit in fruits:
    print(fruit)
```

Построение списка: цикл с `append()` → генератор списков

Классический подход

```
kvadraty = []
for n in range(1, 6):
    kvadraty.append(n
** 2)
print(kvadraty)
```

Современный Python (list comprehension)

```
kvadraty = [n ** 2 for
n in range(1, 6)]
print(kvadraty)
```

Что использовать сегодня: list comprehension (генератор списков) для простых случаев — он существует в Python с версии 2.0, так что это не вопрос новизны, а вопрос стиля: короче и, при некоторой практике, читается как одно предложение («квадрат n для каждого n из диапазона»). Для более сложной логики (с несколькими условиями или побочными эффектами) цикл с `append()` часто остаётся понятнее — это не всегда "хуже", выбирайте то, что легче прочитать. Подробный разбор comprehensions — в §11.20.

Практика: len/min/max/sum, вложенные списки, list comprehension

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-04/index.html>)

Мини-проект — автоматическая разноцветная звезда

Список цветов + цикл + Turtle — звезда, где каждый луч своего цвета.

Соединим списки с Turtle (главы 6–7): нарисуем звезду, где каждый луч — своего цвета из заранее заданного списка.

raznocvetnaya_zvezda.py

```
cveta = ["red", "orange", "yellow", "green", "blue"]

for cvet in cveta:
    artist.pencolor(cvet)
    artist.forward(150)
    artist.right(144) # угол пятиконечной звезды из
главы 6
```

Список любой длины — тот же код

Добавьте в `cveta` ещё пару значений — цикл `for cvet in cveta` сам подстроится под новую длину списка, без изменений остального кода.

★★ Самостоятельная задача**Случайные цвета**

Замените список фиксированных цветов на `random.choice()` из списка — чтобы цвет каждого луча выбирался случайно.

Практика: разноцветная звезда (списки + Turtle)

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/11-05/index.html>)

Кортежи

Почти список, но неизменяемый — и именно поэтому иногда более подходящий выбор.

Кортеж (tuple) — не «список в круглых скобках», а отдельная идея: иногда несколько значений логически образуют ОДНУ запись (координата, RGB-цвет), и хочется быть уверенным, что её случайно не изменят.

kortezhi.py

```
point = (10, 20)
print(point)
print(point[0])    # 10 — индексация работает как у
                   # списков
```

kortezh_nelzya_menyat.py

```
point = (10, 20)
point[0] = 99      # TypeError: 'tuple' object does not
                   # support item assignment
```

Кортеж — последовательность, как и список

len, индексация, срезы, перебор, in — всё это работает у кортежа так же, как у списка (§11.1–§11.2). Разница только одна, но принципиальная: кортеж нельзя изменить.

	list	tuple
Синтаксис	<code>[1, 2, 3]</code>	<code>(1, 2, 3)</code>
Изменяем?	да	нет
Индексация, срезы, len, in	да	да

Кортеж из одного элемента — нужна запятая

odinochnyj_kortezh.py

```
not_a_tuple = (42)
print(type(not_a_tuple))    # <class 'int'>

one = (42,)
print(type(one))            # <class 'tuple'>
```

Кортеж создаёт запятая, а не круглые скобки

`(42)` — это просто число 42 в скобках, как в математике. Кортежем его делает запятая: `(42,)`. Без запятой Python не поймёт, что вы хотели кортеж, и тихо создаст обычное число — без ошибки, что особенно коварно.

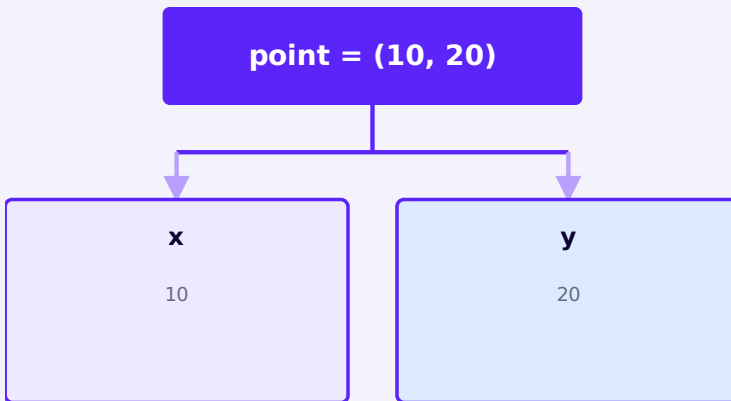
Packing и unpacking

packing.py

```
point = 10, 20    # скобки необязательны — Python
                  «упаковывает» в кортеж сам
print(point, type(point))
```

```
raspakovka.py
```

```
point = (10, 20)
x, y = point
print(x, y)    # 10 20
```



`x, y = point` — распаковка: число имён должно совпадать с числом значений

Мы уже пользовались распаковкой

`artist.position()` в модуле `turtle` возвращает именно такой кортеж `(x, y)` — можно сразу распаковать его в две переменные.

Классический трюк: обмен значений

swap.py

```
a = 1
b = 2
a, b = b, a
print(a, b)    # 2 1
```

Работает благодаря той же паре «packing + unpacking»: справа сначала упаковывается кортеж `(b, a)`, потом он распаковывается в `a, b` — без промежуточной переменной.

Звёздочная распаковка

star_unpacking.py

```
first, *middle, last = [1, 2, 3, 4, 5]
print(first)          # 1
print(middle)         # [2, 3, 4]
print(last)           # 5
```

Звёздочка собирает «всё остальное» в список

`*middle` забирает все значения, не разобранные другими именами, и упаковывает их в обычный список — даже если распаковывается кортеж.

Функции могут возвращать несколько значений сразу

divmod_primer.py

```
quotient, remainder = divmod(17, 5)
print(quotient, remainder)    # 3 2
```

`divmod()` из главы 4 на самом деле возвращает один кортеж `(3, 2)` — просто мы сразу распаковываем его в две переменные.

Нюанс: кортеж может содержать изменяемые объекты

`tuple_s_listom.py`

```
t = ([1, 2], [3, 4])
t[0].append(99)
print(t)  # ([1, 2, 99], [3, 4]) — сработало!
```

Неизменяемость кортежа — только про сами «ячейки»

`t[0] = [...]` (заменить ссылку на другой список) — запрещено. А вот `t[0].append(99)` (изменить сам список, на который кортеж ссылается) — разрешено: кортеж не даёт переставить, ЧТО он хранит по каждой позиции, но не запрещает изменяемым объектам внутри меняться.

Практика: кортежи и распаковка

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-06/index.html>)

zip() и распаковка

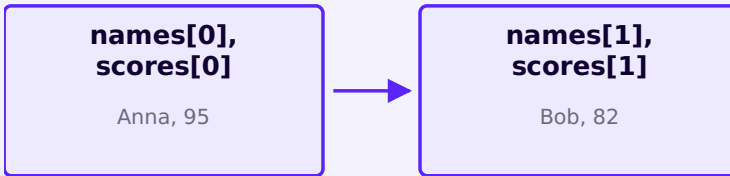
Один инструмент, который часто идёт рука об руку с кортежами и распаковкой: соединить два списка поэлементно.

zip() — соединяем два списка попарно

zip_primer.py

```
names = ["Anna", "Bob"]
scores = [95, 82]

for name, score in zip(names, scores):
    print(name, "-", score)
```



zip() берёт по одному элементу из каждого списка на одинаковой позиции

zip() останавливается на самом коротком

Если списки разной длины, zip() по умолчанию останавливается, как только заканчивается более короткий — «лишние» элементы длинного списка просто игнорируются, без ошибки.

zip() + dict — собрать словарь из двух списков

```
zip_v_dict.py
```

```
names = ["Anna", "Bob", "Maria"]
scores = [95, 82, 91]

result = dict(zip(names, scores))
print(result)    # {'Anna': 95, 'Bob': 82, 'Maria': 91}
```

Распаковка в циклах

Мы уже пользовались этим для словарей (§11.17) — тот же принцип, что и распаковка кортежа из §11.13, только применённая внутри цикла:

```
raspakovka_v_cikle.py
```

```
pary = [("Anna", 95), ("Bob", 82)]
for name, score in pary:
    print(f"{name}: {score}")
```

Практика: zip() и распаковка в циклах

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-18/index.html>)

Множества

Коллекция без повторов и без позиций — быстрый способ убрать дубликаты и сравнивать наборы значений.

Множество (`set`) — коллекция в фигурных скобках, у которой два ключевых отличия от списка: элементы в ней не повторяются, а обращения по числовому индексу нет — множество не позиционная последовательность.

mnozhestva.py

```
chisla = {1, 2, 2, 3, 3, 3}
print(chisla)    # {1, 2, 3} — повторы исчезли сами
```

Зачем нужны множества?

Два самых частых случая: быстро убрать повторы из списка и быстро проверить, есть ли значение в коллекции.

primeneniye_mnozhestv.py

```
spisok_s_povtorami = [1, 2, 2, 3, 1, 4]
unikalnye = set(spisok_s_povtorami)
print(unikalnye)    # {1, 2, 3, 4}
```

chlenstvo_mnozhestva.py

```
allowed = {"admin", "editor", "viewer"}
role = "editor"
if role in allowed:
    print("Доступ разрешён")
```

Множество не «случайно упорядочено» — оно просто не позиционное

Не стоит думать про `set` как про «список со случайным порядком». Правильная модель: у множества **нет** позиций вообще — писать `my_set[0]` нельзя (`TypeError`), и код никогда не должен полагаться на то, в каком порядке множество переберётся циклом.

Пустое множество — частая ловушка

pustoe_mnozhestvo.py

```
print(type({}))          # <class 'dict'> – это пустой
словарь!
print(type(set()))       # <class 'set'> – а это пустое
МНОЖЕСТВО
```

{} — это пустой словарь, а не пустое множество

Фигурные скобки без содержимого Python по умолчанию считает пустым dict (§11.17), потому что словари появились в языке как более базовый случай использования `{}`. Чтобы создать пустое множество, нужно явно написать `set()`.

add, remove, discard, pop, clear

metody_mnozhestv.py

```
tags = {"python", "beginner"}
tags.add("tutorial")
tags.discard("missing")  # без ошибки, даже если
элемент нет
print(tags)
```

Метод	Если элемента нет
<code>.remove(x)</code>	<code>KeyError</code>
<code>.discard(x)</code>	ничего не происходит, ошибки нет

pop() у множества удаляет ПРОИЗВОЛЬНЫЙ элемент

В отличие от `list.pop()`, у множества нет понятия «последний элемент» — `set.pop()` удаляет и возвращает какой-то один элемент, но заранее неизвестно какой.

Операции над множествами

```
operacii_mnozhestv.py
```

```
a = {1, 2, 3}
b = {2, 3, 4}

print(a | b)  # объединение: {1, 2, 3, 4}
print(a & b)  # пересечение: {2, 3}
print(a - b)  # разность: {1}
```

Полная алгебра множеств — в §11.16

Здесь — только знакомство. Венн-диаграммы для всех четырёх операций, подмножества и хешируемость разберём в следующем разделе.

Практика: множества и их операции

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-07/index.html>)

Операции множеств и хешируемость

Четыре операции алгебры множеств на Венн-диаграммах — и почему список нельзя положить внутрь множества.

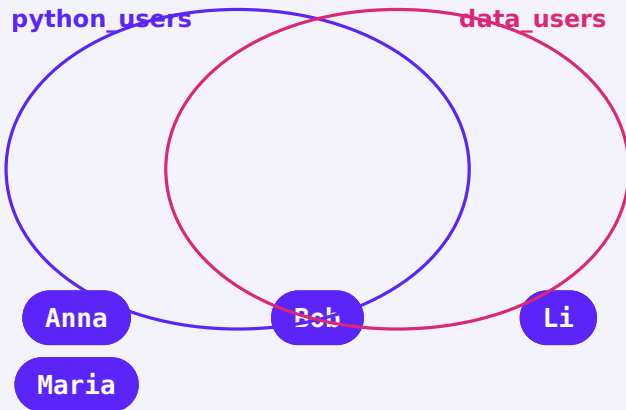
Множества как круги на диаграмме Венна

Две группы участников курсов — удобный пример, на котором видно все четыре операции сразу:

```
dve_gruppy.py
```

```
python_users = ['Anna', 'Bob', 'Maria']  
data_users = ['Bob', 'Li']
```

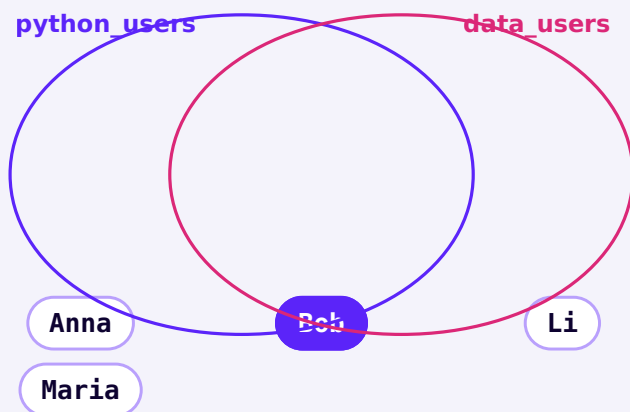
Объединение — union, |



```
union.py
```

```
print(set(python_users) | set(data_users))
```

Пересечение — intersection, &



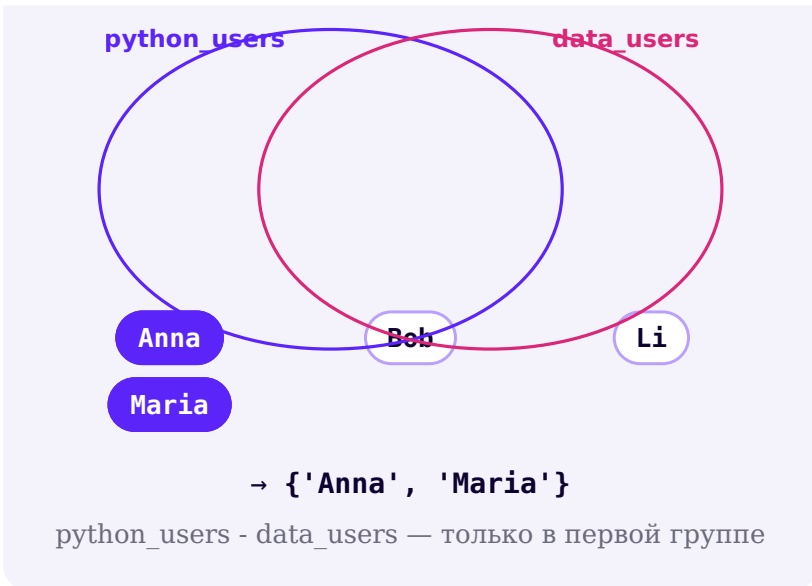
→ {'Bob'}

python_users & data_users — кто есть в ОБЕИХ группах

```
intersection.py
```

```
print(set(python_users) & set(data_users))
```

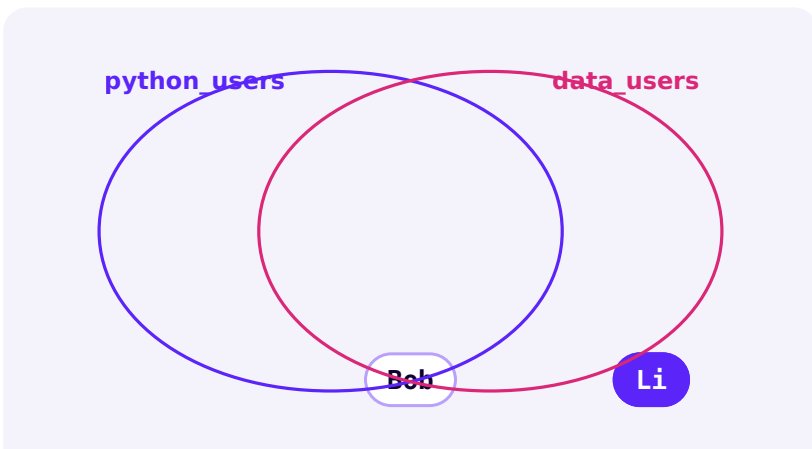
Разность — difference, -



```
difference.py
```

```
print(set(python_users) - set(data_users))
```

Симметричная разность — symmetric_difference,
[^]



Anna

Maria

→ {'Anna', 'Maria', 'Li'}

`python_users ^ data_users` — в одной группе, но не в обеих сразу

`symmetric_difference.py`

```
print(set(python_users) ^ set(data_users))
```

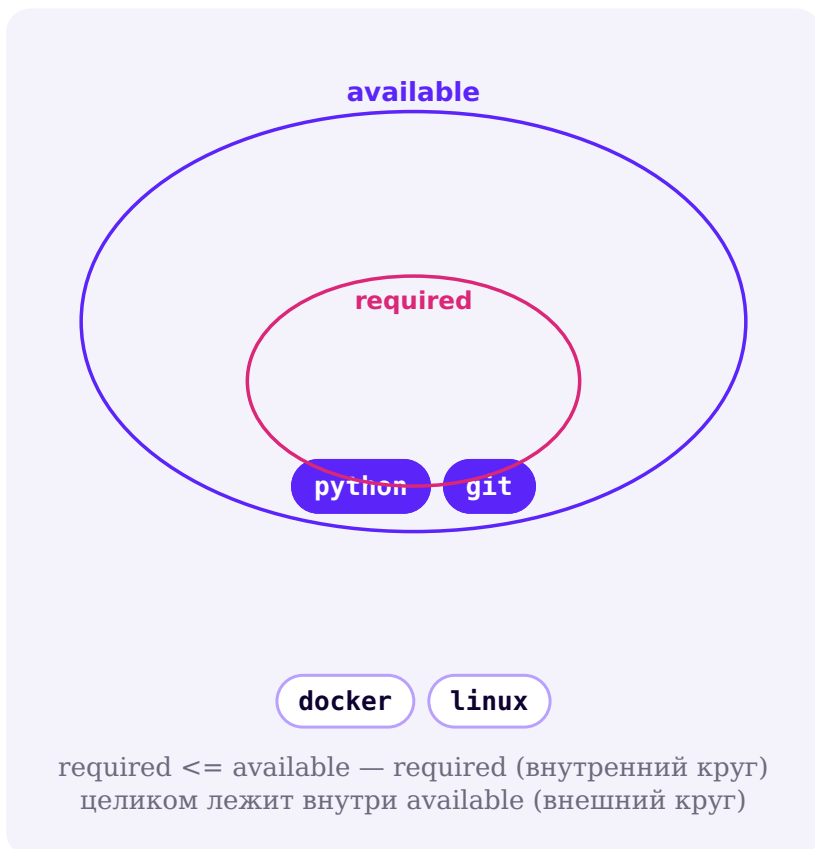
Операция	Оператор	Метод
объединение	<code>a b</code>	<code>a.union(b)</code>
пересечение	<code>a & b</code>	<code>a.intersection(b)</code>
разность	<code>a - b</code>	<code>a.difference(b)</code>
симметричная разность	<code>a ^ b</code>	<code>a.symmetric_difference(b)</code>

Подмножество и надмножество

`podmnozhestvo.py`

```
required = {"python", "git"}
available = {"python", "git", "docker", "linux"}

print(required <= available)      # True – required
ЦЕЛИКОМ входит в available
print(required.issubset(available)) # то же самое
```

Оператор / метод	Значение
a <= b / a.issubset(b)	все элементы a есть и в b
a >= b / a.issuperset(b)	все элементы b есть и в a
a.isdisjoint(b)	у a и b нет общих элементов вообще

Хешируемость: почему в множестве нельзя хранить список

```
hashability_error.py
```

```
bad = {[1, 2], [3, 4]}  
# TypeError: cannot use 'list' as a set element  
(unhashable type: 'list')
```

Элементы множества и ключи словаря должны быть достаточно «стабильными»

Python должен быть уверен, что значение внутри множества (или ключ словаря) не изменится незаметно, пока лежит там. Формальный термин для такой стабильности — **хешируемость (hashable)**. Изменяемые типы (список, словарь, множество) хешируемыми не бывают именно поэтому.

Что можно класть в множество / использовать как ключ словаря

ОБЫЧНО ХЕШИРУЕМЫ

```
int, float, str  
bool, bytes  
tuple — если все элементы тоже  
хешируемы  
frozenset
```

НЕ ХЕШИРУЕМЫ

```
list
dict
set
```

frozenset — неизменяемое множество

frozenset_primer.py

```
zamorozhennoe = frozenset({"python", "git"})
print(zamorozhennoe)
# zamorozhennoe.add("docker") # AttributeError –
# метода add() нет
```

Когда пригодится frozenset

frozenset нужен, когда множество само должно быть хешируемым — например, чтобы использовать его как элемент другого множества или как ключ словаря, что с обычным set невозможно.

Практика: алгебра множеств, подмножества, хешируемость

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/11-19/index.html) (<https://www.cartesianschool.org/practice/11-19/index.html>)

Словари

Самая гибкая встроенная коллекция Python — доступ по ключу вместо числового индекса.

От позиции к ключу

У списка доступ идёт по числовой позиции. У словаря — по **ключу**, обычно строке:

	list	dict
Модель доступа	позиция → значение	ключ → значение
Пример	<code>names[0]</code>	<code>student["name"]</code>

Словарь (`dict`) — самая гибкая коллекция: хранит пары «ключ — значение» вместо простых значений по порядку.

slovari.py

```
student = {  
    "name": "Cartesian",  
    "age": 12,  
    "city": "Москва",  
}  
print(student["name"])    # Cartesian
```

"name"**"Cartesian"****"age"****12****"city"****"Москва"**

Словарь — таблица «ключ → значение», а не пронумерованные ячейки

Повторяющийся ключ в литерале — побеждает последнее значение

Если в фигурных скобках дважды написать один и тот же ключ, ошибки не будет — останется только значение, написанное позже.

Добавление и изменение значений

`izmenenie_slovary.py`

```
student["age"] = 13          # изменить существующее  
                             # значение  
student["grade"] = "7 класс" # добавить новый ключ  
print(student)
```

Доступ по ключу: [] и get()

dostup_get.py

```
print(student["name"])          # Cartesian
print(student.get("email"))     # None — ключа нет, но
                                # ошибки тоже нет
print(student.get("email", "нет email")) # с явным
                                # значением по умолчанию
```

KeyError — обращение к несуществующему ключу

`student["phone"]`, если такого ключа нет, вызовет `KeyError`. Более безопасный способ — метод `student.get("phone")`, который просто вернёт `None` вместо ошибки. Квадратные скобки не «хуже» — они уместны, когда отсутствие ключа само по себе является ошибкой в программе.

Перебор словаря

perebor_slovara.py

```
for key, value in student.items():
    print(f"{key}: {value}")
```

Проверка наличия ключа

proverka_klyucha.py

```
print("name" in student)      # True
print("phone" in student)     # False
```

Практика: словари — создание, изменение, перебор

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/11-08/index.html\)](https://www.cartesianschool.org/practice/11-08/index.html)

Методы словарей

`pop`, `update`, `setdefault` и представления `keys/values/items` — инструменты, без которых не обходится ни один реальный словарь.

`pop()` и `popitem()`

`dict_pop.py`

```
student = {"name": "Cartesian", "age": 13, "grade":
"7 класс"}
age = student.pop("age")
print(student, age)
# {'name': 'Cartesian', 'grade': '7 класс'} 13
```

`dict_popitem.py`

```
last_pair = student.popitem()
print(last_pair) # ('grade', '7 класс') — последняя
добавленная пара
```

`pop()` у словаря — по ключу, не по позиции

В отличие от `list.pop(index)`, у словаря `.pop(key)` принимает КЛЮЧ, а не числовую позицию — позиций у словаря просто нет. Можно передать значение по умолчанию вторым аргументом, если ключа может не быть: `student.pop("phone", None)`.

update() — слить несколько пар сразу

dict_update.py

```
student = {"name": "Cartesian", "age": 13}
student.update({"age": 14, "city": "Москва"})
print(student)  # age заменился, city добавился
```

setdefault() — добавить значение, только если ключа ещё нет

setdefault.py

```
counts = {}
word = "python"
counts.setdefault(word, 0)
counts[word] += 1
print(counts)  # {'python': 1}
```

setdefault() = «если ключа нет — создай со значением по умолчанию»

`counts.setdefault(word, 0)` ничего не делает, если `word` уже есть в словаре, и создаёт запись со значением 0, если ключа не было — в обоих случаях после вызова можно смело писать `counts[word] += 1`. Полный алгоритм подсчёта частоты слов с таким подходом — в §11.24.

keys(), values(), items() — не обычные списки

```
keys_values_items.py
```

```
student = {"name": "Cartesian", "age": 13}
print(student.keys())      # dict_keys(['name', 'age'])
print(student.values())    # dict_values(['Cartesian',
13])
print(student.items())     # dict_items([('name',
'Cartesian'), ('age', 13)])
```

Это «живые» представления, а не списки

.keys(), .values() и .items() возвращают объекты-представления (dict_keys и т.д.) — они отражают словарь в реальном времени. Для перебора циклом это не важно, но если нужен именно список, оберните в `list(student.keys())`.

Порядок словаря

Современный Python сохраняет порядок вставки

Начиная с Python 3.7 порядок пар в словаре гарантированно совпадает с порядком, в котором их добавили — это официальная гарантия языка, а не случайность реализации. Но словарь остаётся **отображением** (доступ по ключу), а не позиционной коллекцией — не стоит писать код, который полагается на числовые позиции словаря.

Проверка `in` — только по ключам

membership_dict.py

```
student = {"name": "Cartesian", "age": 13}
print("name" in student)           # True — есть такой
КЛЮЧ
print("Cartesian" in student)      # False — это
значение, не ключ!
print("Cartesian" in student.values()) # True — а
вот так корректно проверить значение
```

`in` у словаря проверяет ключи, а не значения

Частая ошибка новичка: `value in my_dict` проверяет, есть ли `value` среди КЛЮЧЕЙ. Чтобы проверить значения, нужно явно написать `value in my_dict.values()`.

Ключи словаря тоже должны быть хешируемыми

hashable_keys.py

```
{["x", "y"]: 1}
# TypeError: cannot use 'list' as a dict key
(unhashable type: 'list')
```

Тот же принцип хешируемости из §11.16 — список нельзя использовать как ключ словаря по той же причине, по которой его нельзя положить в множество.

Практика: `pop`, `update`, `setdefault`, `keys/values/items`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-20/index.html>)

Вложенные структуры

Настоящие данные редко бывают плоскими: списки словарей и словари внутри словарей — норма, а не исключение.

Вложенный словарь

vlozhennyj_slovar.py

```
student = {  
    "name": "Anna",  
    "scores": {"math": 95, "python": 100},  
}  
print(student["scores"]["python"]) # 100
```

• student

- name → "Anna"
- scores
 - math → 95
 - python → 100

student['scores']['python'] — сначала внешний ключ, потом внутренний

Список словарей — самый частый реальный паттерн

```
spisok_slovaroj.py
```

```
students = [  
    {"name": "Anna", "score": 95},  
    {"name": "Bob", "score": 82},  
]  
  
for student in students:  
    print(student["name"], student["score"])
```

```
● students (список)  
  ● [0]  
    ● name → "Anna"  
    ● score → 95  
  ● [1]  
    ● name → "Bob"  
    ● score → 82
```

Список словарей — так часто выглядят настоящие данные (записи из базы, ответ API)

Список словарей — это уже почти JSON

Ровно так организованы данные, приходящие от многих веб-сервисов (JSON) — подробно про JSON поговорим в одной из следующих глав, но структура «список записей, у каждой записи свои поля» уже полностью знакома.

Словарь списков — обратный вариант

```
slovar_spiskov.py
```

```
classroom = {
    "students": ["Anna", "Bob"],
    "scores": [95, 82],
}
```

Оба представления валидны — какое выбрать, зависит от того, как с данными будут работать дальше: список словарей удобнее, если часто нужна ОДНА запись целиком (найти Bob'a и все его данные); словарь списков — если чаще нужна ОДНА колонка целиком (все имена или все оценки).

Практика: вложенные словари и список словарей

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/11-21/index.html) (<https://www.cartesianschool.org/practice/11-21/index.html>)

Преобразования и comprehensions

От list()/tuple()/set() до компактного способа строить новые коллекции одной строкой — но только после того, как понятен обычный цикл.

Преобразования между коллекциями

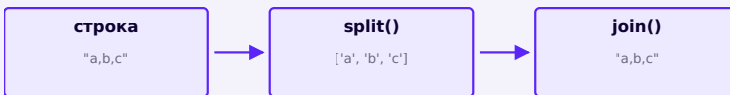
```
preobrazovaniya.py
```

```
print(list("Python"))          # ['P', 'y', 't',
'h', 'o', 'n']
print(tuple([1, 2, 3]))        # (1, 2, 3)
print(set([1, 1, 2, 3]))       # {1, 2, 3}
```

dict() требует пары «ключ, значение»

`dict(...)` превращает в словарь не любую коллекцию, а только такую, из которой можно собрать пары: например, `dict(zip(a, b))` из §11.14. Просто `dict([1, 2, 3])` вызовет ошибку.

Строка ↔ список — самое частое преобразование



`text.split()` → список, `' '.join(список)` → строка обратно

split_join.py

```

text = "python git linux"
words = text.split()
print(words)           # ['python', 'git', 'linux']
print(" ".join(words)) # "python git linux"
  
```

sorted() для разных коллекций

sorted_raznoe.py

```

print(sorted((3, 1, 2)))           # [1, 2, 3] — из
кортежа
print(sorted({3, 1, 2}))           # [1, 2, 3] — из
множества
print(sorted({"b": 1, "a": 2}))    # ['a', 'b'] — из
словаря сортируются КЛЮЧИ
  
```

sorted() всегда возвращает список

Независимо от того, что было на входе — кортеж, множество или словарь — `sorted()` всегда отдаёт обычный `list`.

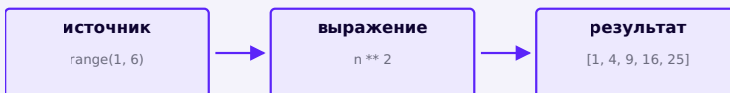
List comprehension — уже знакомо, теперь по шагам

comprehension_klassika.py

```
kvadraty = []
for n in range(1, 6):
    kvadraty.append(n ** 2)
print(kvadraty)
```

comprehension_novyj.py

```
kvadraty = [n ** 2 for n in range(1, 6)]
print(kvadraty)
```

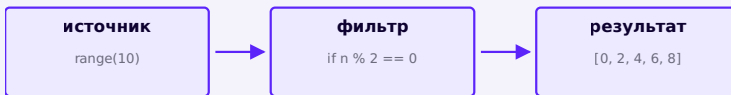


[`выражение for n in источник`] — то же самое, что цикл, но в одну строку

Comprehension с условием

comprehension_uslovie.py

```
chetnye = [n for n in range(10) if n % 2 == 0]
print(chetnye)  # [0, 2, 4, 6, 8]
```



Условие в конце — фильтр: элемент попадёт в результат, только если условие True

Set и dict comprehension

set_comprehension.py

```
unikalnye_bukvy = {ch.lower() for ch in "Python"}
print(unikalnye_bukvy)
```

dict_comprehension.py

```
kvadratny_slovar = {n: n ** 2 for n in range(5)}
print(kvadratny_slovar)
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Та же идея «источник → выражение → результат», только результат оформляется в фигурных скобках — с двоеточием получаем словарь, без — множество.

Чуть глубже — генераторные выражения

`(n ** 2 for n in range(5))` — генераторное выражение, похожее на comprehension, но не строящее список целиком в памяти сразу, а отдающее значения по одному. Пригодится позже, когда будем говорить об итераторах и работе с большими объёмами данных.

Сборка списка: цикл vs comprehension

Цикл с `append()`

```
kuby = []
for n in range(1, 6):
    if n % 2 == 0:
        kuby.append(n
** 3)
```

List comprehension

```
kuby = [n ** 3 for n in
range(1, 6) if n % 2 ==
0]
```

Что использовать сегодня: comprehension для простых преобразований и фильтров — короче и читается как одна мысль. Но comprehension — это НЕ универсальный инструмент глубокого копирования и не заменяет цикл, если внутри нужны побочные эффекты (`print`, запись в файл, несколько условий с разной логикой) — тогда обычный цикл с `append()` читается понятнее.

Практика: преобразования и comprehensions

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/11-22/index.html\)](https://www.cartesianschool.org/practice/11-22/index.html)

Мини-проект — бесконечные цвета

Словарь как «переводчик» — от слова к конкретному цвету Turtle.

Словарь отлично подходит для «перевода» одного значения в другое — например, названия цвета в его код. Нарисуем фигуру, «раскрашенную по словарю».

```
beskonechnye_cveta.py
```

```

cvetovaya_karta = {
    "огонь": "red",
    "трава": "green",
    "небо": "blue",
}

zapros = "небо"
artist.pencolor(cvetovaya_karta[zapros])
artist.circle(50)

```

★★ Самостоятельная задача

Добавьте свои цвета

Добавьте в `cvetovaya_karta` ещё 3-4 пары «слово — цвет» и нарисуйте для каждого отдельную фигуру.

Практика: бесконечные цвета (словари + Turtle)

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/11-09/index.html>)

Как выбрать правильную структуру

Теперь, когда все четыре коллекции знакомы по отдельности, — как решить, какую использовать в конкретной задаче.

Таблица изменяемости

Тип	Упорядочен?	Изменяем?	Повторы?	Доступ
<code>list</code>	да	да	да	по индексу

Тип	Упорядо- чен?	Изменя- ем?	Повторы?	Доступ
tuple	да	нет	да	по индекс- су
set	нет пози- ций	да	нет — только уникаль- ные	по член- ству (in)
frozenset	нет пози- ций	нет	нет — только уникаль- ные	по член- ству (in)
dict	сохраняет порядок вставки	да	ключи уникаль- ны	по ключу

Таблица хешируемости

Что можно класть в множество / использовать как ключ словаря

ОБЫЧНО ХЕШИРУЕМЫ

int, float, str

bool, bytes

tuple (если элементы хешируемы)

frozenset

НЕ ХЕШИРУЕМЫ

list
dict
set

Итоговый ориентир выбора**Полный ориентир выбора структуры**

Нужна пара «ключ → значение»? → **dict**

Иначе: нужны только уникальные значения, порядок не важен? → **set**

Иначе: нужен порядок, значения точно не изменятся? → **tuple**

Иначе: нужен порядок и придётся менять содержимое? → **list**

Правило большого пальца — сначала думайте о задаче, потом о синтаксисе

Истинность коллекций (truthiness)

Из главы 9: пустая коллекция — `False` в логическом контексте, непустая — `True`, независимо от значений внутри:

Пусто → <code>False</code>	Не пусто → <code>True</code>
<code>[]</code>	<code>[0]</code>
<code>()</code>	<code>("",)</code>
<code>set()</code>	<code>{0}</code>
<code>{}</code>	<code>{"x": None}</code>

`truthiness_kollekcij.py`

```
items = []
if items:
    print("Есть элементы")
else:
    print("Список пуст")
```

if items: — идиоматичнее, чем if len(items) > 0:

Обе записи работают одинаково, но `if items:` прямо спрашивает «список непустой?», без промежуточного подсчёта длины — это стандартный стиль в Python-коде.

Изменение коллекции во время перебора — ещё одна ловушка

mutation_during_iteration.py

```
items = [2, 4, 6, 8]
for item in items:
    if item % 2 == 0:
        items.remove(item)
print(items)  # [4, 8] — а не [], хотя условие
              подходит КАЖДОМУ элементу!
```

Не меняйте размер коллекции, пока перебираете её тем же циклом

Удаление элементов списка во время перебора этого же списка сдвигает индексы прямо «под ногами» у цикла — часть элементов будет пропущена. Для словаря или множества Python вообще выбросит `RuntimeError: dictionary changed size during iteration`. Безопасные способы: собирать результат в НОВУЮ коллекцию, использовать comprehension, или перебирать явную копию — `for item in items.copy():`.

Практика: выбор структуры данных по сценарию

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-23/index.html>)

Отладка коллекций: 14 типичных ошибок

`IndexError`, `KeyError`, путаница `append/extend`, aliasing, поверхностная копия и другие ловушки — в одном месте, с примерами и разбором.

14 типичных ошибок при работе со списками, кортежами, множествами и словарями — каждая с примером и объяснением. Первые две разберём отдельно с диаграммой, остальные — компактно.

IndexError — визуально

`items = ['A', 'B', 'C']` — допустимые индексы только 0, 1, 2 (и -1, -2, -3)

```
indexerror.py
```

```
items = ['A', 'B', 'C']  
print(items[3])  
# IndexError: list index out of range
```

KeyError — визуально

`'name'`



`'Anna'`

У `student` есть только ключ `'name'` — обращение по ключу `'email'` не находит ничего и вызывает `KeyError`

`keyerror.py`

```
student = {'name': 'Anna'}  
print(student['email'])  
# KeyError: 'email'
```

Ещё 12 типичных ошибок

1 • IndexError

`bug.py`

```
items = ['A', 'B', 'C']  
print(items[3])
```

Допустимые индексы — от 0 до $\text{len}(\text{items}) - 1 = 2$.
Индекс 3 не существует.

2 • KeyError

`bug.py`

```
student = {'name': 'Anna'}  
print(student['email'])
```

Такого ключа в словаре нет. Безопаснее:
`student.get('email')`.

3 • ValueError от `index()/remove()`


```
bug.py
```

```
fruits = ['яблоко', 'банан']  
fruits.remove('вишня')
```

'вишня' нет в списке — `remove()` ищет по значению и не находит его. Проверьте через `in` перед вызовом.

4 • `TypeError: unhashable type`

```
bug.py
```

```
bad = {[1, 2]: 'значение'}
```

Список нельзя использовать как ключ словаря (или элемент множества) — он изменяем, а значит не хешируем (§11.16).

5 • `append()` вместо `extend()`

```
bug.py
```

```
a = [1, 2]  
a.append([3, 4])  
print(a)    # [1, 2, [3, 4]], а не [1, 2, 3, 4]
```

`append()` всегда добавляет один объект. Чтобы добавить элементы ИЗ списка — нужен `extend()`.

6 · `x = x.sort()` — потеря списка

bug.py

```
numbers = [3, 1, 2]
numbers = numbers.sort()
print(numbers)    # None
```

`sort()` мутирует список на месте и возвращает `None`. Присваивание результата обратно стирает список.

7 · Aliasing вместо копии

bug.py

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)    # тоже [1, 2, 3, 4]!
```

`b = a` не копирует список — оба имени указывают на один и тот же объект (§11.9). Нужно `b = a.copy()`.

8 · Поверхностная копия и вложенный список

bug.py

```
original = [[1, 2]]
copy_ = original.copy()
copy_[0][0] = 99
print(original)    # [[99, 2]] — тоже изменился!
```

.copy() копирует только внешний список — вложенные списки остаются общими (§11.10).
Нужен copy.deepcopy().

9 · `[[0]*3]*3` — общая строка

bug.py

```
grid = [[0] * 3] * 3
grid[0][0] = 1
print(grid)    # изменились ВСЕ строки
```

*3 копирует ссылку на один и тот же внутренний список трижды. Используйте `[[0]*3 for _ in range(3)]`.

10 · Изменение списка во время перебора

bug.py

```
items = [1, 2, 3, 4]
for x in items:
    if x % 2 == 0:
        items.remove(x)
print(items)  # не то, что ожидали
```

Удаление элемента сдвигает индексы прямо во время перебора — часть значений пропускается. Стройте новый список или перебирайте копию.

11 · {} вместо set()

bug.py

```
empty = {}
print(type(empty))  # <class 'dict'>, а не set!
```

Пустой словарь и пустое множество выглядят по-разному только в непустом виде. Пустое множество — всегда set().

12 · Кортеж из одного элемента без запятой

bug.py

```
point = (42)
print(type(point))  # <class 'int'>
```

Кортеж создаёт запятая, а не скобки. Нужно (42,).

13 · `in` у словаря проверяет ключи, не значения

bug.py

```
student = {'name': 'Anna'}  
print('Anna' in student)    # False!
```

'Anna' — значение, а не ключ. Для проверки значений: `'Anna' in student.values()`.

14 · Перепутан порядок вложенного индекса

bug.py

```
matrix = [[1, 2, 3], [4, 5, 6]]  
print(matrix[2][0])    # IndexError
```

У `matrix` всего 2 строки (индексы 0 и 1) по 3 столбца — сначала строка, потом столбец, не наоборот.

Метод отладки: трасса по шагам

- Перед запуском предскажите, что будет в переменной — потом сравните с реальным выводом.
- Для ошибок с индексами/ключами явно выпишите допустимый диапазон или список ключей.
- Если результат «странный, но без ошибки» — подозревайте aliasing или поверхностную копию.
- `print()` после каждого шага — самый надёжный способ найти момент, где ожидание разошлось с реальностью.

Практика: находим и исправляем ошибки в коллекциях

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/11-24/index.html) → (<https://www.cartesianschool.org/practice/11-24/index.html>)

Мини-проект — подсчёт частоты слов

Строки, циклы и словарь-счётчик в одной классической задаче — с ручным алгоритмом и превью готового инструмента из стандартной библиотеки.

Классическая задача, которая объединяет почти всё из этой главы: строки (глава 8), циклы (глава 10) и словарь как счётчик.

Алгоритм по шагам

chastota_slov.py

```
text = "python is great and python is fun"
words = text.lower().split()

counts = {}
for word in words:
    counts[word] = counts.get(word, 0) + 1

print(counts)
```

- word = 'python'**
counts.get('python', 0) + 1 = 1 → counts = {'python': 1}
- word = 'is'**
counts = {'python': 1, 'is': 1}
- word = 'great'**
counts = {'python': 1, 'is': 1, 'great': 1}
- word = 'and'**
counts = {'...', 'and': 1}
- word = 'python' снова**
counts.get('python', 0) + 1 = 2 → counts['python'] = 2

counts.get(word, 0) + 1 — если слова ещё нет, get() вернёт 0, и счётчик начнётся с 1

Почему именно `.get(word, 0)`, а не `counts[word]`

На первой встрече слова ключа `word` в `counts` ещё нет — `counts[word]` вызвал бы `KeyError`. `.get(word, 0)` возвращает 0 для нового слова, и код одинаково работает и для новых, и для уже встречавшихся слов.

Тот же алгоритм через `setdefault()`

`chastota_setdefault.py`

```
counts = {}
for word in words:
    counts.setdefault(word, 0)
    counts[word] += 1
```

Чуть глубже — стандартная библиотека уже решила эту задачу

`counter_preview.py`

```
from collections import Counter

counts = Counter(words)
print(counts)           # Counter({'python': 2,
                          'is': 2, 'great': 1, 'and': 1, 'fun': 1})
print(counts.most_common(2)) # [('python', 2),
                              ('is', 2)]
```


Ручной алгоритм важнее, чем готовая функция

Counter из стандартной библиотеки делает то же самое в одну строку — и в реальном коде часто лучше использовать именно его. Но алгоритм на `.get(word, 0) + 1` стоило пройти руками: он показывает, как вообще устроен подсчёт через словарь, а не только то, что для этого есть готовый инструмент.

Практика: подсчёт частоты слов в тексте

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-25/index.html>)

Мини-проекты с коллекциями

Четыре коротких, но настоящих применения того, что мы изучили: контакты, игровое поле, список класса и сравнение множеств.

Записная книжка контактов (dict)

zapisnaya_knizhka.py

```
contacts = {
    "Anna": "anna@example.com",
    "Bob": "bob@example.com",
}

contacts["Maria"] = "maria@example.com" # добавить
contacts["Anna"] = "anna.new@example.com" # изменить
del contacts["Bob"] # удалить

for name, email in contacts.items():
    print(f"{name}: {email}")
```

Уже мини-база данных

Записная книжка — это, по сути, крошечная база данных в памяти: словарь как хранилище, ключ как уникальный идентификатор записи.

Игровое поле (вложенный список)

igrovoe_pole.py

```
board = [
    [".", ".", "."],
    [".", "X", "."],
    [".", ".", "."],
]

for row in board:
    print(" ".join(row))
```

	0	1	2
0	.	.	.
1	.	X	.
2	.	.	.

`board[1][1] = 'X'` — та же модель матрицы, что и в §11.11

Список класса (список словарей)

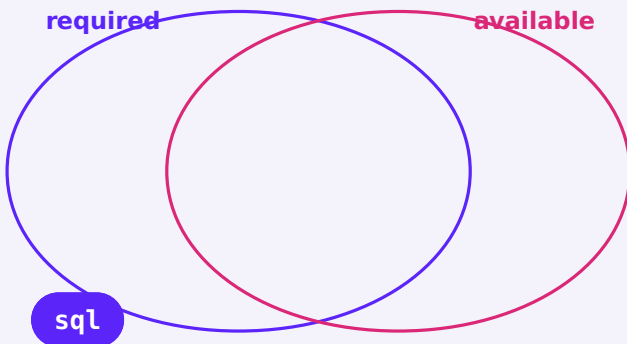
spisok_klassa.py

```
roster = [  
    {"name": "Anna", "score": 95},  
    {"name": "Bob", "score": 82},  
    {"name": "Maria", "score": 91},  
]  
  
total = sum(student["score"] for student in roster)  
average = total / len(roster)  
print(f"Средний балл: {average:.1f}")
```

Сравнение множеств: чего не хватает

sravnenie_navykov.py

```
required = {"python", "git", "sql"}  
available = {"python", "git"}  
  
missing = required - available  
common = required & available  
print("Не хватает:", missing)  
print("Уже есть:", common)
```



`python``git``→ {'sql'}`

required - available — каких навыков не хватает

Практика: мини-проекты с коллекциями

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-26/index.html>)

Мини-проект — перестановка имени и фамилии

Собираем `split()`, распаковку и строки из главы 8 в одной короткой, но полезной программе — и подводим полные итоги главы.

Финальный мини-проект главы: разбить полное имя на части и собрать заново в другом порядке — «Фамилия Имя» вместо «Имя Фамилия», используя методы строк из главы 8 и списки из этой главы.

perestanovka_imeni.py

```

full_name =
input("Введите имя и фамилию через пробел: ")
parts = full_name.split()  # split() возвращает
                             список
name, surname = parts      # распаковка, как у
                             кортежей

print(f"{surname} {name}")

```

split() возвращает список

Мы видели `.split()` в главе 8, но не заостряли внимание на том, что результат — это именно `list`. Теперь, зная про списки, можно распаковывать его так же, как распаковывали кортежи в §11.13.

★★★ Задача повышенной сложности

Три части имени

Обработайте случай с отчеством (три слова): «Имя Отчество Фамилия» → «Фамилия И.О.» с инициалами.

Практика: перестановка имени и фамилии

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/11-10/index.html>)

Итоги главы

Итоговый инструментарий главы 11

Нужна пара «ключ → значение»? → **dict**

Нужны только уникальные значения, порядок не важен?

→ **set**

Нужен порядок, значения точно не изменятся?

→ **tuple**

Нужен порядок, и придётся менять содержимое?

→ **list**

Нужен неизменяемый набор уникальных значений?

→ **frozenset**

Нужны позиция и значение вместе при переборе?

→ **enumerate(...)**

Нужно перебрать несколько коллекций попарно?

→ **zip(...)**

Нужна новая коллекция, преобразованная из старой?

→ **comprehension**

Нужна
независимая
внешняя
копия? → `.copy()` / `list(...)` / `[:]`

Нужна
полностью
независимая
копия
вложенных
структур? → `copy.deepcopy(...)` — с осторожностью

Полная версия этой карты выбора — в §11.22

Что мы узнали в этой главе

- **Список** (`list`) — упорядоченная, изменяемая коллекция в `[]` : индексы, срезы, `append/extend/insert`, `remove/pop/clear`.
- **Кортеж** (`tuple`) — как список, но неизменяемый, в `()` : `packing/unpacking`, звёздочная распаковка, кортеж из одного элемента требует запятой.
- **Множество** (`set`) — уникальные значения без позиций, в `{}` (но `{}` без содержимого — это `dict`, а не `set`!): `union/intersection/difference/symmetric_difference`.
- **Словарь** (`dict`) — пары «ключ: значение», сохраняет порядок вставки, доступ по ключу, а не по позиции.
- `b = a` для изменяемых объектов создаёт ВТОРОЕ ИМЯ того же объекта (aliasing), а не копию — для копии нужен `.copy()` , а для вложенных структур — `.deepcopy()` .
- Хешируемыми (и годными в качестве ключей словаря или элементов множества) бывают только неизменяемые типы: числа, строки, кортежи из хешируемых элементов, `frozenset`.
- Генератор списков [выражение for элемент in коллекция] — компактная альтернатива циклу с `append()` , но не замена циклу везде, где нужна более сложная логика.
- Выбор структуры данных — это в первую очередь вопрос к задаче («нужен ли порядок?», «нужны ли уникальные значения?», «нужен ли ключ?»), а не вопрос синтаксиса.

Множество увлекательных мини- проектов!

Первая настоящая проектная лаборатория курса: 16 проектов, которые заставляют работать вместе всё, что вы изучили в главах 1-11 — условия, циклы, строки, числа, random, списки/кортежи/множества/словари и Turtle. Не новая теория, а то, как строить программу из идеи: декомпозиция, алгоритм до кода, разработка маленькими шагами, тесты и отладка.

12.1 Что такое проект

12.2 Строим проект по шагам

12.3 Проект: чётное или нечётное

12.4 Проект: достаточно ли чаевых?

12.5 Проект: угадай число, версия 3

12.6 Проект: анализатор текста и частота слов

12.7 Проект: записная книжка

12.8 Проекты: журнал оценок и корзина покупок

12.9 Проект: викторина

Данные vs алгоритм, рефакторинг

12.10 Проекты: консоль команд и проверка пароля

12.11 Проект: студия многоугольников

12.12 Проект: рождественская ёлка

12.13 Проект: спирали!

12.14 Проект: сложная мандала

12.15 Проект: пиксельная графика по сетке

12.16 Проект: гонка Turtle

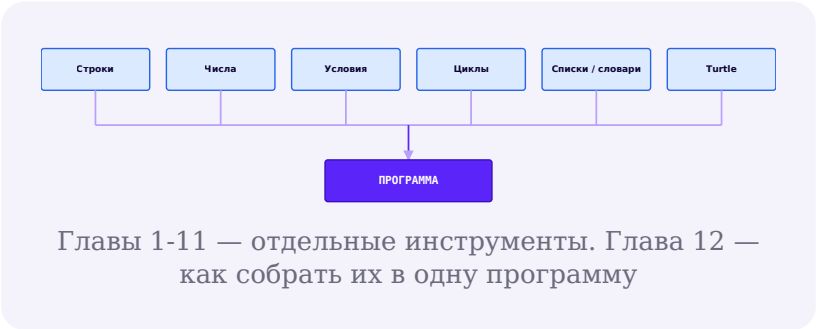
Итоги главы

Что такое проект

До сих пор мы изучали инструменты по отдельности. Теперь учимся складывать их в настоящую программу — начиная с того, как вообще подступиться к большой задаче.

От инструментов — к программе

Главы 1-11 научили вас инструментам по отдельности: строкам, числам, условиям, циклам, Turtle, спискам и словарям. Эта глава не добавляет новых инструментов — она учит **складывать их вместе**, чтобы получилась настоящая, работающая программа.



Чем проект отличается от упражнения

	Упражнение	Проект
Пример	Вычислите <code>7 % 3</code>	Постройте конвертер секунд в часы/минуты/секунды с проверкой ввода
Входные данные	обычно нет	есть, и их нужно продумать заранее
Шагов	один	несколько, связанных друг с другом
Результат	одно значение	оформленный вывод, реакция на разные случаи
Ошибки	почти невозможны	нужно продумать, что может пойти не так

Проект — это не «код побольше»

Разница не в количестве строк, а в том, что у проекта есть **цель, входные данные, правила и результат** — и всё это нужно продумать до того, как написана первая строка кода.

Декомпозиция: разбиваем большую задачу на маленькие

Задача «Постройте игру-угадайку» слишком большая, чтобы решить её одним куском кода. Разобьём на понятные шаги:



Декомпозиция: одна большая задача → восемь маленьких, понятных шагов

Декомпозиция

Декомпозиция — разделить большую задачу на маленькие понятные части. Это один из самых важных навыков в программировании: гораздо проще написать (и проверить) восемь маленьких шагов, чем один запутанный кусок кода на сто строк.

Шаблон проекта, который мы будем использовать

Каждый крупный проект этой главы устроен по одной и той же схеме — не обязательно с этими же самыми заголовками, но в этом же порядке мысли:

Повторяющийся ритм каждого проекта главы

1 · ЧТО СОЗДАЁМ

Цель проекта

Как выглядит готовый результат

2 · ДАННЫЕ

Входные данные

Какая структура их хранит

3 · АЛГОРИТМ

Порядок действий

Блок-схема, если это уместно

4 · РЕАЛИЗАЦИЯ

По одному этапу за раз

Проверка после каждого этапа

5 · ПРОВЕРКА

Типичные ошибки

Debug Lab

Что можно улучшить

Требования и «что значит „готово“»

Прежде чем писать код, полезно явно сформулировать требования — что программа обязана делать, чтобы считаться готовой. Пример для будущей игры-угадайки:

Требование	Проверка
компьютер выбирает целое число от 1 до 100	секрет — int в диапазоне [1, 100]
игрок вводит числа-попытки	input() + int()
программа говорит «мало» / «много»	if/elif/else
программа завершается на правильном ответе	цикл прекращается при guess == secret
попытки считаются	counter увеличивается на каждой попытке

Чек-лист приёмки

Такой список называют **чек-листом приёмки** — до кода мы уже знаем, что именно должно быть верно, чтобы сказать «проект готов». Не обязательно оформлять его формально каждый раз — но держать в голове стоит всегда.

Разминка: статистика трёх чисел

Прежде чем переходить к крупным проектам, потренируем сам процесс — от задачи до кода — на программе из нескольких строк.

Задача: получить три числа и вывести минимальное, максимальное, сумму и среднее.

```
statistika_treh_chisel.py
```

```
a = float(input("Первое число: "))
b = float(input("Второе число: "))
c = float(input("Третье число: "))

chisla = [a, b, c]

print(f"Минимум: {min(chisla)}")
print(f"Максимум: {max(chisla)}")
print(f"Сумма: {sum(chisla)}")
print(f"Среднее: {sum(chisla) / len(chisla):.2f}")
```

Даже такая маленькая программа — уже проект

У неё есть входные данные (три числа), структура для их хранения (список — глава 11), правило обработки (min/max/sum/len — глава 11) и оформленный результат (f-строки — глава 8). Дальше в главе — то же самое, но крупнее.

Практика: статистика трёх чисел — от задачи до кода

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

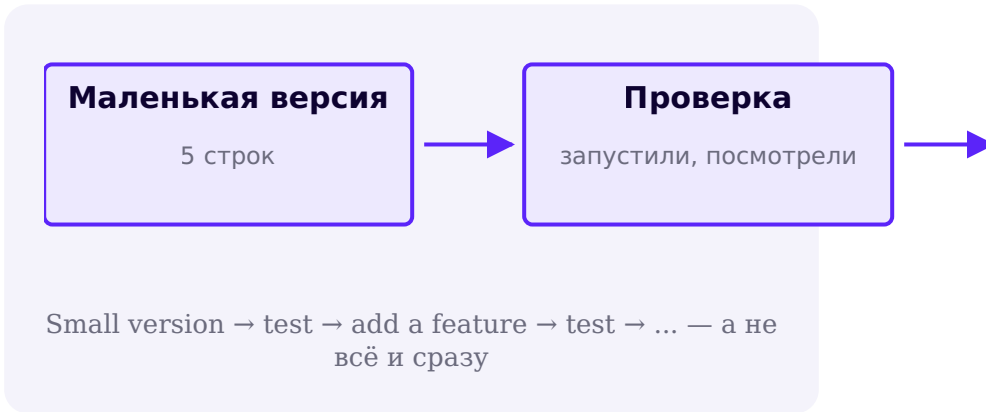
Открыть практику → (<https://www.cartesianschool.org/practice/12-07/index.html>)

Строим проект по шагам

Маленькими проверяемыми шагами вместо 80 строк за раз — и системный способ найти и исправить ошибку, когда что-то пошло не так.

Не пишите 80 строк, а потом нажимайте «Запустить»

Профессиональная привычка, которую стоит выработать с первого крупного проекта: **инкрементальная разработка** — маленькими шагами, каждый из которых сразу проверяется.



Работающая маленькая версия лучше, чем неработающая большая

Если после каждого маленького шага программа запускается и делает то, что ожидалось — вы всегда точно знаете, где искать причину, если что-то сломалось на следующем шаге.

Версии: сначала работает, потом — лучше

Версия	Что в ней есть
V1	минимальная работающая версия — ядро идеи, без украшений
V2	более надёжная проверка ввода, обработка граничных случаев
V3	дополнительные возможности, улучшенный вывод

Такую первую работающую версию иногда называют **MVP** (minimal working version) — «сначала добьёмся маленькой, но работающей версии», а уже потом улучшаем.

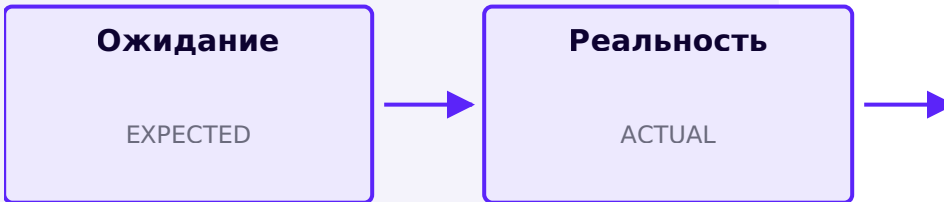
Проверяем проект на примерах

Для каждого нетривиального проекта полезно заранее продумать несколько сценариев — что подать на вход и что должно получиться на выходе:

Сценарий	Вход	Ожидаемый результат
секрет 50, попытка меньше	guess = 20	«слишком мало»
секрет 50, попытка больше	guess = 70	«слишком много»
секрет 50, попытка верна	guess = 50	победа, цикл завершается

Граничные случаи

Обычных примеров недостаточно — стоит явно проверить: пустой ввод, ноль, отрицательное число там, где оно не ожидается, повтор в данных, пустой список, неизвестная команда, значение точно на границе диапазона. Мы вернёмся к этой идее в каждом крупном проекте главы.

Воркфлоу отладки

Ожидание vs реальность → первое расхождение → причина → исправление

Пошагово:

Шесть шагов отладки — работают для любого проекта этой главы

1

Воспроизвести ошибку
получить её снова, стабильно

2

Осмотреть значения
`print()` нужных переменных

3

Найти первое неверное состояние
где ожидание разошлось с фактом

4

Изолировать условие/цикл
какая именно строка виновата

5

Исправить одну причину
не переписывать всё

6

Запустить снова
убедиться, что починилось

print()-отладка

print_otladka.py

```
for word in words:
    print("DEBUG:", word, counts.get(word))    #
    временная диагностика
    counts[word] = counts.get(word, 0) + 1
```

Уберите отладочные print() после того, как нашли причину

Временный print("DEBUG:", ...) внутри цикла — самый надёжный способ увидеть, что происходит на самом деле. Уберите такие строки, когда баг найден и исправлен — иначе они засоряют настоящий вывод программы.

Debug

Lab: найдите ошибку

В программе ниже счёт должен расти на каждом правильном ответе, но почему-то всегда остаётся 1. Прежде чем читать дальше — попробуйте найти причину сами.

```
debug_lab_schet.py
```

```
answers = ["python", "git", "sql"]
user_answers = ["python", "python", "sql"]

for correct, given in zip(answers, user_answers):
    score = 0
    if given == correct:
        score += 1

print("Итоговый счёт:", score)
```

Причина: счётчик обнуляется внутри цикла

`score = 0` стоит ВНУТРИ тела цикла — значит, он пересоздаётся на каждой итерации, и предыдущий прогресс теряется. Счётчик нужно инициализировать ОДИН раз, до цикла — та же ошибка, что и «почему cart total сбрасывается на каждой итерации» из главы 10.

Чек-лист готовности проекта

Прежде чем сказать «проект готов»

- Программа выполняет то, что требовалось (чек-лист приёмки)?
- Обычный, «счастливый» ввод работает правильно?
- Проверены граничные случаи (пусто, ноль, повтор, неизвестная команда)?
- Имена переменных понятны без дополнительных объяснений?
- Нет ли повторяющегося куска кода, который можно было бы убрать?
- Использована ли подходящая коллекция (список / множество / словарь)?
- Цикл действительно останавливается в нужный момент?
- Условия проверяют именно то, что нужно?
- Вывод понятен — пользователь видит, что произошло?
- Сможет ли другой человек прочитать и понять эту программу?

Практика: Debug Lab — найдите и исправьте ошибку счётчика

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/12-08/index.html>)

Проект: чётное или нечётное

Первый проект главы закрепляет условия и оператор % из ранних глав — маленький, но с полным циклом «данные → алгоритм → код».

ИСПОЛЬЗУЕМ

```
input()  int()  %  if/else  
list comprehension
```

УРОВЕНЬ

★ разминка

РЕЗУЛЬТАТ

текстовая программа

Что создаём

Первый настоящий проект главы — маленький, но с полным циклом: данные → алгоритм → код → результат.

Часть 1 — Ваше число чётное или нечётное?

chetnoe_ili_nechetnoe.py

```
number = int(input("Введите число: "))

if number % 2 == 0:
    print(f"{number} — чётное.")
else:
    print(f"{number} — нечётное.")
```

Введите число: 17

17 — нечётное.

Детерминированный пример выполнения

Часть 2 — выводим чётные или нечётные числа из диапазона

chetnye_iz_diapazona.py

```
nachalo = int(input("Начало диапазона: "))
koniec = int(input("Конец диапазона: "))

chetnye = [n for n in range(nachalo, konec + 1) if n
% 2 == 0]
print("Чётные числа:", chetnye)
```

Какие темы здесь встретились

input() и int() — глава 8; % — глава 5; if/else — глава 9; генератор списков — глава 11.

Практика: чётное или нечётное

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/12-01/index.html\)](https://www.cartesianschool.org/practice/12-01/index.html)

Проект: достаточно ли чаевых оставляет ваша мама?

Считаем процент чаевых от суммы счёта и оцениваем щедрость через цепочку `elif`.

ИСПОЛЬЗУЕМ

`input()` `float()` арифметика `if/elif/else`
f-строки

УРОВЕНЬ

★ разминка

РЕЗУЛЬТАТ

текстовая программа

Что создаём

Стандартные чаевые в ресторане — 15–20% от счёта. Проверим, укладывается ли конкретная сумма чаевых в этот диапазон.

```
chaevye.py
```

```
schet = float(input("Сумма счёта: "))
chaevye = float(input("Сумма чаевых: "))

procent = (chaevye / schet) * 100

if procent < 15:
    print(f"Маловато — всего {procent:.1f}%. Обычно
оставляют 15-20%.")
elif procent <= 20:
    print(f"В самый раз — {procent:.1f}%!")
else:
    print(f"Очень щедро — целых {procent:.1f}%!")
```

Сумма счёта: 40

Сумма чаевых: 8

В самый раз — 20.0%!

Детерминированный пример выполнения

★★ Самостоятельная задача

Своя граница щедрости

Добавьте четвёртую категорию — «сказочно щедро» — для чаевых больше 30%.

Практика: вычисляем и оцениваем процент чаевых

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/12-02/index.html>)

Проект: угадай число, версия 3

Полный цикл разработки на знакомой игре: требования → алгоритм → блок-схема → трасса состояния → код.

ИСПОЛЬЗУЕМ

random while if/elif/else input()/int()
счётчик

УРОВЕНЬ

★★ средний

РЕЗУЛЬТАТ

текстовая игра

Что создаём

Финальная, отполированная версия игры-угадайки из глав 9-10 — с подсчётом попыток и аккуратным выводом.

Загадано число от 1 до 100. Попробуйте угадать!

Ваша попытка: 50

Слишком много!

Ваша попытка: 25

Слишком мало!

Ваша попытка: 37

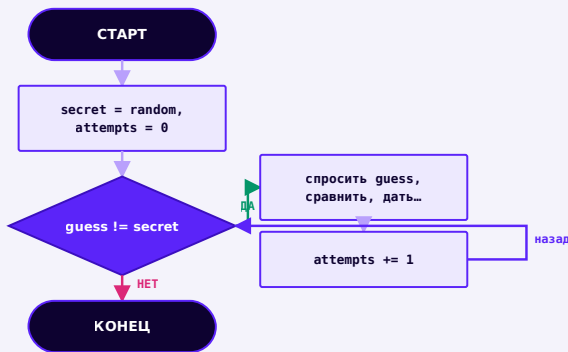
Поздравляем! Загадано было 37. Попыток: 3

Детерминированный пример выполнения

Требования

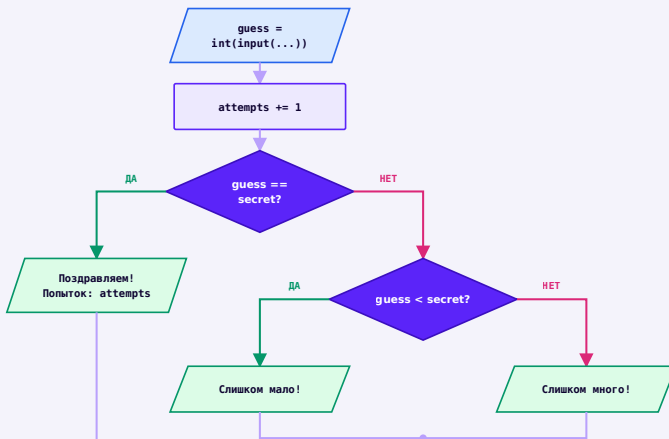
Требование	Как проверим
компьютер выбирает целое число от 1 до 100	<code>random.randint(1, 100)</code>
игрок вводит попытки, пока не угадает	<code>while guess != secret</code>
после каждой попытки — подсказка мало/много	<code>if/elif/else</code>
попытки считаются	<code>attempts += 1</code>
игра завершается на правильном ответе	цикл заканчивается, печатается итог

Алгоритм: внешний цикл



Пока не угадано — спросить попытку, сравнить, подсказать, увеличить счётчик

Алгоритм: что происходит внутри одной попытки



Одна попытка: сравнить и подсказать — весь этот блок повторяется циклом `while` сверху

Трасса состояния

Попытка	guess	secret	Результат
1	50	37	слишком много
2	25	37	слишком мало
3	37	37	угадано, цикл завершается

Финальный код

ugadaj_chislo_v3.py

```
import random

secret = random.randint(1, 100)
attempts = 0
guess = None

print("Загадано число от 1 до 100. Попробуйте угадать!")

while guess != secret:
    guess = int(input("Ваша попытка: "))
    attempts += 1
    if guess < secret:
        print("Слишком мало!")
    elif guess > secret:
        print("Слишком много!")
    else:
        print(f"Поздравляем! Загадано было {secret}.")
    Попыток: {attempts})
```

В практике число фиксировано, а не случайно

На странице показан настоящий `random.randint(1, 100)` — именно так должна работать игра. В ноутбуке практики секретное число зафиксировано заранее, а ввод подставляется автоматически — иначе автоматическая проверка результата была бы недетерминированной.

Debug

Lab

Что произойдёт, если строку `attempts = 0` случайно перенести внутрь цикла `while`? Проверьте предположение, прежде чем читать дальше.

attempts всегда будет равен 1

Ровно та же ошибка, что и в §12.2 Debug Lab: счётчик, обнуляемый внутри цикла, никогда не накапливает значение — на каждой итерации он создаётся заново.

Уровень 3: ограничение попыток

Усложним требования: игра заканчивается поражением после 10 неудачных попыток.

```
ugadaj_chislo_limit.py
```

```
max_attempts = 10

while guess != secret and attempts < max_attempts:
    ...

if guess == secret:
    print(f"Победа за {attempts} попыток!")
else:
    print(f"Числа закончились. Было загадано {secret}.")
```


Практика: угадай число, версия 3

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/12-09/index.html) → (<https://www.cartesianschool.org/practice/12-09/index.html>)

Практика (★★★): ограничение числа попыток

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/12-10/index.html) → (<https://www.cartesianschool.org/practice/12-10/index.html>)

Проект: анализатор текста и частота слов

Строки, циклы, множества и словари — впервые вместе, в одной программе, которая рассказывает о тексте больше, чем видно на первый взгляд.

ИСПОЛЬЗУЕМ

strings циклы list set dict

УРОВЕНЬ

★★★ интеграционный

РЕЗУЛЬТАТ

ТЕКСТОВЫЙ ОТЧЁТ

Что создаём

Программу, которая получает предложение и считает про него всё интересное: сколько символов, слов, уникальных слов и какое слово встречается чаще всего.

Введите текст: Python is great and python is fun

Символов: 33

Слов: 7

Уникальных слов: 5

Самое частое слово: 'python' – 2 раз(а)

Детерминированный пример выполнения

Конвейер обработки данных



Текст → нормализация → список слов → множество уникальных слов

Почему set()

Множество естественно устраняет повторы (глава 11) — количество уникальных слов равно просто `len(set(words))`, без единой ручной проверки.

Этап 1 — базовые подсчёты

```
analizator_etap1.py
```

```
text = input("Введите текст: ")

words = text.split()

print("Символов:", len(text))
print("Слов:", len(words))
```

Этап 2 — уникальные слова

```
analizator_etap2.py
```

```
normalized = text.lower().split()
unikalnye = set(normalized)

print("Уникальных слов:", len(unikalnye))
```

Без `.lower()` 'Python' и 'python' — разные слова

Строки сравниваются посимвольно и с учётом регистра (глава 8) — `"Python" != "python"`. Если забыть `.lower()` перед подсчётом уникальных слов, они посчитаются как два разных слова.

Этап 3 — частота слов

Тот же алгоритм, что подробно разбирался в главе 11 (§11.24) — теперь как часть более крупной программы:

```
chastota_slov.py
```

```
counts = {}
for word in normalized:
    counts[word] = counts.get(word, 0) + 1
```

word	старое значение	новое значение
python	0 (get вернул 0)	1
is	0	1
great	0	1
and	0	1
python	1	2
is	1	2

Находим самое частое слово

```
samoe_chastoe.py
```

```
samoe_chastoe = max(counts, key=counts.get)
print(f"Самое частое слово: {samoe_chastoe!r} –
{counts[samoe_chastoe]} раз(a)")
```

max() с key — не только для чисел

`max(counts, key=counts.get)` перебирает КЛЮЧИ словаря (глава 11) и выбирает тот, для которого `counts.get(ключ)` максимален — компактный способ найти «самое частое» без ручного цикла со сравнением.

Финальный код

analizator_teksta.py

```
text = input("Введите текст: ")
normalized = text.lower().split()

unikalnye = set(normalized)

counts = {}
for word in normalized:
    counts[word] = counts.get(word, 0) + 1
samoe_chastoe = max(counts, key=counts.get)

print("Символов:", len(text))
print("Слов:", len(normalized))
print("Уникальных слов:", len(unikalnye))
print(f"Самое частое слово: {samoe_chastoe!r} – {counts[samoe_chastoe]} раз(a)")
```

Debug

Lab

Если посчитать `len(unikalnye)` из слов БЕЗ `.lower()`, а частоту слов — уже ПОСЛЕ приведения к нижнему регистру, эти два числа могут не вйтсь друг с другом. Почему?

Используйте один и тот же нормализованный список везде

Если `unikalnye` считается из необработанных слов, а `counts` — из `normalized`, то «Python» и «python» попадут в `unikalnye` как два разных слова, а в `counts` — как одно. Всегда считайте от одного и того же, уже нормализованного, списка.

Практика: анализатор текста — символы, слова, уникальные слова

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/12-11/index.html>)

Практика: частота слов и самое частое слово

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/12-12/index.html>)

Проект: записная книжка

Словарь как естественное хранилище «имя → телефон» — и первый взгляд на интерактивное меню, управляемое командами.

ИСПОЛЬЗУЕМ

dict if/elif циклы f-строки

УРОВЕНЬ

★★ средний

РЕЗУЛЬТАТ

текстовое меню

Что создаём

Записную книжку контактов в памяти программы: показать все контакты, добавить, найти, изменить, удалить.

Модель данных

● contacts (dict)

● "Anna" → "+48 111 111 111"

● "Bob" → "+48 222 222 222"

Ключ — имя, значение — телефон. Почему dict?
Потому что ищем контакт по имени, а не по позиции

Почему словарь, а не список

Список пришлось бы перебирать целиком в поисках нужного имени. Словарь сразу даёт доступ по ключу (глава 11) — `contacts["Anna"]`, без перебора.

Действия

`zapisnaya_knizhka.py`

```
contacts = {
    "Anna": "+48 111 111 111",
    "Bob": "+48 222 222 222",
}

# показать все контакты
for name, phone in contacts.items():
    print(f"{name}: {phone}")

# добавить контакт
contacts["Maria"] = "+48 333 333 333"

# найти контакт
zapros = "Anna"
if zapros in contacts:
    print(f"Найдено: {contacts[zapros]}")
else:
    print("Контакт не найден")

# изменить контакт
contacts["Anna"] = "+48 111 000 000"

# удалить контакт
del contacts["Bob"]

print("Контактов осталось:", len(contacts))
```


Уровень 2: интерактивное меню

То же самое, но управляется командами вместо жёстко прописанной последовательности действий — предвестник консоли команд из §12.10:

```
zapisnaya_knizhka_menu.py
```

```
contacts = {}

while True:
    command = input("Команда (add/show/exit): ").strip().lower()
    if command == "exit":
        break
    elif command == "add":
        name = input("Имя: ")
        phone = input("Телефон: ")
        contacts[name] = phone
    elif command == "show":
        for name, phone in contacts.items():
            print(f"{name}: {phone}")
```

Когда программа завершится, contacts исчезнет

`contacts` живёт только в памяти, пока работает программа — это обычная переменная (глава 3). Чтобы данные сохранялись между запусками, понадобятся файлы — тема одной из следующих глав.

Чуть глубже — контакт с несколькими полями

Если контакту нужно больше одного значения (телефон И email), внешний словарь может хранить вложенные словари (глава 11, §11.19):

```
zapisnaya_knizhka_vlozhennaya.py
```

```
contacts = {  
    "Anna": {"phone": "+48 111 111 111", "email":  
        "anna@example.com"},  
}  
print(contacts["Anna"]["email"])
```

Debug

Lab

```
debug_lab_contacts.py
```

```
contacts = {"Anna": "+48 111 111 111"}  
print("+48 111 111 111" in contacts) # False!
```

in у словаря проверяет ключи, а не значения

Та же ловушка из главы 11 (§11.19): `in` ищет среди КЛЮЧЕЙ. Чтобы проверить, есть ли такой телефон среди значений, нужно `" +48 111 111 111" in contacts.values()`.

Практика: записная книжка — добавить, найти, изменить, удалить

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/12-13/index.html) (<https://www.cartesianschool.org/practice/12-13/index.html>)

Проекты: журнал оценок и корзина покупок

Одна и та же модель данных — список словарей — и один и тот же приём — накопитель, применённый в двух разных задачах.

ИСПОЛЬЗУЕМ

list dict циклы накопитель

УРОВЕНЬ

★★★ интеграционный

РЕЗУЛЬТАТ

отчёт / чек

Два проекта на одной странице специально: у обоих одна и та же модель данных — " **список словарей** " — и один и тот же приём — **накопитель** (глава 10), только применённый по-разному.

Проект: журнал оценок

```

● students (список)
  ● [0]
    ● name → "Anna"
    ● score → 95
  ● [1]
    ● name → "Bob"
    ● score → 82

```

Список словарей — почему list? Потому что ученики образуют упорядоченную последовательность записей

zhurnal_ocenok.py

```
students = [
    {"name": "Anna", "score": 95},
    {"name": "Bob", "score": 82},
    {"name": "Maria", "score": 91},
    {"name": "Leo", "score": 58},
]

for student in students:
    print(f"{student['name']}: {student['score']}")

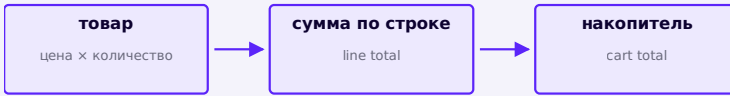
scores = [student["score"] for student in students]
average = sum(scores) / len(scores)
print(f"Средний балл: {average:.1f}")
print("Максимум:", max(scores))
print("Минимум:", min(scores))

otlichniki = [s for s in students if s["score"] >= 90]
print("Отличников:", len(otlichniki))
```

Классификация через if/elif

Если нужна не просто фильтрация, а оценка «отлично/хорошо/пересдача», к каждому студенту можно применить цепочку if/elif/else из главы 9 — это не про реальные школьные критерии, а про тренировку алгоритма классификации.

Проект: корзина покупок



Каждый товар даёт свою сумму, накопитель складывает их все вместе

korzina_pokupok.py

```

cart = [
    {"name": "Молоко", "price": 4.50, "qty": 2},
    {"name": "Хлеб", "price": 2.20, "qty": 1},
    {"name": "Сыр", "price": 9.90, "qty": 1},
]

total = 0
for item in cart:
    line_total = item["price"] * item["qty"]
    total += line_total
    print(f"{item['name']:<10} {item['qty']} x {item['price']:.2f} = {line_total:.2f}")

print(f"Итого: {total:.2f}")
  
```

```

Молоко      2 x 4.50 = 9.00
Хлеб        1 x 2.20 = 2.20
Сыр         1 x 9.90 = 9.90
Итого: 21.10
  
```

Чек — детерминированный пример выполнения

Это учебная арифметика, не Decimal

Для настоящих денежных расчётов используют более точные типы (например `decimal.Decimal`), чтобы избежать неточностей `float` (глава 5). Здесь используется обычная арифметика — целей главы 12 это не меняет: фокус на структуре программы, а не на денежной точности.

Debug

Lab

Почему в этом коде `total` в конце равен сумме только ПОСЛЕДНЕГО товара?

```
debug_lab_korzina.py
```

```
for item in cart:
    total = 0
    total += item["price"] * item["qty"]
```

Накопитель обнуляется внутри цикла

Та же ошибка, что уже встречалась в §12.2 и §12.5: `total = 0` стоит внутри тела цикла, поэтому пересоздаётся на каждой итерации. Инициализация накопителя должна быть ОДИН раз, до цикла.

Практика: журнал оценок — среднее, максимум, минимум, отличники

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/12-14/index.html) → (<https://www.cartesianschool.org/practice/12-14/index.html>)

Практика: корзина покупок — чек и итоговая сумма

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/12-15/index.html) → (<https://www.cartesianschool.org/practice/12-15/index.html>)

Проект: викторина

Самый важный архитектурный урок главы: вопросы — это данные, а не код, и алгоритм не должен меняться, когда меняются данные.

ИСПОЛЬЗУЕМ

`list dict for input if счётчик`

УРОВЕНЬ

★★★★ challenge

РЕЗУЛЬТАТ

текстовая викторина

Что создаём

Викторину из нескольких вопросов: программа задаёт вопрос, принимает ответ, сравнивает его с правильным, считает счёт и в конце показывает результат.

Вопрос 1: Столица Франции?

Ваш ответ: paris

Верно!

Вопрос 2: $7 * 8$?

Ваш ответ: 54

Неверно. Правильный ответ: 56

Счёт: 1 из 2

Детерминированный пример выполнения

Данные vs алгоритм — главный урок этого проекта

Есть соблазн написать викторину так:

Три вопроса: захардкоженная логика → данные + цикл

```
Вопросы «защиты» в код
question1 = "Столица
Франции?"
answer1 = "paris"
user1 = input(question1
+ " ").strip().lower()
if user1 == answer1:
    score += 1

question2 = "7 * 8?"
answer2 = "56"
user2 = input(question2
+ " ").strip().lower()
if user2 == answer2:
    score += 1

# ...и так для каждого
следующего вопроса
```

```
Вопросы — данные,
алгоритм один
questions = [
    {"question":
"Столица Франции?",
"answer": "paris"},
    {"question": "7 *
8?", "answer": "56"},
]

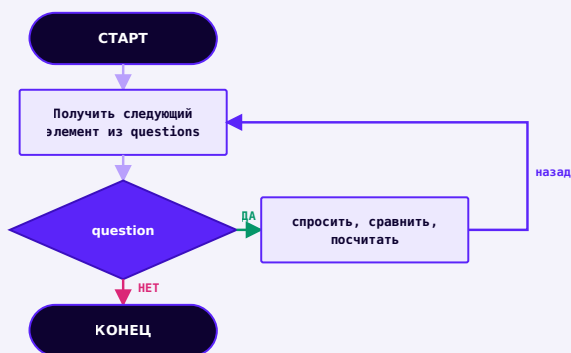
score = 0
for q in questions:
    user_answer =
input(q["question"] + "
").strip().lower()
    if user_answer ==
q["answer"]:
        score += 1
```

Что использовать сегодня: хранить вопросы как ДАННЫЕ (список словарей), а не как повторяющийся код. Если вопросов станет 20 вместо 2, левый вариант пришлось бы переписывать заново, а правый — просто дополнить список. **Алгоритм не меняется, когда меняются данные** — в этом суть *data-driven programming*.

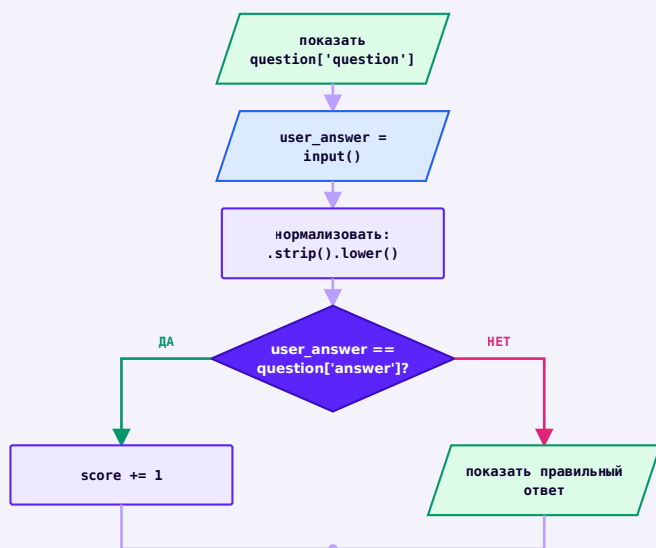
Рефакторинг

Переход от левого варианта к правому называется **рефакторингом** — мы улучшаем внутреннюю структуру программы, не меняя то, что она должна делать. Программа с двумя вопросами ведёт себя одинаково в обеих версиях — правая просто лучше устроена и легче расширяется.

Алгоритм



`for question in questions` — один и тот же алгоритм для любого числа вопросов



Одна итерация цикла — этот блок повторяется for-циклом сверху, для каждого вопроса

Финальный код

viktorina.py

```
questions = [
    {"question": "Столица Франции?", "answer":
"paris"},
    {"question": "7 * 8?", "answer": "56"},
    {"question": "Язык, который мы изучаем?",
"answer": "python"},
]

score = 0
for q in questions:
    user_answer = input(q["question"] + "
").strip().lower()
    if user_answer == q["answer"]:
        print("    Верно!")
        score += 1
    else:
        print(f"    Неверно. Правильный ответ:
{q['answer']}")

print(f"Счёт: {score} из {len(questions)}")
```

Уровень 4: процент правильных ответов

viktorina_procent.py

```
procent = score / len(questions) * 100
print(f"Результат: {procent:.0f}%")
```

Debug

Lab

Что произойдёт, если убрать `.strip().lower()` при нормализации ответа?

«Paris» и «paris» — разные строки

Без нормализации регистр и лишние пробелы делают правильный ответ «неверным» — та же чувствительность строк к регистру и пробелам, что и в главе 8. Всегда нормализуйте пользовательский ввод перед сравнением.

Практика: викторина из списка вопросов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/12-16/index.html) → (<https://www.cartesianschool.org/practice/12-16/index.html>)

Практика (★★★★): процент правильных ответов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/12-17/index.html) → (<https://www.cartesianschool.org/practice/12-17/index.html>)

Проекты: консоль команд и проверка пароля

Два способа читать пользовательский ввод в цикле: команда за командой и символ за символом, накапливая состояние.

ИСПОЛЬЗУЕМ

```
while True    if/elif/else    break    strings
bool
```

УРОВЕНЬ

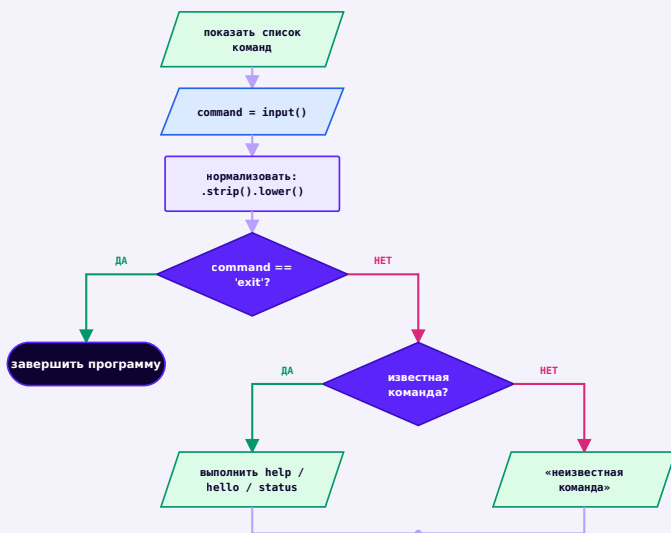
★★★ интеграционный

РЕЗУЛЬТАТ

текстовая консоль / проверка

Проект: консоль команд

Простое текстовое приложение с командами: `help`, `hello`, `status`, `exit`. Этот паттерн — «бесконечный цикл, читающий команды» — встречается в меню, консолях, ботах, играх и утилитах командной строки.



Пока не exit — прочитайте команду, распознать её, выполнить, повторить

```
konsoł_komand.py
```

```
history = []

while True:
    command = input("> ").strip().lower()
    history.append(command)

    if command == "exit":
        print("До встречи!")
        break
    elif command == "help":
        print("Команды: help, hello, status, exit")
    elif command == "hello":
        print("Привет!")
    elif command == "status":
        print(f"Команд выполнено: {len(history)}")
    else:
        print(f"Неизвестная команда: {command}")
```

```
> help
Команды: help, hello, status, exit
> hello
Привет!
> exit
До встречи!
```

Детерминированный пример выполнения

history — обычный список

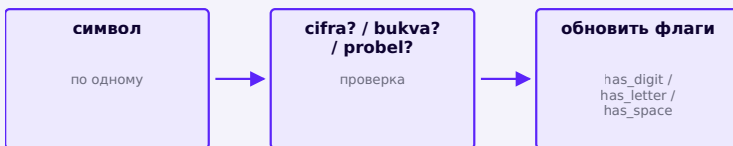
`history` копит все введённые команды в порядке поступления — ровно то, для чего создан список (глава 11): упорядоченная, изменяемая коллекция.

Проект: проверка имени пользователя / пароля

Это учебное упражнение по строкам, а не защита данных

Ниже — тренировка работы со строками и булевыми флагами, а НЕ полноценная политика безопасности паролей. Настоящая защита учётных записей требует значительно большего (хеширование, соль, менеджеры паролей, лимиты попыток) — тем этого курса пока не касаемся.

Правила (учебные): минимум 8 символов, есть хотя бы одна цифра, есть хотя бы одна буква, нет пробелов.



Перебираем строку посимвольно, накапливая три булевых флага

```
validator_parolya.py
```

```
password = input("Придумайте пароль: ")

has_digit = False
has_letter = False
has_space = False

for ch in password:
    if ch.isdigit():
        has_digit = True
    elif ch.isalpha():
        has_letter = True
```

```

    elif ch == " ":
        has_space = True

dlinnyj_dostatochno = len(password) >= 8

if dlinnyj_dostatochno and has_digit and has_letter
and not has_space:
    print("Пароль подходит под учебные правила.")
else:
    print("Пароль не подходит:")
    if not dlinnyj_dostatochno:
        print("- слишком короткий (нужно 8+
символов)")
    if not has_digit:
        print("- нет ни одной цифры")
    if not has_letter:
        print("- нет ни одной буквы")
    if has_space:
        print("- содержит пробел")

```

Накопление состояния через булевы флаги

has_digit, has_letter и has_space — это накопители (глава 10), только булевы: каждый символ может ПЕРЕКЛЮЧИТЬ флаг в True, и он остаётся True до конца перебора.

Debug

Lab

Если использовать `elif` вместо трёх отдельных `if` при проверке символа — некоторые символы могут обрабатываться неправильно. Например, что если правило было бы «есть спецсимвол»?

`elif` проверяет только одну ветку на символ

Пока правила взаимоисключающие (цифра/буква/пробел — символ не может быть сразу двумя из них), `elif` работает верно. Но если добавить проверку ещё одного, НЕ взаимоисключающего свойства (например, «является заглавной буквой»), её нужно проверять отдельным `if`, а не веткой в той же цепочке `elif`.

Практика: консоль команд с историей

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/12-18/index.html>)

Практика: проверка пароля по учебным правилам

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/12-19/index.html>)

Проект: студия многоугольников

Одна и та же формула и один и тот же цикл рисуют треугольник, квадрат, пятиугольник, шестиугольник или восьмиугольник — в зависимости от одного числа.

ИСПОЛЬЗУЕМ

Turtle input()/int() for формула

УРОВЕНЬ

★★★ интеграционный

РЕЗУЛЬТАТ

геометрическая фигура

Что создаём

Программу, которая рисует ЛЮБОЙ правильный многоугольник — не переписывая код, а меняя одно число.

Формула

Правильный многоугольник с `storony` сторонами всегда поворачивает на один и тот же угол между сторонами:

```
formula_ugla.py
```

```
ugol = 360 / storony
```

storony	ugol
3	120°
4	90°
5	72°
6	60°
8	45°

Один алгоритм — пять результатов

```
studiya_mnogougolnikov.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=300, height=300)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(3)

storony = 5
dlina = 75
ugol = 360 / storony

for _ in range(storony):
    artist.forward(dlina)
    artist.right(ugol)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Правильный пятиугольник, нарисованный Turtle

storony = 5 → правильный пятиугольник

```
studiya_mnogougolnikov.py
```

```
storony = int(input("Сколько сторон? "))
dlina = 300 / storony
ugol = 360 / storony

for _ in range(storony):
    artist.forward(dlina)
    artist.right(ugol)
```

Данные меняются — алгоритм нет

Та же идея, что и в проекте «Викторина» (§12.9): цикл `for _ in range(storony)` и формула `ugol = 360 / storony` не меняются — меняется только введённое число.

```
storony = 3
```

```
storony = 3
```

```
storony = 4
```

```
storony = 4
```

```
storony = 5
```

```
storony = 5
```

```
storony = 6
```

```
storony = 6
```

```
storony = 8
```

```
storony = 8
```

Один и тот же код, разное значение `storony` — реально сгенерированные результаты

Debug

Lab

Что произойдёт, если ввести `storony = 1` или `storony = 2` ?

Формула работает только для `storony >= 3`

Многоугольника с одной или двумя сторонами не существует геометрически — код не упадёт с ошибкой ($360 / 1 = 360$), но нарисует не то, что ожидалось. Полноценная защита от такого ввода потребует проверки `if storony < 3` перед рисованием.

Практика: вычисляем угол поворота для многоугольника

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/12-20/index.html>)

Практика: рисуем многоугольник по введённому числу сторон

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/12-21/index.html>)

Проект: рождественская ёлка

Несколько треугольных ярусов уменьшающегося размера, нарисованных циклом.

ИСПОЛЬЗУЕМ

Turtle for begin_fill/end_fill

УРОВЕНЬ

★★ средний

РЕЗУЛЬТАТ

графика Turtle

Что создаём

Нарисуем ёлку из треугольных «ярусов» уменьшающегося размера — используя цикл и фигуры из главы 6.

elka.py

```

import turtle

screen = turtle.Screen()
screen.setup(width=340, height=380)
artist = turtle.Turtle()
artist.speed(0)
artist.hideturtle()
artist.pencolor("green")
artist.fillcolor("green")

yarusy = 4
shirina = 120

artist.penup()
artist.goto(0, 100)
artist.pendown()

for yarus in range(yarusy):
    artist.begin_fill()
    artist.setheading(240)
    artist.forward(shirina)
    artist.setheading(0)
    artist.forward(shirina)
    artist.setheading(120)
    artist.forward(shirina)
    artist.end_fill()

    artist.penup()
    artist.setheading(270)
    artist.forward(30)
    artist.pendown()
    shirina -= 20

artist.pencolor("brown")
artist.fillcolor("brown")
artist.setheading(270)
artist.penup()
artist.forward(10)

```

```
artist.pendown()
artist.begin_fill()
for _ in range(2):
    artist.forward(40)
    artist.right(90)
    artist.forward(20)
    artist.right(90)
artist.end_fill()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Рождественская ёлка, нарисованная Turtle из
уменьшающихся треугольных ярусов

Четыре яруса уменьшающегося размера + ствол

Каждый ярус — тот же треугольник, что и в главе 6
Формула угла поворота ($360 / 3 = 120$) — та же
самая, что и для любого правильного многоугольника
из главы 6 (и из проекта «Студия многоугольников»,
§12.11). Ёлка — это просто несколько треугольников
уменьшающегося размера, нарисованных друг под
другом в цикле.

Практика: рисуем ёлку

Модуль turtle открывает нативное окно Python — выполните
локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/12-03/index.html) ([https://www.cartesianschool.org/
practice/12-03/index.html](https://www.cartesianschool.org/practice/12-03/index.html))

Проект: спирали!

Один и тот же принцип — пять разных узоров, в
зависимости от угла поворота.

ИСПОЛЬЗУЕМ

Turtle for random (одна из версий)

УРОВЕНЬ

★★★ интеграционный

РЕЗУЛЬТАТ

графика Turtle · генеративное искусство

Что создаём

Пять вариаций одной идеи — фигура, каждый шаг которой немного больше предыдущего. Один принцип из главы 10 («шаг + поворот + увеличение переменной»), пять углов поворота.

Квадратная спираль

kvadratnaya_spiral.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=480)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(2)

dlina = 5
for _ in range(60):
    artist.forward(dlina)
    artist.right(90)
    dlina += 3

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Квадратная спираль Turtle

right(90) на каждом шаге

Случайная спираль

sluchaynaya_spiral.py

```
import turtle
import random

random.seed(4)
screen = turtle.Screen()
screen.setup(width=480, height=480)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#DB2777")
artist.pensize(2)

dlina = 5
for _ in range(60):
    artist.forward(dlina)
    artist.right(random.randint(80, 100))
    dlina += 3

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Спираль со случайно дрожащим углом поворота

Угол «дрожит» случайно в диапазоне 80-100°

Треугольная спираль

treugolnaya_spiral.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=480)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#059669")
artist.pensize(2)

dlina = 5
for _ in range(60):
    artist.forward(dlina)
    artist.right(120)
    dlina += 3

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Треугольная спираль Turtle

right(120) на каждом шаге

Звёздная спираль

zvezdnaya_spiral.py

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=480)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(2)

dlina = 5
for _ in range(100):
    artist.forward(dlina)
    artist.right(144)
    dlina += 2

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Звёздная спираль Turtle

right(144) — угол пятиконечной звезды

Круговая спираль

```
krugovaya_spiral.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=480, height=480)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#DB2777")
artist.pensize(2)

radius = 5
for _ in range(60):
    artist.circle(radius, 90)
    radius += 3

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Круговая спираль из дуг Turtle

`circle(radius, 90)` с растущим радиусом

Один общий принцип

Все пять спиралей — одна и та же идея из главы 10 («шаг + поворот + увеличение переменной») с разным углом поворота. Освоив один вариант, вы фактически освоили все пять.

Практика: спирали

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/12-04/index.html>)

Проект: сложная мандала — полностью автоматизированная

Финальная версия мандалы из глав 6 и 10 — со случайными цветами и кругами на концах лучей.

ИСПОЛЬЗУЕМ

Turtle for random.choice

УРОВЕНЬ

★★★★ challenge

РЕЗУЛЬТАТ

графика Turtle · генеративное искусство

Что создаём

Финальная эволюция мандалы из глав 6 и 10 — добавим случайный цвет на каждый луч и сделаем полностью автоматической, без единого «магического числа», прописанного вручную.

Как строится узор: от одного луча к полной мандале

Тот же принцип поэтапного построения, что и в главе 10: сначала один элемент, потом несколько, потом полный узор.

```
odin_luch.py
```

```
artist.circle(20)  
artist.forward(150)  
artist.forward(-150)
```

slozhnaya_mandala.py

```
import turtle
import random

random.seed(11)
screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)

luchi = 36
shag_ugla = 360 / luchi
cveta = ["red", "orange", "purple", "blue", "green"]

for i in range(luchi):
    artist.pencolor(random.choice(cveta))
    artist.setheading(i * shag_ugla)
    artist.forward(150)
    artist.circle(20)
    artist.forward(-150)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Сложная мандала Turtle из 36 лучей случайных цветов с кругами на концах

36 лучей, случайный цвет на каждом (seed зафиксирован для документации)

slozhnaya_mandala.py

```
import random

luchi = 36
shag_ugla = 360 / luchi
cveta = ["red", "orange", "purple", "blue", "green"]

for i in range(luchi):
    artist.pencolor(random.choice(cveta))
    artist.setheading(i * shag_ugla)
    artist.forward(150)
    artist.circle(20)
    artist.forward(-150)
```

luchi определяет всё остальное

Измените только `luchi` — угол шага, число повторов цикла и даже плотность узора пересчитаются автоматически, ничего больше менять не нужно. Это и называется «полностью автоматизировано».

Практика: сложная мандала со случайными цветами

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/12-05/index.html>)

Проект: пиксельная графика по сетке

Данные → алгоритм → графика: вложенный список решает, что нарисовать, а вложенный цикл — как.

ИСПОЛЬЗУЕМ

nested list nested for Turtle

УРОВЕНЬ

★★★★ challenge

РЕЗУЛЬТАТ

пиксельная графика

Что создаём

Мост между данными и графикой: матрица из нулей и единиц — и программа, которая рисует по ней картинку.

0 — пусто, 1 — закрашенный квадрат.

	0	1	2	3	4
0	0	1	0	1	0
1	1	1	1	1	1
2	1	1	1	1	1
3	0	1	1	1	0
4	0	0	1	0	0

picture — вложенный список: 0 = пусто, 1 =
закрашенный квадрат

```
setka_piksel_art.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=360, height=360)
artist = turtle.Turtle()
artist.speed(0)
artist.hideturtle()
artist.penup()

picture = [
    [0, 1, 0, 1, 0],
    [1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1],
    [0, 1, 1, 1, 0],
    [0, 0, 1, 0, 0],
]

razmer = 40
for row_index, row in enumerate(picture):
    for col_index, value in enumerate(row):
        if value == 1:
            x = -100 + col_index * razmer
            y = 100 - row_index * razmer
            artist.goto(x, y)
            artist.pendown()
            artist.fillcolor("#DB2777")
            artist.begin_fill()
            for _ in range(4):
                artist.forward(razmer)
                artist.right(90)
            artist.end_fill()
            artist.penup()

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Сердце из закрашенных квадратов, нарисованное по матрице нулей и единиц

Та же матрица, нарисованная вложенным циклом

Алгоритм

setka_piksel_art.py

```
picture = [  
    [0, 1, 0, 1, 0],  
    [1, 1, 1, 1, 1],  
    [1, 1, 1, 1, 1],  
    [0, 1, 1, 1, 0],  
    [0, 0, 1, 0, 0],  
]  
  
razmer = 40  
for row_index, row in enumerate(picture):  
    for col_index, value in enumerate(row):  
        if value == 1:  
            x = -100 + col_index * razmer  
            y = 100 - row_index * razmer  
            artist.goto(x, y)  
            artist.pendown()  
            artist.begin_fill()  
            for _ in range(4):  
                artist.forward(razmer)  
                artist.right(90)  
            artist.end_fill()  
            artist.penup()
```

Вложенный цикл — строка за строкой, столбец за столбцом

Внешний цикл (глава 10) идёт по строкам матрицы, внутренний — по значениям внутри строки; та же схема, что и у матрицы из главы 11 (§11.11), только теперь на каждую единицу рисуется квадрат, а не просто печатается число.

★★★ Задача повышенной сложности**Измените ОДНО: своя картинка**

Замените `picture` на свою собственную матрицу 5×5 (или другого размера) из нулей и единиц — буква, смайлик, любой узор. Алгоритм менять не нужно.

Debug

Lab

Что произойдёт, если в формуле координат перепутать местами `row_index` и `col_index` ?

```
debug_lab_grid.py
```

```
x = -100 + row_index * razmer    # было col_index
y = 100 - col_index * razmer     # было row_index
```

Картинка окажется зеркально отражённой (транспонированной)

Строки и столбцы поменяются местами — там, где должна быть широкая горизонтальная часть сердца, получится вытянутая вертикальная. Формулы координат нужно очень аккуратно сверять с тем, что означает каждый индекс.

Практика: пиксельная графика по своей матрице

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/12-22/index.html>)

Проект: гонка Turtle с использованием циклов

Несколько черепашек на одном экране одновременно — и подведение итогов всей проектной лаборатории.

ИСПОЛЬЗУЕМ

Turtle list while random

УРОВЕНЬ

★★★★ challenge

РЕЗУЛЬТАТ

графика Turtle

Что создаём

Помните из главы 6, что экран и черепашка — разные объекты, и черепашек может быть несколько? Настало время это использовать: гонка из нескольких черепашек со случайным шагом.

gonka_turtle.py

```

import turtle
import random

random.seed(2)
screen = turtle.Screen()
screen.setup(500, 400)

cveta = ["red", "blue", "green", "orange"]
uchastniki = []

for i, cvet in enumerate(cveta):
    t = turtle.Turtle()
    t.shape("turtle")
    t.color(cvet)
    t.penup()
    t.goto(-200, i * 40 - 60)
    uchastniki.append(t)

finish_line = 200
pobeditel = None

while pobeditel is None:
    for t in uchastniki:
        t.forward(random.randint(1, 10))
        if t.xcor() >= finish_line:
            pobeditel = t.pencolor()
            break

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Четыре цветные черепашки Turtle на разных позициях в конце гонки

Четыре черепашки в момент завершения гонки (seed зафиксирован для документации)

gonka_turtle.py

```
import random

screen = turtle.Screen()
screen.setup(500, 400)

cveta = ["red", "blue", "green", "orange"]
uchastniki = []

for i, cvet in enumerate(cveta):
    t = turtle.Turtle()
    t.shape("turtle")
    t.color(cvet)
    t.penup()
    t.goto(-200, i * 40 - 60)
    uchastniki.append(t)

finish_line = 200
pobeditel = None

while pobeditel is None:
    for t in uchastniki:
        t.forward(random.randint(1, 10))
        if t.xcor() >= finish_line:
            pobeditel = t.pencolor()
            break

print(f"Победила черепашка цвета {pobeditel}!")
```

Список черепашек — тот же список из главы 11
 uchastniki — обычный список Python (глава 11), просто хранящий не числа, а объекты turtle.Turtle(). Цикл for t in uchastniki перебирает его точно так же, как перебирал бы список чисел или строк.

★★★ Задача повышенной сложности

Финишная лента

Нарисуйте вертикальную линию на финише ($x=200$) перед началом гонки, используя отдельную черепашку-«судью».

Практика: гонка Turtle

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/12-06/index.html>)

Итоги главы

Что мы теперь умеем строить

- Что мы теперь умеем строить
 - Текстовые программы
 - Анализатор текста и частота слов
 - Викторина
 - Консоль команд
 - Записная книжка
 - Угадай число, версия 3
 - Программы с данными
 - Журнал оценок
 - Корзина покупок
 - Проверка пароля
 - Графика

- Студия многоугольников
- Пиксельная графика по сетке
- Спирали и сложная мандала
- Рождественская ёлка
- Гонка Turtle

16 проектов — и всё это работает на одних и тех же инструментах глав 1-11

Всё это питается одними и теми же инструментами глав 1-11

СТРОКИ

`input()`, `split()`, `.lower()`
форматирование f-строками

ЧИСЛА

арифметика, округление
`random`

УСЛОВИЯ

`if / elif / else`
булевы флаги

ЦИКЛЫ

for, while, break
счётчики, накопители

КОЛЛЕКЦИИ

list, dict, set
список словарей

TURTLE

формулы поворота
вложенные циклы → графика

Как далеко мы продвинулись

Глава 1

```
print("Hello")
```

Глава 12

интерактивная игра, анализатор текста, викторина, записная книжка, структурированные данные, графический генератор

Чек-лист готовности проекта

Полный чек-лист — в §12.2 «Строим проект по шагам». Перед тем как считать любой из 16 проектов главы законченным, полезно пройти по нему ещё раз.

Что дальше: зачем нужны функции

Присмотритесь к проектам этой главы: нормализация ввода (`.strip().lower()`) повторяется в анализаторе текста, в консоли команд и в викторине. Накопитель, который нужно завести до цикла — та же самая ошибка чинилась трижды в разных Debug Lab. Формула угла поворота Turtle появляется и в ёлке, и в мандале, и в студии многоугольников.

Скоро мы это упростим

Сейчас, если один и тот же блок кода нужен в нескольких местах, его приходится копировать. В следующей главе — **«Автоматизация с помощью функций»** — мы научимся давать группе команд имя и использовать её многократно, не копируя код заново. Ничего из глав 1-12 при этом не устареет — функции просто дадут более аккуратный способ организовать то, что мы уже умеем писать.

Что мы закрепили в этой главе

- **Декомпозиция** — разбиение большой задачи на маленькие понятные шаги.
- **Инкрементальная разработка** — маленькими шагами, каждый сразу проверяется, а не 80 строк за раз.
- **Данные отдельно от алгоритма** — викторина и студия многоугольников показали, что одна и та же логика работает для любых входных данных.
- **Рефакторинг** — улучшение структуры программы без изменения её поведения.
- Условия, циклы, строки, числа, `random`, списки/множества/словари и `Turtle` — впервые работали вместе, в одной программе, а не по отдельности.
- Системный воркфлоу отладки: ожидание vs реальность → первое расхождение → причина → исправление.
- Сложные результаты (викторина, анализатор текста, мандала) — всегда комбинация нескольких простых, уже знакомых приёмов.

Автоматизация с помощью функций

Функции · декомпозиция · параметры · `return` · области видимости. Научимся превращать большие программы в понятные части: давать алгоритмам имена, передавать им данные, получать результаты, управлять областью видимости и переиспользовать код без копирования — идея, которая переносится на любой другой язык программирования.

13.1 Зачем программе функции

13.2 Настоящая автоматизация. Первая функция

13.3 Что происходит во время вызова

13.4 Параметр и аргумент

13.5 Изменяемые и неизменяемые аргументы

13.6 Позиционные и именованные аргументы

13.7 Значения по умолчанию и `*args`

Ловушка изменяемого значения по умолчанию

13.8 `*args`, `**kwargs` и распаковка

13.9 Positional-only и keyword-only

13.10 Возвращаем ответ

13.11 Вход, работа, выход: чистые функции

13.12 Глобальные и локальные переменные

13.13 Вложенные функции и nonlocal

13.14 Стек вызовов и traceback

13.15 Проектируем хорошую функцию

13.16 Докстринги и подсказки типов

13.17 Функции как объекты

13.18 Лямбда-функции

13.19 Функции как конвейер

13.20 Рефакторим проекты главы 12

13.21 Debug Lab: типичные ошибки функций

13.22 Тестируем функции

13.23 Мини-проект — домашнее задание по математике

13.24 Мини-проект — анализатор текста v2

13.25 Мини-проекты — конвертер и утилиты коллекций

13.26 Мини-проект — Turtle Function Studio

Итоги главы

Зачем программе функции

Циклы автоматизируют повторение в потоке выполнения. Функции автоматизируют переиспользование поведения — и это не одно и то же.

Проблема, которую циклы не решают

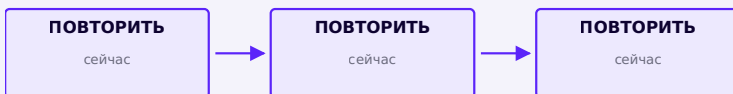
В главе 12 наши проекты становились всё крупнее — и в них стали появляться повторяющиеся логические блоки. Например, в викторине один и тот же набор действий — «спросить вопрос, нормализовать ответ, сравнить, посчитать» — фактически повторяется на каждой итерации. Цикл (глава 10) отлично с этим справляется, **пока повторение происходит подряд, в одном месте.**

Но что, если тот же самый набор действий нужен:

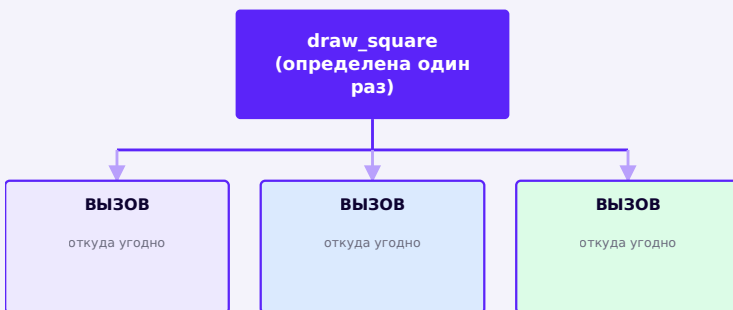
- в **разных местах** программы — не подряд;
- с **разными данными** каждый раз;
- из **разных проектов** вообще;
- **по требованию**, а не по расписанию цикла?

Цикл здесь не поможет — он умеет повторять только то, что стоит прямо у него в теле. Нужен другой инструмент: способ **дать этому действию имя** и вызывать его откуда угодно.

Цикл vs функция



Цикл: один поток повторяющегося выполнения



Функция: определена один раз, вызывается по требованию

Не альтернативы, а дополнение друг друга

Циклы автоматизируют повторение в потоке выполнения. Функции автоматизируют переиспользование поведения. Совсем скоро мы увидим, как функция вызывается ИЗНУТРИ цикла — они прекрасно работают вместе, а не вместо друг друга.

Функции — это декомпозиция

В главе 12 мы уже разбивали большую задачу на маленькие шаги — это называлось **декомпозицией**. Функция — это способ не просто мысленно выделить шаг, а буквально дать ему имя в коде:

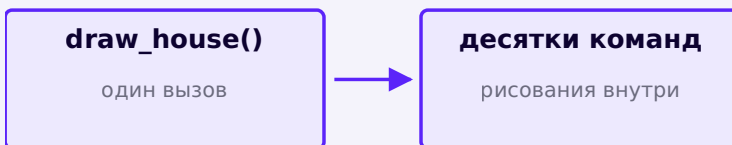


Большая задача → части с понятными именами —
это и есть декомпозиция в коде

Каждая функция — это осмысленное имя для одной части большего алгоритма.

Абстракция: полезный уровень детализации

Вы уже пользуетесь `print(...)`, не думая о том, как именно текст оказывается на экране. Точно так же вызов `draw_house()` мог бы скрыть десятки команд рисования крыши, стен и окон:



Вызов прячет детали реализации за понятным именем

Абстракция — не «спрятать всё», а выбрать полезный уровень детализации

Абстракция означает, что мы можем пользоваться осмысленной операцией, не думая каждый раз о каждой детали её реализации. Это не значит, что детали недоступны или неважны — просто для большинства задач достаточно знать ЧТО делает `draw_house()`, а не КАК именно.

Вы уже вызывали функции — с самой первой главы

`print()`, `input()`, `len()`, `type()`, `int()`, `range()`, `sorted()`, `min()`, `max()` — все они функции, просто **встроенные** в Python. Всё это время вы их **ВЫЗЫВАЛИ**. Теперь научимся их **СОЗДАВАТЬ**:

Встроенная функция	Ваша будущая функция
<code>len("Python")</code>	<code>calculate_area(10, 5)</code>

Общая модель одна и та же для обеих:



Практика: цикл или функция — что решает задачу

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-09/index.html>)

Настоящая автоматизация

Функции переиспользуют набор действий столько раз, сколько нужно — без копирования кода.

Настоящая автоматизация

Циклы из главы 10 автоматизировали повторение одного и того же кода. Но что если один и тот же *набор действий* нужен в разных местах программы — не подряд, а время от времени? Копировать код каждый раз — плохая идея: любое исправление придётся вносить во все копии. **Функции** решают эту проблему раз и навсегда.

Наша первая функция

```
pervaya_funkciya.py
```

```
def privetstvie():  
    print("Привет, Python!")  
  
privetstvie()  
privetstvie()  
privetstvie()
```

Анатомия def

Часть	Значение
<code>def</code>	ключевое слово: «дальше идёт определение функции»
<code>privetstvie</code>	имя функции — по нему функцию будут вызывать
<code>()</code>	список параметров (здесь пуст — функция не принимает данных)
<code>:</code>	начало тела функции
<code>print(...)</code>	тело — код с отступом, который выполнится при вызове

`def` **определяет** функцию — записывает её код, но пока не выполняет его. Функция выполняется только тогда, когда вы её **вызываете** — по имени со скобками: `privetstvie()`.

Определение — это ещё не вызов

Если написать только `def privetstvie(): ...` и не добавить `privetstvie()` ниже, программа не выведет ничего — Python просто запомнит функцию, но не запустит её.

Определение выполняется один раз, по-особому

Когда Python доходит до строки `def privetstvie():`, тело функции **не выполняется** как обычные последовательные команды. Вместо этого создаётся объект функции, привязанный к имени `privetstvie` — и выполнение программы продолжается СРАЗУ ПОСЛЕ определения. Тело запустится только позже, когда произойдёт вызов.

Функция как объект

Мы уже знаем модель «имя указывает на объект» (глава 3). Функция — не исключение:

`privetstvie`



ФУНКЦИЯ-ОБЪЕКТ

После `def` имя `privetstvie` указывает на объект функции

```
imya_bez_skobok.py
```

```
print(privetstvie)
# <function privetstvie at 0x...> — сам объект функции
privetstvie()      # Привет, Python! — а это ВЫЗОВ
```

privetstvie и privetstvie() — не одно и то же

Без скобок — обращение к самому объекту функции (её можно передать, сохранить, сравнить). Со скобками — команда «выполни её прямо сейчас». Это различие станет особенно важным в §13.17, когда функции начнут передаваться как обычные значения.

Практика: определяем и вызываем первую функцию

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/13-01/index.html\)](https://www.cartesianschool.org/practice/13-01/index.html)

Что происходит во время вызова

Вызов функции — это переход управления внутрь тела функции и обязательный возврат к месту вызова, а не «выполнение где-то в стороне».

Вызов — это не «выполнение где-то в стороне»

Разберём по шагам, что реально происходит, когда программа доходит до `greet()`:

```
poryadok_vypolneniya.py
```

```
def greet():  
    print("Привет")  
  
print("До")  
greet()  
print("После")
```

```
До  
Привет  
После
```

Порядок вывода — сверху вниз, как и порядок выполнения строк

Управление потоком выполнения

Вызов передаёт управление в тело функции; после её завершения управление возвращается точно на следующую строку

Учащийся должен буквально **увидеть**: вызов → переход внутрь функции → выполнение тела → возврат к месту вызова → продолжение со следующей строки. Функция не выполняется «где-то в стороне» — выполнение программы буквально ныряет внутрь неё и возвращается обратно.

Место вызова

Строка, где стоит `greet()`, называется **местом вызова** (call site). Именно сюда возвращается управление, когда функция заканчивается:

```
mesto-vyzova.py
```

```
result = calculate(2, 3)
# ↑ это место вызова — после calculate() выполнение
# продолжится СРАЗУ ЗДЕСЬ
```

Практика: предсказываем порядок вывода

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/13-10/index.html\)](https://www.cartesianschool.org/practice/13-10/index.html)

Зачем нужны функции?

Параметр — это имя в определении. Аргумент — значение, переданное при вызове. Разница между ними — фундамент всего, что будет дальше.

Зачем нужны функции?

- **Код не повторяется** — исправление вносится в одном месте.
- **Программу легче читать** — имя функции объясняет, что происходит, не заставляя вникать в детали реализации.
- **Легче искать ошибки** — если что-то сломалось в приветствии, вы точно знаете, где искать: внутри `privetstvie()`.

Каждый раз делаем что-то новое!

Функция без входных данных всегда делает одно и то же. Чтобы функция вела себя по-разному в зависимости от ситуации, ей передают **аргументы** — значения в скобках при вызове:

```
argumenty.py

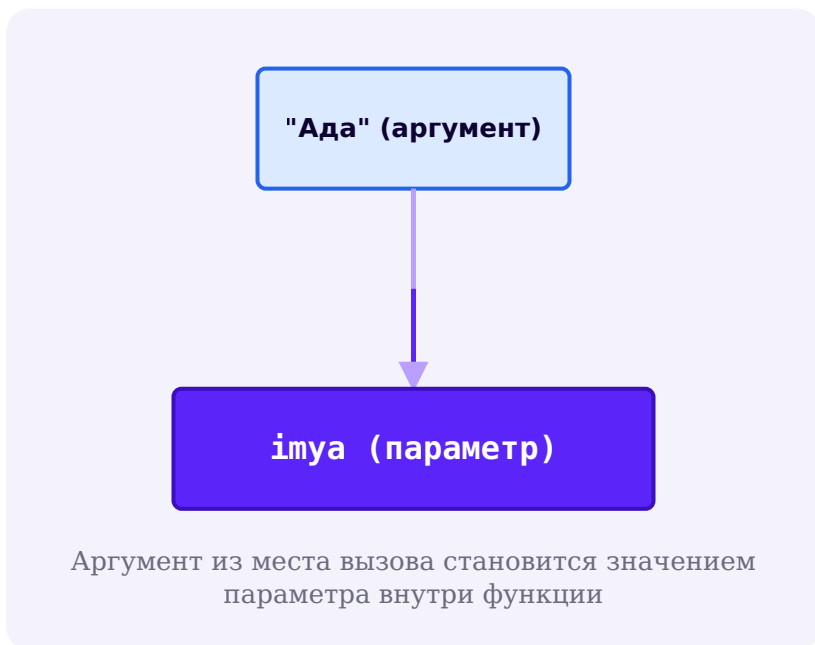
def privetstvie(imya):
    print(f"Привет, {imya}!")

privetstvie("Ада")
privetstvie("Cartesian")
```

Параметр vs аргумент — точное различие

Эти два слова легко перепутать, но у них разные роли:

	Где встречается	Что это
Параметр	в определении: <code>def</code> <code>privetstvie(imya</code> <code>):</code>	локальное имя внутри функции
Аргумент	в вызове: <code>privetstvie("Ада</code> <code>")</code>	объект/значение, переданное при вызове



Параметры — это локальные имена

Продолжим уже знакомую модель «имя указывает на объект» (глава 3): вызов функции создаёт новую привязку параметра к переданному объекту:



«Передача по значению» и «по ссылке» — вводят в заблуждение

Не думайте о Python как о языке, где «всё передаётся по ссылке» или «всё передаётся по значению» — обе формулировки слишком неточны и здесь не нужны. Точнее так: **при вызове функции её параметры привязываются к тем же объектам, что и переданные аргументы** — та же модель «имя → объект», что мы используем с самой главы 3. Иногда это называют *call by sharing*, но специальный термин запоминать не обязательно — важна сама модель.

Без аргументов?

Функция без параметров в скобках — например, `privetstvie()` из §13.2 — тоже совершенно нормальна: не каждой функции нужны входные данные.

Практика: параметр vs аргумент

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/13-02/index.html\)](https://www.cartesianschool.org/practice/13-02/index.html)

Изменяемые и неизменяемые аргументы

Один из самых важных уроков главы: присваивание параметру и мутация объекта, на который он указывает, — это два совершенно разных действия.

Неизменяемый аргумент: rebinding не выходит наружу

```
immutable_argument.py
```

```
def add_one(number):  
    number += 1  
    print(number)  
  
x = 10  
add_one(x)  
print(x)
```

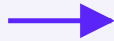
```
11
```

```
10
```

`add_one(x)` печатает 11, но `x` снаружи остался 10

x

10

До вызова: $x \rightarrow 10$ **x**

10

number

11

Внутри `add_one`: `number += 1` привязывает `number` к НОВОМУ объекту 11 — `x` это не затрагивает

Не «функция получает копию x»

Точнее: `number` сначала указывает туда же, куда и `x` (на 10). Но `number += 1` — это `number = number + 1`: создаётся НОВЫЙ объект 11, и `number` начинает указывать на него. `x` по-прежнему указывает на старый объект 10 — они разошлись.

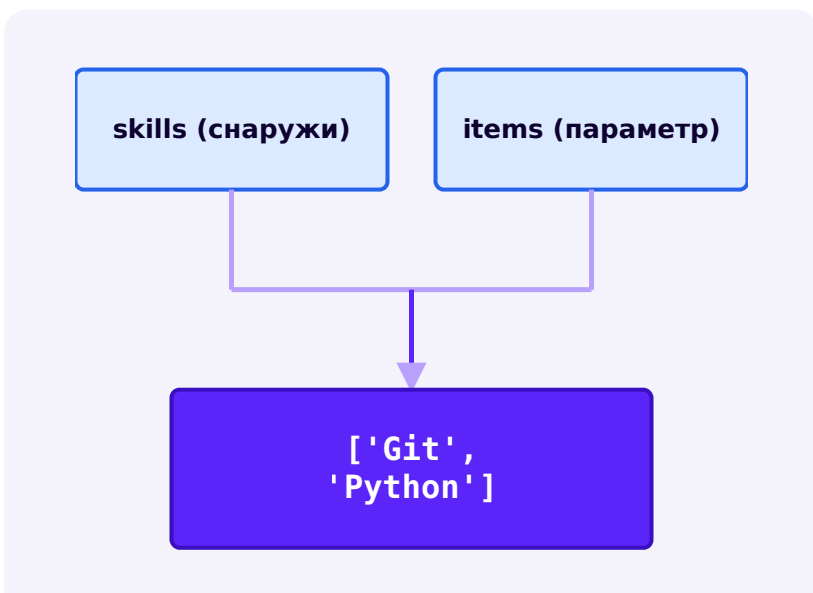
Изменяемый аргумент: мутация видна снаружи

```
mutable_argument.py
```

```
def add_item(items):  
    items.append("Python")  
  
skills = ["Git"]  
add_item(skills)  
print(skills)
```

```
['Git', 'Python']
```

skills изменился, хотя мы вызвали функцию, а не присвоили skills новую



`skills` и `items` — два имени ОДНОГО списка;
`items.append(...)` меняет его для обоих

Это не магия — это тот же `aliasing` из главы 11

`items` внутри функции указывает на ТОТ ЖЕ объект-список, что и `skills` снаружи. `.append(...)` мутирует этот общий объект — поэтому изменение видно через оба имени. Это ровно тот же принцип `aliasing`, что мы разбирали в §11.9, только теперь одно из «двух имён» — параметр функции.

Rebinding vs mutation — сравниваем напрямую

`rebinding_vs_mutation.py`

```
def replace(items):
    items = ["new"]      # rebinding: items теперь
                          # указывает на ДРУГОЙ список

def modify(items):
    items.append("new")  # mutation: меняется ТОТ ЖЕ
                          # список
```

	<code>replace(items)</code>	<code>modify(items)</code>
Что происходит	переприсваивание локального имени <code>items</code>	изменение объек- та, на который <code>items</code> указывает
Видно снаружи после вызова?	нет — снаружи всё как было	да — тот же объ- ект изменился

Главное практическое правило

Присваивание параметру (`items = ...`) отвязывает локальное имя от аргумента — снаружи ничего не меняется. Вызов мутлирующего метода (`.append()` , `.sort()` , `[i] = ...`) меняет сам объект — и это видно снаружи, если объект изменяемый.

Практика: rebinding vs mutation

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-11/index.html>)

Позиционные и именованные аргументы

Два способа сообщить функции, какой аргумент к какому параметру относится — у каждого свои плюсы.

Несколько параметров

```
neskolko_parametrov.py
```

```
def rectangle_area(width, height):
    return width * height

print(rectangle_area(10, 5))    # 50
```

Аргумент	Параметр
10	width
5	height

Позиционные аргументы

По умолчанию Python сопоставляет аргументы с параметрами **по порядку** — первый аргумент попадает в первый параметр, и так далее:

pozicionnye.py

```
create_user("Anna", 25)
# position 1 → name = "Anna"
# position 2 → age = 25
```

Перепутанный порядок — не всегда ошибка Python, но всегда ошибка логики

`create_user(25, "Anna")` может не вызвать никакой ошибки типов — Python не запрещает передать число туда, где обычно передают строку. Но результат окажется логически неверным: `name` получит `25`, а `age` — `"Anna"`. Позиционные аргументы удобны, но требуют помнить точный порядок.

Именованные аргументы

Аргумент можно передать по имени параметра — тогда порядок уже не важен:

imennye.py

```
create_user(
    name="Anna",
    age=25,
)
```

Зачем именованные аргументы

ЧИТАЕМОСТЬ

видно, что есть что, прямо в вызове

ПОРЯДОК НЕВАЖЕН

age=25, name="Anna" — сработает так же

УДОБСТВО API

особенно при многих параметрах

Смешиваем позиционные и именованные

smeshannye.py

```
def function(width, height, color):  
    ...  
  
function(10, height=20, color="red")    # МОЖНО
```

Правило порядка

Позиционные аргументы всегда идут ПЕРЕД именованными в вызове — `function(width=10, 20, "red")` вызовет `SyntaxError`. И один и тот же параметр нельзя получить дважды — `function(10, 20, width=5)` вызовет `TypeError`, так как `width` уже получил значение позиционно.

Практика: позиционные и именованные аргументы

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-12/index.html>)

Нет аргументов? Слишком много аргументов!

Значения по умолчанию делают аргумент необязательным — но изменяемое значение по умолчанию скрывает одну из самых известных ловушек Python.

Нет аргументов? Что делать!

Если вызвать функцию без обязательного аргумента, Python сообщит об ошибке. Чтобы аргумент можно было пропустить, ему задают **значение по умолчанию**:

znachenie_po_umolchaniyu.py

```
def privetstvie(imya="друг"):
    print(f"Привет, {imya}!")

privetstvie()           # Привет, друг!
privetstvie("Ада")      # Привет, Ада!
```

Сигнатура	Роль
<code>imya</code>	обязательный параметр — если бы не было умолчания
<code>imya="друг"</code>	необязательный параметр со значением по умолчанию

Ловушка изменяемого значения по умолчанию

Это один из самых известных сюрпризов Python — и его нужно разобрать не торопясь.

mutable_default_trap.py

```
def add_task(task, tasks=[]):
    tasks.append(task)
    return tasks

print(add_task("A"))
print(add_task("B"))
```

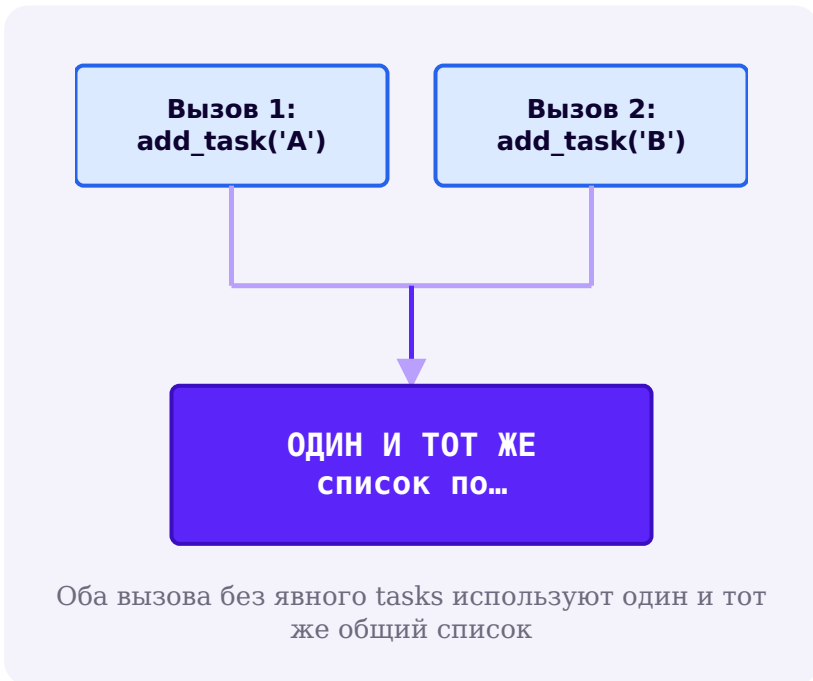
```
['A']
['A', 'B']
```

Второй вызов не начинается с пустого списка — В добавился к тому же списку, что и А!

Список по умолчанию создаётся ОДИН РАЗ — когда выполняется def

`tasks=[]` вычисляется в момент **определения** функции, а не заново при каждом вызове. Получается один и тот же список-объект по умолчанию, общий для всех вызовов, где аргумент не передан явно.

Список по умолчанию создаётся один раз, когда Python выполняет саму строку `def`



Правильный паттерн: `None` как отметка «не передано»

`mutable_default_fix.py`

```
def add_task(task, tasks=None):  
    if tasks is None:  
        tasks = []  
  
    tasks.append(task)  
    return tasks
```

None — безопасный сигнальный аргумент по умолчанию

None — неизменяемый одиночный объект, поэтому его можно безопасно использовать как значение по умолчанию. Проверка `if tasks is None` (глава 9: `is None`) создаёт НОВЫЙ пустой список при каждом вызове без аргумента — ровно то поведение, которое интуитивно ожидалось. Это не единственный способ решить проблему, но самый распространённый.

Слишком много аргументов!

Иногда заранее неизвестно, сколько аргументов понадобится передать. Символ `*` перед именем параметра собирает **любое** количество аргументов в один кортеж (глава 11):

args.py

```
def summa_vseh(*chisla):
    itog = 0
    for n in chisla:
        itog += n
    return itog

print(summa_vseh(1, 2))           # 3
print(summa_vseh(1, 2, 3, 4, 5)) # 15 — сколько
                                  угодно аргументов
```

Именованные аргументы — напоминание

Аргументы можно передавать и по имени, а не только по порядку: `privetstvie(imya="Ада")` — подробнее об этом в §13.6.

Практика: значения по умолчанию и ловушка изменяемого умолчания

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-04/index.html>)

***args, **kwargs и распаковка**

*args собирает лишние позиционные аргументы в кортеж, **kwargs — лишние именованные в словарь. Те же символы на месте вызова распаковывают коллекцию обратно в аргументы.

***args — произвольное число позиционных аргументов**

args_povtorenie.py

```
def total(*numbers):
    return sum(numbers)

total(1, 2, 3, 4)
```

numbers



(1, 2, 3, 4)

Внутри функции numbers — обычный кортеж (глава 11)

args — просто общепринятое имя

Работает именно символ `*` перед параметром — имя `args` лишь общепринятое соглашение. `def total(*numbers)` работает совершенно так же, как `def total(*args)`.

****kwargs — произвольное число именованных аргументов**

kwargs.py

```
def show_profile(**fields):  
    for key, value in fields.items():  
        print(f"{key}: {value}")  
  
show_profile(  
    name="Anna",  
    city="Warsaw",  
)
```

fields



```
{"name": "Anna",  
 "city": ...
```

Внутри функции `fields` — обычный словарь (глава 11)

**** собирает лишние именованные аргументы в словарь**

Любые именованные аргументы, для которых нет отдельного параметра, попадают в словарь `fields` — ключ - имя аргумента, значение - переданное значение.

*args vs **kwargs

```
vyzov_s_oboimi.py
```

```
func(10, 20, color="red", size=3)
```

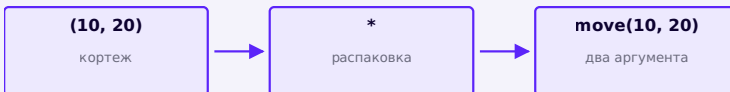
	*args	**kwargs
Собирает	лишние позиционные аргументы	лишние именованные аргументы
Тип внутри функции	tuple	dict
В примере выше	(10, 20)	{'color': 'red', 'size': 3}

Распаковка аргументов на месте вызова

Тот же символ `*` / `**`, но теперь на стороне ВЫЗОВА, а не определения — связываем с распаковкой из главы 11:

```
raspakovka_pri_vyzove.py
```

```
point = (10, 20)
move(*point)  # эквивалентно move(10, 20)
```



* перед аргументом при вызове раскладывает кортеж/список на отдельные позиционные аргументы

raspakovka_slovary.py

```
options = {
    "color": "red",
    "size": 3,
}
draw(**options)    # эквивалентно draw(color="red",
size=3)
```

* и ** встречаются в нескольких разных контекстах

Контекст	Пример	Что делает
Определение функции	<code>def f(*args, **kwargs):</code>	собирает лишние аргументы
Вызов функции	<code>f(*values, **mapping)</code>	распаковывает коллекцию в аргументы
Литерал коллекции (глава 11)	<code>[*a, *b]</code>	объединяет коллекции

Один символ, разный смысл в зависимости от контекста

* и ** не означают везде буквально одну и ту же операцию — их роль зависит от того, где они стоят: в определении функции, в вызове или в литерале коллекции.

Практика: *args, **kwargs и распаковка при вызове

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-13/index.html>)

Positional-only и keyword-only

Иногда хочется заставить аргумент передаваться только по имени (или только по позиции) — сигнатура функции умеет это явно требовать.

Keyword-only параметры — практически уже сейчас

Бывают параметры, которые ХОЧЕТСЯ заставить передавать только по имени — обычно это необязательные флаги или настройки, где голая позиция путает читателя:

keyword_only.py

```
def draw_rectangle(width, height, *, color="blue",
    filled=False):
    ...

draw_rectangle(100, 50, color="red")    # МОЖНО
```

Одинокая * в списке параметров

Всё, что стоит ПОСЛЕ одинокого * в определении функции, обязано передаваться по имени при вызове — позиционно эти параметры передать нельзя.

```
bez_keyword_only.py
```

```
draw_rectangle(100, 50, "red", True)
# что здесь True? filled? Неочевидно.
```

```
s_keyword_only.py
```

```
draw_rectangle(
    100,
    50,
    color="red",
    filled=True,
)
# сразу понятно, что есть что
```

Второй вариант читается и понимается значительно быстрее — именно ради этого существуют keyword-only параметры.

Чуть глубже — positional-only параметры

Реже, но тоже встречается обратная ситуация: параметр, который МОЖНО передавать только позиционно, без имени.

```
positional_only.py
```

```
def function(x, /):
    ...
```

Одинокая / в списке параметров

Всё, что стоит ДО / , можно передать только позиционно — имя параметра нельзя использовать при вызове. Так делают, когда автор функции не хочет, чтобы имя параметра стало частью «контракта» — его можно будет свободно менять в будущем, не ломая код, который её вызывает.

Анатомия полной сигнатуры

```
signatura.py
```

```
def draw(
    x, y, /,
    width, height,
    *,
    color="blue",
    filled=False,
):
    ...
```

Одна сигнатура, три зоны

X, Y

ТОЛЬКО ПОЗИЦИОННО

ДО /

WIDTH, HEIGHT

позиционно ИЛИ по имени
между / и *

COLOR, FILLED

только по имени
после *

Не обязательно запоминать наизусть с первого раза

Это продвинутая, но ценная грамотность чтения API — она особенно пригодится при чтении документации сторонних библиотек. Для собственных небольших функций чаще всего достаточно обычных параметров без / и одинокой * — используйте их осознанно, когда это действительно улучшает читаемость вызова.

Практика: keyword-only параметры

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-14/index.html>)

Возвращаем ответ

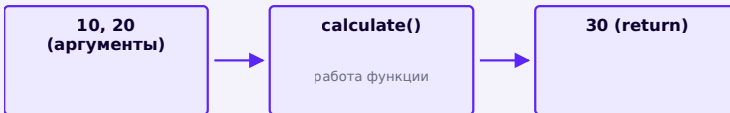
return передаёт результат работы функции обратно туда, откуда она была вызвана — а функция без return всё равно кое-что возвращает: None.

До сих пор наши функции только печатали текст. Но часто нужно не вывести результат на экран, а **вернуть** его — чтобы использовать дальше в программе. Ключевое слово `return` делает именно это:

return.py

```
def summa(a, b):
    return a + b

result = summa(5, 7)
print(result)          # 12
print(summa(2, 3) * 10) # результат функции можно
                        # использовать сразу — 50
```



Вход → работа функции → результат уходит обратно
в место вызова

`print()` внутри функции — это не то же самое, что `return`

Функция, которая только печатает результат, ничего не *возвращает* — попытка сохранить её результат в переменную даст `None` :

```
print_vs_return.py
```

```
def summa_pechataet(a, b):
    print(a + b)    # выводит на экран, но не
                   # возвращает

x = summa_pechataet(5, 7)  # на экране появится 12
print(x)                 # но x — это None!
```

	print(...) внутри функции	return ... внутри функции
Результат	текст на экране	объект уходит в место вызова
x = f(...)	x станет None	x станет вычисленным значением

Неявный None

Если функция доходит до конца тела, ни разу не выполнив `return`, Python сам возвращает `None`:

```
implicit_none.py
```

```
def hello():
    print("Hello")

result = hello()
print(result)
```

```
Hello
```

```
None
```

`hello()` ничего не `return`-ит — Python сам подставляет `None`

None — настоящий объект, а не «пустота»

`None` — это конкретный объект Python, обозначающий отсутствие значения в данном контексте (глава 9). Функция без `return` не «ничего не возвращает» в смысле полного отсутствия результата — она возвращает именно объект `None`.

Голый `return`

`return` без выражения после него тоже возвращает `None` — и сразу завершает функцию. Полезно для досрочного выхода:

bare_return.py

```
def process(value):
    if value is None:
        return
    print(value)
```

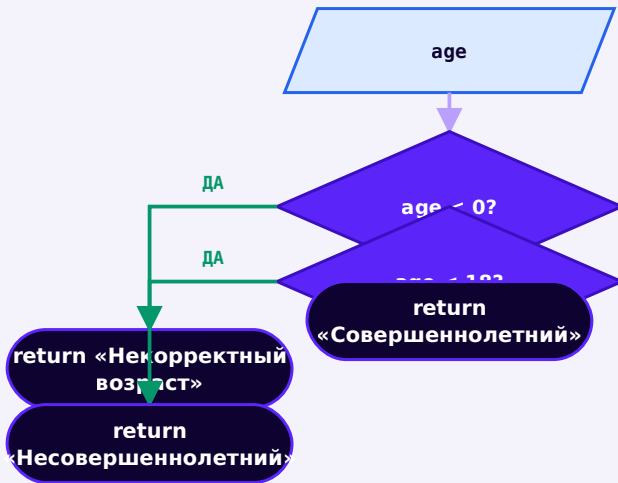
Ранний `return`

early_return.py

```
def classify_age(age):
    if age < 0:
        return "Некорректный возраст"

    if age < 18:
        return "Несовершеннолетний"

    return "Совершеннолетний"
```

Каждый `return` завершает функцию немедленно — код после него в этом вызове не выполнится

return сразу завершает функцию

Как только выполняется `return`, функция немедленно заканчивает работу — код после него внутри функции не выполнится.

Возвращаем несколько значений

`neskolko_znachenij.py`

```

def min_max(numbers):
    return min(numbers), max(numbers)

low, high = min_max([3, 7, 1, 9])
print(low, high)    # 1 9
  
```

На самом деле возвращается один кортеж

`return a, b` на уровне Python создаёт и возвращает ОДИН объект — кортеж `(a, b)` (глава 11). Функция не «возвращает два объекта одновременно» — она возвращает один кортеж, который затем можно распаковать в две переменные при вызове, точно как любой другой кортеж.

Практика: `return`, `implicit None`, ранний `return`

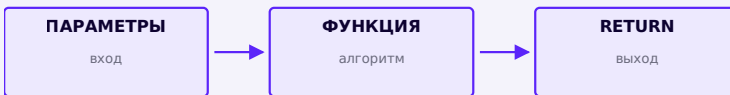
Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-03/index.html>)

Вход, работа, выход: чистые функции

Не каждая функция обязана что-то возвращать — но полезно чётко понимать, ради чего написана каждая конкретная функция: ради результата или ради эффекта.

Универсальная модель функции



Параметры → тело функции → `return` — модель, применимая к любой функции с результатом

Пример	Вход	Выход
<code>convert_temperature</code>	градусы Цельсия	градусы Фаренгейта

Пример	Вход	Выход
<code>normalize</code>	сырой текст	очищенный текст
<code>calculate_average</code>	список чисел	одно число

Два законных вида функций

Не каждая функция обязана возвращать значение — у функций есть две законные роли:

Функция-команда / эффект	Функция-вычисление
<code>show_menu()</code> , <code>draw_square()</code> , <code>print_report()</code>	<code>area(...)</code> , <code>normalize(...)</code> , <code>average(...)</code>
главная цель — эффект (вывод, рисование)	главная цель — вернуть значение

Оба вида — легитимны

Не каждая функция обязана что-то возвращать осмысленное — `draw_square()` вполне нормальна, даже если она ничего не `return`-ит: её работа — эффект на экране, а не значение.

Побочный эффект

Побочный эффект — это когда функция меняет что-то ВНЕ своего возвращаемого результата: печатает на экран, рисует Turtle, мутирует переданный список, позже — пишет в файл.

```
storonnij_effekt.py
```

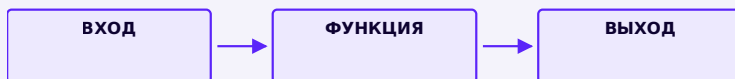
```
def add_item(items):  
    items.append("Python")    # побочный эффект:  
    мутирует переданный список
```

Чистые функции

Чистая функция — результат зависит только от аргументов, и функция намеренно ничего не меняет вне себя:

```
chistaya_funkciya.py
```

```
def rectangle_area(width, height):  
    return width * height
```



Чистая функция: только вход → результат



Функция с побочным эффектом: результат ИЛИ/И эффект вовне

Почему чистые функции удобны

ЛЕГЧЕ ПОНИМАТЬ

результат зависит только от входа

ЛЕГЧЕ ТЕСТИРОВАТЬ

не нужен экран/файл/сеть — просто
вызов

ПРЕДСКАЗУЕМОСТЬ

один и тот же вход → всегда один и тот
же выход

Не каждая функция ДОЛЖНА быть чистой

Графика, ввод/вывод, работа с интерфейсом — всё это по своей природе требует побочных эффектов.

Чистота — полезное свойство там, где оно естественно, а не универсальное правило, которому обязана подчиняться каждая функция программы.

Практика: чистые функции vs функции с побочным эффектом

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-15/index.html>)

Глобальные и локальные переменные

Где «живёт» переменная и в каком порядке Python её ищет — модель LEGB избавит от многих загадочных ошибок.

Переменные внутри функций

Переменная, созданная внутри функции, называется **локальной** — она существует только пока функция выполняется и недоступна снаружи:

```
lokalnye_peremennye.py
```

```
def moya_funkciya():
    message = "Я живу только внутри функции"
    print(message)

moya_funkciya()
print(message)  # NameError: message не определена
здесь
```

Глобальные переменные

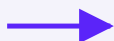
Глобальная переменная объявлена вне всех функций — и доступна для чтения внутри любой из них:

```
globalnye_peremennye.py
```

```
course_name = "Python"

def lesson():
    topic = "Функции"
    print(course_name)  # чтение глобальной –
                        # работает без всяких оговорок
    print(topic)

lesson()
```

`course_name``"Python"``lesson`

ФУНКЦИЯ-ОБЪЕКТ

Глобальное пространство имён

`topic``"Функции"`Локальное пространство имён `lesson()` во время
вызова**Функция может смотреть наружу, но не наоборот**

Функция может ЧИТАТЬ глобальные имена, но переменные, созданные внутри неё, не становятся автоматически видны снаружи.

Затенение (shadowing)

shadowing.py

```
name = "GLOBAL"

def demo():
    name = "LOCAL"
    print(name)

demo()
print(name)
```

LOCAL
GLOBAL

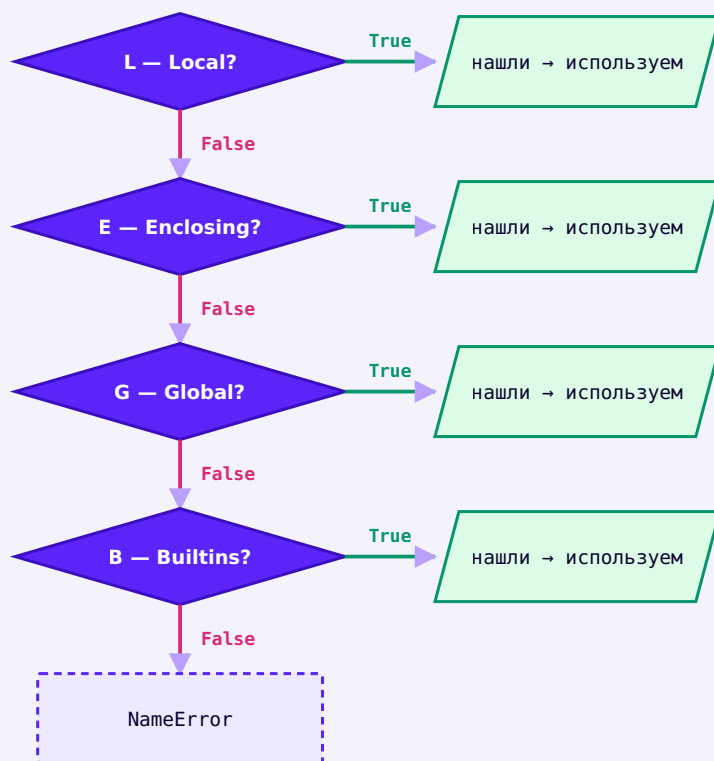
Локальное имя `name` существует только внутри `demo()` и не трогает глобальное

Локальное имя затеняет глобальное с тем же именем

Внутри `demo()` создаётся СВОЁ, отдельное локальное имя `name` — оно временно «заслоняет» глобальное `name` для кода внутри функции, но не заменяет и не удаляет его.

LEGB: как Python ищет имя

Когда Python встречается имя, он ищет его по чёткому порядку — эту модель называют **LEGB**:



Local → Enclosing → Global → Builtins — поиск останавливается на первом совпадении

Буква	Что это	Пример
L — Local	имена внутри текущей функции	<code>topic</code> внутри <code>lesson()</code>
E — Enclosing	имена внешней функции для вложенной (§13.13)	рассмотрим дальше

Буква	Что это	Пример
G — Global	имена на уровне модуля	<code>course_name</code>
B — Builtins	встроенные имена Python	<code>len</code> , <code>print</code>

UnboundLocalError — почему это НЕ случайность

unbound_local.py

```
count = 10

def increment():
    count += 1

increment()
```

Присваивание где-либо в теле делает имя локальным ВО ВСЁМ теле функции

Python видит `count += 1` (это `count = count + 1`) и заранее решает: раз `count` присваивается внутри функции, значит, это ЛОКАЛЬНОЕ имя — на протяжении ВСЕГО тела функции, а не только после присваивания. Но правая часть `count + 1` пытается ПРОЧИТАТЬ `count` ДО того, как локальное значение вообще появилось — отсюда `UnboundLocalError`. Это не случайное поведение, а прямое следствие того, как Python заранее размечает локальные имена функции.

global — если действительно нужно изменить глобальную переменную

global_keyword.py

```
count = 10

def increment():
    global count
    count += 1

increment()
print(count)  # 11
```

global нужен только для присваивания, не для чтения

Просто ПРОЧИТАТЬ глобальную переменную можно без `global` (мы это уже делали с `course_name` выше). `global` нужен, только когда функция собирается ИЗМЕНИТЬ (присвоить) глобальное имя.

global работает, но чаще есть способ лучше

`global` — не запрещённая, но зачастую не лучшая практика: функции, которые читают параметры и возвращают результат через `return`, легче тестировать и переиспользовать, потому что их поведение не зависит от скрытого внешнего состояния. Это не абсолютное правило «глобальные переменные — всегда плохо» — иногда общее состояние действительно уместно, — но по умолчанию стоит предпочитать параметры и `return`.

Практика: локальные, глобальные переменные и LEGB

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-05/index.html>)

Вложенные функции и nonlocal

Функция может быть определена прямо внутри другой функции — а `nonlocal` позволяет ей изменить имя из объемлющей функции, а не только прочесть его.

Функция внутри функции

Функцию можно определить прямо внутри тела другой функции:

```
vlozhennaya_funkciya.py
```

```
def outer():  
    message = "Hello"  
  
    def inner():  
        print(message)  
  
    inner()  
  
outer()
```

inner видит message — это и есть Enclosing из LEGB
`inner()` не имеет своего `message`, но находит его в объемлющей функции `outer()` — это буква «Е» (Enclosing) из ладдера LEGB (§13.12).

Отступ — это не украшение, а объявление вложенности

Если случайно сдвинуть `def inner():` на уровень отступа меньше, чем нужно, она перестанет быть вложенной в `outer` — и наоборот. Отступ буквально определяет, какая функция находится «внутри» какой.

nonlocal

Как и с глобальными переменными, ПРОЧИТАТЬ имя из объемлющей функции можно свободно, а вот ИЗМЕНИТЬ (присвоить) — нужно явно попросить:

```
nonlocal.py

def outer():
    count = 0

    def inner():
        nonlocal count
        count += 1

    inner()
    inner()
    print(count)

outer()
```

2

`nonlocal` позволил `inner()` изменить `count` из `outer()`, а не создать свою локальную копию

nonlocal — это НЕ global

`nonlocal` ищет имя в БЛИЖАЙШЕЙ подходящей объемлющей функции — не на уровне модуля. Если убрать `nonlocal count`, получится та же ошибка `UnboundLocalError`, что и в §13.12 — по той же самой причине: присваивание внутри `inner` сделало бы `count` локальным именем самой `inner`.

Чуть глубже — замыкание (closure)

Вложенная функция способна «запомнить» значения из объемлющей функции даже после того, как внешний вызов уже завершился:

closure.py

```
def make_greeter(name):
    def greet():
        print(f"Привет, {name}!")
    return greet

greet_anna = make_greeter("Anna")
greet_anna()
# Привет, Anna! — хотя make_greeter уже закончила
# работу
```

Замыкание — тема на будущее

Это называется **замыканием** — `greet` «уносит с собой» доступ к `name`. Здесь достаточно увидеть, что так вообще бывает — подробный разбор замыканий и их практических применений (декораторы и не только) не входит в эту главу.

Практика: вложенные функции и `nonlocal`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/13-16/index.html) → (<https://www.cartesianschool.org/practice/13-16/index.html>)

Стек вызовов и traceback

Функции, вызывающие функции, образуют стек ожидающих вызовов — и traceback из главы 3 на самом деле показывает именно этот стек в момент ошибки.

Кадр вызова (call frame)

Когда функция вызывается, Python создаёт временный «контейнер» для этого конкретного вызова — с параметрами, локальными именами и отметкой, куда вернуться после завершения:

```
call_frame.py
```

```
def area(width, height):  
    return width * height  
  
area(5, 3)
```

width



5

height



3

Кадр вызова `area(5, 3)`: параметры существуют, пока этот конкретный вызов не завершится

Функции могут вызывать другие функции

```
vlozhennye_vyzovy.py
```

```
def calculate_tax(amount):  
    return amount * 0.2  
  
def calculate_invoice(amount):  
    tax = calculate_tax(amount)  
    return amount + tax  
  
calculate_invoice(100)
```

Пока `calculate_tax()` выполняется, вызов `calculate_invoice()` не заканчивается — он «ждёт» на паузе. Python отслеживает все ожидающие вызовы в структуре, которую называют **стеком вызовов**.

Стек растёт и уменьшается

Шаг 1 — выполняется основная программа

Шаг 2 — `main` вызвала `calculate_invoice()`

Шаг 3 — `calculate_invoice` вызвала `calculate_tax()`;
стек на пике

Шаг 4 — `calculate_tax()` вернула результат и исчезла
со стека

Последним вошёл — первым вышел

Стек вызовов растёт с каждым новым вызовом и уменьшается с каждым завершением — самый недавний вызов всегда наверху и завершается первым. Это называют принципом LIFO (Last In, First Out).

Traceback — это и есть снимок стека вызовов

Вспомним отладку из главы 3: когда возникает ошибка, Python печатает `traceback` — и это буквально показывает цепочку вызовов, которая привела к ошибке:

```
traceback_primer.py
```

```
def divide(a, b):  
    return a / b  
  
def calculate():  
    return divide(10, 0)  
  
def main():  
    calculate()  
  
main()
```

```
Traceback (most recent call last):  
  File "example.py", line 8, in <module>  
    main()  
  File "example.py", line 6, in main  
    calculate()  
  File "example.py", line 4, in calculate  
    return divide(10, 0)  
  File "example.py", line 2, in divide  
    return a / b  
ZeroDivisionError: division by zero
```

Каждая строка traceback — это один кадр из стека вызовов на момент ошибки

Читаем traceback снизу вверх

Самая нижняя строка — сама ошибка. Строка над ней — где именно она произошла (`divide`). Выше — кто её вызвал (`calculate`), и ещё выше — кто вызвал того (`main`). Traceback — это ровно тот стек вызовов, который мы только что визуализировали, показанный в момент, когда что-то пошло не так.

Практика: читаем стек вызовов и traceback

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/13-17/index.html) → (https://www.cartesianschool.org/practice/13-17/index.html)

Проектируем хорошую функцию

Прежде чем писать реализацию, полезно спроектировать функцию: одна ясная обязанность, понятное имя, чёткий контракт.

Хорошие имена — глаголы-действия

Хорошо	Плохо
<code>calculate_total()</code>	<code>do_it()</code>
<code>normalize_text()</code>	<code>func1()</code>
<code>draw_polygon()</code>	<code>thing()</code>
<code>find_max_score()</code>	<code>test2()</code>

Функции обычно представляют ДЕЙСТВИЕ — имя должно быть глаголом или глагольной фразой. Для функций, возвращающих `True / False`, хорошо работают префиксы `is_`, `has_`, `can_`, `contains_` — они прямо сообщают, что вернётся булево значение.

Одна функция — одна понятная обязанность

`plohaya_dekompoziciya.py`

```
def process_everything():  
    # спрашивает ввод  
    # проверяет  
    # считает  
    # печатает  
    # рисует  
    # обновляет пять структур данных  
    ...
```

`horoshaya_dekompoziciya.py`

```
def read_answer(): ...  
def normalize_answer(answer): ...  
def check_answer(answer, correct): ...  
def show_result(correct): ...
```

Это ориентир, а не догма

Небольшие программы вполне могут разумно объединять несколько простых действий в одной функции — не нужно дробить каждую строку в отдельную функцию искусственно (пример того, ЧЕГО не стоит делать, — в §13.19). Цель — ясность, а не количество функций.

Сколько строк должна занимать функция?

Не существует правила вида «максимум 10 строк». Вместо количества строк спрашивайте:

Вопросы вместо количества строк

1

Есть ли у неё одна ясная цель?

2

Объясняет ли имя, что она делает?

3

Легко ли её протестировать отдельно?

4

Есть ли внутри сильно повторяющаяся логика?

5

Понятны ли входы и выход читателю?

Контракт функции

Прежде чем писать реализацию, полезно явно сформулировать контракт — что функция принимает, что возвращает, и есть ли у неё побочные эффекты:

Часть контракта	<code>calculate_discount(price, percent)</code>
Вход	price: число, percent: число
Выход	цена со скидкой
Побочные эффекты	нет
Пример	<code>calculate_discount(100, 10) → 90</code>

Предусловия

Некоторые функции ожидают осмысленный вход — например, `draw_polygon(sides, length)` предполагает `sides >= 3` и `length > 0`. Это **предусловия** — требования к входным данным, которые стоит проверить обычными условиями:

```
predusloviya.py
```

```
def polygon_angle(sides):  
    if sides < 3:  
        return None  
  
    return 360 / sides
```

None — не единственный вариант

Возврат `None` при неверном входе — простой и рабочий вариант на этом уровне курса. Более строгие способы (например, поднять исключение) рассматриваются в главах, посвящённых обработке ошибок.

Чек-лист проектирования функции

Прежде чем написать `def`

- Какую ОДНУ работу она выполняет?
- Какие данные ей нужны? Это и есть параметры.
- Возвращает ли она результат — и какой?
- Меняет ли она что-то намеренно (побочный эффект)?
- Как её назвать, чтобы имя объясняло действие?
- На каких входных данных её стоит проверить?

Практика: проектируем функцию по контракту

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-18/index.html>)

Докстринги и подсказки типов

Профессиональная привычка, которую стоит выработать рано: документировать функцию так, чтобы её можно было понять, не читая реализацию.

Докстринг — документация функции

docstring.py

```
def rectangle_area(width, height):
    """Возвращает площадь прямоугольника."""
    return width * height
```

	Комментарий (# ...)	Докстринг ("""...""")
Объясняет	почему код написан именно так	что делает функция — как API
Виден снаружи?	нет, только в исходнике	да, через help() и __doc__

help() показывает докстринг во время работы программы

help_primer.py

```
help(rectangle_area)
print(rectangle_area.__doc__)
```

```
Help on function rectangle_area in module __main__:

rectangle_area(width, height)
    Возвращает площадь прямоугольника.
```

help() достаёт докстринг прямо из объекта функции — та же идея, что мы уже видели у встроенных функций

Type hints — подсказки типов

```
type_hints.py
```

```
def rectangle_area(width: float, height: float) -> float:
    return width * height
```

Часть	Значение
<code>width: float</code>	ожидаемый тип параметра
<code>-> float</code>	ожидаемый тип возвращаемого значения

Подсказки типов не проверяются автоматически во время выполнения

`def add(a: int, b: int) -> int:` НЕ помешает вызвать `add("2", "3")` — Python не станет автоматически приводить строки к числам и не выбросит ошибку сам по себе. Аннотации типов — это документация и подсказка для инструментов разработки, а не runtime-проверка.

Аннотации коллекций

```
annotacii_kollekcij.py
```

```
def average(scores: list[float]) -> float:
    return sum(scores) / len(scores)

def count_words(text: str) -> dict[str, int]:
    ...
```

Необязательное значение в аннотации

```
optional_annotation.py
```

```
def find_user(name: str) -> str | None:
    ...
```

str | None читается как «строка или None»

Такая аннотация говорит: функция либо вернёт строку, либо None, если ничего не найдено. Это не отдельная тема — просто способ явно описать словами то, что мы уже видели на практике (например, у `dict.get()`).

Аннотации не отменяют ловушку изменяемого умолчания

```
annotacii_ne_spasayut.py
```

```
def add(item: str, items: list[str] = []):
    ...
    # всё ещё та же ловушка из §13.7!
```

Аннотации типов — это документация, а не защита от ошибок времени выполнения

Даже с полными аннотациями

`items: list[str] = []` остаётся тем же изменяемым значением по умолчанию, что и раньше — аннотация ничего не меняет в реальном поведении функции.

Практика: докстрынги и подсказки типов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/13-19/index.html) → (<https://www.cartesianschool.org/practice/13-19/index.html>)

Функции как объекты

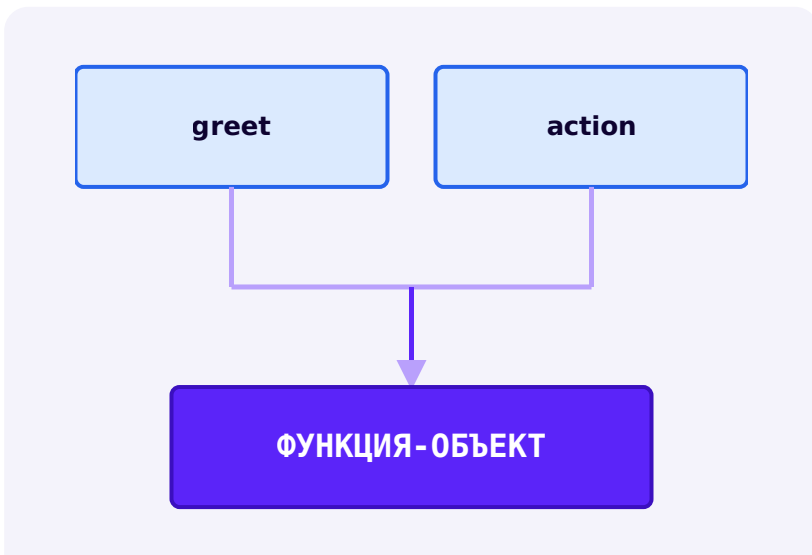
Функция — такой же объект, как список или строка: её можно сохранить под другим именем, положить в словарь, передать другой функции.

Функцию можно сохранить под другим именем

Вернёмся к различию из §13.2 — `greet` и `greet()` — и пойдём на шаг дальше:

```
funkciya_kak_obekt.py
```

```
def greet(name):  
    return f"Привет, {name}"  
  
action = greet  
  
print(action("Anna"))  # Привет, Анна
```



`greet` и `action` — два имени одного и того же объекта функции, точно как с любым другим объектом (глава 3)

Это то же самое *aliasing*, что и всегда

`action = greet` не копирует функцию и не создаёт вторую — оба имени указывают на один и тот же объект функции. Функции в Python — это **объекты первого класса**: их можно сохранять в переменных, класть в коллекции, передавать в другие функции — как любые другие значения.

Функции в коллекциях

`funkcii_v_slozare.py`

```
def double(x):
    return x * 2

def square(x):
    return x ** 2

operations = {
    "double": double,
    "square": square,
}

operation = operations["square"]
result = operation(5)
print(result)    # 25
```

Словарь функций + глава 11

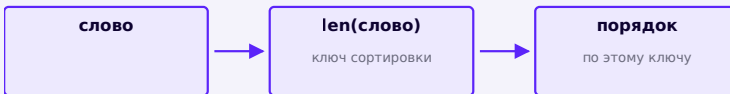
`operations` — обычный словарь (глава 11), просто хранящий не числа и не строки, а объекты функций.

Передача функции как аргумента

Вы уже это делали, даже не задумываясь:

```
sorted_key.py
```

```
words = ["python", "я", "программирование"]  
print(sorted(words, key=len))
```



`sorted()` вызывает переданную функцию `len` для каждого элемента, чтобы получить ключ сортировки

`len` — это ФУНКЦИЯ-ОБЪЕКТ, а не результат её вызова

`key=len` передаёт саму функцию `len` — БЕЗ скобок. `sorted()` сама вызовет `len(...)` для каждого элемента, когда придёт время сравнивать. Если написать `key=len()` — это была бы попытка вызвать `len` прямо сейчас, без аргумента, что приведёт к ошибке.

Термин:

функция высшего порядка

Функция, которая принимает другую функцию как аргумент (как `sorted()`) или возвращает функцию (как `make_greeter()` из §13.13), называется **функцией высшего порядка**. Специальная теория функционального программирования здесь не нужна — достаточно узнавать этот паттерн, когда он встречается.

Практика: функции как объекты первого класса

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/13-20/index.html) → (<https://www.cartesianschool.org/practice/13-20/index.html>)

Лямбда-функции

Маленький безымянный объект функции для одного выражения — полезен в узких случаях, а не как более современная замена `def`.

В §13.20 мы передавали `len` как аргумент в `sorted()`. А что, если нужное правило сортировки не существует как готовая функция — и заводить ради него отдельный `def` кажется излишним? Для этого существует **`lambda`**.

lambda создаёт маленький безымянный объект функции

```
lambda_sortirovka.py
```

```
students = [{"name": "Anna", "score": 82}, {"name":  
"Bob", "score": 95}]  
students.sort(key=lambda student: student["score"])
```

Не «более короткий def», а инструмент для конкретной ситуации

Главная задача lambda — передать маленькое правило/преобразование туда, где полноценная именованная функция была бы избыточной. Это НЕ более современная замена def — это отдельный инструмент для узкого случая.

Анатомия lambda

```
lambda_anatomiya.py
```

```
lambda x: x ** 2
```

Часть	Значение
lambda	ключевое слово — создать функцию-выражение
x	параметр (без скобок)
x ** 2	единственное выражение — его результат возвращается автоматически


```
lambda_vyzov.py
```

```
kvadrat = lambda x: x ** 2  
print(kvadrat(5))    # 25
```

Никакого явного `return` и никаких блочных инструкций внутри — только одно выражение.

Ограничения `lambda`

Когда `lambda` НЕ подходит — берите `def`

ТОЛЬКО ВЫРАЖЕНИЕ

ни одной инструкции
ни присваивания, ни `print`

НЕ ДЛЯ СЛОЖНОЙ ЛОГИКИ

ветвление с эффектами
длинные вычисления

БЕЗ ИМЕНИ И ДОКСТРИНГА

нечего показать в `traceback`
нечего документировать

Если логика заслуживает имени — используйте def

Как только правило становится достаточно важным, чтобы его стоило переиспользовать, протестировать отдельно или объяснить докстрингом — самое время превратить `lambda` в обычную функцию с `def`.

def → lambda: сравнение стиля, а не «улучшение»

Простая функция: def → lambda

Обычная функция (def)

```
def kvadrat(x):
    return x ** 2

print(kvadrat(5))
```

Лямбда-функция

```
kvadrat = lambda x: x
** 2

print(kvadrat(5))
```

Что использовать сегодня: обычную функцию с `def` — она читается яснее, у неё есть нормальное имя для отладки, и в неё легко добавить несколько строк логики или комментариев. `lambda` удобна только для одной короткой строки без имени — например, как аргумент функции `sorted()` или `max()`. Присваивание `lambda` имени (`kvadrat = lambda x: ...`) не более «современно», чем `def` — во многих реальных проектах такой стиль сочли бы менее читаемым, а не прогрессивным.

Необязательно: `map()` и `filter()`

Раз вы уже знаете comprehensions (глава 11), полезно увидеть их альтернативу:

`map_primer.py`

```
numbers = [1, 2, 3, 4]
kvadraty = list(map(lambda n: n ** 2, numbers))
print(kvadraty)
```

`comprehension_ekvivalent.py`

```
numbers = [1, 2, 3, 4]
kvadraty = [n ** 2 for n in numbers]
print(kvadraty)
```

```
filter_primer.py
```

```
chetnye = list(filter(lambda n: n % 2 == 0, numbers))
```

```
comprehension_s_if.py
```

```
chetnye = [n for n in numbers if n % 2 == 0]
```

Python чаще предпочитает comprehensions

Для простых преобразований и фильтров comprehension обычно читается яснее, чем `map()` / `filter()` с `lambda`. Знать оба стиля полезно — вы встретите оба в реальном коде, — но по умолчанию тянитесь к comprehension, если задача простая.

Практика: лямбда-функции

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/13-06/index.html) (<https://www.cartesianschool.org/practice/13-06/index.html>)

Функции как конвейер

Несколько маленьких функций, каждая с ясным входом и выходом, могут вместе выполнить крупную задачу — не хуже одного длинного скрипта, но гораздо понятнее.

Результат одной функции — вход для следующей

konvejer.py

```
clean = normalize_text(input_text)
words = split_words(clean)
counts = count_words(words)
```

СЫРОЙ ТЕКСТ



normalize_text



Каждая функция — одна понятная стадия;
результат одной становится входом следующей

Это архитектурный приём, а не просто удобство

Разбиение на такие стадии — мощный способ организовать программу: каждая функция решает одну понятную задачу, у каждой ясный вход и выход, и стадии можно тестировать по отдельности (§13.24).

Граф вызовов (call graph)

Когда функции вызывают другие функции, получившуюся структуру можно изобразить как дерево — **граф вызовов**:

● MAIN

```

● normalize_text
  ● analyze_text
    ● count_words
    ● count_unique
  ● show_report

```

Граф вызовов — какая функция кого вызывает;
показывает структуру программы целиком

Мы вернёмся к этой идее в §13.22, когда будем рефакторить реальные проекты главы 12 в похожую структуру.

Практика: конвейер из функций

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/13-21/index.html) → (https://www.cartesianschool.org/
practice/13-21/index.html)

Рефакторим проекты главы 12

Возьмём три готовых проекта прошлой главы и превратим их из одного длинного скрипта в набор понятных функций — поведение снаружи останется тем же.

Рефакторинг: меняем структуру, не меняем поведение

Рефакторинг

Рефакторинг — улучшение внутренней структуры программы без намеренного изменения того, что она должна делать снаружи. Мы уже видели этот термин в главе 12 (§12.9, Викторина) — здесь применим его к нескольким проектам подряд.

Викторина

viktorina_do.py

```
# ДО — один длинный скрипт (глава 12, §12.9)
score = 0
for q in questions:
    user_answer = input(q["question"] + "
").strip().lower()
    if user_answer == q["answer"]:
        score += 1
```

viktorina_posle.py

```
def ask_question(question):
    return input(question["question"] + "
").strip().lower()

def check_answer(user_answer, question):
    return user_answer == question["answer"]

def run_quiz(questions):
    score = 0
    for q in questions:
        user_answer = ask_question(q)
        if check_answer(user_answer, q):
            score += 1
    return score
```

Анализатор текста

```
analizator_posle.py
```

```
def normalize_text(text):  
    return text.lower().split()  
  
def count_words(words):  
    return len(words)  
  
def count_unique(words):  
    return len(set(words))
```

Записная книжка

```
zapisnaya_knizhka_posle.py
```

```
def add_contact(contacts, name, phone):  
    contacts[name] = phone  
  
def find_contact(contacts, name):  
    return contacts.get(name)  
  
def remove_contact(contacts, name):  
    del contacts[name]
```


Главная программа становится читаемым описанием алгоритма

do_glavnaya.py

```
text = input("Текст: ")
text = text.lower()
words = text.split()
count = len(words)
unique = set(words)
unique_count = len(unique)
counts = {}
for word in words:
    counts[word] = counts.get(word, 0) + 1
# ... ещё 40 строк ...
```

posle_glavnaya.py

```
text = input("Текст: ")
clean = normalize_text(text)
stats = analyze_text(clean)
show_report(stats)
```

Главный выигрыш рефакторинга — читаемость верхнего уровня

Правая версия читается почти как обычное предложение: «получить текст, нормализовать, проанализировать, показать отчёт». Вся сложность спрятана внутри отдельных функций — и каждую из них можно понять, протестировать и исправить независимо.

★★★ Задача повышенной сложности

Turtle тоже рефакторится

Возьмите код ёлки или мандалы из главы 12 и выделите отдельную функцию для одного яруса/луча — точно так же, как студия многоугольников в §13.26 выделяет `draw_polygon()`.

Практика: рефакторинг проекта из главы 12

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-22/index.html>)

Debug Lab: типичные ошибки функций

Забытый `return`, `return` внутри цикла на первой итерации, случайно мутированный аргумент — десять ошибок, которые встречаются в реальном коде снова и снова.

Десять типичных ошибок, связанных с функциями, — каждая с примером и объяснением. Первые две уже разбирались отдельно (§13.1, §13.13) — здесь для полноты списка, компактно; остальные восемь — подробно.

1 · Случайный вызов до определения

bug.py

```
greet()
```

```
def greet():
    print('Привет')
```

На момент вызова `greet()` ещё не определена — Python выполняет код сверху вниз, и `NameError` возникает раньше, чем строка `def` успевает выполниться.

2 · Незаметный `def` внутри `def`

```
bug.py

def outer():
    ...

    def helper():
        ...
```

Отступ определяет вложенность. Если `helper` случайно оказался с отступом внутри `outer`, он не будет виден снаружи как самостоятельная функция — только через `outer`.

3 · Функция всегда возвращает `None`

```
bug.py

def calculate(x):
    print(x * 2)

result = calculate(5) + 1
```

`calculate` печатает, но не возвращает — `result` станет `None`, и `None + 1` вызовет `TypeError`.

4 • Не все пути функции возвращают значение

bug.py

```
def sign(number):  
    if number > 0:  
        return 'positive'  
    # а если number <= 0?
```

Для отрицательных и нулевых чисел функция доходит до конца без return — и молча возвращает None.

5 • return внутри цикла на первой итерации

bug.py

```
def contains_even(numbers):  
    for n in numbers:  
        if n % 2 == 0:  
            return True  
        else:  
            return False
```

return в ветке else срабатывает уже на первом элементе — проверяются не все числа. Правильно: return True внутри if, а return False — после всего цикла, с тем же отступом, что и for.

6 • return перепутан с break

`bug.py`

```
def find_first_even(numbers):  
    for n in numbers:  
        if n % 2 == 0:  
            break    # а не return n!
```

`break` просто останавливает цикл — `n` нужно ещё явно вернуть отдельной строкой после цикла. `return` сразу выходит из ВСЕЙ функции, `break` — только из ближайшего цикла.

7 · Неожиданная мутация переданного списка

`bug.py`

```
def sorted_names(names):  
    names.sort()  
    return names
```

`names.sort()` мутирует список вызывающего кода на месте. Если это не входит в план, безопаснее `return sorted(names)` — новый список, оригинал не тронут.

8 · Забыли скопировать перед изменением

bug.py

```
def normalize_items(items):
    items[0] = items[0].strip()
    return items
```

Если исходные данные нужно сохранить нетронутыми, начните с `items = items.copy()` — но помните про поверхностность копии (глава 11, §11.10) для вложенных структур.

9 · Глобальное состояние усложняет тестирование

bug.py

```
score = 0

def add_point():
    global score
    score += 1
```

Работает, но такую функцию сложнее тестировать изолированно — её результат зависит от скрытой глобальной переменной. `def add_point(score): return score + 1` тестируется одной строкой, без всякой настройки.

10 · Параметр затеняет встроенное имя

bug.py

```
def total(list):
    return sum(list)
```

`list` — имя встроенного типа. Затенение его параметром работает, но сбивает с толку читателя и ломает доступ к настоящему `list()` внутри этой функции. Лучше: `def total(numbers):`

Метод отладки функций

- Печатает функция или возвращает? Проверьте на бумаге, прежде чем читать код.
- Проверены ли ВСЕ пути выполнения функции, включая неявный «конец без `return`»?
- `return` внутри цикла — точно ли он должен сработать именно на этой итерации?
- Мутирует ли функция переданный аргумент намеренно? Если нет — скопируйте.
- Не перепутано ли имя параметра со встроенным именем Python?

Практика: находим и исправляем ошибки в функциях

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-23/index.html>)

Тестируем функции

Функцию можно проверить отдельно от всей программы — без пользовательского ввода и без запуска приложения целиком.

Главное практическое преимущество функций

```
testiruemaya_funkciya.py
```

```
def rectangle_area(w, h):
    return w * h
```

```
proverki.py
```

```
rectangle_area(2, 3) == 6
rectangle_area(0, 5) == 0
rectangle_area(10, 1) == 10
```

Не нужен ни пользовательский ввод, ни всё приложение целиком

Функцию можно проверить в изоляции — вызвать напрямую с конкретными значениями и сравнить с ожидаемым результатом. Это и есть идея **тестирования отдельных единиц** кода.

Таблица тестов

Вход	Ожидание	Факт	Пройден?
<code>classify_score(95)</code>	«отлично»	«отлично»	

Вход	Ожидание	Факт	Пройден?
<code>classify_score(60)</code>	«хорошо»	«хорошо»	
<code>classify_score(0)</code>	«пересдача»	«пересдача»	
<code>classify_score(100)</code>	граница сверху	?	проверить

Граничные случаи — из главы 9

Тестовая таблица — прямое продолжение граничного тестирования условий из главы 9: проверяем не только «обычные» значения, но и границы диапазонов.

assert — лёгкая проверка предположения

```
assert_primer.py
```

```
assert rectangle_area(2, 3) == 6
assert rectangle_area(0, 5) == 0
print("Все проверки пройдены")
```

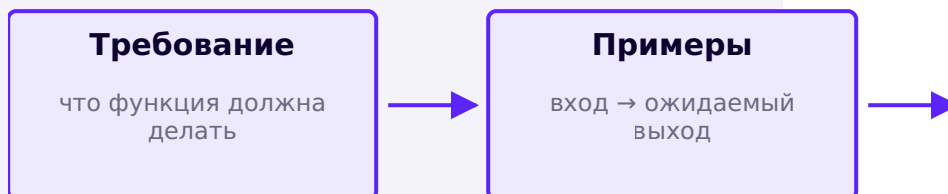
```
assert_padaet.py
```

```
assert rectangle_area(2, 3) == 7
# AssertionError
```

assert — инструмент разработчика, не проверка пользовательского ввода

`assert` проверяет предположение и поднимает `AssertionError`, если оно ложно — это удобно для самопроверки во время разработки и отладки. Это НЕ механизм валидации пользовательского ввода и не защита безопасности — для проверки данных, полученных от пользователя, используются обычные условия (глава 9), как мы уже делали.

Рабочий процесс: контракт → примеры → реализация → проверка



Этот порядок мысли — прямой предвестник полноценного unit-тестирования, которое встретится в будущих главах

Практика: тестируем функцию по таблице

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/13-24/index.html\)](https://www.cartesianschool.org/practice/13-24/index.html)

Мини-проект — выполняем домашнее задание по математике с Python

Функции для генерации и проверки примеров на умножение — практика на все приёмы главы, с чистой логикой проверки, отделённой от ввода-вывода.

ИСПОЛЬЗУЕМ

def return *args (опционально) random
циклы

УРОВЕНЬ

★★★ интеграционный

РЕЗУЛЬТАТ

текстовый тренажёр

Соберём функции, аргументы и `return` в одном полезном инструменте: генераторе примеров для тренировки таблицы умножения. Разобьём задачу на функции по смыслу, а не оставим одним куском:

● MAIN

- `sgenerirovat_primer`
- `proverit_otvet`

Граф вызовов: главная программа опирается на две маленькие, независимо проверяемые функции

`domashnee_zadanie.py`

```
import random

def sgenerirovat_primer():
    a = random.randint(2, 9)
    b = random.randint(2, 9)
    return a, b, a * b

def proverit_otvet(pravilnyj_otvet,
otvet_polzovatelya):
    return pravilnyj_otvet == otvet_polzovatelya

a, b, pravilnyj_otvet = sgenerirovat_primer()
otvet = int(input(f"Сколько будет {a} x {b}? "))

if proverit_otvet(pravilnyj_otvet, otvet):
    print("Верно!")
else:
    print(f"Неверно – правильный ответ: {pravilnyj_otvet}")
```

Функция возвращает сразу три значения

`return a, b, a * b` на самом деле возвращает один кортеж `(a, b, a * b)` — мы распаковываем его в три переменные сразу, как в §13.10 и главе 11.

`sgenerirovat_primer` и `proverit_otvet` тестируются отдельно

`proverit_otvet(56, 56)` и `proverit_otvet(56, 50)` можно вызвать напрямую и проверить результат — без единого `input()`. Это ровно идея из §13.24: чистая логика проверки отделена от ввода/вывода.

★★ Самостоятельная задача

Счётчик правильных ответов

Оберните генерацию примера в цикл на 5 вопросов подряд и посчитайте, сколько из них решено верно.

Практика: генератор примеров по математике

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-07/index.html>)

Мини-проект — анализатор текста, версия 2

Тот же анализатор текста из главы 12 — но теперь как чистый, тестируемый конвейер функций.

ИСПОЛЬЗУЕМ

def return list set dict

УРОВЕНЬ

★★★ интеграционный

РЕЗУЛЬТАТ

ТЕКСТОВЫЙ ОТЧЁТ

Полный рефакторинг анализатора текста из главы 12 (§12.6) в конвейер из чистых функций — прямое продолжение идеи §13.21.

● MAIN

- normalize_text
- split_words
- word_frequency
- build_summary

Каждая стадия — отдельная, независимо тестируемая функция

analizator_v2.py

```
def normalize_text(text):
    return text.lower()
```

```

def split_words(text):
    return text.split()

def word_frequency(words):
    counts = {}
    for word in words:
        counts[word] = counts.get(word, 0) + 1
    return counts

def build_summary(text):
    clean = normalize_text(text)
    words = split_words(clean)
    counts = word_frequency(words)
    return {
        "total_words": len(words),
        "unique_words": len(set(words)),
        "counts": counts,
    }

summary =
build_summary("Python is great and python is fun")
print(summary)

```

build_summary — это конвейер, вызывающий остальные три

Точно паттерн из §13.21: `build_summary` ничего не вычисляет сама — она передаёт данные по цепочке `normalize_text` → `split_words` → `word_frequency` и собирает итог в словарь.

Практика: анализатор текста как конвейер функций

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-25/index.html>)

Мини-проекты — конвертер единиц и утилиты коллекций

Два небольших проекта, полностью состоящих из чистых функций — образцовых кандидатов для независимого тестирования.

ИСПОЛЬЗУЕМ

`def` `return` `type hints` `assert`

УРОВЕНЬ

★★ средний

РЕЗУЛЬТАТ

набор чистых функций

Конвертер единиц измерения

Маленький проект из полностью чистых функций — идеальных кандидатов для тестирования из §13.24:

konverter.py

```
def celsius_to_fahrenheit(celsius: float) -> float:
    return celsius * 9 / 5 + 32

def fahrenheit_to_celsius(fahrenheit: float) -> float:
    return (fahrenheit - 32) * 5 / 9

def km_to_miles(km: float) -> float:
    return km * 0.621371

assert celsius_to_fahrenheit(0) == 32
assert celsius_to_fahrenheit(100) == 212
assert round(km_to_miles(10), 2) == 6.21
```

Ни одного input(), ни одного print() внутри самих функций

Все три функции — чистые: результат целиком зависит от аргумента, побочных эффектов нет. Именно поэтому их можно проверить через `assert`, не запуская вообще ничего похожего на полноценную программу.

Утилиты для работы с коллекциями

Набор маленьких, сфокусированных функций поверх списков и словарей из главы 11:

kolleksiya_utility.py

```
def average(scores: list[float]) -> float:
    return sum(scores) / len(scores)

def count_above(scores: list[float], threshold:
float) -> int:
    return len([s for s in scores if s > threshold])

def unique_words(text: str) -> set[str]:
    return set(text.lower().split())

def find_top_score(students: list[dict]) -> dict:
    return max(students, key=lambda s: s["score"])
```

find_top_score использует функцию как аргумент

key=lambda s: s["score"] — то же самое сочетание max() / sorted() + lambda из §13.20 и §13.26, только теперь на списке словарей (глава 11, §11.14).

★★ Самостоятельная задача**Ещё одна утилита**

Напишите find_students_below(students, threshold) — список имён учеников с баллом ниже threshold. Проверьте её на 2-3 примерах через assert.

Практика: конвертер единиц измерения

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-26/index.html>)

Практика: утилиты для списков и словарей

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/13-27/index.html>)

Мини-проект — Turtle Function Studio

Фигуры из глав 6 и 10 становятся настоящей переиспользуемой функцией с явными параметрами — и подводим итоги всей главы о функциях.

ИСПОЛЬЗУЕМ

def параметры циклы Turtle

УРОВЕНЬ

★★★★ challenge

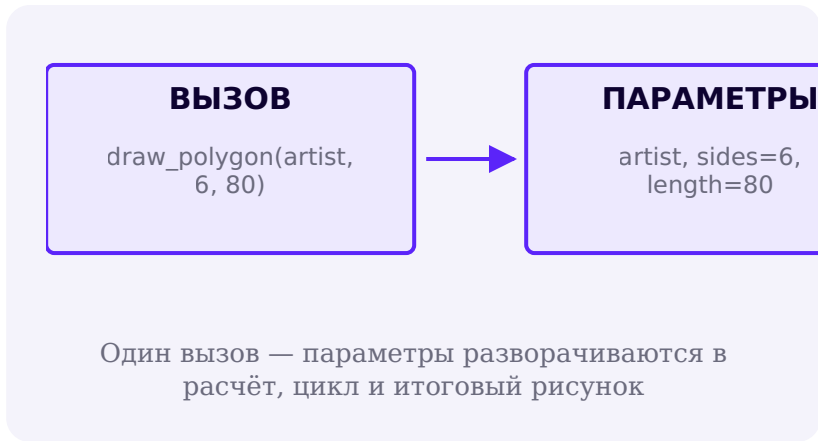
РЕЗУЛЬТАТ

графика Turtle

Что создаём

Финальный проект главы: превратим повторяющийся код рисования фигур из глав 6 и 10 в настоящую переиспользуемую функцию с параметрами.

Поток данных функции



`draw_polygon.py`

```
import turtle

screen = turtle.Screen()
screen.setup(width=300, height=300)
artist = turtle.Turtle()
artist.speed(0)
artist.pencolor("#5B24F9")
artist.pensize(3)

def draw_polygon(artist, sides, length):
    angle = 360 / sides
    for _ in range(sides):
        artist.forward(length)
        artist.right(angle)

draw_polygon(artist, 6, 80)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Правильный шестиугольник, нарисованный функцией `draw_polygon`

`draw_polygon(artist, 6, 80)` — правильный
шестиугольник

Почему функция принимает `artist` явно

`figura_funkciya.py`

```
def draw_polygon(artist, sides, length):
    angle = 360 / sides
    for _ in range(sides):
        artist.forward(length)
        artist.right(angle)

draw_polygon(artist, 4, 100)      # квадрат
draw_polygon(artist, 3, 100)     # треугольник, без
повторения кода!
draw_polygon(artist, 8, 60)      # восьмиугольник
```

Явный параметр `artist` — не лишняя формальность

Функция могла бы полагаться на глобальную переменную `artist`, как в главе 10. Но параметр делает зависимость ЯВНОЙ (§13.15): сразу видно, что функция работает именно с той черепашкой, которую ей передали, — её можно переиспользовать с другой черепашкой (например, во время гонки Turtle из §12.6, где черепашек несколько) без единого изменения кода внутри.

Функция внутри цикла: то же поведение, новый параметр на каждом шаге

`figury_v_cikle.py`

```
cveta = ["#5B24F9", "#DB2777", "#059669"]
for i, sides in enumerate(range(3, 9)):
    artist.pencolor(cveta[i % 3])
    draw_polygon(artist, sides, 70)
    artist.right(15)
```

```
figury_v_cikle.py
```

```
import turtle

screen = turtle.Screen()
screen.setup(width=420, height=420)
artist = turtle.Turtle()
artist.speed(0)
artist.pensize(2)

def draw_polygon(artist, sides, length):
    angle = 360 / sides
    for _ in range(sides):
        artist.forward(length)
        artist.right(angle)

cveta = ["#5B24F9", "#DB2777", "#059669"]
for i, sides in enumerate(range(3, 9)):
    artist.pencolor(cveta[i % 3])
    draw_polygon(artist, sides, 70)
    artist.right(15)

screen.exitonclick()
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Веер из шести повернутых многоугольников от треугольника до восьмиугольника, нарисованный циклом, вызывающим `draw_polygon`

Цикл вызывает `draw_polygon` с новым `sides` на каждом шаге — веер многоугольников

Цикл автоматизирует повторение, функция — поведение

Ровно идея §13.1: цикл решает, СКОЛЬКО раз и с какими данными вызвать `draw_polygon`; сама функция решает, КАК нарисовать многоугольник с этими данными. Ни один из двух инструментов не заменяет другой.

★★★ Задача повышенной сложности**Функция с позицией**

Добавьте функции параметры `x`, `y` (со значениями по умолчанию `0`, `0`) — чтобы можно было указать, откуда начинать рисовать фигуру.

Практика: Turtle Function Studio

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/13-08/index.html>)

Итоги главы

Инструментарий проектирования функции

Итоговый инструментарий главы 13

Нужно повторно использовать один и тот же набор действий? → **выделите функцию**

Функции нужны входные
данные?

→ **параметры**

Нужно вернуть результат для
дальнейшего использования?

→ **return**

Аргументов может
быть неизвестное
количество?

→ ***args / **kwargs**

Некоторые
аргументы стоит
сделать
необязательными?

→ **значения по умолчанию (не изме**

Некоторые
аргументы
должны
передаваться
только по
имени?

→ **keyword-only, после ***

Функция
должна вести
себя
предсказуемо
и легко
тестироваться?

→ **чистая функция, без побочных эффек**

Нужно
передать
саму
функцию
как
значение?

→ **функция – объект первого класса**

Нужно крошечное правило
для `sorted()/max()`?

→ **lambda**

Полный чек-лист проектирования — в §13.18

Что мы узнали в этой главе

- Функция — именованный переиспользуемый кусок поведения: получает данные (параметры), работает с ними, возвращает результат (`return`).
- Вызов функции передаёт управление в её тело и обязательно возвращается к месту вызова — функция не выполняется «где-то в стороне».
- Параметр — имя в определении, аргумент — значение в вызове. Присваивание параметру и мутация переданного объекта — два разных действия с разными последствиями снаружи.
- Изменяемое значение по умолчанию создаётся ОДИН РАЗ при определении функции — используйте `None` как сигнальное значение вместо `[]` или `{}` .
- `*args` и `**kwargs` собирают лишние аргументы в кортеж и словарь соответственно; та же `*/**` на месте вызова распаковывает коллекцию обратно.
- Функция без `return` всё равно возвращает `None` — это не «ничего», а конкретный объект.
- Python ищет имя по правилу LEGB: Local → Enclosing → Global → Builtins.
- Функции, вызывающие функции, образуют стек вызовов — и `traceback` из главы 3 показывает именно этот стек в момент ошибки.
- Функции — объекты первого класса: их можно сохранять под другим именем, класть в коллекции, передавать другим функциям (`sorted(key=...)`).
- `lambda` — маленькая безымянная функция для одного выражения, не более «современная» замена `def` .

- Рефакторинг большого скрипта в набор функций делает главную программу читаемым описанием алгоритма, а каждую часть — независимо тестируемой.

Что дальше

Мы научились группировать ПОВЕДЕНИЕ в функции. Но во многих наших проектах данные и поведение, которое с ними работает, естественно ходят парой — контакт со своим телефоном, ученик со своим баллом, черепашка со своим цветом и позицией. В следующей главе — **«Создаём объекты реального мира»** — мы научимся объединять данные и функции, которые с ними работают, в одну именованную структуру.

Создаём объекты реального мира

Классы, объекты, атрибуты и методы — основы объектно-ориентированного программирования. От уже знакомых объектов (черепашка, строка, список) — к собственным классам: состояние и поведение, `self` и `__init__`, инкапсуляция и `property`, композиция и наследование, `super()`, полиморфизм и `duck typing`, специальные методы, `dataclasses` — и к вопросу, который важнее любого синтаксиса: когда классу вообще стоит появляться в программе.

14.1 Что такое ООП?

Давайте это докажем!

14.2 Классы и объекты со своими значениями

Объекты со своими значениями

14.3 Управляем объектами. Действия объектов

14.4 Гонка Turtle с объектами

14.5 `self` и связывание методов

14.6 `__init__` и создание объекта

14.7 Атрибуты экземпляра и класса

14.8 Ловушка общего изменяемого атрибута

14.9 Мини-проект: Player

14.10 Инкапсуляция

14.11 property: вычисляемые атрибуты

14.12 Мини-проект: Rectangle

14.13 Композиция: объекты внутри объектов

14.14 Мини-проект: Корзина покупок v2

14.15 Наследование

14.16 super() и порядок разрешения методов

14.17 Переопределение методов

14.18 Полиморфизм

14.19 Duck typing

14.20 Мини-проект: полиморфные фигуры

14.21 Специальные методы

14.22 Практика: применяем __str__ и __eq__

14.23 dataclasses

14.24 Практика: dataclass

14.25 Класс или не класс? Проектируем модели

14.26 Мини-проект: гонка Turtle v2

Итоги главы

Что такое объектно-ориентированное программирование?

Вы уже пользовались объектами много глав подряд — просто ещё не называли их так.

Объект — то, что уже знакомо, но с новым именем

Присмотритесь к коду из предыдущих глав:
`artist.forward(100)` , `"Python".upper()` ,
`[1,2,3].append(4)` — во всех трёх у нас есть какой-то **объект** (черепашка, строка, список) и команда, которую мы у него вызываем через точку. Всё это время вы уже работали с объектами — этот раздел просто даёт знакомой идее точное имя.

У любого объекта в Python есть две стороны:

Сторона объекта	Пример у artist (черепашки)
Состояние — данные, которые объект хранит прямо сейчас	текущие координаты, цвет, направление
Поведение — действия, которые объект умеет выполнять	<code>forward()</code> , <code>right()</code> , <code>color()</code>

Объектно-ориентированное программирование (ООП) — это способ организовать код вокруг таких объектов: данные и действия, которые к ним относятся, живут **вместе**, а не разбросаны по отдельным переменным и функциям.

Тип объекта: `type()`

У каждого объекта в Python есть тип — узнать его можно встроенной функцией `type()`, с которой вы уже знакомы:

`tip_obekta.py`

```
artist = __import__("turtle").Turtle()
print(type(artist))      # <class 'turtle.Turtle'>
print(type("Python"))    # <class 'str'>
print(type([1, 2, 3]))   # <class 'list'>
```

Слово `class` в каждом ответе — не случайность. Тип объекта — это и есть его **класс**.

Давайте это докажем!

Вы уже пользовались объектами с самой главы 6, даже не называя их так:

`vy_uzhe_polzovalis_obektami.py`

```
import turtle

artist = turtle.Turtle()  # artist — объект класса Turtle
artist.forward(100)       # forward() — действие (метод) этого объекта
artist.color("red")       # у объекта есть и своё состояние — например, цвет
```

`turtle.Turtle` — это **класс**: чертёж, шаблон, описывающий, какими бывают черепашки и что они умеют. Каждый вызов `turtle.Turtle()` создаёт новый,

независимый **объект** (говорят также «экземпляр класса») по этому чертежу — вспомните главу 12, где несколько независимых черепашек участвовали в гонке одновременно: у каждой было своё состояние (позиция, цвет), хотя все они были черепашками одного и того же класса.

Класс описывает БУДУЩИЕ объекты, а не один конкретный

`turtle.Turtle` сам по себе не имеет позиции на экране — позиция появляется только у КАЖДОГО созданного объекта `turtle.Turtle()` отдельно. Класс — это описание того, какими будут объекты, а не сам объект.

Практика: находим объекты в уже знакомом коде

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-01/index.html>)

Классы

Пишем свой первый класс — чертёж, по которому Python будет создавать объекты.

Классы

Определим свой собственный класс — чертёж для объекта «Собака»:

```
klass_sobaka.py
```

```
class Sobaka:
    def __init__(self, klichka, vozrast):
        self.klichka = klichka
        self.vozrast = vozrast

rex = Sobaka("Рекс", 3)
print(rex.klichka, rex.vozrast)
```

`__init__` — специальный метод, который автоматически выполняется при создании объекта (`Sobaka("Рекс", 3)`) и настраивает его начальное состояние. `self` — ссылка на сам создаваемый объект; Python передаёт её автоматически, и первым параметром `__init__` (и любого другого метода) она должна быть всегда. Подробно, что здесь происходит «под капотом», разберём в разделах 14.5-14.6 — сейчас достаточно рабочей модели.

Класс vs объект — аналогия с формой для печенья

Класс — как форма для печенья: одна и та же форма может «вырезать» сколько угодно печений (объектов) — каждое своё, но по одному и тому же шаблону.

Где аналогия ломается

Форма для печенья — пассивный инструмент: она сама ничего не делает и не хранит состояния. Класс в Python — не пассивный шаблон, а самостоятельный объект: у него можно спросить `type(rex)`, ему самому можно назначать атрибуты, его можно передать в переменную. Пользуйтесь аналогией, чтобы запомнить «один чертёж → много экземпляров», но не воспринимайте её буквально дальше этого.

Класс `Sobaka` — описывает БУДУЩИЕ атрибуты, но ещё не содержит ни одного значения

Объекты со своими значениями

Каждый объект хранит свои собственные значения атрибутов, независимо от других объектов того же класса:

```
raznye_obekty.py
```

```
rex = Sobaka("Рекс", 3)
sharik = Sobaka("Шарик", 5)

print(rex.klichka, rex.vozrast)      # Рекс 3
print(sharik.klichka, sharik.vozrast) # Шарик 5 –
                                     независимо от rex
```

rex : Собака`klichka = 'Рекс'``vozrast = 3`

Объект `rex` — конкретные значения

sharik : Собака`klichka = 'Шарик'``vozrast = 5`

Объект `sharik` — свои значения, тот же класс

Один класс — сколько угодно объектов, и у каждого своё, независимое состояние. Именно это отличает объект от простого словаря с похожими ключами: объект *принадлежит* своему классу и получает через него доступ к общим методам (раздел 14.3).

Практика: создаём собственные классы

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/14-02/index.html) (<https://www.cartesianschool.org/practice/14-02/index.html>)

Управляем объектами

Атрибуты можно менять после создания объекта — а методы дают объекту собственные действия.

Управляем объектами

Значения атрибутов можно менять уже после создания объекта — так же, как менять значение обычной переменной:

```
меняем_атрибуты.py
```

```
rex = Sobaka("Рекс", 3)
rex.vozrast = 4 # у Рекса был день рождения
print(rex.vozrast)
```

Объекты выполняют действия

Кроме данных (атрибутов), у класса могут быть свои действия — **методы**: обычные функции, определённые внутри класса, которые имеют доступ к `self` и, через него, к атрибутам объекта:

metody.py

```

class Sobaka:
    def __init__(self, klichka, vozrast):
        self.klichka = klichka
        self.vozrast = vozrast

    def layat(self):
        print(f"{self.klichka} говорит: Гав-гав!")

    def prazdnovat_den_rozhdeniya(self):
        self.vozrast += 1
        print(f"Теперь {self.klichka} исполнилось {self.vozrast}!")

rex = Sobaka("Пекс", 3)
rex.layat()
rex.prazdnovat_den_rozhdeniya()

```

Знакомая запись

`rex.layat()` устроена точно так же, как `artist.forward(100)` или `"текст".upper()` — методы объектов, которыми вы уже давно пользуетесь, устроены абсолютно так же, как метод `layat()`, который вы только что написали сами.

Класс `Sobaka` — теперь с методами

self — это не ключевое слово

`self` — обычное имя параметра, выбранное по соглашению, а не зарезервированное слово Python (в отличие от `if` или `class`). Технически метод заработает и с именем `this` или `obj` — но используйте `self` всегда: так его называет весь код на Python, и любое другое имя будет путать других читающих ваш код. Подробно, откуда `self` берётся при вызове, разберём в разделе 14.5.

Практика: методы и изменение атрибутов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-03/index.html>)

Гонка Turtle с объектами

Переписываем гонку из главы 12 с собственным классом — и впервые видим композицию в деле.

Вернёмся к гонке черепашек из главы 12 — на этот раз обернём каждого участника в свой собственный класс, а не просто в объект `turtle.Turtle` напрямую.

```
gonka_s_klassom.py
```

```
import turtle
import random

random.seed(5)
screen = turtle.Screen()
screen.setup(500, 400)

class Uchastnik:
    def __init__(self, cvet, startovyj_y):
        self.t = turtle.Turtle()
        self.t.shape("turtle")
        self.t.color(cvet)
        self.t.penup()
        self.t.goto(-200, startovyj_y)
        self.cvet = cvet

    def sdelat_shag(self):
        self.t.forward(random.randint(1, 10))

    def finishiroval(self, finish_line):
        return self.t.xcor() >= finish_line

cveta = ["red", "blue", "green", "orange"]
uchastniki = [Uchastnik(cvet, i * 40 - 60) for i,
cvet in enumerate(cveta)]

pobeditel = None
while pobeditel is None:
    for u in uchastniki:
        u.sdelat_shag()
        if u.finishiroval(200):
            pobeditel = u.cvet
            break

screen.exitonclick()
```

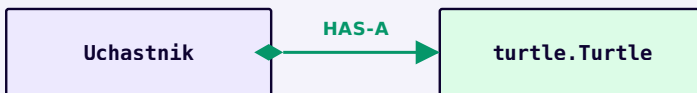

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Четыре черепашки разных цветов на финише гонки, каждая — отдельный объект класса `Uchastnik`

Четыре объекта `Uchastnik` — независимое состояние, общий класс

Зачем оборачивать `Turtle` в свой класс?

В главе 12 логика гонки (движение, проверка финиша) была разбросана по основному коду. Теперь каждый участник — самостоятельный объект `Uchastnik`, который сам знает, как сделать шаг и как проверить, финишировал ли он. Обратите внимание: `Uchastnik` не является черепашкой — он ХРАНИТ черепашку в атрибуте `self.t`. Это называется **композицией** — формально разберём её в разделе 14.13.



`Uchastnik` ХРАНИТ объект `turtle.Turtle` — а не является им

★★★ Задача повышенной сложности

Счёт очков

Добавьте классу `Uchastnik` атрибут `ochki` и метод `nabrat_ochki(n)`, увеличивающий счёт — начислите очки победителю в конце гонки.

Практика: гонка Turtle с объектами

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/14-04/index.html>)

self и связывание методов

`rex.layat()` и `Sobaka.layat(rex)` — один и тот же вызов. Разбираемся, откуда `self` берётся на самом деле.

Две равносильные записи одного вызова

`rex.layat()` — не единственный способ вызвать этот метод. Тот же самый вызов можно записать и через класс, передав объект вручную первым аргументом:

Одно и то же действие — два способа записи

Через объект (обычная запись)

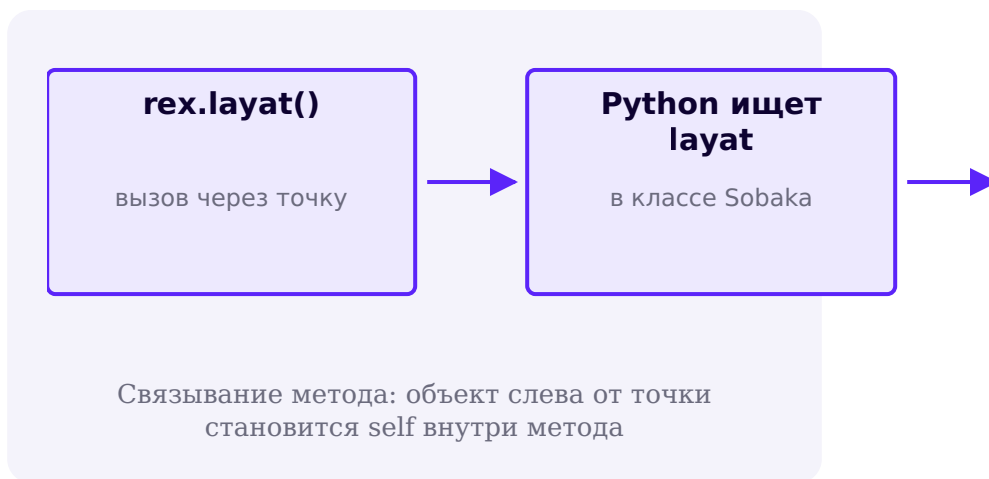
```
rex.layat()
```

Через класс (то, что происходит на самом деле)

```
Sobaka.layat(rex)
```

Что использовать сегодня: На письме всегда используйте запись через объект — `rex.layat()`. Вторая форма показана только для того, чтобы стало видно, откуда берётся `self`: это и есть `rex`, переданный автоматически первым аргументом.

Что происходит при вызове `rex.layat()`



self — снова не ключевое слово

Как уже говорилось в разделе 14.3: `self` — это просто имя первого параметра метода, выбранное по соглашению. Python подставляет в него объект слева от точки НЕЗАВИСИМО от того, как вы его назвали — но называйте его `self` всегда, это единственное имя, которое ожидает увидеть любой человек, читающий код на Python.

DEBUG LAB 1: ЗАБЫЛИ SELF В ОПРЕДЕЛЕНИИ МЕТОДА

```
bez_self.py
```

```
class Собака:
    def __init__(self, кличка):
        self.кличка = кличка

    def layat():
        # забыли self первым параметром
        print("Гав-гав!")

rex = Собака("Пекс")
rex.layat()
```

```
Traceback (most recent call last):
```

```
File "bez_self.py", line 9, in
    rex.layat()
```

```
TypeError: Собака.layat() takes 0 positional argument
```

Что видно на экране

Python всё равно попытался подставить `rex` первым аргументом при вызове `rex.layat()` — связывание метода происходит ВСЕГДА, а не только когда вы этого ожидаете. Раз в определении метода не было параметра, чтобы его принять, — Python сообщает, что аргументов передано на один больше, чем метод способен принять.

ИСПРАВЛЕННЫЙ КОД

s_self.py

```
class Собака:  
    def __init__(self, кличка):  
        self.кличка = кличка  
  
    def лаят(self):  
        print(f"{self.кличка}: Гав-гав!")  
  
rex = Собака("Пекс")  
rex.лаят()
```

Практика: self и связывание методов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

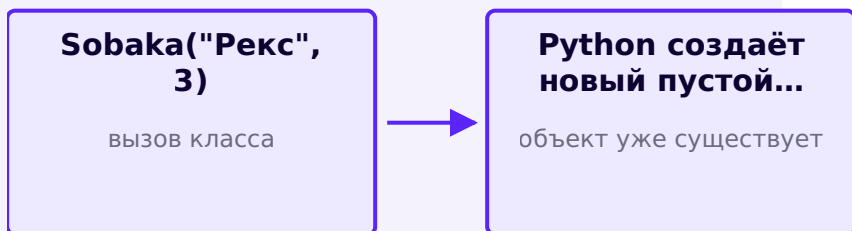
Открыть практику → (<https://www.cartesianschool.org/practice/14-05/index.html>)

__init__ и создание объекта

__init__ не создаёт объект — он настраивает объект, который уже существует.

Что на самом деле происходит при

`Sobaka("Рекс", 3)`



`__init__` НЕ создаёт объект — он настраивает объект, который Python уже создал

Частое заблуждение: «`__init__` создаёт объект»

Это не так. К моменту вызова `__init__` объект уже существует — иначе Python не смог бы передать его первым аргументом (`self`)! Роль `__init__` — не создать объект, а **настроить его начальное состояние**: имя честнее было бы «инициализатор», а не «конструктор» — так его и называют в документации Python.

Значения по умолчанию в `__init__`

`__init__` — обычная функция (глава 13 применима целиком): параметры могут иметь значения по умолчанию:

```
init_s_umolchaniem.py
```

```
class Собака:
    def __init__(self, кличка, возраст=1):
        self.кличка = кличка
        self.возраст = возраст

shchenok = Собака("Бим")           # возраст = 1 по
умолчанию
rex = Собака("Рекс", возраст=3)     # явно указан
```

__init__ может вызывать другие методы

```
init_vyzyvaet_metod.py
```

```
class Собака:
    def __init__(self, кличка, возраст=1):
        self.кличка = кличка
        self.возраст = возраст
        self.privet()           # метод можно вызвать
уже внутри __init__

    def privet(self):
        print(f"Новый пёс на сцене: {self.кличка}!")

rex = Собака("Рекс")           # печатает приветствие сразу
при создании
```

**DEBUG LAB 2: SELF.X = X ЗАПИСАЛИ
ПРОСТО КАК X = X**

poteryannyj_atribut.py

```
class Sobaka:
    def __init__(self, klichka, vozrast):
        klichka = klichka      # это просто
        локальная переменная!
        self.vozrast = vozrast

rex = Sobaka("Пекс", 3)
print(rex.klichka)
```

```
Traceback (most recent call last):
  File "poteryannyj_atribut.py", line 7, in
    print(rex.klichka)
AttributeError: 'Sobaka' object has no attribute 'klichka'
```

Что видно на экране

Строка `klichka = klichka` без `self.` слева создаёт обычную **ЛОКАЛЬНУЮ** переменную внутри `__init__`, которая исчезает, как только `__init__` заканчивается — на объекте она никогда не сохранялась. Чтобы значение осталось частью объекта **НАВСЕГДА**, слева от `=` обязательно должно стоять `self.имя_атрибута`.

ИСПРАВЛЕННЫЙ КОД


```
atribut_ispravlen.py
```

```
class Sobaka:
    def __init__(self, klichka, vozrast):
        self.klichka = klichka
        self.vozrast = vozrast

rex = Sobaka("Пекс", 3)
print(rex.klichka)
```

Практика: `__init__` и создание объекта

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-06/index.html>)

Атрибуты экземпляра и класса

Один атрибут может жить в объекте, а другой — в самом классе, общим на всех. Это не одно и то же.

Два места, где может жить атрибут

До сих пор все атрибуты создавались внутри `__init__` через `self.имя = ...` — это **атрибуты экземпляра**, у каждого объекта свои. Но атрибут можно объявить и прямо в теле класса, без `self` — тогда это **атрибут класса**, один на все объекты сразу:

atribut_klassa.py

```
class Sobaka:
    vid = "Собака"
    # атрибут КЛАССА — один на всех

    def __init__(self, klichka, vozrast):
        self.klichka = klichka
        # атрибут ЭКЗЕМПЛЯРА — свой у каждого
        self.vozrast = vozrast

rex = Sobaka("Рекс", 3)
sharik = Sobaka("Шарик", 5)
print(rex.vid, sharik.vid)  # Собака Собака — общий
                             атрибут
```

Атрибут класса объявлен в теле класса; атрибуты экземпляра — внутри `__init__`

rex : Sobaka

```
vid = 'Собака'  
klichka = 'Рекс'  
vozrast = 3
```

rex — vid читается с класса

sharik : Sobaka

```
vid = 'Собака'  
klichka = 'Шарик'  
vozrast = 5
```

sharik — тот же vid, но не своя копия

Частое заблуждение: «атрибуты класса копируются в каждый объект»

Это не так. `vid` хранится ОДИН раз — в самом классе `Sobaka`. Когда вы пишете `rex.vid`, Python сначала ищет `vid` среди атрибутов ЭКЗЕМПЛЯРА `rex` — не находит — и только затем смотрит на класс `Sobaka`, где и находит общее значение. Ни один байт `vid` не копируется в `rex`.

Если присвоить атрибуту с тем же именем значение ЧЕРЕЗ ОБЪЕКТ — Python создаст новый атрибут ЭКЗЕМПЛЯРА, который отныне «затеняет» атрибут класса именно для этого объекта, не трогая остальные:

zatenenie.py

```
rex.vid = "Дворянга"
# создаёт vid у САМОГО rex, не трогая класс
print(rex.vid, sharik.vid) # Дворянга Собака –
sharik не изменился
```

Практика: атрибуты экземпляра и класса

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-07/index.html>)

Ловушка общего изменяемого атрибута

Один список «на двоих» — классический баг ООП-новичка. Разбираем, откуда он берётся и как его избежать.

Когда «общий атрибут» превращается в баг

Атрибут класса с НЕИЗМЕНЯЕМЫМ значением (число, строка) безопасен — присваивание через объект просто создаёт новый атрибут экземпляра (раздел 14.7). Но атрибут класса со **ИЗМЕНЯЕМЫМ** значением — список или словарь — устроен иначе: если его не ПЕРЕПРИСВОИТЬ, а ИЗМЕНИТЬ на месте (`.append()` , `[key] = ...`), это затронет объект класса, который используют ВСЕ экземпляры сразу.

DEBUG LAB 3: ОБЩИЙ ИЗМЕНЯЕМЫЙ СПИСОК КАК АТТРИБУТ КЛАССА

obshchaya_korzina.py

```
class Korzina:
    tovary = []          # атрибут КЛАССА –
                        # один список на все корзины!

    def dobavit(self, tovar):
        self.tovary.append(tovar)
    # .append() меняет список НА МЕСТЕ

k1 = Korzina()
k2 = Korzina()
k1.dobavit("Хлеб")
print(k2.tovary)
```

```
>>> print(k2.tovary)
['Хлеб']
```

Что видно на экране

`k2` ни разу не вызывал `dobavit()`, но «Хлеб» уже в его корзине! Причина: `self.tovary.append(...)` не создаёт новый список — он ИЗМЕНЯЕТ существующий, а этот список один-единственный, унаследован обоими объектами от класса `Korzina`.

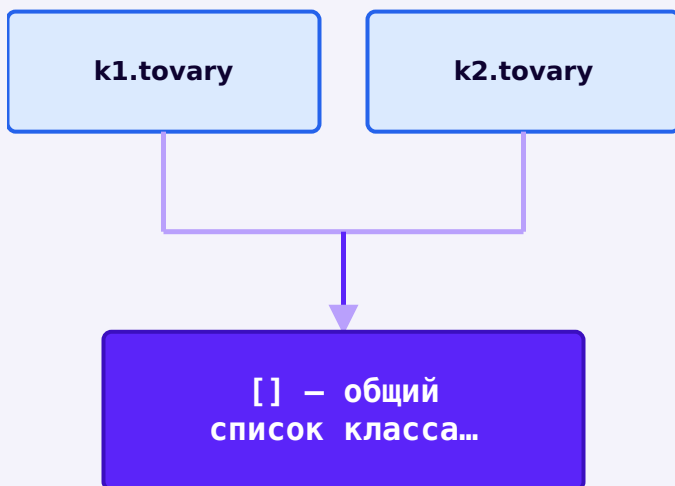
ИСПРАВЛЕННЫЙ КОД

`svoya_korzina.py`

```
class Korzina:
    def __init__(self):
        self.tovary = []    # атрибут
        ЭКЗЕМПЛЯРА — свой список у каждой корзины

    def dobavit(self, tovar):
        self.tovary.append(tovar)

k1 = Korzina()
k2 = Korzina()
k1.dobavit("Хлеб")
print(k2.tovary)    # [] — пусто, как и должно
                    быть
```



Без `self.tovary = []` в `__init__` оба обращения ведут к
ОДНОМУ И ТОМУ ЖЕ списку

Правило: изменяемые значения — только в `__init__`

Если атрибуту нужно значение `list`, `dict` или `set` — создавайте его внутри `__init__` через `self.имя = []`, а не в теле класса. Это тот же самый принцип, что и «ловушка изменяемого значения по умолчанию» у параметров функций (глава 13) — и грабли те же самые.

Практика: находим общий изменяемый атрибут

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-08/index.html>)

Мини-проект: Player

Модель игрока с очками здоровья и счётю — первый самостоятельный класс главы.

Мини-проект: Player

Соберём всё из разделов 14.1-14.8 в один настоящий класс — модель игрока с очками здоровья и счётю:

player.py

```
class Player:
    def __init__(self, name, health=100):
        self.name = name
        self.health = health
        self.score = 0

    def take_damage(self, amount):
        self.health -= amount
        if self.health < 0:
            self.health = 0

    def heal(self, amount):
        self.health += amount
        if self.health > 100:
            self.health = 100

    def add_score(self, points):
        self.score += points
```

Класс Player

dva_igroka.py

```
anna = Player("Anna")
bob = Player("Bob")
anna.take_damage(30)
anna.add_score(10)

print(anna.health, anna.score)    # 70 10
print(bob.health, bob.score)      # 100 0 – bob не
                                  пострадал
```

anna : Player

```
name = 'Anna'
```

```
health = 70
```

```
score = 10
```

anna — своё состояние

bob : Player

```
name = 'Bob'
```

```
health = 100
```

```
score = 0
```

bob — не затронут действиями anna

**DEBUG LAB 4: == СРАВНИВАЕТ НЕ
ЗНАЧЕНИЯ, А САМИ ОБЪЕКТЫ**

```
sравнение_игроков.py
```

```
a1 = Player("Anna")
a2 = Player("Anna")    # те же имя, здоровье и
                        # счёт, что у a1

print(a1 == a2)
```

```
>>> print(a1 == a2)
False
```

Что видно на экране

Хотя у `a1` и `a2` одинаковые значения ВСЕХ атрибутов, `==` по умолчанию сравнивает объекты по **идентичности** — это два РАЗНЫХ объекта в памяти, поэтому результат `False`, даже если содержимое совпадает полностью. Чтобы `==` сравнивал значения, классу нужен специальный метод `__eq__` — дойдём до него в разделе 14.21.

ИСПРАВЛЕННЫЙ КОД

```
sравнение_cherez_pole.py
```

```
# Пока сравниваем нужные поля вручную:
print(a1.name == a2.name and a1.health ==
a2.health)
```

★★ Самостоятельная задача**is_alive**

Добавьте классу Player метод `is_alive(self)`, возвращающий `True`, если `health` больше 0, и `False` иначе.

Практика: класс Player

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/14-09/index.html\)](https://www.cartesianschool.org/practice/14-09/index.html)

Инкапсуляция

Атрибуты доступны напрямую — и это может сломать состояние объекта. Разбираемся, как этого избежать.

Проблема: атрибуты доступны напрямую

Ничто не мешает изменить атрибут объекта в обход всех методов — напрямую, снаружи класса:

DEBUG LAB 5: ПРЯМОЕ ПРИСВАИВАНИЕ ЛОМАЕТ СОСТОЯНИЕ ОБЪЕКТА

```
slo manoe_zdorove.py
```

```
anna = Player("Anna")
anna.health = -50      # напрямую, в обход
                        take_damage()
print(anna.health)
```

```
>>> print(anna.health)
-50
```

Что видно на экране

Метод `take_damage()` аккуратно не даёт `health` уйти ниже нуля — но НИЧТО не заставляет пользоваться именно этим методом. Присваивание `anna.health = -50` обходит всю защитную логику напрямую, и объект оказывается в состоянии, которое сам класс считал невозможным.

ИСПРАВЛЕННЫЙ КОД

cherez_metod.py

```
anna.take_damage(150)    # health корректно
                          # останавливается на 0
print(anna.health)
```

Инкапсуляция — идея хранить состояние объекта «внутри» и разрешать менять его только через методы, которые могут проверить, что новое значение осмысленно.

Соглашение об именах: `_internal` и `__name`

Запись	Что означает по соглашению
<code>name</code>	публичный атрибут — пользуйтесь снаружи свободно
<code>_name</code>	«внутреннее» — не трогайте снаружи, хотя технически доступ есть
<code>__name</code>	включает подмену имени (name mangling) — доступ снаружи усложнён, но не исчезает

```
dvojnoe_podcherkivanie.py
```

```
class Konto:
    def __init__(self, balans):
        self.__balans = balans

schet = Konto(100)
print(schet._Konto__balans)  # 100 — доступ ЕСТЬ,
                             # просто имя изменено
```

Частое заблуждение: «`__private` по-настоящему приватно»

Это не так — в Python нет настоящей приватности на уровне языка. `__balans` автоматически переименовывается в `_Konto__balans` (name mangling) — это защищает в основном от СЛУЧАЙНОГО совпадения имён при наследовании, а не от намеренного доступа снаружи. Инкапсуляция в Python — это соглашение и дисциплина команды, а не запрет со стороны интерпретатора.

Практика: инкапсуляция

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/14-10/index.html) (<https://www.cartesianschool.org/practice/14-10/index.html>)

property: вычисляемые атрибуты

@property позволяет вызывать метод так, будто это обычный атрибут — с проверкой при присваивании и без изменения синтаксиса снаружи.

property: проверка при записи, без изменения синтаксиса чтения

Декоратор @property позволяет обращаться к методу ТАК ЖЕ, как к обычному атрибуту — без круглых скобок:

property_getter.py

```
class Krug:
    def __init__(self, radius):
        self._radius = radius

    @property
    def ploshchad(self):
        return 3.14159 * self._radius ** 2

krug = Krug(10)
print(krug.ploshchad)  # 314.159 — без скобок, хотя
                        это метод!
```

ploshchad нигде не хранится — она вычисляется заново при каждом обращении. Снаружи это неотличимо от обычного атрибута.

@x.setter: проверка при присваивании

property_setter.py

```

class Krug:
    def __init__(self, radius):
        self.radius = radius      # проходит через
        setter ниже

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value <= 0:
            raise ValueError("радиус должен быть
положительным")
        self._radius = value

krug = Krug(10)
krug.radius = 5      # проходит проверку
krug.radius = -1     # ValueError — присваивание
отклонено

```

Главная польза property: код снаружи не меняется

Если сегодня `radius` — обычный атрибут, а завтра понадобилась проверка — можно превратить его в `property`, и весь код, который писал `krug.radius = 5`, продолжит работать без единой правки. Именно поэтому в Python принято начинать с обычных атрибутов и добавлять `property` только когда проверка действительно понадобилась — а не оборачивать в `property` вообще всё с самого начала.

**DEBUG LAB 6: ЗАБЫЛИ ДЕКОРАТОР
@RADIUS.SETTER**

bez_setter.py

```

class Krug:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    def radius(self, value):      # забыли
        @radius.setter сверху!
        self._radius = value

krug = Krug(10)
krug.radius = 5

```

```

>>> krug.radius = 5
>>> print(krug.radius)
5

```

Что видно на экране

Ошибки не будет — и это как раз самое коварное. Без декоратора `@radius.setter` вторая функция `radius` — это отдельный обычный метод, который просто ЗАМЕНИЛ собой `property` сверху (то же имя в теле класса!). После этого `Krug.radius` — уже не `property`, а обычная функция, и `krug.radius = 5` просто создаёт атрибут ЭКЗЕМПЛЯРА `radius`, молча затеняя и метод, и всю проверку из §14.11 — печатается число `5`, а не результат `getter`'а.

ИСПРАВЛЕННЫЙ КОД

```
s_setter.py
```

```
@radius.setter
def radius(self, value):
    self._radius = value
```

Практика: property

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/14-11/index.html\)](https://www.cartesianschool.org/practice/14-11/index.html)

Мини-проект: Rectangle

Проверенные размеры и вычисляемые площадь с периметром — property в реальном классе.

Мини-проект: Rectangle

Класс прямоугольника с проверенными размерами и вычисляемыми площадью/периметром:

```
rectangle.py
```

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    @property
    def width(self):
        return self._width

    @width.setter
    def width(self, value):
        if value <= 0:
            raise ValueError("ширина должна быть
```

```

положительной")
    self._width = value

    @property
    def height(self):
        return self._height

    @height.setter
    def height(self, value):
        if value <= 0:
            raise ValueError("высота должна быть
положительной")
        self._height = value

    @property
    def area(self):
        return self.width * self.height

    @property
    def perimeter(self):
        return 2 * (self.width + self.height)

```

Rectangle — все четыре доступны как атрибуты, но два из них проверяют значение, два вычисляются

ispolzovanie.py

```

r = Rectangle(10, 4)
print(r.area, r.perimeter)    # 40 28
r.width = 6
print(r.area)                  # 24 — пересчиталось
само
r.width = -1                    # ValueError

```

★★ Самостоятельная задача

is_square

Добавьте Rectangle property `is_square`, возвращающую `True`, если `width` равна `height`.

Практика: класс Rectangle

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

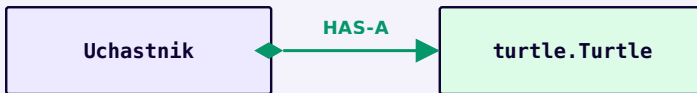
Открыть практику → (<https://www.cartesianschool.org/practice/14-12/index.html>)

Композиция: объекты внутри объектов

Один объект может ХРАНИТЬ другие объекты, отвечая за них — так был устроен `Uchastnik` ещё в разделе 14.4.

Объект как атрибут другого объекта

В разделе 14.4 класс `Uchastnik` хранил объект `turtle.Turtle` в своём атрибуте `self.t` — не наследовал от него, а именно ХРАНИЛ. Это называется **композицией**: один объект строится ИЗ других объектов, отвечая за них.



Композиция: `Uchastnik` ХРАНИТ `Turtle` в своём атрибуте `self.t`

Правило именования отношений: HAS-A

Композицию легко проверить вопросом «X — это Y?» или «X ИМЕЕТ Y?». Uchastnik не является черепашкой (HAS-A верно, IS-A неверно) — поэтому это композиция, а не наследование (о нём — в разделе 14.15).

Объектный граф: композиция может идти в глубину

Объект может хранить не только один объект, а целый список объектов — и те, в свою очередь, могут хранить свои:

● Zakaz

- Tovar('Книга', 590)
- Tovar('Ручка', 90)
- Tovar('Тетрадь', 120)

Zakaz хранит СПИСОК объектов Tovar — композиция, где владелец хранит несколько объектов сразу

DEBUG LAB 7: ПЕРЕПУТАЛИ УРОВЕНЬ ВЛОЖЕННОСТИ АТТРИБУТА

```
propushchennyj_uroven.py
```

```
class Mashina:
    def __init__(self):
        self.dvigatel = Dvigatel()

mashina = Mashina()
mashina.start()      # а где именно определён
start()?
```

```
Traceback (most recent call last):
  File "propushchennyj_uroven.py", line 5, in
    mashina.start()
AttributeError: 'Mashina' object has no attribute 'start'
```

Что видно на экране

`start()` определён у `Dvigatel`, а не у `Mashina` — при композиции методы вложенного объекта НЕ становятся автоматически методами владельца. Нужно обратиться явно, через тот атрибут, в котором вложенный объект хранится.

ИСПРАВЛЕННЫЙ КОД

```
cherez_atribut.py
```

```
mashina.dvigatel.start()  # явно: через
self.dvigatel
```

Практика: композиция

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/14-13/index.html) (<https://www.cartesianschool.org/practice/14-13/index.html>)

Мини-проект: Корзина покупок v2

Список словарей из главы 12 становится списком объектов `Tovar` внутри `Korzina`.

Мини-проект: Корзина покупок v2

В главе 12 корзина покупок хранилась как список словарей. Теперь у нас есть инструменты, чтобы сделать её надёжнее: класс `Tovar` для одного товара и класс `Korzina`, который хранит СПИСОК объектов `Tovar` — композиция из раздела 14.13 в действии.

korzina_v2.py

```

class Tovar:
    def __init__(self, nazvanie, tsena):
        self.nazvanie = nazvanie
        self.tsena = tsena

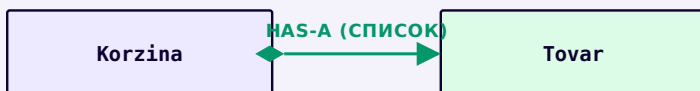
class Korzina:
    def __init__(self):
        self.tovary = []      # атрибут ЭКЗЕМПЛЯРА –
                             # раздел 14.8!

    def dobavit_tovar(self, tovar):
        self.tovary.append(tovar)

    def obshchaya_summa(self):
        return sum(t.tsena for t in self.tovary)

    def kolichество_tovarov(self):
        return len(self.tovary)

```



Korzina хранит список объектов Tovar


```
ispolzovanie.py
```

```
korzina = Korzina()
korzina.dobavit_tovar(Tovar("Книга", 590))
korzina.dobavit_tovar(Tovar("Ручка", 90))

print(korzina.kolichestvo_tovarov())    # 2
print(korzina.obshchaya_summa())        # 680
```

Почему это лучше словаря

У объекта `Tovar` всегда есть ровно `nazvanie` и `tsena` — опечатка в ключе словаря (`tovar["cena"]` вместо `tovar["tsena"]`) обнаружится только во время выполнения, а обращение к несуществующему атрибуту объекта — тоже во время выполнения, но с классом легче добавить проверки и методы (подсчёт суммы, скидки) в одном понятном месте.

★★ Самостоятельная задача

udalit_tovar

Добавьте `Korzina` метод `udalit_tovar(self, nazvanie)`, удаляющий из `self.tovary` первый товар с этим названием.

Практика: Корзина покупок v2

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/14-14/index.html\)](https://www.cartesianschool.org/practice/14-14/index.html)

Наследование

Когда один класс — это более частный случай другого, наследование позволяет не переписывать общее поведение заново.

Строим класс на основе существующего

Если два класса имеют много общего, но один — более общее понятие, а другой — его частный случай, отношение между ними — **IS-A** («Кошка — это животное»). Для такого отношения в Python есть **наследование**:

nasledovanie.py

```
class Zhivotnoe:
    def __init__(self, klichka):
        self.klichka = klichka

    def predstavitsya(self):
        print(f"Я {self.klichka}")

class Sobaka(Zhivotnoe):      # Собака — это Zhivotnoe
    def zvuk(self):
        return "Гав!"

class Koshka(Zhivotnoe):     # Коска — это тоже
                              Zhivotnoe
    def zvuk(self):
        return "Мяу!"

rex = Sobaka("Рекс")
rex.predstavitsya()         # унаследовано от Zhivotnoe —
                             # работает без переопределения
print(rex.zvuk())           # своё, определено в Sobaka
```

● Zhivotnoe

● Sobaka

● Koshka

Sobaka и Koshka наследуют от Zhivotnoe — общее поведение (predstavitsya) не нужно писать дважды



IS-A: открытый треугольник указывает на родителя

Наследование — это переиспользование, а не обязанность

Sobaka получает `predstavitsya()` БЕСПЛАТНО, ничего не переписывая — в этом весь смысл. Но наследование оправдано только когда отношение реально IS-A. Если тянет унаследовать класс просто чтобы «одолжить» пару методов без настоящего родства понятий — это сигнал, что нужна композиция (раздел 14.13), а не наследование.

**DEBUG LAB 8: ЗАБЫЛИ ВЫЗВАТЬ
SUPER().__INIT__()**

```
propushchennyj_super.py
```

```
class Zhivotnoe:
    def __init__(self, klichka):
        self.klichka = klichka

class Sobaka(Zhivotnoe):
    def __init__(self, klichka, poroda):
        self.poroda = poroda      # klichka из
        Zhivotnoe не настроен!

rex = Sobaka("Рекс", "Дворняга")
print(rex.klichka)
```

```
Traceback (most recent call last):
  File "propushchennyj_super.py", line 9, in
    print(rex.klichka)
AttributeError: 'Sobaka' object has no attribute 'klichka'
```

Что видно на экране

Когда дочерний класс определяет СОБСТВЕННЫЙ `__init__`, он полностью ЗАМЕНЯЕТ родительский `__init__` — родительский код настройки атрибутов сам по себе больше не выполняется. Чтобы получить и то, и другое, дочерний `__init__` должен явно вызвать родительский — разберём как именно, в разделе 14.16.

ИСПРАВЛЕННЫЙ КОД

s_super.py

```
class Sobaka(Zhivotnoe):
    def __init__(self, klichka, poroda):
        super().__init__(klichka)  # сначала
        настраиваем родительскую часть
        self.poroda = poroda
```

Практика: наследование

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-15/index.html>)

super() и порядок разрешения методов

super() позволяет дочернему классу воспользоваться методом родителя изнутри переопределения — а не заново писать его с нуля.

super(): позвать родителя, а не заменить его

super() даёт доступ к методам родительского класса ИЗНУТРИ переопределённого метода — обычно чтобы ДОПОЛНИТЬ поведение родителя, а не заменить его целиком:

super_init.py

```

class Zhivotnoe:
    def __init__(self, klichka):
        self.klichka = klichka

class Sobaka(Zhivotnoe):
    def __init__(self, klichka, poroda):
        super().__init__(klichka) # выполняет
        Zhivotnoe.__init__(self, klichka)
        self.poroda = poroda
# затем добавляет своё

rex = Sobaka("Рекс", "Дворняга")
print(rex.klichka, rex.poroda)
# Рекс Дворняга – оба атрибута на месте

```

Частое заблуждение: «super() возвращает объект родительского класса»

Это не так. `super()` возвращает специальный промежуточный объект (прокси), который умеет находить методы РОДИТЕЛЯ для ТЕКУЩЕГО `self` — `self` остаётся объектом класса `Sobaka`, он не превращается в `Zhivotnoe`. Просто поиск метода на этот раз начинается не с `Sobaka`, а на уровень выше.

Порядок разрешения методов (MRO)

При обычном одиночном наследовании порядок поиска метода простой и предсказуемый: сначала сам класс объекта, затем его родитель, затем родитель родителя — и так до `object`, базового класса для всех классов в Python:



MRO для одиночного наследования —
предсказуемый порядок поиска

DEBUG LAB 9: ИСПОЛЬЗУЮТ АТРИБУТ ДО ВЫЗОВА SUPER().__INIT__()

nepravilnyj_poryadok.py

```

class Sobaka(Zhivotnoe):
    def __init__(self, klichka, poroda):
        self.poroda = poroda
        print(f"Порода: {self.poroda},
кличка: {self.klichka}") # klichka ещё нет!
        super().__init__(klichka)
  
```

```

Traceback (most recent call last):
  print(f"Порода: {self.poroda}, кличка: {self.klichka}")
AttributeError: 'Sobaka' object has no attribute 'klichka'
  
```

Что видно на экране

`super().__init__(klichka)` стоит ПОСЛЕ строки,
которая уже пытается прочитать `self.klichka` —
на этот момент родительский `__init__` ещё не

выполнялся, атрибут ещё не создан. Порядок вызовов внутри `__init__` выполняется строго сверху вниз, как и в любой другой функции.

ИСПРАВЛЕННЫЙ КОД

`pravilnyj_poryadok.py`

```
class Sobaka(Zhivotnoe):
    def __init__(self, klichka, poroda):
        super().__init__(klichka)  # сначала
        родительская часть
        self.poroda = poroda
        print(f"Порода: {self.poroda},
        кличка: {self.klichka}")
```

Практика: `super()`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-16/index.html>)

Переопределение методов

Дочерний класс может заменить метод родителя целиком — или дополнить его через `super()`.

Переопределение: свой вариант метода родителя

Дочерний класс может определить метод с ТЕМ ЖЕ именем, что и у родителя — это называется **переопределением** (override). Python при поиске метода находит версию дочернего класса первой (раздел 14.16, MRO) и использует именно её:

pereopredelenie.py

```

class Zhivotnoe:
    def zvuk(self):
        print("Какое-то животное издаёт звук")

class Sobaka(Zhivotnoe):
    def zvuk(self):      # полностью заменяет
        родительскую версию
        print("Гав!")

Sobaka().zvuk()      # Гав! — версия Zhivotnoe даже не
                     вызывается

```

Иногда нужно не полностью заменить поведение родителя, а ДОПОЛНИТЬ его — тогда переопределённый метод вызывает `super().метод()` и добавляет что-то своё:

rasshirenie.py

```

class Zhivotnoe:
    def zvuk(self):
        print("Животное подаёт голос:")

class Sobaka(Zhivotnoe):
    def zvuk(self):
        super().zvuk()      # сначала родительская
        часть
        print("Гав!")      # затем своя

Sobaka().zvuk()
# Животное подаёт голос:
# Гав!

```

**DEBUG LAB 10: ПЕРЕОПРЕДЕЛИЛИ МЕТОД,
НО ЗАБЫЛИ РАСШИРИТЬ ЧЕРЕЗ SUPER()**

poteryannoe_povedenie.py

```
class Zhivotnoe:
    def zvuk(self):
        print("Животное подаёт голос:")

class Sobaka(Zhivotnoe):
    def zvuk(self):
        print("Гав!")      # хотели ДОПОЛНИТЬ,
                            # но случайно заменили целиком

Sobaka().zvuk()
```

```
>>> Sobaka().zvuk()
Гав!
```

Что видно на экране

Строка «Животное подаёт голос:» пропала — переопределение по умолчанию ЗАМЕНЯЕТ родительский метод целиком, а не дополняет его автоматически. Если задумывалось именно дополнение, вызов `super().zvuk()` нужно прописать явно — Python никогда не вызывает родительскую версию сам, если вы её не попросили.

ИСПРАВЛЕННЫЙ КОД

s_rasshireniem.py

```
class Sobaka(Zhivotnoe):
    def zvuk(self):
        super().zvuk()
        print("Гав!")
```

Практика: переопределение методов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/14-17/index.html\)](https://www.cartesianschool.org/practice/14-17/index.html)

Полиморфизм

Одна и та же строка кода — `.zvuk()` — ведёт себя по-разному в зависимости от того, какой именно объект её вызвал.

Один вызов, разные результаты

Полиморфизм («много форм») — когда один и тот же вызов метода даёт разное поведение в зависимости от того, у объекта какого класса он вызван:

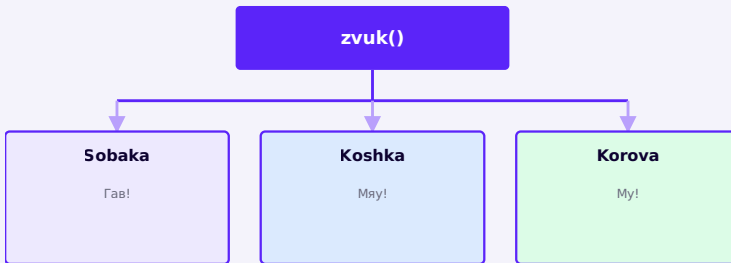
polimorfizm.py

```
class Собака:
    def звук(self):
        return "Гав!"

class Котика:
    def звук(self):
        return "Мяу!"

class Корова:
    def звук(self):
        return "Му!"

zhivotnye = [Собака(), Котика(), Корова()]
for zh in животные:
    print(zh.звук())    # каждый раз вызывается СВОЯ
                        версия звук()
```



Один и тот же вызов `.zvuk()` — разный результат для каждого класса

Цикл не знает и не должен знать, с каким классом работает

Строка `zh.zvuk()` внутри цикла ОДНА — ей не нужно проверять `if isinstance(zh, Sobaka): ...` и так для каждого класса. Именно в этом ценность полиморфизма: код, который ИСПОЛЬЗУЕТ объекты, остаётся простым и не растёт с каждым новым классом животного.

Практика: полиморфизм

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-18/index.html>)

Duck typing

Python не проверяет, от какого класса унаследован объект — только то, есть ли у него нужный метод.

«Если это выглядит как утка и крикает как утка...»

В разделе 14.18 все классы разделяли общего родителя не всегда. На самом деле Python вообще не проверяет, от какого класса объект унаследован, чтобы вызвать метод — важно только, ЕСТЬ ли у объекта нужный метод:

duck_typing.py

```
class Sobaka:                                # никак не связана с
Robot
    def predstavitsya(self):
        print("Гав! Я собака.")

class Robot:                                # никак не связана с
Sobaka
    def predstavitsya(self):
        print("БМП. Я робот.")

def poznamomit(obj):
    obj.predstavitsya()
# неважно, какого obj класса — важно, что метод есть

poznamomit(Sobaka())
poznamomit(Robot())
```

Частое заблуждение: «полиморфизм требует наследования»

Это не так, и duck typing — прямое тому доказательство. `Sobaka` и `Robot` НЕ имеют общего родителя (кроме неявного `object`), но `poznamomit()` одинаково успешно работает с обоими. В Python интерфейс — это фактическое наличие нужного метода, а не формальная запись в дереве наследования.

Название пришло из выражения «если оно выглядит как утка, плавает как утка и крикает как утка — вероятно, это и есть утка»: неважно, как объект был создан, важно, что он УМЕЕТ.

DEBUG LAB 11: НЕ У ВСЕХ ОБЪЕКТОВ В СПИСКЕ ЕСТЬ ОЖИДАЕМЫЙ МЕТОД

```
nesovmestimyj_obekt.py
```

```
zhivotnye = [Sobaka(), Robot(), "просто  
строка"]  
for zh in животные:  
    zh.predstavitsya()
```

```
Гав! Я собака.
```

```
БИП. Я робот.
```

```
Traceback (most recent call last):
```

```
AttributeError: 'str' object has no attribute 'predstavitsya'
```

Что видно на экране

Duck typing не проверяет наличие метода заранее — Python просто пытается вызвать `predstavitsya()` и падает в момент вызова, если метода нет. Гибкость duck typing не отменяет ответственность: класть в один список объекты, которые ДОЛЖНЫ поддерживать общий метод, нужно осознанно.

ИСПРАВЛЕННЫЙ КОД

```
s_proverkoj.py
```

```
for zh in zhivotnye:
    if hasattr(zh, "predstavitsya"):
        zh.predstavitsya()
```

Практика: duck typing

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/14-19/index.html\)](https://www.cartesianschool.org/practice/14-19/index.html)

Мини-проект: полиморфные фигуры

Общий предок задаёт интерфейс, каждая фигура реализует его по-своему — наследование и полиморфизм вместе.

Мини-проект: полиморфные фигуры

Общий предок `Figura` хранит имя фигуры; каждая конкретная фигура переопределяет `ploshchad()` по-своему — наследование (14.15) и полиморфизм (14.18) вместе:

```
figury.py
```

```
class Figura:
    def __init__(self, nazvanie):
        self.nazvanie = nazvanie

    def ploshchad(self):
        raise NotImplementedError
```

```

class Krug(Figura):
    def __init__(self, radius):
        super().__init__("круг")
        self.radius = radius

    def ploshchad(self):
        return 3.14159 * self.radius ** 2

class Pryamougolnik(Figura):
    def __init__(self, width, height):
        super().__init__("прямоугольник")
        self.width = width
        self.height = height

    def ploshchad(self):
        return self.width * self.height

class Treugolnik(Figura):
    def __init__(self, base, height):
        super().__init__("треугольник")
        self.base = base
        self.height = height

    def ploshchad(self):
        return 0.5 * self.base * self.height

```

● Figura

- Krug
- Pryamougolnik
- Treugolnik

Все три фигуры — IS-A Figura, каждая переопределяет `ploshchad()` по-своему


```
ispolzovanie.py
```

```
figury = [Krug(5), Pryanougolnik(4, 6), Treugolnik(8,
3)]
for f in figury:
    print(f"{f.nazvanie}: {f.ploshchad():.2f}")

# круг: 78.54
# прямоугольник: 24.00
# треугольник: 12.00
```

raise NotImplementedError — сигнал «доделай в наследнике»

Figura.ploshchad() намеренно не вычисляет ничего полезного — она существует только чтобы задать ОБЩИЙ ИНТЕРФЕЙС, который обязаны реализовать наследники. Если забыть переопределить ploshchad() в новой фигуре, ошибка обнаружится сразу же при первом вызове, а не тихо выдаст неверную площадь.

★★ Самостоятельная задача

perimetr

Добавьте Figura метод-заглушку perimetr() (raise NotImplementedError) и реализуйте его во всех трёх фигурах. Подсказка для Treugolnik: base и height одних не хватает для точного периметра — считайте его равнобедренным и найдите боковую сторону по теореме Пифагора.

Практика: полиморфные фигуры

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-20/index.html>)

Специальные методы

`__init__` — не единственный дандер-метод. `__str__` и `__eq__` подключают объект к `print()` и `==`.

Как объекты подключаются к встроенным операциям

Когда вы пишете `print(rex)` или `len(text)`, Python на самом деле вызывает специальный метод объекта с именем вида `__имя__` («дандер», double underscore). Вы уже пользовались одним таким методом с самого начала главы — `__init__`.

Что вы пишете	Что Python вызывает на самом деле
<code>Sobaka("Пекс", 3)</code>	<code>__init__(self, ...)</code>
<code>print(rex)</code>	<code>__str__(self)</code>
<code>len(korzina)</code>	<code>__len__(self)</code>
<code>a + b</code>	<code>__add__(self, other)</code>
<code>a == b</code>	<code>__eq__(self, other)</code>

`__str__` и `__repr__`

Без специального метода `print(объект)` выводит малополезную строку вроде `<__main__.Sobaka object at 0x7f...>`. `__str__` позволяет задать понятный человеку вид:

str_metod.py

```

class Собака:
    def __init__(self, кличка, возраст):
        self.кличка = кличка
        self.возраст = возраст

    def __str__(self):
        return f"{self.кличка} ({self.возраст} года)"

rex = Собака("Пекс", 3)
print(rex)  # Пекс (3 года) – а не адрес в памяти

```

__eq__: сравнение по значению, а не по идентичности

В разделе 14.9 `a1 == a2` у двух `Player` с одинаковыми значениями оказалось `False` — потому что `==` по умолчанию сравнивает объекты как разные ячейки памяти. `__eq__` это меняет:

eq_metod.py

```

class Собака:
    def __init__(self, кличка, возраст):
        self.кличка = кличка
        self.возраст = возраст

    def __eq__(self, other):
        return self.кличка == other.кличка and
               self.возраст == other.возраст

Собака("Пекс", 3) == Собака("Пекс", 3)  # теперь True

```

DEBUG LAB 12: __EQ__ ОПРЕДЕЛИЛИ, А ОБЪЕКТ СТАЛ UNHASHABLE

```
unhashable_obekt.py
```

```
class Tochka:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return self.x == other.x
    and self.y == other.y

tochki = {Tochka(1, 2), Tochka(3, 4)} #
положили объекты в множество
```

```
Traceback (most recent call last):
TypeError: unhashable type: 'Tochka'
```

Что видно на экране

У объекта по умолчанию есть автоматический `__hash__`, основанный на его адресе в памяти. Как только вы определяете свой `__eq__`, Python СНИМАЕТ автоматический `__hash__` — ведь если объекты «равны» по значению, у равных объектов логично ожидать и одинаковый хеш, а старый адресный хеш этому больше не соответствует. Объект без хеша нельзя положить в `set` или использовать как ключ `dict`.

ИСПРАВЛЕННЫЙ КОД

`s_hash.py`

```
def __eq__(self, other):  
    return self.x == other.x  
and self.y == other.y  
  
def __hash__(self):  
    return hash((self.x, self.y))
```

Практика: специальные методы

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-21/index.html>)

Практика: применяем `__str__` и `__eq__`

Небольшой класс `Точка` — площадка, чтобы закрепить
оба метода из раздела 14.21.

Практика: точка на плоскости

Применим оба метода из раздела 14.21 к небольшому
классу `Точка` — координата `x`, `y`:

tochka.py

```

class Tochka:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

a = Tochka(1, 2)
b = Tochka(1, 2)
c = Tochka(5, 5)

print(a)           # (1, 2)
print(a == b)      # True — совпадают координаты
print(a == c)      # False

```

f-string и __str__ работают вместе

f"Точка: {a}" вызывает тот же самый `__str__`, что и `print(a)` — любое место, где Python нужно превратить объект в строку для человека, проходит через этот метод.

★★ Самостоятельная задача**rasstoyanie**

Добавьте `Tochka` метод `rasstoyanie_do(self, other)`, возвращающий евклидово расстояние до другой точки: $((self.x - other.x) ** 2 + (self.y - other.y) ** 2) ** 0.5$.

Практика: __str__ и __eq__ на практике

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-22/index.html>)

dataclasses

Классы, которые в основном хранят данные, не обязаны писать `__init__` и `__repr__` вручную.

Меньше шаблонного кода для классов-данных

Классы вроде `Tochka` из раздела 14.22 — в основном шаблонный код: `__init__`, который просто копирует параметры в `self`, плюс `__str__` и `__eq__`, которые почти всегда выглядят одинаково. Модуль `dataclasses` умеет сгенерировать всё это автоматически:

Класс-данные: вручную и через `@dataclass`

Обычный класс

```
class Tochka:
    def __init__(self,
x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return
f"Tochka(x={self.x},
y={self.y})"

    def __eq__(self,
other):

return self.x ==
other.x and self.y ==
other.y
```

С `@dataclass`

```
from dataclasses import
dataclass

@dataclass
class Tochka:
    x: int
    y: int
# __init__, __repr__ и
__eq__ сгенерированы
автоматически
```

Что использовать сегодня: `@dataclass` — там, где класс в основном ХРАНИТ данные и не выигрывает от ручного контроля над `__init__` / `__repr__` / `__eq__`. Он всё ещё обычный класс — методы добавляются в тело точно так же, как и всегда.

```
dataclass_s_metodom.py
```

```
from dataclasses import dataclass

@dataclass
class Tochka:
    x: int
    y: int

    def rasstoyanie_do(self, other):
        return ((self.x - other.x) ** 2 + (self.y -
other.y) ** 2) ** 0.5

a = Tochka(0, 0)
b = Tochka(3, 4)
print(a)                                # Tochka(x=0, y=0) –
__repr__ уже готов
print(a.rasstoyanie_do(b))              # 5.0
```

Частое заблуждение: «dataclass — это не настоящий класс» или «в dataclass нельзя писать методы»

Оба неверны. @dataclass — это обычный декоратор класса: он лишь ДОБАВЛЯЕТ автосгенерированные `__init__`, `__repr__`, `__eq__` поверх обычного тела класса. Методы, property и вообще всё, что можно писать в обычном классе, точно так же пишется и в dataclass.

DEBUG LAB 13: ИЗМЕНЯЕМОЕ ЗНАЧЕНИЕ ПО УМОЛЧАНИЮ В DATACLASS


```
izmenyaemoe_polei_umolchanie.py

from dataclasses import dataclass

@dataclass
class Korzina:
    tovary: list = []
# похоже на ловушку из раздела 14.8!
```

```
Traceback (most recent call last):
  ValueError: mutable default for field tovary is not
```

Что видно на экране

Python `dataclasses` **УМЫШЛЕННО** запрещает изменяемое значение по умолчанию прямо в аннотации — это тот же самый риск общего списка на все объекты, что и в разделе 14.8, только для `dataclass` он обнаруживается сразу, ошибкой при определении класса, а не тихим багом во время выполнения.

ИСПРАВЛЕННЫЙ КОД

```
s_default_factory.py

from dataclasses import dataclass, field

@dataclass
class Korzina:
    tovary: list =
    field(default_factory=list) # новый список
для КАЖДОГО объекта
```

Практика: dataclasses

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/14-23/index.html) (<https://www.cartesianschool.org/practice/14-23/index.html>)

Практика: dataclass

Товар главы 14.14, но короче — @dataclass берёт на себя шаблонный код.

Практика: Товар как dataclass

Перепишем `Товар` из раздела 14.14 через `@dataclass`:

`товар_dataclass.py`

```
from dataclasses import dataclass

@dataclass
class Товар:
    название: str
    tsena: float

    def so_skidkoj(self, protsent):
        return self.tsena * (1 - protsent / 100)

knigi = Товар("Книга", 590)
print(knigi)                                #
Товар(название='Книга', tsena=590)
print(knigi.so_skidkoj(10))                 # 531.0
```

`__eq__` достаётся бесплатно

`Товар("Книга", 590) == Товар("Книга", 590)` — True, хотя мы не писали `__eq__` вручную ни разу: `dataclass` сравнивает по значениям всех полей автоматически.

★★ Самостоятельная задача

Zakaz как dataclass

Определите @dataclass Zakaz с полями `tovar: Tovar` и `kolichество: int`, и методом `summa(self)`, возвращающим `self.tovar.tsena * self.kolichество`.

Практика: dataclass Tovar

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/14-24/index.html>)

Класс или не класс? Проектируем модели

Не каждая задача требует класса — а неправильный выбор виден не сразу, а только когда код становится сложнее читать.

Не каждая задача требует класса

После 24 разделов про классы легко решить, что классом нужно оборачивать вообще всё. Это не так — класс полезен, когда есть И состояние, И поведение, которые имеет смысл держать вместе, и когда объектов этого вида будет несколько, с независимой историей.

Класс или не класс?

Нужно и хранить
данные, и
выполнять над
ними действия?

→ → вероятно, класс

Будет несколько независимых экземпляров с разным состоянием? → → **класс**

Это разовое вычисление без хранимого состояния? → → **обычная функция**

Данные без всякого поведения, читаются один раз? → → **dict / tuple / dataclass без методов**

Нужна проверка при изменении значения? → → **property (раздел 14.11)**

DEBUG LAB 14: КЛАСС РАДИ ОДНОГО ВЫЧИСЛЕНИЯ БЕЗ СОСТОЯНИЯ

```
izlishnij_klass.py
```

```
class SredneeArifmeticheskoe:
    def __init__(self, chisla):
        self.chisla = chisla

    def vychislit(self):
        return sum(self.chisla) /
len(self.chisla)

rezultat = SredneeArifmeticheskoe([4, 8,
15]).vychislit()
```

```
# Код работает правильно — ошибка не во время выполнения
# а в самом ДИЗАЙНЕ: слишком много церемоний для одного
```

Что видно на экране

Формально это рабочий код, но здесь нет ни настоящего состояния (объект создаётся и сразу используется один раз), ни нескольких экземпляров — просто вычисление, притворяющееся классом. Обычная функция читается и тестируется проще: `srednee(chisla)`. Класс оправдан, когда объект живёт какое-то время и меняет состояние — как `Player` из раздела 14.9, а не когда он существует одну строчку.

ИСПРАВЛЕННЫЙ КОД

```
prostaya_funkciya.py
```

```
def srednee_arifmeticheskoe(chisla):
    return sum(chisla) / len(chisla)

rezultat = srednee_arifmeticheskoe([4, 8, 15])
```

Признаки, что классу самое место

Объект несколько раз меняет состояние ПОСЛЕ создания (не только при `__init__`); в программе будет больше одного такого объекта одновременно; данные и действия над ними логически связаны настолько, что разделять их неудобно. Если ни один признак не выполняется — начните с функции или словаря, класс всегда можно добавить позже, когда он реально понадобится.

★★ Самостоятельная задача

Функция или класс?

Для каждого сценария решите, что уместнее — функция или класс, и обоснуйте: (1) перевод температуры из Цельсия в Фаренгейт; (2) банковский счёт с балансом и историей операций; (3) проверка, является ли число простым.

Практика: класс или не класс

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/14-25/index.html) → (<https://www.cartesianschool.org/practice/14-25/index.html>)

Мини-проект: гонка Turtle v2

Гонка сама становится объектом — управляет участниками, а не дублирует их логику.

Мини-проект: гонка Turtle v2

Расширим проект из раздела 14.4. На этот раз не только участники (`Uchastnik`) — сама гонка тоже становится объектом: класс `Gonka` хранит список участников, рисует финишную линию и определяет порядок финиша — ещё один пример композиции (раздел 14.13), но уже объект, управляющий другими объектами.

```
gonka_v2.py
```

```
import turtle
import random

random.seed(7)
screen = turtle.Screen()
screen.setup(520, 420)

class Uchastnik:
    def __init__(self, cvet, startovyj_y):
        self.t = turtle.Turtle()
        self.t.shape("turtle")
        self.t.color(cvet)
        self.t.penup()
        self.t.goto(-220, startovyj_y)
        self.cvet = cvet
        self.finishiroval = False

    def sdelat_shag(self):
        if not self.finishiroval:
            self.t.forward(random.randint(1, 10))

    def proverit_finish(self, finish_x):
        if self.t.xcor() >= finish_x:
            self.finishiroval = True
        return self.finishiroval

class Gonka:
    def __init__(self, cveta, finish_x):
        self.finish_x = finish_x
        self.uchastniki = [
            Uchastnik(cvet, i * 50 - 75) for i, cvet
in enumerate(cveta)
        ]
        self.rezultaty = []

    def narisovat_finish(self):
        liniya = turtle.Turtle()
```



```

        liniya.hideturtle()
        liniya.penup()
        liniya.goto(self.finish_x, -120)
        liniya.pendown()
        liniya.pencolor("#0D0230")
        liniya.pensize(3)
        liniya.setheading(90)
        liniya.forward(240)

    def sygrat_do_kontsa(self):
        self.narisovat_finish()
        while len(self.rezultaty) <
len(self.uchastniki):
            for u in self.uchastniki:
                u.sdelat_shag()
                if u.proverit_finish(self.finish_x)
and u.cvet not in self.rezultaty:
                    self.rezultaty.append(u.cvet)

gonka = Gonka(["red", "blue", "green"], 210)
gonka.sygrat_do_kontsa()

screen.exitonclick()

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Гри черепашки на дорожках рядом с вертикальной финишной линией

Gonka хранит трёх Uchastnik и сама рисует финишную линию



Gonka хранит список объектов Uchastnik и управляет гонкой в целом

Объекты, сотрудничающие друг с другом

Gonka не дублирует логику шага или проверки финиша — она делегирует её каждому Uchastnik (`u.sdelat_shag()`, `u.proverit_finish(...)`) и только координирует их. Это типичная для ООП картина: не один гигантский класс, который умеет всё, а несколько небольших классов, каждый отвечает за своё, и сотрудничающих через понятные методы друг друга.

★★★ Задача повышенной сложности

Таблица результатов

Добавьте Gonka метод `tablitsa_rezultatov(self)`, возвращающий `self.rezultaty` в виде пронумерованного текста («1 место: red», «2 место: blue», ...).

Практика: гонка Turtle v2

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/14-26/index.html>)

Итоги главы

От первого объекта — к моделям из нескольких сотрудничающих классов. Собираем всё воедино.

Инструментарий ООП, который у вас теперь есть

Итоговая карта главы 14

ОСНОВЫ

`class`, `__init__`, `self`
атрибуты и методы
экземпляр vs класс (14.7)

ИНКАПСУЛЯЦИЯ

`_internal`, `__name`
`property` и `@x.setter`

ОТНОШЕНИЯ ОБЪЕКТОВ

композиция – HAS-A (14.13)
наследование – IS-A (14.15)
`super()` (14.16)

ГИБКОСТЬ ПОВЕДЕНИЯ

полиморфизм (14.18)
duck typing (14.19)

СОВРЕМЕННЫЙ PYTHON

специальные методы (14.21)

@dataclass (14.23)

ПРОЕКТИРОВАНИЕ

класс или не класс? (14.25)

Путь, который мы прошли

- Объект = состояние + поведение
 - Класс описывает будущие объекты
 - `__init__` настраивает состояние
 - `self` связывает метод с объектом
 - Отношения между объектами
 - Композиция: объект ХРАНИТ объект
 - Наследование: объект ЯВЛЯЕТСЯ объектом
 - Гибкость поведения
 - Полиморфизм
 - Duck typing

От одного объекта — к моделям из нескольких
сотрудничающих классов

Что мы узнали в этой главе

- ООП организует код вокруг **объектов** — состояния и поведения, собранных вместе.
- `class` определяет чертёж; `__init__` настраивает состояние уже существующего объекта, а не создаёт его.
- `self` — не ключевое слово, а объект слева от точки, подставленный автоматически при связывании метода.
- Атрибут экземпляра — свой у каждого объекта; атрибут класса — один на всех, и изменяемые значения там опасны.
- Инкапсуляция и `property` защищают состояние, не меняя синтаксис снаружи.
- Композиция (HAS-A) и наследование (IS-A) — два разных способа связать классы; выбор между ними определяется реальным отношением понятий, а не удобством.
- Полиморфизм и `duck typing` позволяют коду работать с разными объектами одинаково, не проверяя их класс явно.
- Специальные методы подключают объект к `print()`, `==`, `len()` и другим встроенным операциям; `@dataclass` убирает шаблонный код там, где он не нужен.
- Не каждая задача требует класса — это тоже часть проектирования, а не исключение из правил.

Дальше — глава 15: Python и файлы

Все объекты этой главы жили только пока работала программа: закрыли Python — и `Player`, и `Korzina` исчезли вместе с процессом. В следующей главе научимся сохранять данные на диск — в текстовые файлы и файлы CSV — чтобы состояние переживало перезапуск программы, а не терялось каждый раз.

Python и файлы

Путь данных от объекта Python до файла на диске — и обратно. Файловая система, переносимые пути через `pathlib`, безопасное чтение и запись текста и байтов в UTF-8, JSON и CSV — и как сохранить состояние программы между запусками, а не только на время одного сеанса.

15.1 Зачем нужны файлы? Открытие и чтение

15.2 Строка за строкой

15.3 Создание новых файлов

15.4 Мини-проект: дневник заметок

15.5 Файл, папка и файловая система

15.6 Пути: абсолютные и относительные

15.7 Текущая рабочая директория (CWD)

15.8 `pathlib.Path`: пути как объекты

15.9 Разбираем путь: `name`, `stem`, `suffix`, `parent`

15.10 Практика: пути и CWD

15.11 `open()` возвращает объект файла

- 15.12 Жизненный цикл файла и with
- 15.13 Курсор файла: tell() и seek()
- 15.14 Читаем файлы: read(), readline(), readlines()
- 15.15 Пишем в файлы: write() и режимы r/w/a/x
- 15.16 Текст, bytes и кодировка UTF-8
- 15.17 Бинарные файлы и переносы строк
- 15.18 pathlib: read_text/write_text/read_bytes/write_bytes
- 15.19 Папки: exists(), is_file(), mkdir()
- 15.20 Поиск файлов: iterdir() и glob()
- 15.21 Переименование, копирование, удаление
- 15.22 Ошибки файловой системы
- 15.23 Большие файлы и потоковая обработка
- 15.24 Как выбрать формат хранения данных
- 15.25 JSON: сохраняем структуры данных
- 15.26 Мини-проект: сохраняем Player
- 15.27 CSV: таблицы в текстовом виде
- 15.28 Безопасная работа с файлами
- 15.29 Мини-проект: таблица рекордов

15.30 Браузер и локальный диск

15.31 Итоги главы: инструментарий работы с файлами

Зачем нужны файлы?

Переменные исчезают вместе с программой — файлы позволяют сохранить данные надолго.

Куда пропадают переменные?

Небольшая программа:

```
schet_igry.py

score = 1200
player = "Anna"

print(score)
```

Программа заканчивается. Где теперь `score` и `player`? Запустите программу ещё раз — она снова начнёт со значения `score = 1200`, как будто предыдущего запуска не было.

Точная формулировка

Не «всё в памяти мгновенно стирается» — а точнее: **следующий запуск программы не наследует автоматически объекты Python из предыдущего процесса.** Пока программа выполняется, её объекты живут в памяти запущенного процесса; когда процесс завершается, эта память освобождается операционной системой.



Чтобы данные **пережили завершение программы** — список результатов игры, текст, настройки — их нужно записать в **файл** на диске (или другом постоянном хранилище) и прочитать обратно при следующем запуске.

Что такое файл?

Файл — это **именованный набор данных**, который хранится в файловой системе.

Чтобы программа могла открыть нужный файл, ей нужно указать, где этот файл находится. Для этого используется **путь к файлу** — запись, которая показывает его расположение среди папок и файлов. Например:

```
primer_puti.py
```

```
# data/results.txt
```

Здесь `data` — это папка, а `results.txt` — файл внутри неё.

Путь — это адрес файла

Для первого знакомства путь можно представить как адрес файла в файловой системе — так же, как адрес дома помогает найти его среди улиц и номеров. Это лишь сравнение для интуиции: путь — это текстовая запись, а не физический адрес в памяти компьютера. Абсолютные и относительные пути подробно разберём в разделе 15.6.

В конечном счёте содержимое файла представлено байтами. Если файл содержит текст, Python должен знать, как преобразовать эти байты в символы и обратно. Правило такого преобразования называется **кодировкой** — подробно поговорим об этом в разделе 15.16, вместе с различием `str` и `bytes`.

Открытие и чтение существующих файлов

`chtenie_fajla.py`

```
file = open("privet.txt", "r", encoding="utf-8") #  
"r" — режим чтения (read)  
content = file.read()  
print(content)  
file.close() # обязательно закрыть файл после работы
```

Не забывайте `file.close()`

Незакрытый файл может не сохранить изменения на диск или заблокировать доступ к нему для других программ. Забыть `close()` — очень частая ошибка новичков.

Современный способ: `with`

Ручное закрытие → менеджер контекста `with`

Классический подход

```
file =
open("privet.txt", "r",
encoding="utf-8")
content = file.read()
print(content)
file.close()  # легко
забыть!
```

Современный Python
(`with`)

```
with open("privet.txt",
"r", encoding="utf-8")
as file:
    content =
file.read()
    print(content)
# файл закроется САМ,
даже если внутри блока
произойдёт ошибка
```

Что использовать сегодня: `with` — он существует в Python с версии 2.5, так что дело не в новизне, а в надёжности: файл закроется автоматически при выходе из блока `with`, даже если внутри что-то пойдёт не так. Начиная с этой главы в книге мы используем именно `with` и всегда указываем `encoding="utf-8"` явно (подробнее — в 15.16).

Практика: чтение файлов через `with`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-01/index.html>)

Строка за строкой

Часто удобнее обработать файл по одной строке за раз, а не целиком.

Читать весь файл сразу одним куском не всегда удобно — особенно если файл большой. Часто нужнее обработать его **построчно**:

stroka_za_strokoj.py

```
with open("spisok.txt", "r", encoding="utf-8") as
file:
    for line in file:
        print(line.strip())
```

Что именно убирает strip()

Каждая строка файла, кроме, возможно, последней, заканчивается невидимым символом перевода строки `\n`. Но `.strip()` (глава 8) убирает **все** пробельные символы по обеим сторонам строки — пробелы, табуляции и переносы строк, — а не только этот один перевод строки. Если данные в файле могут осмысленно начинаться или заканчиваться пробелом, слепой `.strip()` может незаметно испортить их. Если цель — убрать именно завершающий перевод строки и ничего больше, используйте `line.rstrip("\n")`.

DEBUG LAB 1: STRIP() УБИРАЕТ БОЛЬШЕ, ЧЕМ КАЖЕТСЯ

city.py

```
with open("city.txt", "w",
encoding="utf-8") as file:

    file.write("    важная цитата с пробелами по
краям \n")

with open("city.txt", "r",
encoding="utf-8") as file:
    for line in file:
        print(f"[{line.strip()}]")
```

[важная цитата с пробелами по краям]

Что видно на экране

Программа вывела цитату без ведущих и завершающих пробелов — хотя автор специально их поставил. `.strip()` убирает пробелы по краям вместе с переводом строки, и не различает «случайный мусор» и «намеренный пробел»: она просто убирает всё пробельное по обоим концам строки.

ИСПРАВЛЕННЫЙ КОД

`citaty_fixed.py`

```
with open("citaty.txt", "r",
encoding="utf-8") as file:
    for line in file:
        print(f"[{line.rstrip(chr(10))}]")
# убираем только перевод строки
```

Все строки сразу — списком

`readlines.py`

```
with open("spisok.txt", "r", encoding="utf-8") as
file:
    lines = file.readlines()

print(len(lines), "строк")
print(lines[0]) # первая строка — вместе со своим \n
```

Символы переноса строки — не только \n

В Python текстовые файлы читаются в режиме универсальных переносов строк: даже если на диске строки разделены последовательностью `\r\n` (так исторически принято в Windows), при чтении в текстовом режиме Python отдаёт вам `\n`. Подробно про текстовый и бинарный режимы — в 15.17.

Практика: построчное чтение файлов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/15-02/index.html) (<https://www.cartesianschool.org/practice/15-02/index.html>)

Создание новых файлов

Записываем данные на диск — и знакомимся с современным способом работать с путями.

Создание новых файлов

Чтобы записать что-то в файл, откройте его в режиме записи `"w"` (write) — если файла не существует, Python создаст его сам; если существует — его прежнее содержимое стирается **уже в момент открытия**, ещё до первого `write()`:

```
zapis_fajla.py
```

```
with open("rezultaty.txt", "w", encoding="utf-8") as file:
    file.write("Уровень 1: 100 очков\n")
    file.write("Уровень 2: 250 очков\n")
```

ДО

Уровень 1: 50 очков
(старая игра)

`open(..., "w")`

**ПОСЛЕ ОТКРЫТИЯ "W"**

(пустой файл)

Режим "w" стирает старое содержимое сразу при открытии файла — до первой записи.

Работа с файлами

Три основных режима открытия файла:

- "r" — чтение (read); ошибка, если файла не существует.
- "w" — запись (write); создаёт новый файл или полностью стирает существующий сразу при открытии.
- "a" — дозапись (append); добавляет текст в конец файла, не стирая то, что уже есть.

dozapis.py

```
with open("rezultaty.txt", "a", encoding="utf-8") as
file:
    file.write("Уровень 3: 400 очков\n")
```

"a" — это не «вставить куда захочу»

Дозапись всегда добавляет текст в конец файла. Она не умеет вставлять текст в середину или начало существующего содержимого.

Пути к файлам: строка или pathlib

Путь к файлу: обычная строка → pathlib

Классический подход

```
file_path = "data/
rezultaty.txt"
with open(file_path,
"r",
encoding="utf-8") as
f:
    print(f.read())
```

Современный Python
(pathlib)

```
from pathlib import Path

file_path = Path("data") /
"rezultaty.txt"
print(file_path.exists())
# удобные методы прямо у
пути
with file_path.open("r",
encoding="utf-8") as f:
    print(f.read())
```

Что использовать сегодня: `pathlib` для путей к файлам — он появился в Python 3.4 и с тех пор считается предпочтительным способом. Строковые пути всё ещё повсеместно работают и встречаются в старом коде, но `pathlib` избавляет от ручной сборки пути через `+` и добавляет полезные методы вроде `.exists()` прямо у самого пути. Подробно разберём его в разделе 15.8.

Практика: запись, дозапись и pathlib

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-03/index.html>)

Мини-проект: дневник заметок

Дневник заметок, сохраняющийся между запусками программы — первая настоящая персистентность.

Соберём чтение и запись в одном небольшом проекте — дневнике заметок, который сохраняется между запусками программы.

```
dnevnik_zametok.py
```

```
from pathlib import Path

fajl_zametok = Path("zametki.txt")

novaya_zametka = input("Новая заметка: ")

with fajl_zametok.open("a", encoding="utf-8") as f:
    f.write(novaya_zametka + "\n")

print("Все заметки:")
with fajl_zametok.open("r", encoding="utf-8") as f:
    for line in f:
        print("-", line.strip())
```

Почему дозапись, а не перезапись

Режим "a" добавляет заметку, не стирая предыдущие — именно так и должен вести себя настоящий дневник: каждый запуск программы добавляет новую запись, а не начинает всё заново.

★★ Самостоятельная задача

Нумерация заметок

Добавьте номер к каждой заметке при чтении — например, «1. Первая заметка», «2. Вторая заметка».

Практика: дневник заметок

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-04/index.html>)

Файл, папка и файловая система

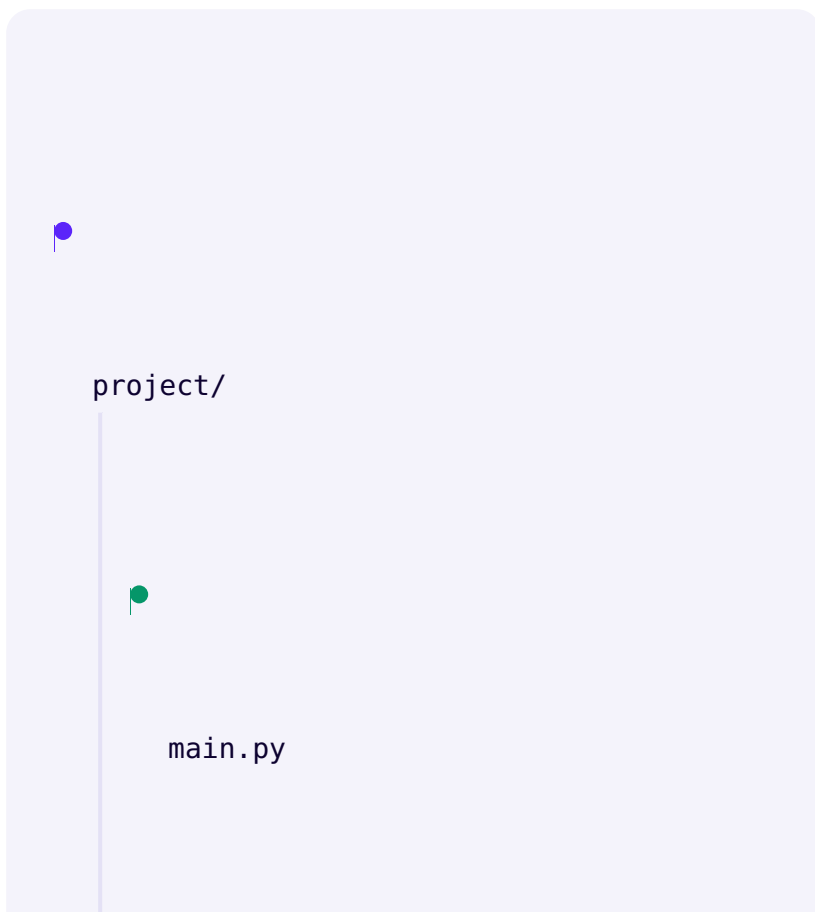
Файлы хранят данные, папки организуют файловую систему в дерево.

Файл и папка

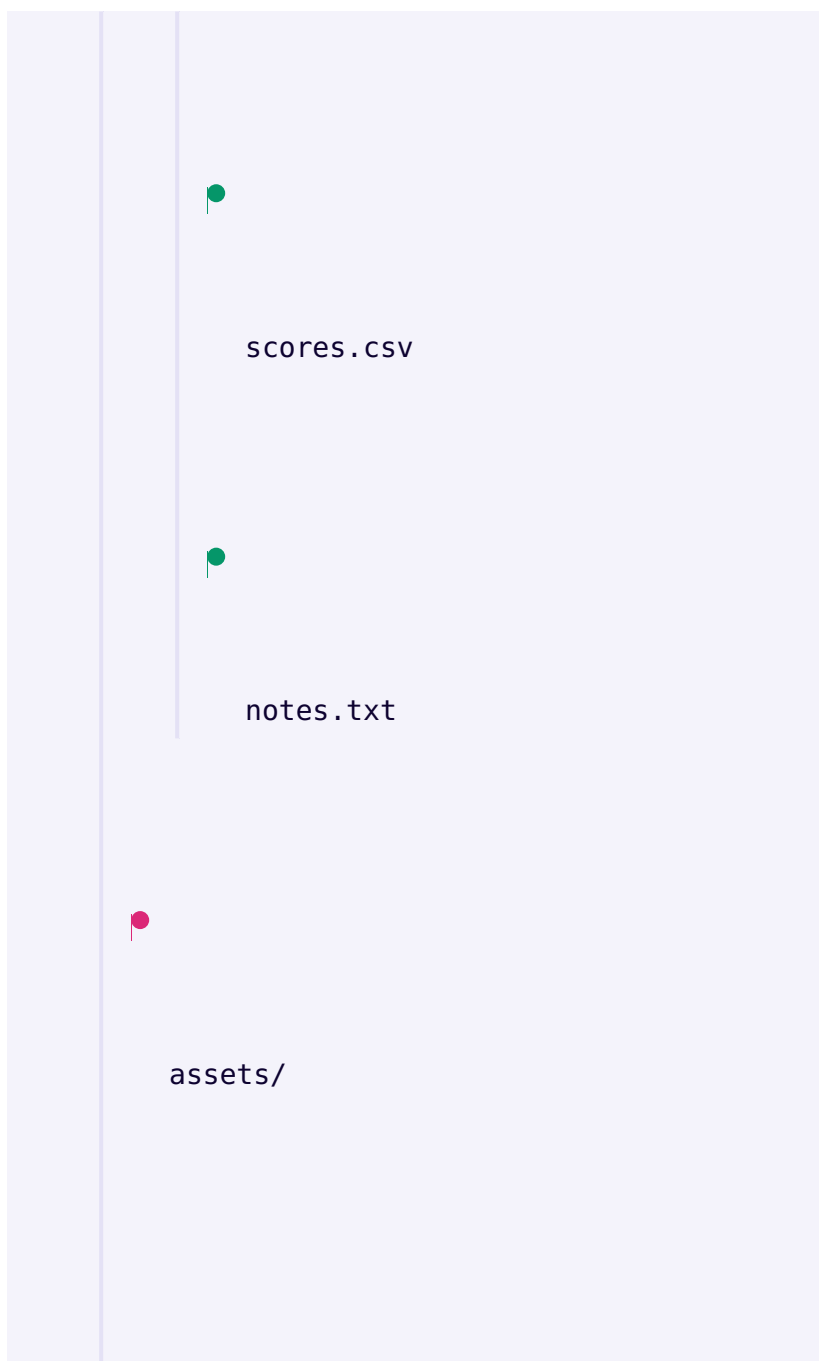
Файловая система различает два основных вида элементов:

- **Файл** — хранит данные (текст, код, картинку, что угодно).
- **Папка** (директория) — не хранит данные сама, а организует другие файлы и папки, вложенные в неё.

Путь говорит, где именно в этой организации находится конкретный элемент — подробно про пути поговорим в следующем разделе.









Файловая система — иерархия от корня

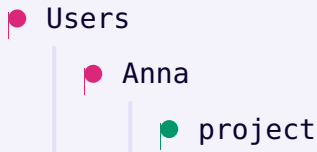
На Linux/macOS всё дерево файлов растёт из одного корня — `/`:



На Windows у каждого диска свой корень, например `C:\`:



C:\\



Windows: диск как корень (например, C:\\).

POSIX и Windows — не соперники

Не нужно спорить, какой вариант «правильный» — они устроены по-разному, и реальным кодом с обоими вариантами вам поможет работать не запоминание конкретных разделителей, а `pathlib` (раздел 15.8), который сам собирает переносимый путь для той системы, где выполняется программа.

Практика: файлы, папки и дерево путей

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/15-05/index.html\)](https://www.cartesianschool.org/practice/15-05/index.html)

Пути: абсолютные и относительные

Абсолютный путь однозначен всегда. Относительный — только относительно чего-то.

Анатомия пути

Путь — это адрес элемента файловой системы, записанный как последовательность имён папок, разделённых слешем, и (для файла) имени файла в конце:

`/home/anna/project/data/
scores.txt`

РОДИТЕЛЬСКАЯ ПАПКА

`/home/anna/project/data`

ИМЯ ФАЙЛА

`scores.txt`

Путь состоит из цепочки папок и имени файла —
подробный разбор имени файла в 15.9.

Абсолютный путь

Абсолютный путь называет местоположение файла, начиная от **корня** файловой системы (или буквы диска на Windows) — он однозначен независимо от того, откуда его читают:

```
absolyutnye_puti.py
```

```
# POSIX (Linux/macOS)  
# /home/anna/project/data.txt  
  
# Windows  
# C:\\Users\\Anna\\project\\data.txt
```


Не зашивайте абсолютные пути в реальный код

Такие пути показаны здесь только как иллюстрация устройства пути. В настоящем коде курса (и в ваших будущих программах) абсолютный путь конкретного компьютера почти никогда не подходит — он не будет существовать на другом компьютере.

Относительный путь — относительно чего?

Путь вида:

```
otnositelny_put.py
```

```
# data/scores.txt
```

не называет место в файловой системе однозначно, пока не ответить на главный вопрос: **относительно чего?** Ответ — раздел 15.7: относительный путь разрешается относительно **текущей рабочей директории** программы, если API или документация не говорят иного.

Практика: абсолютные и относительные пути

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/15-06/index.html) → (<https://www.cartesianschool.org/practice/15-06/index.html>)

Текущая рабочая директория — CWD

Один из самых частых источников файловых ошибок: относительный путь ведёт не туда, куда вы думали.

Текущая рабочая директория (CWD)

Узнать текущую рабочую директорию программы:

```
cwd.py
```

```
from pathlib import Path

print(Path.cwd())
```

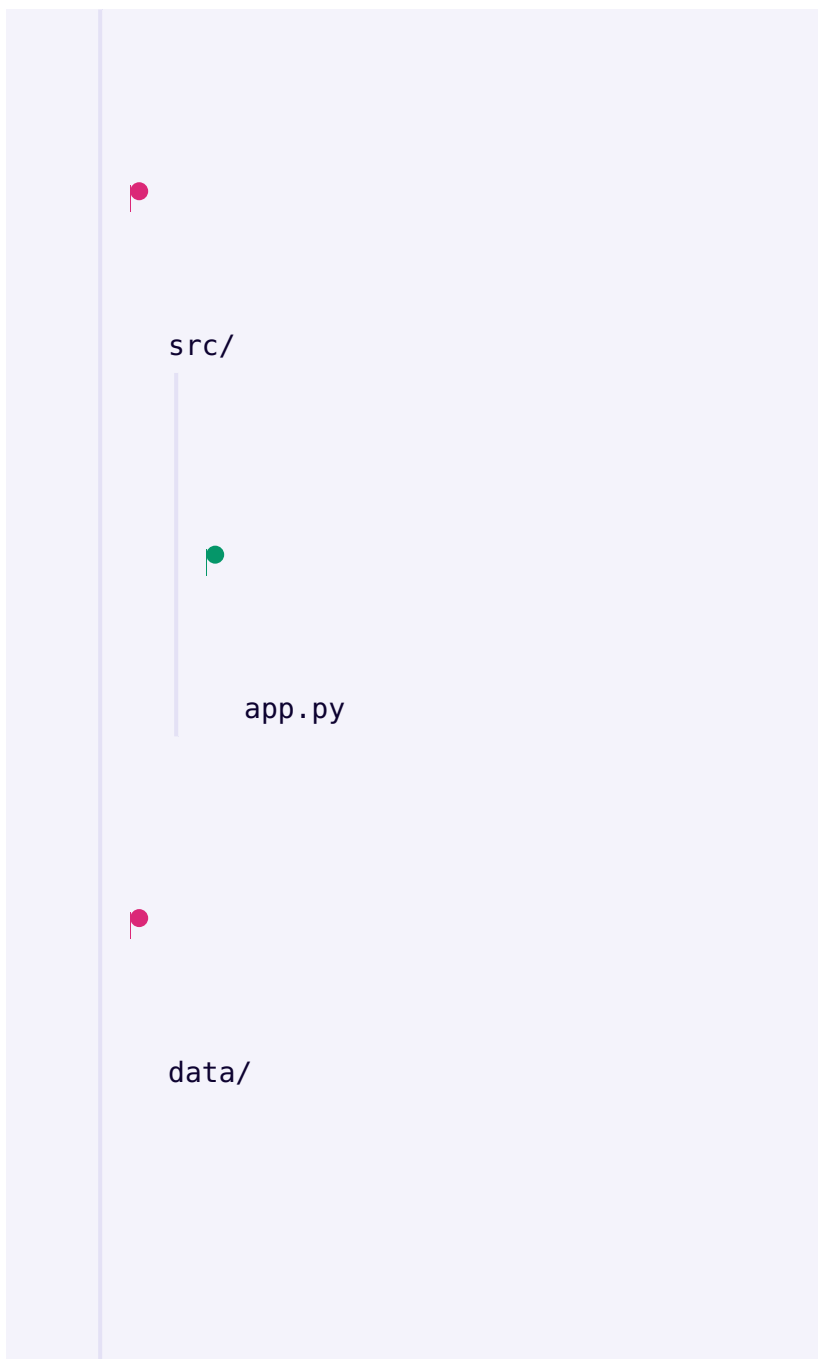
Текущая рабочая директория (CWD) — это папка, относительно которой интерпретируются обычные относительные пути в программе.

CWD — это НЕ «папка со скриптом»

Это одна из самых частых ошибок с путями в реальном коде. CWD и папка, где лежит файл `.py`, могут совпадать — а могут и не совпадать. CWD определяется тем, **откуда была запущена программа**, а не тем, где физически лежит её исходный файл.



project/ ← CWD (запуск `python src/app.py` отсюда)





`config.json`

Скрипт лежит в `src/app.py`, но CWD — это `project/`,
если запустить программу отсюда.

`app_py_fragment.py`

```
# внутри src/app.py, если программа запущена из
project/:
from pathlib import Path

путь = Path("data/config.json")
# указывает на project/data/config.json — НЕ на
project/src/data/config.json
```

**DEBUG LAB 2: CWD — ЭТО НЕ ПАПКА СО
СКРИПТОМ**

```
app_wrong.py
```

```
# файл лежит в project/src/app.py
from pathlib import Path

config = Path("config.json")
# предполагаем: "рядом со скриптом"
print(config.read_text(encoding="utf-8"))
```

```
$ cd project
$ python src/app.py
Traceback (most recent call last):
  File "src/app.py", line 10, in <module>
    print(config.read_text(encoding="utf-8"))
  File "/usr/lib/python3.8/pathlib.py", line 1245, in read_text
    with self.open(mode='r', encoding=encoding, errors=errors) as f:
  File "/usr/lib/python3.8/pathlib.py", line 1230, in open
    return self._io.open(self, mode, buffering, text, **kwargs)
FileNotFoundError: [Errno 2] No such file or directory: 'config.json'
```

Что видно на экране

Файл `config.json` лежит в `project/src/`, рядом со скриптом. Но программа запущена из `project/` — значит, CWD равен `project/`, и относительный путь `"config.json"` ищется в `project/config.json`, которого нет.

ИСПРАВЛЕННЫЙ КОД

```
app_fixed.py
```

```
# надёжный способ — путь относительно самого
# файла, а не относительно CWD
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
config = BASE_DIR / "config.json"
print(config.read_text(encoding="utf-8"))
```

Папка скрипта: `__file__`

Для обычных `.py`-файлов узнать папку, где лежит сам исходный файл, можно так:

```
script_dir.py
```

```
from pathlib import Path
```

```
BASE_DIR = Path(__file__).resolve().parent
```

`__file__` не гарантирован всегда

`__file__` обычно определён в обычных скриптах и модулях, но интерактивная оболочка Python и некоторые окружения ноутбуков могут не задавать эту переменную. Не считайте её универсально доступной в любом контексте выполнения кода.

Практика: CWD и папка скрипта на вашем компьютере

Локальная практика — запустите настоящий `.py`-файл из разных папок и наблюдайте, что меняется

Открыть практику → (<https://www.cartesianschool.org/practice/15-07/index.html>)

pathlib.Path: пути как объекты

Путь — не просто текст. Это объект со своими атрибутами и методами.

Почему pathlib

Путь — это не просто строка, это **предметная область со своими правилами**: у него есть родитель, имя, расширение, он может существовать или не существовать. Вместо ручной сборки строки через `+`:

Путь как строка → путь как объект

Классический подход

```
file_path = "data" +
"/" + "players" + "/" +
"anna.json"
# на Windows придётся
использовать "\", легко
ошибиться
```

Современный Python
(pathlib)

```
from pathlib import Path

file_path =
Path("data") /
"players" / "anna.json"
# сам подставит
правильный разделитель
для текущей ОС
```

Что использовать сегодня: `pathlib` появился в Python 3.4 и с тех пор считается предпочтительным способом работы с путями: он избавляет от ручной сборки строк, ошибок с разделителями и добавляет удобные методы прямо у самого пути.

Путь — не содержимое файла**`Path(...)` не читает файл**

`Path("notes.txt")` создаёт объект-путь — ссылку на место в файловой системе. Он ничего не читает и не открывает сам по себе, пока вы не вызовете у него метод вроде `.open()` или `.read_text()`.

Path как объект: атрибуты и методы

Это прямая связь с главой 14: `Path` — обычный Python-объект со своими атрибутами и методами.

`pathlib.Path` — объект с методами, а не просто текст пути (полный разбор атрибутов — в 15.9).

Path.home()

```
path_home.py
```

```
from pathlib import Path

print(Path.home())  # только для справки
```

Не пишите практические файлы в домашнюю папку

`Path.home()` полезен для справки, но все практики этого курса записывают файлы только в собственную рабочую директорию — никогда напрямую в домашнюю папку пользователя.

Практика: собираем пути через `pathlib`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/15-08/index.html\)](https://www.cartesianschool.org/practice/15-08/index.html)

Разбираем путь: `name`, `stem`, `suffix`, `parent`

У пути, как у любого объекта, есть свои атрибуты — и они отвечают на конкретные вопросы о файле.

Разбираем путь на части

```
razbor_puti.py
```

```
from pathlib import Path

path = Path("/home/anna/project/data/scores.txt")

print(path.name)      # scores.txt
print(path.stem)      # scores
print(path.suffix)    # .txt
print(path.parent)    # /home/anna/project/data
```

**/home/anna/project/data/
scores.txt**

PARENT

/home/anna/project/data

NAME

scores.txt

STEM

scores

SUFFIX

.txt

name = stem + suffix; parent — родительская папка.

Точка и две точки

- `.` — текущая папка.

- `..` — родительская папка.

```
dot_dotdot.py
```

```
from pathlib import Path

print(Path("."))
print(Path(".."))
```

Не увлекайтесь `.././../..`

Длинные цепочки `..` сложно читать и легко сломать при переносе кода в другое место. Там, где это практично, предпочитайте явную базовую папку (как `BASE_DIR` из раздела 15.7) вместо нескольких `..` подряд.

Расширение — это соглашение, а не гарантия

Суффикс не гарантирует формат содержимого

`.txt`, `.json`, `.jpg` — это соглашения об имени файла, а не магическая проверка того, что внутри. Файл `report.txt` технически может содержать что угодно. Не проверяйте формат содержимого только по расширению имени файла.

Практика: `name`, `stem`, `suffix`, `parent`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-09/index.html>)

Практика: пути и CWD

Прежде чем двигаться к самому файлу — закрепим уверенное владение путями.

Соберём вместе всё, что мы теперь знаем о путях: абсолютные и относительные пути, CWD, `pathlib.Path` и разбор пути на части.

```
puteshestvie_po_putyam.py
```

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent if
"__file__" in globals() else Path.cwd()
data_dir = BASE_DIR / "data"
data_dir.mkdir(exist_ok=True)

file_path = data_dir / "otchet.txt"
print("Абсолютный путь:", file_path.resolve())
print("Имя:", file_path.name, "| Расширение:",
file_path.suffix)
print("Существует?", file_path.exists())
```

Path.resolve()

`path.resolve()` строит абсолютный, разрешённый вариант пути — удобно для отладки, чтобы увидеть, куда путь указывает на самом деле. Это не инструмент безопасности — не полагайтесь на него как на «санитайзер» пользовательского ввода (вернёмся к этому в разделе 15.28).

★ Базовая практика

Путь к самому себе

Выведите `BASE_DIR`, `.parent` от `BASE_DIR` и `.parent.parent` от `BASE_DIR` — понаблюдайте, как подниматься по дереву папок.

★★ Самостоятельная задача

Проверка перед чтением

Постройте путь к несуществующему файлу в `data/` и, используя `.exists()`, выведите одно из двух сообщений: «файл найден» или «файла нет — сначала создайте его».

Практика: пути и CWD — итоговая тренировка

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-10/index.html>)

open() возвращает объект файла

Файл в Python — не магия, а объект со своим состоянием и методами.

open() возвращает объект

Это ещё одна прямая связь с главой 14. Результат `open(...)` — не «магический канал», а обычный Python-объект — объект файла:

```
file_object.py
```

```
with open("privet.txt", "r", encoding="utf-8") as file:
    print(type(file))
```

`file` — экземпляр класса, отвечающего за чтение/запись текста конкретного открытого файла.

Переменная `file` в наших примерах — это имя, которое мы сами выбираем; важно то, что оно ссылается на объект со своим состоянием (`.closed`, `.mode`, `.name`) и методами (`.read()`, `.write()` и другими).

Точную иерархию классов не запоминаем

У Python есть отдельные внутренние классы для текстовых и бинарных файлов, но для повседневной работы достаточно знать: `open()` возвращает объект файла с предсказуемым набором методов — точную иерархию классов ввода-вывода запоминать не нужно.

Практика: объект файла и его атрибуты

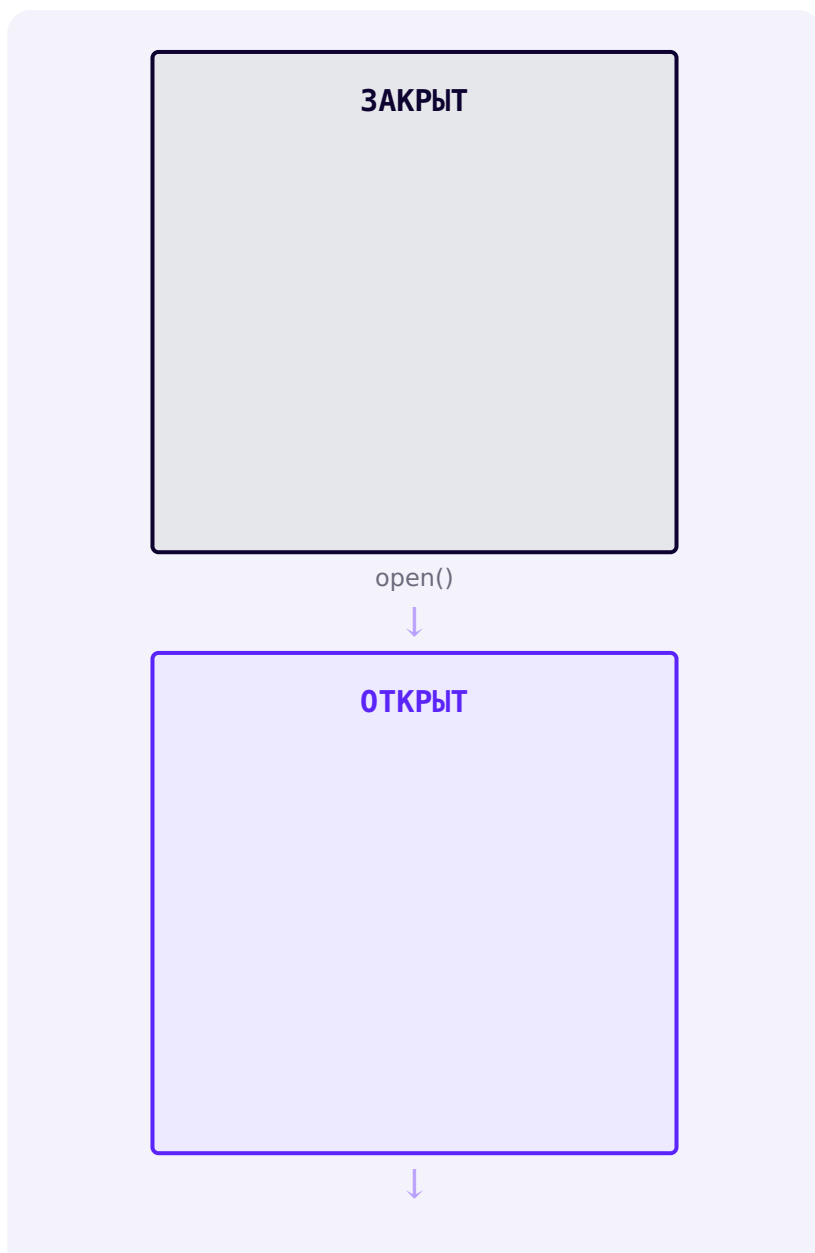
Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

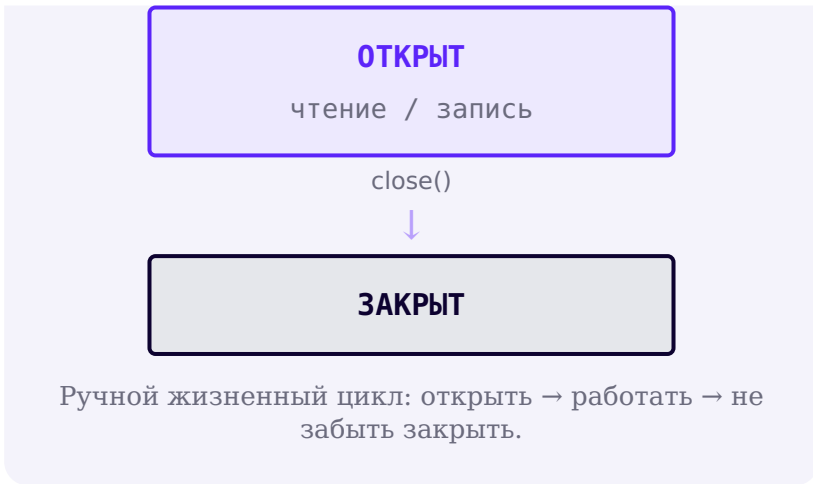
[Открыть практику →](https://www.cartesianschool.org/practice/15-11/index.html) (<https://www.cartesianschool.org/practice/15-11/index.html>)

Жизненный цикл файла и with

`with` — не просто удобный синтаксис, а гарантия корректного закрытия ресурса.

Жизненный цикл файла





С менеджером контекста



```
with_protokol.py
```

```
with open("privet.txt", "r", encoding="utf-8") as
file:
    print(file.closed)    # False — файл открыт внутри
                           блока
print(file.closed)       # True — закрылся сам при
                           выходе из блока
```

with — это не просто «модный синтаксис»

`with` появился в Python 2.5 — дело не в новизне, а в надёжном управлении ресурсом: даже если внутри блока произойдёт исключение, протокол менеджера контекста всё равно выполнит корректное закрытие файла.

Что происходит под капотом

Это часть объектной модели Python: объекты, участвующие в `with`, поддерживают специальное поведение при входе и выходе из блока — методы `__enter__` и `__exit__`. Пока не нужно уметь писать свои такие объекты — достаточно знать, что файлы — лишь одно из применений этого протокола, а не какая-то функциональность, придуманная специально для файлов.

Ручное закрытие — исторический вариант

```
ruchnoe_zakrytie.py
```

```
file = open("privet.txt", "r", encoding="utf-8")
content = file.read()
file.close()    # нужно не забыть — и выполнить даже
                 при ошибке выше
```


with — не просто новее, а надёжнее

Ручное открытие/закрытие всё ещё работает и встречается в старом коде, но `with` рекомендуется не потому, что он новее, а потому, что закрытие происходит гарантированно, даже при ошибке.

Практика: жизненный цикл файла

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-12/index.html>)

Курсор файла: tell() и seek()

У открытого файла есть текущая позиция — она двигается вперёд, но сама назад не возвращается.

Курсор файла

У открытого файла есть текущая **позиция чтения/записи** — курсор. Он двигается вперёд по мере чтения или записи:

`cursor.py`

```
with open("alfavit.txt", "w", encoding="utf-8") as f:
    f.write("ABCDE")

with open("alfavit.txt", "r", encoding="utf-8") as f:
    print(f.read(2))    # AB — курсор передвинулся на 2
    print(f.read(2))    # CD — курсор передвинулся ещё
на 2
    print(f.read())      # E — до конца файла
    print(repr(f.read())) # '' — курсор уже в конце
```



Курсор перед первым `read(2)`: позиция 0.



После `read(2)`: курсор на позиции 2, прочитано «AB».



После второго `read(2)`: курсор на позиции 4, прочитано «ABCD».



После `read()`: курсор в конце файла (EOF), позиция 5.

DEBUG LAB 3: КУРСОР УЖЕ В КОНЦЕ ФАЙЛА

`posle_eof.py`

```
with open("alfavit.txt", "r",
encoding="utf-8") as f:
    print(f.read())
    print(f.read())    # ожидаем увидеть то же
                        # самое ещё раз?
```

ABCDE
(пустая строка)

Что видно на экране

Второй `f.read()` вернул пустую строку, а не «ABCDE» снова. Курсор не возвращается в начало сам — после первого `read()` он остался в конце файла (EOF), и там больше нечего читать.

ИСПРАВЛЕННЫЙ КОД

vozvrat_v_nachalo.py

```

with open("alfavit.txt", "r",
encoding="utf-8") as f:
    print(f.read())
    f.seek(0)          # вручную возвращаем
                        # курсор в начало
    print(f.read())    # теперь снова ABCDE

```

tell() и seek()

tell_seek.py

```

with open("alfavit.txt", "r", encoding="utf-8") as f:
    f.read(2)
    print(f.tell())    # 2 — текущая позиция курсора
    f.seek(0)          # вернуться в начало
    print(f.read())    # ABCDE — читаем заново

```

seek() в текстовом режиме — не «номер символа»

Для простых примеров с ASCII-текстом смещение, которое возвращает `tell()`, интуитивно совпадает с числом прочитанных символов. Но в общем случае для текста в UTF-8 это неверно: один символ (особенно кириллица или эмодзи) может занимать несколько байт (раздел 15.16), и произвольные смещения не соответствуют напрямую индексам символов. Для произвольных байтовых смещений понятнее и безопаснее работать в бинарном режиме (раздел 15.17). Безопасное использование `seek()` в текстовом режиме — `seek(0)`, чтобы начать чтение сначала.

Практика: курсор файла, tell() и seek()

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/15-13/index.html\)](https://www.cartesianschool.org/practice/15-13/index.html)

Читаем файлы: read(), readline(), readlines()

Один и тот же файл можно прочитать по-разному — и выбор способа — это выбор, а не формальность.

read() — весь файл целиком

read_ves.py

```
with open("spisok.txt", "r", encoding="utf-8") as f:  
    content = f.read()
```

read() — не всегда плохо

Для небольших файлов читать всё сразу — просто и удобно. Для очень больших файлов (гигабайты логов) это может занять много памяти — тогда лучше читать построчно или порциями (раздел 15.23). Не одно решение всегда правильное — важно выбирать под задачу.

readline() — по одной строке

readline.py

```
with open("spisok.txt", "r", encoding="utf-8") as f:
    print(f.readline()) # первая строка
    print(f.readline()) # вторая строка
    # ...
    print(repr(f.readline())) # '' — когда строк
    больше нет
```

DEBUG LAB 4: READ() ВМЕСТО READLINE()

odna_stroka.py

```
with open("spisok.txt", "r",
encoding="utf-8") as f:
    first_line = f.read() # хотели одну
    строку
    second_line = f.read()
    print("Первая:", first_line)
    print("Вторая:", repr(second_line))
```

Первая: яблоки\пхлеб\пмолоко\псыр\n
Вторая: ''

Что видно на экране

Ожидалось, что каждый вызов вернёт одну строку — но `read()` без аргумента читает **весь оставшийся файл**, а не одну строку. Курсор сразу оказался в конце, и второй вызов вернул пустую строку.

ИСПРАВЛЕННЫЙ КОД

odna_stroka_fixed.py

```

with open("spisok.txt", "r",
encoding="utf-8") as f:
    first_line = f.readline()
    second_line = f.readline()
    print("Первая:", first_line.strip())
    print("Вторая:", second_line.strip())

```

Цикл по файлу — предпочтительный способ

cikl_po_fajlu.py

```

with open("spisok.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())

```

Объект файла — итерируемый

Как и список или строка (глава 10), объект файла можно обходить циклом `for` напрямую — Python сам читает по одной строке за раз, не загружая весь файл в память.

readlines() — все строки списком

```
readlines_spisok.py
```

```
with open("spisok.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()

print(len(lines), "строк")
```

Способ	Что возвращает	Память
for line in file	по одной строке за раз	экономно — подходит для больших файлов
file.readlines()	список всех строк сразу	весь список строк в памяти одновременно

Практика: read(), readline(), readlines()

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/15-14/index.html\)](https://www.cartesianschool.org/practice/15-14/index.html)

Пишем в файлы: write() и режимы r/w/a/x

Каждый режим открытия — это осознанный выбор, а не деталь синтаксиса.

write() не добавляет перевод строки сам

write_bez_n.py

```
with open("log.txt", "w", encoding="utf-8") as f:  
    f.write("Hello")  
    f.write("World")
```

ЗАПИСАЛИ

```
f.write("Hello")  
f.write("World")
```

→

→

НА ДИСКЕ

HelloWorld

write() не вставляет \n сам — если нужен перевод строки, пишите его явно: f.write("Hello\n").

Режимы открытия — обзор

Четыре базовых режима

OPEN(ПУТЬ, "R")

Чтение.

Ошибка, если файл не существует.

OPEN(ПУТЬ, "W")

Запись.

Создаёт новый файл или очищает
существующий
УЖЕ ПРИ ОТКРЫТИИ.

OPEN(ПУТЬ, "A")

Дозапись в конец.

Создаёт файл, если его нет.

OPEN(ПУТЬ, "X")

Создаёт новый файл.

FileExistsError, если файл уже
существует.

Дальше добавляются модификаторы t/b/+ (раздел
15.17 и далее).

**DEBUG LAB 5: "W" СТИРАЕТ ФАЙЛ ЕЩЁ ДО
ЗАПИСИ**

perezapis.py

```
# rezultaty.txt уже содержит важные результаты
# предыдущей игры
with open("rezultaty.txt", "w",
encoding="utf-8") as f:
    pass    # ничего даже не записали
```

(файл rezultaty.txt теперь пуст)

Что видно на экране

Файл стал пустым, хотя мы ничего и не записали внутри блока. Открытие в режиме "w" стирает прежнее содержимое **в момент открытия**, а не в момент первого `write()`.

ИСПРАВЛЕННЫЙ КОД

dozapis_vместo_w.py

```
# если цель — добавить, а не заменить, нужен
# режим "a", а не "w"
with open("rezultaty.txt", "a",
encoding="utf-8") as f:
    f.write("Новый результат\n")
```

Режим "x" — создать, но не заменить

```
rezhim_x.py
```

```
try:
    with open("save.json", "x", encoding="utf-8") as
f:
    f.write("{}")
except FileExistsError:
    print("Файл сохранения уже существует — не
перезаписываем его случайно")
```

x — защита от случайной перезаписи

Если важно не перезаписать существующий файл по ошибке — режим "x" явно сообщит об этом через `FileExistsError`, вместо тихой перезаписи, которую сделал бы режим "w".

DEBUG LAB 6: WRITELINES() НЕ ДОБАВЛЯЕТ ПЕРЕНОСЫ СТРОК

```
writelines_lovushka.py
```

```
stroki = ["Anna", "Bob", "Carlos"]
with open("igroki.txt", "w",
encoding="utf-8") as f:
    f.writelines(stroki)
```

(файл `igroki.txt` содержит одну строку: `AnnaBobCarlos`)

Что видно на экране

Ожидались три строки, а получилась одна слитая строка. `writelines()` — несмотря на название — **не добавляет переносы строк сама**, она просто записывает элементы один за другим. Если нужны отдельные строки, перевод строки нужно включить в каждый элемент заранее.

ИСПРАВЛЕННЫЙ КОД

```
writelines_fixed.py
```

```
stroki = ["Anna", "Bob", "Carlos"]
with open("igroki.txt", "w",
encoding="utf-8") as f:
    f.writelines(s + "\n" for s in stroki)
```

Чуть

глубже: модификаторы +

`r+`, `w+`, `a+` открывают файл сразу для чтения и записи — но базовая семантика усечения/дозаписи каждого режима (`w` стирает, `a` добавляет в конец) продолжает действовать и здесь. Это необязательный материал — базовых `r/w/a/x` достаточно для большинства программ.

Практика: write() и режимы r/w/a/x

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-15/index.html>)

Текст, bytes и кодировка UTF-8

Файлы хранят байты. То, как байты превращаются в текст, — это кодировка, и она не универсальна по умолчанию.

Текст и bytes — разные типы

str_vs_bytes.py

```
print(type("Python"))      # <class 'str'>
print(type(b"Python"))     # <class 'bytes'>
```

bytes — не «строка без Unicode»

bytes — это последовательность целых чисел от 0 до 255 (двоичные данные), а не урезанная версия строки. Текстовая строка (str) и бинарные данные (bytes) — два разных типа с разным назначением.

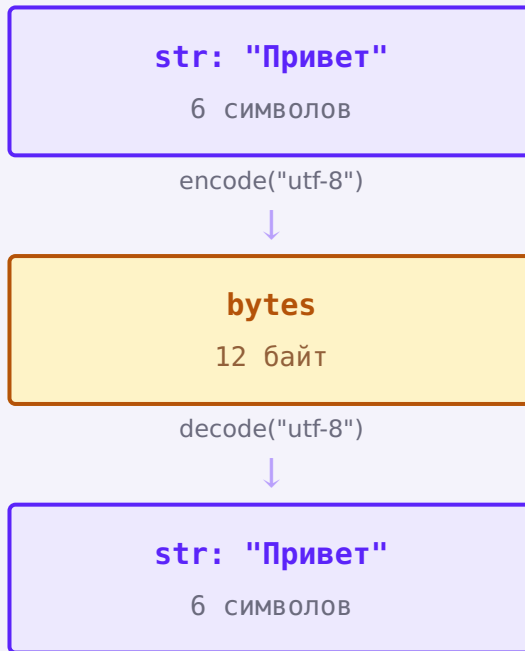
Кодирование и декодирование

Чтобы записать текст в файл, его строковое представление превращают в байты — это называется **кодированием (encode)**. При чтении происходит обратное — **декодирование (decode)**:

```
encode_decode.py
```

```
text = "Привет"
b = text.encode("utf-8")
print(b)                # b'\xd0\x9f\xd1\x80\xd0\x
                        \xb8\xd0\xb2\xd0\xb5\xd1\x82'
print(len(text))        # 6 — шесть символов
print(len(b))           # 12 — двенадцать байт!

obratno = b.decode("utf-8")
print(obratno)          # Привет
```



Кириллица в UTF-8 занимает больше байт, чем символов — реально выполненный пример.

Unicode ≠ UTF-8

Unicode — это система, сопоставляющая символам числовые коды (какой символ значит какое число). UTF-8 — лишь один из способов записать эти числа байтами; есть и другие кодировки. В этом курсе мы всегда явно выбираем UTF-8 как стандарт для текстовых файлов — но это выбор, а не единственно возможный вариант.

Когда формат текста заранее известен как UTF-8 (а в этом курсе это так почти всегда), явное указание `encoding="utf-8"` делает поведение программы предсказуемым и переносимым — вместо того, чтобы зависеть от кодировки по умолчанию, которая определяется платформой и настройками окружения выполнения.

Почему кодировка важна

DEBUG LAB 7: ФАЙЛ ОТКРЫТ БЕЗ УКАЗАНИЯ ENCODING

```
bez_encoding.py
```

```
with open("privet.txt", "w") as f:    # без
    encoding!
    f.write("Привет!")
```

```
# Работает по-разному на разных компьютерах и системах
# кодировка по умолчанию НЕ гарантировано UTF-8 везде
```

Что видно на экране

Без явного `encoding="utf-8"` Python использует кодировку по умолчанию, зависящую от платформы и настроек окружения — а не гарантированно UTF-8. Файл, записанный на одном компьютере, может быть неправильно прочитан на другом, если их кодировки по умолчанию различаются.

ИСПРАВЛЕННЫЙ КОД

`s_encoding.py`

```
with open("privet.txt", "w",
encoding="utf-8") as f:
    f.write("Привет!") # кодировка указана
                        явно и предсказуема всегда
```

DEBUG LAB 8: UNICODEDECODEERROR: НЕВЕРНАЯ КОДИРОВКА ПРИ ЧТЕНИИ

`nevernaya_kodirovka.py`

```
data = "Привет".encode("utf-8")
with open("soobshenie.bin", "wb") as f:
    f.write(data)

# читаем как текст в ДРУГОЙ кодировке
with open("soobshenie.bin", "r",
encoding="ascii") as f:
    print(f.read())
```

```
Traceback (most recent call last):
UnicodeDecodeError: 'ascii' codec can't decode byte
```

Что видно на экране

Байты были закодированы в UTF-8, а прочитаны как будто в ASCII — кодировки не совпали, и Python не смог превратить эти байты обратно в корректные символы. Решение — читать файл в той же кодировке, в которой он был записан.

ИСПРАВЛЕННЫЙ КОД

```
pravilnaya_kodirovka.py

with open("soobshenie.bin", "r",
encoding="utf-8") as f:
    print(f.read())    # Привет
```

Чуть глубже: параметр `errors=`

У `open()` есть параметр `errors` (`"strict"` по умолчанию, также `"replace"`, `"ignore"`) для случаев несовпадения кодировки.

errors="ignore" не универсальное решение

`errors="ignore"` молча выбрасывает нераспознанные байты — это может незаметно потерять часть данных. Не используйте его как способ «просто убрать ошибку» без понимания, что именно теряется.

Практика: текст, bytes и UTF-8

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/15-16/index.html) → (https://www.cartesianschool.org/practice/15-16/index.html)

Бинарные файлы и переносы строк

Не всё, что лежит в файле, — текст. И даже текст занимает на диске больше, чем кажется.

Бинарный режим

Добавьте `b` к режиму, чтобы работать с файлом как с чистыми байтами, без какой-либо текстовой кодировки:

```
binaryn_fajl.py
```

```
data = bytes([0, 1, 2, 3, 255])

with open("signal.bin", "wb") as f:
    f.write(data)

with open("signal.bin", "rb") as f:
    print(f.read())    # b'|\x00|\x01|\x02|\x03|\xff'
```

Режим	Что получаем в Python	Пример
<code>"rt" / "r"</code>	str (текст)	"Python"

Режим	Что получаем в Python	Пример
"rb"	bytes (двоичные данные)	b'Python'

Не декодируйте произвольные бинарные данные как текст

PNG, JPEG или PDF не станут читаемыми от `open(..., encoding="utf-8")` — это не текстовые данные, и попытка прочитать их как текст приведёт к ошибке или мусору. Для таких форматов используйте бинарный режим (`"rb"`) или специализированную библиотеку для этого формата.

DEBUG LAB 9: БИНАРНЫЕ ДАННЫЕ, ПРОЧИТАННЫЕ КАК ТЕКСТ

```
bin_kak_text.py
```

```
data = bytes([0, 1, 2, 3, 255])
with open("signal.bin", "wb") as f:
    f.write(data)

with open("signal.bin", "r",
encoding="utf-8") as f:
    print(f.read())
```

```
Traceback (most recent call last):
```

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0
```

Что видно на экране

Байт `0xff` не образует корректный символ в UTF-8 — эти данные никогда и не были текстом. Файл нужно открывать в бинарном режиме (`"rb"`), а не текстовом.

ИСПРАВЛЕННЫЙ КОД

```
bin_pravilno.py
```

```
with open("signal.bin", "rb") as f:
    print(f.read())    # b'\\x00\\x01\\x02\\
\\x03\\xff'
```

Переносы строк и текстовый режим

`\n` внутри Python — универсальный перевод строки

На разных операционных системах исторически применяются разные последовательности для перевода строки на диске (например, `\r\n` в файлах, созданных на Windows). Текстовый режим Python читает такие файлы в режиме универсальных переносов строк и отдаёт вам во всех случаях `\n` — вам не нужно обрабатывать оба варианта вручную в обычном текстовом коде.

Символы против байтов — реальный пример

```
simvoly_vs_bajty.py
```

```
text = "Питон "
print(len(text))           # 6 — количество
СИМВОЛОВ
print(len(text.encode("utf-8"))) # 14 — количество
байт в UTF-8
```

Длина текста ≠ размер файла в байтах

Кириллица и эмодзи занимают в UTF-8 несколько байт на один символ. `len(text)` считает символы, а не байты — размер файла на диске может быть заметно больше, чем количество символов в строке.

Практика: бинарные файлы и переносы строк

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-17/index.html>)

pathlib: read_text, write_text, read_bytes, write_bytes

Для «прочитать всё» и «записать всё» `pathlib` даёт короткую запись — но не отменяет `open()`.

Удобные методы pathlib для целого файла

read_write_text.py

```
from pathlib import Path

path = Path("nastroyki.txt")
path.write_text("Привет!", encoding="utf-8")
print(path.read_text(encoding="utf-8"))  # Привет!
```

read_write_bytes.py

```
from pathlib import Path

path = Path("signal.bin")
path.write_bytes(bytes([0, 1, 2]))
print(path.read_bytes())  # b'\x00\x01\x02'
```

write_text()/write_bytes() заменяют содержимое целиком

Как и режим `"w"`, эти методы полностью заменяют содержимое файла — они не подходят, когда нужна дозапись.

open() vs удобные методы pathlib — когда что

Что выбрать для конкретной задачи

Небольшой
файл
целиком,
без
поточковой
обработки

→ `path.read_text() / write_text()`

Нужна
построчная
обработка
или
большой
файл

→ `with path.open(...)` как в разделах 1

Целиком
бинарные
данные
без
потока

→ `path.read_bytes() / write_bytes()`

Нужен
особый
режим,
например
дозапись

→ `open(путь, "a", ...)`

pathlib не заменяет open() полностью

Path добавляет удобные методы для частых случаев целиком прочитать/записать файл — но потоковая построчная обработка и специальные режимы всё ещё держатся на `open()` / `path.open()`. `pathlib` не «заменяет `open()`» и делает его ненужным — они решают разные задачи.

Практика: read_text, write_text, read_bytes, write_bytes

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-18/index.html>)

Папки: exists(), is_file(), mkdir()

Прежде чем работать с путём, часто нужно понять, что перед нами — и создать недостающие папки.

Проверяем, что перед нами

```
exists_is_file_is_dir.py
```

```
from pathlib import Path

p = Path("data")
print(p.exists())      # существует ли путь вообще
print(p.is_file())     # это обычный файл?
print(p.is_dir())      # это папка?
```

Три исхода проверки пути

`exists() → False` → **путь не существует**

`exists() → True, is_file() → True` → **обычный файл**

`exists() → True, is_dir() → True` → **папка**

exists() — это не гарантия на будущее

Между проверкой `if path.exists():` и следующей операцией файловая система может измениться — файл может быть удалён другим процессом, права доступа могут отличаться. Проверка существования полезна для логики программы, но не доказывает, что следующая операция обязательно пройдёт без ошибок (вернёмся к этому в разделе 15.22).

Создание папок

`mkdir_prostoj.py`

```
from pathlib import Path

Path("data").mkdir()
```

mkdir_nadezhny.py

```
from pathlib import Path
```

```
Path("data/users").mkdir(parents=True, exist_ok=True)  
# parents=True — создать все промежуточные папки,  
# если их ещё нет  
# exist_ok=True — не считать ошибкой, если папка уже  
# существует
```

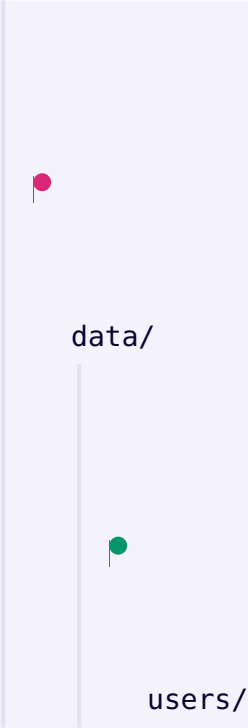


project/ (до)

До: project/ — пустой проект.



project/ (после)



data/

users/

После `Path("data/users").mkdir(parents=True, exist_ok=True)`: обе папки созданы за один вызов.

DEBUG LAB 10: ЗАПИСЬ В НЕСУЩЕСТВУЮЩУЮ ПАПКУ

propushena_papka.py

```
from pathlib import Path

path = Path("data/users/anna.json")
path.write_text("{} ", encoding="utf-8")  #
папки data/users ещё нет!
```

```
Traceback (most recent call last):
  FileNotFounError: [Errno 2] No such file or directory
```

Что видно на экране

Файл можно создать только внутри папки, которая уже существует. `write_text()` / `open(..., "w")` создают файл, но не создают недостающие родительские папки автоматически.

ИСПРАВЛЕННЫЙ КОД

папка_zagotovlena.py

```
from pathlib import Path

path = Path("data/users/anna.json")
path.parent.mkdir(parents=True,
exist_ok=True) # сначала — папки
path.write_text("{} ",
encoding="utf-8") # потом — файл
```

Практика: `exists()`, `is_file()`, `mkdir()`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-19/index.html>)

Поиск файлов: `iterdir()` и `glob()`

Часто нужно не открыть конкретный файл, а сначала понять, что вообще лежит в папке.

Список содержимого папки

iterdir.py

```
from pathlib import Path

for item in sorted(Path("data").iterdir()):
    print(item.name, "-", "папка" if item.is_dir()
          else "файл")
```

Порядок не гарантирован

`iterdir()` не обещает какой-либо конкретный порядок элементов. Если порядок важен для вывода или для теста — обращайтесь в `sorted(...)`, как в примере выше.

Поиск файлов по шаблону: `glob()`

glob_prosty.py

```
from pathlib import Path

for f in sorted(Path("data").glob("*.txt")):
    print(f.name)
```

Файл в data/	Подходит под "*.txt"?
a.txt	да
b.csv	нет
c.txt	да
notes.md	нет

glob() — один уровень, rglob() — рекурсивно

`Path("data").glob("*.txt")` ищет только в самой папке `data`. Чтобы искать также во всех вложенных папках, используйте `Path("data").rglob("*.txt")`. Не запускайте рекурсивный поиск на больших системных папках — это не игрушка для «просканировать весь диск».

Мини-проект: отчёт по папке

Соберите список содержимого учебной папки: имя, тип (файл/папка), расширение и размер (для файлов) — используя только `iterdir()` и атрибуты `Path`, без рекурсивного удаления или изменения содержимого.

`otchet_po_papke.py`

```
from pathlib import Path

def otchet(papka: Path) -> list[str]:
    stroki = []
    for item in sorted(papka.iterdir()):
        vid = "папка" if item.is_dir() else "файл"
        razmer = f", {item.stat().st_size} байт" if
item.is_file() else ""
        stroki.append(f"{item.name} — {vid}{razmer}")
    return stroki

for stroka in otchet(Path("data")):
    print(stroka)
```

★★ Самостоятельная задача**Только .json**

Измените отчёт так, чтобы он показывал только файлы с расширением `.json`, используя `glob("*.json")` вместо `iterdir()`.

Практика: iterdir(), glob() и отчёт по папке

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-20/index.html>)

Переименование, копирование, удаление

Эти операции по-настоящему меняют файловую систему — и не всегда обратимы.

Переименование

rename.py

```
from pathlib import Path

old_path = Path("chernovik.txt")
old_path.rename("gotovo.txt")
```

Файловые операции могут завершиться ошибкой

Переименование, копирование и удаление — это обращения к операционной системе, а не гарантированно успешные действия Python. Они могут завершиться ошибкой по многим причинам файловой системы или прав доступа (раздел 15.22) — не считайте их непременно безотказными.

replace() — для «заменить целевой файл»

replace.py

```
from pathlib import Path

novy_fajl = Path("save_new.json")
tselevoy_fajl = Path("save.json")
novy_fajl.replace(tselevoy_fajl)  # заменяет целевой
файл, если он уже существует
```

Мы вернёмся к этому в разделе 15.28 — он лежит в основе безопасного паттерна сохранения «сначала во временный файл, потом заменить основной».

Копирование — shutil

shutil_copy.py

```
import shutil

shutil.copy("save.json", "save_backup.json")
```

pathlib и shutil дополняют друг друга

pathlib отлично работает с самими путями, а высокоуровневое копирование файлов обычно берёт на себя модуль стандартной библиотеки shutil (shutil.copy, shutil.copy2, shutil.move).

Удаление — с максимальной осторожностью

Удаление — разрушительная операция

`path.unlink()` удаляет файл **без возможности отмены** средствами самой программы. `path.rmdir()` удаляет только **пустую** папку. В этом курсе мы никогда не используем рекурсивное удаление (`shutil.rmtree`) как учебное упражнение — и все примеры удаления в практике работают только с файлами, созданными самой практикой в её собственной рабочей папке, никогда с произвольными путями пользователя.

`unlink_bezопасno.py`

```
from pathlib import Path

vremenny_fajl = Path("chernovik_kopiya.txt")
if vremenny_fajl.exists():
    vremenny_fajl.unlink()
```

Практика: переименование, копирование, удаление

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/15-21/index.html\)](https://www.cartesianschool.org/practice/15-21/index.html)

Ошибки файловой системы

Файловые операции обращаются к внешнему миру — и внешний мир не всегда отвечает так, как хочется.

Частые ошибки файловой системы

Исключение	Когда возникает
FileNotFoundError	путь не существует, а операция ожидала существующий файл
FileExistsError	режим "x" (или похожая операция) — а файл уже есть
PermissionError	нет прав на чтение/запись этого пути
IsADirectoryError	путь — папка, а операция ожидала файл
NotADirectoryError	путь — файл, а операция ожидала папку
UnicodeDecodeError	байты не удаётся декодировать выбранной кодировкой (15.16)

Общий предок — `OSError`

Большинство файловых исключений — это разновидности `OSError`. Знать всю иерархию классов исключений не обязательно — достаточно уверенно распознавать перечисленные выше по имени и понимать, когда каждое из них возникает.

Чек-лист: файл не найден

1. Что сейчас является CWD? (`Path.cwd()` , раздел 15.7)
2. Какой именно путь был построен в коде?
3. Существует ли родительская папка этого пути?
4. Нет ли опечатки в имени?

5. Путь абсолютный или относительный — и относительно чего?
6. Совпадает ли регистр букв в имени файла?

Регистр имени файла — не мелочь

Поведение по чувствительности к регистру различается между файловыми системами. Не считайте, что `Data.txt` и `data.txt` — обязательно один и тот же файл: пишите код так, чтобы имя совпадало точно.

DEBUG LAB 11: ОТКРЫЛИ ПАПКУ КАК ФАЙЛ

`папка_kak_fajl.py`

```
from pathlib import Path

Path("arhiv").mkdir(exist_ok=True)
with open("arhiv", "r", encoding="utf-8") as
f:    # "arhiv" — это папка!
    print(f.read())
```

Traceback (most recent call last):

IsADirectoryError: [Errno 21] Is a directory: 'arhiv'

Что видно на экране

`"arhiv"` — папка, а не файл, и её нельзя открыть как файл для чтения текста. Перед открытием стоило проверить `.is_file()`.

ИСПРАВЛЕННЫЙ КОД

```
proverka_pered_otkrytiem.py
```

```
from pathlib import Path

path = Path("arhiv")
if path.is_file():
    with path.open("r", encoding="utf-8") as f:
        print(f.read())
else:
    print(f"{path} — не обычный файл")
```

Обрабатываем именно ту ошибку, которую ожидаем

```
targeted_except.py
```

```
from pathlib import Path

path = Path("nastroyki.json")
try:
    text = path.read_text(encoding="utf-8")
except FileNotFoundError:
    print("Файл настроек не найден — используем значения по умолчанию")
    text = "{}"
```

Никогда `except: pass`

Пустой перехват всех исключений подряд молча прячет реальные проблемы программы. Ловите конкретное ожидаемое исключение — как `FileNotFoundError` выше — а не всё подряд.

Чуть

глубже: EAFP и LBYL

Два стиля: **проверить заранее** (`if path.exists():`) или **попробовать и обработать ошибку** (`try/except`). В Python операции с файлами часто пишут вторым способом — именно потому, что состояние файловой системы может измениться между проверкой и самой операцией (раздел 15.19). Запоминать сами английские сокращения не обязательно — важна идея.

Практика: ошибки файловой системы

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-22/index.html>)

Большие файлы и потоковая обработка

Способ чтения файла — это решение, которое должно учитывать его размер.

Размер файла

```
razmer_fajla.py
```

```
from pathlib import Path

path = Path("spisok.txt")
print(path.stat().st_size, "байт")
```

Размер в байтах — не длина текста

`.stat().st_size` — это размер в байтах на диске, а не количество символов. Для текста в UTF-8 с кириллицей или эмодзи эти числа, как мы видели в разделе 15.17, обычно не совпадают.

Маленький файл vs большой файл

Выбор способа чтения по размеру файла

Файл
заведомо
небольшой
(настройки,
короткая
заметка)

→ `read_text()` / `read()` целиком — просто

Файл
может
быть
очень
большим
(журнал,
лог за
год)

→ **построчная обработка – цикл `for line in`**

`readlines()` на действительно большом файле

Файл в 1 гигабайт логов — плохой повод для `readlines()`: весь список строк придётся держать в памяти одновременно. Построчный цикл `for line in file` обрабатывает файл по одной строке, не накапливая их все сразу.

Чуть глубже: чтение бинарных данных порциями

Для очень больших бинарных файлов иногда читают не всё сразу, а порциями (*chunks*) фиксированного размера:

chunked_read.py

```

with open("bolshoy_fajl.bin", "rb") as f:
    while True:
        chunk = f.read(8192)    # порция по 8192 байта
        if not chunk:
            break
        # обработать chunk здесь

```

Это необязательный, более глубокий материал — для большинства учебных и небольших прикладных задач построчного чтения текста достаточно.

Мини-проект: анализатор текстового файла

Посчитаем строки, слова и символы файла потоково — без загрузки всего файла в память сразу как единой строки:

analizator_fajla.py

```

from pathlib import Path

def analiz_fajla(path: Path) -> dict[str, int]:
    stroki = 0
    slova = 0
    simvoly = 0
    with path.open("r", encoding="utf-8") as f:
        for line in f:
            stroki += 1
            slova += len(line.split())
            simvoly += len(line)
    return {"строки": stroki, "слова": slova,
            "символы": simvoly}

print(analiz_fajla(Path("spisok.txt")))

```

★★ Самостоятельная задача

Байты отдельно

Добавьте в отчёт четвёртое число — размер файла в байтах через `.stat().st_size` — и сравните его с количеством символов.

Практика: большие файлы и анализатор текста

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-23/index.html>)

Как выбрать формат хранения данных

Прежде чем сохранять данные, стоит спросить: а в каком виде их сохранить?

Один и тот же вопрос, разные форматы

Как из главы 11 знаком словарь:

```
igrok_slovar.py
```

```
igrok = {
    "name": "Anna",
    "score": 1200,
    "skills": ["Python", "Git"],
}
```

Как сохранить такую структуру в файл? `str(igrok)` технически работает, но это не настоящий формат для обмена данными — его неудобно и небезопасно разбирать обратно. Нужен **формат хранения**, у которого есть чёткие правила записи и чтения.

Сериализация

Сериализация и десериализация

Сериализация — превращение структуры Python (словаря, списка, объекта) в представление, которое можно сохранить или передать (текст или байты).

Десериализация — восстановление структуры данных обратно из этого представления.



Сериализация превращает объекты Python в сохраняемый текст.

Как выбрать формат

Какой формат хранения выбрать

Простой
человекочитаемый
текст — заметки,
лог

→ **обычный текстовый файл (главы**

Таблица со
строками и
столбцами

→ **CSV (раздел 15.27)**

Вложенная
структура —
словари,
списки,
настройки,
сохранение
игры

→ **JSON (раздел 15.25)**

Картинка,
звук,
произвольные
не-текстовые
данные

→ **бинарный формат (раздел 15.17)**

Ни один формат не «лучший всегда»

Выбор формата — это решение под конкретную задачу, а не вопрос моды. Плоский текст подходит для заметок, но плохо подходит для вложенных структур; JSON отлично сохраняет структуру, но плохо подходит для таблиц с тысячами одинаковых строк, где куда удобнее CSV.

Практика: выбираем формат хранения

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/15-24/index.html\)](https://www.cartesianschool.org/practice/15-24/index.html)

JSON: сохраняем структуры данных

Словари и списки из главы 11 наконец получают надёжный способ пережить перезапуск программы.

Модуль json

json_dump.py

```
import json

data = {"name": "Anna", "score": 1200}

with open("igrok.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2)
```

```
$ cat igrok.json
{
  "name": "Anna",
  "score": 1200
}
```

Реально сохранённый файл — самый обычный текст, его можно открыть в любом редакторе.

```
json_load.py
```

```
import json

with open("igrok.json", "r", encoding="utf-8") as f:
    data = json.load(f)

print(type(data))    # <class 'dict'>
print(data["name"])  # Anna
```

dump/load vs dumps/loads

Функция	Что делает
<code>json.dump(data, file)</code>	пишет JSON прямо в открытый файловый объект
<code>json.dumps(data)</code>	возвращает JSON как обычную строку str
<code>json.load(file)</code>	читает JSON из открытого файлового объекта
<code>json.loads(text)</code>	разбирает JSON из готовой строки str

Соответствие типов JSON ↔ Python

JSON	Python
object	dict
array	list
string	str
number	int / float

JSON	Python
true / false	True / False
null	None

Что JSON не умеет напрямую

JSON не сохраняет произвольный объект Python сам `set`, экземпляр собственного класса, `bytes`, `complex` — все они не имеют прямого представления в JSON. `json.dump()` не умеет «магически» сохранить произвольный объект Python — сначала нужно самостоятельно привести данные к словарям/спискам/строкам/числам.

Кортеж при чтении вернётся списком

Если сохранить `tuple` как JSON-массив, при загрузке обратно вы получите обычный `list`, а не `tuple` — JSON не различает эти два типа Python, у него есть только один вид «массив».

DEBUG LAB 12: НЕКОРРЕКТНЫЙ JSON В ФАЙЛЕ

`sloanny_json.py`

```
with open("nastroyki.json", "w",
encoding="utf-8") as f:
    f.write('{"theme": "dark",}}') # лишняя
запятая перед }

import json
with open("nastroyki.json", "r",
encoding="utf-8") as f:
    data = json.load(f)
```

```
Traceback (most recent call last):
  json.decoder.JSONDecodeError: Illegal trailing comma
```

Что видно на экране

Наличие файла не означает, что его содержимое — корректный JSON: файл мог быть повреждён, отредактирован вручную или записан с ошибкой в другом месте программы. `json.JSONDecodeError` — сигнал именно об этом, а не о проблеме с путём или правами доступа.

ИСПРАВЛЕННЫЙ КОД

```
json_bez_zapyatoj.py
```

```
with open("nastroyki.json", "w",
encoding="utf-8") as f:
    f.write('{"theme": "dark"}')
```

Мини-проект: менеджер настроек

Настройки приложения, которые нужно будет использовать в графическом приложении главы 16 — с понятными значениями по умолчанию, если файла ещё нет:

```
nastroyki_menedzher.py
```

```
import json
from pathlib import Path

DEFAULT_SETTINGS = {"theme": "light", "language":
"ru", "window_width": 900}
```



```
def load_settings(path: Path) -> dict:
    if not path.exists():
        return dict(DEFAULT_SETTINGS)
    with path.open("r", encoding="utf-8") as f:
        return json.load(f)

def save_settings(path: Path, settings: dict) -> None:
    with path.open("w", encoding="utf-8") as f:
        json.dump(settings, f, ensure_ascii=False,
            indent=2)

settings_path = Path("nastroyki.json")
settings = load_settings(settings_path)
settings["theme"] = "dark"
save_settings(settings_path, settings)
```

Готовим почву для главы 16

Именно так графическое приложение на Tkinter (глава 16) сможет запоминать настройки пользователя между запусками — загружать их при старте и сохранять при изменении, без единой строчки кода интерфейса в этой функции.

Практика: JSON и менеджер настроек

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/15-25/index.html\)](https://www.cartesianschool.org/practice/15-25/index.html)

Мини-проект: сохраняем Player

Объект из главы 14 наконец умеет переживать перезапуск программы.

Соберём вместе главу 14 (dataclasses, объекты) и главу 15 (JSON, файлы): сохраним и загрузим настоящего игрового персонажа.

```
player_dataclass.py
```

```
from dataclasses import dataclass, asdict

@dataclass
class Player:
    name: str
    score: int
    inventory: list[str]
```

Сохранение

```
save_player.py
```

```
import json
from dataclasses import asdict
from pathlib import Path

player = Player(name="Anna", score=1200,
inventory=["меч", "щит"])

with Path("save.json").open("w", encoding="utf-8") as
f:
    json.dump(asdict(player), f, ensure_ascii=False,
indent=2)
```

asdict() — короткий путь от dataclass к словарию

`dataclasses.asdict()` (глава 14) превращает экземпляр `dataclass` в обычный словарь — если все значения внутри уже JSON-совместимы (строки, числа, списки, словари), результат можно передать прямо в `json.dump()`.

Загрузка

load_player.py

```
import json
from pathlib import Path

with Path("save.json").open("r", encoding="utf-8") as f:
    data = json.load(f)

loaded_player = Player(**data)
print(loaded_player)
```

player : Player

name = "Anna"
score = 1200

asdict() + json.dump()



save.json

JSON-текст на диске



программа завершается

json.load() + Player(**data)



loaded_player : Player

```
name = "Anna"
score = 1200
```

`loaded_player` — НЕ тот же объект, что `player`: это новый экземпляр, восстановленный из файла.

Это не «тот же самый» объект

После перезапуска программы `loaded_player` — это **новый** экземпляр `Player`, построенный из данных файла, а не тот объект, что существовал в предыдущем запуске программы. Он просто имеет такое же состояние — и в этом и есть смысл персистентности: не сохранить сам объект Python, а сохранить достаточно данных, чтобы восстановить эквивалентное состояние.

★★★ Задача повышенной сложности

Дальше самостоятельно: книга контактов на JSON

Возьмите контактную книгу главы 12 (словарь имя → телефон) и добавьте ей сохранение/загрузку через JSON, по образцу этого раздела — теперь контакты будут переживать перезапуск программы.

Практика: сохраняем и загружаем Player

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/15-26/index.html\)](https://www.cartesianschool.org/practice/15-26/index.html)

CSV: таблицы в текстовом виде

Табличные данные заслуживают формат, который понимает, что такое поле — а не просто разделитель.

Таблица в текстовом виде

rekordy.csv

```
name,score,level
Anna,1200,5
Bob,900,4
```

CSV (comma-separated values) — простой текстовый формат для таблиц: строки — записи, значения в строке разделены запятыми. Подходит для экспорта в таблицы, отчётов, наборов данных.

Почему НЕ `split(",")`

DEBUG LAB 13: CSV, РАЗОБРАННЫЙ ЧЕРЕЗ `SPLIT(",")`

csv_split_lovushka.py

```
stroka = 'Anna,"Отлично, продолжай!'"
chasti = stroka.split(",")
print(chasti)
```

```
['Anna', '"Отлично, продолжай!"]
```

Что видно на экране

Поле в кавычках само содержало запятую — как и полноценно допускает формат CSV. `split(",")` ничего не знает про кавычки в CSV и слепо режет по каждой запятой, разрывая одно поле на два.

Разбирать CSV нужно модулем `csv`, который правильно обрабатывает кавычки и запятые внутри полей.

ИСПРАВЛЕННЫЙ КОД

`csv_modul_pravilno.py`

```
import csv
import io

stroka = 'Anna,"Отлично, продолжай!'"
reader = csv.reader(io.StringIO(stroka))
print(next(reader))  # ['Anna', 'Отлично,
                     продолжай!']
```

csv.reader и csv.DictReader

`csv_reader.py`

```
import csv

with open("rekordy.csv", "r", encoding="utf-8",
          newline="") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Зачем newline=""

Документация модуля `csv` рекомендует открывать файл с `newline=""`, чтобы сам модуль полностью управлял переносами строк внутри полей — без этого возможна двойная обработка переносов на некоторых платформах.

csv_dictreader.py

```
import csv

with open("rekordy.csv", "r", encoding="utf-8",
          newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], int(row["score"]))
```

Значения CSV — всегда строки, пока вы их не преобразуете

`row["score"]` — это строка `"1200"`, а не число `1200`, даже если оно похоже на число визуально. CSV не хранит информацию о типе — приведение (`int(...)`, `float(...)`) нужно делать самостоятельно.

Запись CSV

csv_writer.py

```
import csv

rows = [
    {"name": "Anna", "score": 1200},
    {"name": "Bob", "score": 900},
]

with open("novye_rekordy.csv", "w", encoding="utf-8",
          newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name",
                                           "score"])
    writer.writeheader()
    writer.writerows(rows)
```

CSV vs JSON — и CSV — не Excel

	CSV	JSON
Лучше подходит для	однородных таблиц (строки/столбцы)	вложенных структур
Типы данных	всё — строки текста	строки, числа, булевы, вложенность

CSV — это не файл Excel

`.csv` — обычный текстовый формат, а не формат-книга `.xlsx` с листами, формулами и форматированием — тем занимаются другие, специализированные библиотеки.

Практика: CSV — `csv.reader`, `DictReader`, `writer`

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-27/index.html>)

Безопасная работа с файлами

Запись в файл — действие с последствиями. Разумная осторожность стоит нескольких лишних строк кода.

Перед записью — спросите себя

1. Я хочу заменить содержимое или дозаписать?
2. Или создать только если файла ещё нет?
3. Нужна ли резервная копия перед изменением?
4. Это точно тот путь, который я имею в виду?

Режим — осознанный выбор, не заготовка из примера

"r", "w", "a", "x" — каждый несёт разные последствия для существующего содержимого (раздел 15.15). Копировать режим из примера без вопроса «а что именно мне сейчас нужно?» — источник потери данных.

Практики этого курса — только в песочнице

Обязательное правило безопасности практик

Все файловые практики этого курса читают, создают, изменяют и удаляют файлы **только внутри собственной рабочей папки практики**. Никогда — файлы репозитория курса, документы, которые вы уже сохранили, домашнюю папку пользователя или произвольные абсолютные пути. Это касается и ваших собственных программ: не выполняйте разрушительные операции над путём, который пришёл откуда-то извне, не проверив его.

Путь наружу из своей папки — path traversal

```
put_naruzhu.py
```

```

mya_fajla = input("Имя файла для сохранения: ")
# если пользователь введёт ".././important.txt" —
# куда это укажет?
path = Path("data") / mya_fajla

```

Наивное объединение пути — риск выйти за пределы папки

Если имя файла приходит от пользователя и содержит `..`, простое объединение с базовой папкой может увести путь за её пределы, в совершенно другое место файловой системы. Мы не строим здесь полноценный фильтр безопасности — важно просто знать о самом риске и не доверять пользовательскому имени файла как безопасному само по себе.

Профессиональнее: безопасное сохранение

Идея, снижающая риск оставить основной файл в повреждённом промежуточном состоянии: сначала записать во **временный** файл в той же папке, а затем заменить основной файл целиком:

bezopasnoe_sohranenie.py

```
import json
from pathlib import Path

def bezopasno_sohranit(path: Path, data: dict) ->
None:
    vremenny = path.with_suffix(path.suffix + ".tmp")
    with vremenny.open("w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False,
            indent=2)
    vremenny.replace(path)  # заменяем основной файл
    только когда всё уже записано
```

Не абсолютная гарантия — а сниженный риск

Это снижает риск оставить `path` в частично записанном состоянии при обрыве записи, но не является абсолютной гарантией сохранности данных на любом оборудовании и любой файловой системе.

Резервная копия перед заменой

backup_pered_zamenoj.py

```
import shutil
from pathlib import Path

path = Path("save.json")
if path.exists():
    shutil.copy(path, path.with_suffix(".json.bak"))
```

Одна резервная копия, не бесконечная цепочка

Одной актуальной резервной копии перед заменой обычно достаточно для учебных и небольших прикладных задач — не нужно плодить бесконечно растущую цепочку файлов `.bak.bak.bak`.

Практика: безопасная запись и резервная копия

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-28/index.html>)

Мини-проект: таблица рекордов

Загрузить → изменить → сохранить — рабочий цикл почти любой персистентной программы.

Соберём файлы, списки, словари, функции (глава 13) и сортировку (глава 11) в одном практическом проекте — таблице рекордов.



отсортированный список

сохранение

**rekordy.csv**

обновлённый файл

Загрузить → добавить → отсортировать → сохранить
— типичный конвейер персистентных данных.

tablitsa_rekordov.py

```
import csv
from pathlib import Path

def load_rekordy(path: Path) -> list[dict]:
    if not path.exists():
        return []
    with path.open("r", encoding="utf-8", newline="")
as f:
        reader = csv.DictReader(f)
        return [{"name": row["name"], "score":
int(row["score"])} for row in reader]

def save_rekordy(path: Path, rekordy: list[dict]) ->
None:
    with path.open("w", encoding="utf-8", newline="")
as f:
        writer = csv.DictWriter(f,
fieldnames=["name", "score"])
        writer.writeheader()
        writer.writerows(rekordy)

def top_n(rekordy: list[dict], n: int) -> list[dict]:
    return sorted(rekordy, key=lambda r: r["score"],
reverse=True)[:n]
```

```

path = Path("rekordy.csv")
rekordy = load_rekordy(path)
rekordy.append({"name": "Carlos", "score": 1500})
save_rekordy(path, rekordy)

for zapis in top_n(rekordy, 3):
    print(zapis["name"], zapis["score"])

```

★★ Самостоятельная задача

Без повторов имени

Измените `load_rekordy`/добавление так, чтобы при повторном сохранении результата уже существующего игрока обновлялся его рекорд, а не добавлялась вторая строка с тем же именем.

★★★ Задача повышенной сложности

На JSON вместо CSV

Перепишите тот же проект так, чтобы `rekordy.csv` стал `rekordy.json` — сравните, что изменилось в `load/save`, а что осталось прежним (сортировка, `top_n`).

Практика: таблица рекордов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/15-29/index.html>)

Браузер и локальный диск: две файловые системы

"Файл создан" и "файл появился на моём компьютере" — не всегда одно и то же.

Две разные файловые системы

Практики этого курса выполняются в браузере через Pyodide — настоящий Python, но работающий внутри веб-страницы, а не как обычная программа на вашем компьютере.

Файлы в браузерной практике — не файлы на вашем компьютере

Когда практика этого курса выполняет `Path("hello.txt").write_text(...)`, поведение `open / pathlib` настоящее — запись, чтение и дозапись работают по реальным правилам. Но сам файл существует только во **временной файловой системе Python внутри этой вкладки браузера**. Он не появляется в `~/Downloads`, не виден в проводнике/Finder и исчезает при нажатии «Сбросить среду» или закрытии вкладки.

код практики

```
Path("hello.txt").write_text(...)
```

запись



виртуальная ФС Pyodide

существует внутри вкладки браузера



НЕ ваш реальный диск

Браузерная практика пишет в виртуальную файловую систему — не на настоящий диск компьютера.

DEBUG LAB 14: ФАЙЛ СОЗДАН В PYODIDE, НО ЕГО НЕТ НА ДИСКЕ

putanitsa_fs.py

```
# выполнено ВНУТРИ браузерной практики курса
from pathlib import Path

Path("moy_fajl.txt").write_text("привет",
encoding="utf-8")
# ожидание: "теперь этот файл лежит у меня на
компьютере"
```

```
# Файл действительно создан и читаем внутри этой вкладки
# но его нет ни в проводнике/Finder, ни в реальной до
```

Что видно на экране

Виртуальная файловая система Pyodide полностью реальна для самой программы внутри вкладки — но она изолирована от настоящего диска компьютера. Путать «файл создан программой» с «файл появился на моём компьютере» — источник путаницы именно в браузерных средах выполнения Python.

ИСПРАВЛЕННЫЙ КОД


```
kak-poluchit-nastoyaschiy-fajl.py
```

```
# для файла на настоящем диске — запустите тот же код ЛОКАЛЬНО
# (VS Code / PyCharm / Jupyter на вашем компьютере, раздел 15.30)
from pathlib import Path

Path("moy_fajl.txt").write_text("привет",
encoding="utf-8")
```

Когда важна настоящая персистентность на диске

Для этого раздела — практика, требующая локального запуска: настоящего файла на настоящем диске, который переживёт даже закрытие редактора.

Практика: настоящая персистентность на вашем компьютере

Локальная практика — скачайте `.ipynb` и выполните его в VS Code/PyCharm/Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/15-30/index.html\)](https://www.cartesianschool.org/practice/15-30/index.html)

1. Скачайте ноутбук этого раздела и откройте его локально (VS Code, PyCharm или Jupyter).
2. Выполните ячейку, создающую папку `data/` и записывающую в неё UTF-8-файл.
3. Остановите выполнение (или закройте редактор).
4. Откройте файл в проводнике/Finder или в самом редакторе — убедитесь, что он на месте.
5. Запустите ноутбук ещё раз и прочитайте тот же файл — данные из первого запуска на месте, потому что это настоящий файл на настоящем диске.

Итоги главы: инструментарий работы с файлами

От «куда пропадают переменные» до JSON, CSV и настоящей персистентности — полная карта главы.

Инструментарий работы с файлами

Что выбрать для конкретной задачи

Нужен путь → `pathlib.Path`

Нужна текущая рабочая директория → `Path.cwd()`

Нужна папка рядом со своим .ру-файлом → `Path(__file__).resolve().parent`

Небольшой UTF-8-текстовый файл целиком → `path.read_text() / write_text()`

Построчная
обработка
или особый
режим

→ `with path.open(режим, encoding="utf-`

Дозапись без
потери
старого
содержимого

→ `open(путь, "a", ...)`

Создать, но не
заменить
существующий
файл

→ `open(путь, "x", ...)`

Бинарные
данные
целиком

→ `path.read_bytes() / write_bytes()`

Список содержимого
папки

→ `path.iterdir()`

Поиск файлов
по шаблону

→ `path.glob("*.txt")`

Вложенная структура
данных

→ `модуль json`

Таблица со строками и столбцами

→ **модуль csv**

Много
пользователей /
транзакции

→ **вероятно, база данных – не просто**

Глава 15 целиком

ПЕРСИСТЕНТНОСТЬ

память процесса – временна
устойчивое хранилище переживает
завершение процесса
загрузка ≠ тот же объект

ФАЙЛОВАЯ СИСТЕМА

файл vs папка
абсолютный vs относительный путь
CWD ≠ папка со скриптом

PATHLIB.PATH

путь как объект

```
name / stem / suffix / parent  
read_text/write_text/read_bytes/  
write_bytes
```

ФАЙЛ И WITH

`open()` → объект файла
`with` гарантирует закрытие
курсор движается вперёд, `tell()/seek()`

ТЕКСТ И БАЙТЫ

`str` кодируется в `bytes`
UTF-8 — явный выбор курса
символы ≠ байты

СТРУКТУРИРОВАННЫЕ ДАННЫЕ

JSON — вложенные структуры
CSV — таблицы, не `split(",")`
безопасное сохранение — `temp` → `replace`

● Работа с файлами

Персистентность

- память временна
- хранилище переживает завершение процесса

Пути

- абсолютные и относительные
- CWD
- `pathlib.Path`

Файл

- `open/with`
- курсор, `tell/seek`
- режимы `r/w/a/x`

Текст и байты

- `str/bytes`
- UTF-8

Структурированные данные

- JSON
- CSV

Карта главы 15 целиком.

Что дальше

Глава 16: Tkinter-приложение

использует то, что мы уже умеем



`load_settings() /`
`save_settings()`



`settings.json`

Персистентность, освоенная в этой главе,
становится памятью будущего графического
приложения.

В главе 16 («Создаём классные приложения с Tkinter») мы построим первое настоящее графическое приложение — и оно сможет запоминать настройки пользователя между запусками именно через `load_settings() / save_settings()` из раздела 15.25, без единой новой строчки про файлы.

Что мы узнали в этой главе

- Объекты Python живут в памяти процесса и не переживают его завершение — данные в устойчивом файловом хранилище могут пережить завершение процесса.
- Путь — не содержимое файла: `Path(...)` лишь ссылается на место в файловой системе.
- Относительный путь разрешается относительно текущей рабочей директории (CWD) — а не папки со скриптом.
- `pathlib.Path` — путь как объект: `name/stem/suffix/parent` и удобные методы вместо ручной сборки строк.
- `open()` возвращает объект файла; `with` гарантирует его закрытие даже при ошибке.
- У файла есть курсор — позиция чтения/записи, которая двигается вперёд и не возвращается сама; для этого есть `seek()`.
- Режимы `r/w/a/x` — разный риск для существующего содержимого; `"w"` стирает файл уже при открытии.
- Текст кодируется в байты — всегда явно указывайте `encoding="utf-8"`, не полагайтесь на кодировку по умолчанию.
- JSON сохраняет вложенные структуры Python; CSV — таблицы; ни `split(",")`, ни `str(словарь)` не заменяют настоящий формат сериализации.
- Файлы браузерной практики (Pyodide) — не файлы на вашем реальном компьютере.

Создаём классные приложения с Tkinter

Научимся строить настоящие оконные приложения: разберём событийную модель, дерево виджетов, адаптивную компоновку, формы, меню, диалоги, таймеры и архитектуру приложения — и свяжем интерфейс с функциями, объектами и файлами из предыдущих глав.

16.1 Tkinter — правильно всё настраиваем!

16.2 Метки, кнопки и pack

16.3 Поля ввода

16.4 Переменные Tkinter

16.5 Множество вариантов!

16.6 Меню

16.7 Строки и столбцы — grid

16.8 Мини-проект: калькулятор чаевых

16.9 От терминала к GUI: событийная модель

16.10 Как работает событийный цикл и mainloop

16.11 Виджет и дерево интерфейса

16.12 tk и ttk

16.13 Frame и LabelFrame

16.14 pack подробно

16.15 Адаптивный grid

16.16 Виджеты выбора

16.17 Progressbar и Notebook

16.18 Messagebox и диалоги

16.19 filedialog и pathlib

16.20 Toplevel: несколько окон

16.21 Focus, клавиатура и доступность

16.22 after(): таймеры без блокировки

16.23 Валидация ввода

16.24 Архитектура приложения

16.25 Класс приложения и настройки

16.26 Мини-проект: счётчик кликов

16.27 Мини-проект: конвертер температур

16.28 Мини-проект: таймер обратного отсчёта

16.29 Мини-проект: список задач

16.30 Мини-проект: редактор заметок

16.31 Tip Calculator Pro

16.32 Отладка интерфейса и качество GUI

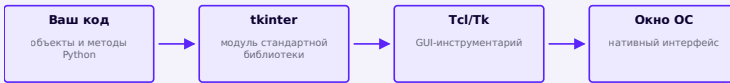
16.33 Итоги главы: инструментарий Tkinter

Tkinter — правильно всё настраиваем!

Первое настоящее оконное приложение — и переход от последовательного выполнения к событийной модели.

До сих пор все программы либо выводили текст в терминал (главы 1–5, 8–15), либо рисовали на холсте Turtle (главы 6–7, 12). Терминальная программа выполняется **строка за строкой**: она делает шаг, ждёт ввода через `input()`, делает следующий шаг — управление всегда у программы. Оконное приложение устроено иначе: оно строит интерфейс один раз, а дальше **ждёт действий пользователя** и реагирует на них в произвольном порядке — пользователь может нажать любую кнопку, ввести текст в любое поле, изменить размер окна. Это называется **событийно-ориентированным программированием** (event-driven programming), и подробно мы разберём эту модель в разделах 16.9–16.10.

Модуль для оконных приложений называется `tkinter` — это интерфейс Python к графической библиотеке Tcl/Tk, и, как и `turtle`, он входит в стандартную поставку Python:



tkinter — мост между Python и системным окружением окон, а не переизобретённый GUI с нуля.

Проверка: доступен ли Tkinter?

Не каждая установка Python обязательно имеет рабочее окружение Tk. Быстрая диагностика:

```
proverka_tkinter.py
```

```
python -m tkinter
```

Если Tkinter и Tk доступны — появится маленькое тестовое окно. Если появляется ошибка импорта или окно не открывается — с локальной установкой Python/Tk нужно разобраться отдельно (в проверенных инструкциях для вашей ОС), прежде чем продолжать эту главу.

Первое окно

```
pervoe_okno.py
```

```
import tkinter as tk

root = tk.Tk()
root.title("Моё первое приложение")

root.mainloop()  # запускает цикл обработки событий
```

`tk.Tk()` создаёт главное окно приложения — примерно как `turtle.Screen()` создавал холст для рисования. Это прямая связь с главой 14: результат `tk.Tk()` — обычный Python-объект со своими методами.

`root` — экземпляр `Tk` с собственными методами; это ООП главы 14 в реальной библиотеке.

root — распространённое соглашение об имени

Главное окно принято называть `root` («корень») — это не обязательное правило языка, а широко принятое соглашение, которое вы увидите почти в любом примере с Tkinter.

Обычно — один root на приложение

В обычном приложении создаётся **один** объект `Tk()` на весь процесс. Для дополнительных окон (настройки, диалоги) используется `tk.Toplevel(...)` (раздел 16.20), а не второй `Tk()`.

mainloop() — не «замирание», а цикл обработки событий

`root.mainloop()` запускает цикл, который активно обрабатывает события: клики, ввод текста, изменение размера окна. Программа не «зависает» и не «засыпает» — она ждёт и реагирует, оставаясь отзывчивой, пока не будет закрыта. Подробно разберём это в разделе 16.10.

Практика: создаём первое окно

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-01/index.html>)

Метки, кнопки и их размещение

Первые интерактивные виджеты — и то, как Tkinter размещает их в окне.

Виджет — визуальный объект интерфейса

Виджет (widget) — любой видимый/интерактивный элемент интерфейса: метка, кнопка, поле ввода. Как и в главе 14, каждый виджет — экземпляр класса со своими атрибутами и методами. **Метка** (`Label`) показывает текст, **кнопка** (`Button`) реагирует на клик.

```
metki_knopki.py
```

```
import tkinter as tk

root = tk.Tk()

label = tk.Label(root, text="Привет, Tkinter!")
label.pack()

def na_knopku_nazhali():
    print("Кнопку нажали!")

button = tk.Button(root, text="Нажми меня",
                    command=na_knopku_nazhali)
button.pack()

root.mainloop()
```

Окно с меткой «Привет, Tkinter!» и кнопкой «Нажми меня»

Результат выполнения кода выше — настоящее окно Tkinter. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

Создание виджета — это не то же самое, что его показ

`tk.Button(root, ...)` лишь создаёт объект кнопки. Он не появится в окне, пока вы не разместите его через **менеджер геометрии** — `.pack()` здесь, или `.grid()` (раздел 16.7), или `.place()` (раздел 16.15).

command — функция, а не вызов функции

Параметр `command` связывает кнопку с функцией — важно передать саму функцию, **без скобок**:

`command=na_knopku_nazhali`, а не
`command=na_knopku_nazhali()`.

Частая ошибка: скобки в command

`na_knopku_nazhali` — это сама функция как объект (глава 13). `na_knopku_nazhali()` — это *вызов* функции прямо сейчас. `command=na_knopku_nazhali()` вызовет функцию **немедленно**, один раз, при создании кнопки — и свяжет кнопку с тем, что функция *вернула* (обычно `None`), а не с самой функцией. Клики по кнопке после этого ничего не вызовут.

Обычная кнопка рядом с недоступной (disabled) кнопкой

`state="disabled"` делает кнопку визуалью и функционально недоступной. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

state="disabled"

Кнопку можно сделать временно недоступной — `button.state(["disabled"])` у `ttk`-виджетов или `state="disabled"` при создании. Полезно, пока данные ещё не готовы для действия (раздел 16.23 — валидация).

Подробно о pack

`.pack()` — самый простой способ разместить виджет в окне. По умолчанию виджеты укладываются друг под другом сверху вниз, но есть полезные параметры:

```
pack_podrobno.py
```

```
label.pack(side="left", padx=10, pady=10)
# side: top (по умолчанию), bottom, left, right
# padx/pady: внешний отступ по горизонтали/вертикали
```

Подробнее про `fill/expand` и группировку через `Frame` — в разделах 16.13-16.14.

Практика: Label, Button и pack()

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/16-02/index.html) (<https://www.cartesianschool.org/practice/16-02/index.html>)

Множество полей ввода

`Entry` для одной строки, `Text` — для нескольких: два способа получить текст от пользователя.

Множество полей ввода

Поле ввода (`Entry`) — однострочное текстовое поле, куда пользователь может что-то напечатать:


```
polya_vvoda.py
```

```
import tkinter as tk

root = tk.Tk()

label = tk.Label(root, text="Введите имя:")
label.pack()

entry = tk.Entry(root)
entry.pack()

def pokazat_imya():
    imya = entry.get()    # достаём текст из поля
    print(f"Привет, {imya}!")

button = tk.Button(root, text="Отправить",
                    command=pokazat_imya)
button.pack()

root.mainloop()
```

`entry.get()` возвращает текущий текст поля — обычную строку, с которой можно работать, как с любой другой (глава 8). `entry.insert(0, "текст")` вставляет текст программно, `entry.delete(0, "end")` очищает поле.

Одна строка текста. Строка за строкой

`Entry` подходит для короткого текста в одну строку — имени, числа, пароля. Для многострочного текста используют другой виджет, `Text`:

`Entry` с текстом «Cartesian» рядом с `Text` из трёх строк

`Entry` — всегда одна строка; `Text` — сколько угодно строк. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

```
mnogostrochnyj_tekst.py
```

```
text_box = tk.Text(root, height=5, width=30)
text_box.pack()

def pokazat_tekst():
    content = text_box.get("1.0", "end-1c")  # с
    начала до конца, без завершающего \n
    print(repr(content))
```

Индекс "1.0" — не опечатка

У Text своя система индексов: "1.0" означает «строка 1, символ 0» — то есть самое начало текста. Это не связано с числами float из главы 4 — просто текстовая метка позиции, и обычная индексация строк Python (text[0]) к Text не применяется.

"end" против "end-1c"

Text всегда добавляет один завершающий символ перевода строки в конец своего содержимого. `.get("1.0", "end")` включает этот лишний `\n`, которого пользователь не печатал. Если нужен именно видимый пользователем текст без этого технического дописка — используйте `.get("1.0", "end-1c")` («конец минус один символ»).

Практика: Entry и Text

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-03/index.html>)

Переменные Tkinter

Специальные переменные, которые сами обновляют связанные с ними виджеты на экране.

Обычные переменные Python не «знают», когда их значение используется в интерфейсе — связь с виджетом нужно устанавливать явно, каждый раз заново. Для

случаев, когда несколько виджетов должны отражать одно и то же значение синхронно, Tkinter предлагает свои специальные переменные: `StringVar`, `IntVar`, `BooleanVar`, `DoubleVar`.

```
peremennye_tkinter.py
```

```
import tkinter as tk

root = tk.Tk()

imya = tk.StringVar(value="Гость")

label = tk.Label(root, textvariable=imya)  # метка
сама обновится при смене imya
label.pack()

def smenit_imya():
    imya.set("Cartesian")

button = tk.Button(root, text="Сменить имя",
command=smenit_imya)
button.pack()

root.mainloop()
```

Значение достают методом `.get()` и меняют методом `.set()` — а виджеты, связанные через `textvariable` (или `variable` у `Radiobutton/Checkbutton`), обновляются на экране автоматически.

imya : StringVar

значение = "Гость"

связаны через одну переменную



```
label (textvariable=имя)
```

Один StringVar может связывать сразу несколько виджетов — все они видят одно значение.

Tk-переменные не заменяют обычные переменные Python

StringVar и подобные — специальный инструмент именно для связи с виджетами, а не universal-замена всех переменных программы. Обычные `int`, `str`, `list` по-прежнему отлично подходят для хранения данных приложения, которые не отображаются напрямую в виджете.

Чуть

глубже: `trace_add`

Можно реагировать на каждое изменение Tk-переменной:

```
trace_add.py
```

```
def pri_izmenenii(*args):
    print("Новое значение:", имя.get())

имя.trace_add("write", pri_izmenenii)
```

Это удобно для «реактивных» интерфейсов, но не обязательно на старте — большинство форм прекрасно обходятся обычным чтением значения по кнопке.

Практика: StringVar и связанные виджеты

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-04/index.html>)

Множество вариантов!

Переключатели для выбора одного варианта и флажки для независимых да/нет решений.

Для выбора из нескольких вариантов у Tkinter есть переключатели (`Radiobutton` — выбрать один из нескольких) и флажки (`Checkbutton` — включить/выключить каждый независимо).

Три переключателя: Чай, Кофе (выбран), Вода

Один выбранный вариант из группы — закрашенный кружок. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

radiobutton.py

```
import tkinter as tk

root = tk.Tk()
vybor = tk.StringVar(value="chay")

tk.Radiobutton(root, text="Чай", variable=vybor,
value="chay").pack()
tk.Radiobutton(root, text="Кофе", variable=vybor,
value="kofe").pack()

def pokazat_vybor():
    print(f"Вы выбрали: {vybor.get()}")

tk.Button(root, text="Готово",
command=pokazat_vybor).pack()
root.mainloop()
```

Группа — это общая переменная

Все переключатели одной группы должны использовать **одну и ту же** переменную (`variable=vybor` у обоих) и **разные** значения (`value="chay"` / `value="kofe"`). Так Tkinter понимает, что это одна группа взаимоисключающих вариантов.

DEBUG LAB 1: У КАЖДОГО RADIOBUTTON — СВОЯ ПЕРЕМЕННАЯ

```
slo mannaya_gruppa.py
```

```
chay_var = tk.StringVar()
kofe_var = tk.StringVar()

tk.Radiobutton(root, text="Чай",
variable=chay_var, value="chay").pack()
tk.Radiobutton(root, text="Кофе",
variable=kofe_var, value="kofe").pack()
```

Оба переключателя можно выбрать **ОДНОВРЕМЕННО** —
 # это уже не взаимоисключающий выбор, а два независимых

Что видно на экране

У каждой кнопки — своя отдельная переменная, поэтому Tkinter не видит связи между ними: выбор одной никак не влияет на другую. Групповое поведение переключателей держится именно на общей переменной, а не на визуальной похожести кнопок.

ИСПРАВЛЕННЫЙ КОД

```
gruppa_fixed.py
```

```
vybor = tk.StringVar(value="chay")

tk.Radiobutton(root, text="Чай",
variable=vybor, value="chay").pack()
tk.Radiobutton(root, text="Кофе",
variable=vybor, value="kofe").pack()
```

Checkbox — независимый выбор

Два флажка: снятый и установленный

Два состояния одного и того же виджета — снят/установлен. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

checkboxbutton.py

```
saharok = tk.BooleanVar(value=False)
tk.Checkbutton(root, text="С сахаром",
variable=saharok).pack()

def pokazat_flazhok():
    print(f"С сахаром: {saharok.get()}")
```

Checkbox — не Radiobutton

`Checkbox` — независимое да/нет решение, не связанное с другими флажками. Не путайте его с группой `Radiobutton`, где выбор одного варианта исключает остальные.

Практика: Radiobutton и Checkbox

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-05/index.html>)

Меню

Настоящее приложение почти всегда начинается с меню в верхней части окна.

Настоящее приложение редко обходится без меню в верхней части окна. В Tkinter меню строится из объекта `Menu` :

menu.py

```

import tkinter as tk

root = tk.Tk()

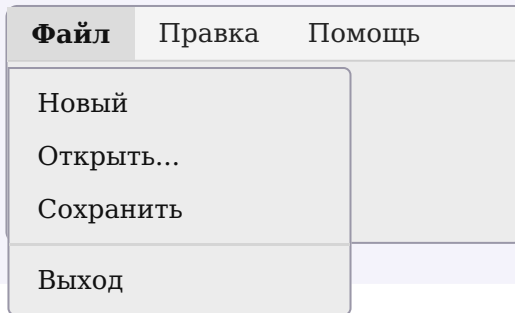
def novyj_fajl():
    print("Создаём новый файл...")

menu_bar = tk.Menu(root)
fajl_menu = tk.Menu(menu_bar, tearoff=0)
fajl_menu.add_command(label="Новый",
                      command=novyj_fajl)
fajl_menu.add_separator()
fajl_menu.add_command(label="Выход",
                      command=root.destroy)
menu_bar.add_cascade(label="Файл", menu=fajl_menu)

root.config(menu=menu_bar)
root.mainloop()

```

**СХЕМАТИЧЕСКОЕ ИЗОБРАЖЕНИЕ (НЕ
РЕАЛЬНЫЙ СКРИНШОТ)**



Внешний вид меню зависит от ОС

Настоящий вид строки меню и открытого меню зависит от операционной системы, версии Tk и темы — курс не может воспроизвести headless-скриншот этого элемента стабильно во всех средах, поэтому здесь используется схематическое изображение с точной структурой (строка меню → пункт → открытое меню → пункты → разделитель), а не фотографический скриншот.

tearoff=0

По умолчанию Tkinter добавляет в начало каждого меню пунктирную линию, позволяющую «оторвать» меню в отдельное окно. `tearoff=0` отключает именно эту возможность отрывать меню — если в вашем интерфейсе она не нужна (а обычно не нужна), это частый и осознанный выбор, а не универсальное правило «всегда так делай».

root.quit() против root.destroy()

Метод	Что делает
<code>root.quit()</code>	останавливает текущий <code>mainloop()</code> — но само окно и виджеты остаются существовать
<code>root.destroy()</code>	уничтожает окно и всё дерево виджетов целиком, также останавливая <code>mainloop()</code>

Это не взаимозаменяемые синонимы

`root.quit()` просит текущий `mainloop()` завершиться — сам объект `root` и виджеты при этом не уничтожаются. `root.destroy()` идёт дальше: уничтожает дерево виджетов и само окно, что тоже останавливает `mainloop()`. Ни один из них не завершает сам процесс Python напрямую — после возврата из `mainloop()` код теоретически может продолжаться (например, что-то сохранить перед выходом); процесс завершается позже, естественным образом, если после этого просто не осталось кода. Для обычного пункта меню «Выход» в этом курсе понятнее и надёжнее `root.destroy`.

Акселераторы — это надпись, не привязка клавиш

`accelerator.py`

```
fajl_menu.add_command(label="Сохранить",
accelerator="Ctrl+S", command=sohranit)
```

accelerator сам по себе не создаёт горячую клавишу

`accelerator="Ctrl+S"` лишь выводит текст подсказки рядом с пунктом меню. Он **не** связывает автоматически комбинацию клавиш с командой — для этого нужен отдельный механизм привязки событий (`.bind(...)`), который подробно разберём в главе 17.

Практика: строим меню приложения

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-06/index.html>)

Строки и столбцы — grid

Для форм с несколькими колонками `grid()` удобнее, чем `pack()`.

`.pack()` прекрасно подходит для простых случаев, но у него есть предел: сложные формы с несколькими колонками собрать сложно. Для этого есть `.grid()` — размещение по строкам и столбцам, как в таблице:

grid.py

```
import tkinter as tk

root = tk.Tk()

tk.Label(root, text="Имя:").grid(row=0, column=0)
tk.Entry(root).grid(row=0, column=1)

tk.Label(root, text="Email:").grid(row=1, column=0)
tk.Entry(root).grid(row=1, column=1)

tk.Button(root, text="Отправить").grid(row=2,
column=0, columnspan=2)

root.mainloop()
```

Позиция	0	1
row=0	Label «Имя:»	Entry
row=1	Label «Email:»	Entry
row=2	Button (columnspan=2 — занимает обе ко- лонки)	

Не смешивайте pack() и grid() в одном контейнере

Виджеты внутри одного и того же родительского окна (или фрейма) должны использовать либо только `pack()`, либо только `grid()` — смешение вызывает ошибку. Разные контейнеры (например, вложенные `Frame`, раздел 16.13) могут использовать разные способы размещения независимо друг от друга — подробная схема «хорошо/плохо» будет в разделе 16.15.

О том, как сделать такую форму отзывчивой к изменению размера окна — в разделе 16.15 «Адаптивный grid».

Практика: компоновка через grid()

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-07/index.html>)

Мини-проект — калькулятор чаевых

Первое полноценное приложение книги — с полями ввода, кнопкой и результатом на экране.

Соберём всё изученное в главе в одном настоящем приложении: калькулятор чаевых с полем ввода, кнопкой и меткой результата. Это **фундамент** — в разделе 16.31 мы вернёмся к нему и превратим в полноценное «Tip Calculator Pro» с классом приложения, валидацией и сохранением настроек.

```
kalkulyator_chaevyh.py
```

```
import tkinter as tk

root = tk.Tk()
root.title("Калькулятор чаевых")

tk.Label(root, text="Сумма счёта:").grid(row=0,
```

```

column=0, padx=10, pady=10)
schet_entry = tk.Entry(root)
schet_entry.grid(row=0, column=1)

tk.Label(root, text="Процент чаевых:").grid(row=1,
column=0, padx=10, pady=10)
procent_entry = tk.Entry(root)
procent_entry.grid(row=1, column=1)

rezultat_label = tk.Label(root, text="",
font=("Arial", 14, "bold"))
rezultat_label.grid(row=3, column=0, columnspan=2,
pady=10)

def poschitat():
    schet = float(schet_entry.get())
    procent = float(procent_entry.get())
    chaevye = schet * procent / 100
    rezultat_label.config(text=f"Чаевые:
{chaevye:.2f}")

tk.Button(root, text="Посчитать",
command=poschitat).grid(row=2, column=0, columnspan=2)

root.mainloop()

```

Какие темы здесь встретились

grid() — раздел 16.7; float() — глава 4;
 форматирование :.2f — глава 8; сама формула
 чаевых — глава 12, проект 12-2.

★★★ Задача повышенной сложности

Проверка ввода

Оберните вычисление в try/except (забегая немного вперёд — подробно об этом в главе 21) — чтобы приложение не падало, если пользователь введёт не число, а текст. В разделе 16.23 мы разберём это системно.

Практика: калькулятор чаевых

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

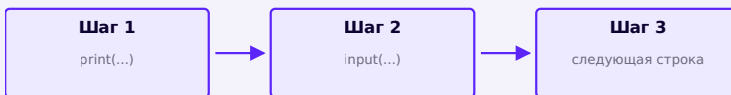
Открыть практику → (<https://www.cartesianschool.org/practice/16-08/index.html>)

От терминала к GUI: событийная модель

Главный интеллектуальный переход этой главы: управление больше не течёт одной сплошной линией.

Терминал: последовательное выполнение

Терминальная программа выполняется предсказуемо, строка за строкой — управление всегда у самой программы:



Терминал: программа сама решает, когда и что делать дальше.

GUI: событийно-ориентированное выполнение

Оконное приложение строит интерфейс один раз, а затем **ждёт события** — и на каждое реагирует отдельным, заранее зарегистрированным кусочком кода:

инициализация интерфейса

переход в цикл событий



mainloop()

пользователь нажал кнопку



событие: клик



Источники событий

Откуда берутся события

ПОЛЬЗОВАТЕЛЬ

клик мышью
ввод текста
действие с окном

СИСТЕМА

перерисовка
изменение размера окна

ПРОГРАММА

таймер / `after()` — раздел 16.22

Событие, `callback`, `command` — не синонимы

Термин	Что это
Событие (event)	то, что произошло — клик, ввод, таймер
Callback	функция/вызываемый объект, зарегистрированный заранее для вызова позже
command	один из способов Tkinter связать <code>callback</code> с конкретным виджетом

Регистрация `callback` — это ещё не его выполнение:

registraciya_vs_vypolnenie.py

```
log = []

def on_click():
    log.append("clicked")

registered = on_click    # регистрация: callback
                        # запомнен, но НЕ вызван
print(log)               # []

registered()             # диспетчеризация: callback
                        # выполняется именно сейчас
print(log)               # ['clicked']
```

Это прямая связь с главой 13

Функция как объект, который можно передать и вызвать позже — ровно то же самое «функция без скобок», о котором шла речь в разделе 16.2 про `command`. Событийная модель Tkinter целиком построена на этой идее.

Практика: регистрация и диспетчеризация callback

Проверяется без tkinter — чистой логикой на функциях-объектах

Открыть практику → (<https://www.cartesianschool.org/practice/16-09/index.html>)

Как работает событийный цикл и mainloop

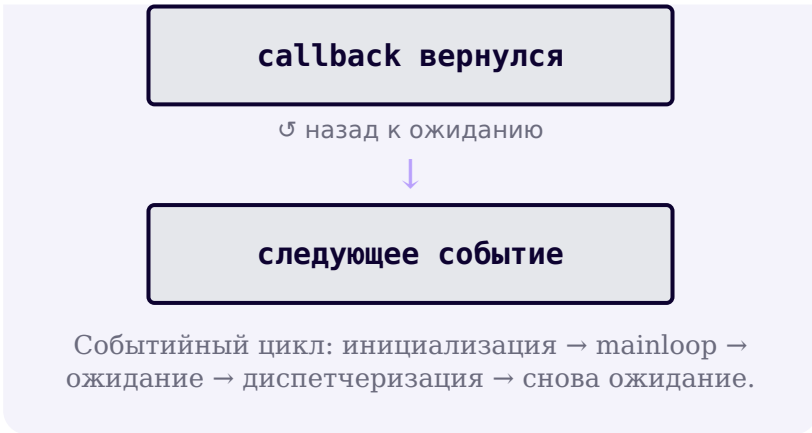
`mainloop()` — не чёрный ящик и не «замирание»: это цикл, который активно ждёт и обрабатывает события.

Событийный цикл — не Python while True

Что на самом деле происходит внутри `mainloop()`

Не думайте о `mainloop()` как о вашем собственном `while True: ...` — это внутренний цикл обработки событий Tcl/Tk. Модель ниже — концептуальная, для понимания порядка событий, а не описание реальной реализации.





Порядок событий — предсказуемый, но не последовательный в коде

sobytijnyj_cikl.py

```
log = []

def on_click(): log.append("click")
def on_timer(): log.append("timer")
def on_type(): log.append("type")

handlers = {"click": on_click, "timer": on_timer,
            "type": on_type}

def run_event_loop(queue):
    for event in queue:
        handlers[event]()

run_event_loop(["click", "timer", "type"])
print(log)      # ['click', 'timer', 'type']
```

Реальные события Tkinter приходят по мере действий пользователя и системы — но идея та же: очередь событий обрабатывается по одному, и каждое запускает свой callback.

Долгий callback — это уже отдельная тема

Пока обработчик выполняется, событийный цикл ждёт его завершения, прежде чем перейти к следующему событию. Что происходит, если callback работает слишком долго — подробно в разделах 16.22 и 16.24.

Практика: порядок обработки событий

Проверяется без tkinter — моделируем очередь событий на чистом Python

Открыть практику → (<https://www.cartesianschool.org/practice/16-10/index.html>)

Виджет и дерево интерфейса

Интерфейс — не плоский список элементов, а дерево вложенных контейнеров.

У каждого виджета есть родитель

Виджеты образуют дерево: у каждого, кроме самого корня, есть родитель (master), который определяет контекст размещения и жизненного цикла:

```

● root
  ● menu_bar
  ● main_frame
    ● title_label
    ● name_entry
    ● save_button

```

Типичное дерево небольшого приложения: `root` → `main_frame` → отдельные виджеты формы.

`derevo_widgetov.py`

```
import tkinter as tk

root = tk.Tk()
main_frame = tk.Frame(root)
main_frame.pack()

title_label = tk.Label(main_frame, text="Форма")
title_label.pack()
name_entry = tk.Entry(main_frame)
name_entry.pack()
```

Первый аргумент почти любого виджета — его родитель (`main_frame` выше). Родитель определяет, в каком контейнере разместится виджет — подробно про `Frame` в разделе 16.13.

Жизненный цикл виджета



Создание виджета создаёт сам объект; менеджер геометрии включает его в раскладку окна.

Создание — это создание объекта, размещение — это раскладка

Создание виджета (`tk.Button(...)`) создаёт сам Python-объект — как и создание любого другого объекта (глава 14). Менеджер геометрии (`pack` , `grid` или `place`) включает этот объект в раскладку окна, чтобы пользователь мог его увидеть и с ним взаимодействовать. Объект существует в обоих случаях — различается лишь то, участвует ли он в раскладке видимого окна.

`destroy()` — не единственный способ скрыть виджет

`.destroy()` уничтожает виджет безвозвратно. Если нужно лишь временно убрать виджет с экрана, есть более лёгкие варианты (например, `.grid_remove()`) — но для большинства первых приложений деструктивное удаление ненужных виджетов и есть правильный выбор.

Практика: дерево виджетов как данные

Проверяется без `tkinter` — моделируем дерево вложенными данными Python

Открыть практику → (<https://www.cartesianschool.org/practice/16-11/index.html>)

tk и ttk: классические и тематизированные виджеты

Начиная с этого раздела в новых примерах мы предпочитаем `ttk` там, где существует тематизированный аналог — но сначала посмотрим, как выглядят виджеты вместе.

Витрина виджетов Tkinter

Прежде чем разбирать детали каждого виджета отдельно (разделы 16.13–16.22) — вот как они выглядят вместе, в одном настоящем окне:

Витрина: Label, Entry, Button, Checkbutton, Radiobutton, Combobox, Spinbox, Scale, Progressbar, Notebook в одном окне

Реальное окно с представительным набором виджетов главы. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

Дальше в этой главе каждый из этих виджетов получит отдельный раздел с собственными состояниями и примерами кода.

tk и ttk — два набора виджетов

	tk	ttk
Что это	классические виджеты Tkinter	тематизированный (themed) набор поверх Tk
Импорт	<code>import tkinter as tk</code>	<code>from tkinter import ttk</code>
Внешний вид	фиксированный классический стиль	может адаптироваться под тему/платформу
Стилизация	<code>fg= / bg=</code>	через <code>ttk.Style()</code>

Одна и та же форма (Label, Entry, Button) собрана дважды: классическим tk и тематизированным ttk

На этой теме курса разница едва заметна — так и должно быть: ttk не гарантирует «более современный» вид на любой платформе/теме. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

```
tk_vs_ttk.py
```

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()

ttk.Label(root, text="Современный ttk.Label").pack()
ttk.Button(root, text="ttk.Button").pack()
ttk.Entry(root).pack()
```

Для нового кода — ttk, где есть аналог

Начиная с этого раздела в новых примерах мы предпочитаем `ttk.Label`, `ttk.Button`, `ttk.Entry` и другие тематизированные виджеты там, где существует тематизированный аналог. Но **ttk не заменяет** весь Tk целиком — виджеты вроде `tk.Text`, `tk.Canvas` и `tk.Menu` остаются классическими, потому что у них нет тематизированного аналога.

ttk не гарантирует «более современный» вид

Тема оформления ttk зависит от операционной системы и установленных тем Tk. На некоторых платформах/темах разница между tk и ttk почти незаметна визуально (как на скриншоте выше) — говорите «тематизированный», а не «современный» или «красивее».

Не используйте fg/bg на ttk-виджетах как на классических

Классические tk-виджеты понимают `fg=` / `bg=` напрямую. Виджеты ttk стилизуются иначе — через объект `ttk.Style()` (следующий раздел). Не переносите привычки классического Tk на ttk буквально.

from tkinter import * — не в продакшн-стиле

Импорт: звёздочка → явные имена

Не рекомендуется

```
from tkinter import *
```

```
root = Tk()
Button(root,
text="Кнопка") #
откуда взялся Button —
неочевидно
```

Рекомендуется

```
import tkinter as tk
from tkinter import ttk
```

```
root = tk.Tk()
ttk.Button(root,
text="Кнопка") #
источник имени виден
сразу
```

Что использовать сегодня: `from tkinter import *` работает, но затрудняет чтение кода: непонятно, какому модулю принадлежит каждое имя, и легко случайно перекрыть встроенные имена Python. В примерах курса, начиная с этой главы, — только явные импорты.

ttk.Style() — стилизация тематизированных виджетов

Обычная кнопка рядом с кнопкой, стилизованной через `Accent.TButton`

Слева — стиль по умолчанию, справа — собственный именованный стиль. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

```
ttk_style.py
```

```
style = ttk.Style()
print(style.theme_use())    # активная тема, например
                             'clam' или 'default'

style.configure(
    "Accent.TButton",
    font=("TkDefaultFont", 11, "bold"),
    padding=8,
)

button = ttk.Button(
    root,
    text="Сохранить",
    style="Accent.TButton",
)
```

ttk.Style() — не CSS

`style.configure(...)` принимает только опции, которые поддерживает конкретный виджет и активная тема Tk — это не универсальный CSS-подобный язык стилей, где можно менять произвольное свойство произвольного элемента. Именованный стиль ("Accent.TButton") применяется через параметр `style=` у самого виджета.



**Cartesian
Purple**
#5B24F9



Cartesian Pink
#DB2777



Success Green
#059669



Warning Amber
#B45309

Цвет — сначала увиденный, потом числовой

Прежде чем писать `#5B24F9` в коде, взгляните на реальный цветовой образец выше — так проще запомнить и различать похожие оттенки, чем по одним шестнадцатеричным кодам.

Canvas — рисование произвольных фигур

Canvas с линией, прямоугольником, овалом и подписью координат

Результат кода ниже. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

canvas_basics.py

```
canvas = tk.Canvas(root, width=260, height=170,
background="white")
canvas.pack(padx=10, pady=10)

canvas.create_line(10, 10, 240, 10, fill="#5B24F9",
width=2)
rect_id = canvas.create_rectangle(10, 30, 90, 90,
outline="#DB2777", width=2)
canvas.create_oval(130, 30, 210, 90,
outline="#059669", width=2)
canvas.create_text(130, 130, text="(0,0) — верхний
левый угол")
```

Ось Y растёт вниз, а не вверх

Начало координат (0, 0) — верхний левый угол Canvas. x растёт вправо, а y растёт **вниз** — это противоположно привычной математической системе координат, где y растёт вверх. Так устроена экранная система координат в большинстве графических библиотек, не только Tkinter.

create_* возвращает идентификатор элемента

rect_id = canvas.create_rectangle(...) — Canvas хранит нарисованные элементы и возвращает идентификатор, по которому потом можно изменить или удалить именно этот элемент (canvas.itemconfig(rect_id, ...) , canvas.delete(rect_id)) — пригодится для будущих игровых интерфейсов.

Canvas — не второй Turtle

Canvas — обычный виджет Tkinter для рисования произвольных фигур внутри окна приложения, а не отдельный модуль с собственным миром координат, как turtle (главы 6-7). Это краткое знакомство, а не полноценный курс по Canvas.

PhotoImage — изображения в виджетах

Label с маленьким сгенерированным изображением-шахматкой

Изображение сгенерировано прямо в Python, без внешнего файла. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

photoimage.py

```
image = tk.PhotoImage(file="icon.png")
label = ttk.Label(root, image=image)
label.pack()
```

DEBUG LAB 2: ИЗОБРАЖЕНИЕ ИСЧЕЗАЕТ, ХОТЯ КОД «ПРАВИЛЬНЫЙ»

photoimage_propadaet.py

```
def build_ui():
    image = tk.PhotoImage(file="icon.png")
    # локальная переменная функции
    label = ttk.Label(root, image=image)
    label.pack()

build_ui()
```

Окно открывается, но картинка не отображается —
на месте изображения пусто.

Что видно на экране

После завершения `build_ui()` локальное имя `image` исчезает. Если на объект `PhotoImage` больше не остаётся Python-ссылок, объект может быть освобождён — виджет хранит ссылку на него только внутри Tcl, а не в самом Python. Поэтому приложение должно хранить ссылку на `PhotoImage` столько времени, сколько изображение должно отображаться.

ИСПРАВЛЕННЫЙ КОД

photoimage_fixed.py

```
class App:
    def __init__(self, root):
        self.logo_image =
tk.PhotoImage(file="icon.png")    # хранится в
self -
        self.logo_label = ttk.Label(root,
image=self.logo_image)    # живёт, пока жив
self
        self.logo_label.pack()
```

Практика: собираем форму из tk и ttk

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-12/index.html>)

Frame и LabelFrame: организуем интерфейс

Прежде чем размещать десятки виджетов, разбейте окно на понятные регионы.

Frame — структура прежде компоновки

`ttk.Frame` — контейнер без собственного видимого содержимого, который группирует связанные виджеты в осмысленные регионы:

```

● root
  ● toolbar_frame
  ● content_frame
  ● status_frame

```

Разбиение окна на именованные регионы упрощает и код, и мышление о layout.

frame.py

```

import tkinter as tk
from tkinter import ttk

root = tk.Tk()

toolbar_frame = ttk.Frame(root)
content_frame = ttk.Frame(root)
status_frame = ttk.Frame(root)

toolbar_frame.pack(side="top", fill="x")
content_frame.pack(side="top", fill="both",
expand=True)
status_frame.pack(side="bottom", fill="x")

```

Внутри каждого фрейма можно независимо использовать `pack()` или `grid()` — правило «не смешивать» действует в пределах одного родителя, а не всего окна (подробная схема — в разделе 16.15).

LabelFrame — визуально подписанная группа

Верхняя панель с кнопками Открыть/Сохранить, ниже — подписанная рамка «Настройки» с двумя флажками

Toolbar через Frame (без рамки) и LabelFrame с видимой подписью «Настройки». Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

labelframe.py

```
nastrojki = ttk.LabelFrame(root, text="Настройки")
nastrojki.pack(padx=10, pady=10, fill="x")

ttk.Checkbutton(nastrojki, text="Тёмная
тема").pack(anchor="w")
```

LabelFrame — не для каждой мелочи

Используйте LabelFrame для действительно связанной группы настроек, а не для каждого отдельного виджета — иначе интерфейс превращается в лестницу рамок.

Практика: организуем интерфейс через Frame

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-13/index.html>)

pack подробно: fill, expand и вложенные фреймы

pack — это не просто «сверху вниз»: у него есть управление свободным пространством.

pack: как распределяется пространство

`side` определяет, с какой стороны укладывается очередной виджет, `fill` — растягивать ли его на всю доступную ширину/высоту, а `expand` — забирать ли ему лишнее свободное место:

Параметр	Значение	Эффект
fill	"x"	растянуть виджет по ширине выделенной ему полосы
fill	"y"	растянуть виджет по высоте выделенной ему полосы
fill	"both"	растянуть и по ширине, и по высоте
expand	True	отдать виджету дополнительное свободное место контейнера

pack_fill_expand.py

```
ttk.Label(root, text="Верх",
background="#E7DEFF").pack(side="top", fill="x")
ttk.Label(root, text="Центр",
background="#B9A0FC").pack(side="top", fill="both",
expand=True)
ttk.Label(root, text="Низ",
background="#E7DEFF").pack(side="bottom", fill="x")
```

Внешний отступ и внутренний — не одно и то же
`padx / pady` у `.pack(...)` — это отступ **снаружи** виджета, между ним и соседями. Отступ **внутри** самого виджета (между его рамкой и содержимым) настраивается отдельно — например, через `padding=` у `ttk.Frame`.

Вложенные фреймы + pack

`vlozhennye_frejmy.py`

```
forma_frame = ttk.Frame(root, padding=10)
forma_frame.pack(fill="x")

ttk.Label(forma_frame, text="Имя:").pack(side="left")
ttk.Entry(forma_frame).pack(side="left", fill="x",
expand=True)
```

Метка прижата слева, а поле ввода забирает всё оставшееся горизонтальное пространство — типичная комбинация `side="left"` + `fill="x"` + `expand=True` для формы «подпись — растягивающееся поле».

Практика: pack с fill и expand

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-14/index.html>)

Адаптивный grid: sticky и weight

Форма не обязана оставаться одного размера навсегда — `grid` умеет подстраиваться.

Адаптивный grid: кто получает лишнее место

По умолчанию строки и столбцы `grid()` не растут при увеличении окна. Чтобы форма подстраивалась под размер окна, столбцам/строкам нужно явно назначить **вес**:

`adaptivny_grid.py`

```
root.columnconfigure(1, weight=1)  # растягивается
именно колонка 1 (с полями ввода)
root.rowconfigure(0, weight=1)

ttk.Label(root, text="Имя:").grid(row=0, column=0,
sticky="w")
ttk.Entry(root).grid(row=0, column=1, sticky="ew")
# растягивается по ширине ячейки
```

Две одинаковые по ширине формы: слева Entry узкое, справа растянуто на всю ширину

Одна и та же ширина окна (340px) — разница только в `columnconfigure(weight=1)` и `sticky="ew"`. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

`raspredelenie_vesa.py`

```
def raspredelit_prostranstvo(weights, extra_space):
    total = sum(weights)
    if total == 0:
        return [0] * len(weights)
    return [extra_space * w / total for w in weights]

print(raspredelit_prostranstvo([1, 2], 300))  #
[100.0, 200.0]
```

weight — это пропорция, не пиксели

Вес определяет **долю** дополнительного пространства, а не абсолютный размер. Колонка с весом 2 получит вдвое больше лишнего места, чем колонка с весом 1 — но обе продолжают вмещать свой обычный минимальный контент.

sticky — к какому краю ячейки прилипает виджет

sticky	Эффект
"w" / "e" / "n" / "s"	прилипает к одному краю ячейки (запад/восток/север/юг)
"ew"	растягивается по ширине ячейки
"nsew"	растягивается по всей ячейке — и по ширине, и по высоте

Правило одного родителя

✓ Верно

```
• root (pack)
  |
  • frame (grid внутри)
    |
    • label
    • entry
```

✗ Ошибка

```
• root
  |
  • widget_a (.pack())
  • widget_b (.grid())
```


Смешивать `pack()` и `grid()` нельзя у виджетов с **одним и тем же** родителем — но разные контейнеры (`root` использует `pack()`, а вложенный `Frame` внутри — `grid()`) полностью независимы друг от друга.

place() — точное позиционирование

place.py

```
label.place(x=20, y=10)
label2.place(relx=0.5, rely=0.5, anchor="center")  #
            центр контейнера
```

place() — не первый выбор для обычных форм

`place()` удобен для точных наложений и особых случаев, но не подстраивается под размер окна автоматически, как `grid()` с весами. Для обычных адаптивных форм — `grid()`.

Как выбрать менеджер геометрии

pack vs grid vs place

Простое вертикальное
расположение / регионы

→ **pack()**

Форма/таблица/адаптивные
колонки

→ **grid()**

Точное или особое позиционирование, → **place()** – осознанно наложения

Сложный экран → **Frame + разные менеджеры в разных ро**

Практика: распределение веса в адаптивном grid

Проверяется без tkinter — чистая арифметика распределения пространства

Открыть практику → (<https://www.cartesianschool.org/practice/16-15/index.html>)

Виджеты выбора: Combobox, Listbox, Spinbox, Scale

Каждый из этих виджетов решает свой вариант одной задачи: выбрать значение из ограниченного множества.

Combobox — выбор из списка

Закрытый Combobox с выбранным значением
«Маленький»

Закрытое состояние. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

```
combobox.py
```

```
variant = ttk.Combobox(root, values=["Маленький",  
"Средний", "Большой"], state="readonly")  
variant.current(0)  
variant.pack()
```

Открытый выпадающий список Combobox с тремя вариантами

Открытое состояние (выпадающий список) — реальный скриншот, не имитация. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

state="readonly"

Без `state="readonly"` пользователь сможет напечатать в поле произвольный текст, не входящий в список. Если нужен выбор именно из предложенных вариантов — используйте `"readonly"`.

Listbox — список для выбора одного/нескольких элементов

Listbox с четырьмя элементами, «Хлеб» выделен

Один элемент выделен (виден по подсветке). Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

listbox.py

```

spisok = tk.Listbox(root)
spisok.insert("end", "Молоко")
spisok.insert("end", "Хлеб")
spisok.insert("end", "Яблоки")
spisok.pack()

def pokazat_vybor():
    indeksy = spisok.curselection()
    print([spisok.get(i) for i in indeksy])

```

Spinbox — поле со стрелками для пошагового выбора

Spinbox со значением 3 и стрелками вверх/вниз

Стрелки справа увеличивают/уменьшают значение по шагу. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

spinbox.py

```

kolichество = ttk.Spinbox(root, from_=1, to=10)
kolichество.pack()

```

from_/to — не строгая защита от любого текста

Spinbox — это поле, похожее на Entry, с добавленными стрелками, которые пошагово перебирают значения из заданного диапазона. Параметры from_/to задают диапазон для самих стрелок — они не являются универсальной гарантией, что пользователь не введёт с клавиатуры произвольный текст напрямую в поле. Если нужен строгий выбор только из диапазона — потребуется дополнительно продумать валидацию или readonly-подобное состояние.

Scale — ползунок для значения из диапазона

Scale — горизонтальный ползунок со значением 50 в метке рядом

Ползунок (track) и бегунок (thumb); значение показано в соседней метке. Пример внешнего вида на среде курса.

На вашей ОС шрифты, размеры и тема могут немного отличаться.

scale.py

```
gromkost = ttk.Scale(root, from_=0, to=100,
orient="horizontal")
gromkost.set(50)
gromkost.pack()
```

Scale возвращает float

.get() у Scale возвращает число с плавающей точкой, даже если визуально ползунок стоит на целом делении — округляйте явно (round(...)), если нужно целое число.

Практика: Combobox, Listbox, Spinbox, Scale

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/16-16/index.html\)](https://www.cartesianschool.org/practice/16-16/index.html)

Progressbar и вкладки Notebook

Прогресс должен быть честным, а вкладки — естественным способом уместить много настроек.

Progressbar — прогресс, а не украшение

Четыре Progressbar на 0%, 35%, 70% и 100%

Реальные состояния одного и того же виджета при разных значениях `value`. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

`progressbar.py`

```
progress = ttk.Progressbar(root, mode="determinate",
                           maximum=100, value=0)
progress.pack(fill="x")

def obnovit_progress(procent):
    progress["value"] = procent
```

Режим	Когда использовать
determinate	известно точное количество этапов/процент выполнения
indeterminate	работа идёт, но её объём заранее не известен

Не подделывайте прогресс, которого не знаете

Если приложение реально не может оценить процент выполнения — используйте `mode="indeterminate"` (бегущая полоса), а не произвольные числа в `determinate`. Ложный прогресс хуже честного «неизвестно, сколько осталось».

```
progressbar_indeterminate.py
```

```
progress = ttk.Progressbar(root, mode="indeterminate")
progress.pack(fill="x")

progress.start()    # запускает анимацию бегущей
                    # ... долгая операция ...
                    # останавливает анимацию
progress.stop()
```

indeterminate не анимируется сам по себе

Создание `Progressbar(mode="indeterminate")` не запускает анимацию автоматически — движение бегущей полосы начинается только после явного вызова `.start()` и останавливается по `.stop()`.

Notebook — вкладки

Notebook с тремя вкладками: Общие, Внешний вид, Файлы — активна первая

Три вкладки, активна «Общие» с флажком «Автосохранение» внутри. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

```
notebook.py
```

```
vkladki = ttk.Notebook(root)
vkladki.pack(fill="both", expand=True)

obshaya_vkladka = ttk.Frame(vkladki)
vneshnij_vid_vkladka = ttk.Frame(vkladki)
fajly_vkladka = ttk.Frame(vkladki)

vkladki.add(obshaya_vkladka, text="Общие")
vkladki.add(vneshnij_vid_vkladka, text="Внешний вид")
vkladki.add(fajly_vkladka, text="Файлы")
```

Каждая вкладка — обычный `Frame`, внутри которого можно независимо использовать `pack()` или `grid()` — прекрасно подходит для окна настроек (раздел 16.25). На скриншоте выше видно ровно то, что описывает термин «вкладка» — переключаемая область содержимого внутри одного окна, а не несколько отдельных окон.

Практика: Progressbar и Notebook

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-17/index.html>)

Messagebox и диалоги

Пользователь должен узнавать об успехе, предупреждении или ошибке — не из терминала.

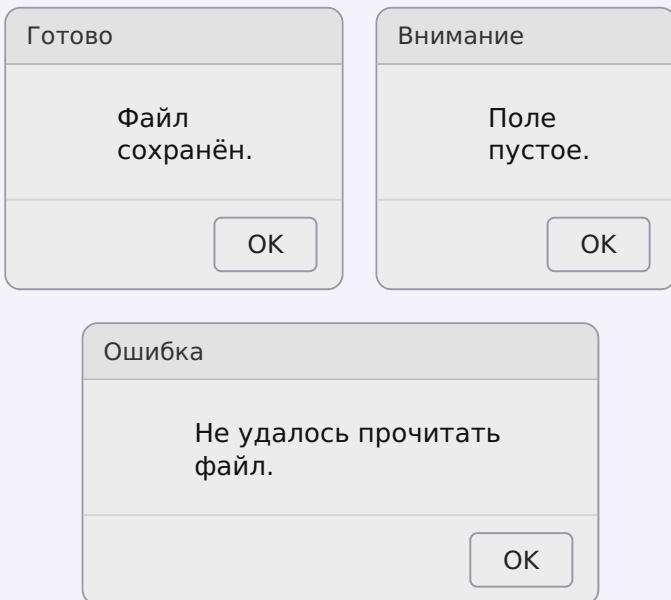
messagebox — стандартные диалоги уведомлений

```
messagebox.py
```

```
from tkinter import messagebox

messagebox.showinfo("Готово", "Файл сохранён.")
messagebox.showwarning("Внимание", "Поле пустое.")
messagebox.showerror("Ошибка", "Не удалось прочитать файл.")
```

СХЕМАТИЧЕСКОЕ ИЗОБРАЖЕНИЕ (НЕ РЕАЛЬНЫЙ СКРИНШОТ)



Три типа уведомлений — разная иконка и смысл, один и тот же принцип вызова.

Внешний вид диалогов зависит от ОС

Настоящий вид этих диалогов зависит от операционной системы, версии Tk и выбранной темы — точную нативную картинку headless-среда курса не может воспроизвести стабильно, поэтому здесь используется схематическое изображение, а не скриншот.

Диалоги с двумя вариантами — читайте возвращаемое значение

```
messagebox_yesno.py
```

```
soglasen = messagebox.askyesno("Подтверждение",  
                                "Удалить эту заметку?")  
if soglasen:  
    udalit_zametku()
```

Не угадывайте результат по названию кнопки

Не предполагайте заранее, что вернёт диалог — читайте фактическое возвращаемое значение (`True` / `False` для `askyesno`) и реагируйте на него, а не на предположение «пользователь наверняка нажмёт да».

Три исхода: Да / Нет / Отмена

Для сценария «выход с несохранёнными изменениями» одного «да/нет» мало — пользователь может передумать закрывать программу вообще:

```
messagebox_yesnocancel.py
```

```
answer = messagebox.askyesnocancel(
    "Несохранённые изменения",
    "Сохранить изменения перед выходом?",
)

if answer is None:
    pass # Отмена — не выходим вообще, пользователь
        передумал
elif answer:
    save_document()
    root.destroy()
else:
    root.destroy() # Нет — выходим, не сохраняя
```

Возвращённое значение	Что нажал пользователь
<code>True</code>	Да
<code>False</code>	Нет
<code>None</code>	Отмена (или закрыл диалог крестиком)

Частая ошибка — перепутать Да/Нет с сохранить/закрыть

«Да» отвечает на вопрос «сохранить?», а не «выйти?». Не пишите `if answer: save() else: root.destroy()` — тогда «Нет» молча закроет окно, даже не спросив, а «Да» сохранит, но не закроет. Полную рабочую версию для реального редактора — с учётом обоих действий и отмены — смотрите в разделе 16.30.

Модальность

Стандартные диалоги `messagebox` — модальные: они блокируют взаимодействие с остальным приложением, пока не будут закрыты. Для собственных модальных окон есть более тонкие инструменты (`transient`, `grab_set`) — раздел 16.20, помечены как продвинутые.

Практика: messagebox и диалоги выбора

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-18/index.html>)

Открываем и сохраняем файлы: `filedialog` и `pathlib`

Диалог выбора файла отдаёт путь — а дальше работает всё, что мы уже знаем из главы 15.

Прямая связь с главой 15: диалог выбора файла отдаёт путь, а дальше работает всё то же `pathlib`.

`filedialog.py`

```
from tkinter import filedialog
from pathlib import Path

filename =
filedialog.askopenfilename(filetypes=[("Текстовые
файлы", "*.txt")])
if not filename:
    pass    # пользователь нажал «Отмена» — ничего не
           делаем
else:
    text = Path(filename).read_text(encoding="utf-8")
```

Не забывайте про отмену

Если пользователь закрыл диалог без выбора файла, `askopenfilename()` вернёт **пустую строку**, а не `None` и не путь. Попытка сразу открыть `Path("")` — частая ошибка. Всегда проверяйте результат перед использованием: `if not filename: return`.

Сохранение файла

`filedialog_save.py`

```
filename =  
filedialog.asksaveasfilename(defaulttextextension=".txt")  
if filename:  
    Path(filename).write_text(text_widget.get("1.0",  
"end-1c"), encoding="utf-8")
```

нажали «Открыть»

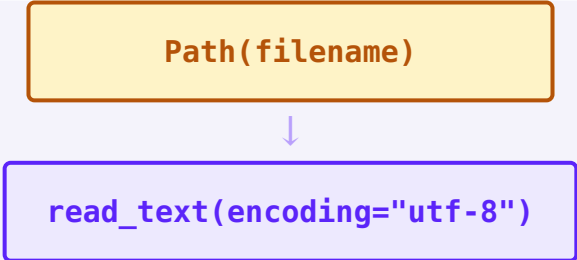
показывается диалог

`filedialog`

отмена?

да — ничего не делаем

`return`



```
Path(filename)
```

```
read_text(encoding="utf-8")
```

Правильный порядок: сначала проверка отмены, потом работа с путём.

Практика: `filedialog` + `pathlib`

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-19/index.html>)

Toplevel: несколько окон

Настоящему приложению почти всегда нужно больше одного окна — но не больше одного `root`.

Toplevel — дополнительное окно

Главное окно приложения и отдельное окно настроек рядом

`Toplevel` — это отдельное настоящее окно, а не панель внутри главного. Пример внешнего вида на среде курса.

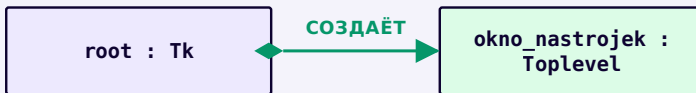
На вашей ОС шрифты, размеры и тема могут немного отличаться.

Для второго окна (настройки, «О программе», диалог) используется `tk.Toplevel`:

toplevel.py

```
def otkryt_nastrojki():
    okno_nastrojek = tk.Toplevel(root)
    okno_nastrojek.title("Настройки")
    ttk.Label(okno_nastrojek, text="Здесь будут
настройки").pack(padx=20, pady=20)

    ttk.Button(root, text="Настройки",
command=otkryt_nastrojki).pack()
```



Один root, дополнительные окна — через Toplevel

Для обычного приложения используется **один** корневой `Tk()`, а дополнительные окна создаются через `Toplevel`. Технически создать несколько независимых `Tk()` возможно — каждый заводит свой отдельный интерпретатор Tcl, — но для обычного единого приложения это обычно не то, что нужно, и не рекомендуемая структура. Практическое правило: одно приложение → один `root` → `Toplevel` для остальных окон.

Чуть

глубже: модальное окно

Некоторые диалоги должны блокировать взаимодействие с родительским окном, пока не будут закрыты:

`modalnoe_okno.py`

```
okno = tk.Toplevel(root)
okno.transient(root)    # окно принадлежит родителю
                        визуально
okno.grab_set()
# забирает весь ввод, пока не закрыто
root.wait_window(okno)  # родитель ждёт закрытия
                        этого окна
```

Не обязательно на старте

Собственные модальные окна — продвинутая техника. Для большинства учебных и небольших приложений обычного `Toplevel` без `grab_set()` вполне достаточно.

Практика: окно настроек через `Toplevel`

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-20/index.html>)

Focus, клавиатура и основы доступности

Не у каждого пользователя мышь — а понятная форма должна работать и без неё.

Фокус ввода

Клавиатурный ввод всегда идёт только одному виджету — тому, у которого сейчас **фокус**:

```
focus.py
```

```
entry.focus_set()      # передать фокус этому полю  
                        программно  
print(root.focus_get()) # какой виджет в фокусе  
                        сейчас
```

Готовим почву для главы 17

Фокус — важное понятие для дальнейшей работы с клавиатурными событиями (`.bind("<Key>", ...)`), которые подробно разберём в главе 17.

Порядок перехода между полями (Tab)

Пользователь должен уметь заполнить форму, используя только клавиатуру — переходя между полями клавишей `Tab` в разумном порядке.

Порядок Tab — не просто «в порядке создания»

Клавиатурный обход следует правилам обхода фокуса Tk по дереву виджетов и их положению, а также тому, участвует ли конкретный виджет в получении фокуса вообще (параметр `takefocus`). Порядок создания влияет на естественную структуру простой формы, но не является полным правилом.

Практический beginner-совет: стройте элементы формы в логичном порядке, обязательно **проверяйте** переход по Tab вручную, избегайте лишних фокусируемых виджетов и используйте `takefocus` осознанно там, где нужно явно включить или исключить виджет из обхода.

Основы доступности

Небольшой, но настоящий список

МЕТКИ

у каждого поля — видимая подпись
не только текст-заглушка (`placeholder`)

СОСТОЯНИЕ

не только цветом — текстом/иконкой
тоже
`disabled` — когда действие недоступно

КЛАВИАТУРА

разумный порядок Tab

не только мышь как единственный путь

СООБЩЕНИЯ

понятный текст ошибки

что пошло не так и как исправить

Tkinter сам не решает доступность за вас

Использование Tkinter не гарантирует доступный интерфейс на всех платформах автоматически.

Перечисленные принципы — то, что зависит от решений в вашем коде и разметке, а не от самого факта использования GUI-библиотеки.

```
proverka_dostupnosti.py
```

```
forma = [
    {"name": "name_entry", "has_label": True,
    "tab_index": 0},
    {"name": "email_entry", "has_label": True,
    "tab_index": 1},
    {"name": "submit_button", "has_label": True,
    "tab_index": 2},
]

def vse_imeyut_metku(widgets):
    return all(w["has_label"] for w in widgets)

def poryadok_posledovatelen(widgets):
    indeksy = [w["tab_index"] for w in widgets]
    return indeksy == sorted(indeksy)
```

Практика: проверка доступности формы

Проверяется без tkinter — на модели формы как списка данных

Открыть практику → (<https://www.cartesianschool.org/practice/16-21/index.html>)

after(): таймеры без блокировки

Нужно подождать — не значит нужно остановить всё приложение.

after() — отложенный вызов без блокировки

after_prosto.py

```
after_id = root.after(1000, lambda: print("Прошла  
секунда"))  
# программа продолжает обрабатывать другие события,  
ожидаая эту секунду
```

событийный цикл



запланировать через after()



другие события продолжают
обрабатываться

callback вызывается



время подошло

after() ставит callback в очередь на будущее — и не блокирует всё остальное в ожидании.

«Через секунду» — не хронометр с точностью до миллисекунды

`after(1000, callback)` означает «сделай callback готовым к выполнению примерно через 1000 мс, когда событийный цикл сможет его обработать» — не гарантию исполнения ровно в указанную миллисекунду. Если в этот момент цикл занят другим событием, вызов произойдёт чуть позже.

Повторяющийся таймер (self-rescheduling)

tik.py

```
after_id = None

def tik():
    global after_id
    label.config(text=f"Секунд: {schet.get()}")
    schet.set(schet.get() + 1)
    after_id = root.after(1000, tik)    # сам себя
    планирует заново — вот и повторение
```

Не только for/while — это тоже повторение

Из главы 10 мы знаем циклы `for/while`. Повторяющийся вызов через self-scheduling `after()` — ещё один способ организовать повторение, устроенный на событиях, а не на последовательном блоке кода.

Каждый новый after() должен обновлять сохранённый id

У `tik()` — новый вызов `after()` на каждом тике, и, значит, новый идентификатор. Если не переприсваивать `after_id` заново при каждом вызове (как выше), сохранённая переменная быстро устареет — будет указывать на уже сработавший вызов, а не на актуально ожидающий, и попытка остановить таймер по этому `id` ничего не остановит.

Остановка таймера: `after_cancel`

`after_cancel.py`

```
def stop():
    global after_id
    if after_id is not None:
        root.after_cancel(after_id)
        after_id = None
```

Не потеряйте идентификатор

Чтобы остановить запланированный вызов, нужно сохранить **актуальный** идентификатор, который вернул последний вызов `after()`. Без него отменить именно этот запланированный вызов не получится. Полную трёхсостоятельную модель (остановлен/идёт/завершён) с корректным `after_id` смотрите в мини-проекте «Таймер» (раздел 16.28).

**DEBUG LAB 3: ПОВТОРНЫЙ ЗАПУСК
СОЗДАЁТ ДУБЛИРУЮЩИЕСЯ ТАЙМЕРЫ**

duplicate_after.py

```
def start():
    tik()
    # каждый клик по «Старт» запускает ЕЩЁ один
    цикл tik()

ttk.Button(root, text="Старт",
            command=start).pack()
```

После нескольких нажатий «Старт» счётчик начинает п
работает уже несколько параллельных цепочек after()

Что видно на экране

Каждый клик запускает новую независимую цепочку самопланирующихся вызовов `tik()`, и они не заменяют друг друга, а работают одновременно. Нужно блокировать повторный старт через общий флаг `running` — и этот же флаг должна проверять сама `tik()`, иначе цепочка не остановится даже после `stop()`.

ИСПРАВЛЕННЫЙ КОД

duplicate_after_fixed.py

```
running = False
after_id = None

def tik():
    global after_id, running
    if not running:
        return # stop() уже сбросил флаг —
        # новую цепочку не планируем
    label.config(text=f"Секунд:
{schet.get()}")
```



```

schet.set(schet.get() + 1)
after_id = root.after(1000, tik)

def start():
    global running
    if running:
        return # уже запущено — повторный
        клик игнорируем
    running = True
    tik()

def stop():
    global running, after_id
    running = False
    if after_id is not None:
        root.after_cancel(after_id)
        after_id = None

```

Никогда `time.sleep()` внутри `callback`

`time.sleep(...)` в обработчике останавливает событийный цикл целиком на всё время сна — интерфейс перестаёт отвечать. Подробный разбор — в разделе 16.32.

Практика: таймер обратного отсчёта через `after()`

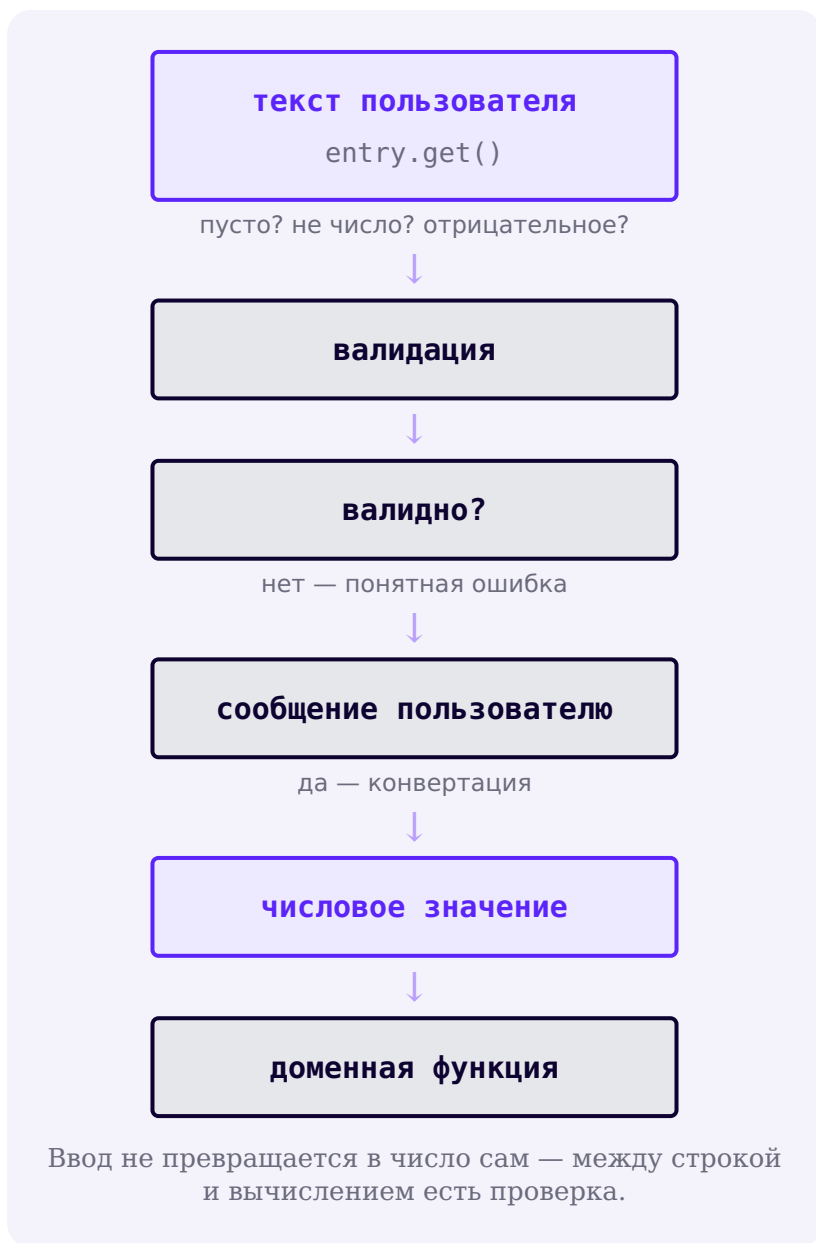
Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-22/index.html>)

Валидация ввода и обратная связь

Сырой текст пользователя не становится числом сам — между ними должна быть проверка.

Валидация — прежде чем считать



Превью try/except

Это минимальное практическое превью try/except. Пока используем его как готовый шаблон для проверки пользовательского ввода. Подробно исключения будут изучены в главе 21.

parse_number.py

```
def parse_number(text):
    text = text.strip()
    if not text:
        return False, None, "Поле не должно быть пустым"
    try:
        return True, float(text), ""
    except ValueError:
        return False, None, "Введите число"

print(parse_number(""))          # (False, None, ...)
print(parse_number("abc"))       # (False, None, ...)
print(parse_number("-5"))        # (True, -5.0, '') —
    отрицательное число ЧИСЛОМ является!
print(parse_number("100"))       # (True, 100.0, '')
```

Не всякое число обязано быть положительным

`parse_number` проверяет только «это вообще число?» — и намеренно принимает отрицательные значения и ноль: они тоже числа. Например, температура в градусах Цельсия может быть отрицательной (раздел 16.27) — требование «положительное» специфично для конкретной задачи (суммы денег, количества людей), а не свойство чисел вообще.

Специализированная валидация поверх `parse_number`

`validate_positive.py`

```
def validate_positive_amount(text):
    ok, value, message = parse_number(text)
    if not ok:
        return False, message
    if value <= 0:
        return False, "Число должно быть больше нуля"
    return True, ""

def validate_positive_int(text):
    ok, value, message = parse_number(text)
    if not ok:
        return False, message
    if value != int(value):
        return False, "Введите целое число"
    if int(value) < 1:
        return False, "Число должно быть не меньше 1"
    return True, ""
```

Разные задачи — разные функции

Сумма счёта должна быть положительным числом — `validate_positive_amount`. Количество человек должно быть положительным **целым** — `validate_positive_int` дополнительно отвергает 2.5. Обе функции переиспользуют `parse_number`, а не дублируют разбор текста.

Как показать ошибку пользователю

```
status_label.py
```

```
ok, message =  
validate_positive_amount(schet_entry.get())  
if not ok:  
    status_label.config(text=message,  
                        foreground="#DB2777")  
    return
```

Ошибка — на экране, а не только в терминале

GUI-пользователь обычно не смотрит в терминал. Показывайте ошибку через виджет-статус или `messagebox.showerror(...)` (раздел 16.18) — печать `print(...)` в терминал не считается обратной связью пользователю в оконном приложении.

Чуть

глубже: `validatecommand`

У Tkinter есть встроенный механизм валидации прямо на уровне виджета (`validate=`, `validatecommand=`, `register(...)`) — синтаксис у него не самый интуитивный, поэтому для большинства форм в этом курсе достаточно ручной проверки перед вычислением, как выше.

Практика: parse_number и специализированная валидация

Проверяется без tkinter — на чистых функциях

Открыть практику → (<https://www.cartesianschool.org/practice/16-23/index.html>)

Архитектура приложения: логика отдельно от виджетов

Лучшая GUI-логика — та, которую можно протестировать, даже не открывая окно.

Чистая логика отдельно от виджетов

Прямая связь с главой 13: доменную функцию можно написать и протестировать **независимо** от интерфейса — просто вход и выход, без единого виджета:

```
chistaya_logika.py
```

```
def calculate_tip(amount, percent, people):  
    total_tip = amount * percent / 100  
    return total_tip / people  
  
print(calculate_tip(1000, 15, 2))    # 75.0
```

callback (обработчик кнопки)



прочитать значения виджетов



проверить (раздел 16.23)

чистая функция — без единого виджета внутри



```
calculate_tip(...)
```



показать результат в виджете

Callback — тонкий слой-посредник; вся логика вычисления живёт вне интерфейса.

Callback не должен разрастаться

Если обработчик кнопки занимает сотню строк с вложенными вычислениями — вероятно, часть этой логики стоит вынести в отдельную функцию, которую можно понять и проверить без запуска окна.

**Чуть
глубже: ловушка позднего связывания
в lambda**

**DEBUG LAB 4: ВСЕ КНОПКИ В ЦИКЛЕ
«ЗАПОМИНАЮТ» ОДНО И ТО ЖЕ ЗНАЧЕНИЕ**


```
pozdnnee_svyazyvanie.py
```

```
for i in range(3):
    ttk.Button(root, text=str(i),
               command=lambda: print(i)).pack()
```

Все три кнопки выведут одно и то же число — последнее
а не 0, 1, 2 соответственно.

Что видно на экране

`lambda: print(i)` обращается к переменной `i` из окружающей области видимости **в момент клика**, а не в момент создания кнопки (глава 13, замыкания). К моменту клика цикл уже закончился, и `i` равно своему последнему значению — 2 — для всех трёх кнопок сразу.

ИСПРАВЛЕННЫЙ КОД

```
pozdnnee_svyazyvanie_fixed.py
```

```
for i in range(3):
    ttk.Button(root, text=str(i),
               command=lambda i=i: print(i)).pack()
# i=i создаёт значение по умолчанию,
# зафиксированное ИМЕННО в момент создания
# lambda
```

```
proverka_lambda_fix.py
```

```
callbacks = []
for i in range(3):
    callbacks.append(lambda i=i: i)

values = [cb() for cb in callbacks]
print(values)    # [0, 1, 2]
```

Практика: чистая логика и ловушка lambda

Проверяется без tkinter — на функциях и замыканиях

Открыть практику → (<https://www.cartesianschool.org/practice/16-24/index.html>)

Класс приложения и персистентные настройки

Приложение — тоже объект: со своим состоянием, виджетами-атрибутами и методами-обработчиками.

Класс приложения — композиция, а не наследование Tk

Прямая связь с главой 14. Для приложений сложнее пары виджетов удобно собрать всё в один класс:

klass_prilozheniya.py

```
class TipCalculatorApp:
    def __init__(self, root):
        self.root = root
        self.amount_var = tk.StringVar()
        self.build_ui()

    def build_ui(self):
        ttk.Entry(self.root,
textvariable=self.amount_var).pack()
        ttk.Button(self.root, text="Считать",
command=self.on_calculate).pack()

    def on_calculate(self):
        ...

root = tk.Tk()
app = TipCalculatorApp(root)
root.mainloop()
```

app : TipCalculatorApp

```
root = Tk
amount_var = StringVar
result_var = StringVar
```

Объектный граф приложения: App HAS-A root, а не App IS-A Tk.

App содержит root, а не наследует от Tk

`class App: c self.root = root` — основная модель этого курса: зависимость от Tk видна явно в конструкторе. Наследование `class App(tk.Tk):` тоже возможно и встречается на практике — но композиция здесь проще для рассуждений, не «неправильный» вариант против «правильного».

Не каждый виджет — атрибут self

Сохраняйте в `self.виджет` только то, к чему понадобится обратиться позже из другого метода (поле ввода, метка результата). Временные виджеты, созданные и размещённые в одном месте, можно оставить локальными переменными `build_ui()`.

Персистентные настройки — мост к главе 15

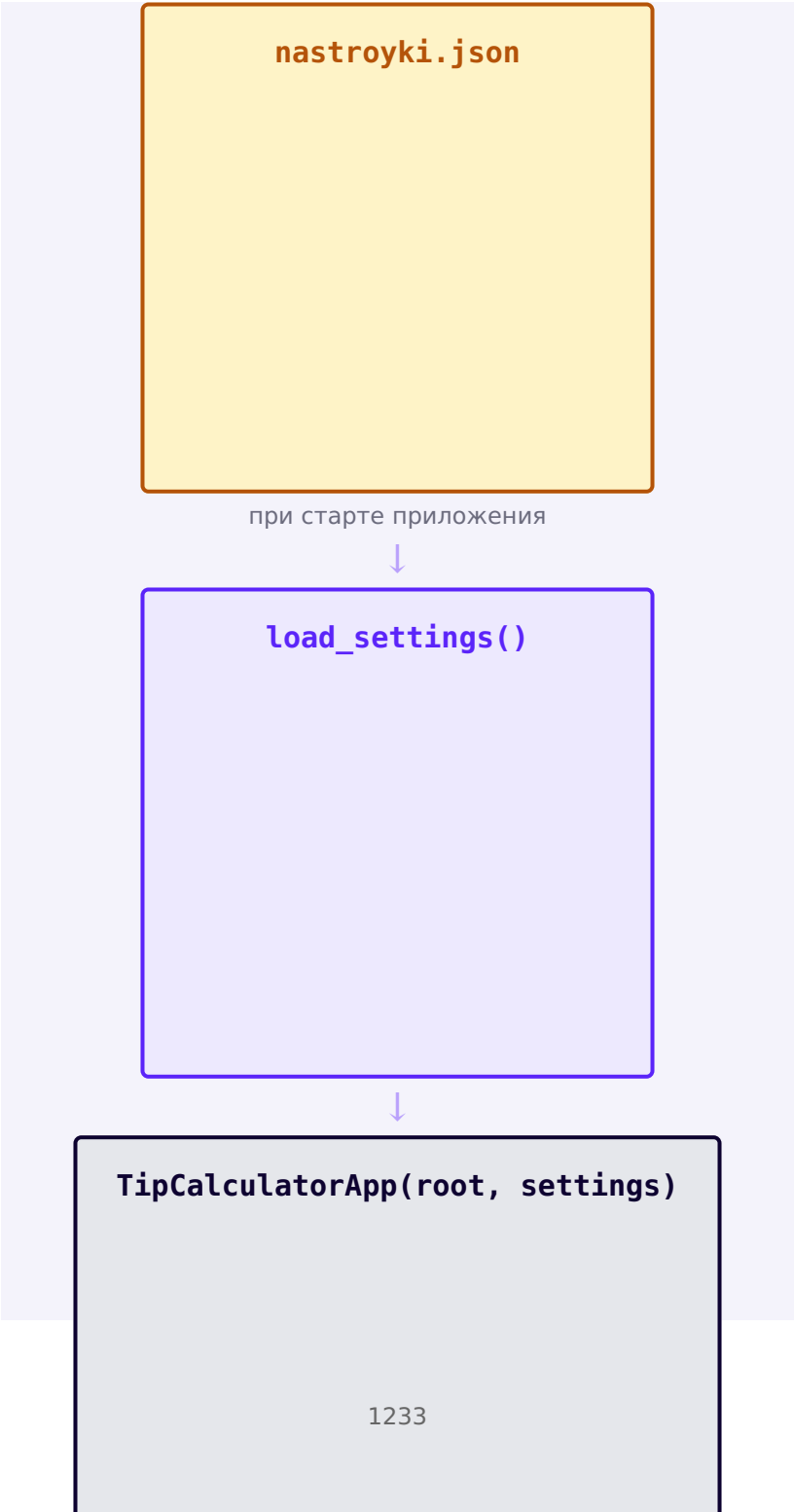
nastroyki_gui.py

```
import json
from pathlib import Path

DEFAULT_SETTINGS = {"theme": "light", "window_width":
900}

def load_settings(path):
    if not path.exists():
        return dict(DEFAULT_SETTINGS)
    with path.open("r", encoding="utf-8") as f:
        return json.load(f)

def save_settings(path, settings):
    with path.open("w", encoding="utf-8") as f:
        json.dump(settings, f, ensure_ascii=False,
indent=2)
```



↓

пользователь меняет настройку

save_settings() перед закрытием

↓

nastroyki.json

Ни одна строчка про JSON и файлы не нова — всё уже было в главе 15.

Ни единой новой идеи о файлах

load_settings() / save_settings() не используют ничего, кроме того, что уже изучено в главе 15 — json, pathlib, безопасные значения по умолчанию. GUI просто вызывает эти функции при старте и перед закрытием.

Практика: load_settings/save_settings для GUI

Проверяется без tkinter — переносим функции persistence главы 15

Открыть практику → (<https://www.cartesianschool.org/practice/16-25/index.html>)

Мини-проект: счётчик кликов

Маленький проект — но уже настоящее взаимодействие: клик → callback → изменение состояния → обновление экрана.

Самый маленький настоящий проект главы: счётчик кликов. Хорошая возможность увидеть оба способа хранить число, связанное с виджетом.

Окно счётчика кликов со значением 3 и кнопкой +1

Результат выполнения кода ниже, после трёх кликов. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

```
schetchik_intvar.py
```

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title("Счётчик кликов")

schet = tk.IntVar(value=0)
ttk.Label(root, textvariable=schet, font=("Arial",
24)).pack(pady=20)

def na_klik():
    schet.set(schet.get() + 1)

ttk.Button(root, text="+1",
command=na_klik).pack(pady=10)

root.mainloop()
```

Второй способ — без Tk-переменной

Тот же счётчик можно собрать с обычным `int` и явным `label.config(text=...)` после каждого изменения — работает одинаково хорошо; `IntVar` удобен именно когда значение нужно синхронизировать сразу с несколькими виджетами (раздел 16.4).

**DEBUG LAB 5: ЗАБЫЛИ MAINLOOP() —
ОКНО НЕ РАБОТАЕТ КАК НАДО**

bez_mainloop.py

```
root = tk.Tk()
ttk.Button(root, text="+1").pack()
# забыли root.mainloop()!
```

Скрипт завершается почти сразу – окно либо не появ
либо мгновенно закрывается, не дожидаясь никаких де

Что видно на экране

Без `root.mainloop()` программа не переходит в цикл обработки событий (раздел 16.10) — она просто линейно доходит до конца скрипта и завершается, как обычная терминальная программа. Виджеты созданы и даже размещены, но никто не ждёт кликов по ним.

ИСПРАВЛЕННЫЙ КОД**s_mainloop.py**

```
root = tk.Tk()
ttk.Button(root, text="+1").pack()
root.mainloop() # без этого вызова событий
не будет вообще
```

Практика: счётчик кликов

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-26/index.html>)

Мини-проект: конвертер температур

Простая форма — но по образцовой архитектуре: чистая функция преобразования отдельно от интерфейса.

Форма «ввод → проверка → чистая функция → результат» на практике: конвертер температур.

konverter_temperatur.py

```
def celsius_v_farengejty(celsius):
    return celsius * 9 / 5 + 32

def farengejty_v_celsius(farengejty):
    return (farengejty - 32) * 5 / 9

print(round(celsius_v_farengejty(100), 1))    # 212.0
print(round(celsius_v_farengejty(-40), 1))   # -40.0
print(round(celsius_v_farengejty(0), 1))     # 32.0
```

Температура не обязана быть положительной

0°C, −5°C, −40°C — всё это совершенно нормальные температуры. Конвертер температур не должен переиспользовать `validate_positive_amount` (раздел 16.23) — она отвергла бы отрицательный и нулевой ввод как «ошибку». Здесь нужна просто общая проверка «это число?» — `parse_number`.

konverter_gui.py

```
import tkinter as tk
from tkinter import ttk

def celsius_v_farengejty(celsius):
    return celsius * 9 / 5 + 32

def parse_number(text):
    text = text.strip()
    if not text:
```

```

        return False, None, "Поле не должно быть
пустым"
    try:
        return True, float(text), ""
    except ValueError:
        return False, None, "Введите число"

root = tk.Tk()
celsius_var = tk.StringVar()
result_var = tk.StringVar()

def on_convert():
    ok, value, message =
parse_number(celsius_var.get())
    if not ok:
        result_var.set(message)
    return
    farengejty = celsius_v_farengejty(value)
    result_var.set(f"{farengejty:.1f} °F")

ttk.Entry(root, textvariable=celsius_var).pack()
ttk.Button(root, text="В Фаренгейты",
command=on_convert).pack()
ttk.Label(root, textvariable=result_var).pack()

```

Конвертер температур: введено -40, результат -40.0 °F

−40°C = −40°F — единственная точка, где шкалы Цельсия и Фаренгейта совпадают. Результат выполнения кода выше. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

parse_number уже знаком

Функция `parse_number` — та же самая, что мы написали в разделе 16.23. Один раз написанная и проверенная функция разбора числа пригождается в нескольких проектах — здесь без дополнительного требования «положительное».

★★ Самостоятельная задача

Оба направления

Добавьте второе поле и кнопку «В Цельсии», использующую `farengfejty_v_celsius` — по образцу существующей кнопки.

Практика: конвертер температур

Проверяется без `tkinter` — на чистых функциях преобразования, включая отрицательные значения

Открыть практику → (<https://www.cartesianschool.org/practice/16-27/index.html>)

Мини-проект: таймер обратного отсчёта

`after()` в деле: обратный отсчёт без единой блокирующей операции.

Таймер обратного отсчёта — образцовая демонстрация `after()` и состояния приложения.

`format_time.py`

```
def format_time(sekundy):
    minuty, ostatok = divmod(sekundy, 60)
    return f"{minuty:02d}:{ostatok:02d}"

print(format_time(65))    # 01:05
print(format_time(600))  # 10:00
```

Окно таймера: 00:42 и кнопки Старт, Стоп, Сброс

Стартовое отображение — до первого тика. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

Три состояния таймера



tajmer_gui.py

```
class TimerApp:
    def __init__(self, root, start_seconds=60):
        self.root = root
```

```

        self.start_seconds = start_seconds
        self.remaining = start_seconds
        self.running = False
        self.after_id = None
        self.label_var =
tk.StringVar(value=format_time(self.remaining))
        ttk.Label(root,
textvariable=self.label_var).pack()
        ttk.Button(root, text="Старт",
command=self.start).pack()
        ttk.Button(root, text="Стоп",
command=self.stop).pack()
        ttk.Button(root, text="Сброс",
command=self.reset).pack()

    def start(self):
        if self.running or self.remaining <= 0:
            return # уже идёт, или уже FINISHED –
                сначала Сброс
        self.running = True
        self._schedule_tick()

    def _schedule_tick(self):
        self.after_id = self.root.after(1000,
self.tick)

    def tick(self):
        self.remaining -= 1

self.label_var.set(format_time(self.remaining))
        if self.remaining <= 0:
            self.running = False
            self.after_id = None # FINISHED –
                планировать больше нечего
            return
        self._schedule_tick()

    def stop(self):
        self.running = False
        if self.after_id is not None:
            self.root.after_cancel(self.after_id)
            self.after_id = None

```

```
def reset(self):
    self.stop()
    self.remaining = self.start_seconds

self.label_var.set(format_time(self.remaining))
```

after(1000, ...) — «примерно через секунду», не хронометр

`after(1000, callback)` означает «сделай callback готовым к выполнению примерно через 1000 мс, когда событийный цикл сможет его обработать» — не гарантию исполнения ровно в указанную миллисекунду. Если цикл в этот момент занят другим событием, вызов произойдёт чуть позже.

DEBUG LAB 6: ТАЙМЕР ПРОДОЛЖАЕТ ТИКАТЬ ПОСЛЕ «СТОП»

tajmer_bez_proverki.py

```
def tick(self):
    self.remaining -= 1

self.label_var.set(format_time(self.remaining))
self.after_id = self.root.after(1000,
self.tick)  # планирует себя ВСЕГДА

def stop(self):
    self.running = False  # состояние
поменяли, но tick() его не проверяет
```

После нажатия «Стоп» таймер продолжает уменьшать ге
флаг `running` изменился, но ничто не проверяет его в

Что видно на экране

`stop()` лишь меняет флаг `running`, но сам метод `tick()` не проверяет его перед тем, как запланировать себя заново — цепочка `after()` продолжает жить независимо от флага.

ИСПРАВЛЕННЫЙ КОД

tajmer_s_proverkoj.py

```
def tick(self):
    if not self.running:  # проверяем
        состояние ПЕРЕД тем как продолжить
        return
    self.remaining -= 1

    self.label_var.set(format_time(self.remaining))
    self.after_id = self.root.after(1000,
    self.tick)
```

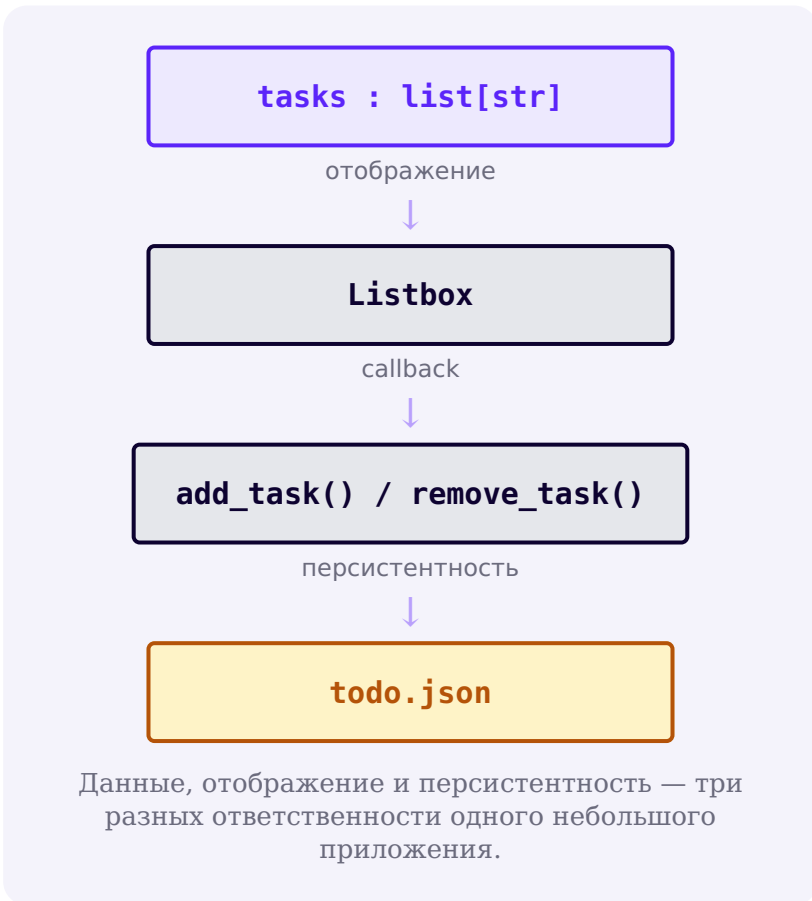
Практика: таймер обратного отсчёта

Открыть практику → (<https://www.cartesianschool.org/practice/16-28/index.html>)

Мини-проект: список задач

Простой список строк — и уже целое маленькое приложение с сохранением между запусками.

Список задач — интеграция главы 11 (списки), главы 15 (JSON) и Tkinter (Listbox).



todo_logika.py

```

import json
from pathlib import Path

def load_tasks(path):
    if not path.exists():
        return []
    with path.open("r", encoding="utf-8") as f:
        return json.load(f)

def save_tasks(path, tasks):

```



```

with path.open("w", encoding="utf-8") as f:
    json.dump(tasks, f, ensure_ascii=False,
indent=2)

def add_task(tasks, text):
    if not text.strip():
        return tasks
    return tasks + [text.strip()]

def remove_task(tasks, index):
    return tasks[:index] + tasks[index + 1:]

```

Список задач: три задачи, вторая выделена, поле ввода и кнопки + / –

Результат выполнения кода ниже. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

Отдельные функции-callback без класса — источник ошибки

Callback вроде `on_add()`, объявленный сам по себе (не внутри другой функции), не может использовать `nonlocal tasks` — `nonlocal` требует переменную из объемлющей функции, а не из глобальной области. Здесь уместнее класс приложения (глава 14) — `self.tasks` решает эту проблему естественно.

`todo_gui.py`

```

class TodoApp:
    def __init__(self, root, path):
        self.root = root
        self.path = path
        self.tasks = load_tasks(path)
        self.new_task_var = tk.StringVar()
        self.listbox = tk.Listbox(root, height=6)
        self.listbox.pack()
        entry = ttk.Entry(root,
textvariable=self.new_task_var)

```

```

        entry.pack(side="left", fill="x", expand=True)
        ttk.Button(root, text="+",
command=self.on_add).pack(side="left")
        ttk.Button(root, text="-",
command=self.on_delete).pack(side="left")
        self.refresh_listbox()

    def refresh_listbox(self):
        self.listbox.delete(0, "end")
        for task in self.tasks:
            self.listbox.insert("end", task)

    def on_add(self):
        self.tasks = add_task(self.tasks,
self.new_task_var.get())
        self.new_task_var.set("")
        self.refresh_listbox()
        save_tasks(self.path, self.tasks)

    def on_delete(self):
        selected = self.listbox.curselection()
        if not selected:
            return
        self.tasks = remove_task(self.tasks,
selected[0])
        self.refresh_listbox()
        save_tasks(self.path, self.tasks)

```

Логика проверяема без единого виджета

`add_task`, `remove_task`, `load_tasks`, `save_tasks` — обычные функции со списками и словарями. Именно их и тестирует практика этого раздела — ровно то разделение, о котором шла речь в разделе 16.24.

Практика: логика списка задач

Проверяется без `tkinter` — на функциях работы со списком задач

Открыть практику → (<https://www.cartesianschool.org/practice/16-29/index.html>)

Мини-проект: редактор заметок

Дневник заметок из главы 15 наконец получает настоящее окно и меню.

Прямое продолжение дневника заметок из главы 15 (15.4) — теперь с настоящим окном, многострочным полем и меню «Файл».

Окно редактора заметок с текстом заметки

Результат выполнения кода ниже. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

redaktor_zametok.py

```
class NotesEditorApp:
    def __init__(self, root):
        self.root = root
        self.current_path = None
        self.dirty = False
        self.text_widget = ScrolledText(root,
wrap="word")
        self.text_widget.pack(fill="both",
expand=True)
        self.text_widget.bind("<<Modified>>",
self.on_text_modified)
        self.build_menu()
        self.root.protocol("WM_DELETE_WINDOW",
self.on_exit)

    def on_text_modified(self, event):
        self.dirty = True
        self.text_widget.edit_modified(False) #
сбрасываем встроенный флаг Tk

    def confirm_discard_if_dirty(self):
        """True — можно продолжать (New/Open/Exit).
False — пользователь передумал."""
```

```

        if not self.dirty:
            return True
        answer = messagebox.askyesnocancel(
            "Несохранённые изменения",
            "Сохранить изменения перед продолжением?",
        )
        if answer is None:  # Cancel – прервать
            текущее действие
            return False
        if answer:          # Yes – продолжить,
            только если сохранение реально удалось
            return self.save_as()
        return True        # No – продолжить, не
            сохраняя

    def on_exit(self):
        if self.confirm_discard_if_dirty():
            self.root.destroy()

    def save_as(self):
        """True – файл сохранён. False – пользователь
        отменил диалог "Сохранить как"."""
        filename =
        filedialog.asksaveasfilename(defaulttextextension=".txt")
        if not filename:
            return False
        self.current_path = Path(filename)

        self.current_path.write_text(self.text_widget.get("1.0",
            "end-1c"), encoding="utf-8")
        self.dirty = False
        return True

```

Три исхода одного вопроса

`messagebox.askyesnocancel(...)` возвращает `True` (Да — сохранить), `False` (Нет — не сохранять) или `None` (Отмена/закрыт диалог — прервать текущее действие полностью). Обычный `askyesno` умеет только два первых варианта — для «Выход с несохранёнными изменениями» нужен именно третий: возможность передумать закрывать приложение вообще.

Yes — это ещё не сохранено

Если пользователь выбрал «Да», но затем нажал «Отмена» в диалоге `asksaveasfilename`, файл не сохранён — и продолжать действие (закрывать окно, открывать другой файл) нельзя, иначе несохранённый текст будет потерян. Поэтому `confirm_discard_if_dirty()` возвращает результат самого `save_as()`, а не `True` сразу после вызова «Да».

Ничего принципиально нового про файлы

`filedialog` (16.19), `Path.read_text/write_text(encoding="utf-8")` (глава 15), `ScrolledText` (готовая связка `Text` + `Scrollbar`) — все части уже знакомы, здесь они просто собраны в одно приложение.

★★ Самостоятельная задача**Новый и Открыть тоже спрашивают**

Добавьте методы `new_file()` и `open_file()`, каждый из которых сначала вызывает `confirm_discard_if_dirty()` — и продолжает только если он вернул `True`.

Практика: редактор заметок

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/16-30/index.html>)

Tip Calculator Pro: финальная версия

Тот же проект, что и в разделе 16.8 — но теперь со всем, что мы изучили после него.

Вернёмся к калькулятору чаевых (раздел 16.8) и соберём в нём всё, что изучили в этой главе — по стадиям, а не одним огромным финальным куском кода.

Путь от фундамента до Pro

V1-V2 (16.8)

Entry + Button + grid()
чистая формула чаевых

V3

Combobox с типовыми процентами вместо
Entry

V4

поле «количество человек» — делим
чаевые

V5

```
validate_positive_amount/  
validate_positive_int (16.23)
```

V6

меню «Файл» с пунктом «Выход» (16.6)

V7

load_settings/save_settings –
запоминаем последний процент (16.25)

V8

класс TipCalculatorApp – всё внутри
одного объекта (16.25)

Настройки сохраняются относительно текущей рабочей директории

`SETTINGS_PATH = Path("tip_calculator_settings.json")` — относительный путь: файл появится там, откуда запущена программа (глава 15, раздел 15.7 про CWD). Для учебного проекта это осознанный, явный выбор — не забытая неопределённость. В реальном распространяемом приложении для этого обычно выбирают выделенную папку данных приложения; здесь это намеренно оставлено простым.

```
tip_calculator_pro.py
```

```
import json
import tkinter as tk
from tkinter import ttk
from pathlib import Path

SETTINGS_PATH = Path("tip_calculator_settings.json")
DEFAULT_SETTINGS = {"last_percent": "15"}

def load_settings():
    if not SETTINGS_PATH.exists():
        return dict(DEFAULT_SETTINGS)
    with SETTINGS_PATH.open("r", encoding="utf-8") as f:
        return json.load(f)

def save_settings(settings):
    with SETTINGS_PATH.open("w", encoding="utf-8") as f:
        json.dump(settings, f, ensure_ascii=False,
            indent=2)

def parse_number(text):
    text = text.strip()
    if not text:
        return False, None, "Поле не должно быть пустым"
    try:
```



```

        return True, float(text), ""
    except ValueError:
        return False, None, "Введите число"

def validate_positive_amount(text):
    ok, value, message = parse_number(text)
    if not ok:
        return False, message
    if value <= 0:
        return False, "Число должно быть больше нуля"
    return True, ""

def validate_positive_int(text):
    ok, value, message = parse_number(text)
    if not ok:
        return False, message
    if value != int(value):
        return False, "Введите целое число"
    if int(value) < 1:
        return False, "Число должно быть не меньше 1"
    return True, ""

def calculate_tip(amount, percent, people):
    return (amount * percent / 100) / people

class TipCalculatorApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Tip Calculator Pro")
        self.settings = load_settings()
        self.amount_var = tk.StringVar()
        self.percent_var =
tk.StringVar(value=self.settings["last_percent"])
        self.people_var = tk.StringVar(value="1")
        self.result_var = tk.StringVar()
        self.build_ui()
        self.root.protocol("WM_DELETE_WINDOW",
self.on_close)

    def build_ui(self):
        frame = ttk.Frame(self.root, padding=12)
        frame.grid(row=0, column=0, sticky="nsew")
        self.root.columnconfigure(0, weight=1)

```

```

        ttk.Label(frame, text="Сумма
счёта:").grid(row=0, column=0, sticky="w")
        ttk.Entry(frame,
textvariable=self.amount_var).grid(row=0, column=1,
sticky="ew")

        ttk.Label(frame, text="Процент
чаевых:").grid(row=1, column=0, sticky="w")
        ttk.Combobox(frame,
textvariable=self.percent_var, values=["10", "15",
"20"],
state="readonly").grid(row=1, column=1,
sticky="ew")

        ttk.Label(frame, text="Количество
человек:").grid(row=2, column=0, sticky="w")
        ttk.Entry(frame,
textvariable=self.people_var).grid(row=2, column=1,
sticky="ew")

        ttk.Button(frame, text="Посчитать",
command=self.on_calculate).grid(
row=3, column=0, columnspan=2, pady=8)
        ttk.Label(frame,
textvariable=self.result_var).grid(row=4, column=0,
columnspan=2)
        frame.columnconfigure(1, weight=1)

    def on_calculate(self):
        ok, message =
validate_positive_amount(self.amount_var.get())
        if not ok:
            self.result_var.set(message)
            return
        ok_people, message_people =
validate_positive_int(self.people_var.get())
        if not ok_people:
            self.result_var.set("Количество человек:
" + message_people)
            return
        chaevye = calculate_tip(
float(self.amount_var.get()),

```

```

        float(self.percent_var.get()),
        int(self.people_var.get()),
    )
    self.result_var.set(f"Чаевые с человека:
{chaevye:.2f}")
    self.settings["last_percent"] =
self.percent_var.get()

    def on_close(self):
        save_settings(self.settings)
        self.root.destroy()

root = tk.Tk()
app = TipCalculatorApp(root)
root.mainloop()

```

Количество человек — положительное целое, не любое число

2.5 человека не бывает. `validate_positive_int` (раздел 16.23) принимает 1, 2, 5, 10 — и отвергает 0, -1, 2.5, "abc" и пустую строку с понятным сообщением об ошибке.

Окно Tip Calculator Pro с суммой счёта 1000, процентом 15, количеством человек 2 и результатом 75.00

Результат выполнения кода выше — настоящее окно, а не иллюстрация. Пример внешнего вида на среде курса. На вашей ОС шрифты, размеры и тема могут немного отличаться.

```
app : TipCalculatorApp
```

```
root = Tk
settings = dict
amount_var = StringVar
percent_var = StringVar
people_var = StringVar
result_var = StringVar
```

Финальный объектный граф: одно приложение, явные зависимости, ни одной глобальной переменной.

Практика: Tip Calculator Pro

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/16-31/index.html\)](https://www.cartesianschool.org/practice/16-31/index.html)

Отладка интерфейса и качество GUI

Отзывчивость интерфейса — не автоматическое следствие использования Tkinter, а результат конкретных решений в коде.

Событийный цикл нельзя занимать надолго

DEBUG LAB 7: TIME.SLEEP() ВНУТРИ CALLBACK ЗАМОРАЖИВАЕТ ИНТЕРФЕЙС

sleep_v_callback.py

```
import time

def on_click():
    time.sleep(5) # "подождать 5 секунд"
    label.config(text="Готово!")
```

```
# На все 5 секунд окно перестаёт перерисовываться и
# событийный цикл заблокирован внутри time.sleep(), а
```

Что видно на экране

`time.sleep(...)` останавливает **весь поток**, в котором работает событийный цикл — а Tkinter обычно работает в одном потоке. Пока поток спит, цикл не может обработать вообще ничего: ни перерисовку, ни клики, ни закрытие окна.

ИСПРАВЛЕННЫЙ КОД

```
after_vmesto_sleep.py
```

```
def on_click():
    label.config(text="Ждём...")
    root.after(5000, lambda:
label.config(text="Готово!"))
    # событийный цикл продолжает работать все
    эти 5 секунд
```

DEBUG LAB 8: WHILE TRUE ВМЕСТО СОБЫТИЙНОГО ЦИКЛА

```
while_true_gui.py
```

```
while True:
    if button_nazhata():
        obrabotat_klik()
```

```
# Такого кода не должно быть в Tkinter-приложении воо
# у Tk уже есть собственный событийный цикл внутри ма
```

Что видно на экране

Tkinter уже предоставляет полноценный событийный цикл. Собственный `while True` для «опроса» состояния виджетов — не нужная и не работающая архитектура: она либо блокирует `mainloop()`, либо просто избыточна.

ИСПРАВЛЕННЫЙ КОД

```
mainloop_vmesto_while.py
```

```
button.config(command=obrabotat_klik)
root.mainloop()  # сам обрабатывает события,
включая клики по button
```

DEBUG LAB 9: ДОЛГОЕ ВЫЧИСЛЕНИЕ НАПРЯМУЮ В CALLBACK

```
dolgoe_vychislenie.py
```

```
def on_click():
    result = obrabotat_million_zapisej()  #
    секунды вычислений внутри callback
    label.config(text=str(result))
```

```
# Интерфейс "подвисает" на всё время вычисления — ровно
# как и с time.sleep(), просто по другой причине.
```

Что видно на экране

Любая тяжёлая по времени операция внутри callback — не только `time.sleep()` — занимает событийный цикл на всё это время. Для настоящего долгих задач: разбивать работу на части, планируемые через `after()`, или выносить в отдельный поток/процесс с безопасной передачей результата обратно — это уже продвинутая тема, не требующая полной реализации в этом курсе.

ИСПРАВЛЕННЫЙ КОД

```
razbivka_cherez_after.py
```

```
def obrabotat_chast(indeks):
    if indeks >= len(dannye):
        label.config(text="Готово")
        return
    obrabotat_odnu_zapis(dannye[indeks])
    root.after(0, obrabotat_chast, indeks +
1)  # следующая часть — на следующем такте
цикла
```

Обновление виджетов — из потока событийного цикла

Если превью с рабочим потоком вообще встречается в вашем коде — результат нужно безопасно передавать обратно и обновлять виджеты именно из событийного цикла Tkinter, а не напрямую из другого потока. Полная многопоточная архитектура — вне рамок этой главы.

Тестируем то, что можно протестировать без окна

```
test_domennomj_logiki.py
```

```
def test_calculate_tip():
    assert calculate_tip(100, 10, 2) == 5.0

test_calculate_tip()
print("Доменная логика проверена без единого
виджета.")
```


Чек-лист перед тем, как считать GUI-проект готовым

- ☐ Окно открывается
- ☐ Все виджеты видны
- ☐ Порядок Tab по клавиатуре разумен
- ☐ Кнопка вызывает ожидаемый callback
- ☐ Некорректный ввод обрабатывается понятным сообщением
- ☐ Изменение размера окна не ломает раскладку
- ☐ Заккрытие окна завершает процесс корректно
- ☐ Данные сохраняются там, где это нужно

Практика: блокирующий vs неблокирующий обработчик

Проверяется без tkinter — на модели очереди событий

Открыть практику → (<https://www.cartesianschool.org/practice/16-32/index.html>)

Итоги главы: инструментарий Tkinter

От «программа ждёт событий, а не выполняется по порядку» до класса приложения с персистентными настройками.

Визуальный контакт-лист виджетов

Прежде чем выбирать по названию — вспомните, как это выглядит:

Глава 16 — контакт-лист

Label / Entry / Button

текст, поле ввода, кнопка

витрина виджетов

Checkbox

независимый флажок

снят/установлен

Radiobutton

группа взаимоисключающих вариантов

выбран один из трёх

Combobox

выбор из выпадающего списка

открытый список

Listbox

список с выделением элемента

выделенный элемент

Spinbox / Scale

число со стрелками / ползунок

Spinbox

Scale

Progressbar

индикатор выполнения

0-100%

Notebook

переключаемые вкладки

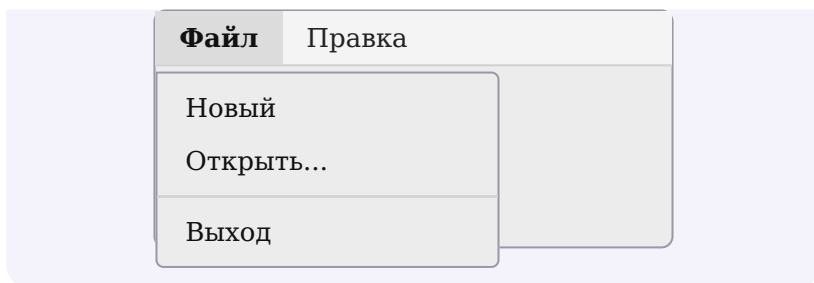
три вкладки

Toplevel

отдельное второе окно

главное + настройки

**СХЕМАТИЧЕСКОЕ ИЗОБРАЖЕНИЕ (НЕ
РЕАЛЬНЫЙ СКРИНШОТ)**



Инструментарий Tkinter

Что выбрать для конкретной задачи

Нужно
главное
окно
приложения

→ `tk.Tk()` – один на процесс

Нужно ещё одно
окно

→ `tk.Toplevel(root)`

Нужен
тематизированный
виджет формы

→ `ttk.Label/Button/Entry/Combobox`

Нужен
многострочный
редактор

→ `tk.Text / ScrolledText`

Простое вертикальное
расположение

→ `pack()`

Форма/
таблица/
адаптивные
колонки

→ `grid() + weight/sticky`

Точное
позиционирование

→ `place()` – осознанно

Общее
состояние
нескольких
виджетов

→ `StringVar/IntVar/BooleanVar/DoubleVar`

Уведомление
пользователя

→ `messagebox`

Выбор файла

→ `filedialog + pathlib`

Отложенный/
повторяющийся
вызов без
блокировки

→ `root.after(...)`

Персистентные
настройки → **JSON + pathlib (глава 15)**

Структура
крупного
приложения → **класс App + мелкие callback + чистая**

Обработка
клавиатуры/
мышь
напрямую → **глава 17 – .bind(...)**

Глава 16 целиком

СОБЫТИЙНАЯ МОДЕЛЬ

`mainloop()` – цикл, а не заморозка
`callback` $\neq$ `command` $\neq$ `event`
`function` без скобок

ДЕРЕВО ВИДЖЕТОВ

родитель определяет контекст
`create` → `configure` → размещение →
интерактивность

ТК И ТТК

ttk – тема + современные виджеты
стиль через `ttk.Style()`, не `fg/bg`

КОМПОНОВКА

pack – простые регионы
grid + weight – адаптивные формы
не смешивать в одном родителе

ВВОД И СОСТОЯНИЕ

Entry/Text, `end-1c`
Tk-переменные ≠ замена Python-
переменных

ДИАЛОГИ И ОКНА

messagebox/filedialog – проверяйте
результат
Toplevel, не второй Tk()

ОТЗЫВЧИВОСТЬ

`after()` вместо `sleep()`

долгая работа – не в одном callback

АРХИТЕКТУРА

чистая логика отдельно от виджетов

App HAS-A root

настройки – через JSON главы 15

- Tkinter-приложение
 - Событийная модель
 - mainloop
 - callback/command
 - Интерфейс
 - дерево виджетов
 - tk/ttk
 - pack/grid/place
 - Данные
 - Tk-переменные
 - валидация
 - Диалоги
 - messagebox
 - filedialog

- Toplevel
- Отзывчивость
 - after()
 - не sleep()
- Архитектура
 - класс приложения
 - персистентные настройки

Карта главы 16 целиком.

Последовательное внутри событийного

Точная формулировка важнее эффектной: инициализационный код по-прежнему выполняется последовательно, строка за строкой — и каждый отдельный callback тоже выполняется последовательно, когда его вызывают. Меняется не это, а **общая модель управления**: именно событийный цикл решает, какой callback будет вызван следующим, в ответ на событие — а не сам код, идущий одной сплошной линией сверху вниз.

старт программы



инициализация

выполняется последовательно
строка за строкой



mainloop()



Что дальше

Мы теперь умеем: строить `root` и дерево виджетов, связывать `callback` через `command`, компоновать формы через `pack` и адаптивный `grid`, хранить общее состояние в Tk-переменных, показывать диалоги, открывать/сохранять файлы, планировать отложенные вызовы через `after()` и собирать всё в класс приложения с персистентными настройками.

В главе 17 («Проект: игра «Крестики-нолики» с Tkinter») мы построим полноценную игру — и познакомимся с более общим механизмом привязки событий, `.bind("<Button-1>", ...)`, который реагирует на клик в любом месте виджета (например, на конкретную клетку холста), а не только на предопределённое действие вроде `command` у кнопки.

Что мы узнали в этой главе

- Инициализация и каждый callback по-прежнему выполняются последовательно — меняется общая модель управления: событийный цикл решает, какой callback вызвать следующим, в ответ на события.
- `root.mainloop()` запускает цикл обработки событий, а не «замирает» — программа остаётся отзывчивой.
- `command=` функция связывает callback с виджетом — без скобок после имени функции.
- Каждый виджет создаётся, настраивается и отдельно размещается менеджером геометрии — `pack()`, `grid()` или `place()`, но не смешивая `pack/grid` в одном родителе.
- `ttk` даёт тематизированные виджеты и стилизацию через `ttk.Style()` — но не заменяет `tk.Text/Canvas/Menu`.
- Tk-переменные (`StringVar` и другие) связывают несколько виджетов с одним значением — и не заменяют обычные переменные Python.
- `messagebox` и `filedialog` возвращают реальные значения, которые нужно проверять (в том числе на отмену), а не предполагать.
- `after()` планирует отложенный или повторяющийся вызов без блокировки — `time.sleep()` в callback замораживает весь интерфейс.
- Доменную логику стоит писать как чистые функции, проверяемые без единого виджета — callback лишь читает ввод, вызывает логику и обновляет экран.
- Персистентные настройки GUI-приложения используют те же функции `JSON+pathlib`, что и глава 15 — ничего принципиально нового.

Проект: игра «Крестики-нолики» с Tkinter

Построим первую полноценную GUI-игру как настоящий небольшой программный проект: разберём систему событий Tkinter, отделим игровое состояние от виджетов, реализуем правила и тесты, добавим управление мышью и клавиатурой, визуальную обратную связь, счёт матчей и финальную архитектуру приложения.

17.1 Привязка событий — делаем приложения динамическими!

17.2 Игра «Крестики-нолики» — объяснение

Настраиваем Tkinter

17.3 Создаём глобальные переменные

Создаём кнопки

17.4 При нажатии на кнопку рисуем на ней

17.5 Проверяем после каждого хода, победил ли игрок

17.6 Новая игра и первая полная версия

17.7 Event, callback, command и binding

17.8 Объект Event

17.9 Синтаксис событий Tk

17.10 command vs bind — что выбирать

17.11 Focus и клавиатура

17.12 Mouse Enter/Leave и hover

17.13 Модель игрового состояния

17.14 Индексы, строки и столбцы

17.15 Правильный алгоритм хода

17.16 Восемь выигрышных линий

17.17 Победа, ничья и terminal state

17.18 От widgets-as-state к board model

17.19 GameState с dataclass

17.20 Архитектура TicTacToeApp

17.21 Адаптивное игровое поле

17.22 Визуальный стиль X/O

17.23 Hover preview через bind()

17.24 Управление клавиатурой

17.25 Подсветка выигрышной линии

17.26 Счёт матчей

17.27 New Round vs New Match

17.28 Тестируем игру без Tkinter

17.29 Debug Labs

17.30 Visual effects и after()

17.31 Tic-Tac-Toe Pro — итоги главы

Привязка событий — делаем приложения динамическими!

`command` — не единственный способ реагировать на действия пользователя: `bind()` открывает доступ к событиям любого рода.

В главе 16 кнопки реагировали на клик через параметр `command` — это самый частый, но не единственный способ связать код с действием пользователя. **Привязка событий** (`.bind()`) позволяет реагировать на что угодно: нажатие клавиши, движение мыши, наведение курсора.

```
privyazka_sobytij.py
```

```
import tkinter as tk

root = tk.Tk()
label = tk.Label(root, text="Нажмите любую клавишу",
font=("Arial", 14))
label.pack(padx=20, pady=20)

def na_klavishu(event):
    label.config(text=f"Вы нажали: {event.keysym}")

root.bind("<Key>", na_klavishu)
root.mainloop()
```

Когда callback зарегистрирован через `.bind()`, Tkinter передаёт ему объект `Event` с подробностями о произошедшем событии: какая клавиша нажата (`event.keysym`), где кликнула мышь, и так далее. Это правило именно для `.bind()`: коллбэк, переданный в `command=` (как в главе 16), обычно вызывается без объекта `Event` — раздел 17.8 разберёт разницу подробнее.

Пригодится в главе 19

Та же идея — связать клавишу с функцией — вернётся в главе 19: змейка будет реагировать на нажатия клавиш со стрелками. У Turtle для этого есть свой, более простой интерфейс — `screen.listen()` и `screen.onkeypress()`, — но принцип тот же самый.

Практика: привязка событий клавиатуры

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-01/index.html>)

Игра «Крестики-нолики» — объяснение

Разберём план из шести шагов, по которому мы соберём игру — от пустого окна до готовой программы.

Игра «Крестики-нолики» — объяснение

Правила знакомы почти всем: поле 3×3 , два игрока по очереди ставят «X» и «O», первый, выстроивший три своих символа подряд — по горизонтали, вертикали или диагонали — побеждает. Соберём эту игру шаг за шагом — так, как её строил бы разработчик, а не одним куском кода сразу:

1. Настроить окно;
2. Создать переменные состояния игры;
3. Создать поле из девяти кнопок;
4. Реагировать на клик — рисовать X или O;
5. Проверять после каждого хода, есть ли победитель;
6. Добавить кнопку «Новая игра».

Пустое окно с полем из девяти кнопок и статусом Ход игрока: X

Забегая вперёд: примерно так будет выглядеть окно к концу раздела 17.6, после всех шести шагов. Дальше эта же страница пока не выполняет весь этот код — только настраивает окно (шаг 1).

Настраиваем Tkinter

nastrojka_okna.py

```
import tkinter as tk

root = tk.Tk()
root.title("Крестики-нолики")
```

Практика: начинаем собирать игру

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/17-03/index.html) (<https://www.cartesianschool.org/practice/17-03/index.html>)

Создаём глобальные переменные

Игре нужна память между ходами — заведём переменные состояния и построим поле из кнопок циклом.

Создаём глобальные переменные

Игре нужно помнить своё состояние между ходами — чей сейчас ход, закончена ли игра, и сами кнопки поля, чтобы менять их текст:

globalnye_peremennye.py

```
tekuschij_igrok = "X"
polya = [] # список из 9 кнопок поля
igra_okonchena = False
```

Создаём кнопки

Поле 3×3 — это девять кнопок, расположенных через `grid()` (глава 16). Создадим их циклом, а не девятью одинаковыми строками кода вручную (глава 10):

`sozdaem_knopki.py`

```
pole_frame = tk.Frame(root)
pole_frame.grid(row=1, column=0, columnspan=3)

for indeks in range(9):
    knopka = tk.Button(
        pole_frame,
        text="",
        font=("Arial", 24, "bold"),
        width=3, height=1,
    )
    knopka.grid(row=indeks // 3, column=indeks % 3)
    polya.append(knopka)
```

`indeks // 3` и `indeks % 3` — координаты из номера

Числа от 0 до 8 нужно превратить в строку и столбец таблицы 3×3. Целочисленное деление `//` (глава 5) даёт номер строки (0,0,0,1,1,1,2,2,2), а остаток `%` — номер столбца (0,1,2,0,1,2,0,1,2).

Девять пустых кнопок в сетке 3×3 и статус Ход игрока: X

Реальное окно полного файла прототипа: девять пустых кнопок, расставленных этим циклом.

Практика: переменные состояния и поле из кнопок

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-03/index.html>)

При нажатии на кнопку рисуем на ней

Каждая кнопка должна знать свой номер — здесь прячется одна из самых известных тонкостей Python.

Каждой кнопке нужна своя функция-обработчик клика, которая знает *какая именно* кнопка нажата. Хитрость в том, чтобы связать `command` с номером кнопки — для этого используем лямбда-функцию из главы 13:

risuem_na_knopke.py

```
def na_knopku_nazhali(indeks):
    global tekuschij_igrok

    if polya[indeks]["text"] != "":
        return # клетка уже занята

    polya[indeks]["text"] = tekuschij_igrok
    tekuschij_igrok = "0" if tekuschij_igrok == "X"
    else "X"

# при создании каждой кнопки:
knopka.config(command=lambda i=indeks:
na_knopku_nazhali(i))
```

Зачем `lambda i=indeks`, а не просто `lambda: na_knopku_nazhali(indeks)`?

Без `i=indeks` все девять кнопок запомнили бы одну и ту же переменную `indeks` — а не её значение в момент создания. К моменту клика цикл давно завершится, и `indeks` будет равен 8 для всех кнопок сразу. Значение по умолчанию `i=indeks` «замораживает» текущее значение при создании каждой лямбды — классическая и важная тонкость Python.

На кнопке клетки 0 появилась X, статус сменился на Ход игрока: O

Реальное окно полного файла прототипа: клик по первой клетке — на ней нарисован X, ход перешёл к O.

Практика: обработка клика по клетке поля

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-04/index.html>)

Проверяем после каждого хода, победил ли игрок

Восемь возможных выигрышных линий — три строки, три столбца, две диагонали.

После каждого хода нужно проверить: не выстроились ли три одинаковых символа подряд. Всего в игре восемь возможных «выигрышных линий» — три строки, три столбца и две диагонали:

proverka_pobedy.py

```
def proverit_pobedu():
    linii_pobedy = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8), # строки
        (0, 3, 6), (1, 4, 7), (2, 5, 8), # столбцы
        (0, 4, 8), (2, 4, 6),           # диагонали
    ]
    for a, b, c in linii_pobedy:
        znacheniya = (polya[a]["text"], polya[b]
["text"], polya[c]["text"])
        if znacheniya[0] != "" and znacheniya[0] ==
znacheniya[1] == znacheniya[2]:
            return znacheniya[0] # 'X' или 'O' —
есть победитель
    return None # победителя пока нет
```

Список кортежей — компактный способ описать 8 линий

Каждая линия — кортеж (глава 11) из трёх индексов поля. Список из восьми таких кортежей заменяет восемь отдельных проверок `if`, которые пришлось бы писать вручную.

Верхняя строка занята X, статус говорит Победил игрок X!

Реальное окно полного файла прототипа: X собрал верхнюю строку — `proverit_pobedu()` нашла победителя.

Ничья

Если все девять клеток заполнены, а победителя нет — это ничья:

```
proverka_nichyej.py
```

```
def pole_zapolнено():
    return all(кнопка["text"] != "" for кнопка in
    polya)
```

Поле полностью заполнено X и O без выигрышной линии, статус говорит Ничья!

Реальное окно полного файла прототипа: все девять клеток заняты, но выигрышной линии нет.

Практика: проверка победы и ничьей

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/17-05/index.html>)

Новая игра — и первая полная версия

Финальный штрих первого прототипа — сброс игры без перезапуска программы. Но глава только начинается.

Кнопка новой игры

Чтобы не перезапускать программу заново после каждой партии, добавим кнопку сброса — она возвращает все переменные и все клетки в начальное состояние:

novaya_igra.py

```
def novaya_igra():
    global tekuschij_igrok, igra_okonchena
    tekuschij_igrok = "X"
    igra_okonchena = False
    for knopka in polya:
        knopka.config(text="")
    status_label.config(text=f"Ход игрока: {tekuschij_igrok}")
```

Поле снова пустое, статус Ход игрока: X, после победы X в предыдущей партии

Реальное окно полного файла прототипа: нажата «Новая игра» после победы X — поле и статус сброшены.

Полная программа

Соберём все части в одну программу — она уже полностью проверена и доступна отдельным файлом в этой книге:

[projects/tkinter/tic-tac-toe/tic_tac_toe_basic.py](#)

Запустите игру у себя

Скачайте файл и запустите его через `python tic_tac_toe_basic.py` в терминале (глава 3) — либо кнопкой Run в VS Code или PyCharm.

Это ПЕРВЫЙ рабочий прототип, не финал главы

Мы построили настоящую играющую программу — с ходами, победой, ничьей и сбросом. Это честная, полная маленькая игра. Но начиная со следующего раздела мы будем смотреть на неё внимательнее: откуда узнаёт callback, что его вообще кто-то вызвал; почему хранить состояние в тексте кнопки не лучшая идея надолго; как сделать интерфейс, который реагирует на мышь и клавиатуру одним и тем же кодом. К концу главы (раздел 17.31) та же самая игра станет `tic_tac_toe.py` — с отдельной моделью состояния, подсветкой победной линии, счётом матчей и полными тестами.

★★ Самостоятельная задача

Счёт побед

Добавьте метки счёта побед X и O, увеличивайте нужный счётчик при каждой победе — счётчик не должен обнуляться кнопкой «Новая игра».

★★★ Задача повышенной сложности

Подсветка выигрышной линии

Когда находится победитель, измените цвет фона трёх выигрышных кнопок — потребуется вернуть из `proverit_pobedu()` не только победителя, но и индексы линии.

Практика: новая игра и первая полная версия

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-06/index.html>)

Итоги раздела

Что мы узнали

- `.bind()` реагирует на любые события — не только клик по кнопке.
- Большие проекты строят по шагам: состояние → интерфейс → обработка событий → правила → сброс.
- Лямбда с параметром по умолчанию (`lambda i=indeks: ...`) «замораживает» значение переменной цикла в момент создания.
- Список кортежей — удобный способ описать несколько похожих проверок (восемь выигрышных линий) без повторения кода.
- У полноценной игры почти всегда есть способ начать заново, не перезапуская программу.
- Это только начало главы — дальше мы разберём систему событий Tkinter глубже и перестроим саму игру на более надёжной архитектуре.

Event, callback, command и binding

«Кнопки Tkinter — это и есть игра» — неверная модель. Начнём с вопроса: кто вообще вызывает ваш callback?

Кто вызывает on_click()?

Сравним два знакомых способа получить действие от пользователя.

posledovatelno.py

```
inya = input("Как вас зовут? ")
print(f"Привет, {inya}!")
```

Здесь всё просто: программа сама, в известной строке кода, спрашивает и ждёт ответа.

gui.py

```
def on_click():
    print("Кнопка нажата")

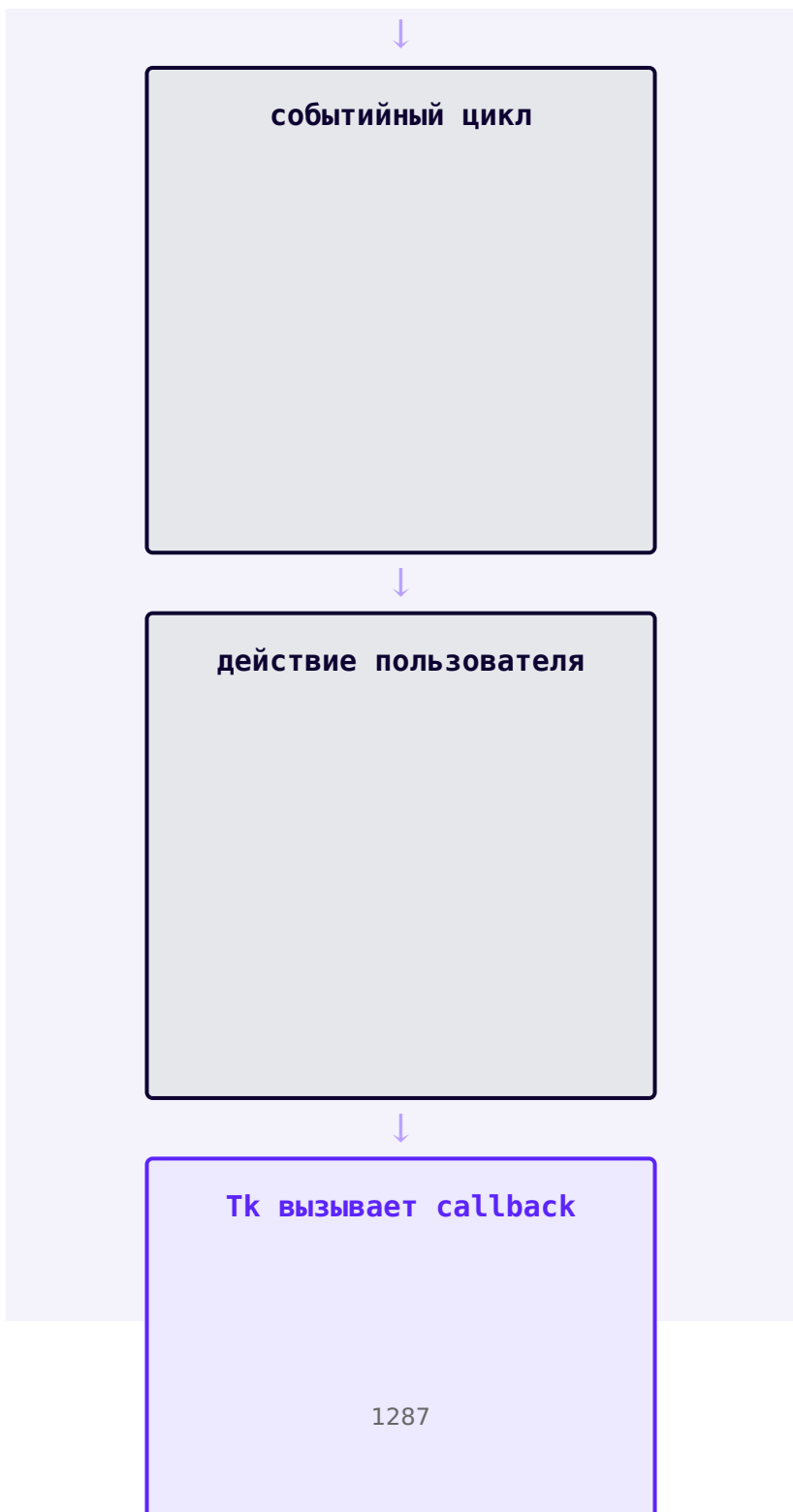
button = ttk.Button(root, text="OK", command=on_click)
```

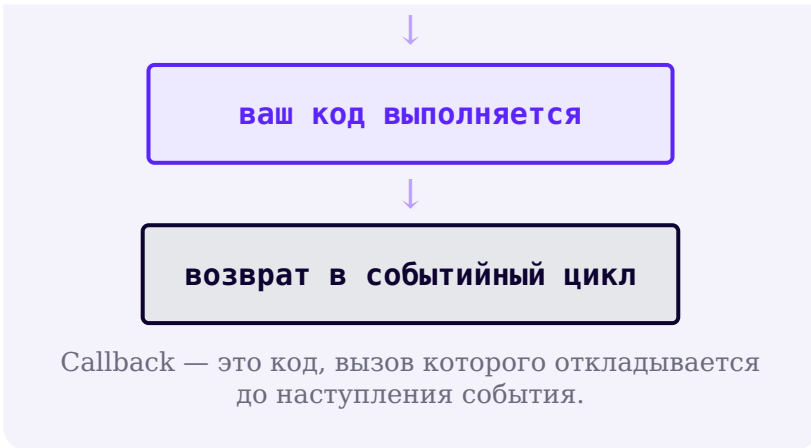
Кто вызывает on_click() ? Не мы — в коде нет строки on_click(). Функция может выполняться через секунду, через час или никогда — в зависимости от того, нажмёт ли пользователь кнопку.

ваш код

регистрирует callback

ждёт / обрабатывает



**Ответ**

Callback вызывает **система событий Tk** — в момент, когда происходит соответствующее действие пользователя (или другое событие). Не ваш код, не интерпретатор Python сам по себе.

Event, callback, command, binding — четыре разных слова

Термин	Что это
EVENT (событие)	Факт: что-то произошло — клик, нажатие клавиши, наведение курсора
CALLBACK (обратный вызов)	Функция, которую вызывают В ОТВЕТ на событие
COMMAND (команда)	Высокоуровневый крючок конкретного виджета для его основного действия (например, клика по Button)

Термин	Что это
BINDING (привязка)	Связь между событием (последовательностью) и callback-функцией

Они связаны, но не взаимозаменяемы

Событие — это ЧТО произошло. Callback — функция, которая ОТРЕАГИРУЕТ. Command — частный случай, специфичный для конкретного виджета. Binding — механизм, который соединяет событие с callback-ом. Спутать эти слова — источник путаницы в объяснении собственного кода.

Практика: классификация event/callback/command/binding

Автоматическая проверка — классифицируем короткие примеры кода по этим четырём терминам

[Открыть практику → \(https://www.cartesianschool.org/practice/17-07/index.html\)](https://www.cartesianschool.org/practice/17-07/index.html)

Объект Event

Callback из bind() получает Event; callback из command= обычно вызывается без него. Разберём, что внутри этого объекта.

Не каждый callback получает event

Частое заблуждение: «обработчик события всегда принимает event». Это неточно.

command= против bind()**command=** — БЕЗ event

```
def new_game():
    ...
```

```
ttk.Button(root,
text="Новая игра",
command=new_game)
```

bind() — С event

```
def on_key(event):
    ...
```

```
root.bind("<KeyPress>",
on_key)
```

Что использовать сегодня: `command=` вызывает функцию БЕЗ аргументов. `.bind(sequence, callback)` вызывает функцию С ОДНИМ аргументом — объектом `Event`. Перепутать сигнатуру — частая причина `TypeError`.

Объект Event — что внутри

event : Event

```
widget = виджет-источник
type = тип события
keysym = 'Return', 'a', ...
char = текстовый символ
keycode = числовой код
x = координата X внутри виджета
y = координата Y внутри виджета
```

Не каждое поле имеет смысл для каждого события — подробнее ниже.

event.x не имеет смысла для нажатия клавиши

`Event` — один класс на все типы событий, поэтому у него много полей сразу. Но `event.x / event.y` осмысленны для событий мыши, а не клавиатуры; `event.keysym / event.char` — для клавиатуры, а не для наведения мыши. Не читайте поле, если не уверены, что оно применимо к этому типу события.

event.widget

```
event_widget.py
```

```
def on_enter(event):
    print(event.widget)  # какой именно виджет
                          получил событие
```

`event.widget` — виджет, к которому относится событие. Это источник UI-события, а не игровое состояние (важное различие — раздел 17.18).

Практика: инспектор объекта Event

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-08/index.html>)

Синтаксис событий Tk

`<модификатор-тип-деталь>` — не всегда нужны все три части сразу.

Анатомия последовательности событий

Строка вроде `"<Control-Button-1>"` — это не магическое заклинание, а структура `<модификатор-тип-деталь>`, где часть элементов может отсутствовать:

Запись	Что означает
<code><KeyPress></code>	любая клавиша нажата (то же, что <code><Key></code>)
<code><Return></code>	конкретная клавиша Enter
<code><Escape></code>	конкретная клавиша Escape
<code><Button-1></code>	нажатие левой кнопки мыши

Запись	Что означает
<code><Enter></code>	курсор вошёл в границы виджета
<code><Leave></code>	курсор покинул границы виджета
<code><Control-r></code>	модификатор Control + клавиша r

Не нужно запоминать всю грамматику Tk целиком

У Tk десятки поддерживаемых последовательностей событий. Для главы 17 достаточно этого небольшого набора — при необходимости остальные ищутся в официальной документации Tcl/Tk по мере надобности, а не заучиваются заранее.

Button-1 — левая кнопка мыши

button_1_demo.py

```
def on_raw_click(event):
    print("Клик мышью в координатах", event.x,
          event.y)

label.bind("<Button-1>", on_raw_click)
```

Не главный способ активировать клетки игры

`<Button-1>` полезен для демонстрации низкоуровневого события мыши — но для клеток игрового поля мы всё равно будем использовать `command=` у Button (раздел 17.10). Здесь это просто иллюстрация синтаксиса.

Практика: синтаксис событий

Автоматическая проверка — сопоставляем описание действия с правильной строкой последовательности события

Открыть практику → (<https://www.cartesianschool.org/practice/17-09/index.html>)

command vs bind — что выбирать

Не «bind существует, значит используем его везде» — у каждого инструмента своя работа.

command= или bind() — как выбирать

Что выбрать?

Нужно выполнить основное действие Button → **command=**

Нужны координаты движения мыши → **bind("<Motion>", ...)**

Нужно знать, какая именно клавиша нажата → **bind("<Key>", ...)**

Нужна
реакция
на
наведение → `bind("<Enter>"/"<Leave>", ...)`
курсора
(без
клика)

Виджет вообще не
предоставляет `command` → `bind()`
(например, `Label`, `Frame`)

Не заменяйте `Button.command` на `bind('<Button-1>', ...)` по умолчанию

Это профессиональное правило. `command=` у `Button` — семантическая активация: она описывает основное действие кнопки, а встроенное поведение класса `Button` решает, какие способы активации вызывают эту команду. Конкретные клавиши могут отличаться у `tk.Button` и `ttk.Button`, а также между платформами и темами. Голая привязка `<Button-1>` описывает только конкретное событие мыши и обходит семантическое действие виджета. Замена `command` на `bind` «просто потому что можно» — типичная архитектурная ошибка, не стилистическая мелочь.

Как это распределится в игре

Три задачи — три разных инструмента

ХОД ПО КЛЕТКЕ

`command=` у Button
основное действие виджета

НАВЕДЕНИЕ — ПРЕВЬЮ

`bind(<Enter>/<Leave>)`
нет готового `command` для этого

КЛАВИШИ 1-9 / R

`bind(<Key>)` на root
не привязано к одному виджету

Практика: `command` vs `bind` — что выбрать

Автоматическая проверка — для набора сценариев выбираем правильный инструмент

Открыть практику → (<https://www.cartesianschool.org/practice/17-10/index.html>)

Focus и клавиатура

Окно существует — не значит, что оно получает все нажатия клавиш. Нужен фокус.

Клавиатурным событиям нужен адресат

Окно, в котором есть привязка к клавише, не значит, что нажатие ЛЮБОЙ клавиши где угодно попадёт в этот обработчик. Каждое клавиатурное событие сначала приходит в виджет, у которого сейчас **фокус ввода** — а уже оттуда Tk проверяет привязки по цепочке *bindtags*: сам виджет → его класс → окно верхнего уровня (root) → "all".

focus_demo.py

```
print(root.focus_get()) # какой виджет сейчас в
фокусе (или None)

entry.focus_set()      # явно передать фокус этому
виджету
```

«Привязка не работает» часто означает «класс виджета перехватил событие раньше»

`root.bind("<Key>", ...)` стоит в цепочке ПОСЛЕ привязок самого сфокусированного виджета и его класса — не вместо них. Если фокус на поле ввода, класс `Entry` первым обрабатывает нажатие цифры (вписывает символ в поле); привязка на `root` при этом обычно ВСЁ РАВНО срабатывает следом, если только обработчик явно не вернул `"break"`, прервав цепочку. Значит символ и появится в поле ввода, И одновременно уйдёт в игровую логику — редко то, чего вы хотели. Смотрите Debug Lab 1 (раздел 17.29).

Для главы 17 клавиатурные привязки повешены на `root` — но, как только что было показано, фокус на `Entry` или `Text` сам по себе НЕ отключает привязку на `root` (у нашей

игры таких полей нет — все клетки это кнопки). В приложении с текстовыми полями важно не «удерживать фокус на root», а осознанно решить: должны ли игровые горячие клавиши работать, пока пользователь печатает текст? Если нет — обработчик может проверить `focus_get()` и сам проигнорировать нажатие, когда фокус находится на текстовом поле.

Практика: `focus_get()` и `focus_set()`

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-11/index.html>)

Mouse Enter/Leave и hover

Наведение курсора — событие само по себе, без клика. Идеально для визуальных подсказок.

Enter / Leave — идеальны для визуальных эффектов



```
<Leave>
on_cell_leave(index)
```

Наведение — не клик: ни одно из этих событий не должно менять игровое состояние.

hover_bind.py

```
button.bind(
    "<Enter>",
    lambda _event, i=index: self.on_cell_enter(i),
)
```

_event — намеренно неиспользуемое имя

Callback от `.bind()` обязан принять аргумент `event`, даже если он не нужен внутри. Имя `_event` (с подчёркиванием) — общепринятый сигнал «параметр существует по требованию сигнатуры, но сознательно не используется».

Binding scope — где действует привязка

Вызов	Область действия
<code>widget.bind(...)</code>	только для этого конкретного виджета
<code>root.bind(...)</code>	для главного окна (например, глобальные клавиши игры)

ЧУТЬ ГЛУБЖЕ — `bind_class` / `bind_all`

Tk также поддерживает `bind_class(...)` (все виджеты одного класса) и `bind_all(...)` (буквально всё приложение). Для главы 17 они не нужны — упомянуты, чтобы вы знали, что они существуют, если понадобятся в собственных проектах.

ЧУТЬ ГЛУБЖЕ — `add="+"` и `unbind()`

По умолчанию `bind(sequence, callback)` ЗАМЕНЯЕТ предыдущий обработчик этой же последовательности на этом виджете. Необязательный третий аргумент `add="+"` вместо замены добавляет ЕЩЁ ОДИН обработчик — оба будут вызваны. `widget.unbind(sequence)` убирает привязку целиком. Для главы 17 не нужны — но Debug Lab 17 (раздел 17.29) показывает, почему неосторожный `add="+"` может незаметно запустить один и тот же код дважды.

Практика: `hover` через `Enter/Leave`

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-12/index.html>)

Модель игрового состояния

Tkinter показывает игру и передаёт ввод пользователя. Игровое СОСТОЯНИЕ — отдельная вещь.

Что вообще должна помнить программа?

```
state : GameState
```

```
board = ['', '', '', '', '', '', '', '', '']
current_player = 'X'
game_over = False
winner = None
winning_line = None
score_x = 0
score_o = 0
draws = 0
```

Это данные — ни одной кнопки Tkinter здесь нет.

Виджеты — НЕ каноническое игровое состояние

Кнопки на экране ОТОБРАЖАЮТ состояние — они не являются им. Если наведение мыши временно рисует «X» на кнопке (раздел 17.23), это не значит, что ход сделан — настоящее состояние живёт отдельно от того, что нарисовано на экране прямо сейчас.

Прототип: state в глобальных переменных

Раздел 17.3 использовал ровно такой подход:

```
prototip_globals.py
```

```
tekuschij_igrok = "X"
polya = []
igra_okonchena = False
```


Прототип: сначала делаем состояние видимым

Глобальные переменные — приемлемый выбор для самого первого маленького прототипа: их легко читать и легко объяснить. Проблема появляется, когда проект растёт — глобальные переменные создают скрытые связи между функциями, которые неявно ожидают, что кто-то другой уже их изменил в правильном порядке. Это не значит «глобальные переменные — зло»: это значит, что у них есть компромисс между простотой и сопровождаемостью, который стоит замечать.

Путь этой главы

От глобальных переменных к GameState

V1 — ГЛОБАЛЬНЫЕ (17.3)

3 отдельные переменные
просто, но растёт бесконтрольно

V2 — СЛОВАРЬ

```
{'board': [...], ...}
```

один объект вместо трёх переменных

V3 — GAMESTATE (17.19)

```
@dataclass
```

типизировано, читаемо, тестируемо

Одно состояние — разные видимые моменты партии

Пустое поле и статус Ход игрока: X

Реальное состояние канонического приложения: начало раунда, ход X.

После первого хода X статус сменился на Ход игрока: O

Реальное состояние того же приложения после валидного хода X: модель переключила `current_player` на O.

Пять отметок X и O на поле, партия продолжается

Реальное промежуточное состояние канонического приложения: `board` хранит отметки, `status_var` показывает текущего игрока.

Практика: моделируем игровое состояние

Автоматическая проверка — строим и проверяем словарь-состояние без Tkinter

[Открыть практику](https://www.cartesianschool.org/practice/17-13/index.html) → (<https://www.cartesianschool.org/practice/17-13/index.html>)

Индексы, строки и столбцы

Поле выглядит как таблица 3×3 — но в памяти это один список из девяти элементов.

Поле — плоский список из девяти элементов

0	1	2
3	4	5
6	7	8

Индексы клеток поля 0..8 — слева направо, сверху вниз.

Хотя визуально поле — таблица 3×3 , в памяти это ОДИН плоский список из девяти элементов (`board[0]` .. `board[8]`), как и в разделе 17.3.

Индекс → строка и столбец

```
indeks_v_koordinaty.py
```

```
row = index // 3  
column = index % 3
```

index	row	column
0	0	0
1	0	1
2	0	2
3	1	0
4	1	1
5	1	2
6	2	0
7	2	1
8	2	2

Обратное направление: строка и столбец → индекс

koordinaty_v_indeks.py

```
index = row * 3 + column
```

Пригодится не только здесь

Формула $\text{row} * \text{ширина} + \text{column}$ — общий приём для любой прямоугольной сетки, хранящейся плоским списком (изображения, карты уровней, электронные таблицы).

Практика: индекс, строка, столбец

Автоматическая проверка — прямое и обратное преобразование координат для всех 9 клеток

Открыть практику → (<https://www.cartesianschool.org/practice/17-14/index.html>)

Правильный алгоритм хода

Один путь для любого способа сделать ход — проверка, ход, правила, смена игрока.

Валидный ход — что это?

`attempt_move(index)` — единый путь для мыши и клавиатуры (раздел 17.24). Разберём его в двух шагах: сначала фильтр валидности, потом — что происходит после того, как ход уже принят.

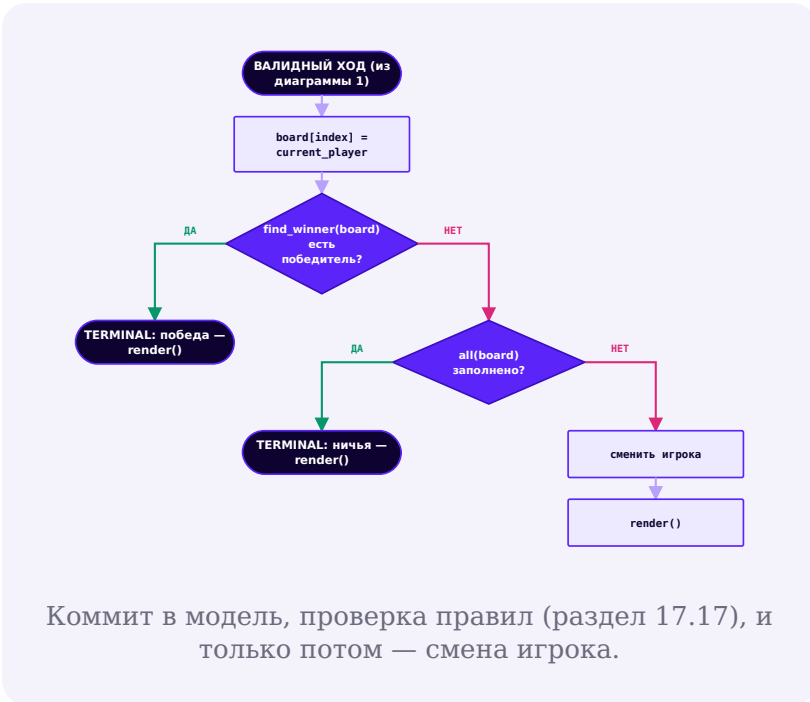
Диаграмма 1 — фильтр валидности хода



Невалидный ход НЕ переключает игрока

Это частая ошибка (Debug Lab 7, раздел 17.29): если клик проигнорирован — очередь хода должна остаться прежней. Переключение игрока происходит только ПОСЛЕ успешного хода.

Диаграмма 2 — что происходит после валидного хода



Видимый результат валидного хода

Пустое поле перед первым ходом

Реальное окно канонического приложения до хода: модель board содержит девять пустых строк.

X занял центральную клетку, очередь перешла к O

То же реальное приложение после `attempt_move(4)`:
отметка записана в модель, затем `render()` обновил поле и статус.

Переключение игрока — три эквивалентных способа

`smena_igroka_razvernuto.py`

```
if current_player == "X":  
    current_player = "O"  
else:  
    current_player = "X"
```

`smena_igroka_kratko.py`

```
current_player = "O" if current_player == "X" else "X"
```

Сначала читаемость, лаконичность потом

Обе версии делают одно и то же. Развёрнутая явно читается как «если/иначе» — с ней проще начинать. Короткая версия (условное выражение, глава 9; его часто называют тернарным оператором) — не «умнее», просто компактнее, когда вы уже уверенно её читаете.

Практика: логика одного хода

Автоматическая проверка — валидация хода и переключение игрока на чистых функциях

[Открыть практику →](https://www.cartesianschool.org/practice/17-15/index.html) (<https://www.cartesianschool.org/practice/17-15/index.html>)

Восемь выигрышных линий

Три строки, три столбца, две диагонали — и одна чистая функция, которая их все проверяет.

Все восемь выигрышных линий

Строки

X	X	X

Строка 1

X	X	X

Строка 2

X	X	X

Строка 3

Столбцы

X		
X		
X		

Столбец 1



	X	
	X	
	X	

Столбец 2

		X
		X
		X

Столбец 3

Диагонали



Константа WINNING_LINES

winning_lines.py

```
WINNING_LINES = (  
    (0, 1, 2), (3, 4, 5), (6, 7, 8), # строки  
    (0, 3, 6), (1, 4, 7), (2, 5, 8), # столбцы  
    (0, 4, 8), (2, 4, 6),           # диагонали  
)
```

ЗАГЛАВНЫМИ — это данные правил, а не алгоритм

`WINNING_LINES` написана ЗАГЛАВНЫМИ буквами: так в Python принято называть константы модуля (соглашение PEP 8 — договорённость между программистами, а не запрет интерпретатора на изменение). Это данные о правилах игры, отдельные от кода, который их использует. `find_winner()` сам по себе не знает о размере 3×3 — он просто перебирает готовые линии. Но константа решает не всё: индексная арифметика (`// 3`, `% 3`), длина `[""] * 9` и сама 3×3 -сетка виджетов тоже жёстко зашиты на размер поля — реальная поддержка другого размера потребовала бы менять всё это, а не только `WINNING_LINES`.

`find_winner()` — чистая функция

`find_winner.py`

```
def find_winner(board):
    for a, b, c in WINNING_LINES:
        mark = board[a]
        if mark and mark == board[b] == board[c]:
            return mark, (a, b, c)
    return None, None
```

Возвращает И победителя, И саму линию

Раздел 17.5 возвращал только победителя.

`find_winner()` возвращает пару (`winner`, `winning_line`) — индексы линии нужны, чтобы подсветить именно её (раздел 17.25), не пересчитывая победителя заново.

Ничего не знает про Tkinter

`find_winner(board)` принимает обычный список строк — никаких кнопок, никакого `root`. Это то, что делает функцию тестируемой без единого открытого окна (раздел 17.28).

Практика: find_winner для всех восьми линий

Автоматическая проверка — find_winner на каждой из восьми линий для X и O

Открыть практику → (<https://www.cartesianschool.org/practice/17-16/index.html>)

Победа, ничья и terminal state

Заполненное поле — не всегда ничья. Порядок проверки — часть правил игры.

is_draw() — чистая функция

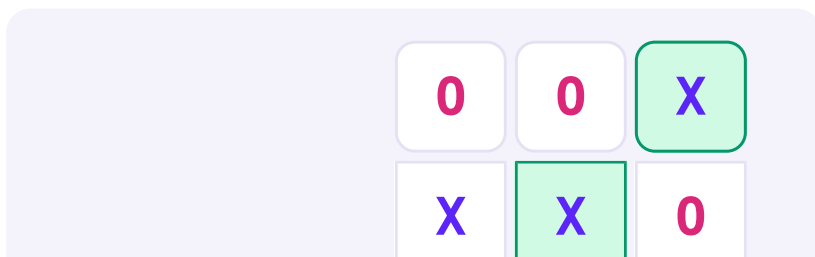
is_draw.py

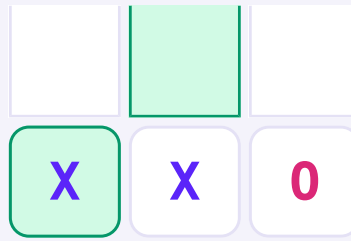
```
def is_draw(board):
    winner, _ = find_winner(board)
    return winner is None and all(board)
```

«Поле заполнено» — недостаточное условие ничьей

Заполненное поле само по себе не гарантирует ничью — если последний ход одновременно выигрывает линию, это ПОБЕДА, а не ничья. Порядок проверки решает всё.

Последний ход: победа побеждает ничью





9-й ход одновременно заполняет поле И выигрывает диагональ 2-4-6.

Обязательное упражнение-предсказание

Прежде чем читать код — предскажите: если 9-й ход заполняет ПОСЛЕДНЮЮ пустую клетку И выстраивает линию, что покажет программа? «Ничья» или «Победа X»? Правильный ответ — «Победа», если `find_winner()` проверяется РАНЬШЕ `is_draw()`. Именно поэтому `is_draw` внутри себя сначала зовёт `find_winner` и только потом смотрит на заполненность.

Terminal state

ПОБЕДА и НИЧЬЯ — истинные тупики: как только партия туда попала, обратного пути в «игра продолжается» нет. Ниже это показано буквально — у терминальных узлов нет исходящих стрелок.



Три терминальных результата в реальном окне

X выиграл верхнюю строку

Реальное состояние полной версии: победа X, выигрышная линия подсвечена, поле заблокировано.

O выиграл средний столбец

Реальное состояние полной версии: та же логика `find_winner()` работает для O.

Полное поле без выигрышной линии, статус Ничья

Реальное состояние полной версии: поле заполнено, победителя нет, дополнительные ходы заблокированы.

Практика: победа против ничьей — порядок проверки

Автоматическая проверка — в том числе тест на «последний ход выигрывает, а не заканчивается ничьей»

Открыть практику → (<https://www.cartesianschool.org/practice/17-17/index.html>)

От widgets-as-state к board model

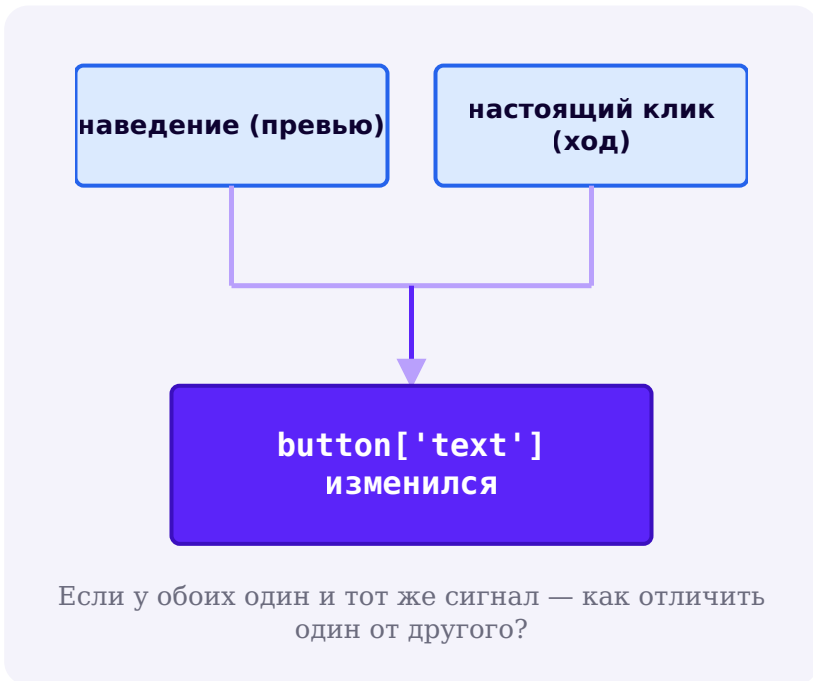
Наведение мыши временно рисует X на пустой кнопке. Если это и есть «состояние» — игра сломана.

Раздел 17.3 хранил состояние В ТЕКСТЕ кнопки

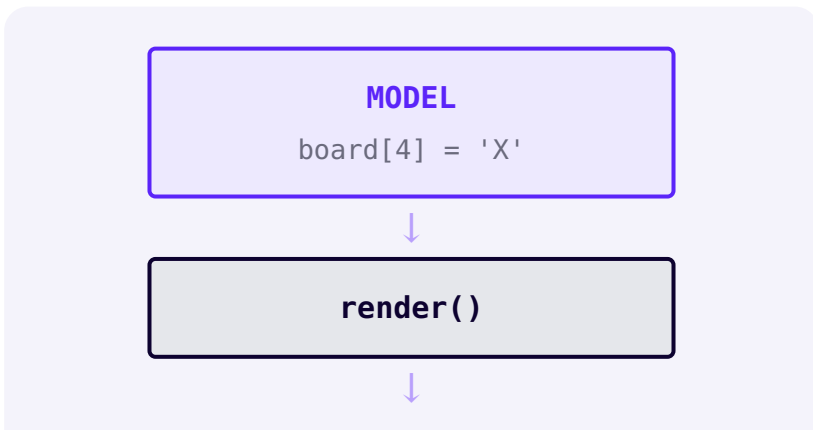
`widget_kak_istochnik.py`

```
# text кнопки — единственное место, где хранится
#занята клетка или нет"
if polya[indeks]["text"] != "":
    return
```

Это работало для маленького прототипа. Но представьте эффект наведения (раздел 17.23): он временно показывает «X» на пустой кнопке, ещё не сделав хода. Если состояние ЖИВЁТ в тексте кнопки — программа больше не может отличить «наведение» от «настоящего хода».



Модель отдельно, кнопка — только отображение



VIEW

`buttons[4]` показывает 'X'

Данные текут в одну сторону: модель → `render()` → виджет. Никогда наоборот.

Кнопка может ВРЕМЕННО показывать не то, что в модели

Во время наведения `buttons[4]["text"] == "X"`, а `board[4] == ""` — и это НОРМАЛЬНО, если модель остаётся источником истины. Проблема начинается только тогда, когда код (например, проверка победителя) читает состояние из текста кнопки вместо модели — см. Debug Lab 3, раздел 17.29.

board = список строк — окончательная модель

`board_model.py`

```
board = ["", "", "", "", "", "", "", "", "", ""]
# board[4] = "X" — это НАСТОЯЩИЙ ход, кнопка лишь
отображает это значение
```

Практика: модель против виджета-как-состояния

Автоматическая проверка — на игрушечной модели виджета без Tkinter показываем, почему `widget-as-state` ломается

Открыть практику → (<https://www.cartesianschool.org/practice/17-18/index.html>)

GameState с dataclass

От прототипных глобальных переменных — к одному типизированному объекту состояния.

GameState — @dataclass (глава 14)

game_state.py

```
from dataclasses import dataclass, field

@dataclass
class GameState:
    board: list[str] = field(default_factory=lambda:
[""] * 9)
    current_player: str = "X"
    game_over: bool = False
    winner: str | None = None
    winning_line: tuple[int, int, int] | None = None
    score_x: int = 0
    score_o: int = 0
    draws: int = 0
```

НЕ board: list[str] = [""] * 9

Для @dataclass это даже не тихий бар: Python замечает изменяемое значение по умолчанию (list, dict, set) прямо при определении класса и сразу отказывает — `ValueError: mutable default <class 'list'> for field board is not allowed: use default_factory`. Класс с такой строкой вообще не создастся, значит до «распаривания между экземплярами» (глава 14 разбирала эту ловушку для обычных изменяемых аргументов по умолчанию) дело не дойдёт. `field(default_factory=lambda: [""] * 9)` — единственный способ, которым dataclass разрешит дать полю новый список при каждом создании экземпляра.

GameState хранит ТОЛЬКО данные партии — ни одного виджета внутри.

Практика: GameState как dataclass

Автоматическая проверка — умолчания, независимость экземпляров, поля состояния

Открыть практику → (<https://www.cartesianschool.org/practice/17-19/index.html>)

Архитектура TicTacToeApp

app.root, а не class TicTacToeApp(tk.Tk) — та же композиция, что и в главе 16.

app HAS-A root — не наследование от Tk

Композиция против наследования

Возможно, но не нужно

```
class
TicTacToeApp(tk.Tk):
    def __init__(self):
super().__init__()
    ...
```

Как в главе 16

```
class TicTacToeApp:
    def __init__(self,
root):
        self.root = root
    ...
```

Что использовать сегодня: Наследование от `tk.Tk` технически работает, но не нужно и не согласуется с тем, как строились приложения в главе 16. `app.root` (композиция, «HAS-A») — тот же паттерн, что и у Tip Calculator Pro и редактора заметок.

Объектный граф приложения

```
app : TicTacToeApp

root = Tk
state = GameState
buttons = list[Button]
status_var = StringVar
score_var = StringVar
```

app хранит виджеты и ССЫЛКУ на состояние — не наоборот.

Домен против UI — разделение ответственности

Чистая доменная логика (без Tkinter)	UI-логика (внутри TicTacToeApp)
find_winner(board)	on_cell_click / attempt_move — вызывает домен, потом render
is_draw(board)	on_cell_enter / on_cell_leave — прерыву
index → row/column	render() — рисует виджеты ИЗ модели (state)
	new_round() / new_match() — жизненный цикл

Проверка «можно ли протестировать без окна?»

Хороший тест архитектуры: если для проверки функции обязательно открывать окно Tkinter — это, вероятно, UI-логика. Если можно вызвать её с обычными списком/строкой и получить результат — это доменная логика, и её МОЖНО было бы вынести в отдельный модуль.

Практика: собираем объектный граф TicTacToeApp

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/17-20/index.html) → (<https://www.cartesianschool.org/practice/17-20/index.html>)

Адаптивное игровое поле

Игра не должна разваливаться при изменении размера окна — `grid()` и `weight` решают это так же, как в главе 16.

Поле не должно превращаться в хрупкую конструкцию из виджетов с фиксированным размером

adaptivnoe_pole.py

```
# outer — родительский контейнер (build_ui); строка 1
— это строка поля
outer.rowconfigure(1, weight=1)

board_frame = ttk.Frame(outer)
board_frame.grid(row=1, column=0, columnspan=3,
sticky="nsew")
for i in range(3):
    board_frame.rowconfigure(i, weight=1)
    board_frame.columnconfigure(i, weight=1)

for index in range(9):
    btn = tk.Button(board_frame, text="",
font=("Arial", 28, "bold"), width=3, height=1)
    btn.grid(row=index // 3, column=index % 3,
sticky="nsew", padx=3, pady=3)
```

sticky='nsew' + weight — на КАЖДОМ уровне вложенности

Те же приёмы адаптивного `grid()`, что и в разделе 16.15 — но их нужно применить на обоих уровнях сразу. Внутри `board_frame` уже недостаточно дать вес строкам/столбцам, если сам `board_frame` не растягивается в своей ячейке `outer`: нужны И `outer.rowconfigure(1, weight=1)` на родителе, И `sticky="nsew"` на самом `board_frame.grid()`. Пропустить любое из двух — и поле перестанет расти вместе с окном по вертикали.

Не обещаем идеально квадратные клетки на любой платформе

`width=3, height=1` у `tk.Button` задаёт размер в текстовых единицах (символах/строках шрифта), а не в пикселях — точное соотношение сторон зависит от шрифта и темы конкретной ОС. Мы делаем поле адаптивным и опрятным — не гарантируем математически точный квадрат на всех платформах одновременно.

Видно на реальном окне, а не только в коде

Слева — обычный размер окна, справа — то же окно после увеличения: то же состояние партии, но клетки заметно крупнее

Одно и то же приложение и одно и то же состояние партии. Слева — исходный размер окна, справа — то же окно после увеличения. Клетки получают дополнительное пространство благодаря `weight` и `sticky="nsew"`.

Практика: адаптивное поле при изменении размера окна

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-21/index.html>)

Визуальный стиль X/O

X и O должны различаться с первого взгляда — и не только по цвету.

Палитра игры — сначала увиденная, потом числовая



Игрок X

#5B24F9



Игрок O

#DB2777



**Победная
линия**

#059669



**Недоступно (конец
игры)**

#6B6B7D

X — фиолетовый (#5B24F9), O — розовый (#DB2777) — те же фирменные цвета курса, что и везде на сайте. Победная линия подсвечивается мягким зелёным фоном, а после конца игры отметки становятся приглушёнными — виджет `disabled` сигнализирует «здесь больше нельзя ходить».

Почему здесь используется классический `tk.Button`, а не `ttk`

Осознанный выбор, а не «забыли про `ttk`»

Глава 16 учит предпочитать `ttk` там, где есть тематизированный аналог (раздел 16.12). Но покраска отдельной клетки в цвет игрока напрямую через `fg=` / `bg=` — именно то, что `tk.Button` делает просто и предсказуемо, а тема `ttk` обычно перекрывает подобную покраску по виджету через `ttk.Style()`. Рамка, статус и счёт при этом всё равно построены на `ttk` — выбор виджета принят осознанно ради конкретной задачи, не по привычке.

```
cvet_otmetki.py
```

```
MARK_COLOR = {"X": "#5B24F9", "O": "#DB2777"}
```

```
btn.config(text=mark, fg=MARK_COLOR.get(mark,
"#0D0230"))
```

Не только цвет — ещё и буква, и текст статуса

Победителя видно не только по зелёной подсветке: буквы «X»/«O» остаются на месте, а статус явно пишет «Победил игрок X!». Это важное правило доступности: нельзя полагаться только на цвет как на единственный сигнал.

Практика: покраска X и O

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-22/index.html>)

Hover preview через `bind()`

Показать возможный ход — не значит его сделать. Именно этот эффект доказывает, зачем нужна модель.

Наведение — сигнатурный визуальный эффект главы

Наведение на пустую клетку показывает бледный превью X

Реальное окно полной версии: ход X, временный превью не изменяет board. Фрагмент ниже отвечает только за эффект наведения.

После хода X наведение показывает бледный превью O

Реальное окно той же версии на ходе O: визуальный слой меняется вместе с `current_player`, но пустая клетка в модели остаётся пустой.

`hover_preview.py`

```
def on_cell_enter(self, index):
    state = self.state
    if state.game_over or state.board[index]:
        return # не показываем превью — конец игры
    или клетка занята

    self.buttons[index].config(text=state.current_player,
                               fg=HOVER_COLOR)

def on_cell_leave(self, index):
    self.render() # не 'text=""' — а render() из
МОДЕЛИ
```

on_cell_leave НЕ должен просто стирать текст

Наивная реализация `on_cell_leave` вида `self.buttons[index].config(text="")` сотрёт уже СДЕЛАННЫЙ ход, если игрок наводит мышь на занятую клетку каким-то другим путём — Debug Lab 13 (раздел 17.29) разбирает это подробно. Правильный код всегда перерисовывает из модели, а не гадает, что стереть.

Доказательство: модель не меняется

`hover_ne_menyaet_model.py`

```
assert app.state.board[4] == ""
app.on_cell_enter(4)
assert app.state.board[4] == ""      # ПОСЛЕ наведения
— всё ещё пусто
app.on_cell_leave(4)
assert app.state.board[4] == ""      # и после ухода
курсора — тоже
```

Практика: hover preview на реальном поле

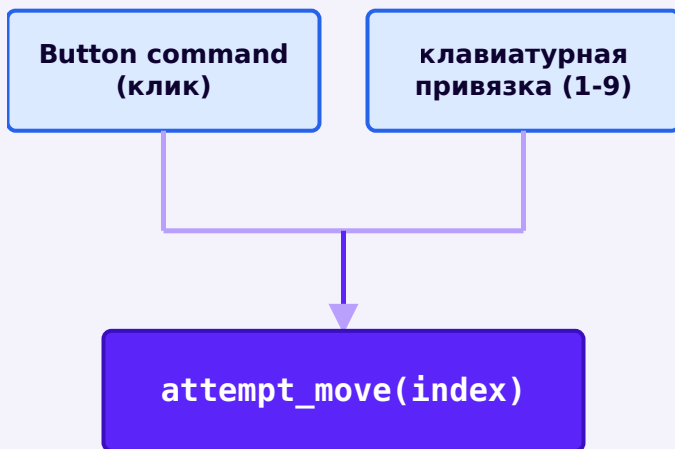
Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-23/index.html>)

Управление клавиатурой

1-9 — те же ходы, что и клик мышью, через тот же самый код.

Мышь и клавиатура сходятся в одной точке



Оба пути ведут в ОДНУ И ТУ ЖЕ функцию — правила не дублируются.

keyboard_control.py

```
def on_key(self, event):
    if event.ksysym in ("r", "R"):
        self.new_round()
        return
    if event.char and event.char.isdigit():
        n = int(event.char)
        if 1 <= n <= 9:
            self.attempt_move(n - 1)    # клавиша 1 ->
индекс 0, ... 9 -> индекс 8
```

Клавиатура не получает свою копию правил

Обработчик клавиатуры ТОЛЬКО переводит нажатую клавишу в номер клетки и зовёт `attempt_move()` — ту же функцию, что вызывает клик мышью. Если бы клавиатурный путь копировал проверку победителя отдельно, два набора правил рано или поздно разошлись бы (Debug Lab 9, раздел 17.29 — похожая ошибка с опечаткой в линии).

Клавиша	Индекс клетки
1	0
2	1
3	2
4	3
5	4
6	5
7	6
8	7
9	8

keysym против char — здесь выбран char

Раздел 17.8 показал разницу: `event.char` — фактический введённый символ. Для цифровых клавиш верхнего ряда он предсказуемо равен самой цифре; с дополнительной цифровой клавиатурой (NumPad) поведение обычно совпадает, если включён NumLock, но может отличаться в зависимости от платформы, раскладки и состояния NumLock — если клавиатура важна для вашего проекта, проверьте оба варианта на целевой системе. Здесь `event.char` всё равно удобнее, чем сравнение по `event.keysym`.

Практика: клавиши 1-9 и R

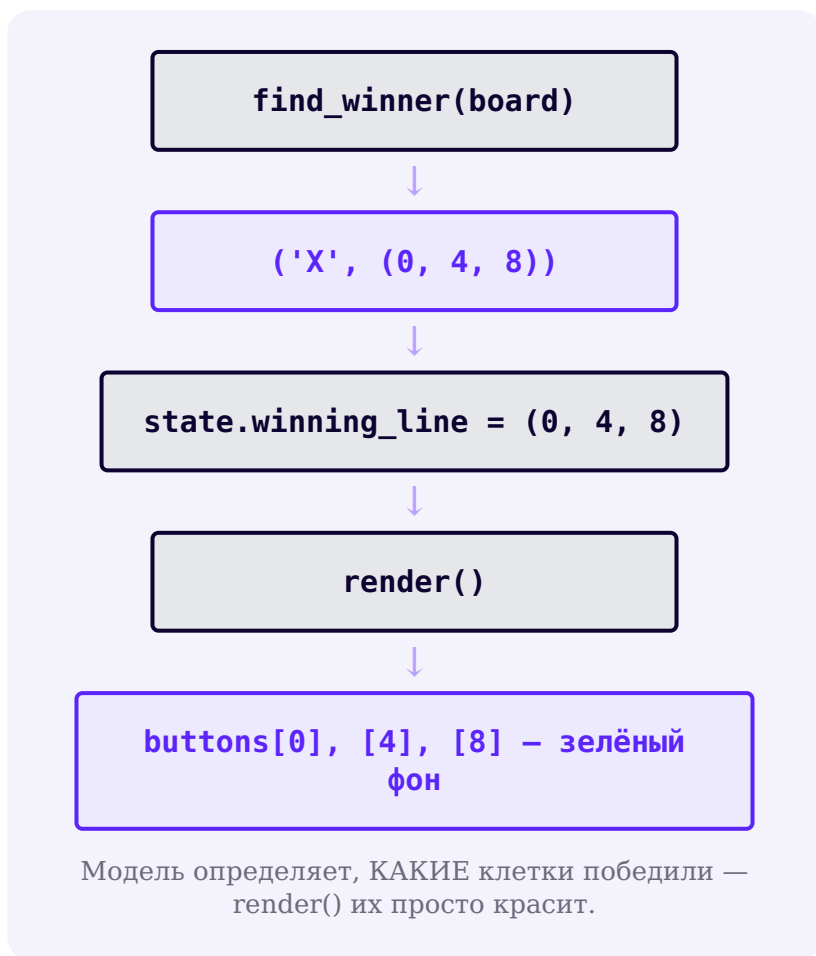
Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/17-24/index.html\)](https://www.cartesianschool.org/practice/17-24/index.html)

Подсветка выигрышной линии

Модель уже знает, какие три клетки победили — `render()` их просто красит.

Данные о линии ведут к подсветке




```
podsvetka_linii.py
```

```
for index, btn in enumerate(self.buttons):
    is_win_cell = state.winning_line is not None and
index in state.winning_line
    btn.config(bg=WIN_BG if is_win_cell else
NEUTRAL_BG)
```

Подсвечиваем ИМЕННО линию — не любые совпавшие символы

Если на поле есть, скажем, четыре «X», но победили только три из них по диагонали — подсвечивается СТРОГО `winning_line`, а не каждая клетка с меткой «X». Проверка `index in state.winning_line` гарантирует это.

Диагональ 0-4-8 подсвечена зелёным, статус говорит Победил игрок X

Реальное окно полной версии игры: победная диагональ подсвечена, буквы остаются видимыми. Фрагмент выше — это только код самой подсветки, а не весь путь до победы.

Практика: подсветка выигрышной линии

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-25/index.html>)

Счёт матчей

Партия заканчивается — матч продолжается: счёт живёт дольше одного раунда.

Счёт живёт в GameState, а не в случайной отдельной переменной

Строка счёта: X: 2, O: 1, Ничьи: 1

Реальное окно полной версии игры: счёт матча под игровым полем. Фрагмент ниже показывает, как значения счёта попадают в эту строку.

schet.py

```
if winner == "X":
    state.score_x += 1
else:
    state.score_o += 1

self.score_var.set(f"X: {state.score_x} | 0: {state.score_o} | Ничьи: {state.draws}")
```

Осторожно с редким символом-разделителем в Tkinter-метках

Красивый символ-маркер «•» в headless-окружениях с урезанным набором шрифтов иногда превращается в набор мусорных символов — как выяснилось при подготовке скриншотов этой самой главы. Простой ASCII-разделитель " | " работает предсказуемо везде.

Счёт увеличивается ровно один раз за партию — внутри `attempt_move()`, сразу после того как найден победитель или зафиксирована ничья, а не где-то ещё.

Практика: учёт счёта матча

Автоматическая проверка — чистые функции обновления счёта по результату раунда

Открыть практику → (<https://www.cartesianschool.org/practice/17-26/index.html>)

New Round vs New Match

«Новая игра» — это раунд или матч целиком? В этой игре — принципиально разные вещи.

Два разных «сброса» — и это не одно и то же

New Round (новый раунд)	New Match (новый матч)
Очищает поле, текущего игрока, <code>game_over</code> , <code>winner</code> , <code>winning_line</code>	Делает всё то же, что New Round
СОХРАНЯЕТ счёт (<code>score_x</code> , <code>score_o</code> , <code>draws</code>)	И ДОПОЛНИТЕЛЬНО обнуляет счёт
Вызывается после каждой партии	Вызывается, когда матч целиком завершён

```
new_round_vs_new_match.py

def new_round(self):
    self.state.board = [""] * 9
    self.state.current_player = "X"
    self.state.game_over = False
    self.state.winner = None
    self.state.winning_line = None
    self.render()

def new_match(self):
    self.state.score_x = 0
    self.state.score_o = 0
    self.state.draws = 0
    self.new_round()    # New Match полностью включает
                        # в себя New Round
```

Перепутать их — реальная ошибка (Debug Lab 12, раздел 17.29)

Если кнопка «Новая игра» вызывает то, что обнуляет счёт — игроки теряют турнирный счёт после каждой партии. Если кнопка «Новый матч» не обнуляет счёт — старый счёт тянется в новый турнир. Названия функций должны отражать это различие однозначно.

После New Round поле пустое, а счёт X:1 сохранён

Реальное окно полной версии после `new_round()`: состояние раунда сброшено, счёт матча сохранён. Код выше — фрагмент методов полного приложения.

После New Match поле пустое и весь счёт равен нулю

Реальное окно полной версии после `new_match()`: сброшены и раунд, и счёт X/O/ничьих. Код выше — фрагмент методов полного приложения.

Необязательное расширение: сохраняем счёт матча в JSON

Ниже показан только фрагмент — тело функции `save_scores()` из `tic_tac_toe.py`. Полный файл уже содержит нужные `import json`, `from pathlib import Path` и определение `GameState` — здесь они не повторяются.

фрагмент `save_scores()` из `tic_tac_toe.py`

```
SCORES_PATH = Path("tic_tac_toe_scores.json")  #
относительно текущей рабочей директории

def save_scores(state):
    with SCORES_PATH.open("w", encoding="utf-8") as f:
        json.dump({"x": state.score_x, "o":
state.score_o, "draws": state.draws}, f,
                    ensure_ascii=False, indent=2)
```

Необязательно для первой играющей версии

Персистентный счёт — приятное расширение (глава 15: `pathlib` + `JSON`), но НЕ требование для рабочей игры. В `tic_tac_toe.py` это включается явным флагом `persist_scores=True` — по умолчанию выключено.

Практика: New Round, New Match и сохранение счёта

Автоматическая проверка — семантика сброса раунда/матча и сохранение `JSON` на виртуальной файловой системе браузера

Открыть практику → (<https://www.cartesianschool.org/practice/17-27/index.html>)

Тестируем игру без Tkinter

Чистая логика — прямая награда за архитектуру: тестируем правила игры без единого открытого окна.

Почему чистая логика легко тестируется

test_bez_okna.py

```
board = ["X", "X", "X", "", "", "", "", "", ""]
winner, line = find_winner(board)
assert winner == "X"
# Ни одного открытого окна Tkinter – просто список
# строк и обычная функция.
```

Прямая награда за архитектуру раздела 17.20

Это возможно именно потому, что `find_winner / is_draw` ничего не знают про Tkinter. Если бы победитель определялся прямо внутри обработчика клика по кнопке — тестировать пришлось бы через настоящее окно и настоящие клики.

Полная тестовая матрица игры

Проверка	Ожидаемый результат
X выигрывает все 8 линий	<code>find_winner</code> возвращает ('X', линия) для каждой
O выигрывает все 8 линий	<code>find_winner</code> возвращает ('O', линия) для каждой
Пустое/неполное поле	<code>find_winner</code> возвращает (None, None)
Известная ничья	<code>is_draw(board)</code> is True
Последний ход одновременно выигрывает	<code>find_winner</code> побеждает, <code>is_draw</code> is False
Ход в занятую клетку	<code>attempt_move</code> ничего не меняет, <code>current_player</code> тот же

Проверка	Ожидаемый результат
Ход после <code>game_over</code>	<code>attempt_move</code> ничего не меняет
<code>new_round()</code>	поле пустое, <code>game_over=False</code> , счёт СОХРАНЁН
<code>new_match()</code>	то же самое, но счёт ОБНУЛЁН

Проверяйте все восемь линий, а не одну строку наугад

Тест только для одной строки не поймает опечатку в описании диагонали — Debug Lab 9 (раздел 17.29) устроен именно так, чтобы одна строка теста его бы не заметила.

Практика: полный набор тестов без Tkinter

Автоматическая проверка — вся регрессионная матрица игры: 8 линий × 2 игрока, ничья, последний ход, невалидный ход, сброс

[Открыть практику](https://www.cartesianschool.org/practice/17-28/index.html) → (<https://www.cartesianschool.org/practice/17-28/index.html>)

Debug Labs — типичные ошибки событийной игры

Каждая ошибка здесь встречается в реальных студенческих проектах — научитесь узнавать симптом раньше, чем откроете отладчик.

Небольшая коллекция типичных ошибок событийно-управляемых игр — каждая с симптомом и исправлением. Часть из них вы уже видели раньше в главе; здесь они собраны как справочник.

DEBUG LAB 1: СВОЯ ПРИВЯЗКА ВИДЖЕТА С BREAK ГЛУШИТ КЛАВИШИ ИГРЫ

focus_problem.py

```
root.bind("<Key>", on_key)

entry = tk.Entry(root)
entry.pack()
entry.bind("<Key>", lambda e: "break")  #
останавливает цепочку bindtags
entry.focus_set()
```

```
# Пока фокус в entry, нажатия цифр не долетают до on_key
# entry.bind(..., 'break') обрывает цепочку прямо здесь
```

Что видно на экране

Клавиатурное событие сначала идёт в сфокусированный виджет и дальше по цепочке `bindtags`: сам виджет → его класс → `root` → `"all"` (раздел 17.11). Если КАКОЙ-ТО обработчик на этом пути явно вернёт `"break"` — дальнейшие привязки в цепочке (включая `root.bind`) для этого события просто не вызовутся. Без такого `"break"` событие обычно продолжило бы путь и до `root` тоже — фокус сам по себе не блокирует привязку верхнего уровня.

ИСПРАВЛЕННЫЙ КОД


```
focus_fixed.py
```

```
root.bind("<Key>", on_key)

entry = tk.Entry(root)
entry.pack()
entry.focus_set()
# без своей <Key>-привязки событие доходит и
до root
```

DEBUG LAB 2: LAMBDA БЕЗ I=INDEKS — ПОЗДНЯЯ ПРИВЯЗКА

```
lambda_pozdnyaya_privyazka.py
```

```
for indeks in range(9):
    btn = tk.Button(frame, command=lambda:
attempt_move(indeks))
    btn.grid(row=indeks // 3, column=indeks %
3)
```

```
# Клик по ЛЮБОЙ из девяти кнопок ставит отметку в клетке
# потому что все lambda читают ОДНУ И ТУ ЖЕ переменную
```

Что видно на экране

К моменту клика цикл давно закончился, и `indeks` равен 8 для всех девяти замыканий разом — они не «запомнили» значение на момент создания, они ссылаются на переменную. Раздел 17.4 разбирал именно эту ошибку.

ИСПРАВЛЕННЫЙ КОД

lambda_fixed.py

```
for indeks in range(9):
    btn = tk.Button(frame, command=lambda
i=indeks: attempt_move(i))
    btn.grid(row=indeks // 3, column=indeks %
3)
```

DEBUG LAB 3: НАВЕДЕНИЕ ПУТАЕТ ПРЕВЬЮ
С НАСТОЯЩИМ ХОДОМ

hover_kak_hod.py

```
def on_cell_enter(self, index):

self.buttons[index].config(text=self.state.curren

def find_winner_from_widgets(self):
    board = [b["text"] for b in
self.buttons] # читает ТЕКСТ КНОПОК
    return find_winner(board)
```

Просто наведя курсор на три пустые клетки одной лин
можно получить объявление победы без единого клика.

Что видно на экране

Если проверка победителя читает состояние из `button["text"]`, а наведение временно пишет туда текст — превью становится неотличимо от настоящего хода. Раздел 17.18 объясняет, почему модель обязана быть источником истины.

ИСПРАВЛЕННЫЙ КОД

`hover_ne_hod.py`

```
def on_cell_enter(self, index):
    if self.state.game_over or
self.state.board[index]:
        return

self.buttons[index].config(text=self.state.current
fg=HOVER_COLOR)

def find_winner_call(self):
    return find_winner(self.state.board) #
читает МОДЕЛЬ, не кнопки
```

DEBUG LAB 4: ИГРОК ПЕРЕКЛЮЧАЕТСЯ ДО ПРОВЕРКИ ПОБЕДИТЕЛЯ

`smena_do_proverki.py`

```
state.board[index] = state.current_player
state.current_player = "0" if
state.current_player == "X" else "X"

winner, line = find_winner(state.board)
if winner:
    status_var.set(f"Победил игрок
{winner}!") # но current_player уже сменился
```

```
# Статус верный ('Победил X'), но если где-то дальше
# используется state.current_player как 'кто только'
```

Что видно на экране

Порядок операций важен: смена игрока должна произойти ПОСЛЕ проверки победителя и ТОЛЬКО если игра продолжается. Иначе любой код, который смотрит на `current_player` сразу после хода, увидит уже следующего игрока, а не автора выигрышного хода.

ИСПРАВЛЕННЫЙ КОД

`smena_posle_proverki.py`

```
state.board[index] = state.current_player
winner, line = find_winner(state.board)
if winner:
    state.winner = winner    # 'кто выиграл'
    зафиксировано ДО смены игрока
    state.game_over = True
elif not is_draw(state.board):
    state.current_player = "0" if
state.current_player == "X" else "X"
```

DEBUG LAB 5: НИЧЬЯ ПРОВЕРЯЕТСЯ РАНЬШЕ ПОБЕДЫ

```
nichya_do_pobedy.py
```

```
if all(state.board):
    status_var.set("Ничья!")
elif find_winner(state.board)[0]:
    status_var.set("Победа!")
```

```
# Девятый ход одновременно заполняет поле и выигрывает
# программа объявляет 'Ничья!', хотя есть явный победитель
```

Что видно на экране

Раздел 17.17 показывал именно этот сценарий. Проверка «заполнено ли поле» должна идти ПОСЛЕ проверки победителя, иначе выигрышный последний ход теряется за ложной ничьей.

ИСПРАВЛЕННЫЙ КОД

```
pobeda_do_nichyej.py
```

```
winner, line = find_winner(state.board)
if winner:
    status_var.set(f"Победил игрок {winner}!")
elif all(state.board):
    status_var.set("Ничья!")
```

DEBUG LAB 6: ХОД В ЗАНЯТУЮ КЛЕТКУ РАЗРЕШЁН

zanyataya_kletka.py

```
def attempt_move(index):
    state.board[index] =
    state.current_player  # нет проверки!
```

Повторный клик по уже занятой клетке перезаписывает
(или наоборот) – чужой ход стирается.

Что видно на экране

Валидация должна идти ПЕРВОЙ строкой, до любого изменения состояния — раздел 17.15 формулирует это как обязательный первый шаг алгоритма хода.

ИСПРАВЛЕННЫЙ КОД

zanyataya_kletka_fixed.py

```
def attempt_move(index):
    if state.game_over or state.board[index]:
        return
    state.board[index] = state.current_player
```

DEBUG LAB 7: ИГРОК ПЕРЕКЛЮЧАЕТСЯ ДАЖЕ ПРИ НЕВАЛИДНОМ КЛИКЕ

```
smena_pri_nevalidnom_klike.py
```

```
def attempt_move(index):
    if not state.board[index]:
        state.board[index] =
state.current_player
        state.current_player = "0" if
state.current_player == "X" else "X"    #
снаружи if!
```

```
# Клик по занятой клетке ничего не рисует —
# но ход всё равно передаётся сопернику. Игрок теряет
```

Что видно на экране

Строка смены игрока оказалась ВНЕ блока `if` — она выполняется независимо от того, был ли ход валидным. Раздел 17.15: невалидный ход не должен переключать игрока.

ИСПРАВЛЕННЫЙ КОД

```
smena_pri_nevalidnom_klike_fixed.py
```

```
def attempt_move(index):
    if state.board[index]:
        return
    state.board[index] = state.current_player
    state.current_player = "0" if
state.current_player == "X" else "X"
```

DEBUG LAB 8: НЕТ ПРОВЕРКИ GAME_OVER — ИГРА ПРОДОЛЖАЕТСЯ ПОСЛЕ ПОБЕДЫ

net_game_over.py

```
def attempt_move(index):
    if state.board[index]:
        return
    state.board[index] = state.current_player
    # ... проверка победителя, но НЕТ return/
    # guard в начале функции
```

После объявления победы X можно продолжать кликать
и даже 'перезаписать' исход партии дополнительными

Что видно на экране

Без явной проверки `state.game_over` в начале `attempt_move()` ничего не мешает продолжать партию после того, как она уже закончилась.

ИСПРАВЛЕННЫЙ КОД

game_over_guard.py

```
def attempt_move(index):
    if state.game_over or state.board[index]:
        return
    ...
```


DEBUG LAB 9: ОПЕЧАТКА В WINNING_LINES`opechatka_v_linii.py`

```

WINNING_LINES = (
    (0, 1, 2), (3, 4, 5), (6, 7, 8),
    (0, 3, 6), (1, 4, 7), (2, 5, 8),
    (0, 4, 6), (2, 4, 6),
    # первая диагональ должна быть (0, 4, 8)!
)

```

X ставит 0, 4 и 8 — три подряд по настоящей диагонали
 # но игра не объявляет победителя, потому что такой к

Что видно на экране

Опечатка тихая: код синтаксически верный и даже проходит поверхностный тест на «какая-то» диагональ. Только регрессионный тест, проверяющий ВСЕ восемь линий (раздел 17.28), поймал бы конкретно эту ошибку.

ИСПРАВЛЕННЫЙ КОД`linii_fixed.py`

```

WINNING_LINES = (
    (0, 1, 2), (3, 4, 5), (6, 7, 8),
    (0, 3, 6), (1, 4, 7), (2, 5, 8),
    (0, 4, 8), (2, 4, 6),
)

```

DEBUG LAB 10: СБРОСИЛИ ИНТЕРФЕЙС, НО НЕ МОДЕЛЬ

reset_tolko_ui.py

```
def new_round(self):
    for btn in self.buttons:
        btn.config(text="") # кнопки
очищены...
    # ...но self.state.board по-прежнему
    хранит старые X/O!
```

```
# Поле выглядит пустым, но следующий attempt_move()
# натывается на 'клетка уже занята' там, где на экран
```

Что видно на экране

Визуальный сброс — не то же самое, что сброс модели. Раздел 17.18: кнопки лишь отображают состояние; если очистить только их, модель продолжает врать об истинном состоянии партии.

ИСПРАВЛЕННЫЙ КОД

reset_model_i_ui.py

```
def new_round(self):
    self.state.board = [""] * 9
    self.state.current_player = "X"
    self.state.game_over = False
    self.render() # UI обновляется ИЗ
    модели, а не отдельно
```

DEBUG LAB 11: СБРОСИЛИ МОДЕЛЬ, НО НЕ ВЫЗВАЛИ RENDER()`reset_tolko_model.py`

```
def new_round(self):
    self.state.board = [""] * 9
    self.state.current_player = "X"
    self.state.game_over = False
    # забыли self.render() в конце!
```

Модель для нового раунда готова правильно —
 # но на экране всё ещё видны старые X и O из прошлой

Что видно на экране

Обратная сторона той же ошибки: без вызова `render()` изменения модели никогда не попадают на экран. Раздел 17.20 формулирует этот принцип: `render()` — место, которое рисует виджеты ИЗ модели (state), и его нужно вызывать явно после каждого изменения этой модели.

ИСПРАВЛЕННЫЙ КОД`reset_s_render.py`

```
def new_round(self):
    self.state.board = [""] * 9
    self.state.current_player = "X"
    self.state.game_over = False
    self.render()
```

DEBUG LAB 12: «НОВАЯ ИГРА» СЛУЧАЙНО ОБНУЛЯЕТ СЧЁТ МАТЧА

```
novaya_igra_obnulyaet_schet.py
```

```
def new_round(self):
    self.state.score_x = 0
    # это должно быть только в new_match()!
    self.state.score_o = 0
    self.state.board = [""] * 9
```

После каждой победы счёт тут же обнуляется —
турнирный счёт невозможно накопить.

Что видно на экране

Раздел 17.27: New Round и New Match — разные операции. Обнуление счёта принадлежит **ИСКЛЮЧИТЕЛЬНО** `new_match()`.

ИСПРАВЛЕННЫЙ КОД

```
new_round_bez_scheta.py
```

```
def new_round(self):
    self.state.board = [""] * 9
    # счёт (score_x/score_o/draws) здесь не трогаем
```

DEBUG LAB 13: ON_CELL_LEAVE СТИРАЕТ УЖЕ СДЕЛАННЫЙ ХОД

```
leave_stiraet_hod.py
```

```
def on_cell_leave(self, index):
    self.buttons[index].config(text="")    #
    наивно "убрать превью"
```

```
# После того как игрок делает настоящий ход и потом у
# отметка на клетке пропадает с экрана — хотя в модел
```

Что видно на экране

`on_cell_leave` не знает, было ли на клетке превью или настоящий ход — он просто стирает текст безусловно. Правильный подход: не гадать, а перерисовать из модели, которая точно знает истину.

ИСПРАВЛЕННЫЙ КОД

```
leave_fixed.py
```

```
def on_cell_leave(self, index):
    self.render()    # модель решает, что
    должно быть на клетке
```

DEBUG LAB 14: EVENT.WIDGET ИСПОЛЬЗУЕТСЯ КАК ИГРОВОЕ СОСТОЯНИЕ

widget_kak_state.py

```
def on_click(self, event):

    event.widget.config(text=self.state.current_player)
    # где здесь запись в board? Её нет —
    состояние живёт ТОЛЬКО в виджете
```

```
# find_winner(self.state.board) никогда не видит эти
# победа не определяется, потому что модель не меняла
```

Что видно на экране

`event.widget` — источник UI-события (раздел 17.8), а не домен игры. Запись хода обязана попасть в `state.board`, иначе вся доменная логика (поиск победителя, ничьей) работает вслепую.

ИСПРАВЛЕННЫЙ КОД

widget_i_model.py

```
def on_click(self, event, index):
    self.attempt_move(index) # меняет
    state.board, потом render()
```

DEBUG LAB 15: ГОЛЫЙ <BUTTON-1> ПОДМЕНЯЕТ СЕМАНТИЧЕСКОЕ ДЕЙСТВИЕ BUTTON

goloj_button1.py

```
btn.bind("<Button-1>", lambda e, i=index:
attempt_move(i))
# command= не задан вообще
```

```
# Обработчик связан только с конкретным событием мыши
# Основное действие Button больше не описано через co
```

Что видно на экране

Раздел 17.10: `command=` описывает основное действие Button, а встроенное поведение класса виджета решает, какие способы активации его вызывают. Конкретные клавиши зависят от вида Button, темы и платформы. Голая привязка `<Button-1>` знает только о клике мышью и обходит эту семантику — поэтому не используйте её вместо `command` по умолчанию.

ИСПРАВЛЕННЫЙ КОД

command_fixed.py

```
btn.config(command=lambda i=index:
attempt_move(i))
```

DEBUG LAB 16: TIME.SLEEP() В АНИМАЦИИ ПОБЕДЫ ЗАМОРАЖИВАЕТ ОКНО

sleep_v_animacii.py

```
import time

def pulse_winning_line(self):
    for _ in range(6):
        self.buttons[0].config(bg="green")
        time.sleep(0.15)
        self.buttons[0].config(bg="white")
        time.sleep(0.15)
```

Окно перестаёт отвечать почти на две секунды анимации
та же ошибка, что и в главе 16 (раздел 16.32).

Что видно на экране

`time.sleep()` блокирует событийный цикл целиком. Для анимации без блокировки нужен self-rescheduling через `after()` — раздел 17.30.

ИСПРАВЛЕННЫЙ КОД

after_fixed.py

```
def pulse_winning_line(self, tick=0):
    color = PULSE_BG if tick == 0 else WIN_BG
    for i in self.state.winning_line:
        self.buttons[i].config(bg=color)
    if tick == 0:
        self.root.after(400,
            self.pulse_winning_line, 1)
```


DEBUG LAB 17: ДОПОЛНИТЕЛЬНЫЙ BIND(..., ADD="+") ПОВЕРХ COMMAND= ВЫЗЫВАЕТ RESET ДВАЖДЫ

dublirovannaya_privyazka.py

```
btn = ttk.Button(controls, text="Новый
паунд", command=self.new_round)
btn.bind("<Button-1>", lambda e:
self.new_round(), add="+")
# теперь один клик запускает new_round() ЧЕРЕЗ
command И через bind — дважды подряд
```

Само по себе new_round() дважды подряд не ломает иг
но если бы это была, например, функция увеличения с

Что видно на экране

`add="+"` добавляет ЕЩЁ ОДИН обработчик, не заменяя существующий (раздел 17.12 упоминал это как «чуть глубже»). Если один и тот же клик уже обрабатывается через `command=`, дублирующая привязка через `bind(..., add="+")` на том же событии запускает логику повторно — используйте `add="+"` осознанно, только когда действительно нужно НЕСКОЛЬКО независимых обработчиков одного события.

ИСПРАВЛЕННЫЙ КОД

```
bez_dublirovaniya.py
```

```
btn = ttk.Button(controls, text="Новый  
раунд", command=self.new_round)
```

и всё — одной привязки достаточно

Практика: находим баг по симптому

Автоматическая проверка — для набора описанных симптомов выбираем правильную причину/исправление

Открыть практику → (<https://www.cartesianschool.org/practice/17-29/index.html>)

Visual effects и after()

Один короткий, неблокирующий эффект — победа должна быть замечена, а не устроить дискотеку.

Один ненавязчивый эффект — не фреймворк анимации

После победы клетки выигрышной линии один раз получают мягкий акцент и спокойно возвращаются к постоянной подсветке. На реальном окне это выглядит как краткий переход между accent-состоянием и базовой подсветкой победы; сайт не может показать видео-анимацию, поэтому ниже — реальные кадры этого перехода, а локальное приложение переключает их через `after()`. Различимых кадров всего два: третий снимок совпадает со вторым не случайно — эффект заканчивается именно на постоянной подсветке победы и дальше ничего не меняет.

Начало эффекта: выигрышная линия в мягком accent-цвете PULSE_BG

Реальный первый кадр: мягкий акцент PULSE_BG.

Через 400 мс линия возвращается к постоянной подсветке WIN_BG

Реальный второй кадр: постоянная подсветка WIN_BG.

После эффекта линия остаётся в постоянном цвете подсветки

Установившееся состояние: картинка совпадает со вторым кадром, потому что эффект на нём и заканчивается.

Эффект намеренно сдержанный — никакого резкого высококонтрастного мигания.

pulse_winning_line.py

```
def pulse_winning_line(self, tick=0):
    if not self.state.winning_line:
        return
    color = PULSE_BG if tick == 0 else WIN_BG
    for index in self.state.winning_line:
        self.buttons[index].config(bg=color)
    if tick == 0:
        self._pulse_job = self.root.after(400,
        self.pulse_winning_line, 1)
    else:
        self._pulse_job = None
```

Самопланирующийся `after()` — знакомый приём из главы 16

Та же техника `self-rescheduling`, что и в разделе 16.22: каждый вызов `pulse_winning_line` сам планирует свой следующий тик через `after()`, пока не достигнет предела — событийный цикл ни на миг не блокируется.

Отменяйте запланированный `after()` при новом раунде

Если игрок нажимает «Новый раунд» посреди анимации, недоделанный `after()` может сработать уже на пустом поле нового раунда. `cancel_pulse()` вызывает `after_cancel()` перед сбросом — та же дисциплина трёх состояний, что и у таймера в разделе 16.28.

`cancel_pulse.py`

```
def cancel_pulse(self):
    if self._pulse_job is not None:
        self.root.after_cancel(self._pulse_job)
        self._pulse_job = None
```

Никакого `time.sleep()`: один спокойный переход

Эффект не мигает серией: линия получает мягкий `accent-цвет` и через 400 мс остаётся в постоянной подсветке победы. Даже короткие эффекты должны быть ненавязчивыми и не отвлекать от статуса «Победил игрок X/O». Для продукта с формальными требованиями доступности параметры и возможность отключения эффектов проверяют отдельно.

Практика: ненавязчивая анимация победы

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-30/index.html>)

Tic-Tac-Toe Pro — полная программа и итоги главы

Та же игра, что и в разделе 17.6 — но теперь с моделью, правилами, тестами и архитектурой, которые выдержат рост проекта.

Итоговая программа

Финальное окно Tic-Tac-Toe Pro с игрой в процессе и счётом X:1 O:2 Ничьи:1

Реальное окно финальной версии — модель, правила, счёт, подсветка и клавиатура вместе. Ниже — структура всего файла целиком, а не одного фрагмента.

Файл целиком, самодостаточный и без невидимых зависимостей от других уроков:

[projects/](#)

[tkinter/tic-tac-toe/tic_tac_toe.py](#)

tic_tac_toe.py — структура

```
WINNING_LINES = (...)

def find_winner(board): ...
def is_draw(board): ...
def load_scores(): ...      # необязательное
                             сохранение счёта (17.27)
def save_scores(state): ...
```

```

@dataclass
class GameState:
    ...

class TicTacToeApp:
    def __init__(self, root, *,
persist_scores=False): ...
    def build_ui(self): ...
    def build_board(self, outer): ...
    def attempt_move(self, index): ...
    def on_cell_enter(self, index): ...
    def on_cell_leave(self, index): ...
    def on_key(self, event): ...
    def render(self): ...
    def new_round(self): ...
    def new_match(self): ...
    def pulse_winning_line(self, tick=0): ...
    def cancel_pulse(self): ...      # отменяет
запланированный after() (17.30)
    def on_close(self): ...

def main():
    root = tk.Tk()
    app = TicTacToeApp(root)
    root.mainloop()

if __name__ == "__main__":
    main()

```

Запустите игру у себя

python tic_tac_toe.py в терминале — либо кнопкой Run в VS Code или PyCharm. Сравните с tic_tac_toe_basic.py (раздел 17.6) — тот же результат для игрока, совершенно другая внутренняя архитектура.

Чек-лист готовой игры

MODEL

- поле (`board`) отделено от виджетов
- текущий игрок — явное поле состояния
- terminal state (`game_over/winner`) — явное поле, а не вывод из виджетов

RULES

- невалидные ходы отклоняются и не переключают игрока
- все 8 выигрышных линий протестированы для X и O
- ничья проверяется ПОСЛЕ победы

UI

- статус хода виден всегда
- победитель виден не только по цвету
- поле адаптивно к изменению размера окна
- клавиатура работает

АРХИТЕКТУРА

- callback-и короткие: валидация → модель → render
- чистые правила тестируются без окна
- render() задаёт канонический вид из модели; временные эффекты накладываются поверх

Мост к главе 18

Дальше: КУДА, а не только КТО и ЧТО

В этой игре события помогли нам понять, КТО действовал и ЧТО должно произойти. Дальше события мыши дадут нам ещё и **КООРДИНАТЫ** — `event.x` и `event.y`. В следующей главе координаты мыши станут основой рисования на Canvas.

Практика: Tic-Tac-Toe Pro целиком

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/17-31/index.html>)

Итоги главы 17

Что мы построили и чему научились

- Событие, `callback`, `command` и `binding` — четыре разных, связанных понятия; `callback` из `command` = обычно не получает `Event`; `callback`, зарегистрированный через `bind()`, получает его.
- `command` = описывает основное действие `Button`; `bind()` — явную реакцию на конкретное событие мыши или клавиатуры. Встроенные способы активации `Button` зависят от класса виджета и платформы, поэтому `command` и `bind` не следует считать взаимозаменяемыми.
- Игровое СОСТОЯНИЕ — не то же самое, что виджеты, которые его отображают: наведение мыши доказывает это нагляднее всего.
- `find_winner(board)` и `is_draw(board)` — чистые функции, тестируемые без единого открытого окна; порядок «сначала победа, потом ничья» — часть правил, а не деталь реализации.
- Мышь (`command`=) и клавиатура (`bind` на `root`) сходятся в одной функции `attempt_move()` — правила игры существуют только в одном месте.
- `render()` задаёт каноническое базовое отображение ИЗ модели. Наведение мыши и короткий акцент победы временно меняют представление поверх него, не меняя `GameState`; следующий `render()` снова полностью восстанавливает вид из модели.
- Ненавязчивые визуальные эффекты (`hover-превью`, подсветка линии, пульс победы) стоят на прочной архитектуре — они возможны именно потому, что модель отделена от вида.

Проект: приложение для рисования с Tkinter

Полноценный редактор рисования на Canvas: карандаш, фигуры, цвет, отмена действий, сохранение и загрузка — от первого прототипа до архитектуры уровня приложения.

18.1 Что мы строим? Начало работы

18.2 Экран и холст (Canvas)

18.3 Панель инструментов и параметры

18.4 Позиция мыши и рисуем линии

18.5 Квадраты, прямоугольники, круги и овалы

18.6 Выбираем размер и цвет

18.7 Первая полная программа

18.8 Canvas хранит объекты, а не пиксели

18.9 Система координат Canvas

18.10 Item ID: что возвращает create_*

18.11 Теги: группируем и выбираем элементы

18.12 Жизненный цикл жеста мыши

18.13 Состояние рисования

18.14 Карандаш: непрерывный штрих

18.15 Инструмент «Линия»

18.16 Геометрия прямоугольника

18.17 Геометрия овала

18.18 Живое превью: coords()

18.19 itemconfig, move, delete

18.20 Порядок наложения

18.21 Система цвета

18.22 Толщина кисти

18.23 Ластик

18.24 Отмена действий (Undo)

18.25 Повтор действий (Redo)

18.26 Очистка холста

18.27 Горячие клавиши

18.28 Строка состояния

18.29 Архитектура PaintApp

18.30 Сохранение и загрузка JSON

18.31 Debug Labs

18.32 Paint Pro — итоги главы

Приложение для рисования — объяснение

Так будет выглядеть готовый проект. Разберём путь к нему по шагам — от пустого холста до архитектуры уровня приложения.

Готовое приложение: холст с несколькими фигурами, панель инструментов с выбранным инструментом, палитра цветов, ползунок толщины и строка состояния

К концу главы мы соберём это приложение шаг за шагом.

Приложение для рисования — объяснение

Соберём собственный маленький «Paint»: холст, на котором можно рисовать линии, прямоугольники, овалы и произвольные каракули мышью, с выбором цвета и толщины. Начнём с того же плана, что и в главе 17 — сначала работающий прототип, потом более внимательный взгляд на то, как он устроен изнутри:

1. Холст для рисования (виджет `Canvas`);
2. Панель инструментов — кнопки выбора фигуры;
3. Реакция на движение и клики мыши;
4. Собственно рисование линий, прямоугольников, овалов;
5. Выбор толщины и цвета;

6. Кнопка очистки холста и свободное рисование.

Это даст честно работающую программу уже через несколько разделов — раздел 18.7. А дальше глава возвращается назад и разбирает Canvas куда внимательнее: что на самом деле хранит холст, как устроены координаты, что значит «отменить действие» и как из кучи глобальных переменных вырастает архитектура целого приложения.

Начинаем работу

nachalo.py

```
import tkinter as tk

root = tk.Tk()
root.title("Рисовалка")
```

Практика: начинаем собирать рисовалку

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-02/index.html>)

Настраиваем экран. Создаём холст

Виджет Canvas — прямоугольная область для рисования внутри окна Tkinter.

Настраиваем экран

nastrojka_ekrana.py

```
root.title("Рисовалка")
```

Создаём холст

Виджет `Canvas` — прямоугольная область, на которой можно рисовать линии, фигуры и текст по координатам, как в `Turtle` (главы 6–7), только внутри окна `Tkinter`:

```
sozdaem_holst.py
```

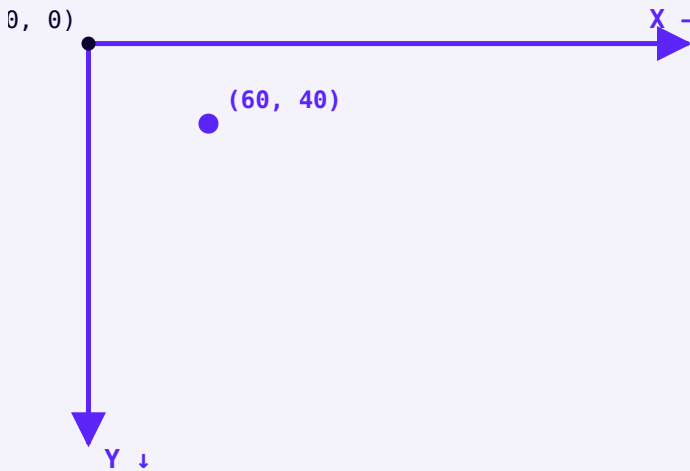
```
canvas = tk.Canvas(root, width=600, height=400,  
bg="white")  
canvas.pack()
```

Реальное окно: пустой белый холст 600×400 внутри окна `Tkinter`

Реальное окно: пустой холст сразу после запуска — рисовать пока нечем, но область уже есть.

Координаты `Canvas` — как у экрана, не как у `Turtle`

У `Canvas` точка (0, 0) — левый верхний угол, а не центр, как у экрана `Turtle`, и ось `Y` растёт **вниз**, а не вверх. Раздел 18.9 разберёт это подробнее и сравнит обе системы координат в лоб.



Точка (60, 40): 60 пикселей ВПРАВО и 40 пикселей ВНИЗ от левого верхнего угла холста.

Практика: создаём Canvas

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-02/index.html>)

Создаём первое меню (фигуры)

Панель инструментов с кнопками выбора фигуры — и переменные, хранящие текущее состояние.

Создаём первое меню (фигуры)

Панель инструментов — обычный `Frame` (глава 16) с кнопками, каждая из которых выбирает свою фигуру:

menu_figur.py

```

toolbar = tk.Frame(root)
toolbar.pack(side="top", fill="x")

def vybrat_figuru(figura):
    global tekuschaya_figura
    tekuschaya_figura = figura

tk.Button(toolbar, text="Линия", command=lambda:
vybrat_figuru("linia")).pack(side="left")
tk.Button(toolbar, text="Прямоугольник",
command=lambda:
vybrat_figuru("pryamougolnik")).pack(side="left")
tk.Button(toolbar, text="Овал", command=lambda:
vybrat_figuru("oval")).pack(side="left")

```

Реальное окно: панель инструментов с кнопками Линия, Прямоугольник, Овал над пустым холстом

Реальное окно: панель инструментов уже видна, хотя рисовать фигуры мы пока не умеем.

Заставляем параметры рисования работать!

Заведём глобальные переменные для текущей фигуры, цвета и толщины линии — их будут менять кнопки панели инструментов и читать функции рисования:

parametry.py

```

tekuschaya_figura = "linia"
tekuschij_cvet = "black"
tolschina = 3

```


Прототип: глобальные переменные — не финал архитектуры

Три глобальные переменные легко читать и объяснить, пока приложение маленькое — тот же выбор мы уже видели в первом прототипе крестиков-ноликов (глава 17). Раздел 18.13 покажет, во что это вырастает, когда инструментов, действий и истории становится больше.

Практика: панель инструментов и параметры рисования

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-03/index.html>)

Получаем позицию мыши

События мыши позволяют превратить движение курсора в настоящую линию на холсте.

Получаем позицию мыши

Как и в главе 17, реагировать на мышь помогает `.bind()` — только вместо клавиатурных событий здесь события мыши: `<Button-1>` (нажатие левой кнопки), `<B1-Motion>` (движение с зажатой левой кнопкой), `<ButtonRelease-1>` (кнопку отпустили).

`poziciya_myshi.py`

```
def pokazat_poziciyu(event):
    pozicia_label.config(text=f"x={event.x},
y={event.y}")

canvas.bind("<Motion>", pokazat_poziciyu)
```

`event.x` и `event.y` — координаты курсора относительно холста, в тех же единицах, что и у самого `Canvas`.

Рисуем линии

Три события вместе создают эффект «рисования от точки до точки»: запоминаем начало на `<Button-1>`, рисуем линию к текущей позиции на каждом `<B1-Motion>`:

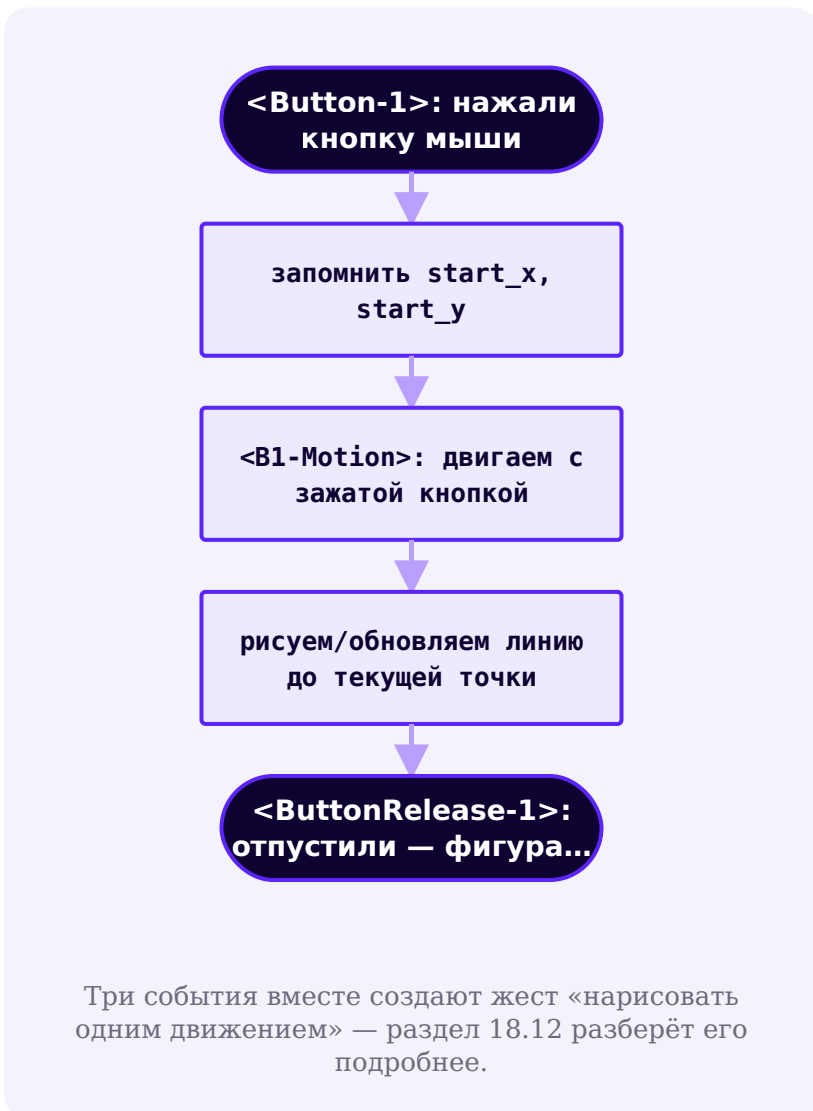
```
risuem_linii.py
```

```
start_x, start_y = None, None

def nachalo_risovaniya(event):
    global start_x, start_y
    start_x, start_y = event.x, event.y

def vo_vremya_risovaniya(event):
    canvas.create_line(start_x, start_y, event.x,
event.y, fill=tekuschij_cvet, width=tolschina)

canvas.bind("<Button-1>", nachalo_risovaniya)
canvas.bind("<B1-Motion>", vo_vremya_risovaniya)
```



Реальное окно: курсор нажат в точке начала линии, отметка на холсте ещё не появилась

Реальное окно: момент нажатия — `start_x/start_y` запомнены.

Реальное окно: готовая прямая линия между точкой нажатия и точкой отпущения

Реальное окно: кнопку отпустили — линия нарисована.

`create_line` — как `forward()` у `Turtle`, только по координатам

`canvas.create_line(x1, y1, x2, y2)` рисует прямую между двумя точками — концептуально то же самое, что `goto()` у `Turtle` из главы 6, только без «черепашки», которая сама туда едет.

Практика: позиция мыши и рисование линий

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-04/index.html>)

Квадраты и прямоугольники! Круги и овалы!

`Canvas` умеет рисовать готовые фигуры по двум углам — не только линии.

Прямоугольники и овалы рисуются похожим образом — `Canvas` умеет строить их сразу по двум противоположным углам, без ручного вычисления сторон:

```
pryamougolniki_ovaly.py
```

```
def vo_vremya_risovaniya(event):
    if tekuschaya_figura == "pryamougolnik":
        canvas.create_rectangle(
            start_x, start_y, event.x, event.y,
            outline=tekuschij_cvet, width=tolschina,
        )
    elif tekuschaya_figura == "oval":
        canvas.create_oval(
            start_x, start_y, event.x, event.y,
            outline=tekuschij_cvet, width=tolschina,
        )
```

Реальное окно: прямоугольник, нарисованный между точкой нажатия и точкой отпускания

Реальное окно: `create_rectangle()` по двум противоположным углам.

Реальное окно: овал, вписанный в область между точкой нажатия и точкой отпускания

Реальное окно: `create_oval()` по тем же двум углам — подробнее об этом в разделе 18.17.

Промежуточные фигуры множатся

Если рисовать так, как показано выше, при каждом движении мыши будет создаваться **новая** фигура поверх предыдущей — а не одна, растущая вместе с движением. В полной программе (раздел 18.7) эта проблема решена: предыдущая «черновая» фигура удаляется перед тем, как нарисовать новую. Раздел 18.18 показывает более аккуратный способ — обновлять один и тот же элемент через `coords()`, вместо того чтобы пересоздавать его заново.

Практика: прямоугольники и овалы

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-05/index.html>)

Выбираем размер! Очень много цветов!

Ползунок толщины и палитра цветов, построенная циклом по списку.

Выбираем размер!

Виджет `Scale` — ползунок для выбора числа в диапазоне, отлично подходит для толщины линии:

`vybor_razmera.py`

```
def vybrat_tolschinu(znachenie):
    global tolschina
    tolschina = int(znachenie)

    tolschina_scale = tk.Scale(toolbar, from_=1, to=10,
                                orient="horizontal", command=vybrat_tolschinu)
    tolschina_scale.set(3)
    tolschina_scale.pack(side="left")
```

command у Scale получает значение, а не событие

В отличие от `.bind()`, обработчик `Scale` получает готовое строковое значение ползунка напрямую — поэтому `vybrat_tolschinu(znachenie)` сразу превращает его в число через `int()` (глава 4).

Реальный холст: четыре линии одного цвета толщиной 1, 3, 8 и 16 пикселей, подписанные числами

Реальный холст: одна и та же линия при разной толщине — числа в коде превращаются в конкретную разницу на экране.

Очень много цветов!

Палитру цветов легко построить циклом (глава 10) по списку названий (глава 11) — вместо того, чтобы создавать каждую кнопку вручную:

vybor_cveta.py

```
cveta = ["black", "red", "blue", "green", "orange",
"purple"]
for cvet in cveta:
    tk.Button(toolbar, bg=cvet, width=2,
command=lambda c=cvet:
vybrat_cvet(c)).pack(side="left")
```

Реальное окно: панель инструментов с рядом цветных квадратных кнопок палитры

Реальное окно: шесть кнопок-квадратов, каждая цвета кнопки bg=cvet — палитра видна раньше, чем результат рисования ею.

lambda c=cvet — та же тонкость, что и в главе 17

Без c=cvet все кнопки палитры выбирали бы **последний** цвет из списка — та же самая ловушка с лямбдами внутри цикла, что мы уже разбирали в проекте «Крестики-нолики».

Практика: Scale и палитра цветов

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-06/index.html>)

Я закончил рисовать!

Первая полная программа

Последние штрихи первого прототипа — очистка холста и свободное рисование — и работающая программа целиком.

Я закончил рисовать!

Осталось добавить кнопку очистки холста и режим «свободного рисования» (карандаш) — для него точки рисуются на каждое движение мыши без привязки к начальной точке:

`ochistka_i_svobodno.py`

```
def ochistit_holst():
    canvas.delete("all")

def vo_vremya_risovaniya(event):
    if tekuschaya_figura == "svobodno":
        canvas.create_oval(
            event.x - tolschina, event.y - tolschina,
            event.x + tolschina, event.y + tolschina,
            fill=tekuschij_cvet,
            outline=tekuschij_cvet,
        )
    return
# ... остальные фигуры, как в разделе 18.5
```

Реальный холст: штрих из отдельных кружков с заметными промежутками между ними

Реальный холст: кружок на каждое движение мыши — при резком жесте между ними видны разрывы.

Отдельные кружки — не одна линия

Такой штрих рисует не непрерывную линию, а россыпь маленьких кружков-точек, которые иногда даже не соприкасаются друг с другом, если мышь двигалась быстро между двумя событиями `<B1-Motion>`. Раздел 18.14 показывает, как получить настоящий непрерывный штрих — соединяя каждую новую точку с предыдущей.

Первая полная программа

Полная версия первого прототипа, включая исправленную отрисовку «черновых» фигур во время перетаскивания мыши, доступна отдельным файлом:

[projects/
tkinter/paint-app/paint_app_basic.py](#)

Реальное окно: холст с несколькими линиями, прямоугольником и овалом разных цветов — первый прототип рисовалки

Реальное окно первого прототипа: несколько фигур разных цветов на одном холсте.

Честная, но не окончательная версия

Мы построили настоящую работающую программу — с холстом, фигурами, цветом и толщиной. Но состояние здесь по-прежнему живёт в глобальных переменных, «отменить» ничего нельзя, а «черновая» фигура во время перетаскивания просто удаляется и создаётся заново. Начиная со следующего раздела глава смотрит на Canvas куда внимательнее: что он на самом деле хранит, как отменить действие по-настоящему и как вырастить из этого прототипа приложение уровня PaintApp (раздел 18.32).

★★ Самостоятельная задача**Кнопка «Отменить»**

Сохраняйте `id` каждой нарисованной фигуры (`canvas.create_*` возвращает `id`) в список — и добавьте кнопку, удаляющую последнюю через `canvas.delete(id)`. Раздел 18.24 разберёт эту идею как настоящую архитектуру.

★★★ Задача повышенной сложности**Сохранение рисунка**

Изучите модуль `tkinter.filedialog` и попробуйте добавить кнопку «Сохранить» — как минимум сохраняющую список нарисованных фигур в текстовый файл (глава 15). Раздел 18.30 покажет полное решение через JSON.

Практика: первая полная программа

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-07/index.html>)

Итоги раздела

Что мы узнали

- Canvas — виджет Tkinter для рисования линий и фигур по координатам; координаты растут вправо и вниз от левого верхнего угла.
- `canvas.create_line/rectangle/oval(...)` рисуют готовые фигуры по координатам.
- События мыши (`<Button-1>` , `<B1-Motion>` , `<ButtonRelease-1>`) отслеживают клик, перетаскивание и отпускание кнопки мыши.
- Scale — ползунок для выбора числа в диапазоне.
- Палитра из нескольких похожих кнопок эффективнее строится циклом, чем вручную одна за другой.
- Это только первый прототип — дальше глава разбирает Canvas и архитектуру приложения куда внимательнее.

Canvas хранит объекты, а не пиксели

`create_line()` не просто рисует — он добавляет элемент в список, который Canvas помнит сам.

Canvas не «красит пиксели» — он хранит объекты

Первый прототип уже рисует, но легко представить его неправильно: будто `create_line()` просто закрашивает пиксели на картинке, как кисть в графическом редакторе. На самом деле Canvas устроен иначе — и от этого зависит почти всё остальное в этой главе: отмена действий, редактирование фигур, порядок наложения.

```
canvas.create_rectangle(...)
```



Canvas

сохраняет фигуру как элемент



```
item_id = 3
```

1384



экран

`create_*` не просто рисует — он добавляет ЭЛЕМЕНТ в список, который Canvas хранит сам.

Возвращённое число — **item ID**, идентификатор именно этого элемента внутри Canvas (раздел 18.10). Пока фигура не удалена явно, Canvas помнит о ней и может её перерисовать, переместить или изменить — даже если ваш код давно закончил с ней работать.

canvas : Canvas

```
item #1 = line (0, 0,...
100, 60)

item #2 = rectangle...
(10, 80, 120, 140)

item #3 = oval (30,...
20, 90, 70)
```

Canvas хранит СПИСОК элементов — примерно так, если бы у него были свои внутренние «объекты».

Реальное окно: холст с тремя нарисованными фигурами — линией, прямоугольником и овалом

То же самое реальное окно из раздела 18.7: три видимых фигуры — три хранимых элемента.

Три разных слоя — не путайте их

Слой	Что это
Модель приложения (Python)	Ваши переменные и структуры данных — то, что вы сами придумали
Элемент Canvas (item)	Запись внутри Canvas с типом, координатами и параметрами — создаётся через <code>create_*</code>
Пиксели на экране	Итоговая картинка, которую видит пользователь — результат отрисовки элемента

item ID — это НЕ идентификатор Python-объекта

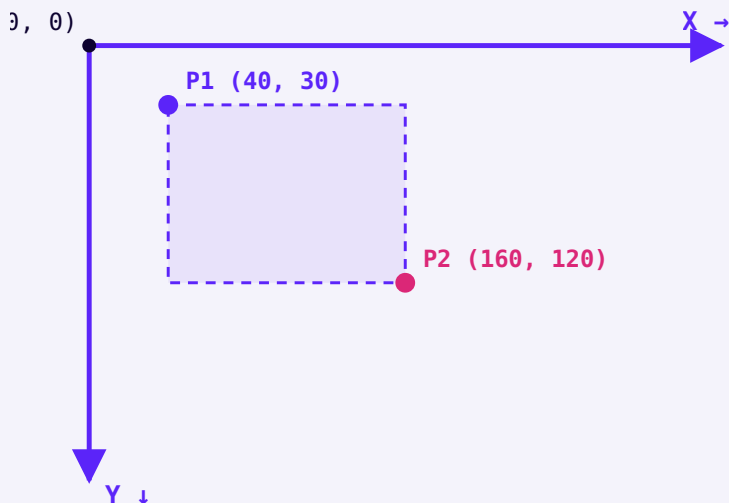
`item_id = canvas.create_rectangle(...)` возвращает обычное целое число — внутренний номер записи внутри Canvas, а не `id()` какого-то Python-объекта и не индекс в списке, который ведёте вы сами. Canvas выдаёт номера по своим правилам (обычно по возрастанию) — полагаться на конкретные значения не стоит, только на то, что каждый `item_id` уникален для одного элемента.

Система координат Canvas

Левый верхний угол — начало координат. X растёт вправо, Y — вниз. Не как у Turtle.

Координаты растут вправо и вниз

У Canvas точка `(0, 0)` — левый верхний угол. Первая координата (X) растёт вправо, вторая (Y) — вниз. Любая фигура задаётся через точки в этой системе:



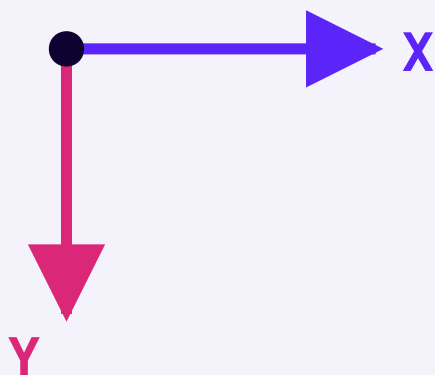
Прямоугольник по двум противоположным углам P1 и P2 — ровно то, что делает `create_rectangle(P1x, P1y, P2x, P2y)`.

Реальное окно: холст с подписанными точками и линией между ними, показывающими рост координат вправо и вниз

Реальное окно: две точки и линия между ними — движение мыши вправо увеличивает x, движение вниз увеличивает y.

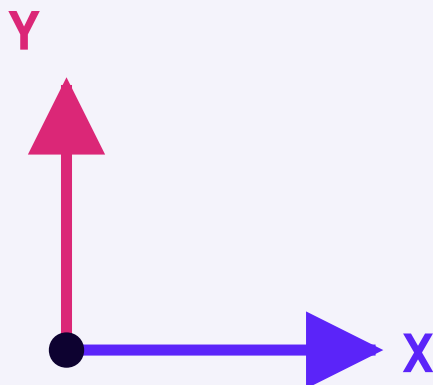
Turtle vs Canvas — сравним в лоб

Если до этой главы вы рисовали Turtle (главы 6–7), интуиция может подвести: там начало координат — центр экрана, а Y растёт вверх, как в математике на бумаге. У Canvas — иначе, и это не «более правильная» или «менее правильная» система, а просто другое соглашение, которое Tkinter унаследовал от оконных систем, где строки экрана нумеруются сверху вниз.

Canvas (Tkinter)

Начало координат — левый верхний угол, Y растёт
ВНИЗ

Turtle (главы 6-7)



Начало координат — центр экрана, Y растёт ВВЕРХ

event.x / event.y — те же координаты, что и у самого Canvas

Координаты события мыши (глава 17 — `event.x`, `event.y`) и координаты, которые принимает `create_*`, — одна и та же система: пиксели относительно левого верхнего угла холста. Именно поэтому `canvas.create_line(start_x, start_y, event.x, event.y)` работает без какого-либо пересчёта.

Практика: координаты Canvas

Автоматическая проверка — определяем направление движения курсора по координатам Y

Открыть практику → (<https://www.cartesianschool.org/practice/18-09/index.html>)

Item ID: что возвращает `create_*`

Число, которое `create_*` возвращает — не пустая формальность: это способ вернуться к фигуре позже.

`create_*` возвращает идентификатор

Каждый вызов `canvas.create_line/rectangle/oval(...)` возвращает целое число — **item ID** только что созданного элемента. Если его не сохранить, фигура всё равно нарисуеться, но обратиться к ней позже будет нечем:

```
item_id.py
```

```
item_id = canvas.create_rectangle(10, 10, 100, 60,
outline="blue")
print(item_id) # например, 1 — конкретное число
               # зависит от того, сколько элементов уже создано
```

Этот номер пригождается для трёх вещей, которые мы разберём дальше в главе:

Зачем помнить `item_id`

```
COORDS(ITEM_ID, ...)
```

изменить координаты существующего
элемента
раздел 18.18–18.19

ITEMCONFIG(ITEM_ID, ...)

изменить цвет/толщину без пересоздания
раздел 18.19

DELETE(ITEM_ID)

убрать именно этот элемент, не все
остальные
разделы 18.19, 18.24

item_id — не индекс списка и не id() объекта Python

Соблазн считать, что `item_id` — это позиция в списке («первая фигура — id 0», «вторая — id 1») ошибочен: Canvas выдаёт номера по своим правилам и не обязан начинать с нуля или идти без пропусков после удаления элементов. Не полагайтесь на конкретные значения — только на то, что каждый `item_id` уникально ссылается на один элемент.

Практика: item ID

Автоматическая проверка — храним фигуры по `item_id` как по ключу словаря, а не по индексу списка

Открыть практику → (<https://www.cartesianschool.org/practice/18-10/index.html>)

Теги: группируем и выбираем элементы

`item_id` указывает на один элемент. Тег может стоять сразу на многих — и работать с группой одной командой.

Тег — это имя, а не число

`item_id` указывает на ОДИН конкретный элемент. Но часто нужно обратиться сразу к нескольким — например, ко всем нарисованным фигурам сразу, не трогая временную «черновую» фигуру превью. Для этого у Canvas есть теги:

tegi.py

```
canvas.create_line(
    0, 0, 100, 60,
    fill="black", tags=("shape", "stroke"),
)
```

Один элемент может носить сразу несколько тегов. Дальше к тегу можно обращаться вместо `item_id` почти везде, где Canvas ожидает адрес элемента:

operacii_s_tegami.py

```
canvas.delete("preview")           # убрать все
элементы с тегом "preview"
canvas.itemconfig("shape", width=2)
# изменить толщину у ВСЕХ элементов с тегом "shape"
```

item_id против тега

ITEM_ID

указывает на **ОДИН** конкретный элемент
число, которое возвращает create_*

ТЕГ

может стоять сразу на многих элементах
строка, которую придумываете вы

тег "shape"

```
├ item 12  
├ item 13  
└ item 14
```

тег "preview"

```
└ item 15
```

В финальном приложении теги пригодятся дважды

Раздел 18.20 использует теги, чтобы показать порядок наложения фигур друг на друга. А черновой элемент превью (раздел 18.18) получает собственный тег "preview", чтобы его можно было убрать одной командой, не перебирая все остальные фигуры.

Практика: теги Canvas

Автоматическая проверка — собираем группы элементов по тегам, где один тег держит сразу несколько id

[Открыть практику →](https://www.cartesianschool.org/practice/18-11/index.html) (<https://www.cartesianschool.org/practice/18-11/index.html>)

Жизненный цикл жеста мыши

Рисование одной фигуры — это последовательность из трёх разных событий, а не одно.

Один жест, три события

Рисование одной фигуры мышью — это не одно событие, а последовательность из трёх разных типов, которые вместе образуют жест «нарисовать»:



Событие	Когда срабатывает
<code><Button-1></code>	ровно один раз — в момент нажатия левой кнопки
<code><B1-Motion></code>	много раз подряд, пока кнопка зажата и мышь движется
<code><ButtonRelease-1></code>	ровно один раз — в момент отпускания кнопки

<Motion> — это НЕ <B1-Motion>;

`<Motion>` срабатывает при ЛЮБОМ движении мыши над холстом — даже если ни одна кнопка не нажата.

`<B1-Motion>` — только при движении с зажатой ЛЕВОЙ кнопкой. Перепутать их — частая причина, по которой рисование начинается без клика (раздел 18.31, Debug Lab).

Все нужные координаты приходят через тот же объект `Event`, что и в главе 17 — `event.x`, `event.y`. `event.widget` тоже доступен, хотя для одного холста он обычно и так очевиден.

Клик без перетаскивания — не забыть про этот случай

Что если пользователь нажал и сразу отпустил кнопку в одной точке, не подвинув мышь? `<B1-Motion>` в этом случае не произойдёт ни разу — событие сработает, только если мышь действительно сдвинулась. Финальное приложение (раздел 18.32) явно проверяет это: если точка начала и точка отпускания почти совпадают, линия, прямоугольник или овал попросту не создаётся — вместо невидимой фигуры нулевого размера.

Практика: жест мыши press → drag → release

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-12/index.html>)

Состояние рисования

Инструмент, цвет, толщина и точка начала жеста — параметры, а не сами фигуры.

Что должна помнить программа между событиями

Три переменные из раздела 18.3 (`tekuschaya_figura`, `tekuschij_cvet`, `tolschina`) — честный прототип, но по мере роста приложения к ним добавляются точка начала жеста, последняя точка карандаша, id черновой фигуры превью — и глобальных переменных становится слишком много, чтобы держать их связь в голове.

Один объект вместо восьми отдельных переменных — те же данные, но с понятной границей.

Новое: Enum — заранее известный список вариантов

Пока инструмент хранится строкой, **любое** слово выглядит допустимым: `"pryamougolnik"`, `"pryamougolnick"`, `"квадрат"`. Python не возражает — он и не знает, какие варианты вы считаете правильными. Опечатка всплывёт гораздо позже и совсем не там: значение просто не совпадёт ни с одной веткой `if`, и нажатие мыши тихо перестанет что-либо рисовать.

Но у инструмента конечное, заранее известное число вариантов — карандаш, линия, прямоугольник, овал, ластик. Для таких случаев в Python есть **перечисление** (`Enum` из модуля `enum`): вы один раз выписываете все допустимые значения, и дальше пользуетесь только ими.

```
enum_vpervye.py
```

```
from enum import Enum

class Tool(Enum):
    PENCIL = "pencil"
    LINE = "line"
    RECTANGLE = "rectangle"

print(Tool.RECTANGLE)           # Tool.RECTANGLE
print(Tool.RECTANGLE.value)     # rectangle
print(Tool.RECTANGLE is Tool.PENCIL) # False
print(Tool.RECTANGL)           # AttributeError —
                                опечатка видна СРАЗУ
```

Три вещи, которые стоит запомнить про `Enum`:

- `Tool.RECTANGLE` — это **член** перечисления, отдельное значение, а не строка. Его удобно передавать и хранить;
- `.value` достаёт строку, которую вы за ним закрепили — она пригодится, когда фигуру нужно будет записать в JSON (раздел 18.30);
- члены сравнивают через `is`, а не `==`: член перечисления существует ровно в одном экземпляре, поэтому `tool is Tool.ERASER` — самая точная и быстрая проверка.

Главное отличие от строки: `Tool.RECTANGL` — это `AttributeError` при первом же обращении, а `"pryamougolnick"` — совершенно «нормальная» строка, о которой Python промолчит.

Собираем DrawingState

Теперь соберём оба инструмента вместе: Enum для инструмента и @dataclass (раздел 14.23) для самого объекта состояния — он избавляет от рукописного __init__ и сразу даёт понятный repr:

drawing_state.py

```
from dataclasses import dataclass
from enum import Enum

class Tool(Enum):
    PENCIL = "pencil"
    LINE = "line"
    RECTANGLE = "rectangle"
    OVAL = "oval"
    ERASER = "eraser"

@dataclass
class DrawingState:
    tool: Tool = Tool.PENCIL
    color: str = "#111827"
    width: int = 4
    start_x: float | None = None
    start_y: float | None = None
```

Enum вместо строк — когда это того стоит

tekuschaya_figura = "pryamougolnik" работает, но опечатку в строке Python не заметит, пока код не выполнится и фигура тихо не окажется «неизвестной». Tool.RECTANGLE — то же самое по смыслу, но опечатку поймает уже редактор или python -c "import module". Enum здесь оправдан ровно потому, что вариантов инструмента конечное известное число. Если вариант всего один или их список постоянно меняется — обычной строки достаточно, перечисление только добавит церемоний.

Реальное окно: кнопка «Карандаш» на панели инструментов выглядит нажатой (утоплена), остальные — обычные

Реальное окно: выбран Карандаш — кнопка визуально «утоплена», а не просто где-то в памяти хранится строка.

Реальное окно: кнопка «Линия» выглядит нажатой (утоплена), кнопка «Карандаш» вернулась в обычное состояние

Реальное окно: переключились на Линию — прежняя кнопка вернулась в обычный вид, новая «утоплена».

Выбранный инструмент виден, а не только хранится в переменной

Состояние — не только то, что помнит Python. Пользователь должен ВИДЕТЬ, какой инструмент выбран сейчас, не глядя в код. В финальном приложении `set_tool()` одновременно меняет `self.state.tool` И `relief` нужной кнопки — та же связка «модель → отражение на экране», что и `render()` в главе 17.

Путь этой главы

От глобальных переменных к PaintApp

V1 — ГЛОБАЛЬНЫЕ (18.3)

3 отдельные переменные
просто, но растёт бесконтрольно

V2 — DRAWINGSTATE

@dataclass

параметры инструмента в одном месте

V3 — PAINTAPP (18.29)

DrawingState + документ фигур

полная архитектура приложения

Практика: моделируем состояние рисования

Автоматическая проверка — выбор инструмента и параметров без Tkinter

Открыть практику → (<https://www.cartesianschool.org/practice/18-13/index.html>)

Карандаш: непрерывный штрих

Отдельные кружки создают разрывы. Соединяем каждую точку с предыдущей — и штрих становится одной линией.

Почему кружки на каждое движение — плохая идея

Раздел 18.7 рисовал свободный штрих кружками — по одному на каждое событие `<B1-Motion>`. Работает, но у этого подхода есть три проблемы:

- между кружками остаются разрывы, если мышь двигалась быстрее, чем приходили события;

- сама линия выглядит «бугристой», а не гладкой;
- кружок ничего не знает о том, где курсор был мгновение назад, — путь между двумя соседними событиями просто теряется.

Реальный холст: штрих из отдельных кружков с заметными промежутками между ними

То же самое реальное окно из раздела 18.7: рассыпь кружков вместо линии.

Лучшая модель: помнить предыдущую точку

Вместо кружка на каждое событие — соединяем каждую новую точку с предыдущей отрезком прямой. Отрезков много, но вместе они образуют одну непрерывную линию:

karandash.py

```
def on_drag(self, event):
    self.canvas.create_line(
        self.state.last_x, self.state.last_y, event.x,
        event.y,
        fill=self.state.color, width=self.state.width,
        capstyle=tk.ROUND,
    )
    self.state.last_x, self.state.last_y = event.x,
    event.y
```

Отсюда код — это методы класса PaintApp

Начиная с этого раздела примеры пишутся как методы: `def on_drag(self, event)`, `self.state`, `self.canvas`. Все они принадлежат классу `PaintApp`, который мы соберём целиком в разделе 18.29 — до тех пор достаточно читать `self.state.color` как «текущий цвет приложения».

Реальный холст: непрерывная плавная линия без разрывов, повторяющая движение мыши

Реальное окно финальной версии: та же самая рука, но линия без разрывов — потому что рисуются отрезки, а не отдельные точки.

Параметр	Зачем
<code>capstyle=tk.ROUND</code>	скругляет концы каждого отрезка — стыки между ними не выглядят «зубчатыми»
<code>width</code>	толщина штриха; чем она больше, тем заметнее была бы «зубчатость» стыков без <code>capstyle</code>

А как же `smooth=True`? Здесь он не сработает

У `create_line()` есть параметр `smooth=True` — Tk сглаживает ломаную, проводя через её точки кривую. Но сглаживать он умеет только **внутренние** точки одной линии, а каждый наш отрезок состоит ровно из ДВУХ точек — между ними нет ни одного угла. Добавить `smooth=True` в этот код можно, и Tk его молча примет, но на экране не изменится ровно ничего. Параметр стал бы полезен, если бы весь штрих был ОДНОЙ длинной ломаной — такой вариант описан ниже.

Куда это можно развить: один штрих — одна ломаная

Сейчас каждое движение мыши создаёт отдельный элемент Canvas. Можно вместо этого создать линию один раз на `<Button-1>` и на каждом движении дописывать к ней новую пару координат через `coords()` (раздел 18.18): тогда длинный штрих — это один элемент вместо сотни, а `smooth=True` наконец получает внутренние точки, которые действительно можно сгладить. В этой главе оставлен более простой вариант с отдельными отрезками — он короче и его легче отлаживать.

Практика: непрерывный штрих карандаша

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-14/index.html>)

Инструмент «Линия»

Точка → пунктирный черновик, который обновляется на лету → финальная сплошная линия.

Линия — не то же самое, что карандаш

У карандаша нет заранее известного конца: штрих завершается там, где пользователь отпустит кнопку, и линия строится из множества маленьких отрезков «на лету». У инструмента «Линия» всё иначе — есть ровно ОДНА прямая от точки нажатия до точки отпускания, и до самого конца жеста она — только черновик, который может ещё много раз поменяться.

```
instrument_linia.py
```

```
def on_press(self, event):
    self.state.start_x, self.state.start_y = event.x,
    event.y
    self.state.preview_id = self.canvas.create_line(
```



```

        event.x, event.y, event.x, event.y,
        fill=self.state.color, width=self.state.width,
dash=(4, 2),
    )

    def on_drag(self, event):
        self.canvas.coords(
            self.state.preview_id,
            self.state.start_x, self.state.start_y,
            event.x, event.y,
        )

    def on_release(self, event):
        self.canvas.delete(self.state.preview_id)
        self.canvas.create_line(
            self.state.start_x, self.state.start_y,
            event.x, event.y,
            fill=self.state.color, width=self.state.width,
        )

```

Реальное окно: пунктирная линия-превью тянется от точки нажатия к текущей позиции курсора

Реальное окно: во время перетаскивания — пунктирная черновая линия, ещё не финальная.

Реальное окно: сплошная финальная линия на месте бывшего пунктирного превью

Реальное окно: кнопку отпустили — сплошная линия заменила пунктирный черновик.

dash=(4, 2) — визуальный сигнал «это ещё не финал»

Пунктир у превью — не техническое требование Canvas, а сознательный выбор: пользователь должен на глаз отличать черновую фигуру от готовой. Раздел 18.18 обобщает этот приём «создать превью → обновлять coords() → зафиксировать на release» для прямоугольника и овала — с линией он работает точно так же.

Практика: инструмент «Линия» с превью

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

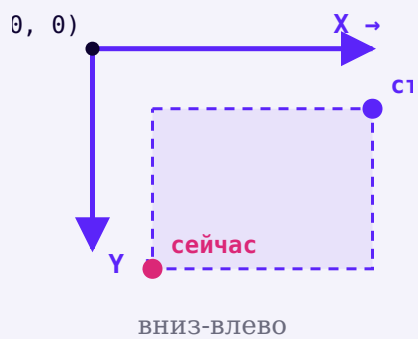
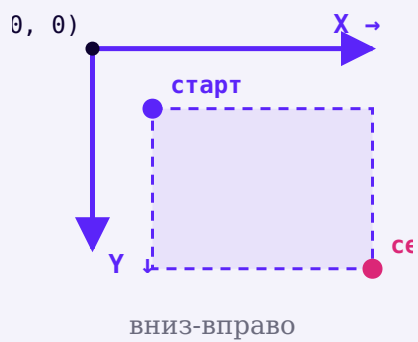
Открыть практику → (<https://www.cartesianschool.org/practice/18-15/index.html>)

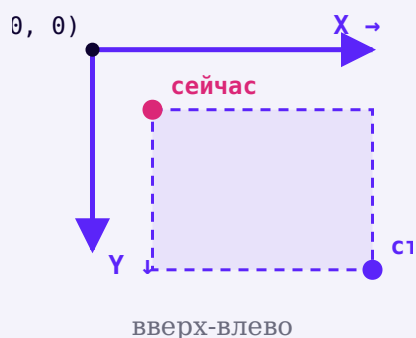
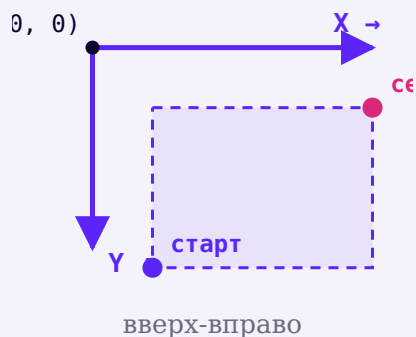
Геометрия прямоугольника

Мышь можно тянуть в любую из четырёх сторон — прямоугольник должен получиться одинаковым.

Два угла — четыре возможных направления

`canvas.create_rectangle(x1, y1, x2, y2)` рисует прямоугольник между двумя точками — но пользователь может тянуть мышь в любую сторону от точки старта: вниз-вправо, вниз-влево, вверх-вправо, вверх-влево. Прямоугольник получается один и тот же — Тк сам разбирается, какая координата больше:





Но если координаты нужны и вашему коду — например, чтобы сохранить фигуру в документе (раздел 18.13) с предсказуемым порядком точек, — стоит явно привести их к одному виду:

```
normalize_bounds.py
```

```
def normalize_bounds(x1, y1, x2, y2):

    """Приводит две произвольные противоположные точки к
    (left, top, right, bottom)."""
    return min(x1, x2), min(y1, y2), max(x1, x2),
    max(y1, y2)
```

Canvas это уже делает сам, а `normalize_bounds()` — для вашего кода

`create_rectangle()` прекрасно рисует прямоугольник, даже если $x1 > x2$ — сам Tk не путается в направлении. `normalize_bounds()` нужна не Canvas, а вашей модели документа: чтобы одна и та же фигура, нарисованная в любую сторону, сохранялась в JSON (раздел 18.30) в одном и том же предсказуемом виде.

Практика: `normalize_bounds()` для всех направлений перетаскивания

Автоматическая проверка — приводим координаты к (left, top, right, bottom) без Tkinter

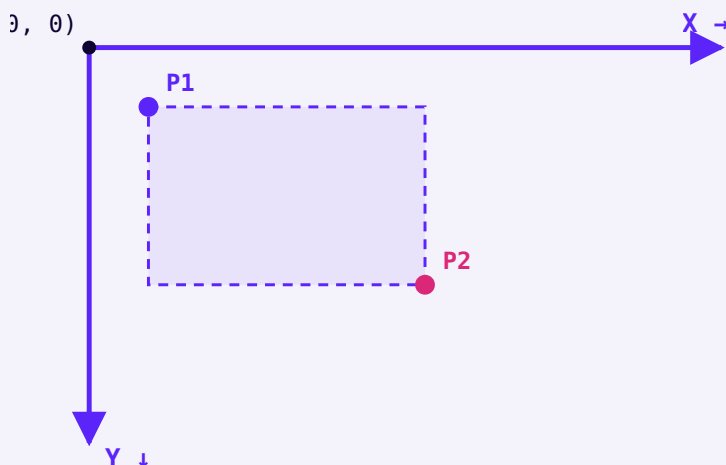
Открыть практику → (<https://www.cartesianschool.org/practice/18-16/index.html>)

Геометрия овала

`create_oval` принимает те же четыре координаты, что и `create_rectangle` — овал вписан в невидимые границы.

Овал — это прямоугольник с закруглёнными углами до предела

`canvas.create_oval(x1, y1, x2, y2)` принимает ровно те же четыре числа, что и `create_rectangle()` — но рисует не прямоугольник, а эллипс, **вписанный** в него. Прямоугольник при этом не рисуется — он просто определяет границы, внутри которых строится овал.



Пунктирный прямоугольник — это те же координаты, что вы передали в `create_oval()`; сам овал строится внутри него.

Реальное окно: овал, аккуратно вписанный в невидимую прямоугольную область между двумя точками

Реальное окно: `create_oval(x1, y1, x2, y2)` — овал касается всех четырёх сторон невидимого прямоугольника.

Не «центр + радиус», а именно ограничивающий прямоугольник

У Canvas нет отдельного `create_circle()`: круг — это просто овал, у которого ограничивающий прямоугольник — квадрат ($x2 - x1 == y2 - y1$).

Если приложению всё-таки нужны «центр + радиус» (например, для игровой логики), их несложно пересчитать в границы: $x1, y1 = cx - r, cy - r$ и $x2, y2 = cx + r, cy + r$.

Практика: ограничивающий прямоугольник овала

Автоматическая проверка — переводим координаты между «центр+радиус» и границами `create_oval`

Открыть практику → (<https://www.cartesianschool.org/practice/18-17/index.html>)

Живое превью: coords()

Один элемент создаётся один раз и много раз обновляется — вместо потока одnorазовых фигур.

Не пересоздавать фигуру на каждое движение

Наивный подход к превью — на каждом `<Bl-Motion>` удалять старую черновую фигуру и создавать новую. Работает, но заставляет Canvas без остановки выбрасывать и заново строить элементы — а заодно теряет любые индивидуальные настройки, которые вы, возможно, ставили только один раз.

**on_press: создать
превью ОДИН раз**



**on_drag:
coords(preview_id,...**



**on_drag:
coords(preview_id,...**



**on_drag:
coords(preview_id,...**



**on_release:
зафиксировать...**

Создать один раз → много раз обновить →
зафиксировать (CREATE ONCE → UPDATE MANY
TIMES → COMMIT) — один и тот же элемент, а не
поток одноразовых.

zhivoe_prevyu.py

```

def on_press(self, event):
    self.state.preview_id =
self.canvas.create_rectangle(
    event.x, event.y, event.x, event.y,
    outline=self.state.color,
    width=self.state.width, dash=(4, 2),
)

def on_drag(self, event):
    self.canvas.coords(
        self.state.preview_id,
        self.state.start_x, self.state.start_y,
        event.x, event.y,
    )

```

Реальное окно: пунктирный прямоугольник-превью на полпути перетаскивания

Реальное окно: один и тот же элемент `preview_id` меняет размер на каждый `coords()` — не создаётся заново.

Реальное окно: пунктирный овал-превью на полпути перетаскивания

Реальное окно: та же самая идея работает и для овала — меняется только форма, которую рисует `create_oval()`.

coords() принимает столько чисел, сколько нужно фигуре

Для линии и прямоугольника — четыре числа (две точки). Для многоточечной ломаной (например, растущего карандашного контура) — сколько угодно пар `x, y`. Главное — вызывать `coords()` с тем же `item_id`, что вернул исходный `create_*`.

Практика: живое превью через coords()

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-18/index.html>)

itemconfig, move, delete

Элемент Canvas не обязательно пересоздавать — его можно изменить на месте.

Четыре способа обратиться к уже нарисованному

Метод	Что меняет
<code>canvas.coords(id, ...)</code>	координаты элемента — форму и положение
<code>canvas.itemconfig(id, ...)</code>	визуальные параметры — цвет, толщину, стиль — без изменения формы
<code>canvas.move(id, dx, dy)</code>	сдвигает элемент НА dx, dy — а не задаёт абсолютную позицию
<code>canvas.delete(id)</code>	убирает элемент насовсем

redaktirovanie.py

```

item_id = canvas.create_rectangle(10, 10, 80, 60,
outline="black", width=2)

canvas.itemconfig(item_id, outline="blue", width=4)
# тот же прямоугольник, другой вид
canvas.move(item_id, 30, 0)
# сдвинуть на 30px вправо
canvas.coords(item_id, 10, 10, 150,
100) # изменить сами границы
canvas.delete(item_id)
# убрать совсем

```

move(id, dx, dy) — это смещение, а не координата

`canvas.move(item_id, 30, 0)` означает «сдвинь на 30 пикселей вправо от текущего положения», а не «помести на $x=30$ ». Вызвать `move()` дважды подряд с одними и теми же аргументами сдвинет элемент на общие 60 пикселей, а не оставит его на месте.

Все три способа — `coords()`, `itemconfig()`, `move()` — работают и по `item_id`, и по тегу (раздел 18.11): `canvas.itemconfig("preview", ...)` изменит сразу все элементы с этим тегом.

Практика: coords, itemconfig, move, delete

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-19/index.html>)

Порядок наложения

Позже нарисованный элемент оказывается сверху — если не сказать Canvas иначе.

Что нарисовано позже — оказывается сверху

Элементы Canvas хранятся не только как список, но и в определённом ПОРЯДКЕ — том самом, в котором их создали. Если два элемента перекрываются, верхним окажется тот, что был создан позже:

Реальное окно: три перекрывающихся фигуры — прямоугольник, овал и линия — овал поверх прямоугольника, линия поверх обоих

Реальное окно: порядок создания — прямоугольник, затем овал, затем линия. Линия оказалась сверху.

Этот порядок можно изменить явно — поднять элемент выше или опустить ниже остальных:

`poryadok_sloev.py`

```
canvas.tag_raise(rect_id)          # поднять
прямоугольник НАД всеми остальными
canvas.tag_lower(rect_id)          # опустить
прямоугольник ПОД всеми остальными
canvas.tag_raise(rect_id, oval_id) # поднять
прямоугольник только НАД конкретным овалом
```

Реальное окно: та же тройка фигур, но прямоугольник теперь поверх овала и линии

Реальное окно: после `tag_raise(rect_id)` — прямоугольник теперь выше остальных.

Небольшой намёк на слои графических редакторов

Настоящие графические редакторы строят на этой идее целую систему слоёв — со скрыванием, блокировкой, группами. Здесь мы не реализуем ничего из этого: `tag_raise` / `tag_lower` — лишь демонстрация того, что порядок наложения существует и им можно управлять, если понадобится.

Практика: порядок наложения элементов

Автоматическая проверка — предсказываем итоговый порядок после `tag_raise`/`tag_lower`

[Открыть практику](https://www.cartesianschool.org/practice/18-20/index.html) → (<https://www.cartesianschool.org/practice/18-20/index.html>)

Система цвета

Шесть быстрых цветов и `colorchooser` для любого другого — цвет нужно видеть, а не только читать hex-код.

Цвет нужно видеть, а не только читать в коде

Шесть цветов палитры финального приложения — не абстрактные строки, а конкретные оттенки:



Чёрный
#111827



Красный
#dc2626



Синий
#2563eb



Зелёный
#16a34a



Оранжевый
#f59e0b



Фиолетовый
#7c3aed

Реальное окно: ряд цветных кнопок палитры над холстом

Реальное окно: та же палитра — каждая кнопка красится
в свой `bg=hex_color`.

Свой цвет — через `colorchooser`

Готовая палитра — это только шесть вариантов. Модуль `tkinter.colorchooser` открывает нативный системный диалог выбора любого цвета:

custom_color.py

```

from tkinter import colorchooser

def choose_custom_color(self):
    _rgb, hex_color =
colorchooser.askcolor(color=self.state.color,
title="Выберите цвет")
    if hex_color is not None:
        self.set_color(hex_color)

```

СХЕМА ДИАЛОГА (НЕ НАСТОЯЩИЙ СКРИНШОТ)

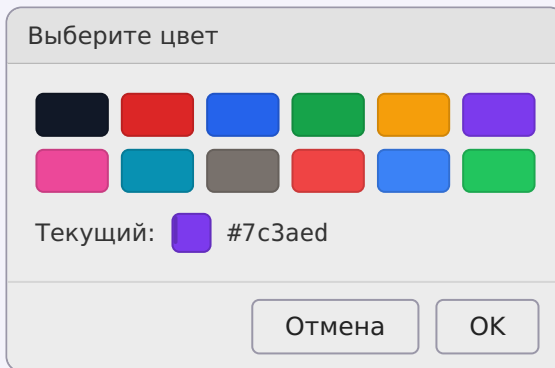


Схема нативного диалога colorchooser — под headless Xvfb он не рендерится предсказуемо без реального клика пользователя, поэтому здесь показана его структура, а не скриншот.

askcolor() может вернуть (None, None)

Если пользователь нажимает «Отмена», `askcolor()` возвращает `(None, None)`, а не какой-то цвет по умолчанию. Код обязан проверить `hex_color is not None` перед использованием — иначе следующий вызов, ожидающий строку вроде `"#7c3aed"`, получит `None` и упадёт с ошибкой.

Реальное окно: текущий цвет в панели инструментов изменился на нестандартный оттенок, выбранный через диалог

Реальное окно: результат выбора пользовательского цвета — сам диалог в кадр не попал, но эффект от него виден по-настоящему.

Практика: палитра и пользовательский цвет

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-21/index.html>)

Толщина кисти

Число в коде и толщина линии на экране — покажем разницу вживую, а не только опишем.

Толщина в коде — и толщина на экране

Четыре линии одного цвета, нарисованные с разной толщиной — разница видна сразу, без необходимости представлять её по числу:

Реальный холст: четыре горизонтальные линии одного цвета толщиной 1, 3, 8 и 16 пикселей с подписанными числами рядом

Реальный холст: `width=1, 3, 8, 16` — одна и та же линия, разная толщина.

`tolschina_kisti.py`

```
self.width_scale = tk.Scale(
    toolbar, from_=1, to=20, orient="horizontal",
    command=self.set_width,
)
self.width_scale.set(4)

def set_width(self, value):
    self.state.width = int(value)
```

command у Scale получает строку, а не Event

Как и в разделе 18.6, обработчик Scale получает готовое значение ползунка напрямую строкой — не объект Event, как у `.bind()`. Отсюда явный `int(value)`.

Толщина хранится в DrawingState, а не только в самом Scale

`self.state.width` — источник истины, который читают `on_press` / `on_drag`. `tk.Scale` — просто удобный виджет для его изменения, а не хранилище само по себе: та же логика «виджет отображает состояние, а не является им», что и в главе 17.

Практика: ползунок толщины кисти

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-22/index.html>)

Ластик

На Canvas «стереть» — не единственно возможное действие. Разберём, какой смысл выбран здесь и почему.

Что вообще значит «стереть» на Canvas?

В графическом редакторе с обычным растровым изображением ластик очевиден: закрасить пиксели цветом фона. Но Canvas хранит не пиксели, а элементы (раздел 18.8) — и у «стереть» здесь есть несколько разных честных значений:

Три честных смысла слова «ластик»

Закрасить
область
цветом
фона

→ ластик = карандаш цветом фона

Удалить
весь
элемент,
которого
коснулись

→ нужен `hit-test` – `find_overlapping()`

Удалить
только
часть
штриха
под
курсором

→ сложно: пришлось бы резать элемент на

Для этого курса выбран первый, самый предсказуемый вариант: ластик — это тот же карандаш (раздел 18.14), только цвет штриха принудительно равен цвету фона холста:

```
lastik.py
```

```
def on_drag(self, event):
    tool = self.state.tool
    if tool in (Tool.PENCIL, Tool.ERASER):
        color = CANVAS_BG if tool is Tool.ERASER else
self.state.color
        self.canvas.create_line(
            self.state.last_x, self.state.last_y,
event.x, event.y,
            fill=color, width=self.state.width,
capstyle=tk.ROUND,
        )
        self.state.last_x, self.state.last_y =
event.x, event.y
```

Реальное окно: холст с нарисованными фигурами до применения ластика

Реальное окно: холст с фигурами — до ластика.

Реальное окно: тот же холст с белой полосой на месте штриха ластика поверх фигур

Реальное окно: после ластика — белая полоса поверх части фигур, а не настоящее удаление затронутых элементов.

Это НЕ настоящее удаление затронутых элементов

Такой ластик рисует **НОВЫЙ** элемент цвета фона поверх старых — сами старые фигуры остаются в документе, просто визуально перекрыты. Если фон холста когда-нибудь изменится (например, вы добавите фоновое изображение), «стёртые» линии снова станут видны — это не баг, а прямое следствие выбранной модели.

Настоящее удаление — необязательное расширение

Метод `canvas.find_overlapping(x1, y1, x2, y2)` находит все элементы в прямоугольной области под курсором ластика — их можно удалить по-настоящему через `canvas.delete(item_id)`. Это честная, но заметно более сложная альтернатива: она требует решить, что делать, если ластик задел только **ЧАСТЬ** длинной фигуры. В этой главе она не реализована, но упомянута как реальное направление развития.

Практика: ластик как штрих цветом фона

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-23/index.html>)

Отмена действий (Undo)

Один пользовательский жест может состоять из многих элементов Canvas — отменяться он должен целиком.

Отмена по действиям, а не по элементам Canvas

Наивная реализация Undo — просто удалить последний созданный элемент Canvas. Проблема: один карандашный штрих состоит из **МНОГИХ** отдельных

отрезков (раздел 18.14) — «отменить» удалит только последний крошечный кусочек штриха, а не весь его целиком, чего пользователь точно не ожидает.

`istoriya_deystviy.py`

```
undo_stack = [
    [item_id_1, item_id_2, item_id_3], # один
    карандашный штрих – три отрезка
    [item_id_4], # один
    прямоугольник – один элемент
]
```

Финальное приложение решает это через документ (раздел 18.13, 18.29): каждое пользовательское действие — один или несколько объектов `Shape` — целиком добавляется в документ и целиком же может быть отменено:

`undo.py`

```
def _commit_action(self, shapes):
    self.document.extend(shapes)
    self.undo_stack.append(shapes)
    # Одно действие – даже если shapes из многих отрезков
    self.redo_stack.clear()
    self.render_document()

def undo(self):
    if not self.undo_stack:
        return
    shapes = self.undo_stack.pop()
    del self.document[len(self.document) -
len(shapes):]
    self.redo_stack.append(shapes)
    self.render_document()
```

Реальное окно: холст с несколькими фигурами, включая последний нарисованный прямоугольник

Реальное окно: до Ctrl+Z — виден весь рисунок, включая последнее действие.

Реальное окно: тот же холст без последнего прямоугольника, остальные фигуры на месте

Реальное окно: после Ctrl+Z — последнее ДЕЙСТВИЕ убрано целиком, предыдущие фигуры не тронуты.

render_document() — то же render(), что и в главе 17

После отмены документ меняется, а холст просто ПЕРЕРИСОВЫВАЕТСЯ из него заново — не точечное удаление конкретных элементов Canvas. Тот же принцип «модель меняют действия, Canvas только отображает текущее состояние», что и `render()` в игре «Крестики-нолики».

Практика: стек отмены по действиям

Автоматическая проверка — `undo` убирает ВСЕ элементы одного действия, а не один

Открыть практику → (<https://www.cartesianschool.org/practice/18-24/index.html>)

Повтор действий (Redo)

Redo — второй стек рядом с Undo. Новое действие после отмены должно его опустошить.

Redo — второй стек, а не «отмена отмены»

Раздел 18.24 отменяет действие, снимая его с `undo_stack`. Чтобы вернуть его назад, само действие нужно куда-то временно положить — во второй стек, `redo_stack`:

redo.py

```
def redo(self):
    if not self.redo_stack:
        return
    shapes = self.redo_stack.pop()
    self.document.extend(shapes)
    self.undo_stack.append(shapes)
    self.render_document()
```

Реальное окно: прямоугольник, ранее убраный через Undo, снова виден на холсте после Redo

Реальное окно: после Ctrl+Y — действие, отменённое в разделе 18.24, восстановлено.

Новое действие обязано опустошить redo_stack

Если после Undo пользователь рисует что-то НОВОЕ, старая ветка «которую можно было бы повторить» перестаёт иметь смысл — вернуть её означало бы применить действие к документу, который уже стал другим. Поэтому `_commit_action()` (раздел 18.24) всегда очищает `redo_stack`:

commit_ochischaet_redo.py

```
def _commit_action(self, shapes):
    self.document.extend(shapes)
    self.undo_stack.append(shapes)
    self.redo_stack.clear() # новое действие делает
    старую ветку redo недействительной
    self.render_document()
```

Забывтый `redo_stack.clear()` — незаметная, но реальная ошибка

Без этой строки Redo мог бы «воскресить» фигуру, которая никак не связана с текущим состоянием документа — потому что после Undo пользователь успел нарисовать что-то ещё, а старое отменённое действие всё ещё лежит в очереди «повторить». Раздел 18.31 разбирает этот случай как отдельный Debug Lab.

Практика: стек повтора и его очистка новым действием

Автоматическая проверка — redo восстанавливает действие, новое действие обнуляет redo

Открыть практику → (<https://www.cartesianschool.org/practice/18-25/index.html>)

Очистка холста

Необратимое действие заслуживает подтверждения — и обязано обнулить историю отмены вместе с документом.

«Очистить» — необратимое действие

Кнопка «Очистить холст» удаляет ВСЕ рисунок разом. В отличие от Undo, здесь нет предыдущего состояния, к которому можно легко вернуться, — если история отмены тоже очищается вместе с документом. Поэтому перед необратимым действием стоит спросить подтверждение, но только если есть что терять:


```
ochistka_holsta.py
```

```
def clear_canvas(self):
    if self.document and not messagebox.askyesno(
        "Очистить холст", "Удалить весь рисунок без
возможности отмены?"):
        return
    self.document.clear()
    self.undo_stack.clear()
    self.redo_stack.clear()
    self.render_document()
```

Очистка холста, забывшая очистить историю

Если удалить только фигуры (`self.document.clear()`), но не тронуть `undo_stack`, — `undo()` после очистки внешне не сделает НИЧЕГО (убирать из пустого документа нечего), зато положит устаревшее действие в `redo_stack`. И тогда `Ctrl+Y` вернёт на холст фигуры, которые пользователь только что удалил «Очистить». Раздел 18.31 разбирает этот случай отдельно: история обязана обнуляться вместе с документом, который она описывает.

Спрашиваем подтверждение не всегда

Если `self.document` уже пуст, спрашивать «точно очистить?» не о чем — лишнее диалоговое окно только раздражает. Проверка `if self.document and ...` пропускает подтверждение именно в этом случае.

Практика: безопасная очистка холста

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-26/index.html>)

Горячие клавиши

Ctrl+Z, Ctrl+Y и другие быстрые команды — привязаны на root, а не только на холст.

Быстрые команды с клавиатуры

Сочетание	Действие
Ctrl+Z	Undo — отменить последнее действие
Ctrl+Y	Redo — повторить отменённое действие
Ctrl+N	очистить холст (с подтверждением)
Ctrl+S	сохранить рисунок в JSON
Ctrl+O	открыть сохранённый рисунок

```
goryachie_klavishi.py
```

```
self.root.bind("<Control-z>", lambda _e: self.undo())
self.root.bind("<Control-y>", lambda _e: self.redo())
self.root.bind("<Control-s>", lambda _e:
self.save_document())
self.root.bind("<Control-o>", lambda _e:
self.load_document())
self.root.bind("<Control-n>", lambda _e:
self.clear_canvas())
```

Сочетания клавиш — не универсальны между операционными системами

`Ctrl` — стандарт для Windows и Linux. На macOS пользователи по привычке ждут `Cmd`, а не `Ctrl` — Tkinter не подменяет одно другим автоматически. Полная кроссплатформенная поддержка потребовала бы отдельно проверять `sys.platform` и привязывать оба варианта — эта глава этим не занимается, но важно не обещать того, чего код не делает.

Привязка на root, а не только на Canvas — вспомним главу 17

Горячие клавиши повешены на `self.root`, а не на `self.canvas` — они должны работать независимо от того, какой именно виджет окна сейчас в фокусе. Если бы в приложении было текстовое поле (например, для подписи рисунка), пришлось бы учитывать те же правила фокуса и `bindtags`, что разбирает раздел 17.11 — горячие клавиши не должны перехватывать ввод текста в поле, где пользователь печатает.

Практика: горячие клавиши приложения

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-27/index.html>)

Строка состояния

Инструмент, координаты, цвет и толщина — сразу видны, без необходимости лезть в код.

Одна строка — четыре факта сразу

Реальное окно: строка состояния внизу окна показывает инструмент, координаты курсора, текущий цвет и толщину

Реальное окно: строка состояния под холстом — инструмент, координаты, цвет, толщина.

stroka_sostoyaniya.py

```
def _update_status(self, x, y):
    coords = f"x={x} y={y}" if x is not None else
    "x=- y=-"
    self.status_var.set(
        f"Инструмент: {T00L_LABELS[self.state.tool]}
    | {coords} | "
        f"Цвет: {self.state.color} | Толщина:
    {self.state.width}"
    )
```

Строка обновляется в трёх разных местах: при смене инструмента, при движении мыши (<Motion>) и при выборе цвета или толщины — везде, где меняется хотя бы одно из четырёх значений.

Живое доказательство пользы event.x / event.y

Координаты курсора — не абстракция «где-то в коде»: строка состояния делает их видимыми в реальном времени, буквально на каждое движение мыши. Это тот же event.x / event.y , который используют on_press / on_drag для самого рисования.

Практика: строка состояния приложения

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-28/index.html>)

Архитектура PaintApp

app.root, состояние инструмента, документ фигур и Canvas — три разных слоя одного приложения.

app HAS-A root — тот же паттерн, что и в главах 16-17

Композиция против наследования

Возможно, но не нужно

```
class PaintApp(tk.Tk):
    def __init__(self):
        super().__init__()
        ...
```

Как в главах 16-17

```
class PaintApp:
    def __init__(self,
        root):
        self.root = root
        ...
```

Что использовать сегодня: Наследование от `tk.Tk` технически работает, но не согласуется с тем, как строились приложения в этой книге начиная с главы 16. `app.root` — тот же паттерн, что у калькулятора чаевых и крестиков-ноликов.

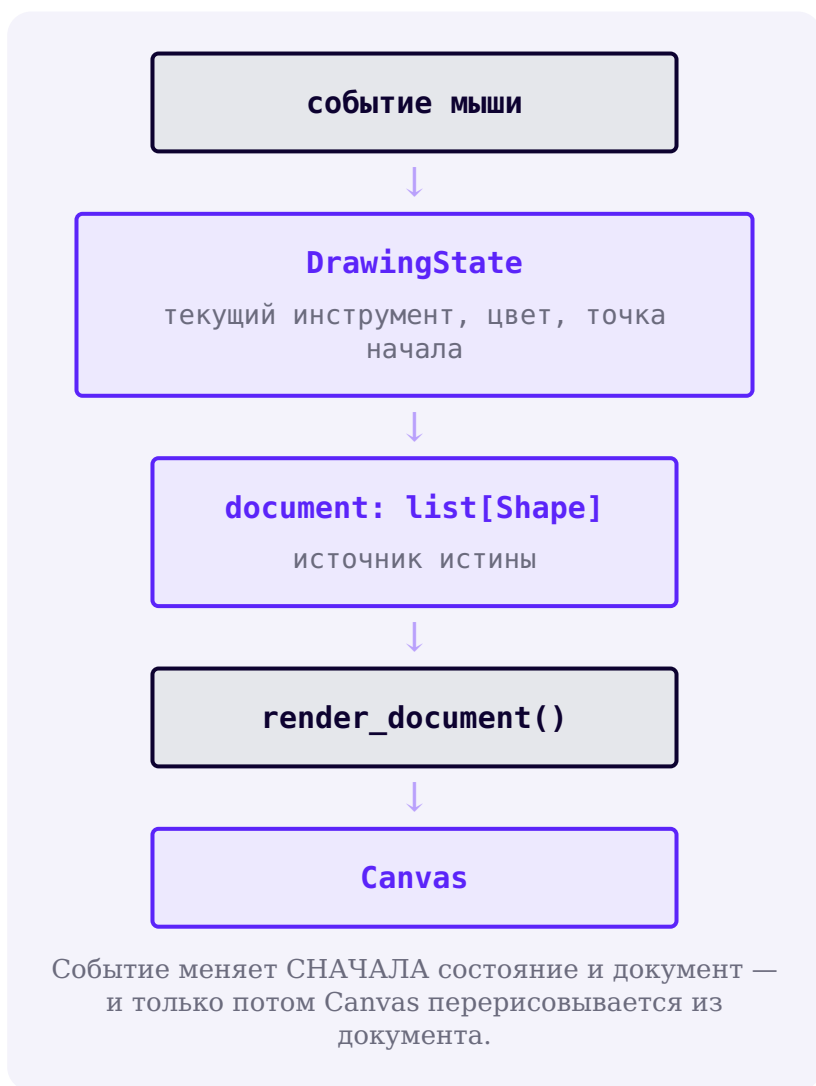
Три обязанности, три группы методов

app : PaintApp

```
root = Tk
state = DrawingState
document = list[Shape]
undo_stack = list[list[Shape]]
redo_stack = list[list[Shape]]
canvas = Canvas
```

app хранит виджеты, параметры инструмента И документ — но не путает их друг с другом.

Группа методов	За что отвечает
<code>build_ui / build_toolbar</code>	построение виджетов — один раз, при запуске
<code>on_press / on_drag / on_release</code>	события мыши → изменения state и document
<code>render_document</code>	документ → Canvas (единственное место, которое рисует по-настоящему)
<code>undo / redo / clear_canvas</code>	управление историей документа
<code>save_document / load_document</code>	документ ↔ файл на диске



UI-состояние, документ и отображение — три разных слоя

`DrawingState` — что выбрано ПРЯМО СЕЙЧАС (инструмент, цвет). `document` — что уже НАРИСОВАНО и что переживает смену инструмента.

`Canvas` — то, что видно на экране, производное от документа. Смешать первое со вторым — частая причина путаницы: например, хранить историю Undo внутри `DrawingState` вместо отдельного списка не позволило бы отменять действия, сделанные ДО того, как пользователь сменил инструмент.

Практика: собираем объектный граф PaintApp

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-29/index.html>)

Сохранение и загрузка JSON

Сохраняется не картинка, а документ фигур — `Canvas` умеет заново построить из него холст в любой момент.

Canvas не сохраняет рисунок как файл — документ сохраняет

Важная честная оговорка: `Canvas` сам по себе не умеет экспортировать себя в обычный PNG или JPEG — надёжный кроссплатформенный растровый экспорт требует дополнительных зависимостей и выходит за рамки этой главы. Вместо этого мы сохраняем ДОКУМЕНТ — список фигур, из которого `Canvas` и так строится через `render_document()` (раздел 18.29):

```
document: list[Shape]
```


save_document()



JSON-файл на диске

Сохранение.

JSON-файл на диске



load_document()



document: list[Shape]



render_document()



Canvas

Загрузка — обязательно заканчивается перерисовкой, а не просто чтением файла.

drawing.json

```
{
  "version": 1,
  "canvas": {"background": "#ffffff"},
  "items": [
    {
      "kind": "line",
      "coords": [34, 52, 180, 90],
      "color": "#2563eb",
      "width": 4
    },
    {
      "kind": "rectangle",
      "coords": [80, 120, 220, 210],
      "color": "#dc2626",
      "width": 3
    }
  ]
}
```

sohranenie_json.py

```
def save_document(self):
    path_str = filedialog.asksaveasfilename(
        defaultextension=".json",
        filetypes=[("Рисунок JSON", "*.json")]
    )
    if not path_str: # пользователь нажал "Отмена" —
        path_str == ""
    return
    data = {
        "version": 1,
        "canvas": {"background": CANVAS_BG},
        "items": [shape.to_dict() for shape in
self.document],
    }
    Path(path_str).write_text(json.dumps(data,
ensure_ascii=False, indent=2), encoding="utf-8")
```

Реальное окно: рисунок с несколькими фигурами непосредственно перед сохранением

Реальное окно: рисунок перед Ctrl+S — этот же документ окажется в JSON-файле.

Реальное окно: тот же самый рисунок, восстановленный после загрузки JSON в новом сеансе

Реальное окно: тот же рисунок после Ctrl+O в другом запуске программы — восстановлен из JSON, а не из памяти прежнего процесса.

Отмена диалога — не ошибка, а нормальный путь

И `asksaveasfilename()`, и `askopenfilename()` возвращают пустую строку `""`, если пользователь нажал «Отмена» — не `None` и не бросают исключение. Код обязан проверить это ДО того, как передавать путь в `Path(...): Path("")` — валидный, но бессмысленный объект, указывающий на текущую директорию, а не на файл, который выбрал пользователь.

А если файл повреждён?

Открывает файл пользователь — значит, выбрать он может что угодно: чужой JSON, картинку, недописанный при сбое файл, отредактированный вручную рисунок с опечаткой. Глава 15 (раздел 15.25) уже показывала, что `json.load()` сообщает об этом через `json.JSONDecodeError`. Здесь к нему добавляются ещё несколько возможностей: ключа `"items"` может не быть вовсе, у фигуры может не хватать поля, а цвет может оказаться строкой, которой Tk не знает.

Важно не просто «не упасть», а не потерять рисунок, над которым пользователь уже работал. Поэтому загрузка разбирает файл в **локальный** список и заменяет документ только тогда, когда разбор целиком удался:

zagruzka_bezопасно.py

```
def load_from_path(self, path):
    try:
        data =
        json.loads(path.read_text(encoding="utf-8"))
        shapes = [Shape.from_dict(item) for item in
        data["items"]]
        for shape in shapes:
            self.canvas.wininfo_rgb(shape.color) #
            цвет существует?
        except json.JSONDecodeError as exc:
            messagebox.showerror("Не удалось открыть
            файл", f"Это не корректный JSON.\n{exc}")
            return
        except (KeyError, TypeError, ValueError,
        tk.TclError) as exc:
            messagebox.showerror("Не удалось открыть
            файл", f"Это не сохранённый рисунок.\n{exc}")
            return

    # Разбор удался — только теперь меняем состояние
    приложения.
    self.document = shapes
    self.undo_stack.clear()
    self.redo_stack.clear()
    self.render_document()
```

Сначала разобрать, потом заменять — а не наоборот

Соблазн написать `self.document = [...]` сразу, а `try` навесить только на `json.loads()`. Тогда файл с неизвестным цветом пройдёт разбор, документ уже будет заменён — и упадёт `render_document()`. Итог: прежний рисунок потерян, история отмены очищена, а каждая следующая попытка нарисовать хоть что-нибудь снова падает, потому что перерисовка каждый раз натывается на ту же плохую фигуру. Проверка ДО присваивания стоит одной лишней переменной и спасает работу пользователя.

Проверять цвет умеет только холст

Shape сознательно ничего не знает о Tkinter (раздел 18.29), поэтому строку цвета он проверить не может — для него "#2563eb" и "не-цвет" одинаково похожи на цвет. А вот `canvas.winfo_rgb(color)` спрашивает у самого Tk и бросает `TclError`, если такого цвета нет. Разделение обязанностей сохранено: модель проверяет структуру, холст — то, что относится к рисованию.

Практика: сохранение и загрузка рисунка в JSON

Модуль `tkinter` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/18-30/index.html>)

Debug Labs — типичные ошибки рисовалки

Каждая ошибка здесь встречается в реальных студенческих проектах — научитесь узнавать симптом раньше, чем откроете отладчик.

Небольшая коллекция типичных ошибок рисовалок на Canvas — каждая с симптомом и исправлением. Часть из них вы уже видели раньше в главе; здесь они собраны как справочник.

DEBUG LAB 1: У РАСТЁТ ВВЕРХ, КАК У TURTLE

```
turtle_privychka.py
```

```
def on_drag(event):
    # 'привычная' логика: чем выше на экране,
    тем БОЛЬШЕ Y
    if event.y > start_y:
        status.config(text="Двигаемся
        ВВЕРХ") # неверно для Canvas!
```

```
# Курсор двигается вниз по экрану, а статус говорит "ВВЕРХ"
# логика была написана в терминах Turtle, а не Canvas
```

Что видно на экране

У Canvas Y растёт ВНИЗ (раздел 18.9) —

`event.y > start_y` означает «курсор ниже, чем был», а не выше. Перенос интуиции с Turtle без пересчёта знака — частая ошибка именно у тех, кто уже уверенно рисовал Turtle в главах 6–7.

ИСПРАВЛЕННЫЙ КОД

```
canvas_naправление.py
```

```
def on_drag(event):
    if event.y > start_y:
        status.config(text="Двигаемся ВНИЗ")
    # верно: Y растёт вниз
```

DEBUG LAB 2: НЕ ЗАПОМНИЛИ ТОЧКУ НАЧАЛА ЖЕСТА

```
zabyli_start.py
```

```
def on_press(event):
    pass # забыли сохранить event.x, event.y

def on_drag(event):
    canvas.create_line(start_x, start_y,
event.x, event.y) # start_x не определена
```

```
# NameError: name 'start_x' is not defined —
# программа падает на первом же движении мыши после н
```

Что видно на экране

`on_press()` обязан сохранить точку начала (раздел 18.12) — без неё `on_drag()` просто не может знать, откуда рисовать линию или превью фигуры.

ИСПРАВЛЕННЫЙ КОД

```
sohranili_start.py
```

```
def on_press(event):
    global start_x, start_y
    start_x, start_y = event.x, event.y
```

DEBUG LAB 3: <MOTION> ВМЕСТО <B1-MOTION>

```
lyuboe_dvizhenie.py
```

```
canvas.bind("<Motion>", on_drag)
# работает при ЛЮБОМ движении мыши
```

```
# Линия рисуется, даже если кнопка мыши вообще не нажата
# достаточно просто провести курсором над холстом.
```

Что видно на экране

`<Motion>` реагирует на любое движение мыши над виджетом. Для рисования только во время перетаскивания нужен именно `<B1-Motion>` — раздел 18.12 объясняет разницу.

ИСПРАВЛЕННЫЙ КОД

```
b1_motion.py
```

```
canvas.bind("<B1-Motion>", on_drag)
# только при зажатой левой кнопке
```

DEBUG LAB 4: ТЫСЯЧИ НЕСВЯЗАННЫХ ТОЧЕК ВМЕСТО ЛИНИИ


```
tochki_vmesto_linii.py
```

```
def on_drag(event):
    canvas.create_oval(
        event.x - 2, event.y - 2, event.x +
        2, event.y + 2,
        fill="black",
    )
```

```
# Быстрое движение мыши оставляет заметные разрывы ме
# штрих выглядит рваным, а не гладким.
```

Что видно на экране

Кружок на каждое отдельное событие не учитывает путь МЕЖДУ двумя последовательными точками. Раздел 18.14 показывает решение — соединять новую точку с предыдущей через `create_line()`.

ИСПРАВЛЕННЫЙ КОД

```
linii_mezhdu_tochkami.py
```

```
def on_drag(event):
    canvas.create_line(last_x, last_y,
        event.x, event.y, width=3, capstyle=tk.ROUND)

# last_x, last_y обновляются после каждого
отрезка
```

DEBUG LAB 5: ПРЕВЬЮ ПЕРЕСОЗДАЁТСЯ НА КАЖДЫЙ MOTION БЕЗ НЕОБХОДИМОСТИ

```
peresozdanie_prevyu.py
```

```
def on_drag(event):
    canvas.delete(preview_id)
    preview_id =
    canvas.create_rectangle(start_x, start_y,
    event.x, event.y)
```

```
# Работает, но на каждое из десятков событий Motion b
# Canvas выбрасывает и заново строит один и тот же эл
```

Что видно на экране

Удалить и создать заново — не ошибка в смысле «сломано», но лишняя работа: у Canvas уже есть `coords()` именно для обновления существующего элемента (раздел 18.18) — без пересоздания.

ИСПРАВЛЕННЫЙ КОД

```
coords_vmesto_peresozdaniya.py
```

```
def on_drag(event):
    canvas.coords(preview_id, start_x,
    start_y, event.x, event.y)
```

DEBUG LAB 6: ПОТЕРЯЛИ ID ПРЕВЬЮ-ФИГУРЫ

poteryan_id.py

```
def on_press(event):
    canvas.create_rectangle(event.x, event.y,
event.x, event.y, dash=(4, 2))
    # id не сохранён – preview_id нигде не
    появился

def on_drag(event):
    canvas.coords(preview_id, ...) #
    NameError: preview_id не определена
```

NameError на первом же движении мыши –
ссылаться не на что, id черновой фигуры потерян.

Что видно на экране

`create_*`() возвращает `item_id` (раздел 18.10), но только если его СОХРАНИТЬ — иначе фигура нарисована, но обратиться к ней позже будет нечем.

ИСПРАВЛЕННЫЙ КОД

sohranili_id.py

```
def on_press(event):
    global preview_id
    preview_id =
    canvas.create_rectangle(event.x, event.y,
event.x, event.y, dash=(4, 2))
```

DEBUG LAB 7: ПРЯМОУГОЛЬНИК «ПЕРЕВОРАЧИВАЕТСЯ» ПРИ ПЕРЕТАСКИВАНИИ ВВЕРХ-ВЛЕВО

```
ne_normalizovano.py
```

```
def get_bounds():
    return start_x, start_y, current_x,
    current_y # без учёта направления

left, top, right, bottom = get_bounds()
if left > right:
    raise ValueError("это невозможно") # но
    перетаскивание вверх-влево именно это и даёт!
```

```
# Приложение падает с ValueError, если пользователь н
# перетаскивание из правого нижнего угла будущей фигу
```

Что видно на экране

Мышь можно тянуть в любую из четырёх сторон (раздел 18.16) — код, ожидающий `left <= right` без предварительной нормализации, работает только для одного из четырёх направлений.

ИСПРАВЛЕННЫЙ КОД

```
normalizovano.py
```

```
left, top, right, bottom =
normalize_bounds(start_x, start_y, current_x,
current_y)
# теперь left <= right и top <= bottom
гарантированно, независимо от направления
```

DEBUG LAB 8: ITEM_ID ИСПОЛЬЗОВАН КАК ИНДЕКС СПИСКА

id_kak_indeks.py

```
shapes = []
item_id = canvas.create_rectangle(10, 10, 50,
50)
shapes.append("rectangle")
print(shapes[item_id]) # IndexError, если
item_id больше len(shapes) - 1
```

```
# IndexError: list index out of range –
# item_id не совпадает с порядковым номером в списке
```

Что видно на экране

`item_id` — внутренний номер Canvas (раздел 18.10), а не позиция в вашем собственном списке фигур. Использовать его как индекс `shapes[item_id]` совпадёт только случайно.

ИСПРАВЛЕННЫЙ КОД

id_kak_klyuch_slovara.py

```
shapes_by_id = {}
item_id = canvas.create_rectangle(10, 10, 50,
50)
shapes_by_id[item_id] = "rectangle" #
словарь – id как КЛЮЧ, а не индекс
```

DEBUG LAB 9: UNDO УБИРАЕТ ТОЛЬКО ПОСЛЕДНИЙ ОТРЕЗОК ШТРИХА, А НЕ ВСЕШТРИХ

chastichnaya_otmena.py

```
def undo(self):
    item_id = self.undo_stack.pop()    #
    предполагает ОДИН id на действие
    self.canvas.delete(item_id)
```

После Ctrl+Z длинный карандашный штрих теряет только
последний маленький отрезок, а не исчезает целиком.

Что видно на экране

Один карандашный штрих — это МНОГО отрезков `create_line()` (раздел 18.14), а не один элемент. Undo обязан хранить список `id` (или фигур) на КАЖДОЕ действие целиком, а не один `id` (раздел 18.24).

ИСПРАВЛЕННЫЙ КОД

otmena_celym_deystviem.py

```
def undo(self):
    shapes = self.undo_stack.pop()    # список
    фигур ОДНОГО действия
    del self.document[len(self.document) -
    len(shapes):]
    self.render_document()
```

DEBUG LAB 10: REDO ВОСКРЕШАЕТ НЕСВЯЗАННОЕ ДЕЙСТВИЕ

```
redo_bez_ochistki.py
```

```
def _commit_action(self, shapes):
    self.document.extend(shapes)
    self.undo_stack.append(shapes)
    # забыли self.redo_stack.clear()
```

```
# Undo, затем новый рисунок, затем Redo —
# и на холсте внезапно появляется фигура, никак не св
```

Что видно на экране

Если после Undo нарисовать что-то новое, старая ветка redo больше не соответствует текущему документу. Раздел 18.25 требует явно очищать redo_stack при любом новом действии.

ИСПРАВЛЕННЫЙ КОД

```
redo_s_ochistkoj.py
```

```
def _commit_action(self, shapes):
    self.document.extend(shapes)
    self.undo_stack.append(shapes)
    self.redo_stack.clear()
```

DEBUG LAB 11: ОЧИСТИЛИ ХОЛСТ, НО НЕ ИСТОРИЮ

```
ochistka_bez_istorii.py
```

```
def clear_canvas(self):
    self.canvas.delete("all")
    self.document.clear()
    # undo_stack и redo_stack не тронуты!
```

```
# После 'Очистить' нажатие Ctrl+Z не делает ничего ви
# а следующий Ctrl+Y возвращает на холст фигуры, кото
# пользователь только что удалил кнопкой 'Очистить'.
```

Что видно на экране

Ошибка тихая: `del document[len(document) - len(shapes):]` на пустом документе — это `del document[-3:]`, что ничего не удаляет и ничего не ломает. Зато отменённое «действие» уезжает в `redo_stack`, и следующий `Ctrl+Y` возвращает на холст фигуры, стёртые кнопкой «Очистить». История описывает документ, которого больше нет: как только `document` очищается, `undo_stack` и `redo_stack` обязаны очиститься вместе с ним (раздел 18.26).

ИСПРАВЛЕННЫЙ КОД


```
ochistka_s_istoriej.py
```

```
def clear_canvas(self):
    self.document.clear()
    self.undo_stack.clear()
    self.redo_stack.clear()
    self.render_document()
```

DEBUG LAB 12: ОТМЕНА COLORCHOOSER ЗАПИСЫВАЕТ NONE КАК ЦВЕТ

```
otmena_colorchooser.py
```

```
def choose_custom_color(self):
    _rgb, hex_color = colorchooser.askcolor()
    self.set_color(hex_color)
    # не проверили hex_color на None!
```

```
# Пользователь нажал 'Отмена' в диалоге
# следующая же попытка нарисовать линию
```

```
падает: fill=
```

Что видно на экране

`askcolor()` возвращает `(None, None)` при отмене (раздел 18.21) — присвоить это как текущий цвет без проверки значит сломать рисование при следующей же попытке.

ИСПРАВЛЕННЫЙ КОД

```
proverka_otmeny.py
```

```
def choose_custom_color(self):
    _rgb, hex_color = colorchooser.askcolor()
    if hex_color is not None:
        self.set_color(hex_color)
```

DEBUG LAB 13: ОТМЕНА ДИАЛОГА СОХРАНЕНИЯ — PATH('') ВМЕСТО ВЫХОДА

```
otmena_sohraneniya.py
```

```
def save_document(self):
    path_str = filedialog.asksaveasfilename()
    Path(path_str).write_text(...) # не
    проверили пустую строку!
```

```
# Пользователь нажал 'Отмена' в диалоге сохранения —
# программа пытается записать файл по пути текущей ди...
```

Что видно на экране

`asksaveasfilename()` / `askopenfilename()`
возвращают пустую строку при отмене, а не `None`
и не исключение (раздел 18.30) — код обязан
проверить это перед использованием пути.

ИСПРАВЛЕННЫЙ КОД

```
proverka_otmeny_faila.py
```

```
def save_document(self):
    path_str = filedialog.asksaveasfilename()
    if not path_str:
        return
    Path(path_str).write_text(...)
```

DEBUG LAB 14: JSON ЗАГРУЖЕН, НО CANVAS НЕ ПЕРЕРИСОВАН

```
zagruzka_bez_render.py
```

```
def load_from_path(self, path):
    data =
    json.loads(path.read_text(encoding="utf-8"))
    self.document = [Shape.from_dict(item)
    for item in data["items"]]
    # забыли self.render_document()!
```

```
# self.document правильно заполнен новыми фигурами,
# но на экране всё ещё виден старый рисунок — или пусто
```

Что видно на экране

Как и в главе 17: изменение модели без вызова отрисовки не долетает до экрана.

`load_from_path()` обязан заканчиваться `render_document()` (раздел 18.30).

ИСПРАВЛЕННЫЙ КОД

zagruzka_s_render.py

```
def load_from_path(self, path):
    data =
    json.loads(path.read_text(encoding="utf-8"))
    shapes = [Shape.from_dict(item) for item
in data["items"]]
    self.document = shapes
    self.undo_stack.clear()
    self.redo_stack.clear()
    self.render_document()
# без этой строки экран не узнает о загрузке
# (полная версия с обработкой
повреждённого файла – в разделе 18.30)
```

DEBUG LAB 16: ПОВРЕЖДЁННЫЙ ФАЙЛ СТИРАЕТ ТЕКУЩИЙ РИСУНОК

zagruzka_bez_proverki.py

```
def load_from_path(self, path):
    data =
    json.loads(path.read_text(encoding="utf-8"))
    self.document = [Shape.from_dict(i) for i
in data["items"]] # уже заменили!
    self.undo_stack.clear()
    self.render_document() # ...и только
здесь падаем на плохом цвете
```

```
# Открыли чужой JSON — рисунок, над которым работали
# Ctrl+Z его не возвращает, а каждая новая линия снова
# _tkinter.TclError: unknown color name "not-a-color"
```

Что видно на экране

Документ заменён ДО того, как стало известно, что файл рисуется. Разбирайте файл в локальную переменную и присваивайте `self.document` только после успеха (раздел 18.30) — иначе одна попытка открыть не тот файл уносит и рисунок, и историю, и возможность рисовать дальше.

ИСПРАВЛЕННЫЙ КОД

zagruzka_s_proverkoj.py

```
def load_from_path(self, path):
    try:
        data =
        json.loads(path.read_text(encoding="utf-8"))
        shapes = [Shape.from_dict(i) for i in
        data["items"]]
        for shape in shapes:
            self.canvas.wininfo_rgb(shape.color)
        except (json.JSONDecodeError, KeyError,
        TypeError, ValueError, tk.TclError) as exc:
            messagebox.showerror("Не удалось
            открыть файл", str(exc))
        return
        self.document = shapes
        self.render_document()
```

Реальное окно: то же приложение в увеличенном виде — холст занимает больше места, но ранее нарисованные фигуры остались того же размера в тех же абсолютных координатах

Реальное окно после увеличения: холст стал больше, но нарисованные фигуры не выросли вместе с ним — именно то, что показывает Debug Lab 15 ниже.

DEBUG LAB 15: ИЗМЕНЕНИЕ РАЗМЕРА ОКНА НЕ МАСШТАБИРУЕТ РИСУНОК

`resize_kak_masshtab.py`

```
# ожидание: если растянуть окно, старые фигуры  
увеличатся вместе с ним  
canvas.grid(row=1, column=0, sticky="nsew")  
# ...но существующие create_line/rectangle/  
oval координаты никто не пересчитывает
```

```
# Сам виджет Canvas становится больше вместе с окном  
# но уже нарисованные фигуры остаются в прежних абсол
```

Что видно на экране

`sticky="nsew"` и веса строк/столбцов (глава 17) заставляют РАСТИ сам виджет Canvas — это не то же самое, что автоматическое масштабирование координат уже нарисованных фигур. Раздел 18.29 явно разбирает это различие: увеличенный холст не означает автоматически увеличенный рисунок.

ИСПРАВЛЕННЫЙ КОД

```
resize_kak_bolshe_mesta.py
```

```
# Честная модель: увеличенное окно даёт БОЛЬШЕ
# МЕСТА для новых фигур,
# а не растягивает уже существующие — если
# нужно последнее, координаты
# пришлось бы пересчитывать вручную при
# изменении размера.
```

Практика: воспроизводим и чиним баги рисовалки

Автоматическая проверка — Debug Lab 11 (очистка и история) и Debug Lab 10 (redo после нового действия)

Открыть практику → (<https://www.cartesianschool.org/practice/18-31/index.html>)

Paint Pro — полная программа и итоги главы

Тот же холст, что и в разделе 18.7 — но теперь с документом, историей отмены и архитектурой, которая выдержит рост проекта.

Итоговая программа

Финальное окно Paint Pro с несколькими нарисованными фигурами, панелью инструментов, палитрой, ползунком толщины и строкой состояния

Реальное окно финальной версии — та же картинка, что открывала главу в разделе 18.1, но теперь мы знаем, из чего она построена.

Файл целиком, самодостаточный и без невидимых зависимостей от других уроков:

[tkinter/paint-app/paint_app.py](#)
[projects/](#)

paint_app.py – структура

```

class Tool(Enum): ...

@dataclass
class Shape: ...

@dataclass
class DrawingState: ...

def normalize_bounds(x1, y1, x2, y2): ...

class PaintApp:
    def __init__(self, root): ...
    def build_ui(self): ...
    def set_tool(self, tool): ...
    def set_color(self, hex_color): ...
    def choose_custom_color(self): ...
    def set_width(self, value): ...
    def on_press(self, event): ...
    def on_drag(self, event): ...
    def on_release(self, event): ...
    def render_document(self): ...
    def undo(self): ...
    def redo(self): ...
    def clear_canvas(self): ...
    def save_document(self): ...
    def load_document(self): ...

def main():
    root = tk.Tk()

```



```

app = PaintApp(root)
root.mainloop()

if __name__ == "__main__":
    main()

```

Запустите приложение у себя

python paint_app.py в терминале — либо кнопкой Run в VS Code или PyCharm. Сравните с paint_app_basic.py (раздел 18.7) — тот же холст и та же идея, но совершенно другая внутренняя архитектура.

Чек-лист готового приложения

МОДЕЛЬ

- документ (document) — список фигур, источник истины
- DrawingState — параметры инструмента, отдельно от самих фигур
- Canvas — только отображение, перерисовывается из документа

ИНСТРУМЕНТЫ

- карандаш непрерывным штрихом
- линия, прямоугольник, овал с живым превью
- ластик как штрих цветом фона
- выбор инструмента виден на экране, не только в памяти

ИСТОРИЯ

- undo/redo по действиям, а не по отдельным элементам Canvas
- новое действие очищает redo_stack
- очистка холста обнуляет историю вместе с документом

СОХРАНЕНИЕ

- сохранение и загрузка рисунка через JSON
- отмена диалогов сохранения/открытия/выбора цвета обработана честно
- строка состояния и горячие клавиши

Мост к главе 19

Дальше: движение и повторение вместо ожидания клика

Рисовалка целиком построена вокруг ожидания действий пользователя — программа реагирует, но сама не «живёт». В следующей главе — игра «Змейка» на Turtle — появится собственный игровой цикл, где программа сама двигает объекты кадр за кадром, независимо от того, нажимает ли пользователь что-то прямо сейчас.

Практика: Paint Pro целиком

Модуль tkinter открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/18-32/index.html) (<https://www.cartesianschool.org/practice/18-32/index.html>)

Итоги главы 18

Что мы построили и чему научились

- Canvas хранит элементы (items), а не пиксели — `create_*` добавляет запись в список, который помнит сам виджет, и возвращает её `item_id`.
- Координаты Canvas растут вправо и вниз от левого верхнего угла — не как у Turtle, где начало в центре, а Y растёт вверх.
- Рисование мышью — жест из трёх событий: `press` запоминает начало, `drag` обновляет черновик, `release` фиксирует результат.
- Живое превью — это ОДИН элемент, который создаётся один раз и обновляется через `coords()`, а не пересоздаётся на каждое движение.
- Прямоугольник и овал задаются двумя противоположными точками; овал вписан в получившийся прямоугольник, а не задан центром и радиусом.
- Undo/redo работают по логическим ДЕЙСТВИЯМ пользователя, а не по отдельным элементам Canvas — один карандашный штрих отменяется целиком.
- Документ (список фигур) — источник истины; Canvas каждый раз перерисовывается из него заново через `render_document()`, тот же принцип, что и `render()` в главе 17.
- Сохраняется не картинка, а документ — как JSON, который можно загрузить и перестроить заново в любой момент.

Проект: игра «Змейка» с Turtle

От первого работающего прототипа до модели игры с игровым тиком, паузой, рестартом, счётом и проверяемыми правилами — классическая «Змейка» на Turtle как введение в программирование игр реального времени.

19.1 Игра «Змейка»

19.2 Настраиваем экран Turtle

19.3 Рисуем голову

19.4 Клавиши и движение головы

19.5 Запускаем табло счёта

19.6 Наша змейка ест!

19.7 Проверка столкновений

19.8 Полный код первого прототипа

19.9 Мир игры как сетка

19.10 Координаты клетки и пиксели Turtle

19.11 Направление как вектор

19.12 Один игровой тик

19.13 Настоящий игровой цикл

19.14 Время, скорость и задержка

19.15 Состояние игры

19.16 Голова, тело и модель Snake

19.17 Почему тело движется с хвоста

19.18 Еда и свободная клетка

19.19 Столкновение со стеной

19.20 Столкновение с собой

19.21 Game Over как состояние

19.22 Pause / Resume

19.23 Restart / New Game

19.24 Скорость и рост сложности

19.25 High Score

19.26 Чистая игровая логика без Turtle

19.27 GameState с dataclass

19.28 SnakeApp: отделяем модель от визуализации

19.29 Render: модель → Turtle

19.30 Тестируем правила игры

19.31 Debug Labs

19.32 Snake Pro — финальная игра

19.33 Итоги главы

Игра «Змейка»

Классическая игра — движение, еда и столкновения — на уже знакомом модуле `turtle`.

Реальное окно: змейка из нескольких сегментов, яблоко, счёт и рекорд на тёмном поле

К концу главы эта игра будет работать на собственной модели состояния, игровом цикле и проверяемых правилах.

Игра «Змейка»

«Змейка» — одна из самых узнаваемых игр в истории программирования. Змейка непрерывно движется по полю, съедает появляющиеся яблоки и растёт с каждым съеденным яблоком; игра заканчивается при столкновении со стеной или с собственным хвостом. Соберём её на уже знакомом модуле `turtle` (главы 6–7, 12) — только на этот раз черепашка станет не художником, а персонажем игры.

Как и в главе 18, план знакомый: сначала честно работающий процедурный прототип (разделы 19.1–19.8), потом внимательный разбор того, как устроена игра на самом деле — сетка, направление как данные, игровой тик, состояние, архитектура (разделы 19.9–19.33).

Импортируем необходимые модули

```
importy.py
```

```
import random
import turtle
```

`random` понадобится для случайного положения яблока (глава 5), `turtle` — для самой графики.

Практика: начинаем собирать игру

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/19-02/index.html>)

Настраиваем экран Turtle

Отключаем автообновление экрана и заводим переменные, которые будут помнить состояние игры.

Настраиваем экран Turtle

Игре нужен предсказуемый, контролируемый экран — и, что важно, отключённое автообновление: обновлять картинку мы будем вручную, ровно раз за игровой шаг, а не при каждом мелком движении:

```
nastrojka_ekrana.py
```

```
screen = turtle.Screen()
screen.title("Змейка")
screen.bgcolor("black")
screen.setup(width=600, height=600)
screen.tracer(0) # отключаем автообновление
```


Зачем tracer(0)?

Без `tracer(0)` Turtle перерисовывает экран после *каждого* отдельного движения — для игры, где нужно двигать голову и несколько сегментов тела на каждом шаге, это выглядело бы мерцающим и медленным. `tracer(0)` + ручной `screen.update()` в конце каждого шага дают куда более плавную анимацию.

Создаём и инициализируем необходимые переменные

```
переменные.py
```

```
RAZMER_SHAGA = 20
GRANICA = 280

napravlenie = "stop"
schet = 0
igra_okonchena = False
segmenty = [] # тело змейки
```

ЗАГЛАВНЫЕ_БУКВЫ — соглашение для констант

`RAZMER_SHAGA` и `GRANICA` не меняются в процессе игры — по соглашению такие «постоянные» переменные принято называть заглавными буквами. Само по себе это ничего не меняет технически, но сигнализирует читателю: «это значение не должно меняться».

Реальное окно: чёрный игровой экран 600×600 с надписью «Счёт: 0» и одним красным яблоком

Реальное окно: экран настроен и переменные заведены.

Яблоко и табло на снимке появятся в коде только в разделах 19.3 и 19.5 — здесь важен сам настроенный экран.

Практика: настройка экрана и переменных

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/19-02/index.html) (<https://www.cartesianschool.org/practice/19-02/index.html>)

Рисуем голову

Голова и яблоко — обычные объекты Turtle с разной формой и цветом.

Рисуем голову

Голова змейки — обычный объект `turtle.Turtle()`, знакомый ещё с главы 6, только с квадратной формой и без рисования линий (перо поднято):

`golova.py`

```
golova = turtle.Turtle()
golova.speed(0)
golova.shape("square")
golova.color("white")
golova.penup()
golova.goto(0, 0)
```

Рисуем первое яблоко

Яблоко — тоже черепашка, только круглая и красная, и появляется в случайном месте поля, выровненном по сетке шага (`random.randrange()` с шагом `RAZMER_SHAGA`, чтобы яблоко всегда оказывалось «в клетке»):

yabloko.py

```

yabloko = turtle.Turtle()
yabloko.speed(0)
yabloko.shape("circle")
yabloko.color("red")
yabloko.penup()

def novoe_yabloko():
    x = random.randrange(-GRANICA, GRANICA +
RAZMER_SHAGA, RAZMER_SHAGA)
    y = random.randrange(-GRANICA, GRANICA +
RAZMER_SHAGA, RAZMER_SHAGA)
    yabloko.goto(x, y)

novoe_yabloko()

```

Почему GRANICA + RAZMER_SHAGA, а не просто GRANICA

У `randrange()` правая граница *исключается* — `randrange(-280, 280, 20)` выдаёт значения только до 260 включительно. Змейка при этом спокойно доезжает до 280 (раздел 19.19), так что без `+ RAZMER_SHAGA` крайний ряд клеток справа и сверху был бы для яблока недостижим — поле получилось бы незаметно несимметричным.

Реальное окно: белая квадратная голова змейки в центре, красное круглое яблоко рядом

Реальное окно: голова и яблоко — два независимых объекта Turtle, оба с поднятым пером.

Практика: голова и яблоко

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/19-03/index.html>)

Регистрирует ли экран нажатия клавиш со стрелками?

Подключаем клавиатуру и заставляем голову двигаться — с защитой от мгновенного разворота на 180°.

Регистрирует ли экран нажатия клавиш со стрелками?

Чтобы Turtle реагировал на клавиатуру, ему нужно явно разрешить «слушать» события (`screen.listen()`) и связать конкретные клавиши с функциями через `onkeypress()` — идея та же, что и `.bind()` у Tkinter в главе 17, только с более простым, специализированным интерфейсом:

klavishi.py

```
def idti_vverh():
    global napravlenie
    if napravlenie != "down":    # нельзя развернуться
на 180° мгновенно
        napravlenie = "up"

def idti_vniz():
    global napravlenie
    if napravlenie != "up":
        napravlenie = "down"

# idti_vlevo() и idti_vpravo() устроены аналогично

screen.listen()
screen.onkeypress(idti_vverh, "Up")
screen.onkeypress(idti_vniz, "Down")
screen.onkeypress(idti_vlevo, "Left")
screen.onkeypress(idti_vpravo, "Right")
```

Почему нельзя просто развернуться на 180°?

Если змейка двигалась вправо, а игрок нажмёт «влево», голова мгновенно «наедет» на первый сегмент собственного тела — это выглядело бы как мгновенный проигрыш без видимой причины. Проверка `if napravlenie != "down":` запрещает разворот на месте, разрешая только повороты на 90°.

Заставляем голову змейки двигаться

`dvizhenie_golovu.py`

```
def dvigat_golovu():
    if napravlenie == "up":
        golova.sety(golova.ycor() + RAZMER_SHAGA)
    elif napravlenie == "down":
        golova.sety(golova.ycor() - RAZMER_SHAGA)
    elif napravlenie == "left":
        golova.setx(golova.xcor() - RAZMER_SHAGA)
    elif napravlenie == "right":
        golova.setx(golova.xcor() + RAZMER_SHAGA)
```

sety()/setx() — половина от goto()

`sety(y)` меняет только координату Y, оставляя X прежним — удобнее, чем вычислять обе координаты через `goto()` каждый раз.

Реальное окно: голова змейки сдвинулась вправо от центра поля на несколько шагов

Реальное окно: после нескольких вызовов `dvigat_golovu()` с направлением `right`.

Практика: клавиши и движение головы

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/19-04/index.html>)

Запускаем табло счёта

Отдельная черепашка без формы, которая только показывает текст текущего счёта.

Счёт удобно показывать отдельной черепашкой без формы (`hideturtle()`), которая ничего не рисует, кроме текста (`write()` из главы 7):

`tablo_scheta.py`

```
tablo = turtle.Turtle()
tablo.speed(0)
tablo.color("white")
tablo.penup()
tablo.hideturtle()
tablo.goto(0, 260)

def obnovit_tablo():
    tablo.clear() # стираем старую надпись перед
    # новой
    tablo.write(f"Счёт: {schet}", align="center",
    font=("Arial", 16, "normal"))

obnovit_tablo()
```

`clear()` перед `write()` — обязательно

Без `tablo.clear()` каждый новый счёт накладывался бы поверх предыдущего — цифры быстро превратились бы в нечитаемую кашу.

Табло стоит прямо в игровом поле — пока что

`tablo.goto(0, 260)` ставит надпись на координату $y = 260$, а это *легальная клетка*: змейка может проехать прямо сквозь текст. Для первого прототипа это терпимо, но раздел 19.32 вынесет табло в отдельную полосу HUD над полем — туда, куда змейка по правилам попасть не может.

Реальное окно: игровое поле с надписью «Счёт: 30»
вверху, змейка из трёх сегментов

Реальное окно: `obnovit_tablo()` перерисовывает счёт после каждого съеденного яблока.

Практика: табло и поедание яблок

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/19-06/index.html>)

Наша змейка ест!

Съеденное яблоко добавляет сегмент — и каждый сегмент должен правильно следовать за предыдущим.

Наша змейка ест!

Проверяем, достаточно ли близко голова оказалась к яблоку (метод `.distance()` считает расстояние между двумя черепашками) — если да, яблоко «съедено»: появляется новое яблоко, змейка растёт, счёт увеличивается:

eda.py

```
def proverit_edu():
    global schet
    if golova.distance(yabloko) < RAZMER_SHAGA:
        novoe_yabloko()
        dobavit_segment()
        schet += 10
        obnovit_tablo()
```

Заставляем двигаться всю змейку

Каждый сегмент тела должен занять место *предыдущего* сегмента (а первый — место головы) — это создаёт эффект «змейка ползёт целиком», а не «части двигаются независимо»:

dobavit_segment.py

```
def dobavit_segment():
    novyj = turtle.Turtle()
    novyj.speed(0)
    novyj.shape("square")
    novyj.color("grey")
    novyj.penup()
    segmenty.append(novyj)
```

dvigat_telo.py

```
def dvigat_telo():
    # начинаем с хвоста и идём к голове, чтобы не
    # потерять позиции по пути
    for indeks in range(len(segmenty) - 1, 0, -1):
        x = segmenty[indeks - 1].xcor()
        y = segmenty[indeks - 1].ycor()
        segmenty[indeks].goto(x, y)

    if segmenty:
        segmenty[0].goto(golova.xcor(), golova.ycor())
```


Порядок вызовов решает всё

`dvigat_telo()` обязательно вызывается **до** `dvigat_golovu()` — иначе первый сегмент «унаследует» уже *новую* позицию головы вместо старой, и тело слипнется с головой в одну точку.

Два реальных окна рядом: слева голова рядом с яблоком, справа яблоко исчезло и появился один серый сегмент тела

Реальное окно до и после `proverit_edu()`: яблоко съедено, счёт вырос, добавился сегмент.

Практика: поедание яблок и движение тела

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/19-06/index.html\)](https://www.cartesianschool.org/practice/19-06/index.html)

Проверка столкновений

Игра заканчивается при столкновении со стеной или с собственным хвостом.

Игра заканчивается при одном из двух столкновений: со стеной поля или с собственным телом.

stolknoveniya.py

```
def proverit_stolknoveniya():
    global igra_okonchena

    # столкновение со стеной
    if abs(golova.xcor()) > GRANICA or
abs(golova.ycor()) > GRANICA:
        igra_okonchena = True

    # столкновение с собственным телом
    for segment in segmenty:
        if segment.distance(golova) < RAZMER_SHAGA /
2:
            igra_okonchena = True
```

abs() снова экономит код

`abs(golova.xcor()) > GRANICA` проверяет сразу обе стены (левую и правую) одним сравнением — вместо `golova.xcor() > GRANICA or golova.xcor() < -GRANICA`. Сама встроенная функция `abs()` знакома по разделу 5.4 — здесь она впервые работает не как «модуль числа», а как способ сравнить расстояние до границы независимо от знака координаты.

Реальное окно: голова змейки у самого правого края тёмного поля, счёт остаётся прежним

Реальное окно: голова вышла за GRANICA — `proverit_stolknoveniya()` выставила `igra_okonchena` в `True`.

★★ Самостоятельная задача

Ускорение игры

Увеличивайте скорость движения (уменьшайте задержку между шагами) на каждые 50 очков — игра станет постепенно сложнее.

Практика: столкновения

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/19-07/index.html\)](https://www.cartesianschool.org/practice/19-07/index.html)

Полный код первого прототипа

Собираем все части в единый игровой цикл — и переходим от честного прототипа к внимательному разбору того, как устроена игра на самом деле.

Соберём один игровой шаг из всех частей главы — эта функция и есть сердце игры, вызываемое снова и снова, пока игра не закончится:

igrovoj_shag.py

```
import time

ZADERZHKA_SEK = 0.12  # пауза между шагами

def igrovoj_shag():
    if igra_okonchena:
        return False

    dvigat_telo()
    dvigat_golovu()
    proverit_edu()
    proverit_stolknoveniya()
    screen.update()
    return not igra_okonchena

def glavnyj_cikl():
    global napravlenie
    napravlenie = "right"
    while igrovoj_shag():
        screen.update()
        time.sleep(ZADERZHKA_SEK)
    tablo.goto(0, 0)
```

```
tablo.write(f"Игра окончена! Счёт: {schet}",
align="center", font=("Arial", 20, "bold"))
```

Без `time.sleep()` игра закончилась бы раньше, чем вы успели бы нажать клавишу

Цикл `while` крутится со скоростью Python, а не со скоростью человека: от центра до стены всего 14 шагов, и без паузы они проходят примерно за две тысячных секунды — окно успело бы только моргнуть надписью «Игра окончена! Счёт: 0».

`time.sleep(0.12)` задаёт человеческий темп: примерно восемь шагов в секунду. Здесь `sleep()` уместен именно потому, что цикл и так блокирующий — пока он работает, в программе всё равно ничего другого не происходит, а `screen.update()` в конце каждого шага успевает разобрать накопившиеся нажатия клавиш. В архитектуре на `screen.ontimer()` (раздел 19.13) тот же самый `sleep()` уже станет ошибкой — Debug Lab 4 в разделе 19.31 разбирает, почему.

Реальное окно: собранная игра — счёт 30, змейка из трёх сегментов, яблоко на поле

Реальное окно первого прототипа: всё вместе — движение, еда, счёт.

Полная, уже собранная и проверенная программа первого прототипа доступна отдельным файлом:

[projects/turtle/](#)[snake/snake_basic.py](#)**Запустите игру у себя**

`python snake_basic.py` в терминале — управление стрелками клавиатуры.

Честная, но не окончательная версия

Мы построили настоящую работающую игру — с движением, едой, счётом и столкновениями. Но `while igrovoj_shag(): ...` — блокирующий цикл с фиксированной задержкой: скорость нельзя менять по ходу партии, а паузу и рестарт пришлось бы вплетать прямо в тело цикла. Всё состояние при этом живёт в глобальных переменных модуля. Начиная со следующего раздела глава смотрит на игру внимательнее: что такое сетка и игровой тик по-настоящему, как устроить паузу и рестарт без второй параллельной цепочки таймеров, и как вырастить из этого прототипа архитектуру уровня `SnakeApp` (раздел 19.32).

★★★ Задача повышенной сложности**Уровни сложности**

Добавьте выбор уровня сложности перед стартом игры (`input()` из главы 8) — влияющий на начальную скорость через задержку между шагами.

Практика: полная игра

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/19-08/index.html>)

Итоги первого прототипа

Что мы узнали в первых восьми разделах

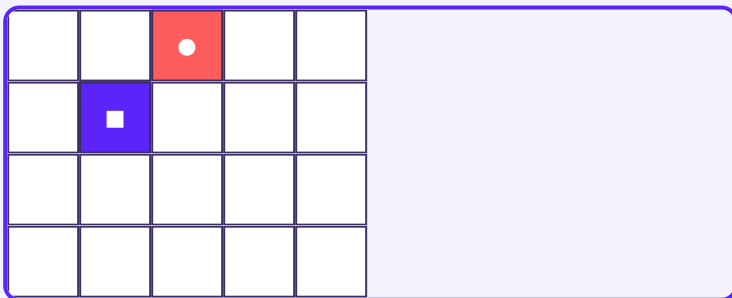
- `screen.tracer(0)` + ручной `screen.update()` — стандартный приём для плавной анимации в играх на Turtle.
- `screen.onkeypress()` связывает клавиши со стрелками с функциями изменения направления; запрет разворота на 180° предотвращает мгновенное самостолкновение.
- Тело змейки — список отдельных сегментов, каждый из которых следует за предыдущим; порядок обновления (сначала тело, потом голова) критически важен.
- `.distance()` между двумя черепашками — удобный способ проверить, находятся ли они «достаточно близко» (для еды и столкновений).
- Игровой цикл — функция, вызываемая снова и снова, пока не наступит условие конца игры.
- Это только первый прототип — дальше глава разбирает сетку, тик, состояние и архитектуру куда внимательнее.

Мир игры как сетка

Голова, тело и яблоко всегда стоят в узле решётки — движение это переход к соседнему узлу, не скольжение.

Змейка живёт не в произвольных координатах

Первый прототип уже двигает голову шагами по 20 пикселей, но легко не заметить, почему именно так: змейка не может оказаться в точке (137, 52) — только в точках, кратных `RAZMER_SHAGA`. Игровое поле — это не холст, где можно нарисовать линию под любым углом (как Canvas в главе 18), а **дискретная сетка**: конечный набор клеток, и каждый объект игры — голова, сегмент тела, яблоко — всегда стоит ровно в одной из них. Рядом со словом «сетка» дальше в главе встретится и «решётка» — это одно и то же: узлы решётки и есть клетки сетки.



Легальные позиции — узлы решётки с шагом `RAZMER_SHAGA`, а не любая точка поля.

При `RAZMER_SHAGA = 20` легальные координаты по каждой оси — `..., -40, -20, 0, 20, 40, ...`. Движение на шаг — это переход к соседнему узлу решётки, а не плавное скольжение на произвольное расстояние.

Та же идея, что и в главе 18 — но без пикселей между клетками

Canvas тоже работал в целых пикселях, но фигуру можно было сдвинуть на 1 пиксель. У змейки шаг фиксирован: `RAZMER_SHAGA` — это не «минимальная единица экрана», а сама единица игрового мира. Это и определяет всё остальное в главе: как размещается еда, как обнаруживается столкновение, как выглядит движение тела.

Практика: легальные позиции сетки

Автоматическая проверка — функция `is_on_grid()`: кратна ли точка шагу сетки

Открыть практику → (<https://www.cartesianschool.org/practice/19-09/index.html>)

Координаты клетки и пиксели Turtle

Отдельный перевод «клетка → пиксель» не нужен — координаты и так всегда кратны шагу.

Клетка сетки против пикселя Turtle

У Turtle нет отдельного понятия «клетка» — есть только пиксельные координаты `(x, y)`, те же самые, что и в главах 6–7. Можно было бы завести отдельную систему координат клеток — `col = x // STEP`, `row = y // STEP` — и переводить одно в другое при каждом обращении к экрану.

```
pixel_to_cell.py
```

```
def cell_to_pixel(col, row, step=20):
    return col * step, row * step

def pixel_to_cell(x, y, step=20):
    return x // step, y // step
```


Целочисленное деление отрицательных чисел — не то же самое, что округление к нулю

`-10 // 20` в Python равно `-1`, а не `0` — Python округляет деление вниз (к минус бесконечности), а не к нулю. Формула перевода пикселей в клетки, написанная без учёта этого, будет неверна ровно в половине поля — той, что с отрицательными координатами.

Для этой главы выбрано более простое решение: логическая позиция змейки и яблока — это сразу пиксельные координаты Turtle, уже кратные STEP. Отдельной системы «клетка (col, row)» нет вообще — (x, y) одновременно и то, что мы храним в модели, и то, что понимает `turtle.goto()`.

`logicheskaya_poziciya.py`

```
STEP = 20
head = (0, 0)           # уже пиксели Turtle — и уже
                        позиция клетки
head = (head[0] + STEP, head[1]) # шаг вправо —
                        сразу в пикселях
```

Меньше кода — меньше мест для ошибки

Перевод «клетка ↔ пиксель» — лишний шаг, который пришлось бы делать в обе стороны на каждом тике: один раз при движении, второй — при отрисовке. Раз координаты и так всегда кратны STEP, этот перевод не добавляет никакой новой информации — только риск обратной ошибки округления из примера выше.

Практика: клетка и пиксели

Автоматическая проверка — функция `pixel_to_cell()`: округление координат в клетки с отрицательными числами

Открыть практику → (<https://www.cartesianschool.org/practice/19-10/index.html>)

Направление как вектор

Направление — это не строка, которую сравнивают в `if/elif`, а данные: пара чисел (`dx`, `dy`).

От четырёх `if/elif` к одному словарию

Первый прототип определял движение головы четырьмя ветками `if/elif` (раздел 19.4) — читаемо, но при добавлении новой логики (например, диагонального движения в другой игре) пришлось бы дублировать структуру снова. Направление можно представить иначе — как **вектор смещения**:



UP

(0, +STEP)



DOWN

(0, -STEP)



LEFT

(-STEP, 0)



RIGHT

(+STEP, 0)

direction_vectors.py

```

DIRECTION_VECTORS = {
    Direction.UP: (0, STEP),
    Direction.DOWN: (0, -STEP),
    Direction.LEFT: (-STEP, 0),
    Direction.RIGHT: (STEP, 0),
}

def next_head(head, direction):
    dx, dy = DIRECTION_VECTORS[direction]
    return (head[0] + dx, head[1] + dy)

```

if/elif против словаря векторов

Явно, но растёт с числом направлений

```

if napravlenie == "up":

    golova.sety(golova.ycor()
+ RAZMER_SHAGA)
elif napravlenie ==
"down":

    golova.sety(golova.ycor()
- RAZMER_SHAGA)
# ...и так для left, right

```

Один словарь, одна формула

```

dx, dy =
DIRECTION_VECTORS[direction]
new_x, new_y = x + dx, y
dy

```

Что использовать сегодня: Оба варианта дают одинаковый результат для четырёх направлений. Разница появляется в том, как код растёт: словарь — это данные, а не ветвление, поэтому тестировать и расширять его проще, не трогая саму формулу движения.

Практика: направление как вектор

Автоматическая проверка — функция `next_head()` для всех четырёх направлений

Открыть практику → (<https://www.cartesianschool.org/practice/19-11/index.html>)

Один игровой тик

Тик — это вся цепочка правил целиком, а не одно движение головы по экрану.

Игровой тик — одно дискретное обновление игры

Игровой тик — не «один кадр анимации», а один полный цикл правил: применить направление, подвинуть голову, проверить еду, проверить столкновения, обновить модель. Отрисовка — отдельный, хоть и обычно смежный шаг:

применить направление



next_head()



проверка еды

съедено? → grow=True

**Тик и render — разные понятия, даже если происходят вместе**

В этой простой игре каждый тик заканчивается отрисовкой — но это решение, а не обязательное правило. `game_tick()` отвечает за ПРАВИЛА: что изменилось в модели. `render()` отвечает только за то, чтобы показать текущую модель на экране. Раздел 19.29 разберёт эту границу подробнее — она станет важна, когда логика будет тестироваться без Turtle вообще (раздел 19.26).

Практика: порядок шагов одного тика

Автоматическая проверка — функция `tick_order()`: что происходит раньше, проверка еды или сборка тела

Открыть практику → (<https://www.cartesianschool.org/practice/19-12/index.html>)

Настоящий игровой цикл

`screen.ontimer()` планирует следующий тик и сразу возвращает управление — вместо того чтобы блокировать программу в `while`.

while — не единственный способ повторять тики

Первый прототип запускает игру через `while` `igrovoj_shag(): ... с time.sleep()` в теле — простой и понятный блокирующий цикл. Здесь легко сделать неверный вывод, так что стоит проговорить его отдельно: `screen.update()` внутри цикла **ДЕЙСТВИТЕЛЬНО** обрабатывает накопившиеся события Tkinter, включая нажатия клавиш — значит, обработчик, привязанный через `onkeypress()`, технически может сработать прямо во время этого вызова. Проблема такого цикла не в том, что клавиатура «не успевает» — а в том, что он *владеет* управлением от начала и до конца партии и ни у кого не спрашивает разрешения. Скорость в нём — одна константа `ZADERZHKA_SEK`, которую по ходу игры не поменять; а паузу и перезапуск пришлось бы вручную вплетать в тело цикла отдельными флагами и проверками на каждой итерации. И всё это время программа не может заниматься ничем другим: она сидит внутри `sleep()`.

`ontimer_cikl.py`

```
def game_tick(self):
    if self.state.status is not GameState.RUNNING:
        return
    # ...применить направление, подвинуть голову,
    # проверить еду и столкновения...
    self.render()
    self.screen.ontimer(self.game_tick,
        self.state.delay_ms)
```

screen.ontimer() планирует следующий тик, а не блокирует программу

`screen.ontimer(callback, delay_ms)` просит цикл событий Tkinter (тот же самый, что и в главе 17) вызвать `callback` примерно через `delay_ms` — и сразу возвращает управление. Между тиками программа полностью свободна: клавиатура, пауза, изменение скорости обрабатываются как обычные события, а не ждут своей очереди внутри цикла.

time.sleep() в игровом цикле на Tkinter/Turtle — плохая идея

`time.sleep()` останавливает весь процесс, включая обработку событий — окно перестанет реагировать на клавиатуру и закрытие ровно на время сна.
`screen.ontimer()` ничего не блокирует: он передаёт часы циклу событий, который в это время продолжает обслуживать всё остальное.

Каждый вызов `game_tick()` сам планирует следующий через `ontimer()` — получается цепочка, а не цикл в привычном смысле. Здесь есть тонкость: `ontimer()` только планирует будущий вызов — он не удаляется сам по себе, если статус игры поменяется. Раздел 19.22 показывает, почему поэтому пауза не может полагаться на одну лишь проверку `status` внутри тика — и что происходит, если положиться только на неё.

Практика: игровой цикл на ontimer()

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/19-13/index.html>)

Время, скорость и задержка

`delay_ms` — не точное время, а приблизительный интервал между тиками, который уменьшается по мере роста счёта.

delay_ms — не гарантия, а просьба

`screen.ontimer(game_tick, 120)` означает «вызови `game_tick` примерно через 120 миллисекунд, когда цикл событий сможет это сделать» — не «ровно через 120.000 мс». Если в этот момент цикл событий занят чем-то ещё (например, перерисовкой), вызов немного задержится. Для игры такого масштаба разница обычно незаметна, но обещать точное время не стоит.

delay_ms	Ощущение
200	медленно — удобно для первого знакомства с управлением
140	нормальная скорость — значение по умолчанию (BASE_DELAY_MS)
120	скорость после 100 очков — игра уже заметно подгоняет
70	быстро — требует точных и быстрых реакций

`calculate_delay.py`

```
def calculate_delay(score, *, base_ms=140, min_ms=60,
    step_score=50, step_ms=10):
    steps = score // step_score
    return max(min_ms, base_ms - steps * step_ms)
```

max() — обязательная защита от нуля и отрицательной задержки

Без `max(min_ms, ...)` достаточно высокий счёт рано или поздно довёл бы задержку до нуля или отрицательного числа. `screen.ontimer(callback, 0)` формально не упадёт, но игра станет неиграбельно быстрой; отрицательное значение — и вовсе поведение, которое не стоит проверять на практике. `min_ms` — жёсткая нижняя граница скорости.

Практика: задержка тика по счёту

Автоматическая проверка — функция `calculate_delay()`: задержка падает со счётом и не опускается ниже минимума

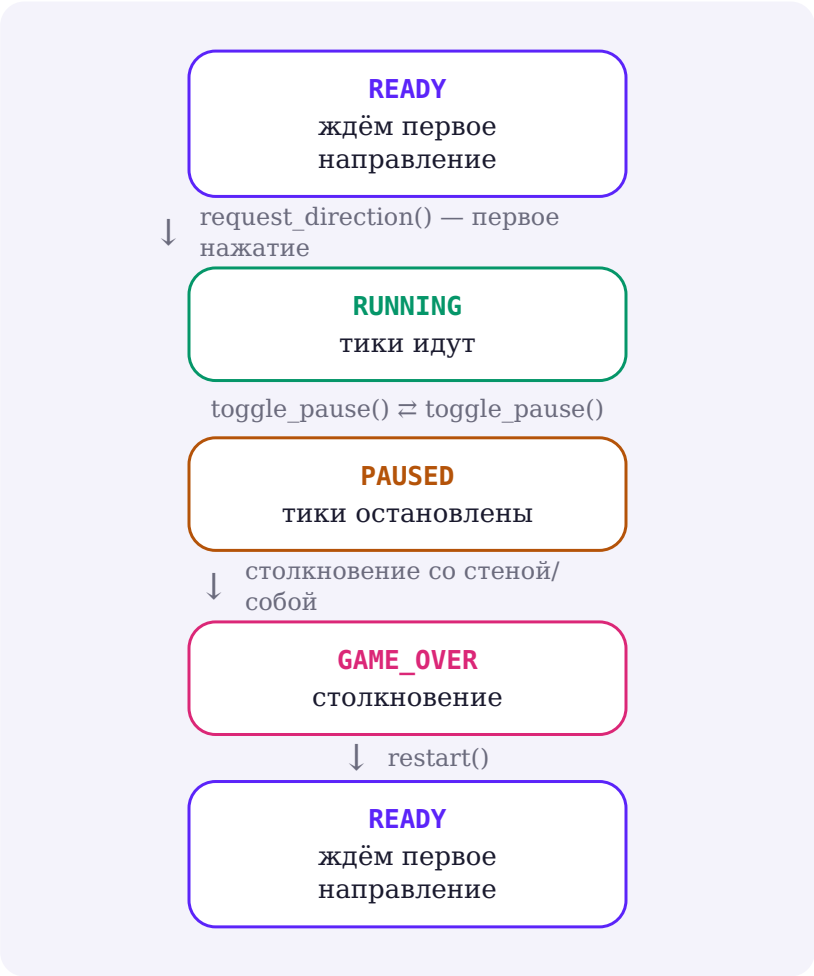
Открыть практику → (<https://www.cartesianschool.org/practice/19-14/index.html>)

Состояние игры

Не просто «игра идёт или закончилась» — четыре чётких состояния с ясными переходами между ними.

Игра — это не только «идёт» или «не идёт»

Первый прототип знал только `igra_okonchena` — булево да/нет. Финальная версия различает четыре состояния обычной партии, и переходы между ними — это не случайность, а чёткие правила:



Состояние	Что происходит
READY	змеяка на месте, ждём первое нажатие направления — тики ещё не идут
RUNNING	тики планируются друг за другом через <code>ontimer()</code>

Состояние	Что происходит
PAUSED	тики остановлены, модель не меняется, экран показывает «ПАУЗА»
GAME_OVER	столкновение произошло; ждём restart()
WON	поле заполнено целиком, свободной клетки для еды не осталось — раздел 19.18

Пятое состояние — редкое, но настоящее

В `GameStatus` финальной программы пять членов, а не четыре: кроме четырёх состояний обычной партии есть `WON` — змейка заняла буквально каждую клетку поля. Оно почти недостижимо в реальной игре, но это полноценное терминальное состояние, а не «почти `GAME_OVER`»: раздел 19.18 показывает, откуда оно берётся, а раздел 19.21 — чем терминальные состояния похожи друг на друга.

`GameStatus` как `Enum` — та же идея, что и `Tool` в главе 18

Четыре состояния — конечный известный список, поэтому `Enum` здесь так же оправдан, как `Tool` для инструментов рисовалки: опечатку в имени состояния поймает редактор, а не тихий баг посреди игры.

Практика: переходы состояния игры

Автоматическая проверка — функция `can_transition()`: какие переходы между `READY/RUNNING/PAUSED/GAME_OVER` допустимы

Открыть практику → (<https://www.cartesianschool.org/practice/19-15/index.html>)

Голова, тело и модель Snake

Список координат вместо списка Turtle-объектов — `snake[0]` это голова, остальное — тело.

Змейка как список позиций, а не список черепашек

Первый прототип хранит тело как список *объектов Turtle* — состояние игры и графика неразрывно связаны, ту же проблему глава 18 уже разбирала для Canvas (раздел 18.8). Финальная модель хранит змейку как обычный список координат:

`model_snake.py`

```
snake = [
    (0, 0),          # snake[0] — голова
    (-20, 0),
    (-40, 0),
]
```

`snake[0]` — голова; `snake[1:]` — тело. Ни одного объекта *Turtle* внутри.

Отрисовка превращает позиции в Turtle-сегменты — не наоборот

Модель ничего не знает о том, как выглядит змейка на экране. `render()` (раздел 19.29) читает `snake` и раскладывает координаты по уже существующим Turtle-сегментам — то же разделение «модель $\neq$ виджеты», что и `render_document()` в PaintApp главы 18.

Практика: список позиций как модель Snake

Автоматическая проверка — функции `head_of()` и `body_of()`:
`snake[0]` это голова, `snake[1:]` — тело

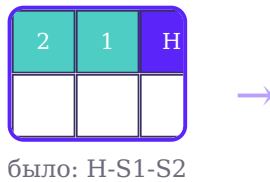
Открыть практику → (<https://www.cartesianschool.org/practice/19-16/index.html>)

Почему тело движется с хвоста

Обновление в обратном порядке схлопывает все сегменты в одну точку — разберём, почему.

Порядок обновления сегментов — не деталь, а необходимость

Раздел 19.6 уже показал правило: тело обновляется с хвоста, каждый сегмент занимает место *предыдущего*. Разберём, что случится, если поменять порядок на противоположный — от головы к хвосту:



стало (правильно): каждый унаследовал позицию предыдущего

ot_hvosta.py

```
def dvigat_telo():
    for indeks in range(len(segmenty) - 1, 0, -1):
        # segmenty[indeks] получает СТАРУЮ позицию
        segmenty[indeks - 1]
        x = segmenty[indeks - 1].xcor()
        y = segmenty[indeks - 1].ycor()
        segmenty[indeks].goto(x, y)
```

От головы к хвосту — сегменты схлопываются в одну точку

Если пойти в обратном порядке, `segmenty[0]` первым получит позицию головы. На следующей итерации `segmenty[1]` прочтает координаты `segmenty[0]` — но это уже НОВАЯ, только что перезаписанная позиция, а не старая. Через несколько итераций все сегменты окажутся в одной и той же точке — змейка визуально схлопнется в квадрат.

Раздел 19.18 покажет более элегантную альтернативу этому циклу — модель «новая голова + старое тело», которая вообще не двигает существующие сегменты по отдельности.

Практика: правильный порядок обновления тела

Автоматическая проверка — функции `move_from_tail()` и `move_from_head_BROKEN()`: порядок обновления решает всё

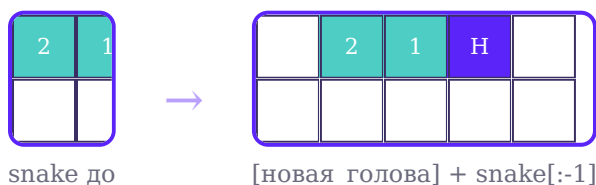
Открыть практику → (<https://www.cartesianschool.org/practice/19-17/index.html>)

Еда и свободная клетка

Новая голова плюс старое тело строит движение за один проход — и та же идея множества свободных клеток решает, куда честно поставить еду.

Эlegantная альтернатива циклу по сегментам

Вместо того чтобы двигать каждый существующий сегмент по отдельности (раздел 19.17), можно построить новый список целиком — новая голова спереди, старое тело позади, без последнего элемента (если змейка не растёт):



move_snake.py

```
def move_snake(snake, new_head, *, grow):
    if grow:
        return [new_head, *snake]
    return [new_head, *snake[:-1]]
```

Хвост при этом естественным образом «отваливается» — `snake[:-1]` отбрасывает последний элемент. Никакого цикла, никакого риска перепутать порядок обновления: результат строится за один проход, а не мутирует существующие позиции одну за другой.

Еда — случайная свободная клетка

В разделе 19.3 клетка для яблока выбиралась без проверки — при достаточно длинной змейке яблоко иногда оказывалось бы прямо внутри тела. Честная версия выбирает только среди по-настоящему свободных клеток:

choose_food.py

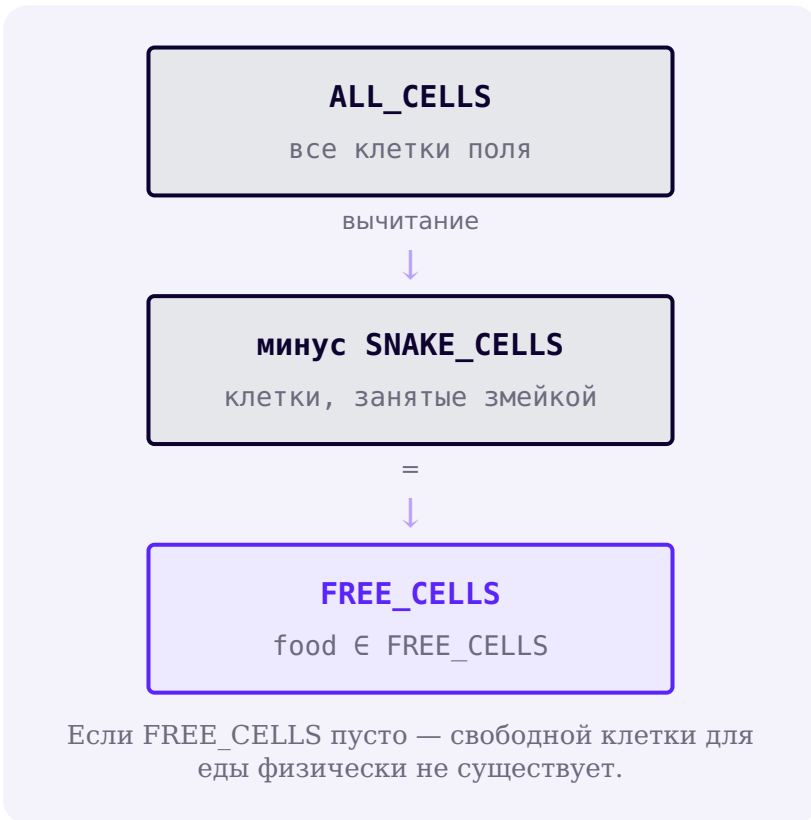
```
def choose_food(snake, rng, *, half=280, step=20):
    occupied = set(snake)
    free = tuple(
        cell for cell in all_cells(half=half,
step=step) if cell not in occupied
    )
    if not free:
        return None
    return rng.choice(free)
```

rng как параметр — та же идея пригодится в разделе 19.30

choose_food() принимает готовый random.Random, а не читает глобальный random модуль напрямую. В реальной игре передают random.Random() — настоящую случайность; в тестах — random.Random(seed) с фиксированным зерном, чтобы результат был предсказуем и его можно было проверить assert-ом.

Инвариант: еда — только из свободных клеток

Контракт choose_food() можно записать формулой: `food ∈ FREE_CELLS`, где `FREE_CELLS = ALL_CELLS - SNAKE_CELLS`. Это должно оставаться верным ВСЕГДА, включая крайний случай, до которого редко доходят на практике, но который обязан быть предусмотрен: змейка выросла настолько, что заняла буквально каждую клетку поля. Тогда `FREE_CELLS` пусто, и честного ответа, где поставить еду, не существует.



Нельзя тайком подставить клетку змейки вместо **None**

Соблазнительное, но неверное решение — вернуть, например, `snake[0]`, когда свободных клеток не осталось: тогда `food == snake[0]`, и инвариант «еда никогда не внутри змейки» нарушается прямо в самом же коде, который эту еду выбирает. Честный ответ — `None`: свободной клетки для еды в самом деле нет, и вызывающий код (раздел 19.32) обязан явно это обработать, а не притворяться, что еда по-прежнему существует.

В `SnakeApp.game_tick()` это выглядит так: если змейка съела еду и `choose_food()` вернула `None`, игра переходит в терминальное состояние `GameStatus.WON` — поле заполнено целиком, дальше двигаться некуда, и это победа, а не ошибка. `state.food` в этом состоянии тоже становится `None`, и отрисовка (раздел 19.29) обязана явно проверить это перед тем, как рисовать еду на экране.

Реальное окно: игровое поле целиком заполнено телом змейки, сверху табло со счётом, по центру надпись ПОБЕДА и итоговый счёт

Реальное окно после того, как змейка честно заняла каждую легальную клетку поля — `GameStatus.WON`, а не еда внутри тела.

Практика: еда только на свободной клетке

Автоматическая проверка — функции `move_snake()` и `choose_food()`: еда никогда не внутри змейки

Открыть практику → (<https://www.cartesianschool.org/practice/19-18/index.html>)

Столкновение со стеной

GRANICA — предел координаты центра головы, а не размер окна; граница включена в легальную область.

Граница — координата центра сегмента, а не края экрана

`GRANICA = 280` — это не размер окна (600×600), а предел для координаты центра головы. При толщине сегмента в 20 пикселей голова с центром ровно в 280 всё ещё целиком помещается на видимом поле; на 300 она бы уже выходила за край.

```
is_wall_collision.py
```

```
def is_wall_collision(head, *, half=280):
    x, y = head
    return abs(x) > half or abs(y) > half
```

Позиция головы	is_wall_collision()
(280, 0)	False — легальная граница
(-280, -280)	False — легальная граница, угол поля
(300, 0)	True — уже за пределами

Строгое сравнение (>), не (>=) — граница включена

Если бы проверка использовала `>=`, легальная позиция 280 уже считалась бы столкновением — змейка не смогла бы доехать до самого края поля. Раздел 19.30 разбирает эти пограничные случаи как отдельные тесты, а не только на словах.

Практика: граница поля

Автоматическая проверка — функция `is_wall_collision()` на самой границе и за ней

Открыть практику → (<https://www.cartesianschool.org/practice/19-19/index.html>)

Столкновение с собой

Проверять нужно тело после хода, а не до — иначе легальный заезд в клетку освободившегося хвоста ошибочно засчитывается столкновением.

Заезд в клетку хвоста — не всегда столкновение

Классическое правило «Змейки»: если змейка не растёт, хвост в этот же тик освобождает свою клетку — заехать туда головой законно. Если растёт — хвост никуда не девается, и та же самая клетка уже занята.

is_self_collision.py

```
def is_self_collision(new_head, body_after_move):
    # body_after_move – тело ПОСЛЕ move_snake(), без
    # самой головы
    return new_head in body_after_move
```

Проверять нужно тело ПОСЛЕ движения, а не до

Если проверить `new_head in snake[1:]` на СТАРОМ списке (до `move_snake()`), клетка старого хвоста всё ещё будет в списке — и легальный заезд в неё ошибочно посчитается столкновением. Правильная проверка идёт против `move_snake(...)[1:]` — тела, каким оно станет ПОСЛЕ хода, а не каким было до него.

Ситуация	is_self_collision()
Голова заезжает в клетку старого хвоста, змейка не растёт	False — хвост уже освободил клетку
Голова заезжает в клетку старого хвоста, змейка растёт	True — хвост никуда не делся
Голова заезжает в клетку середины тела	True — настоящее столкновение

Практика: столкновение с собственным телом

Автоматическая проверка — функция `is_self_collision()`: заезд в клетку хвоста допустим, в середину тела — нет

Открыть практику → (<https://www.cartesianschool.org/practice/19-20/index.html>)

Game Over как состояние

Столкновение не просто останавливает движение — оно переводит игру в отдельное состояние `GAME_OVER`.

Game Over — состояние, а не только надпись на экране

Реальное окно: неподвижная змейка S-образной формы, поверх поля крупная надпись `GAME OVER` и счёт

Реальное окно: столкновение с собственным телом переводит игру в `GAME_OVER` — та же надпись появилась бы и при столкновении со стеной.

Когда `is_wall_collision()` или `is_self_collision()` возвращают `True`, тик не просто останавливает движение — он переводит `state.status` в `GAME_OVER` и рисует оверлей поверх последнего кадра игры:

game_over.py

```
if is_wall_collision(head):
    state.status = GameState.GAME_OVER
    self.render()
    self._show_overlay("GAME OVER", f"Счёт: {state.score} | R — новая игра")
    return
```

GAME_OVER не запускает больше тиков

`game_tick()` в самом начале проверяет `if state.status is not RUNNING: return` — `GAME_OVER` сюда попадает точно так же, как `PAUSED`.

Единственный способ снова сдвинуться с места — `restart()` (раздел 19.23), который явно создаёт новое состояние.

`GAME_OVER` — не единственный терминальный статус: раздел 19.18 показывает второй, куда более редкий случай — `GameStatus.WON`, когда змейка занимает буквально всё поле. Оба останавливают тики одним и тем же способом, отличается только причина и текст оверлея.

Практика: переход в GAME_OVER

Автоматическая проверка — функция `resolve_tick()`: какие столкновения переводят игру в `GAME_OVER`

[Открыть практику](https://www.cartesianschool.org/practice/19-21/index.html) → (<https://www.cartesianschool.org/practice/19-21/index.html>)

Pause / Resume

Пауза замораживает модель на месте — возобновление продолжает ту же игру, не начинает новую.

Пауза останавливает тики, а не рисует поверх бегущей игры

Два реальных окна рядом: слева игра идёт, справа надпись ПАУЗА и подсказка `Space` — продолжить поверх того же кадра

Реальное окно: `toggle_pause()` не двигает змейку дальше — кадр справа тот же самый, что и слева, только с оверлеем.

Клавиша `Space` переключает игру между `RUNNING` и `PAUSED`:

toggle_pause.py

```
def toggle_pause(self):
    if self.state.status is GameState.RUNNING:
        self._generation += 1 # инвалидирует уже
        запланированный тик немедленно
        self.state.status = GameState.PAUSED
        self._show_overlay("ПАУЗА", "Space –
        продолжить")
    elif self.state.status is GameState.PAUSED:
        self._generation += 1 # новое поколение –
        ровно одна свежая цепочка тиков
        self.state.status = GameState.RUNNING
        self._clear_overlay()
        self._schedule_next_tick()
```

Пауза не пересоздаёт змейку

`toggle_pause()` не трогает `state.snake`, `state.score` или `state.food` — меняется только `status`. Возобновление продолжает ту же самую игру с того же места, а не начинает новую.

Почему одной проверки `status` недостаточно

`screen.ontimer(callback, delay_ms)` только ПЛАНИРУЕТ будущий вызов — сам факт, что `status` сменился на `PAUSED`, никак не удаляет уже запланированный `callback` из очереди Tkinter: он всё ещё будет вызван, просто ничего полезного не сделает, если внутри проверить статус. Это почти работает — но не совсем, и разница проявляется именно там, где её сложнее всего заметить: что если возобновление (Resume) произойдёт РАНЬШЕ, чем сработает вызов тика, запланированный ещё до паузы?

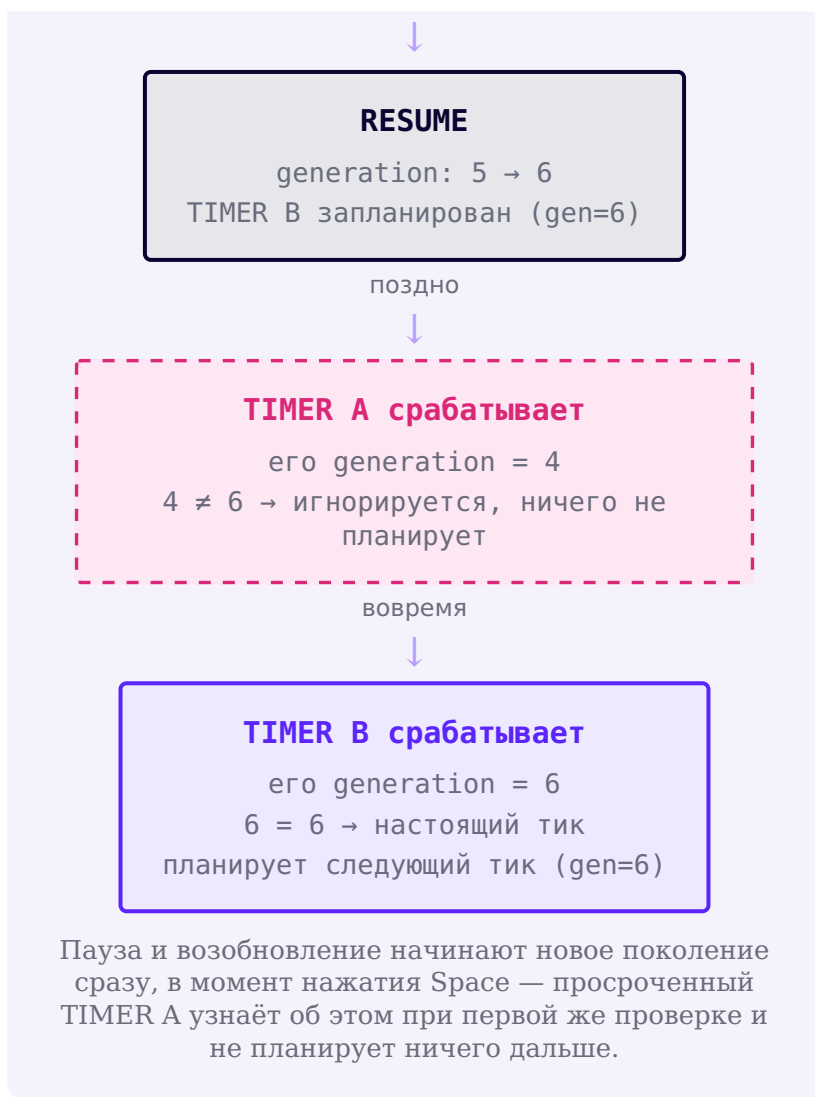
Реальный баг: Pause → Resume до того, как сработал старый тик

Без инвалидации поколения именно в момент паузы получается: старый тик всё ещё запланирован → Resume планирует ВТОРОЙ, свой собственный тик → оба тика рано или поздно срабатывают, каждый планирует следующий сам за себя → в игре тикают ДВЕ независимые цепочки одновременно, и змейка начинает двигаться вдвое чаще, чем должна.

Проверка `status is RUNNING` внутри тика эту гонку не ловит: к моменту, когда старый тик наконец срабатывает, игра уже снова RUNNING — проверка проходит, и тик как ни в чём не бывало планирует продолжение.

Поэтому `toggle_pause()` увеличивает `self._generation` на КАЖДОМ переключении — и при постановке на паузу, и при возобновлении, — а не только при перезапуске. Пауза инвалидирует запланированный тик сразу, в момент нажатия `Space`, а не полагается на то, что он сам себя узнает, когда наконец сработает. Возобновление получает собственное новое поколение и планирует ровно один свежий тик — какая бы цепочка ни существовала раньше, она уже мертва.



**Практика: pause не двигает состояние**

Автоматическая проверка — функция `game_tick()`: тик во время PAUSED не меняет змейку

Открыть практику → (<https://www.cartesianschool.org/practice/19-22/index.html>)

Restart / New Game

Новая игра обязана оборвать старую цепочку тиков — иначе на мгновение окажутся две независимые цепочки тиков сразу.

Restart обязан оборвать старую цепочку тиков

Реальное окно: свежая игра с одной головой в центре, табло Счёт: 0 Рекорд: 40

Реальное окно после restart(): свежая змейка, счёт обнулён, рекорд остался.

Restart не просто возвращает змейку в исходную позицию — раздел 19.22 предупреждал: если старая цепочка `ontimer()` всё ещё тикает, у игры на мгновение оказалось бы две независимые цепочки обновлений одновременно (глава 16 уже разбирала похожую проблему с повторным нажатием кнопки).

restart.py

```
def restart(self):
    self._generation += 1 # обрывает любую ещё
    тикающую цепочку прошлой игры
    high_score = self.state.high_score
    self.state = new_game_state(self.rng,
    high_score=high_score)
    self._clear_overlay()
    self.render()
```

generation_guard.py

```
def _schedule_next_tick(self):
    generation = self._generation
    self.screen.ontimer(lambda:
self._on_timer(generation), self.state.delay_ms)

def _on_timer(self, generation):
    if generation != self._generation:
        return # просроченная цепочка от игры до
restart() — сама себя гасит
    self.game_tick()
    if self.state.status is GameState.RUNNING:
        self._schedule_next_tick()
```

Поколение (generation) — простой счётчик, не сложная архитектура

Каждый `restart()` увеличивает `_generation` на единицу. Любой уже запланированный `_on_timer()` запомнил СТАРОЕ значение — когда он наконец сработает, число не совпадёт, и он тихо ничего не сделает вместо того, чтобы двигать уже несуществующую змейку прошлой игры.

`high_score` — единственное поле, которое `restart` переносит из старого состояния в новое; всё остальное создаётся заново через `new_game_state()` (раздел 19.25).

Практика: restart и просроченные тики

Автоматическая проверка — функции `new_game_state()` и `restart()`: рекорд переживает рестарт, поколение растёт

Открыть практику → (<https://www.cartesianschool.org/practice/19-23/index.html>)

Скорость и рост сложности

Каждое яблоко не только увеличивает счёт, но и пересчитывает задержку — игра постепенно ускоряется.

Игра ускоряется вместе со счётом

Реальное окно: змейка из пяти сегментов, счёт 450, яблоко в верхней части поля

Реальное окно: при высоком счёте `calculate_delay()` уже вернула минимальное значение — игра идёт на предельной скорости.

Раздел 19.14 уже показал формулу — здесь она подключается к самой игре: после каждого съеденного яблока пересчитывается не только счёт, но и задержка следующего тика:

`speed_progression.py`

```
if grow:
    state.score += FOOD_SCORE
    state.high_score = max(state.high_score,
                           state.score)
    state.food = choose_food(state.snake, self.rng)
    state.delay_ms = calculate_delay(state.score)
```

Новое значение `delay_ms` вступит в силу на следующем вызове `_schedule_next_tick()` — предыдущий тик уже был запланирован со старой задержкой, и это нормально: скорость меняется плавно, не рывком.

`min_ms` — сознательный предел, а не случайность

Раздел 19.14 уже объяснял `max(min_ms, ...)` — здесь видно, зачем он нужен на практике: без него достаточно долгая игра рано или поздно довела бы задержку до нуля.

Практика: скорость растёт со счётом

Автоматическая проверка — функция `eat_food()`: каждое яблоко пересчитывает и счёт, и задержку

Открыть практику → (<https://www.cartesianschool.org/practice/19-24/index.html>)

High Score

`score` и `high_score` — разные поля с разным временем жизни: одно обнуляется при рестарте, другое переживает его.

Рекорд переживает рестарт — счёт нет

Реальное окно: одна голова в центре поля, табло сверху показывает Счёт: 0 Рекорд: 40

Реальное окно: после `restart()` счёт обнулился, а рекорд остался прежним.

`score` и `high_score` — два разных поля с разным временем жизни. Раздел 19.23 уже показал, что `restart()` явно передаёт `high_score` в новое состояние, а не создаёт его заново:

`high_score.py`

```
def new_game_state(rng, *, high_score=0):
    snake = [(0, 0)]
    return GameState(
        snake=snake,
        # ...
        score=0,                # всегда с нуля
        high_score=high_score,  # передан вызывающим
        # ...
    )
```

кодом

Поле	Что происходит при restart()
score	всегда сбрасывается в 0 — новая игра начинается с чистого счёта
high_score	передаётся из прошлого state.high_score — переживает рестарт

high_score обновляется в момент max(), а не в конце игры

state.high_score = max(state.high_score, state.score) вызывается сразу при каждом съеденном яблоке (раздел 19.24) — рекорд не нужно «подводить» отдельно в конце: он уже точен в любой момент игры, включая экран Game Over.

Практика: рекорд переживает рестарт

Автоматическая проверка — функция update_high_score(): рекорд только растёт

[Открыть практику →](https://www.cartesianschool.org/practice/19-25/index.html) (https://www.cartesianschool.org/practice/19-25/index.html)

Чистая игровая логика без Turtle

Правила игры — обычные функции над числами и списками; Turtle нужен только для того, чтобы их показать.

Ни одна из функций правил не открывает окно

Оглянемся на все функции, разобранные с раздела 19.9: `next_head()`, `move_snake()`, `is_wall_collision()`, `is_self_collision()`, `choose_food()`, `calculate_delay()`. Ни одна из них не создаёт `turtle.Turtle()`, не читает `golova.xcor()` и вообще ничего не знает про экран:

Правила игры — обычные функции над числами и списками

NEXT_HEAD(HEAD, DIRECTION)

кортеж → кортеж

раздел 19.11

MOVE_SNAKE(SNAKE, HEAD, GROW)

список → список

раздел 19.18

IS_WALL_COLLISION(HEAD)

кортеж → bool

раздел 19.19

IS_SELF_COLLISION(HEAD, BODY)

кортеж, список → bool
раздел 19.20

CHOOSE_FOOD(SNAKE, RNG)

список → кортеж
раздел 19.18

CALCULATE_DELAY(SCORE)

int → int
раздел 19.14

Это не случайность, а сознательное разделение: правила «Змейки» — это математика над координатами, а Turtle — только один из способов эту математику *показать*. Раздел 19.30 использует ровно это свойство, чтобы протестировать правила напрямую, без единого настоящего окна.

Тот же принцип, что и в главе 18

`normalize_bounds()` и `Shape` в `PaintApp` (глава 18) были ровно такими же — чистыми функциями и данными без `tkinter` внутри. Пользы от этого две: логику проще тестировать, и её проще понять — читая функцию, не нужно держать в голове состояние целого окна.

Практика: какие функции — чистая логика

Автоматическая проверка — функция `is_pure_logic()`:
отличаем правила без Turtle от кода, которому нужен экран

Открыть практику → (<https://www.cartesianschool.org/practice/19-26/index.html>)

GameState с dataclass

Одна структура данных описывает всю игру целиком — без единого объекта Turtle внутри.

Одна структура вместо восьми отдельных переменных

Собираем все поля, разобранные по частям — змейку, направление, еду, счёт, статус, скорость — в один `@dataclass`, ту же идею, что и `DrawingState` в главе 18:

Ни одного объекта Turtle — только данные, которые полностью описывают текущую игру.

gamestate.py

```
@dataclass
class GameState:
    snake: list[Position] =
    field(default_factory=lambda: [(0, 0)])
    direction: Direction = Direction.RIGHT
    next_direction: Direction = Direction.RIGHT
    food: Position | None = (0, 0)
    score: int = 0
    high_score: int = 0
    status: GameStatus = GameStatus.READY
    delay_ms: int = BASE_DELAY_MS
```

Почему food вообще может быть None

На девяносто девяти играх из ста `food` — обычная клетка. Но раздел 19.18 показывает честный крайний случай: если змейка займёт буквально всё поле, свободной клетки для новой еды не останется. Тип `Position | None` делает этот случай видимым в самой сигнатуре, а не спрятанным — код, который читает `state.food`, обязан явно решить, что делать, если еды нет.

Соблазн сохранить сюда Turtle-объект — частая ошибка

Если положить `head_turtle` внутрь `GameState`, тесты из раздела 19.30 перестанут работать без настоящего окна, а «модель» перестанет быть моделью — она снова срастётся с отображением, как в первом прототипе. Раздел 19.31 разбирает эту ошибку как отдельный Debug Lab.

`next_direction` — отдельное от `direction` поле неспроста: раздел 19.31 (Debug Labs 5–6) объясняет, зачем клавиатура запрашивает направление, а не меняет его напрямую.

Практика: конструируем и сравниваем GameState

Автоматическая проверка — класс `GameState`: создание, сравнение и `dataclasses.replace()`

Открыть практику → (<https://www.cartesianschool.org/practice/19-27/index.html>)

SnakeApp: отделяем модель от визуализации

`app.state` — модель игры; `app.screen` и Turtle-объекты — только то, что видно на экране.

app HAS-A screen — тот же паттерн, что и в главах 16-18

app : SnakeApp

```
screen = Screen
state = GameState
rng = random.Random
head = Turtle
food_turtle = Turtle
_segment_pool = list[Turtle]
```

app хранит и виджеты Turtle, и модель (state) — но не путает их друг с другом.

Группа методов	За что отвечает
bind_keys	построение обработчиков клавиатуры — один раз, при запуске
request_direction	клавиша → запрос направления, без движения прямо сейчас
game_tick	один тик игры → изменения state
render	state → Turtle (единственное место, которое рисует по-настоящему)

Группа методов	За что отвечает
<code>toggle_pause</code> / <code>restart</code>	управление жизненным циклом игры

SnakeApp не наследуется от turtle.Screen

Как и `PaintApp(tk.Tk)` в главе 18, наследование здесь технически возможно, но не выбрано: `self.screen` — отдельный атрибут, а не сам объект `SnakeApp`. Композиция делает границу между «моей логикой» и «внутренностями Turtle» явной.

Практика: собираем объектный граф SnakeApp

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/19-28/index.html>)

Render: модель → Turtle

Один и тот же пул Turtle-сегментов переиспользуется на каждом тике — новые создаются, только когда змейка выросла.

render() не создаёт новые сегменты на каждом тике

Реальное окно: тонкая фиолетовая сетка на поле, подписанные клетки `head cell` и `food cell`

Реальное окно: та же сетка из раздела 19.9, но теперь видно, как голова и еда — просто клетки, которые `render()` превращает в конкретные Turtle-объекты.

Змейка растёт, и напрашивается простое решение: создавать новый `turtle.Turtle()` на каждое съеденное яблоко — ровно так и делал первый прототип (раздел

19.6). Но объекты Turtle никуда не деваются: после `restart()` их окажется столько же, сколько было в самой длинной партии, и каждый из них экран продолжит обслуживать. Поэтому вводим **пул сегментов**: один раз созданные черепашки переиспользуются от кадра к кадру, новые добавляются только когда их действительно не хватает, а лишние прячутся, а не удаляются.

`render.py`

```
def render(self):
    self.head.goto(*self.state.snake[0])

    body = self.state.snake[1:]
    self._ensure_segment_pool(len(body))
    for i, segment in enumerate(self._segment_pool):
        if i < len(body):
            segment.showturtle()
            segment.goto(*body[i])
        else:
            segment.hideturtle()

    if self.state.food is not None:
        self.food_turtle.showturtle()
        self.food_turtle.goto(*self.state.food)
    else:
        self.food_turtle.hideturtle()  # WON —
        свободных клеток не осталось

    self._render_scoreboard()
    self.screen.update()
```

Проверка `food is not None` здесь обязательна

Раздел 19.18 обещал, что отрисовка честно обработает случай, когда свободной клетки для еды не осталось, — вот это место. Без проверки `self.food_turtle.goto(*self.state.food)` при `food = None` падает с `TypeError: turtle.TNavigator.goto() argument after * must be an iterable, not NoneType` — ровно в момент победы, в самом редком и самом обидном состоянии игры.

Пул растёт, но никогда не уменьшается сам по себе

`_ensure_segment_pool()` добавляет новые Turtle-сегменты, только если их не хватает — уже существующие переиспользуются. Когда змейка становится короче (после `restart()`), лишние сегменты не удаляются, а просто прячутся через `hideturtle()` — на следующей долгой игре они снова понадобятся, и пересоздавать их не придётся.

Практика: `render()` и пул сегментов

Модуль `turtle` открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/19-29/index.html>)

Тестируем правила игры

Раз правила — чистые функции, их можно проверить обычным `assert`, без окна и без `Xvfb`.

Проверяем правила напрямую, без единого окна

Раздел 19.26 показал, что правила игры — чистые функции. Значит, их можно проверить обычным `assert`, как в главе 3 — без `Xvfb`, без `turtle.Screen()`, без ожидания, пока что-то нарисуеться:

```
test_snake_logic.py
```

```
def test_move_snake_without_growth_drops_tail():
    snake = [(0, 0), (-20, 0), (-40, 0)]
    result = move_snake(snake, (20, 0), grow=False)
    assert result == [(20, 0), (0, 0), (-20, 0)]

def test_wall_collision_boundary_is_safe():
    assert is_wall_collision((280, 280)) is False

def test_wall_collision_beyond_boundary():
    assert is_wall_collision((300, 0)) is True
```

Пограничные случаи — отдельные тесты, а не один общий

«Безопасно ровно на границе» и «столкновение чуть дальше границы» — два разных теста, не один: если объединить их в одну проверку, тест может остаться зелёным даже при ошибке ровно в этой точке (раздел 19.19 разбирал, почему граница включена).

Тестировать сценарии, для которых важна случайность (выбор еды), помогает `random.Random(seed)` с зафиксированным зерном — раздел 19.18 уже объяснял, зачем `choose_food()` принимает готовый генератор, а не читает глобальный `random` напрямую:

```
test_food.py
```

```
def test_choose_food_never_lands_on_snake():
    rng = random.Random(1)
    snake = [(0, 0), (-20, 0), (-40, 0)]
    for _ in range(50):
        food = choose_food(snake, rng)
        assert food not in snake
```


Практика: пишем тест правила игры

Автоматическая проверка — функция `test_boundary_is_safe()`:
тест обязан поймать сломанную версию

Открыть практику → (<https://www.cartesianschool.org/practice/19-30/index.html>)

Debug Labs — типичные ошибки «Змейки»

Каждая ошибка здесь встречается в реальных студенческих проектах — научитесь узнавать симптом раньше, чем откроете отладчик.

Небольшая коллекция типичных ошибок «Змейки» — каждая с симптомом и исправлением. Часть из них вы уже видели раньше в главе; здесь они собраны как справочник.

DEBUG LAB 1: ЗАБЫЛИ `SCREEN.TRACER(0)`

`bez_tracer.py`

```
screen = turtle.Screen()
screen.title("Змейка")
# tracer(0) не вызван
```

```
# Игра работает, но каждое движение головы и КАЖДОГО
# перерисовывается отдельно — заметное мерцание и замедление
```

Что видно на экране

Без `tracer(0)` Turtle обновляет экран после каждого отдельного вызова `goto()` — раздел 19.2 уже объяснял, зачем нужно отключить автообновление и делать это вручную, ровно раз за тик.

ИСПРАВЛЕННЫЙ КОД

`s_tracer.py`

```
screen = turtle.Screen()
screen.title("Змейка")
screen.tracer(0)
```

DEBUG LAB 2: ЗАБЫЛИ `SCREEN.UPDATE()` В КОНЦЕ `RENDER()`

`bez_update.py`

```
def render(self):
    self.head.goto(*self.state.snake[0])
    # ...остальная отрисовка...
    # self.screen.update() не вызван
```

Модель меняется на каждом тике (можно проверить `print`)
но на экране змейка стоит неподвижно.

Что видно на экране

С `tracer(0)` ручной `screen.update()` — единственный момент, когда изменения долетают до экрана. Тот же принцип, что и в главе 17: изменение модели без вызова отрисовки невидимо пользователю.

ИСПРАВЛЕННЫЙ КОД

`s_update.py`

```
def render(self):
    self.head.goto(*self.state.snake[0])
    # ...остальная отрисовка...
    self.screen.update()
```

DEBUG LAB 3: БЛОКИРУЮЩИЙ ЦИКЛ УСЛОЖНЯЕТ ПАУЗУ И СКОРОСТЬ

`busy_loop.py`

```
while igrovoj_shag():
    screen.update()
    time.sleep(ZADERZHKA_SEK) #
    фиксированная задержка
    # цикл сам решает, когда именно проверять
    события — паузу, скорость
    # и перезапуск приходится вручную вплетать в
    его тело
```

```
# Игру нельзя поставить на паузу: событие обрабатывае
# но цикл ничего не проверяет и продолжает крутиться
# Скорость задана одной константой и по ходу партии н
```

Что видно на экране

Раздел 19.13 разбирал это подробно:

`screen.update()` внутри `while` **ДЕЙСТВИТЕЛЬНО** обрабатывает события Tkinter, включая нажатия клавиш, — обработчик паузы технически может сработать. Но сам цикл не спрашивает, нужно ли ему остановиться: паузу, скорость и перезапуск пришлось бы вручную проверять на каждой итерации. `screen.ontimer()` устроен иначе: каждый тик — это отдельный запланированный вызов, а не итерация одного бесконечного цикла, и решение «делать следующий тик или нет» принимается один раз, явно, а не проверяется заново внутри `while`.

ИСПРАВЛЕННЫЙ КОД

`ontimer_vmesto_busy.py`

```
def game_tick(self):
    if self.state.status is not
    GameState.RUNNING:
        return
    # ...
    self.screen.ontimer(self.game_tick,
    self.state.delay_ms)
```

DEBUG LAB 4: TIME.SLEEP() ВНУТРИ ИГРОВОГО ЦИКЛА

s_sleep.py

```
def game_tick(self):
    # ...
    time.sleep(self.state.delay_ms / 1000)
    self.game_tick()
```

```
# Окно замирает НАВСЕГДА, а не на delay_ms: тик зовёт
# без условия выхода, поэтому пауз становится бесконечной
# а примерно через 1000 вызовов программа падает с Re...
```

Что видно на экране

Ошибок здесь сразу две. Первая: `time.sleep()` блокирует ВСЬ процесс, включая цикл событий Tkinter, на котором держится Turtle. Вторая, менее заметная: `game_tick()` вызывает саму себя напрямую, без всякого условия выхода — это обычная бесконечная рекурсия, и стек кончится раньше, чем игрок дожждётся хоть одного отклика. `screen.ontimer()` решает обе: он ничего не блокирует и не наращивает стек — раздел 19.13 объясняет разницу подробно.

ИСПРАВЛЕННЫЙ КОД

```
bez_sleep.py
```

```
def game_tick(self):
    # ...
    self.screen.ontimer(self.game_tick,
self.state.delay_ms)
```

DEBUG LAB 5: РАЗВОРОТ НА 180° РАЗРЕШЁН

```
bez_is_reverse.py
```

```
def request_direction(self, direction):
    self.state.next_direction = direction    #
    без проверки!
```

```
# Змейка едет вправо, игрок нажимает Left +
# голова немедленно врезается в первый сегмент собственн
```

Что видно на экране

Без `is_reverse()` ничто не мешает развернуть змейку прямо в её же тело за один тик — раздел 19.4 (и 19.11) объясняли, почему разворот на 180° должен быть запрещён на уровне ввода, а не только совпадением по случайности.

ИСПРАВЛЕННЫЙ КОД

```
s_is_reverse.py
```

```
def request_direction(self, direction):
    if is_reverse(self.state.direction,
direction):
        return
    self.state.next_direction = direction
```

DEBUG LAB 6: БЫСТРАЯ ПАРА КЛАВИШ ПРОНОСИТ РАЗВОРОТ МИМО ПРОВЕРКИ

```
proverka_ne_protiv_togo.py
```

```
def request_direction(self, direction):
    if is_reverse(self.state.next_direction,
direction): # против next_direction!
        return
    self.state.next_direction = direction
```

```
# Змейка едет вправо. Игрок быстро нажимает Up, потом
# Left проходит проверку, потому что next_direction y
```

Что видно на экране

Ловушка тонкая: проверка обязана идти против `state.direction` — направления, которое СЕЙЧАС применяется на тике — а не против уже изменённого `next_direction`, иначе вторая клавиша между тиками может протащить нелегальный разворот.

ИСПРАВЛЕННЫЙ КОД

```
proverka_protiv_direction.py
```

```
def request_direction(self, direction):
    if is_reverse(self.state.direction,
direction):
        return
    self.state.next_direction = direction
```

DEBUG LAB 7: ЕДА НЕ ВЫРОВНЕНА ПО СЕТКЕ

```
neverno_vyrovnnennaya_ed.py
```

```
def choose_food(snake, rng, *, half=280):
    x = rng.randint(-half, half) # любое
целое, не кратное STEP!
    y = rng.randint(-half, half)
    return (x, y)
```

```
# Яблоко появляется в точке вроде (137, -52) —
# голова змейки никогда не попадёт туда ровно, съест
```

Что видно на экране

Раздел 19.9 объяснял: легальные позиции — узлы решётки с шагом `STEP`. `choose_food()` обязана выбирать из того же множества клеток, что и `next_head()` — иначе математически невозможно попасть точно в клетку еды.

ИСПРАВЛЕННЫЙ КОД

vyrovnennaya_eda.py

```
def choose_food(snake, rng, *, half=280,
step=20):
    free = tuple(c for c in
all_cells(half=half, step=step) if c not in
set(snake))
    return rng.choice(free)
```

DEBUG LAB 8: ЕДА ПОЯВЛЯЕТСЯ ВНУТРИ ТЕЛА ЗМЕЙКИ

eda_bez_proverki.py

```
def choose_food(snake, rng, *, half=280,
step=20):
    coords = range(-half, half + 1, step)
    return (rng.choice(coords),
rng.choice(coords)) # не проверяет snake!
```

При достаточно длинной змейке яблоко иногда появляе
прямо внутри собственного тела — заведомо несъедобн

Что видно на экране

Раздел 19.18 разбирал именно эту проблему: без исключения занятых клеток `choose_food()` честно выбирает случайно среди ВСЕХ клеток, включая занятые телом — что технически случайно, но нечестно по отношению к игроку.

ИСПРАВЛЕННЫЙ КОД

`eda_s_proverkoj.py`

```
def choose_food(snake, rng, *, half=280,
step=20):
    occupied = set(snake)
    free = tuple(c for c in
all_cells(half=half, step=step) if c not in
occupied)
    return rng.choice(free)
```

DEBUG LAB 9: ТЕЛО ОБНОВЛЯЕТСЯ ОТ ГОЛОВЫ К ХВОСТУ

`ot_golovy_k_hvostu.py`

```
def dvigat_telo():
    for indeks in range(len(segmenty) - 1):
        # неверное направление цикла!
        segmenty[indeks].goto(segmenty[indeks
+ 1].xcor(), segmenty[indeks + 1].ycor())
```

```
# Через несколько шагов все сегменты тела
# визуально схлопываются в одну точку.
```

Что видно на экране

Раздел 19.17 разбирал это подробно: обновление обязано идти с хвоста к голове, иначе каждый следующий сегмент читает уже перезаписанную, а не старую позицию соседа.

ИСПРАВЛЕННЫЙ КОД

s_hvosta_k_golove.py

```
def dvigat_telo():
    for indeks in range(len(segmenty) - 1, 0,
-1):
        segmenty[indeks].goto(segmenty[indeks
- 1].xcor(), segmenty[indeks - 1].ycor())
```

DEBUG LAB 10: ТАБЛО СЧЁТА БЕЗ CLEAR()

tablo_bez_clear.py

```
def obnovit_tablo():
    tablo.write(f"Счёт: {{schet}}",
align="center", font=("Arial", 16, "normal"))
    # tablo.clear() не вызван
```

После нескольких съеденных яблок надпись на табло
превращается в нечитаемую кашу из наложенных друг на друга

Что видно на экране

Раздел 19.5 объяснял: `write()` не заменяет предыдущий текст, а рисует поверх него. `clear()` обязателен перед каждой новой надписью той же черепашки.

ИСПРАВЛЕННЫЙ КОД

`tablo_s_clear.py`

```
def obnovit_tablo():
    tablo.clear()
    tablo.write(f"Счёт: {{schet}}",
    align="center", font=("Arial", 16, "normal"))
```

DEBUG LAB 11: ГРАНИЦА ПРОВЕРЕНА НЕСТРОГО — ГОЛОВА НЕ ДОЕЗЖАЕТ ДО КРАЯ

`granica_s_ravno.py`

```
def is_wall_collision(head, *, half=280):
    x, y = head
    return abs(x) >= half or abs(y) >= half
# >=, не >
```

```
# Игра заканчивается на одну клетку раньше настоящей
# змейка никогда не может доехать до последнего легала
```

Что видно на экране

Раздел 19.19 объяснял: `GRANICA` — координата центра СЕГМЕНТА, который на ней всё ещё целиком помещается на поле. `>=` ошибочно исключает саму границу из легальной области.

ИСПРАВЛЕННЫЙ КОД

```
granica_strogo.py
```

```
def is_wall_collision(head, *, half=280):
    x, y = head
    return abs(x) > half or abs(y) > half
```

DEBUG LAB 12: САМОСТОЛКНОВЕНИЕ ПРОВЕРЕНО ПО СТАРОМУ ТЕЛУ

```
stolknovenie_po_staromu.py
```

```
grow = head == state.food
if is_self_collision(head,
state.snake[1:]): # ещё ДО move_snake()!
    state.status = GameStatus.GAME_OVER
```

```
# Змейка не растёт, а игрок пытается захватить в клетку
# хвост как раз освобождает в этом же тике — игра ошибается
```

Что видно на экране

Раздел 19.20 разбирал именно это: клетка старого хвоста всё ещё в `state.snake` ДО `move_snake()`. Проверять нужно тело ПОСЛЕ хода — `new_snake[1:]` — где хвост уже честно отброшен, если змейка не растёт.

ИСПРАВЛЕННЫЙ КОД

`stolknovenie_po_novomu.py`

```
new_snake = move_snake(state.snake, head,
grow=grow)
if is_self_collision(head, new_snake[1:]):
    state.status = GameStatus.GAME_OVER
```

DEBUG LAB 13: RESTART ОСТАВЛЯЕТ СТАРУЮ ЦЕПОЧКУ ТИКОВ ЖИВОЙ

`restart_bez_generation.py`

```
def restart(self):
    self.state = new_game_state(self.rng)
    self.render()
    # generation не увеличен — старый
    _on_timer() всё ещё запланирован!
```

Через мгновение после Restart на экране внезапно по
фигура/движение от уже несуществующей прошлой игры.

Что видно на экране

Раздел 19.23 разобрал этот случай — классический баг «двух параллельных таймерных цепочек», знакомый ещё по главе 16. Без счётчика поколений старый `_on_timer()` ничем не отличим от нового — оба тикают одновременно.

ИСПРАВЛЕННЫЙ КОД

restart_s_generation.py

```
def restart(self):
    self._generation += 1
    self.state = new_game_state(self.rng,
    high_score=self.state.high_score)
    self.render()
```

DEBUG LAB 14: ПАУЗА МЕНЯЕТ ЭКРАН, НО ИГРА ПРОДОЛЖАЕТ ДВИГАТЬСЯ

pauza_tolko_vizualno.py

```
def toggle_pause(self):
    self._show_overlay("ПАУЗА", "Space –
    продолжить")
    # state.status не изменён – game_tick()
    ничего не знает о паузе!
```

Оверлей «ПАУЗА» показан, но змейка под ним
продолжает двигаться и может врезаться в стену.

Что видно на экране

Раздел 19.22 объяснял: оверлей — это только то, что ВИДНО. Реальная остановка происходит из-за проверки `status is not RUNNING` в самом начале `game_tick()` — без смены `state.status` эта проверка никогда не сработает.

ИСПРАВЛЕННЫЙ КОД

`pauza_po_statusu.py`

```
def toggle_pause(self):
    if self.state.status is
GameState.RUNNING:
        self.state.status = GameState.PAUSED
        self._show_overlay("ПАУЗА", "Space –
продолжить")
```

DEBUG LAB 15: НОВАЯ ИГРА НЕ СБРАСЫВАЕТ НАПРАВЛЕНИЕ

`restart_bez_napravleniya.py`

```
def restart(self):
    self._generation += 1
    self.state.snake = [(0, 0)] # правит
поле точно, а не создаёт state заново
    self.state.score = 0
    # direction/next_direction остались от
прошлой игры!
```



```
# Новая змейка стоит в центре, но при первом же движе
# уезжает в направлении, в котором закончилась ПРОШЛ
```

Что видно на экране

Точечные правки существующего `state` легко забывают одно из полей. `new_game_state()` (раздел 19.25) создаёт структуру целиком заново — забыть отдельное поле в новом объекте невозможно, оно либо есть в конструкторе, либо код не запустится.

ИСПРАВЛЕННЫЙ КОД

restart_novym_state.py

```
def restart(self):
    self._generation += 1
    self.state = new_game_state(self.rng,
                                high_score=self.state.high_score)
```

DEBUG LAB 16: РЕКОРД ОБНУЛЯЕТСЯ ВМЕСТЕ СО СЧЁТОМ

restart_teryaet_rekord.py

```
def restart(self):
    self._generation += 1
    self.state = new_game_state(self.rng)    #
    high_score не передан — снова 0!
```

```
# Игрок набрал 90 очков, проиграл, нажал R —
# табло снова показывает «Рекорд: 0», как будто игра
```

Что видно на экране

Раздел 19.25 объяснял: `high_score` обязан быть явно передан из старого `state` в новый. `new_game_state()` без аргумента `high_score` использует значение по умолчанию — ноль.

ИСПРАВЛЕННЫЙ КОД

restart_s_rekordom.py

```
def restart(self):
    self._generation += 1
    self.state = new_game_state(self.rng,
                                high_score=self.state.high_score)
```

DEBUG LAB 17: GAMESTATE ХРАНИТ ОБЪЕКТ TURTLE

gamestate_s_turtle.py

```
@dataclass
class GameState:
    snake: list[Position]
    head_turtle: turtle.Turtle # объект
    Turtle внутри модели!
```

```
# Тесты из раздела 19.30 падают с ошибкой создания Turtle
# хотя проверяют только математику координат — окно
```

Что видно на экране

Раздел 19.27 предупреждал об этом явно:

`GameState` — домен данных, а не контейнер для виджетов. Как только внутри появляется `turtle.Turtle`, создать состояние без реального окна становится невозможно — вся польза чистой логики (раздел 19.26) исчезает.

ИСПРАВЛЕННЫЙ КОД

gamestate_bez_turtle.py

```
@dataclass
class GameState:
    snake: list[Position]
    # объекты Turtle живут в SnakeApp, не в
    GameState
```

DEBUG LAB 18: ТИК ПЛАНИРУЕТ САМ СЕБЯ, ДАЖЕ КОГДА ИГРА УЖЕ НЕ RUNNING

```
tik_planiruet_vsegda.py
```

```
def _on_timer(self, generation):
    if generation != self._generation:
        return
    self.game_tick()
    self._schedule_next_tick()    # без
    проверки статуса!
```

```
# После Game Over или паузы змейка выглядит остановленной
# но цепочка ontimer() продолжает тикать вхолостую в
```

Что видно на экране

`game_tick()` сама по себе безопасна — она просто ничего не делает при `status != RUNNING`. Но если следующий тик планируется БЕЗУСЛОВНО, цепочка никогда не остановится сама, продолжая впустую расходовать таймер даже после конца игры.

ИСПРАВЛЕННЫЙ КОД

```
tik_planiruet_esli_running.py
```

```
def _on_timer(self, generation):
    if generation != self._generation:
        return
    self.game_tick()
    if self.state.status is
    GameState.RUNNING:
        self._schedule_next_tick()
```

Практика: находим баг «Змейки» по симптому

Автоматическая проверка — функция `diagnose()`: по симптому находим причину, неизвестный симптом тоже обрабатываем

Открыть практику → (<https://www.cartesianschool.org/practice/19-31/index.html>)

Snake Pro — финальная игра

Тот же самый прототип из раздела 19.1 — но теперь с моделью состояния, игровым тиком и архитектурой, которая выдержит рост проекта.

Итоговая программа

Финальное окно Snake Pro: несколько сегментов тела, яблоко, счёт и рекорд на тёмном поле

Реальное окно финальной версии — та же самая идея, что открывала главу в разделе 19.1, но теперь мы знаем, из чего она построена.

Файл целиком, самодостаточный и без невидимых зависимостей от других уроков:

`snake/snake.py` [projects/turtle/](#)

`snake.py` — структура

```

class Direction(Enum): ...
class GameState(Enum): ...

@dataclass
class GameState: ...

def next_head(head, direction): ...
def move_snake(snake, new_head, *, grow): ...
def is_wall_collision(head): ...
def is_self_collision(new_head, body_after_move): ...
def choose_food(snake, rng): ...
def calculate_delay(score): ...
def new_game_state(rng, *, high_score=0): ...

class SnakeApp:
    def __init__(self, *, rng=None): ...
    def bind_keys(self): ...
    def request_direction(self, direction): ...
    def game_tick(self): ...
    def toggle_pause(self): ...
    def restart(self): ...
    def render(self): ...
    def run(self): ...

def main():
    app = SnakeApp()
    app.run()

if __name__ == "__main__":
    main()

```

Запустите игру у себя

python snake.py в терминале. Стрелки или WASD — движение, Space — пауза, R — новая игра. Сравните с snake_basic.py (раздел 19.8) — та же идея игры, но совершенно другая внутренняя архитектура.

Чек-лист готового приложения

МОДЕЛЬ

- GameState — snake, direction, food: Position | None, score, status, delay_ms, без Turtle внутри
- чистые функции правил — next_head, move_snake, is_wall_collision, is_self_collision, choose_food, calculate_delay
- новая голова + старое тело вместо цикла по сегментам

ЦИКЛ

- тик через screen.ontimer(), без блокирующего цикла и time.sleep()
- клавиатура запрашивает направление, тик его применяет
- поколение (generation) инвалидируется сразу и при паузе, и при возобновлении — не только при restart()
- ровно одна активная цепочка тиков в любой момент времени

ФУНКЦИИ ИГРЫ

- pause/resume без пересоздания змейки
- restart с сохранением рекорда

- скорость растёт вместе со счётом, не опускаясь ниже минимума
- заполненное поле — явный `GameStatus.WON`, а не еда внутри змейки
- HUD — отдельная полоса над полем, а не поверх легальных клеток

ТЕСТИРОВАНИЕ

- правила протестированы без единого настоящего окна
- детерминированная еда через `random.Random(seed)`
- `render()` переиспользует пул Turtle-сегментов

Палитра игры

Финальная игра использует всего пять цветов — не для того, чтобы выглядеть скромно, а потому что различать голову, тело и еду проще по контрасту, а не по количеству оттенков:



Фон
#000000



Голова
#FFFFFF



Тело
#4ECDC4



Еда
#FF5D5D



Текст
#FFFFFF

Практика: Snake Pro целиком

Модуль turtle открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/19-32/index.html\)](https://www.cartesianschool.org/practice/19-32/index.html)

Итоги главы

От блокирующего цикла с глобальными переменными до модели состояния с игровым тиком, паузой, рестартом и проверяемыми правилами.

Итоги главы 19

Что мы построили и чему научились

- Игровой мир — дискретная решётка с шагом STEP; легальные позиции всегда кратны шагу, а не произвольны.
- Направление — не строка для сравнения в if/elif, а вектор смещения (DIRECTION_VECTORS), и запрет разворота на 180° проверяется против ТЕКУЩЕГО направления, а не уже запрошенного следующего.
- Игровой тик — вся цепочка правил целиком (направление → движение → еда → столкновения → рендер), а отрисовка — концептуально отдельный шаг.
- screen.ontimer() планирует следующий тик и сразу возвращает управление — вместо того чтобы блокировать программу в цикле while с time.sleep().
- Модель змейки — список логических позиций; новая голова плюс старое тело строит движение за один проход, без цикла с риском перепутать порядок.
- Столкновение с собой проверяется против тела ПОСЛЕ хода — иначе легальный заезд в клетку освободившегося хвоста ошибочно считается проигрышем.
- Явные состояния READY/RUNNING/PAUSED/GAME_OVER — пауза и рестарт не просто меняют картинку, а переключают состояние, от которого зависит, тикает ли игра вообще.
- Поколение (generation) защищает restart() от параллельных цепочек ontimer() — тот же класс проблемы, что и повторное нажатие кнопки в главе 16.
- GameState не хранит ни одного объекта Turtle — правила игры протестированы обычным assert, без единого настоящего окна.

Мост к главе 20

Дальше: полноценный игровой движок вместо Turtle

«Змейка» построена на том же Tkinter-цикле событий, что и рисовалка из главы 18 — удобно для учебных целей, но Turtle никогда не проектировался как игровой движок. В следующей главе — Pygame — появятся настоящие спрайты, обработка коллизий на уровне библиотеки и цикл кадров, спроектированный именно для игр реального времени.

Разработка игр с Pygame

Введение в разработку игр — жанры, платформы, мобильный геймдев — и Pygame как инструмент для практики основных принципов GameDev: игровой цикл, время кадра, спрайты, столкновения и архитектура своей игры.

20.1 Pygame и pygame-ce: что это и как установить

Устанавливаем и импортируем Pygame

20.2 Создаём окно игры и первый игровой цикл

Фон и очистка кадра

20.3 Рисуем первые игровые объекты

20.4 Управляем объектом с клавиатуры

События нажатия клавиш

20.5 Мини-проект: прыгающий мяч

20.6 Как устроена разработка игр

20.7 Жанры игр и игровые механики

20.8 Библиотека, фреймворк или движок: где находится Pygame

20.9 Игровые платформы: компьютеры, консоли и портативные устройства

20.10 Веб-игры, XR и облачный гейминг

20.11 Мобильные игры: сенсорный ввод и виртуальное управление

20.12 Экран мобильной игры: ориентация, пропорции и High-DPI

20.13 Ограничения мобильных игр: батарея, память и жизненный цикл

20.14 Как работает игровой цикл: ввод, обновление и отрисовка

20.15 FPS и время кадра

20.16 Delta time: движение независимо от FPS

20.17 Surface: поверхность для изображения и кадра

20.18 Rect: позиция, размер и координаты

20.19 Спрайты, изображения и ресурсы игры

20.20 Обработка ввода: события и удержание клавиш

20.21 Столкновения и хитбоксы

20.22 Основы игровой физики: скорость, гравитация и отскок

20.23 Спрайтовая анимация: кадры и спрайт-лист

20.24 Звуковые эффекты и музыка

20.25 Состояния игры

20.26 Структура небольшой игры: класс Game

20.27 Производительность: FPS и бюджет кадра

20.28 Как отлаживать игры

20.29 Собираем настольную версию игры

20.30 Запускаем Pygame-игру в браузере

20.31 Что реально возможно на Android, iOS и консолях

20.32 Профессии в разработке игр и портфолио

20.33 Финальный проект: «Прыгающий мяч» с архитектурой Game

Итоги главы

Pygame и pygame-ce

Специализированный инструмент для игр — с более тонким контролем над кадрами анимации, чем у Turtle и Tkinter.

Что такое Pygame и pygame-ce

Pygame — библиотека специально для игр: она даёт гораздо больше контроля над кадрами анимации, столкновениями и звуком, чем Turtle (главы 6–7, 12, 19)

или Tkinter (главы 16–18). В отличие от них, Pygame не входит в стандартную поставку Python — его нужно установить отдельно.

У Pygame есть два пакета с почти одинаковым названием, и это стоит сразу прояснить. Классический pygame — исходный проект. **pygame-ce** (Community Edition) — его активно поддерживаемое продолжение: API во многом совместим с классическим pygame, большинство базовых примеров pygame работает без изменений, а импорт остаётся тем же самым — `import pygame`. При этом pygame-ce получает новые версии и исправления быстрее и надёжнее устанавливается на актуальных версиях Python. В этой книге мы устанавливаем и используем pygame-ce.

Устанавливаем и импортируем Pygame

```
ustanovka.txt
```

```
pip install pygame-ce
```

Пакет и имя модуля — разные вещи

Название пакета (то, что вы ставите через pip) — pygame-ce. **Имя модуля** (то, что вы импортируете в коде) — по-прежнему pygame, без изменений: pygame-ce специально сохранил это имя, чтобы существующий код на pygame работал без правок. Весь код в этой книге пишется как `import pygame`.

Не держите pygame и pygame-ce в одном окружении одновременно

Оба пакета устанавливаются под одним и тем же именем модуля `pygame`, поэтому если в окружении случайно оказались сразу оба, непредсказуемо, какой из них реально импортируется. Если раньше вы уже устанавливали классический `pygame`, сначала удалите его: `pip uninstall pygame`, и только потом ставьте `pip install pygame-ce`. Новое обычное виртуальное окружение (глава 15) помогает изолировать зависимости проекта от глобальных пакетов. Но и внутри него не устанавливайте одновременно `pygame` и `pygame-ce`.

```
import pygame.py
```

```
import pygame
```

```
pygame.init() # в проектах этой книги мы начинаем
настройку Pygame с этого вызова
```

Практика: установка и первый импорт

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-02/index.html>)

Создаём окно игры и первый игровой цикл

Игровой цикл — сердце любой игры на Pygame: он пишется вручную, кадр за кадром.

Создаём окно игры

Экран Pygame — обычная поверхность (`Surface`) заданного размера:

igrovoj_ekran.py

```
import pygame

pygame.init()
screen = pygame.display.set_mode((600, 400))
pygame.display.set_caption("Моя первая игра")
```

Реальное окно Pygame: тёмно-синий экран 600×400, залитый цветом фона

Реальное окно: экран создан и залит цветом — уже настоящая, пусть и пустая, игра.

Игровой цикл

В отличие от Tkinter с его `mainloop()`, в Pygame игровой цикл пишут вручную — это даёт полный контроль над тем, что происходит на каждом кадре:

igrovoj_cikl.py

```
clock = pygame.time.Clock()
rabotaet = True

while работаet:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            работаet = False

    pygame.display.flip()    # показать нарисованный
    кадр
    clock.tick(60)          # не больше 60 кадров в
    секунду

pygame.quit()
```

Зачем нужен pygame.QUIT?

Без обработки события `pygame.QUIT` (нажатие на крестик окна) программа не узнает, что пользователь хочет закрыть игру, и окно перестанет отвечать. Обработка событий в начале каждого кадра цикла — обязательная часть любой программы на Pygame.

Фон и очистка кадра

`screen.fill(цвет)` закрашивает весь экран цветом (в виде кортежа RGB, глава 11) — обычно первая команда в каждом кадре, иначе на экране останутся «следы» от предыдущего кадра:

```
zalivka_fona.py
```

```
CVET_FONA = (20, 20, 40)
```

```
screen.fill(CVET_FONA)
```

Реальное окно: голубой квадрат ближе к правому краю тёмного экрана после серии кадров

Реальное окно: квадрат сдвигается на каждом кадре — то же самое движение, что и в разделе 20.4, но здесь важен именно сам цикл.

Практика: игровой цикл и заливка фона

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-02/index.html>)

Рисуем первые игровые объекты

Первые игровые объекты — простые фигуры, рисуемые заново на каждом кадре.

Игровые объекты в Pygame рисуют прямо на экране каждый кадр — простыми фигурами (кругами, прямоугольниками), как мы делали в Turtle, или готовыми изображениями. Начнём с фигур — они не требуют внешних файлов и отлично подходят для первых игр. Полноценных персонажей с изображениями и анимацией мы соберём позже (разделы 20.19 и 20.23) — здесь же важен сам принцип: объект на экране — это просто фигура, которую рисуют заново на каждом кадре в нужном месте.

personazhi.py

```
CVET_IGROKA = (100, 200, 255)
CVET_VRAGA = (255, 80, 80)

x_igroka, y_igroka = 300, 350
x_vraga, y_vraga = 300, 50

# внутри игрового цикла, после screen.fill(...):
pygame.draw.rect(screen, CVET_IGROKA, (x_igroka,
y_igroka, 40, 40))
pygame.draw.circle(screen, CVET_VRAGA, (x_vraga,
y_vraga), 20)
```

Реальное окно: голубой квадрат игрока внизу экрана и красный круглый врагверху на чёрном фоне

Реальное окно: те же две фигуры, что и в коде — прямоугольник и круг, каждый со своим цветом.

`pygame.draw.rect()` принимает кортеж (x, y, ширина, высота) — (x, y) это левый верхний угол, как у Canvas в главе 18, а не центр, как у `circle()`.

Rect — прямоугольник как объект

Для более сложных игр Pygame предлагает класс `pygame.Rect` — он хранит позицию и размер вместе и умеет проверять пересечения (`.colliderect()`) — то, что понадобится для космического шутера в главе 21.

Реальное окно: три цветных круга в ряд — красный, зелёный, синий — на чёрном фоне

Реальное окно: три врага разных цветов в ряд, нарисованные одним циклом `for`.

Практика: рисуем первые игровые объекты

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-03/index.html>)

Управляем объектом с клавиатуры

Движение — это просто изменение координат на каждом кадре; клавиатуру можно проверять двумя разными способами.

Движение объекта

«Движение» в Pygame — это просто изменение переменных позиции на каждом кадре перед тем, как объект будет нарисован заново:

```
peremeshenie.py
```

```
x_igroka += skorost_x
pygame.draw.rect(screen, CVET_IGROKA, (x_igroka,
y_igroka, 40, 40))
```

При реальных окнах рядом: квадрат игрока смещается вправо от кадра к кадру

Реальное окно: одна и та же переменная `x_igroka` в трёх последовательных снимках — движение и есть изменение координаты.

События нажатия клавиш

Есть два способа узнать о клавиатуре. Первый — через события, как в `pygame.event.get()` (удобно для разовых действий вроде прыжка):

`sobytiya_klavish.py`

```
for event in pygame.event.get():
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_SPACE:
            print("Прыжок!")
```

Второй способ — проверка текущего состояния клавиш на каждом кадре (удобнее для непрерывного движения, например, персонажа влево-вправо):

`sostoyanie_klavish.py`

```
klavishi = pygame.key.get_pressed()
if klavishi[pygame.K_LEFT]:
    x_igroka -= 5
if klavishi[pygame.K_RIGHT]:
    x_igroka += 5
```

Какой способ выбрать?

События (`event.type == pygame.KEYDOWN`) подходят для действий «один раз за нажатие» — прыжок, выстрел, пауза. `get_pressed()` подходит для непрерывного движения, пока клавиша зажата, — именно так реализовано управление кораблём в главе 21.

Практика: движение и клавиши

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/20-04/index.html\)](https://www.cartesianschool.org/practice/20-04/index.html)

Мини-проект: прыгающий мяч

Первая настоящая мини-игра на Pygame — движение, отскоки и игровой цикл в одном коротком проекте.

Соберём всё в один мини-проект: мяч, который отскакивает от всех четырёх стен экрана.

```
прыгающий_мяч.py
```

```
import pygame

SHIRINA, VYSOTA = 600, 400
RADIUS = 20

pygame.init()
screen = pygame.display.set_mode((SHIRINA, VYSOTA))
clock = pygame.time.Clock()

x, y = SHIRINA // 2, VYSOTA // 2
dx, dy = 4, 3

rabotaet = True
while работаet:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            работаet = False

    x += dx
    y += dy

    if x - RADIUS < 0 or x + RADIUS > SHIRINA:
        dx = -dx
    if y - RADIUS < 0 or y + RADIUS > VYSOTA:
```

```

        dy = -dy

    screen.fill((20, 20, 40))
    pygame.draw.circle(screen, (255, 100, 100),
        (int(x), int(y)), RADIUS)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()

```

Гри реальных окна рядом: красный мяч в разных точках тёмно-синего поля

Реальное окно: тот же мяч из `projects/pygame/bouncing-ball/bouncing_ball_basic.py` в трёх последовательных положениях траектории.

dx = -dx — отражение одной строкой

Смена знака скорости на противоположный — стандартный приём отражения от стены: движение продолжается с той же скоростью, только в обратную сторону, без дополнительных проверок направления.

Полный, уже проверенный файл — отдельно:

[projects/
pygame/bouncing-ball/bouncing_ball_basic.py](#)

★★ Самостоятельная задача

Меняем цвет при отскоке

При каждом отскоке от стены выбирайте новый случайный цвет мяча (`random.choice()` из главы 5/11).

★★★ Задача повышенной сложности

Два мяча

Добавьте второй мяч с собственными x , y , dx , dy — оба должны двигаться и отскакивать независимо.

Практика: прыгающий мяч

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-05/index.html>)

Ограничения этой версии

Мяч уже прыгает, но у этой версии есть слабые места: скорость отскока зависит от частоты кадров конкретного компьютера, а весь код лежит в глобальных переменных без какой-либо структуры. Дальше в этой главе мы разберём, откуда берутся оба ограничения и как до конца главы вырастить этот же проект в профессионально организованную версию — с классом `Game`, паузой и движением, которое не зависит от частоты кадров.

Как устроена разработка игр

Прежде чем углубляться в Pygame — посмотрим, что такое разработка игр как ремесло: роли, масштаб и путь от идеи до релиза.

Мяч из прошлого раздела уже прыгает — самое время отступить на шаг назад и посмотреть, в какой мир мы только что вошли. Разработка игр (гейм-девелопмент, геймдев) — это не один навык, а ремесло на стыке программирования, дизайна и искусства. Небольшую игру вполне может создать один человек — вы только что это сделали. По мере роста проекта обычно растёт и число специализированных ролей, а крупные коммерческие игры создают большие команды.

Роли в разработке игры

В крупной студии за каждую из этих ролей отвечает отдельный человек или целая команда; в инди-проекте один и тот же человек часто совмещает три-четыре роли сразу — и вы, работая над мини-проектами этой главы, уже побывали в роли программиста, дизайнера и тестировщика одновременно.

Кто делает игру

ГЕЙМ-ДИЗАЙНЕР

Придумывает правила игры

Балансирует сложность

Пишет документ игрового дизайна

ПРОГРАММИСТ

Реализует правила в коде

Пишет игровой цикл

Оптимизирует производительность

ХУДОЖНИК / АНИМАТОР

Рисует спрайты и фон

Готовит анимацию персонажей

Задаёт визуальный стиль

ЗВУКОРЕЖИССЁР

Пишет музыку

Записывает звуковые эффекты

Сводит звук под события игры

ПРОДЮСЕР

Планирует сроки

Управляет командой

Следит за бюджетом

QA-ТЕСТИРОВЩИК

Ищет баги

Проверяет баланс

Играет в игру сотни раз до релиза

Инди и AAA

Инди-игры обычно создают независимые разработчики или небольшие команды, часто без крупного издателя. «AAA» — условное обозначение крупных коммерческих проектов с большими командами, значительными бюджетами и длительным производственным циклом. Это не строгие категории с чёткой границей: реальные проекты сильно отличаются по масштабу, и немало команд находится где-то посередине.

Характеристика	Инди-проект	Крупный AAA-проект
Команда	От одного разработчика до небольшой или средней команды	Обычно крупная команда специалистов разных направлений
Срок разработки	От нескольких месяцев до нескольких лет	Обычно несколько лет
Принятие решений	Решения часто принимаются быстрее и меньшим числом людей	Изменения могут требовать согласования между несколькими командами
Инструменты	Готовые движки, библиотеки и доступные инструменты разработки	Готовые или собственные технологии и внутренние инструменты
Сильная сторона	Гибкость и свобода эксперимента	Масштаб производства и большие ресурсы

Ругайте для первых инди-прототипов

Учебные проекты этой главы написаны в том же духе, что и многие инди-игры: маленькая команда (в вашем случае — один человек), общедоступный инструмент, быстрые эксперименты. Разница между учебным проектом и реальным инди-релизом обычно не в инструменте, а в том, сколько итераций отделяет прототип от готовой игры.

Путь от идеи до релиза

Ни одна игра не появляется сразу готовой. У процесса есть распространённые стадии — в конкретном проекте их названия и границы могут отличаться, но общая логика похожа и у AAA-студии, и у одного человека с ноутбуком:

Концепт

идея, ключевая механика
на бумаге или в голове



Пре-продакшн

план, оценка объёма работы
выбор инструментов и технологий

если механика не работает — переделать концепт



Прототип

проверяет саму механику

минимум кода, часто одноразового

показывает, каким будет качество релиза



Vertical slice

небольшой готовый кусок игры
близко к качеству финального релиза



Продакшн

основной объём кода и контента
уровни, персонажи, звук



Альфа

в конкретном проекте критерии могут отличаться

обычно: основной функционал уже есть, тестирование внутри команды



Бета

обычно: контент почти не меняется
тестирование снаружи, охота за багами



Релиз

игра выходит к игрокам

жизнь игры продолжается после релиза



Пострелизная поддержка

патчи, баланс, новый контент

Мяч, отскакивающий от стен, — это уже рабочий прототип: минимальная механика, которую можно попробовать. Это один из распространённых вариантов пути от идеи до релиза — не единственно верная схема.

Прототип и vertical slice — разные задачи

Прототип отвечает на вопрос «работает ли сама идея» — код в нём почти всегда одноразовый, а качество картинки и текста не имеет значения. Vertical slice отвечает на другой вопрос: «как будет выглядеть и ощущаться готовая игра» — это уже небольшой, но полноценный кусок, близкий к целевому качеству, и он часто заодно проверяет производственный конвейер проекта. На практике команда может выстроить техническую архитектуру (раздел 20.26) раньше, ещё на этапе прототипа, а задачи прототипа и vertical slice — пересекаться: разные проекты организуют эти стадии по-своему.

Жанры игр и игровые механики

Жанр — это в первую очередь механика: что именно игрок делает от кадра к кадру.

Игры принято делить на жанры — не строгую научную классификацию, а удобный общий язык: когда игрок слышит «платформер» или «головоломка», у него уже

есть ожидания о том, что внутри. Жанр — это в первую очередь про **механику** (что игрок делает), а не про сюжет или картинку.

Экшен

Экшен-игры строятся вокруг быстрой реакции и координации — это скорее большая семья, чем один жанр:

Экшен

ПЛАТФОРМЕР

Бег и прыжки по уровню

Точный тайминг прыжка

Пример механики: гравитация + столкновение с платформой

ШУТЕР

Стрельба по целям

Быстрая реакция

Пример: пуля — это ещё один движущийся объект

ФАЙТИНГ

Поединок один на один или команда на команду

Точные тайминги ударов и блоков

Пример: конечный автомат состояний персонажа

BEAT 'EM UP

Бой с несколькими противниками сразу

Продвижение по уровню волнами врагов

Пример: список активных врагов, проверка столкновений с каждым

Приключения и ролевые игры

Приключения и RPG

ПРИКЛЮЧЕНИЕ (ADVENTURE)

Исследование мира и решение задач по сюжету

Реакция обычно не критична

Пример: состояние игры как машина состояний

POINT-AND-CLICK КВЕСТ

Взаимодействие с объектами сцены
кликом

Головоломки, встроенные в сюжет

Пример: список интерактивных объектов
на экране

ACTION RPG

Развитие персонажа плюс бой в реальном
времени

Характеристики и прогресс

Пример: инвентарь – структура данных
из главы 11

ПОШАГОВАЯ RPG

Развитие персонажа плюс бой по очереди
Меньше нагрузки на реакцию, больше –
на тактику

Пример: очередь ходов персонажей

ТАКТИЧЕСКАЯ RPG

Бой на клетчатом поле

Позиция и порядок ходов решают исход

Пример: игровое поле – сетка клеток с занятостью

Стратегии и симуляторы

Стратегии и симуляторы

RTS (СТРАТЕГИЯ В РЕАЛЬНОМ ВРЕМЕНИ)

Управление ресурсами и юнитами без остановки игры

Планирование одновременно с реакцией

Пример: юниты – список объектов с состоянием

ПОШАГОВАЯ СТРАТЕГИЯ

Планирование на много ходов вперёд

Время между ходами не ограничено

Пример: очередь ходов, как в пошаговой RPG

4X

Освоение территории, развитие,
дипломатия, война

Обычно длинные партии на большой карте

Название – от четырёх целей: eXplore,
eXpand, eXploit, eXterminate

TOWER DEFENSE

Расстановка защитных объектов на пути
врагов

Волны противников по расписанию

Пример: список башен, список врагов,
проверка дальности

СИМУЛЯТОР ТРАНСПОРТА

Управление одной машиной, самолётом и
похожим

Физика движения – центральная механика

Пример: разгон, торможение, инерция

СИМУЛЯТОР ЖИЗНИ

Моделирование быта или отношений персонажей

Прогресс измеряется показателями, а не боем

Пример: набор числовых характеристик персонажа

МЕНЕДЖМЕНТ / CITY BUILDER

Строительство и управление системой (город, ферма, база)

Баланс ресурсов важнее реакции

Пример: числовая модель + визуализация

Ещё жанры, которые стоит знать

Ещё жанры

СПОРТИВНЫЕ ИГРЫ

Симуляция реального вида спорта

Правила задаёт сам вид спорта

ГОНКИ

Управление транспортом на трассе

Соревнование на время

Пример: движение с ускорением и
торможением

ГОЛОВОЛОМКА

Логика важнее реакции

Чёткие правила победы

Пример: игровое поле – сетка состояний

АРКАДА

Простые правила, растущая сложность

Счёт и рекорды

Пример: наш прыгающий мяч

SURVIVAL

Выживание во враждебной среде

Управление ресурсами: здоровье, голод,
укрытие

SANDBOX

Свобода действий важнее заданной цели
Игрок сам ставит себе задачи

ROGUELIKE / ROGUELITE

Случайно генерируемые уровни
Смерть обычно начинает партию заново
Пример: random из главы 5, но на уровне целого забега

ЦИФРОВЫЕ КАРТОЧНЫЕ И НАСТОЛЬНЫЕ ИГРЫ

Цифровая версия карточной или настольной механики
Пример: колода и рука — списки карт

ММО / ОНЛАЙН- МНОГОПОЛЬЗОВАТЕЛЬСКИЕ

Много игроков в общем мире
одновременно
Требует сетевого взаимодействия — тема отдельной книги

IDLE / INCREMENTAL

Прогресс идёт даже без активных действий игрока

Пример: числа растут по формуле, привязанной к времени

КАЗУАЛЬНЫЕ И HYPERCASUAL

Очень простые правила, короткая сессия

Часто одна механика на всю игру — как наш прыгающий мяч

Жанры смешиваются

На практике мало какая игра — это чистый один жанр из учебника: шутер с элементами RPG (прокачка оружия), платформер-головоломка (уровень решается прыжками в правильном порядке), стратегия в реальном времени с элементами симулятора. Границы между соседними жанрами в списке выше часто размыты — Roguelike пересекается с RPG, tower defense — со стратегией, idle — с симулятором. Классификация — инструмент общения, а не клетка, в которую обязана поместиться идея.

С чего начать первый проект?

Люблю
точный
тайминг → Платформер / шутер / аркада
и
реакцию

Люблю
просчитывать
наперёд, а не → Стратегия / головоломка
реагировать

Хочу
рассказывать
историю через → Приключение / RPG
игру

Хочу смоделировать что-то
из реального мира → Симулятор

Жанр и первый проект

Прыгающий мяч из раздела 20.5 — учебная аркада: простое правило (отскок от стен) плюс счёт очков. Проект следующей главы — космический шутер — устроен так же по духу, но добавляет стрельбу и столкновения с врагами. Понимание жанра заранее помогает предсказать, какие механики (главы 20.21–20.22) вам понадобятся раньше остальных.

Библиотека, фреймворк или движок: где находится Pygame

Pygame — это библиотека, а не движок: она даёт кирпичики, а структуру игры вы строите сами.

«Разработка игр» звучит как один инструмент, но на самом деле это целая лестница инструментов с разной степенью готовой структуры — и Pygame стоит на конкретной ступеньке этой лестницы, не на самом верху и не на самом низу.

Три уровня инструментов

От кирпичиков к готовой мастерской

БИБЛИОТЕКА

Даёт отдельные «кирпичики»: окно, отрисовку, звук, ввод
Структуру приложения строите сами
Pygame — именно библиотека

ФРЕЙМВОРК

Задаёт общую структуру приложения
Вы заполняете предусмотренные места своим кодом

Пример духа подхода: Flask из главы 22 для веба

движок

Полная среда: визуальный редактор, физика, свет, звук из коробки
Пишете код меньше, настраиваете компоненты больше

Примеры: универсальные игровые движки с редактором сцен

Pygame — это библиотека: она даёт окно, поверхность для рисования, обработку клавиш и звук, но игровой цикл, столкновения и структуру всего проекта вы пишете сами, своими руками. Это осознанный компромисс между уровнем контроля и скоростью старта:

	Pygame (библиотека)	Типичный игровой движок
Кривая обучения	Нужно понимать цикл, кадры, координаты	Многое можно сделать без кода вовсе
Контроль над деталями	Более прямой контроль над циклом и низкоуровневые решениями	Больше готовой структуры; степень контроля зависит от движка

	Pygame (библиотека)	Типичный игровой движок
Визуальный редактор сцен	Нет — сцена собирается кодом	Обычно есть
Встроенная физика/свет	Нет — реализуете сами или подключаете отдельно	Обычно есть из коробки
Что получаете взамен	Глубокое понимание того, как игра устроена изнутри	Скорость сборки крупного проекта

Pygame и SDL

Сам Pygame не рисует пиксели и не говорит со звуковой картой напрямую — под капотом он опирается на библиотеку **SDL** (Simple DirectMedia Layer), написанную на C: она отвечает за низкоуровневую работу с окном, вводом, графикой и звуком на разных операционных системах. Pygame даёт этому питоновскую, удобную обёртку.

Зачем учиться на библиотеке, если есть движки

Понятия, которые вы осваиваете в этой главе — игровой цикл, кадр, время между кадрами, система координат, столкновение прямоугольников — переносятся между инструментами разработки игр в своих основных идеях, включая крупные визуальные движки. Но конкретная архитектура, API, физический timestep, модель сцен и система столкновений могут сильно отличаться, а движок к тому же часто прячет эти понятия за интерфейсом кнопок и слайдеров. Изучив их напрямую через код, вы будете лучше понимать, что происходит за этими кнопками — а не только уметь их нажимать.

Игровые платформы: компьютеры, консоли и портативные устройства

Игра пишется под конкретное устройство — и Rugame нацелен прежде всего на десктоп.

Игра почти никогда не пишется «вообще», а пишется под конкретную платформу. От платформы зависит и ввод (клавиатура или тачскрин?), и распространение (файл для скачивания или магазин приложений?), и технические ограничения (сколько памяти доступно?). Стоит сразу различать два разных вопроса: на каком **устройстве** запускается игра — и каким **способом** она до этого устройства доходит. Первое — это десктоп, мобильные, консоли, портативные устройства, XR; второе (например, облачный гейминг из раздела 20.10) — это скорее способ доставки игры, чем отдельный тип устройства: облачный сервис в итоге всё равно исполняет игру на каком-то настоящем компьютере или сервере.

Целевые устройства

Целевые устройства

ДЕСКТОП

Windows, macOS, Linux

Клавиатура, мышь, геймпад

Прямая раздача файла или магазин вроде Steam

КОНСОЛИ

PlayStation, Xbox, Nintendo и похожие
Официальный набор разработчика (SDK) и
сертификация
Издание требует соглашения с
производителем консоли

ПОРТАТИВНЫЕ УСТРОЙСТВА

Портативная консоль производителя или
портативный ПК с полноценной ОС
Тачскрин, физические кнопки или и то и
другое
Часто ограничены по батарее и памяти
сильнее настольного устройства

МОБИЛЬНЫЕ

Android, iOS
Тачскрин, акселерометр
Магазины приложений – раздел 20.11–
20.13

ВЕБ

Запускается прямо в браузере

Не требует установки

WebAssembly – раздел 20.30

XR (VR/AR)

Гарнитуры виртуальной и дополненной реальности

Сtereo-рендеринг, отслеживание головы и рук

Отдельная, узкоспециализированная разработка

Портативное устройство — не всегда игровая консоль

Под «портативным устройством» может скрываться две разные вещи: портативная консоль производителя (со своей закрытой экосистемой и SDK, как у больших консолей) или портативный ПК с обычной настольной операционной системой, на котором способна запускаться любая десктопная программа, включая написанную на PUGAME. Разница принципиальна для распространения игры, и её стоит уточнять для конкретного устройства, а не считать «портативное» одной однородной категорией.

Десктоп — родная платформа Pygame

Pygame изначально и прежде всего — инструмент для десктопных игр на Windows, macOS и Linux. Всё, что вы написали в этой главе, запускается как обычная программа: двойной клик по файлу (после упаковки, раздел 20.29) или запуск через `python имя_файла.py`.

Консоли и портативные консоли

Здесь нужна честность: у Pygame нет официальной поддержки игровых консолей. Издание игры на консоли требует официального набора инструментов разработчика (SDK) от производителя самой консоли, подписания соглашения о разработке и прохождения сертификации — это отдельный, закрытый процесс, не имеющий отношения к Pygame или Python вообще.

Консоли — не путь Pygame

Если конечная цель — выпустить игру на консоли, Pygame не тот инструмент, с которого стоит начинать этот конкретный путь: консольная разработка почти всегда идёт через официальные движки и SDK производителя. Но навыки, полученные здесь — игровой цикл, работа с кадрами, архитектура — переносятся в любой из этих инструментов напрямую, без потерь.

Веб-игры, XR и облачный гейминг

Три способа добраться до игрока, устроенные совсем не как обычный десктопный запуск.

Веб: игра прямо в браузере

Веб-игра не требует установки — игрок открывает ссылку и играет прямо в браузере. Между вашим Python-кодом на Pygame и браузером стоит несколько слоёв, а не один волшебный переключатель:

Исходный код на Pygame

обычный `import pygame`



Python-рантайм, собранный под WebAssembly

готовит инструмент вроде `pygbag` —
раздел 20.30



WebAssembly (WASM)

компактный байт-код, который браузер
умеет исполнять быстро



Браузер

Canvas для картинки, Web Audio для
звука, обработчики ввода

WebAssembly — это формат байт-кода, который исполняет браузер; сам по себе он не запускает обычный десктопный Python — для этого нужен отдельно собранный рантайм и инструмент вроде `pygbag`.

XR: виртуальная и дополненная реальность

XR (extended reality) объединяет виртуальную реальность (VR, полностью цифровое окружение в гарнитуре) и дополненную реальность (AR, цифровые объекты поверх реального мира). Разработка под XR требует стерео-рендеринга (отдельная картинка для каждого глаза), отслеживания положения головы и рук и жёстких требований к частоте кадров, чтобы не вызвать у игрока укачивание. Это отдельная, узкоспециализированная область разработки — Rугame не предназначен для XR.

Облачный гейминг — способ доставки, а не отдельное устройство

При облачном гейминге игра выполняется на сервере в дата-центре — фактически на обычном десктопном или серверном оборудовании, — а на устройство игрока в реальном времени передаётся только видеопоток, как видеозвонок с игрой на том конце. Поэтому облачный гейминг честнее описывать не как ещё одну платформу наравне с десктопом или мобильными, а как способ *доставки* уже существующей десктопной (или консольной) версии игры до экрана, на котором её физически не запускали. Это снимает требования к мощности устройства игрока, но требует стабильного и быстрого интернет-соединения.

Сводка: где Rugame подходит напрямую

Куда	Типичный ввод	Как игра попадает к игроку	Rugame — подходящий инструмент?
Десктоп	Клавиатура, мышь, геймпад	Скачивание файла или магазин вроде Steam	Да — родная платформа
Консоли	Геймпад производителя	Официальный магазин консоли, через SDK	Нет — нужен официальный SDK
Портативные консоли	Тачскрин или кнопки	Магазин устройства	Нет напрямую — см. раздел 20.13
Мобильные	Тачскрин, акселерометр	App Store, Google Play	Только через инструменты сообщества — раздел 20.31
Веб	Клавиатура, мышь, тач	Ссылка в браузере	Да, через rugarbag — раздел 20.30
XR	Контроллеры, отслеживание рук	Магазин гарнитуры	Нет — нужен XR-движок

Облачный гейминг в эту таблицу намеренно не включён: он не меняет ответ на вопрос «подходит ли Rugarbag», а лишь добавляет ещё один способ доставить уже готовую десктопную сборку до экрана игрока.

Где Pygame сильнее всего

Среди целевых устройств Pygame увереннее всего чувствует себя на десктопе и способен, с оговорками и через инструменты сообщества, дотянуться до веба и мобильных устройств (разделы 20.11–20.13, 20.30–20.31). Знать эти границы заранее избавляет от разочарования на позднем этапе проекта — гораздо лучше выбрать целевое устройство под инструмент, чем упереться в инструмент, не подходящий под уже выбранное устройство.

Мобильные игры: сенсорный ввод и виртуальное управление

На мобильном устройстве основной способ ввода — сенсорный экран, и это меняет правила управления игрой.

Мобильные игры охватывают огромную аудиторию, и у телефонов и планшетов другой основной способ ввода: встроенный сенсорный экран вместо клавиатуры и мыши. Внешние клавиатура, мышь или геймпад тоже могут быть подключены к телефону или планшету, но рассчитывать на них как на основной способ управления не стоит — игра должна полноценно работать с одним лишь тачскрином. Начнём с того, как он устроен.

Тачскрин и мультитач

Прикосновение к экрану устройство сообщает игре примерно так же, как Pygame сообщает о клавише — событием с координатами. Ключевая разница в том, что у экрана может быть сразу **несколько** одновременных точек касания (мультитач) — например, один палец двигает персонажа, а другой в это же время нажимает кнопку стрельбы. В настольном Pygame у события мыши всегда одна точка и одна кнопка; в мобильной разработке приходится с самого начала проектировать под несколько независимых точек касания сразу.

Виртуальные элементы управления

Без физической клавиатуры кнопки движения, прыжка и стрельбы приходится рисовать прямо на экране — виртуальный джойстик, виртуальные кнопки. Это не мелочь, а отдельная часть гейм-дизайна: виртуальные элементы управления должны быть достаточно крупными для пальца (палец гораздо неточнее указателя мыши), не перекрывать важную часть экрана и оставаться отзывчивыми даже при быстрых движениях.

Реальное окно: круглый виртуальный джойстик слева и две круглые виртуальные кнопки справа на тёмном фоне

Реальное окно: мокап виртуальных элементов управления, нарисованный обычными `pygame.draw.circle()` — крупные зоны под палец, а не под курсор мыши.

	Клавиатура + мышь (десктоп)	Тачскрин (мобиль- ные)
Точность указа- ния	Высокая — пик- сель под курсором	Ниже — палец шире курсора
Число одновре- менных вводов	Ограничено коли- чеством клавиш	Несколько независимых каса- ний (мультиач)
Тактильная обратная связь	Физический ход клавиши при на- жатии	Экран плоский, но устройство может ответить про- граммной вибра- цией
Место на экране	Не занимает игро- вое поле	Виртуальные кнопки перекры- вают часть экрана

Игровое действие как абстракция

Чтобы одна и та же игровая логика работала и с клавиатурой, и с геймпадом, и с тачскрином, полезно отделить *игровое действие* («двигаться вправо», «прыгнуть») от конкретного физического ввода, который его вызывает. Тогда «прыгнуть» может означать нажатие пробела на десктопе, кнопку A на геймпаде или тап по виртуальной кнопке на телефоне — а код игры, который реагирует на «прыгнуть», не меняется вообще:

input_action.py

```
# Схематично: один и тот же вызов не зависит от
# источника ввода
def obrabotat_dejstviya(igra):
    if igra.dejstvie_nazhato("прыжок"):
        igrok.prygnut()

# dejstvie_nazhato() внутри проверяет либо клавишу,
# либо геймпад,
# либо попадание тапа в область виртуальной кнопки —
# игрок этого не видит
```

Абстракция ввода — не только для мобильных

Такое разделение полезно даже для чисто десктопной игры: оно позволяет позже добавить поддержку геймпада или перенастраиваемые клавиши, не переписывая игровую логику. Архитектура класса `Game` в разделе 20.26 закладывает именно это разделение с самого начала.

Экран мобильной игры: ориентация, пропорции и High-DPI

Мобильный экран не похож на десктопное окно: он поворачивается, у него нет стандартного соотношения сторон и часто высокая плотность пикселей.

На десктопе игра почти всегда получает окно фиксированного или свободно меняемого размера, которое выбирает сам игрок. Мобильный экран устроен иначе — и у него есть сразу несколько собственных источников сложности: ориентация, соотношение сторон, безопасная область и высокая плотность пикселей.

Ориентация экрана

Телефон можно держать вертикально (portrait) или горизонтально (landscape), и игрок вправе повернуть устройство в любой момент. Игра должна либо явно ограничить себя одной ориентацией (частый выбор для аркад и головоломок), либо честно пересчитывать раскладку экрана при повороте — то есть повторно решать, куда поместить игровое поле и виртуальные кнопки управления из раздела 20.11.

Соотношение сторон и безопасная область

Экраны разных телефонов сильно отличаются по соотношению сторон — в отличие от десктопных мониторов, где почти всегда широкоформатный прямоугольник, здесь диапазон гораздо шире, от почти квадратных до сильно вытянутых. Дополнительно у многих современных экранов есть вырезы под камеру и закруглённые углы — область экрана, физически видимую пользователю, но небезопасную для важных элементов интерфейса. Разработчики называют пригодную для контента часть экрана **safe area** (безопасная область) — счёт и важные кнопки размещают внутри неё, а не впритык к самому краю экрана.

Реальное окно: тёмно-красная рамка по краю экрана и зелёная безопасная зона с подписью safe area внутри неё, с отступом от края

Реальное окно: безопасная область — прямоугольник с отступом от каждого края, куда и помещают счёт и важные кнопки.

Тот же принцип — на самом сайте книги

Идея safe area знакома вам не только из мобильной разработки: сайт этой книги тоже проектируется с оглядкой на маленькие экраны — контент не прижимается к самому краю на маленьких телефонах, а оставляет отступ по краям именно по этой причине.

Разрешение и плотность пикселей — разные вещи

Разрешение — это просто количество пикселей экрана, например 2400×1080 . **Плотность пикселей** (PPI/DPI, pixels per inch) — это то, насколько плотно эти пиксели упакованы на реальном физическом размере экрана. У современных телефонов физический экран небольшой, а пикселей на нём часто больше, чем на заметно более крупном настольном мониторе — то есть плотность выше при сопоставимом или даже меньшем разрешении. Экран с высокой плотностью принято называть high-DPI.

Отсюда третье понятие — **логический размер**: то, в каких единицах ваш код и интерфейс думают об экране, в отличие от того, сколько на нём физических пикселей на самом деле:

Логическое разрешение игры

в этих единицах вы задаёте позиции и размеры в коде



Масштабирование

система или движок пересчитывают логические координаты в физические



Физические пиксели экрана

то, что реально светится на high-DPI дисплее

Изображение, нарисованное «один логический пиксель — один физический», на high-DPI экране выглядит мелким; поэтому растровые изображения часто готовят в нескольких разрешениях, а интерфейсные элементы, где это возможно, рисуют векторно.

Особенность экра-на	Десктоп	Мобильные
Ориентация	Обычно фиксиро-ванная, не меняет-ся во время игры	Может поменять-ся в любой момент
Соотношение сторон	Стандартный уз-кий диапазон	Широкий разброс между устройства-ми
Вырезы и за-круглённые углы	Практически не встречаются	Обычное дело — нужна safe area
Плотность пик-селей	Обычно ниже	Часто заметно выше — нужен масштаб под high-DPI

Ограничения мобильных игр: батарея, память и жизненный цикл

Батарея, перегрев, память, жизненный цикл приложения — и реальный ответ на вопрос о публикации Pygame-игры в App Store и Google Play.

Разберём ограничения мобильной разработки, которые не видны на первый взгляд, и вопрос, который рано или поздно возникает у каждого, кто дошёл до этого места: можно ли просто взять и опубликовать Pygame-проект в App Store и Google Play?

Батарея и перегрев

Телефон работает от ограниченной батареи и имеет ограниченный тепловой бюджет — заметно более жёсткий, чем у ноутбука, который часто подключён к розетке или несёт батарею большей ёмкости. Игровой цикл, который держит процессор загруженным на полную мощность без остановки (см. раздел 20.15 про `clock.tick()`), длительной высокой нагрузкой CPU/GPU ускоряет разряд батареи и может привести к `thermal throttling` — системному снижению частоты процессора для защиты от перегрева, из-за которого игра начинает тормозить не из-за плохого кода, а из-за самой этой защиты.

Память

Мобильные устройства обычно располагают заметно меньшим объёмом оперативной памяти, чем настольный компьютер, и операционная система гораздо агрессивнее следит за приложениями, которые расходуют её слишком много.

Жизненный цикл приложения

Настольные приложения тоже не работают в вакууме: они могут терять фокус, засыпать вместе с компьютером или закрываться из-за сбоя. Но на телефоне операционная система управляет жизненным циклом приложения заметно активнее и чаще: она может в любой момент отправить игру в фон (пришёл звонок, игрок свернул приложение) и позже либо вернуть её в точности с того места, либо — если памяти не хватает — выгрузить из памяти полностью, и при повторном запуске игра стартует заново. Мобильная игра должна корректно переживать переходы между foreground и background. Если потеря текущего состояния важна для игрока, приложение должно сохранять нужный прогресс или checkpoint до возможного завершения процесса.

Магазины приложений, реклама и покупки внутри игры

Публикация на телефон почти всегда означает публикацию через магазин приложений — Google Play для Android или App Store для iOS. У обоих есть процесс проверки перед публикацией, правила оформления страницы приложения и (для многих категорий игр) типичная бизнес-модель через показ рекламы или покупки внутри игры (in-app purchases, IAP) — это отдельная большая тема бизнеса мобильных игр, выходящая за рамки этой книги, но полезно знать, что она существует и влияет на то, как устроены многие мобильные игры «изнутри».

Публикация Pygame-игры на Android и iOS

У Pygame нет встроенного экспорта на Android и iOS

У Pygame и pygame-ce нет официальной, встроенной команды «собрать APK» или «собрать приложение для iOS». На момент проверки в основном репозитории стороннего инструмента `python-for-android` нет принятого официального рецепта pygame-ce — есть только открытые community pull request'ы и пользовательские рецепты, поэтому экспериментальная сборка на Android возможна, но её нельзя считать стандартным или гарантированно поддерживаемым путём. Публикация на iOS возможна через отдельный инструмент сообщества (шаблон проекта плюс сборка в Xcode), автор которого сам описывает его как недостаточно протестированный и не готовый к продакшену, а сборки для App Store лично не проверял. Оба пути требуют заметно больше шагов, чем `pip install`, и результат стоит проверять на реальном устройстве, а не только предполагать по документации инструмента.

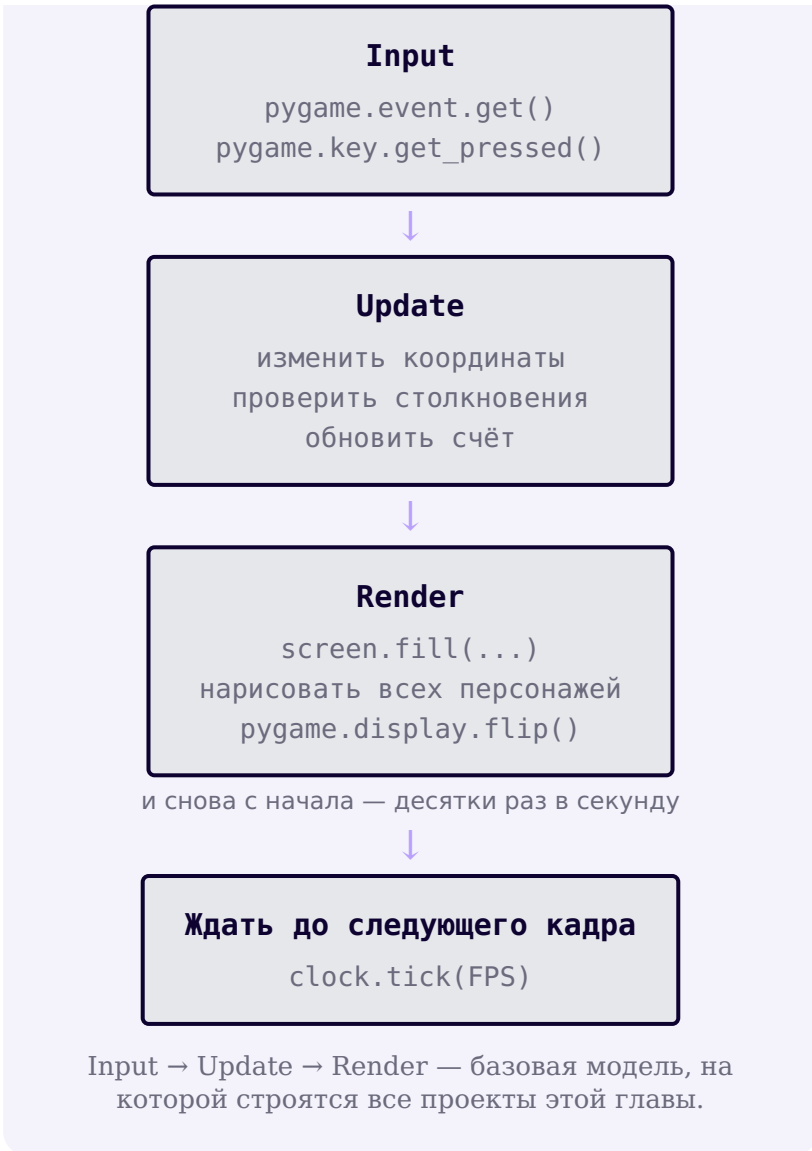
Это не значит, что мобильный Pygame-проект невозможен — раздел 20.31 подробно разбирает оба пути (для Android и для iOS) и то, как они выглядят на практике. Но важно закладывать это в план проекта заранее, а не в последний момент: если конечная цель — магазины приложений, ограничения этого раздела (энергопотребление, память, жизненный цикл, сборка через инструменты сообщества) стоит учитывать с самого начала работы над проектом, а не пытаться исправить их постфактум, когда игра уже написана под десктопные предположения.

Ограничение	Десктоп	Мобильные
Источник питания	Часто розетка или батарея, рассчитанная на долгую работу	Всегда ограниченная батарея
Реакция на нагрузку	Редко становится проблемой	Перегрев снижает производительность
Оперативная память	Обычно много	Заметно меньше, строже лимиты
Управление жизненным циклом со стороны ОС	Менее активное	Активное — фон, выгрузка, перезапуск
Путь публикации	Прямая раздача файла или Steam	Магазин приложений плюс сборка сторонним инструментом

Как работает игровой цикл: ввод, обновление и отрисовка

Базовая модель игрового цикла в этой книге — три повторяющиеся фазы: обработать ввод, обновить состояние, нарисовать кадр.

В разделе 20.2 вы уже написали игровой цикл — но теперь, зная контекст всей главы, стоит назвать его части точными именами. Для наших Rugame-игр мы используем базовую модель: любой цикл в этой книге, от простейшего мяча до финального проекта, состоит из трёх повторяющихся фаз.



Порядок фаз важен: ввод обрабатывается раньше обновления состояния (иначе персонаж реагирует на клавишу с опозданием на кадр), а обновление — раньше

отрисовки (иначе на экране окажется прошлый, ещё не обновлённый кадр). В разделе 20.26 эти три фазы станут тремя методами класса `Game`.

У больших движков схема сложнее

`Input` → `Update` → `Render` — надёжная база для учебных и небольших проектов, но не единственно возможная схема. Крупные игровые движки нередко используют фиксированный шаг физики отдельно от частоты кадров экрана, несколько разных фаз обновления (например, отдельно логику и отдельно анимацию), интерполяцию между кадрами для более плавной картинки, очереди событий с приоритетами и иногда несколько потоков выполнения. Здесь эти усложнения намеренно не нужны — трёх фаз достаточно, чтобы понять принцип, а к более сложным схемам можно вернуться позже, когда проект их действительно потребует.

Забыли `pygame.init()`

`pygame.init()` не единственный способ инициализировать `Pygame` — можно инициализировать нужные модули по отдельности, — но в проектах этой книги мы всегда начинаем настройку с него: это удобный способ разом инициализировать все модули, которым нужна инициализация. Если его пропустить, часть модулей (в первую очередь звук) не будет готова к работе, а сообщения об ошибке иногда указывают не на место, где не хватает `pygame.init()`, а на совершенно другую строку — где `Pygame` впервые понадобился неинициализированный модуль.

Очередь событий не обработана — окно "зависает"

Операционная система считает окно зависшим, если оно долго не забирает свои события из очереди — `pygame.event.get()` должен вызываться каждый кадр, даже если внутри цикла `for event in ...` вы ничего не делаете с большинством событий. Пропуск этого вызова на несколько секунд — частая причина пометки "Не отвечает" в заголовке окна.

Практика: три фазы игрового цикла

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-14/index.html>)

Кадры в секунду и время кадра

FPS и время кадра — две стороны одной величины, а `clock.tick()` лишь придерживает цикл, но не ускоряет его.

FPS (frames per second, кадров в секунду) — сколько раз в секунду игра успевает пройти полный цикл Input → Update → Render и показать новый кадр игроку. При прочих равных, если дисплей и система вывода способны показать дополнительные кадры, более высокая частота кадров может сделать движение визуально плавнее — но у частоты кадров есть естественный физический предел, завязанный на экран и время, которое требуется коду каждого кадра.

Время кадра

Время кадра (frame time) — это то, сколько миллисекунд занял один проход цикла. FPS и время кадра — две стороны одной величины:

vremya_kadra.py

```
# frame_time — в секундах, fps — кадров в секунду
frame_time = 1 / 60          # 60 кадров в секунду ->
секунд на кадр
fps = 1 / frame_time         # и обратно: секунд на
кадр -> кадров в секунду

print(frame_time)           # 0.016666... секунды, то есть
                             # примерно 16.7 мс на кадр
print(fps)                  # 60.0
```

`clock.tick(60)` из раздела 20.2 не ускоряет игру — наоборот, он искусственно *придерживает* цикл, если тот попытается выполниться быстрее 60 раз в секунду, и ничего не может сделать, если код одного кадра сам по себе занимает больше времени, чем отведённый бюджет. Реальный, измеренный FPS можно узнать через `clock.get_fps()`, но не сразу: он усредняет последние десять вызовов `clock.tick()` и до десятого кадра честно возвращает `0.0`. Ноль в первых кадрах — это не сломанный счётчик, а ещё не набранная статистика.

Нет ограничения частоты — цикл ест 100% процессора

Если ничего не сдерживает частоту цикла, он превращается в простой `busy loop` и выполняется настолько быстро, насколько способен процессор — часто тысячи кадров в секунду, без какой-либо пользы для игрока, зато с полной загрузкой ядра процессора и повышенным расходом батареи на ноутбуке или телефоне (раздел 20.13). В игровом цикле этой книги решением служит `clock.tick(FPS)` в конце каждого кадра — другие движки и игры могут управлять темпом иначе: через `vsync`, фиксированный `timestep`-планировщик или собственную систему тайминга движка.

Практика: FPS и бюджет кадра

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/20-15/index.html>)

Delta time: движение независимо от FPS

Скорость в «пикселях за кадр» зависит от чужого железа. Скорость в «пикселях за секунду», умноженная на delta time, — нет.

В мини-проекте раздела 20.5 мяч двигался так: `x += dx` на каждом кадре. Это работает — но только пока FPS постоянен. У этого подхода есть скрытая проблема, которая рано или поздно проявляется на реальных устройствах.

Проблема: движение, привязанное к кадрам

Если `dx` — это «пикселей за кадр», то итоговая скорость мяча в пикселях за секунду напрямую зависит от того, сколько кадров реально успевает пройти за секунду, а не от того, что вы попросили у `clock.tick()`.

`clock.tick(60)` из раздела 20.5 задаёт только *верхний предел* — 60 кадров в секунду, не больше — но ничего не может сделать, если устройство не справляется и с этим. На игровом ноутбуке разработчика игра стабильно держит все 60 FPS; на слабом или перегретом (раздел 20.13) устройстве игрока код одного кадра может не укладываться в бюджет, и реальная частота проседает, скажем, до 30 FPS. С кодом «пикселей за кадр» это означает, что мяч у такого игрока движется вдвое медленнее, чем у разработчика, при абсолютно одинаковом коде — игра, которая ощущается правильно на одном устройстве, может оказаться заметно медленнее или быстрее на другом.

Решение: delta time

Delta time (сокращённо `dt`) — время, реально прошедшее с предыдущего кадра, в секундах.

`clock.tick(FPS)` не только ограничивает частоту кадров, но и *возвращает* число миллисекунд, прошедших с предыдущего вызова `clock.tick()`, — то есть полное время кадра, включая ожидание, если оно вообще было:

`delta_time.py`

```
clock = pygame.time.Clock()

while работает:
    dt_ms = clock.tick(FPS)      # сколько миллисекунд
    прошло с прошлого кадра
    dt = dt_ms / 1000           # переводим в секунды

    x += skorost_x * dt         # skorost_x теперь в
    пикселях за СЕКУНДУ, не за кадр
```

Теперь `skorost_x` означает «пикселей в секунду» — величину, не зависящую от FPS вообще. На 30, 60 или 120 кадрах в секунду мяч за одну и ту же реальную секунду пройдёт одно и то же расстояние: на медленном устройстве шаг каждого кадра будет крупнее, на быстром — мельче, но их сумма за секунду останется одинаковой.

Важно, что `tick()` сообщает именно *полное* время кадра, а не то, сколько он прождал. Если устройство не справляется, ждать ему вообще не приходится — и тогда `tick(60)` вернёт не 16, а, скажем, 30 миллисекунд: ровно столько, сколько заняла работа кадра. Именно поэтому delta time и спасает на слабом устройстве: чем медленнее кадр, тем больше `dt` и тем крупнее шаг. (Отдельно время только полезной работы, без ожидания, доступно через `clock.get_rawtime()`.)

Спутали миллисекунды и секунды

`clock.tick(FPS)` возвращает время в **миллисекундах**, а не в секундах. Если забыть разделить на 1000, скорость в коде окажется завышена ровно в тысячу раз — персонаж улетает за пределы экрана за один кадр. Это одна из самых частых ошибок при первом переходе на `delta time`, и симптом у неё всегда один и тот же: движение внезапно становится "телепортацией".

Диагональное движение быстрее осевого

Если персонаж одновременно двигается и по X, и по Y на полную скорость (`x += v * dt` и `y += v * dt` одновременно), то по диагонали он на самом деле перемещается в $\sqrt{2} \approx 1.41$ раза быстрее, чем строго по одной оси — обе составляющие складываются геометрически. Правильное решение — нормализовать вектор направления (привести его длину к 1) перед умножением на скорость, чтобы общая скорость оставалась одинаковой в любом направлении.

Практика: движение, независимое от FPS

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/20-16/index.html>)

Surface: поверхность для изображения и кадра

Экран — это Surface, и можно создавать другие Surface в памяти, а потом наносить их друг на друга через `blit()`.

`screen`, который вы получаете из `pygame.display.set_mode()`, — это объект типа `pygame.Surface`: прямоугольная область пикселей, на

которой можно рисовать. Экран — не единственная Surface в игре: можно создавать дополнительные поверхности в памяти и работать с ними точно так же.

sozdanie_surface.py

```
# Дополнительная поверхность 100x100, отдельная от
экрана
sprajt = pygame.Surface((100, 100))
sprajt.fill((255, 100, 100))

# Наносим (blit) содержимое sprajt поверх экрана в
точке (50, 50)
screen.blit(sprajt, (50, 50))
```

Реальное окно: розовый квадрат 100×100 нанесён поверх тёмно-синего фона в точке (50, 50)

Реальное окно: результат `screen.blit(sprajt, (50, 50))` — вторая Surface нанесена ровно в указанную точку.

`blit()` — основная операция композиции в Pygame: «нарисовать содержимое одной Surface поверх другой в заданной точке». Загруженное изображение (`pygame.image.load()`, раздел 20.19) — это тоже Surface, и на экран оно попадает тем же самым вызовом `blit()`.

Прозрачность

У Surface есть два разных способа задать прозрачность. Первый — **colorkey**: один конкретный цвет объявляется «прозрачным» и не рисуется вовсе (полезно для простой спрайтовой графики без сглаженных краёв). Второй — **per-pixel alpha**: у каждого пикселя своя степень прозрачности, что даёт мягкие полупрозрачные края, но требует изображения в формате с альфа-каналом (обычно PNG) и поверхности, созданной с флагом `pygame.SRCALPHA`.

Реальное окно: полупрозрачный красный слой поверх чёрного фона выглядит тёмно-бордовым, а не чистым красным

Реальное окно: per-pixel alpha в действии — итоговый цвет это смесь фона и полупрозрачного слоя, а не чистый цвет слоя.

Экран не залит — остаются "следы" от предыдущего кадра

Если пропустить `screen.fill(...)` в начале кадра, новая отрисовка просто накладывается на предыдущую — движущийся объект оставляет за собой шлейф из старых позиций вместо чистого кадра. Обратная ошибка — вызвать `fill()`, но забыть `pygame.display.flip()`: тогда нарисованный кадр так и останется невидимым, а окно будет показывать сплошной чёрный (или ранее залитый) цвет.

Реальное окно: цепочка красных кругов остаётся на экране, потому что фон между кадрами ни разу не был перезалит

Реальное окно: намеренно воспроизведённый баг «забыли `screen.fill()`» — каждый новый круг остаётся поверх предыдущих.

Прозрачность "потерялась" после конвертации изображения

`pygame.image.load()` сама по себе сохраняет альфа-канал загруженной картинки. Прозрачность реально теряется, если после загрузки вызвать `.convert()` вместо `.convert_alpha()`: `.convert()` создаёт копию в формате пикселей экрана, но без per-pixel alpha, и края спрайта отрисовываются со сплошным фоновым цветом вместо мягкого перехода. Тот же результат — если создать новую поверхность вручную (`pygame.Surface(size)`) без флага `pygame.SRCALPHA`, когда нужен именно per-pixel alpha.

Практика: Surface, blit и прозрачность

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-17/index.html>)

Rect: позиция, размер и координаты

`pygame.Rect` хранит позицию и размер вместе, а ось Y на экране растёт вниз — в отличие от Turtle и математики на бумаге.

Мы уже упоминали `pygame.Rect` в разделе 20.3 как способ хранить позицию и размер вместе. Пора познакомиться с ним по-настоящему — и заодно разобраться, как в Pygame устроена система координат.

pygame.Rect

rect_osnovy.py

```
igrok_rect = pygame.Rect(100, 200, 40, 40)  # x, y,
                                              ширина, высота

print(igrok_rect.x, igrok_rect.y)           # левый
                                              верхний угол
print(igrok_rect.center)                    # центр —
                                              кортеж (x, y)
print(igrok_rect.right, igrok_rect.bottom)  # правый
                                              и нижний край

igrok_rect.x += 5                           # передвинуть весь
                                              прямоугольник вправо
igrok_rect.center = (300, 300)              # или сразу задать
                                              новый центр
```

`Rect` удобен именно тем, что не заставляет вручную помнить четыре отдельные переменные (`x`, `y`, `ширина`, `высота`) и пересчитывать границы объекта самостоятельно — за это отвечают готовые атрибуты вроде `.right`, `.center`, `.bottom`.

Система координат: мир и экран

В Pygame точка `(0, 0)` — это левый верхний угол экрана, а ось `Y` растёт **вниз**, а не вверх. Это важно держать в голове, особенно после Turtle (главы 6–7, 12 и 19), где ось `Y` растёт вверх, как в математике на бумаге:

	Математика / Turtle	Pygame
Начало координат (0, 0)	Центр экрана	Левый верхний угол
Направление роста Y	Вверх	Вниз

	Математика / Turtle	Pygame
"Прыжок вверх" в коде	у увеличивается	у уменьшается

Отсюда частая на первых порах путаница: код «прыжка» персонажа выглядит как `y -= skorost_pryzhka`, а не `y +=` — потому что «выше на экране» означает «меньшее значение Y».

Мировые координаты против экранных

В играх с прокруткой (скроллящийся уровень, камера, следящая за героем) полезно различать *мировые* координаты объекта (его положение в игровом мире целиком) и *экранные* координаты (где именно он рисуется на видимом сейчас куске экрана) — они совпадают, только пока камера не двигается. В проектах этой книги камера неподвижна, поэтому мировые и экранные координаты — это одно и то же, но в больших играх это разделение — стандартная часть архитектуры.

Практика: Rect и координаты

Проверяется прямо в браузере — установка Pygame не требуется

[Открыть практику](https://www.cartesianschool.org/practice/20-18/index.html) → (<https://www.cartesianschool.org/practice/20-18/index.html>)

Спрайты, изображения и ресурсы игры

Готовые изображения загружаются одной строкой — но путь к ним должен считаться от расположения кода, а не от того, откуда его запустили.

Фигуры, нарисованные `pygame.draw`, отлично подходят для первых шагов, но настоящие игры почти всегда используют готовые изображения — ассеты.

Загружаются они одной строкой:

```
zagruzka_izobrazheniya.py
```

```
igrok_kartinka = pygame.image.load("assets/
igrok.png").convert_alpha()
screen.blit(igrok_kartinka, (x, y))
```

`pygame.image.load()` уже полностью загружает изображение — `.convert_alpha()` после неё не обязательна технически, но для картинок с прозрачностью её настоятельно рекомендуют вызывать сразу после инициализации экрана: она создаёт копию изображения в формате пикселей, оптимальном для повторной отрисовки именно на этом экране, и сохраняет при этом per-pixel alpha (раздел 20.17). Без неё изображение с прозрачностью обычно продолжает работать, но может отрисовываться заметно медленнее при частых вызовах `blit()`.

«Спрайт» здесь — это изображение, а не класс Pygame

В этой главе **спрайт** означает просто картинку игрового объекта: мы загружаем её и рисуем через `blit()`. У Pygame есть и одноимённый класс `pygame.sprite.Sprite` вместе с группами `pygame.sprite.Group`, которые умеют обновлять и проверять столкновения сразу для многих объектов, — но он нам здесь не нужен, и мы к нему вернёмся в главе 21, где объектов станет заметно больше.

Откуда игра знает, где лежат файлы

Путь `"assets/igrok.png"` — **относительный**: он отсчитывается от текущей рабочей директории процесса (CWD), а не от расположения самого файла с кодом. Пока вы запускаете игру из той же папки, где лежит код, разницы не заметно — но стоит запустить её из другого места (например, через ярлык или через `python папка/игра.py` из соседней директории), и загрузка падает с

`FileNotFoundError`. Надёжное решение — модуль `pathlib` (глава 15): строить путь от расположения самого файла с кодом, а не от того, откуда его запустили.

```
nadezhnyj_put.py
```

```
from pathlib import Path
```

```
BASE_DIR = Path(__file__).resolve().parent
igrok_kartinka = pygame.image.load(BASE_DIR /
"assets" / "igrok.png").convert_alpha()
```

Путь к ассету зависит от текущей директории запуска

"У меня всё работает" и "на другом компьютере `FileNotFoundError`" — классический симптом относительного пути, построенного без `Path(__file__)`. Это особенно легко упустить при упаковке в исполняемый файл (раздел 20.29), где рабочая директория процесса может вообще не совпадать с папкой, откуда запущен исполняемый файл.

Организация папки проекта

Устоявшееся соглашение — держать код и ассеты отдельно, группируя ассеты по типу:

```
struktura_proekta.txt
```

```
moya_igra/
  igra.py
  assets/
    images/
      igrok.png
      vrag.png
    sounds/
      prygok.wav
```

Практика: загрузка спрайтов и надёжные пути

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-19/index.html>)

Обработка ввода: события и удержание клавиш

Разовое действие — событие. Непрерывное действие — проверка состояния. Перепутать их — частый источник багов управления.

В разделе 20.4 вы уже видели оба способа проверки клавиатуры. Теперь — точнее о том, для какого именно действия годится каждый из них, и что происходит, если их перепутать.

События или состояние?

Действие
происходит
один раз за
нажатие
(прыжок,
пауза,
выстрел)

→ `pygame.event.get() + KEYDOWN`

Действие
длится,
пока
клавиша
зажата → `pygame.key.get_pressed()`
(бег
влево/
вправо)

KEYDOWN для непрерывного движения — персонаж дёргается

Событие `KEYDOWN` срабатывает один раз в момент нажатия, а не пока клавиша удерживается. Если завязать на него движение персонажа, он сдвинется ровно на один шаг за нажатие и застынет, даже если игрок продолжает держать клавишу — для непрерывного движения нужен `pygame.key.get_pressed()`, проверяемый каждый кадр.

get_pressed() для разового действия — оружие стреляет очередями

Обратная ошибка: если проверять `klavishi[pygame.K_SPACE]` каждый кадр и стрелять при каждом истинном значении, при 60 FPS одно удержание пробела за секунду произведёт около 60 выстрелов — клавиша ведь остаётся нажатой всё это время. Для разового действия по нажатию нужно именно событие `KEYDOWN`, а не проверка текущего состояния.

Мышь устроена так же двойственно

У мыши тот же выбор: событие `pygame.MOUSEBUTTONDOWN` — для разового клика, а `pygame.mouse.get_pressed()` — для проверки, зажата ли кнопка прямо сейчас (полезно, например, для рисования линии, пока кнопка мыши удерживается). Текущие координаты курсора в любой момент доступны через `pygame.mouse.get_pos()`.

Мост к геймпаду и тачскрину

Абстракция игрового действия из раздела 20.11 ("прыжок" вместо конкретной клавиши) особенно окупается именно здесь: под ней могут одинаково скрываться и `KEYDOWN` с пробелом, и кнопка геймпада (`pygame.JOYBUTTONDOWN`), и тап по виртуальной кнопке на тачскрине — код самой игры при этом не меняется ни на строку.

Практика: события и состояние клавиатуры

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-20/index.html>)

Столкновения и хитбоксы

`colliderect()` под капотом — четыре сравнения чисел, а хитбокс не обязан совпадать с видимой картинкой пиксель в пиксель.

Столкновение — момент, когда два игровых объекта пересекаются в пространстве, и игра должна на это отреагировать: мяч отскакивает от стены, персонаж подбирает монету, снаряд поражает врага. У

`pygame.Rect` для этого есть готовый метод:


```
colliderect.py
```

```
if igrok_rect.colliderect(vrag_rect):
    print("Столкновение!")
```

Как это устроено на самом деле

`colliderect()` проверяет пересечение двух прямоугольников, выровненных по осям (AABB, axis-aligned bounding box) — метод, который под капотом сводится к четырём простым сравнениям чисел:

```
aabb_vruchnuyu.py
```

```
def pryamougolniki_peresekayutsya(a, b):
    ax1, ay1, aw, ah = a
    bx1, by1, bw, bh = b
    ax2, ay2 = ax1 + aw, ay1 + ah
    bx2, by2 = bx1 + bw, by1 + bh
    return ax1 < bx2 and ax2 > bx1 and ay1 < by2 and
ay2 > by1
```

Два прямоугольника **не** пересекаются, если один полностью левее, правее, выше или ниже другого — проверка выше как раз убеждается, что ни одно из этих четырёх условий не выполняется.

Реальное окно: голубой и жёлтый квадраты частично перекрываются — `colliderect()` возвращает `True`

Реальное окно: перекрывающиеся прямоугольники — второй закрашен жёлтым, потому что `colliderect()` вернула `True`.

Реальное окно: голубой и красный квадраты далеко друг от друга — `colliderect()` возвращает `False`

Реальное окно: прямоугольники не пересекаются — второй остался красным.

Хитбокс — не всегда то же самое, что рисунок

«Хитбокс» — прямоугольник (или другая фигура), который используется именно для проверки столкновений, и он не обязан по размеру совпадать с видимым рисунком персонажа один в один. У круглого мяча с прозрачными краями изображение обычно квадратное — если считать столкновение по полному квадрату изображения, игрок будет видеть несправедливые «попадания мимо» там, где на глаз ничего не коснулось.

Хитбокс не совпадает с картинкой — столкновения ощущаются "нечестными"

Частая причина: `Rect` берут напрямую из размера всего загруженного изображения, включая прозрачные поля вокруг самого персонажа. Решение — уменьшить хитбокс относительно видимой картинки через `rect.inflate(-20, -20) : inflate()` меняет *общий* размер прямоугольника, сохраняя центр, поэтому `-20` по ширине и высоте убирает ровно по 10 пикселей с каждой стороны, а не 20. Конкретный отступ подбирают на глаз под каждый спрайт.

Реальное окно: красный мяч с двумя рамками поверх — большой красной по границе изображения и меньшей зелёной внутри неё

Реальное окно: красная рамка — полный размер изображения, зелёная — уменьшенный хитбокс, ближе к реальным видимым краям мяча.

Практика: проверка пересечения прямоугольников

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/20-21/index.html>)

Основы игровой физики: скорость, гравитация и отскок

Гравитация — это накапливающаяся скорость, а не мгновенный прыжок координаты; отскок и трение добавляются той же логикой через delta time.

Мяч из раздела 20.5 меняет направление на противоположное при ударе о стену — это уже простейшая физика. Добавим к ней ещё две идеи, которые чаще всего нужны в аркадных играх: гравитацию и трение.

Единицы измерения

В формулах этого раздела три разные величины, и их легко перепутать местами. Позиция (x , y) измеряется в пикселях. Скорость — в пикселях в секунду (px/s): именно в этих единицах, как вы уже знаете из раздела 20.16, задают `skorost_x` / `skorost_y` при движении через delta time. Ускорение — в пикселях в секунду за секунду, то есть (px/s)/s, что обычно и пишут как px/s^2 : это то, насколько сама скорость меняется каждую секунду. Гравитация ниже — как раз пример ускорения.

Гравитация как накопление скорости

Гравитация — это не «прибавить к Y большое число один раз», а постоянное, маленькое ускорение, которое на каждом кадре немного увеличивает вертикальную скорость. Сама позиция меняется уже за счёт этой скорости — с использованием delta time из раздела 20.16:

gravitaciya.py

```

GRAVITACIYA = 900      # пикселей в секунду за секунду
                        (ускорение)

skorost_y += GRAVITACIYA * dt    # скорость растёт с
каждым кадром
y += skorost_y * dt             # позиция меняется за
счёт накопленной скорости

```

Именно поэтому в реальной жизни (и в играх с честной гравитацией) падение начинается медленно и разгоняется — это прямое следствие постоянного накопления скорости, а не настройки «скорость падения» одним числом.

Гри реальных окна рядом: оранжевый шарик падает всё быстрее сверху вниз

Реальное окно: три момента одного и того же падения — расстояние между положениями растёт, потому что скорость накапливается.

Отскок с потерей энергии

Реальный мяч не отскакивает вечно с одной и той же высоты — часть энергии теряется при ударе. В коде это одна дополнительная строка после разворота скорости:

otskok_s_zatuhaniem.py

```

KOEFF_OTSKOKA = 0.8    # мяч сохраняет 80% скорости
после отскока

if y + radius > VYSOTA:
    y = VYSOTA - radius
    skorost_y = -skorost_y * KOEFF_OTSKOKA

```

Без множителя затухания `KOEFF_OTSKOKA` мяч отскакивал бы на одну и ту же высоту бесконечно — с ним же высота отскоков естественно уменьшается с каждым ударом, пока мяч почти не остановится. Такой множитель затухания скорости от удара к удару — устоявшийся в играх приём и называется **restitution** (коэффициент восстановления, буквально «коэффициент отскока»): значение `1.0` соответствует идеализированному отскоку без потерь на исходной скорости, а любое значение между 0 и 1 — затухающему отскоку с постепенной потерей энергии.

Трение

Трение работает похожим образом для горизонтального движения — скорость постепенно уменьшается по модулю (но не меняет знак), пока не станет равна нулю. Важно, чтобы трение, как и гравитация, было задано ускорением в px/s^2 и применялось через `delta time` — иначе, если умножать скорость на постоянный коэффициент каждый кадр, торможение будет зависеть от FPS точно так же, как и нескорректированное движение из раздела 20.16: на 120 FPS такое умножение сработает вдвое чаще, чем на 60 FPS, и объект остановится заметно быстрее:

```
trenie.py
```

```
TRENIE = 300 # пикселей в секунду за секунду –
             ускорение торможения

if skorost_x > 0:
    skorost_x = max(0.0, skorost_x - TRENIE * dt)
elif skorost_x < 0:
    skorost_x = min(0.0, skorost_x + TRENIE * dt)
```

`max()` / `min()` здесь не дают торможению «проскочить» через ноль и разогнать объект в обратную сторону — скорость останавливается ровно на нуле, а не продолжает уменьшаться дальше.

Это не симулятор физики целиком

Настоящие физические движки (Box2D и похожие) решают гораздо более общую задачу — вращение, массу, трение между произвольными формами, столкновения многих тел сразу. Формулы этого раздела — упрощённая, но абсолютно рабочая модель именно для аркадных игр вроде тех, что вы пишете в этой главе, и понимание их — хорошая основа для перехода к настоящему физическому движку позже, если он понадобится.

Практика: гравитация и отскок с затуханием

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/20-22/index.html>)

Спрайтовая анимация: кадры и спрайт-лист

Анимация — это быстрая смена кадров одного изображения по собственному таймеру, не совпадающему с частотой кадров игры.

До сих пор персонажи были статичными фигурами или одной неподвижной картинкой. Анимация — это быстрая смена нескольких кадров рисунка, создающая иллюзию движения, — тот же принцип, что и в мультфильме или флипбуке.

Спрайт-лист

Спрайт-лист (sprite sheet) — одно изображение, в котором кадры анимации выложены в ряд. Чтобы показать нужный кадр, из общего изображения вырезается прямоугольная область через `subsurface()` :

```
kadr_iz_lista.py
```

```
SHIRINA_KADRA, VYSOTA_KADRA = 32, 32
sprajt_list = pygame.image.load("assets/
hodba.png").convert_alpha()

def poluchit_kadr(nomer):
    x = nomer * SHIRINA_KADRA
    return sprajt_list.subsurface((x, 0,
SHIRINA_KADRA, VYSOTA_KADRA))
```

Таймер анимации

Кадры анимации не должны меняться каждый кадр игрового цикла — это слишком быстро (при 60 FPS это 60 смен рисунка в секунду). Вместо этого копится время в накопителе (аккумуляторе), и кадр анимации переключается по своему собственному, более медленному таймеру:

```
tajmer_animacii.py
```

```
INTERVAL_KADRA = 0.12 # секунд на один кадр анимации
tekushij_kadr = 0
vremya_animacii = 0.0

vremya_animacii += dt
while vremya_animacii >= INTERVAL_KADRA:
    vremya_animacii -= INTERVAL_KADRA
    tekushij_kadr = (tekushij_kadr + 1) %
KOLICHESTVO_KADROV
```

% KOLICHESTVO_KADROV (остаток от деления, глава 4) заставляет счётчик кадра заиклиться: после последнего кадра анимация начинается заново с нулевого. Здесь важен именно `while`, а не `if`: если один игровой кадр из-за просадки FPS длится дольше, чем сразу несколько интервалов анимации, `while` переключит кадр анимации нужное число раз подряд и сохранит остаток времени в

`vremya_animacii` для следующего прохода. `if` вместо этого продвинул бы анимацию только на один кадр и молча потерял лишнее накопленное время.

Четыре реальных окна рядом: у голубой фигурки нога от кадра к кадру смещается вправо, отмеряя шаг

Реальное окно: четыре кадра одной и той же простой анимации ходьбы, нарисованные вручную для этого примера — тот же принцип, что и у настоящего спрайт-листа.

Обновление после отрисовки — анимация отстаёт на кадр

Если код, который переключает `tekushij_kadr`, стоит **после** отрисовки текущего кадра, игрок на экране всегда видит предыдущее состояние анимации, а не только что вычисленное. Порядок Update → Render из раздела 20.14 существует именно для того, чтобы избежать подобной рассинхронизации на один кадр.

Практика: спрайт-лист и таймер анимации

Rugame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-23/index.html>)

Звуковые эффекты и музыка

Короткие эффекты и фоновая музыка в Rugame — два разных инструмента с разной внутренней механикой воспроизведения.

За звук в Rugame отвечает отдельный подмодуль `rugame.mixer`. У него, как и у остального Rugame, есть собственная обязательная инициализация — обычно она

уже включена в общий `pygame.init()`, но при проблемах со звуком стоит убедиться в этом явно через `pygame.mixer.init()`.

Короткие звуки и музыка — не одно и то же

У Pygame два разных инструмента для звука, и они не взаимозаменяемы:

	<code>pygame.mixer.Sound</code>	<code>pygame.mixer.music</code>
Для чего	Короткие эффекты: прыжок, выстрел, монета	Фоновая музыка
Загрузка	Полностью в память сразу	Проигрывается потоково, не грузится целиком
Одновременных звуков	Несколько сразу, на разных каналах	Один поток музыки за раз
Типичный вызов	<code>sound.play()</code>	<code>pygame.mixer.music.play(loops=-1)</code>

`zvuk_i_muzyka.py`

```
zvuk_pryzhka = pygame.mixer.Sound("assets/sounds/
pryok.wav")
```

```
pygame.mixer.music.load("assets/sounds/fon.mp3")
pygame.mixer.music.play(loops=-1) # -1 значит
"зациклить бесконечно"
```

```
# внутри игрового цикла, в момент прыжка:
zvuk_pryzhka.play()
```

Звук загружается каждый кадр — заметные подтормаживания

`pygame.mixer.Sound(...)` читает файл с диска — это заметно медленнее, чем воспроизвести уже загруженный звук. Все звуки нужно загружать один раз, до начала игрового цикла (как и изображения из раздела 20.19), и хранить в переменных или словаре, а внутри цикла — только вызывать `.play()` у уже загруженного объекта. Загрузка ассета внутри цикла — одна из самых частых причин необъяснимых просадок кадров в первых проектах на Pygame.

Практика: звуковые эффекты и фоновая музыка

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-24/index.html>)

Состояния игры

Меню, геймплей, пауза и game over — это разные состояния игры, и код должен явно знать, в каком из них находится прямо сейчас.

Уже сейчас у ваших проектов есть как минимум одно неявное состояние — «игра идёт». На практике почти любая игра проходит через несколько чётко различимых состояний, и в каждом из них ведёт себя по-своему.

Типичные состояния игры

МЕНЮ

Показывает заголовок и кнопку "Играть"

Не двигает никаких игровых объектов

Ждёт нажатия клавиши или клика

ИГРА ИДЁТ

Обновляет позиции, проверяет столкновения
Реагирует на управление игрока
Основное состояние геймплея

ПАУЗА

Не обновляет позиции объектов
Продолжает отрисовывать последний кадр
Ждёт команды на возобновление

GAME OVER

Показывает финальный счёт
Не принимает игровое управление
Предлагает начать заново

Состояние как перечисление

Знакомый по главе 19 приём — хранить текущее состояние как значение `enum.Enum` (глава 18), а не как несколько независимых булевых флагов вроде `na_pauze` и `igra_okonchena` по отдельности. У перечисления

невозможно оказаться сразу в двух состояниях одновременно — у отдельных булевых флагов такая ошибка возможна совершенно случайно:

```
sostoyaniya_enum.py
```

```
from enum import Enum, auto

class SostoyanieIgry(Enum):
    MENU = auto()
    IGRA = auto()
    PAUZA = auto()
    GAME_OVER = auto()

tekushee_sostoyanie = SostoyanieIgry.MENU
```

Переходы между состояниями

Состояния этой мини-игры образуют не прямую линию, а граф: из `PAUZA` игра возвращается обратно в `IGRA`, а не идёт дальше вперёд, и то же самое с возвратом из `GAME_OVER` в `MENU`. Явный список разрешённых переходов надёжнее рисовать таблицей «откуда → куда», где у каждой строки есть свой, ничем не перепутанный триггер:

Из состояния	В состояние	Когда
MENU	IGRA	нажали «Играть»
IGRA	PAUZA	нажали паузу
IGRA	GAME_OVER	столкновение / поражение
PAUZA	IGRA	нажали паузу ещё раз (снять с паузы)
GAME_OVER	MENU	нажали «Заново»

Важно, что не каждый переход разрешён: из `MENU` нельзя напрямую попасть в `GAME_OVER`, минуя саму игру, и из `PAUZA` нельзя попасть в `GAME_OVER` напрямую — сначала игра обязана вернуться в `IGRA`. Код, который обрабатывает ввод и обновление, должен явно проверять текущее состояние, прежде чем выполнять действия, разрешённые только в нём — это ровно то же самое, зачем в главе 19 понадобился явный `GameStatus` для змейки.

Четыре реальных окна рядом: экран меню с надписью и подсказкой, игровой процесс с мячом и счётю, мяч с жёлтым оверлеем `ПАУЗА` по центру и экран `GAME OVER` с итоговым счётю

Реальное окно: четыре состояния мини-игры — меню, игра, пауза, `game over`. Это четыре отдельных снимка, а не один непрерывный забег, поэтому счёт на них не продолжает друг друга.

Практика: состояния игры и переходы

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/20-25/index.html>)

Структура небольшой игры: класс `Game`

Три фазы игрового цикла становятся тремя методами одного класса — тем же каркасом, что и у финального проекта этой главы и у следующей.

До сих пор весь код жил в глобальных переменных и одном плоском цикле — нормально для учебного мини-проекта, но неудобно, как только игра обрастает состояниями (раздел 20.25), несколькими объектами и хоть какой-то паузой. Соберём всё, что вы узнали в этой

главе, в один класс — тот самый архитектурный каркас, на котором построен финальный проект раздела 20.33 и космический шутер главы 21.

Каждый метод отвечает ровно за одну фазу игрового цикла из раздела 20.14 — и не более того.

Это набросок каркаса, а не полный файл: он опускает импорт `pygame`, определение `SHIRINA / VYSOTA`, класс `SostoyanieIgrы` из раздела 20.25 и тело метода `toggle_pause()` — всё это в реальном проекте, конечно, должно быть определено. Полный, действительно запускаемый пример такого класса — финальный проект раздела 20.33.

fragment_class_game.py

```
class Game:
    def __init__(self):
        pygame.init()
        self.screen =
pygame.display.set_mode((SHIRINA, VYSOTA))
        self.clock = pygame.time.Clock()
        self.state = SostoyanieIgrы.MENU
        self.running = True

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                self.running = False
            elif event.type == pygame.KEYDOWN and
event.key == pygame.K_p:
                self.toggle_pause()

    def update(self, dt):
        if self.state is not SostoyanieIgrы.IGRA:
            return # на паузе или в меню ничего не
```

двигаем

здесь — обновление позиций, столкновений, счёта

```
def render(self):
    self.screen.fill((20, 20, 40))
    # здесь — отрисовка всех объектов текущего
    # состояния
    pygame.display.flip()

def run(self):
    while self.running:
        dt = self.clock.tick(60) / 1000
        self.handle_events()
        self.update(dt)
        self.render()
    pygame.quit()
```

`run()` — единственное место, где виден весь цикл целиком, и он дословно повторяет три фазы из раздела 20.14: `handle_events()` это Input, `update(dt)` это Update, `render()` это Render. Именно так организован финальный `bouncing_ball.py` (раздел 20.33) — и в этом же стиле в главе 21 написан космический шутер.

Пауза не останавливает управление

Если проверку `self.state is not SostoyanieIgrы.IGRA` поставить только в `render()`, а не в `update()`, экран во время паузы визуально замрёт правильно — но объекты продолжают двигаться в памяти, и после снятия паузы игра "телепортируется" туда, куда они успели дойти незаметно для игрока. Проверка состояния должна стоять именно в `update()`, а не только в отрисовке.

Практика: собираем класс Game

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/20-26/index.html>)

Производительность: FPS и бюджет кадра

У каждого кадра есть жёсткий бюджет времени в миллисекундах — и почти всегда полезнее его измерить, чем гадать, что именно медленное.

Раздел 20.15 уже показал, что 60 FPS означает бюджет примерно 16.7 миллисекунды на кадр целиком — включая обработку ввода, обновление и отрисовку. Если код одного кадра занимает больше этого бюджета, кадры теряются, что бы ни было выставлено в `clock.tick()`.

```
byudzheth_kadra.py

# Бюджет кадра для разных целевых FPS
for fps in (30, 60, 120):
    byudzheth_ms = 1000 / fps
    print(f"{{{fps}}} FPS -> {{{byudzheth_ms:.2f}}} мс на кадр")
```

Куда обычно уходит бюджет кадра

Частая причина просадки	Почему это медленно	Что делать вместо этого
Загрузка файла внутри цикла	Чтение с диска в тысячи раз медленнее операций в памяти	Загружать все ассеты один раз до цикла (разделы 20.19, 20.24)
Создание новой Surface каждый кадр	Выделение памяти под изображение — не бесплатная операция	Создавать поверхности один раз и переиспользовать

Частая причина просадки	Почему это медленно	Что делать вместо этого
Отрисовка объектов за пределами экрана	Каждый вызов <code>blit()</code> выполняет работу по композиции кадра независимо от того, видно ли её результат	Пропускать отрисовку объектов, которых не видно
Проверка столкновений "каждый с каждым"	Число проверок растёт квадратично с числом объектов	Сокращать число проверок (например, делить экран на зоны)

Как измерить, а не гадать

`clock.get_fps()` из раздела 20.15 — самый простой способ увидеть реальную производительность прямо во время игры. Для более точного разбора, какая именно часть кода медленная, в Python есть встроенный профилировщик `cProfile` — он показывает, сколько времени было потрачено в каждой функции, а не только итоговый FPS.

Сначала измерить, потом оптимизировать

Первая реакция на просадку FPS — переписать "на глаз" то, что кажется медленным. Гораздо надёжнее сначала измерить (через `get_fps()` или `cProfile`), где именно уходит время: интуиция о "медленном коде" в программировании вообще и в играх в частности ошибается на удивление часто.

Практика: считаем бюджет кадра

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/20-27/index.html>)

Как отлаживать игры

Справочник по симптому: большинство багов Pygame-проектов не бросают исключение — игра просто ведёт себя не так, как ожидалось.

По ходу главы вам уже встретилось больше десятка типичных багов Pygame-проектов — по одному-два на каждую новую тему. Здесь они собраны в одну таблицу — как справочник, к которому удобно вернуться, когда что-то в собственном проекте ведёт себя необъяснимо.

Симптом	Обычная причина	Где подробнее
Окно помечено "Не отвечает"	<code>pygame.event.get()</code> не вызывается каждый кадр	20.14
Модули Pygame работают со сбоями с самого начала	Забыт <code>pygame.init()</code>	20.14
Процессор загружен на 100% без причины	Забыт <code>clock.tick(FPS)</code>	20.15
Персонаж телепортируется на весь экран	<code>dt</code> не переведён из миллисекунд в секунды	20.16
По диагонали персонаж быстрее, чем по прямой	Вектор скорости не нормализован	20.16
На экране остаются "следы" от предыдущих кадров	Забыт <code>screen.fill()</code> в начале кадра	20.17

Симптом	Обычная причина	Где подробнее
Окно показывает только чёрный экран	Забыт <code>pygame.display.flip()</code>	20.17
Прозрачность картинки пропала	Вызван <code>.convert()</code> вместо <code>.convert_alpha()</code> , или Surface создана без <code>pygame.SRCALPHA</code>	20.17
FileNotFoundError на чужом компьютере	Путь к ассету не построен через <code>pathlib</code>	20.19
Персонаж дёргается вместо плавного движения	<code>KEYDOWN</code> использован для непрерывного действия	20.20
Оружие стреляет очередями от одного нажатия	<code>get_pressed()</code> использован для разового действия	20.20
Столкновения ощущаются нечестными	Хитбокс не уменьшен относительно рисунка	20.21
Анимация визуально отстаёт на кадр	Обновление кадра анимации стоит после отрисовки	20.23
Заметные подтормаживания при каждом звуке	<code>Sound()</code> создаётся заново каждый кадр вместо один раз	20.24

Симптом	Обычная причина	Где подробнее
Игра "телепортируется" после снятия паузы	Пауза проверяется только в <code>render()</code> , не в <code>update()</code>	20.26

Общий метод отладки игр

Почти все баги из этой таблицы объединяет одно: они не бросают исключение и не показывают традиционное сообщение об ошибке — игра просто ведёт себя не так, как ожидалось. Для такого класса багов особенно полезны:

`otladka_pechatyu.py`

```
# Временный print() — самый быстрый способ увидеть
# реальные значения
print(f"dt={{dt:.4f}} x={{x:.1f}}
      skorost_x={{skorost_x:.1f}}")

# Временная отрисовка хитбокса поверх спрайта —
# увидеть его границы глазами
pygame.draw.rect(screen, (255, 0, 0), hitbox, width=2)
```

Отладочная отрисовка — не забыть убрать

Временный `pygame.draw.rect(..., width=2)` для хитбокса — один из самых полезных приёмов отладки столкновений, но его легко забыть в готовом проекте. Полезная привычка — держать такие вставки под одним общим флагом вроде `OTLADKA = False`, чтобы включать и выключать их разом, а не искать по всему файлу.

Практика: диагностика по симптому

Проверяется прямо в браузере — установка Pygame не требуется

[Открыть практику](https://www.cartesianschool.org/practice/20-28/index.html) → (<https://www.cartesianschool.org/practice/20-28/index.html>)

Собираем настольную версию игры

PyInstaller собирает игру в самостоятельный исполняемый файл — но только под ту операционную систему, на которой сам запущен.

Игрок, для которого вы делаете игру, почти никогда не станет устанавливать Python и `pip install pygame-ce` только чтобы её запустить. Игру нужно упаковать в исполняемый файл, который работает сам по себе.

PyInstaller

Один из самых распространённых инструментов для этого на десктопе — `PyInstaller`: он анализирует вашу программу, находит все нужные модули (включая сам `Pygame`) и собирает единый исполняемый файл вместе с интерпретатором Python внутри — играть в такую игру можно без установленного Python на компьютере игрока.

```
pyinstaller.txt
```

```
pip install pyinstaller
pyinstaller --onefile --add-data "assets:assets"
игра.py
```

`--onefile` собирает всё в один файл вместо папки с зависимостями. `--add-data` явно указывает включить папку `assets` в сборку — без этого флага `PyInstaller` упакует только код, а изображения и звуки останутся снаружи готового файла.

Двоеточие в --add-data

В актуальных версиях PyInstaller источник и назначение разделяются двоеточием (`"assets:assets"`) на всех операционных системах, включая Windows. В версиях до 6.0 разделитель зависел от системы, и на Windows нужно было писать точку с запятой: `"assets;assets"` . Если вы работаете со старой закреплённой версией и сборка не находит ассеты — проверьте в первую очередь именно этот символ.

Кроссплатформенность — с оговоркой**PyInstaller не кросс-компилирует**

PyInstaller собирает исполняемый файл под ту операционную систему, на которой сам запущен — сборка под Windows делается на Windows, под macOS — на macOS, под Linux — на Linux. Он не умеет собрать `.exe` для Windows, работая при этом на Linux, и наоборот. Для по-настоящему кроссплатформенного релиза нужен доступ к каждой из целевых операционных систем — либо напрямую, либо через виртуальную машину или облачный сервис сборки.

Скрытые импорты

Pygame устроен так, что часть его модулей подключается не совсем обычным для Python образом — PyInstaller иногда не находит их автоматическим анализом и пропускает из сборки. Если готовый исполняемый файл падает с `ModuleNotFoundError` , хотя из исходного кода игра запускается нормально, обычно помогает явно указать такой модуль через `--hidden-import` .

Проверяйте собранный файл, а не только исходный код

Работающая игра при запуске через `python igra.py` и работающий собранный исполняемый файл — не одно и то же гарантированно: разница в путях к ассетам (раздел 20.19) и в скрытых импортах проявляется именно на этапе сборки. Всегда стоит один раз запустить готовый файл на чистом компьютере (или хотя бы из другой папки) перед тем, как считать релиз готовым.

Запускаем Pygame-игру в браузере

`pygbag` — активно поддерживаемый инструмент сообщества для сборки Pygame-игры в WebAssembly, не официальная часть `pygame-ce`.

Раздел 20.10 уже упоминал: веб-игра на Pygame попадает в браузер через технологию WebAssembly. Инструмент, который делает эту сборку на практике, называется **`pygbag`** — активно поддерживаемый проект сообщества, а не официальная часть самого `pygame-ce`.

`pygbag.txt`

```
pip install pygbag
pygbag moya_igra/
```

Команда запускает локальный веб-сервер и открывает игру прямо в браузере — удобно для проверки перед публикацией на постоянный хостинг. Обратите внимание: аргумент — это **папка** проекта, а не отдельный файл.

Файл с игровым циклом обязан называться main.py

pygame ищет внутри папки проекта строго файл `main.py` — это имя зашито в сборку, и переименовать его нельзя. Коварство в том, что при другом имени (`igra.py`, `game.py`) сборка завершится успешно и без единого предупреждения, но в браузере игра просто не запустится: загрузчик будет искать `main.py`, которого в собранном архиве нет. Если после сборки страница открывается, но остаётся пустой, — первым делом проверьте имя файла.

pygame — инструмент сообщества, не часть pygame-се

Как и с мобильными сборками (раздел 20.13, раздел 20.31), важно понимать границу ответственности: pygame разрабатывается и поддерживается отдельно от pygame-се. Он активно развивается и на практике хорошо справляется с типичными 2D-играми на Pygame, но при обновлении версии Pygame стоит перепроверять совместимость с текущей версией pygame, а не считать её гарантированной навсегда.

Что в вебе устроено иначе

Браузерная версия игры работает не совсем как обычный запуск на десктопе — код выполняется внутри песочницы браузера, из которой:

	Обычный запуск на десктопе	Сборка через pygame
Доступ к файловой системе	Полный	Ограничен песочницей браузера
Главный цикл	<code>while True</code> в вашем коде	Асинхронный, с <code>await asyncio.sleep(0)</code> на каждом кадре

	Обычный запуск на десктопе	Сборка через <code>rugbag</code>
Установка	Требуется Python и <code>pip install</code>	Не требуется — открыл ссылку и играешь
Производительность	Родная скорость процессора	Может отличаться от настольной — стоит измерять на целевом устройстве

Главная практическая разница в коде — цикл игры должен стать асинхронным (`async def main()` с `await asyncio.sleep(0)` в конце каждого кадра), чтобы не блокировать поток браузера: `rugbag` ожидает именно такую структуру программы. Важно, что `await` здесь *добавляется рядом с `clock.tick()`*, а не заменяет его — в примерах самого `rugbag` ограничение частоты кадров остаётся на месте, а строка с `await` идёт сразу после него.

Насколько браузерная сборка медленнее настольной, заранее сказать нельзя: это зависит от браузера, устройства игрока и самой игры. Производительность стоит измерять на целевом устройстве, а не закладывать какой-то определённый процент потерь заранее.

Что реально возможно на Android, iOS и консолях

Android доступен только через экспериментальную интеграцию сообщества, не принятую в основной репозиторий `python-for-android`. iOS — через инструмент, который его собственный автор называет экспериментальным.

Раздел 20.13 уже дал ответ в общих чертах: `Rugame` не публикуется на мобильные платформы одной командой. Здесь — подробнее о том, как выглядят оба реальных пути на практике, а не только то, что они существуют.

Android — python-for-android, Buildozer и экспериментальные рецепты

`python-for-android` — это инфраструктура для упаковки Python-приложений под Android; поверх неё обычно используют `Buildozer` — более высокоуровневый инструмент, который берёт настройки из простого конфигурационного файла и запускает сборку `.apk` за вас, подключая нужные рецепты. На момент проверки в основном репозитории `python-for-android` нет принятого официального рецепта `pygame-ce` — есть только открытые `community pull request`'ы, добавляющие такой рецепт, и пользовательские рецепты, которые можно подключить вручную. Поэтому обычная установка `Buildozer` сама по себе не даёт готовой поддержки `pygame-ce`: собрать APK с `pygame-ce` можно, подключив один из этих рецептов вручную (например, через форк или локальный `custom`-рецепт), но это экспериментальная, а не стандартная или гарантированно поддерживаемая интеграция — совместимость стоит проверять заново на момент сборки и обязательно тестировать на реальном устройстве. Процесс в любом случае заметно длиннее, чем упаковка под десктоп (раздел 20.29): нужен набор инструментов Android (Android SDK/NDK), сборка занимает существенно больше времени, а итоговый размер файла заметно больше, чем сам код игры — внутрь APK упаковывается целая среда выполнения Python.

iOS — через rугame-ios, экспериментальный инструмент

ругame-ios — экспериментальный проект, не готовый к публикации

Для iOS есть инструмент сообщества rугame-ios — шаблон проекта, который встраивает Rугame-игру в стандартный проект Xcode. Его собственная документация прямо описывает его как недостаточно протестированный и не готовый к продакшн-использованию: автор инструмента сообщает, что сборка для реальной публикации в App Store им лично не проверялась. Это не "один из двух равноценных путей", а заметно менее зрелый инструмент, чем python-for-android — воспринимать его стоит как отправную точку для экспериментов, а не как готовый маршрут публикации.

Помимо статуса самого инструмента, публикация на iOS требует компьютера Mac (Xcode доступен только на macOS), сборки и подписи через сам Xcode и, как и любое iOS-приложение, платного аккаунта разработчика Apple. Инструмент поддерживает не каждую версию rугame-ce — конкретную совместимость стоит сверять с документацией инструмента на момент сборки, а не полагаться на память из прошлого проекта.

	Android	iOS
Инструмент	python-for-android + Buildozer (сообщество)	ругame-ios (сообщество, экспериментальный)
Официальная поддержка rугame-ce	Нет — рецепт не принят в основной репозиторий	Нет

	Android	iOS
Готовность к публикации	Экспериментальная сборка через community pull request'ы или custom-рецепт; требует проверки актуального состояния рецепта и инструментов	Автор описывает как не готовый к продакшену
Нужная система для сборки	Windows, macOS или Linux	Только macOS (нужен Xcode)
Итоговый формат	.apk / .aab	Проект Xcode → .ipa
Публикация	Google Play	App Store, нужен аккаунт разработчика Apple

Консоли

Для игровых консолей (раздел 20.9) заметной, поддерживаемой сообществом переходной цепочки вроде `python-for-android` не существует. Публикация на консоли остаётся отдельным, закрытым процессом через официальный SDK производителя — это не связано с выбором Pygame конкретно, так же устроена публикация игр, написанных на любом инструменте.

Планируйте мобильную сборку заранее, а не в конце

Оба мобильных пути добавляют реальную инженерную работу поверх готовой десктопной игры — это не финальный экспорт нажатием одной кнопки, а отдельный этап проекта со своими требованиями к окружению сборки, причём готовность этих путей заметно различается между Android и iOS. Решение выпустить игру и на телефоны стоит принимать в начале работы над игрой (учитывая ограничения раздела 20.13 — батарею, память, жизненный цикл, ввод), а не пытаться пристроить его после того, как весь код уже написан под десктопные предположения.

Профессии в разработке игр и портфолио

Разработка игр — семейство смежных профессий, и завершённые проекты в этой области — практичный способ показать реальный навык.

Разработка игр — не одна профессия, а целое семейство смежных специальностей (раздел 20.6 уже перечислил основные роли). Если этот мини-проект вам понравился, вот несколько направлений, куда можно расти дальше.

Направления в разработке игр

ГЕЙМПЛЕЙ-ПРОГРАММИСТ

Реализует механики игры

Начать: этот проект → собственные
небольшие игры → более сложный движок

ГЕЙМ-ДИЗАЙНЕР

Проектирует правила и баланс

Начать: анализировать, что делает существующие игры увлекательными

ТЕХНИЧЕСКИЙ ХУДОЖНИК

Мост между графикой и кодом

Начать: шейдеры, спрайты, анимация – раздел 20.23

ИНЖЕНЕР ДВИЖКОВ

Строит инструменты для других разработчиков

Начать: понимание того, как устроен сам Ругаме изнутри

QA / ТЕСТИРОВЩИК

Находит баги до релиза

Начать: раздел 20.28 – системная отладка на чужом коде тоже развивает этот навык

Зачем разработчику игр портфолио

Для многих junior-позиций в геймдеве завершённые проекты — практическое доказательство реального навыка: работодатель может сам открыть и посмотреть, что вы построили и как это работает, а не только прочитать список пройденных курсов. При этом требования сильно различаются между компаниями и ролями: где-то формальное образование обязательно, где-то важнее опыт, и портфолио не во всех случаях заменяет диплом — это ещё один способ показать практический уровень, а не гарантированный пропуск в профессию. Прыгающий мяч и финальный проект этой главы (раздел 20.33) — маленький, но настоящий первый пункт такого портфолио: доведённый до конца, а не брошенный на середине.

Следующий шаг уже в этой книге

Глава 21 — космический шутер на Rугame — использует ровно ту же архитектуру класса `Game` (раздел 20.26), что и финальный проект этой главы, но добавляет врагов, снаряды и более сложные столкновения. Это естественное продолжение того же портфолио, а не отдельный проект с нуля.

Финальный проект: «Прыгающий мяч» с архитектурой `Game`

Тот же мяч, что и в разделе 20.5 — но теперь с архитектурой, которая переживёт рост проекта в нечто большее, чем один файл.

Вернёмся к прыгающему мячу из раздела 20.5 в последний раз — но теперь пересоберём его заново, применив всё, что глава добавила по пути: архитектуру класса `Game` (раздел 20.26), движение через `delta time` (раздел 20.16) и явные состояния игры (раздел 20.25).

```
myach : BouncingBallGame
```

```
state = SostoyanieIgry.IGRA
x, y = 38.7, 328.8
vx, vy = -182.5, ...
-122.9 # пикселей в
секунду
otskokov = 7
schet = 70
```

Реальное состояние работающей игры в конкретный момент времени — то, что на самом деле хранится в памяти между кадрами. Обратите внимание на две закономерности: счёт всегда равен числу отскоков, умноженному на 10, а длина вектора скорости остаётся равной 220 пикселям в секунду — отскок разворачивает направление, но не меняет величину.

Что изменилось по сравнению с версией 20.5

	bouncing_ball_basic.py (20.5)	bouncing_ball.py (эта версия, Pro)
Организация кода	Глобальные переменные, плоские функции	Класс Game с методами handle_events/update/render/run

	bouncing_ball_basic.py (20.5)	bouncing_ball.py (эта версия, Pro)
Движение	$x += dx$ за кадр — зависит от FPS	$x += vx * dt$ — не зависит от FPS (раздел 20.16)
Пауза	Нет	Есть, останавливает <code>update()</code> , не только отрисовку
Рестарт	Нет — нужно перезапускать программу	Клавишей, без перезапуска процесса
Счёт	Нет	Счёт очков и счётчик отскоков
Взаимодействие мышью	Нет	Клик мышью — необязательный разворот мяча в сторону клика

bouncing_ball_pro_fragment.py

```
class BouncingBallGame:
    def __init__(self):
        pygame.init()
        self.screen =
        pygame.display.set_mode((SHIRINA, VYSOTA))
        self.clock = pygame.time.Clock()
        self.state = SostoyanieIgry.IGRA
        self.running = True
        self._reset_myach()

    def update(self, dt):
        if self.state is not SostoyanieIgry.IGRA:
            return
        self.x += self.vx * dt
        self.y += self.vy * dt
        if self.x - RADIUS < 0 or self.x + RADIUS >
SHIRINA:
```

```

        self.vx = -self.vx
        self.otskokov += 1
    if self.y - RADIUS < 0 or self.y + RADIUS >
VYSOTA:
        self.vy = -self.vy
        self.otskokov += 1

```

Реальное окно: красный мяч у верхнего края тёмно-синего поля, вверху слева текст «Счёт: 10 Отскоков: 1»

Реальное окно: финальная Pro-версия во время игры — счёт и число отскоков растут вместе, как и должны по правилу из счёта очков: один отскок это десять очков.

Реальное окно: тот же мяч и счёт, поверх — жёлтая надпись «ПАУЗА — Р, чтобы продолжить» по центру экрана

Реальное окно: состояние ПАУЗА — оверлей поверх замершего кадра, ровно то же положение мяча, что и до нажатия паузы.

Полный, уже проверенный файл — отдельно:

[projects/pygame/bouncing-ball/bouncing_ball.py](#)

★★ Самостоятельная задача

Столкновение мяча с мячом

Добавьте второй мяч и столкновение между двумя мячами через `colliderect()` (раздел 20.21), а не только со стенами.

★★★ Задача повышенной сложности

Меню и Game Over

Добавьте состояния MENU и GAME_OVER (раздел 20.25): игра стартует в MENU по нажатию клавиши, а после определённого числа отскоков переходит в GAME_OVER с финальным счётом на экране.

Практика: собираем финальную Pro-версию

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/20-33/index.html\)](https://www.cartesianschool.org/practice/20-33/index.html)

Итоги главы

Что мы узнали в этой главе

- Pygame — библиотека, а не движок: она даёт окно, отрисовку, ввод и звук, а структуру игры вы строите сами (раздел 20.8).
- Разработка игр — целый мир ролей, жанров и платформ, и Pygame уверенно закрывает прежде всего десктоп, а веб и мобильные — через отдельные инструменты сообщества (разделы 20.6–20.13, 20.29–20.31).
- В этой главе мы строили игровой цикл по базовой схеме Input → Update → Render, повторяемой десятки раз в секунду (раздел 20.14).
- Движение в «пикселях за кадр» зависит от чужого железа; движение через delta time — нет (раздел 20.16).
- `pygame.Rect` и `colliderect()` — основа столкновений, а хитбокс не обязан совпадать с рисунком (раздел 20.21).
- Явные состояния игры (`enum`) и разделение `handle_events()`, `update(dt)`, `render()` и `run()` — простая и удобная архитектура для небольших Pygame-проектов и хорошая основа для следующей главы (разделы 20.25–20.26).

Проект: космический шутер с Rugame

Самый крупный проект книги — корабль, враги, стрельба, счёт, жизни, сложность и полноценный конец игры, собранные в архитектуру на классах и дельта-тайме.

21.1 План игры и подготовка проекта

Импортируем необходимые модули

21.2 Игровой цикл и корабль игрока

Создаём космический корабль

21.3 Движение корабля и появление врагов

Создаём и перемещаем врагов

21.4 Стрельба: создаём и двигаем пули

21.5 Счёт и информация на экране

21.6 Попадания и столкновения

Когда враг уничтожает корабль

21.7 Завершение игры и экран «Игра окончена»

21.8 Первая рабочая версия игры

21.9 Разделяем игру на классы

21.10 Ресурсы игры: графика и звук

21.11 Точное движение с Vector2

21.12 Игровое поле и интерфейс игрока

21.13 Скорострельность и интервал между выстрелами

21.14 Как появляются враги

21.15 Попадания во врагов и начисление очков

21.16 Жизни, урон и временная неуязвимость

21.17 Как растёт сложность игры

21.18 Меню, игра, пауза и завершение

21.19 Перезапуск игры без перезапуска программы

21.20 Анимация взрыва

21.21 Звуки выстрела, попадания и взрыва

21.22 Звёздный фон и порядок отрисовки

21.23 Как находить и исправлять ошибки в шутере

21.24 Как сделать игру удобной для автоматических тестов

21.25 Собираем финальную версию игры

21.26 Что мы построили и как развивать игру дальше

План игры и подготовка проекта

Самая крупная игра книги — все части уже знакомы по главе 20, здесь они впервые работают вместе.

Этот проект — самая крупная игра книги: корабль игрока, враги, спускающиеся сверху, стрельба, уничтожение врагов, счёт очков, жизни, растущая сложность и полноценный конец игры с перезапуском. Все составные части уже знакомы по главе 20 — `Rect`, `colliderect()`, спрайты, состояния игры, `delta time` — здесь они впервые работают вместе, в одном учебном проекте.

Реальный кадр готовой игры: синий корабль внизу, несколько врагов разных типов и пуль на игровом поле, HUD со счётом и жизнями сверху

Реальный кадр финальной версии игры — то, что получится в итоге этой главы.

Импортируем необходимые модули

```
importy.py
```

```
import random
from dataclasses import dataclass
from enum import Enum, auto
from pathlib import Path

import pygame
```

`dataclasses` (глава 14) и `enum` (глава 18) здесь не случайны: класс-описание типа врага и явные состояния игры — тот же приём, что `SostoyanieIgru` в разделе 20.25.

Инициализируем всё необходимое

`inicializaciya.py`

```
SHIRINA, VYSOTA = 480, 720
FPS = 60

pygame.init()
screen = pygame.display.set_mode((SHIRINA, VYSOTA))
pygame.display.set_caption("Космический шутер")
clock = pygame.time.Clock()
shrift = pygame.font.SysFont(None, 28)
```

`pygame.font` — тот же модуль, что и в главе 20

`pygame.font.SysFont(None, 28)` создаёт шрифт системным по умолчанию, размером 28 — понадобится для табло счёта в разделе 21.5. Полная финальная версия использует сразу три размера шрифта — обычный, крупный (заголовки экранов) и мелкий (подписи).

План проекта

Прежде чем писать код, полезно увидеть карту того, что получится в итоге. Финальная версия (раздел 21.25) состоит из пяти классов с чёткими обязанностями:

Пять классов финальной игры

GAME

Владеет всем состоянием игры
Игровой цикл: события → ввод →
обновление → отрисовка
Переключает состояния MENU/PLAYING/
PAUSED/GAME_OVER

PLAYER

Позиция как Vector2 (раздел 21.11)
Движение, ограниченное игровым полем
Интервал между выстрелами и таймер
неуязвимости

BULLET

Летит вверх с постоянной скоростью
Удаляет себя за пределами игрового
поля

ENEMY

Летит вниз с индивидуальной скоростью
Несёт очки за уничтожение

EXPLOSION

Анимация из нескольких кадров
Удаляет себя после последнего кадра

Этот раздел и разделы 21.2–21.8 повторяют исторический порядок изложения из бумажной книги и намеренно начинают проще, чем финальная версия — с обычных функций и списков, как в главе 20. С раздела 21.9 книга объясняет, как и почему этот код вырастает в архитектуру на классах.

Практика: импорт и инициализация

Rugame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/21-01/index.html>)

Игровой цикл и корабль игрока

План того, что должно происходить на каждом кадре — и первый персонаж игры.

Игровой цикл

Как и в главе 20, цикл — сердце игры. На этот раз он на каждом кадре обновляет сразу несколько вещей: положение корабля, врагов, пуль — и проверяет столкновения между ними. С самого первого кадра цикл считает *delta time* (сокращённо *dt*) — раздел 20.16 уже объяснял, зачем: скорость, заданная в пикселях в секунду и применённая через *dt*, не зависит от частоты кадров конкретного компьютера:

igrovoj_cikl_plan.py

```

while работает:
    dt = clock.tick(FPS) / 1000.0    # секунд с
    прошлого кадра

    # 1. обработать события (выход, пауза, выстрел по
    нажатию)

    # 2. обработать зажатые клавиши (движение корабля,
    удержание огня)
    # 3. обновить положение врагов и пуль по dt,
    проверить столкновения
    # 4. нарисовать всё заново

```

dt здесь с самого начала, а не добавляется позже

Мини-проект прыгающего мяча (раздел 20.5) начинался с движения «пикселей за кадр» и переходил на delta time только в разделе 20.16. Здесь так делать не будем: раз глава 20 уже объяснила, почему движение «пикселей за кадр» зависит от чужого железа, космический шутер с первого движущегося объекта считает скорость в пикселях в секунду (px/s) и применяет её через dt.

Создаём космический корабль

Вместо отдельных переменных `x`, `y` корабля удобно использовать `pygame.Rect` — он сразу хранит позицию и размер вместе и понадобится для проверки столкновений:

```
korabl.py
```

```
KORABL_SHIRINA, KORABL_VYSOTA = 44, 44

korabl = pygame.Rect(
    SHIRINA // 2 - KORABL_SHIRINA // 2,
    VYSOTA - KORABL_VYSOTA - 20,
    KORABL_SHIRINA,
    KORABL_VYSOTA,
)

# в игровом цикле:
pygame.draw.rect(screen, (80, 220, 120), korabl)
```

Реальное окно: крупный план синего треугольного корабля с оранжевым свечением двигателя внизу

Финальная версия рисует корабль собственным спрайтом (раздел 21.10), а не прямоугольником — но Rect остаётся его хитбоксом.

Rect хранит четыре числа — и много полезных свойств

Rect(x, y, ширина, высота) дополнительно вычисляет `.centerx`, `.bottom`, `.top` и другие удобные координаты — они пригодятся уже в следующем разделе.

Практика: Rect и первый корабль

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/21-02/index.html>)

Движение корабля и появление врагов

Скорость в пикселях в секунду с самого начала — и враги, появляющиеся сверху с течением времени.

Перемещаем космический корабль

Управление — через `get_pressed()` из главы 20 (непрерывное движение, пока клавиша зажата). Скорость сразу задана в пикселях в секунду (px/s), а позиция корабля хранится отдельным дробным числом — `Rect` понимает только целые координаты, и обновлять его напрямую по чуть-чуть на каждом кадре значило бы терять дробную часть движения:

`dvizhenie_korablya.py`

```
KORABL_SKOROST = 260.0 # px/s

korabl_x = float(korabl.x)
# дробная копия позиции — сам Rect хранит только целые числа

def obrabotat_klavishi(korabl_x, klavishi, dt):
    napravlenie = klavishi[pygame.K_RIGHT] -
klavishi[pygame.K_LEFT]
    korabl_x += napravlenie * KORABL_SKOROST * dt
    return max(0.0, min(korabl_x, SHIRINA -
KORABL_SHIRINA))

# в игровом цикле:
korabl_x = obrabotat_klavishi(korabl_x, klavishi, dt)
korabl.x = round(korabl_x)
```

max/min — тот же приём ограничения, что и в главе 20

`max(0.0, min(korabl_x, SHIRINA - KORABL_SHIRINA))` гарантирует, что `korabl_x` никогда не выйдет за пределы `[0, SHIRINA - KORABL_SHIRINA]` — тот же самый приём «зажимания» значения в диапазон, что мы использовали для прыгающего мяча.

napravlenie — число -1, 0 или 1

`klavishi[pygame.K_RIGHT] - klavishi[pygame.K_LEFT]` — булевы значения `True / False` ведут себя как 1/0 (глава 4), поэтому разность даёт ровно то, что нужно: 1, если зажата только правая стрелка, -1, если только левая, и 0, если обе или ни одна. Раздел 21.11 обобщит этот же приём на два измерения сразу через `Vector2`.

Создаём и перемещаем врагов

Враг тоже появится позже как отдельный спрайт-класс (раздел 21.9), а пока — словарь с `Rect` и дробной координатой `Y`, по той же причине, что и у корабля. Врагов будет много, и они появляются со временем — значит, нужен список (глава 11) и таймер в секундах до следующего появления:

`vragi.py`

```
VRAG_SHIRINA, VRAG_VYSOTA = 32, 28
VRAG_SKOROST = 150.0 # px/s
INTERVAL_POYAVLENIYA_VRAGA = 0.75 # секунд между
новыми врагами

vragi = []
vremya_do_vraga = INTERVAL_POYAVLENIYA_VRAGA

def sozdat_vraga():
    x = random.randint(0, SHIRINA - VRAG_SHIRINA)
    return {
        "rect": pygame.Rect(x, -VRAG_VYSOTA,
```

```
VRAG_SHIRINA, VRAG_VYSOTA),
    "y": float(-VRAG_VYSOTA),
    }}

# в игровом цикле, каждый кадр:
vremya_do_vraga -= dt
if vremya_do_vraga <= 0.0:
    vragi.append(sozdat_vraga())
    vremya_do_vraga += INTERVAL_POYAVLENIYA_VRAGA

for vrag in vragi:
    vrag["y"] += VRAG_SKOROST * dt
    vrag["rect"].y = round(vrag["y"])
```

Реальное окно: маленький оранжевый треугольный враг спускается сверху к синему кораблю игрока

Реальное окно — маленький «разведчик» (scout), один из двух типов врагов финальной версии (раздел 21.17).

y = -VRAG_VYSOTA — враг появляется чуть выше экрана

Отрицательная стартовая координата Y означает, что враг рождается чуть выше видимой области и плавно «въезжает» в кадр сверху — а не появляется резко посередине экрана.

+= interval, а не = interval — вот что сохраняет остаток

После появления врага мы прибавляем интервал к текущему значению таймера, а не присваиваем интервал заново. Поэтому небольшое превышение времени не теряется: если `vremya_do_vraga` ушёл в `-0.01`, следующий отсчёт начнётся именно с этого небольшого долга, а не с чистого интервала. В разделе 21.14 этот же принцип будет обобщён с помощью `while` на случай, когда один обрабатываемый шаг может охватить сразу несколько интервалов.

Практика: движение корабля и врагов

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/21-03/index.html>)

Стрельба: создаём и двигаем пули

Пробел создаёт пулю у носа корабля — она улетает вверх, пока не покинет экран.

Пули — словарь с `Rect` и дробной координатой `Y`, появляющийся у носа корабля при нажатии пробела и улетающий вверх со скоростью в пикселях в секунду:

strelba.py

```
PULYA_SHIRINA, PULYA_VYSOTA = 6, 18
PULYA_SKOROST = 560.0 # px/s

puli = []

def vystrelit():
    pula_rect = pygame.Rect(
        korabl.centerx - PULYA_SHIRINA // 2,
        korabl.top,
        PULYA_SHIRINA,
        PULYA_VYSOTA,
    )
    puli.append({"rect": pula_rect, "y":
float(pula_rect.y)})

# в обработке событий:
if event.type == pygame.KEYDOWN and event.key ==
pygame.K_SPACE:
    vystrelit()

# в обновлении кадра:
for pula in puli:
    pula["y"] -= PULYA_SKOROST * dt
```



```
pulya["rect"].y = round(pulya["y"])
puli = [p for p in puli if p["rect"].bottom > 0] #
убираем улетевшие за экран
```

Реальное окно: маленькая жёлтая пуля вылетает из носа синего корабля вверх

Реальное окно — пуля появляется точно у носа корабля и летит вверх.

Список через генератор списков — чистка «мусора»

[p for p in puli if p["rect"].bottom > 0]
(генератор списков из главы 11) оставляет только те пули, что ещё видны на экране — без этой строки список пуль рос бы бесконечно, замедляя игру всё сильнее.

KEYDOWN — одно событие на одно нажатие

Событие `pygame.KEYDOWN` возникает один раз в момент физического нажатия клавиши. Автоматический повтор клавиш в Pygame по умолчанию выключен (включить его можно только явно, через `pygame.key.set_repeat(...)`, а мы этого не делаем) — значит, удержание пробела не порождает новый `KEYDOWN` на каждом кадре. Здесь это даёт ровно один выстрел за одно нажатие: удобно для одиночной стрельбы, но не подходит для удержания огня — этим займётся раздел 21.13.

Практика: стрельба и движение пуль

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/21-04/index.html>)

Счёт и информация на экране

Текст в Pygame рисуется в два шага: сначала `render()`, потом `blit()` на экран.

Счёт выводится готовым шрифтом Pygame (`pygame.font` , раздел 21.1) — в отличие от Turtle, здесь текст сначала «отрисовывается» в отдельное изображение (`render()`), а затем накладывается на экран (`blit()`):

```
tablo_scheta.py
```

```
schet = 0

# в отрисовке кадра:
tablo = shrift.render(f"Счёт: {schet}", True, (255,
255, 255))
screen.blit(tablo, (10, 10))
```

`render()` + `blit()` — тот же принцип, что `write()` у Turtle

`render(текст, сглаживание, цвет)` создаёт изображение текста; `screen.blit(изображение, позиция)` накладывается его на экран — концептуально то же самое, что `artist.write()` у Turtle из главы 7, просто в два явных шага вместо одного.

Реальное окно: верхняя полоса интерфейса со счётом 100 слева и числом жизней справа на тёмном фоне

Финальная версия выделяет счёт и жизни в отдельную HUD-полосу поверху (раздел 21.12) — то же табло, просто отделённое от игрового поля.

Практика: включает вывод счёта

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/21-03/index.html>)

Попадания и столкновения

`colliderect()` проверяет столкновения — попадание уничтожает врага, а враг может уничтожить корабль.

Уничтожаем врагов

Столкновение пули с врагом проверяет готовый метод `Rect.colliderect()` — при попадании оба удаляются из своих списков, а счёт увеличивается:

`unichtozhenie_vragov.py`

```
novye_puli = []
novye_vragi = list(vragi)
for pula in puli:
    popala = False
    for vrag in list(novye_vragi):
        if pula.colliderect(vrag):
            novye_vragi.remove(vrag)
            schet += 10
            popala = True
            break
    if not popala:
        novye_puli.append(pula)
puli = novye_puli
vragi = novye_vragi
```

Почему не удалять элементы прямо во время перебора списка?

Удаление элемента из списка `vragi` прямо внутри цикла `for vrag in vragi:` может пропустить соседние элементы — список «сдвигается» под ногами цикла. Безопаснее собрать **новые** списки уцелевших пуль и врагов, как показано выше. Финальная версия (раздел 21.9) решает эту же задачу без ручных списков — через `pygame.sprite.Group` и `groupcollide()`.

Реальное окно: оранжевая вспышка взрыва в точке, где пуля попала во врага, счёт увеличился до 100

Реальное окно — момент попадания: враг уничтожен, счёт увеличился ровно один раз.

Когда враг уничтожает корабль

Если враг долетел до низа экрана или столкнулся с кораблём — игра заканчивается:

`unichtozhenie_korablya.py`

```
for vrag in vragi:
    if vrag.bottom >= VYSOTA or
vrag.colliderect(korabl):
    igra_okonchena = True
    break
```

Мгновенный конец игры — тоже временное решение

Здесь одно столкновение сразу заканчивает игру. Финальная версия (раздел 21.16) заменяет это на систему жизни с временной неуязвимостью после удара — так три врага, столкнувшихся с кораблём в один момент, не отнимают сразу три жизни.

Практика: столкновения пуль, врагов и корабля

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/21-06/index.html>)

Завершение игры и экран «Игра окончена»

Каждый кадр рисуется заново целиком — а при окончании игры на экране появляется финальная надпись.

Рисуем кадр заново

После всех проверок и обновлений кадр рисуется заново целиком — фон, корабль, пули, враги и счёт, в таком порядке (чтобы более поздние элементы оказывались поверх более ранних):

```
pererisovka.py
```

```
screen.fill((10, 10, 20))
pygame.draw.rect(screen, (80, 220, 120), korabl)
for pulya in puli:
    pygame.draw.rect(screen, (240, 220, 80), pulya)
for vrag in vragi:
    pygame.draw.rect(screen, (230, 60, 60), vrag)

tablo = shrift.render(f"Счёт: {schet}", True, (255, 255, 255))
screen.blit(tablo, (10, 10))

pygame.display.flip()
```

Экран «Игра окончена»

Когда `igra_okonchena` становится истинной, вместо обычной игровой логики выводится финальный экран:

```
game_over.py
```

```
if igra_okonchena:
    nadpis = shrift_bolshoj.render("ИГРА ОКОНЧЕНА",
    True, (255, 255, 255))
    rect = nadpis.get_rect(center=(SHIRINA // 2,
    VYSOTA // 2))
    screen.blit(nadpis, rect)
```

Реальное окно: полупрозрачная тёмная накладка на весь экран, крупная надпись ИГРА ОКОНЧЕНА, счёт и подсказка Enter — заново

Финальная версия добавляет к экрану Game Over счёт, рекорд сессии и явную подсказку, как начать заново (раздел 21.19).

get_rect(center=...) — удобное центрирование текста

Вместо ручного вычисления координат текста по его ширине и высоте, `get_rect(center=(x, y))` сам находит нужный левый верхний угол так, чтобы текст оказался центрирован ровно в точке `(x, y)`.

Из этого экрана нет выхода без перезапуска Python

У переменной `igra_okonchena` нет обратного пути — единственный способ сыграть ещё раз — запустить программу заново. Раздел 21.18 заменит одну булеву переменную на явный `enum.Enum` с четырьмя состояниями (MENU/PLAYING/PAUSED/GAME_OVER) и понятными переходами между ними, включая переход обратно в игру.

Практика: включает финальный экран

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/21-06/index.html>)

Первая рабочая версия игры

Разделы 21.1–21.7 вместе дают рабочую игру, в которую уже можно играть — а глава продолжается дальше.

Собрав разделы 21.1–21.7 вместе, получаем первую рабочую версию игры, в которую уже можно играть — именно на ней заканчивается глава в бумажной книге. Движение корабля, врагов и пуль в ней уже не зависит от частоты кадров: скорости заданы в пикселях в секунду, а появление врагов — таймером в секундах, как и в разделах 21.2–21.4. Дальше, с раздела 21.9, книга продолжает в цифровой версии: тот же проект вырастает в архитектуру на классах, обзаводится собственной графикой и звуком, системой жизни, ростом сложности и полноценными состояниями игры.

Цифровое продолжение проекта — ниже в оглавлении главы

Разделы 21.9–21.26 в боковом меню — это не отдельная тема, а следующий этап разработки той же самой игры: та же механика, но код становится чище, точнее и надёжнее на каждом шаге.

★★ Самостоятельная задача

Жизни вместо мгновенного конца

Добавьте переменную `zhizni = 3` — при столкновении корабля с врагом отнимайте одну жизнь и убирайте врага, вместо немедленного окончания игры. (Раздел 21.16 показывает готовое решение с временной неуязвимостью.)

★★★ Задача повышенной сложности

Уровни сложности

Постепенно уменьшайте интервал появления врагов по мере роста счёта — враги должны появляться всё чаще. (Раздел 21.17 показывает, как сделать это без резких скачков.)

Практика: первая рабочая версия

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

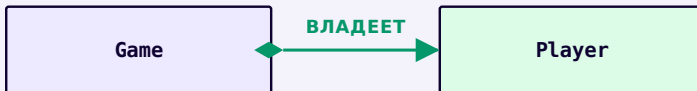
[Открыть практику → \(https://www.cartesianschool.org/practice/21-08/index.html\)](https://www.cartesianschool.org/practice/21-08/index.html)

Разделяем игру на классы

Пять классов с чёткими обязанностями — вместо одних глобальных переменных и списков.

Исторический код разделов 21.1–21.7 хранит корабль, врагов и пули как отдельные переменные и списки `Rect`, а всю логику — прямо в теле игрового цикла. Это работает для маленькой игры, но плохо растёт: чем больше правил, тем труднее понять, какой код за что отвечает. Финальная версия делит игру на пять классов с чёткими обязанностями.

Game владеет всем состоянием игры и координирует остальные классы — сам он не рисует врага и не двигает пулю напрямую.



Game хранит один экземпляр Player.



Game хранит по одной `pygame.sprite.Group` для Bullet, Enemy и Explosion.

`Bullet`, `Enemy` и `Explosion` — наследники `pygame.sprite.Sprite` (раздел 20.19 уже показал, зачем спрайты нужны: у объекта появляются `.image` и `.rect`, которые понимают готовые функции столкновений и группы). Хранить их в `pygame.sprite.Group`, а не в обычном списке, удобно по той же причине, что и в разделе 21.6: группа сама умеет удалять объект (`.kill()`) без риска «сломать» цикл перебора, а `pygame.sprite.groupcollide()` (раздел 21.15) проверяет столкновения между целыми группами одним вызовом.

fragment_game_init.py

```

class Game:
    def __init__(self, *, rng=None, debug=False):
        pygame.init()
        self.screen =
pygame.display.set_mode((SHIRINA, VYSOTA))
        self.clock = pygame.time.Clock()
        self.assets = AssetStore(IMAGE_DIR, AUDIO_DIR)
        self.rng = rng if rng is not None else
random.Random()

        self.state = GameStatus.MENU
        self.bullets = pygame.sprite.Group()
        self.enemies = pygame.sprite.Group()
        self.explosions = pygame.sprite.Group()
        self.player = self._make_player()
        self.score = 0
        self.lives = STARTING_LIVES

```

Это набросок, а не полный файл

Полный, готовый к запуску класс `Game` — в файле `projects/pygame/space-shooter/space_shooter.py`, который подробно разбирает раздел 21.25.

Эта архитектура — не универсальный закон

Разделение на `Game/Player/Bullet/Enemy/Explosion` — удобное решение именно для проекта такого размера, а не единственно правильный способ строить любую игру. Раздел 20.8 уже объяснял: у разных инструментов и проектов архитектура может сильно отличаться.

Практика: набросок класса `Game`

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/21-09/index.html>)

Ресурсы игры: графика и звук

Настоящие спрайты вместо прямоугольников — и класс, который загружает их ровно один раз.

До сих пор корабль, враги и пули рисовались простыми прямоугольниками и кругами — удобно для первых шагов (раздел 20.3). Финальная версия использует собственные спрайты, нарисованные специально для этого проекта, вместо геометрических заглушек.

Спрайты проекта

Реальное окно: маленький оранжевый враг-разведчик слева и более крупный розово-красный враг-истребитель справа, синий корабль игрока снизу

Три собственных спрайта одновременно: разведчик (scout), истребитель (fighter) и корабль игрока — разные силуэты и цвета, чтобы враги считывались с одного взгляда.

Все изображения сохранены как файлы `.png` с прозрачным фоном. При загрузке они попадают в `Surface`, которая поддерживает альфа-канал (флаг `pygame.SRCALPHA` при создании такой `Surface`, раздел 20.17, — это свойство самой `Surface` в `pygame`, а не PNG-файла), поэтому прозрачные пиксели остаются прозрачными и на экране. Спрайты нарисованы напрямую через `pygame.draw` в отдельном скрипте `scripts/generate_chapter_21_assets.py`, а не скачаны откуда-то — простые геометрические фигуры, без единого существующего коммерческого спрайта.

Файл	Роль	Особенность
player_ship.png	Корабль игрока	Стреловидный силуэт, светлая кабина, свечение двигателя
enemy_scout.png	Быстрый, дешёвый враг	Маленький, оранжево-красный треугольник
enemy_fighter.png	Медленный, дорогой враг	Крупнее, пурпурно-красный, с боковыми модулями
bullet.png	Снаряд игрока	Маленькая яркая жёлтая капсула
explosion_sheet.png	Анимация взрыва	6 кадров в ряд — раздел 21.20 режет их через subsurface()

AssetStore — загрузка один раз

Раздел 20.24 уже предупреждал: загрузка файла внутри игрового цикла — частая причина просадок кадров. Здесь всё загружается один раз в момент создания игры, в отдельном классе:

fragment_asset_store.py

```

class AssetStore:
    def __init__(self, image_dir, audio_dir):
        self.images = {}
        for key in ("player_ship", "enemy_scout",
                    "enemy_fighter", "bullet"):
            self.images[key] =
pygame.image.load(image_dir /
f"{{key}}.png").convert_alpha()

        self.sounds = {}
        for key in ("laser", "explosion",
                    "player_hit"):
            path = audio_dir / f"{{key}}.wav"
            try:
                self.sounds[key] =
pygame.mixer.Sound(str(path))
            except (pygame.error, FileNotFoundError):
                self.sounds[key] = None

```

convert_alpha() — тот же смысл, что и в главе 20

pygame.image.load(...) уже сохраняет прозрачность загруженного PNG сама по себе — .convert_alpha() не «создаёт» прозрачность, а лишь готовит копию в формате пикселей экрана для быстрой повторной отрисовки (раздел 20.19). Здесь она вызывается сразу после загрузки, пока изображений мало и это не нужно откладывать.

Звук — необязательное условие для запуска игры

Если в системе нет звукового устройства (например, при автоматическом тестировании),

pygame.mixer.Sound(...) может завершиться ошибкой — тогда self.sounds[key] остаётся None, а игра всё равно запускается и работает, просто без звука. Раздел 21.21 показывает, как проигрывание учитывает это на каждом вызове.

Точное движение с Vector2

Rect хранит только целые числа — позиция живёт отдельно, как Vector2 с плавающей точкой.

`pygame.Rect` удобен для столкновений и отрисовки, но хранит только целые числа. Если на каждом кадре прибавлять к его координате маленькое дробное значение, оно молча теряется при округлении — движение на очень низкой скорости может вообще не сдвинуться с места, хотя код выглядит правильно.

Vector2 (float)

`position += velocity * dt`
не теряет дробную часть



round()

округление только в момент отрисовки



Rect (int)

используется для столкновений и
`blit()`

Позиция всегда живёт как Vector2 с плавающей точкой; Rect каждый кадр пересобирается ИЗ неё, а не обновляется напрямую.

fragment_player_move.py

```

class Player(pygame.sprite.Sprite):
    def __init__(self, image, center):
        super().__init__()
        self.image = image
        self.rect = self.image.get_rect()
        self.position = pygame.Vector2(center)
        self.rect.center = (round(self.position.x),
round(self.position.y))

    def move(self, direction, dt, playfield):
        if direction.length_squared() > 0:
            direction = direction.normalize()
            self.position += direction * self.speed * dt
            self.rect.center = (round(self.position.x),
round(self.position.y))

```

direction.normalize() — то же диагональное правило, что и в разделе 20.16

Вектор направления (1, 1) (вправо-вниз одновременно) без нормализации по длине больше единицы — $\sqrt{2} \approx 1.41$. Если не привести его к единичной длине, корабль по диагонали двигался бы на 41% быстрее, чем строго по одной оси.

direction.length_squared() > 0, а не > 0 сразу для длины

У этих двух строк разные задачи. Проверка > 0 — то, что не даёт вызвать `normalize()` у нулевого вектора (когда игрок вообще не нажимает клавиши движения): нормализация делит вектор на его длину, а длина нулевого вектора равна нулю. А

`length_squared()` (квадрат длины) вместо `length()` нужен только затем, чтобы не извлекать лишний квадратный корень, когда достаточно знать, ненулевой ли вектор — обычная микрооптимизация в реальном коде на `Vector2`.

Игровое поле не отпускает корабль

Каждое движение сразу ограничивается границами игрового поля (раздел 21.12) — так же, как `max(0, min(...))` в разделе 21.3, но теперь применяется и по X, и по Y, потому что корабль двигается во всех четырёх направлениях.

Практика: Vector2, нормализация и clamp

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/21-11/index.html>)

Игровое поле и интерфейс игрока

Счёт и жизни живут в своей полосе сверху — игровые объекты в неё физически не заходят.

Глава 19 уже показывала, что бывает, если интерфейс перекрывает игровое поле: важный текст может оказаться под движущимся объектом. Финальная версия шутера с самого начала разделяет экран на две чётко разные области: игровое поле и полосу с показаниями для игрока — счёт, жизни, номером волны. Такую полосу поверх игрового мира в индустрии принято называть HUD (Heads-Up Display, «приборная панель» — интерфейс, который не является частью самой игры, а просто показывает её текущее состояние).

Реальное окно: верхняя тёмная полоса HUD со счёт и жизнями отделена тонкой линией от основного игрового поля со звёздным фоном и кораблём внизу

Реальное окно — полоса интерфейса сверху (64 пикселя) отделена тонкой линией от игрового поля; ни один враг или снаряд не может оказаться выше этой границы.


```
fragment_playfield.py
```

```
HUD_HEIGHT = 64
```

```
self.playfield = pygame.Rect(0, HUD_HEIGHT, SHIRINA,  
VYSOTA - HUD_HEIGHT)
```

Игровое поле — обычный `Rect`, а не просто число отступа: у него сразу есть `.top`, `.bottom`, `.left`, `.right` — то же самое, что возвращает `korabl.top` в разделе 21.2, только теперь описывает не один объект, а всю разрешённую область движения.

Что где происходит	Область
Счёт, жизни, волна	HUD-полоса (0 — <code>HUD_HEIGHT</code>)
Корабль, враги, пули, взрывы	Игровое поле (<code>HUD_HEIGHT</code> — низ экрана)
Появление врагов	Чуть выше верхней границы игрового поля
Побег врага (раздел 21.16)	Пересечение нижней границы игрового поля

Корабль ограничен именно игровым полем, а не всем окном

`Player.move()` (раздел 21.11) зажимает позицию в границах `self.playfield`, а не всего окна `SHIRINA × VYSOTA` — иначе корабль мог бы заехать под HUD и спрятать сам себя за текстом счёта.

Скорострельность и интервал между выстрелами

Удержание пробела стреляет с ограниченной частотой — в секундах, а не в кадрах.

Раздел 21.4 стрелял по событию `KEYDOWN` — оно происходит ровно один раз на одно физическое нажатие пробела, поэтому даже долгое удержание клавиши там давало только один выстрел. Финальной версии игры нужна стрельба очередями, пока пробел зажат, а для этого требуется другой источник данных — `pygame.key.get_pressed()`, которая на каждом обновлении сообщает, зажата ли клавиша прямо сейчас. Без ограничений это означало бы выстрел на каждом обновлении игры, то есть скорострельность, зависящую от FPS: 60 выстрелов в секунду при 60 FPS и 120 — при 120 FPS. Поэтому стрельба через `get_pressed()` всегда сопровождается минимальным интервалом между выстрелами (`cooldown`, в профессиональной речи — кулдаун), заданным в секундах.

```
fragment_fire_cooldown.py
```

```
FIRE_INTERVAL = 0.20 # секунд между выстрелами

self.fire_cooldown = max(0.0, self.fire_cooldown - dt)

if keys[pygame.K_SPACE] and self.fire_cooldown <= 0.0:
    self._spawn_bullet()
    self.fire_cooldown = FIRE_INTERVAL
```

Интервал измеряется в секундах, а не в кадрах

Если бы вместо `fire_cooldown -= dt` счётчик уменьшался на единицу каждый кадр (`kadrov_do_vystrela -= 1`), скорострельность зависела бы от FPS точно так же, как нескорректированное движение из раздела 20.16: на 120 FPS корабль стрелял бы вдвое чаще, чем на 60 FPS, при одном и том же коде.

Почему здесь не используется `while`, как в таймере появления врагов

Раздел 21.14 покажет финальный таймер появления врагов: он продвигает время через `+= interval`, сохраняя остаток, а `while` там нужен на случай, если один обрабатываемый шаг способен охватить сразу несколько интервалов. Интервал между выстрелами устроен иначе:

	Таймер появления врагов	Интервал между выстрелами игрока
Источник события	Автономная симуляция (сама игра решает, когда)	Ввод игрока (человек решает, когда)
Долгий обрабатываемый шаг	Если шаг способен охватить несколько интервалов, таймер может обработать несколько событий подряд	не должен выстрелить сразу несколько пуль одним кадром
Приём в коде	<code>while</code> : продолжать, пока накоплено время	<code>if</code> : не больше одного выстрела за кадр

В текущей игре второй столбец не наступает

`MAX_DT` не позволяет одному обработанному шагу стать длиннее минимального интервала появления врагов (подробности — в разделе 21.14) — поэтому в этой конкретной игре ни пауза отладчика, ни зависание ОС не приводят к тому, что игра «наверстывает» всё накопленное реальное время залпом из нескольких врагов сразу.

Это осознанный выбор, а не недосмотр

После действительно долгого кадра (пауза отладчика, зависание ОС) внезапный залп из десяти пуль ощущался бы как ошибка, а не как честное поведение — поэтому интервал между выстрелами игрока нарочно даёт максимум один новый выстрел за одно обновление, даже если `dt` оказался большим.

Практика: интервал между выстрелами

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/21-13/index.html>)

Как появляются враги

Таймер появления в секундах — с тем же `while`-накопителем, что и таймер анимации в главе 20.

В разделе 21.3 мы уже измеряли время до следующего врага в секундах: таймер уменьшался на `dt` каждый кадр, а при срабатывании прибавлял интервал заново, а не присваивал его — так остаток не терялся. Теперь перенесём тот же принцип в финальную архитектуру игры и сделаем таймер пригодным для интервала, который меняется вместе со сложностью.

```
fragment_spawn_timer.py
```

```
self.spawn_timer -= dt
while self.spawn_timer <= 0.0:
    self._spawn_enemy()
    self.spawn_timer +=
interval_poyavleniya_vraga(self.score)
```

Два независимых механизма, а не один

self.spawn_timer += interval_poyavleniya_vraga(self.score) — то, что нужно на каждом кадре: остаток времени сверх интервала не выбрасывается, а переносится на следующего врага, точно так же, как в разделе 21.3 и как таймер анимации в разделе 20.23. Это сохранение остатка работает и с обычным if — оно не требует while. Сам while нужен для другого: если бы один обрабатываемый шаг длился дольше сразу нескольких интервалов появления врагов, он честно создал бы несколько врагов подряд за один вызов, а if продвинул бы появление только одного врага и молча потерял остальное накопленное время.

В этой конкретной игре второй случай пока не наступает

Финальная версия игры ограничивает `dt` значением `MAX_DT = 0.05` секунды перед каждым обновлением (защита от скачка после паузы отладчика или зависания ОС), а нижняя граница интервала появления врагов — `MIN_SPAWN_INTERVAL = 0.35` секунды: даже самый долгий допустимый кадр короче любого возможного интервала между врагами. Значит, тело `while` в этой игре сейчас никогда не выполняется больше одного раза за кадр — и написанный через `if` код вёл бы себя здесь ровно так же. `while` оставлен как более общий и надёжный приём: он не сломается, если позже поднять `MAX_DT` или уменьшить интервал появления врагов сильнее, чем сейчас, а `if` в этом случае пришлось бы менять.

Где именно появляется враг

Координата `X` должна гарантировать, что враг полностью помещается внутри игрового поля — без частичного «обрезания» по краю:

```
fragment_spawn_x.py
```

```
def x_poyavleniya_vraga(rng, shirina_vraga, pole):
    levaya, pravaya = pole.left, pole.right -
    shirina_vraga
    if pravaya <= levaya:
        return float(levaya)
    return rng.uniform(levaya, pravaya)
```

Это отдельная, чистая функция — принимает `rng` явным аргументом (раздел 21.24 объясняет, зачем), а не обращается к глобальному `random` напрямую, поэтому её удобно тестировать без запуска всей игры.

Два типа врагов

Реальное окно: несколько врагов разных размеров и цветов спускаются по игровому полю группой

Реальное окно — волна врагов: маленькие быстрые разведчики и более крупные истребители появляются вперемешку.

	Scout (разведчик)	Fighter (истребитель)
Скорость	Выше	Ниже
Размер	Меньше	Больше
Очки за уничтожение	100	200
Вероятность появления	Выше в начале игры	Растёт вместе со счётом (раздел 21.17)

```
fragment_enemy_spec.py

@dataclass(frozen=True)
class EnemySpec:
    name: str
    base_speed: float
    points: int
    image_key: str

ENEMY_SPECS = {
    "scout": EnemySpec("scout", base_speed=150.0,
points=100, image_key="enemy_scout"),
    "fighter": EnemySpec("fighter", base_speed=85.0,
points=200, image_key="enemy_fighter"),
}
```

EnemySpec описывает тип, а не отдельного врага

@dataclass из главы 14 автоматически создаёт конструктор для четырёх полей. Параметр frozen=True запрещает менять эти поля после создания объекта: общая спецификация scout или fighter остаётся неизменной, а координаты и текущая скорость хранятся уже в каждом экземпляре Enemy. Чтобы добавить тип врага в упражнении 21.25, создайте новый EnemySpec, а не изменяйте существующий.

Практика: таймер появления врагов

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/21-14/index.html>)

Попадания во врагов и начисление очков

groupcollide() проверяет столкновения целых групп одним вызовом — но очки нужно начислять аккуратно.

Раздел 21.6 проверял столкновения пуль и врагов вручную, вложенными циклами по спискам. Группы спрайтов (раздел 21.9) решают эту же задачу одним вызовом:

```
fragment_bullet_enemy_collisions.py
```

```
def resolve_bullet_enemy_collisions(self):
    collisions =
pygame.sprite.groupcollide(self.bullets,
self.enemies, True, True)
    destroyed = {{vrag for vragi in
collisions.values() for vrag in vragi}}
    for vrag in destroyed:
        self._spawn_explosion(vrag.rect.center)
    return ochki_za_unichtozhennyh(destroyed)
```



```
pygame.sprite.groupcollide(a, b, True, True)
```

сравнивает каждый спрайт из `a` с каждым спрайтом из `b` и возвращает словарь: пуля → список врагов, которых она задела. Оба флага `True` означают «удалить из своей группы при столкновении» — сама группа берёт на себя ту же задачу, что вручную решали новые списки в разделе 21.6.

Реальное окно: вспышка взрыва там, где пуля попала во врага, счёт сверху увеличился

Реальное окно — момент попадания: и пуля, и враг уже удалены из своих групп, взрыв запущен.

Почему подсчёт очков идёт через `set`, а не через сумму по всем парам

Если сразу несколько пуль задела одного и того же врага в одном обновлении, `groupcollide()` вернёт этого врага в списках для каждой из них — но физически это один и тот же уничтоженный враг. Собирая всех задетых врагов в множество (`set`) перед подсчётом очков, каждый враг засчитывается ровно один раз, сколько бы пуль его ни задела одновременно.

Практика: столкновения и подсчёт очков

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/21-15/index.html>)

Жизни, урон и временная неуязвимость

Столкновения и побег врага — два разных источника потери жизней с разными правилами.

Раздел 21.6 заканчивал игру от одного столкновения корабля с врагом. Финальная версия вместо этого считает жизни — но здесь есть менее очевидная ловушка: что, если сразу несколько врагов столкнутся с кораблём в один и тот же момент?

```
fragment_player_damage.py
```

```
def resolve_enemy_player_collisions(self):
    hit = pygame.sprite.spritecollide(self.player,
    self.enemies, True)
    for vrag in hit:
        self._spawn_explosion(vrag.rect.center)
    return len(hit) > 0

# в обновлении кадра:
collided = self.resolve_enemy_player_collisions()
if collided and not self.player.is_invulnerable:
    self.lives -= 1
    self.player.take_hit()
```

Три врага одновременно — минус одна жизнь, а не минус три

`spritecollide()` удаляет все столкнувшиеся вражеские спрайты безусловно — но урон кораблю отдельная проверка списывает не более одной жизни за одно обновление кадра, независимо от того, сколько врагов задела корабль одновременно. Без этого разделения три одновременно налетевших врага отняли бы три жизни за один кадр, что ощущалось бы несправедливо резко.

Временная неуязвимость

После удара включается короткое окно неуязвимости — во время него столкновения продолжают убирать врагов, но больше не отнимают жизни:

```
fragment_invulnerability.py
```

```
PLAYER_INVULNERABLE_SECONDS = 1.2

@property
def is_invulnerable(self):
    return self.invulnerable_timer > 0.0

def update_timers(self, dt):
    self.invulnerable_timer = max(0.0,
    self.invulnerable_timer - dt)

def take_hit(self):
    self.invulnerable_timer =
    PLAYER_INVULNERABLE_SECONDS
```

Реальное окно: тонкое голубое кольцо вокруг корабля игрока — визуальный индикатор временной неуязвимости

Реальное окно — ровное кольцо вокруг корабля, а не мигание: спокойный, не раздражающий индикатор неуязвимости.

Ровное кольцо вместо мигания

Мигающий спрайт — распространённый приём, но при быстром моргании он утомляет глаза. Здесь вместо этого — постоянное кольцо вокруг корабля, которое просто исчезает, когда `invulnerable_timer` достигает нуля.

Побег врага за нижнюю границу

У игры есть второй, отдельный источник потери жизней. Враг считается сбежавшим, только когда его спрайт целиком пересёк нижнюю границу игрового поля (`vrag.rect.top > self.playfield.bottom`). Каждый такой враг удаляется и стоит игроку одну жизнь:

```
fragment_enemy_escapes.py
```

```
def resolve_enemy_escapes(self):
    escaped = [
        vrag for vrag in self.enemies
        if vrag.rect.top > self.playfield.bottom
    ]
    for vrag in escaped:
        vrag.kill()
    return len(escaped)
```

```
# в обновлении кадра:
```

```
self.lives -= self.resolve_enemy_escapes()
```

Ограничение «одна жизнь за кадр» относится только к столкновениям

Побег — не удар по кораблю, поэтому временная неуязвимость его не блокирует. Если в одном обновлении поле покинули два врага, игра списывает две жизни; если жизней не осталось, состояние переходит в `GAME_OVER`. Это намеренное правило: столкнувшиеся одновременно враги образуют одно событие удара, а каждый пропущенный враг — отдельный результат, который учитывается ровно один раз.

Практика: урон и неуязвимость

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/21-16/index.html>)

Как растёт сложность игры

Три независимые формулы от счёта — с ограничением сверху и снизу, без резких скачков.

Постоянная сложность быстро становится или слишком лёгкой, или слишком тяжёлой. Финальная версия растит сложность плавно, через три независимые чистые функции от текущего счёта.

```
fragment_difficulty.py
```

```
def interval_poyavleniya_vraga(score):
    return
    max(MIN_SPAWN_INTERVAL, BASE_SPAWN_INTERVAL - score *
        SPAWN_INTERVAL_SCORE_FACTOR)

def mnozhitel_skorosti_vraga(score):
    return 1.0 + min(MAX_ENEMY_SPEED_BONUS, score *
        ENEMY_SPEED_SCORE_FACTOR)

def veroyatnost_istrebitelya(score):
    return min(MAX_FIGHTER_PROBABILITY, score *
        FIGHTER_PROBABILITY_SCORE_FACTOR)
```

Функция	Растёт со счётом	Ограничена сверху/снизу
interval_poyavleniya_vraga	Уменьшается — враги появляются чаще	Не ниже MIN_SPAWN_INTERVAL
mnozhitel_skorosti_vraga	Увеличивается — враги быстрее	Не выше 1.0 + MAX_ENEMY_SPEED_BONUS
veroyatnost_istrebitelya	Увеличивается — больше истребителей	Не выше MAX_FIGHTER_PROBABILITY

Нижняя и верхняя границы — не случайность

Без `max()` / `min()` интервал появления врагов на очень высоком счёте мог бы уйти в ноль или отрицательное число. Тогда `while self.spawn_timer <= 0.0`: в разделе 21.14 никогда не завершился бы: прибавление неположительного интервала не может поднять таймер выше нуля, и цикл создавал бы врагов бесконечно на одном и том же кадре. Ограничения защищают формулы от собственного роста.

Два реальных окна рядом: слева редкие маленькие враги на низком счёте, справа — заметно больше врагов, включая крупные истребители, на высоком счёте

Реальное сравнение — та же самая формула сложности, применённая к низкому и высокому счёту.

Номер волны — только для интерфейса

```
fragment_wave.py
```

```
WAVE_SCORE_STEP = 500

def nomer_volny(score):
    return 1 + score // WAVE_SCORE_STEP
```

Волна — не отдельная система с собственными правилами, а просто удобный способ показать игроку прогресс: раз в 500 очков номер волны на HUD увеличивается, хотя сама сложность растёт непрерывно, без резких скачков в этот момент.

Практика: формулы сложности

Проверяется прямо в браузере — установка Pygame не требуется

Открыть практику → (<https://www.cartesianschool.org/practice/21-17/index.html>)

Меню, игра, пауза и завершение

Явное перечисление GameState и таблица разрешённых переходов вместо одной булевой переменной.

Раздел 21.7 хранил конец игры в одной переменной `igra_okonchena` без пути назад. Финальная версия использует явное перечисление — тот же приём, что и `SostoyanieIгры` в разделе 20.25:

```
fragment_game_status.py

class GameState(Enum):
    MENU = auto()
    PLAYING = auto()
    PAUSED = auto()
    GAME_OVER = auto()
```

Из состояния	В состояние	Когда
MENU	PLAYING	нажали Enter
PLAYING	PAUSED	нажали P
PAUSED	PLAYING	нажали P ещё раз
PLAYING	GAME_OVER	жизни закончи- лись
GAME_OVER	PLAYING	нажали Enter (пе- резапуск)

Выход (`Esc` или закрытие окна) работает из любого состояния — поэтому в таблице выше он не привязан к конкретной строке: это не переход между игровыми состояниями, а завершение самой программы.

Три реальных окна рядом: пустое игровое поле, экран паузы с полупрозрачной накладкой, экран Игра окончена со счётом

Реальные кадры трёх разных состояний одной и той же игры.

Управление кораблём должно знать о состоянии

Если проверку состояния забыть в `handle_input()`, корабль продолжит двигаться и стрелять даже на экране паузы или Game Over — тот же класс ошибки, что раздел 21.7 уже почти совершил с `igra_okonchena`.

Перезапуск игры без перезапуска программы

Перезапуск обязан сбросить каждый кусочек переходного состояния — не только счёт.

У раздела 21.7 не было пути назад из Game Over. Финальная версия перезапускается прямо в процессе — но перезапуск обязан сбросить каждый кусочек переходного состояния, не только счёт.

fragment_start_new_game.py

```
def start_new_game(self):
    self.bullets.empty()
    self.enemies.empty()
    self.explosions.empty()
    self.player = self._make_player()
    self.score = 0
    self.lives = STARTING_LIVES
    self.spawn_timer = interval_poyavleniya_vraga(0)
    self._reset_stars()
    self.state = GameStatus.PLAYING
```


Сбрасывается	Сохраняется
Позиция корабля	Рекорд сессии (high_score)
Все пули, враги, взрывы	
Счёт и жизни	
Таймер появления врагов и интервал между выстрелами	
Таймер неуязвимости	

Забывтая пуля из прошлой игры — реальная ошибка, не гипотетическая

Если `self.bullets.empty()` пропустить, старые пули из предыдущей попытки останутся висеть на экране новой игры — визуально безобидно, но явно неправильно, и легко упустить при ручном тестировании, если специально не проверить перезапуск с непустыми группами.

Реальное окно: чистое игровое поле сразу после перезапуска, счёт 0, три жизни

Реальное окно сразу после перезапуска — счёт, жизни, таймеры и все группы спрайтов возвращены к начальному состоянию.

Анимация взрыва

Тот же `while`-накопитель времени, что и в главе 20 — теперь для взрыва, а не для ходьбы.

Взрыв — такой же спрайт, как пуля или враг, но вместо движения у него собственная анимация из нескольких кадров. Раздел 20.23 уже показывал правильный приём для таймера анимации — здесь он применяется во второй раз, теперь для взрыва:

fragment_explosion.py

```

class Explosion(pygame.sprite.Sprite):
    def __init__(self, frames, center):
        super().__init__()
        self.frames = frames
        self.frame_index = 0
        self.animation_time = 0.0
        self.image = self.frames[0]
        self.rect = self.image.get_rect(center=center)

    def update(self, dt):
        self.animation_time += dt
        while self.animation_time >=
EXPLOSION_FRAME_INTERVAL:
            self.animation_time -=
EXPLOSION_FRAME_INTERVAL
            self.frame_index += 1
            if self.frame_index >= len(self.frames):
                self.kill()
            return
        self.image = self.frames[self.frame_index]

```

Три реальных окна рядом: пуля летит вверх, враг перед попаданием, оранжевая вспышка взрыва после попадания

Реальная последовательность: выстрел, сближение с врагом, взрыв в момент попадания.

kill() сразу после последнего кадра

Как только `frame_index` выходит за пределы списка кадров, спрайт удаляет себя из всех групп (`.kill()`) — без этого завершённые взрывы копились бы в `self.explosions` бесконечно, занимая память и время отрисовки без всякой пользы.

Практика: таймер анимации взрыва

Проверяется прямо в браузере — установка Pygame не требуется

[Открыть практику](https://www.cartesianschool.org/practice/21-20/index.html) → (<https://www.cartesianschool.org/practice/21-20/index.html>)

Звуки выстрела, попадания и взрыва

Три коротких звука — загруженные один раз, и молча пропускаемые без звуковой карты.

Три коротких звука дополняют игру: выстрел, взрыв и попадание по кораблю. Все они — оригинальные, синтезированные напрямую в Python через встроенный модуль `wave` (короткие синус-сигналы и затухающий шум), без единой скачанной из интернета записи.

```
fragment_play_sound.py
```

```
def play(self, key):
    sound = self.sounds.get(key)
    if sound is not None:
        sound.play()
```

```
# при выстреле:
self.assets.play("laser")
```

```
# при уничтожении врага:
self.assets.play("explosion")
```

```
# при ударе по кораблю:
self.assets.play("player_hit")
```

sound может быть None — и это нормально

Раздел 21.10 уже показал: если звуковое устройство недоступно, `AssetStore` хранит `None` вместо объекта `Sound`. Проверка `if sound is not None` перед каждым `.play()` гарантирует, что игра (и автоматические тесты, раздел 21.24) не падает без звуковой карты — просто играет молча.

Звук загружается один раз, как и изображения

Раздел 20.24 уже предупреждал:

`pygame.mixer.Sound(...)` читает файл с диска — вызывать его внутри игрового цикла на каждый выстрел означало бы читать один и тот же файл заново десятки раз в секунду. Здесь загрузка происходит один раз в `AssetStore.__init__()`, а внутри цикла вызывается только `.play()` у уже загруженного объекта.

Звёздный фон и порядок отрисовки

Процедурные звёзды вместо готового изображения — и явный порядок слоёв, чтобы HUD всегда оставался поверх.

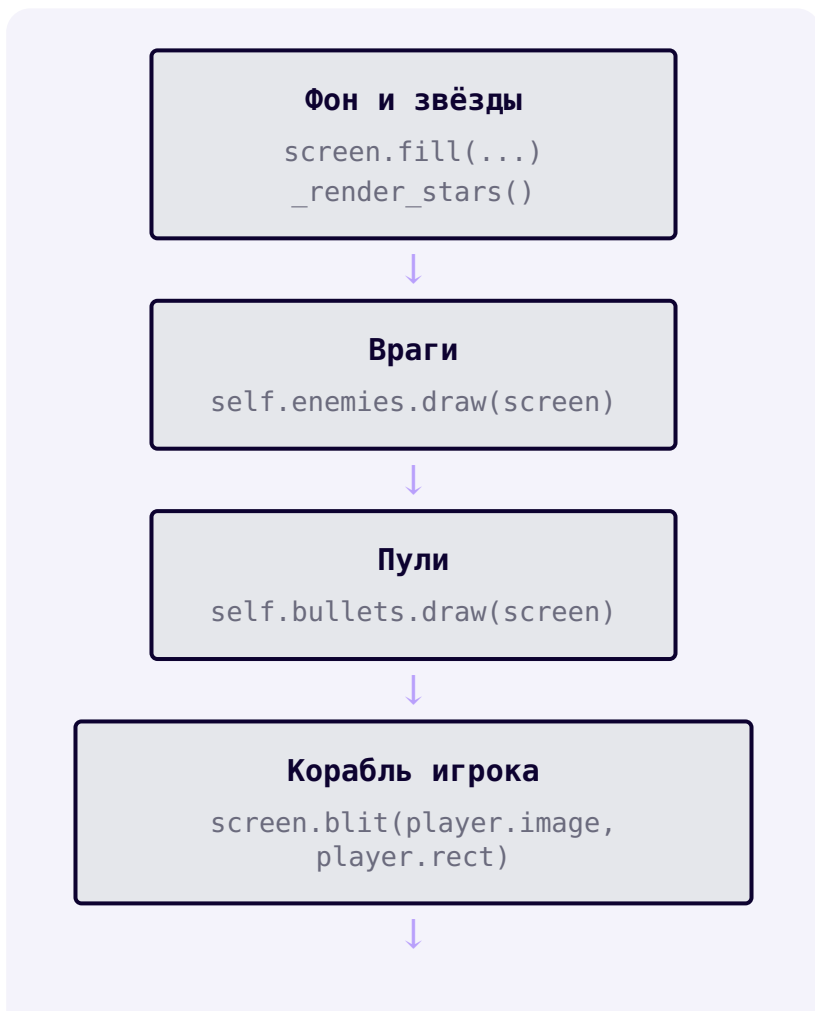
Последний штрих атмосферы — звёздный фон. Вместо готового изображения звёзды рисуются процедурно: список маленьких точек с собственной скоростью и цветом, каждая из которых закидывается сверху вниз.

```
fragment_stars.py
```

```
def _update_stars(self, dt):
    for star in self.stars:
        star.y += star.speed * dt
        if star.y > self.playfield.bottom:
            star.y = self.playfield.top
            star.x = self.rng.uniform(0, SHIRINA)
```

У разных звёзд — разная скорость (от 20 до 90 px/s), поэтому они создают лёгкое ощущение глубины: быстрые точки ближе, медленные — дальше, тот же зрительный приём, что параллакс в более сложных играх, только без отдельных слоёв.

Порядок отрисовки решает, что оказывается «поверх»





Порядок — линейный список, а не случайное совпадение

Раздел 20.13 (мобильный опыт) и раздел 21.12 уже подчёркивали: интерфейс не должен теряться под игровыми объектами. Раздел HUD и оверлей рисуются здесь последними именно поэтому — они физически не могут оказаться под врагом или взрывом.

Как находить и исправлять ошибки в шутере

Справочник по симптому — плюс подробный разбор трёх самых незаметных ошибок проекта.

По ходу главы уже встретилось больше двадцати способов случайно сломать шутер — от старых, основанных на кадрах, приёмах до тонких ошибок с двойным подсчётом очков. Здесь они собраны в один справочник.

Симптом	Причина	Где подробнее
Скорость корабля зависит от FPS	Движение задано «пикселей за кадр», а не px/s через dt	21.11
Медленное движение вообще не сдвигается	Позиция хранится только в Rect (целые числа) — дробная часть теряется	21.11
По диагонали корабль быстрее, чем по прямой	Вектор направления не нормализован	21.11
Корабль вылетает за пределы игрового поля	Забыт clamp по X и/или Y в <code>Player.move()</code>	21.11
Корабль заезжает под HUD	Границы движения — всё окно, а не <code>self.playfield</code>	21.12
Пробел стреляет каждый отрисованный кадр	Нет проверки <code>fire_cooldown</code> перед выстрелом	21.13
Скорострельность зависит от FPS	Интервал между выстрелами измеряется кадрами ($- = 1$), а не секундами ($- = dt$)	21.13

Симптом	Причина	Где подробнее
После долгой паузы отладчика — залп из десятков пуль	Интервал между выстрелами реализован через while вместо if	21.13
Пули летят вечно, игра тормозит сильнее с каждой минутой	Bullet не удаляет себя за пределами игрового поля	21.4 (удаление пуль за экраном)
Появление врагов зависит от FPS	Таймер появления врагов считает кадры, а не секунды	21.14
После просадки FPS «пропадают» запланированные враги	Таймер появления врагов использует if вместо while	21.14
Враг появляется наполовину за краем экрана	x_poyavleniya_vrag а не учитывает ширину врага	21.14
Один враг помечен уничтоженным дважды	Список мутируется прямо во время перебора (как и в разделе 21.6)	21.15
Одному врагу засчитывают очки дважды	Подсчёт идёт по всем парам пуля-враг, а не по множеству уникальных врагов	21.15
Три одновременных врага отнимают три жизни	Урон применяется за каждое столкновение, а не один раз за кадр	21.16

Симптом	Причина	Где подробнее
Неуязвимость «тикает» на паузе	Таймер неуязвимости обновляется вне <code>update_playing()</code> , не проверяя <code>state</code>	21.16, 21.18
На экране Game Over корабль всё ещё двигается	<code>handle_input()</code> не проверяет <code>self.state</code> перед обработкой клавиш	21.18
После перезапуска остаются старые пули или враги	<code>start_new_game()</code> не очищает все группы спрайтов	21.19
После перезапуска враг появляется мгновенно	<code>spawn_timer</code> не сброшен к начальному интервалу	21.19
Анимация взрыва «проглатывает» кадры на слабом устройстве	<code>Explosion.update()</code> использует <code>if</code> вместо <code>while</code>	21.20
Игра подтормаживает перед каждым выстрелом или взрывом	Изображение или звук загружается внутри игрового цикла, а не в <code>AssetStore</code>	21.10, 21.21
Игра падает без звуковой карты	<code>Sound.play()</code> вызывается без проверки на <code>None</code>	21.21

Симптом	Причина	Где подробнее
HUD перекрыт врагом или взрывом	Порядок отрисовки не выносит HUD и оверлей последними слоями	21.22
rect и position со временем «расходятся»	Rect обновлён напрямую, а не пересобран из position	21.11
Тест столкновений то проходит, то падает без изменений в коде	Random используется напрямую, без переданного seed	21.24

Встроенный отладочный оверлей

Нажмите F3, чтобы включить или выключить отладочный оверлей финальной игры. Он рисует красные границы Rect корабля, врагов и пуль, а внизу показывает фактический FPS. Так можно увидеть реальные хитбоксы при разборе столкновений и отличить просадку частоты кадров от ошибки в расчёте движения через dt . Оверлей меняет только отрисовку: состояние игрового мира, таймеры и столкновения остаются прежними.

Разберём три самых незаметных ошибки подробнее

Субпиксельное движение теряется на медленных объектах

Симптом: объект со скоростью меньше одного пикселя за кадр вообще не двигается. **Ломаная версия:** `self.rect.x += self.speed * dt` — Rect хранит только целые числа, и дробное приращение округляется в ноль на каждом кадре по отдельности. **Почему:** округление происходит каждый раз, а не один раз в конце. **Исправление:** накапливать движение в `self.position` (Vector2, раздел 21.11), а `rect.center` пересобирать из неё через `round()` каждый кадр. **Правило:** позиция — всегда float, Rect — производная от неё, никогда не наоборот.

Один враг засчитан дважды

Симптом: счёт иногда прыгает на 200 вместо 100 за одного разведчика. **Ломаная версия:** `for pula, vragi in collisions.items(): schet += sum(v.points for v in vragi)` — суммирование по всем парам пуля-враг. **Почему:** если две пули задела одного врага в одном обновлении, он попадёт в списки обеих пуль. **Исправление:** собрать всех задетых врагов в `set` перед подсчётом очков (раздел 21.15). **Правило:** считать очки по уникальным уничтоженным объектам, а не по числу столкновений.

Три врага одновременно отнимают три жизни

Симптом: лобовое столкновение с группой врагов сразу переводит игру в Game Over, хотя жизней было три. **Ломаная версия:** `for vrag in hit: self.lives -= 1` — урон внутри цикла по столкнувшимся врагам.

Почему: цикл списывает жизнь за каждого врага, а не за сам факт столкновения. **Исправление:** проверить "было ли хоть одно столкновение" один раз после цикла и списать не больше одной жизни за кадр (раздел 21.16). **Правило:** урон измеряется событием «задели», а не количеством объектов, которые это сделали одновременно.

Практика: найдите и исправьте ошибку

Rugame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/21-08/index.html>)

Как сделать игру удобной для автоматических тестов

Чистые функции, `Game(rng=...)` и запуск без окна — игра, которую можно проверить автоматически.

Игру размером с этот проект нельзя проверить только запуском вручную — слишком много комбинаций состояний. Финальная версия написана так, чтобы её можно было тестировать автоматически, без единого настоящего окна.

Чистые функции: логика без Rugame

Формулы сложности (раздел 21.17), координата появления врага (раздел 21.14) и подсчёт очков (раздел 21.15) — обычные функции от чисел и списков, без обращения к `self` или экрану:

```
fragment_pure_functions.py
```

```
def ochki_za_unichtozhennyh(vragi):
    return sum(vrag.points for vrag in vragi)

# Чистой функции не нужен настоящий Enemy со спрайтом
# и картинкой –
# достаточно любого объекта с атрибутом points:
class ZaglushkaVraga:
    def __init__(self, points):
        self.points = points

assert ochki_za_unichtozhennyh([]) == 0
assert ochki_za_unichtozhennyh(
    [ZaglushkaVraga(points=100),
     ZaglushkaVraga(points=200)]
) == 300
```

Это не обход правил, а прямое следствие того, что `ochki_za_unichtozhennyh` вообще не проверяет, какого класса переданные объекты — ей нужен только атрибут `.points` у каждого. Такую функцию можно протестировать любым объектом с этим атрибутом, вплоть до простой заглушки, без запуска Pygame и без создания настоящих врагов.

Управляемая случайность: `Game(rng=...)`

Игра принимает генератор случайных чисел явным параметром, а не обращается к глобальному `random` напрямую:

```
fragment_rng_injection.py
```

```
class Game:
    def __init__(self, *, rng=None):
        self.rng = rng if rng is not None else
        random.Random()

# в реальной игре — обычная непредсказуемая
# случайность:
game = Game()

# в тестах и при генерации скриншотов —
# воспроизводимая:
game = Game(rng=random.Random(42))
```

Один и тот же seed — один и тот же результат

random.Random(42) с фиксированным зерном (seed) даёт одну и ту же последовательность случайных чисел при каждом запуске — поэтому появление врагов, их тип и позиция становятся воспроизводимыми: удобно и для тестов, и для детерминированных сценариев — например, для скриншотов, снятых с конкретного, заранее известного состояния игры.

Запуск без окна: «пустой» SDL-драйвер (headless)

Тот же приём, что и во всех тестах главы 20: переменные окружения переключают Pygame на «пустой» (dummy) видео- и аудиодрайвер SDL — окно нигде физически не появляется, но вся логика (Surface, Rect, столкновения, звук) работает по-настоящему. Такой запуск без настоящего окна в англоязычной терминологии называют headless — «без головы», то есть без экрана и без пользовательского интерфейса.

```
fragment_headless_env.py
```

```
import os

os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
os.environ.setdefault("SDL_AUDIODRIVER", "dummy")

import space_shooter as ss

game = ss.Game() # реальный Game, без настоящего
окна
```

Никакой тяжёлой работы при простом импорте модуля

`space_shooter.py` не вызывает `pygame.init()` или `Game().run()` на уровне модуля — вся инициализация происходит внутри `Game.__init__()`, а бесконечный цикл — только внутри `if __name__ == "__main__":`. Иначе просто `import space_shooter` в тесте запустил бы настоящую игру.

Собираем финальную версию игры

Все разделы 21.9–21.24 — в одном читаемом, готовом к запуску файле.

Все разделы 21.9–21.24 собираются в один файл — `projects/pygame/space-shooter/space_shooter.py`. Он не разбит на несколько модулей: проект достаточно компактен, чтобы один читаемый файл был понятнее, чем россыпь из десятка мелких.

[projects/pygame/space-shooter/space_shooter.py](#)

game : Game

```
state = GameState.PLAYING
score = 640
lives = 3
player.position = (240.0, 612.0)
bullets = 3 спрайта
enemies = 4 спрайта
```

Реальное состояние работающей игры в конкретный момент — то, что на самом деле хранится в памяти между кадрами.

Что изменилось по сравнению с первой рабочей версией (21.8)

	Раздел 21.8 (checkpoint)	Финальная версия (21.25)
Организация кода	Функции и словари/списки поверх Rect	Классы Game/Player/Bullet/Enemy/Explosion поверх pygame.sprite.Group
Движение	$px/s + dt$, дробные координаты хранятся отдельно от Rect	$px/s + dt$ через Vector2 внутри классов Player/Bullet/Enemy
Графика	Закрашенные прямоугольники	Собственные PNG-спрайты
Звук	Нет	Три оригинальных синтезированных звука
Конец игры	Одно столкновение — мгновенный конец	Система жизней с неуязвимостью
Сложность	Постоянная	Растёт плавно вместе со счётом
Состояния	Одна булева переменная	GameStatus: MENU/PLAYING/PAUSED/GAME_OVER
Перезапуск	Только перезапуск Python	Enter на экране Game Over

	Раздел 21.8 (checkpoint)	Финальная версия (21.25)
Взрывы	Нет	Анимация из нескольких кадров
Тесты	7 регрессионных тестов таймера и движения (раздел 21.3)	38 тестов класса Game и вспомогательных функций, включая FPS-независимость

Реальное окно: насыщенный кадр геймплея — синий корабль, несколько врагов и пуль, звёздный фон, читаемый HUD со счётом 640

Реальный кадр финальной версии — тот же корабль, что и в разделе 21.2, но выросший в полноценную игру.

★★★ Задача повышенной сложности

Ещё один тип врага

Добавьте третий EnemySpec — например, "bomber": медленный, но с большим количеством очков и собственным спрайтом (расширьте scripts/generate_chapter_21_assets.py).

★★★ Задача повышенной сложности

Бонусы на игровом поле

Добавьте класс PowerUp: подбираемый объект, временно уменьшающий FIRE_INTERVAL после столкновения с кораблём — используйте тот же приём таймера, что и для неуязвимости.

Практика: собираем и запускаем финальную игру

Pygame открывает нативное окно Python — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/21-25/index.html>)

Что мы построили и как развивать игру дальше

От первого окна в главе 20 до тестируемой, озвученной игры с собственной графикой.

Космический шутер завершён — а вместе с ним закрывается и вторая большая тема курса: полноценная разработка игр на RUGAME, от первого окна в главе 20 до тестируемой, озвученной игры с собственной графикой здесь.

Что мы построили в этой главе

- Полноценную игру из пяти классов: `Game` , `Player` , `Bullet` , `Enemy` , `Explosion` — вместо глобальных переменных и списков.
- Точное движение через `Vector2` и `delta time` — тот же принцип из главы 20, применённый к настоящему проекту с несколькими типами объектов сразу.
- Таймеры, которые различают автономную симуляцию (появление врагов, анимация — сохраняют остаток времени сверх интервала через `while`) и реакцию на ввод игрока (интервал между выстрелами — не более одного события за кадр).
- Столкновения через `pygame.sprite.groupcollide()` и `spritecollide()` , с защитой от двойного подсчёта очков и от потери нескольких жизней за одно одновременное столкновение.
- Явные состояния игры (`GameStatus`) и контракт перезапуска, сбрасывающий действительно всё переходное состояние — не только счёт.
- Собственную графику и звук, загруженные один раз, и игру, которую можно проверить автоматически — через внедрение генератора случайных чисел и запуск без окна под «пустым» SDL-драйвером (`headless`).

Куда развивать проект дальше

Идеи для самостоятельного развития

БОЛЬШЕ ТИПОВ ВРАГОВ

Собственные паттерны движения (зигзаг, наведение на корабль)

Собственные спрайты и очки

БОНУСЫ И УСИЛЕНИЯ

Временные эффекты через тот же приём таймера, что и неуязвимость

Щит, двойной выстрел, дополнительная жизнь

БОССЫ

Крупный враг с собственным здоровьем (не одно попадание)

Собственный набор атак

СОХРАНЕНИЕ РЕКОРДА

Запись `high_score` в файл между запусками (глава 15)

Не только в рамках одной сессии

Следующая глава меняет направление: от локальных игр на Pygame — к веб-разработке на Python. Принципы, освоенные здесь — чёткое разделение обязанностей между классами, явные состояния, тестируемый код — пригодятся и там, просто в других декорациях.

Веб-разработка с Python

Введение в устройство веб-приложений: браузер и сервер, HTTP и HTTPS, HTML/CSS/JavaScript, Flask, базы данных, SQL, JSON и API. Более глубокое изучение продолжается в отдельном курсе по веб-разработке на Python.

22.1 Основы веб-разработки: клиент, сервер и Python

22.2 HTML: структура веб-страницы

22.3 CSS: оформление и расположение элементов веб-страницы

22.4 JavaScript: программирование в браузере

22.5 Первое веб-приложение на Flask

22.6 Итоги первого веб-проекта

22.7 Как браузер получает веб-страницу

22.8 URL: домен, путь, параметры и фрагмент

22.9 HTTP: запросы, ответы, методы и коды состояния

22.10 HTTPS: защита соединения с сайтом

22.11 Маршрутизация во Flask: URL и функции-обработчики

22.12 Jinja: шаблоны HTML с данными

22.13 HTML-формы и отправка данных на сервер

22.14 Статические файлы: CSS, изображения и JavaScript

22.15 JSON: формат обмена данными

22.16 HTTP API: обмен данными между программами

22.17 Веб-фреймворки на Python: Flask, Django, FastAPI и другие

22.18 Сравнение Flask, Django и FastAPI

22.19 WSGI и ASGI: связь веб-приложения с сервером

22.20 Зачем веб-приложению база данных

22.21 Реляционные базы данных: таблицы, ключи и связи

22.22 SQL: создаём, читаем и изменяем данные

22.23 SQLite: встраиваемая реляционная СУБД

22.24 PostgreSQL, MySQL/MariaDB и SQLite: основные различия

22.25 NoSQL: основные виды нереляционных баз данных

22.26 ORM: работа с базой данных через объекты Python

22.27 Миграции схемы базы данных

22.28 Перенос данных между базами

22.29 Flask и SQLite: сохраняем данные в базе

22.30 Cookie и сессии: состояние между HTTP-запросами

22.31 Проверка входных данных и обработка ошибок

22.32 Основы безопасности веб-приложений

22.33 Автоматическое тестирование Flask-приложения

22.34 Развёртывание Flask-приложения

22.35 Итоговый проект: список задач на Flask и SQLite

22.36 Дальнейшее изучение веб-разработки на Python

Основы веб-разработки: клиент, сервер и Python

Веб-приложение — это диалог между клиентом и сервером. Разберёмся, кто в нём кто.

Веб-приложение обычно состоит из двух взаимодействующих сторон: клиента и сервера. Клиентом чаще всего является браузер. Он отправляет HTTP-запросы и отображает полученные ответы. Серверная часть принимает запросы, выполняет логику приложения, при необходимости обращается к базе данных и формирует ответ.

Python часто используется для программирования серверной части веб-приложения. В этой главе серверное приложение написано на Python с использованием Flask.

А как же Python в браузере?

Это не означает, что Python принципиально не может выполняться в браузере. Существуют среды на основе WebAssembly, например Pyodide — на нём построена интерактивная практика этой книги, — которые позволяют выполнять Python на стороне клиента. Однако это другая модель выполнения; здесь изучается классическая серверная веб-разработка.

Клиент и сервер

Клиент — программа, которая отправляет запрос и получает на него ответ. **Сервер** — программа, которая принимает запросы и отвечает на них. Слово «сервер» обозначает именно серверную программу (процесс); в разговорной речи им иногда называют и компьютер, на котором эта программа запущена, — из контекста обычно понятно, что имеется в виду.



Один цикл: запрос уходит на сервер, ответ возвращается и отображается в браузере

Этот обмен запросами и ответами подчиняется общему правилу — протоколу **HTTP** (HyperText Transfer Protocol, протокол передачи гипертекста). Подробнее об этом — в разделах 22.7-22.10; здесь достаточно знать, что браузер и сервер обмениваются сообщениями по одним и тем же правилам, независимо от того, кто их написал.

Что делает браузер

Браузер — это клиентская программа, которая по адресу пользователя строит и отправляет HTTP-запрос, получает HTTP-ответ и отображает его: разбирает HTML, применяет CSS, выполняет JavaScript, показывает изображения и другие ресурсы страницы.

Что делает серверное приложение

Серверное приложение принимает запрос, определяет, что нужно сделать (открыть список задач, сохранить новую запись, вернуть данные в формате JSON), при необходимости обращается к базе данных и формирует ответ — чаще всего HTML-страницу или данные.

Где используется Python

В типичном веб-приложении Python работает на серверной стороне: код выполняется на сервере, а браузеру отправляется уже готовый результат. Дальше в этой главе так и есть — Python исполняется сервером, собранным на Flask.

Интернет и Всемирная паутина

Интернет — это сеть, которая физически соединяет компьютеры друг с другом и умеет передавать между ними данные. **Всемирная паутина** (World Wide Web, веб) — один из способов использовать эту сеть: сайты, ссылки между ними, протокол HTTP. По тому же интернету работают и другие сервисы — почта, мессенджеры, онлайн-игры, — веб лишь один из них, хотя и самый заметный.

Адрес сайта: домен и IP-адрес

Чтобы браузер знал, к какому серверу обращаться, у сайта есть **домен** — имя вида `python.org`, которое человеку легко запомнить. Но компьютеры в интернете

находят друг друга по числовым адресам — **IP-адресам**. Специальная служба, DNS, превращает домен в IP-адрес перед отправкой запроса. Весь этот путь подробно рассматривается в разделе 22.7, а устройство самого адреса — в разделе 22.8.

Фронтенд и бэкенд

То, что видит и с чем взаимодействует пользователь в браузере — HTML, CSS, JavaScript, — называют **фронтендом** (front — «перед»). Программу, которая работает на сервере и готовит ответы, называют **бэкендом** (back — «тыл»).

Статическая страница и динамическое приложение

Самый простой сайт — это просто файлы `.html`, лежащие на сервере: сервер отдаёт их без изменений, это **статическая** страница. Если же ответ каждый раз собирается заново — с учётом того, кто спросил, что хранится в базе данных, что человек только что отправил в форме, — такое приложение называют **динамическим**. Сайт, который вы соберёте в этой главе на Flask, — динамический: он строит HTML-страницу из данных прямо во время запроса.

В следующих трёх разделах — краткое знакомство с языками фронтенда: HTML (раздел 22.2), CSS (раздел 22.3) и JavaScript (раздел 22.4). Затем — первый бэкенд на Python с библиотекой **Flask** (раздел 22.5).

HTML: структура веб-страницы

HTML — язык разметки, а не программирования: он описывает структуру, а не поведение.

HTML (HyperText Markup Language — язык гипертекстовой разметки) описывает не поведение и не внешний вид, а *структуру* страницы: где заголовок, где

абзац текста, где картинка, где ссылка. HTML — язык разметки, а не язык программирования: в нём нет условий, циклов или переменных, только описание того, что и в каком порядке показать.

stranica.html

```
<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>Моя первая страница</title>
</head>
<body>
  <h1>Привет, мир!</h1>
  <p>Это мой первый сайт на HTML.</p>
  <ul>
    <li>Учу Python</li>
    <li>Учу HTML</li>
  </ul>
  <a href="https://python.org">Официальный сайт
Python</a>
</body>
</html>
```

Страница в браузере без единой строчки CSS: чёрный заголовок, обычный текст, список точками, синяя подчеркнутая ссылка

Тот же файл, открытый в браузере, без единого правила CSS — браузер сам выбирает шрифт, отступы и цвет ссылки по умолчанию.

Теги обычно идут парами

`<h1>` — открывающий тег заголовка, `</h1>` (со слэшем) — закрывающий, а вместе с содержимым между ними они образуют один **элемент** страницы. У некоторых элементов, например `<img>`, нет закрывающего тега и вложенного содержимого — это называют **void-элементом**: вся нужная информация передаётся через атрибуты.

Атрибуты — дополнительные свойства тега

Внутри открывающего тега можно указать **атрибуты** — пары `имя="значение"`. У ссылки `<a>` атрибут `href` задаёт адрес, у картинки `<img>` — `src` задаёт файл, а `alt` задаёт текстовую альтернативу изображения, соответствующую его смыслу на странице: её используют программы для чтения с экрана и она показывается, если файл не загрузился:

```
atributy.html
```

```

```

Частые теги

Тег	Что описывает
<code><h1> ... <h6></code>	Заголовки от самого важного (h1) до самого мелкого (h6)
<code><p></code>	Абзац текста
<code> / </code>	Список и его пункты
<code></code>	Ссылка на другую страницу

Тег	Что описывает
<code></code>	Изображение с текстовым описанием
<code><label></code> / <code><input></code>	Поле формы и его подпись

Заголовки — не просто крупный шрифт

Уровни `h1` - `h6` задают логическую структуру страницы, как оглавление книги: `h1` — главный заголовок, у него на странице обычно один, а `h2` — заголовки разделов внутри. Программы для чтения с экрана и поисковые системы используют именно эту структуру, а не то, крупным ли шрифтом что-то нарисовано.

Формы

Форма собирает то, что ввёл пользователь, и отправляет это на сервер:

forma.html

```
<form>
  <label for="imya">Ваше имя</label>
  <input type="text" id="imya" name="imya">
  <button type="submit">Отправить</button>
</form>
```

`<label for="imya">` связан с `<input id="imya">` по совпадающему идентификатору — клик по подписи ставит курсор в поле, а программа для чтения с экрана озвучивает подпись, когда поле получает фокус. Что происходит с этими данными на сервере — в разделе 22.13.

Практика: структура HTML и стили CSS

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/22-02/index.html\)](https://www.cartesianschool.org/practice/22-02/index.html)

CSS: оформление и расположение элементов веб-страницы

CSS определяет внешний вид и расположение элементов HTML-документа.

CSS (Cascading Style Sheets — каскадные таблицы стилей) — язык таблиц стилей, предназначенный для описания внешнего вида и расположения элементов документа, написанного на HTML или другом языке разметки. С помощью CSS задают цвета, шрифты, размеры, отступы, рамки, расположение элементов и адаптацию страницы к разным размерам экрана.

HTML задаёт структуру документа, а CSS определяет, как эта структура отображается на экране: без собственных стилей браузер применяет стандартное оформление — обычный шрифт, чёрный текст, минимальные отступы. CSS выбирает элементы страницы и задаёт им свойства:

stili.css

```
body {
  font-family: sans-serif;
  background: #f7f7fb;
  color: #1a1a2e;
}

h1 {
  color: #4a2fbd;
}

p {
  line-height: 1.6;
}
```

Каждое правило CSS — это **селектор** (какие элементы выбирает правило: по тегу, классу или идентификатору) и фигурные скобки с парами **свойство: значение**. Подключить CSS-файл к HTML-странице можно одной строкой внутри `<head>`:

podklyuchenie.html

```
<link rel="stylesheet" href="stili.css">
```

Та же страница с подключённым CSS: светло-сиреневый фон, тёмно-фиолетовый заголовок и увеличенный межстрочный интервал у абзаца

Ровно тот же HTML из раздела 22.2, но с подключённым stili.css: фон, цвет заголовка и межстрочный интервал теперь заданы явно, а не оставлены на усмотрение браузера.

Классы — стилизуем не все теги подряд

Селектор по тегу (`p { }`) красит все абзацы разом. Чтобы выделить только некоторые элементы, им дают атрибут `class` , а в CSS обращаются к нему через точку:

```
klass.html
```

```
<p class="preduprezhdenie">Осторожно, ступенька!</p>
```

```
klass.css
```

```
.preduprezhdenie {  
  color: #b3261e;  
  font-weight: 700;  
}
```

Каскад и наследование — два разных механизма

Каскад решает, какое из нескольких конкурирующих правил применить к одному и тому же элементу: в упрощённом случае, когда остальные условия равны, более специфичный селектор побеждает менее специфичный, а при одинаковой специфичности — решает порядок правил в файле. Это часть алгоритма каскада, а не он целиком.

Наследование — отдельный механизм: некоторые свойства (например, `font-family`) элемент по умолчанию перенимает от родителя, если для него самого явно ничего не задано.

Блочная модель

У каждого элемента есть содержимое, а вокруг него — отступ внутри рамки (`padding`), сама рамка (`border`) и отступ снаружи, до соседних элементов (`margin`):

```
blochnaya_model.css
```

```
.kartochka {  
  padding: 16px;  
  border: 1px solid #ccc;  
  margin: 12px;  
}
```

Расположение элементов: `display` и `flexbox`

Свойство `display` определяет, ведёт ли себя элемент как блок на всю ширину (`block` , как `<p>` и `<div>`) или как часть строки (`inline` , как `<a>` и `<span>`). Чтобы выстроить несколько элементов в ряд с равными промежутками, чаще всего используют `flexbox` :

```
flexbox.css
```

```
.panel {  
  display: flex;  
  gap: 12px;  
  align-items: center;  
}
```

Адаптивная вёрстка: одна страница, разные экраны

Медиа-запрос (`media query`) применяет часть правил только при определённых условиях — например, только на узких экранах:

```
adaptivnost.css
```

```
@media (max-width: 480px) {
  .panel {
    flex-direction: column;
  }
}
```

Так одна и та же страница может выстраивать элементы в ряд на широком экране и столбиком — на узком, без отдельного мобильного сайта. Итоговый проект главы (раздел 22.35) использует именно этот приём.

Официальная документация: [MDN — CSS](#).

Практика: структура HTML и стили CSS

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/22-02/index.html\)](https://www.cartesianschool.org/practice/22-02/index.html)

JavaScript: программирование в браузере

Язык программирования, который в браузере отвечает за поведение страницы: реакцию на действия пользователя и изменение содержимого.

JavaScript (JS) — язык программирования, широко используемый в веб-разработке. Ядро языка стандартизовано спецификацией ECMAScript (ECMA-262). В браузере JavaScript выполняет программную логику страницы: обрабатывает события, работает с DOM, изменяет содержимое страницы и может отправлять сетевые запросы.

JavaScript работает не только в браузере

JavaScript используется не только в браузерах. Он также работает в серверных и других средах выполнения, например Node.js. Поэтому JavaScript — не просто «язык для браузера», хотя именно браузерное применение рассматривается в этом разделе.

Основные свойства JavaScript

Что за язык JavaScript

ТИПИЗАЦИЯ И ПАМЯТЬ

Динамическая типизация

Автоматическое управление памятью
(сборка мусора)

МОДЕЛЬ ОБЪЕКТОВ

Прототипная модель объектов

Функции — значения первого класса

СТИЛИ ПРОГРАММИРОВАНИЯ

Императивный стиль

Объектный стиль

Функциональный стиль

ECMAScript, DOM и Web API — не одно и то же

ECMAScript — стандарт, описывающий сам язык: синтаксис, типы, операторы, функции. **DOM** (Document Object Model) и такие функции, как `fetch()`, не входят в ECMAScript — это отдельные программные интерфейсы (Web API), которые браузер предоставляет коду JavaScript в дополнение к самому языку.

JavaScript добавляет странице *поведение*: реакцию на клики, изменение содержимого без перезагрузки страницы, проверку формы перед отправкой. JavaScript нужен не каждой странице: простая статья или документ хорошо работают на одном HTML (плюс CSS для оформления) — браузер отображает такую страницу без единой строчки JavaScript. JavaScript нужен именно там, где странице требуется реагировать на действия пользователя без нового запроса к серверу.

DOM — представление страницы, с которым работает JavaScript

Когда браузер разбирает HTML, он строит из тегов дерево объектов в памяти — DOM (Document Object Model). JavaScript не редактирует исходный текст HTML-файла — он находит нужный узел в этом дереве и меняет его:

povedenie.html

```
<button id="кнопка">Нажми меня</button>
<p id="tekst">Пока ничего не произошло.</p>

<script>
  const кнопка = document.getElementById("кнопка");
  const tekst = document.getElementById("tekst");

  кнопка.addEventListener("click", function () {
    tekst.textContent = "Кнопку нажали!";
  });
</script>
```

Страница с кнопкой «Нажми меня» и текстом «Пока ничего не произошло.» до клика

До клика: обработчик подписан, но событие ещё не произошло.

Та же страница сразу после клика по кнопке: текст сменился на «Кнопку нажали!»

После клика: та же страница, без перезагрузки — JavaScript просто заменил содержимое узла #tekst.

`addEventListener("click", ...)` подписывает функцию на **событие** — клик по кнопке. Событий много: `"click"`, `"submit"` (отправка формы), `"input"` (ввод текста) и другие — код выполняется только тогда, когда событие действительно произошло.

Знакомые конструкции в другом синтаксисе

И в Python, и в JavaScript есть переменные, функции, условия и циклы. Но их системы типов, модели объектов и среды выполнения существенно различаются: например, JavaScript использует прототипную модель объектов и фигурные скобки вместо отступов. Если вы понимаете Python, освоить основы JavaScript будет заметно проще, чем начинать с нуля.

Проверка формы прямо в браузере

JavaScript может проверить поле формы ещё до отправки — и сразу показать подсказку, не дожидаясь ответа сервера:

validaciya.js

```
forma.addEventListener("submit", function (event) {
  if (pole.value.trim() === "") {
    event.preventDefault();
    oshibka.textContent = "Поле не должно быть
пустым";
  }
});
```

`event.preventDefault()` останавливает обычную отправку формы — страница не перезагрузится, пока ошибка не исправлена. Ту же проверку всё равно необходимо повторить и на сервере (раздел 22.31): пользователь может отправить запрос вообще без браузера, в обход любого JavaScript.

fetch() — запрос к серверу без перезагрузки страницы

Функция `fetch()` отправляет HTTP-запрос из JavaScript и получает ответ, не перезагружая страницу целиком:

`fetch.js`

```
fetch("/api/tasks")
  .then(function (response) { return
response.json(); })
  .then(function (data) { console.log(data); });
```

Что такое `/api/tasks` и что именно возвращает такой запрос — в разделе 22.16.

Три языка вместе — HTML, CSS и JavaScript — и есть тот самый «фронтенд», который видит и с которым взаимодействует пользователь. Дальше в главе речь пойдёт о том, что происходит на сервере, — то есть о Python.

Официальная документация: [MDN — JavaScript, ECMA-262 \(ECMAScript\)](#).

Первое веб-приложение на Flask

Flask — лёгкий веб-фреймворк для создания веб-приложений на Python.

Flask — лёгкий веб-фреймворк для создания веб-приложений на языке Python. Он предоставляет маршрутизацию, объекты запроса и ответа, работу с шаблонами Jinja и другие базовые средства, необходимые серверному приложению. Сам Flask не принимает сетевые соединения напрямую — этим занимается отдельный сервер (раздел 22.19 объясняет эту границу подробнее); при локальном запуске Flask временно берёт эту роль на себя через встроенный сервер разработки Werkzeug. Установка — как у любой библиотеки:

`terminal.txt`

```
pip install flask
```

Самое маленькое Flask-приложение

app_minimal.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def glavnaya():
    return "Привет, мир!"

if __name__ == "__main__":
    app.run(debug=True)
```

@app.route("/") — декоратор маршрута

Строчка над функцией — **декоратор**.

@app.route("/") регистрирует маршрут: связывает адрес / с функцией glavnaya(), которая обрабатывает запрос к этому адресу и формирует ответ. Этот механизм подробно рассматривается в разделе 22.11.

Шаблоны — HTML с выражениями Jinja

Возвращать HTML прямо строкой в Python неудобно. Flask использует движок шаблонов **Jinja** (раздел 22.12 рассматривает его подробно) — это отдельный язык шаблонов со своим синтаксисом: обычный HTML-файл, куда можно вставлять выражения и управляющие конструкции Jinja в фигурных скобках `{{ }}`:

```
templates/index.html
```

```
<h1>Мой список задач</h1>
<ul>
  {% for zadacha in zadachi %}
    <li>{{ zadacha }}</li>
  {% endfor %}
</ul>
```

```
app.py
```

```
from flask import Flask, render_template

app = Flask(__name__)
zadachi = ["Выучить основы Python", "Собрать сайт на Flask"]

@app.route("/")
def glavnaya():
    return render_template("index.html",
                           zadachi=zadachi)
```

Динамические адреса

Часть адреса можно превратить в параметр маршрута — Flask передаёт его значение аргументом функции-обработчика:

```
dinamicheskij_marshrut.py
```

```
@app.route("/privet/<imya>")
def privet(imya):
    return render_template("privet.html", imya=imya)
```

/privet/Сергей → imya = "Сергей"

Открыв в браузере адрес `/privet/Сергей`, вы получите в переменной `imya` строку `"Сергей"` — эту часть адреса Flask передаёт функции-обработчику как аргумент.

Страница в браузере: заголовок «Привет, Ада!» и ссылка «Назад к списку задач»

Итоговый проект использует именно этот маршрут: адрес `/privet/` Ада в браузере — и Flask подставляет имя из URL в шаблон.

Принимаем данные из формы

Чтобы пользователь мог что-то отправить на сервер (например, добавить задачу), используется HTML-форма и маршрут, принимающий метод `POST` (почему именно `POST`, а не `GET` — в разделе 22.13):

`dobavlenie.py`

```
from flask import request, redirect, url_for

@app.route("/dobavit", methods=["POST"])
def dobavit():
    novaya_zadacha = request.form.get("zadacha",
    "").strip()
    if novaya_zadacha:
        zadachi.append(novaya_zadacha)
    return redirect(url_for("glavnaya"))
```

Готовый мини-сайт «Список задач»

Маршруты, шаблоны, форма и статический CSS-файл из этого раздела собраны в готовый мини-сайт «Список задач»:

`todo-app/app.py`

`projects/flask/`

Запустите сайт у себя

`python app.py` в терминале внутри `projects/flask/todo-app/`, затем откройте `http://127.0.0.1:5000/` в браузере. Так запускается сервер разработки; чем он отличается от рабочего развёртывания — в разделе 22.34.

Сейчас задачи хранятся в обычном списке Python и исчезают при перезапуске программы — почему это проблема, объясняет раздел 22.20, а разделы 22.20-22.29 постепенно заменят список на базу данных SQLite.

★★ Самостоятельная задача**Счётчик задач**

Добавьте в `index.html` строку, показывающую общее количество задач: `{{ zadachi|length }}`.

★★★ Задача повышенной сложности**Удаление задач**

Добавьте маршрут `/udalit/<int:indeks>`, который удаляет задачу по её номеру в списке и делает редирект на главную.

Практика: маршруты, шаблоны и формы

Модуль `flask` не установлен в браузерном окружении Pyodide — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/22-05/index.html) → (<https://www.cartesianschool.org/practice/22-05/index.html>)

Итоги первого веб-проекта

От HTTP-запроса до собственного сайта на Flask — и продолжение впереди, в цифровом издании главы.

Что мы узнали в этой части главы

- Сайт — это диалог по протоколу HTTP: браузер (клиент) отправляет запрос, сервер отвечает.
- Фронтенд (то, что видит пользователь) строится из трёх языков: HTML (структура), CSS (оформление) и JavaScript (поведение).
- Бэкенд — программа на сервере, которая готовит ответы. На Python для этого часто используют библиотеку Flask.
- `@app.route("/адрес")` связывает адрес с функцией, которая на него отвечает.
- Шаблоны Jinja позволяют вставлять данные Python прямо в HTML через `{{ }}` и `{% %}`.
- Формы с методом POST и `request.form` позволяют принимать данные, которые пользователь ввёл в браузере.

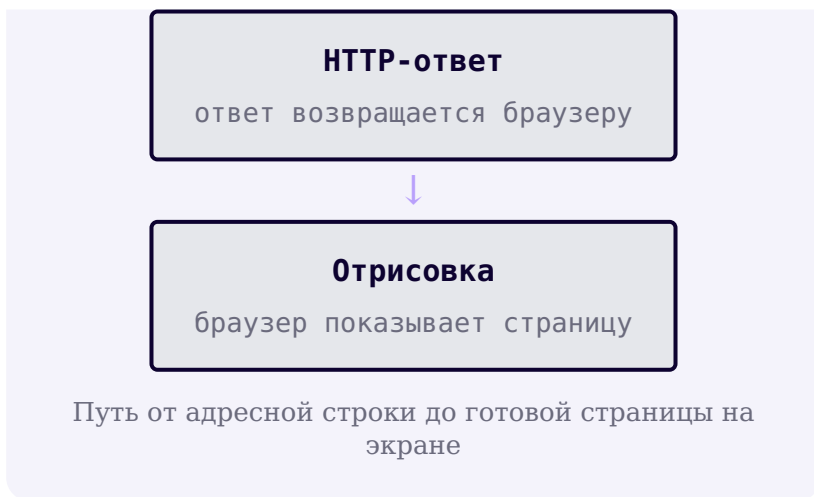
На этом заканчивается исторический печатный раздел главы — но не сама глава. Список задач пока хранится в обычном списке Python и исчезает при каждом перезапуске: страница полностью рабочая, но данные в ней не переживают перезапуск программы. Цифровое издание книги продолжает ту же тему дальше: как устроен путь запроса до сервера и обратно, как устроены HTTP и HTTPS, как связать Flask с базой данных и как проверить готовое приложение тестами — начиная с раздела 22.7 в боковом меню.

Как браузер получает веб-страницу

От нажатия Enter в адресной строке до готовой страницы на экране — путь одного запроса.

Раздел 22.1 показал общую картину: браузер отправляет запрос, сервер отвечает. Теперь разберём этот путь по шагам — что в точности происходит между нажатием Enter в адресной строке и появлением страницы на экране.





DNS: как доменное имя связывается с сетевыми адресами

Домен вроде `python.org` удобен человеку, но компьютеры в интернете находят друг друга по числовым **IP-адресам**. Служба **DNS** (Domain Name System — система доменных имён) хранит соответствия между доменами и записями, которые помогают найти нужный сервер, и превращает домен в IP-адрес прежде, чем браузер сможет отправить запрос.

Один домен — не обязательно один постоянный IP-адрес

У крупных сайтов один и тот же домен на практике может отвечать разными IP-адресами в разное время или в разных регионах — например, чтобы распределять нагрузку между несколькими серверами или ускорить доставку данных пользователям в разных частях света. Модель «один домен — ровно один сервер навсегда» верна только для самых простых случаев.

Соединение и запрос

Получив IP-адрес, браузер устанавливает соединение с сервером и отправляет **HTTP-запрос** — его устройство подробно рассматривается в разделе 22.9. Сервер (в нашем случае — приложение на Flask) обрабатывает запрос и отправляет **HTTP-ответ**: обычно HTML-страницу, но иногда — данные в формате JSON (раздел 22.15), изображение или файл.

Отрисовка

Получив ответ, браузер разбирает HTML и строит из него дерево DOM (раздел 22.4). По мере разбора он находит ссылки на CSS и JavaScript и запрашивает их отдельными HTTP-запросами (раздел 22.14); отображение страницы при этом может обновляться постепенно, по мере получения нужных данных, а не только после того, как загрузится буквально всё.

Мы намеренно не разбираем TCP/IP на уровне пакетов

У соединения между браузером и сервером есть более глубокие технические уровни — разбиение данных на пакеты, подтверждение доставки и так далее. Для веб-разработки на уровне этой главы достаточно мысленной модели «браузер посылает запрос — сервер отвечает»; сетевые протоколы этого уровня — тема отдельного курса.

URL: домен, путь, параметры и фрагмент

У каждого адреса в вебе есть чёткая структура — разберём её по частям на одном примере.

URL (Uniform Resource Locator — единый указатель ресурса) — это полный адрес чего-то в вебе: страницы, изображения, API-ответа. У него есть чёткая структура, и каждая часть отвечает за своё:

https://example.com:443/products/42?sort=price#details

СХЕМА
https

ХОСТ (ДОМЕН)
example.com

ПОРТ
443

ПУТЬ
/products/42

СТРОКА ЗАПРОСА
?sort=price

ФРАГМЕНТ
#details

Разбор одного URL по частям

Часть	Значение в приме- ре	За что отвечает
Схема	https	Каким протоколом пользоваться (раз- дел 22.10)
Хост (домен)	example.com	К какому серверу обращаться

Часть	Значение в примере	За что отвечает
Порт	443	Номер сетевого порта, к которому выполняется подключение — часто не пишется явно, потому что у схемы есть порт по умолчанию
Путь	/products/42	Какой ресурс на сервере нужен
Строка запроса	?sort=price	Дополнительные параметры в формате ключ=значение
Фрагмент	#details	Место внутри самой страницы

Фрагмент не уходит на сервер

Часть адреса после `#` обрабатывает сам браузер — обычно чтобы прокрутить страницу к элементу с таким идентификатором. Она не передаётся на сервер как часть HTTP-запроса: сервер, ответивший на `/products/42?sort=price#details`, физически не видит `#details` — он получает запрос только на `/products/42?sort=price`.

Строка запроса — несколько параметров

После `?` параметры перечисляются через `&`: `?sort=price&page=2` — два параметра, `sort` и `page`. Flask читает их через `request.args`:

```
query_params.py
```

```
@app.route("/tovary")
def tovary():
    sortirovka = request.args.get("sort", "name")
    return f"Сортировка: {{sortirovka}}"
```

Практика: разбор URL

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/22-08/index.html\)](https://www.cartesianschool.org/practice/22-08/index.html)

HTTP: запросы, ответы, методы и коды состояния

Общий набор правил, по которым браузер и сервер обмениваются сообщениями.

HTTP (HyperText Transfer Protocol) — прикладной протокол обмена сообщениями между клиентом и сервером в вебе: он задаёт общий набор правил, по которым браузер и сервер обмениваются запросами и ответами. Каждое сообщение — запрос или ответ — состоит из похожих частей.

HTTP-запрос	HTTP-ответ
Метод: что нужно сделать (GET, POST...)	Код состояния: как всё прошло (200, 404...)
Путь: какой ресурс нужен	Заголовки: метаданные ответа
Заголовки: метаданные запроса	Тело: содержимое — HTML, JSON, файл...
Тело: необязательные данные (например, форма)	

Основные методы HTTP

Метод	Обычный смысл
GET	Получить данные — открыть страницу, посмотреть список
POST	Отправить новые данные — добавить задачу, отправить форму
PUT	Полностью заменить существующий ресурс
PATCH	Частично изменить существующий ресурс
DELETE	Удалить ресурс

Это распространённые соглашения, а не физический закон: ни один из методов не заставляет сервер вести себя определённым образом — Flask выполнит любой код, какой вы напишете в обработчике, независимо от метода. Но следование этим соглашениям делает приложение понятным для других разработчиков и для инструментов вроде браузерного кеша. Не каждое приложение использует все пять методов — маленький сайт вполне может обойтись только GET и POST, как в разделе 22.5.

GET не «безопаснее» POST — и наоборот

GET и POST не отличаются по защите передаваемых данных: это определяет протокол (HTTP или HTTPS, раздел 22.10), а не метод. Разница в другом: по определению HTTP GET предназначен для получения данных и не должен вызывать побочные изменения на сервере — именно на этом основано поведение браузеров, кеширующих прокси и других инструментов, которые считают GET безопасным для повтора: обновление страницы или добавление в закладки не должно ничего менять. Поэтому удаление задачи через GET-ссылку — плохая идея (правильный вариант показывает раздел 22.35): случайное обновление страницы браузером не должно ничего удалять.

Коды состояния: как прошёл запрос

Диапазон	Смысл	Примеры
2xx	Успех	200 OK, 201 Created
3xx	Перенаправление	302 Found, 303 See Other — «ресурс нужно смотреть по другому адресу»
4xx	Ошибка клиента	400 Bad Request, 404 Not Found
5xx	Ошибка сервера	500 Internal Server Error

Раздел 22.13 показывает код 303 в деле — как часть паттерна POST-Redirect-GET, а раздел 22.31 — коды 400 и 404 в обработке ошибок ввода.

Тело сообщения — не только HTML

Тело HTTP-ответа может содержать HTML-страницу, но так же легко — данные в формате JSON (раздел 22.15), изображение, CSS-файл, JavaScript-файл или любой другой файл. Именно заголовок Content-Type сообщает браузеру, как понимать тело ответа — `text/html`, `application/json`, `image/png` и так далее.

Практика: классификация кодов состояния

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/22-09/index.html\)](https://www.cartesianschool.org/practice/22-09/index.html)

HTTPS: защита соединения с сайтом

HTTPS защищает канал между браузером и сервером — но не заменяет безопасность самого приложения.

HTTPS — использование протокола HTTP поверх TLS (Transport Layer Security): те же сообщения запрос-ответ, но соединение, по которому они передаются, защищено. Это важно, потому что обычный HTTP-запрос идёт по сети без транспортного шифрования: любой, кто технически способен перехватить трафик между браузером и сервером (например, в общей Wi-Fi сети), может его прочитать или незаметно изменить.

	HTTP	HTTPS
Данные в пути	Без шифрования	Зашифрованы
Можно ли перехватить и прочесть	Да, технически возможно	Практически нет — без ключа шифрования содержимое нечитаемо

	HTTP	HTTPS
Можно ли незаметно подменить данные в пути	Да	Нет — подмена нарушает проверку целостности, и соединение считается недействительным
Подтверждение личности сервера	Нет	Сертификат подтверждает, что сервер — тот, за кого себя выдаёт

TLS решает три задачи: **конфиденциальность** (данные в пути нечитаемы для посторонних), **целостность** (получатель может обнаружить, что данные были изменены в пути) и **подлинность сервера** (сертификат подтверждает, что вы говорите именно с тем сервером, а не с подменённым).

HTTPS защищает канал, а не само приложение

HTTPS не делает сайт «безопасным» в широком смысле. Он защищает данные *по пути* между браузером и сервером — но если в самом приложении есть ошибка (например, SQL-инъекция — раздел 22.32), HTTPS её никак не устраняет: данные доедут до сервера в целостности и там же будут неправильно обработаны. Это разные слои защиты, и один не заменяет другой.

Мы намеренно не разбираем математику шифрования — для веб-разработки на уровне этой главы достаточно знать, зачем HTTPS нужен и что именно он защищает. В разработке локально (раздел 22.34) чаще всего используют обычный HTTP на своём компьютере — шифровать канал внутри одной машины не требуется; HTTPS становится нужен, когда приложение реально развёрнуто и доступно через интернет.

Маршрутизация во Flask: URL и функции-обработчики

От HTTP-запроса до вызова нужной функции — устройство маршрутизации Flask.

Раздел 22.5 уже показал `@app.route("/")` — но что именно Flask делает с этим декоратором? Разберём весь путь одного запроса внутри приложения.



Каждый вызов `@app.route(...)` регистрирует **правило** (URL rule) — сам шаблон пути, например `/task/<int:task_id>`. Совокупность всех правил называют **маршрутизацией** (routing). У каждого правила есть также **эндпоинт** (endpoint) — символическое имя, под

которым Flask знает связанную функцию: по умолчанию это имя самой функции. Именно по эндпоинту, а не по URL напрямую, `url_for(...)` строит адрес — это и позволяет менять сам путь, не трогая код, который на него ссылается. Когда приходит запрос, Flask ищет среди зарегистрированных правил подходящее по пути и методу и вызывает связанную с ним **функцию-обработчик** — вы уже видели этот механизм в разделе 22.5, просто без названий.

Параметр пути — типизированная часть URL

Часть пути в угловых скобках Flask передаёт в функцию как аргумент. Можно указать её ожидаемый тип прямо в правиле:

path_param.py

```
@app.route("/task/<int:task_id>")
def poluchit_zadachu(task_id):
    return f"Задача №{task_id}"
```

<int:task_id> — нечисловой адрес отклоняется до вызова функции

Если открыть `/task/abc`, Flask не вызовет `poluchit_zadachu()` вообще — правило `<int:task_id>` требует целое число, и запрос получит стандартный ответ 404 (раздел 22.9) прежде, чем дойдёт до вашего кода.

Функция-обработчик возвращает содержимое ответа

То, что возвращает функция, становится телом HTTP-ответа: строка, отрендеренный шаблон (раздел 22.12) или, например, словарь — такой словарь Flask преобразует в JSON (раздел 22.15-22.16). Именно

поэтому обработчики называют *функциями представления* (view functions): их задача — подготовить представление данных для конкретного запроса.

Практика: маршруты и параметры пути

Модуль flask не установлен в браузерном окружении Pyodide — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/22-11/index.html\)](https://www.cartesianschool.org/practice/22-11/index.html)

Jinja: шаблоны HTML с данными

HTML-шаблон с выражениями Jinja и данными из Python — и встроенная защита от случайно выполненной разметки.

Раздел 22.5 уже использовал шаблоны — теперь разберём, как устроен движок **Jinja**, который Flask использует по умолчанию. Шаблон — обычный HTML-файл, куда вставлены два вида конструкций:

`{{ выражение }}` (вывести значение) и `{% команда %}` (условие, цикл, наследование).

templates/spisok.html

```
<ul>
  {% for zadacha in zadachi %}
    <li>{{ zadacha.title }}</li>
  {% endfor %}
</ul>

{% if not zadachi %}
  <p>Задач пока нет.</p>
{% endif %}
```

Автоэкранирование — Jinja защищает вас по умолчанию

Если значение, которое вставляет `{{ }}`, содержит HTML-разметку (например, пользователь ввёл в поле `<b>`), Jinja по умолчанию экранирует специальные символы — превращает `<` в `<` и так далее. Браузер покажет это как обычный текст, а не выполнит как разметку. Это называют **автоэкранированием** (autoescaping), и оно включено для HTML-шаблонов Flask по умолчанию.

|safe отключает именно эту защиту

У Jinja есть фильтр `|safe`, который отключает автоэкранирование для конкретного значения — Jinja вставит его как есть, включая любую разметку. Это уместно для текста, который вы полностью контролируете сами (например, HTML, зашитый в код приложения), но **не для данных, которые ввёл пользователь**: к чему это приводит на практике, показывает раздел 22.32. Если сомневаетесь — не добавляйте `|safe`.

Наследование шаблонов — общий каркас страницы

Чтобы не повторять `<head>`, навигацию и подключение CSS в каждом шаблоне, заводят базовый шаблон с именованными блоками, которые заполняют дочерние шаблоны:

```
templates/base.html
```

```
<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>{% block title %}Список задач{% endblock %}</
title>
  <link rel="stylesheet" href="{{ url_for('static',
filename='style.css') }}">
</head>
<body>
  {% block content %}{% endblock %}
</body>
</html>
```

```
templates/index.html
```

```
{% extends "base.html" %}

{% block content %}
  <h1>Мой список задач</h1>
  ...
{% endblock %}
```

`{% extends "base.html" %}` говорит: «возьми базовый шаблон и подставь моё содержимое в отмеченные блоки». Итоговый проект главы (раздел 22.35) использует именно эту структуру — `base.html` и несколько дочерних шаблонов.

Практика: шаблоны и автоэкранирование

Модуль flask не установлен в браузерном окружении Pyodide — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/22-12/index.html) → (https://www.cartesianschool.org/practice/22-12/index.html)

HTML-формы и отправка данных на сервер

От HTML-формы до сохранённых данных — и почему после сохранения нужен ещё один запрос.

Раздел 22.2 показал HTML-форму, раздел 22.5 — маршрут, который её принимает. Теперь разберём этот путь целиком, с одной важной деталью: что делать *после* того, как форма обработана.

templates/forma.html

```
<form action="{{ url_for('dobavit') }}" method="post">
  <label for="zadacha">Новая задача</label>
  <input type="text" id="zadacha" name="zadacha"
required>
  <button type="submit">Добавить</button>
</form>
```

`method="post"` говорит браузеру отправить данные формы как тело POST-запроса, а не как часть адреса. Атрибут `name="zadacha"` у поля определяет ключ, под которым значение придёт на сервер.

app.py

```
@app.route("/dobavit", methods=["POST"])
def dobavit():
    novaya_zadacha = request.form.get("zadacha",
    "").strip()
    if novaya_zadacha:
        zadachi.append(novaya_zadacha)
    return redirect(url_for("glavnaya"), code=303)
```

`request.form.get("zadacha", "")` достаёт значение поля по имени; второй аргумент — что вернуть, если поля вообще не было. Какие ещё проверки стоит добавить перед тем, как сохранять введённое значение — в разделе 22.31.

POST-Redirect-GET — зачем нужен redirect в конце

Обработчик формы заканчивается не отрисовкой страницы, а `redirect(url_for("glavnaya"), code=303)` — HTTP-ответом с кодом 303 See Other (раздел 22.9), который сообщает браузеру адрес, откуда нужно запросить результат методом GET. Браузер выполняет второй, уже безопасный запрос и показывает страницу.



POST-Redirect-GET: два отдельных запроса вместо одного

Без redirect повторная отправка дублирует данные

Если обработчик POST-запроса сразу возвращает готовую страницу (а не делает redirect), браузер связывает уже показанную страницу именно с этим POST-запросом — и обновление страницы (F5) отправит форму ещё раз, молча продублировав задачу. Redirect гарантирует, что после ответа сервера текущей операцией браузера окажется безопасный GET, который можно повторять сколько угодно.

Практика: форма и POST-Redirect-GET

Модуль flask не установлен в браузерном окружении Pyodide — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/22-13/index.html\)](https://www.cartesianschool.org/practice/22-13/index.html)

Статические файлы: CSS, изображения и JavaScript

Одна страница — несколько независимых запросов: HTML собирается заново, статика отдаётся как есть.

Открывая страницу со стилями, браузер на самом деле делает несколько отдельных HTTP-запросов: один за HTML, ещё по одному — за каждый CSS-файл, изображение и JavaScript-файл, на которые страница ссылается. Такие файлы называют **статическими**: сервер отдаёт их без изменений, в отличие от HTML, который Jinja рендерит заново для каждого запроса.



Flask по умолчанию ожидает статические файлы в папке `static/` рядом с приложением и отдаёт их по адресам вида `/static/имя_файла`:

```
структура_proekta.txt
```

```
todo-app/
  app.py
  templates/
    base.html
  static/
    style.css
```

Адрес статического файла удобнее не прописывать в шаблоне вручную, а получать через `url_for('static', filename='style.css')` — эта функция строит URL на основе конфигурации маршрутизации Flask, а не просто склеивает строку. Это заодно избавляет от лишней работы, если структура папок когда-нибудь изменится:

```
templates/base.html
```

```
<link rel="stylesheet" href="{{ url_for('static',
filename='style.css') }}">
```

templates/ и static/ — разные папки, разные роли

templates/ хранит файлы, которые Jinja рендерит заново под каждый запрос, подставляя в них данные из Python. static/ хранит готовые файлы, которые отдаются как есть, без обработки, — CSS, изображения, JavaScript, шрифты.

JSON: формат обмена данными

Распространённый текстовый формат для структурированных данных, который понимают программы на разных языках.

JSON (JavaScript Object Notation) — текстовый формат для передачи структурированных данных между программами. Название напоминает про JavaScript, но JSON давно используется программами на любых языках, включая Python, — это просто текстовый формат, а не код, который где-то выполняется.

Значение JSON	Соответствие в Python
объект <code>{{"ключ": значение}}</code>	<code>dict</code>
массив <code>[1, 2, 3]</code>	<code>list</code>
строка <code>"текст"</code>	<code>str</code>
число <code>42</code> / <code>3.14</code>	<code>int</code> / <code>float</code>
<code>true</code> / <code>false</code>	<code>True</code> / <code>False</code>
<code>null</code>	<code>None</code>

JSON — не то же самое, что запись словаря Python

У JSON строгий синтаксис: ключи и строковые значения обязаны быть в двойных кавычках.

`{{'name': 'Ada'}}` — это распечатка словаря Python, а не JSON: в корректном JSON она выглядит как `{{"name": "Ada"}}`. Одинарные кавычки, `True` с большой буквы, завершающая запятая перед закрывающей скобкой — всё это ломает JSON, даже если выглядит почти правильно.

Модуль json — стандартная библиотека Python

```
json_primer.py
```

```
import json

zadacha = {"title": "Купить хлеб", "done": False}

# Python -> текст JSON
tekst = json.dumps(zadacha, ensure_ascii=False)
print(tekst)  # {"title": "Купить хлеб", "done": false}

# текст JSON -> Python
obratno = json.loads(tekst)
print(obratno["title"])  # Купить хлеб
```

ensure_ascii=False — чтобы кириллица оставалась кириллицей

По умолчанию `json.dumps()` заменяет все не-ASCII символы на escape-последовательности вроде `\u041a` — валидный, но нечитаемый для человека JSON.

`ensure_ascii=False` оставляет кириллицу как есть; результат остаётся тем же самым JSON, просто удобным для чтения.

Ограничения формата JSON

У формата нет отдельного типа для даты и времени — их обычно передают строкой в оговорённом формате (например, "2026-08-23") и разбирают уже на своей стороне. Нет и типа для двоичных данных (изображение, файл) — их обычно кодируют в текст (например, Base64) или передают отдельным запросом. А ещё JSON не различает целые и вещественные числа так строго, как Python: очень большие или очень точные числа при передаче между разными языками программирования иногда требуют аккуратности.

JSON — формат данных, а не база данных и не API сам по себе

JSON описывает, *как записать* данные в виде текста. Он не хранит данные постоянно сам по себе (для этого нужна база данных — раздел 22.20) и не является программным интерфейсом сам по себе (для этого нужен код, который его обрабатывает — пример такого кода есть в разделе 22.16). JSON — распространённый текстовый формат обмена данными между разными частями системы.

Практика: преобразование данных между Python и JSON

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/22-15/index.html\)](https://www.cartesianschool.org/practice/22-15/index.html)

HTTP API: обмен данными между программами

Тот же сервер, тот же Flask — но получатель ответа теперь другая программа, а не браузер человека.

До сих пор Flask-приложение отвечало HTML-страницами — их читает человек через браузер. Но у ответа сервера может быть и другой получатель: мобильное

приложение, другой сервер или JavaScript-код на той же странице (раздел 22.4 уже показал `fetch()`). Такой ответ обычно приходит не в виде HTML, а в виде данных — чаще всего JSON.

Программный интерфейс, через который одна программа получает данные от другой, называют **API** (Application Programming Interface — программный интерфейс приложения). Конкретный адрес внутри API тоже называют **эндпоинтом** — это более широкое, общеупотребительное значение слова, чем эндпоинт-имя маршрута Flask из раздела 22.11: там речь шла о внутреннем идентификаторе для `url_for()` , здесь — просто о конкретном URL, который отдаёт данные, а не HTML-страницу.

	Обычный маршрут	API-эндпоинт
Кто читает ответ	Человек через браузер	Программа: JavaScript, приложение, другой сервер
Формат ответа	HTML	Чаще всего JSON
Content-Type ответа	text/html	application/json

Маленький JSON-эндпоинт на Flask

```
app.py

from flask import jsonify

@app.route("/api/tasks")
def api_tasks():
    return jsonify([{"id": z["id"], "title":
z["title"], "done": z["done"]} for z in zadachi])
```

`jsonify(...)` делает две вещи сразу: превращает данные Python в текст JSON (как `json.dumps()` из раздела 22.15) и выставляет заголовок ответа `Content-Type: application/json`, чтобы получатель знал, как читать тело ответа.

Ответ `/api/tasks` в браузере: массив JSON-объектов с полями `done`, `id` и `title`, кириллица в `title` показана как экранированные последовательности вида `\u0418`

Реальный ответ GET `/api/tasks` итогового проекта. Это валидный JSON — но по умолчанию `jsonify()` экранирует кириллицу в `\uXXXX`-последовательности, ровно как описывал раздел 22.15 про `json.dumps()` без `ensure_ascii=False`.

REST — распространённый стиль, а не обязательный стандарт

Многие API строят в стиле **REST** (Representational State Transfer): путь называет ресурс (`/api/tasks`), а метод определяет действие над ним — GET получает список, POST создаёт новую запись. Это популярное и полезное соглашение, но не единственно возможный способ построить API — важно понимать сам принцип «данные вместо HTML», а не заучивать REST как жёсткое правило.

Раздел 22.35 добавит такой эндпоинт в итоговый проект — GET `/api/tasks`, возвращающий текущий список задач в формате JSON.

Практика: форматируем данные для API-ответа

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/22-16/index.html\)](https://www.cartesianschool.org/practice/22-16/index.html)

Веб-фреймворки на Python: Flask, Django, FastAPI и другие

Веб-фреймворк — набор готовых средств для создания веб-приложений. У Flask, Django, FastAPI и Starlette разный набор встроенных возможностей.

Веб-фреймворк — набор библиотек, правил и готовых компонентов, которые упрощают создание веб-приложений: маршрутизацию, обработку HTTP-запросов и ответов, работу с шаблонами, проверку данных, доступ к базе данных и другие типичные задачи. Flask — не единственный веб-фреймворк для Python; у остальных другой набор встроенных возможностей.

Flask

Flask — лёгкий WSGI-фреймворк для создания веб-приложений на Python (раздел 22.19). Его основное ядро сравнительно невелико, а дополнительные возможности обычно подключаются библиотеками и расширениями.

Django

Django — полнофункциональный веб-фреймворк на Python. В его состав входят ORM, система миграций, маршрутизация, шаблоны, средства аутентификации и административный интерфейс.

FastAPI

FastAPI — веб-фреймворк для создания HTTP API на Python. Он использует аннотации типов Python для описания и проверки данных и может автоматически формировать документацию API на основе OpenAPI.

Starlette

Starlette — лёгкий ASGI-фреймворк и инструментарий для создания асинхронных веб-сервисов на Python. FastAPI использует Starlette как один из своих базовых компонентов.

Каждый фреймворк берёт на себя часть типовой работы: маршрутизацию (кто отвечает за какой адрес — раздел 22.11), обработку запроса и ответа, иногда — шаблоны, проверку входных данных, работу с базой данных, авторизацию, инструменты для тестирования. Разница между фреймворками — в том, *сколько именно* он берёт на себя и насколько легко заменить любую из этих частей на свою.

Другие веб-фреймворки для Python

Кроме Flask, Django, FastAPI и Starlette, в веб-разработке на Python используются и другие проекты. Ни один из них не рассматривается в этой главе подробно — они упомянуты, чтобы вы узнавали названия, встретив их в реальных проектах.

Фреймворк	Основной фокус	Интерфейс/модель	Типичное применение
Litestar	Производительность и строгая типизация	ASGI	API-сервисы, где важны типы и валидация
Quart	API, совместимый с Flask	ASGI	Перенос Flask-приложений на асинхронную модель

Фреймворк	Основной фокус	Интерфейс/модель	Типичное применение
Pyramid	Гибкая архитектура без навязанной структуры	WSGI	Проекты, растущие от небольшого скрипта до крупного приложения
Tornado	Асинхронный ввод-вывод, долгоживущие соединения	Собственный асинхронный движок + ASGI/WSGI-совместимость	Веб-сокеты, длительные соединения
Bottle	Минимализм, один файл	WSGI	Совсем маленькие сервисы и прото-типы

Не выбирайте фреймворк по рейтингам популярности

Бенчмарки производительности и списки «самых быстрых фреймворков» сильно зависят от того, что именно измеряется, и быстро устаревают. Разумнее спрашивать: что должно делать именно моё приложение, какой опыт уже есть у команды, что здесь уже используется. Более предметное сравнение по конкретным задачам, а не по абстрактному «что лучше», — в разделе 22.18.

Официальная документация: [Flask](#), [Django](#), [FastAPI](#), [Starlette](#).

Сравнение Flask, Django и FastAPI

Flask, Django и FastAPI решают разные классы задач и предоставляют разный набор встроенных возможностей.

Раздел 22.17 показал общую карту. Теперь сравним три самых заметных фреймворка — Flask, Django и FastAPI — по конкретным задачам, а не по общим впечатлениям.

Критерий	Flask	Django	FastAPI
Назначение	Общего назначения, часто сайты с HTML-страницами	Общего назначения, приложения вокруг базы данных	Прежде всего HTTP API
Маршрутизация	Встроенная	Встроенная	Встроенная
Шаблоны HTML	Jinja	Встроенный движок шаблонов	Не основная задача — фреймворк ориентирован на API
ORM	Нет своего — через сторонний ORM (раздел 22.26)	Да, встроенный	Нет своего — через сторонний ORM
Миграции	Нет своих — через сторонний инструмент	Да, встроенные (makemigrations/migrate)	Нет своих — через сторонний инструмент
Административный интерфейс	Нет своего	Да, встроенный автоматический	Нет своего
Аутентификация	Нет своей — через расширения	Да, встроенная система пользователей	Нет своей — через сторонние библиотеки

Критерий	Flask	Django	FastAPI
Проверка данных	Вручную или через расширения	Через формы и сериализаторы Django/DRF	Встроенная, на основе подсказок типов Python
OpenAPI	Нет своей	Нет своей в ядре	Да, встроенная, автоматическая
WSGI / ASGI (раздел 22.19)	WSGI, с ограниченной поддержкой асинх-обработчиков	Поддерживает и WSGI, и ASGI режимы развёртывания	ASGI изначально
Типичные проекты	Небольшие и средние сайты, простые API	Крупные приложения с базой данных и админкой	HTTP API, сервисы с типизированными данными

Django REST Framework — отдельный проект, а не часть ядра Django

Django из коробки умеет отдавать HTML-страницы и имеет админ-панель, но полноценный инструмент для REST API — это отдельный пакет, Django REST Framework (DRF), который устанавливают и подключают самостоятельно. Он не входит в состав самого Django и не устанавливается вместе с ним автоматически.

Отправная точка для выбора

Нужен сайт с формами,
шаблонами, без готовой
админки

→ **Flask**

Нужна бизнес-система с базой
данных и админ-панелью «из
коробки»

→ **Django**

Нужен API-сервис с
автоматической
документацией

→ **FastAPI**

Нужен низкоуровневый
контроль над ASGI-
приложением

→ **Starlette**

Реальные проекты часто смешивают инструменты
— это стартовые ориентиры, а не жёсткие правила

Почему для учебного проекта выбран Flask

В этой главе первый серверный проект построен на Flask. Flask предоставляет основные средства веб-приложения — маршрутизацию, обработку запросов и ответов и работу с шаблонами — без большого набора обязательных подсистем. Поэтому на небольшом учебном проекте удобно последовательно рассмотреть каждый из этих механизмов.

Django предоставляет больше готовых компонентов, например ORM, миграции, аутентификацию и административный интерфейс. FastAPI ориентирован прежде всего на создание HTTP API и тесно использует аннотации типов Python.

Flask выбран здесь как подходящий инструмент для первого учебного веб-приложения. Это не означает, что Flask является лучшим решением для любого проекта.

Практика: выбор фреймворка по задаче

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/22-18/index.html\)](https://www.cartesianschool.org/practice/22-18/index.html)

WSGI и ASGI: связь веб-приложения с сервером

Фреймворк описывает, что делать с запросом. WSGI и ASGI — соглашение о том, как запрос до него доходит.

Раздел 22.18 упомянул WSGI и ASGI как техническую разницу между фреймворками. Разберём, что они означают на уровне идеи, без деталей реализации.

WSGI (Web Server Gateway Interface) и **ASGI** (Asynchronous Server Gateway Interface) — стандартные интерфейсы, описывающие, как сервер приложений передаёт HTTP-запрос Python-приложению и получает от него ответ. В обычном развёртывании сетевые соединения принимает сервер приложений или другая серверная инфраструктура, а не сам фреймворк (Flask, Django, FastAPI) напрямую — WSGI/ASGI и есть то общее соглашение между ними.

	WSGI	ASGI
Полное название	Web Server Gateway Interface	Asynchronous Server Gateway Interface

	WSGI	ASGI
Модель обработки запроса	Синхронная — один запрос обрабатывается целиком, прежде чем начать следующий в этом потоке	Поддерживает асинхронную обработку — приложение может ждать медленную операцию, не блокируя весь процесс
Долгоживущие соединения (WebSocket)	Не рассчитан на такой сценарий	Поддерживается совместимыми фреймворками и серверами
Кто использует	Flask	Starlette, FastAPI

Django поддерживает WSGI и ASGI

Современный Django поддерживает оба режима развёртывания — традиционный WSGI и ASGI — в зависимости от того, как настроен проект. Это не значит, что каждая часть Django одинаково хорошо работает асинхронно, но сам фреймворк не привязан жёстко только к одному протоколу.

Flask и async — не то же самое, что ASGI-фреймворк

Flask умеет вызывать `async def`-обработчики, но само приложение при этом остаётся WSGI-приложением по архитектуре: для выполнения такого обработчика Flask запускает событийный цикл и выполняет в нём корутину, занимая при этом один обработчик запроса на всё время её работы. Это удобно, когда конкретному обработчику нужно ждать медленную операцию ввода-вывода, но не превращает Flask в полноценный ASGI-фреймворк вроде FastAPI — если приложению принципиально важна асинхронная модель на всех уровнях, ASGI-фреймворк подойдёт лучше.

Фреймворк и сервер приложений — разные вещи

Сам Flask или FastAPI описывает *что* делать с запросом. Отдельная программа — **сервер приложений** — отвечает за то, чтобы реально принять соединение по сети и передать запрос фреймворку по протоколу WSGI или ASGI:

Протокол	Примеры серверов приложений
WSGI	Gunicorn, Waitress
ASGI	Uvicorn, Hypercorn

К этому различию мы вернёмся в разделе 22.34 — сервер разработки, встроенный во Flask, и сервер приложений для рабочего окружения решают одну и ту же задачу по-разному.

Практика: WSGI или ASGI?

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/22-19/index.html) (<https://www.cartesianschool.org/practice/22-19/index.html>)

Зачем веб-приложению база данных

Список Python в памяти хранит данные только до перезапуска процесса — а рано или поздно процесс перезапускается.

Список задач в разделе 22.5 хранится вот так:


```
app.py
```

```
zadachi = ["Выучить основы Python", "Собрать сайт на Flask"]
```

Это обычная переменная Python — список, который живёт в оперативной памяти, пока работает процесс приложения. У такого хранения есть ограничения, которые незаметны на маленьком примере, но становятся серьёзной проблемой по мере роста приложения и нагрузки на него.

Проблема	Что происходит
Перезапуск	Остановили процесс Python (обновили код, перезагрузили сервер) — список исчез вместе с ним
Несколько процессов	Рабочее развёртывание (раздел 22.34) часто запускает несколько процессов приложения одновременно — у каждого своя копия памяти, свой список, они не видят изменений друг друга
Запрос данных	Список умеет перебор и фильтрацию средствами Python, но не умеет быстро находить нужные записи среди миллионов без полного перебора
Согласованность при одновременных изменениях	Два запроса, пришедшие почти одновременно, могут непредсказуемо испортить список, если оба меняют его в один и тот же момент

СУБД (система управления базами данных) — программная система, создающая, хранящая, обрабатывающая запросы и управляющая базами данных; **база данных** — это организованные хранимые данные, с которыми работает СУБД. СУБД предоставляет механизмы постоянного хранения, запросов, транзакций, ограничений и согласованной работы с данными — она решает именно эти проблемы. Конкретные гарантии при этом зависят от выбранной СУБД, спроектированной схемы, использования транзакций и кода самого приложения: СУБД не устраняет проблему автоматически сама по себе, если ей не пользоваться правильно.

Персистентность — данные, которые переживают перезапуск программы

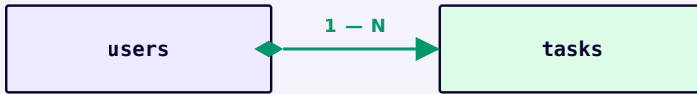
Свойство хранить данные так, что они не исчезают после завершения процесса, называют **сохранением данных между запусками**, или *persistence*. Список Python в памяти этим свойством не обладает; файл на диске или база данных — обладают.

Разделы 22.21-22.28 разбирают, как устроены базы данных и на каком языке с ними работают, а раздел 22.29 заменяет список Python в проекте главы на базу данных SQLite.

Реляционные базы данных: таблицы, ключи и связи

Данные в строках и столбцах — и связи между таблицами через первичные и внешние ключи.

Самый распространённый вид базы данных — **реляционная**: данные хранятся в **таблицах**, похожих на лист расчётной таблицы. Каждая **строка** — одна запись, каждый **столбец** — одно её свойство с заранее заданным типом.



Одному пользователю `users` может соответствовать несколько строк `tasks`

id	title	done
1	Выучить основы Python	1
2	Собрать сайт на Flask	0

Такая таблица — `tasks` : у каждой строки есть `id` — уникальный номер, по которому её можно однозначно найти. Столбец, который делает это (обычно `id`), называют **первичным ключом** (primary key).

Связь между таблицами — внешний ключ

Если бы у каждой задачи был владелец, вторая таблица `users` хранила бы пользователей, а таблица `tasks` — ссылку на строку в ней:

id	title	done	user_id
1	Выучить основы Python	1	7

id	title	done	user_id
2	Собрать сайт на Flask	0	7

Столбец `user_id`, который ссылается на `id` в другой таблице, называют **внешним ключом** (foreign key).

Откуда взялось слово «реляционная»

Название происходит от математического понятия relation (отношение), лежащего в основе реляционной модели данных: отношение — это множество кортежей одинаковой структуры, а на практике его представляют в виде таблицы. Связи между таблицами через внешние ключи — важная часть проектирования баз данных, но не источник названия модели: одна таблица без единого внешнего ключа всё равно остаётся реляционной.

Итоговый проект остаётся без пользователей — и это осознанно

Как раздел 22.31 объясняет подробнее: чтобы не расширять эту главу до полноценной аутентификации, итоговый проект (раздел 22.35) использует только одну таблицу — `tasks`, без `users` и без внешних ключей. Связи между таблицами — важная идея, но не обязательный элемент самого маленького работающего приложения.

SQL: создаём, читаем и изменяем данные

Один язык — и создание таблиц, и чтение, и изменение, и удаление данных.

SQL (Structured Query Language — язык структурированных запросов) — язык определения структуры данных и выполнения запросов к реляционной базе данных: им создают таблицы, читают, добавляют, изменяют и удаляют строки. У SQL есть общий стандарт,

но у каждой конкретной системы (SQLite, PostgreSQL, MySQL) — свои расширения и особенности поверх него, поэтому говорят не о «разных языках SQL», а о разных **диалектах** одного языка. Примеры на этой странице выполняются в SQLite (раздел 22.23).

CREATE TABLE — описываем структуру

sozдание_tablicy.sql

```
CREATE TABLE tasks (  
    id INTEGER PRIMARY KEY,  
    title TEXT NOT NULL,  
    done INTEGER NOT NULL DEFAULT 0  
);
```

NOT NULL запрещает именно значение NULL, а не пустую строку

NOT NULL запрещает записывать в столбец специальное SQL-значение NULL («значение неизвестно / отсутствует»). Пустая строка '' — это по-прежнему допустимое значение типа TEXT, и NOT NULL её не отклонит. Если приложению нужно запретить именно пустой (или состоящий из пробелов) заголовок задачи, для этого нужна отдельная проверка — на стороне приложения (раздел 22.31) или явное ограничение CHECK, как показывает раздел 22.29.

DEFAULT 0 задаёт значение по умолчанию, если его не указали явно.

Четыре операции CRUD

Действие	SQL	Что делает
Create	INSERT	Добавить новую строку

Действие	SQL	Что делает
Read	SELECT	Прочитать одну или несколько строк
Update	UPDATE	Изменить существующую строку
Delete	DELETE	Удалить строку

```
insert.sql
```

```
INSERT INTO tasks (title) VALUES ('Купить хлеб');
```

```
select.sql
```

```
SELECT id, title, done FROM tasks WHERE done = 0 ORDER BY id LIMIT 10;
```

`WHERE` оставляет только подходящие строки, `ORDER BY` задаёт порядок, `LIMIT` ограничивает количество результатов.

```
update.sql
```

```
UPDATE tasks SET done = 1 WHERE id = 1;
```

```
delete.sql
```

```
DELETE FROM tasks WHERE id = 1;
```

SQL из Python: модуль sqlite3

sql_iz_python.py

```
import sqlite3

baza = sqlite3.connect("zadachi.db")
baza.execute(
    "INSERT INTO tasks (title) VALUES (?)",
    ("Купить хлеб",),
)
baza.commit()

stroki = baza.execute("SELECT id, title, done FROM
tasks").fetchall()
for stroka in stroki:
    print(stroka)
```

Знак вопроса вместо f-строки — не стилистический выбор

? в SQL-строке — это **параметризованный запрос**: значение подставляет сам модуль `sqlite3`, безопасно, отдельно от текста запроса. Если вместо этого собрать запрос через f-строку с пользовательским вводом внутри, это открывает путь к SQL-инъекции — что именно может пойти не так и почему параметризация обязательна, а не желательна, показывает раздел 22.32.

Практика: CRUD на SQLite

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/22-22/index.html) → (https://www.cartesianschool.org/practice/22-22/index.html)

SQLite: встраиваемая реляционная СУБД

SQLite — реляционная СУБД, работающая как библиотека внутри процесса приложения, без отдельного сервера.

СУБД — система управления базами данных: программное обеспечение, которое создаёт, хранит, обрабатывает запросы и управляет базами данных.

SQLite — встраиваемая реляционная СУБД, реализованная в виде программной библиотеки. В отличие от PostgreSQL или MySQL, SQLite не требует отдельного процесса сервера базы данных: приложение обращается к библиотеке SQLite непосредственно, внутри собственного процесса.

При обычном файловом использовании содержимое базы SQLite хранится в одном основном файле. Во время работы SQLite может создавать дополнительные служебные файлы, например `rollback journal` или `WAL`. SQLite также поддерживает базу данных в оперативной памяти (`:memory:`), которая не сохраняется на диск вовсе — это удобно, например, для тестов (раздел 22.33).

Для работы с SQLite в Python используется модуль `sqlite3`, предоставляющий интерфейс DB-API 2.0. В обычных сборках CPython он доступен без установки отдельного пакета через `pip`, однако технически `sqlite3` относится к опциональным модулям конкретной сборки интерпретатора и использует библиотеку SQLite, установленную в системе или собранную вместе с Python.

	Обычная клиент-серверная СУБД	SQLite
Отдельный процесс-сервер	Да, работает постоянно	Нет — работает внутри процесса приложения

	Обычная клиент-серверная СУБД	SQLite
Основное хранение	Управляется сервером базы данных	Основной файл базы данных (возможны служебные файлы journal/WAL)
Настройка для старта	Установить и запустить сервер, настроить доступ	Не требуется отдельная настройка сервера
Несколько серверов приложения пишут одновременно	Штатный сценарий	Ограниченная поддержка одновременной записи

Где SQLite подходит особенно хорошо

SQLite — полноценный SQL-движок, которым пользуются и очень крупные приложения — как основное хранилище локальных данных или как часть более крупной системы. SQLite хорошо подходит для локальных приложений, обучения, прототипов, тестов (раздел 22.33) и многих небольших и средних развёртываний, где модели одновременного доступа SQLite достаточно. Клиент-серверную базу выбирают не потому, что SQLite чем-то хуже как движок, а когда нескольким серверам приложения нужно надёжно писать в одну базу параллельно — эту разницу подробнее рассматривает раздел 22.24.

Подключение и курсор

podklyuchenie.py

```
import sqlite3

baza = sqlite3.connect("zadachi.db")
baza.row_factory = sqlite3.Row # доступ к столбцам
по имени

kursor = baza.execute("SELECT id, title FROM tasks")
for stroka in kursor:
    print(stroka["id"], stroka["title"])

baza.close()
```

`sqlite3.connect(...)` открывает (или создаёт, если файла ещё нет) файл базы данных. `row_factory = sqlite3.Row` позволяет обращаться к столбцам результата по имени, а не только по числовому индексу — удобнее и надёжнее, особенно если порядок столбцов в запросе когда-нибудь изменится.

Отдельное подключение на каждый поток

По умолчанию одно соединение `sqlite3.connect(...)` нельзя без осторожности использовать сразу из нескольких потоков. Практичный способ организовать подключения во Flask-приложении — открывать соединение под каждый запрос и закрывать его сразу после — показывает раздел 22.29.

Официальная документация: [SQLite, Python — модуль sqlite3](#).

Практика: подключение к SQLite через sqlite3

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/22-23/index.html) → (<https://www.cartesianschool.org/practice/22-23/index.html>)

PostgreSQL, MySQL/MariaDB и SQLite: основные различия

Эти СУБД различаются архитектурой, возможностями и требованиями к эксплуатации — выбор зависит от требований проекта.

SQLite — не единственная реляционная СУБД. Разберём, чем от неё и друг от друга отличаются два широко используемых клиент-серверных варианта.

PostgreSQL

PostgreSQL — объектно-реляционная СУБД с клиент-серверной архитектурой: приложение подключается к отдельно запущенному серверу PostgreSQL по сети. Известна широким набором встроенных типов данных и расширяемостью.

MySQL

MySQL — реляционная клиент-серверная СУБД, одна из самых распространённых в веб-разработке.

MariaDB

MariaDB — отдельная реляционная СУБД, исторически возникшая как форк MySQL и во многом остающаяся с ним совместимой.

SQLite

SQLite — встраиваемая реляционная СУБД: она работает как библиотека внутри процесса приложения, без отдельного сервера (раздел 22.23).

MySQL и MariaDB — не взаимозаменяемые синонимы

MariaDB возникла как форк MySQL и во многом остаётся с ним совместимой, но это два отдельных проекта с собственной историей разработки — они не идентичны и не всегда развиваются синхронно. В документации и в вакансиях их иногда упоминают вместе («MySQL/MariaDB»), но для конкретного проекта стоит уточнять, какая именно из двух систем используется.

Выбор между PostgreSQL, MySQL/MariaDB и SQLite почти никогда не сводится к «какая из них лучше в целом» — обычно это вопрос требований конкретного проекта: сколько серверов приложения будут писать в базу одновременно, какие возможности SQL нужны, что уже используется в команде и в её опыте, как будет устроено развёртывание. Ни одно из этих решений не является универсально правильным по умолчанию.

Официальная документация: [PostgreSQL](#), [MySQL](#), [MariaDB](#), [SQLite](#).

NoSQL: основные виды нереляционных баз данных

NoSQL объединяет несколько нереляционных моделей хранения: ключ-значение, документы, ширококолоночные хранилища и графы.

NoSQL — общее название для нереляционных систем управления базами данных, использующих модели данных, отличные от классической реляционной модели таблиц.

NoSQL — «не только SQL», а не «никогда не SQL»

Термин NoSQL часто трактуют как Not Only SQL — «не только SQL». Это не название одного языка и не единый стандарт: под NoSQL объединяют несколько разных семейств СУБД. Многие нереляционные системы всё равно поддерживают SQL-подобные или собственные языки запросов — название описывает отход от классической реляционной модели таблиц, а не полный отказ от идеи структурированных запросов.

Реляционные базы (разделы 22.21-22.24) хранят данные в таблицах с заранее заданной структурой. Иногда такая структура неудобна — например, если форма записей сильно отличается от документа к документу, нужна очень быстрая работа по ключу, или данные естественно представляются как узлы и связи. Ниже — четыре основных семейства NoSQL-СУБД, у каждого своя модель данных.

Ключ — значение

Каждая запись доступна по уникальному ключу. Значением может быть строка, число, структура данных или другой объект, в зависимости от СУБД.

Пример: Redis — сервер структур данных, в котором каждый объект имеет ключ; Redis поддерживает строки, списки, множества, хеши, потоки и другие структуры данных.

```
primer_redis.txt
```

```
SET session:42 '{"user_id": 42, "expires":  
"2026-08-24"}'  
GET session:42
```

Типичное применение: кеш, счётчики, хранение сессий/состояния, очереди и потоки данных — часто как дополнение к основной базе, а не её замена.

Документоориентированные базы данных

Данные хранятся в документах, состоящих из полей и значений. Документы могут содержать вложенные структуры.

Пример: MongoDB хранит документы в формате BSON, который по структуре близок к JSON.

```
primer_dokument.json
```

```
{{
  "name": "Механическая клавиатура",
  "price": 8900,
  "characteristics": {"switches": "red", "layout":
"ru"}}
}}
```

Удобны, когда форма записей может меняться от документа к документу.

Ширококолоночные базы данных

Данные организуются вокруг ключей разделов (partition keys) и строк с набором столбцов. Такая модель рассчитана на распределённое хранение больших объёмов данных и заранее продуманные шаблоны запросов.

Пример: Apache Cassandra — распределённая ширококолоночная СУБД, доступная через собственный язык запросов CQL.

```
primer_wide_column.txt
```

```
Партиция: device_id = 42
Строки внутри партиции упорядочены по date
```

Типичное применение: события и метрики с высокой скоростью записи, распределённые по устройству, пользователю или другому ключу раздела.

Графовые базы данных

Данные представлены узлами, связями между ними и свойствами. Такая модель удобна, когда сами связи являются важной частью задачи.

Пример: Neo4j — графовая СУБД, где запросы явно оперируют узлами и связями.

```
primer_graf.txt
```

```
(Ада) - [:ДРУГ] -> (Борис) - [:ДРУГ] -> (Вера)
```

Типичное применение: социальные связи, рекомендации, сети зависимостей, анализ связей при выявлении мошенничества.

Специализированные системы хранения и поиска

Отдельно стоят системы полнотекстового поиска, векторные и временные базы данных — их не относят к четырём каноническим семействам выше. Эти категории могут пересекаться друг с другом и с уже перечисленными моделями, а некоторые продукты сочетают несколько моделей в одной СУБД (multi-model).

Общие особенности NoSQL-систем

У многих NoSQL-СУБД схема данных более гибкая, чем в реляционной модели, — это не значит, что схема отсутствует полностью. Многие NoSQL-СУБД проектируются с расчётом на распределённую работу и горизонтальное масштабирование, но конкретные гарантии (согласованность, поддержка транзакций, устойчивость к сбоям) зависят от продукта и его

настройки. Производительность зависит от модели данных, конкретной СУБД и типа запросов — не существует правила «NoSQL всегда быстрее».

Критерий	Реляционная СУБД	NoSQL-системы
Модель данных	Таблицы, строки, столбцы	Зависит от типа: документы, ключ-значение, wide-column, граф
Схема	Обычно определяется явно	Часто более гибкая; зависит от конкретной СУБД
Связи	Внешние ключи, JOIN и другие реляционные операции	Реализуются по-разному: ссылки, вложенные документы, графовые связи и другое
Язык запросов	SQL и его диалекты	Собственные языки запросов, команды или API; некоторые поддерживают SQL-подобные интерфейсы
Транзакции	Зависят от СУБД, обычно развитая поддержка транзакций	Зависят от конкретной СУБД; многие современные системы также поддерживают транзакции
Масштабирование	Возможны вертикальные и распределённые архитектуры	Многие системы изначально проектируются для распределённой работы

Критерий	Реляционная СУБД	NoSQL-системы
Когда выбирать	Когда реляционная модель хорошо соответствует данным и запросам	Когда конкретная нереляционная модель лучше соответствует данным и нагрузке

Что выбрать

Реляционную СУБД стоит рассматривать, когда данные имеют естественную реляционную структуру, важны связи и ограничения, запросы хорошо ложатся на SQL, а транзакционная целостность важна для задачи. Конкретную NoSQL-СУБД стоит рассматривать, когда одна из моделей — документная, ключ-значение, ширококолоночная или графовая — естественно соответствует задаче, шаблон доступа к данным хорошо понятен заранее, а требования к распределению и масштабированию склоняют выбор в сторону конкретной нереляционной архитектуры.

Для простого приложения достаточно одной подходящей СУБД

Крупные реальные системы нередко используют не одну технологию хранения, но учебный проект стоит начинать с самой простой подходящей. Для обычного CRUD-приложения вроде списка задач не нужны одновременно PostgreSQL, MongoDB, Redis и векторная база — это добавит сложности эксплуатации без реальной выгоды. Итоговый проект этой главы (раздел 22.35) обходится одной таблицей в SQLite.

Официальная документация: [MongoDB](#), [Redis](#), [Apache Cassandra](#), [Neo4j](#).

ORM: работа с базой данных через объекты Python

Строки таблицы как объекты Python — удобно, но ORM всё равно формирует и выполняет обычный SQL.

Раздел 22.22 писал SQL-запросы текстом. **ORM** (Object-Relational Mapper — объектно-реляционное отображение) — это библиотека, которая позволяет работать со строками таблицы как с обычными объектами Python, а не собирать SQL-текст вручную. У разных ORM разный API — вот один и тот же запрос и одна и та же вставка в двух самых заметных ORM экосистемы Python:

Действие	SQL напрямую	Django ORM	SQLAlchemy 2.x
Прочитать невыполненные задачи	<code>SELECT id, title FROM tasks WHERE done = 0</code>	<code>Task.objects.filter(done=False)</code>	<code>select(Task).where(Task.done.is_(False))</code>
Добавить новую задачу	<code>INSERT INTO tasks (title) VALUES (?)</code>	<code>Task.objects.create(title="Купить хлеб")</code>	<code>session.add(Task(title="Купить хлеб"))</code>

SQLAlchemy (описывает себя как «инструментарий SQL и объектно-реляционный преобразователь для Python») используется вместе с разными фреймворками, включая Flask, и распространяется отдельным пакетом. Встроенный **ORM Django** поставляется вместе с самим фреймворком и тесно с ним интегрирован — это разные библиотеки с разным API, а не два имени одного и того же инструмента.

ORM не отменяет необходимость понимать SQL

ORM удобно избавляет от рутины — не нужно вручную собирать текст запроса для каждой операции. Но он не убирает саму реляционную модель: ORM всё равно формирует и выполняет SQL-запросы на основе операций с объектами, и понимание того, как устроены таблицы, связи и запросы (разделы 22.21-22.22), напрямую влияет на то, насколько эффективно и правильно вы используете ORM — особенно когда запрос через ORM работает не так быстро, как ожидалось, и нужно понять, какой именно SQL он выполнил.

Почему в итоговом проекте используется `sqlite3`

Для проекта этой главы (раздел 22.29) выбран прямой модуль `sqlite3`, а не ORM: цель этой главы — понять, что именно происходит с данными на каждом шаге, а не спрятать это за слоем абстракции ещё до того, как эта абстракция станет понятна. ORM — полезный и распространённый инструмент для реальных проектов, но здесь он отвлёл бы от самой идеи главы.

Практика: SQL и его отображение в ORM

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/22-26/index.html\)](https://www.cartesianschool.org/practice/22-26/index.html)

Миграции схемы базы данных

Структура таблицы меняется вместе с приложением — миграция записывает это изменение предсказуемо.

Слово «миграция» в веб-разработке означает две разные вещи, и их легко перепутать. Эта страница — про первую: **миграцию схемы** — изменение структуры базы данных со временем. Вторая — перенос самих данных между базами — рассматривается в разделе 22.28.

Приложение растёт, и структура таблицы, спроектированная в начале, перестаёт хватать. Например, у задачи появляется отметка «выполнено»:

Версия 1	Версия 2
<code>tasks(id, title)</code>	<code>tasks(id, title, done)</code>

Миграция схемы — это записанное изменение структуры, которое можно применить к базе данных так же предсказуемо на любом компьютере: у разработчика локально, у коллеги, на сервере. В SQL для этого есть команда `ALTER TABLE` :

```
migraciya.sql
```

```
ALTER TABLE tasks ADD COLUMN done INTEGER NOT NULL
DEFAULT 0;
```

Старые строки не исчезают — у каждой из них новый столбец `done` получает значение по умолчанию, `0`.

```
migraciya.py
```

```
import sqlite3

baza = sqlite3.connect("zadachi.db")
baza.execute("ALTER TABLE tasks ADD COLUMN done
INTEGER NOT NULL DEFAULT 0")
baza.commit()
```

Инструменты, которые записывают миграции за вас

В маленьком проекте достаточно выполнить `ALTER TABLE` вручную один раз. В более крупном проекте с несколькими разработчиками и несколькими окружениями миграции обычно оформляют отдельными файлами с номерами и порядком применения — чтобы каждое изменение структуры можно было применить, отследить и, если нужно, откатить. В экосистеме SQLAlchemy для этого используют **Alembic** — отдельный инструмент для миграций схемы, который умеет сравнивать текущую структуру базы с описанием в коде и формировать черновик миграции; такой автосгенерированный файл — не готовое решение, а «черновая миграция» (candidate migration), которую нужно проверить и при необходимости поправить вручную перед применением. В Django миграции встроены в сам фреймворк: команда `makemigrations` создаёт файлы миграций по обнаруженным изменениям моделей, а `migrate` применяет их к базе данных.

Миграция схемы — про структуру, а не про содержимое

Миграция схемы отвечает на вопрос «как теперь устроена таблица», а не «что переехало из одной базы в другую». Это принципиально другая задача, чем перенос данных, о котором пойдёт речь в разделе 22.28 — не путайте эти два значения слова «миграция», когда встретите его в реальных проектах.

Практика: миграция схемы SQLite

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/22-27/index.html>)

Перенос данных между базами

Не структура таблицы, а сами записи — как их переносят между базами и почему перенос обязательно проверяют.

Второе значение слова «миграция» — **перенос данных** между базами или системами: например, из старой базы в новую, из одного формата хранения в другой. Не путайте с миграцией схемы из раздела 22.27 — там менялась структура таблицы, здесь переезжают сами записи.



Проверка

сверяем количество и содержимое

Перенос данных: чтение, преобразование, запись,
проверка

JSON (раздел 22.15) удобен как *промежуточный* формат в этом процессе — он текстовый, человекочитаемый и его легко прочитать в любом языке программирования. Но JSON — не универсальный формат переноса без потерь: у него нет отдельного типа для дат, для двоичных данных, и он не хранит ограничения и связи между таблицами (внешние ключи, раздел 22.21) — их приходится восстанавливать отдельно на стороне целевой базы.

Маленький воспроизводимый пример

eksport_v_json.py

```
import json
import sqlite3

istochnik = sqlite3.connect("zadachi.db")
istochnik.row_factory = sqlite3.Row

stroki = источник.execute("SELECT title, done FROM
tasks").fetchall()
dannye = [dict(stroka) for stroka in stroki]

with open("zadachi.json", "w", encoding="utf-8") as
fajl:
    json.dump(dannye, fajl, ensure_ascii=False,
indent=2)
```

```
import_iz_json.py
```

```
import json
import sqlite3

with open("zadachi.json", encoding="utf-8") as fajl:
    dannye = json.load(fajl)

cel = sqlite3.connect("novaya_zadachi.db")
cel.execute("CREATE TABLE tasks (id INTEGER PRIMARY
KEY, title TEXT NOT NULL, done INTEGER NOT NULL
DEFAULT 0)")
for zapis in dannye:
    cel.execute(
        "INSERT INTO tasks (title, done) VALUES
(?, ?)",
        (zapis["title"], zapis["done"]),
    )
cel.commit()
```

Идентификаторы — решение, которое нужно явно продумать и проверить

В примере выше новая база сама назначает новые значения `id` при вставке — старые идентификаторы не переносятся. Это осознанный выбор, не случайность: если приложению важно сохранить исходные `id` (например, на них ссылаются извне), их нужно переносить явно и после этого проверить, что они остались уникальными. Любой перенос данных должен заканчиваться проверкой: совпадает ли количество строк и совпадают ли представительные записи в источнике и в цели.

Перенос между локальными файлами SQLite в этом примере воспроизводит тот же принцип, что и перенос в промышленную базу вроде PostgreSQL, — читать, преобразовывать, записывать, проверять. Отличаются только конкретные инструменты и драйверы подключения; сначала — обязательная резервная копия исходных данных.

Практика: перенос данных через JSON

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/22-28/index.html>)

Flask и SQLite: сохраняем данные в базе

Список Python из раздела 22.5 становится таблицей SQLite, которая переживает перезапуск.

Теперь список Python из раздела 22.5 заменяется базой данных SQLite. Схема простая — одна таблица:

schema.sql

```
CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    done INTEGER NOT NULL DEFAULT 0 CHECK (done IN (0,
1))
);
```

`CHECK (done IN (0, 1))` — дополнительное ограничение: SQLite не даст записать в столбец `done` ничего, кроме 0 или 1, даже если в коде приложения где-то закрадётся ошибка.

Одно подключение на запрос

Открывать соединение с базой на каждый маленький вызов — расточительно, а держать одно общее соединение на всё приложение без осторожности — рискованно при параллельных запросах. Flask предлагает удобное место для данных, которые должны жить ровно один запрос, — объект `g`:

app.py

```

import sqlite3
from flask import Flask, g

app = Flask(__name__)
app.config["DATABASE"] = "zadachi.db"

def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(app.config["DATABASE"])
        g.db.row_factory = sqlite3.Row
    return g.db

@app.teardown_appcontext
def close_db(exception=None):
    db = g.pop("db", None)
    if db is not None:
        db.close()

```

g привязан к контексту приложения, а не буквально к запросу

`g` хранит данные в рамках *контекста приложения* Flask. При обычной обработке HTTP-запроса этот контекст живёт ровно с начала обработки запроса до его конца и создаётся заново для каждого следующего запроса — поэтому на практике `g` ведёт себя как хранилище на один запрос, а не как общая переменная, из которой данные могут случайно «утечь» в другой запрос. `teardown_appcontext` вызывается при закрытии контекста приложения, даже если обработчик завершился с ошибкой, — соединение всегда закрывается.

Маршруты поверх базы данных

app.py

```
@app.route("/")
def glavnaya():
    db = get_db()
    zadachi = db.execute("SELECT id, title, done FROM
tasks ORDER BY id").fetchall()
    return render_template("index.html",
zadachi=zadachi)

@app.route("/dobavit", methods=["POST"])
def dobavit():
    title = clean_title(request.form.get("zadacha",
""))
    if title is None:
        flash("Название задачи не может быть пустым и
не длинее 200 символов.")
        return redirect(url_for("glavnaya"), code=303)
    db = get_db()
    db.execute("INSERT INTO tasks (title) VALUES
(?)", (title,))
    db.commit()
    return redirect(url_for("glavnaya"), code=303)
```

Обратите внимание: `?` в SQL-запросе и значение отдельным аргументом — тот же параметризованный запрос из раздела 22.22, теперь внутри рабочего приложения. `flash(...)` сохраняет сообщение, которое покажется на следующей странице после редиректа — как именно это устроено, показывает следующий раздел, 22.30. `clean_title(...)` — простая функция проверки из раздела 22.31: пустое или слишком длинное название отклоняется ещё до обращения к базе.

Пустой список задач: заголовок «Мой список задач», подпись «Задач пока нет — добавьте первую ниже» и поле ввода

Список задач сразу после `init_db()` — таблица создана, но пуста.

Список задач с одной строкой «Купить хлеб» и пустым круглым чекбоксом слева

После отправки формы: INSERT прошёл, редирект вернул на / — задача видна в списке.

Тот же список: «Купить хлеб» зачёркнут, чекбокс слева с галочкой

После клика по кружку: `UPDATE tasks SET done = 1` — строка помечена как выполненная.

Полный рабочий файл — со всеми маршрутами, обработкой ошибок и API — лежит здесь:

[projects/flask/](#)

[todo-app/app.py](#)

Проверьте сами: перезапустите — задачи никуда не делись

Запустите `python app.py`, добавьте пару задач, остановите процесс (Ctrl+C) и запустите заново. Список останется тем же самым — именно то, чего не хватало версии из раздела 22.5.

Список задач в браузере после перезапуска процесса: «Купить хлеб» по-прежнему отмечен выполненным, ниже добавлена новая строка «Написать тесты для API»

Реальная проверка: процесс Flask был полностью остановлен и запущен заново на том же файле базы данных. «Купить хлеб» остался отмеченным, а новая задача, добавленная уже новым процессом, сохранилась рядом — файл SQLite пережил перезапуск, обычный список Python не пережил бы.

Практика: Flask поверх SQLite

Модуль flask не установлен в браузерном окружении Pyodide — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/22-29/index.html) → (<https://www.cartesianschool.org/practice/22-29/index.html>)

Cookie и сессии: состояние между HTTP-запросами

HTTP-запросы независимы друг от друга — cookie и сессионные механизмы позволяют связать состояние между ними.

HTTP-запросы независимы друг от друга: сам по себе протокол не связывает один запрос со следующим. Но приложению из раздела 22.29 уже нужно сохранять что-то между запросами — например, сообщение об ошибке, которое должно появиться на следующей странице после `flash(...)`. Для этого существуют cookie и сессионные механизмы.

Cookie — маленькое значение, которое хранит браузер

Cookie — небольшой кусочек данных, который сервер просит браузер сохранить, а браузер затем прикладывает к следующим запросам на тот же сайт. Именно так сервер может связать несколько запросов друг с другом как пришедшие от одного и того же браузера.



Сессия — механизм сохранения состояния между запросами

Сессия (session) — механизм уровня приложения, который использует cookie, чтобы связать несколько запросов друг с другом и хранить между ними какие-то данные. Реализации бывают разными: данные сессии можно хранить на сервере (в базе данных или отдельном хранилище, а в cookie передавать только идентификатор), а можно — прямо в самом cookie на стороне браузера.

Flask по умолчанию использует второй вариант и предоставляет готовый объект `session`, который ведёт себя как словарь:

```
session_primer.py
```

```
from flask import session

session["poslednij_vizit"] = "22-30"
# на следующем запросе того же браузера:
session.get("poslednij_vizit") # "22-30"
```

Функция `flash(...)`, использованная в разделе 22.29, устроена поверх того же механизма `session`: сообщение сохраняется в сессии на один следующий запрос и автоматически стирается после того, как его прочитали через `get_flashed_messages()` в шаблоне.

Подписано — не значит зашифровано

По умолчанию Flask хранит содержимое сессии прямо в cookie на стороне браузера, защищённое подписью с помощью `SECRET_KEY`. Подпись гарантирует **подлинность**: если кто-то изменит содержимое cookie, подпись перестанет совпадать, и Flask отклонит такую сессию. Но подпись — **не шифрование**: содержимое сессии по умолчанию можно прочитать, не зная секретный ключ, — оно просто не поддаётся обнаруживаемому изменению. Никогда не кладите в сессию пароли или другие данные, которые должны оставаться нечитаемыми для пользователя, чей браузер их хранит.

Проект этой главы намеренно останавливается здесь: полноценный вход пользователей (аутентификация) добавил бы главе объём отдельного курса. Где эта тема продолжается — в разделе 22.36.

Практика: сообщение через flash() и session

Модуль flask не установлен в браузерном окружении Pyodide — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/22-30/index.html>)

Проверка входных данных и обработка ошибок

Форма в браузере — не единственный способ отправить запрос, поэтому проверка на сервере обязательна.

Раздел 22.4 показал проверку формы в браузере через JavaScript. Это удобно для пользователя, но никогда не единственная линия защиты: запрос можно отправить и вовсе без браузера — например, программой, которая обращается прямо к `/dobavit` или `/api/tasks`, без единой строчки JavaScript. Входные данные необходимо повторно проверять на серверной стороне в каждом маршруте, который их принимает.

Не доверяйте вводу только потому, что он пришёл из вашей формы

HTML-форма из раздела 22.13 задаёт `required` и `maxlength` — но это подсказки для браузера, а не гарантия. Ничто не мешает отправить POST-запрос на `/dobavit` напрямую, в обход формы и любых её ограничений.

Проверка в проекте главы

```
app.py

MAX_TITLE_LENGTH = 200

def clean_title(raw_title):
    title = raw_title.strip()
    if not title or len(title) > MAX_TITLE_LENGTH:
        return None
    return title
```

`strip()` убирает случайные пробелы по краям (например, если пользователь нажал пробел, а затем Enter). Пустая строка после этого и слишком длинная строка — оба случая считаются недопустимыми и возвращают `None`.

HTML-маршрут и API-маршрут реагируют на ошибку по-разному

	HTML-форма (/dobavit)	API (/api/tasks)
Кто читает ответ	Человек в браузере	Программа
Реакция на невалидные данные	flash(...) с понятным текстом и редирект обратно (раздел 22.30)	Код 400 и тело JSON с описанием ошибки
Пример ответа	Страница со списком задач и сообщением сверху	<code>{"error": "title не может быть пустым"}</code>

Список задач с розовым сообщением сверху: «Название задачи не может быть пустым и не длиннее 200 символов.»

Реальный результат отправки формы с полем из одних пробелов: `clean_title()` вернул `None`, `flash()` показал сообщение, редирект вернул на ту же страницу — задача не добавлена.

`app.py`

```
data = request.get_json(silent=True)
if not isinstance(data, dict) or not
instance(data.get("title"), str):
    return jsonify({"error": "поле title обязательно
и должно быть строкой"}), 400
title = clean_title(data["title"])
if title is None:
    return jsonify({"error": "title не может быть
пустым и не длиннее 200 символов"}), 400
```

404 — обращение к тому, чего нет

Если попытаться отметить выполненной или удалить задачу с несуществующим `id`, маршрут вызывает `abort(404)` — Flask сразу возвращает стандартную страницу ошибки (или JSON для API-путей, раздел 22.9), не выполняя код дальше.

Практика: проверяем ввод

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/22-31/index.html>)

Основы безопасности веб-приложений

Несколько частых ошибок и принципов, которые нужно знать даже в маленьком проекте.

Веб-безопасность — большая тема; здесь — только самые частые ошибки и принципы, которые нужно знать даже в маленьком проекте. Глубокое изучение остаётся будущему специализированному курсу (раздел 22.36).

SQL-инъекция

Если собрать SQL-запрос склеиванием строк с пользовательским вводом, часть введённого текста может быть воспринята как часть самого запроса, а не как данные:

`opasno.py`

```
# ТАК ДЕЛАТЬ НЕЛЬЗЯ
db.execute(f"SELECT * FROM tasks WHERE title =
'{{user_input}}'")
```

`bezopasno.py`

```
db.execute("SELECT * FROM tasks WHERE title = ?",
(user_input,))
```

Ограничение `execute()` не делает f-строки безопасными

Метод `sqlite3.execute()` в Python не разрешает несколько SQL-инструкций за один вызов: составная атака вроде `x'; DROP TABLE tasks; --` завершится ошибкой `ProgrammingError`. Это ограничение конкретного Python-драйвера, а не универсальная защита SQL. Оно не спасает от одноинструкционных инъекций: ввод `' OR '1'='1` меняет условие `WHERE` и возвращает все строки. Никогда не вставляйте недоверенный ввод в текст SQL — передавайте его только параметром.

Раздел 22.22 уже показывал этот приём — **параметризованный запрос**, где значение передаётся отдельно от текста SQL и никогда не смешивается с ним.

Модуль `sqlite3` сам отвечает за то, чтобы значение осталось данными, что бы в нём ни было — даже фрагмент, похожий на SQL.

XSS — межсайтовый скриптинг

Если чужой текст попадает в HTML-страницу без обработки, он может содержать исполняемый код, который выполнится в браузере другого пользователя, — например, `<script>...</script>` в названии задачи. Раздел 22.12 уже показал защиту: в автоэкранируемом HTML-контексте Jinja заменяет специальные символы HTML соответствующими `escape`-последовательностями, поэтому введенный текст не интерпретируется как разметка. Это не делает значение безопасным вставлять куда угодно — например, внутрь атрибута `<script>` или URL нужна уже другая защита, — именно поэтому фильтр `|safe`, который вообще отключает экранирование, нельзя применять к вводу, который вы не полностью контролируете сами.

CSRF — подделка межсайтового запроса

CSRF (Cross-Site Request Forgery) актуален там, где браузер автоматически прикладывает учётные данные (например, `cookie` сессии, раздел 22.30) к запросам, а сам запрос меняет состояние на сервере: пользователь, залогиненный на сайте, открывает другую, вредоносную страницу, а та страница теоретически может незаметно отправить от его имени запрос на ваш сайт — браузер сам приложит те же `cookie`. Пока в проекте главы нет входа пользователей, риск CSRF ограничен, но для приложений, где авторизация или состояние опираются на автоматически отправляемые браузером `cookie`, изменяющие состояние запросы должны быть защищены от CSRF соответствующим механизмом — обычно его предоставляет сам фреймворк или расширение.

Пароли

Пароли никогда не хранят как обычный текст — только в виде хеша, полученного через проверенный, специально предназначенный для паролей инструмент. Изобретать собственную схему хеширования паролей не нужно и не стоит — для этого есть готовые, тщательно проверенные библиотеки. Проект этой главы не хранит пароли вообще: раздел 22.30 уже объяснил, почему полноценный вход пользователей остался за пределами этой главы.

Секреты

`SECRET_KEY` приложения, пароли к базе данных, ключи внешних API — всё это не должно попадать в публичный исходный код. Как такие значения обычно передают через переменные окружения вместо того, чтобы записывать их прямо в файл с кодом, — в разделе 22.34.

HTTPS — ещё раз, коротко

Раздел 22.10 уже объяснил: HTTPS защищает данные *по пути* между браузером и сервером, но не устраняет ошибки внутри самого приложения — SQL-инъекцию, XSS или утечку секретов HTTPS не остановит. Это разные, дополняющие друг друга уровни защиты.

Практика: параметризация против инъекции

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/22-32/index.html>)

Автоматическое тестирование Flask-приложения

Тестовый клиент проверяет логику приложения быстро и без обращения к сети.

Проверять каждое изменение вручную через браузер медленно и легко забыть повторить после следующей правки. У Flask есть встроенный **тестовый клиент** — он позволяет отправлять запросы приложению из кода теста и проверять ответ.

Тестовый клиент не открывает сетевой порт

`app.test_client()` не создаёт сетевое HTTP-соединение. Он формирует запрос внутри процесса и передаёт его напрямую обработчику Flask через тестовый интерфейс библиотеки Werkzeug, на которой построен Flask, — быстро и без сети, но с той же логикой обработки запроса. Полный путь запроса через браузер и сеть показывал раздел 22.7 — тестовый клиент сознательно пропускает именно эту часть.

test_app.py

```
import app as todoapp

def test_glavnaya_pustoj_spisok(baza_dannyh):
    client = todoapp.app.test_client()
    otvet = client.get("/")
    assert otvet.status_code == 200
    assert "Задач пока нет" in
otvet.get_data(as_text=True)
```

Проверка JSON-ответа

test_api.py

```
def test_api_tasks_vozvrashchaet_json(baza_dannyh):
    client = todoapp.app.test_client()
    otvet = client.get("/api/tasks")
    assert otvet.status_code == 200
    assert otvet.content_type == "application/json"
    dannye = otvet.get_json()
    assert isinstance(dannye, list)
```

`otvet.get_json()` сам разбирает тело ответа как JSON — это то же самое, что `json.loads(otvet.get_data())` из раздела 22.15, но короче.

Отдельная база данных для тестов

Тесты не должны трогать ту же базу данных, с которой вы работаете вручную, — иначе прогон тестов может стереть или испортить сохранённые в ней задачи. Перед каждым тестом создаётся временная база данных, а после — удаляется:

`conftest.py`

```
import tempfile
import os
import pytest

import app as todoapp

@pytest.fixture
def baza_dannyh():
    fd, put = tempfile.mkstemp()
    todoapp.app.config["DATABASE"] = put
    with todoapp.app.app_context():
        todoapp.init_db()
    yield put
    os.close(fd)
    os.unlink(put)
```

Полный набор тестов проекта — в `tests/test_chapter22_todo_app.py` в репозитории книги; он проверяет маршруты, сохранение в базе, валидацию, JSON API и защиту от SQL-инъекции и XSS из разделов 22.31-22.32.

Практика: тесты для Flask-приложения

Модуль flask не установлен в браузерном окружении Pyodide — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику → \(https://www.cartesianschool.org/practice/22-33/index.html\)](https://www.cartesianschool.org/practice/22-33/index.html)

Развёртывание Flask-приложения

Сервер разработки Flask удобен локально — но не для того, что доступно всему интернету.

Раздел 22.5 запускал приложение командой `python app.py`, которая внутри вызывает `app.run(debug=True)`. Это **сервер разработки** (development server) — встроенный во Flask, удобный для локальной работы, но не предназначенный для реальной эксплуатации. Раздел «Deploying to Production» официальной документации Flask прямо предупреждает об этом.

	Среда разработки	Рабочее развёртывание
Кто использует	Вы сами, локально	Реальные пользователи через интернет
Сервер	Встроенный сервер разработки Flask	Отдельный сервер приложений (раздел 22.19)
debug=True	Удобно — подробные ошибки, автоперезапуск	Опасно — может показать посторонним внутренние детали кода и данные
Секреты (SECRET_KEY и т.п.)	Можно временное значение для локальной работы	Обязательно отдельное, не публикуемое значение

debug=True в открытом приложении — реальная уязвимость

В режиме отладки Flask может показать интерактивный отладчик с полным доступом к выполнению кода прямо в браузере при необработанной ошибке. Это огромное удобство локально и серьёзный риск, если приложение доступно кому-то ещё, — отладочный режим должен быть выключен в любом развёртывании, доступном не только вам.

Путь запроса до рабочего приложения



База данных

отдельное надёжное хранилище

Тот же Flask-код — но между браузером и им теперь несколько дополнительных слоёв

Конфигурация через переменные окружения

Проект этой главы уже читает путь к базе данных и секретный ключ из переменных окружения, с явным резервным значением для локальной разработки:

app.py

```
app.config["DATABASE"] =
os.environ.get("TODO_APP_DB", str(DEFAULT_DB_PATH))
app.config["SECRET_KEY"] = os.environ.get(
    "TODO_APP_SECRET_KEY", "dev-only-secret-do-not-
    use-in-production"
)
```

Так один и тот же код работает и локально (без единой настройки), и в другом окружении — там, где нужно, значения переменных окружения просто задают иначе, не трогая сам код.

Логи, несколько процессов, персистентность

У рабочего развёртывания есть и другие практические заботы, которые выходят за рамки этой главы: **логи** (записанная история происходящего, чтобы разбирать проблемы постфактум), запуск **нескольких процессов или потоков** одновременно для обработки параллельных запросов, и то, что база данных должна физически

находиться на надёжном, не исчезающем хранилище. Мы намеренно не превращаем эту главу в учебник по эксплуатации — где эта тема продолжается, показывает раздел 22.36.

Официальная документация: [Flask — Deploying to Production](#).

Итоговый проект: список задач на Flask и SQLite

Тот же список задач, с которого началась глава, — теперь с постоянным хранением, JSON API и проверкой ввода.

Всё, что разбирали разделы 22.7-22.34, сходится в одном рабочем приложении — том же списке задач, с которого началась глава, теперь с постоянным хранением в SQLite.

projects/flask/
todo-app/app.py

Файл	Роль
app.py	Маршруты, работа с базой данных, API
schema.sql	Структура таблицы tasks
templates/base.html	Общий каркас страницы, включая сообщения flash()
templates/index.html	Список задач, форма добавления

Файл	Роль
<code>templates/privet.html</code>	Историческая страница приветствия из раздела 22.5
<code>templates/404.html</code>	Страница «не найдено»
<code>static/style.css</code>	Собственные стили, без внешних CDN

Что умеет готовое приложение

Пять возможностей одного маленького приложения

СПИСОК ЗАДАЧ

GET / — показывает все задачи

Пустое состояние, если задач нет

ДОБАВЛЕНИЕ

POST /dobavit

Проверка ввода (раздел 22.31)

POST-Redirect-GET (раздел 22.13)

ОТМЕТКА ВЫПОЛНЕНИЯ

POST /vypolnit/

Переключает done между 0 и 1

УДАЛЕНИЕ

POST /udalit/

Полностью убирает строку из базы

JSON API

GET /api/tasks – список

POST /api/tasks – добавление с JSON-телом

Итоговый список задач в браузере: две выполненные (зачёркнутые) задачи вверху и две ещё не выполненные ниже, поле добавления и кнопка «Поздороваться» снизу

Локально запущенное Flask-приложение в браузере — итоговый вид проекта: выполненные и невыполненные задачи, форма добавления и ссылка на историческую страницу /privet/<имя> из раздела 22.5.

Удаление и отметка выполнения — тоже POST, не GET

Раздел 22.9 уже объяснял: GET не должен менять состояние на сервере, потому что браузер может повторить его без предупреждения — например, при обновлении страницы. Поэтому у кнопок «Выполнено» и «Удалить» в шаблоне — собственные маленькие формы с `method="post"`, а не простые ссылки `<a href="/udalit/...">`.

Запуск

```
terminal.txt
```

```
cd projects/flask/todo-app
python app.py
```

Первый запуск сам создаёт файл базы данных и таблицу — отдельная команда для этого не нужна. Откройте `http://127.0.0.1:5000/` : добавьте задачу, отметьте её выполненной, удалите, обновите страницу посреди процесса, перезапустите сам процесс и убедитесь, что список остался прежним.

Приложение работает и на узких экранах: раскладка CSS (раздел 22.3) перестраивается под мобильную ширину без горизонтальной прокрутки страницы.

★★ Самостоятельная задача

Счётчик оставшихся задач

Добавьте в `index.html` строку с числом невыполненных задач — `{{ zadachi|selectattr('done', 'equalto', 0)|list|length }}`.

★★★ Задача повышенной сложности

Сортировка по статусу

Измените запрос в `glavnaya()`, чтобы невыполненные задачи всегда показывались раньше выполненных — `ORDER BY done, id`.

Практика: работаем с готовым проектом

Модуль `flask` не установлен в браузерном окружении `Pyodide` — выполните локально в `VS Code`, `PyCharm` или `Jupyter`

Открыть практику → (<https://www.cartesianschool.org/practice/22-35/index.html>)

Дальнейшее изучение веб-разработки на Python

От первого HTTP-запроса до работающего приложения с базой данных — и куда двигаться дальше.

Что теперь понятно после этой главы

- Как устроен диалог браузера и сервера: запрос, ответ, HTTP и HTTPS.
- Из чего состоит фронтенд — HTML, CSS, JavaScript — и что каждый из них делает.
- Как Flask превращает функции Python в обработчики HTTP-запросов через маршруты.
- Как шаблоны Jinja собирают HTML из данных, и почему автоэкранирование включено по умолчанию.
- Что такое JSON и API, и чем ответ с данными отличается от HTML-страницы.
- Чем отличаются друг от друга Flask, Django и FastAPI — и что такое WSGI и ASGI.
- Зачем нужна база данных, что такое SQL, и чем реляционные базы отличаются от нереляционных.
- Что такое миграция схемы и перенос данных — и почему это два разных значения одного слова.
- Базовые принципы безопасности: параметризованные запросы, автоэкранирование, секреты.
- Как проверить Flask-приложение тестами и чем сервер разработки отличается от рабочего развертывания.

Вы прошли путь от обмена сообщениями по HTTP между браузером и сервером до работающего приложения с базой данных, JSON API и тестами — солидная основа, но далеко не вся веб-разработка. Этой теме будет посвящён

отдельный курс Cartesian School «Веб-разработка на Python: от HTTP до развёртывания» (курс готовится). Он продолжит материал этой главы значительно глубже.

Чем займётся будущий специализированный курс

ФРЕЙМВОРКИ ГЛУБЖЕ

Архитектура Flask за пределами основ
Django целиком
FastAPI и типизированные API

БАЗЫ ДАННЫХ

PostgreSQL на практике
SQLAlchemy и Alembic
ORM и миграции Django

API И ОБМЕН ДАННЫМИ

REST подробно
OpenAPI
Аутентификация и авторизация

ПАРАЛЛЕЛЬНОСТЬ И РЕАЛЬНОЕ ВРЕМЯ

Асинхронный Python

ASGI на практике

WebSocket

ИНФРАСТРУКТУРА СЕРВИСА

Фоновые задачи

Redis и кеширование

Docker, обратные прокси, CI/CD

ЭКСПЛУАТАЦИЯ

Конфигурация и секреты

Логи, метрики, наблюдаемость

Рабочее развёртывание

А следующая глава книги возвращается к настольным мини-проектам на Python — совсем другая часть языка в деле.

Первый проект на GitHub: SafeSort

Пишем программу, которая наводит порядок в файлах, — и доводим её до состояния, готового для GitHub: тесты, история коммитов, Pull Request.

23.1 Что мы будем создавать: SafeSort

Что такое Git и что такое GitHub

Устанавливаем Git

Первая настройка Git

Учётная запись GitHub

HTTPS, SSH и аутентификация

Настраиваем SSH

Создаём репозиторий SafeSort

Клонировем репозиторий

Локальный и удалённый репозиторий

Ветка, HEAD и origin/main

Repository и GitHub Project

Лучшие практики Project

Создаём GitHub Project

Board, Table и Roadmap

Поля Project

Черновики: задача без Issue

Превращаем черновик в Issue

Создаём Issues

Первый цикл: Issue → Branch → PR

Необязательно · Копируем Project

Необязательно · Редактируем элементы

Необязательно · Фильтруем и группируем

Необязательно · Представления

Необязательно · Автоматизация

Необязательно · Архив

Необязательно · Шаблоны

Необязательно · Insights

23.2 Создаём репозиторий проекта

23.3 Первый README проекта

23.4 Планируем структуру Python-пакета

23.5 `pyproject.toml` и установка проекта

23.6 Командная строка `SafeSort`

23.7 `pathlib`: работаем с путями и каталогами

23.8 Сканируем каталог

23.9 Какие каталоги не нужно сканировать

23.10 Определяем категорию файла

23.11 От анализа к плану действий

23.12 Режим предварительного просмотра

23.13 Безопасно перемещаем файлы

23.14 Что делать, если имя уже занято

23.15 Журнал выполненных операций

23.16 Отмена последней операции

23.17 Поиск одинаковых файлов

23.18 SHA-256 и хеш содержимого файла

23.19 Находим группы дубликатов

23.20 Обработываем ошибки файловой системы

23.21 Добавляем журнал работы программы

23.22 Настройки проекта

23.23 Пишем первые автоматические тесты

23.24 Проверяем сканирование и классификацию

23.25 Проверяем перемещение и отмену

23.26 Проверяем поиск дубликатов

23.27 Проверяем интерфейс командной строки

23.28 Самопроверка изменений и дисциплина коммитов

23.29 GitHub: применяем Issue, ветку и Pull Request

23.30 GitHub Actions: автоматически запускаем тесты

23.31 Документация, версия и первый релиз

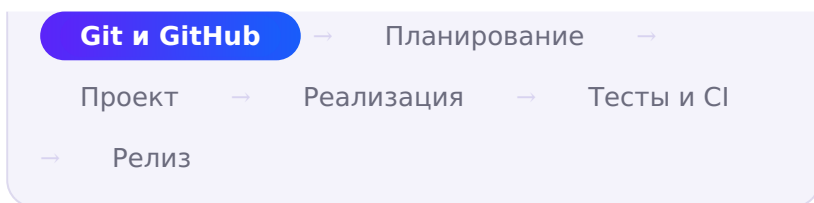
23.32 Итоги: полный путь проекта от идеи до релиза

Приложение: шесть мини-проектов для GitHub

Что мы будем создавать: SafeSort

Программа, которая наводит порядок в файлах, — сначала показывает, что сделает, и только потом делает это по команде.

SAFESORT · ЧАСТЬ 1 ИЗ 6



До сих пор мы писали программы по частям: функции, игры, интерфейсы. Теперь соберём один проект целиком и доведём его до состояния, которое можно разместить на GitHub. Программа называется **SafeSort**, и вот что она делает.

БЫЛО

Downloads/

report.pdf

photo.jpg

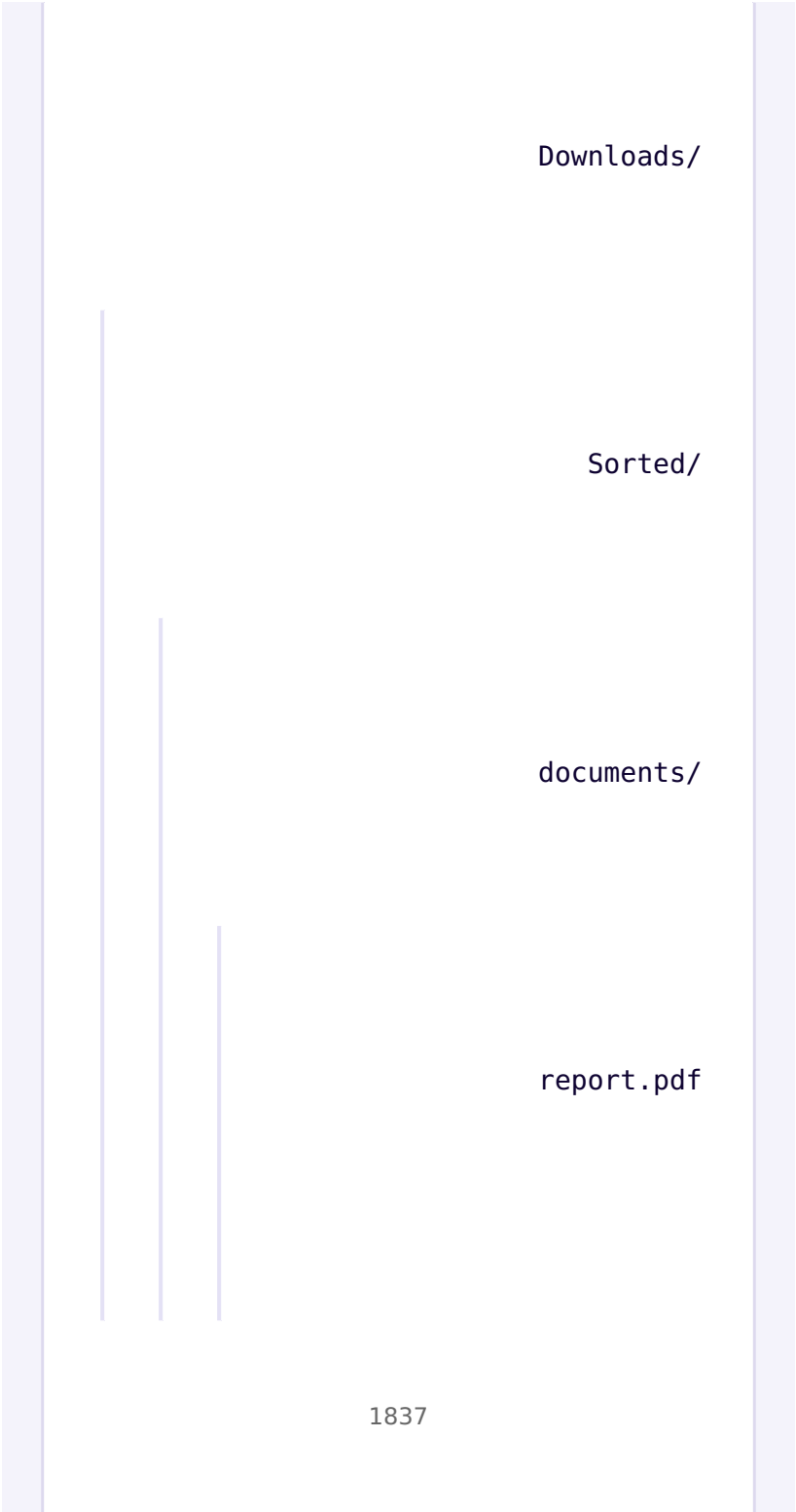
archive.zip

notes.txt



copy_of_notes.txt

СТАЛО

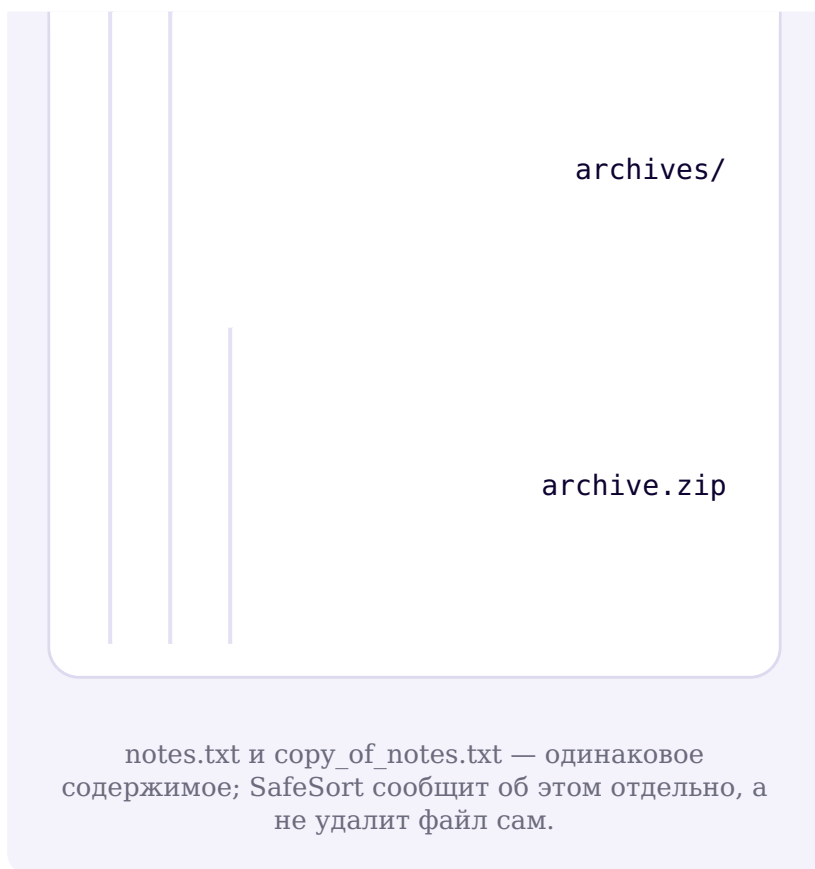


notes.txt

copy_of_notes.txt

images/

photo.jpg



Сначала программа только показывает, что собирается сделать. Переместить файлы может исключительно отдельная, явно вызванная команда:



ТОЛЬКО ЧИТАЮТ

scan — показывает найденные файлы
 plan — показывает план перемещений
 duplicates — показывает совпадения

МЕНЯЮТ ФАЙЛЫ

apply — перемещает файлы
 undo — возвращает файлы на место



```
~/safesort $ safesort plan ~/Downloads
5 move operations planned.
No files have been changed.
```

Файловая система после этой команды осталась прежней — `plan` только описывает будущие перемещения. Это различие между «показать» и «сделать» — главная идея всего проекта, и мы будем возвращаться к ней ещё не раз.

Что должна уметь первая версия

Программа должна уметь:

- показать найденные файлы, разложенные по категориям;

- показать план будущих перемещений, ничего не меняя;
- выполнить перемещения только по явной команде;
- найти файлы с одинаковым содержимым;
- отменить последнюю выполненную операцию.

Как программа должна себя вести:

- не изменять файлы без явной команды;
- не перезаписывать существующий файл молча;
- одинаковый ввод даёт одинаковый план — никакой случайности;
- вся обработка происходит локально, без сети;
- логику можно проверить автоматическими тестами, не трогая настоящие файлы.

Первый список в разработке принято называть **функциональными требованиями** — что программа делает. Второй — **нефункциональными**: не «что», а «как», какими свойствами обладает поведение программы независимо от команды.

Что не будем делать в первой версии

Чтобы закончить проект, а не растягивать его бесконечно, сразу ограничим задачу:

Не входит в первую версию	Почему
Автоматическое удаление дубликатов	Удаление данных без явного подтверждения — риск, а не удобство
Графический интерфейс	Командную строку проще реализовать, протестировать и объяснить
Загрузка файлов в облако	Программа работает только с локальной файловой системой

Не входит в первую версию	Почему
Классификация по содержанию файла	Классификация по расширению уже решает основную задачу

Коротко

- scan, plan и duplicates только читают файловую систему; apply сортирует файлы, а undo выполняет обратное перемещение.
- «Показать план» и «выполнить план» — разные шаги, и это различие определяет всю архитектуру программы.
- Функциональные требования — что программа делает; нефункциональные — какими свойствами обладает её поведение.

Git ≠ GitHub Что такое Git и что такое GitHub

Git и GitHub не синонимы: Git работает локально, а GitHub предоставляет сетевой сервис поверх репозитория.

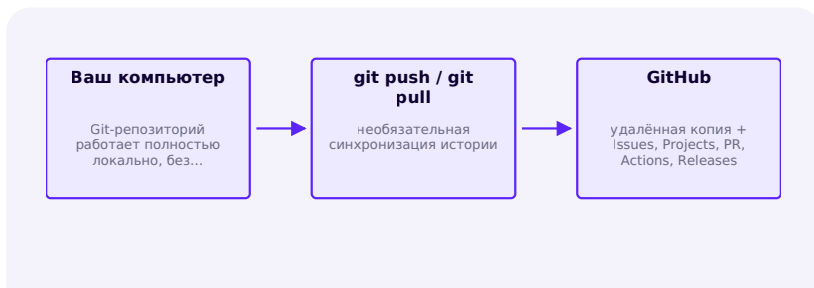
SafeSort · Часть 1 из 6 **Git и GitHub**

≠ GitHub

Предыдущий раздел показал, что будет делать SafeSort. Прежде чем писать код, разберёмся с инструментами, вокруг которых построена вся оставшаяся часть главы, и с четырьмя похожими словами, которые легко перепутать. Git и GitHub: разные продукты, а не два имени одной и той же вещи.

Понятие	Что это
Python-проект	исходный код и файлы SafeSort на диске, с которыми мы работаем
Git-репозиторий	история изменений этих файлов, хранящаяся в скрытом каталоге .git
GitHub-репозиторий	Git-репозиторий на сервере GitHub вместе с веб-интерфейсом
GitHub Project	отдельный инструмент планирования: Issues, доска и представления

Git работает полностью локально на вашем компьютере и хранит историю изменений файлов. Ему не нужны ни аккаунт GitHub, ни интернет, ни удалённый репозиторий. Можно установить Git, создать репозиторий, сделать сто коммитов и ни разу не подключиться к сети. **GitHub** работает как отдельный интернет-сервис поверх Git: он хранит удалённую копию репозитория и добавляет то, чего у самого Git нет: Issues, Pull Request, Projects, Actions и Releases. GitHub построен вокруг Git, но обратной зависимости нет: Git прекрасно работает без GitHub, а вот GitHub без Git, напротив, работать не может. Этот сервис хранит и обслуживает Git-репозитории.



Git работает с локальной историей самостоятельно. GitHub предоставляет сетевой сервис поверх Git. Зависимость односторонняя: Git не нуждается в GitHub, а GitHub использует Git-репозитории.

Репозиторий на GitHub — не то же самое, что GitHub Project

Это одна из самых частых путаниц у начинающих. Репозиторий хранит код и его историю. GitHub Project служит отдельным, необязательным инструментом планирования: можно иметь репозиторий вообще без Project, и можно завести Project, объединяющий Issues сразу из нескольких репозиториев. Часть II этой главы разберёт эту разницу подробно.

Официальные ресурсы Git

У проекта Git есть собственный сайт, справочник и учебная книга. Они отвечают на разные вопросы, поэтому полезно сразу знать, куда идти за установщиком, точным синтаксисом команды или цельным объяснением идеи.

Ресурс	Для чего он нужен
Официальный сайт Git	Загрузки Git, сведения о проекте и ссылки на учебные материалы.
Справочная документация Git	Точное описание отдельных команд: <code>git init</code> , <code>git clone</code> , <code>git add</code> , <code>git commit</code> , <code>git branch</code> , <code>git switch</code> , <code>git fetch</code> , <code>git pull</code> и <code>git push</code> .

Ресурс	Для чего он нужен
Книга Pro Git	Последовательное объяснение модели Git. Это учебная книга, а не только перечень параметров команд.
Git Source Code Mirror on GitHub	Зеркало исходного кода Git на GitHub. Сам Git не зависит от GitHub: размещение зеркала показывает, что Git и GitHub остаются разными системами.

Логотип Git

Цветной знак на этой странице взят из официального набора Git без перерисовки и изменения цвета. Git Logo by Jason Long, лицензия [CC BY 3.0](#).

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Официальный сайт Git](#)

[Справочная документация Git](#)

[Книга Pro Git](#)

[Git Source Code Mirror on GitHub](#)

[Git logos](#)

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Start your journey: What is GitHub?](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Git работает локально и следит за историей файлов; ему не нужен интернет.
- GitHub предоставляет удалённую копию репозитория и инструменты Issues, Pull Request, Projects, Actions и Releases.
- Репозиторий хранит код и историю, а GitHub Project служит для планирования. Это независимые понятия.

Устанавливаем Git

Git ставится отдельно от Python, один раз для всей системы — способ зависит от операционной системы.

SafeSort · Часть 1 из 6 **Git и GitHub**

Git — не встроенная часть Python: его нужно установить отдельно, один раз для всей системы. Способ установки зависит от операционной системы.



```
~/safesort $ git --version  
git version 2.47.3
```

Так выглядит успешная проверка — программа сообщает свою версию и ничего больше.

Linux (Debian/Ubuntu)

Документированная команда — см. git-scm.com/download/linux

```
sudo apt update
sudo apt install git
```

git, а не git-all

Пакет `git` уже включает всё нужное для этого курса. Пакет `git-all` — метапакет, добавляющий дополнительные интеграции (например, с Emacs), которые здесь не понадобятся.

Для дистрибутивов на основе Fedora/RHEL:

Документированная команда

```
sudo dnf install git
```

Windows

Официальный установщик — **Git for Windows**: он ставит саму программу `git` и терминал **Git Bash**, в котором работают все команды этой главы без изменений.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Git for Windows](https://git-scm.com/docs)

macOS

На macOS у git-scm.com больше нет единого официального установщика: отдельный бинарный `installer` поддерживался только до версии 2.33.0 (2021 год) и с тех пор не обновляется. git-scm.com прямо

сообщает, что больше не ссылается на него и не планирует возобновлять поддержку. Вместо этого на macOS используют один из трёх вариантов:

Способ	Команда
Xcode Command Line Tools; подходит, если полный Xcode не нужен	<code>xcode-select --install</code>
Homebrew; распространённый менеджер пакетов для macOS	<code>brew install git</code>
MacPorts; альтернативный менеджер пакетов	<code>sudo port install git</code>

Эти сборки поддерживает не `git-scm.com`

`git-scm.com` сам предупреждает: любые дистрибутивы Git, кроме исходного кода, собирают и поддерживают третьи стороны (Apple для Xcode Command Line Tools, команда Homebrew, команда MacPorts), и они не обязаны обновляться сразу вслед за новым релизом Git. Для этого курса подходит любой из трёх вариантов. Команда `git --version` ниже покажет, что реально установилось.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Git for macOS](#)

Проверяем установку

Независимо от системы, результат один и тот же: команда `git --version` в терминале печатает номер версии. Если вместо этого терминал отвечает «command

not found» (или похожим сообщением на Windows) — Git не установлен или его нет в PATH; переустановка обычно решает проблему.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Git — Downloads](#)

Коротко

- Git ставится один раз для системы — отдельно от Python.
- Linux: `sudo apt install git` (или `dnf install git`). Windows: Git for Windows + Git Bash. macOS: Xcode Command Line Tools, Homebrew или MacPorts; единого официального установщика с 2021 года нет.
- Команда `git --version` проверяет результат установки.

Первая настройка Git

Прежде чем сделать первый коммит, Git нужно один раз сказать, кто вы, — это записывается в каждый коммит.

SafeSort · Часть 1 из 6 **Git и GitHub**

Прежде чем сделать первый коммит, Git нужно один раз сообщить имя и email. эта информация записывается в каждый коммит и остаётся в истории навсегда.



```
~/git_demo $ git init
Initialized empty Git repository in .../
git_demo/.git/
```

```
~/git_demo $ git config user.name "Ваше Имя"
~/git_demo $ git config user.email
"you@example.com"
~/git_demo $ git config --local --list
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
user.name=Ваше Имя
user.email=you@example.com
```

Это имя записывается в коммиты, а не только в профиль GitHub

`user.name` / `user.email` обозначают не логин GitHub и не то же самое, что имя в профиле. Это данные, которые буквально попадают в каждый коммит как автор изменения, и их видно всем, кто посмотрит историю, в том числе после публикации на GitHub.

Флаг `--local` относится только к этому одному репозиторию. Чтобы не повторять настройку для каждого нового проекта, обычно используют `--global` — тогда имя и почта применяются ко всем репозиториям на этом компьютере:

Терминал

```
git config --global user.name "Ваше Имя"
git config --global user.email "you@example.com"
```

Имя ветки по умолчанию

Здесь легко ошибиться: сам Git до сих пор называет самую первую ветку нового репозитория `master`, если её имя никак не настроено, — это встроенное поведение не изменилось. В версии 2.28 у Git появилась только сама *настройка* `init.defaultBranch`, позволяющая задать другое имя по умолчанию, — а не новое поведение "из коробки". Имя `main` стало привычным, потому что GitHub, GitLab и многие редакторы настраивают эту опцию сами при установке, но сам Git её не устанавливает. Поэтому Cartesian School задаёт её явно, а не полагается на то, что окажется настроено на конкретном компьютере:

Терминал

```
git config --global init.defaultBranch main
```

Git 2.28 добавил настройку, а не новое поведение

Легко перепутать эти две вещи. Git 2.28 (выпущен в 2020 году) добавил опцию `init.defaultBranch` — раньше такой настройки не существовало вообще, и переименовать первую ветку можно было только вручную после создания репозитория. Но сама опция ничего не меняет, пока её не задать: без неё Git по-прежнему создаёт ветку `master`, в любой версии, вплоть до самой новой. Проверить это можно на чистом окружении без `~/.gitconfig` — там `git init` создаёт именно `master`.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Set up Git](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- `user.name/user.email` записываются в каждый коммит — это данные автора, а не логин GitHub.
- `--global` применяет настройку сразу ко всем репозиториям на компьютере.
- Git 2.28 добавил настройку `init.defaultBranch`, но встроенное имя первой ветки по-прежнему `master` — `main` нужно задать явно.

Создаём и защищаем учётную запись GitHub

Имя пользователя, email и двухфакторная аутентификация — несколько решений стоит принять осознанно с самого начала.

SafeSort · Часть 1 из 6 **Git и GitHub**

Работа с GitHub требует учётной записи. Завести её просто, но пара решений на этом шаге стоит того, чтобы принять их осознанно.

Имя пользователя

Имя пользователя GitHub становится частью адреса каждого вашего репозитория (`github.com/имя/репозиторий`) и видно всем — стоит выбрать что-то, что не жалко будет использовать в резюме или портфолио, а не сиюминутный никнейм.

Email

GitHub требует подтверждённый email. Если не хочется, чтобы личный адрес попадал в историю коммитов при публикации репозитория, GitHub предоставляет приватный `@users.noreply.github.com` -адрес — специально для этого случая.

Пароль и двухфакторная аутентификация

GitHub поддерживает вход по паролю с двухфакторной аутентификацией (2FA) и по passkey. Включить 2FA стоит сразу: учётная запись на GitHub — это не просто набор файлов, а доступ к публикации кода от вашего имени.

Ситуация	Решение
Личный email в коммитах нежелателен	используйте noreply-адрес GitHub
Нужен резервный способ входа	сохраните коды восстановления при включении 2FA
Забыт пароль	используйте официальное восстановление доступа, не сторонние сервисы

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Signing up for a new GitHub account](#)
[About two-factor authentication](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Имя пользователя становится частью адреса каждого репозитория — выбирайте осознанно.
- Приватный noreply-адрес GitHub скрывает личный email из истории коммитов.
- Двухфакторная аутентификация стоит того, чтобы включить её сразу, а не откладывать.

HTTPS, SSH и аутентификация

Браузер, Git по HTTPS и Git по SSH подтверждают личность тремя разными способами — курс использует SSH.

SafeSort · Часть 1 из 6Git и GitHub

Открыть github.com в браузере и работать с Git из терминала — два разных способа подтвердить, что это действительно вы, и у них разные механизмы.



Пароль напрямую для git push больше не работает
Раньше можно было использовать пароль аккаунта прямо при git push по HTTPS — GitHub отключил этот способ. Сейчас HTTPS требует personal access token через credential helper или вход через GitHub CLI (`gh auth login`), а не пароль напрямую.

HTTPS	SSH
URL вида <code>https://github.com/OWNER/REPO.git</code>	URL вида <code>git@github.com:OWNER/REPO.git</code>

HTTPS	SSH
Работает почти везде, включая сети со строгим firewall	Может требовать открытый порт 22
Аутентификация через credential helper / GitHub CLI	Аутентификация через пару SSH-ключей

Курс использует **SSH** как основной путь: один раз настроенная пара ключей работает для любого числа репозиториев без повторного ввода токена — а следующая страница проведёт через настройку целиком. Это не значит, что SSH объективно лучше во всех случаях: в сети с жёстким firewall, блокирующим порт 22, HTTPS может быть единственным рабочим вариантом.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[About authentication to GitHub](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Вход в браузере, Git по HTTPS и Git по SSH — три разных механизма аутентификации.
- Пароль аккаунта напрямую для git push больше не работает — только token или SSH-ключ.
- Курс использует SSH как основной путь; HTTPS остаётся рабочей альтернативой.

Настраиваем SSH

Приватный ключ остаётся на компьютере и доказывает личность, не покидая его; публичный ключ уходит на GitHub.

SafeSort · Часть 1 из 6 **Git и GitHub**



SSH-аутентификация строится на паре ключей: приватный остаётся на вашем компьютере и никогда никому не передаётся, публичный — загружается в настройки GitHub. GitHub проверяет, что у вас есть приватная половина пары, не видя её саму.

Создаём ключ

Документированная команда — см. официальную инструкцию GitHub

```
ssh-keygen -t ed25519 -C "you@example.com"
```

На вопрос о файле для сохранения обычно достаточно нажать Enter (путь по умолчанию), а парольную фразу (passphrase) стоит задать — это дополнительная защита ключа на диске.

Добавляем ключ в ssh-agent

Документированная команда

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

Добавляем публичный ключ в GitHub

Содержимое файла `~/.ssh/id_ed25519.pub` копируется в Settings → SSH and GPG keys → New SSH key.

Проверяем соединение

Документированная команда

```
ssh -T git@github.com  
# Hi USERNAME! You've successfully authenticated, but  
GitHub does not provide shell access.
```

Это сообщение — не ошибка

GitHub не предоставляет интерактивную оболочку по SSH — фраза «does not provide shell access» означает ровно то, что она говорит: соединение и аутентификация прошли успешно, именно это и было целью проверки.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[About SSH](#)

[Generating a new SSH key and adding it to the ssh-agent](#)

[Adding a new SSH key to your GitHub account](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Приватный ключ никогда не покидает компьютер; на GitHub загружается только публичный.
- `ssh-keygen` создаёт пару, `ssh-add` добавляет её в `ssh-agent` на время сессии.
- `ssh -T git@github.com` проверяет соединение — сообщение об отсутствии `shell`-доступа означает успех.

Создаём репозиторий SafeSort на GitHub

Настоящий репозиторий `Cartesian-School/safesort` — тот, что используется во всей оставшейся части главы.

SafeSort · Часть 1 из 6 **Git и GitHub**

ГІТНВВ НАСТОЯЩІЙ РЕПОЗИТОРІЙ ЭТОЙ ГЛАВЫ

Учётная запись готова, аутентификация настроена — пора создать настоящий репозиторий для SafeSort. Именно этот репозиторий, `Cartesian-School/safesort`, используется на всех оставшихся страницах главы: каждый Issue, каждая ветка, каждый Pull Request и финальный релиз, которые здесь показаны, — реальные.

Главная страница реального репозитория `Cartesian-School/safesort` на GitHub: файлы, README, история коммитов, вкладки Issues/Pull requests/Actions/Projects

Реальный репозиторий SafeSort — тот, что используется во всей оставшейся части главы.

Поля формы создания репозитория

Поле	Что оно значит
Owner	аккаунт или организация, которой принадлежит репозиторий
Repository name	часть адреса github.com/OWNER/ИМЯ — короткое, без пробелов
Description	одна строка, видна в списке репозиториях и в поиске
Public / Private	виден ли репозиторий всем или только тем, кого вы пригласили
Add a README	создать ли стартовый README.md сразу при создании
.gitignore template	готовый шаблон исключений для конкретного языка
License	лицензия, под которой распространяется код — например, MIT

Пустой репозиторий или сразу с README?

Есть два пути: создать репозиторий с README/.gitignore/лицензией сразу на GitHub — или создать его пустым и запустить туда уже готовый локальный проект. Эта глава идёт вторым путём: пустой репозиторий на GitHub, а первый коммит (README, LICENSE, .gitignore, pyproject.toml) приходит с локальной машины — так с

самого начала понятно, что именно легло в историю первым коммитом, а не появилось «само» через веб-интерфейс.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

Creating a new repository

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Cartesian-School/safesort — настоящий репозиторий, используемый во всей оставшейся части главы.
- Repository name становится частью адреса; Public/Private определяет видимость.
- Курс создаёт пустой репозиторий на GitHub и пушит в него готовый локальный первый коммит.

Клонируем репозиторий

`git clone` создаёт полную локальную копию репозитория разом — файлы, всю историю и связь с GitHub.

SafeSort · Часть 1 из 6 **Git и GitHub**

Репозиторий существует на GitHub — но, чтобы писать код, его нужно получить на свой компьютер. `git clone` скачивает репозиторий целиком, вместе со всей его историей коммитов, и сразу настраивает связь с GitHub как `origin` (следующая страница разберёт это подробнее).



`git clone` — единственная команда, разом создающая полную локальную копию

```
~/safesort $ git clone https://github.com/
Cartesian-School/safesort.git
Cloning into 'safesort'...
~/safesort $ cd safesort
~/safesort $ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
~/safesort $ git log --oneline -5
fe610cf docs: fill in CHANGELOG for 0.1.0
(#23)
c376fba feat: add command-line interface (#21)
01989e0 feat: add duplicate detection with
byte-level confirmation (#20)
29c6328 feat: record operation manifest and
add undo (#19)
```

```
c5e34d5 feat: add explicit apply operation
(#18)
```

Как эталонный репозиторий Cartesian-School/safesort выглядит сейчас — с уже завершённой историей

Эталонный репозиторий подготовлен для курса заранее

`git log --oneline` сразу после клонирования показывает не пустую историю, а все коммиты, которые уже есть в репозитории на GitHub, — клонирование скачивает полную историю, а не только последнее состояние файлов. И это не пустой репозиторий: Cartesian-School/safesort подготовлен для курса заранее, со всеми Issue, ветками, Pull Request и релизом v0.1.0 уже сделанными. Учебные шаги этой главы показывают, как такой проект строится с нуля, но настоящая история эталонного репозитория — не покадровая запись этих шагов, а её собственная, отдельная и полностью честная последовательность реальных коммитов. Ниже по главе явные пометки различают «РЕАЛЬНОЕ СВИДЕТЕЛЬСТВО РЕПОЗИТОРИЯ» (то, что действительно есть в этой истории) и «УЧЕБНЫЙ ПРИМЕР» (иллюстрация приёма на отдельном тренировочном материале).

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Cloning a repository](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- `git clone` скачивает репозиторий целиком — файлы и всю историю коммитов, не только последнее состояние.
- После клонирования `git status` сразу показывает связь с `origin` — GitHub уже настроен как удалённый репозиторий.
- `git log --oneline` после клонирования сразу показывает настоящую историю проекта.

Локальный и удалённый репозиторий

`origin` — просто имя для удалённого репозитория, которое `git clone` назначает по умолчанию.

SafeSort · Часть 1 из 6 **Git и GitHub**

`origin` — не специальный сервер и не зарезервированное слово Git, а просто имя, которое `git clone` дал одному конкретному удалённому репозиторию по умолчанию. Ничто не мешает называть удалённые репозитории иначе — но `origin` для «того, откуда всё началось» настолько общепринято, что почти никто не выбирает другое имя без веской причины.

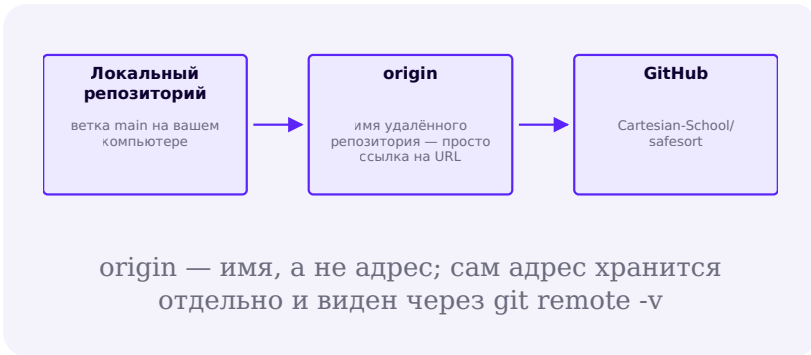


```
~/safesort $ git remote -v
origin https://github.com/Cartesian-School/
safesort.git (fetch)
origin https://github.com/Cartesian-School/
safesort.git (push)
```

Ваш компьютер: локальный репозиторий

⇌ push / pull ⇌

GitHub **GitHub: удалённый репозиторий**



Один локальный репозиторий может иметь несколько удалённых — например, «origin» для основного репозитория и «upstream» для оригинала. **Fork** — это ваша собственная копия чужого репозитория на GitHub: её можно свободно менять, не затрагивая оригинал. Такая связка origin/upstream пригодится, если в домашней практике вы решите работать через собственный fork `python-mini-projects` — это один из двух равноценных вариантов практики, наравне с созданием отдельного репозитория с нуля (см. приложение).

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[About remote repositories](#)

[Managing remote repositories](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- `origin` — имя удалённого репозитория, которое `git clone` назначает по умолчанию, а не специальное слово `Git`.
- `git remote -v` показывает настоящий URL, стоящий за именем `origin`.
- У одного локального репозитория может быть несколько удалённых — например, `origin` и `upstream` при работе с `fork`.

Ветка, HEAD и `origin/main`: как Git понимает, где мы находимся

Сначала отделим текущую локальную ветку от локального знания о GitHub. Тогда `switch`, `fetch`, `pull` и `push` перестанут быть набором магических команд.

SafeSort · Часть 1 из 6 Git и GitHub

Перед первой рабочей веткой соберём одну цельную модель: где лежит изменение, на какой коммит указывает текущая ветка и что именно Git обновляет при `fetch`, `pull` и `push`.



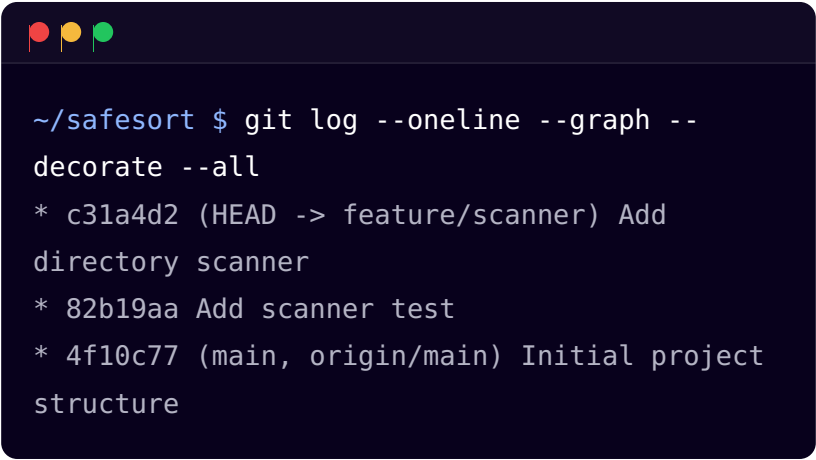
Файл остаётся в `working tree`: `git add` не перемещает его, а фиксирует выбранное содержимое в индексе

`git add` не переносит файл в другое место. Рабочий файл остаётся на диске и может снова измениться. Команда помещает в индекс снимок его *текущего содержимого*, который станет частью следующего коммита.

Коммиты образуют граф

Каждый коммит знает родительский коммит. Поэтому история похожа не на папку с версиями, а на граф.

Веткой называют подвижное имя, указывающее на один коммит. **HEAD** обычно указывает на выбранную ветку. Новый коммит получает текущий коммит родителем и передвигает эту ветку вперёд.



```
~/safesort $ git log --oneline --graph --
decorate --all
* c31a4d2 (HEAD -> feature/scanner) Add
directory scanner
* 82b19aa Add scanner test
* 4f10c77 (main, origin/main) Initial project
structure
```

HEAD указывает на выбранную `feature/scanner`; `main` и `origin/main` пока остались на предыдущем коммите

```

      C3 feature/scanner ← HEAD
      /
C1—C2 main, origin/main
```

ВЕТКА, HEAD И ORIGIN/MAIN: КАК GIT ПОНИМАЕТ, ГДЕ МЫ НАХОДИМСЯ

Имя	Где живёт	Что означает
main	локальный репозиторий	ваша локальная ветка
origin/main	локальный репозиторий	последнее известное Git состояние удалённой main
main на GitHub	удалённый репозиторий	реальная ветка на сервере GitHub

`origin/main` не показывает состояние GitHub в реальном времени. Это локальная remote-tracking ссылка, которая меняется, когда Git получает сведения с сервера, например после `git fetch`.



fetch обновляет знание о remote; pull добавляет интеграцию; push движется в обратную сторону

```
~/safesort $ git status
On branch main
Your branch is up to date with 'origin/main'.
~/safesort $ git branch -vv
* main 4f10c77 [origin/main] Initial project structure
```

```
~/safesort $ git fetch origin
~/safesort $ git log --oneline --graph --
decorate --all
```

Команда	Что показывает
git status	какие файлы изменены, какие уже в staging, какие Git вообще не отслеживает
git diff	построчные изменения в рабочем дереве, ещё не добавленные в staging
git diff --staged	построчные изменения, уже добавленные в staging — то, что попадёт в коммит
git branch -vv	локальные ветки, текущую ветку и их upstream-связи
git log --oneline --graph --decorate --all	граф коммитов и положения ссылок

Коротко

- Веткой называют подвижную ссылку на коммит; HEAD обычно указывает на текущую выбранную ветку.
- main, origin/main и main на GitHub обозначают три различных состояния, а не три написания одного объекта.
- fetch обновляет remote-tracking ссылки; pull добавляет интеграцию; push обновляет удалённую ветку.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Git Branching — Branches in a Nutshell](#)

[git-fetch](#)

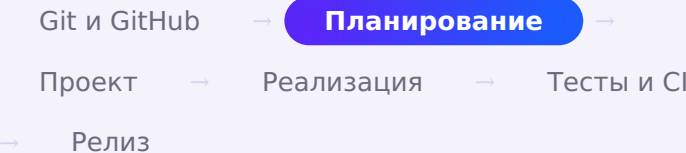
[git-pull](#)

[git-push](#)

Repository и GitHub Project — в чём разница

Repository хранит код; GitHub Project отслеживает статус задач — два разных, независимых объекта.

SAFESORT · ЧАСТЬ 2 ИЗ 6



Часть I уже разделила Git и GitHub. Здесь — вторая частая путаница: репозиторий и **GitHub Project** — тоже разные, независимые понятия, и легко решить, что раз оба слова начинаются с «проект», это одно и то же.

Repository	GitHub Project
код, файлы, история коммитов	задачи, их статус, представления (доска/таблица)
ветки, теги, Pull Request	может объединять Issues сразу из нескольких репозиториях

Repository	GitHub Project
отвечает на вопрос «что уже сделано»	отвечает на вопрос «что нужно сделать и в каком порядке»
обязателен — без него нет кода	необязателен — можно работать вообще без Project



Один Project может охватывать несколько репозиториев

GitHub Project — не часть репозитория и не находится «внутри» него: это отдельный объект, который может показывать Issues и Pull Request сразу из нескольких репозиториев одной организации. Для SafeSort в этой главе используется один Project на один репозиторий — но так бывает не всегда.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[About Projects](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Repository хранит код и историю; GitHub Project отслеживает статус задач — это разные объекты.
- Project не обязателен: можно вести репозиторий вообще без него.
- Один Project может объединять Issues из нескольких репозиториях организации.

Как спланировать Project: лучшие практики

Project легко создать за секунду и так же легко захлестнуть десятком неиспользуемых полей — несколько решений стоит принять заранее.

SafeSort · Часть 2 из 6 **Планирование**

Создать Project — секундное дело; ошибка, которую совершает почти каждый новичок, — сразу добавить десяток полей и три представления «на будущее», а через месяц забросить большинство из них. Прежде чем нажать «New project», стоит принять несколько решений осознанно.

Решить область действия заранее

Project можно привязать к одному репозиторию или объединить в нём несколько. Для SafeSort ответ простой — один Project на один репозиторий, потому что вся работа этой главы происходит в `Cartesian-School/safesort` и нет смысла тянуть в один список задачи из других репозиториях курса.

Меньше полей — лучше

Вместо	Лучше
поле «на всякий случай», которое никто не заполняет	поле, отвечающее на конкретный вопрос (кто важнее, что за часть кода)
свободный текст там, где вариантов на самом деле немного	Single select с заранее известным списком значений
новое представление для каждой идеи «а вдруг пригодится»	одно-два представления, которые реально открывают каждый день

Статус — почти всегда достаточно одного поля для отслеживания прогресса

У GitHub Project уже есть встроенное поле Status. Прежде чем добавлять что-то ещё, стоит спросить: отвечает ли новое поле на вопрос, которого Status не покрывает? Для SafeSort это Priority (что делать в первую очередь) и Area (к какой части кода относится задача) — оба поля появятся дальше в этой части главы.

Поля и представления можно менять позже

Ничего из решённого на этом шаге не высечено в камне: GitHub позволяет добавить, переименовать или удалить поле и представление в любой момент, не теряя уже собранные данные. Ошибка новичка — не «выбрать неправильное поле», а решить всё сразу и никогда не пересматривать.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

Best practices for Projects

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Перед созданием Project стоит решить его область действия — один репозиторий или несколько.
- Меньше полей и представлений, каждое из которых реально используется, лучше десятка «на будущее».
- Status — встроенное поле; собственные поля стоит добавлять только когда Status не отвечает на нужный вопрос.

Создаём GitHub Project

Прежде чем писать код, GitHub Project даёт место для списка задач, связанного с реальными Issues репозитория.

SafeSort · Часть 2 из 6Планирование

Прежде чем писать код SafeSort, полезно составить список того, что нужно сделать, — GitHub Project даёт для этого готовое место, связанное с реальными Issues репозитория.

Что заполняется при создании

Поле	Значение
Owner	организация или аккаунт — здесь Cartesian-School
Title	например, «SafeSort — первый релиз»

Поле	Значение
Template	пустой Project или один из готовых шаблонов (Board, Roadmap...)
Visibility	виден ли Project всем или только участникам организации

Именно так и был создан настоящий Project этой главы — «SafeSort — первый релиз» (Cartesian-School, Project №1): Owner — Cartesian-School, Title — без номера версии, Template — пустой Project. Как и многие рабочие доски планирования, этот Project виден только участникам организации Cartesian-School (Visibility из таблицы выше) — прямой ссылки на него в тексте намеренно нет, чтобы не обещать доступ, которого у читателя нет; всё его содержимое показано ниже на реальных скриншотах.

Table-представление настоящего Project «SafeSort — первый релиз»: все 14 Issues, поля Status, Priority, Area

Настоящий Project SafeSort — первый релиз, созданный под организацией Cartesian-School.

Название без номера версии

Название Project для этой главы — **«SafeSort — первый релиз»**, без номера 0.1.0 : часть VI введёт версии только тогда, когда первая версия действительно будет готова, и называть Project по номеру раньше времени — забегать вперёд без необходимости.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Creating a project](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- GitHub Project создаётся для конкретного владельца (организации или аккаунта) с названием и уровнем видимости.
- Название Project для SafeSort не включает номер версии — версия появляется позже.
- Сразу после создания Project пуст — следующие разделы показывают его представления и поля, а затем, как в него попадают задачи.

Board, Table и Roadmap

Board, Table и Roadmap — разные способы посмотреть на один и тот же набор задач, а не отдельные наборы данных.

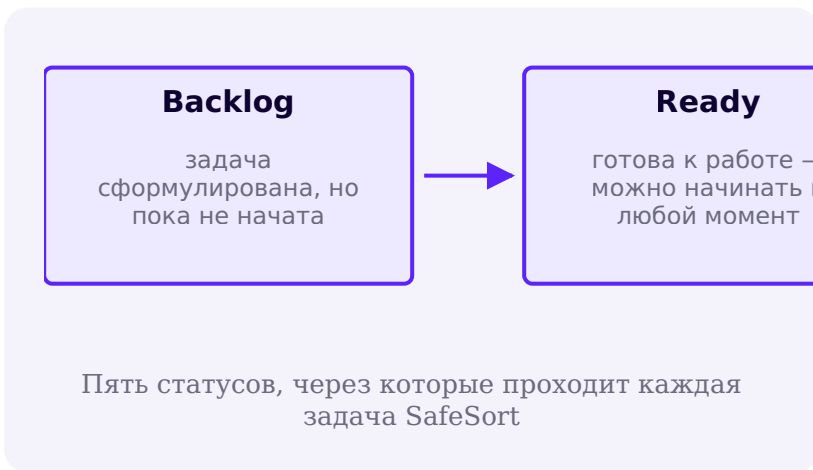
SafeSort · Часть 2 из 6Планирование
Один и тот же набор задач можно смотреть по-разному — GitHub Project называет это **представлениями** (views): один набор элементов, несколько способов на него посмотреть.

Представление	Когда полезно
Board	видеть прогресс по колонкам-статусам: Backlog / Ready / In Progress / In Review / Done
Table	видеть все задачи и их поля сразу — сортировать, фильтровать, группировать
Roadmap	видеть задачи на временной шкале — полезно при датах и дедлайнах

Не каждому проекту нужны все представления

Для SafeSort в этой главе хватает Board (видно, что происходит прямо сейчас) и Table (видно все задачи целиком). Roadmap имеет смысл, когда у задач есть даты начала и окончания, — здесь этого нет, и добавлять его не нужно только потому, что GitHub его предлагает.

Статусы для SafeSort



Board настоящего Project SafeSort: пять колонок-статусов, все 14 задач в Done

Настоящая доска Project SafeSort — все 14 задач уже в Done, потому что релиз 0.1.0 уже вышел.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ[Changing the layout of a view](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Board, Table и Roadmap — представления одного и того же набора задач, а не отдельные наборы данных.
- SafeSort использует пять статусов: Backlog, Ready, In Progress, In Review, Done.
- Представление стоит добавлять только тогда, когда им реально будут пользоваться.

Поля Project: встроенные и пользовательские

Часть полей Project встроена в GitHub, часть можно добавить самим — под конкретный вопрос, который они должны решать.

SafeSort · Часть 2 из 6**Планирование**
Каждый элемент Project — это набор полей: часть из них встроена в GitHub и есть у любого Project, часть можно добавить самим под конкретную задачу.

Встроенные поля

Поле	Откуда берётся
Title	заголовок Issue или Pull Request
Assignees	кому назначена задача в самом Issue
Status	колонка Board — управляет-ся Project, не репозиторием
Labels	метки Issue/PR из репозито-рия

Поле	Откуда берётся
Repository	какой репозиторий, если Project объединяет несколько
Linked pull requests	PR, связанные с Issue через «Closes #N»

Эти поля Project не придумывает сам — он либо читает их из репозитория (Title, Assignees, Labels), либо управляет ими только внутри себя (Status).

Типы пользовательских полей

Тип поля	Когда использовать
Text	короткая произвольная заметка — например, ссылка на обсуждение
Number	числовое значение — например, оценка сложности в часах
Date	конкретная дата — например, дедлайн, если он есть
Single select	фиксированный список значений — то, что выбирают из выпадающего списка
Iteration	повторяющиеся отрезки времени — спринты, недели

Два пользовательских поля SafeSort

Поле	Тип	Значения
Priority	Single select	High / Medium / Low
Area	Single select	Packaging / CLI / Filesystem / Safety / Duplicates / Testing / Documentation / CI

Оба поля — Single select, а не Text: список значений заранее известен и конечен, а Single select ещё и позволяет группировать и фильтровать Table по значению, чего свободный текст не даёт. Iteration здесь не нужен — SafeSort не ведётся спринтами, у задач нет повторяющихся временных отрезков.

Table настоящего Project SafeSort с заполненными колонками Priority и Area для всех 14 задач

Priority и Area заполнены для всех 14 задач настоящего Project SafeSort.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Understanding fields](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Встроенные поля (Title, Assignees, Status, Labels) есть у любого Project без настройки.
- Пользовательские поля бывают текстом, числом, датой, Single select или Iteration.
- SafeSort использует два поля Single select — Priority и Area — потому что список их значений заранее известен и конечен.

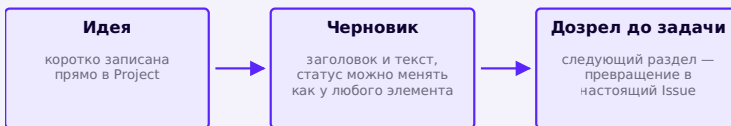
Черновики: задача без Issue

Черновик (draft issue) фиксирует мысль внутри Project раньше, чем она станет формальным Issue репозитория.

SafeSort · Часть 2 из 6Планирование

Не каждая мысль о будущей задаче сразу созревает до полноценного Issue с Problem, Expected outcome и чек-листом Acceptance criteria. Для таких промежуточных заметок у Project есть **черновик** (draft issue) — элемент с заголовком и текстом, который существует только внутри Project и пока не связан ни с одним репозиторием.

Issue	Черновик (draft issue)
живёт в конкретном репозитории	живёт только внутри Project
у него есть номер (#14 и т.д.)	номера нет — это ещё не запись репозитория
виден в списке Issues репозитория	виден только тем, у кого есть доступ к Project
можно связать Pull Request через «Closes #N»	связать Pull Request нельзя, пока не станет Issue



Черновик — способ зафиксировать мысль в Project раньше, чем она станет формальным Issue

У SafeSort черновиков не было — но приём стоит знать

Все 14 задач SafeSort с самого начала были сформулированы достаточно чётко, чтобы сразу стать полноценными Issues (следующие разделы), — черновики в их истории не использовались. Но приём полезен в проектах, где работа идёт не заранее спланированным списком, а по ходу дела: увидели проблему во время разработки — сразу записали черновиком в Project, не отвлекаясь на формулировку полноценного Issue.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Adding items to your project](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

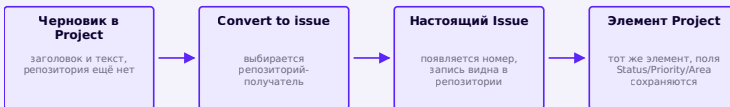
- Черновик (draft issue) — элемент Project без репозитория и без номера, для мыслей, которые ещё не дозрели до Issue.
- У черновика есть заголовок, текст и статус, но нет связи с Pull Request.
- SafeSort обошёлся без черновиков, потому что все 14 задач были сформулированы сразу как Issues.

Превращаем черновик в Issue

Convert to issue превращает черновик в настоящий Issue репозитория одним действием — без ручного копирования текста.

SafeSort · Часть 2 из 6 **Планирование**

Когда черновик созрел до понятной задачи, его можно превратить в настоящий Issue — одним действием прямо из Project, без копирования текста вручную.



Конвертация меняет тип элемента, но не создаёт новый — это тот же элемент Project

Поля не сбрасываются при конвертации

Если у черновика уже был выставлен статус или заполнено поле Priority, после конвертации в Issue эти значения остаются как есть — конвертация меняет только тип элемента (черновик → Issue) и добавляет связь с репозиторием, а не пересоздаёт элемент с нуля.

Репозиторий выбирается в момент конвертации

У черновика изначально нет репозитория — GitHub спрашивает, в какой репозиторий добавить будущий Issue, только когда происходит конвертация. Для Project с несколькими репозиториями это осознанный выбор; для SafeSort, где Project привязан к одному репозиторию, ответ всегда один и тот же — `Cartesian-School/safesort`.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ[Converting draft issues to issues](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Convert to issue превращает черновик в настоящий Issue одним действием, без ручного копирования текста.
- Уже заполненные поля элемента (Status, Priority и другие) сохраняются после конвертации.
- Репозиторий-получатель выбирается в момент конвертации — у черновика его до этого не было.

Создаём Issues и добавляем в Project

Каждая часть SafeSort формулируется как Issue до того, как написана хоть одна строка кода.

SafeSort · Часть 2 из 6**Планирование**

Прежде чем писать код, каждая часть SafeSort формулируется как **Issue** — запись, описывающая задачу, до того как появилась хоть одна строка кода.

Часть Issue	Пример для SafeSort
Title	«Add directory scanner»
Problem	SafeSort не умеет находить файлы в каталоге
Expected outcome	scan() обходит каталог и возвращает список FileInfo
Acceptance criteria	чек-лист: принимает Path, исключает .git/.venv, не следует по символическим ссылкам, покрыт тестами

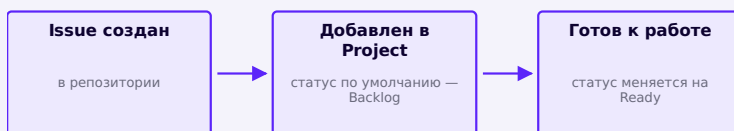
Все 14 задач SafeSort действительно оформлены так в реальном репозитории — не отредактированы задним числом, а созданы как формулировка задачи до начала работы над ней:

Список Issues репозитория Cartesian-School/safesort: 14 закрытых задач, каждая — от scanner до релиза

Настоящие 14 Issues репозитория SafeSort.

Один Issue — не всегда один Pull Request

В репозитории SafeSort 9 из 14 Issues закрыты Pull Request — но не «каждый своим»: коллизии имён (Issue №6) оказались достаточно тесно связаны с планом перемещений (Issue №3), чтобы попасть в один PR №17; то же с манифестом и undo (Issues №7 и №8 — один PR №19). Остальные пять Issues (обработка ошибок файловой системы, настройки и логирование, тестовый набор, CI, подготовка релиза) не имеют собственного выделенного PR: эта работа вошла в код по мере реализации соответствующих модулей и была закрыта записью, объясняющей, где именно она оказалась — реальная история проекта, а не выдуманная ради красивой таблицы «14 Issues → 14 PR».

Issue добавляется в Project

Issue существует в репозитории независимо от Project; добавление в Project — отдельное, необязательное действие

Все 14 Issues репозитория SafeSort действительно добавлены в настоящий Project «SafeSort — первый релиз» (виден только участникам организации, как и отмечено в начале этой части) — это видно на скриншотах Board и Table в следующих разделах этой части главы.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ[About issues](#)[Creating an issue](#)[Adding items to your project](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

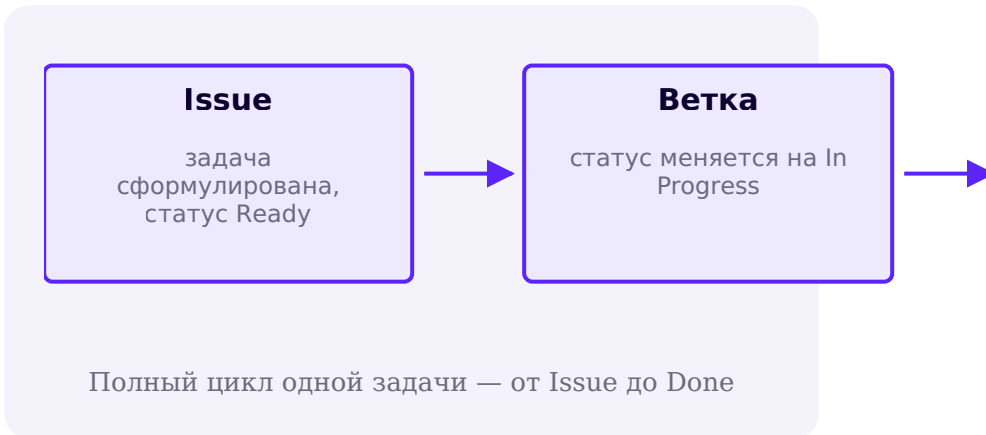
- Issue формулирует задачу — что нужно сделать и как проверить результат — до того, как написан код.
- У каждого Issue SafeSort есть Title, Problem, Expected outcome и чек-лист Acceptance criteria.
- Issue существует в репозитории независимо от Project; добавление в Project — отдельный шаг.

Первый цикл: Issue → Branch → Pull Request

Issue → ветка → код и тесты → Pull Request → CI → слияние — цикл, который повторяется для большинства задач SafeSort, но не по жёсткому правилу «один Issue — один PR».

SafeSort · Часть 2 из 6 **Планирование**

Issue сформулирован — теперь разберём полный цикл его жизни, от постановки задачи до закрытия. Этот цикл повторяется для большинства задач SafeSort, начиная со следующей части главы — но не по жёсткому правилу «один Issue — один Pull Request», а так, как реально удобно ложится работа.



Терминал

```
git switch -c feat/directory-scanner
# ...пишем код и тесты, коммитим изменения...
git push -u origin feat/directory-scanner
```

После push откройте репозиторий в браузере, нажмите **Compare & pull request**, выберите base: main, добавьте заголовок и строку Closes #1 в описание. git работает с историей, а создание Pull Request выполняется в интерфейсе GitHub.

Closes #N закрывает Issue автоматически — а один PR может закрыть сразу несколько

Если тело Pull Request содержит фразу вида Closes #1, GitHub автоматически закрывает Issue №1 в момент слияния этого PR — вручную закрывать Issue не нужно. Ничто не мешает написать Closes #3, Closes #6, если один PR решает сразу две тесно связанные задачи — именно так и произошло в репозитории SafeSort с планом перемещений и обработкой конфликтов имён.

Не каждый Issue закрывается через PR

Формулировка задачи через Issue не обязывает закрывать её именно Pull Request'ом. Если работа была сделана попутно, в рамках другой задачи, или не требовала кода вовсе, Issue можно закрыть вручную — с комментарием, объясняющим, что произошло. Реальная история проекта важнее искусственно ровной таблицы «N задач — N PR».

Следующая часть главы проходит этот цикл по-настоящему, шаг за шагом, для первой реальной задачи — сканера каталогов.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ[About pull requests](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

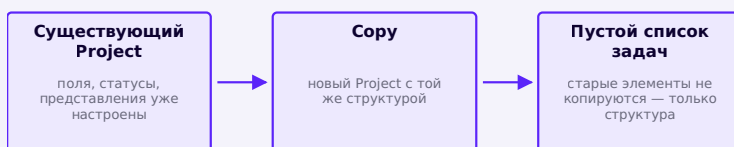
- Issue → ветка → код и тесты → Pull Request → CI и проверка → слияние — полный цикл одной задачи.
- «Closes #N» в описании Pull Request закрывает Issue автоматически при слиянии.
- Этот цикл описывает большинство задач SafeSort, но не жёсткое правило — один PR иногда закрывает несколько тесно связанных Issues, а часть задач закрывается без отдельного PR.

Необязательно: копируем существующий Project

Сору Project переносит структуру — поля, статусы, представления, — но не переносит сами задачи.

SafeSort · Часть 2 из 6 **Планирование**

Создать Project с нуля — не единственный способ начать. Если в организации уже есть Project с нужным набором статусов, полей и представлений, GitHub позволяет **скопировать** его: новый Project получает ту же структуру, но пустой список задач — прошлые Issues, Pull Request и история статусов не переносятся.



Копирование переносит структуру Project, а не его содержимое

SafeSort создаётся с нуля — копировать пока нечего

У Cartesian-School на момент подготовки этой главы ещё не было готового Project с нужной структурой, поэтому Project «SafeSort — первый релиз» создаётся заново (предыдущий раздел), а не копированием. Но у копирования есть реальное применение для курса: после того как этот Project будет готов, он сам может стать заготовкой для планирования следующих больших проектных глав — не нужно будет заново придумывать статусы Backlog / Ready / In Progress / In Review / Done и поля Priority / Area.

Когда копирование окупается

Копирование полезно, когда команда уже выработала удобный набор статусов и полей и не хочет каждый раз собирать его заново, — типичный случай: несколько похожих релизов подряд или несколько похожих учебных

проектов один за другим. Для первого Project в организации копировать попросту не с чего, и создание с нуля — не компромисс, а единственный доступный путь.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

Copying an existing project

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Копирование Project переносит его структуру — поля, статусы, представления — но не сами задачи.
- SafeSort создан с нуля, потому что подходящего Project для копирования ещё не существовало.
- Копирование окупается, когда одна и та же структура нужна для нескольких похожих проектов подряд.

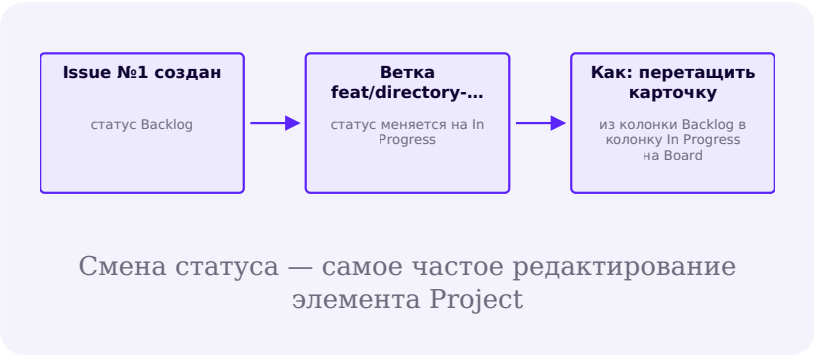
Необязательно: редактируем элементы Project

Статус, приоритет и другие поля элемента можно менять по одному, через панель или сразу массово в Table.

SafeSort · Часть 2 из 6 **Планирование**

Задача редко остаётся неизменной от Backlog до Done — статус, приоритет, а иногда и заголовок меняются по ходу работы. У Project для этого есть три способа, разной степени массовости.

Способ	Когда удобен
Перетащить карточку на Board	поменять только статус одного элемента — самый быстрый способ
Открыть панель элемента	изменить сразу несколько полей одного элемента — Priority, Area, Assignees
Массовое изменение в Table	выделить несколько строк и применить одно значение поля сразу ко всем



Все 14 реальных задач SafeSort уже в статусе Done — релиз 0.1.0 вышел, менять их статус назад ради красивого скриншота значило бы врать про историю проекта. Поэтому ниже — настоящая демонстрация на отдельном демо-элементе, специально добавленном и проведённом через Ready → In Progress → In Review → Done в реальном Project SafeSort, а затем удалённом, чтобы не искажать фактическое состояние доски:

Демо-элемент в колонке Ready настоящего Project SafeSort

1. Ready — демо-карточка добавлена и готова к работе.

Демо-элемент перетащен в колонку In Progress

2. In Progress — та же карточка перетащена в следующую колонку.

Демо-элемент перетащен в колонку In Review

3. In Review — карточка снова перетащена, теперь в предпоследнюю колонку.

Демо-элемент перетащен в колонку Done, счётчик Done стал 15

4. Done — карточка дошла до последней колонки; счётчик Done временно показывает 15 (14 реальных задач + демо-элемент).

Массовое редактирование экономит время на однотипных задачах

Если несколько Issues одновременно переходят в одну и ту же фазу — например, все учебные задания **Практика 23-20–Практика 23-23** готовы к работе одновременно, то выделение нескольких строк в Table и разовое изменение поля Status для всех выделенных быстрее, чем открывать каждый элемент по отдельности.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Editing items in your project](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Перетаскивание карточки на Board — самый быстрый способ изменить статус одного элемента.
- Панель элемента позволяет менять сразу несколько полей одного элемента за раз.
- Массовое редактирование в Table применяет одно значение поля сразу ко всем выделенным элементам.

Необязательно: фильтруем, сортируем и группируем

Фильтр оставляет подмножество элементов, сортировка меняет порядок строк, группировка раскладывает весь список по секциям.

SafeSort · Часть 2 из 6Планирование
Table и Board показывают все элементы Project сразу, но по мере роста списка задач нужен способ увидеть только нужную часть — для этого у представлений есть фильтрация, сортировка и группировка.

Операция	Что делает	Пример для SafeSort
Фильтр	оставляет только элементы, подходящие под условие	показать только Priority: High
Сортировка	меняет порядок строк по значению поля	отсортировать Table по Priority
Группировка	разбивает элементы на секции по значению поля	сгруппировать по Area — все задачи Filesystem вместе

Фильтр и группировка решают разные задачи

Фильтр убирает лишнее и оставляет подмножество — удобно, когда интересна только часть списка (например, только задачи со статусом In Progress прямо сейчас). Группировка не убирает ничего — она организует весь список по разделам, так что видно сразу все задачи, но разложенные по Area или по Priority.

Фильтр — это строка запроса

Фильтр в Project записывается как текстовый запрос вида `status:"In Review" area:CLI` — его можно ввести вручную в строке фильтра или собрать через выпадающие подсказки. Тот же синтаксис используется и в поиске по Issues репозитория, так что навык переносится.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ[Filtering projects](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Фильтр оставляет только элементы, подходящие под условие — остальные временно скрываются из представления.
- Сортировка меняет порядок строк; группировка раскладывает весь список по значению поля, не убирая элементы.
- Фильтр записывается как текстовый запрос вида `status:"In Review"` — тот же синтаксис, что и в поиске по Issues.

Необязательно: управляем представлениями

Настроенные фильтр, сортировку и группировку можно сохранить как именованное представление со своей вкладкой.

SafeSort · Часть 2 из 6Планирование

Настроенные фильтр, сортировку и группировку не нужно собирать заново каждый раз — их можно сохранить как отдельное представление (view) со своей вкладкой и именем.



Действие	Результат
Создать представление	новая вкладка со своим макетом, фильтром и группировкой
Дублировать представление	копия существующей вкладки — удобно как отправная точка для похожей настройки
Переименовать представление	меняет только подпись вкладки

Действие	Результат
Изменить порядок вкладок	перетаскивание вкладок местами

Board и Table для SafeSort — это два сохранённых представления одного Project

Board (статусы по колонкам) и Table (все поля сразу) не дублируют данные — это два разных способа посмотреть на один и тот же список из 14 задач. Изменение элемента в одном представлении сразу видно в другом.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Managing your views](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Представление можно сохранить с именем и своей вкладкой — фильтр и группировка не нужно настраивать заново.
- Board и Table — два сохранённых представления одного и того же списка задач, а не отдельные копии данных.
- Представления можно дублировать, переименовывать и переставлять местами.

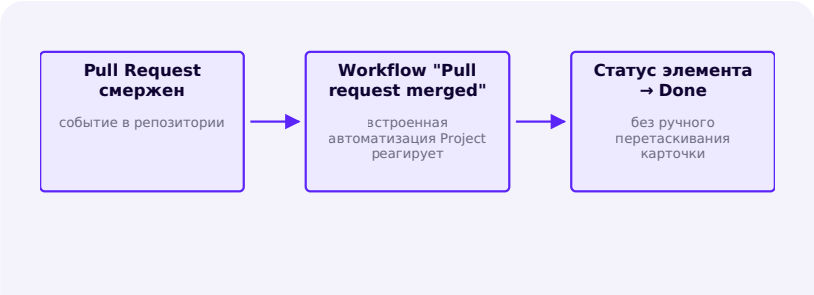
Необязательно: встроенная автоматизация и auto-add

Встроенные workflow сами меняют статус элемента по событиям репозитория и сами добавляют новые Issues в Project.

SafeSort · Часть 2 из 6**Планирование**

Часть переходов между статусами не нужно делать руками — у Project есть встроенные автоматизации (built-in workflows), которые реагируют на события в репозитории.

Встроенный workflow	Что делает
Item added to project	выставляет статус по умолчанию новому элементу
Item reopened	возвращает элемент в заданный статус, если Issue перекроет
Item closed	переводит элемент в статус Done, когда Issue или PR закрыт
Pull request merged	переводит элемент в статус Done при слиянии Pull Request
Auto-add to project	автоматически добавляет в Project новые Issues и PR, подходящие под фильтр
Auto-archive items	архивирует элементы, подходящие под фильтр — например, всё в статусе Done



Встроенные workflow реагируют на события репозитория и сами меняют статус

В настоящем Project SafeSort сработал похожий, но не тот же workflow — все 14 Issues добавлялись уже закрытыми, поэтому включился **Item closed**, а не «Pull request merged»:

Настроенный workflow Item closed → Status: Done в настоящем Project SafeSort

Реальный workflow, который автоматически перевёл все 14 задач в Done.

Auto-add — чтобы не добавлять Issues вручную

Workflow **Auto-add to project** настраивается фильтром — например, «любой новый Issue в репозитории Cartesian-School/safesort ». С таким правилом каждый новый Issue сам попадает в Project со статусом по умолчанию, и не нужно помнить о ручном шаге «добавить в Project» из более раннего раздела главы. Именно так настроен и включён Auto-add в настоящем Project SafeSort:

Включённый workflow Auto-add to project с фильтром is:issue для репозитория safesort

Реальный, включённый Auto-add — фильтр is:issue уже находит все 14 существующих Issues.

Автоматизация не заменяет решения — она освобождает от рутины

Auto-add избавляет только от механического действия «перетащить Issue в Project». Решение о том, в каком статусе должна оказаться задача дальше — Ready она уже или ещё Backlog, — по-прежнему принимает человек.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Using the built-in automations](#)

[Adding items automatically](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Встроенные workflow меняют статус элемента в ответ на событие репозитория — например, слияние Pull Request.
- Auto-add to project автоматически добавляет в Project новые Issues и PR, подходящие под настроенный фильтр.
- Автоматизация убирает рутинные действия, но не решения о том, в каком статусе должна быть задача.

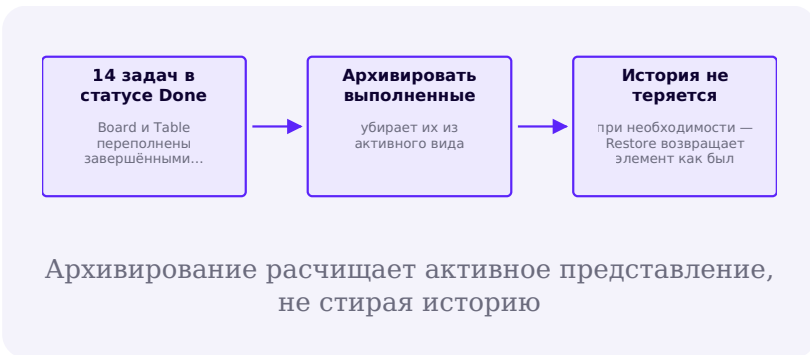
Необязательно: архивируем и восстанавливаем элементы

Архивирование расчищает активное представление от завершённых задач, не стирая их данные и историю.

SafeSort · Часть 2 из 6 **Планирование**

Когда задача закрыта и попала в статус Done, оставлять её карточку на Board навсегда не обязательно — но и удалять запись из Project тоже не стоит. Для этого есть промежуточный шаг: **архивирование**.

Действие	Что происходит
Архивировать	элемент исчезает из активных представлений, но данные сохраняются и его можно восстановить
Восстановить	элемент возвращается во все представления с теми же полями, что были до архивации
Удалить	элемент удаляется из Project безвозвратно — сам Issue в репозитории при этом не затрагивается



Архивирование можно автоматизировать

Workflow **Auto-archive items** из предыдущего раздела делает то же самое по фильтру автоматически — например, архивирует любой элемент, как только его статус становится Done, чтобы Board оставался читаемым без ручной уборки.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Archiving items from your project](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Архивирование убирает элемент из активных представлений, но сохраняет его данные и позволяет восстановить.
- Удаление, в отличие от архивирования, стирает элемент из Project безвозвратно — сам Issue в репозитории не затрагивается.
- Auto-archive items автоматизирует архивирование завершённых задач по фильтру.

Необязательно: шаблоны Project

Project, помеченный шаблоном, появляется в галерее заготовок организации при создании нового Project.

SafeSort · Часть 2 из 6**Планирование**
Организация может пометить готовый Project как **шаблон** — тогда его структура (поля, статусы, представления, но не сами задачи, как и при копировании из более раннего раздела) появляется в списке заготовок, доступных при создании нового Project любому участнику организации.

Сору существующего Project	Project-шаблон
копируется один раз, вручную выбранный Project	виден в галерее шаблонов при создании любого нового Project
нужно знать, какой именно Project копировать	не нужно искать — шаблон предлагается сразу в интерфейсе

Сору существующего Project	Project-шаблон
доступно для любого Project, на который есть права	требуется, чтобы владелец явно включил флаг «сделать шаблоном»

Потенциальный следующий шаг для Cartesian-School, не часть текущей главы

Project «SafeSort — первый релиз» решает конкретную задачу этой главы и сам по себе шаблоном пока не помечен. Но его структура — статусы Backlog/Ready/In Progress/In Review/Done и поля Priority/Area — подошла бы и другим проектным главам курса; пометить его шаблоном для организации Cartesian-School — разумный будущий шаг, а не то, что нужно SafeSort прямо сейчас.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Managing project templates in your organization](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

Коротко

- Project-шаблон появляется в галерее заготовок при создании нового Project — не нужно искать, что копировать.
- Шаблоном становится Project, для которого владелец организации явно включил эту настройку.
- SafeSort пока не помечен шаблоном — это не нужно для текущей главы, но подходит как будущий шаг курса.

Необязательно: Insights и графики Project

Insights строит графики прямо из полей Project — необязательная, более продвинутая возможность.

SafeSort · Часть 2 из 6 **Планирование**

Последний раздел про механику Project — необязательный и более продвинутый: **Insights** строит графики прямо из данных Project, без экспорта в сторонний инструмент.



Настройка графика	Пример для SafeSort
Группировка	по Status — сколько задач в каждой колонке прямо сейчас
Фильтр	например, только Area: CI — сколько задач относится к настройке CI
Тип графика	столбчатая диаграмма, круговая диаграмма и другие — зависит от вопроса

Полезно для больших списков задач, необязательно для четырнадцати

Insights раскрывает свою пользу, когда элементов в Project много и вручную посчитать распределение по статусам или областям неудобно. При 14 задачах SafeSort это распределение видно и просто посмотрев на Board — Insights здесь скорее демонстрация возможности, чем необходимость.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[About insights for Projects](#)

[Creating charts](#)

Перевод и учебная адаптация официальной документации GitHub. Исходный материал — CC BY 4.0.

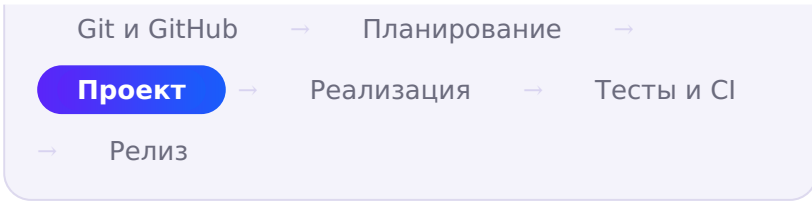
Коротко

- Insights строит графики прямо из полей Project — без экспорта данных в сторонний инструмент.
- График настраивается группировкой, фильтром и типом визуализации.
- Польза Insights растёт вместе с количеством задач — на 14 элементах распределение видно и на глаз.

Первый коммит в клонированном репозитории

После клонирования в рабочем дереве уже есть стартовый набор файлов — первый коммит репозитория SafeSort.

SAFESORT · ЧАСТЬ 3 ИЗ 6



Репозиторий склонирован (часть I), Issues и GitHub Project разобраны (часть II) — самое время заглянуть внутрь каталога, который появился после `git clone`, и понять, что в нём уже есть.

```
~/safesort $ ls -la
CHANGELOG.md
.git/
.github/
.gitignore
LICENSE
pyproject.toml
README.md
src/
tests/
~/safesort $ cat .gitignore
__pycache__/
*.pyc
.venv/
dist/
build/
*.egg-info/
```

The image shows a terminal window with a dark background. The first command executed is `~/safesort $ ls -la`, which lists the contents of the current directory. The output shows several files and directories: `CHANGELOG.md`, `.git/`, `.github/`, `.gitignore`, `LICENSE`, `pyproject.toml`, `README.md`, `src/`, and `tests/`. The second command is `~/safesort $ cat .gitignore`, which displays the contents of the `.gitignore` file. The output lists patterns to be ignored: `__pycache__/`, `*.pyc`, `.venv/`, `dist/`, `build/`, and `*.egg-info/`.

```
.pytest_cache/  
.safesort/
```

safesort/

.git/

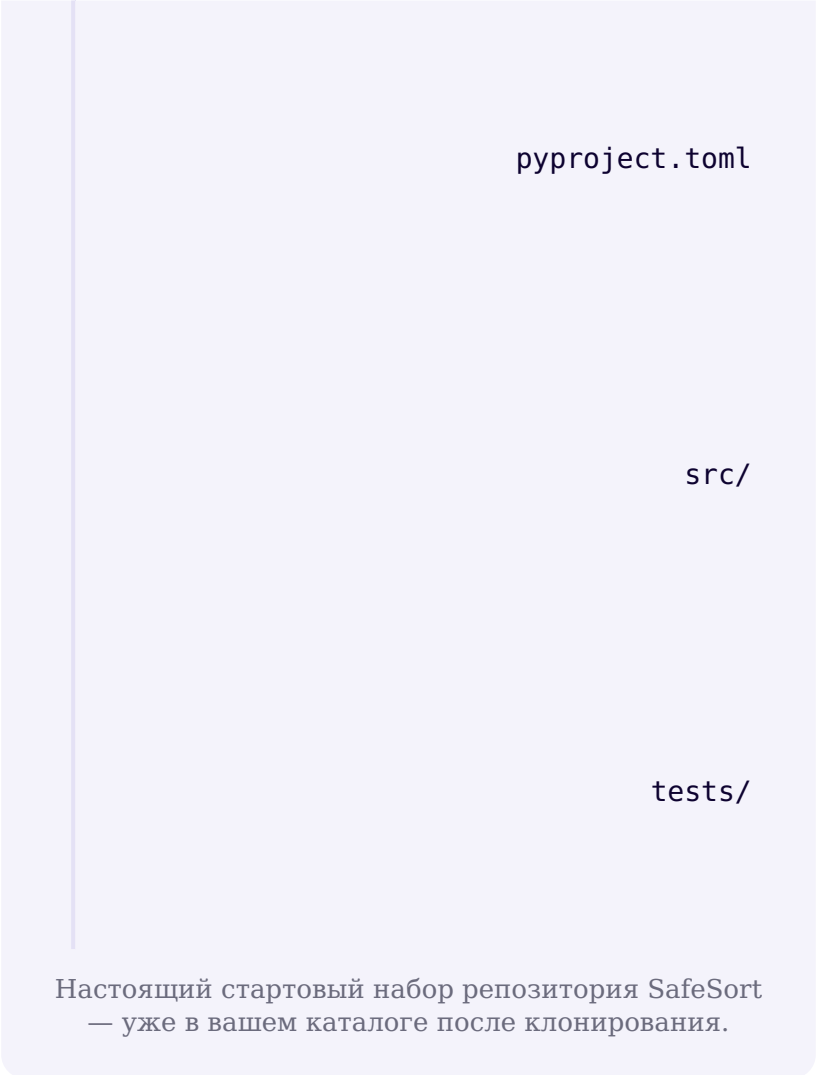
.github/

`.gitignore`

`README.md`

`LICENSE`

`CHANGELOG.md`

A diagram showing a project directory structure. It consists of a light purple rounded rectangle containing three labels: 'pyproject.toml' at the top right, 'src/' in the middle right, and 'tests/' at the bottom right. A thin vertical purple line is on the left side of the rectangle.

pyproject.toml

src/

tests/

Настоящий стартовый набор репозитория SafeSort
— уже в вашем каталоге после клонирования.

Всё это — первый коммит репозитория, `chore: initial project scaffold`: рабочее дерево (файлы на диске) плюс `.git` (история изменений) — вместе их и называют **репозиторием**. Каждый следующий раздел этой части добавляет к этому набору содержание — README дополняется, появляется Python-пакет внутри `src/`, а

`.gitignore` уже сейчас исключает служебные файлы (кеш байткода, виртуальное окружение, сборочные каталоги), чтобы они не засоряли историю.

Коротко

- `git clone` сразу приносит рабочее дерево — файлы на диске — и `.git` — историю изменений; вместе это и есть репозиторий.
- У SafeSort уже есть стартовый набор: README, LICENSE, CHANGELOG, `pyproject.toml`, `.gitignore` — первый коммит репозитория.
- `.gitignore` перечисляет то, что Git не должен отслеживать: временные и сгенерированные файлы.

Первый README проекта

README отвечает на вопросы «что это?» и «как запустить?» ещё до того, как человек откроет код.

SafeSort · Часть 3 из 6 **Проект**

safesort/

A screenshot of a file explorer interface. At the top right, there is a label ".git/". Below it, in the center, is a light blue rounded rectangle containing the text "README.md" followed by the word "НОВОЕ" in blue. Below this rectangle, there is a text block explaining the purpose of the README.md file.

.git/

README.md НОВОЕ

README.md — один из файлов стартового коммита, увиденного в предыдущем разделе.

README — первый файл, который видит человек, открывший репозиторий: GitHub автоматически показывает его содержимое на главной странице проекта. В клонированном репозитории README.md уже существует и уже закоммичен — это часть стартового коммита `chore: initial project scaffold`, который был показан в предыдущем разделе. Сейчас он совсем короткий:

README.md

SafeSort

SafeSort — программа для безопасной сортировки файлов по папкам.

Раз файл уже отслеживается Git, `git status` в чистом рабочем дереве ничего не покажет — нечего добавлять и нечего коммитить. Интересное начнётся, когда в файл внесут изменения: именно так README и растёт по ходу главы — дополняется разделами, а не переписывается с нуля. Добавим первый такой раздел:

README.md

```
# SafeSort
```

```
SafeSort — программа для безопасной сортировки файлов по папкам.
```

```
## Установка
```

```
pip install -e .
```

```
~/safesort $ git diff
```

```
diff --git a/README.md b/README.md
```

```
index f5a2610..d60da39 100644
```

```
--- a/README.md
```

```
+++ b/README.md
```

```
@@ -1,3 +1,7 @@
```

```
# SafeSort
```

```
SafeSort — программа для безопасной сортировки файлов по папкам.
```

```
+
```

```
+## Установка
```

```
+
```

```
+pip install -e .
```

```
~/safesort $ git status
M README.md
```

git diff показывает построчную разницу перед коммитом, git status — что именно изменилось с последнего коммита. По мере того как в проекте появляются установка, команды и тесты, в README добавятся такие же короткие разделы Usage и Tests. Полный README проекта, уже дополненный:

[projects/](#)

[python/safesort/README.md](#).

Без придуманных значков

На странице проекта на GitHub иногда встречаются цветные значки (badge) вида «tests: passing». Такой значок честен только тогда, когда его действительно обслуживает настроенная проверка — мы вернёмся к этому, когда подключим GitHub Actions. Вставлять его раньше значит показывать то, чего на самом деле ещё нет.

Коротко

- README.md — первое, что видит человек, открывший репозиторий на GitHub.
- README можно дополнять по мере роста проекта, а не писать целиком с первого раза.
- git diff показывает построчные изменения, git status — какие файлы затронуты.

Планируем структуру Python-пакета

src-раскладка: пакет лежит в `src/safesort/`, а не в корне репозитория — и почему это осознанный выбор, а не догма.

SafeSort · Часть 3 из 6 **Проект**

Хаотичная структура каталогов не мешает программе работать, но мешает её понимать, тестировать и устанавливать как пакет. Пока код не написан, зафиксируем только каркас — и добавим в него по одному файлу за раз, когда для него появится задача.

`safesort/` **НОВОЕ**

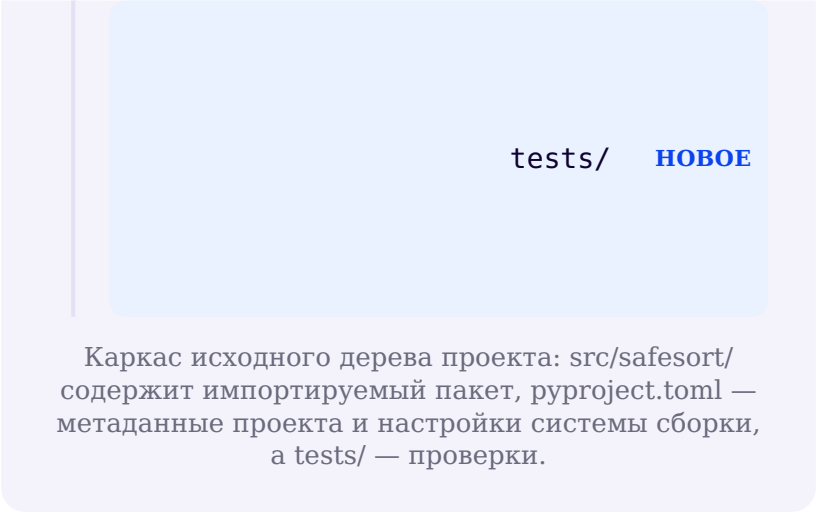
`README.md`

pyproject.toml **HOB0E**

src/ **HOB0E**

safesort/ **HOB0E**

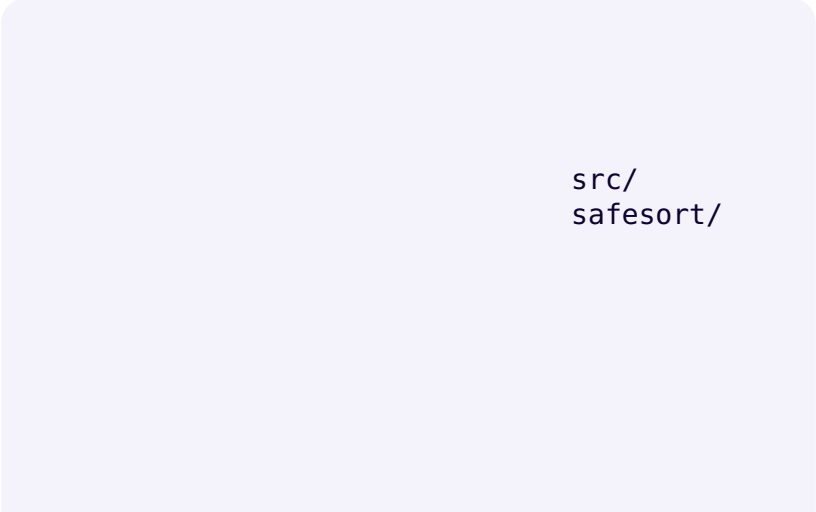
__init__.py **HOB0E**



```
tests/ НОВОЕ
```

Каркас исходного дерева проекта: `src/safesort/` содержит импортируемый пакет, `pyproject.toml` — метаданные проекта и настройки системы сборки, а `tests/` — проверки.

Исходный код лежит внутри `src/`, а не прямо в корне репозитория — это называют **src-раскладкой** (`src layout`). Без этого уровня установленный пакет и каталог исходников совпадали бы, и было бы легко по ошибке протестировать не тот код, что реально установлен — например, если забыть переустановить пакет после правки файла.



```
src/  
safesort/
```

`__init__.py`

`scanner.py`

`classifier.py`

`planner.py`

`executor.py`

`cli.py`

`duplicates.py`

`manifest.py`

config.py

Эти файлы появятся позже, когда для них появится задача — запоминать их сейчас не нужно.

Коротко

- Пакет пока состоит только из README.md, pyproject.toml, src/safesort/__init__.py и tests/.
- src-раскладка кладёт исходный код в src/safesort/, а не в корень репозитория, — так тесты не могут случайно обратиться к установленному коду.
- Остальные модули появятся по одному, каждый — когда для него будет конкретная задача.

pyproject.toml и установка проекта

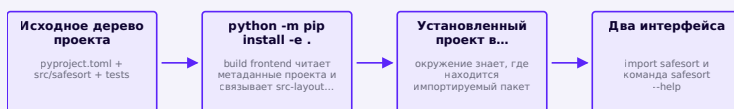
src/safesort импортируется, pyproject.toml описывает Проект, а editable install связывает его с текущим окружением.

SafeSort · Часть 3 из 6 **Проект**

Здесь легко смешать четыре разных объекта.

Импортируемый пакет src/safesort/ содержит `__init__.py` и модули, которые Python загружает

командой `import safesort`. **Проект** (project) — библиотека или приложение, предназначенное для упаковки; **исходное дерево проекта** (project source tree) — его файлы на диске: `pyproject.toml`, `src/`, `tests/` и документация. **Дистрибутивный пакет** (distribution package) — устанавливаемый выпуск проекта; когда речь идёт о конкретном файле, это архив дистрибутива: wheel или sdist. **Установленный проект** — то, что появляется в окружении Python после установки: файлы импортируемого пакета плюс метаданные, по которым инструменты вроде `pip` и `importlib.metadata` узнают о нём.



`pyproject.toml` не превращает каталог в `import package`: он хранит метаданные проекта и настройки, по которым из исходного дерева собирают архивы дистрибутива (wheel/sdist)

При `src-layout` запуск Python из корня репозитория сам по себе не добавляет `src/` в `sys.path`. Поэтому до установки `import safesort` обычно завершится `ModuleNotFoundError`. Это ожидаемое свойство раскладки, а не признак того, что `__init__.py` «не сработал».

```
pyproject.toml
```

```
[project]
name = "safesort"
version = "0.1.0"
description = "A safe, non-destructive command-line
file organizer."
dependencies = []

[project.scripts]
safesort = "safesort.cli:main"
```

Поле `version` обязательно идентифицирует текущую версию пакета. Пока просто запишем `0.1.0` как номер первой учебной версии. Как устроены номера версий и почему здесь три числа, разберём ближе к завершению проекта.

Строка `safesort = "safesort.cli:main"` говорит инструменту установки: после установки создай в окружении команду `safesort`, которая вызывает функцию `main()` из модуля `safesort.cli`. Этого модуля пока не существует; мы напомним его на следующей странице.

Почему `dependencies` остаётся пустым

У SafeSort нет ни одной обязательной сторонней зависимости: `pathlib`, `argparse`, `hashlib`, `shutil` и `json` входят в стандартную библиотеку Python.

Устанавливаем проект в редактируемом режиме

Редактируемая установка (`editable install`) связывает окружение Python с исходным кодом проекта напрямую: изменения в файлах `src/safesort/` становятся видны сразу, без повторной установки.

Терминал (окружение активировано)

```
python -m pip install -e ".[dev]"
python -c "import safesort; print(safesort.__file__)"
safesort --help
```

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Python Tutorial — Packages](#)

[Packaging Python Projects](#)

[Distribution package vs. import package](#)

[src layout vs flat layout](#)

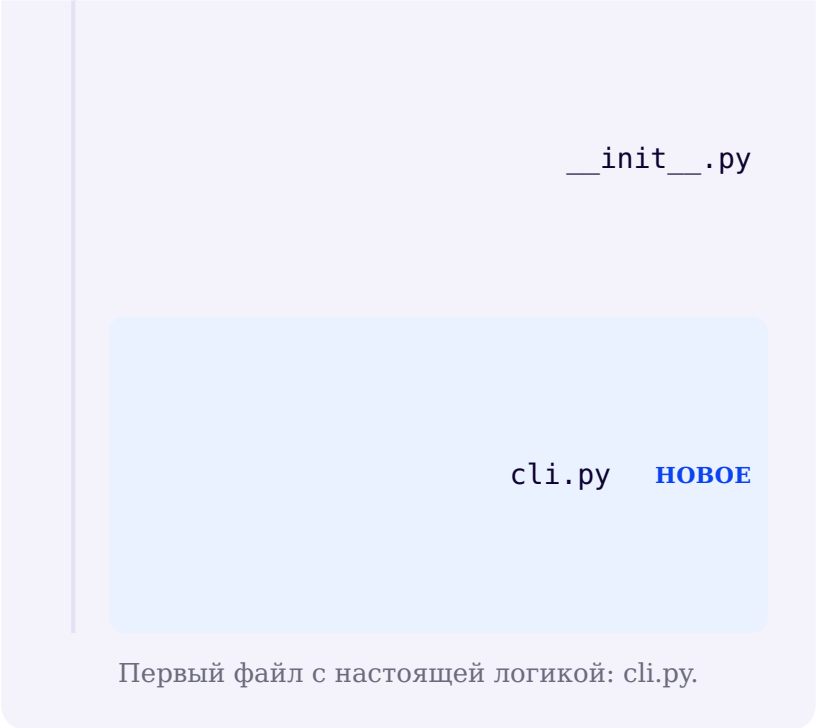
[Writing pyproject.toml](#)

Командная строка SafeSort

argparse разбирает пять подкоманд SafeSort и связывает каждую с отдельной функцией-обработчиком.

SafeSort · Часть 3 из 6 **Проект**

```
src/
safesort/
```



```
__init__.py
```

```
cli.py  НОВОЕ
```

Первый файл с настоящей логикой: cli.py.

SafeSort будет управляться пятью подкомандами: `scan`, `plan`, `apply`, `duplicates` и `undo`. За разбор аргументов командной строки отвечает модуль `argparse` из стандартной библиотеки — он же формирует текст `--help`.

src/safesort/cli.py

```
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="safesort",
        description=(
            "SafeSort: a safe, non-destructive file
organizer. "
            "scan/plan/duplicates are read-only;
'apply' sorts and 'undo' restores files."
        ),
    )
    subparsers =
parser.add_subparsers(dest="command", required=True)

    scan_parser = subparsers.add_parser("scan",
help="...")
    scan_parser.add_argument("root", type=Path,
help="Directory to scan.")
    # ...аналогично для plan, apply, duplicates, undo
    return parser
```

add_subparsers(dest="command", required=True)

dest="command" кладёт название вызванной подкоманды в args.command. required=True заставляет argparse самостоятельно вывести понятную ошибку, если пользователь запустит safesort вообще без подкоманды, — писать эту проверку вручную не нужно.

От разобранных аргументов к результату

Каждой подкоманде соответствует одна функция-обработчик, и словарь связывает имя команды с нужной функцией:

```
src/safesort/cli.py
```

```
_HANDLERS = {
    "scan": cmd_scan,
    "plan": cmd_plan,
    "apply": cmd_apply,
    "duplicates": cmd_duplicates,
    "undo": cmd_undo,
}

def main(argv: list[str] | None = None) -> int:
    parser = build_parser()
    args = parser.parse_args(argv)
    handler = _HANDLERS[args.command]
    return handler(args)
```

Такой словарь — тот же приём, что и в игре «Камень, ножницы, бумага» из приложения к этой главе: вместо цепочки `if args.command == "scan": ... elif ...` нужную функцию просто ищут по ключу.

Каждая обработчик-функция возвращает целое число: 0 при успехе, отличное от нуля значение при ошибке — это и есть код возврата программы, который видит операционная система и любой сценарий, вызывающий `safesort` из другого места.

Установим пакет и запустим команду — `argparse` уже формирует текст помощи сам, без единой написанной вручную строки:



```
~/safesort $ safesort --help
usage: safesort [-h]
{scan,plan,apply,duplicates,undo} ...
```


SafeSort: a safe, non-destructive file organizer. scan/plan/duplicates are read-only; 'apply' sorts and 'undo' restores files.

positional arguments:

{scan,plan,apply,duplicates,undo}

scan List files found under ROOT, grouped by category (read-only).

plan Show the moves that would be made under ROOT, without changing anything (read-only).

apply Move files under ROOT into Sorted/<category>/ and

record an undo manifest.

duplicates Report groups of files with identical content under

ROOT (read-only, never deletes).

undo Undo the most recent 'apply' run recorded under ROOT.

options:

-h, --help show this help message and exit

Практика: разбор аргументов командной строки

Интерактивный ноутбук в браузере: Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/23-07/index.html) (<https://www.cartesianschool.org/practice/23-07/index.html>)

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[argparse — Parser for command-line options](#)

Коротко

- `argparse.ArgumentParser` с `add_subparsers()` разбирает пять подкоманд `SafeSort` и сам формирует текст `--help`.
- `dest="command"` кладёт имя вызванной подкоманды в `args.command`; словарь `_HANDLERS` связывает это имя с нужной функцией.
- Каждый обработчик возвращает 0 при успехе и ненулевое значение при ошибке — это код возврата всей программы.

pathlib: работаем с путями и каталогами

`Path` описывает путь объектом со своими операциями. На нём строится модель данных `SafeSort`.

SAFESORT · ЧАСТЬ 4 ИЗ 6

Git и GitHub



Планирование



Проект



Реализация



Тесты и CI



Релиз

Всё, что SafeSort делает с файлами, начинается с путей. Модуль `pathlib` из стандартной библиотеки описывает путь не строкой, а объектом `Path` с собственными операциями.

Строка	Path
<code>"/home/anna/Downloads/report.pdf"</code>	<code>Path("/home/anna/Downloads/report.pdf")</code>
ручной разбор через <code>.split('/')</code>	готовые атрибуты <code>fajl.parent</code> , <code>fajl.name</code> , <code>fajl.suffix</code> , <code>fajl.stem</code>
конкатенация строк для соединения путей	оператор <code>/</code> собирает путь: <code>koren / 'report.pdf'</code>

```
pathlib_osnovy.py
```

```
from pathlib import Path
```

```
koren = Path("~/Downloads").expanduser()
```

```
fajl = koren / "otchet.pdf"
```

```
>>> fajl.name
'otchet.pdf'
>>> fajl.suffix
'.pdf'
>>> fajl.stem
'otchet'
>>> fajl.parent
PosixPath('/home/anna/Downloads')
```

Оператор / для путей

У класса `Path` переопределён оператор `/`: он не делит числа, а склеивает часть пути с новым именем, автоматически подставляя правильный разделитель для текущей операционной системы. Собирать пути строковой конкатенацией (`koren + "/" + "otchet.pdf"`) не нужно.

Модель данных **SafeSort: FileInfo**

Сканер `SafeSort` не ограничивается списком путей. Он описывает каждый найденный файл небольшим неизменяемым объектом:

```
src/safesort/models.py
```

```
@dataclass(frozen=True)
class FileInfo:
    path: Path
    size: int
    extension: str
```

`frozen=True` запрещает переназначать поля уже созданного `FileInfo`. Это свойство *объекта-значения*, а не защита файловой системы. Функция может получать `frozen`-объект и всё равно записывать файлы, обращаться к сети или выполнять другие побочные эффекты.

Неизменяемые данные не означают чистую функцию

`build_plan()` не меняет диск по отдельному контракту: API возвращает план, реализация не вызывает `move/write`, а тест проверяет отсутствие изменений. `frozen=True` лишь не даёт переназначить поля модели. Это разные границы, и обе нужны.

Практика: операции с Path

Интерактивный ноутбук в браузере: Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/23-08/index.html) (<https://www.cartesianschool.org/practice/23-08/index.html>)

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[pathlib — Object-oriented filesystem paths](#)
[dataclasses](#)

Сканируем каталог

`scan()` строго читает файловую систему — обходит каталог рекурсивно и не меняет ни одного файла.

SafeSort · Часть 4 из 6 **Реализация**

GITHUB ISSUE #1 · ПРОЕКТ «SAFESORT — ПЕРВЫЙ РЕЛИЗ»

Add directory scanner

Area: **Filesystem** Priority: **High**

```
git switch -c feat/directory-scanner
```

Первый шаг SafeSort — обойти каталог и составить список файлов. Функция `scan()` — одна из трёх строго читающих команд SafeSort: она только вызывает `iterdir()` и `stat()`, ни разу не создавая, не перемещая и не удаляя ничего на диске.

Downloads/

report.pdf

cat.jpg

archive.zip



Эта цепочка будет расти: следующие страницы добавят в неё новые звенья.

src/safesort/scanner.py

```
def scan(root: Path, config: Config) ->
list[FileInfo]:
    root = Path(root)
    excluded = config.excluded_names()
    results: list[FileInfo] = []
    _scan_dir(root, excluded, results)
    return results
```

Сам обход рекурсивный: функция `_scan_dir()` заходит в каждый подкаталог и вызывает себя снова:

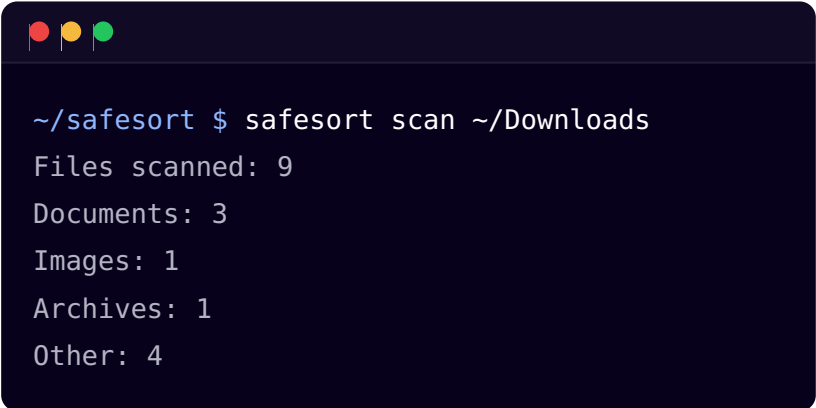
src/safesort/scanner.py

```
for entry in entries:
    if entry.is_symlink():
        continue # символические ссылки пропускаются
    — см. следующую страницу
    if entry.is_dir():
        if entry.name in excluded:
            continue # исключённые каталоги
            пропускаются — см. следующую страницу
        _scan_dir(entry, excluded, results)
    elif entry.is_file():
        size = entry.stat().st_size
        results.append(FileInfo(path=entry, size=size,
            extension=entry.suffix.lower()))
```

Каталог, который нельзя прочитать, не должен остановить всё сканирование

Если для какого-то подкаталога недостаточно прав доступа, `iterdir()` вызывает `PermissionError`. Реализация `_scan_dir()` перехватывает эту ошибку для каждого подкаталога отдельно, записывает предупреждение в журнал и продолжает сканировать остальные каталоги — так один недоступный подкаталог не обрывает весь просмотр целиком. Позже мы рассмотрим это подробнее вместе с остальными ошибками файловой системы.

Проверить сканер можно прямо сейчас — реальный вывод:



```
~/safesort $ safesort scan ~/Downloads
Files scanned: 9
Documents: 3
Images: 1
Archives: 1
Other: 4
```


Тест-чекпойнт: требование становится регрессией

```
tests/test_scanner.py
```

```
def
test_scan_finds_file_without_changing_it(tmp_path):
    source = tmp_path / "report.pdf"
    source.write_bytes(b"report")

    files = scan(tmp_path, Config())

    assert [item.path for item in files] == [source]
    assert source.read_bytes() == b"report"
```

Сначала требование наблюдается на одном примере, затем тест сохраняет его как контракт. Поздняя секция тестирования расширит эту основу fixtures и edge cases.

Практика: сканируем настоящий каталог

Нужен доступ к настоящей файловой системе — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику →](https://www.cartesianschool.org/practice/23-09/index.html) (<https://www.cartesianschool.org/practice/23-09/index.html>)

Какие каталоги не нужно сканировать

Каталог результата и служебный каталог SafeSort исключены всегда — а символические ссылки не отслеживаются вовсе.

SafeSort · Часть 4 из 6 **Реализация**

Downloads/

report.pdf – ✓
сканировать

images – ✓
сканировать/

Sorted – ×
пропустить/

```
.git — x  
пропустить/
```

```
.venv — x  
пропустить/
```

Сканер SafeSort пропускает не только неважные файлы, но и целые каталоги — по двум разным причинам. Первая: некоторые каталоги заведомо не относятся к пользовательским файлам — например, `.git` или `.venv`. Вторая, более тонкая: собственный каталог результата, `Sorted/`, и служебный каталог самого SafeSort, `.safesort/`, должны быть исключены *всегда* — иначе повторный запуск начал бы сортировать уже отсортированные файлы и собственные журналы операций.

```
src/safesort/config.py
```

```
def excluded_names(self) -> frozenset[str]:  
  
    return frozenset(*self.exclude, self.destination,  
                      STATE_DIRNAME})
```

Настраиваемые исключения и обязательные исключения — разные множества

`self.exclude` — список, который пользователь может изменить через файл настроек, который мы рассмотрим позже. Каталог результата (`self.destination`, по умолчанию `Sorted`) и служебный каталог `.safesort` добавляются в исключения динамически, при каждом вызове — их нельзя случайно убрать из списка исключений через настройки.

Символические ссылки: осознанно пропускаются

Символическая ссылка — файл или каталог, который на самом деле указывает на другое место в файловой системе. SafeSort не переходит по символическим ссылкам вообще, ни на файлы, ни на каталоги:

Почему ссылки пропускаются	Что произошло бы иначе
Ссылка может указывать за пределы просканированного каталога	SafeSort мог бы переместить файл, не принадлежащий выбранному каталогу
Ссылка на каталог может создать цикл обхода	Рекурсивное сканирование могло бы никогда не завершиться
Поведение с симлинками не всегда очевидно даже опытным пользователям	Первая версия программы выбирает предсказуемое поведение вместо более сложного

Это решение легко проверить: сканер использует `entry.is_symlink()` раньше любой другой проверки и, если это символическая ссылка, сразу переходит к следующему элементу каталога — эта строка уже встречалась в `_scan_dir()` на предыдущей странице.

Практика: проверка исключённых каталогов

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику →](https://www.cartesianschool.org/practice/23-10/index.html) (<https://www.cartesianschool.org/practice/23-10/index.html>)

ЧЕКПОИНТ · ISSUE #1

```
git commit -m "feat: add directory scanner and configuration"
```

Слился как [Pull Request #15](#). Та же ветка заодно принесла базовый модуль Config, на который позже опираются Issues #10 и #11 (страницы 23-20-23-22).

Статус в реальном Project: **Done**

Определяем категорию файла

Классификация по расширению — практичная эвристика, не доказательство того, что действительно лежит внутри файла.

SafeSort · Часть 4 из 6 **Реализация**

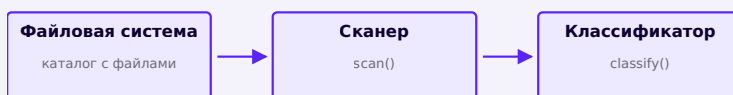
GITHUB ISSUE #2 · PROJECT «SAFESORT — ПЕРВЫЙ РЕЛИЗ»

Add file classification

Area: **Filesystem** Priority: **High**

```
git switch -c feat/classifier
```

Каждому найденному файлу нужно назначить категорию — документы, изображения, видео и так далее. SafeSort определяет категорию по расширению файла: простое и предсказуемое правило, которое покрывает подавляющее большинство практических случаев.



К цепочке присоединяется новое звено — классификатор.

Файл	Категория
report.pdf	documents
photo.jpg	images
backup.zip	archives
unknown.xyz	other

src/safesort/config.py

```

DEFAULT_EXTENSIONS: dict[str, list[str]] = {
    "documents": [".pdf", ".docx", ".txt", ".odt"],
    "images": [".jpg", ".jpeg", ".png", ".webp"],
    "video": [".mp4", ".mkv", ".mov"],
    "audio": [".mp3", ".wav", ".flac"],
    "archives": [".zip", ".tar", ".gz", ".7z"],
    "code": [".py", ".js", ".ts", ".rs", ".java"],
    "data": [".json", ".csv", ".xml"],
}
  
```

```
src/safesort/classifier.py
```

```
OTHER_CATEGORY = "other"

def classify(extension: str, mapping: dict[str,
list[str]]) -> str:
    normalized = extension.lower()
    for category, extensions in mapping.items():
        lowered = {ext.lower() for ext in extensions}
        if normalized in lowered:
            return category
    return OTHER_CATEGORY
```

Классификация по расширению — эвристика, а не доказательство содержимого

Расширение `.pdf` означает лишь то, что имя файла заканчивается на `.pdf` — ничто не мешает переименовать любой файл так, чтобы его расширение не соответствовало реальному формату. SafeSort сознательно не заглядывает внутрь файла: это было бы медленнее, менее предсказуемо и выходит за рамки задачи «разложить файлы по тому, чем они себя называют».

Файл с расширением, которого нет ни в одной категории, попадает в `"other"` — так классификация остаётся полной: у любого файла всегда есть категория, даже если это самая широкая из них.

Практика: `classify()` и собственные категории

Интерактивный ноутбук в браузере: Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/23-11/index.html>)

ЧЕКПОЙНТ · ISSUE #2

```
git commit -m "feat: add file classification"
```

Слился как [Pull Request #16](#) из отдельной самостоятельной ветки, не связанная с остальными Issues.

Статус в реальном Project: **Done**

От анализа к плану действий

План перемещений — обычные данные: его можно вывести на экран, проверить или отбросить, не тронув ни одного файла.

SafeSort · Часть 4 из 6 **Реализация**

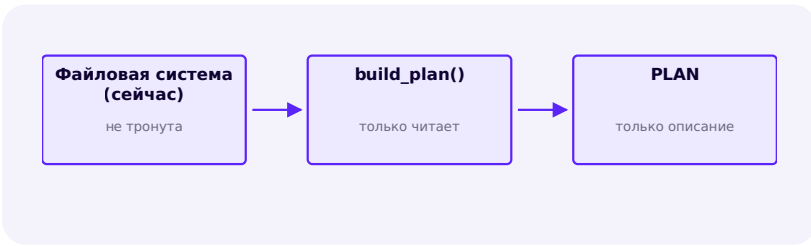
GITHUB **ISSUE #3 · PROJECT «SAFESORT — ПЕРВЫЙ РЕЛИЗ»**

Add safe organization plan

Area: **Safety** Priority: **High**

```
git switch -c feat/plan-and-collisions
```

У SafeSort есть список найденных файлов и правило классификации — но само по себе это ещё не план действий. **План** — список конкретных перемещений: откуда и куда переместится каждый файл, если пользователь подтвердит выполнение.



ТОЛЬКО ЧИТАЮТ

scan / plan / duplicates только читают

МЕНЯЮТ ФАЙЛЫ

apply сортирует; undo восстанавливает

src/safesort/models.py

```
@dataclass(frozen=True)
class MoveOperation:
    source: Path
    destination: Path

@dataclass(frozen=True)
class SortPlan:
    root: Path
    operations: tuple[MoveOperation, ...]
```

План — это данные, а не действие

`SortPlan` не содержит ни одного вызова, который перемещает файлы, — только описание того, что *можно было бы* сделать. Эта мысль — ключевая для всей архитектуры `SafeSort`: пока объект остаётся данными, его можно вывести на экран, проверить тестами, сохранить или просто отбросить, не рискуя ни одним файлом пользователя.

Функция `build_plan()` строит план по списку файлов от сканера, не трогая диск ни разу, кроме одной read-only проверки — существует ли уже файл с таким именем в месте назначения. Что делать, если да, — тема следующего этапа.

```
src/safesort/planner.py
```

```
def build_plan(files: list[FileInfo], root: Path,
               config: Config) -> SortPlan:
    dest_root = Path(root) / config.destination
    operations = []
    for file in files:
        category = classify(file.extension,
                           config.extensions)
        dest_dir = dest_root / category
        candidate = dest_dir / file.path.name
        destination = _resolve_collision(candidate,
                                         reserved)

    operations.append(MoveOperation(source=file.path,
                                    destination=destination))
    return SortPlan(root=root,
                    operations=tuple(operations))
```

Тест-чекпойнт: чистое планирование

```
tests/test_planner.py
```

```
def
test_build_plan_describes_move_without_executing_it(tmp_
    source = tmp_path / "report.pdf"
    source.write_bytes(b"report")
    file = FileInfo(source, source.stat().st_size,
".pdf")

    plan = build_plan([file], tmp_path, Config())

    assert plan.operations[0].destination ==
tmp_path / "Sorted/documents/report.pdf"
    assert source.exists()
    assert not plan.operations[0].destination.exists()
```

Практика: строим план из списка файлов

Интерактивный ноутбук прямо в браузере — Python 3.14
через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/23-12/index.html>)

ЧЕКПОЙНТ · ISSUE #3

```
git commit -m "feat: add safe organization
plan with collision handling"
```

Слился как [Pull Request #17](#). Заголовок PR называет обе вещи сразу: этот же PR закрыл и Issue #6 (обработка конфликта имён), показанную отдельной страницей 23-14, потому что как урок это два разных, самостоятельных понятия, даже если в репозитории они попали в одну ветку.

Статус в реальном Project: **Done**

Режим предварительного просмотра

Команда `plan` — предварительный просмотр без побочных эффектов: тот же план, что и у `apply`, но без единого изменения диска.

SafeSort · Часть 4 из 6 **Реализация**

GITHUB **ISSUE #4 · PROJECT «SAFESORT — ПЕРВЫЙ РЕЛИЗ»**

Add dry-run CLI (scan/plan)

Area: **CLI** Priority: **Medium**

`git switch -c feat/cli`

Команда `plan` — единственное, что нужно сделать с готовым объектом `SortPlan`, чтобы получить полноценный **предварительный просмотр** (dry run): она собирает план и выводит его размер на экран, ни разу не вызывая ничего, что меняет диск.

`src/safesort/cli.py`

```
def cmd_plan(args: argparse.Namespace) -> int:
    root = args.root
    config, code = _load_config(root)
    files = scan(root, config)
    plan = build_plan(files, root, config)
    print(f"{len(plan.operations)} move operations planned.")
    print("No files have been changed.")
    return 0
```



```
~/safesort $ safesort plan ~/Downloads
9 move operations planned.
No files have been changed.
```

Вторая строка вывода — "No files have been changed." — не формальность. Проверим её честно: посчитаем контрольные суммы файлов до и после `plan` и сравним.



```
~ $ sha256sum Downloads/* > before.txt
~ $ safesort plan ~/Downloads
9 move operations planned.
No files have been changed.
~ $ sha256sum Downloads/* > after.txt
~ $ diff before.txt after.txt
```

`diff` не вывел ни строки — файлы действительно не изменились. Позже такую же проверку сделает автоматический тест, а не ручное сравнение.

scan, plan и duplicates используют один и тот же список файлов

Три read-only команды SafeSort — `scan`, `plan` и `duplicates` — начинаются одинаково: вызывают `scan()`, чтобы получить список файлов. Далее их пути расходятся: `plan` строит план, `duplicates` ищет совпадения по содержимому, а `scan` просто считает файлы по категориям.

В истории репозитория cli.py собрали одним PR, в самом конце

Эта книга вводит `cli.py` постепенно: `cmd_plan()` здесь, остальные подкоманды дальше по мере готовности функций, которые они вызывают. В реальном репозитории SafeSort всё было наоборот: `argparse`-обвязка для всех пяти подкоманд (`scan`, `plan`, `apply`, `duplicates`, `undo`) была написана и слита одним PR в самом конце, уже когда все пять функций существовали. Это два разных порядка: порядок, в котором удобно объяснять, и порядок, в котором реально писали код.

Коротко

- `plan` строит тот же `SortPlan`, что и `apply`, но только выводит его размер на экран.
- `scan`, `plan` и `duplicates` начинаются одинаково — с вызова `scan()` — и расходятся дальше.
- Утверждение «файлы не изменены» после `plan` — не текст для красоты, а поведение, которое проверяется автоматическим тестом.

ЧЕКПОЙНТ · ISSUE #4

```
git commit -m "feat: add command-line
interface"
```

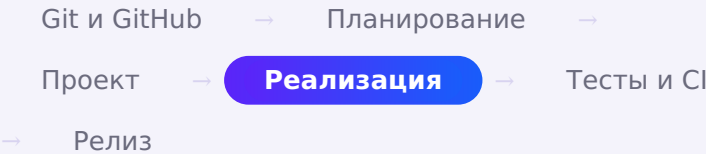
В реальной истории репозитория это [Pull Request #21](#), который слился последним из девяти, после Issue #9 (дубликаты), а не сразу после плана, как в порядке этой книги: к этому моменту в репозитории уже существовали все функции, которые предстоит обвязать интерфейсом командной строки.

Статус в реальном Project: **Done**

Безопасно перемещаем файлы

Прямую сортировку выполняет `apply_plan()`: перед каждым перемещением функция повторно проверяет конфликт. Undo остаётся отдельной обратной операцией.

SAFESORT · ЧАСТЬ 4 ИЗ 6



GITHUB ISSUE #5 · PROJECT «SAFESORT — ПЕРВЫЙ РЕЛИЗ»

Add explicit apply operation

Area: **Safety** Priority: **High**

```
git switch -c feat/apply-operation
```

До сих пор ни одна функция в прямой цепочке сортировки SafeSort не меняла файловую систему. Модуль `executor.py` выполняет прямую операцию: функция `apply_plan()` получает готовый план и перемещает только перечисленные в нём файлы. Обратная операция `undo` тоже перемещает файлы, но восстанавливает их по записанному манифесту.



В прямой цепочке сортировки запись начинается только в executor; undo образует отдельную обратную цепочку.

БЫЛО

Downloads/

report.pdf

photo.jpg

archive.zip



СТАЛО

Downloads/

Sorted/

documents/

report.pdf

images/

photo.jpg



archives/

archive.zip

apply выполняет прямую сортировку; undo позднее
сможет выполнить обратные перемещения по
манифесту.

```
src/safesort/executor.py
```

```
def apply_plan(plan: SortPlan) -> list[CompletedMove]:
    results = []
    for operation in plan.operations:
        source, destination = operation.source,
        operation.destination
        destination.parent.mkdir(parents=True,
        exist_ok=True)

        if destination.exists() or
        destination.is_symlink():
            results.append(CompletedMove(source,
            destination, completed=False,

            error="destination already exists"))
            continue

        shutil.move(str(source), str(destination))
        results.append(CompletedMove(source,
        destination, completed=True))
    return results
```

Партия перемещений — не транзакция базы данных

Если один файл в середине партии не удаётся переместить (например, права доступа изменились между сканированием и выполнением), это не должно остановить обработку остальных файлов и не должно откатывать уже выполненные перемещения — файловая система не умеет так же атомарно откатывать группу операций, как это делает база данных. Поэтому `apply_plan()` обрабатывает каждое перемещение независимо и в конце возвращает честный отчёт о том, что реально произошло с каждым файлом.

Обратите внимание на проверку `destination.exists()` прямо перед перемещением, хотя планировщик уже избегал этого конфликта на этапе построения плана. Между построением плана и его выполнением на диске мог появиться новый файл — например, если

пользователь сам что-то туда положил в этот момент. Без повторной проверки `shutil.move()` в системах на основе POSIX молча перезаписал бы такой файл — а SafeSort не перезаписывает файлы молча ни при каких обстоятельствах.



```
~/safesort $ safesort apply ~/Downloads
Applied 9 moves.
Manifest written to:
.safesort/history/20260824T085400685280.json
```

Тест-чекпойнт: `apply` меняет только заявленный путь

tests/test_executor.py

```
def test_apply_plan_moves_one_file(tmp_path):
    source = tmp_path / "report.pdf"
    destination = tmp_path / "Sorted/documents/
report.pdf"
    source.write_bytes(b"report")
    plan = SortPlan(tmp_path, (MoveOperation(source,
destination),))

    [result] = apply_plan(plan)

    assert result.completed is True
    assert destination.read_bytes() == b"report"
    assert not source.exists()
```

Практика: перемещаем файлы во временном каталоге

Нужен доступ к настоящей файловой системе — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/23-13/index.html>)

ЧЕКПОИНТ · ISSUE #5

```
git commit -m "feat: add explicit apply operation"
```

Слился как [Pull Request #18](#). В описании PR явно указано Closes #5 в описании, поэтому Issue #5 закрылся автоматически в момент слияния.

Статус в реальном Project: **Done**

Что делать, если имя уже занято

Ни на диске, ни внутри одного плана два файла никогда не получают одинаковое имя назначения.

SafeSort · Часть 4 из 6 **Реализация**

GITHUB ISSUE #6 · ПРОЕКТ «SAFESORT — ПЕРВЫЙ РЕЛИЗ»

Handle destination name collisions

Area: **Safety** Priority: **Medium**

```
git switch -c feat/plan-and-collisions
```

Два файла с одинаковым именем могут попасть в одну и ту же категорию — например, два разных `notes.txt` из разных подкаталогов исходного каталога. Если бы SafeSort просто перемещал файл поверх существующего, второй файл исчез бы молча — в каталоге назначения осталось бы одно и то же имя `notes.txt`, но с содержимым только одного из двух файлов, без единого предупреждения об этом.



```
~/safesort $ safesort apply ~/Downloads  
Applied 1 moves.
```

Sorted/
documents/

notes.txt

notes
(1).txt **НОВОЕ**

Файл, который уже был в Sorted/documents/,
остался нетронутым; новый получил свободное
имя.

SafeSort никогда не решает конфликт имён перезаписью:
вместо этого он находит свободное имя по понятной
схеме.

src/safesort/planner.py

```
def _resolve_collision(candidate: Path, reserved:
set[Path]) -> Path:
    if candidate not in reserved and not
candidate.exists():
        return candidate

    stem, suffix, parent = candidate.stem,
candidate.suffix, candidate.parent
    counter = 1
    while True:
        alternative = parent / f"{stem} ({counter})
{suffix}"
        if alternative not in reserved and not
alternative.exists():
            return alternative
        counter += 1
```

Пример

otchet.pdf уже существует в Sorted/documents/
 → следующий otchet.pdf получит имя otchet (1).pdf
 → если и оно занято — otchet (2).pdf, и так далее

reserved — конфликты внутри одного плана, не только на диске

Проверки `not candidate.exists()` достаточно для конфликта с уже существующим файлом на диске — но не для случая, когда *два файла из одного и того же плана* претендуют на одинаковое имя одновременно, пока ни один из них ещё не перемещён. Множество `reserved` запоминает уже занятые в этом плане имена, поэтому второй файл получит `otchet (1).pdf`, даже если на диске пока нет ни одного `otchet.pdf`.

Практика: безопасное имя при конфликте

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/23-14/index.html>)

ЧЕКПОИНТ · ISSUE #6

```
git commit -m "feat: add safe organization
plan with collision handling"
```

Та же ветка и тот же [Pull Request #17](#), который закрыл Issue #3 на странице 23-11. план и обработка конфликта имён были одной, тесно связанной задачей в терминах кода, но остаются двумя разными Issues и двумя разными уроками.

Статус в реальном Project: **Done**

Журнал выполненных операций

Каждый `apply` записывает JSON-манифест того, что действительно произошло — это единственный источник данных для отмены.

SafeSort · Часть 4 из 6 **Реализация**

GITHUB **ISSUE #7 · PROJECT «SAFESORT — ПЕРВЫЙ РЕЛИЗ»**

Record operation manifest

Area: **Safety** Priority: **Medium**

```
git switch -c feat/manifest-and-undo
```

Каждый успешный вызов `apply` оставляет след — файл-манифест в формате JSON, который описывает, что именно было сделано. Без этого журнала команда `undo`, о которой пойдёт речь на следующей странице, не знала бы, что именно нужно отменить.



`apply` не только двигает файлы — он записывает, что именно сделал.

```
.safesort/history/20260824T011640800152.json
```

```
{
  "operation_id": "20260824T011640800152",
  "root": "/home/anna/Downloads",
  "timestamp": "2026-08-24T01:16:40",
  "moves": [
    {
      "source": "/home/anna/Downloads/otchet.pdf",
      "destination": "/home/anna/Downloads/Sorted/
documents/otchet.pdf",
      "completed": true,
      "error": null
    }
  ]
}
```

Путь до манифеста — не случайный: он предсказуемо строится из корня и идентификатора операции, а сам идентификатор — временная метка, отформатированная так, чтобы лексикографический порядок совпадал с хронологическим:

```
src/safesort/manifest.py
```

```
def new_operation_id() -> str:
    return datetime.now().strftime("%Y%m%dT%H%M%S%f")

def history_dir(root: Path) -> Path:
    return Path(root) / STATE_DIRNAME / "history"
```

Почему именно такой формат идентификатора

Строка вида 20260824T011640800152 сортируется как текст точно в том же порядке, в каком операции происходили по времени. Это позволяет функции `find_latest_manifest()` находить последнюю операцию простой сортировкой имён файлов, не читая содержимое каждого манифеста.

Модели `Path` внутри программы нужно превратить в обычные строки, чтобы записать их в JSON — формат, у которого нет типа «путь». Это единственное место в SafeSort, где происходит такое преобразование в обе стороны.

Практика: манифест как обычный JSON

Интерактивный ноутбук прямо в браузере — Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/23-15/index.html) → (https://www.cartesianschool.org/practice/23-15/index.html)

ЧЕКПОИНТ · ISSUE #7

```
git commit -m "feat: record operation manifest and add undo"
```

Слился как [Pull Request #19](#). Заголовок PR показывает, что это одна ветка на две тесно связанные задачи: без манифеста Issue #8 (undo, следующая страница) попросту нечего было бы восстанавливать.

Статус в реальном Project: **Done**

Отмена последней операции

undo восстанавливает файлы из журнала — и отказывается затирать то, что успело появиться на исходном месте.

SafeSort · Часть 4 из 6 **Реализация**

GITHUB **ISSUE #8 · PROJECT «SAFESORT — ПЕРВЫЙ РЕЛИЗ»**

Add undo

Area: **Safety** Priority: **High**

```
git switch -c feat/manifest-and-undo
```

Команда `undo` находит последний манифест, читает список выполненных перемещений и возвращает файлы туда, откуда они были взяты. Здесь действует то же правило, что и во всей программе: **ничего не перезаписывать молча**.



Тот же путь, что и у `apply`, но в обратную сторону.

```
src/safesort/manifest.py
```

```
def undo(manifest: OperationManifest) -> UndoResult:
    restored, conflicts = [], []
    for move in manifest.moves:
        if not move.completed:
            continue
        source, destination = move.source,
        move.destination

        if source.exists() or source.is_symlink():
            conflicts.append(UndoConflict(source,
            destination,
            "a file already exists at the original location"))
            continue

        shutil.move(str(destination), str(source))
        restored.append(CompletedMove(destination,
        source, completed=True))
    return UndoResult(restored=tuple(restored),
    conflicts=tuple(conflicts))
```

Отмена не восстанавливает то, чего не было выполнено

Цикл проверяет `if not move.completed: continue` — перемещения, которые не удались во время `apply`, никогда не касались диска, и отменять для них нечего.

Сначала — обычный успешный откат:



```
~/safesort $ safesort undo ~/Downloads
Restored 9 moves.
```

Конфликт при отмене — реальный сценарий

Что произойдёт, если запустить `undo` ещё раз, когда файлы уже на исходных местах? Простое перемещение назад стёрло бы то, что там уже есть. Вместо этого SafeSort проверяет исходное место *перед* восстановлением каждого файла и, если там уже что-то есть, отказывается восстанавливать именно этот файл:



```
~/safesort $ safesort undo ~/Downloads
ERROR:safesort.manifest:Refusing to undo
Sorted/other/bigfile_b.dat -> bigfile_b.dat: a
file already exists at the original location:
bigfile_b.dat
ERROR:safesort.manifest:Refusing to undo
Sorted/documents/notes.txt -> notes.txt: a
file already exists at the original location:
notes.txt
... (ещё 7 таких строк)
Restored 0 moves.
9 moves could not be restored:
Sorted/other/bigfile_b.dat -> bigfile_b.dat: a
file already exists at the original location:
bigfile_b.dat
... (ещё 8 строк)
```


Отказ — а не тихая перезапись. Каждый конфликт обрабатывается независимо: файлы без конфликта восстановились бы, даже если часть списка отказала.

Тест-чекпойнт: обратная операция

tests/test_manifest.py

```
def test_undo_restores_original_location(tmp_path):
    source = tmp_path / "report.pdf"
    source.write_bytes(b"report")
    plan = build_plan(scan(tmp_path, Config()),
tmp_path, Config())
    moves = apply_plan(plan)
    manifest, _ = write_manifest(tmp_path, moves)

    result = undo(manifest)

    assert len(result.restored) == 1
    assert source.read_bytes() == b"report"
```

Практика: отмена и конфликт при восстановлении

Нужен доступ к настоящей файловой системе — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/23-16/index.html>)

ЧЕКПОЙНТ · ISSUE #8

```
git commit -m "feat: record operation manifest
and add undo"
```

Та же ветка и тот же [Pull Request #19](#), что закрыл Issue #7 на предыдущей странице.

Статус в реальном Project: **Done**

Поиск одинаковых файлов

Дорогое хеширование содержимого делается только там, где оно действительно может изменить ответ — после отбора по размеру.

SafeSort · Часть 4 из 6**Реализация**

GITHUB ISSUE #9 · ПРОЕКТ «SAFESORT — ПЕРВЫЙ РЕЛИЗ»

Add duplicate detection

Area: **Duplicates** Priority: **Medium**

```
git switch -c feat/duplicate-detection
```

Второй крупный компонент SafeSort — поиск файлов с одинаковым содержимым. Задача выглядит просто: если у двух файлов одинаковые байты, они дубликаты. Наивное решение — сравнить содержимое каждого файла с содержимым каждого другого — работает, но для тысяч файлов означает чтение каждого файла снова и снова.

Размер	Файлы
100 KB	a.jpg, b.jpg
240 KB	report.pdf (один файл — дальше не идёт)
3 MB	video.mp4, video-copy.mp4

Дальше в проверку идут только группы из двух и более файлов — одиночный размер сразу отбрасывается, дублировать ему нечего.



Файл с уникальным размером не может быть дубликатом

Если размер файла не совпадает ни с одним другим файлом в просканированном каталоге, у него точно нет дубликата — и вычислять его хеш незачем. Эта проверка почти бесплатна (размер уже известен из FileInfo), а экономит она ровно то, что дороже всего — чтение и хеширование содержимого файлов.

Дальше — сама функция хеширования, а затем то, как результаты хеширования группируются в готовые группы дубликатов и проходят последнее, байтовое подтверждение.

Коротко

- Поиск дубликатов идёт в четыре этапа: по размеру, по хешу, по паре (размер, хеш), затем байтовое подтверждение.
- Хеш вычисляется только для файлов, у которых уже нашёлся хотя бы один файл того же размера.
- `duplicates()` — `read-only` команда: она только сообщает о найденных группах, ничего не удаляя.

SHA-256 и хеш содержимого файла

Одинаковое содержимое всегда даёт одинаковый дайджест SHA-256. Поблочное чтение не требует держать в памяти весь файл разом.

SafeSort · Часть 4 из 6 **Реализация**



Хеш-функция превращает содержимое файла произвольного размера в строку фиксированной длины — **дайджест**. SafeSort использует **SHA-256** из модуля `hashlib`: одинаковые байты всегда дают одинаковый дайджест, а разные дайджесты гарантированно значат разное содержимое. Хеширование **many-to-one**: возможных входов больше, чем 256-битных результатов, поэтому разные входы математически могут иметь один дайджест. Совпадающий дайджест служит сильным фильтром, но не доказывает равенство файлов.

Поскольку SafeSort перемещает настоящие файлы пользователя, он не останавливается на совпадении дайджеста — следующая страница показывает последний шаг, который превращает совпадение хеша в подтверждённый дубликат.

```
src/safesort/duplicates.py
```

```
def sha256_stream(stream: BinaryIO, chunk_size: int =
1024 * 1024) -> str:
    digest = hashlib.sha256()
    while chunk := stream.read(chunk_size):
        digest.update(chunk)
    return digest.hexdigest()
```

Хеширование и шифрование решают разные задачи

SHA-256 спроектирован так, чтобы поиск прообраза по дайджесту был вычислительно неосуществим на практике. Это инженерное свойство стойкости, а не математическое утверждение «обратить невозможно». Хеш-функция помогает ответить на вопрос «может ли это быть одинаковое содержимое», а не скрывает его. SafeSort использует SHA-256 исключительно для сравнения файлов, а не для защиты данных.

При **лавинном эффекте** малое изменение входа обычно меняет много битов выхода; для идеализированного поведения ожидают примерно половину выходных битов. Это не означает, что в каждом опыте обязана измениться каждая шестнадцатеричная цифра.

Зачем читать файл частями

Цикл `while chunk := file.read(chunk_size)` читает файл не целиком, а блоками по мегабайту, и каждый блок сразу добавляет к дайджесту через `digest.update()`. Если бы функция читала файл одним вызовом `file.read()`, для файла в несколько гигабайт программе пришлось бы держать в оперативной памяти всё его

содержимое разом. При поэтапном чтении в памяти в любой момент находится только один блок, независимо от размера файла целиком.

Проверка вручную

```
>>> from pathlib import Path
>>> from safesort.duplicates import sha256_file
>>> sha256_file(Path("otchet.pdf"))
'784cc58b2286b83f67f58ffb1968ca4b80d1d0615863ad9b1ce9c3d'
```

Практика: хеш содержимого по частям

Интерактивный ноутбук в браузере: Python 3.14 через Pyodide, без установки

[Открыть практику](https://www.cartesianschool.org/practice/23-17/index.html) → (https://www.cartesianschool.org/practice/23-17/index.html)

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

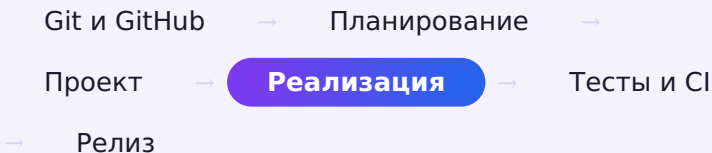
[hashlib](#) — Secure hashes and message digests

[NIST FIPS 180-4](#) — Secure Hash Standard

Находим группы дубликатов

Одна функция превращает список файлов в группы подтверждённых дубликатов и ничего не удаляет.

SAFESORT · ЧАСТЬ 4 ИЗ 6



С хеш-функцией с предыдущей страницы поиск дубликатов группирует файлы сначала по размеру, затем по дайджесту SHA-256. На этом многие реализации останавливаются. SafeSort делает ещё один шаг: прежде чем считать группу подтверждённой, он сравнивает файлы внутри неё побайтово.

src/safesort/duplicates.py

```
def files_equal(path_a: Path, path_b: Path,
                chunk_size: int = DEFAULT_CHUNK_SIZE) -> bool:
    """Финальное подтверждение – совпадающий дайджест
    не гарантия."""
    with path_a.open("rb") as file_a,
         path_b.open("rb") as file_b:
        while True:
            chunk_a = file_a.read(chunk_size)
            chunk_b = file_b.read(chunk_size)
            if chunk_a != chunk_b:
                return False
            if not chunk_a:
                return True

def find_duplicates(files: list[FileInfo]) ->
list[DuplicateGroup]:
    by_size = defaultdict(list)
    for file in files:
        by_size[file.size].append(file)

    groups = []
    for size, candidates in by_size.items():
        if len(candidates) < 2:
            continue
        by_digest = defaultdict(list)
        for candidate in candidates:
            digest = sha256_file(candidate.path)
            by_digest[digest].append(candidate)
        for digest, matched in by_digest.items():
            if len(matched) < 2:
                continue
            exact_groups = []
            for candidate in matched:
```

```

        for exact_group in exact_groups:
            if
files_equal(exact_group[0].path, candidate.path):
                exact_group.append(candidate)
                break
            else:
                exact_groups.append([candidate])
        for exact_group in exact_groups:
            if len(exact_group) >= 2:

groups.append(DuplicateGroup(size=size, digest=digest,
files=tuple(exact_group)))
        return groups

```

Один bucket с одинаковыми размером и дайджестом остаётся только набором кандидатов. Алгоритм сравнивает кандидата с представителем каждой уже найденной группы: при равенстве добавляет его туда, иначе создаёт новый класс. Поэтому даже искусственно вызванная коллизия SHA-256 может дать несколько независимых групп точного содержимого.

Тест-чекпойнт: два одинаковых файла

tests/test_duplicates.py

```

def test_equal_content_forms_one_group(tmp_path):
    a = tmp_path / "a.bin"
    b = tmp_path / "b.bin"
    a.write_bytes(b"same")
    b.write_bytes(b"same")

    groups = find_duplicates(scan(tmp_path, Config()))

    assert len(groups) == 1
    assert {item.path for item in groups[0].files} ==
{a, b}

```


files_equal() — та же идея, что и sha256_file(): читать блоками

Побайтовая сверка читает оба файла кусками по мегабайту и сравнивает кусок с куском, а не загружает файлы в память целиком — то же соображение, что и на предыдущей странице про `sha256_file()`. Как только один из кусков не совпал, сравнение сразу останавливается: незачем дочитывать оставшуюся часть заведомо разных файлов.

Пустые файлы тоже могут быть дубликатами

Два файла нулевого размера побайтово идентичны: у обоих попросту нет байтов. `find_duplicates()` не делает для этого случая никакого исключения — они естественно попадают в одну группу по размеру (0) и в одну группу по дайджесту пустого содержимого. Позже мы проверим это отдельным тестом.

Реальный вывод команды `duplicates` для каталога с тремя парами совпадений:

```
~/safesort $ safesort duplicates ~/Downloads
Found 3 duplicate group(s):
Group 1: 2 files, 2097289 bytes each,
sha256=44f70488694a224316549d0752fe6fdec636df12d
Downloads/bigfile_b.dat
Downloads/bigfile_a.dat
Group 2: 2 files, 0 bytes each,
sha256=e3b0c44298fc1c149afb4c8996fb92427ae41e46
Downloads/empty_b.bin
Downloads/empty_a.bin
```

```
Group 3: 2 files, 28 bytes each,
sha256=f3b439399a805a21e826bbd4111ca4c4615e49293
Downloads/copy_of_notes.txt
Downloads/notes.txt
```

duplicates сообщает о группах и не удаляет файлы

Первая версия программы не удаляет ни один файл из найденной группы, и в коде `find_duplicates()` нет ни одного вызова, который удаляет файлы, даже отключённого или закомментированного.

Автоматическое удаление дубликатов сознательно вынесено за рамки первой версии: решение о том, какой из одинаковых файлов оставить, требует контекста, которого у программы нет.

Практика: группируем файлы в дубликаты

Интерактивный ноутбук в браузере: Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/23-18/index.html>)

ЧЕКПОИНТ · ISSUE #9

```
git commit -m "feat: add duplicate detection
with byte-level confirmation"
```

Слился как [Pull Request #20](#). Три страницы (23-17-23-19) книги соответствуют одному PR в репозитории: группировка по размеру, хеш и байтовое подтверждение относятся к одному Issue, который объяснён по шагам.

Статус в реальном Project: **Done**

Обрабатываем ошибки файловой системы

Конкретные исключения вместо except Exception: программа не скрывает реальные ошибки, а обрабатывает только ожидаемые.

SafeSort · Часть 4 из 6 **Реализация**

GITHUB **ISSUE #10 · PROJECT «SAFESORT — ПЕРВЫЙ РЕЛИЗ»**

Handle filesystem errors

Area: **Filesystem** Priority: **Medium**

Реальная файловая система непредсказуема: файл может исчезнуть между сканированием и чтением, доступ к каталогу может быть запрещён, диск может оказаться неисправен. Python сообщает о таких ситуациях через конкретные классы исключений, и SafeSort ловит именно их — не всё подряд.

Что произошло → какое исключение

Файл пропал между шагами → **FileNotFoundError**

Недостаточно прав доступа → **PermissionError**

Другая ошибка файловой системы

→ **OSError**

Исключение	Когда возникает	Как реагирует SafeSort
FileNotFoundError	Файл или каталог исчез между шагами	Пропускает эту запись, продолжает остальные
PermissionError	Недостаточно прав для чтения или записи	Пропускает эту запись, пишет предупреждение в журнал
OSError	Общая ошибка файловой системы (например, диск переполнен)	Пропускает эту запись, продолжает остальные

except Exception — не решение

Перехват `except Exception: pass` проглотил бы не только ожидаемые ошибки файловой системы, но и настоящие ошибки в самой программе — например, опечатку в имени переменной, — и программа продолжила бы работать, как ни в чём не бывало, скрывая реальную проблему. SafeSort перехватывает только те исключения, которые действительно ожидает в конкретном месте: `FileNotFoundError`, `PermissionError`, `OSError` — и ни разу не перехватывает исключения без указания типа.

Пример из `scanner.py` — чтение каталога, для которого может не быть прав доступа:

```
src/safesort/scanner.py
```

```
try:
    entries = list(directory.iterdir())
except PermissionError:
    logger.warning("Permission denied, skipping
directory: %s", directory)
    return
except FileNotFoundError:
    logger.warning("Directory vanished during scan,
skipping: %s", directory)
    return
except OSError as exc:
    logger.warning("Could not read directory %s: %s",
directory, exc)
    return
```

Каждая ветка перехватывает свой конкретный случай, пишет понятное сообщение в журнал (что это за журнал — на следующей странице) и возвращает управление — сканирование остальных каталогов продолжается как ни в чём не бывало.

Коротко

- SafeSort перехватывает конкретные ожидаемые исключения — `FileNotFoundError`, `PermissionError`, `OSError` — а не всё подряд.
- Каждый перехват сопровождается сообщением в журнал, а не молчаливым игнорированием.
- Ошибка на одном файле или каталоге не должна останавливать обработку остальных.

ЧЕКПОЙНТ · ISSUE #10

```
git commit -m "feat: add directory scanner and
configuration"
```

У этого Issue нет собственного Pull Request с фразой «Closes #10». Обработка ошибок файловой системы попала в тот же коммит, что и сканер (PR #15, страница 23-08), а сам Issue #10 закрыли вручную уже после проверки, без автоматической связки. Не каждая задача закрывается отдельным PR. Так выглядит фактическая история репозитория.

Статус в реальном Project: **Done**

Добавляем журнал работы программы

Итог для пользователя печатается напрямую; диагностика идёт отдельным каналом — через модуль logging.

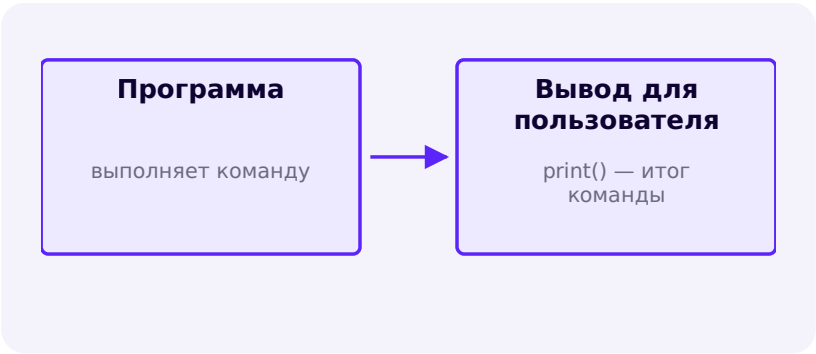
SafeSort · Часть 4 из 6 **Реализация**

GITHUB ISSUE #11 · PROJECT «SAFESORT — ПЕРВЫЙ РЕЛИЗ»

Add configuration and logging

Area: **CLI** Priority: **Low**

В коде SafeSort уже несколько раз встречался вызов `logger.warning (...)`. Это не то же самое, что вывод на экран через `print()`: у программы есть два разных канала сообщений с разным назначением.



	Пользовательский вывод (print)	Журнал (logging)
Кому адресован	Человеку, который вызвал команду	Тому, кто разбирается, что произошло внутри
Что показывает	Итог команды — сколько файлов, сколько перемещений	Диагностику: пропущенные файлы, ошибки доступа
Формат	Короткий, фиксированный	Уровень важности + подробности

```
src/safesort/cli.py
```

```
def _configure_logging() -> None:
    logging.basicConfig(
        level=logging.WARNING,
        format="%(levelname)s: %(name)s: %(message)s",
    )
```

SafeSort использует три уровня важности, каждый для своей ситуации:

Уровень	Когда используется в SafeSort
INFO	Обычные события: файл перемещён, символическая ссылка пропущена
WARNING	Что-то пропущено, но программа продолжает работать: каталог недоступен
ERROR	Операция не удалась: перемещение или восстановление не выполнено

Модуль получает logger по своему имени

Строка `logger = logging.getLogger(__name__)` сохраняет имя модуля в поле `LogRecord.name`. Само по себе это ещё не выводит имя в терминал: его показывает подстановка `%(name)s` в `formatter`. Без него имя осталось бы в записи журнала, но пользователь его не увидел бы.

Коротко

- Пользовательский вывод (print) и диагностический журнал (logging) — два разных канала с разным назначением, их не стоит смешивать.
- Три уровня важности — INFO, WARNING, ERROR — покрывают всё, что нужно SafeSort, без избыточной настройки.
- getLogger(__name__) записывает имя logger; formatter с %(name)s делает его видимым.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ[Logging HOWTO](#)[LogRecord attributes](#)

Настройки проекта

Файл настроек необязателен: без него в силу вступают встроенные значения по умолчанию, а с ним можно переопределить часть поведения.

SafeSort · Часть 4 из 6 **Реализация**

Категории по умолчанию и каталог результата подходят для большинства случаев, но иногда их стоит настроить — например, разложенные файлы должны попадать не в Sorted/, а в каталог с другим именем. Без файла настроек SafeSort использует встроенные значения:

Встроенные значения по умолчанию (нигде в файле не записаны — это поведение программы)

```
destination = "Sorted"
exclude = [".git", ".venv"]
```

Чтобы изменить их, SafeSort ищет необязательный файл `safesort.toml` прямо в корне сканируемого каталога:

Ситуация	Что произойдёт
safesort.toml есть в корне каталога	читаем и применяем настройки
safesort.toml отсутствует	используем встроенные значения по умолчанию

```
safesort.toml – переопределяет имя каталога
результата и добавляет категорию

destination = "Organized"
exclude = [".git", ".venv"]

[extensions]
books = [".epub"]
```

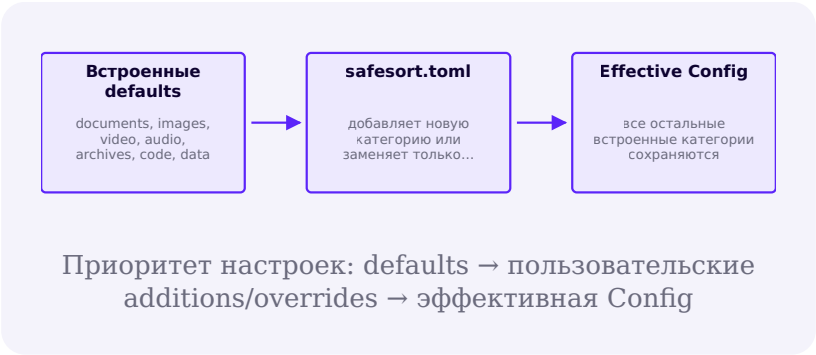


Таблица `[extensions]` работает как overlay. Строка `books = [".epub"]` добавляет категорию books и не удаляет documents или images. Если пользователь явно задаст `documents = [".md"]`, только список documents будет заменён.

safesort.toml остаётся входом конфигурации

Сканер всегда пропускает этот файл. После `plan`, `apply` и повторного запуска конфигурация остаётся в корне, поэтому следующий запуск получает те же настройки.

Ни одна команда SafeSort не требует файла настроек

Если `safesort.toml` не найден, в дело идут встроенные значения по умолчанию. Программа работает предсказуемо и без единой строчки настроек. Файл конфигурации переопределяет `defaults`, но не служит обязательным условием для запуска.

Чтение файла использует `tomllib`, модуль стандартной библиотеки Python для разбора TOML, доступный только на чтение:

`src/safesort/config.py`

```
def load_config(root: Path) -> Config:
    config_path = root / "safesort.toml"
    if not config_path.is_file():
        return Config()

    try:
        with config_path.open("rb") as handle:
            raw = tomllib.load(handle)
    except tomllib.TOMLDecodeError as exc:
        raise ConfigError(f"Could not parse config
file {config_path}: {exc}") from exc
    # ...
```

tomllib.load() принимает открытый файл в бинарном режиме

Обратите внимание на `open("rb")`, а не `open("r")`: `tomllib` сам решает, как декодировать байты файла в текст согласно спецификации TOML, поэтому ожидает на входе именно бинарный поток, а не уже прочитанную строку.

Если файл настроек существует, но содержит некорректный TOML, `SafeSort` не пытается угадать намерение пользователя. Он поднимает понятную ошибку `ConfigError` с указанием файла и причины, вместо того чтобы либо упасть с трудночитаемой трассировкой, либо молча продолжить с настройками по умолчанию.

Практика: читаем и проверяем TOML-настройки

Интерактивный ноутбук в браузере: Python 3.14 через Pyodide, без установки

[Открыть практику → \(https://www.cartesianschool.org/practice/23-19/index.html\)](https://www.cartesianschool.org/practice/23-19/index.html)

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[tomllib — Parse TOML files](#)

ЧЕКПОИНТ · ISSUE #11

```
git commit -m "feat: add directory scanner and configuration"
```

Как и Issue #10, у этой задачи нет собственного PR: логирование и настройки попали в тот же коммит, что и сканер ([PR #15](#)), а Issue #11 закрыли вручную. Книга разносит logging (23-21) и safesort.toml (эта страница) по двум урокам,

потому что это две разные идеи. В репозитории им соответствуют один Issue и один коммит.

Статус в реальном Project: **Done**

Пишем первые автоматические тесты

Тесты SafeSort работают только во временном каталоге `pytest` — ни один из них не трогает настоящие пользовательские файлы.

SAFESORT · ЧАСТЬ 5 ИЗ 6

Git и GitHub



Планирование



Проект



Реализация



Тесты и CI



Релиз

Представим: кто-то случайно сломал классификатор — расширение `.PDF` в верхнем регистре перестало определяться как документ. Как об этом узнать, не проверяя вручную каждый раз?

```
~/safesort $ pytest tests/test_classifier.py
```

```
-q
```

```
.....F.
```

```
===== FAILURES =====
```

```

_____
test_classify_is_case_insensitive
_____

assert classify(".PDF", DEFAULT_EXTENSIONS) ==
"documents"
E AssertionError: assert 'other' ==
'documents'

1 failed, 9 passed in 0.04s

```

RED — тест обнаружил поломку раньше человека.

Возвращаем нормализацию регистра на место — и тот же тест подтверждает исправление:



```

~/safesort $ pytest tests/test_classifier.py
-q
.....
10 passed in 0.02s

```

GREEN — поведение снова соответствует ожиданию.

Код, который сам проверяет другой код и сообщает, если поведение изменилось незаметно для человека, называют **автоматическим тестом**. В Python для этого чаще всего используют `pytest`.

Терминал (окружение активировано)

```
pip install -e ".[dev]"  
pytest tests/ -v
```

Тесты файловой системы никогда не используют настоящие пользовательские каталоги

Тест, который вызывает `apply` прямо на `~/Downloads`, при первом же неудачном запуске способен испортить реальные файлы. Все тесты `SafeSort` работают внутри временного каталога, который `pytest` создаёт и удаляет сам, через встроенную функцию `tmp_path` — она передаётся тестовой функции как обычный параметр:

ТОЛЬКО ЧИТАЮТ

ВРЕМЕННЫЙ КАТАЛОГ `/tmp/pytest-.../` — создаём, перемещаем, удаляем — удаляется автоматически

МЕНЯЮТ ФАЙЛЫ

РЕАЛЬНЫЕ ФАЙЛЫ ПОЛЬЗОВАТЕЛЯ `~/Downloads` — здесь тесты не выполняются никогда

```
tests/test_classifier.py
```

```
from safesort.classifier import classify
from safesort.config import DEFAULT_EXTENSIONS

def test_classify_known_extension():
    # Arrange: расширение и правило классификации уже
    #           готовы, ничего создавать не нужно
    # Act
    rezultat = classify(".pdf", DEFAULT_EXTENSIONS)
    # Assert
    assert rezultat == "documents"
```

Три части — **Arrange** (подготовить), **Act** (вызвать), **Assert** (проверить) — повторяются почти в каждом тесте SafeSort. Функция `classify()` — хороший первый тест не случайно: она чистая, не трогает файловую систему и не зависит ни от чего внешнего. Следующая страница берётся за более сложные тесты — те, что действительно создают файлы во временном каталоге.

Коротко

- Тест — код, который проверяет код: запускает функцию и сравнивает результат с ожидаемым.
- `tmp_path` — временный каталог, который `pytest` создаёт и удаляет для каждого теста сам.
- Ни один тест SafeSort не обращается к настоящему пользовательскому каталогу вроде `~/Downloads`.

Проверяем сканирование и классификацию

Тест на пустом каталоге и тест на повторное сканирование `Sorted/` — простые случаи, которые чаще всего ломаются незаметно.

SafeSort · Часть 5 из 6 **Тесты и CI**

Тесты для `scan()` и `classify()` вместе создают маленькую, полностью контролируемую файловую структуру во временном каталоге, а затем проверяют, что сканер нашёл именно те файлы, которые туда положили — ни больше, ни меньше.

tests/test_scanner.py

```

def test_scan_finds_nested_files(tmp_path):
    (tmp_path / "a").mkdir()
    (tmp_path / "a" / "otchet.pdf").write_text("...")
    (tmp_path / "photo.jpg").write_text("...")

    files = scan(tmp_path, Config())
    names = {f.path.name for f in files}
    assert names == {"otchet.pdf", "photo.jpg"}

def test_scan_skips_destination_directory(tmp_path):
    (tmp_path / "Sorted" /
     "documents").mkdir(parents=True)
    (tmp_path / "Sorted" / "documents" /
     "staryj.pdf").write_text("...")

    files = scan(tmp_path, Config())
    assert files == []
  
```

Пустой каталог — тоже проверяемый случай

Тест на пустом временном каталоге (без единого файла) кажется тривиальным, но именно такие крайние случаи чаще всего ломаются при рефакторинге: убедиться, что `scan()` возвращает пустой список, а не падает с ошибкой, — простой, но реальный тест.

Второй тест выше — прямая проверка требования, которое мы обсуждали раньше: каталог результата никогда не сканируется повторно. Без такого теста регресс (случайный возврат старой, неправильной логики) остался бы незамеченным до тех пор, пока кто-то не запустил бы SafeSort дважды подряд на одном каталоге и не увидел странный результат вручную.

Практика: тестируем сканер и классификатор

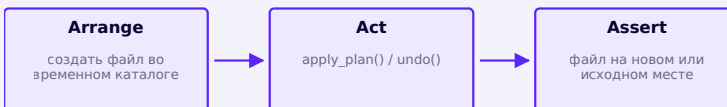
Нужен доступ к настоящей файловой системе — выполните локально в VS Code, PyCharm или Jupyter

[Открыть практику](https://www.cartesianschool.org/practice/23-21/index.html) → (<https://www.cartesianschool.org/practice/23-21/index.html>)

Проверяем перемещение и отмену

Каждый шаг — перемещение, отмену, конфликт при отмене — проверяет отдельный тест, а не один общий сценарий.

SafeSort · Часть 5 из 6 **Тесты и CI**



Проверить перемещение файлов сложнее, чем проверить чистую функцию: нужно создать файлы, применить план, убедиться, что они оказались в новом месте, и только потом проверить отмену. Каждый шаг проверяется отдельным тестом, а не одним большим сценарием — так гораздо понятнее, что именно сломалось, если тест не проходит.

```
tests/test_executor.py
```

```
def
test_apply_plan_moves_file_to_destination(tmp_path):
    source = tmp_path / "otchet.pdf"
    source.write_text(...)
    destination = tmp_path / "Sorted" / "documents" /
"otchet.pdf"
    plan = SortPlan(root=tmp_path,
operations=(MoveOperation(source, destination),))

    results = apply_plan(plan)

    assert results[0].completed is True
    assert not source.exists()
    assert destination.exists()
```

Тест отмены строит план, применяет его, затем вызывает `undo()` и проверяет, что файл оказался в точности там, откуда был взят:

tests/test_manifest.py

```
def test_undo_restores_original_location(tmp_path):
    source = tmp_path / "otchet.pdf"
    source.write_text("...")
    plan = build_plan(scan(tmp_path, Config()),
tmp_path, Config())
    moves = apply_plan(plan)
    manifest_obj, _ = write_manifest(tmp_path, moves)

    result = undo(manifest_obj)

    assert source.exists()
    assert result.conflicts == ()
```

Тест конфликта отмены — не менее важен, чем тест успешной отмены

Одного теста «отмена работает» недостаточно: нужен ещё тест, который специально создаёт конфликт — кладёт новый файл на исходное место перед вызовом `undo()` — и проверяет, что SafeSort отказался его перезаписать, а не проверяет только «счастливый путь».

Практика: тестируем перемещение и отмену

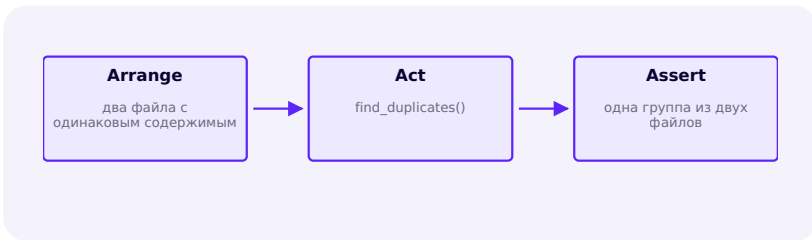
Нужен доступ к настоящей файловой системе — выполните локально в VS Code, PyCharm или Jupyter

Открыть практику → (<https://www.cartesianschool.org/practice/23-20/index.html>)

Проверяем поиск дубликатов

Пустые файлы и файл в несколько мегабайт проверяют два случая, которые легко пропустить в обычном тесте.

SafeSort · Часть 5 из 6 **Тесты и CI**



Тесты для `find_duplicates()` проверяют не только «типичный» случай двух одинаковых файлов, но и два крайних случая, которые легко упустить: файлы нулевого размера и файл заметно больше одного блока чтения.

tests/test_duplicates.py

```

def
test_identical_content_files_are_grouped(tmp_path):
    a = tmp_path / "notes.txt"
    b = tmp_path / "copy_of_notes.txt"
    a.write_text("тот же текст")
    b.write_text("тот же текст")

    groups = find_duplicates(scan(tmp_path, Config()))

    assert len(groups) == 1
    assert len(groups[0].files) == 2

def
test_empty_files_are_duplicates_of_each_other(tmp_path):
    (tmp_path / "a.txt").write_text("")
    (tmp_path / "b.txt").write_text("")

    groups = find_duplicates(scan(tmp_path, Config()))

    assert len(groups) == 1
    assert groups[0].size == 0
  
```

Результат и способ чтения проверяются отдельно

Файл больше одного блока и правильный digest доказывают только результат функции. Такой тест не наблюдает вызовы `read()` : реализация могла бы прочитать весь файл сразу и всё равно вернуть тот же SHA-256. Контракт bounded reads проверяется отдельным instrumented reader.

```
tests/test_duplicates.py
```

```
def
test_sha256_large_file_has_correct_result(tmp_path):
    data = b"x" * (5 * 1024 * 1024) # 5 МБ – больше
одного блока чтения
    path = tmp_path / "bolshoj_fajl.bin"
    path.write_bytes(data)

    assert sha256_file(path) ==
hashlib.sha256(data).hexdigest()
```

```
tests/test_duplicates.py: interaction test
```

```
class RecordingReader(io.BytesIO):
    def __init__(self, data: bytes) -> None:
        super().__init__(data)
        self.requested_sizes = []

    def read(self, size: int = -1) -> bytes:
        self.requested_sizes.append(size)
        return super().read(size)

def test_sha256_stream_uses_multiple_bounded_reads():
    reader = RecordingReader(b"abcdefghij")

    digest = sha256_stream(reader, chunk_size=4)

    assert digest ==
hashlib.sha256(b"abcdefghij").hexdigest()
    assert reader.requested_sizes == [4, 4, 4, 4]
    assert max(reader.requested_sizes) == 4
```

Первый тест относится к типу **result test (black-box)**: он проверяет, *что* вернулось. Второй проверяет interaction/implementation contract: зависимость получила несколько ограниченных `read(4)`. Небольшой записывающий поток делает это наблюдаемым без тяжёлой mocking-инфраструктуры.

Практика: тесты дубликатов и пустых файлов

Интерактивный ноутбук в браузере: Python 3.14 через Pyodide, без установки

Открыть практику → (<https://www.cartesianschool.org/practice/23-22/index.html>)

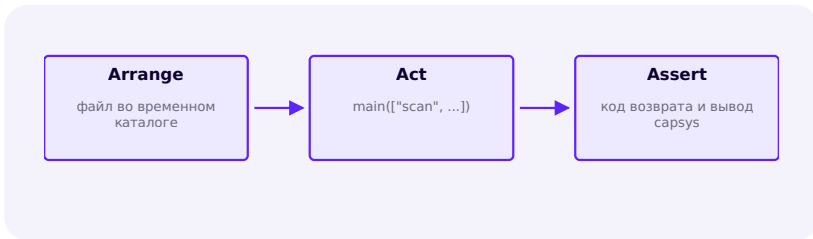
ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[pytest — How to write and report assertions](#)
[unittest.mock — mock object library](#)

Проверяем интерфейс командной строки

carsys перехватывает напечатанный программой текст. Тесты проверяют его как обычную строку без реального терминала.

SafeSort · Часть 5 из 6 **Тесты и CI**



Последний слой SafeSort, который стоит проверить тестами, это сама командная строка: правильно ли `argparse` разбирает аргументы, и правильный ли код возврата получает вызывающая сторона.

tests/test_cli.py

```
def
test_scan_subcommand_returns_zero_on_success(tmp_path,
capsys):
    (tmp_path / "otchet.pdf").write_text("...")

    code = main(["scan", str(tmp_path)])

    assert code == 0
    assert "Files scanned: 1" in
capsys.readouterr().out

def test_missing_path_returns_nonzero(tmp_path):
    code = main(["scan", str(tmp_path / "net-takogo-
kataloga")])

    assert code != 0
```

capsys: встроенная перехватка вывода pytest

Функция `main()` печатает результат через `print()`, а не возвращает текст напрямую. Фикстура `capsys` перехватывает всё, что попало на стандартный вывод и поток ошибок во время теста, и позволяет проверить это как обычную строку.

Проверка `--help` тоже заслуживает отдельного теста. `argparse` печатает справку и намеренно поднимает `SystemExit(0)`: код 0 означает успешное завершение, а не ошибку. Некорректные аргументы тоже приводят к `SystemExit`, но уже с ненулевым кодом, обычно 2.

tests/test_cli.py

```
def test_help_exits_zero():
    with pytest.raises(SystemExit) as excinfo:
        main(["--help"])
    assert excinfo.value.code == 0

def test_invalid_subcommand_exits_nonzero():
    with pytest.raises(SystemExit) as excinfo:
        main(["not-a-command"])
    assert excinfo.value.code != 0
```

Практика: тесты аргументов командной строки

Интерактивный ноутбук в браузере: Python 3.14 через Pyodide, без установки

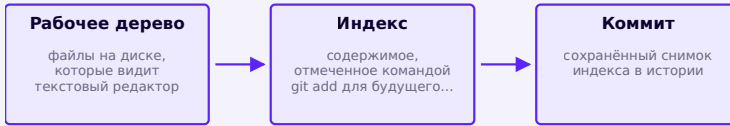
Открыть практику → (<https://www.cartesianschool.org/practice/23-23/index.html>)

Самопроверка изменений и дисциплина коммитов

Git сопровождал SafeSort с первой строчки кода. Теперь соберём привычку проверять изменения перед каждым коммитом.

SafeSort · Часть 5 из 6 Тесты и CI

Git сопровождал SafeSort с самого начала: каждый чекпойнт в Части IV (страницы с 23-08 по 23-22) отмечал коммит и Pull Request, сохранявший очередную функцию в истории. Здесь мы не открываем Git впервые. Мы формализуем дисциплину самопроверки перед коммитом, прежде чем в следующем разделе код пройдёт полный цикл Issue → ветка → Pull Request на GitHub. Между «файл изменён на диске» и «изменение сохранено в истории» есть два промежуточных шага, и понимание разницы между ними экономит немало недоумения в будущем.



git add помещает снимок текущего содержимого в индекс; рабочий файл остаётся на месте и может снова измениться

Терминал

```
git status
git diff
git add src/safesort/duplicates.py tests/
test_duplicates.py
git commit -m "feat: add duplicate-file detection"
```

git diff показывает содержимое изменений

git status перечисляет изменённые файлы, но не показывает содержимое изменений. git diff показывает построчно, что именно добавлено и что удалено. Читайте его перед каждым коммитом, чтобы случайно не закоммитить что-то лишнее: отладочный print(), забытый файл с личными данными, временный код.

Логические коммиты

Один коммит должен описывать одно законченное, осмысленное изменение, а не «конец рабочего дня» или случайный набор всего, что накопилось. Сравните:

Хорошо	Плохо
feat: add directory scanner	update

Хорошо	Плохо
feat: add dry-run sort planning	fix
test: cover undo conflicts	stuff
docs: document safesort.toml options	final-final

Когда не стоит использовать git add .

git add . добавляет в индекс все изменения в текущем каталоге разом. Это удобно, когда коммит действительно должен включать всё, но легко случайно затянуть в коммит что-то не относящееся к делу. git add путь/к/файлу добавляет только нужные файлы и делает коммиты более осмысленными.

Практика: читаем git diff и группируем изменения

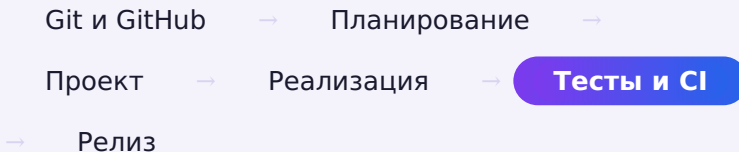
Нужен Git-репозиторий. Выполните практику локально в VS Code, PyCharm или терминале

[Открыть практику](https://www.cartesianschool.org/practice/23-24/index.html) → (https://www.cartesianschool.org/practice/23-24/index.html)

GitHub: применяем Issue, ветку и Pull Request

Разберём Pull Request #20, который прошёл проверку, был слит и закрыл Issue #9.

SAFESORT · ЧАСТЬ 5 ИЗ 6



Часть II уже разобрала цикл Issue → ветка → код → Pull Request → CI → слияние на странице 23-proj-17. Теперь применим эту схему к завершённому изменению SafeSort: Issue #9 «Add duplicate detection». На этот раз разберём и сам Pull Request.

LOCAL GIT	GITHUB
git switch -c feat/duplicate-detection	Issue #9 описывает задачу
редактирование + локальный pytest	удалённая ветка появится после push
git add + git commit	Pull Request сравнивает ветку с main
git push -u origin feat/duplicate-detection	Conversation, Commits, Checks, Files changed
после merge: git fetch / pull	review, решение о merge, закрытие Issue

Граница пересекается на `git push`: локальные коммиты становятся удалённой веткой. Issue и Pull Request являются объектами GitHub, а `switch`, редактирование, тесты, staging и commit происходят локально. После merge локальный Git узнаёт новое состояние через fetch/pull.

Issue #9 в Project

Issue #9 заранее добавили в Project «SafeSort: первый релиз». На странице 23-proj-09 мы уже видели, как Issues попадают в Project. Ниже показана форма GitHub с полями заголовка, описания и метаданных.

Форма создания нового Issue на GitHub: поле заголовка, поле описания, боковая панель Assignees/Labels/Type

Форма Issue на GitHub, в которой был сформулирован Issue #9 репозитория SafeSort.

Ветка feat/duplicate-detection

Работа над Issue #9 шла в изолированной ветке. Тот же приём применялся в каждом чекпойнте Части IV:

Терминал

```
git switch -c feat/duplicate-detection
# ...пишем код и тесты, коммитим изменения...
git push -u origin feat/duplicate-detection
```

Pull Request #20: что сохранил GitHub

Посмотрим на данные репозитория Cartesian-School/safesort :

Поле PR #20	Реальное значение
Заголовок	feat: add duplicate detection with byte-level confirmation
Ветка → main	feat/duplicate-detection → main
Commits	1 коммит; весь Issue #9 уместился в одно логическое изменение
Files changed	2 файла: src/safesort/duplicates.py (+113), tests/test_duplicates.py (+144)
Conversation	«Closes #9» в описании и чек-лист плана тестирования (pytest tests/: 59 passed)

Поле PR #20	Реальное значение
Checks	test: pass, 11s (workflow safesort-tests.yml)
Review	у учебного PR не было отдельного reviewer; это факт истории, а не модель для командной работы
Решение о слиянии	Merged обычным merge commit 01989e0c, не squash и не rebase

Четыре вкладки и два разных вида проверки

Вкладка	На какой вопрос отвечает
Conversation	что обсуждали, какое Issue закрывается, каков итог review
Commits	из каких сохранённых шагов состоит ветка
Checks	прошли ли автоматические workflow и тесты
Files changed	что именно предлагается добавить, изменить или удалить

Reviewer читает изменения и может оставить **comment**, выбрать **Approve** или **Request changes**. CI отвечает «прошли ли автоматические проверки?», а человек отвечает «следует ли принять это изменение?». Ни зелёный Checks не заменяет инженерное review, ни review не заменяет воспроизводимые тесты.

Вкладка Files changed настоящего Pull Request репозитория Cartesian-School/safesort: добавленный файл duplicates.py, статус Merged

Pull Request SafeSort «Add duplicate detection with byte-level confirmation», закрывший Issue №9.

Один PR не всегда соответствует одному Issue

PR #20 показывает простой случай: одна ветка, один коммит, один Issue, автоматически закрытый фразой Closes #9 при слиянии. В истории SafeSort Встречаются и другие варианты: один PR может закрыть два связанных Issue (страницы 23-11 и 23-14 показывают такой пример), и Issues, закрытые вручную без отдельного PR вовсе (страницы 23-20 и 23-22).

PR можно открыть раньше, чем код готов полностью

Pull Request не обязан представлять уже законченную работу. Его можно открыть раньше, чтобы обсудить подход или получить промежуточный отзыв, и явно пометить как черновик. Ожидание «пока всё не будет идеально» не единственный способ работать с Pull Request.

Коротко

- На PR #20 виден тот же цикл Issue, ветки и Pull Request, который был разобран в Части II.
- Files changed, Commits, Checks и Conversation показывают 2 файла, 1 коммит, зелёную проверку и «Closes #9».
- Один PR может закрыть несколько Issues; Issue также можно закрыть вручную без PR.

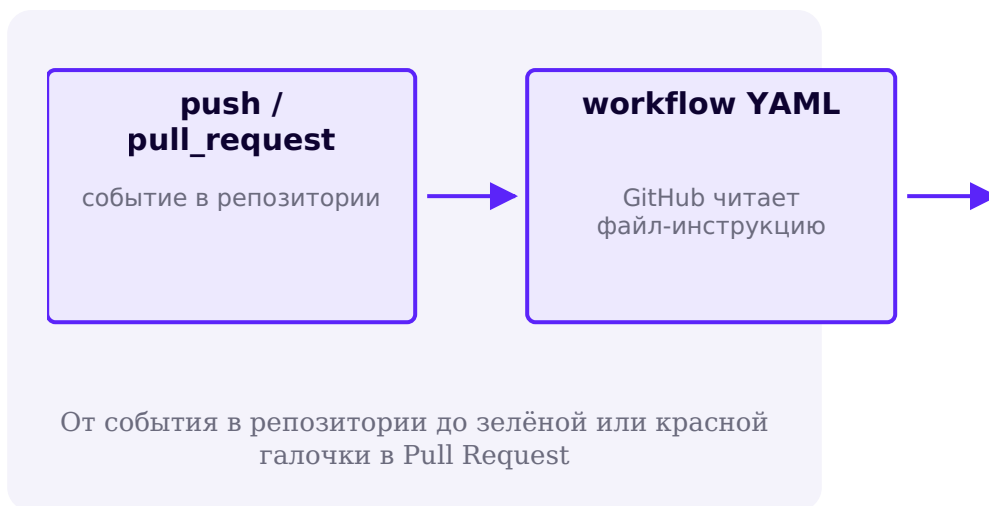
GitHub Actions: автоматически запускаем тесты

Один файл воркфлоу запускает тесты SafeSort автоматически при каждом изменении — без ручной проверки перед Pull Request.

SafeSort · Часть 5 из 6 **Тесты и CI**

GITHUB ПРОВЕРКА (CI) — ЧЕТВЁРТЫЙ ШАГ ИЗ
ДИАГРАММЫ НА ПРОШЛОЙ СТРАНИЦЕ

Проверять тесты вручную перед каждым Pull Request легко забыть. **GitHub Actions** — сервис, который автоматически выполняет заданные действия при определённых событиях в репозитории, например при каждом пуше или открытии Pull Request. Для SafeSort такое действие — запуск тестов.



Список запусков GitHub Actions репозитория Cartesian-School/safesort: 19 реальных запусков; среди двух красных один относится к раннему этапу до рабочего CI, второй — к намеренно сломанному тесту, сразу после которого следует зелёный

Настоящая история CI репозитория SafeSort — 19 запусков: один ранний красный до рабочего CI и учебная пара из намеренно сломанного запуска и зелёного после исправления (раздел ниже).

Вот тот самый файл-инструкция, который GitHub читает перед каждым запуском:

```
.github/workflows/safesort-tests.yml
```

```
name: SafeSort tests

on:
  push:
    branches: ["main"]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository
        uses: actions/checkout@v7

      - name: Set up Python
        uses: actions/setup-python@v7
        with:
          python-version: "3.14"

      - name: Install SafeSort with dev dependencies
        run: pip install -e ".[dev]"

      - name: Run tests
        run: pytest tests/
```

он: без ограничений — потому что весь репозиторий и есть SafeSort

Здесь нет ключа `paths`, ограничивающего запуск по изменённым файлам: в отдельном репозитории `safesort` любой пуш или Pull Request так или иначе касается самого пакета. Такое ограничение нужно только в общем репозитории курса, где `SafeSort` — лишь один из многих подкаталогов и незачем перезапускать его тесты из-за изменений где-то ещё.

Управляемая проверка: специально сломанный тест

Полезно один раз увидеть, как выглядит красная (неудачная) проверка — и как её исправить, — прежде чем столкнуться с этим впервые в реальной ситуации. В истории запусков выше это тот самый красный кружок: временный коммит намеренно вернул регистрозависимое сравнение расширений в классификаторе (тот же дефект, что показан в разделе 23-23 как RED/GREEN пример), затем был отменён следующим же коммитом — `main` ни на секунду не оставался в сломанном состоянии.

Ломаем тест

намеренно меняем
ожидаемое значение на
неверное

Пушим ветку

GitHub Actions
запускается
автоматически

Один раз специально сломать тест — самый быстрый способ научиться читать журнал GitHub Actions

Основная ветка не должна оставаться красной

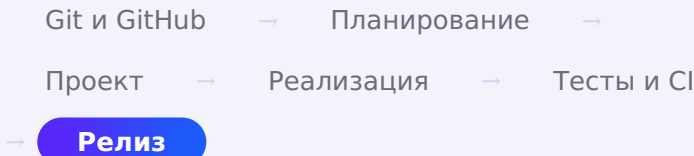
Смысл этого упражнения — увидеть неудачную проверку в управляемых условиях и сразу её исправить, а не оставить `main` в сломанном состоянии. Реальная красная проверка на основной ветке означает, что кто-то другой, кто скачает код прямо сейчас, получит нерабочую версию.

Коротко

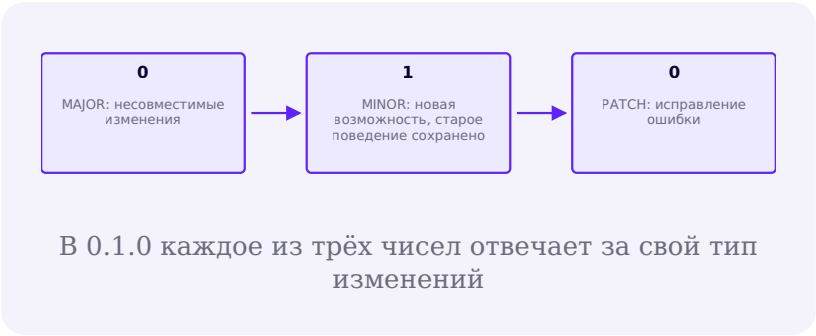
- GitHub Actions запускает заданные действия автоматически — при пуше или открытии Pull Request.
- В отдельном репозитории воркфлоу запускается без ограничения по paths — любое изменение здесь так или иначе касается SafeSort.
- Специально сломанный и затем исправленный тест — быстрый способ научиться читать журнал проверки.

Документация, версия и первый релиз

MAJOR.MINOR.PATCH, CHANGELOG.md, Git tag и GitHub Release дают проекту точку, к которой можно вернуться и на которую можно ссылаться.

SAFESORT · ЧАСТЬ 6 ИЗ 6

Проект готов, проверен тестами и подключён к автоматической проверке. Мы закончили первую пригодную для использования версию SafeSort. Теперь ей нужен номер, чтобы отличать её от будущих изменений. Этот номер мы уже записали в `pyproject.toml` ещё в части III, где он был просто техническим полем. Сейчас разберём, что он означает.



Семантическое версионирование

Эта схема называется **семантическим версионированием** (Semantic Versioning, SemVer), то есть соглашение о записи номера версии как `MAJOR.MINOR.PATCH` :

Часть номера	Меняется, когда
MAJOR	несовместимые изменения; старый способ использования перестает работать
MINOR	добавлена новая возмож- ность, старое поведение со- хранено
PATCH	исправлена ошибка, поведе- ние по сути не изменилось

Соглашение, а не закон физики

Семантическое версионирование задаёт общепринятую договорённость, но не встроено в инструменты правило: ничто технически не мешает нарушить его смысл. Пользы от него ровно столько, сколько сам проект последовательно ему следует. Это ориентир для тех, кто устанавливает пакет и хочет понимать, чего ждать от новой версии.

Первая версия SafeSort имеет номер 0.1.0. Минорная версия ниже 1 традиционно означает «интерфейс ещё может измениться без отдельного согласования».

Что изменилось в этой версии: CHANGELOG

CHANGELOG.md

```
## [0.1.0]
```

```
### Added
```

```
- scan, plan, apply, duplicates, undo commands
```

```
### Safety
```

```
- dry-run by default (scan/plan/duplicates never  
modify files)  
- no automatic duplicate deletion  
- no silent overwrite of existing files
```

CHANGELOG описывает только реальные версии

CHANGELOG не должен придумывать более ранние версии, которых не существовало. Он описывает выпущенные изменения, начиная с первой версии проекта.

Собираем wheel и sdist

Релиз начинается с исходного дерева, но пользователю передают distribution artifacts. Команда `python -m build` создаёт оба стандартных формата:

Терминал (корень репозитория SafeSort)

```
python -m pip install --upgrade build
python -m build
```

dist/

safesort-0.1.0-
py3-none-any.whl

`safesort-0.1.0.tar.gz`

wheel содержит готовый архив Python distribution;
sdist содержит исходники для сборки

Артефакт	Назначение
wheel (.whl)	предсобранный архив distribution: установка обычно не запускает сборку проекта
sdist (.tar.gz)	архив исходников и метаданных, из которого инструмент может собрать wheel

Smoke test в чистом окружении

Editable install проверяет разработку, но не доказывает, что собранный wheel содержит нужные файлы и console script. Поэтому устанавливаем именно artifact в новое окружение:

POSIX shell

```
python -m venv .release-smoke
source .release-smoke/bin/activate
python -m pip install dist/safesort-0.1.0-py3-none-any.whl
python -c "import safesort; print(safesort.__file__)"
safesort --help
deactivate
```

На Windows активация выполняется командой `.release-smoke\Scripts\activate`. Проверяется та же установленная `distribution`, а не исходники из `src/`.

GITHUB GITHUB RELEASE СТРОИТСЯ ПОВЕРХ ТЕГА

Git tag и GitHub Release: разные объекты

Тег (tag) хранит Git-ссылку на конкретный коммит и обычно соответствует номеру версии. Технически тег можно передвинуть или удалить, но опубликованные release-теги проект по политике считает неизменяемыми.

GitHub Release служит отдельным объектом GitHub, созданным вокруг тега. У него есть страница с описанием и прикрепленными artifacts. Push тега сам по себе Release не создаёт.

Тег версии относится ко всему репозиторию целиком, а не к одной его части. Поэтому SafeSort живёт в собственном репозитории `Cartesian-School/safesort`, а не только в подкаталоге курса: `git tag v0.1.0` здесь однозначно означает «версия 0.1.0 SafeSort», без двусмысленности.



```
~/safesort $ git tag -a v0.1.0 -m "SafeSort
0.1.0 – first release"
~/safesort $ git push origin v0.1.0
To https://github.com/Cartesian-School/
safesort.git
* [new tag] v0.1.0 -> v0.1.0
```

Затем в веб-интерфейсе GitHub откройте **Releases** → **Draft a new release**, выберите существующий тег `v0.1.0`, добавьте выдержку из CHANGELOG и прикрепите оба файла из `dist/`. После этого у тега появляется человекочитаемая страница релиза и загружаемые artifacts.

Страница релиза SafeSort 0.1.0 на GitHub: заголовок, метка Latest, описание, команда `pip install`, прикрепленные файлы `wheel` и `sdist`

Релиз SafeSort 0.1.0: тег, описание изменений и собранные пакеты `wheel` и `sdist`.

Релиз создан только после того, как всё остальное было готово

Тег и релиз появились на последнем шаге, когда CI уже был зелёным, CHANGELOG.md описывал реальные изменения, а `editable install`, сборка `wheel/sdist` и команда `safesort --help` были заново проверены на чистом окружении. Без этих проверок номер версии ничего не гарантирует.

Публикация в PyPI остаётся за рамками версии 0.1.0

Установка через `pip install git+https://github.com/Cartesian-School/safesort.git@v0.1.0` достаточна для разработки и личного использования. Публикация пакета в PyPI, чтобы его можно было установить командой `pip install safesort` без ссылки на репозиторий, остаётся отдельной темой с собственными требованиями к учётной записи и публикации. Версия 0.1.0 сознательно её не касается.

Коротко

- MAJOR.MINOR.PATCH задаёт соглашение о номере версии, а не встроенное в инструменты правило.
- `python -m build` создаёт wheel и sdist; чистое окружение проверяет установку wheel и console script.
- Git tag и GitHub Release представляют разные объекты; Release создаётся вокруг тега и хранит artifacts.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Packaging flow](#)

[Packaging Python Projects — Generating distribution archives](#)

[Semantic Versioning 2.0.0](#)

[Managing releases in a repository](#)

Итоги: полный путь проекта от идеи до релиза

От идеи до первого релиза — весь путь пройден один раз целиком, на настоящем проекте.

SafeSort · Часть 6 из 6 **Релиз**

Полный путь проекта

В начале этой главы SafeSort был только идеей. Теперь это настоящий репозиторий на GitHub — [Cartesian-School/safesort](#) — с 14 закрытыми Issues, реальными Pull Request, зелёной проверкой CI и опубликованным релизом 0.1.0. Вот весь путь целиком, с тем, что появилось на каждом шаге:

- Идея**
требования: что программа делает и чего не делает
- Git и GitHub**
установка, аутентификация, настоящий репозиторий SafeSort
- GitHub Project**
14 Issues — задача сформулирована для каждой части
- README.md, pyproject.toml**
первый коммит репозитория — команда safesort становится доступной
- Сканер**
scanner.py — Issue №1, PR №15
- Классификатор**
classifier.py — Issue №2, PR №16
- План и коллизии**
planner.py — Issues №3 и №6 вместе, PR №17
- Apply**
executor.py — Issue №5, PR №18
- Manifest и undo**
manifest.py — Issues №7 и №8 вместе, PR №19
- Дубликаты**
duplicates.py — размер → SHA-256 → байтовое подтверждение, Issue №9, PR №20
- Командная строка**
argparse, пять подкоманд — Issue №4, PR №21 (последней, когда всё остальное уже)
- Ошибки, config, тесты, CI**
Issues №10-13 — часть реализации соответствующих модулей, без отдельного PR
- Версия 0.1.0**
CHANGELOG.md, editable install, wheel и sdist проверены заново — Issue №14
- Релиз**
настоящий тег v0.1.0 и GitHub Release

От идеи до опубликованного релиза — точная история: 9 из 14 Issues закрыты через 7 Pull Request (два PR закрыли по два Issue), остальные пять — без отдельного PR

ЧТО УЖЕ ЕСТЬ

Git и
GitHub
настроены

Настоящий
репозиторий
SafeSort

GitHub
Project
и 14
Issues

Установленный
пакет

Работающая
командная
строка

Безопасные
перемещения
с отменой

Поиск
дубликатов

Автоматические
тесты

GitHub
Actions
(CI)



**Версия 0.1.0 — тег и GitHub Release
опубликованы**



Публикация в PyPI



Классификация по содержимому файла

Что уже умеет SafeSort

Команда	Что делает	Меняет файлы?
scan	находит и подсчитывает файлы по категориям	нет
plan	показывает план перемещений	нет

Команда	Что делает	Меняет файлы?
duplicates	находит файлы с одинаковым содержанием	нет
apply	выполняет перемещения из плана	да, только по явной команде
undo	отменяет последнюю выполненную операцию	да, только восстановление

Что дальше

Версия 0.1.0 сознательно не покрывает всё, что в принципе возможно: самый первый раздел этой главы заранее очертил границы. Дальнейшее развитие SafeSort — за рамками этой главы, но некоторые направления естественно продолжают уже сделанное: публикация пакета в PyPI, классификация не только по расширению, а с осторожной проверкой содержимого файла, интерфейс для настройки, отличный от редактирования TOML-файла вручную.

Приложение к этой главе повторяет тот же самый путь — от задачи до Pull Request — ещё шесть раз, уже самостоятельно, на шести небольших проектах:

[Дополнительная практика: шесть мини-проектов для GitHub.](#)

Что мы узнали в этой главе

- Прежде чем писать код, требования проекта описывают не только то, что программа делает, но и то, что она сознательно не делает.
- Разделение на read-only планирование и две явные операции, прямую apply и обратную undo, защищает от случайных изменений файлов пользователя.
- Поиск дубликатов дорогой операцию — хеширование — делает только там, где она действительно может изменить ответ: после отбора файлов по размеру.
- Автоматические тесты работают только во временных каталогах — ни один тест не должен касаться настоящих файлов пользователя.
- Git и GitHub — часть разработки с самого начала, а не финальный шаг: Issue формулирует задачу, ветка изолирует работу, Pull Request открывает её для проверки, GitHub Actions проверяет тесты автоматически.

Что дальше? Roadmap Python-разработчика

Профессиональная карта развития после курса: фундамент Python, инженерная практика, специализации, портфолио и самостоятельное обучение.

24.1 Чего Вы достигли!

24.2 Первые 30 дней после книги

24.3 Professional Python Core

24.4 Software Engineering

24.5 Как выбрать направление

24.6 Backend: roadmap

24.7 Data Science, ML и AI: roadmap

24.8 Automation и DevOps: roadmap

24.9 Desktop: roadmap

24.10 Game Development: roadmap

24.11 Портфолио разработчика

24.12 Первый вклад в Open Source

24.13 Как учиться дальше

24.14 Следующие курсы Cartesian School

24.15 Ваш следующий шаг

Чего Вы достигли!

Вы прошли путь от первой команды Python до проекта с тестами, Pull Request, CI, пакетом и релизом.

От первой строки до инженерного цикла

В начале курса программа состояла из одной команды `print("Привет!")`. Теперь Вы умеете переводить задачу в данные, функции, модули, интерфейс и тесты. Это важнее количества запомненных методов: Вы научились строить программу по частям и проверять каждую часть.

ЧЕГО ВЫ ДОСТИГЛИ!

- **Вывод и переменные**
Программа хранит состояние и показывает результат.
- **Условия и циклы**
Код принимает решения и повторяет действия.
- **Коллекции и функции**
Данные организованы, логика разделена на операции.
- **Классы, файлы, исключения**
Появились модели, сохранение и явные отказы.
- **Turtle, Tkinter, Pygame, Flask**
Одна основа Python работает в разных интерфейсах.
- **Git, GitHub, Issues, Projects, PR, тесты, CI**
Изменение проходит воспроизводимый инженерный цикл.
- **Пакет и релиз**
Проект можно собрать, установить и передать другому человеку.

Путь по курсу: от первой команды до проекта,
который можно проверить и выпустить.

Что именно Вы теперь можете

Компетенции после Главы 23

МОДЕЛИРОВАТЬ

Выбрать типы данных

Сформулировать инварианты
Разделить состояние и поведение

РЕАЛИЗОВАТЬ

Написать функции и классы
Работать с файлами
Обработать ожидаемые ошибки

СОБИРАТЬ ИНТЕРФЕЙС

Turtle для визуальных моделей
Tkinter для GUI
Pygame для игрового цикла
Flask для HTTP-приложения

ПРОВЕРЯТЬ

Воспроизвести дефект
Написать тест
Проверить граничные случаи

СОТРУДНИЧАТЬ

Issue и ветка
осмысленный commit

Pull Request и review

ПОСТАВЛЯТЬ

pyproject.toml

CI

сборка пакета

версионированный релиз

Компетенцию подтверждает действие, знакомства с термином недостаточно.

Как SafeSort связал все этапы работы

SafeSort в [главе 23](#) был учебным CLI-инструментом автоматизации. Его ценность не в том, чтобы конкурировать с файловым менеджером. На нём Вы прошли полный процесс: уточнили поведение, отделили план от выполнения, ограничили опасные операции, добавили тесты, оформили изменение в ветке, провели его через Pull Request и CI, затем собрали релиз. Итог можно проверить на следующем проекте: тот же процесс должен работать уже без пошаговой инструкции.

Что означает завершение курса

Вводный курс не охватывает весь Python и не готовит к любой профессиональной роли. Теперь у Вас есть фундамент и рабочий способ развивать его через проекты, проверки и разбор ошибок.

Самопроверка без самообмана

Возьмите небольшой незнакомый проект. Сможете ли Вы описать входы и выходы, выделить чистую логику, предусмотреть отказ, написать несколько тестов, сохранить изменение в Git и объяснить решение? Если часть цикла пока требует подсказки, это не обнуляет результат. Она просто становится конкретной целью следующего месяца.

Ваш результат

- Вы читаете задачу как систему данных, правил, интерфейсов и отказов.
- Вы можете довести небольшой проект от идеи до проверяемого изменения.
- Вы знаете границы текущих знаний и умеете превратить их в план работы.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Python 3.14 documentation](#)

[Git reference](#)

[GitHub pull requests documentation](#)

[GitHub Actions documentation](#)

[Python Packaging User Guide](#)

Первые 30 дней после книги

Четыре недели закрепления: один завершённый результат за раз, с тестами и разбором решений.

План на четыре недели

Этот план рассчитан на регулярную практику после книги. Он не претендует на универсальный темп. Если на учёбу есть два коротких занятия в неделю, растяните неделю плана на две. Не сокращайте проверку и разбор ошибок ради календаря.

Неделя	Фокус	Результат недели
1	Повторить основы без копирования: функции, коллекции, файлы, исключения.	Один CLI-мини-проект с README и тестами.
2	Пересобрать знакомую идею другим инструментом.	Один визуальный проект и журнал решений.
3	Добавить проверки проекта: ветка, issue, lint, тесты, CI.	Pull Request в собственном репозитории.
4	Выбрать одну специализацию и провести короткое исследование.	Прототип, ретроспектива и план на следующие 60 дней.

Банк проектов

Выберите один проект на неделю, не все сразу. Ниже нет готовых решений. Минимальная версия задаёт границу завершения, а расширения дают материал на следующий цикл.

Конвертер валют

Идея: Пересчитать сумму между выбранными валютами по явно заданному набору курсов.

Минимальная версия: CLI или Tkinter, проверка числа и кода валюты, курсы в JSON, понятная ошибка при отсутствии пары.

Расширения: Загрузка курса из HTTP API, кэш с временем обновления, Decimal и история операций.

Что закрепляет: словари, функции, JSON, валидация, отделение расчёта от интерфейса

Гонка в Pygame

Идея: Игрок меняет полосу и избегает препятствий, движущихся сверху вниз.

Минимальная версия: Игровой цикл, управление, коллизии, счёт и перезапуск без глобальной мешанины состояния.

Расширения: delta time, возрастающая сложность, звуки, таблица результатов в файле.

Что закрепляет: события, координаты, состояние игры, классы, отладка по кадрам

Вариации узоров Turtle

Идея: Исследовать, как угол, шаг, цвет и повторение меняют геометрический узор.

Минимальная версия: Три параметризованных узора, ввод параметров и сохранение выбранной конфигурации.

Расширения: генератор палитры, симметрия, экспорт изображения, сравнение времени построения.

Что закрепляет: циклы, функции, параметры, координаты, экспериментальное мышление

Змейка на Pygame

Идея: Повторить механику проекта Turtle в другой событийной и графической модели.

Минимальная версия: Сетка, направление, рост, еда, столкновение с границей и собственным телом.

Расширения: пауза, уровни, препятствия, тестируемая логика движения вне Pygame.

Что закрепляет: декомпозиция, коллекции, состояние, перенос модели между фреймворками

Игра на память

Идея: Открывать пары карточек и сохранять совпавшие пары.

Минимальная версия: Перемешанная сетка, не более двух открытых карточек, задержка перед закрытием, завершение партии.

Расширения: несколько размеров поля, таймер, число ходов, воспроизводимое перемешивание для теста.

Что закрепляет: двумерные данные, конечные состояния, события мыши, тестируемая логика

Игра на уклонение

Идея: Персонаж избегает падающих объектов, а сложность постепенно растёт.

Минимальная версия: Движение, генерация препятствий, столкновение, счёт времени и экран окончания.

Расширения: разные типы препятствий, бонусы, пауза, профиль производительности.

Что закрепляет: игровой цикл, случайность, коллизии, управление темпом

Рабочий ритм

- 
- Сформулировать**
одна цель и критерий готовности
 - Разбить**
3 или 4 наблюдаемых шага
 - Реализовать**
маленькими проверяемыми изменениями
 - Проверить**
тест, ручной сценарий, lint
 - Объяснить**
README и короткая ретроспектива

Один цикл самостоятельной практики.

Если застряли, уменьшите задачу до следующего наблюдаемого поведения. Например, сначала покажите неподвижную машинку, затем обработайте одну клавишу, после этого добавьте одно препятствие. Такой разрез сохраняет причинно-следственную связь между кодом и результатом.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

- [Python 3.14 documentation](#)
- [pytest documentation](#)
- [Pygame documentation](#)

Professional Python Core

Идиомы языка, модель итерации, типы, ресурсы, пакеты и конкурентность как общий фундамент Python-разработчика.

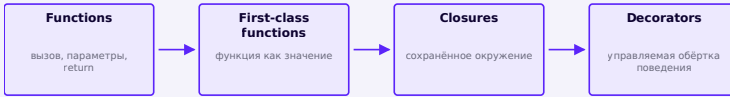
Слой между синтаксисом и специализацией

Professional Python Core нужен во всех направлениях. Его не проходят за один выходной и не изучают изолированными трюками. Каждый блок имеет предпосылку, рабочую задачу и наблюдаемый критерий понимания.

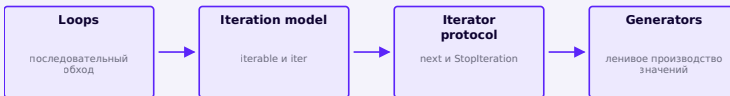
Блок	Зачем и когда	Предпосылка и критерий
Comprehensions, распаковка, срезы	Компактно преобразовывать коллекции и явно раскладывать значения.	После списков и циклов. Вы можете выбрать читаемую форму и не прятать сложную логику в одной строке.

Блок	Зачем и когда	Предпосылка и критерий
Iterable, iterator, generator	Обрабатывать поток значений без обязательного хранения всего результата.	После функций и циклов. Вы объясняете разницу между объектом, итератором и ленивым генератором.
Замыкания и декораторы	Настраивать поведение функций и отделять сквозную логику.	После функций как объектов. Вы сохраняете сигнатуру и понимаете порядок вызовов.
Exceptions и context managers	Гарантировать освобождение ресурса и делать отказ частью контракта.	После try/except и файлов. Вы различаете ожидаемое исключение, cleanup и подавление ошибки.
dataclasses и enums	Описывать данные и конечный набор состояний без лишнего шаблонного кода.	После классов. Инварианты не растворены в наборе строковых литералов.
Typing, aliases, generics, Protocol	Сделать интерфейсы видимыми для читателя и статического анализатора.	После функций и классов. Вы помните, что аннотации сами по себе не проверяются Python во время выполнения.

Зависимости внутри Core



Decorators опираются на функции как объекты и замыкания; изучать их как отдельный синтаксический трюк недостаточно.



Generators становятся понятными после протокола итератора, а не до него.

Стандартная библиотека как рабочий инструмент

Библиотека по назначению, а не по алфавиту

КОЛЛЕКЦИИ

`collections`
`collections.abc`

подходящая структура вместо случайного
list

ВРЕМЯ

datetime
timezone-aware значения
явный формат обмена

ПУТИ И ДАННЫЕ

pathlib
json
кодировка UTF-8
схема и валидация

ТЕКСТ

re
шаблон только там, где он проще явного
разбора
тесты граничных строк

РЕСУРСЫ

contextlib

with
детерминированное освобождение

ДИАГНОСТИКА

logging
уровни и контекст
не заменять логами тесты

Изучайте модуль тогда, когда он решает реальную задачу проекта.

Модули, окружения и поставка

Следующий уровень модульности начинается с ясных импортов и пакета, у которого есть публичный интерфейс. Виртуальное окружение изолирует зависимости проекта. Описание в `pyproject.toml` фиксирует метаданные и инструменты сборки. Версии зависимостей должны быть воспроизводимыми настолько, насколько требует проект; слепое закрепление каждой транзитивной версии и полное отсутствие ограничений создают разные риски.

Асинхронность и конкурентность

Модель	Подходит	Не решает автоматически
asyncio	Много операций ожидания с библиотеками, поддерживающими <code>async</code> .	CPU-bound вычисления и блокирующие вызовы внутри <code>event loop</code> .
threads	Блокирующий I/O и библиотеки с обычным синхронным API.	Упрощение общего изменяемого состояния.
processes	Изоляция и параллельные CPU-bound задачи.	Бесплатный обмен большими данными и мгновенный старт.

Сначала измерьте характер нагрузки. Затем выберите модель и явно определите отмену, тайм-ауты, завершение, повтор операции и границы общего состояния. Само слово `async` не делает программу быстрее.

Порядок освоения

Начните с идиом коллекций и итерационной модели. Затем углубите исключения, контекстные менеджеры, структуры данных и `typing`. Пакеты и окружения изучайте на реальном проекте. Конкурентность добавляйте после появления измеримой задачи.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Python data model](#)

[typing: Support for type hints](#)

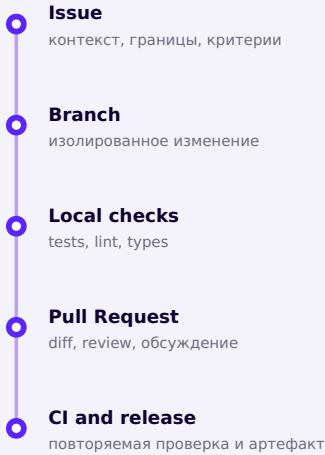
[contextlib: Utilities for with-statement contexts](#)
[The Python Standard Library](#)
[venv: Creation of virtual environments](#)
[Python Packaging User Guide](#)
[asyncio: Asynchronous I/O](#)
[Concurrent Execution](#)

Software Engineering

Git, review, тестирование, статический анализ, диагностика, упаковка и поставка входят в один рабочий процесс.

Как безопасно менять код

В учебном скрипте иногда достаточно получить верный результат один раз. В проекте нужно ещё и безопасно менять код дальше. Рабочий процесс связывает требования, реализацию, проверку и поставку. Инструменты могут меняться. Требования к процессу остаются: историю изменения можно проследить, отказ можно увидеть, а сборку повторить.



Изменение проходит один и тот же проверяемый путь.

Git и review глубже базовых команд

Изучите чтение истории и diff, ветвление, merge, безопасный rebase локальной ветки, revert, теги и разрешение конфликтов. Цель не в коллекции команд. Вы должны понимать, какой граф коммитов получится и как восстановиться после ошибки. Review проверяет контракт, граничные случаи, тесты, читаемость и операционный риск; стиль кода лучше делегировать автоматическим проверкам.

Тестовая стратегия

Средство <code>pytest</code>	Что моделирует	Риск неправильного применения
fixtures	Явную подготовку состояния и ресурсов.	Скрытая сложная <code>fixture</code> делает тест непонятным.
parametrize	Один контракт на наборе входов и границ.	Большая таблица может скрыть разные причины отказа.
mock / monkeypatch	Контролируемую границу: время, сеть, процесс, API.	Mock внутренних деталей связывает тест с реализацией.
coverage	Какие строки или ветви выполнились.	Высокий процент не доказывает качество утверждений.

Стройте пирамиду по риску: много быстрых тестов чистой логики, меньше интеграционных тестов реальных границ и несколько сквозных сценариев. У теста должна быть причина, наблюдаемое утверждение и контролируемая среда.

Автоматические проверки и диагностика

Ruff может выполнять lint и форматирование, а муру проверяет согласованность аннотаций. Выберите правила, зафиксируйте их в `pyproject.toml` и запускайте одинаково локально и в CI. Для дефекта соберите минимальное воспроизведение, прочитайте `traceback`, поставьте точку останова или добавьте

структурированный лог. Профилировщик нужен после измерения проблемы, а не для преждевременной оптимизации.

Пакет, версия и поставка

Проекту нужны ясная структура, метаданные, зависимости, build backend и проверка установки собранного wheel в чистом окружении. Через PyPI публикуют Python-пакеты, но не каждому приложению нужна такая публикация. Semantic Versioning полезен, когда проект объявляет публичный API и может последовательно определить совместимые и несовместимые изменения.

Системный слой

Знания вокруг Python-кода

ПРОТОКОЛЫ

HTTP semantics
status codes
timeouts and retries
идемпотентность

ДАННЫЕ

SQL
схема
индексы

транзакции
миграции

СРЕДА

Linux
shell
processes
permissions
environment variables

БЕЗОПАСНОСТЬ

валидация входа
секреты вне репозитория
минимальные привилегии
обновление зависимостей

CI/CD

повторяемые команды
защищённые секреты
разделение build и deploy
rollback

CONTAINERS

image

runtime config

health check

logs

не путать контейнер и VM

В реальном проекте Python-код взаимодействует с протоколами, данными, ОС и средой эксплуатации.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[pytest documentation](#)

[Ruff documentation](#)

[mypy documentation](#)

[Git reference](#)

[GitHub pull requests documentation](#)

[logging: Logging facility for Python](#)

[The Python Profilers](#)

[Python Packaging User Guide](#)

[Semantic Versioning 2.0.0](#)

[RFC 9110: HTTP Semantics](#)

[PostgreSQL documentation](#)

[GitHub Actions documentation](#)

[Docker Get Started](#)

Как выбрать направление

Короткие практические эксперименты помогают выбрать специализацию по типу ежедневной работы.

Выбирайте тип задач, а не название вакансии

Направление разумно выбирать после нескольких коротких проб. Обратите внимание, какую работу Вы готовы повторять: проектировать HTTP-контракты, исследовать данные, автоматизировать процессы, собирать интерфейсы или настраивать игровой цикл. Интерес к готовому продукту не всегда совпадает с интересом к ежедневной инженерной работе над ним.

Если нравится	Пробный проект	Вероятная ветвь
Контракты, дан- ные, серверное состояние	Небольшое API с БД и тестами	Backend
Измерения, та- блицы, гипоте- зы, модели	Воспроизводимый анализ с baseline и метрикой	Data / ML / AI
Устранять руч- ные операции и связывать ин- струменты	CLI с dry-run,лога- ми и CI	Automation / DevOps
События, фор- мы, локальное состояние, UX	Desktop-приложе- ние с отделённой логикой	Desktop
Интерактив- ность, физика, состояние по кадрам	Завершённая 2D- игра	Game Development

Двухнедельный эксперимент

1. Выберите две ветви, которые действительно интересны.
2. Для каждой выделите по три или четыре занятия и соберите минимальный проект.
3. Запишите, что было интересно в процессе, что утомляло и где не хватало знаний.
4. Сравните не красоту результата, а желание продолжать решать задачи этого типа.
5. Выберите одну основную ветвь на следующие 8 или 12 недель. Решение можно пересмотреть.

Не собирайте стек заранее

Длинный список фреймворков не создаёт специализацию. Сначала нужен один сквозной проект и понимание его фундаментальных границ. Второй инструмент добавляйте, когда можете назвать ограничение первого или требование новой задачи.

Общий контракт любой ветви

Независимо от выбора, сохраняются Python Core и Software Engineering. В каждом проекте должны быть понятный запуск, контролируемые зависимости, тесты критической логики, диагностируемые ошибки, документация решений и история изменений. Эти свойства позволяют переносить опыт между специализациями.

- 
- Проба**
маленький проект
 - Наблюдение**
какая работа увлекает
 - Выбор**
одна ветвь на цикл
 - Глубина**
2 или 3 связанных проекта
 - Переоценка**
по фактам, не по моде

Направление можно сменить после следующего цикла работы.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

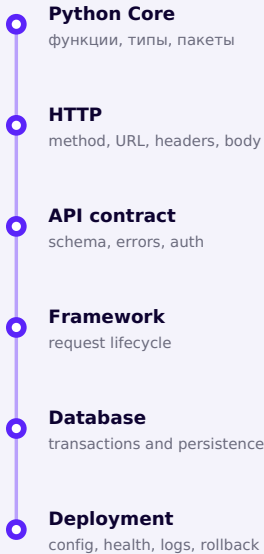
[Python 3.14 documentation](#)

Backend: roadmap

HTTP, API-контракты, SQL, безопасность и эксплуатация перед осознанным выбором web-фреймворка.

Backend начинается с протокола и данных

Backend принимает запрос, проверяет данные и полномочия, выполняет правила предметной области, изменяет состояние и возвращает ответ. Фреймворк помогает выразить этот цикл, но не заменяет понимание HTTP, SQL, транзакций, отказов и безопасности.



Backend-фреймворк имеет смысл после Python Core, HTTP и API-контракта. Затем добавляются БД и требования эксплуатации.

Последовательность обучения

Этап	Изучить	Доказательство
1. Web fundamentals	HTTP semantics, URL, JSON, cookies, browser/server boundary.	Объяснить запрос и ответ без терминов конкретного фреймворка.
2. API and validation	Маршруты, схемы, ошибки, authn/authz, pagination.	Контрактные тесты для успешных и ошибочных ответов.
3. SQL and persistence	Схема, joins, индексы, транзакции, миграции.	Данные сохраняются, ограничения БД защищают инварианты.
4. Operations	Конфигурация, секреты, логи, health checks, deploy, rollback.	Сервис поднимается из чистой среды и диагностирует отказ.

Flask, FastAPI и Django

Инструмент	Сильная сторона	Когда рассматривать
Flask	Небольшое WSGI-ядро и явная композиция расширений.	Обучение границам web-приложения, небольшой сервис, проект с осознанным выбором компонентов.

Инструмент	Сильная сторона	Когда рассматривать
FastAPI	API, построенный вокруг Python type hints, схем и ASGI.	HTTP API с типизированными моделями и асинхронными интеграциями, если они действительно нужны.
Django	Интегрированный набор: ORM, migrations, auth, forms, admin и приглашения проекта.	Приложение, которому полезна согласованная полная платформа.

Flask из [главы 22](#) остаётся действующим и полезным инструментом. Переходить на FastAPI или Django только потому, что они встречаются в другом roadmap, не нужно. Сравнивайте требования: WSGI или ASGI, состав встроенных компонентов, модель данных, размер команды, эксплуатационная среда и стоимость поддержки.

Проект для портфолио

Соберите сервис задач или бронирований: REST API, PostgreSQL, миграции, роли, валидация, тесты доменной логики и API, structured logging, Docker image и CI. Добавьте описание модели угроз и решения о повторе запросов. Не добавляйте микросервисы, очередь и кэш, пока монолит не показывает измеримую потребность в них.

Безопасность проверяется на каждом слое

Проверяйте вход на границе, параметризуйте SQL, храните секреты вне репозитория, ограничивайте права, задавайте тайм-ауты и не раскрывайте внутренние traceback клиенту. Фреймворк снижает часть рисков, но не принимает архитектурные решения за Вас.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[RFC 9110: HTTP Semantics](#)

[Flask documentation](#)

[FastAPI documentation](#)

[Django 6.1 documentation](#)

[PostgreSQL documentation](#)

[Docker Get Started](#)

Data Science, ML и AI: roadmap

Данные, математика, эксперимент, оценка и инженерная поставка без смещения использования модели с её обучением.

Data, ML и AI не являются одним навыком

Аналитик исследует и объясняет данные. ML-инженер строит и предоставляет систему, которая использует обученную модель. Исследователь разрабатывает и проверяет методы. Роли пересекаются, но требуют разной глубины математики, эксперимента и эксплуатации.

Уровень работы с моделью	Что Вы делаете	Что обязаны проверить
Использование готовой модели	Вызываете локальную или внешнюю модель внутри продукта.	Лицензия, данные, стоимость, latency, отказ, качество на своих сценариях.
Адаптация и оценка	Настраиваете pipeline, признаки, параметры или fine-tuning.	Baseline, split без leakage, метрики, воспроизводимость, error analysis.
Обучение и исследование	Проектируете модель или процедуру обучения.	Математические предпосылки, экспериментальный контроль, вычислительный бюджет, статистическая честность.

Зависимости roadmap



Нейронная сеть не предшествует данным, статистике, baseline и оценке модели.

Минимальная научная дисциплина

- Сформулируйте задачу и единицу наблюдения до выбора алгоритма.
- Исследуйте происхождение данных, пропуски, смещения и допустимость использования.
- Зафиксируйте простой baseline. Сложная модель должна выигрывать по заранее выбранной метрике.
- Разделите train, validation и test так, чтобы информация не утекала между частями.

- Смотрите не только на среднее число, но и на ошибки по важным группам и сценариям.
- Сохраняйте код, конфигурацию, seed, версии данных и среды настолько, насколько это нужно для воспроизведения.

Инструменты по роли

Библиотека не заменяет метод

NUMPY

n-dimensional arrays
vectorized operations
численная основа

PANDAS

табличные данные
очистка и преобразование
grouping and joins

MATPLOTLIB

график как проверка гипотезы
подписи, шкалы, uncertainty

SCIKIT-LEARN

preprocessing
classical ML
model selection and evaluation

PYTORCH

tensors
automatic differentiation
neural network training

ENGINEERING

tests for transformations
data contracts
batch or online serving
monitoring

Каждый инструмент отвечает за свою часть
pipeline.

Проект для проверки направления

Возьмите открытый набор данных, сформулируйте один проверяемый вопрос, сделайте data audit, baseline и воспроизводимый pipeline. Для ML добавьте корректное разделение, метрику, error analysis и model card с

ограничениями. Для приложения с готовой AI-моделью добавьте тестовый набор своих сценариев, тайм-аут, обработку недоступности и границу данных. Не называйте демонстрацию обучения доказательством качества в эксплуатации.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[NumPy documentation](#)

[pandas documentation](#)

[Matplotlib documentation](#)

[scikit-learn documentation](#)

[PyTorch documentation](#)

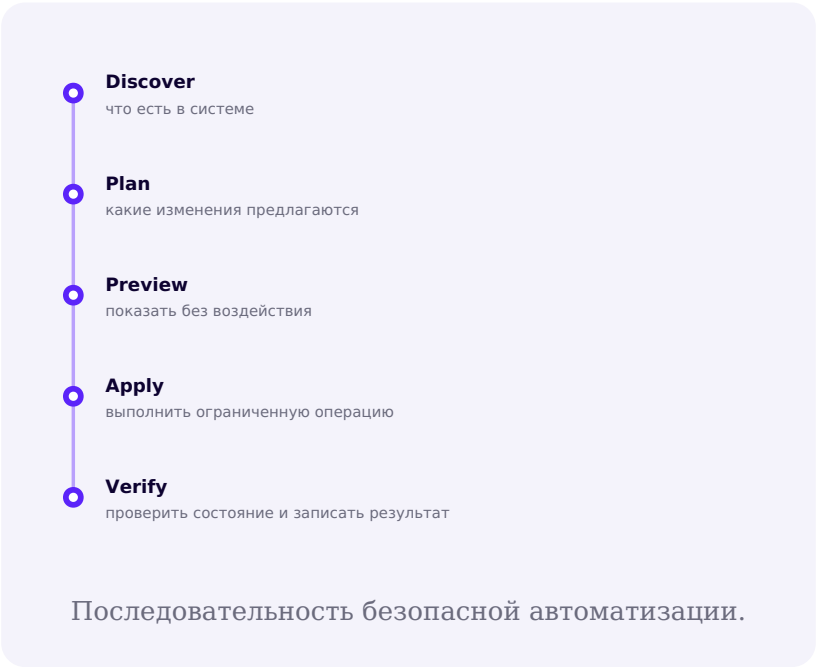
[Python Packaging User Guide](#)

Automation и DevOps: roadmap

CLI, системные границы, интеграции, CI/CD и контейнеры с контролем воздействия и отказов.

Автоматизация начинается с границы воздействия

CLI-инструмент превращает повторяемое ручное действие в проверяемую операцию. Сначала определите вход, наблюдаемый результат и область возможного ущерба. Затем добавьте preview, идиempотентность, логи и код возврата. SafeSort из [главы 23](#) служит таким учебным примером. Это учебный проект, а не готовый файловый менеджер.



Последовательность обучения

Слой	Темы	Критерий
CLI	argparse или другой parser, stdin/stdout/stderr, exit codes, config.	Команда пригодна для shell pipeline и документирует ошибки.
OS boundaries	pathlib, permissions, processes, signals, environment.	Тесты выполняются во временном каталоге и не зависят от домашней директории.

Слой	Темы	Критерий
Integration	HTTP API, auth tokens, rate limits, retries, serialization.	Тайм-ауты и повтор не создают неконтролируемые дубликаты.
CI/CD	workflow, artifact, secrets, environments, deployment gates.	Одна команда проверки работает локально и в CI.
Containers	image layers, runtime config, volumes, health and logs.	Образ воспроизводим, процесс завершается корректно, состояние вынесено явно.

Linux и shell

Изучите файловую модель, владельцев и права, процессы, сигналы, потоки, pipes, перенаправление, переменные окружения и расписание задач. Shell удобен для композиции существующих программ. Python лучше подходит, когда появляются сложные данные, переносимость, тестируемая логика и обработка нескольких классов отказа. Часто правильное решение сочетает оба уровня.

От автоматизации к DevOps

DevOps не сводится к Dockerfile или YAML. Это работа над потоком поставки и обратной связью из эксплуатации. Учитесь строить артефакт один раз, продвигать его между средами, разделять конфигурацию и секреты, проверять health, собирать логи и иметь план rollback. Инфраструктурные изменения требуют review и проверки так же, как код.

Режимы отказа

Автоматизация может повторить ошибку быстрее человека. Ограничивайте область действия, проверяйте пути и идентификаторы перед изменением, избегайте неявных глобальных целей, задавайте тайм-ауты и делайте повтор безопасным там, где это возможно.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[subprocess: Subprocess management](#)

[The Python Standard Library](#)

[GNU Bash manual](#)

[GitHub Actions documentation](#)

[Docker Get Started](#)

Desktop: roadmap

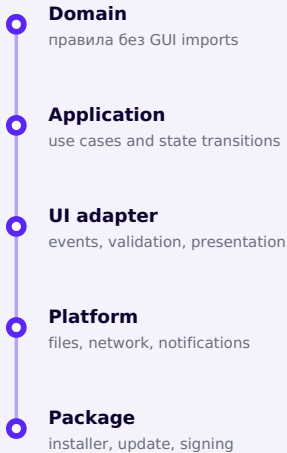
Tkinter и PySide6 в контексте событийной архитектуры, фоновых операций, доступности и поставки приложения.

Когда Tkinter достаточно

Tkinter из [главы 16](#) и последующих проектов остаётся стандартным интерфейсом Python к Tcl/Tk. Он пригоден для обучения событийной модели, внутренних инструментов и приложений с умеренной сложностью интерфейса. Для богатых интерфейсов, сложной модели представления и широкой экосистемы компонентов можно рассматривать PySide6, официальные привязки Python для Qt.

Инструмент	Подходит	Учесть
Tkinter	Небольшие кросс-платформенные GUI, учебные и внутренние инструменты.	Архитектуру, фоновые задачи, packaging и platform testing всё равно проектирует разработчик.
PySide6 / Qt	Более богатые widgets, model/view, QML и крупные desktop-интерфейсы.	Новая объектная модель, event loop, лицензирование компонентов и более сложная поставка.

Архитектура раньше дизайна окон



GUI передаёт события прикладной логике; вся программа не должна жить в обработчиках окна.

План развития desktop-разработчика

- Событийная модель: event loop, callbacks, состояние формы и запрет блокирующей работы в UI thread.
- Композиция интерфейса: layout, widgets, фокус, клавиатурная навигация, accessibility и локализация.
- Архитектура: отделение domain logic, команды, модели представления и адаптеры ресурсов.
- Фоновые операции: cancellation, progress, errors и безопасная передача результата в UI.
- Данные: настройки, миграции локальной схемы, backup и восстановление.
- Поставка: сборка под целевые ОС, зависимости, иконки, installer, обновление и тест на чистой машине.

Проект для проверки направления

Соберите локальный каталог заметок или файлов с поиском. Domain layer должен тестироваться без запуска окна. Добавьте импорт и экспорт, отменяемую долгую операцию, понятное сообщение об ошибке и сборку хотя бы для одной целевой ОС. Отдельно проверьте управление с клавиатуры и ширину окна на небольшом экране.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Python 3.14 documentation](#)

[Qt for Python documentation](#)

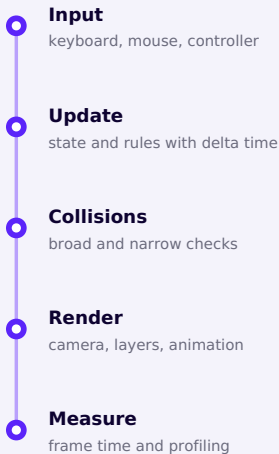
[Concurrent Execution](#)

Game Development: roadmap

Pygame для понимания 2D-цикла, состояния и времени; переход к другим движкам по требованиям цели.

Pygame как хорошая лаборатория 2D

Pygame позволяет изучать игровой цикл, события, координаты, спрайты, коллизии, звук и управление состояниями. Для обучения и самостоятельных 2D-проектов это удобная площадка. При этом Python не доминирует во всех профессиональных игровых движках и классах коммерческих игр. Если цель требует конкретного движка, следующими знаниями могут стать его язык, редактор и pipeline ассетов.



Один кадр игрового цикла; переходы состояния должны оставаться явными.

Порядок углубления

Слой	Темы	Результат
Game state	scene/state machine, pause, restart, save.	Переходы не зависят от случайного набора глобальных флагов.
Time and motion	delta time, fixed timestep where needed, interpolation.	Скорость логики не меняется вместе с FPS.
World	sprites, animation, collisions, camera, tile maps.	Система остаётся измеримой и тестируемой на графиках.
Content	assets, audio, levels, data-driven configuration.	Новый уровень не требует переписывать engine logic.
Delivery	settings, packaging, controller support, performance budget.	Игра запускается на целевой машине и имеет завершённый игровой цикл.

Математика по потребности

Векторы, тригонометрия, преобразования координат и простая физика становятся полезными, когда появляются движение, камера, столкновения и наведение. Изучайте их вместе с наблюдаемой механикой. Для сложной физики сначала определите допустимое приближение и бюджет кадра, затем выбирайте библиотеку или собственную модель.

Проект для портфолио

Доведите небольшую 2D-игру до состояния, в котором есть начало, правила, завершение, звук, настройки и сборка. Добавьте диаграмму состояний, описание архитектуры, короткое видео и таблицу известных ограничений. Завершённый игровой цикл даёт проверяемое доказательство работы, которого нет у прототипа без финального экрана и инструкции запуска.

Когда менять инструмент

Если цель стала связана с конкретным движком, 3D pipeline, платформой или командой, изучите их требования. Опыт с состоянием, временем, координатами, профилированием и Git перенесётся, даже если язык и API изменятся.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Pygame documentation](#)

[The Python Profilers](#)

[Unity Manual: Introduction to programming](#)

[Unreal Engine: Programming with C++](#)

Портфолио разработчика

Осмысленные проекты с доказательствами запуска, проверки и архитектурных решений, а не коллекция повторённых учебных работ.

Портфолио показывает решения

Репозиторий полезен, когда другой разработчик может понять задачу, запустить проект, проверить основные свойства и увидеть, почему архитектура выглядит именно так. Количество проектов не компенсирует отсутствие завершения и объяснения.

Учебные повторы и инженерное портфолио

Учебные повторы	Инженерное портфолио
Повторяет заранее заданные шаги и известный результат.	Начинается с самостоятельно сформулированной проблемы, пользователя и границ.
Показывает только финальный код или снимок экрана.	Даёт воспроизводимую установку, демонстрацию, тесты и проверяемый релиз.
Скрывает ход изменений и причины решений.	Сохраняет историю Git, Issues, небольшие PR и описание архитектурных компромиссов.
Сложно отличить авторские решения от инструкции.	README и documentation объясняют вклад автора, ограничения и дальнейшую работу.

Лестница зрелости проекта

Уровень	Что видно в проекте	Проверяемый результат
Level 1	Скрипт решает одну задачу локально.	Автор может воспроизвести результат.
Level 2	Есть функции, структура каталогов, README и обработка ожидаемых ошибок.	Новый пользователь запускает проект по инструкции.

Уровень	Что видно в проекте	Проверяемый результат
Level 3	Критическая логика покрыта тестами; lint и CI повторяют локальную проверку.	Изменение проходит автоматическую проверку качества.
Level 4	Есть ясные интерфейсы, конфигурация, наблюдаемость, сборка пакета и примечания к релизу.	Артефакт ставится в чистой среде и диагностирует отказ.
Level 5	Описаны компромиссы, ограничения, модель угроз или отказов и эксплуатационный сценарий.	Решение можно обсуждать как инженерную систему.

Эта лестница описывает зрелость демонстрации, а не ранг разработчика. Не каждому проекту нужен Level 5. Один глубокий проект может его достигнуть, а два меньших покажут разнообразие задач.

Как отбирать проекты

Универсального количества проектов и тем более гарантии найма не существует. Выберите минимальный набор, который показывает общий фундамент, выбранную специализацию и качество инженерного процесса. Состав может быть таким:

- один небольшой CLI или библиотека с чистым API и пакетом;
- один основной проект специализации с реальной границей: БД, GUI, data pipeline или game loop;

- один проект, где особенно видны тестирование, CI и диагностика;
- при необходимости один командный или open-source вклад;
- необязательный эксперимент, показывающий исследовательский интерес.

Что должно быть в каждом репозитории

Что должен найти читатель

- README с постановкой проблемы, пользователем, границами решения и статусом проекта.
- Снимки экрана или короткую демонстрацию, подтверждающие основной сценарий без подмены технического описания.
- Воспроизводимую установку: версии, зависимости, команды установки, запуска, тестов и сборки.
- Архитектурную схему, интерфейсы, ключевые компромиссы и известные ограничения.
- Тесты критической логики и CI, повторяющий локальную проверку качества.
- Историю Git, Issues и небольшие изменения, по которым виден ход инженерных решений.
- Версионированный релиз или устанавливаемый артефакт с примечаниями к релизу.
- LICENSE и документацию, достаточные для заявленного способа использования.

Не усложняйте проект ради списка технологий

Не добавляйте Kubernetes, брокер сообщений, микросервисы или нейросеть ради списка технологий. Добавляйте компонент, когда можете сформулировать требование, альтернативы, стоимость и способ проверки. История небольших осмысленных PR делает развитие проекта понятнее, чем один гигантский commit.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Git reference](#)

[GitHub pull requests documentation](#)

[pytest documentation](#)

[GitHub Actions documentation](#)

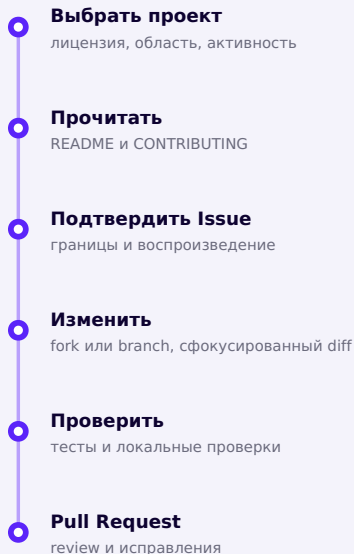
[Python Packaging User Guide](#)

Первый вклад в Open Source

Маленький проверяемый вклад, уважение к правилам проекта, review и лицензиям.

Первый вклад может быть маленьким

Open Source не требует начинать с нового алгоритма в CPython. Исправление опечатки, воспроизводимый bug report, тест для подтверждённого дефекта или уточнение примера может быть полезным вкладом. Ценность определяется правилами конкретного проекта и качеством взаимодействия.



Небольшой вклад проходит техническую проверку и review.

Рабочая последовательность

1. **Выберите проект:** проверьте область, активность, лицензию и соответствие Вашим навыкам.
2. **Прочитайте README:** разберитесь в назначении, установке и поддерживаемых версиях.
3. **Прочитайте CONTRIBUTING:** соблюдайте code of conduct, шаблоны и команды проверки.
4. **Выберите Issue:** найдите небольшую подтверждённую задачу или согласуйте новую.
5. **Воспроизведите:** запишите минимальный сценарий на поддерживаемой версии.
6. **Создайте fork или branch:** следуйте правилам проекта и изолируйте изменение.

7. **Внесите изменение:** держите diff сфокусированным и не смешивайте несвязанные правки.
8. **Выполните тесты:** добавьте проверяемое доказательство и запустите локальные проверки.
9. **Откройте Pull Request:** опишите причину, область и выполненную проверку.
10. **Пройдите review:** отвечайте на вопросы спокойно и обсуждайте требования и компромиссы.
11. **Внесите исправления:** обновите код и тесты; после решения поблагодарите участников.

Социальные навыки являются частью инженерии

У сопровождающих проекта мало времени. Короткий контекст, воспроизводимый пример и сфокусированный diff сокращают время на review. Несогласие обсуждайте через требования и компромиссы. Не принимайте замечание к коду как оценку личности и не требуйте немедленного ответа.

Лицензии и происхождение кода

Публично доступный код не означает разрешение копировать его без условий. Прочитайте лицензию исходного и целевого проекта, сохраняйте обязательные уведомления и не вставляйте код неизвестного происхождения. Если условия непонятны, задайте вопрос сопровождающим проекта или выберите собственную реализацию. Это учебное правило, а не юридическая консультация.

Хорошая первая цель

Ищите задачу, которую можно понять и проверить за несколько занятий. Маленький принятый diff учит чтению чужого кода, правилам проекта и review лучше, чем большой необсуждённый rewrite.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Git reference](#)

[GitHub pull requests documentation](#)

[pytest documentation](#)

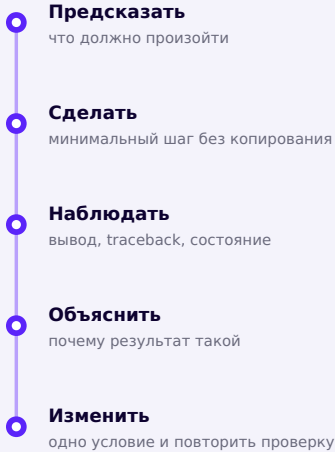
Как учиться дальше

Как выйти из tutorial hell с помощью самостоятельной декомпозиции, отладочных экспериментов и чтения документации.

Что называют tutorial hell

Так иногда называют состояние, в котором человек уверенно повторяет пошаговые уроки, но не может начать собственную задачу без готовой последовательности действий. Это не диагноз и не повод стыдиться. Обычно не хватает перехода от узнавания к самостоятельному решению задач, декомпозиции и отладке.

Цикл самостоятельного обучения



Так ошибка становится частью наблюдаемого эксперимента.

Диагностический протокол

1. Запишите ожидаемый и фактический результат.
2. Сведите проблему к минимальному воспроизводимому примеру.
3. Прочитайте тип исключения и последнюю релевантную строку traceback.
4. Проверьте входы, типы, состояние и предположения на границе дефекта.
5. Сформулируйте одну гипотезу и измените одну переменную.
6. После исправления добавьте тест, который падал бы до него.

7. Запишите причину, а не только финальную строку кода.

Четыре режима документации

Режим	Вопрос читателя	Как использовать
Tutorial	Как впервые пройти путь вместе с автором?	Для знакомства; затем повторить часть без инструкции.
How-to	Как решить конкретную практическую задачу?	Когда цель уже ясна и нужен рабочий рецепт.
Reference	Как точно устроен этот API?	Проверять сигнатуру, параметры, исключения и версионные детали.
Explanation	Почему система устроена так?	Строить модель, сравнивать концепции и понимать компромиссы.

Официальная документация часто разделяет эти режимы. Заметный пример — документация Django: в ней есть отдельные tutorial, topics guides (explanation), how-to guides и reference, и сама схема Diátaxis выросла из опыта переработки именно этой документации. Не пытайтесь читать reference подряд как учебник. Начните с tutorial или explanation, а reference держите рядом во время реализации. Проверяйте версию документации и минимальный пример локально.

От документации к реализации

Начните с официальной документации нужной версии. Tutorial помогает впервые пройти связный путь, explanation строит модель, how-to решает конкретную задачу, а reference фиксирует точный контракт API. Если поведение остаётся неясным, прочитайте исходный код соответствующего участка и связанные тесты. Затем проверьте issue tracker: там могут быть подтверждённые ограничения, исправления и решения сопровождающих проекта. Исходный код и Issues дополняют документацию, но не отменяют публичный контракт и примечания к релизу.

Использование поиска и помощников

Формулируйте запрос через наблюдаемый симптом, версию, минимальный код и уже выполненные проверки. Считайте полученный ответ гипотезой: сопоставьте его с официальным API, выполните тест и объясните каждую строку. Не вставляйте секреты, закрытый код или пользовательские данные в внешнюю систему без разрешения.

Еженедельная ретроспектива

- Что я могу написать и объяснить без образца?
- Какую ошибку я научился диагностировать?
- Какое решение подтвердил тест или измерение?
- Какой следующий пробел ограничивает проект?

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Diátaxis documentation framework](#)

[Python 3.14 documentation](#)

[Django 6.1 documentation](#)

Следующие курсы Cartesian School

Пять направлений, которые находятся в подготовке. Дат выпуска и формы записи пока нет.

Продолжение в Cartesian School

Ниже показаны направления, которые развивают фундамент этой книги. Они находятся в подготовке. Карточки не являются формой записи, не обещают дату выпуска и не показывают фиктивный прогресс. До публикации используйте roadmap этой главы и официальную документацию.

Готовится

Advanced Python

профессиональный Python

Итерационная модель, декораторы, контекстные менеджеры, типизация, архитектура пакетов, конкурентность и профилирование.

Скоро

Python Backend Developer

от HTTP-контракта до эксплуатации

API, SQL, PostgreSQL, аутентификация и авторизация, тестирование, наблюдаемость, контейнер и развёртывание.

Готовится**Python Testing & Software Engineering****изменения, которым можно доверять**

pytest, проектирование тестов, процессы Git, review, Ruff, статическая типизация, сборка пакетов, CI и подготовка релизов.

Скоро**Python for Data & AI****данные, эксперимент и ML-система**

NumPy, pandas, визуализация, baseline, оценка моделей, воспроизводимость и инженерные границы AI-приложений.

Готовится**Python Automation & DevOps****надёжные инструменты и поток поставки**

CLI, Linux, процессы, API-интеграции, безопасные файловые операции, CI/CD, контейнеры и наблюдаемость.

Как выбирать следующий курс

Не выбирайте по длине списка технологий. Сопоставьте предварительные знания с текущими проектами и найдите курс, который закрывает ближайшее ограничение. Если сложно проектировать функции и

данные, сначала укрепите Professional Python Core. Если код работает, но изменения ломают его, приоритетом будут тестирование и Software Engineering. Специализацию выбирайте после небольшого самостоятельного прототипа.

Статус материалов

Все перечисленные курсы ещё не выпущены и явно отмечены статусом «Скоро» или «Готовится». На этой странице нет цены, даты, ссылки на оплату или фиктивной записи.

Ваш следующий шаг

Один выбранный путь, один проверяемый проект и один вопрос о том, каким Python-разработчиком Вы хотите стать.

Выберите один следующий результат

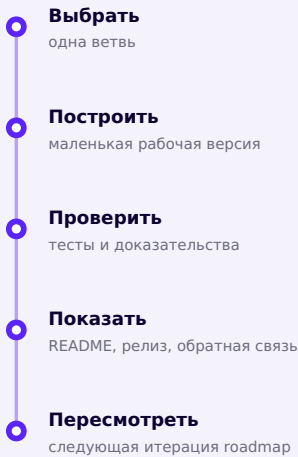
Не нужно проходить пять направлений одновременно. Выберите проект, который можно довести до наблюдаемого результата за ближайшие недели. Опишите пользователя, задачу, границы первой версии и способ проверки. Затем заведите Issue и создайте ветку.

Решение	Запишите сейчас
Направление	Backend, Data / ML / AI, Automation / DevOps, Desktop или Game Development.
Проект	Одно предложение: кто получает какой результат.
Граница	Что войдёт в первую версию и что сознательно не войдёт.
Доказательство	Тест, демонстрационный сценарий, метрика или воспроизводимая сборка.

Решение	Запишите сейчас
Ритм	Реалистичные дни работы и дата короткой ретроспективы.

Вопрос, который остаётся

Ответ не обязан быть окончательным. Он должен быть достаточно конкретным, чтобы сегодня открыть репозиторий, описать Issue и сделать первый проверяемый шаг.



Пересматривайте roadmap после каждой итерации проекта.

Перед началом

- Я могу объяснить, зачем существует проект.
- Я определил минимальный результат и сознательно отложил остальное.
- Я знаю, как проверю критическое поведение.
- Я готов записывать ошибки и решения, а не скрывать их.

ОФИЦИАЛЬНАЯ ДОКУМЕНТАЦИЯ

[Python 3.14 documentation](#)

[Git reference](#)

[pytest documentation](#)

Теперь вопрос уже не:

**«Смогу ли я
программировать?»**

Следующий вопрос:

**«Каким Python-
разработчиком я хочу
стать?»**

**Книга закончилась. Ваш
roadmap — нет.**

Проекты

Двенадцать готовых мини-проектов с открытым исходным кодом — от «Крестики-нолики» до полноценного космического шутера. Исходный код и живая версия каждого — на сайте курса.



Рисовалка

Настоящее приложение для рисования на Tkinter: выбор фигуры, цвета и толщины линии, отрисовка мышью по холсту.

[Tkinter](#)[GUI](#)[Canvas](#)

Глава 18

Теория, из которой вырос этот проект

Практика 18-07

18-07 · Полное приложение

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/tkinter/paint-app/paint_app.py](#)

Запустите локально: `python paint_app.py`



Змейка

Классическая игра «Змейка» на Turtle — движение, поедание яблок, рост тела, столкновения и счёт очков.

Turtle

Игры

Клавиатура

Глава 19

Теория, из которой вырос этот проект

Практика 19-08

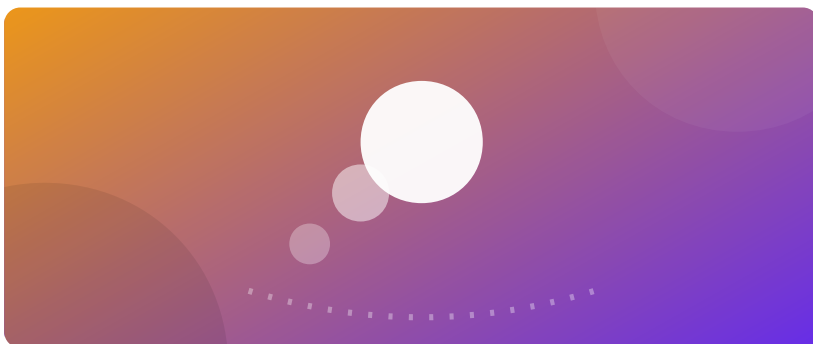
19-08 · Полная игра

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/turtle/snake/snake.py](https://github.com/PyLadies/learnpython/blob/master/projects/turtle/snake/snake.py)

Запустите локально: `python snake.py`



Прыгающий мяч

Первая мини-игра на Pygame: мяч отскакивает от всех четырёх стен экрана в собственном игровом цикле.

Pygame

Игровой цикл

Глава 20

Теория, из которой вырос этот проект

Практика 20-05

20-05 · Мини-проект: прыгающий мяч

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/pygame/bouncing-ball/bouncing_ball.py](#)

Запустите локально: `python bouncing_ball.py`



Космический шутер

Полноценная игра на Pygame: корабль игрока, враги, стрельба, столкновения, счёт очков и экран «Игра окончена».

Pygame

Игры

Клавиатура

Глава 21

Теория, из которой вырос этот проект

Практика 21-08

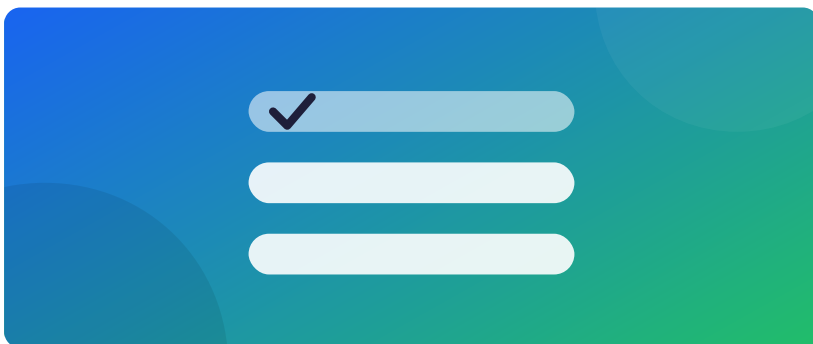
21-08 · Полная игра

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/pygame/space-shooter/space_shooter.py](#)

Запустите локально: `python space_shooter.py`



Список задач

Мини-сайт на Flask со списком задач: маршруты, HTML-шаблоны Jinja2 и приём данных из формы.

[Flask](#)[Веб-разработка](#)[HTML](#)

Глава 22

Теория, из которой вырос этот проект

Практика 22-05

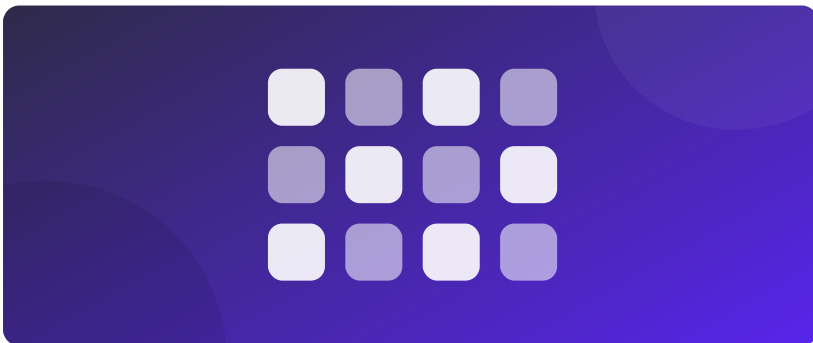
22-05 · Flask в Python

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/flask/todo-app/app.py](https://github.com/roman-yakushev/projects/tree/main/flask/todo-app/app.py)

Запустите локально: `python app.py`



Калькулятор

Работающий калькулятор на Tkinter: сетка кнопок и безопасное вычисление арифметических выражений.

Tkinter

GUI

Глава 23

Теория, из которой вырос этот проект

Практика 23-01

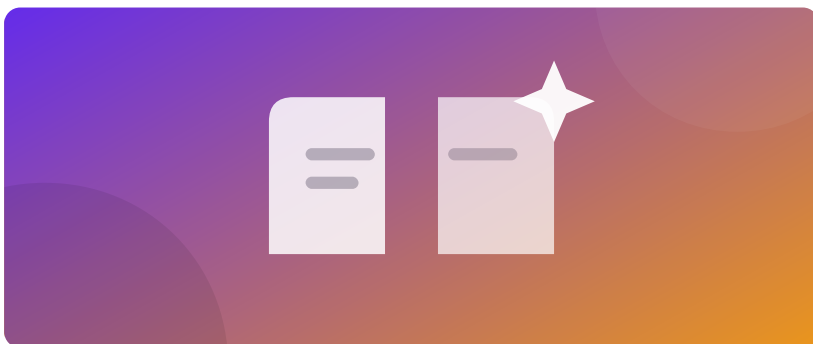
23-01 · Калькулятор с Tkinter

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/tkinter/calculator/calculator.py](https://github.com/PyLadies/Projects/tree/master/projects/tkinter/calculator/calculator.py)

Запустите локально: `python calculator.py`



Генератор случайных историй

Заполняет шаблон предложения случайно выбранными словами из нескольких списков — получаются маленькие абсурдные истории.

[random](#)[Строки](#)

Глава 23

Теория, из которой вырос этот проект

Практика 23-02

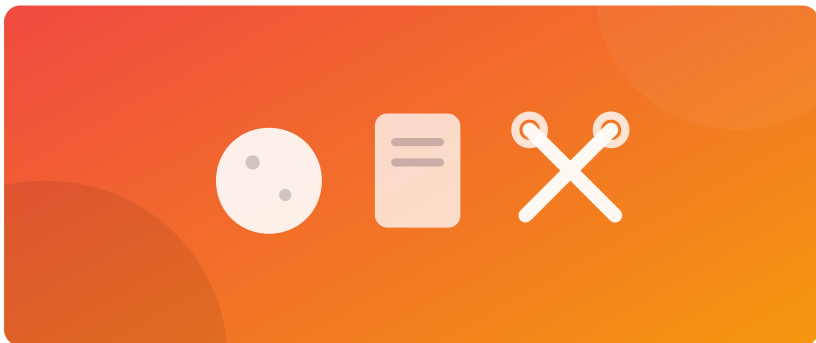
23-02 · Генератор случайных историй

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/console/story-generator/story_generator.py](#)

Запустите локально: `python story_generator.py`



Камень, ножницы, бумага

Классическая игра против компьютера: логика победителя в одном словаре и симуляция раундов.

random

Словари

Глава 23

Теория, из которой вырос этот проект

Практика 23-03

23-03 · Камень, ножницы, бумага

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/console/rock-paper-scissors/rps.py](#)

Запустите локально: `python rps.py`



Отскакивающий мяч — версия с классом

Тот же отскакивающий мяч, переписанный через класс `Myach` — что позволяет легко завести сразу несколько независимых мячей.

Pygame

ООП

Глава 23

Теория, из которой вырос этот проект

Практика 23-04

23-04 · Отскакивающий мяч с Pygame

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/pygame/bouncing-balls-oop/bouncing_balls.py](#)

Запустите локально: `python bouncing_balls.py`



Преобразование температуры

Приложение на Tkinter для перевода температуры между Цельсием, Фаренгейтом и Кельвином.

Tkinter

GUI

Глава 23

Теория, из которой вырос этот проект

Практика 23-05

23-05 · Преобразование температуры

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/tkinter/temperature-converter/temperature_converter.py](#)

Запустите локально: `python temperature_converter.py`



Заметки

Первое знакомство с файлами и Tkinter: текстовое поле с сохранением и загрузкой заметки в обычный файл на диске.

[Tkinter](#)[Файлы](#)

Глава 23

Теория, из которой вырос этот проект

Практика 23-06

23-06 · Файлы и Tkinter

Исходный код

Полный, уже проверенный файл — отдельно:

projects/tkinter/notes-app/notes_app.py

Запустите локально: `python notes_app.py`



Крестики-нолики

Полноценная игра «Крестики-нолики» на Tkinter: сетка кнопок, ходы по очереди и проверка всех восьми выигрышных линий.

Tkinter

Игры

Глава 17

Теория, из которой вырос этот проект

Практика 17-06

17-06 · Новая игра и полная программа

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/tkinter/tic-tac-toe/tic_tac_toe.py](#)

Запустите локально: `python tic_tac_toe.py`



SafeSort

Локальная программа командной строки для безопасной сортировки файлов и поиска дубликатов: план перемещений без изменения файлов, отмена последней операции.

CLI

pathlib

hashlib

Глава 23

Теория, из которой вырос этот проект

Практика 23-08

23-08 · Операции с Path

Исходный код

Полный, уже проверенный файл — отдельно:

[projects/python/safesort/README.md](https://github.com/alekseydudarev/projects/python/safesort/README.md)

Запустите локально: `python README.md`

Предметный указатель

Быстрый способ найти, в какой главе рассматривается нужный термин.

Термины отсортированы по алфавиту. Ссылка ведёт на главу, где термин рассматривается.

Символы и англоязычные термины

break и continue	глава 10	Jinja2, шаблоны Flask	глава 22
CSS	глава 22		
Entry (поле ввода Tkinter)	глава 16	pack, менеджер компоновки Tkinter	глава 16
eval(), безопасное вычисление выражений	глава 23	PyCharm	глава 3
		Pygame	глава 20
f-строки (f-strings)	глава 8	Radiobutton (Tkinter)	глава 23
Flask	глава 22		
for, цикл	глава 10	random, модуль	глава 5
		range()	глава 10
grid, менеджер компоновки Tkinter	глава 16	Rect, pygame.Rect	глава 21
		StringVar (Tkinter)	глава 16
HTML	глава 22	Tkinter	глава 16
HTTP, протокол передачи данных	глава 22	Turtle	глава 6
		VS Code	глава 3
IDLE	глава 3		
if / elif / else	глава 9	while, цикл	глава 10

А—Я

Аргументы функции	глава 13
-----------------------------------	----------

Атрибуты объекта	глава 14	Наследование классов (кратко упомянуто как направление дальнейшего изучения)	глава 24
Булевы значения (True/False)	глава 9	Обработка ошибок (try/except) (пример использования)	глава 23
Ввод данных, input()	глава 8	Объектно-ориентированное программирование (ООП)	глава 14
Вложенные циклы	глава 10	Операторы сравнения	глава 9
Возврат значения, return	глава 13	Отрицательные индексы	глава 8
Глобальные переменные	глава 13	Параметры функции	глава 13
Декоратор (например, @app.route)	глава 22	Переменные	глава 4
Дуги (Turtle)	глава 7	Приоритет операций	глава 5
Замыкания и позднее связывание	глава 17	Регулярные выражения (не рассматриваются подробно — направление для дальнейшего изучения)	глава 24
Индексация строк	глава 8	Свойства объекта (см. «Атрибуты объекта»)	глава 14
Классы и объекты	глава 14	Словари (dict)	глава 11
Конкатенация строк	глава 8	Случайные числа	глава 5
Кортежи (tuple)	глава 11		
Локальные переменные	глава 13		
Лямбда-функции	глава 13		
Массивы (см. «Списки»)	глава 11		
Методы строк	глава 8		
Множества (set)	глава 11		
Модули, импорт	глава 20		

События (events)	глава 17	Файлы, чтение и запись	глава 15
Списки (list)	глава 11	Форматирование строк	глава 8
Срезы списков	глава 11	Функции	глава 13
Срезы строк	глава 8	Целые и дробные числа	глава 4
Столкновения объектов (collision)	глава 19	Циклы	глава 10
Строки (str)	глава 8	Черепашня графика (см. «Turtle»)	глава 6
Таблица истинности (and/or/not)	глава 9		